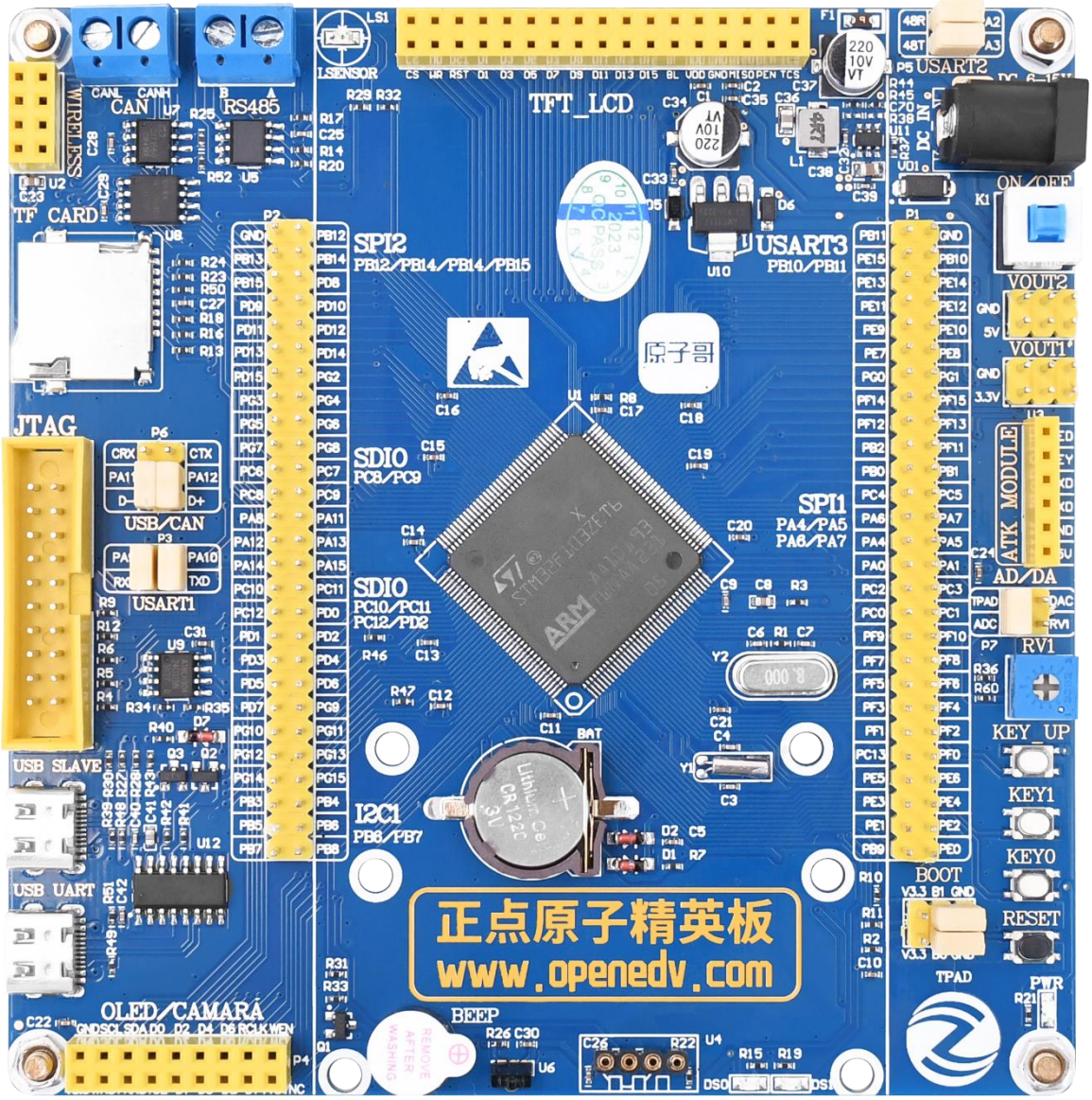
CubeIDE 使用教程

V1.0

-本教程适用于正点原子精英、战舰 开发板



修订历史:

版本	日期	修改内容
V1.0	2024/01/01	第一次发布

正点原子公司名称　：广州市星翼电子科技有限公司
原子哥在线教学平台：www.yuanzige.com
开源电子网 / 论坛：www.openedv.com
正点原子官方网站　：www.alientek.com
正点原子淘宝店铺　：https://openedv.taobao.com
正点原子 B 站视频　：https://space.bilibili.com/394620890

电话：020-38271790　传真：020-36773971

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。
请关注正点原子公众号，资料发布更新我们会通知。

扫码下载“原子哥”APP

扫码关注正点原子公众号

前言 27
第一篇 基础篇 28
第一章 本书学习方法 29
1.1 本书学习顺序 29
1.2 本书参考资料 29
1.3 本书编写规范 29
1.4 本书代码规范 30
1.5 例程资源说明 30
1.6 学习资料查找 31
1.7 给初学者的建议 33
第二章 STM32 简介 35
2.1 初识 STM32 35
2.2 STM32F103 资源简介 36
2.3 STM32F103 设计选型 36
2.3.1 STM32 系列 36
2.3.2 STM32 命名 37
2.3.3 STM32 选型 38
2.3.4 STM32 设计 38
第三章 开发环境搭建 42
3.1 CubeIDE 的简介 42
3.2 常用开发工具简介 42
3.3 CubeIDE 的安装 43
3.3.1 安装 Java 环境 43
3.3.2 STM32CubeIDE 安装 44
3.3.3 STM32CubeIDE 卸载 45
3.4 CH340 USB 虚拟串口驱动安装 47
第四章 STM32CubeIDE 的使用 49
4.1 新建工程 49
4.1.1 创建工程 49
4.1.2 添加用户驱动代码 63
4.1.3 编译工程 64
4.1.4 下载验证 65
4.1.5 仿真调试 69

4.2 STM32CubeIDE 使用技巧 72
4.2.1 STM32CubeIDE 窗口介绍 72
4.2.2 窗口和文本调整 74
4.2.3 代码编辑技巧 80
4.2.4 添加自己的代码文件 88
4.2.5 编译生成 BIN 或 HEX 文件 108
4.2.6 调试技巧 109
4.2.7 CubeIDE 支持打印浮点数设置 127
4.3 无线调试器和 DAP 仿真器的使用 129
4.3.1 DAP 仿真器的使用 129
4.3.2 无线调试器的使用 134
第五章 STM32Cube 固件包 135
5.1 获取 STM32Cube 固件包 135
5.2 STM32CubeF1 固件包目录结构 136
5.2.1 Drivers 文件夹 137
5.2.2 Middlewares 文件夹 137
5.2.3 Projects 文件夹 138
5.2.4 Utilities 文件夹 138
5.2.5 其它文件 138
5.3 CMSIS 文件夹关键文件介绍 138
第六章 认识 HAL 库 142
6.1 初识 HAL 库 142
6.1.1 获取 HAL 库 142
6.1.2 什么是 HAL 库 142
6.1.3 HAL 库能做什么 143
6.1.4 HAL 库和其它库有什么不同 144
6.1.5 怎样学习 HAL 库 145
6.2 HAL 库框架结构 145
6.2.1 HAL 库文件夹结构 145
6.2.2 HAL 库文件介绍 146
6.3 如何使用 HAL 库 150
6.3.1 学会用 HAL 库组织开发工具链 150
6.3.2 HAL 库的用户配置文件 151

6.3.3 stm32f1xx_hal.c 文件..... 153
6.3.4 HAL 库中断处理..... 157
6.3.5 正点原子对 HAL 库用法的个性化修改 158
6.4 HAL 库使用注意事项..... 158
第七章 SYSTEM 文件夹介绍.....160
7.1 sys 文件夹代码介绍 160
7.2 delay 文件夹代码介绍..... 160
第八章 STM32CubeIDE 工程模板分析 161
8.1 STM32CubeIDE 工程文件夹分析 161
1. Binaries..... 162
2. Core 文件夹..... 163
3. Startup 文件夹..... 164
4. Drivers 文件夹 164
5. Debug 文件夹..... 165
6. Release 文件夹..... 165
第九章 STM32 时钟配置.....166
9.1 认识时钟树 166
9.1.1 时钟源 167
9.1.2 锁相环 PLL..... 168
9.1.3 系统时钟 SYSCLK..... 169
9.1.4 时钟信号输出 MCO..... 171
第二篇 实战篇 172
第十章 跑马灯实验 173
10.1 硬件设计 173
1. 例程功能 173
2. 硬件资源 173
3. 原理图 173
10.2 程序设计 173
10.2.1 配套源码路径 173
10.2.2 GPIO 功能引脚配置..... 174
10.2.3 时钟和工程配置（后续实验此步骤基本一样的配置，后面不在过多介绍此步骤） 176
10.2.4 生成工程 178

10.2.5 添加用户驱动代码 180
10.3 下载验证 184
第十一章 蜂鸣器实验 185
11.1 硬件设计 185
1. 例程功能 185
2. 硬件资源 185
3. 原理图 185
11.2 程序设计 185
11.2.1 配套源码路径 185
11.2.2 创建和配置工程 186
11.2.3 时钟和工程配置 188
11.2.4 生成工程 188
11.2.5 添加用户驱动代码 189
11.3 下载验证 190
第十二章 按键输入实验 191
12.1 硬件设计 191
1. 例程功能 191
2. 硬件资源 191
3. 原理图 191
12.2 程序设计 191
12.2.1 配套源码路径 192
12.2.2 GPIO 功能引脚配置 192
12.2.3 时钟和工程配置 193
12.2.4 生成工程 193
12.2.5 添加用户驱动代码 193
12.3 下载验证 196
第十三章 外部中断实验 197
13.1 硬件设计 197
1. 例程功能 197
2. 硬件资源 197
3. 原理图 197
13.2 程序设计 197
13.2.1 配套源码路径 197

13.2.2 GPIO 功能引脚配置 198
13.2.3 NVIC 配置 199
13.2.4 生成工程 199
13.2.5 添加用户驱动代码 200
13.3 下载验证 202
第十四章 串口通信实验 203
14.1 硬件设计 203
1. 例程功能 203
2. 硬件资源 203
3. 原理图 203
14.2 程序设计 204
14.2.1 配套源码路径 204
14.2.2 创建和配置工程 204
14.2.3 配置 USART1 参数 205
14.2.4 配置 NVIC 205
14.2.5 生成初始化代码 206
14.2.6 添加用户驱动代码 206
14.3 下载验证 209
第十五章 独立看门狗（IWDG）实验 211
15.1 硬件设计 211
1. 例程功能 211
2. 硬件资源 211
3. 原理图 211
15.2 程序设计 211
15.2.1 配套源码路径 211
15.2.2 创建和配置工程 211
15.2.3 生成初始化代码 213
15.2.4 添加用户驱动代码 213
15.3 下载验证 215
第十六章 窗口看门狗（WWDG）实验 216
16.1 硬件设计 216
1. 例程功能 216
2. 硬件资源 216

3. 原理图 216
16.2 程序设计 216
16.2.1 配套源码路径 216
16.2.2 创建和配置工程 216
16.2.3 配置 NVIC 218
16.2.4 生成初始化代码 218
16.2.5 添加用户驱动代码 219
16.3 下载验证 220
第十七章 基本定时器实验 221
17.1 硬件设计 221
1. 例程功能 221
2. 硬件资源 221
3. 原理图 221
17.2 程序设计 221
17.2.1 配套源码路径 221
17.2.2 创建和配置工程 221
17.2.3 配置 NVIC 223
17.2.4 生成初始化代码 223
17.2.5 添加用户驱动代码 224
17.3 下载验证 225
第十八章 通用定时器中断实验 226
18.1 硬件设计 226
1. 例程功能 226
2. 硬件资源 226
3. 原理图 226
18.2 程序设计 226
18.2.1 配套源码路径 226
18.2.2 创建和配置工程 226
18.2.3 配置 NVIC 228
18.2.4 生成初始化代码 228
18.2.5 添加用户驱动代码 229
18.3 下载验证 230
第十九章 通用定时器 PWM 输出实验 231

19.1 硬件设计 231
1，例程功能 231
2，硬件资源 231
3，原理图 231
19.2 程序设计 231
19.2.1 配套源码路径 231
19.2.2 创建和配置工程 231
19.2.3 生成初始化代码 234
19.2.4 添加用户驱动代码 234
19.3 下载验证 236
第二十章 通用定时器输入捕获实验 237
20.1 硬件设计 237
1，例程功能 237
2，硬件资源 237
3，原理图 237
20.2 程序设计 237
20.2.1 配套源码路径 237
20.2.2 创建和配置工程 237
20.2.3 配置 TIM5 时基和捕获参数 239
20.2.4 配置 NVIC 240
20.2.5 生成初始化代码 241
20.2.6 添加用户驱动代码 241
20.3 下载验证 245
第二十一章 通用定时器脉冲计数实验 246
21.1 硬件设计 246
1，例程功能 246
2，硬件资源 246
3，原理图 246
21.2 程序设计 246
21.2.1 配套源码路径 246
21.2.2 创建和配置工程 246
21.2.3 配置 TIM2 时基和捕获参数 248
21.2.4 配置 NVIC 250

21.2.5 生成初始化代码 250
21.2.6 添加用户驱动代码 251
21.3 下载验证 254
第二十二章 高级定时器输出指定个数 PWM 实验 255
22.1 硬件设计 255
1， 例程功能 255
2， 硬件资源 255
3， 原理图 255
22.2 程序设计 255
22.2.1 配套源码路径 255
22.2.2 创建和配置工程 255
22.2.3 配置 TIM8 时基等参数 256
22.2.4 配置 NVIC 259
22.2.5 生成初始化代码 259
22.2.6 添加用户驱动代码 260
22.3 下载验证 263
第二十三章 高级定时器输出比较模式实验 264
23.1 硬件设计 264
1， 例程功能 264
2， 硬件资源 264
3， 原理图 264
23.2 程序设计 264
23.2.1 配套源码路径 264
23.2.2 创建和配置工程 264
23.2.3 配置 TIM8 时基等参数 265
23.2.4 生成初始化代码 268
23.2.5 添加用户驱动代码 268
23.3 下载验证 270
第二十四章 高级定时器互补输出带死区控制实验 272
24.1 硬件设计 272
1， 例程功能 272
2， 硬件资源 272
3， 原理图 272

24.2 程序设计 272
24.2.1 配套源码路径 272
24.2.2 创建和配置工程 272
24.2.3 配置 TIM1 时基等参数 273
24.2.4 生成初始化代码 275
24.2.5 添加用户驱动代码 276
24.3 下载验证 278
第二十五章 高级定时器 PWM 输入模式实验 279
25.1 硬件设计 279
1， 例程功能 279
2， 硬件资源 279
3， 原理图 279
25.2 程序设计 279
25.2.1 配套源码路径 279
25.2.2 创建和配置工程 279
25.2.3 配置 TIM3 时基等参数 280
25.2.4 配置 TIM8 时基等参数 282
25.2.5 配置 NVIC 283
25.2.6 生成初始化代码 284
25.2.7 添加用户驱动代码 285
25.3 下载验证 290
第二十六章 电容触摸按键实验 291
26.1 硬件设计 291
1， 例程功能 291
2， 硬件资源 291
3， 原理图 291
26.2 程序设计 292
26.2.1 配套源码路径 292
26.2.2 创建和配置工程 292
26.2.3 配置 TIM5 时基和捕获参数 293
26.2.4 生成初始化代码 294
26.2.5 添加用户驱动代码 295
26.3 下载验证 300
第二十七章 OLED 显示实验 301

27.1 硬件设计 301
1， 例程功能 301
2， 硬件资源 301
3， 原理图 301
27.2 程序设计 302
27.2.1 配套源码路径 302
27.2.2 创建和配置工程 302
27.2.3 生成工程并添加文件 303
27.3 下载验证 313
第二十八章 TFTLCD（MCU 屏）实验 314
28.1 硬件设计 314
1， 例程功能 314
2， 硬件资源 314
3， 原理图 314
28.2 程序设计 315
28.2.1 配套源码路径 315
28.2.2 创建和配置工程 315
28.2.3 配置 FSMC 参数 316
28.2.4 生成工程并添加文件 318
28.3 下载验证 331
第二十九章 USMART 调试组件实验 332
29.1 硬件设计 332
1. 例程功能 332
2. 硬件资源 332
3. 原理图 332
29.2 程序设计 332
29.2.1 配套源码路径 332
29.2.2 创建和配置工程 332
29.2.3 配置 NVIC 334
29.2.4 生成工程并添加文件 334
29.3 下载验证 338
第三十章 RTC 实时时钟实验 340
30.1 硬件设计 340

1. 例程功能 340
2. 硬件资源 340
3. 原理图 340
30.2 程序设计 340
30.2.1 配套源码路径 340
30.2.2 创建和配置工程 340
30.2.3 配置 NVIC 342
30.2.4 生成初始化代码 343
30.2.5 添加用户驱动代码 343
30.3 下载验证 350
第三十一章 PVD 电压监控实验 351
31.1 硬件设计 351
1. 例程功能 351
2. 硬件资源 351
3. 原理图 351
31.2 程序设计 351
31.2.1 配套源码路径 351
31.2.2 创建和配置工程 351
31.2.3 添加文件 352
31.3 下载验证 354
第三十二章 睡眠模式实验 355
32.1 硬件设计 355
1. 例程功能 355
2. 硬件资源 355
3. 原理图 355
32.2 程序设计 355
32.2.1 配套源码路径 355
32.2.2 创建和配置工程 355
32.2.3 配置 NVIC 357
32.2.4 生成并修改代码 357
32.3 下载验证 360
第三十三章 停止模式实验 361
33.1 硬件设计 361

1. 例程功能 361
2. 硬件资源 361
3. 原理图 361
33.2 程序设计 361
33.2.1 配套源码路径 361
33.2.2 创建和配置工程 361
33.2.3 控制逻辑代码实现 362
33.3 下载验证 363
第三十四章 待机模式实验 364
34.1 硬件设计 364
1. 例程功能 364
2. 硬件资源 364
3. 原理图 364
34.2 程序设计 364
34.2.1 配套源码路径 364
34.2.2 创建和配置工程 364
34.2.3 添加用户驱动代码 365
34.3 下载验证 366
第三十五章 DMA 实验 367
35.1 硬件设计 367
1. 例程功能 367
2. 硬件资源 367
3. 原理图 367
35.2 程序设计 367
35.2.1 配套源码路径 367
35.2.2 创建和配置工程 367
35.2.3 生成初始化代码 369
35.2.4 添加用户驱动代码 369
35.3 下载验证 372
第三十六章 单通道 ADC 采集实验 373
36.1 硬件设计 373
1. 例程功能 373
2. 硬件资源 373

3. 原理图 373
36.2 程序设计 374
36.2.1 配套源码路径 374
36.2.2 创建和配置工程 374
36.2.3 生成初始化代码 376
36.2.4 添加用户驱动代码 376
36.3 下载验证 379
第三十七章 单通道 ADC 采集（DMA 读取）实验 380
37.1 硬件设计 380
1. 例程功能 380
2. 硬件资源 380
3. 原理图 380
37.2 程序设计 381
37.2.1 配套源码路径 381
37.2.2 创建和配置工程 381
37.2.3 生成初始化代码 383
37.2.4 添加用户驱动代码 383
37.3 下载验证 387
第三十八章 多通道 ADC 采集（DMA 读取）实验 388
38.1 硬件设计 388
1. 例程功能 388
2. 硬件资源 388
3. 原理图 388
38.2 程序设计 388
38.2.1 配套源码路径 388
38.2.2 创建和配置工程 388
38.2.3 生成初始化代码 391
38.2.4 添加用户驱动代码 391
38.3 下载验证 394
第三十九章 内部温度传感器实验 395
39.1 硬件设计 395
1. 例程功能 395
2. 硬件资源 395

3. 原理图 395
39.2 程序设计 395
39.2.1 配套源码路径 395
39.2.2 创建和配置工程 395
39.2.3 生成初始化代码 397
39.2.4 添加用户驱动代码 397
39.3 下载验证 400
第四十章 光敏传感器实验 401
40.1 硬件设计 401
1. 例程功能 401
2. 硬件资源 401
3. 原理图 401
40.2 程序设计 401
40.2.1 配套源码路径 401
40.2.2 创建和配置工程 402
40.2.3 生成初始化工程 403
40.2.4 添加用户驱动代码 404
40.3 下载验证 407
第四十一章 DAC 输出实验 409
41.1 硬件设计 409
1. 例程功能 409
2. 硬件资源 409
3. 原理图 409
41.2 程序设计 410
41.2.1 配套源码路径 410
41.2.2 创建和配置工程 410
41.2.3 生成初始化工程 412
41.2.4 添加用户驱动代码 413
41.3 下载验证 416
第四十二章 DAC 输出三角波实验 417
42.1 硬件设计 417
1. 例程功能 417
2. 硬件资源 417

3. 原理图 417
42.2 程序设计 417
42.2.1 配套源码路径 417
42.2.2 创建和配置工程 417
42.2.3 生成初始化工程 419
42.2.4 添加用户驱动代码 419
42.3 下载验证 421
第四十三章 DAC 输出正弦波实验 424
43.1 硬件设计 424
1. 例程功能 424
2. 硬件资源 424
3. 原理图 424
43.2 程序设计 424
43.2.1 配套源码路径 424
43.2.2 创建和配置工程 425
43.2.3 生成初始化工程 428
43.2.4 添加用户驱动代码 428
43.3 下载验证 431
第四十四章 IIC 实验 433
44.1 硬件设计 433
1. 例程功能 433
2. 硬件资源 433
3. 原理图 433
44.2 程序设计 433
44.2.1 配套源码路径 433
44.2.2 创建和配置工程 434
44.2.3 添加用户驱动代码 435
44.3 下载验证 443
第四十五章 SPI 实验 445
45.1 硬件设计 445
1. 例程功能 445
2. 硬件资源 445
3. 原理图 445

45.2 程序设计 446
45.2.1 配套源码路径 446
45.2.2 创建和配置工程 446
45.2.3 生成初始化工程 448
45.2.4 添加用户驱动代码 449
45.3 下载验证 455
第四十六章 485 实验 456
46.1 硬件设计 456
1. 例程功能 456
2. 硬件资源 456
3. 原理图 456
46.2 程序设计 457
46.2.1 配套源码路径 457
46.2.2 创建和配置工程 457
46.2.3 生成初始化工程 458
46.2.4 添加用户驱动代码 459
46.3 下载验证 463
第四十七章 CAN 通讯实验 464
47.1 硬件设计 464
1. 例程功能 464
2. 硬件资源 464
3. 原理图 464
47.2 程序设计 465
47.2.1 配套源码路径 465
47.2.2 创建和配置工程 465
47.2.3 生成初始化工程 467
47.2.4 添加用户驱动代码 467
47.3 下载验证 472
第四十八章 触摸屏实验 474
48.1 硬件设计 474
1. 例程功能 474
2. 硬件资源 474
3. 原理图 474

48.2 程序设计 475
48.2.1 配套源码路径 475
48.2.2 创建和配置工程 475
48.2.3 添加用户驱动代码 476
48.3 下载验证 480
第四十九章 红外遥控实验 481
49.1 硬件设计 481
1. 例程功能 481
2. 硬件资源 481
3. 原理图 481
49.2 程序设计 482
49.2.1 配套源码路径 482
49.2.2 创建和配置工程 482
49.2.3 生成初始化工程 485
49.2.4 添加用户驱动代码 485
49.3 下载验证 492
第五十章 DS18B20 数字温度传感器实验 493
50.1 硬件设计 493
1. 例程功能 493
2. 硬件资源 493
3. 原理图 493
50.2 程序设计 494
50.2.1 配套源码路径 494
50.2.2 创建和配置工程 494
50.2.3 添加用户驱动代码 495
50.3 下载验证 501
第五十一章 DHT11 数字温湿度传感器 502
51.1 硬件设计 502
1. 例程功能 502
2. 硬件资源 502
3. 原理图 502
51.2 程序设计 502
51.2.1 配套源码路径 502

51.2.2 创建和配置工程 503
51.2.3 添加用户驱动代码 504
51.3 下载验证 508
第五十二章 无线通信实验 510
52.1 硬件设计 510
1. 例程功能 510
2. 硬件资源 510
3. 原理图 510
52.2 程序设计 511
52.2.1 配套源码路径 511
52.2.2 创建和配置工程 511
52.2.3 生成初始化工程 512
52.2.4 添加用户驱动代码 513
52.3 下载验证 523
第五十三章 FLASH 模拟 EEPROM 实验 525
53.1 硬件设计 525
1. 例程功能 525
2. 硬件资源 525
3. 原理图 525
53.2 程序设计 525
53.2.1 配套源码路径 525
53.2.2 创建和配置工程 525
53.2.3 添加用户驱动代码 526
53.3 下载验证 531
第五十四章 摄像头实验 533
54.1 硬件设计 533
1. 例程功能 533
2. 硬件资源 533
3. 原理图 534
54.2 程序设计 534
54.2.1 配套源码路径 534
54.2.2 创建和配置工程 534
54.2.3 生成初始化工程 536

54.2.4 添加用户驱动代码 537
54.3 下载验证 545
第五十五章 内存管理实验 547
55.1 硬件设计 547
1. 例程功能 547
2. 硬件资源 547
3. 原理图 547
55.2 程序设计 547
55.2.1 配套源码路径 547
55.2.2 创建和配置工程 547
55.2.3 添加用户驱动代码 548
55.3 下载验证 552
第五十六章 SD 卡实验 554
56.1 硬件设计 554
1. 例程功能 554
2. 硬件资源 554
3. 原理图 554
56.2 程序设计 555
56.2.1 配套源码路径 555
56.2.2 创建和配置工程 555
56.2.3 生成初始化工程 556
56.2.4 添加用户驱动代码 557
56.3 下载验证 560
第五十七章 FATFS 实验 563
57.1 硬件设计 563
1. 例程功能 563
2. 硬件资源 563
3. 原理图 563
57.2 程序设计 563
57.2.1 配套源码路径 563
57.2.2 创建和配置工程 563
57.2.3 生成初始化工程 566
57.2.4 添加用户驱动代码 566

57.3 下载验证 575
第五十八章 汉字显示实验 577
58.1 硬件设计 577
1. 例程功能 577
2. 硬件资源 577
3. 原理图 577
58.2 程序设计 577
58.2.1 配套源码路径 577
58.2.2 创建和配置工程 577
58.2.3 添加用户驱动代码 578
58.3 下载验证 581
第五十九章 图片显示实验 582
59.1 硬件设计 582
1. 例程功能 582
2. 硬件资源 582
3. 原理图 582
59.2 程序设计 582
59.2.1 配套源码路径 582
59.2.2 创建和配置工程 582
59.2.3 添加用户驱动代码 583
59.3 下载验证 587
第六十章 照相机实验 589
60.1 硬件设计 589
1. 例程功能 589
2. 硬件资源 589
3. 原理图 589
60.2 程序设计 589
60.2.1 配套源码路径 589
60.2.2 创建和配置工程 590
60.2.3 添加用户驱动代码 590
60.3 下载验证 593
第六十一章 DSP 测试实验 595
61.1 硬件设计 595

1. 例程功能 595
2. 硬件资源 595
3. 原理图 595
61.2 程序设计 595
61.2.1 创建和配置工程 596
61.2.2 添加 DSP 库 596
61.2.3 添加宏定义 597
61.2.4 添加头文件路径 598
61.2.5 添加 Lib 库路径 598
61.2.6 配置 TIM6 599
61.2.7 配置 NVIC 600
61.2.8 添加用户驱动代码 600
61.3 下载验证 604
第六十二章 手写识别实验 605
62.1 硬件设计 605
1. 例程功能 605
2. 硬件资源 605
3. 原理图 605
62.2 程序设计 605
62.2.1 配套源码路径 605
62.2.2 创建和配置工程 605
62.2.3 添加用户驱动代码 606
62.3 下载验证 611
第六十三章 T9 拼音输入法实验 613
63.1 硬件设计 613
1. 例程功能 613
2. 硬件资源 613
3. 原理图 613
63.2 程序设计 613
63.2.1 配套源码路径 613
63.2.2 创建和配置工程 613
63.2.3 添加用户驱动代码 614
63.3 下载验证 619
第六十四章 串口 IAP 实验 620

64.1 硬件设计 620
1. 例程功能 620
2. 硬件资源 620
3. 原理图 620
64.2 程序设计 620
64.2.1 配套源码路径 620
64.2.2 创建和配置工程 621
64.2.3 添加用户驱动代码 621
64.3 下载验证 627
第六十五章 USB 读卡器（Slave）实验 629
65.1 硬件设计 629
1. 例程功能 629
2. 硬件资源 629
3. 原理图 629
65.2 程序设计 630
65.2.1 配套源码路径 630
65.2.2 创建和配置工程 630
65.2.3 生成初始化工程 634
65.2.4 添加用户驱动代码 634
65.3 下载验证 637
第六十六章 USB 虚拟串口（Slave）实验 639
66.1 硬件设计 639
1. 例程功能 639
2. 硬件资源 639
3. 原理图 639
66.2 程序设计 640
66.2.1 配套源码路径 640
66.2.2 创建和配置工程 640
66.2.3 生成初始化工程 643
66.2.4 添加用户驱动代码 644
66.3 下载验证 645
第六十七章 FreeRTOS 移植实验 646
67.1 硬件设计 646

1. 例程功能 646
2. 硬件资源 646
3. 原理图 646
67.2 程序设计 646
67.2.1 创建和配置工程 646
67.2.2 生成初始化工程 648
67.2.3 添加用户驱动代码 649
67.3 下载验证 652

前言

1. 如何学习本书

本书是一本系统性讲解如何使用 STM32CubeIDE 生成对应功能代码的指南。该书共分为 67 个章节，全面覆盖了 STM32CubeIDE 的各个功能的配置。我们将从软件下载、安装、芯片选型、工程创建、编译下载和调试等方面来进行学习。

本课程中我们提供的所有的示例代码均使用 CubeIDE 实现。通过学习本课程，您将会充分了解使用这种开发方式的便捷和高效等特点。

本书的内容按照逐步深入的顺序进行讲解，逐步引导读者使用 CubeIDE 配置并生成对应功能的代码。此外，本书中还包含了大量的实验和配套的例程源码，以帮助读者更好地理解和掌握 CubeIDE 的使用方法。

需要注意的是，本书中的实验并未在原理部分进行详细介绍。相反，我们将重点关注如何使用 STM32CubeIDE 进行配置以实现目标功能。对于希望了解实验原理的读者，建议查阅开发板 A 盘资料中的《HAL 库开发指南》PDF 文档。

2. 本书的配套硬件

本书基于正点原子 STM32F103 系列开发板（精英、战舰）编写，正点原子 STM32 系列开发板为本书的配套开发硬件，使用本书的配套开发硬件完成本书的相关实验，将免去因硬件差异带来的移植等问题。本书配套的正点原子 STM32 系列开发板如下表所示：

开发板型号	内核
精英 STM32F103	ARM Cortex-M3
战舰 STM32F103	ARM Cortex-M3

第一篇 基础篇

万事开头难，如果打好了基础，后面学习就事半功倍了！本篇将详细介绍如何使用STM32CubeIDE，并学习一些相关的基础知识，包括：环境搭建、STM32 入门知识、新建工程、HAL 库介绍、SYSTEM 文件夹介绍等部分。学好了这些基础知识，在后面的例程学习部分，将会有非常大的帮助，能极大的提高大家的学习效率。

如果您是初学者，建议好好学习并理解这些知识，手脑并用，不要漏过任何内容，一遍学不会的可以多学几遍，总之这些知识点都要掌握。

如果您已经学过 STM32 与 STM32CubeIDE 了，本篇内容则可以挑选着学习。

本篇将分为如下章节：

1， 本书学习方法
2， STM32 简介
3，开发环境搭建
4，STM32CubeIDE 的使用
5，STM32CubeIDE 固件包
6，认识 HAL 库
7，SYSTEM 文件夹简介
8，STM32CubeIDE 工程模板分析
9，STM32 时钟配置

第一章 本书学习方法

为了让大家更好的学习和使用本书，本章将给大家介绍一下本书的学习方法，包括：本书的学习顺序、编写规范、代码规范、资料查找、学习建议等内容。

本章将分为如下几个小节：

1.1 本书学习顺序
1.2 本书参考资料
1.3 本书编写规范
1.4 本书代码规范
1.5 例程资源说明
1.6 学习资料查找
1.7 给初学者的建议

1.1 本书学习顺序

为了让大家更好的学习和使用本书，我们做了以下几点考虑：

1，坚持循序渐进的思路编写，从基础到入门，从简单到复杂。

2，将知识进行分类介绍，简化学习过程，包括：基础篇、实战篇。

3，将硬件介绍独立成一个文档（《精英 V2 硬件参考手册.pdf》），本书着重介绍软件知识。

因此，大家在学习本书的时候，我们建议：先通读一遍《精英 V2 硬件参考手册.pdf》，对开发板的硬件资源有个大概了解，然后从基础篇开始，再到入门篇，最后学习提高篇，循序渐进，逐一攻克。

对初学者来说，尤其要按照以上顺序学习，不要跳跃式学习，因为我们书本的知识都是一环扣一环的，如果前面的知识没学好，后面的知识学起来就会很困难。

对于已经学过 STM32 的朋友来说，就可以跳跃式学习了，当然如有不懂，也得翻阅前面的知识点进行巩固。

1.2 本书参考资料

本书的主要参考资料有以下两份文档：

《STM32F10xxx 参考手册_V10（中文版）.pdf》

《Cortex-M3 权威指南》中文版（宋岩 译）

前者是 ST 官方针对 STM32 的一份通用参考资料，重点介绍 STM32 内部资源及使用，寄存器描述等，内容翔实，但是没有实例，也没有对 Cortex-M3 构架进行多少介绍，读者只能根据自己对书本的理解来编写相关代码。

后者是专门介绍 Cortex-M3 构架的书，有简短的实例，但没有专门针对 STM32 的介绍。所以，在学习 STM32 的时候，必须结合这份资料来看。

另外，由于 STM32F103 的中文版是 V10 版本的，而最新的 STM32F103 英文版参考资料已经是 V20 的了，所以大家在遇到一些有问题/矛盾的地方，可以参考最新的英文版参考手册：《STM32 英文参考手册 V20》

这几份参考文档都在我们提供的光盘资料里面可以找到，路径：A 盘→8，STM32 参考资料 文件夹下可以找到。

1.3 本书编写规范

本书通过数十个例程，给大家详细介绍 STM32 的所有功能和外设，按难易程度以及知识结构，我们将本书分为两个篇章：基础篇、和实战篇。

基础篇，主要是一些基础知识介绍，包括开发环境搭建、认识 STM32CubeIDE、新建工程、HAL 库介绍、SYSTEM 文件夹介绍等，这些章节在结构上没有共性，但是互相有关联，

有一个集成的关系在里面，即：必须先学了前面的知识，才好学习后面的知识点。

实战篇，主要涵盖实验实操，旨在通过使用 STM32CubeIDE 来配置并实现相应实验的实验目标。这部分内容占了本书的绝大部分篇幅，而且这些章节在结构上都比较有共性，一般分为 3 个部分，如下：

1，硬件设计

2，程序设计

3，下载验证

硬件设计，包括具体章节的实验具体功能说明、所用到的硬件资源及原理图连接方式，从而知道要做什么？ 需要用到哪些 IO 口？ 是怎么接线的？ 方便程序设计的时候编写驱动代码。

程序设计，一般包括：驱动介绍、配置步骤、配套源码路径、关键代码分析、main 函数讲解等三部分。一点点介绍程序代码是怎么来的，注意事项等，从而学会整个代码。

下载验证，属于实践环节，在完成程序设计后，教大家如何下载并验证我们的例程是否正确？ 完成一个闭环过程。

1.4 本书代码规范

为了方便大家编写高质量代码，我们对本书的代码风格进行了统一，详细的代码规范说明文档，见光盘：A 盘→1，入门资料→《嵌入式单片机 C 代码规范与风格.pdf》，初学者务必好好学习一下这个文档。

总结几个规范的关键点：

1，所有函数/变量名字非特殊情况，一般使用小写字母；

2，注释风格使用 doxgen 风格，除屏蔽外，一律使用 /* */ 方式进行注释；

3，TAB 键统一使用 4 个空格对齐，不使用默认的方式进行对齐；

4，每两个函数之间，一般有且只有一个空行；

5，相对独立的程序块之间，使用一个空行隔开；

6，全局变量命名一般用 g_开头，全局指针命名一般用 p_开头；

7，if、for、while、do、case、switch、default 等语句单独占一行，一般无论有多少行执行语句，都要用加括号：{}。

1.5 例程资源说明

精英 V2 STM32F103 开发板提供的 CubeIDE 例程多达 63 个，我们提供的这些例程，基本都是原创，拥有非常详细的注释，代码风格统一、循序渐进，非常适合初学者入门。

精英开发板的例程列表（实例名称为英文版）如表 1.5.1 所示：

编号	实验名字	编号	实验名字
1	跑马灯实验	21-1	DAC 输出实验
2	蜂鸣器实验	21-2	DAC 输出三角波实验
3	按键输入实验	21-3	DAC 输出正弦波实验
4	外部中断实验	22	IIC 实验
5	串口通信实验	23	SPI 实验
6	独立看门狗实验	24	485 实验
7	窗口看门狗实验	25	CAN 实验
8-1	基本定时器中断实验	26	触摸屏实验
9-1	通用定时器中断实验	27	红外遥控器验
9-2	通用定时器 PWM 输出实验	28	DS18B20 数字温度传感器实验
9-3	通用定时器输入捕获实验	29	DHT11 数字温湿度传感器实验
9-4	通用定时器脉冲计数实验	30	无线通信实验

10-1	高级定时器输出指定个数 PWM 实验	31	FLASH 模拟 EEPROM 实验
10-2	高级定时器输出比较模式实验	32	摄像头实验
10-3	高级定时器互补输出带死区控制实验	33	内存管理实验
10-4	高级定时器 PWM 输入模式实验	34	SD 卡实验
11	电容触摸按键实验	35	FATFS 实验
12	OLED 实验	36	汉字显示实验
13	TFTLCD（MCU 屏）实验	37	图片显示实验
14	USMART 调试实验	38	照相机实验
15	RTC 实验	39-1	DSP BasicMath 测试实验
16-1	PVD 电压监控实验	39-2	DSP FFT 测试实验
16-2	睡眠模式实验	40	手写识别实验
16-3	停止模式实验	41	T9 拼音输入法实验
16-4	待机模式实验	42	串口 IAP 实验
17	DMA 实验	43	USB 读卡器(Slave)实验
18-1	单通道 ADC 采集实验	44	USB 虚拟串口(Slave)实验
18-2	单通道 ADC 采集（DMA 读取）实验	45	FreeRTOS_DEMO
18-3	多通道 ADC 采集（DMA 读取）实验		
18-4	单通道 ADC 过采样（16 位分辨率）		
19	内部温度传感器实验		
20	光敏传感器实验		

表 1.5.1 精英开发板例程表

从上表可以看出，正点原子精英开发板的例程基本上涵盖了 STM32F103ZET6 芯片的所有内部资源，并且外扩展了很多有价值的例程，比如：FLASH 模拟 EEPROM 实验、USMART 调试实验、FreeRTOS 实验、内存管理实验、IAP 实验、拼音输入法实验、手写识别实验等。

而且从上表可以看出，例程安排是循序渐进的，首先从最基础的跑马灯开始，然后一步步深入，从简单到复杂，有利于大家的学习和掌握。所以，正点原子精英 STM32F103 开发板是非常适合初学者的。当然，对于想深入了解 STM32F103 内部资源的朋友，正点原子精英开发板也绝对是一个不错的选择。

1.6 学习资料查找

学习资料包括三个方面：

1，ST 官方的学习资料

ST 官方资料有两个网址：www.stmcu.org.cn 和 www.st.com 。

www.stmcu.org.cn 是 ST 中文社区，里面的资料全部由 ST 中国区的人负责更新和整理，包含了所有 ST 公司的 MCU 资料，比如：STM32F1 最新的芯片文档（参考手册、数据手册、勘误手册、编程手册等）、软件资源（固件库、配置工具、PC 软件等）、硬件资源（各种官方评估板）等，如图 1.6.1 所示：

图 1.6.1 STM32F1 相关资料（stmcu.org）

www.st.com 是 ST 官网，ST 最新最全的资料，一般都是放在该网站，ST 中文社区的资料，一般都是从 ST 官网搬过来的，所以如果你想找最新的 STM32 官方资料，应该在 ST 官网找。对于初学者，一般从 ST 中文社区获取 ST 官方资料就可以了。ST 官网的 STM32F103 资料页面如图 1.6.2 所示（注意：默认是英文语言，需要在网页右上角设置成中文）：

图 1.6.2 STM32F103 官网资料（st.com）

① STM32F103 的硬件相关资源，在：产品→微控制器→STM32 ARM Cortex 32 位微控制器→STM32 主流 MCU→STM32F1 系列→STM32F103→资源 路径下面可以找到。

② STM32F103 的软件相关资源，在：工具与软件→嵌入式软件→微控制器软件→STM32 微控制器软件 路径下面可以找到。

2，正点原子的学习资料

正点原子给大家提供的学习资料，都放在正点原子文档中心，大家可以在文档中心下载所有正点原子最新最全的学习资料，文档中心地址：www.openedv.com/docs/index.html，如图 1.6.3 所示：

图 1.6.3 正点原子文档中心

在文档中心下面，我们可以找到正点原子所有开发板、模块、产品等的详细资料下载地址。

3，正点原子论坛

正点原子论坛，即开源电子网：www.openedv.com/forum.php ，该论坛从 2010 年成立至今，已有 10 年时间，数十万注册用户，STM32 相关帖子数量有 20 多万，每天数百人互动，是一个非常好的 STM32 学习交流平台。

在学习过程中，我们难免会遇到一些问题，有任何问题，大家都可以先去开源电子网搜索一下，基本上你能遇到的问题，我们论坛都有人问过了，所以可以很方便的找到一些参考解决方法。如果实在找不到，你也可以在论坛提问，每天原子哥都会在上面给大家做解答。

不过，大家在论坛发帖的时候，建议先阅读一下提问的智慧，缕清思路，不要乱问一通，提高提问质量，提问的智慧阅读地址：www.openedv.com/thread-7245-1-1.html 。

1.7 给初学者的建议

对于学习 STM32，这里我给大家提以下三点建议：

1，准备一款合适的开发板（强烈建议配仿真器）

任何实验，我们都需要验证，最好的验证方式就是在开发板上面实际跑起来，然后通过仿真器仿真调试，查看具体的执行过程。仿真调试可以加深印象，还可以方便的查找 bug，所以学习 STM32 必备：一个开发板 + 一个仿真器。

另外，开发板在精不在多，学好一款，基本上就够用了。

2，两本参考资料，即《STM32F10xxx 参考手册_V10（中文版）.pdf》和《Cortex-M3 权威指南》。

这两个手册在本章 1.2 节有过介绍，对于我们学习 STM32 和了解 Cortex M3 内核非常有帮助，是学习 STM32 的必备资料，因此初学者尤其要多看这两个手册。

这里建议大家，要多了解一些底层的东西（可结合这两个手册，看看寄存器版本的例程），不要只会使用库，否则，一旦遇到问题，或者换个芯片，就不知道怎么办了。

3，戒骄戒躁，勤思敏行。

学习 STM32 千万不能浮躁，更加不能骄傲，初学者学习 STM32 会遇到很多问题和难点，这个时候千万不能浮躁，不要带情绪，一定要静下心来，缕清思路，逐一攻克。

我就曾经遇到一个问题半个月都没解决的情况，但是这半个月我尝试并掌握了很多解决问题的方法，最终解决问题的时候，其实带来的收获远远大于问题本身。所以不要遇到问题就认怂，就想问别人，问老师，先尝试自己解决一下，比如花个十天半个月去解决一个问题，我相信你也会有很多收获。

学习我们教程的时候，要多思考，多想想为什么要这么写？有没有其他更好的办法？然后，自己去验证，去实践。这里非常重要的一点是要多实践，一定要自己动手写代码，然后再下载到开发板验证，不要只是看看视频，看看例程就算完了，要能做到举一反三，如果自己不实践，不动手写代码，是很难真正学会的。

最后，C 语言是学习 STM32 的必备知识，所以如果 C 语言不过关的朋友，得先好好学习一下 C 语言基础，否则学起来会比较吃力。

第二章 STM32 简介

本章，我们将向大家介绍 STM32 是一个什么东西？有哪些资源？能够做什么？如何选型？等基础知识，让大家对 STM32 有一个大概了解。

本章将分为如下几个小节：

2.1 初识 STM32

2.2 STM32F103 资源简介

2.3 STM32F103 设计选型

2.1 初识 STM32

2007 年 6 月，ST 在北京发布了全球第一款基于 ARM Cortex M3 内核的 32 位通用微控制器芯片：STM32F103，以优异的性能，丰富的资源，超高的性价比，迅速占领市场，从此一鸣惊人，一发不可收拾，截止到 2020 年 6 月，STM32 累计出货量超过 45 亿颗。

精英开发板使用的 STM32F103ZET6 芯片如图 2.1.1 所示：

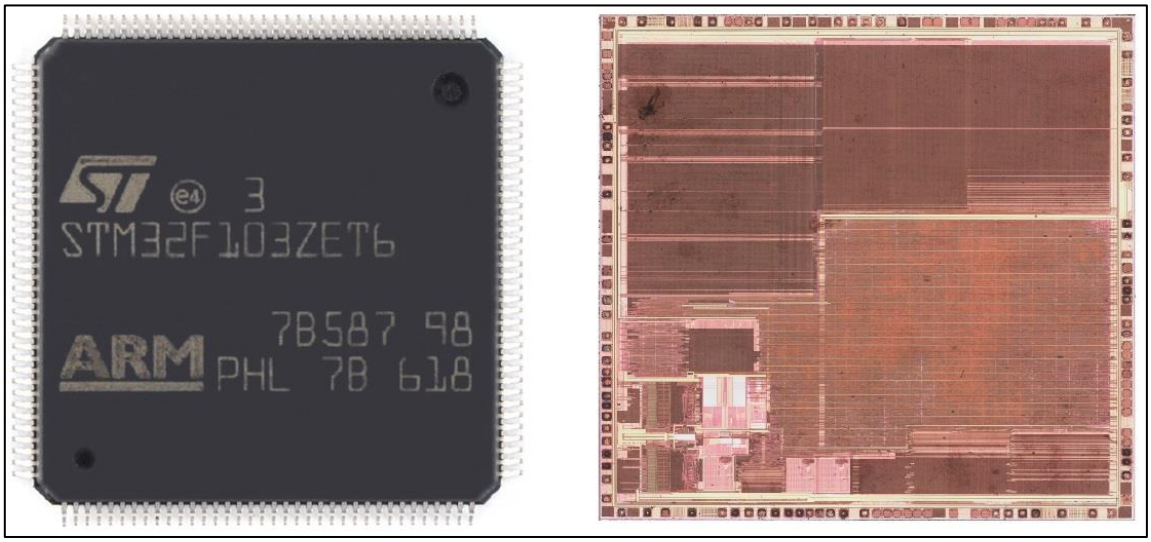


图 2.1.1 STM32F103ZET6 芯片（LQFP144 脚）

上图中，左侧是 STM32F103ZET6 芯片，右侧则是芯片开盖后的图片，即芯片内部视图，可以看到外观看上去平平无奇，但是内部是有很多东西的，需要我们花很多时间和精力去学习掌握。

STM32 的优异性体现在如下几个方面：

1，超低的价格。8 位机的价格，32 位机的性能，是 STM32 最大的优势。

2，超多的外设。STM32 拥有包括：FMC、TIMER、SPI、IIC、USB、CAN、IIS、SDIO、ADC、DAC、RTC、DMA 等众多外设及功能，具有极高的集成度。

3，丰富的型号。STM32 仅 M3 内核就拥有 F100、F101、F102、F103、F105、F107、F207、F217 等 8 个系列上百种型号，具有 QFN、LQFP、BGA 等封装可供选择。同时 STM32 还推出了 STM32L 和 STM32W 等超低功耗和无线应用型的 M3 芯片，另外，ST 还推出了 STM32F4/F7/H7 等更高性能的芯片。

4，优异的实时性能。150 个中断，16 级可编程优先级，并且所有引脚都可以作中断输入。

5，杰出的功耗控制。STM32 各个外设都有自己的独立时钟开关，可以通过关闭相应外设的时钟来降低功耗。

6，极低的开发成本。通过串口即可下载程序，而且相应的仿真器也很便宜，支持 JTAG&SWD 调试接口，最少仅 2 个 IO 口即可实现仿真调试，极大的降低了开发成本。

再来看一个 STM32 与 51 的性能对比如表 2.1.1 所示：

	内核	主频(Mhz)	DMIPS	硬件 FPU
STM32H7	Cortex M7	480	1027	双精度
STM32F7	Cortex M7	216	462	双精度
STM32F4	Cortex M4	168	210	单精度
STM32F1	Cortex M3	72	90	无
STC15	8051	35	35	无

表 2.1.1 STM32 VS 51

这里我们选的 51 是性能比较好的 STC15 系列做为对比，如果换成传统 51，速度会比 STC15 慢 12 倍左右。最强 H7 的 DMIPS 性能约为 STC15 的 30 倍，即便是 STM32F103 也大概有 STC15 性能的 3 倍，由此可见 STM32 的强大，而且最便宜的 STM32F103，价格大概在 5 块多人民币，和 STC15 系列的价格差不多。

简单来说是，价格差不多的情况下，51 能做的，STM32 都能做，51 不能做的，STM32 也能做，因此，越来越多的企业选择使用 STM32 替代 51，所以，大家如果能学会 STM32，找工作的时候，也会有一定的优势。

2.2 STM32F103 资源简介

下面来看看 STM32F103ZET6 具体的内部资源如表 2.2.1 所示：

STM32F103ZET6 资源					
内核	Cortex M3	通用定时器	8	USART	5
主频	72Mhz	高级定时器	2	CAN	1
FLASH	512KB	12 位 ADC	3	SDIO	1
SRAM	64KB	ADC 通道数	18	FSMC	1
封装	LQFP144	12 位 DAC	2	DMA	2
IO 数量	112	SPI	3	RTC	1
工作电压	3.3V	IIC	2	USB 从机	1

表 2.2.1 STM32F103ZET6 内部资源表

由表可知，STM32 内部资源还是非常丰富的，本书将针对这些资源进行详细的使用介绍，并提供丰富的例程，供大家参考学习，相信经过本书的学习，您会对 STM32F103 有一个全面的了解和掌握。

关于 STM32F103 内部资源的详细介绍，请大家参考光盘→A 盘→7，硬件资料→2，芯片资料→STM32F103ZET6.pdf，该文档即 STM32F103 的数据手册，里面有 STM32F103 详细的资源说明和相关性能参数。

2.3 STM32F103 设计选型

STM32 从 2007 年推出至今，已经有 18 个系列，超过 1000 个型号，为了方便大家选择合适的型号设计产品，本节给大家讲讲 STM32 的设计选型。

2.3.1 STM32 系列

STM32 目前总共有 5 大类，18 个系列，如表 2.3.1.1 所示：

大类	系列	内核	特性
主流级 MCU	G0	Cortex M0+	全新入门级 MCU
	G4	Cortex M4	模数混合型 MCU
	F0	Cortex M0	入门级 MCU
	F1	**Cortex M3**	**基础型 MCU**
	F3	Cortex M4	混合信号 MCU
高性能 MCU	F2	Cortex M4	高性能 MCU
	F4	**Cortex M4**	**高性能 MCU**
	F7	**Cortex M7**	**高性能 MCU**

	H7	**Cortex M7**	**超高性能 MCU**，部分型号有双核（M7+M4）
超低功耗 MCU	L0	Cortex M0+	超低功耗 MCU
	L1	Cortex M3	超低功耗 MCU
	L4	**Cortex M4**	**超低功耗 MCU**
	L4+	Cortex M4	超低功耗高性能 MCU
	L5	Cortex M33	超低功耗高性能安全 MCU
	U5	Cortex M33	超低功耗高性能大容量安全 MCU
无线 MCU	WB	Cortex M4 Cortex M0+	双核无线 MCU
	WL	Cortex M4	远程无线 MCU
微处理器 MPU	**MP1**	**Cortex A7** **Cortex M4**	**双核超高性能 MPU**

表 2.3.1.1 STM32 系列分类及说明

可以看到，STM32 主要分两大块，MCU 和 MPU，MCU 就是我们常见的 STM32 微控制器，不能跑 Linux，而 MPU 则是 ST 在 19 年才推出的微处理器，可以跑 Linux。本书重点介绍 MCU 产品，战舰使用的 STM32F103ZET6 属于主流 MCU 分类里面的基础型 F1 系列。

STM32 MCU 提供了包括：基础入门、混合信号、高性能、超低功耗和无线等 5 方面应用的产品型号，我们可以根据自己的实际需要选择合适的 STM32 来设计。比如，我们的产品对性能要求比较高，则可以选择 ST 的高性能 MCU，包括：F2、F4、F7、H7 等 4 个系列的产品；又比如想做超低功耗，则可以选择 ST 的超低功耗 MCU，L 系列的产品。

表 2.3.1.1 中加粗的系列，正点原子都有相应的开发板，包括：主流级 F1、高性能 F4/F7/H7、超低功耗 L4 和微处理器 MP1，大家可以根据自己的需要选择合适的正点原子开发板进行学习。

由于 STM32 系列有很好的兼容性，我们只要能够熟练掌握其中一任何一款 MCU，就可以很方便的学会并使用其他系列的 MCU。比如学好了 STM32F103，再去学 F4/F7/H7 就比较容易学会，由于 STM32F103 系列最早推向市场，资料和教程都是最多的，在市场上的使用也是最为广泛，所以对于没有接触过 STM32 的初学者来说，我们强烈建议先学习 STM32F103，再去学习其他的 STM32 系列。

2.3.2 STM32 命名

STM32 的命名规则如图 2.3.2.1 所示：

图 2.3.2.1 STM32 MCU 命名规则（摘自 STM32 产品选型手册）

如图所示，STM32 的产品名字里面包含了：家族、类别、特定功能、引脚数、闪存容量、

封装、温度范围等重要信息，这些信息可以帮助我们识别和区分 STM32 不同芯片。

精英开发板使用的 STM32F103ZET6，从命名就可以知道如表 2.3.2.1 所示信息：

描述	型号	说明
家族	STM32	ST 公司 32 位 MCU
产品类别	F	基础型
特定功能	103	STM32 基础型
引脚数	Z	144 脚
闪存容量	E	512KB FLASH
封装	T	QFP 封装
温度范围	6	-40～+85℃ 工作温度范围（工业级）

表 2.3.2.1 STM32F103ZET6 型号说明

任何 STM32 型号，都可以按图 2.3.2.1 所示命名规则进行区分解读。

2.3.3 STM32 选型

了解了 STM32 的系列和命名以后，我们再进行 STM32 选型就会比较容易了，这里我们只要遵循：**由高到低，由大到小** 的原则，就可以很方便的完成设计选型了，具体如下：

由高到低原则：在不能评估项目所需性能的时候，可以考虑先选择高性能的 STM32 型号进行开发，比如选择 F4/F7/H7 等，在高性能 STM32 上面完成关键性能（即最需要性能的代码）开发验证，如果满足使用要求，则可以降档，如从 H7→F7→F4→F1，如不满足要求，则可以升档，如从 F4→F7→H7，通过此方法找到最佳性价比的 STM32 系列。

由大到小原则：在不能评估项目所需 FLASH 大小、SRAM 大小、GPIO 数量、定时器等资源需求的时候，可以考虑先选择容量较大的型号进行开发，比如选择 512K/甚至 1M FLASH 的型号进行开发，等到开发完成大部分功能之后，就对项目所需资源有了定论，从而可以根据实际情况进行降档选择（当然极少数情况可能还需要升档），通过此方法，找到最合适的 STM32 型号。

整个选型工作大家可以在正点原子开发板上进行验证，一般我们开发板都是选择容量比较大/资源比较多的型号进行设计的，这样可以免去大家自己设计焊接验证板的麻烦，加快项目开发进度。一些资深工程师，对项目要求认识比较深入的话，甚至都不需要验证了，直接就可以选出最合适的型号，这个效率更高。当然这个需要长期积累和多实践，相信只要大家多学习，多实践，总有一天也能达到这个级别。

2.3.4 STM32 设计

这里我们简单给大家介绍一下 STM32 的原理图设计，上一节我们通过选型原则可以确定项目所需的 STM32 具体型号，但是在选择型号以后，需要先设计原理图，然后再画 PCB、打样、焊接、调试等步骤。这里我们重点介绍如何设计 STM32F103 的原理图。

任何 MCU 部分的原理图设计，其实都遵循：最小系统 + IO 分配 的设计原则。在开始设计原理图之前，我们要通读一遍对 STM32F103 原理图设计非常有用的手册：STM32F103 的数据手册，可以说不看这个数据手册，我们就无法设计 STM32F103 原理图。

1. 数据手册

在设计 STM32F103 原理图的时候，我们需要用到一个非常重要的文档：STM32F103 数据手册，里面对 STM32F103 的 IO 定义和说明有非常详细的描述，是我们设计原理图的基础。战舰开发板所使用的 STM32F103ZET6 芯片数据手册，存放在：A 盘→ 7，硬件资料→2，芯片资料→ STM32F103ZET6.pdf / STM32F103ZET6(中文版).pdf，接下来我们简单介绍一下如何使用该文档。

STM32F103ZET6.pdf 是最新的英文版（V13）STM32 数据手册

STM32F103ZET6(中文版).pdf 是中文版（V5）STM32 数据手册

大家可以根据自己的喜欢来选择合适的版本进行阅读，内容上基本大同小异，从准确性

全面性的角度来说，看 V13 英文版是最好的，从简单，易懂来说，看 V5 中文版也是可以的。

STM32F103ZET6.pdf 数据手册是针对大容量系列（FLASH 容量在 256KB~512KB 之间），主要包括 8 个章节，如表 2.3.4.1 所示：

章节	概要说明
介绍	简单说明数据手册作用：介绍大容量增强型 F103xC/D/E 产品的订购信息和机械特性
规格说明	简单介绍 STM32F103 内部所有资源及外设特点
引脚定义	介绍不同封装的引脚分布、引脚定义等，含引脚特性、复用功能、脚位等
存储器映像	介绍 STM32F103 整个 4GB 存储空间和外设的地址映射关系
电气特性	介绍 STM32F103 的详细电气特性，包括工作电压、电流、温度、各外设资源的电气性能等
封装特性	介绍了 STM32F103 不同封装的封装机械数据（脚距、长短等）、热特性等
订货代码	和 2.3.2 节内容类似，介绍 STM32 具体型号所代表的意义，方便选型订货
版本历史	介绍数据手册不同版本之间的差异和修订内容

表 2.3.4.1 STM32F103 数据手册各章节内容概要

整个 STM32F103 数据手册，对我们开发学习 STM32 来说都比较重要，因此建议大家可以简单的通读一遍这个文档，以加深印象。对于原理图设计，最重要的莫过于引脚定义这一章节了，只有知道了 STM32 的引脚定义，才能开始设计原理图。

STM32F103ZET6 引脚分布如图 2.3.4.1 所示：

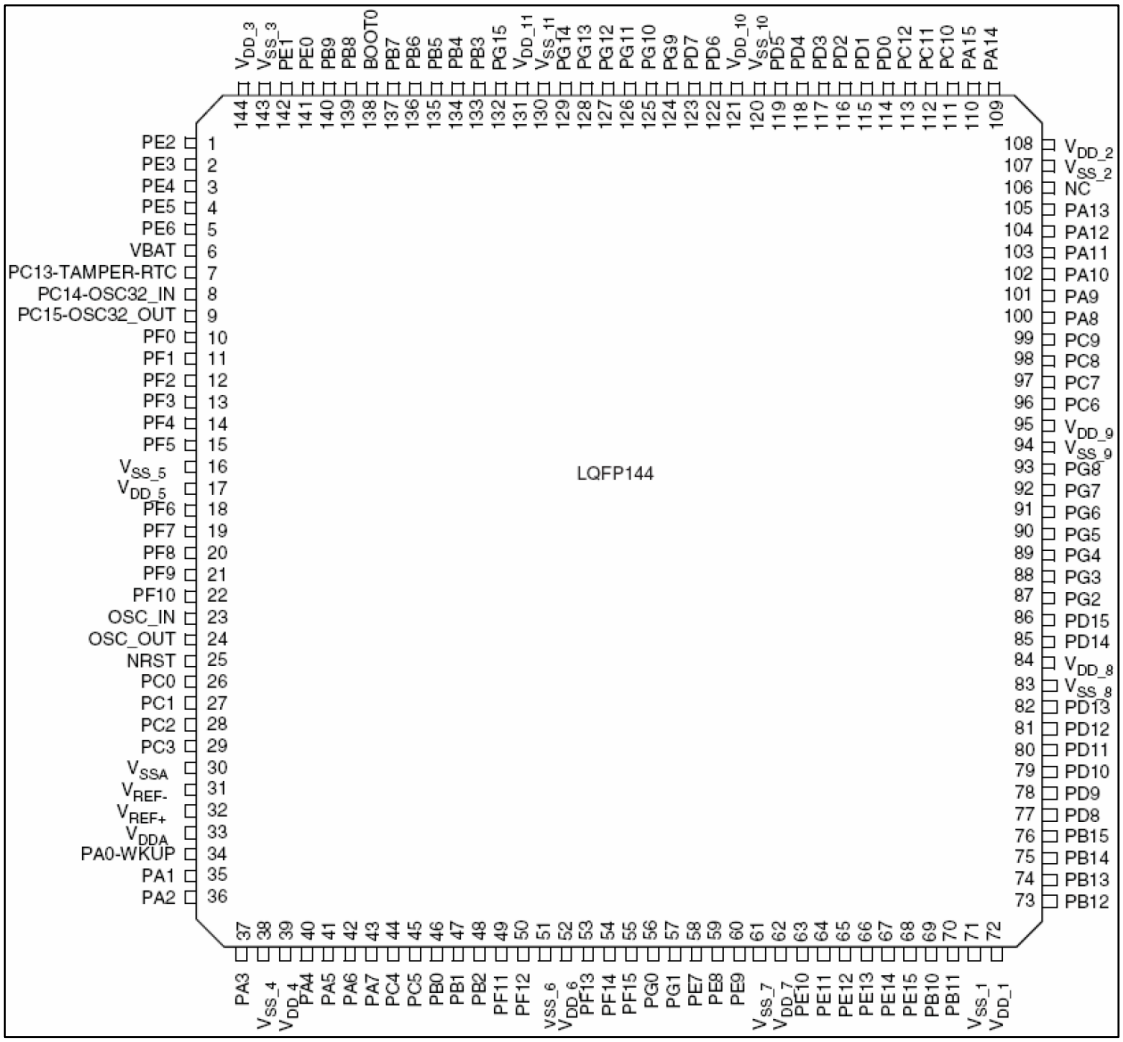


图 2.3.4.1 STM32F103ZET6 芯片引脚分布（摘自 STM32F103 数据手册）

STM32F103ZET6 引脚定义如表 2.3.4.2 所示：

脚位 ①						②	③	④	⑤	⑥ 可选的复用功能	
BGA144	BGA100	WLCSP64	LQFP64	LQFP100	LQFP144	管脚名称	类型(1)	I/O电平(2)	主功能(3)(复位后)	默认复用功能	重定义功能
E10	D8	D8	5	81	114	PD0	I/O	FT	OSC_IN(8)	FSMC_D2(9)	CAN_RX
D10	E8	D7	6	82	115	PD1	I/O	FT	OSC_OUT(8)	FSMC_D3(9)	CAN_TX
E9	B7	A3	54	83	116	PD2	I/O	FT	PD2	TIM3_ETR USART5_RX/SDIO_CMD	
D9	C7	-	-	84	117	PD3	I/O	FT	PD3	FSMC_CLK	USART2_CTS
C9	D7	-	-	85	118	PD4	I/O	FT	PD4	FSMC_NOE	USART2_RTS
B9	B6	-	-	86	119	PD5	I/O	FT	PD5	FSMC_NWE	USART2_TX
E7	-	-	-	-	120	V_{SS_10}	S		V_{SS_10}		
F7	-	-	-	-	121	V_{DD_10}	S		V_{DD_10}		

表 2.3.4.2 STM32F103ZET6 芯片引脚定义（部分）（摘自 STM32F103 数据手册）

引脚定义表的具体说明如表 2.3.4.3 所示：

序号	名称	说明
①	脚位	对应芯片的引脚，LQFP 使用纯数字表示，BGA 使用字母+数字表示 这里列出了 6 种封装的脚位描述，根据实际型号选择合适的封装查阅
②	管脚名称	即对应引脚的名字，PD0~5 表示 GPIO 引脚，VSS_10/VDD_10 表示第 10 组电源引脚，其他类似
③	类型	I/O ：表示输入/输出引脚 I ：表示输入引脚 S ：表示电源引脚
④	IO 电平	FT ：表示 5V 兼容的引脚（可以接 5V/3.3V） 空 ：表示 5V 不兼容引脚（仅可以接 3.3V）
⑤	主功能 (复位后)	复位后，该引脚的默认功能
⑥	可选的 复用功能	默认复用功能：是指开启复用功能后，该引脚默认的复用功能 重定义功能：是指可以通过重映射的复用功能，需设置重映射寄存器

表 2.3.4.3 引脚定义说明

了解了引脚分布和引脚定义以后，我们就可以开始设计 STM32F103 的原理图了。

2. 最小系统

最小系统就是保证 MCU 正常运行的最低要求，一般是指 MCU 的供电、复位、晶振、BOOT 等部分。STM32F103 的最小系统需求如表 2.3.4.4 所示：

类型	引脚名称	说明
电源	VDD/VSS	电源正（VDD）/负（VSS）引脚，给 STM32 供电
	VDDA/VSSA	模拟部分电源正/负引脚，给 STM32 内部模拟部分供电
	VREF+/VREF-	参考电压正/负引脚，给 STM32 内部 ADC/DAC 提供参考电压 100 脚及以上的 STM32F103 型号才有这两个脚
	VBAT	RTC&后备区域供电引脚，给 RTC 和后备区域供电 一般 VBAT 接电池，用于断电维持 RTC 工作 如不需要，直接将 VBAT 接 VDD 即可
复位	NRST	复位引脚，用于复位 STM32，低电平复位
启动	BOOT0/BOOT1	启动选择引脚，一般这两个脚各接一个下拉电阻即可 其他启动配置说明详见后续分析
晶振	OSC_IN / OSC_OUT	外部 HSE 晶振引脚，用于给 STM32 提供高精度系统时钟 如果使用内部 HSI 能满足使用需求，这两个脚可以不接晶振

	OSC32_IN / OSC32_OUT	外部 LSE 晶振引脚，用于给 STM32 内部 RTC 提供时钟 如果使用内部 LSI 能满足使用需求，这两个脚可以不接晶振
调试	SWCLK/SWDIO	SWD 调试引脚，用于调试 STM32 程序，同时 STM32 还支持 JTAG 调试，不过我们不推荐使用！因为 SWD 省 IO！

表 2.3.4.4 STM32F103 最小系统需求

完成以上引脚的设计以后，STM32F103 的最小系统就完成了，关于这些引脚的实际原理图，大家可以参考我们精英开发板的原理图。接下来就可以开始进行 IO 分配了。

3. IO 分配

IO 分配就是在完成最小系统设计以后，根据项目需要对 MCU 的 IO 口进行分配，连接不同的器件，从而实现整体功能。比如：GPIO、IIC、SPI、SDIO、FSMC、USB、中断等。遵循：先分配特定外设 IO，再分配通用 IO，最后微调的原则，见表 2.3.4.5 所示：

分配	外设	说明
特定外设	IIC	IIC 一般用到 2 根线：IIC_SCL 和 IIC_SDA（ST 叫 I2C） 数据手册有 I2C_SCL、I2C_SDA 复用功能的 GPIO 都可选用
	SPI	SPI 用到 4 根线：SPI_CS/MOSI/MISO/SCK 一般 SPI_CS 我们使用通用 GPIO 即可，方便挂多个 SPI 器件 数据手册有 SPI_MOSI/MISO/SCK 复用功能的 GPIO 都可选用
	TIM	根据需要可选：TIM_CH1/2/3/4/ETR/1N/2N/3N/BKIN 等 数据手册有 TIM_CH1/2/3/4/ETR/1N/2N/3N/BKIN 复用功能的 GPIO 都可选用
	USART UART	USART 有 USART_TX/RX/CTS/RTS/CK 信号 UART 仅有 UART_TX/RX 两个信号 一般用到 2 根线：U(S)ART_TX 和 U(S)ART_RX 数据手册有 U(S)ART_TX/RX 复用功能的 GPIO 都可选用
	USB	USB 用到 2 根线：USB_DP 和 USB_DM 数据手册有 USB_DP、USB_DM 复用功能的 GPIO 都可选用
	CAN	CAN 用到 2 根线：CAN_RX 和 CAN_TX 数据手册有 USB_DP、USB_DM 复用功能的 GPIO 都可选用
	ADC	ADC 根据需要可选：ADC_IN0～ADC_IN15 数据手册有 ADC_IN0～ADC_IN15 复用功能的 GPIO 都可选用
	DAC	DAC 根据需要可选：DAC_OUT1 / DAC_OUT2 DAC 固定为：DAC_OUT1 使用 PA4、DAC_OUT2 使用 PA5
	SDIO	SDIO 一般用到 6 根线：SDIO_D0/1/2/3/SCK/CMD 数据手册有 SDIO_D0/1/2/3/SCK/CMD 复用功能的 GPIO 都可选用
	FSMC	根据需要可选：FSMC_D0~15/A0~25/ NBL0~1/NE1~4/NCE2~3/ NOE/NWE/NWAIT/CLK 等 数据手册有 FSMC_D0~15/A0~25/ NBL0~1/NE1~4/NCE2~3/NOE/ NWE/NWAIT/CLK 复用功能的 GPIO 都可选用
通用	GPIO	在完成特定外设的 IO 分配以后，就可以进行 GPIO 分配了 比如将按键、LED、蜂鸣器等仅需要高低电平读取/输出的外设连接到空闲的普通 GPIO 即可
微调	IO	微调主要包括两部分： 1，当 IO 不够用的时候，通用 GPIO 和特定外设可能要共用 IO 口 2，为了方便布线，可能要调整某些 IO 口的位置 这两点，得根据实际情况进行调整设置，做到：尽可能多的可以同时使用所有功能，尽可能方便布线。

表 2.3.4.5 IO 分配

经过以上几个步骤，我们就可以完成 STM32F103 的原理图设计了。

第三章 开发环境搭建

本章，我们将带大家了解什么是 STM32CubeIDE，通过本章的学习，我们将学会搭建 STM32CubeIDE 的环境。

本章将分为如下几个小节：

3.1 CubeIDE 的简介
3.2 常用开发工具简介
3.3 CubeIDE 的安装
3.4 CH340 USB 虚拟串口驱动安装

3.1 CubeIDE 的简介

STM32CubeIDE 是 ST 官方提供的免费软件开发工具，也是 STM32Cube 生态系统的一员大将。它基于 Eclipse®/CDT 框架，GCC 编译工具链和 GDB 调试工具，支持添加第三方功能插件。同时，STM32CubeIDE 集成了 STM32CubeMX 的 STM32 配置和项目创建功能，提供一体化工具体验，大大节省安装和开发时间。用户只需要 STM32CubeIDE 这一个工具，就可以完成从芯片选型，项目配置，代码生成，再到代码编写、编译、调试和烧录的所有工作。可谓是 STM32 的嵌入式开发利器。

图 3.1.1 STM32CubeIDE

3.2 常用开发工具简介

我们开发 STM32 需要用到一些开发工具，如：IDE、仿真器、串口调试助手等。常见的工具如表 3.2.1 所示：

工具	名称	说明
集成开发环境	**CubeIDE**	全名：STM32CubeIDE，是 ST 公司开发的一款集成 STM32CubeMX 和 TrueSTUDIO 的 C/C++开发平台，图形化配置界面，支持数百个插件
仿真器	**DAP**	ARM 公司的开源仿真器，可支持 STM32 仿真调试，且带虚拟串口功能。有高速和低速两个版本，具有免驱、速度快、价格便宜等特点
	STLINK	ST 公司自家的仿真器，支持 STM32 和 STM8 仿真调试，目前最常用的是 ST LINK V2，支持全面、稳定、廉价，是其特点
	JLINK	Segger 公司的仿真器，可支持 STM32 仿真调试，具有稳定、高速的特点，就是价格有点贵。
串口调	**XCOM**	正点原子开发的串口调试助手，具有稳定、功能多、使用简单等特点

试助手	SSCOM	丁丁的串口调试助手，具有稳定、小巧、使用简单等特点。

表 3.2.1 常用开发工具

大家可以根据自己的需要和喜好，选择合适的开发工具。表中加粗部分是我们推荐使用的 STM32 开发工具，即 IDE 推荐使用 CubeIDE、仿真器推荐使用 STLINK 或者 DAP、串口调试助手推荐使用 XCOM。

本书主要是基于 STM32CubeIDE 来开发，所以接下来我们介绍下 STM32CubeIDE 的安装。

3.3 CubeIDE 的安装

3.3.1 安装 Java 环境

安装 CubeIDE 之前需要 JAVA 环境，所以首先需要安装 JAVA8。对于 Java 运行环境，大家可以到 Java 官网 www.java.com 下载最新的 Java 软件，也可以直接从我们光盘复制安装包，路径是“资料盘(A 盘)\6，软件资料\1，软件\Java 安装包”，双击“chromeinstall-8u77.exe”即可进行安装。

图 3.3.1.1 java 安装包

这里大家需要注意，STM32CubeIDE 的 Java 运行环境版本必须是 V1.7 及以上，如果你的电脑安装过 V1.7 以下版本，请先删掉后重新安装最新版本。

对于 Java 运行环境安装，我们这里就不做过多讲解，大家直接双击安装包，根据提示安装即可。安装完成之后提示界面如下图 3.3.1.2：

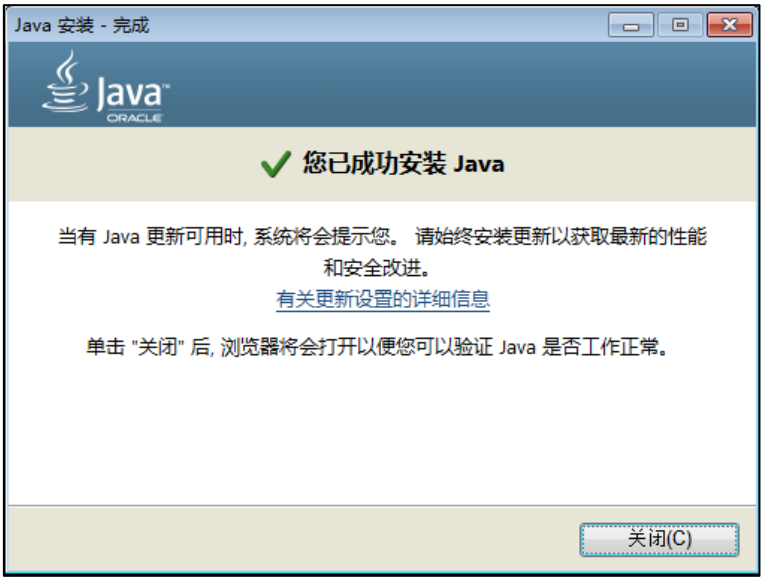


图 3.3.1.2 Java 安装成功提示界面

安装完 Java 运行环境之后，为了检测是否正常安装，我们可以打开 Windows 的命令输入框，输入：java –version 命令，如果显示 Java 版本信息，则安装成功。提示信息如图 3.3.1.3 所示。

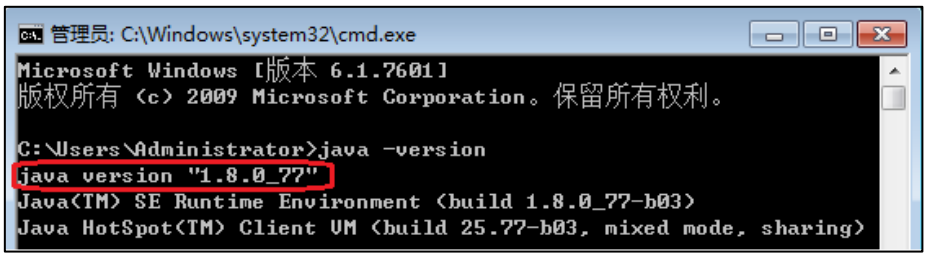


图 3.3.1.3 查看 Java 版本

3.3.2 STM32CubeIDE 安装

一旦 Java 环境安装完成，就可以安装 STM32CubeIDE 软件。STM32CubeIDE 软件的安装方法有两种，第一种方法：通过访问 ST 官方网站下载该软件；第二种方法：使用预先准备好的光盘资料里的软件安装包进行安装。

① 官网下载地址:https://www.st.com/en/development-tools/stm32cubeide.html#st_all-features_sec-nav-tab

图 3.3.2.1 CubeIDE 软件下载

从图 3.3.2.1 中可以看到，STM32CubeIDE 可以支持多种操作系统平台，包括 Windows、Linux 和 MacOS。然而，需要注意的是，该软件仅支持 64 位版本的操作系统。大家根据自己的电脑系统去下载对应版本的 STM32CubeIDE，这里我下载的是 Window 版本的 STM32CubeIDE。

② 资料盘路径："资料盘(A 盘)\6，软件资料\1，软件\6，STM32CubeIDE"，双击"stm32cubeide_1.13.1_x86_64.exe"即可进行安装。

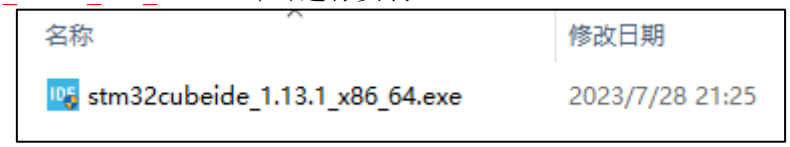


图 3.3.2.2 CubeIDE 安装包

STM32CubeIDE 的安装过程相对简单。在安装过程中，只需要按照提示进行操作即可。需要注意的是，在选择安装路径时如下图 3.3.2.3 所示，一定要注意不要包含中文路径，因为这样可能会导致软件无法正常运行。

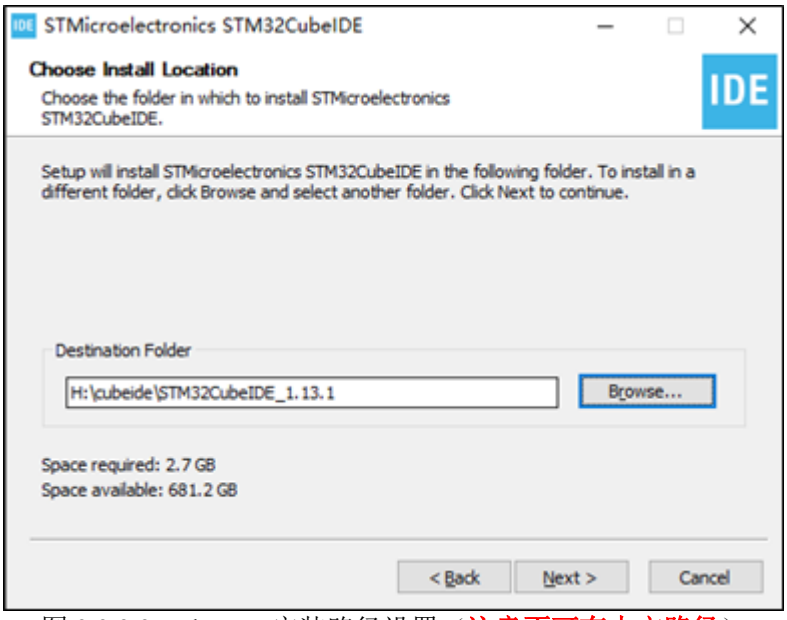


图 3.3.2.3 CubeIDE 安装路径设置（**注意不可有中文路径**）

安装完成后，在我们电脑桌面会显示 CubeIDE 图标，如图 3.3.2.4 所示；

图 3.3.2.4 快捷图标

3.3.3 STM32CubeIDE 卸载

如果不想要 STM32CubeIDE，可以双击 STM32CubeIDE 安装路径下的 uninstall.exe 卸载可执行文件进行卸载，如果已经打开了 STM32CubeIDE 软件，先将其关闭再卸载。这样卸载的好处是，软件卸载的干净，一些注册表信息也同时会被清除，如果用其它软件卸载工具来卸载的话，可能会存在卸载不干净的情况，例如注册表信息可能没有被清除。

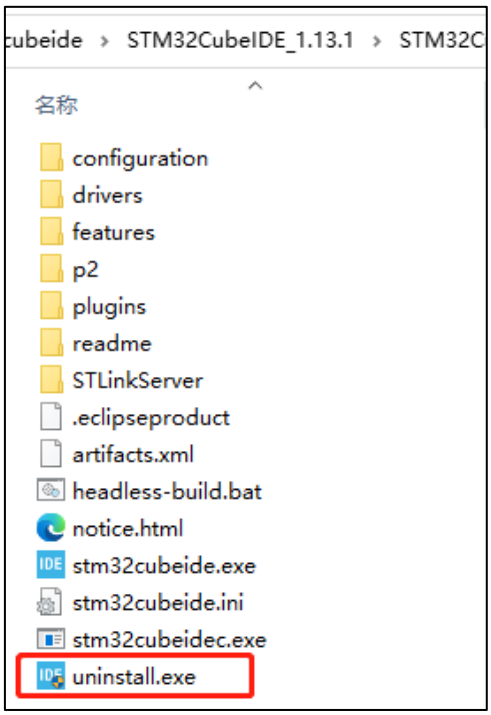


图 3.3.3.1 uninstall.exe 卸载程序

点击 uninstall.exe 卸载可执行文件进行卸载，弹出如下界面，我们选择 Uninstall 进行卸载。

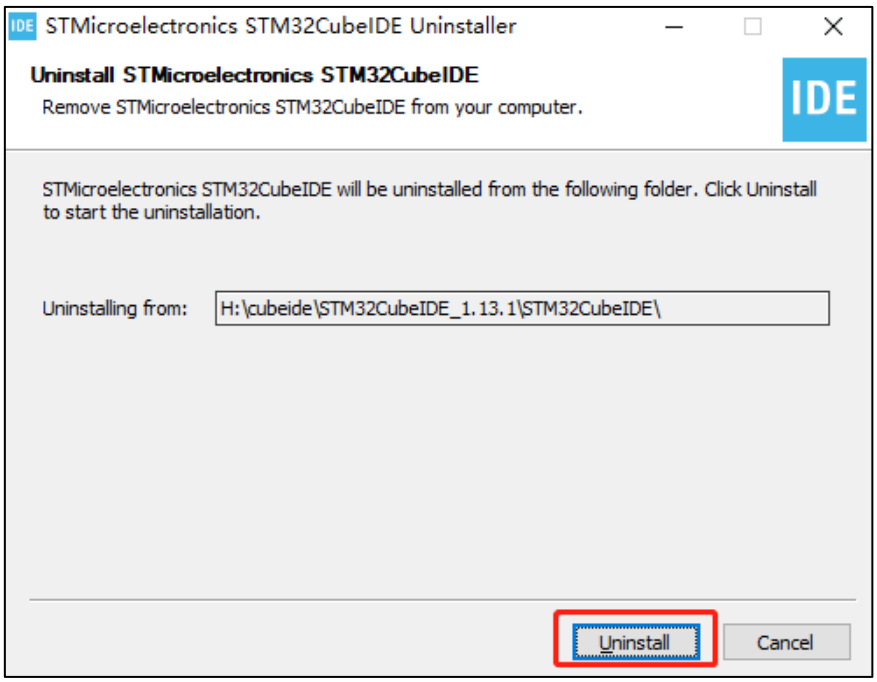


图 3.3.3.2 卸载界面

待进度条完成后，点击 Close 关闭，软件卸载完毕。

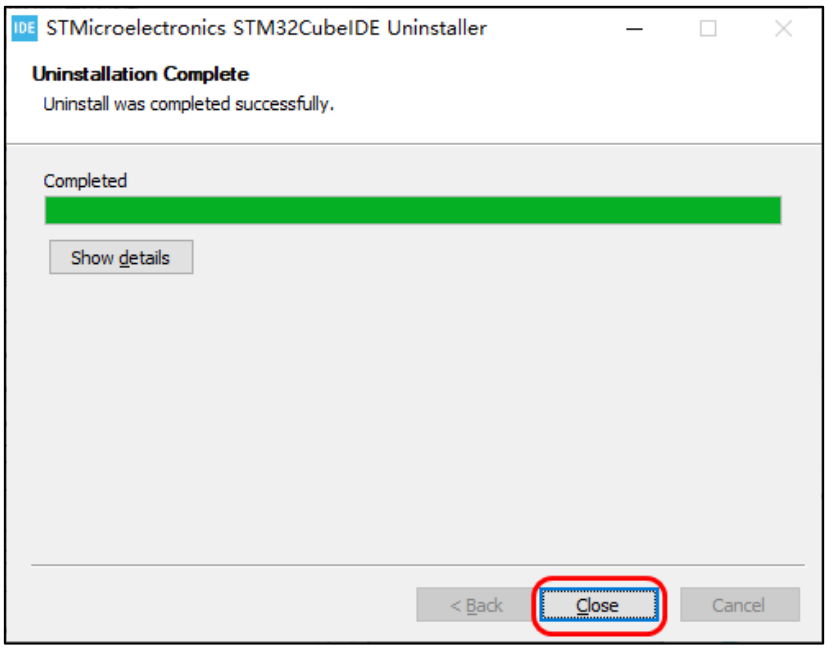


图 3.3.3.3 卸载完毕

3.4 CH340 USB 虚拟串口驱动安装

安装 CH340 USB 虚拟串口驱动，以便我们使用电脑通过 USB 和 STM32 进行串口通信。开发板使用的 USB 虚拟串口芯片是 CH340C，其驱动我们已经放在开发板光盘 A-基础资料→3、软件→ CH340 驱动(USB 串口驱动)_XP_WIN7 共用 文件夹里面，如图 3.4.1 所示：

图 3.4.1 CH340 驱动

双击 SETUP.EXE 进行安装，安装完成后，如图 3.4.2 所示：

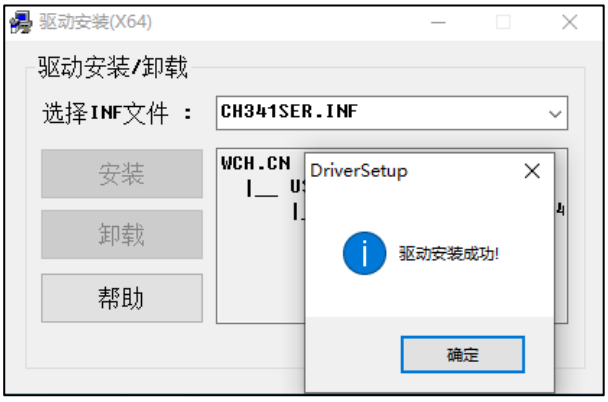


图 3.4.2 驱动安装成功

在驱动安装成功之后，将开发板的 USB_UART 接口通过 USB 连接到电脑，此时电脑就会自动给其安装驱动了。在安装完成之后，可以在电脑的设备管理器里面找到 USB 串口（如果找不到，则重启下电脑），如图 3.4.3 所示：

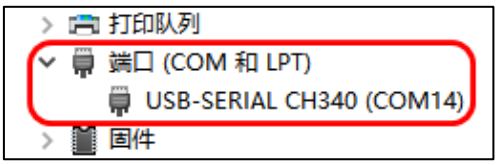


图 3.4.3 设备管理器显示 CH340 USB 虚拟串口

在图 3.4.3 中可以看到，我们的 USB 虚拟串口被识别为 COM14，这里需要注意的是：不同电脑可能不一样，你的可能是 COM4、COM5 等，但是 USB-SERIAL CH340，这个一定是一样的。如果没找到 USB 串口，则有可能是你安装有误，或者系统不兼容。

在安装完 CH340 USB 虚拟串口以后，我们就可以使用串口调试助手，比如 XCOM，和我们的开发板，通过串口进行通信了，这个我们在后续内容再给大家介绍。至此，STM32 的开发环境就搭建完成了。

第四章 STM32CubeIDE 的使用

本章将通过第一个工程实验，带领大家熟悉 STM32CubeIDE 的使用，同时会介绍软件的一些使用技巧，大家平时可以多操作，掌握软件的操作方法，为后面的实验打下基础。

本章将分为如下几个小节：

4.1 新建工程

4.2 STM32CubeIDE 使用技巧

4.3 无线调试器和 DAP 仿真器的使用

4.1 新建工程

本小节我们将使用 STM32CubeIDE 新建一个跑马灯工程（以精英开发板为例），实现 LED0 和 LED1 每 500ms 翻转一次。本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ f103_test。

下面，我们通过这个实验来熟悉 STM32CubeIDE 的基本操作流程，包括新建工程、配置工程、编译工程、下载验证和仿真调试。

4.1.1 创建工程

首先双击打开我们之前安装好的 STM32CubeIDE 软件，并在弹出的窗口中选择一个合适的工作路径。需要注意的是，该工作路径不能包含中文字符，否则可能会导致软件无法正常运行。完成路径的选择后，点击 Launch 按钮，如下图所示：

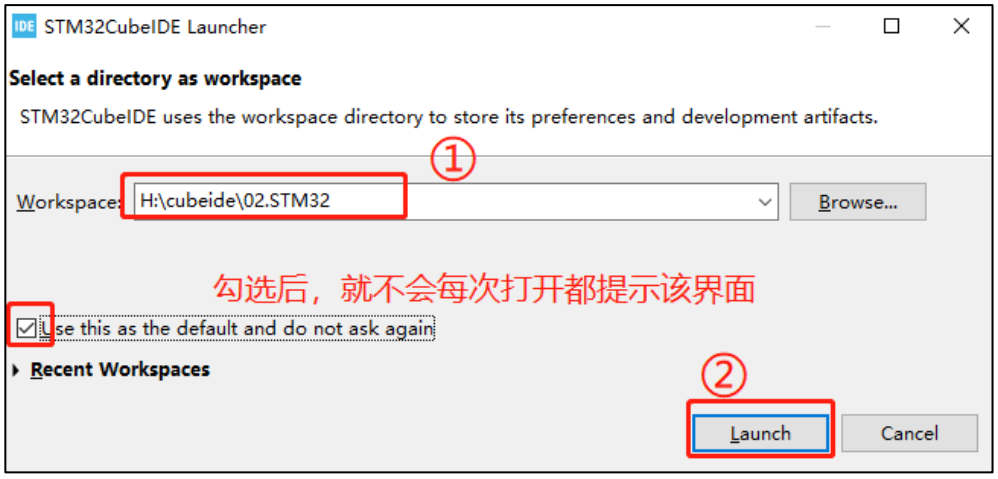


图 4.1.1.1 工作路径设置（**注意不要包含中文路径**）

第一次启动 STM32CubeIDE 以后，会进入如图 4.1.1.2 界面：

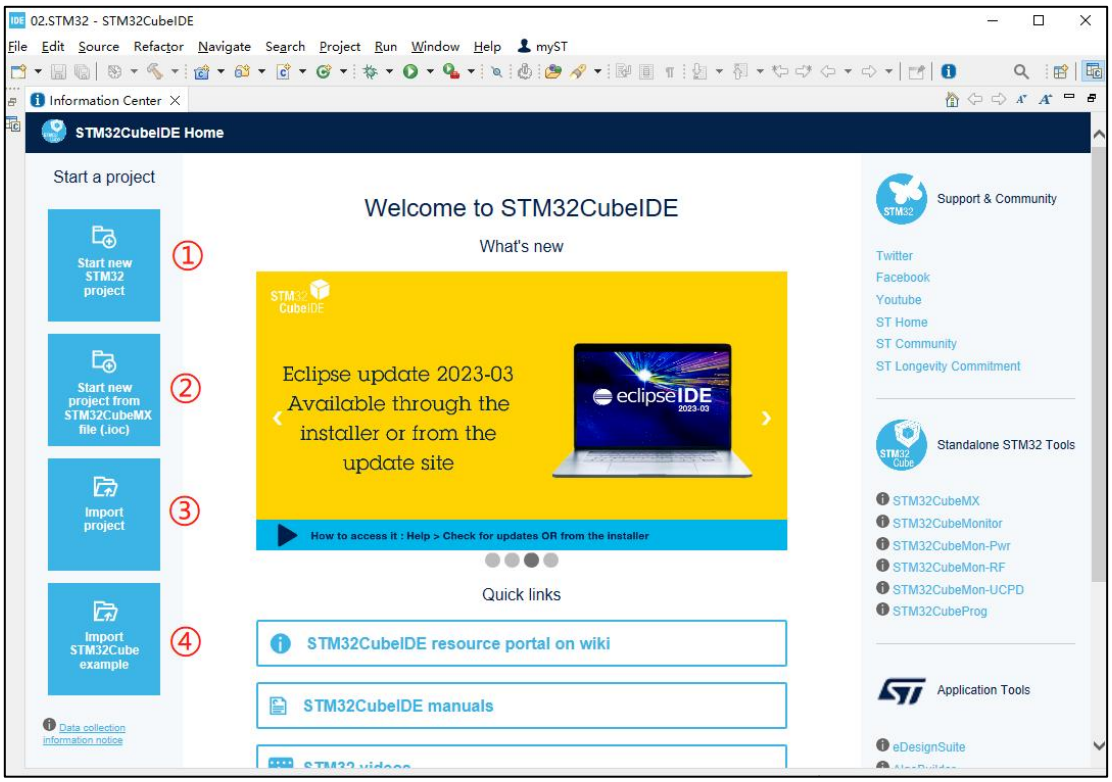


图 4.1.1.2 CubeIDE 开始界面

图 4.1.1.2 展示了 CubeIDE 开始界面的主要功能选项，包括四个方面的内容：

① 新建 CubeIDE 工程：允许用户创建新的工程，并在其中添加源代码、配置选项等。

② 打开现有的 CubeMX 工程：用户可以选择打开现有的 CubeMX 工程文件，该文件后缀名为 ioc。

③ 打开现有的 CubeIDE 工程：用户可以选择打开已经存在的 CubeIDE 工程文件。

④ 打开 ST 官方提供的配置示例：用户可以选择打开 ST 官方提供的一些配置示例，这些示例通常包含了一些常用的外设配置，方便用户快速搭建自己的工程。

我们可以点击①处的 Start new STM32 project 来创建一个新的工程，也可以点击菜单栏的 File→New→STM32 Project 进行创建：

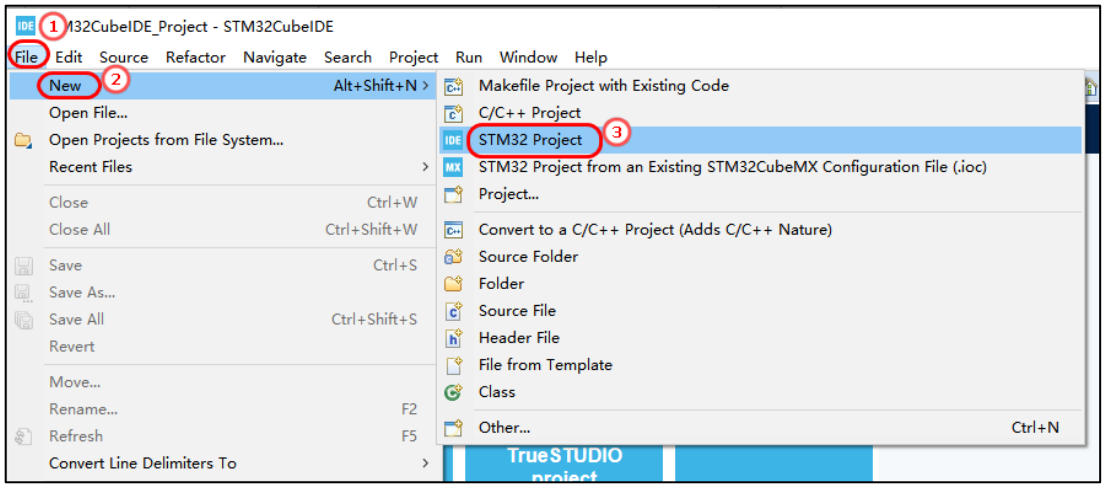


图 4.1.1.3 新建一个 STM32 工程

如图 4.1.1.4，我们进入了 STM32CubeMX 插件界面：

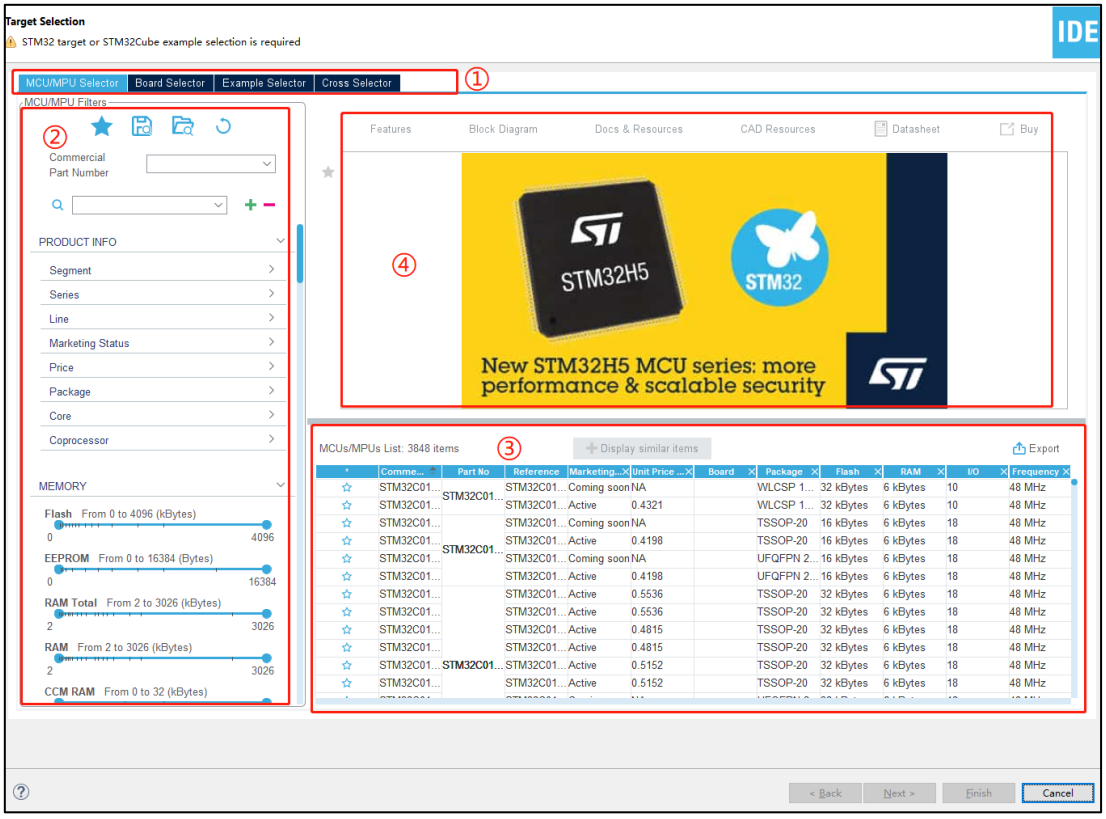


图 4.1.1.4 CubeMX 插件界面

图中的红框部分内容介绍如表 4.1.1.1 所示：

标号	选项	概要说明
①	MCU/MPU Selector	MCU/MPU 芯片选择窗口，我们创建项目一般在此窗口进行
	Board Selector	ST 官方的一些开发板选择窗口
	Example Selector	示例程序选择窗口，ST 官方开发板的一些现有示例
	Cross Selector	交叉引用工具窗口，在此窗口选择对应的 CPU 型号，则会弹出一个相同系列的 CPU 型号的资源对比界面
②	Part Number	芯片型号选择，输入芯片型号可以筛选出符合条件的芯片型号
	Core	内核类型选择，例如 Cortex-M0、Cortex-M3、Cortex-M4、Cortex-M33 和 Arm Cortex-M7 单核，还有 Cortex-M7 和 Cortex-M4 以及 STM32MP1 系列对应的 Cortex-A7+Cortex-M4 双核
	Series	STM 芯片各个系列，包含 ST 的各个产品系列，从 STM32F0 系列到 STM32MP1 系列，以及 STM32WB 和 STM32WL 系列
	Line	STM 芯片各个产品线，从 STM32F0x0 产品线到 STM32MP151、STM32MP153、STM32MP157 产品线，还有其他 STM32WBxM 模块
	Package	ST 各个产品线的封装类型，例如目前 STM32MP1 系类的封装类型有 LFBGA354、LFBGA448、TFBGA257 和 TFBGA361
	Other	其它选项，例如价格范围、IO 引脚数范围、Flash 和 Ram 大小范围以及主频范围等，可以手动配置对应的范围来进行筛选
	Peripheral	周边设备选项，例如可选有无 ADC，有无 FMC，有无 TFT LCD 等，勾选好以后会列出符合条件的产品型号

③	匹配结果列表	显示我们根据②处筛选条件的匹配结果
④	Features	芯片特性介绍，详细介绍芯片有哪些资源
	Block Diagram	CPU 功能框图介绍，以框图的形式列出芯片的整体资源
	Docs &Resources	文档资源浏览，可以下载和在线查看和芯片有关的手册，例如数据手册、参考手册、编程手册等
	Datasheet	数据手册下载区，点击可以下载对应芯片的数据手册
	Buy	ST 官方产品购买链接

表 4.1.1.1 CubeMX 插件界面介绍

（1）选择芯片型号

我们选择①处的 MCU/MPU Selector，然后需要根据所使用的硬件选择相应的芯片型号，这里我们是以精英开发板为例（主芯片为：STM32F103ZET6），所以在②处输入 STM32F103ZET6 然后回车，在③处找到我们要的芯片型号，就可以直接点击下一步，如下图所示：

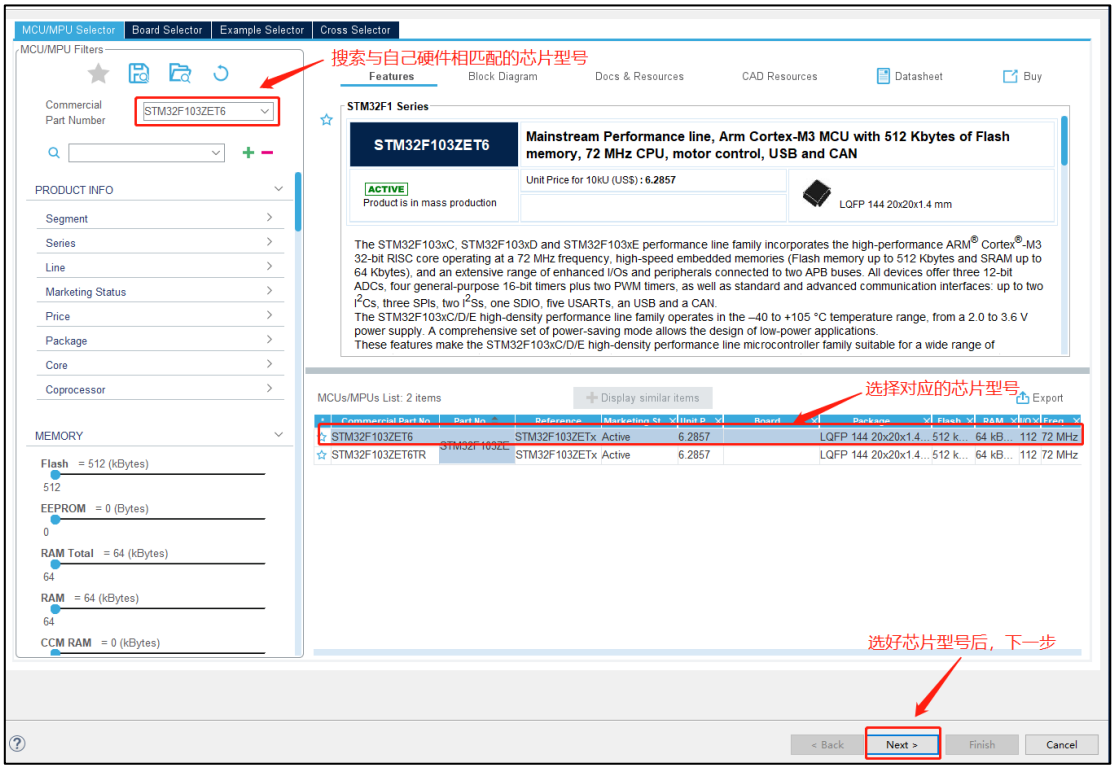


图 4.1.1.5 选择对应的芯片型号

接着进入 CubeIDE 工程设置界面：

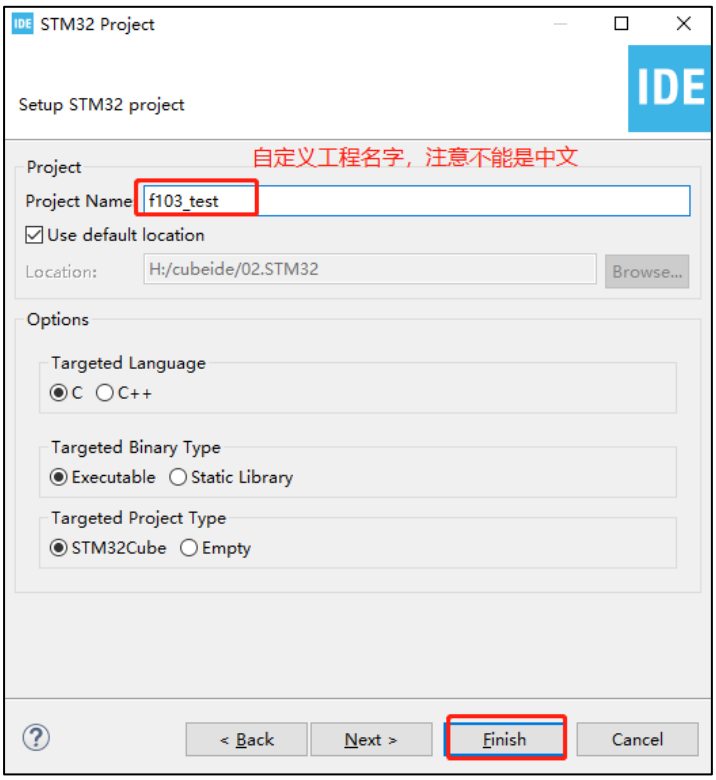


图 4.1.1.6 工程设置界面

工程设置界面，包括设置工程名称、开发语言等选项。需要注意的是，工程名称不能包含中文字符，否则可能会导致软件无法正常运行，其余选项一般默认即可，然后点击 Finish 后弹出一个选择打开此透视图的界面（其实就是工程界面），我们选中 Yes 选择打开，可以勾选 Remember my decision 选项框，下次就不会再跳出此提示界面。

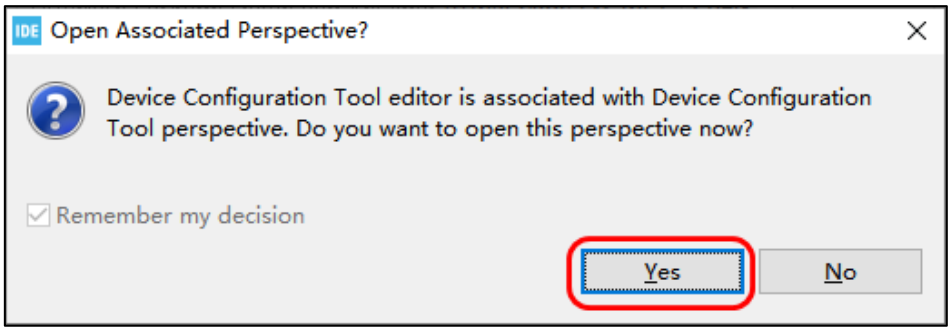


图 4.1.1.7 打开透视图界面

接着就来到如下界面：

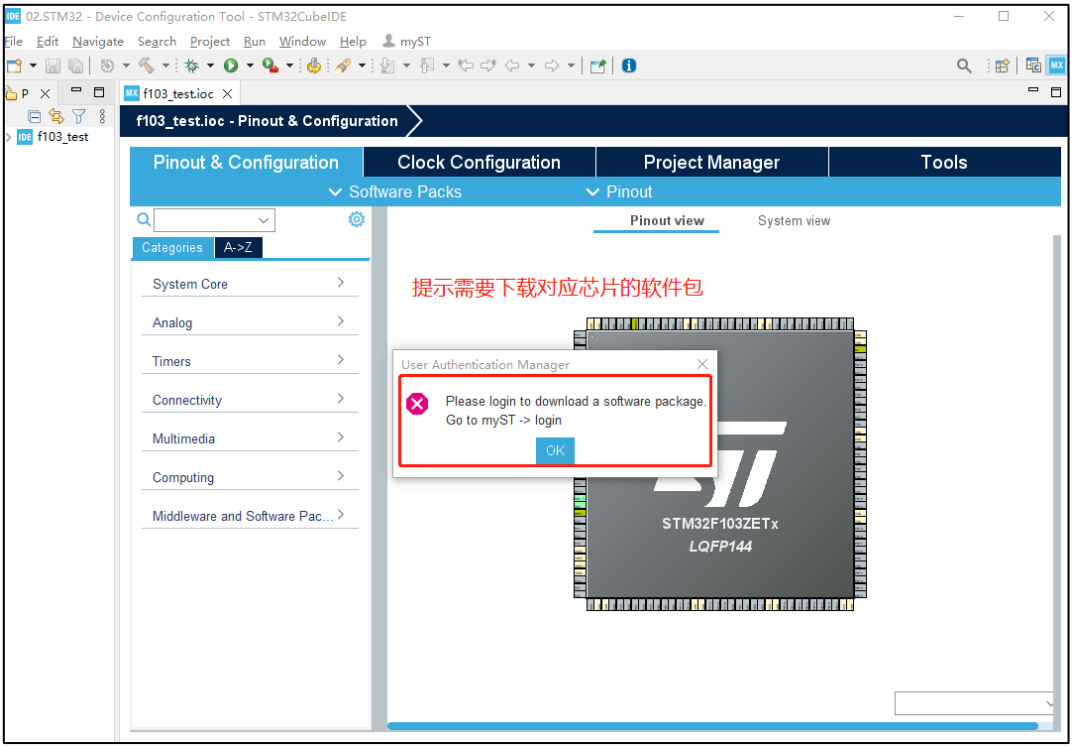


图 4.1.1.8 软件包下载提示

第一次打开该软件时，由于芯片的软件包我们是没有下载的，所以会提示请让我们登录账号后下载软件包，我们直接点击 OK 即可，在后续操作中我们再进行下载。

图 4.1.1.9 功能配置

图 4.1.1.9 的界面功能和 STM32CubeMX 的功能窗口完全相同，可以在该界面进行相应的外设功能配置、系统时钟配置、工程管理设置等操作。其中：

① **Pinout&Configuration**：外设配置栏用于配置与外设相关的参数，包括 GPIO 的设

置、数字/模拟转换、外设初始化等。其中包含的配置项非常丰富，可以进行多种配置操作，例如第三方组件和实时操作系统的移植等。

（2）设置时钟来源

在该项首先我们需要设置 RCC 时钟源，这里我们设置一下高低速时钟源均由外部晶振产生，如下图：

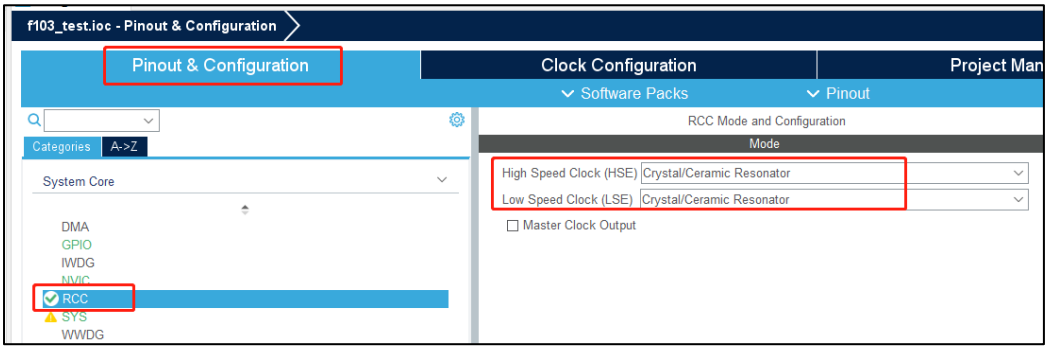


图 4.1.1.10 外设配置栏界面

（3）GPIO 配置

由于我们本章的目的，是要实现精英开发板的 LED0（PB5）和 LED1（PE5）每 500ms 翻转一次，所以还需要配置下 GPIO 的模式以及上下拉等，如下图：

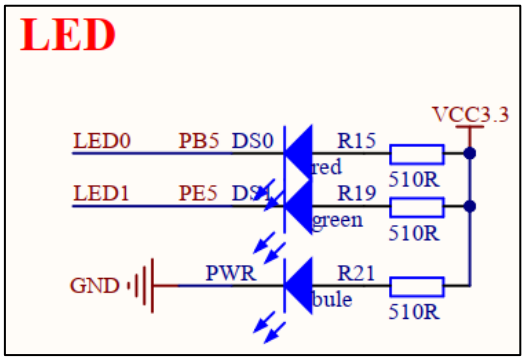


图 4.1.1.11 精英开发板 LED 原理图

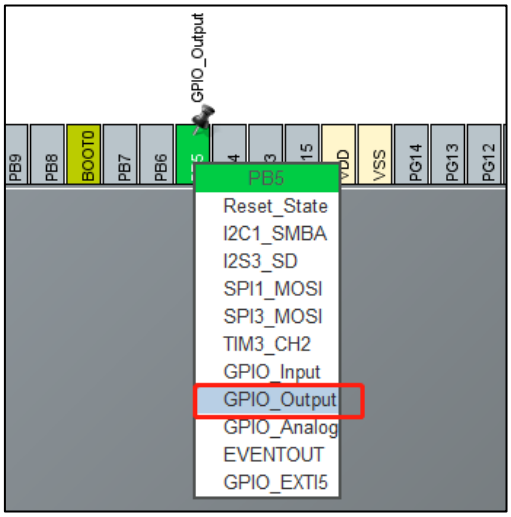


图 4.1.1.12 配置 IO 为输出模式（以 PB5 为例）

本章使用了板载的 LED0 和 LED1，这些 LED 已经连接到开发板上，不需要进行额外的

操作。由于这两个 LED 的一端均被上拉，因此只需要将另一端连接到低电平即可点亮，而将其连接到高电平则会熄灭。所以依据这个特性，我可以设置这两个 IO 的默认输出电平为高（默认熄灭状态），如下：

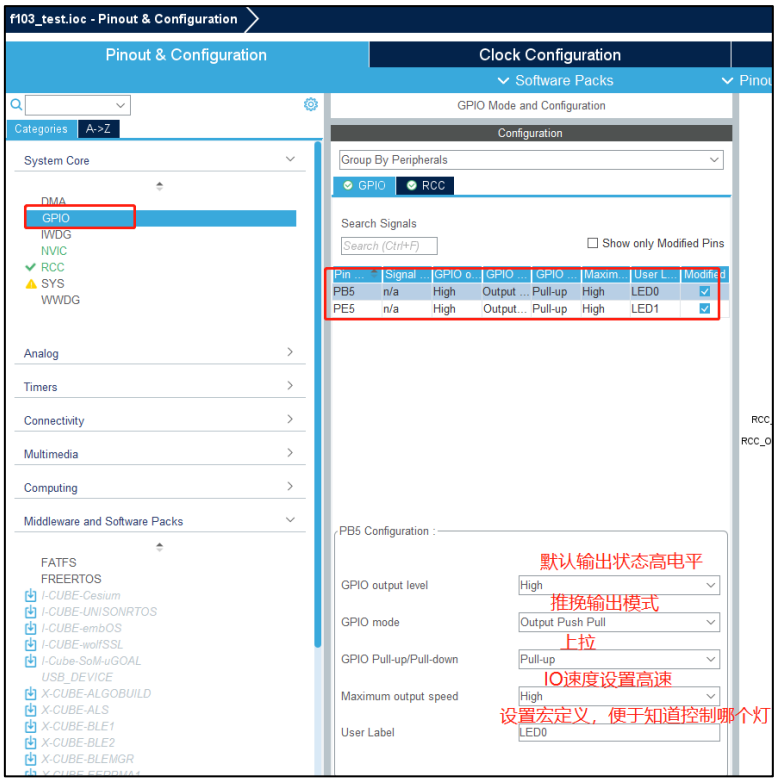


图 4.1.1.13 IO 配置

（4）仿真调试配置

GPIO 配置好后，接着还需配置仿真调试接口。在这里打开 SYS 选项，并进行以下设置：

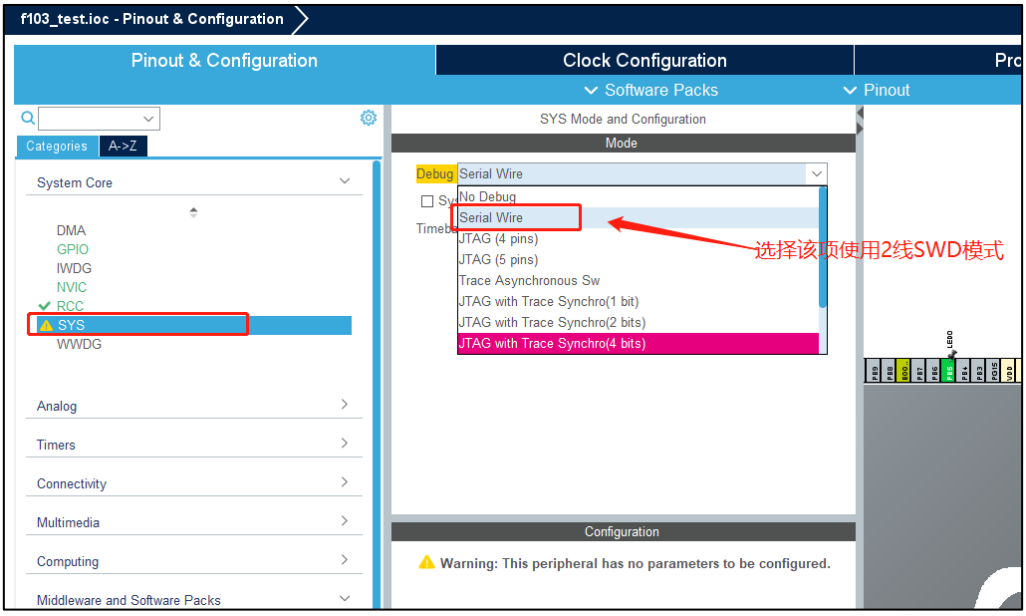


图 4.1.1.14 调试接口设置

STM32 一般支持串行接口和 JTAG 接口两种调试方式。JTAG 调试接口（JTAG-DP）需

要 5 个引脚，而 SW 调试接口（SW-DP）只需要 2 个引脚，两者之间存在复用引脚。在 CubeIDE 中，可以选择 5pin 或 4pin 的 JTAG 接口，其中 5pin 接口比 4pin 多了一个复位引脚。SW 调试端口采用同步串行协议，SWCLK 引脚为从主机到从机的时钟信号，SWDIO 为双向数据信号。考虑成本效益，当采用引脚较少 SW 调试端口的方案显然具有更高的性价比。

（5）时钟树配置

② **Clock Configuration**：该选项为时钟树的配置图，可以直接选择各个节点的分频系数来确定外设的时钟频率。对于初学者来说，时钟树的配置可能会比较复杂，但该选项提供了直观的图形表示，使得配置变得非常方便。在本例中，我们以精英开发板为例，由于板载的外部高速晶振为 8M，且官方推荐主频时钟为 72MHz，因此具体设置如下图所示。

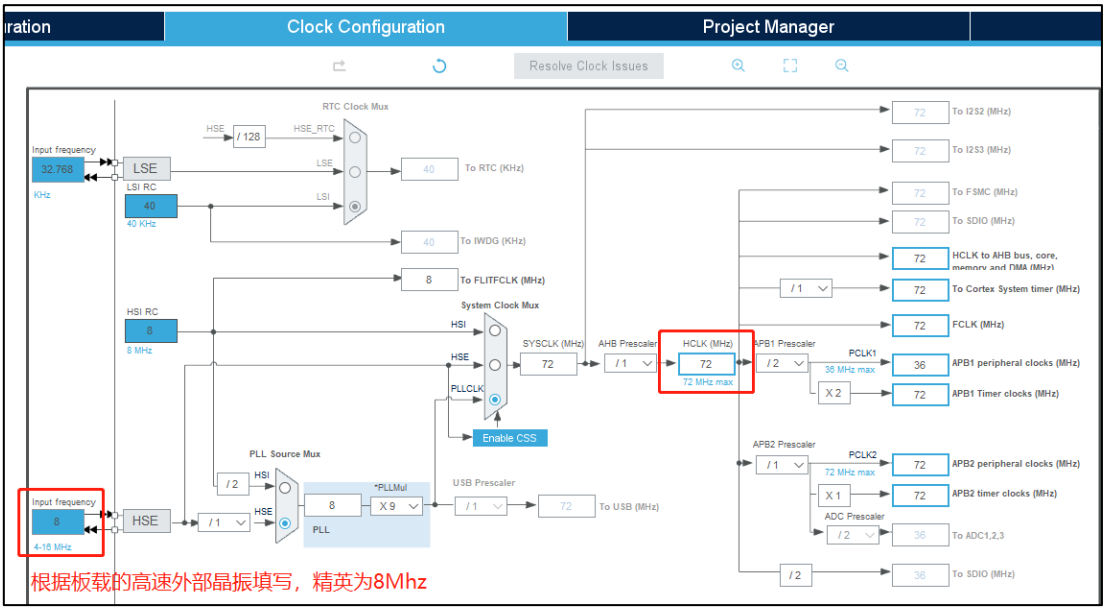


图 4.1.1.15 时钟树的配置

（6）工程配置

③ **Project Manager**：在工程设置选项中，通常只在创建新工程时进行配置，因为这些设置一般不会改变，而且容易被忽略。以下是一些常见的配置点，其他地方可以不用过多关注。

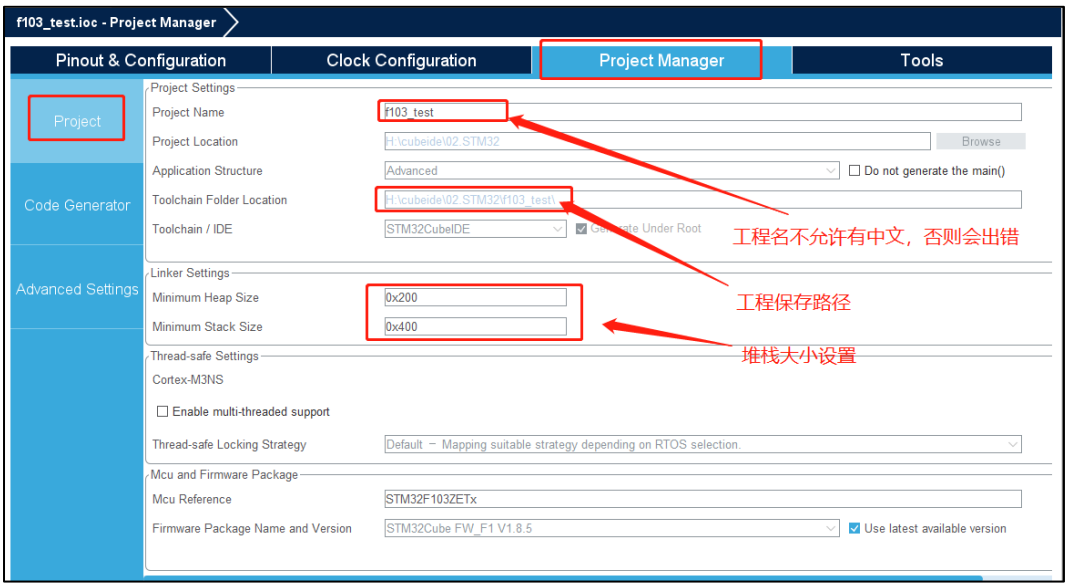


图 4.1.1.16 工程设置界面

图 4.1.1.17 文件格式设置

④ **Tool**：在工具选项中，主要用于计算芯片功耗，该功能相对较少使用，一般直接默认设置即可，如下：

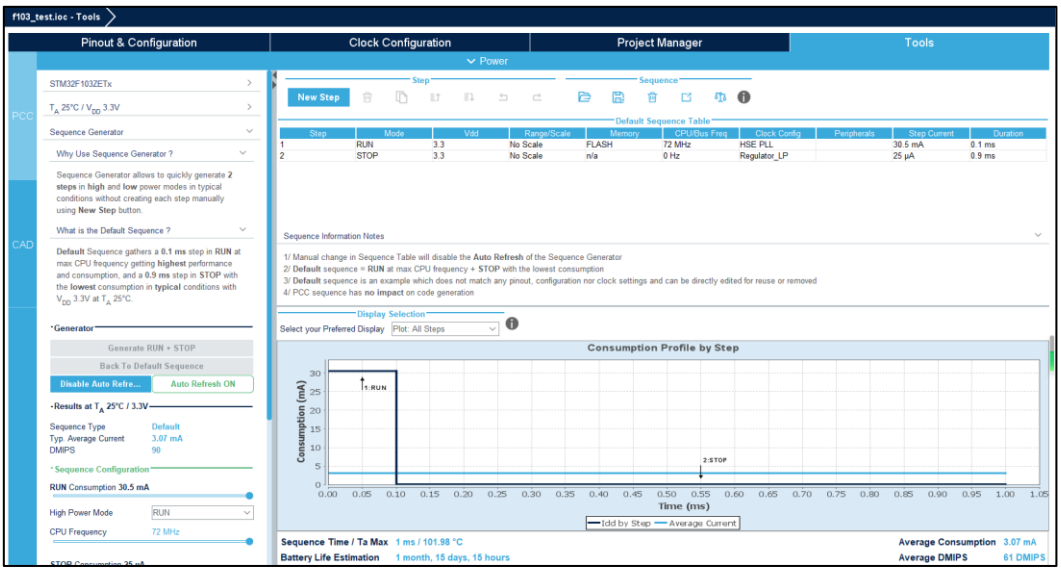


图 4.1.1.18 工具界面设置

（7）生成工程

完成以上配置后，我们可以直接点击 按钮生成相应的代码文件。

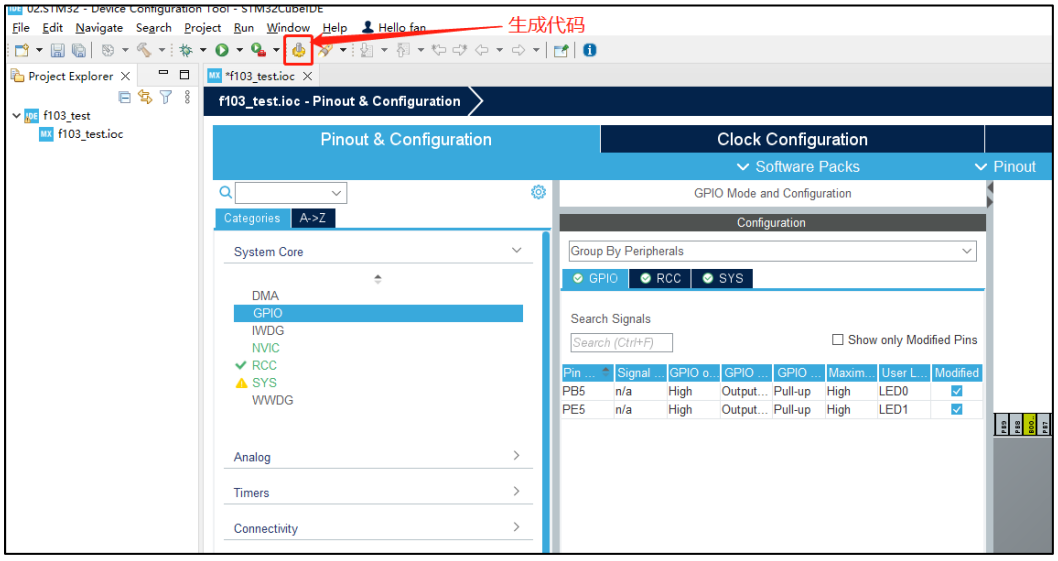


图 4.1.1.19 生成代码

（8）安装固件包

当我们首次使用 CubeIDE 生成代码时，如果该软件发现尚未安装与所选芯片相对应的软件包，则会自动下载并安装该软件包（下载时间过长，需要耐心等待）。下载完成后，我们就可以开始生成代码文件了。如下图：

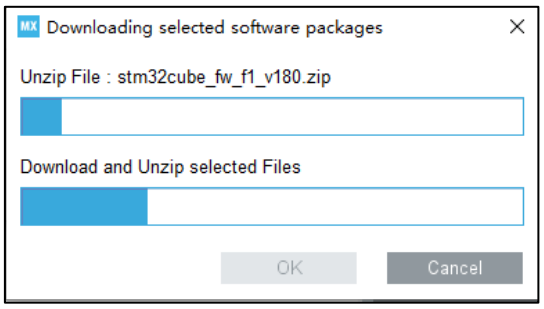


图 4.1.1.20 下载软件包

如果网络不好，下载时间过长，可以在 A 盘资料中，找到我们已经下载好的 Cube 软件包，直接安装。如下图，点击 Help-->Manage embedded software packages：

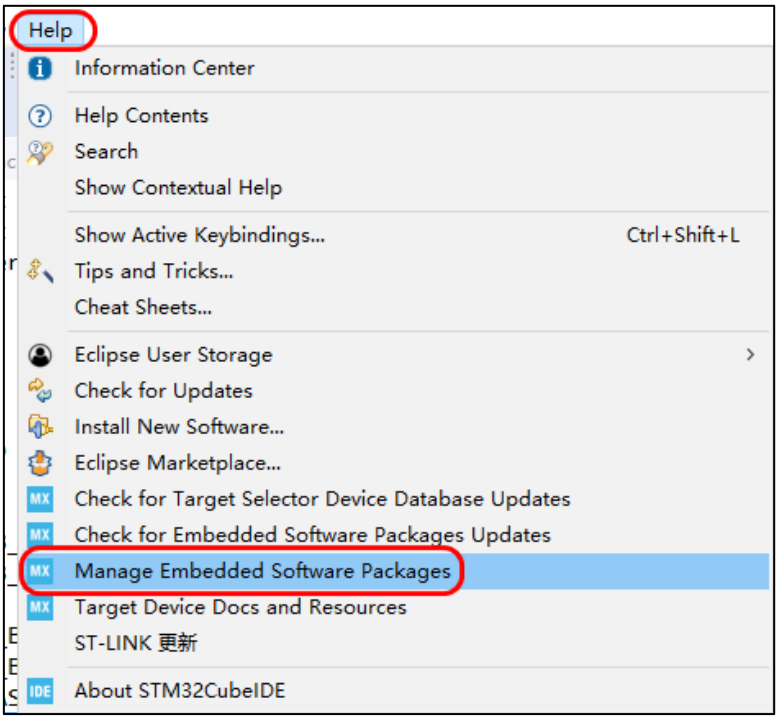


图 4.1.1.21 管理固件包

然后弹出管理界面，在该窗口找到 STMF1 列表选项，勾选 1.8.5 版本（目前最新版本是 1.8.5 版本，也可以选择自己需要的版本）。关联 STM32Cube 固件包有两个方法，如下图所示。

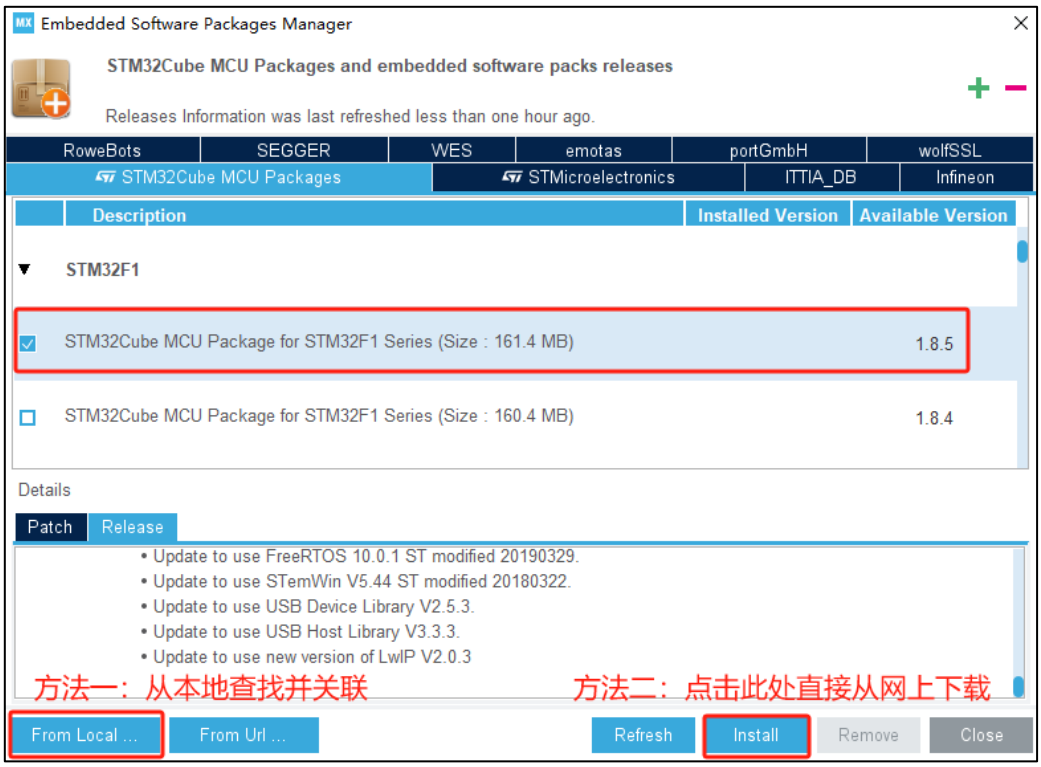


图 4.1.1.22 下载和关联 STM32Cube 固件包

方法二，就是我们前面说讲到的生成代码时，当软件检测到还未下载相应的软件包时，会自动下载 Cube 包，在该界面中，用户可以自由选择所需下载的 Cube 软件包版本号。

如果是方法一：点击后，弹出下面的窗口，然后选择光盘中的对应的固件包，注意这里是以压缩包的形式来选择的固件包版本，如下图所示。

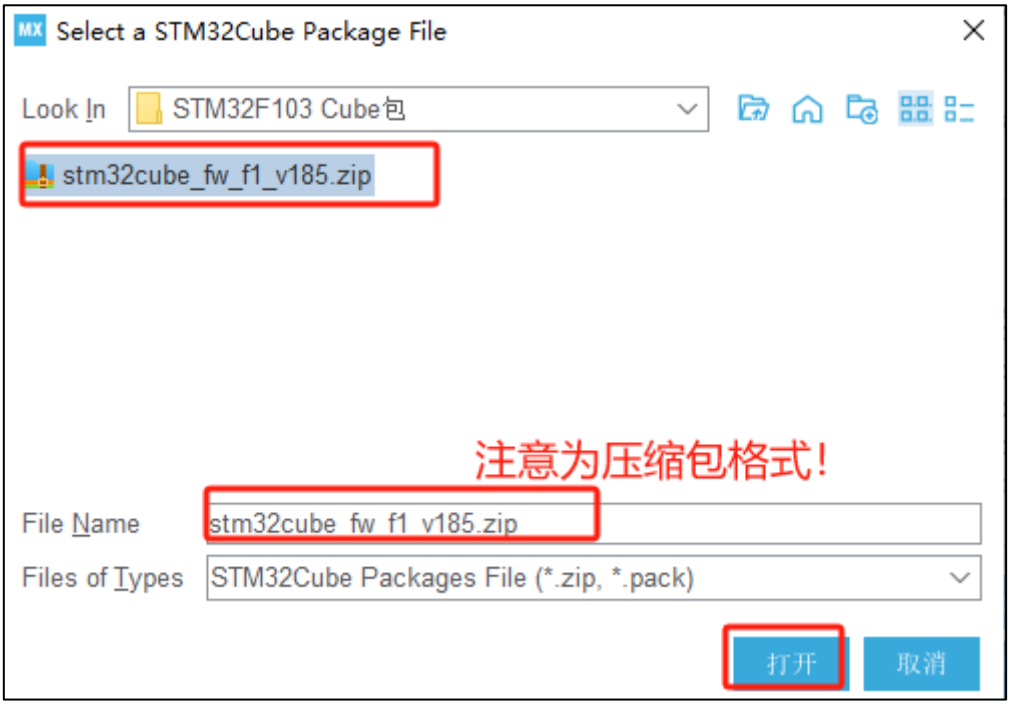


图 4.1.1.23 关联本地 STM32Cube 固件包

关联成功后，软件包界面的对应选项框，会变成绿色，如下图所示：

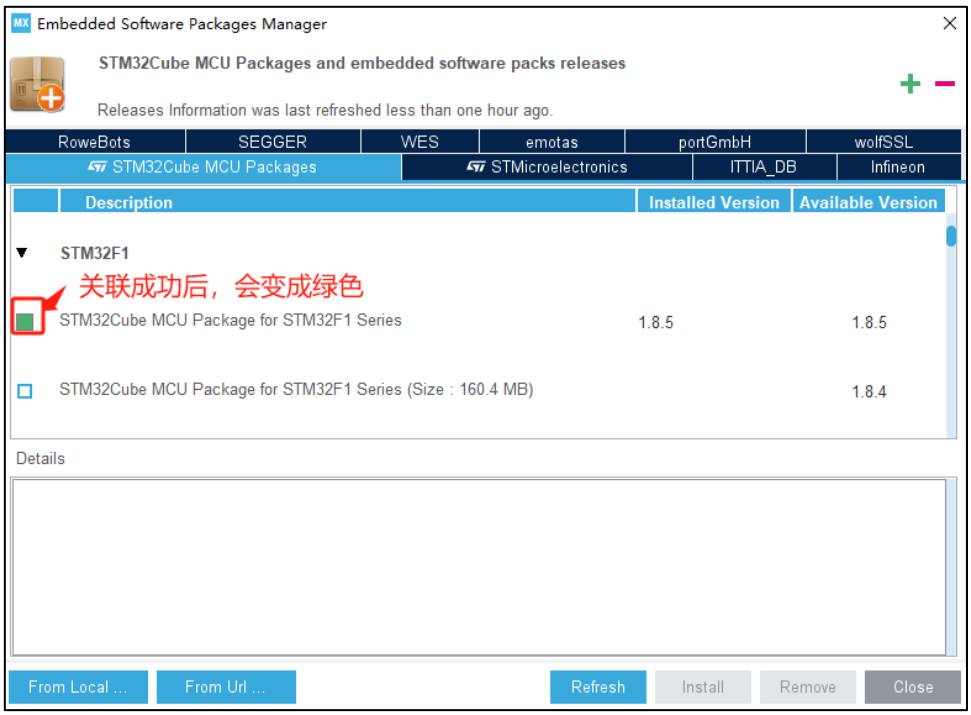


图 4.1.1.24 关联成功

关联成功后，就可以正常生成代码了!!

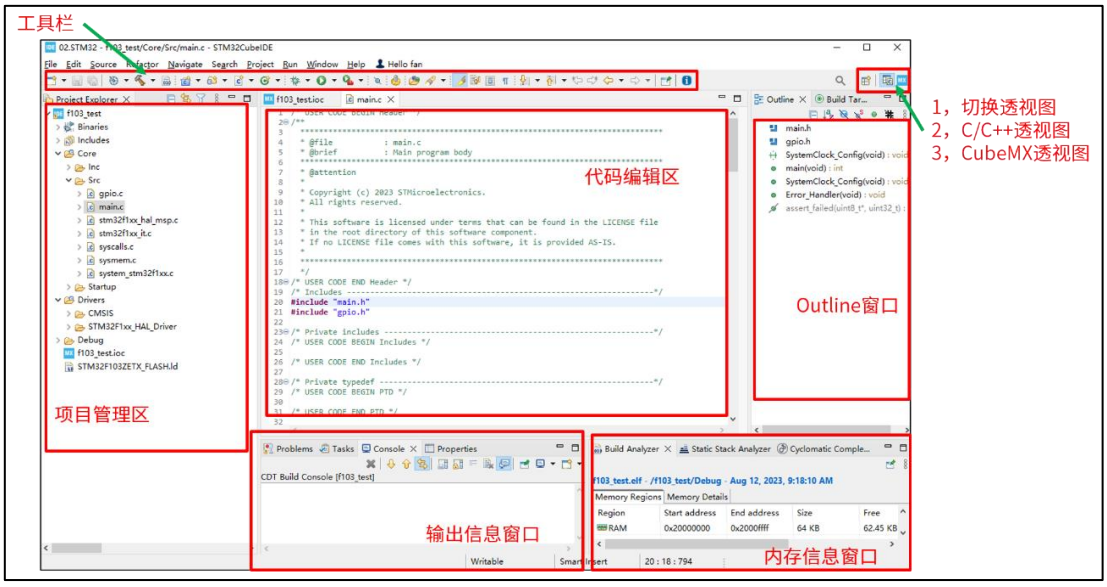


图 4.1.1.25 代码生成后的窗口界面

图 4.1.1.25 是代码生成后的窗口界面，以下是 CubeIDE 的功能窗口的介绍：

- 工具栏：主要包含了一些常用的功能按钮，用于快速访问和操作 CubeIDE 中的各种功能，在后续章节中，我们将详细介绍工具栏的各个功能。
- 项目管理窗口：用于管理嵌入式系统的项目，包括添加、删除、修改文件等操作。
- 代码编辑窗口：用于编辑嵌入式系统的源代码，支持多种编程语言，如 C、C++等，也是用户实现代码逻辑的编写窗口。
- 编译输出窗口：用于显示嵌入式系统的编译生成的代码和链接库的输出信息，如生成的 HEX 文件或 ELF 文件的内容以及编译结果等信息。
- 内存信息窗口：用于查看嵌入式系统的内存信息，包括内存使用情况，内存信息

等。

- Outline 窗口：用于显示当前打开的源文件或工程文件的文件结构和代码结构。Outline 窗口可以帮助用户更好地管理和组织代码，方便用户查看代码的结构和关系，以及快速定位代码中的各个部分。

4.1.2 添加用户驱动代码

在前面，我们使用 STM32CubeIDE 生成了基于 HAL 库的 GPIO 初始化代码，还没有控制 LED0 和 LED1 点亮和熄灭的逻辑代码，接下来我们在这个工程的基础上去手动完成这部分内容。

下面直接基于 HAL 库添加控制逻辑代码，实现精英开发板的 LED0 和 LED1 每 500ms 翻转一次。

我们在 main.c 函数的 while 循环函数里添加如下代码，**注意代码添加的位置，要添加在成对出现的 BEGIN 和 END 的区域之间，这样可以避免下次修改.ioc 文件以后再重新生成初始化代码的时，自己额外添加的代码被清空了**。

```
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------
*/

  /* Reset of all peripherals, Initializes the Flash interface and the
Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
      HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET);
      HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
      HAL_Delay(500);
      HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET);
      HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
      HAL_Delay(500);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

在程序中，经常会看到由 BEGIN 和 END 组成的注释对，需要注意的是，这些注释对是用来限定用户程序的。当 CubeIDE 检测到相应的标识时，它会保持用户程序段内容不改变，从而保存用户程序。只改变 CubeIDE 支配的程序段内容。

切记不要在注释对之外的地方写程序，否则，在重新生成工程时，程序将会全部丢失，除非您已经做好了备份。此外，在外设初始化的.c 文件（例如初始化 SPI 生成的 spi.c 文件）中，在重新生成工程时，请勿关闭相关外设。否则，相应的外设初始化.c 文件将丢失，而.c 文件内的程序也将丢失。

4.1.3 编译工程

到目前为止，我们已完成了软件的编写工作。接着可点击工具栏的小锤子 编译程序，编译过程将会在下方的 Consle 窗口输出：如下：

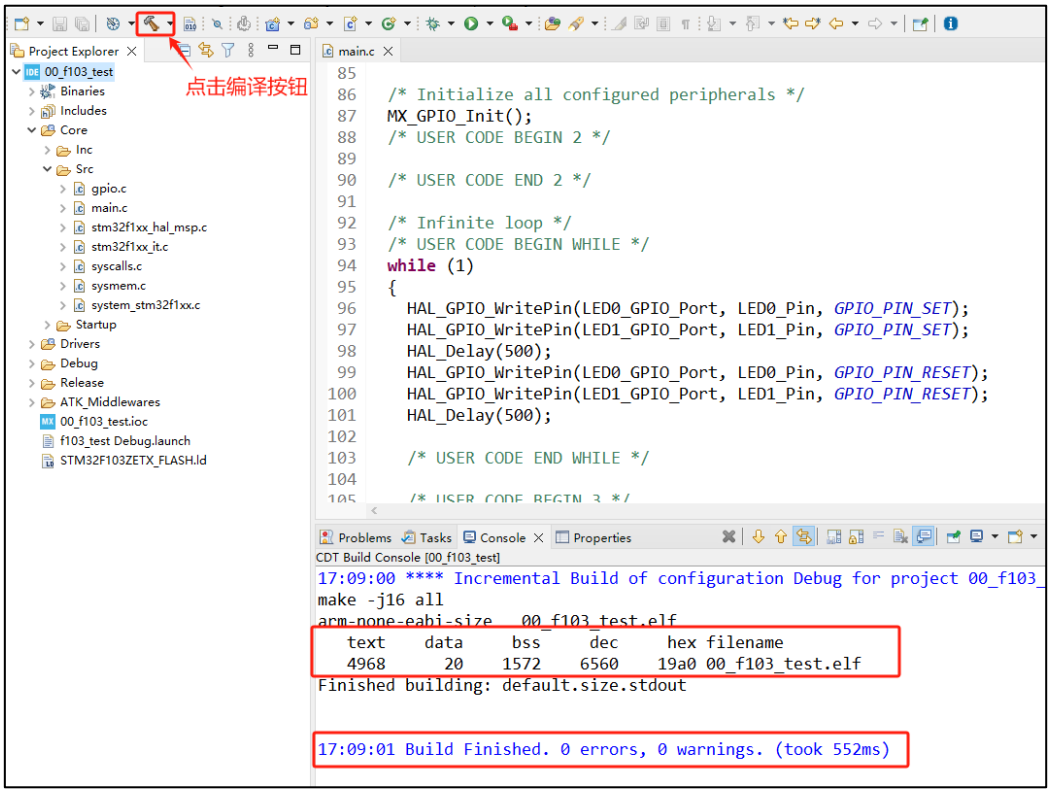


图 4.1.3.1 编译程序

编译结束后提示 Build Finished. 0 errors，0 warnings，没有报错，这里我们解释一下，编译结果里面的几个数据的意义：

text 是代码段，用来存放程序执行代码的一块内存区域。

data(data segment)段，是已经初始化的变量，属于静态内存分配。

bss(block started by symbol)段，是未初始化的全局变量，属于静态内存分配。

dec(decimal 的缩写，即十进制数)，是 text，data 和 bss 的算术和。

hex 就是十六进制的意思，其对应的数值是 dec 的十六进制格式。

以上是在程序没有运行前（程序没有加载到内存前）的 3 段信息。

程序的大小就是 text 代码段的大小。

综上可知编译 0 错误，0 警告，代表程序语法没有任何问题。下一节我们将开始进行程序的下载和调试，验证程序功能是否正常。

4.1.4 下载验证

1. 下载配置

开发板通过 JTAG 线和 ST-Link 连接（如果您使用的是 DAP 下载器可查看 4.3 小节），ST-Link 另外一端接电脑的 USB 接口。插上配套电源线给开发板供电。如图 4.1.4.1 所示。

图 4.1.4.1 开发板和 ST-Link 连接图

① STLINK 通过 USB 线连接电脑，且仿真器的红灯常亮（如果红灯闪烁，说明没有安装驱动，请安装仿真器驱动）。然后用 20P 的灰排线连接到开发板的 JTAG 座上。

② 确保开发板已经正常供电，蓝色电源灯亮起。

③ 确保 BOOT0 和 BOOT1 均接 GND。

硬件接好以后，开发板上电，点击 STM32CubeIDE 菜单栏的下的 Run→Run Configurations...进入下载配置界面。

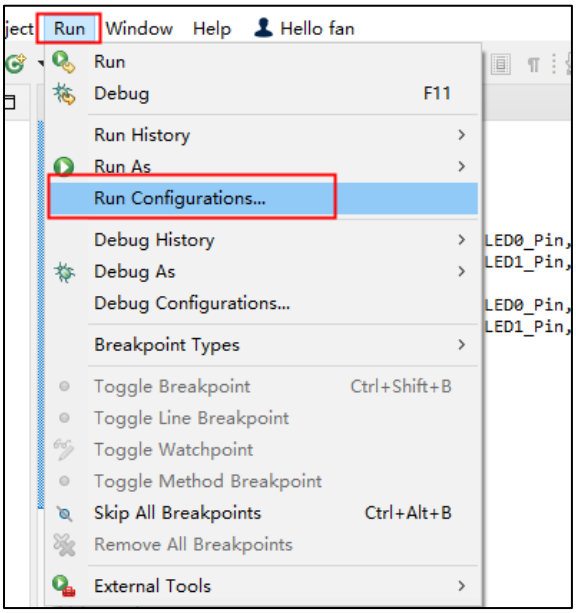


图 4.1.4.2 进入下载配置界面

进入的 Run Configurations...界面如下图。

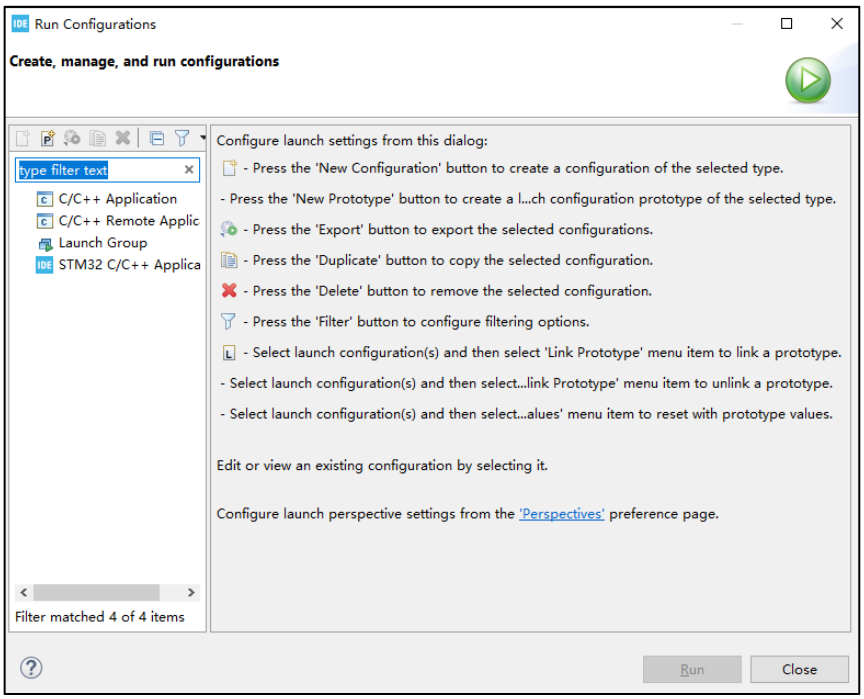


图 4.1.4.3 下载配置界面

下面，我们新增一个 Debug 配置项：

双击图 4.1.4.4 中①处的 STM32 Cortex-M C/C++ Application 就会自动创建和打开名为 f103_test Debug 的配置项。②处会默认选择 Debug 下编译生成的 f103_test.elf 文件，如果没有自动选择对应的.elf 文件，可以再点击右边选择框搜索对应文件，注意的是，此项不能空，因为我们下载以及调试的时候需要将对应的.elf 文件下载到目标板卡的 FLASH 中运行。如果工程没有编译成功，将搜不到.elf 文件。

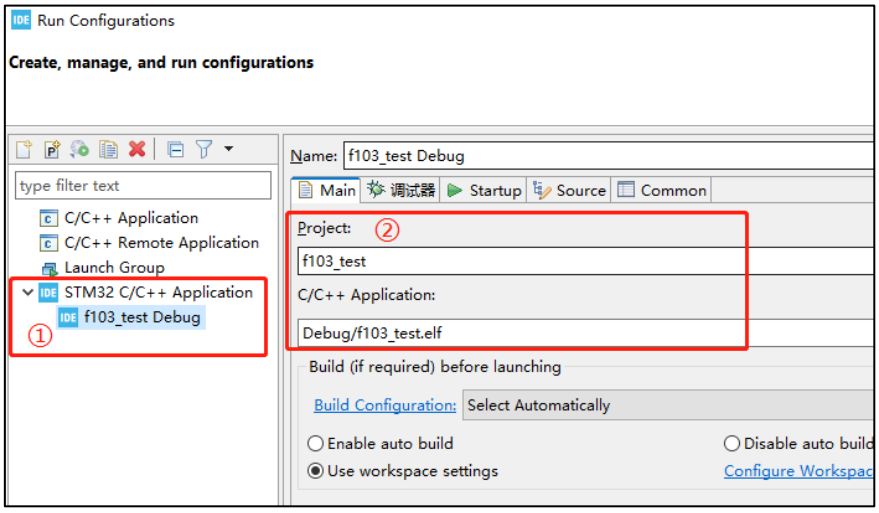


图 4.1.4.4 新增一个 Debug 配置项

接着，在这个界面我们先配置一下调试选项，点击调试器，进行如下配置：

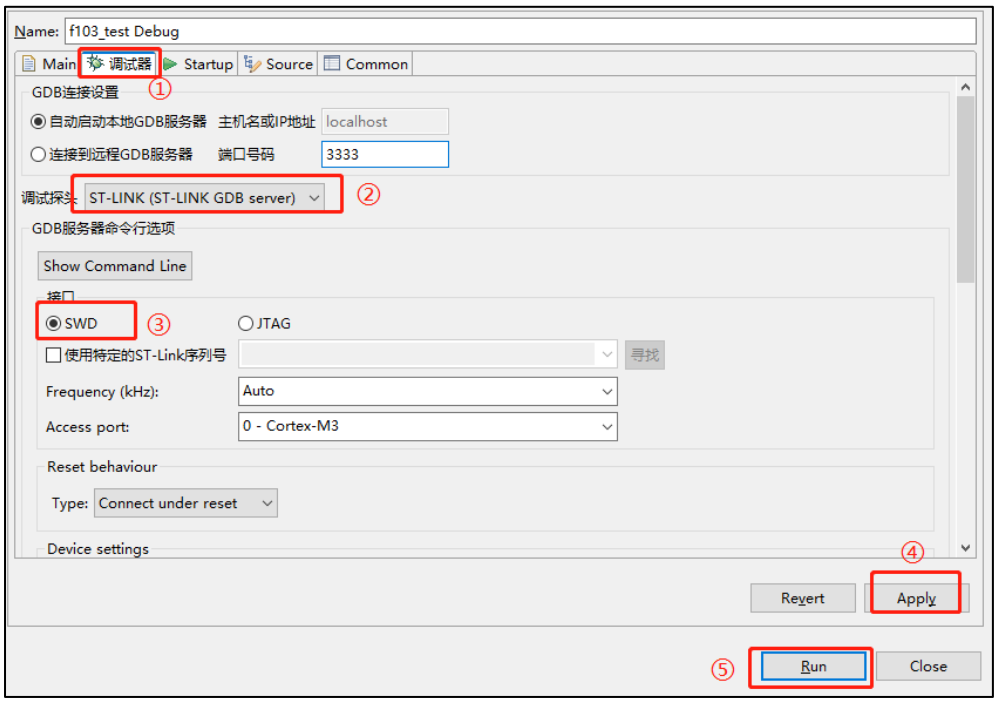


图 4.1.4.5 配置下载选项

调试探头选择 ST-LINK (ST-LINK GDB server)，我们用 ST-Link 来进行调试。下载接口，一般是根据实际接线，如果用的是 SWD 接口，则选中 SWD，若是 JTAG，则选 JTAG，我们这里选择 SWD，其它选项保持默认不变，然后点击 Apply→Run 开始下载。

2. 更新 ST-LINK 固件

第一次在 STM32CubeIDE 用 ST-Link 下载程序的话，系统会提示更新 ST-LINK 固件，点击 Yes 选择更新：

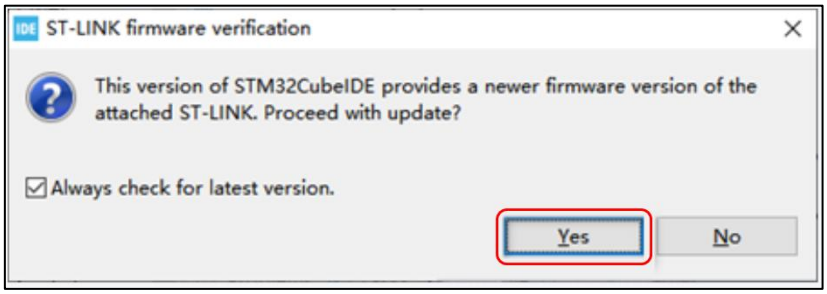


图 4.1.4.6 系统提示更新 ST-LINK 固件

弹出如下 STLinkUpgrade 对话框（本实验用的是 ST-Link，如果是 J-Link 的话会提示下载更新 J-Link 的的驱动）：

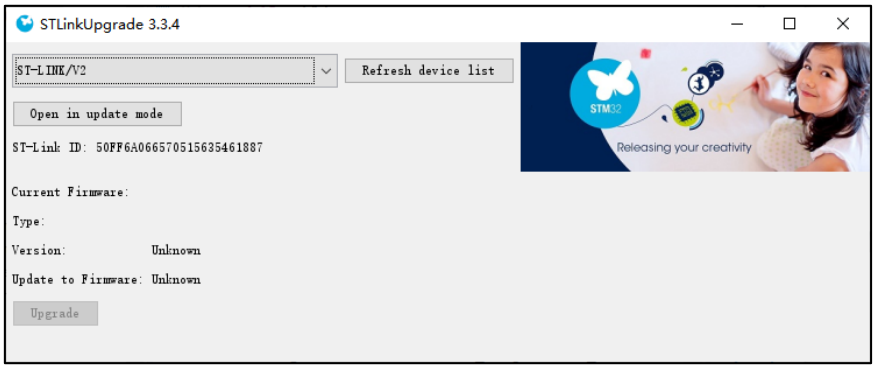


图 4.1.4.7 STLinkUpgrade 对话框

我们先拔掉然后再重新连上连接下载器的 USB 线，接着点击 Open in update mode，弹出如下界面（如果不这样操作，点击 Open in update mode 以后可能提示 ST-Link is not in the DFU mode.Please restart it，即要求复位一下下载器。）：

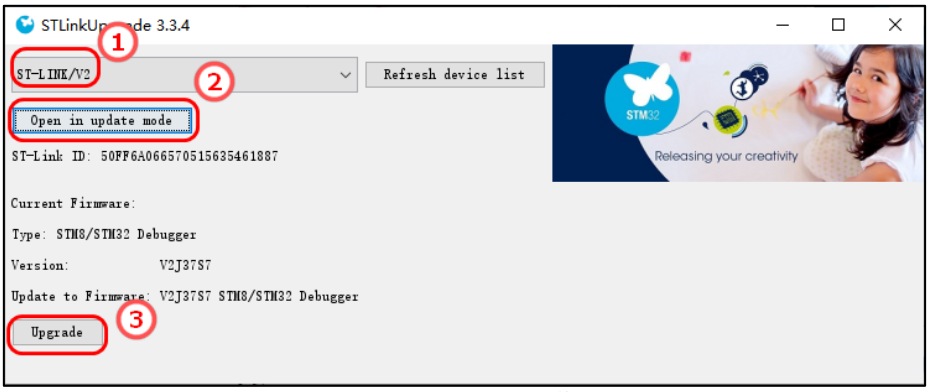


图 4.1.4.8 驱动更新操作顺序

点击 Upgrade 更新 ST-Link 固件，等待更新进度条加载完成就更新成功了。也可以直接在菜单栏的 Help 处进行更新，点击 Help→ST-LINK 更新同样也可以进入前面的 STLinkUpgrade 对话框，后续的操作和前面的一样。

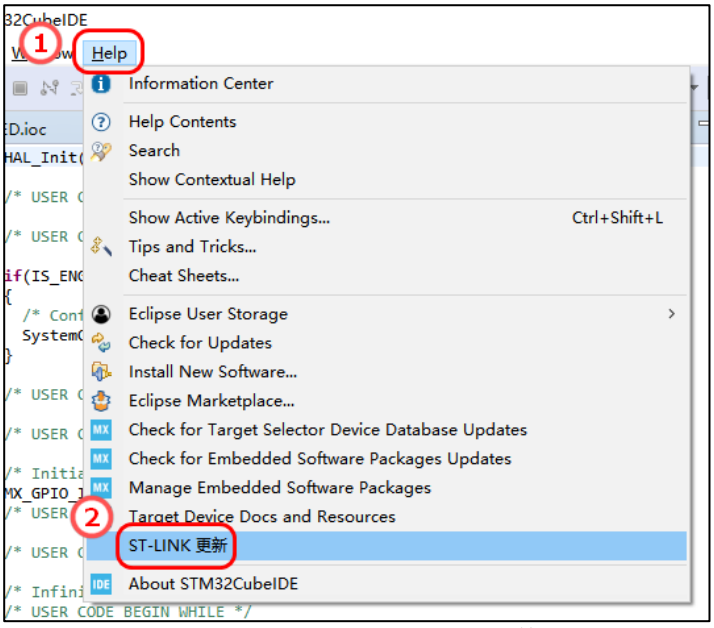


图 4.1.4.9 手动更新 ST-Link 固件

3. 程序下载

更新完 ST-Link 固件后，回到工程界面，点击 Run→Run Configurations→Run 下载程序，或直接点击菜单栏的播放图形 ，直接下载程序！

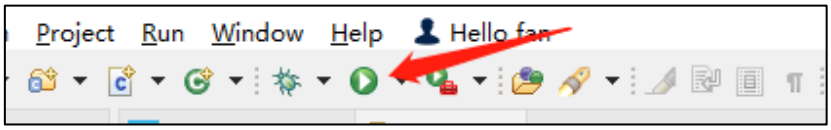


图 4.1.4.10 下载程序

如下在 Console 窗口，提示 Download verified successfully 代表下载成功！

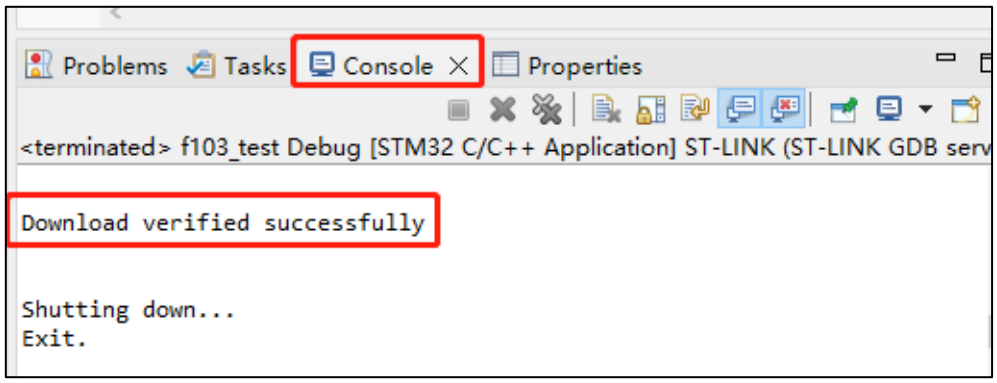


图 4.1.4.11 下载成功提示

可观察到，此时我们的精英开发板上的 LED0 和 LED1 会每 500ms 闪烁一次！代表我们的实验成功！

图 4.1.4.12 下载验证

4.1.5 仿真调试

在经过上一小节的配置后，我们到工程界面，点击 Run→Debug Configurations→Ddebug 进行调试或者可以直接点击菜单栏中的小爬虫 进入仿真调试界面，会出现如下提示是否切换到 Debug 透视图，也就是调试的界面，点击 Switch 进行切换，勾选左边的 Remember my decision，下次就不会再弹出此提示。

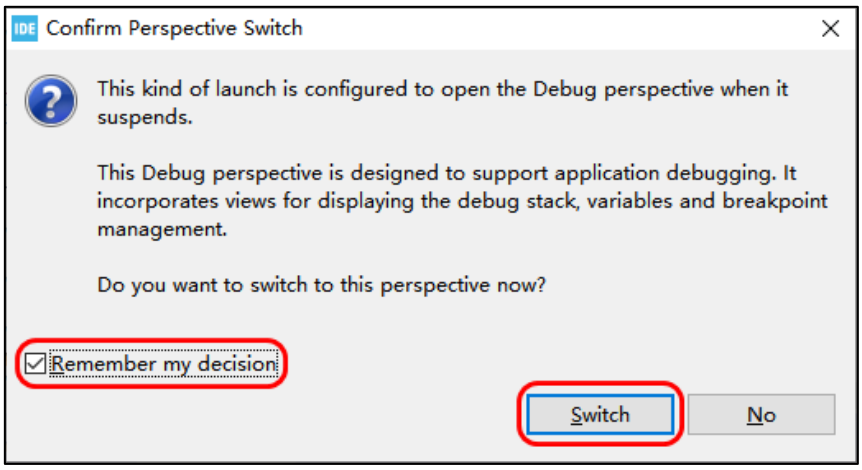


图 4.1.5.1 切换到 Debug 透视图

调试界面如下，我们介绍界面上常用的一些功能：

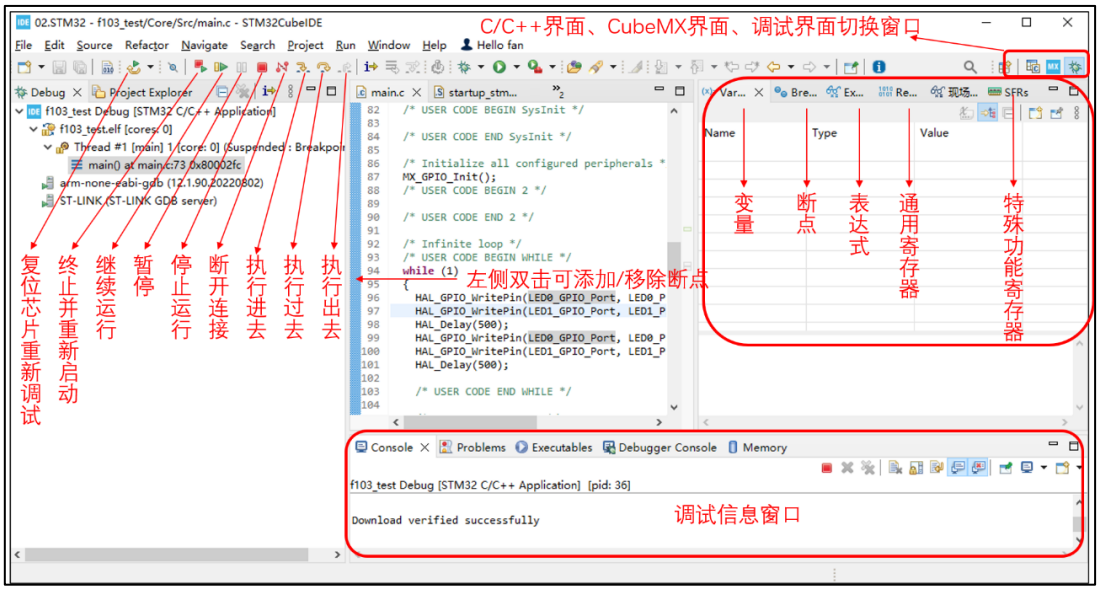


图 4.1.5.2 调试界面介绍

我们点击继续运行按钮来运行调试，可以看到开发板底板的 LED0 和 LED1 没 500ms 闪烁一次，点击即可终止调试并退出调试界面。

接下来，我们将介绍调试窗口的主要功能：

复位芯片重新调试：其功能等同于硬件上按复位按钮。相当于实现了一次硬复位。按下该按钮之后，代码会重新从头开始执行。

终止并重新启动：终止当前正常执行的程序，退出调试功能，然后重新下载程序进入调试功能。

继续运行：该按钮用来快速执行到断点处，有时候你并不需要观看每步是怎么执行的，而是想快速的执行到程序的某个地方看结果，这个按钮就可以实现这样的功能，前提是你在查看的地方设置了断点。

暂停：此按钮在程序一直执行的时候会变为有效，通过按该按钮，就可以使程序停止下来，进入到单步调试状态。

停止运行：退出调试功能。

断开连接：断开仿真器和开发板的连接，与停止运行有点类似。

执行进去：该按钮用来实现执行到某个函数里面去的功能，在没有函数的情况下，是等同于执行过去按钮的。

执行过去：在碰到有函数的地方，通过该按钮就可以单步执行过这个函数，而不进入这个函数单步执行。

执行出去：该按钮是在进入了函数单步调试的时候，有时候可能不必再执行该函数的剩余部分了，通过该按钮就可以一步执行完该函数的余部分，并跳出函数，回到函数被调用的位置。

C/C++界面、CubeMX 界面、调试界面切换窗口：可以通过该窗口切换到对应的透视界面。

Debug 工具条上的其他几个按钮用的比较少，我们这里就不介绍了。以上介绍的是比较常用的，当然也不是每次都用得着这么多，具体看你程序调试的时候有没有必要观看这些东西，来决定要不要看。

4.2 STM32CubeIDE 使用技巧

在前面，我们通过第一个实验初步了解了 STM32CubeIDE 的开发步骤，下面我们对这个 IDE 的开发环境做一个简单的介绍。在以后的学习中，不能纸上谈兵，大家应结合实验多操作，积累更多的实战经验。

4.2.1 STM32CubeIDE 窗口介绍

1. 菜单栏

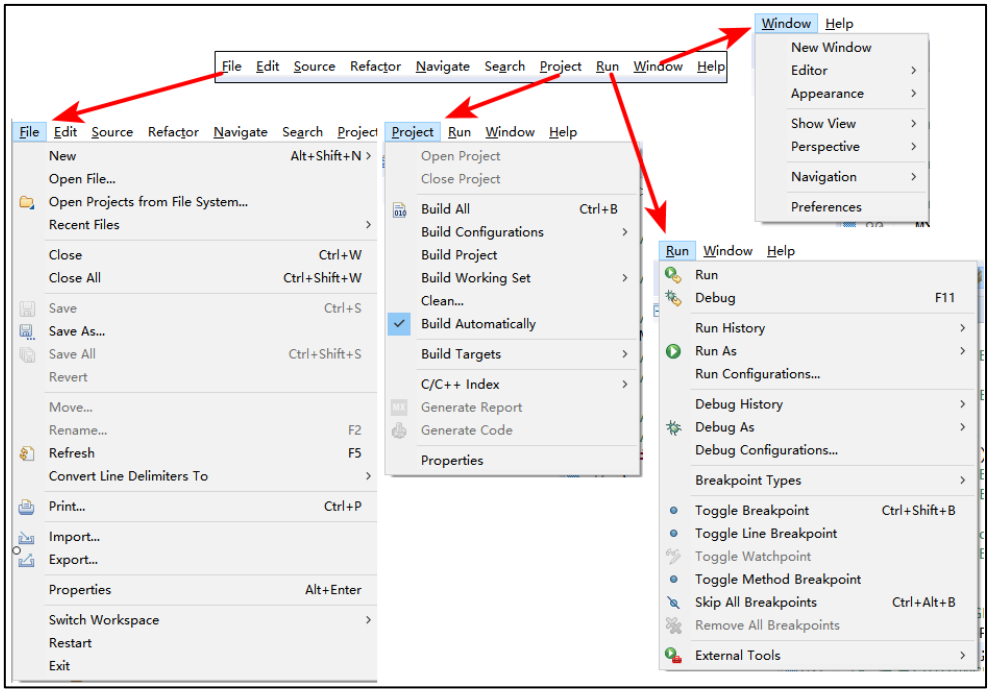


图 4.2.1.1 菜单栏

菜单栏功能简单说明如下表：

菜单	说明
File	新建文件、工程，关闭文件、工程，导入文件、工程，保存、退出工程等操作
Edit	撤销、复制、粘贴、选择、查找、智能能插入等操作
Source	添加、删除代码块注释，代码左移、右移，代码对齐、缩进，格式化代码等操作
Refactor	提取局部变量、常数、函数，构建工程历史记录
Navigate	打开函数声明，查看函数调用关系，退回上一次浏览的位置等操作
Search	查找功能
Project	编译、清除、配置工程
Run	运行、调试、配置工程，添加、取消断点等操作
Windows	打开、隐藏菜单栏，打开某个窗口，窗口风格设置等操作
Help	帮助窗口

表 4.2.1.1 菜单栏功能介绍表

上面介绍的菜单栏我们在实践中会经常用到。如果是第一次使用 STM32CubeIDE，也可以通过 Help 菜单获取更多的帮助信息。例如，通过 Information Center 可以后返回到 Home 界面，然后可以访问 STM32CubeIDE 入门指南，了解 IDE 的使用，访问 ST 支持和社区。此外，还可以通过 Help 菜单检查嵌入式软件包更新、管理嵌入式软件包、更新 ST-LINK 驱动等等。

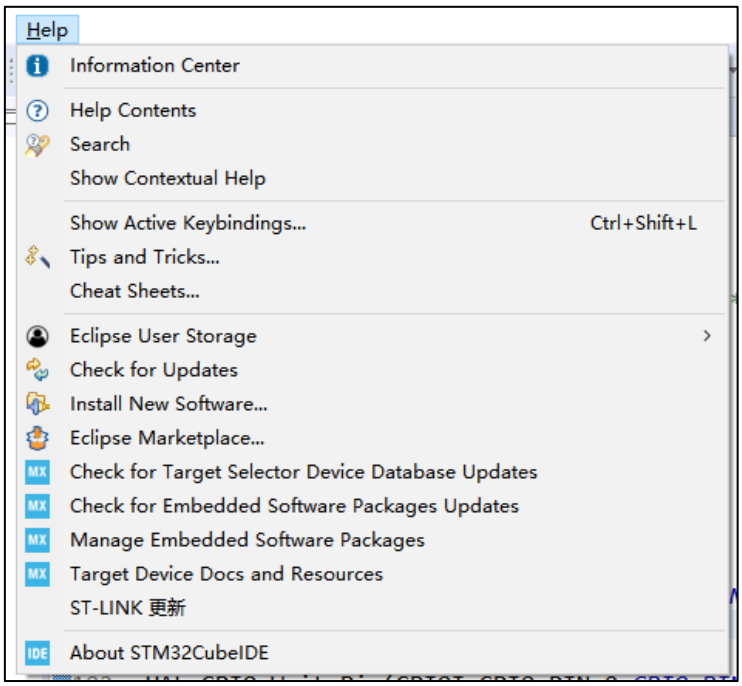


图 4.2.1.2 Help 菜单

2. 工具栏

图 4.2.1.3 工具栏

鼠标放到对应工具选项就会有英文提示，根据提示可以知道此工具的用途。其中，窗口切换工具可以进行窗口切换，例如此时在调试界面，如果想返回 STM32CubeMX 界面，点击 MX 窗口切换图标即可。

3. Debug 窗口

在前面我们已经调试工程，调试窗口如下：

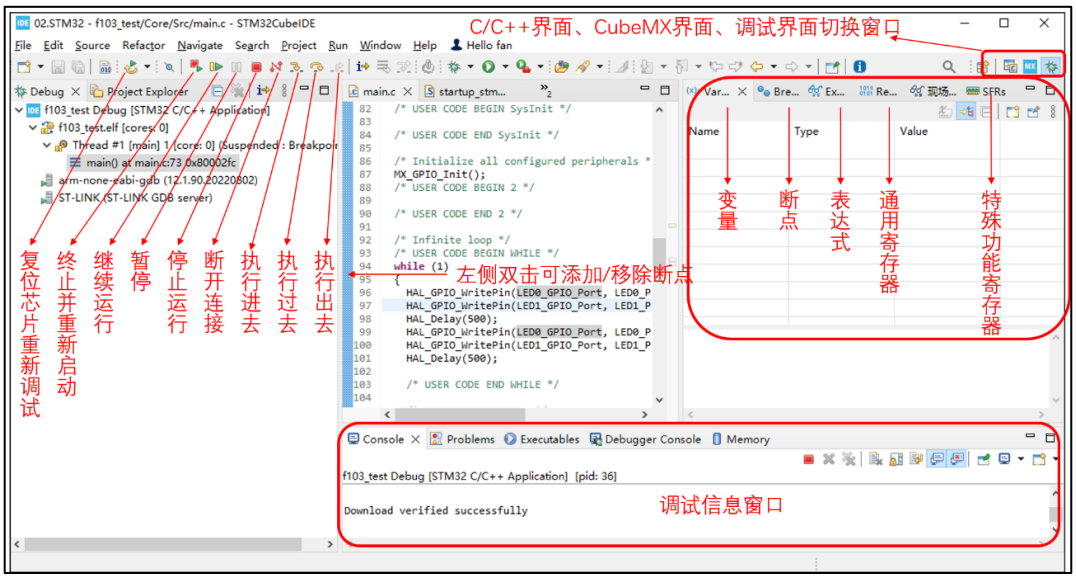


图 4.2.1.4 调试窗口

4.2.2 窗口和文本调整

1. 修改函数窗口字体和大小

如果函数窗口显示的代码的字体太小，想调大一些，可以点击代码界面，然后按下键盘的 Ctrl+“-”来减小字体，按 Ctrl+Shift+“+”可以增大字体。也可以通过点击 Window→preperences→进入 Preferences 界面来修改。

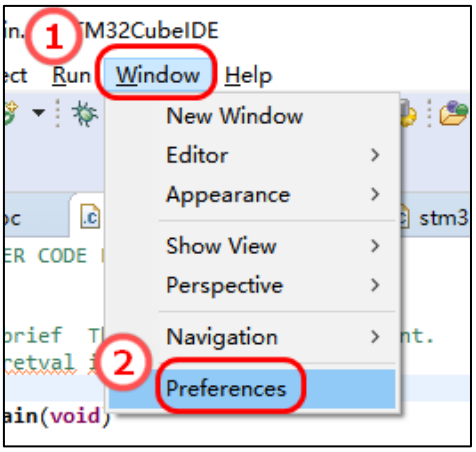


图 4.2.2.1 选择工程属性

进入 Preferences 界面后，点击 General→Appearance→Colors and Fonts→进入 Colors and Fonts 配置选项，里边主要是整个窗口的颜色和字体配置，在选项里的 Basic→Text Front→Edit→选择字体、字形、大小→点击确定→如果效果满意了再点击 Apply 或者 Apply and Close 实现修改，窗口的代码发生变化。如果不想要设置的效果，可以再进入窗口，选择 Restore Defaults 恢复默认值，再点击 Apply and Close 使修改生效并退出。

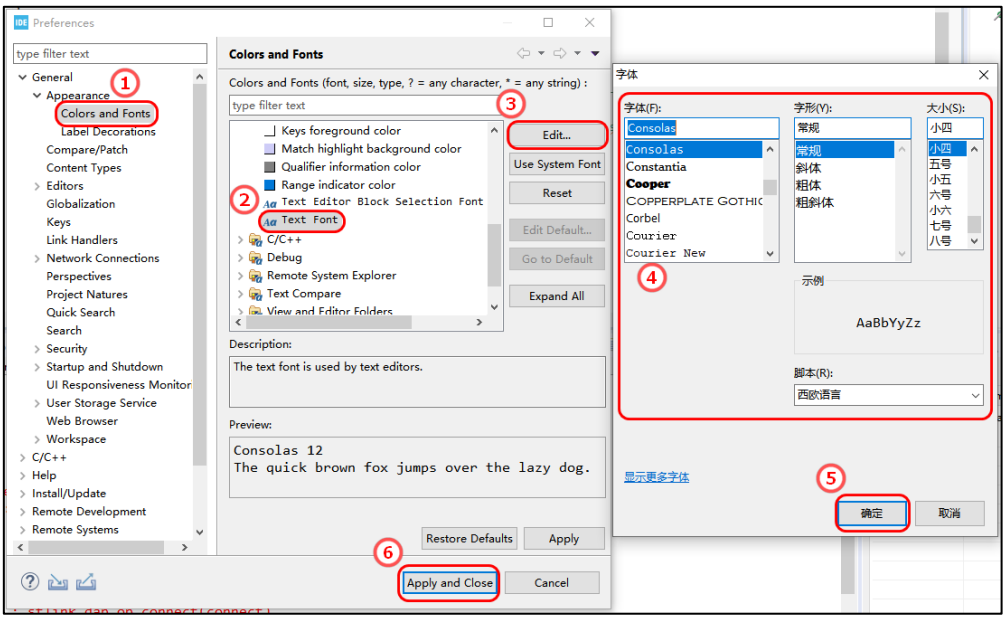


图 4.2.2.2 修改窗口字体

2. 修改函数窗口字体颜色和风格

进入 Preferences 以后，也可以直接在搜索框里输入 color，在搜索到的选项里边选择 C/C++→Editor→Syntax Coloring→Code，可以修改代码的风格，例如关键字、数字等等字体的颜色和风格，例如要修改代码中的数字显示红色，选择字体加粗，可以在 Preview 里查看效果，如果效果满意了再点击 Apply and Close 确认修改，如果要切换回默认设置，可以点击 Restore Defaults 恢复默认值，再点击 Apply and Close 确认修改。

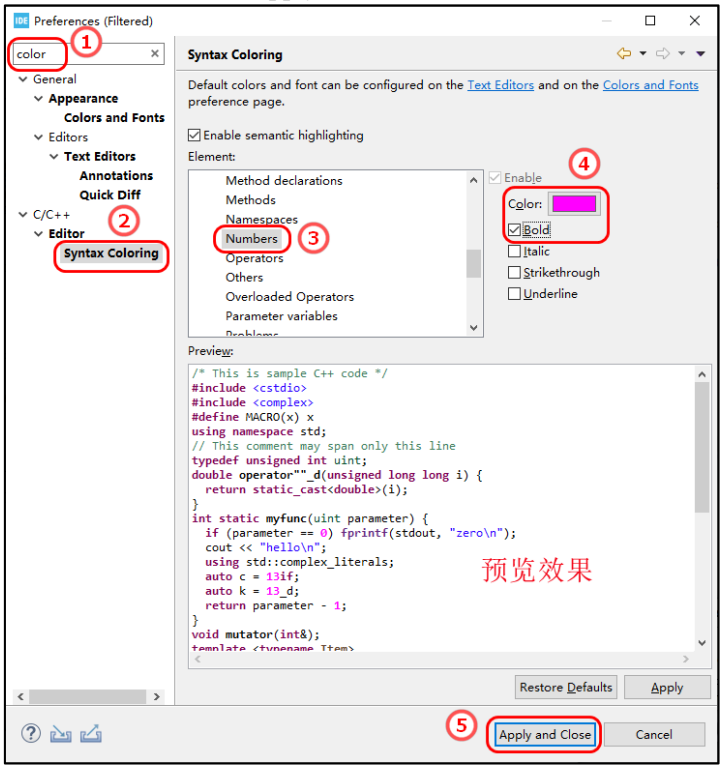


图 4.2.2.3 修改函数窗口字体颜色和风格

3. 修改调试窗口 Console 的字体和大小

如果只是想修改调试窗口 Console 的字体大小，可以修改 Colors and Fonts 选项里的 Debug→Console font (overrides default:Text Font)。

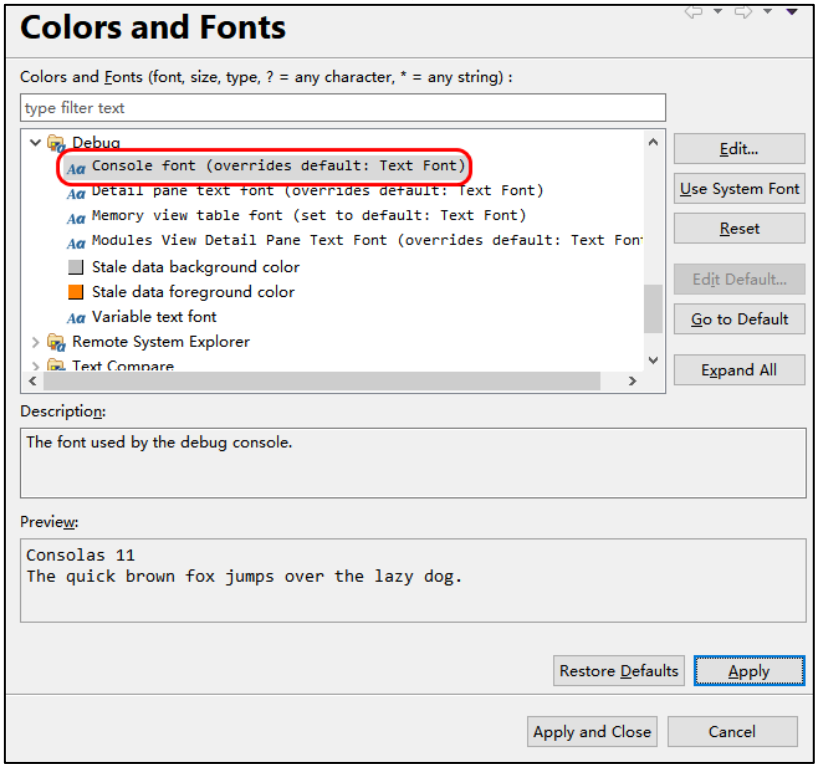


图 4.2.2.4 修改 Console 的字体和大小

4. 修改调试窗口 Console 的字体颜色

如果不喜欢调试窗口的字体颜色，可以选择 Window→Preferences→General→Appearance →Run/Debug→Console 来进行修改，也可以直接在 Preferences 里搜索 Console 找到设置的位置，或者可以在调试窗口 Console 右键选择 Preferences，如图：

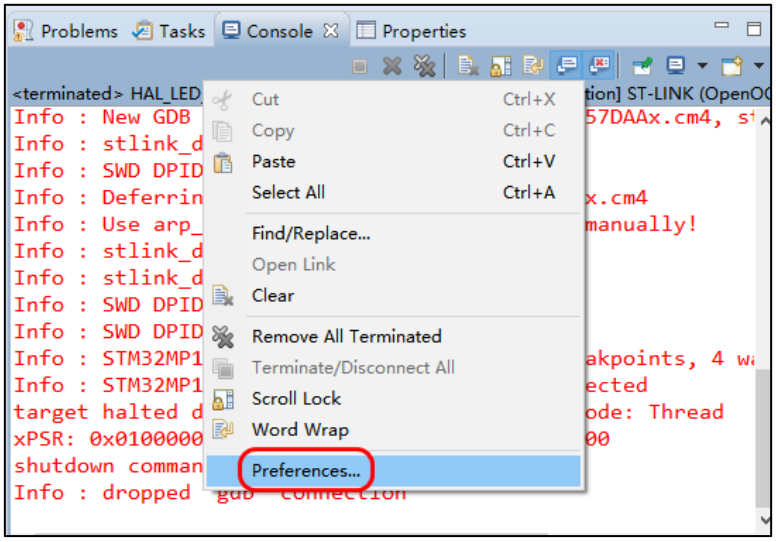


图 4.2.2.5 选择 Console 的属性配置

进入 Debug Console Settings 设置界面，选择 Standard Error text color 进行修改，如果不喜欢修改的效果，可以点击 Restore Defaults 恢复默认值。

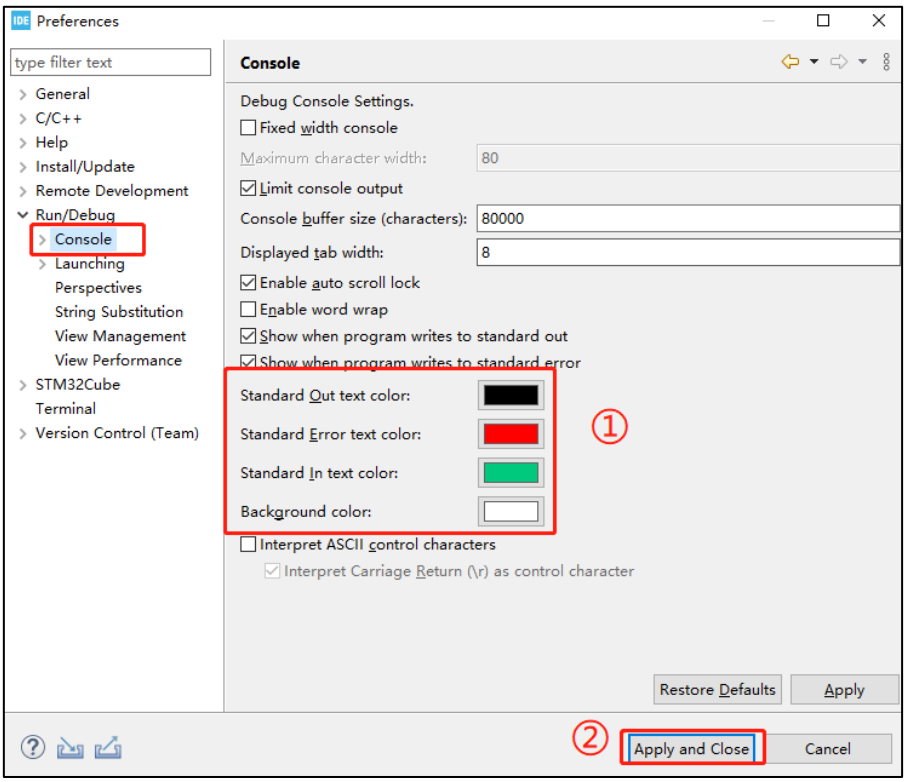


图 4.2.2.6 修改 Console 字体颜色

5. 修改文字编码风格

如果要修改工程的文字编码风格，可以选中想要修改的工程，再点击 Project →Properties→ 进入工程配置界面，选中 Resource→默认文件编码风格是 UTF-8，我们也可以选择其它的编码风格，如图：

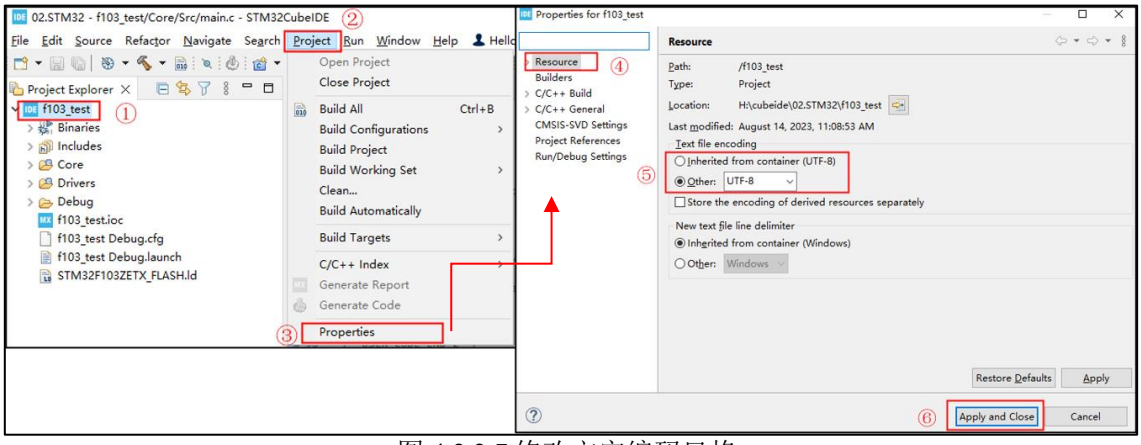


图 4.2.2.7 修改文字编码风格

6. 修改界面背景颜色

如果要修改代码显示区域的背景颜色，可以在 Windows→Preferences 中进行配置：

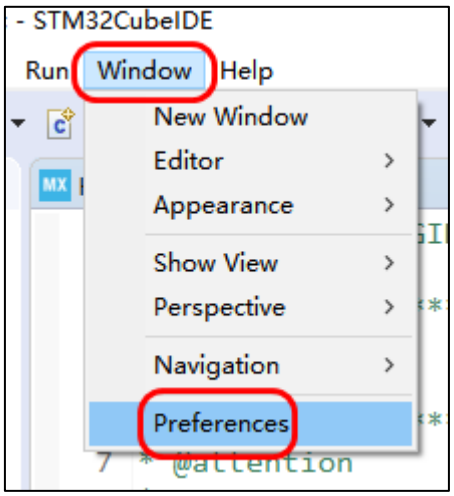


图 4.2.2.8 选中界面属性

进入 Preferences 配置界面以后，依次选择 General→Text Editors→Background color→ 在 Color 处选择一个背景颜色，然后点击 Apply 查看显示效果，如图：

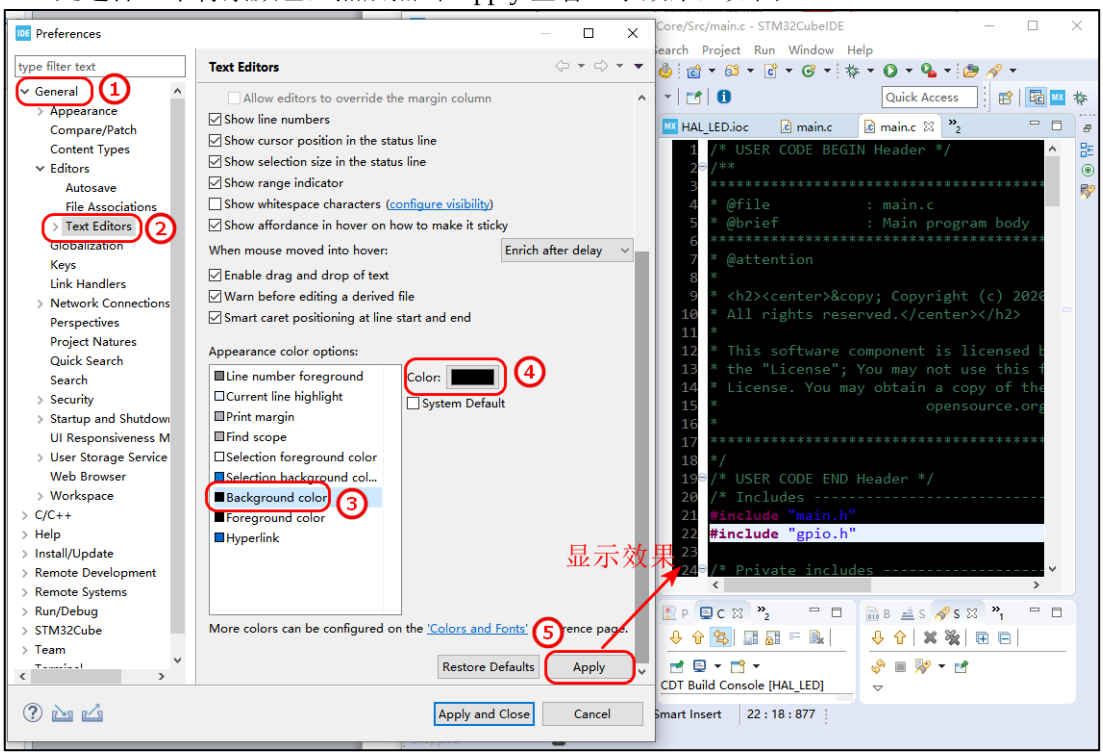


图 4.2.2.9 修改界面背景颜色

7. 窗口&文本样式文件的导出及导入

当我们成功配置好上述设置后，我们可以得到一个符合个人喜好的 STM32CubeIDE 窗口风格和代码颜色设置等。然而，如果我们更换电脑或卸载重装了该软件，恢复之前的配置将变得非常麻烦。因此，我们可以采用另一种方法来解决这个问题。我们可以将已经配置好的窗口风格和文本样式文件导出，然后在新安装 STM32CubeIDE 的电脑上直接导入该文件，这样我们就可以立即回到之前的 IDE 风格设置，从而避免了繁琐的重新配置过程。

窗口风格和文本样式文件导出步骤如下：

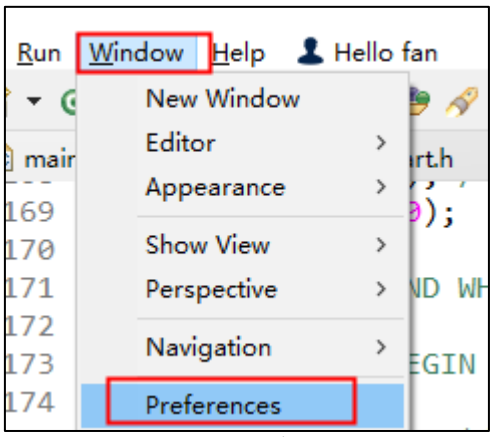


图 4.2.2.10 进入窗口配置项

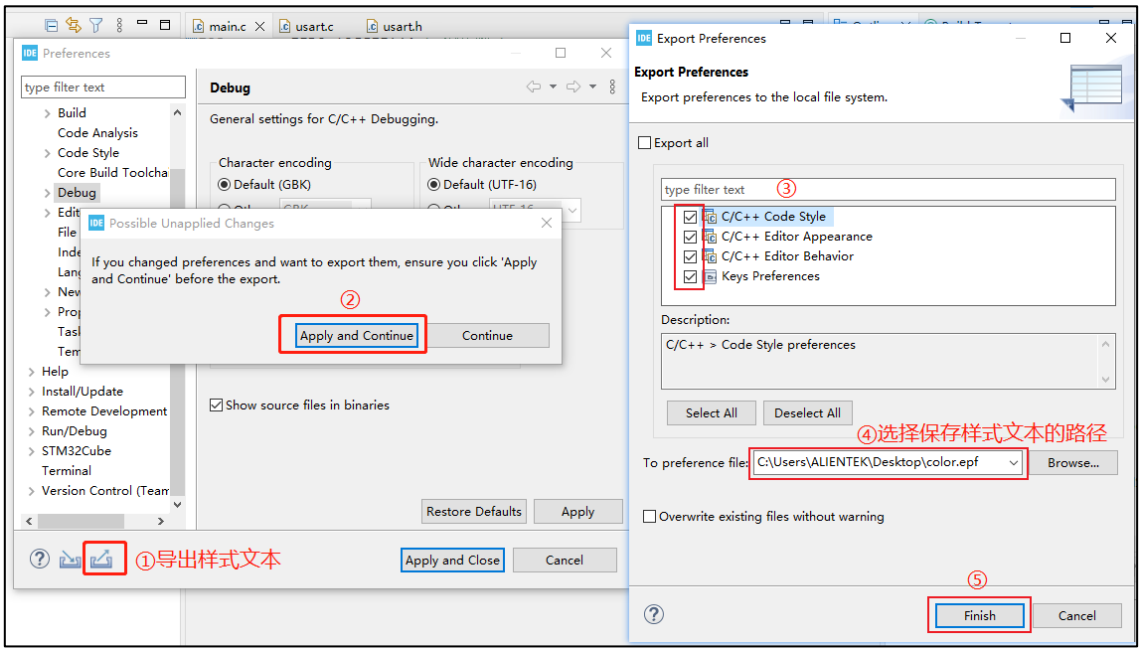


图 4.2.2.11 导出样式文本

通过以上设置即可得到后缀为.epf 的样式文件，如图所示：

图 4.2.2.12 样式文件

接下来我们演示下导入样式文件的过程，同样的进入 Window 的 Preferences 选项，之后按照如下步骤操作：

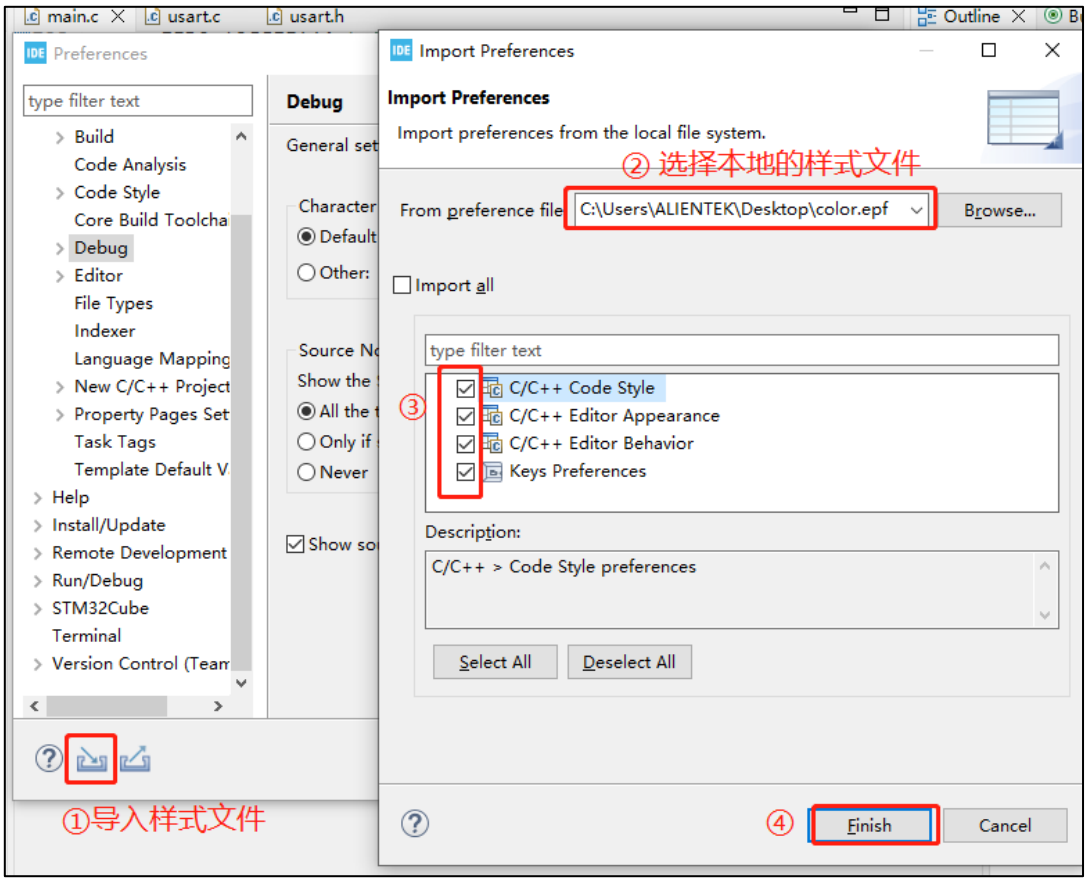


图 4.2.2.13 导入样式文件

这样即使是在新安装 STM32CubeIDE 的电脑上也可立即回到之前的 IDE 风格设置，从而避免了繁琐的重新配置过程。

4.2.3 代码编辑技巧

1. 常用快捷键

先来看看 STM32CubeIDE 常用的快捷键，大家可以多操作，掌握常用的快捷键。

快捷键	快捷键说明	快捷键	快捷键说明
Ctrl+S	保存	Ctrl+C	复制
Ctrl+V	粘贴	Ctrl+X	剪切
Ctrl+“7”	快速注释行	Alt+“/”	补全代码
Shift+Tab	代码对齐	Ctrl+D	删除整行
Ctrl+“-”	缩小字体	Ctrl+ Shift +“+”	放大字体
Ctrl+鼠标左键	跳转到声明处	Ctrl+L	跳转至某行
F3	跳转到声明处	Shift+Enter	在下一行新增空行
F2	展开详细信息		

Ctrl+Tab	源文件/头文件切换	Ctrl+E	快速选择已打开文件
Ctrl+Q	跳回最新编辑点	Alt+“←”/“→”	前一个/后一个页面
Ctrl+鼠标滚轮	快速滑动	Ctrl+F	文件内内搜索
Ctrl+↑/↓	快速滑动	Ctrl+H	项目内搜索
Ctrl+M	最大化/默认当前窗口	Ctrl+O	显示大纲（方便跳转）
Ctrl+W	关闭当前窗口	Ctrl+B	编译工程
F11	启动调试	Ctrl+F2	终止运行
F5	跳进函数（调试时）	F6	单步运行（调试时）
F7	跳出函数（调试时）	F8	全速运行（调试时）
Ctrl+Z	向后撤回	Ctrl+Y	向前撤回

表 4.2.3.1 CubeIDE 常用快捷键

1.文件中搜索

如果我们要在某个文件中搜索一个字符串，例如要在 main.c 文件中搜索是否有 HAL_Init 函数，先打开 main.c 文件，然后按下快捷键“Ctrl+F”打开搜索框，在搜索框中输入 HAL_Init，然后按下回车开始搜索。

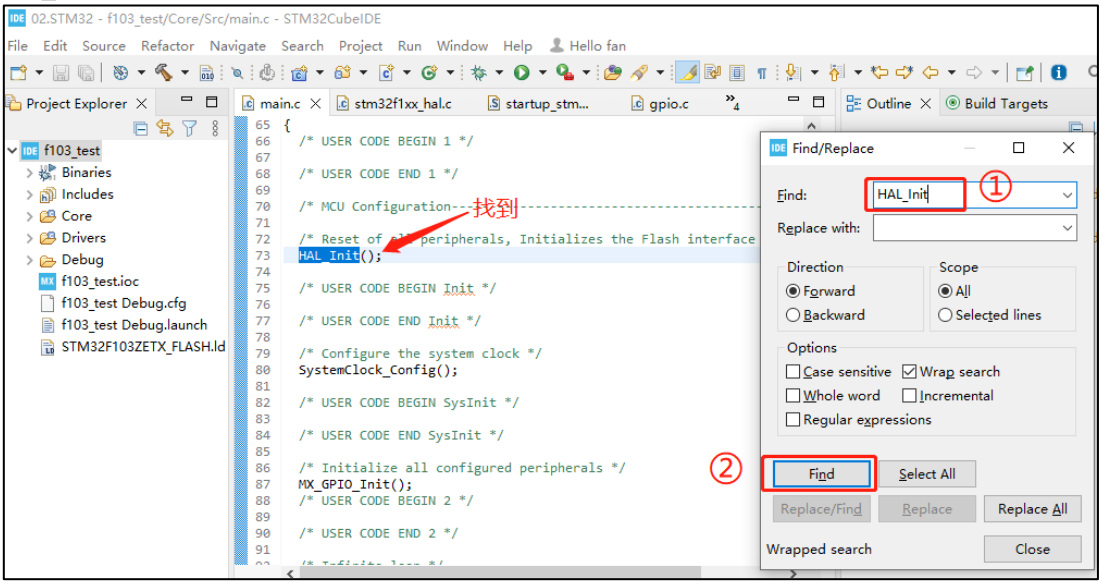


图 4.2.3.1 在文件中搜索

1.1 工程中搜索

我们同样在工程中搜索 HAL_Init 函数，按下“Ctrl+H”快捷键，弹出的搜索框，如图：

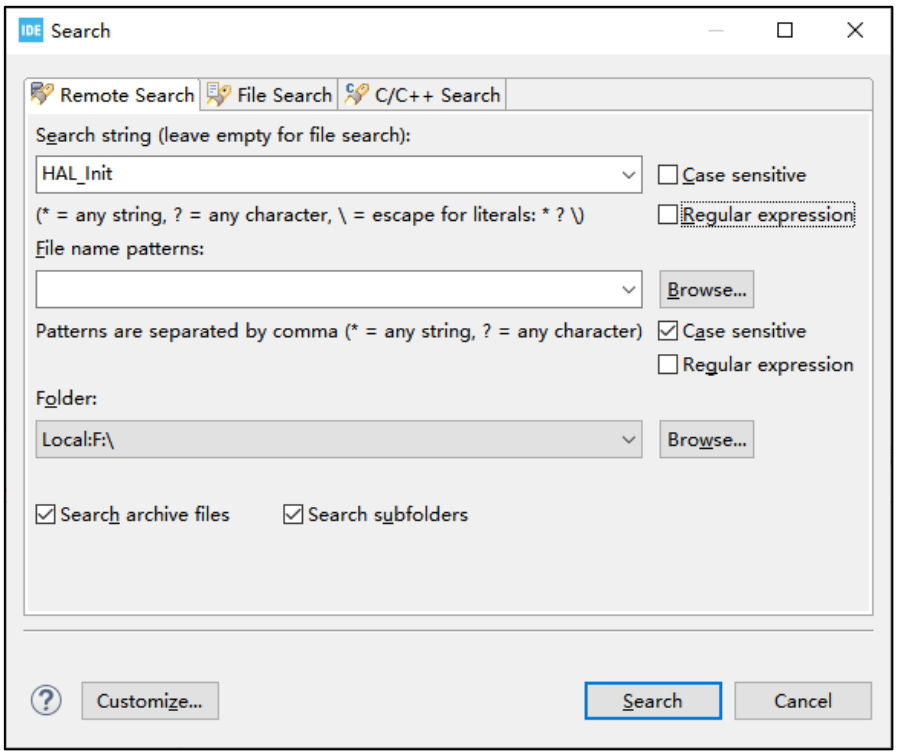


图 4.2.3.2 工程搜索界面

Remote Search 搜索框可以搜索整个工作区或者电脑的其它盘的文件，即可以选择搜索的范围，不过一般不建议用此项来搜索，可能会报内存错误，也可能会导致软件崩溃。

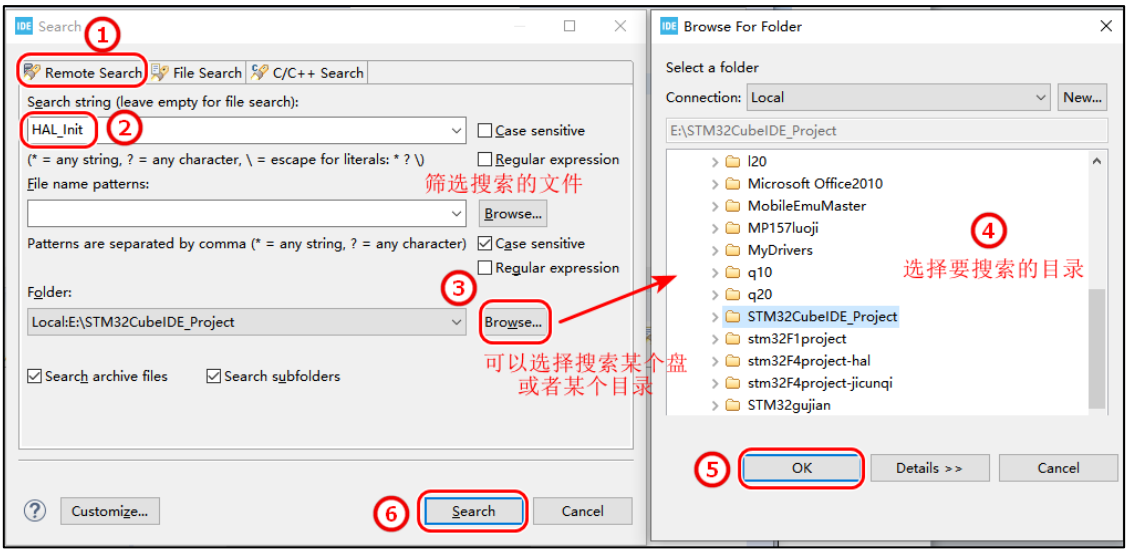


图 4.2.3.3 Remote Search 搜索框

File Search 和 C/C++ Search 这两个是我们常用的搜索项。C/C++ Search 是搜索整个工程的 C/C++文件，并在右下角列出搜索到的文件。File Search 搜索范围是整个工作区的每个文件，在右下角会列出包含 HAL_Init 字符串的文件名字，例如我们用 File Search 搜索结果：

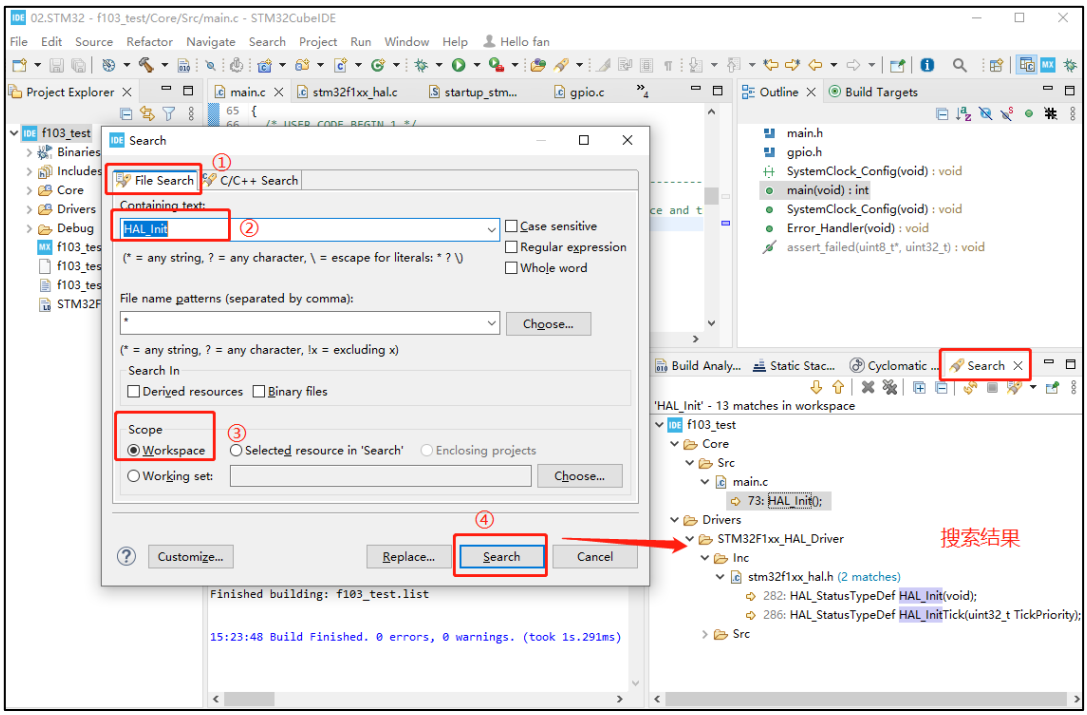


图 4.2.3.4 File Search 搜索框

当然，也可以直接点击菜单栏的搜索项搜索：

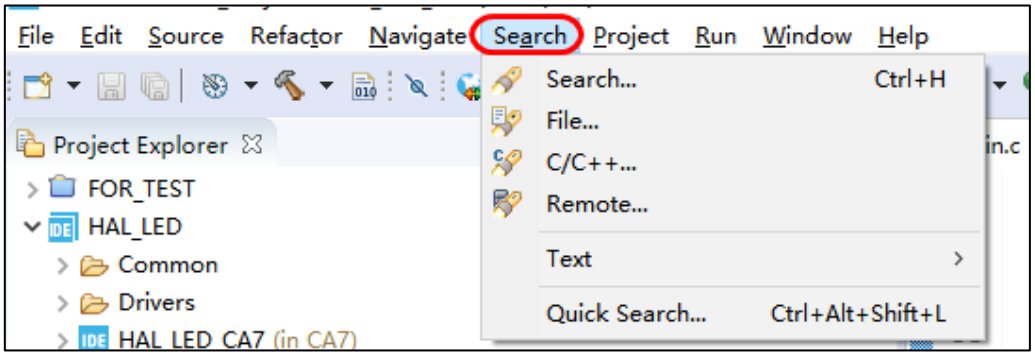


图 4.2.3.5 菜单栏搜索

2. 自定义快捷键

在 Window→Preferences→General→Keys 下可以按照如下步骤自定义快捷键，例如自定义编译工程的快捷键，选中 Command 中的 Build Project 选项，然后鼠标点击 Binding 方框，按下键盘的“Shift+B”组合键，在 Binding 处会显示按下的组合键，点击 Apply and Close 实现修改，以后编译工程我们直接可以选中要编译的工程，再按下“Shift+B”组合键来实现。

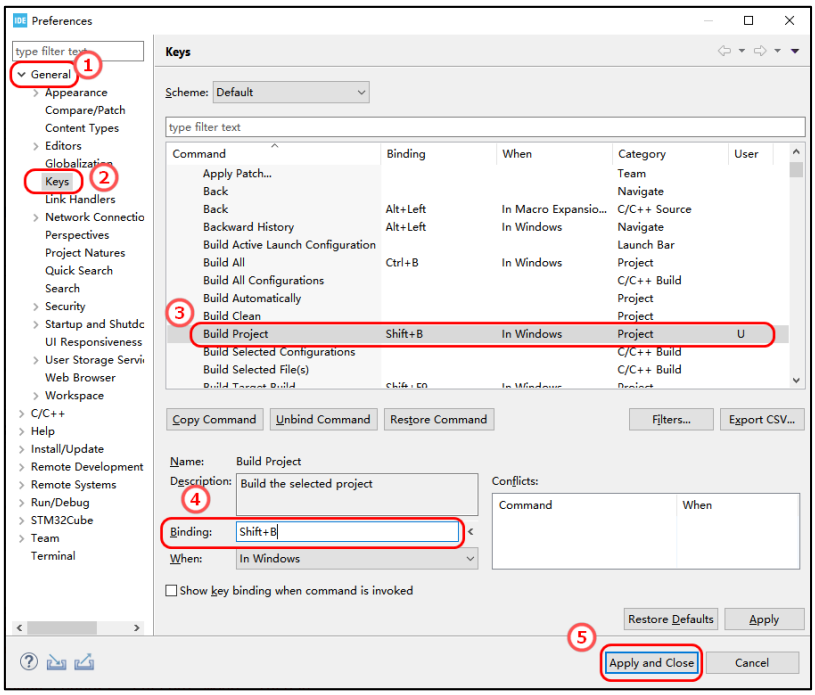


图 4.2.3.6 自定义快捷键

3. 快速注释行

选中要注释的代码，按下键盘的“Ctrl+7”，即可用“//”注释掉选中的代码，再次按下键盘的“Ctrl+ 7”，即可取消注释。

```
93    /* USER CODE BEGIN WHILE */
94    while (1)
95    {
96  //  HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET);
97  //  HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
98  //  HAL_Delay(500);
99  //  HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET);
100 //  HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
101 //  HAL_Delay(500);
102
```

图 4.2.3.7 快速注释行

4. 代码对齐

选中要调整的代码，按下“Shift+Tab”组合键，代码可以整体往左边移动，按下 Tab 键，代码可以整体往右边移动。

5. 删除整行

光标放在要删除的行上，按下“Ctrl+D”组合键可以将光标所在的行删除。

6. 查看/跳转函数的声明

选中要查看的函数，按下 F2 可以显示函数的声明，或者直接按下 F3 跳转到函数的声明。

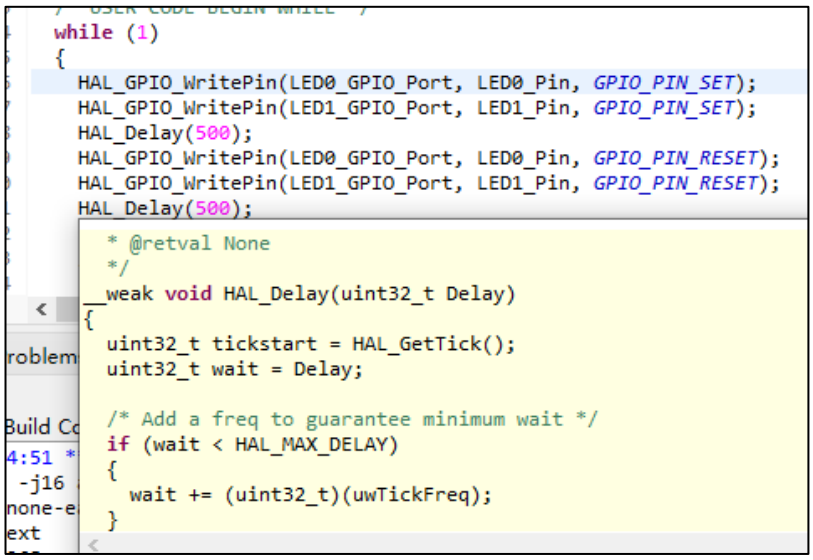


图 4.2.3.8 查看函数声明

7. 代码格式化

选中 Window→Preferences→C/C++→Code Style→Formatter 按照如下配置，在③处选择自己喜欢的代码格式。

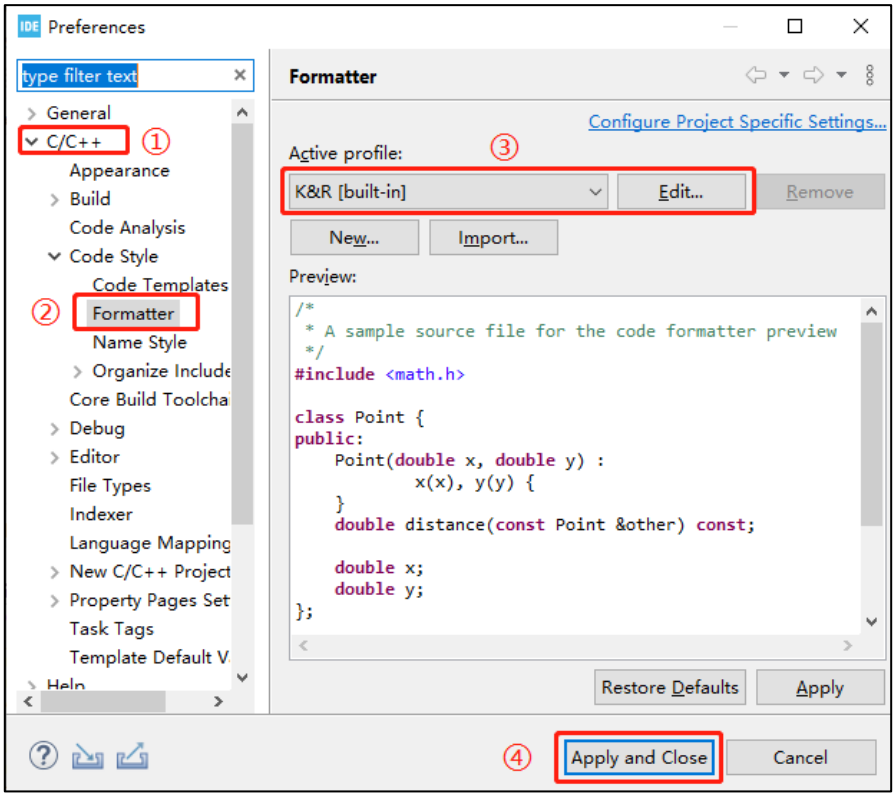


图 4.2.3.9 代码格式化

8. 手动代码补全

按下“Alt+ /”组合键可以手动触发代码提示功能，当我们编辑代码的时候，可以按下“Alt+ /”组合键，然后从弹出的选项列表里边选择我们想要的函数，双击即可补全。

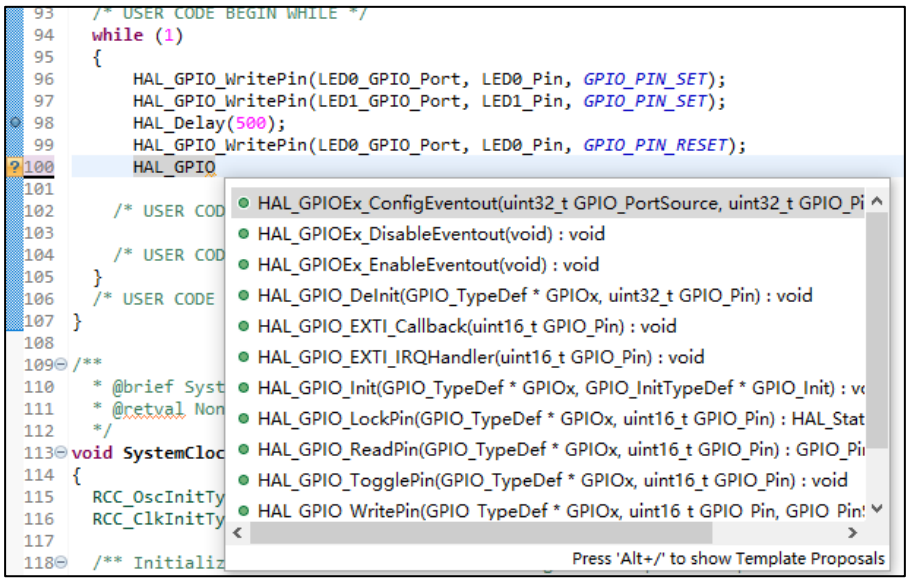


图 4.2.3.10 手动代码补全

9. 自动代码补全

以上是需要每次按下“Alt+ /”组合键才可以开启代码提示功能，如果我们想实现系统自动开启代码补全功能的话，需要在线安装一个 Eclipse 插件，安装完以后还需要配置和导出配置好的插件，因为操作过程复杂也很花时间，这里直接给出插件，大家直接用即可。

注：

此插件目前在 STM32CubeIDE 1.13.1 版本上测试过可用，其他版本没有做过测试。

先关闭 STM32CubeIDE，找到 STM32CubeIDE 的安装目录下的 STM32CubeIDE\plugins 路径，将 org.eclipse.cdt.ui_8.0.100.202301281552.jar 插件先重命名了以后作为备份（万一这次操作不成功还可以再替换回去），我们重命名为 org.eclipse.cdt.ui_8.0.100.202301281552abc.jar，然后再将开发板光盘 A-基础资料\3、软件下的 org.eclipse.cdt.ui_8.0.100.202301281552.jar 拷贝进去。

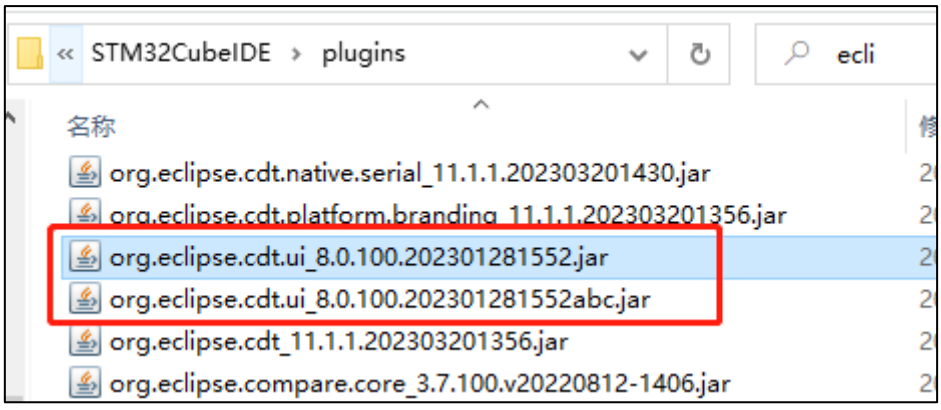


图 4.2.3.11 CubeIDE 插件目录

拷贝完成，重启 STM32CubeIDE，在窗口代码区输入代码，可以看到当我们输入代码的时候，不需要再按下“Alt+ /”组合键来手动触发代码补全功能，系统会自动补全。

10. 比较文件代码

如果要对比两个文件的代码，或者要对比这次修改的文件和上次修改的文件代码差异，可以先选中要对比的其中一个文件，右键后选择 Compare With，如果选择 Each Other，表示和其它文件进行对比，选中要对比的文件进行对比就可以了。如果选择 Local History，则表示和此文件以前的版本进行对比。

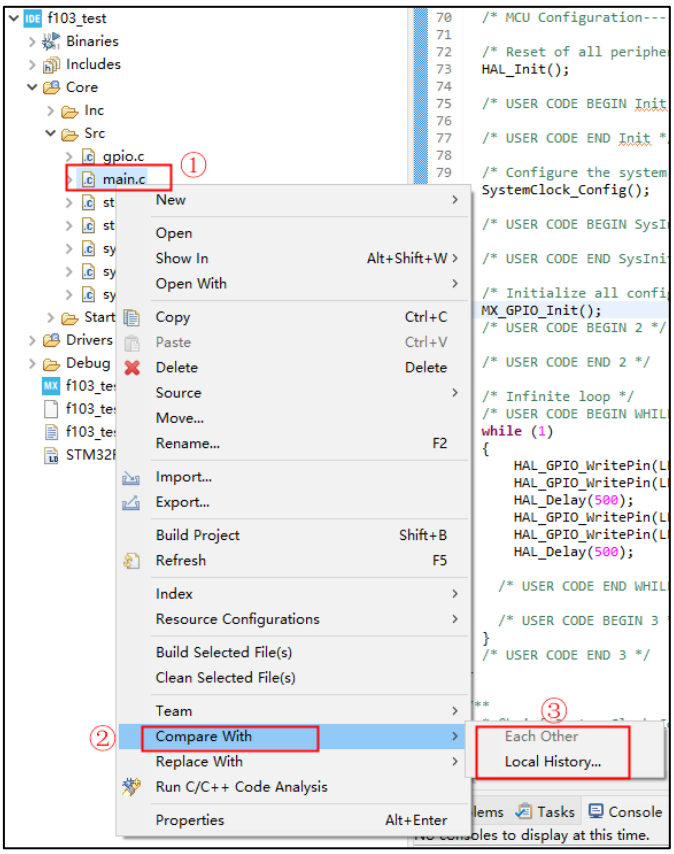


图 4.2.3.12 选择代码对比项

如下图，选择 Local History，则和以前的修改版本进行对比，如下图。

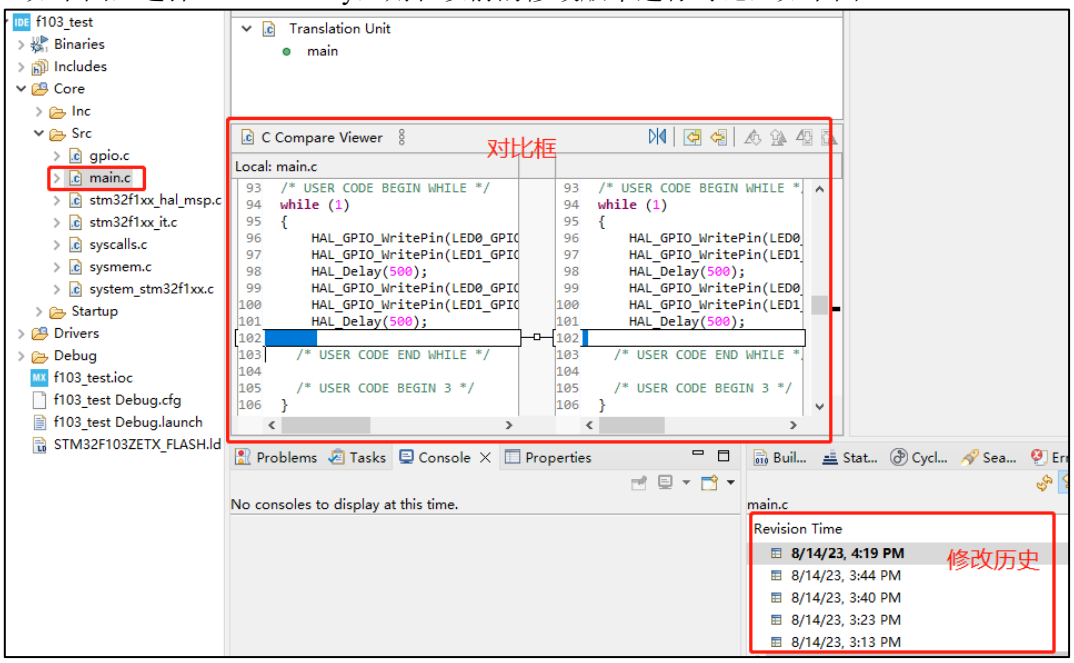


图 4.2.3.13 代码比对窗口

11. 查看修改历史

如果要查看某个文件的修改历史记录，可以选中文件，右键，在下拉选项中选中 Team 查看历史记录。

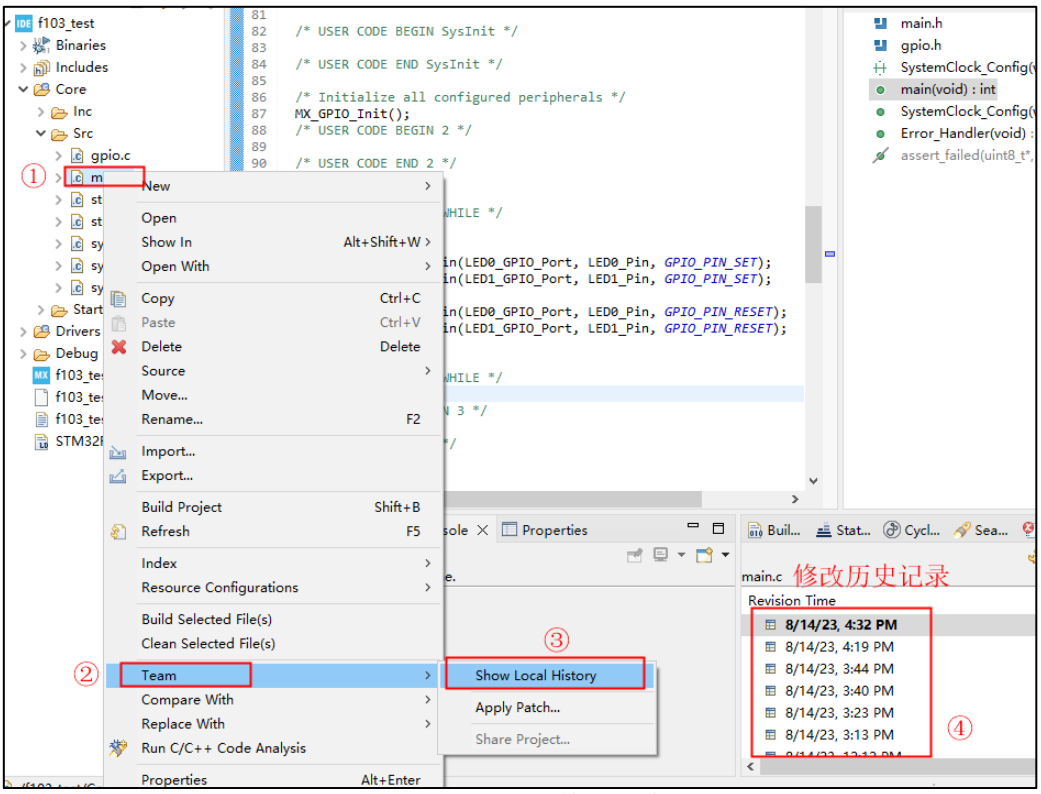


图 4.2.3.14 显示修改历史

找到修改历史以后，可以选中某一项，右键，可以选择打开此历史记录或者和当前的文件对比：

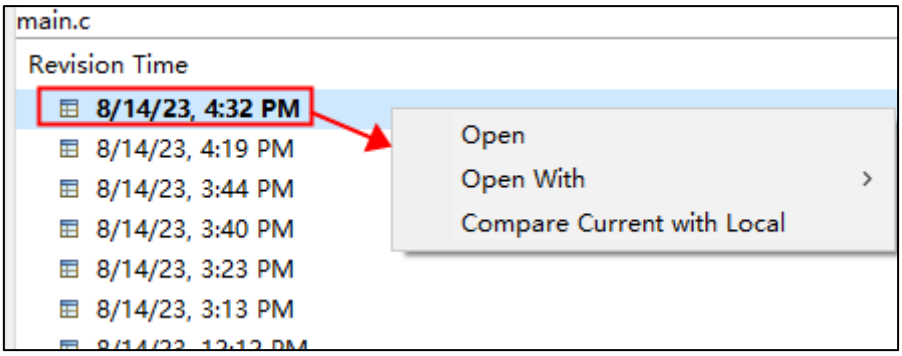


图 4.2.3.15 打开修改历史代码

4.2.4 添加自己的代码文件

在复杂的项目中，STM32CubeMX 插件生成的工程目录是无法满足我们的需求的，很多时候我们需要在工程中新建一个一个文件夹，每个文件夹放置对应的文件，我们往新增的文件里添加我们的代码，实现控制逻辑，然后再在 main.c 函数里调用这些文件。这样做的好处是工程结构逻辑清晰，例如当我们的工程有很多的外设控制逻辑时，采用这样的添加方法一目了然，而且也不会因为修改.ioc 文件不小心清除掉这些代码。下面我们介绍如何直接在工程界面新建这些文件夹/文件，或者直接在工程中导入已有的文件的文件夹。

1. 新建文件夹

选中需要添加文件夹的工程，右键，选择 Properties，如图：

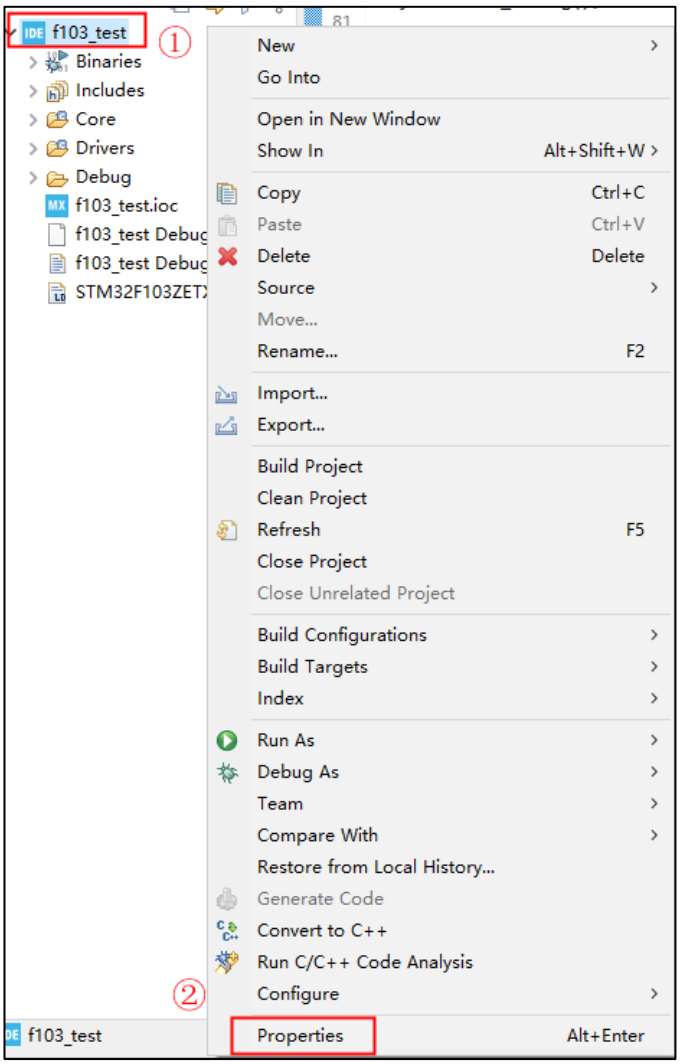


图 4.2.4.1 f103_test 工程属性选项

进入 Properties for f103_test 配置项，依次按照如下操作顺序添加 BSP 文件夹：选择 C/C++General→Paths and Symbols→Includes→GNU C→Add→ 输入文件夹名字 BSP→OK→Apply→Apply and Close。

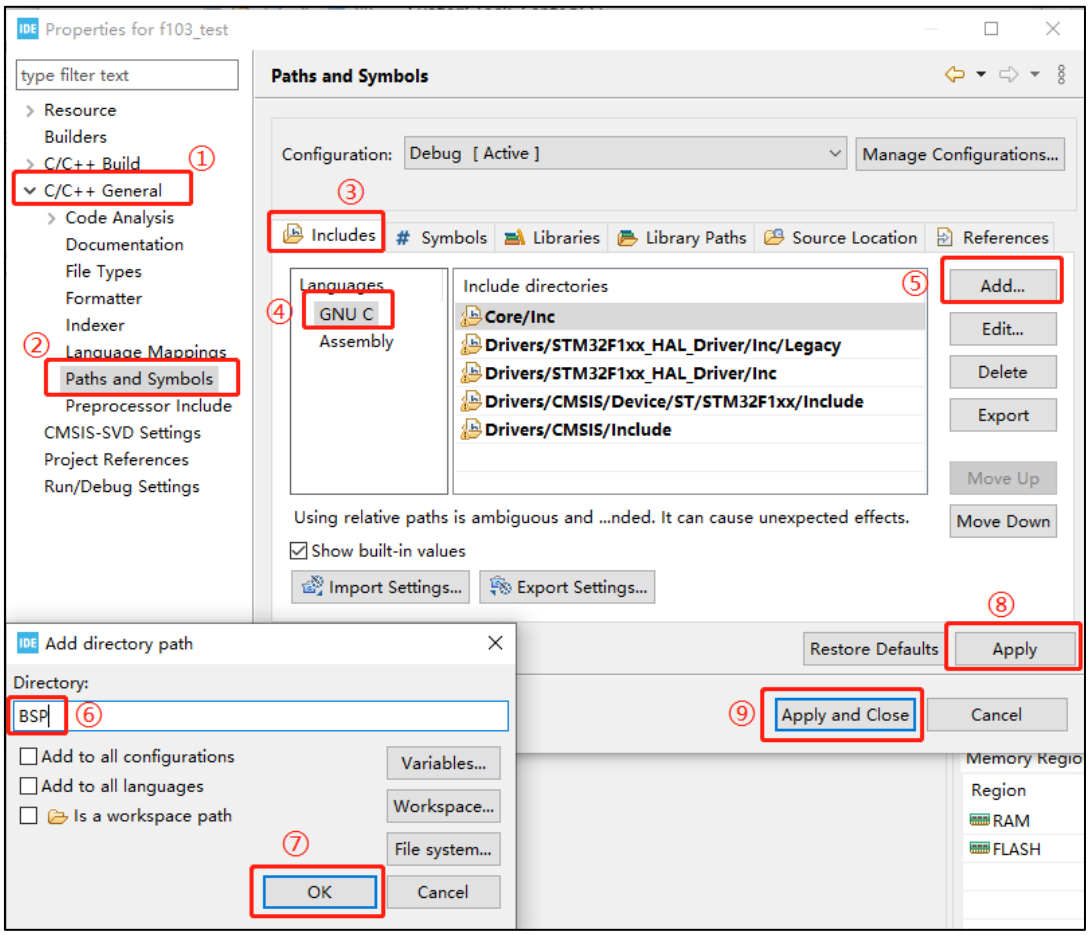


图 4.2.4.2 工程文件夹配置步骤

在点击 Apply 以后，可以在 Includes 选项中多出 BSP：

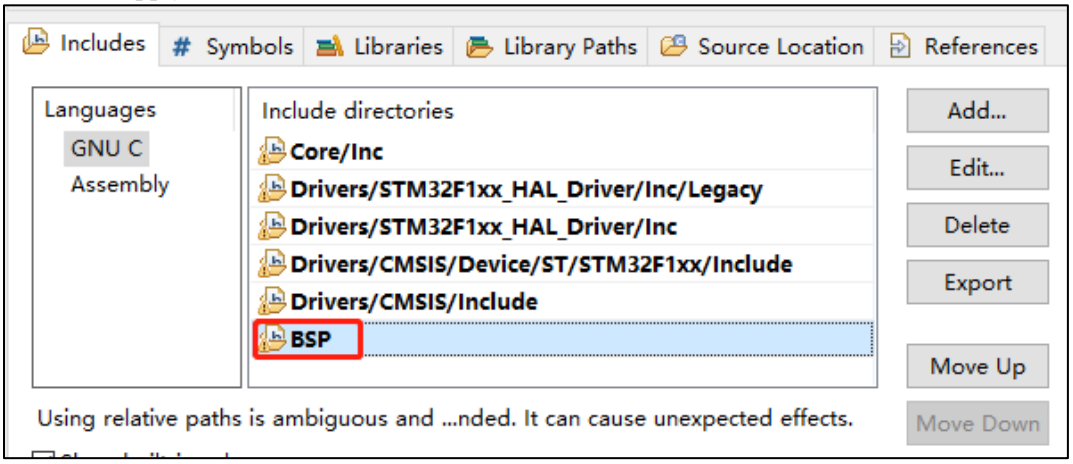


图 4.2.4.3 添加好的工程文件夹

按照同样的操作方式再次进入 Properties 界面，依次按照如下操作顺序对添加的 BSP 文件夹进行链接，即链接到 f103_test 工程中：选择 C/C++General→Paths and Symbols→Source Location→Link Folder→输入文件夹名字 BSP→OK→Apply→Apply and Close。

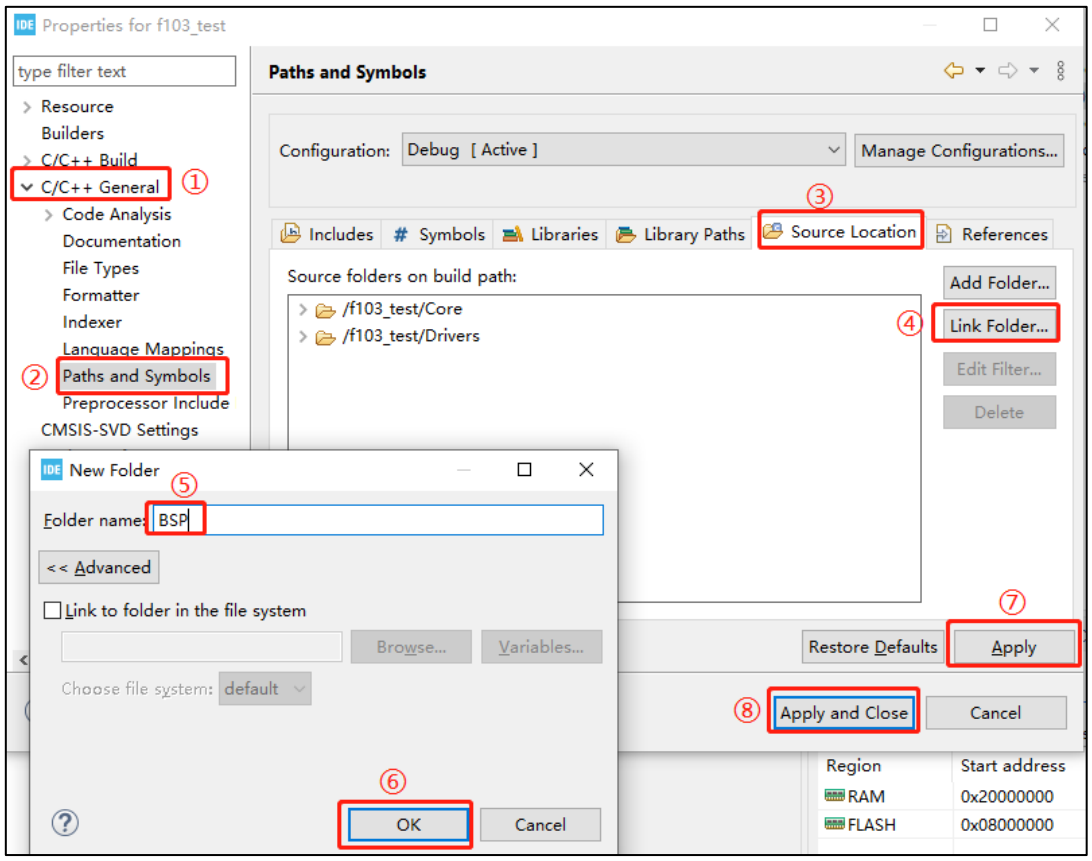


图 4.2.4.4 对添加的文件夹进行关联

在点击 Apply 的时候，可以在 Source Location 选项中多出“/ f103_test/BSP”：

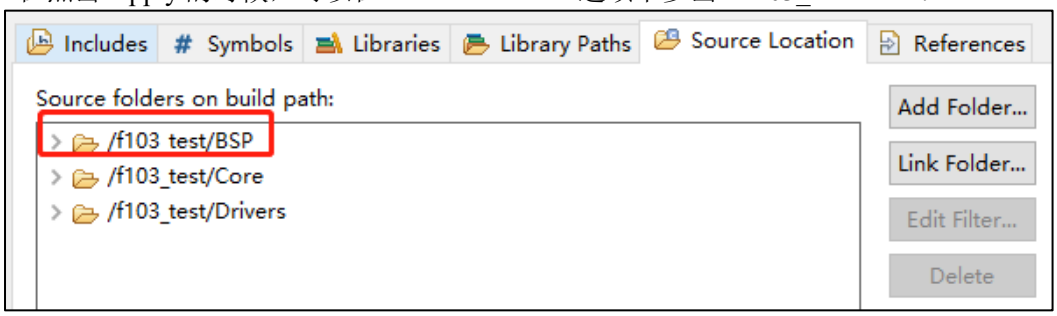


图 4.2.4.5 显示添加的文件夹路径

回到工程界面，我们看到 f103_test 工程下多出了一个文件夹 BSP，注意此 BSP 文件夹的左上角多了一个“C”字眼的字符，表示这个文件夹和工程关联成功（即前面进行链接后的效果）。如果没有看到 BSP 文件夹，可以选中 f103_test 工程，右键，选择 Refresh 更新一下工程。

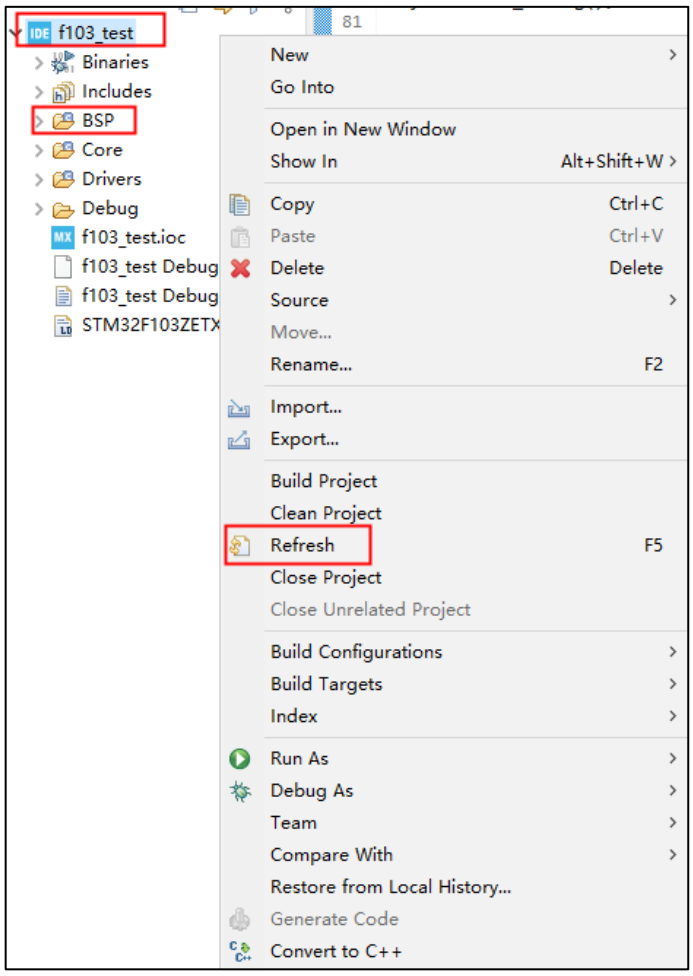


图 4.2.4.6 更新工程目录

2. 新建文件

添加完文件夹，我们在文件夹中添加自己的驱动文件。选中 BSP 文件夹，右键，选择 New 创建一个文件，我们先创建头文件，选择 Header File（选择 File 选项也是可以的）。

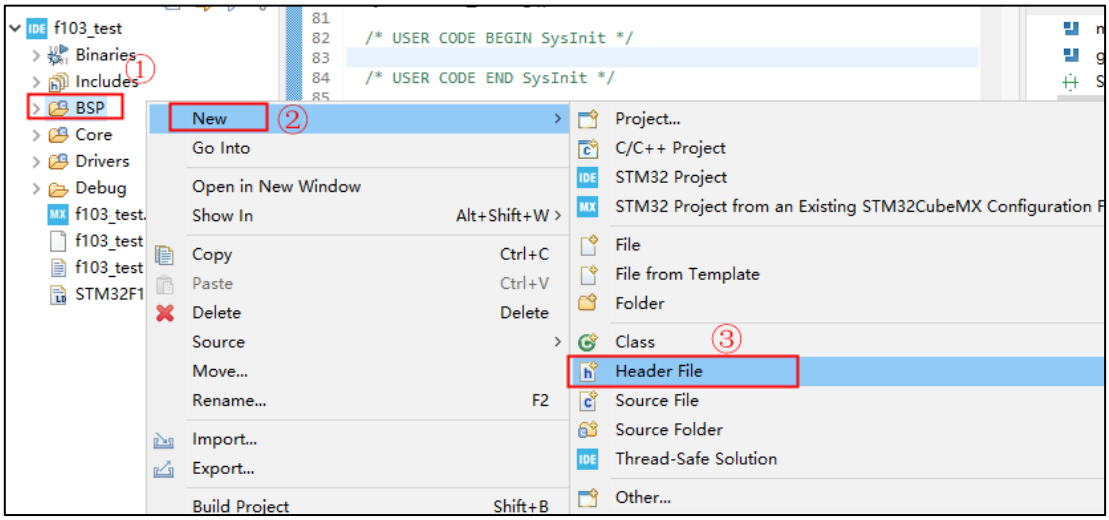


图 4.2.4.7 新建头文件

输入 Header File 的文件名，我们取名为 led.h。

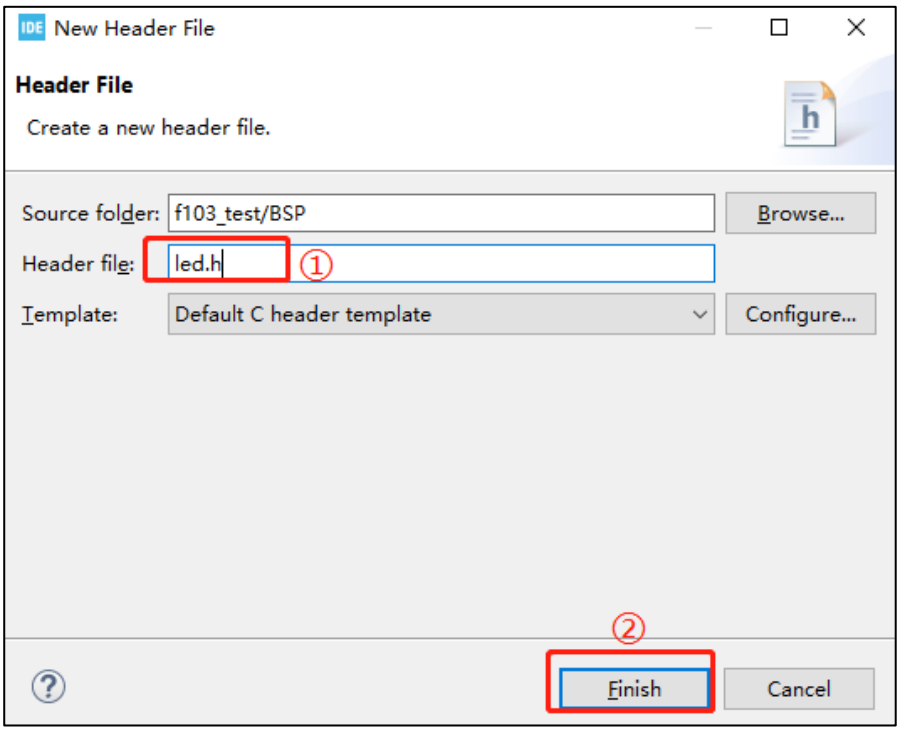


图 4.2.4.8 设置头文件名字

按照同样的方法来创建源文件：

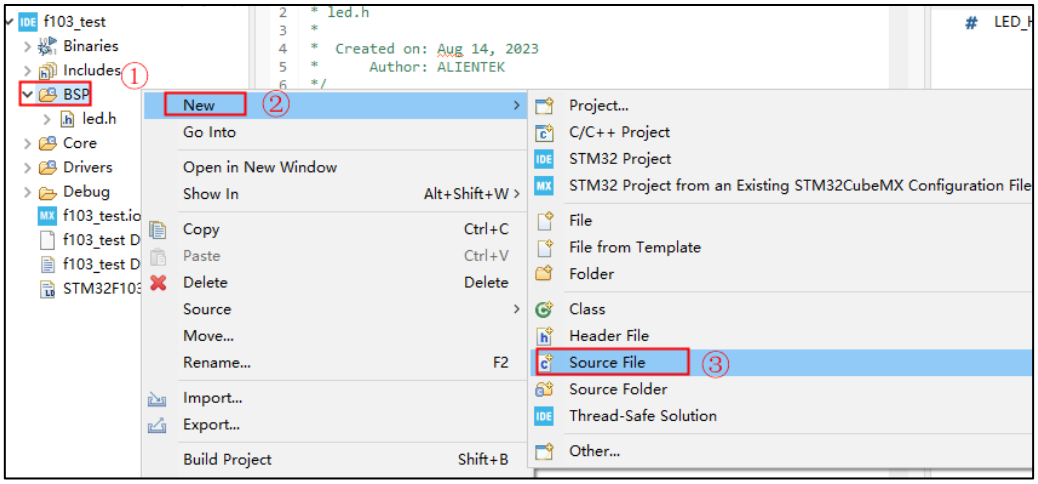


图 4.2.4.9 新建源文件

我们设置源文件的名字为 led.c：

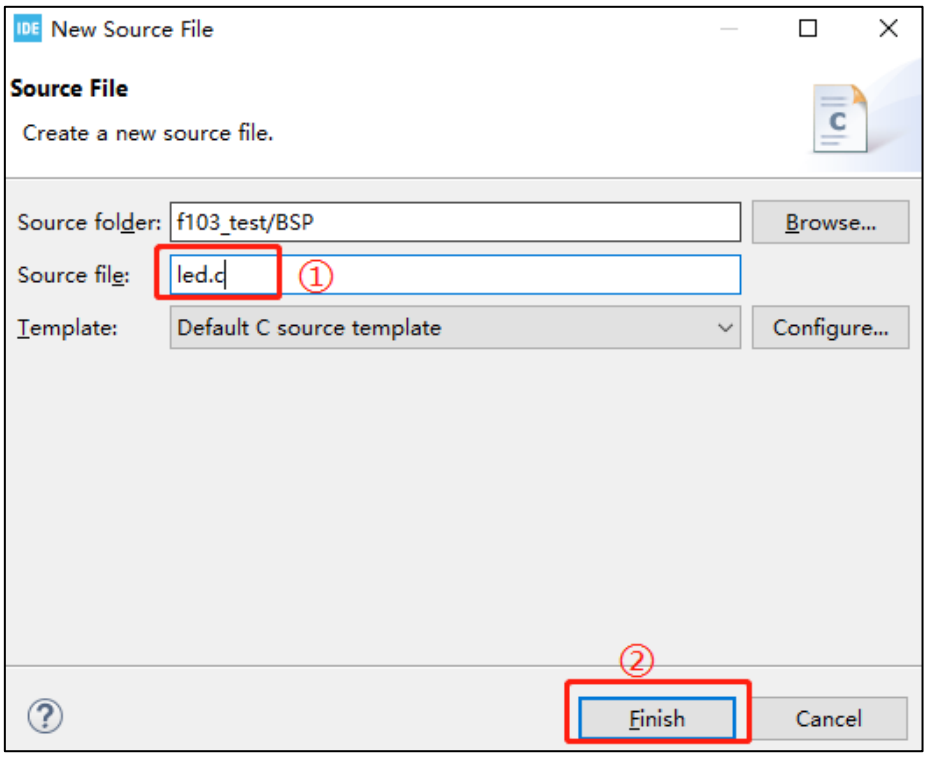


图 4.2.4.10 设置源文件名字

按照上述操作，我们添加了自己的驱动文件，我们在 main.c 文件中包含新建的 led.h 文件，保存修改以后编译工程，在 Console 中编译成功。

```
/* USER CODE BEGIN Includes */
#include "led.h"
/* USER CODE END Includes */
```

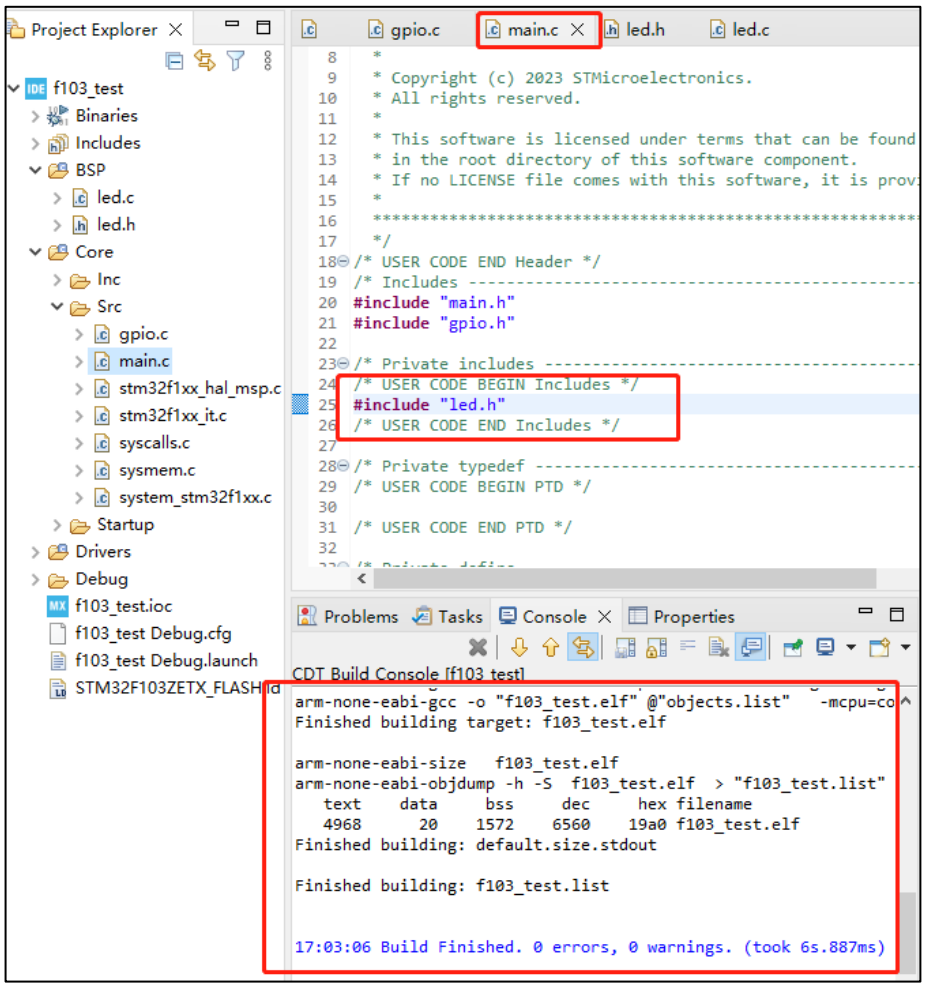


图 4.2.4.11 添加头文件路径

3. 另一种新建文件的方法

按照上述添加自己的文件的好处是，在工程中包含文件不用写文件的绝对路径，在编译工程时不会找不到头文件，因为前面已经做好了关联。如果是采用下面的方式添加，在添加文件的时候必须要写出文件的绝对路径，否则编译的时候会提示找不到头文件。下面我们尝试这种操作。

选中 f103_test 工程，选中 New，选中 Folder，即新建一个文件夹：

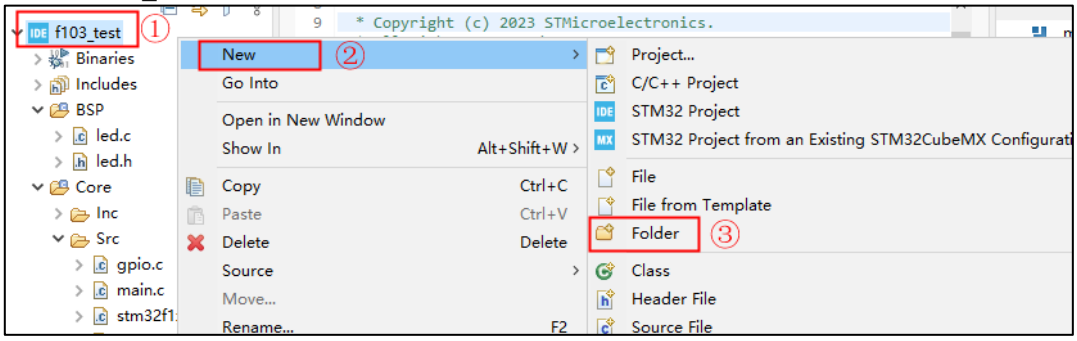


图 4.2.4.12 在工程中新建文件夹

文件夹的名字命名为 UART：

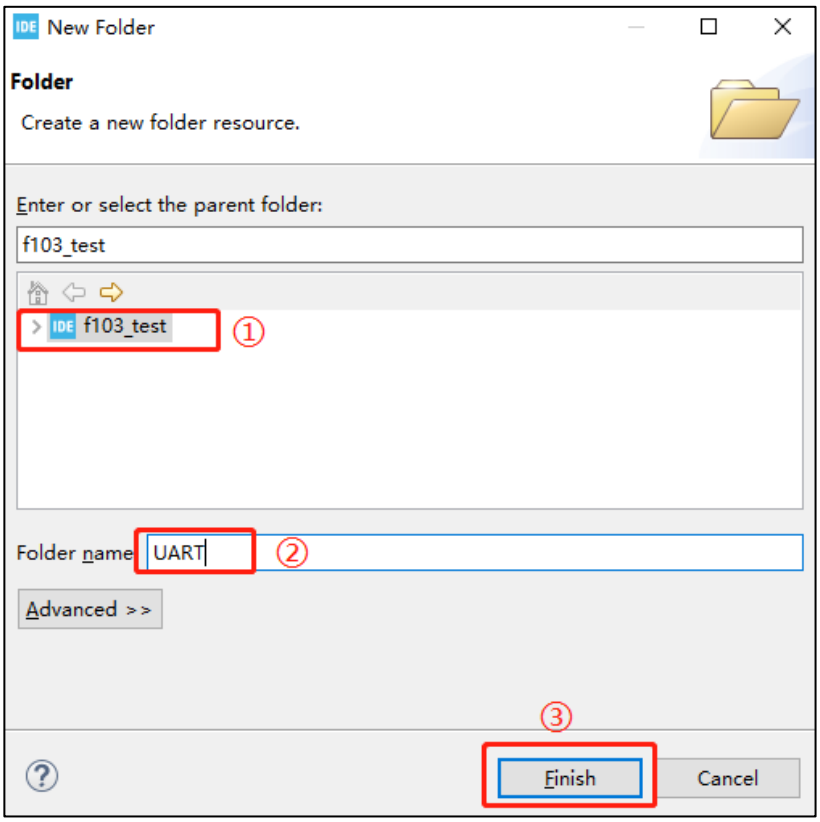


图 4.2.4.13 给新建的文件夹命名

选中 UART，右键，选择 New→File 来新建一个文件：

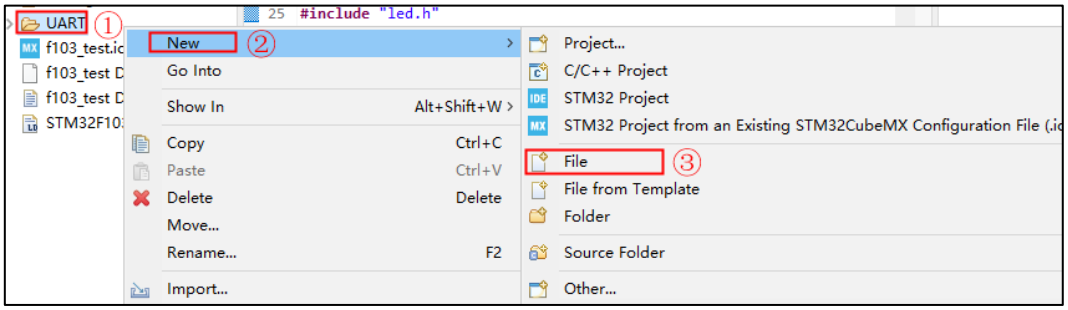


图 4.2.4.14 在文件夹中新建一个文件

文件名字为 uart.h：

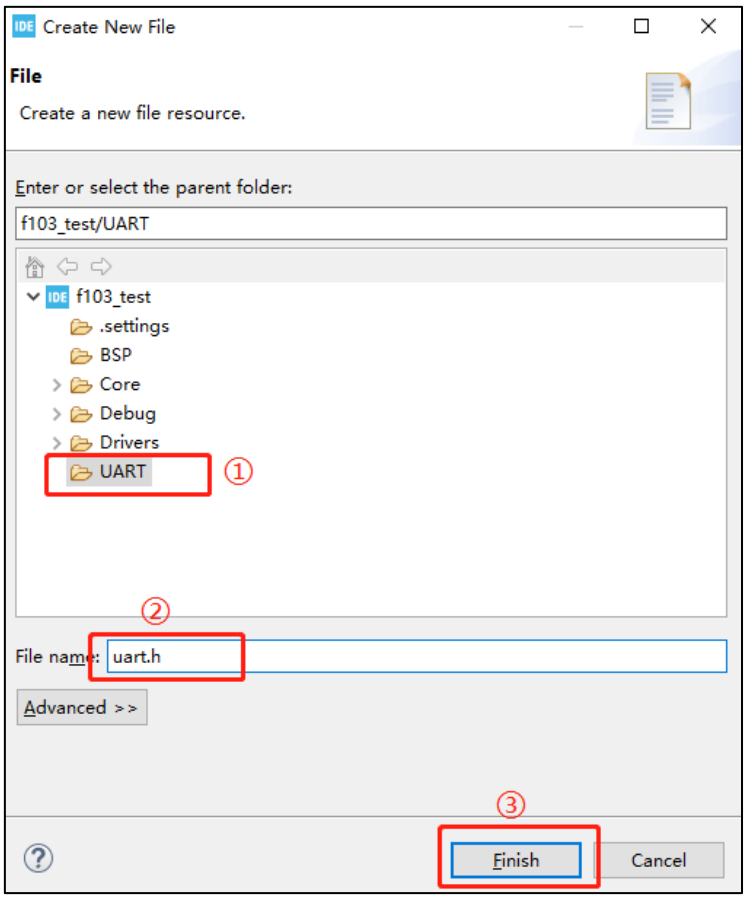


图 4.2.4.15 设置头文件名字

按照同样的方法新建文件 uart.c 文件。文件新建完成，我们在 main.c 中包含 uart.h 头文件。

```
/* USER CODE BEGIN Includes */
#include "led.h"
#include "uart.h"
/* USER CODE END Includes */
```

保存修改后，编译后报错找不到 uart.h 文件：

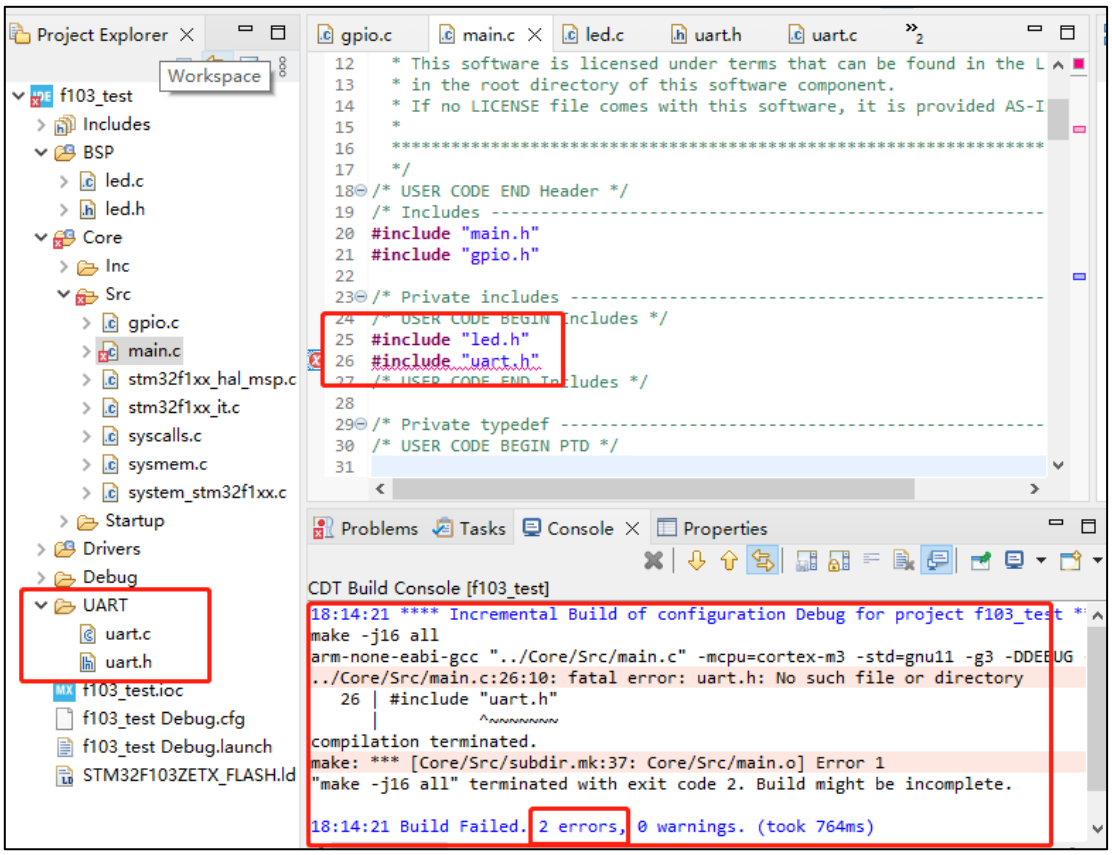


图 4.2.4.16 编译报错

我们注释掉前面添加的#include "uart.h"这段代码，修改成将 uart.h 的文件在工程中的绝对路径（这个路径是相对于 main.c 来说的）添加上去：

```
/* USER CODE BEGIN Includes */
#include "led.h"
#include "../../UART/uart.h"
/* USER CODE END Includes */
```

修改保存，进行编译，编译不再报错了：

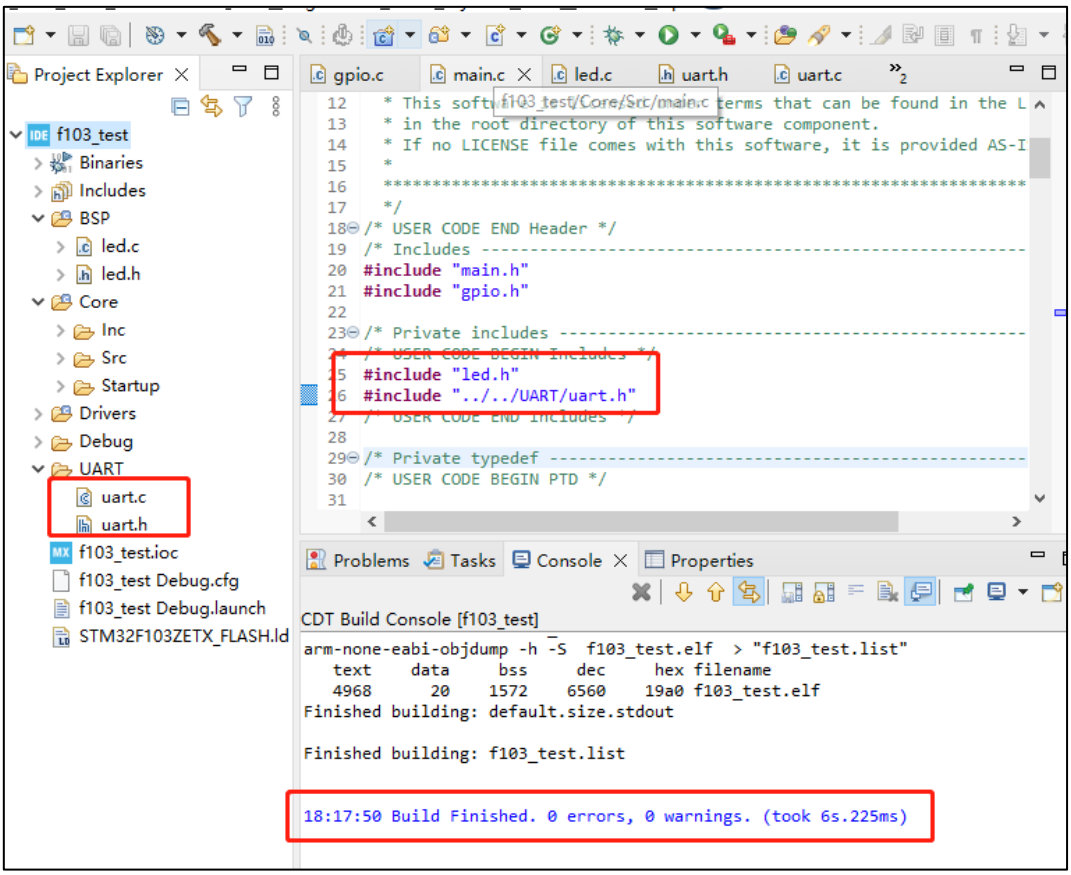


图 4.2.4.17 编译成功

在以后添加文件的操作中，大家多注意这方面的操作，编译报错找不到头文件的话，可以写上文件路径的绝对地址。

4. 导入已有文件

前面是在工程中直接新建文件，如果想在工程中导入已有的文件，可以选中工程中要导入文件的文件夹，然后右键选择 Import，例如要在前面新建的 BSP 文件夹中导入 STM32CubeIDE 工作目录下的 CubeIDE_TEST 文件夹里的 test.c 和 test.h 两个文件（CubeIDE_TEST 文件夹的文件是笔者自己创建的）。

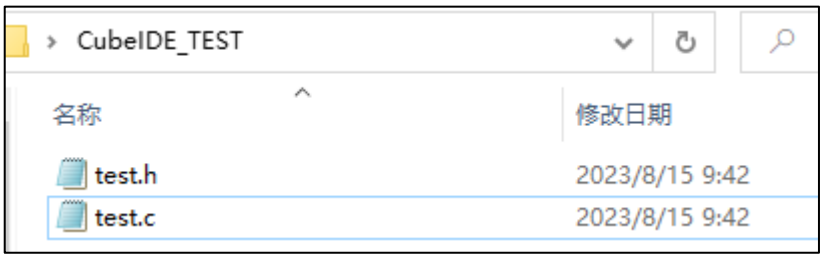


图 4.2.4.18 笔者自己新建的文件

选中工程中 BSP 文件夹，右键选择 Import：

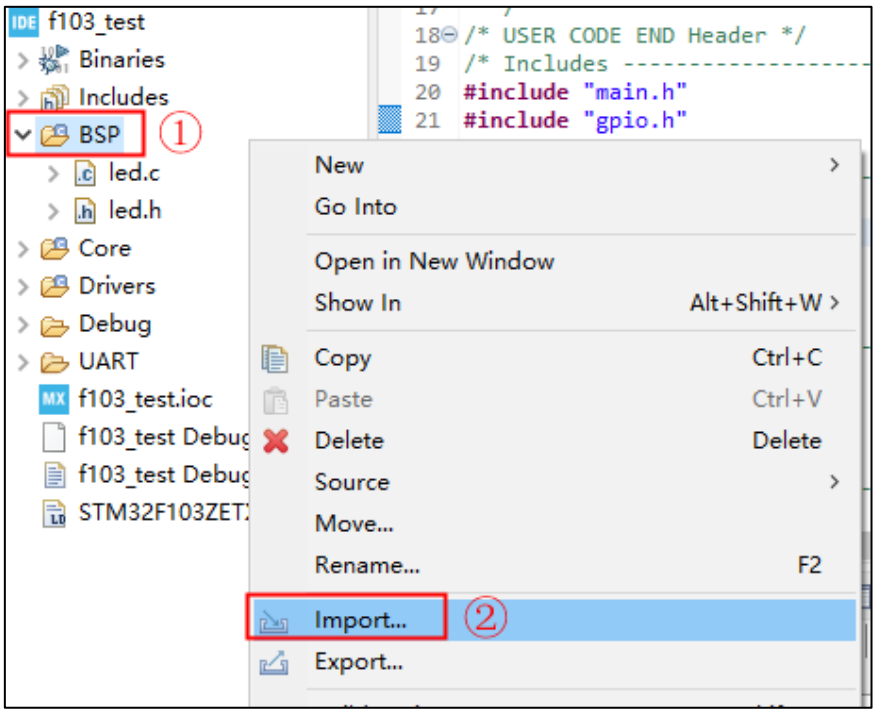


图 4.2.4.19 选择导入文件夹

选择 General→File System，点击 Next：

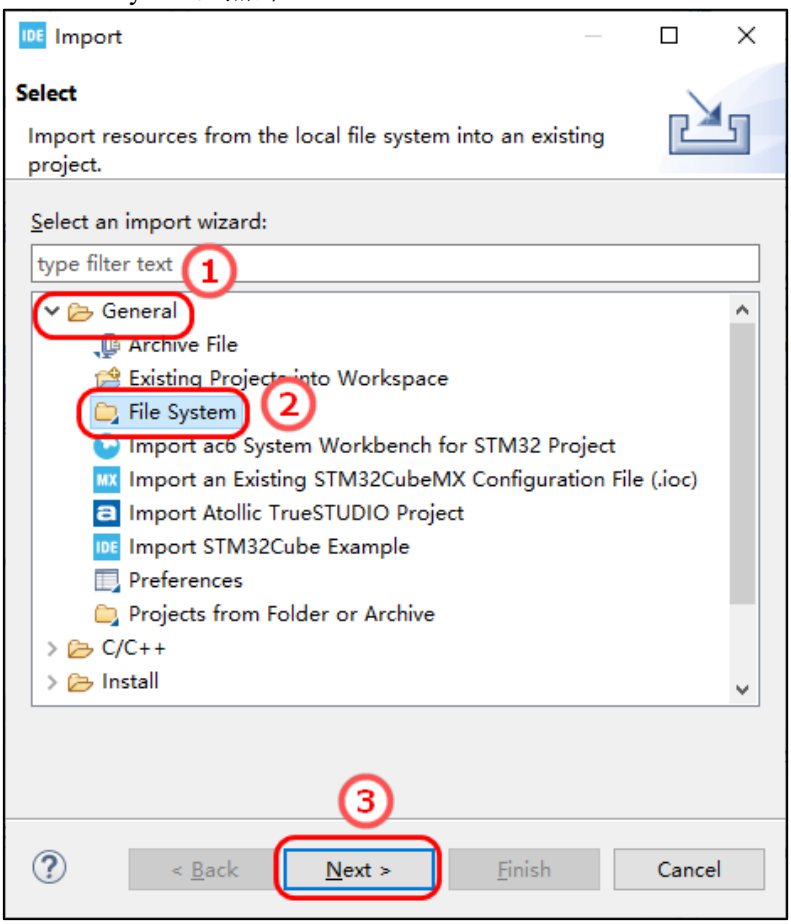


图 4.2.4.20 选择 File System

按照如下步骤选择 TEST 文件夹的两个文件：

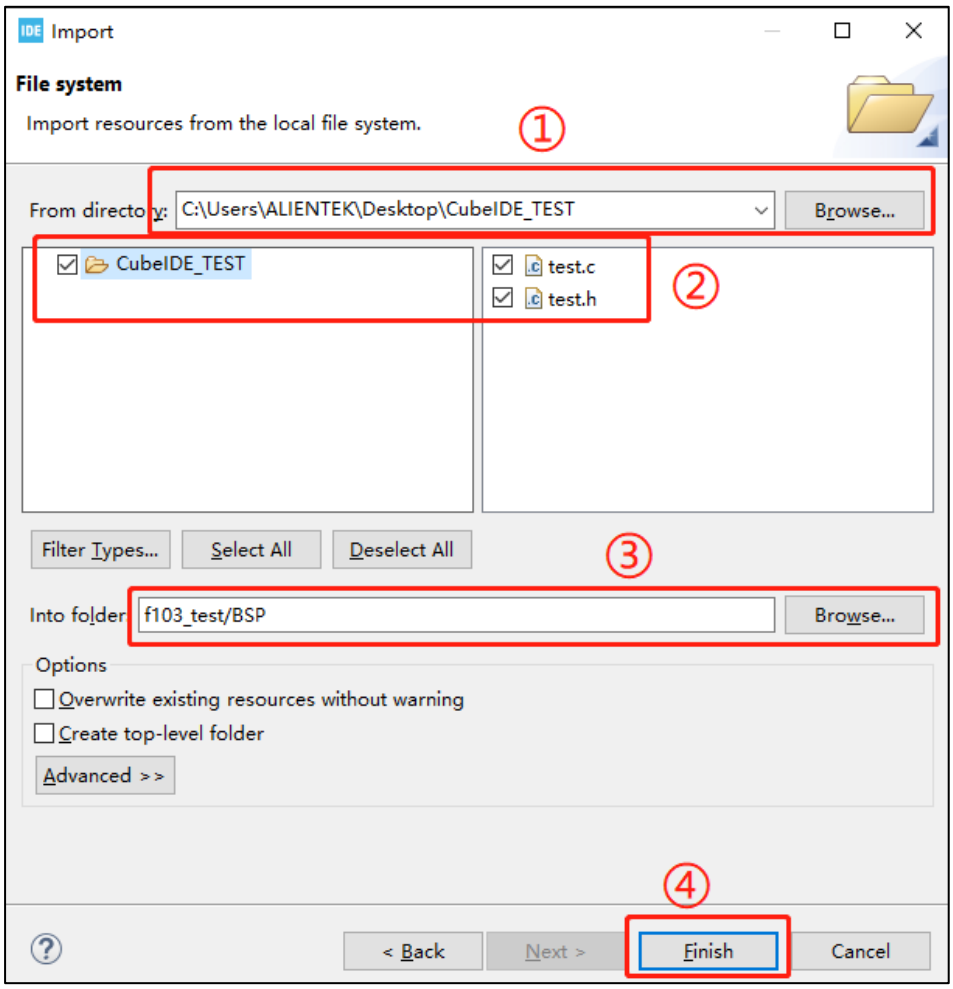


图 4.2.4.21 选中要导入的文件

点击 Finish 添加完成。这种添加的方式相当于拷贝文件，即将文件拷贝到 f103_test 工程的 BSP 目录下：

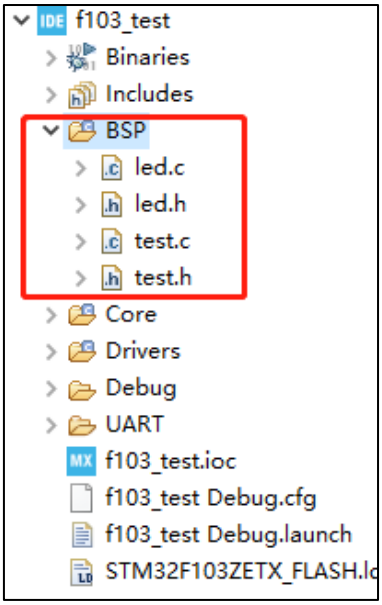


图 4.2.4.22 文件导入成功

采用这种方式导入头文件，可以直接在 main.c 函数中直接包含 test.h 文件，添加完然后保存修改了再编译，可以编译通过。

```
/* USER CODE BEGIN Includes */
#include "led.h"
#include "../../UART/uart.h"
#include "test.h"
/* USER CODE END Includes */
```

如果添加的文件不想要，可以选中文件，然后右键选中 Delete 删除：

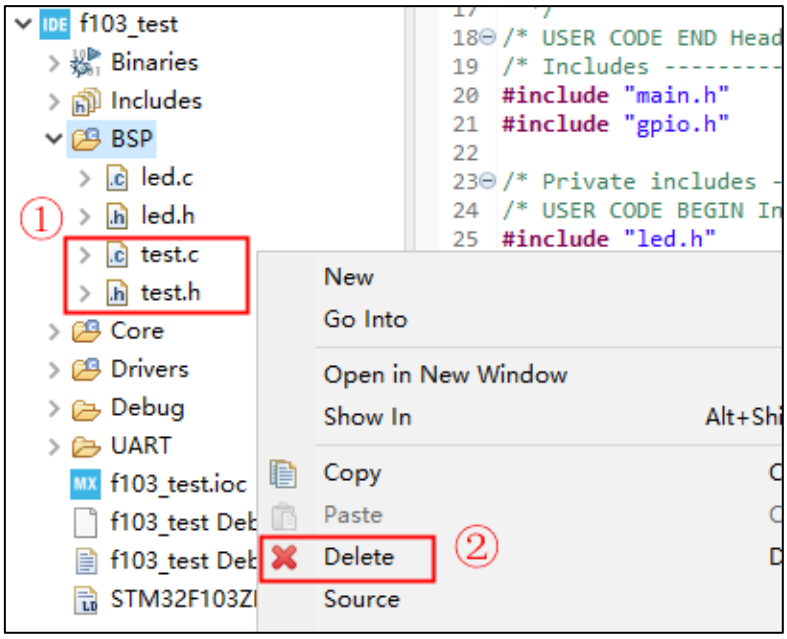


图 4.2.4.23 删除选中的文件

也可以直接选中 STM32CubeIDE 工作目录下的 CubeIDE_TEST 文件夹里的 test.c 和 test.h 两个文件，然后拖拽到工程中，例如直接将上述两个文件拖拽到工程的 BSP 目录下，然后弹出如下选项，如果选择 Copy files 的话，效果和前面的拷贝方式一样。

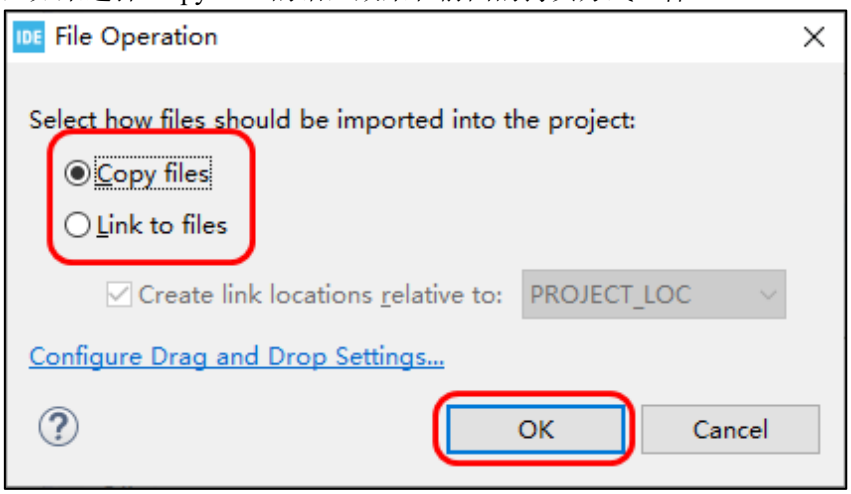


图 4.2.4.24 拖拽文件到工程中

如果选择的是 Link to files 的话，在 main.c 中#include "test.h"以后编译会报错找不到 test.h 文件，需要在 Paths and Symbols 中导入头文件路径才可以编译通过：

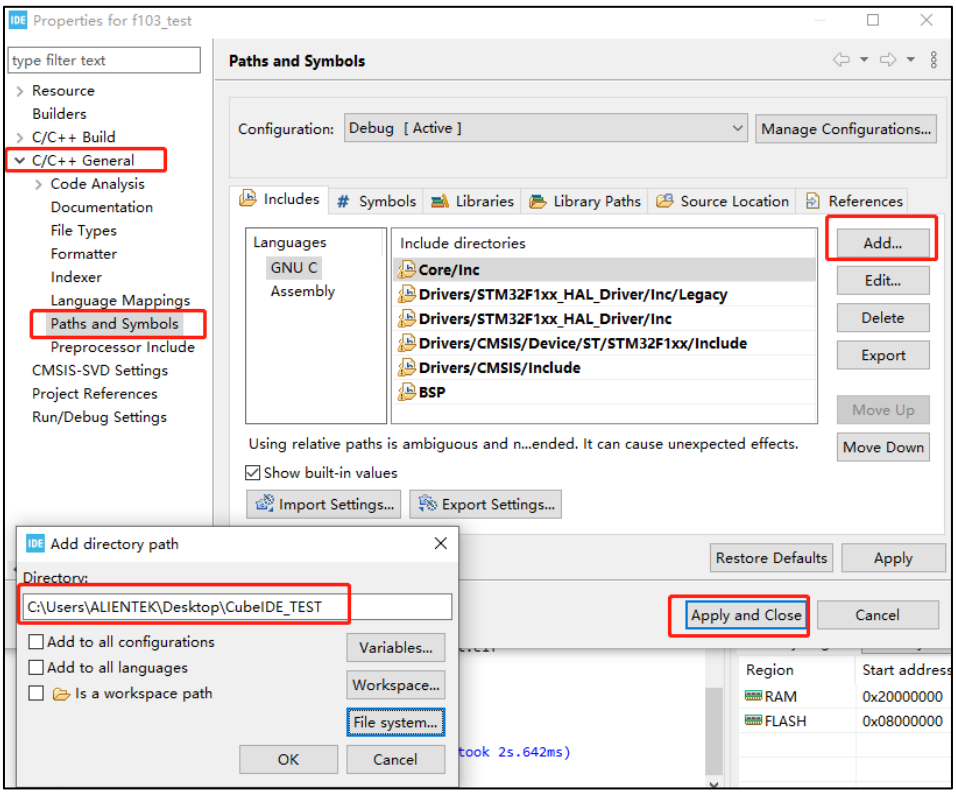


图 4.2.4.25 导入头文件路径

5. 导入已有文件的文件夹

如果已有一个包含源代码文件的文件夹，我们需要将其导入工程中，例如笔者已有的 SYSTEM 文件夹。可以先将文件夹拷贝到 f103_test 工程目录下。

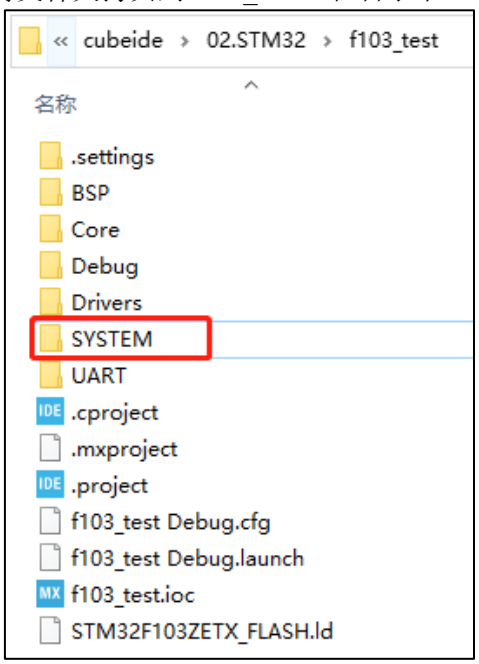


图 4.2.4.26 笔者的 SYSTEM 文件夹

拷贝好 SYSTEM 文件夹后，刷新一下工程，可以看到在 f103_test 工程下多了 SYSTEM 文件夹：

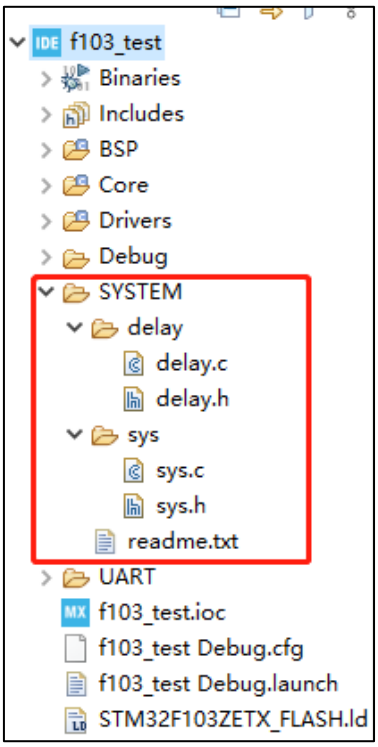


图 4.2.4.27 工程中多了一个 SYSTEM 文件夹

选择 f103_test 工程，右键选中 Properties，依次按照如下操作顺序添加 SYSTEM 文件夹中的 delay 文件夹和 sys 文件夹，这一步主要就是往工程中添加头文件：选择 C/C++General→Paths and Symbols→Includes→GNU C→Add→File system→选中 delay 文件夹→点击 Apply。

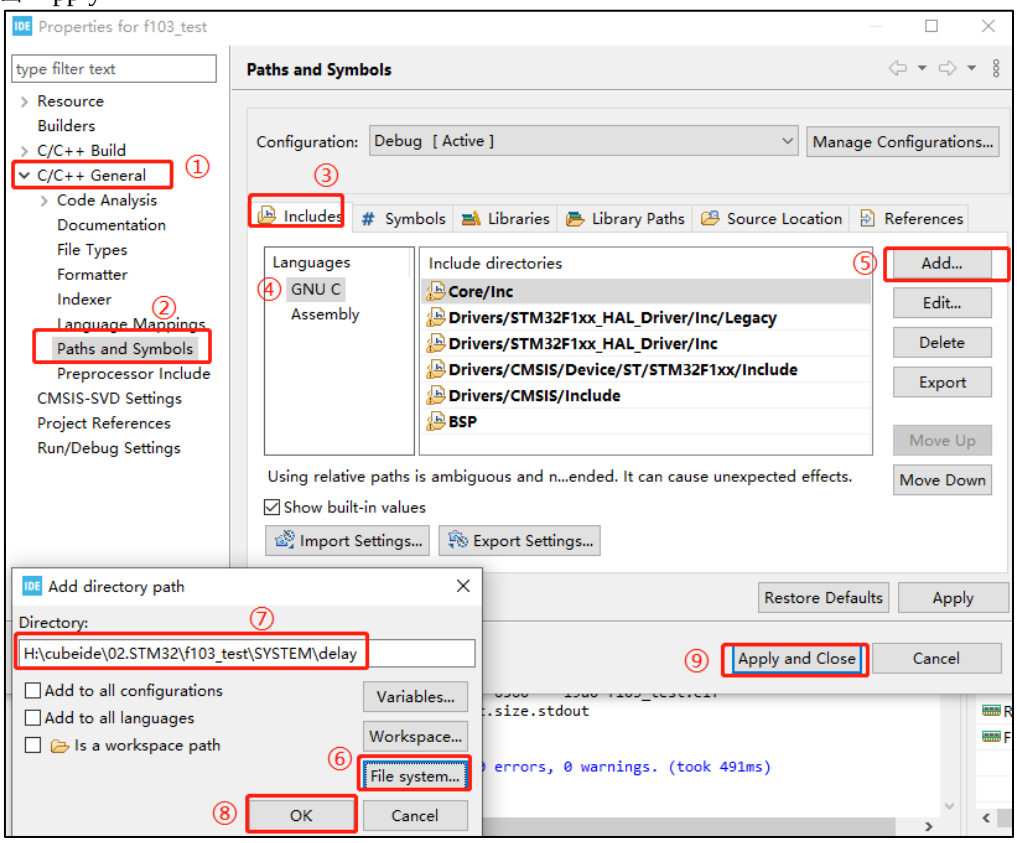


图 4.2.4.28 在工程中添加 delay 文件夹

接着按照同样的操作方法添加 sys 文件夹：

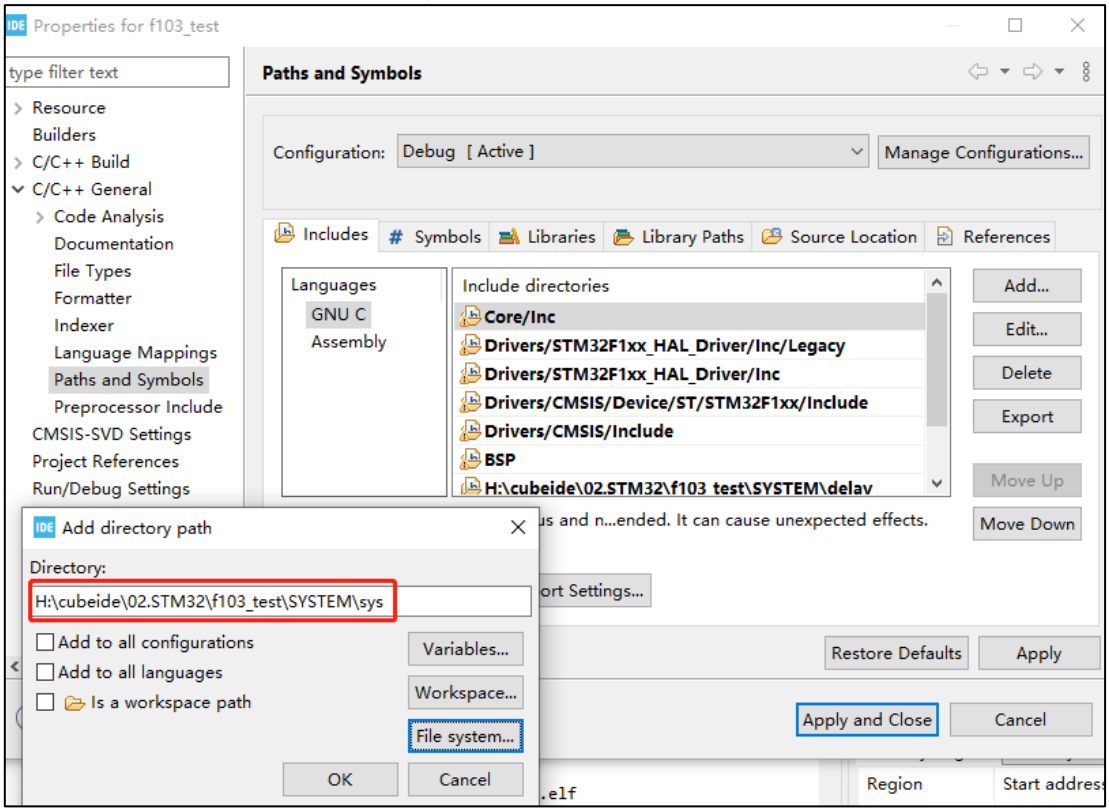


图 4.2.4.29 在工程中添加 sys 文件夹

添加完成后，在 Includes 中看到前面添加的路径：

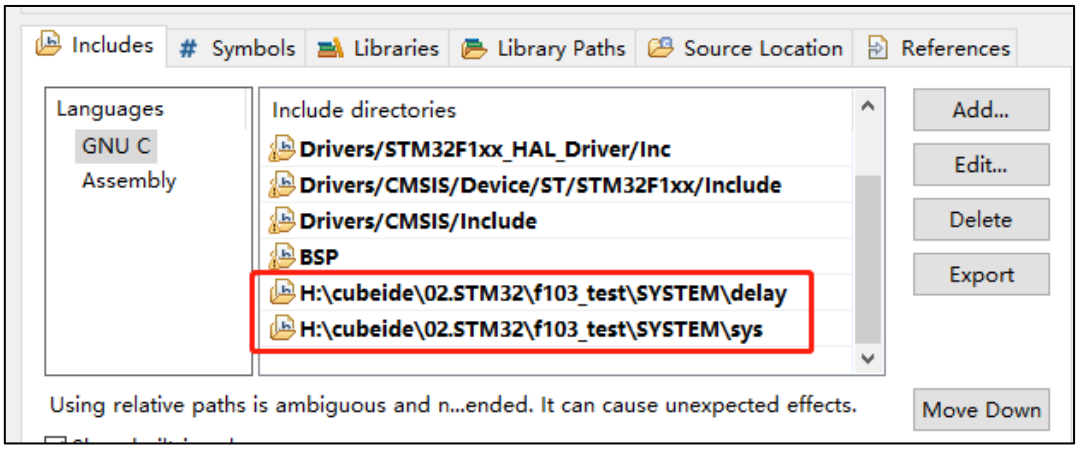


图 4.2.4.30 已添加的文件夹路径

再次进入 Properties for f103_test 配置项，依次按照如下操作顺序添加 SYSTEM 文件夹，这一步主要就是将 SYSTEM 文件夹和工程关联起来：选择 C/C++General→Paths and Symbols→Source Location→Add Folder→选择 SYSTEM 文件夹→OK→Apply and Close。

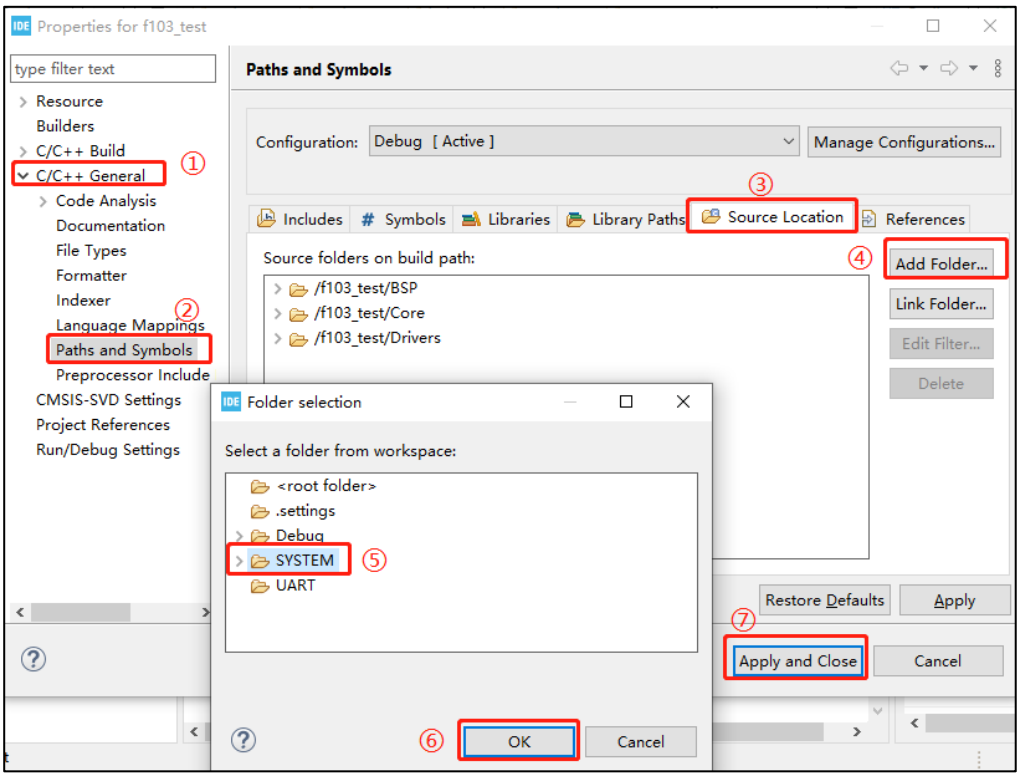


图 4.2.4.31 SYSTEM 文件夹和工程关联

回到工程窗口，我们发现 SYSTEM 文件夹左上角多了一个 C 符号，打开工程的 Includes 目录，发现多出了 f103_test/SYSTEM/delay 和 f103_test/SYSTEM/sys 两个路径，我们在 main.c 函数中包含 delay.h 和 sys.h 两个头文件：

```
/* USER CODE BEGIN Includes */
#include "led.h"
#include "../../UART/uart.h"
#include "delay.h"
#include "sys.h"
/* USER CODE END Includes */
```

修改完 main.c 代码，保存，并编译，如下图编译没有报错，添加成功。

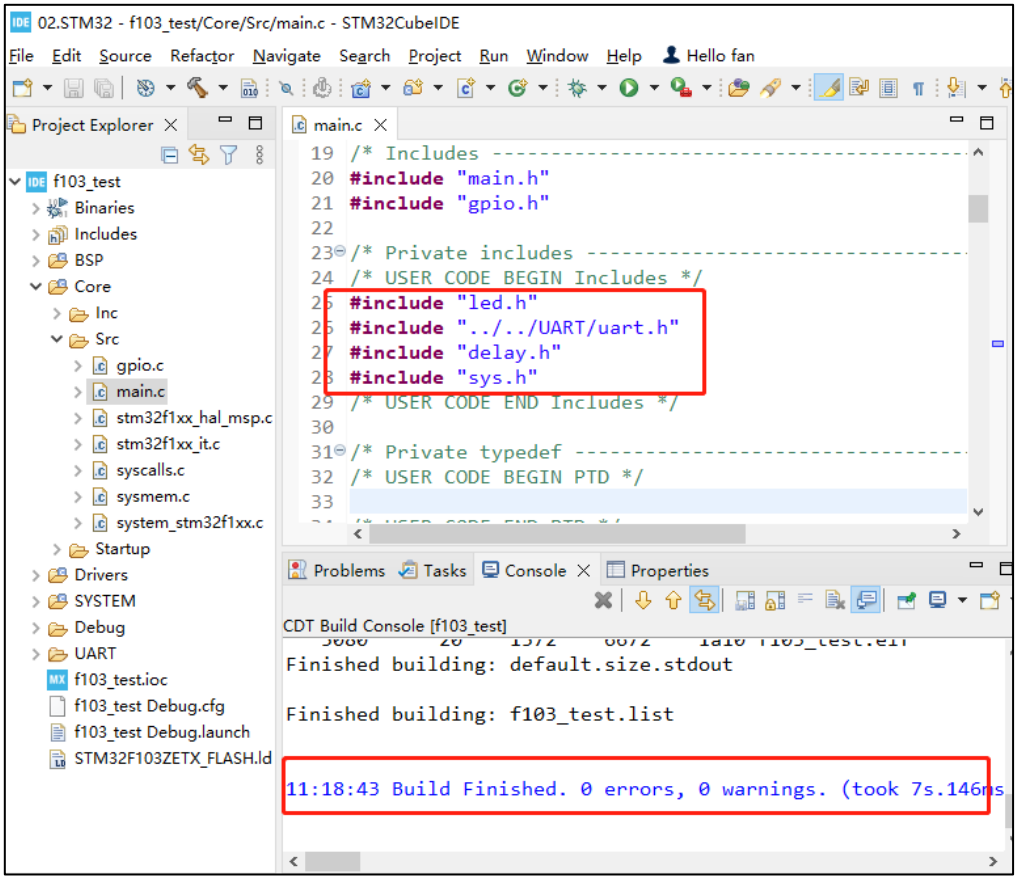


图 4.2.4.32 编译成功

此时我们打开刚刚导入的文件，例如打开 delay.c 文件，会发现原先的一些中文注释已经变成了乱码，如图所示。

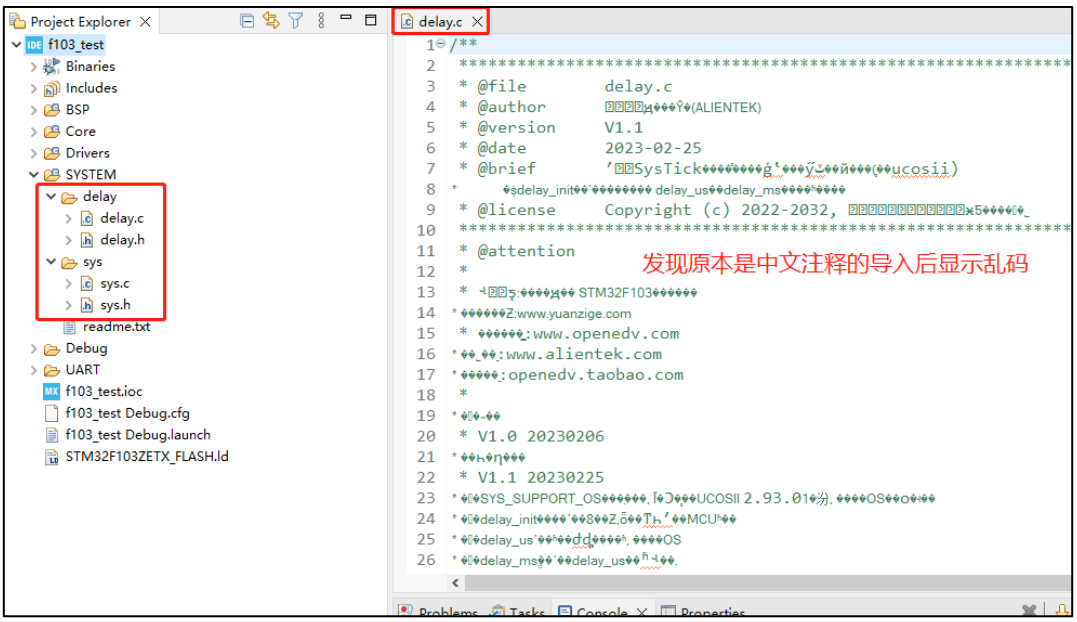


图 4.2.4.33 中文注释乱码显示

解决方法就是选择 f103_test 工程，右键选中 Properties，选择 Resource，在④处选择 GBK，如果没有该选项，就直接填入 GBK，然后点击⑤处即可。如下图：

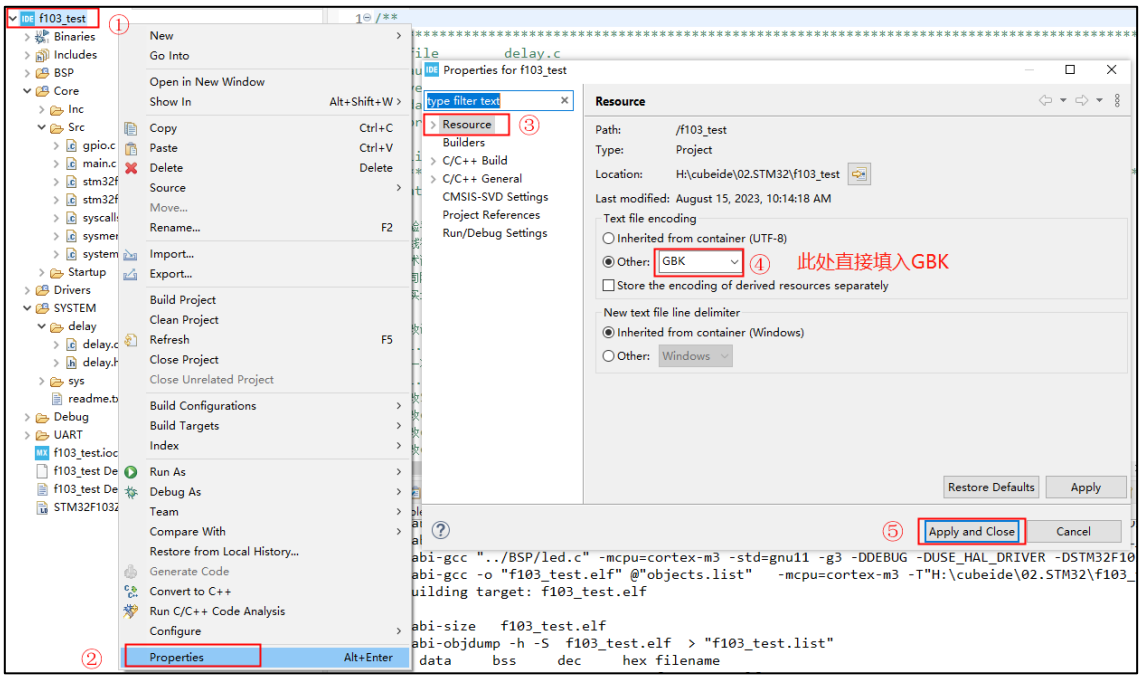


图 4.2.4.34 修改成 GBK

此时可以观察到 delay.c 文件中的中文注释已经恢复正常显示。

图 4.2.4.35 delay.c 中文注释正常显示

4.2.5 编译生成 BIN 或 HEX 文件

如何让编译生成 BIN 或 HEX 文件？

默认情况下 STM32CubeIDE 编译后生成的是 ELF 文件，如果想要生成 HEX 或 BIN 文件的话需要按照如下配置。选中工程，然后右键选择 Properties，或者点击菜单栏的 File→Properties→C/C++ Build→Settings→MCU Post build outputs→勾选 Convert to binary file(-O binary)和 Convert to Intel Hex file(-O ihex)。

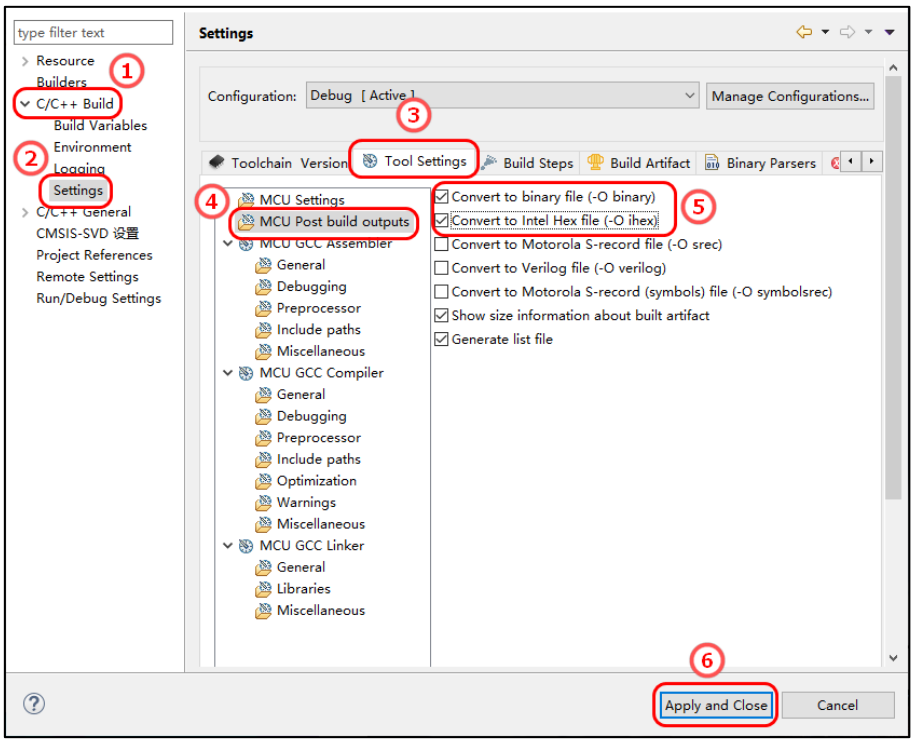


图 4.2.5.1 设置编译生成 BIN 和 HEX 文件

配置好后，保存配置，重新编译工程，在 f103_test 工程的 Debug 目录下生成 bin 文件、elf 文件和 hex 文件。

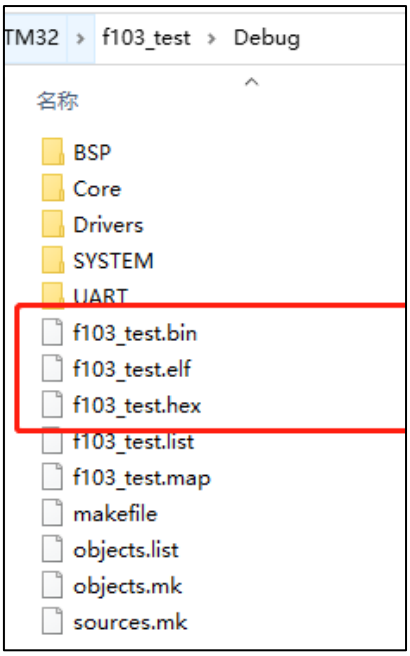


图 4.2.5.2 编译生成的文件

4.2.6 调试技巧

下面，我们讲解一些基本的调试技巧，掌握这些调试技巧可以帮助我们进行快速地开发，在后面的实验中，我们还会结合实验来掌握一些调试的方法。

1. 启用断言

在项目开发阶段，我们可能会借助断言来判断某个参数是否符合要求，那么就会开启断

言进行调试。什么是断言？关于断言我们后面的固件库分析和实验中会讲解到，下面我们先讲解怎么开启断言。

打开.ioc 文件进入 STM32CubeMX 插件配置界面，在 Project Manager 配置界面的 Code Generator 处配置 HAL 库，勾选 Enable Full Assert 开启断言，按下“Ctrl+S”组合键保存修改，重新编译工程。

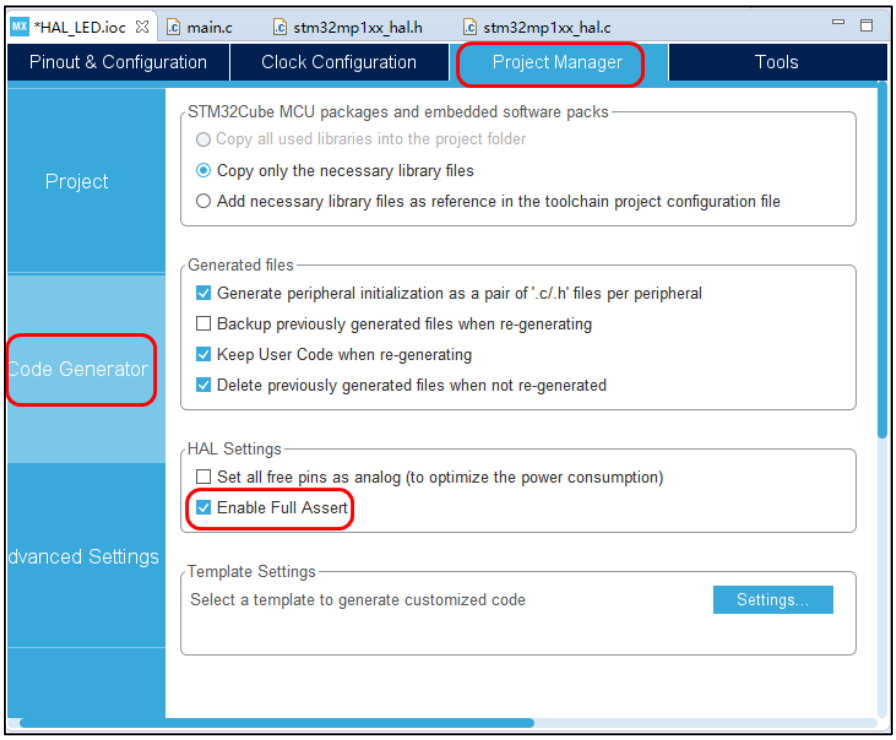


图 4.2.6.1 开启断言

也可以通过手动添加宏定义 USE_FULL_ASSERT 的方式开启断言。

2. 添加宏定义

如果工程中要定义某一个宏，可以直接在 STM32CubeIDE 上添加。选中 f103_test 工程右键然后点击 Properties 进入工程属性设置，点击 C/C++ General→Paths and Symbols→Symbols→Add→添加自己要定义的宏，这里为了演示，填写 FOR_TEST→点击 OK 确定添加→Apply。

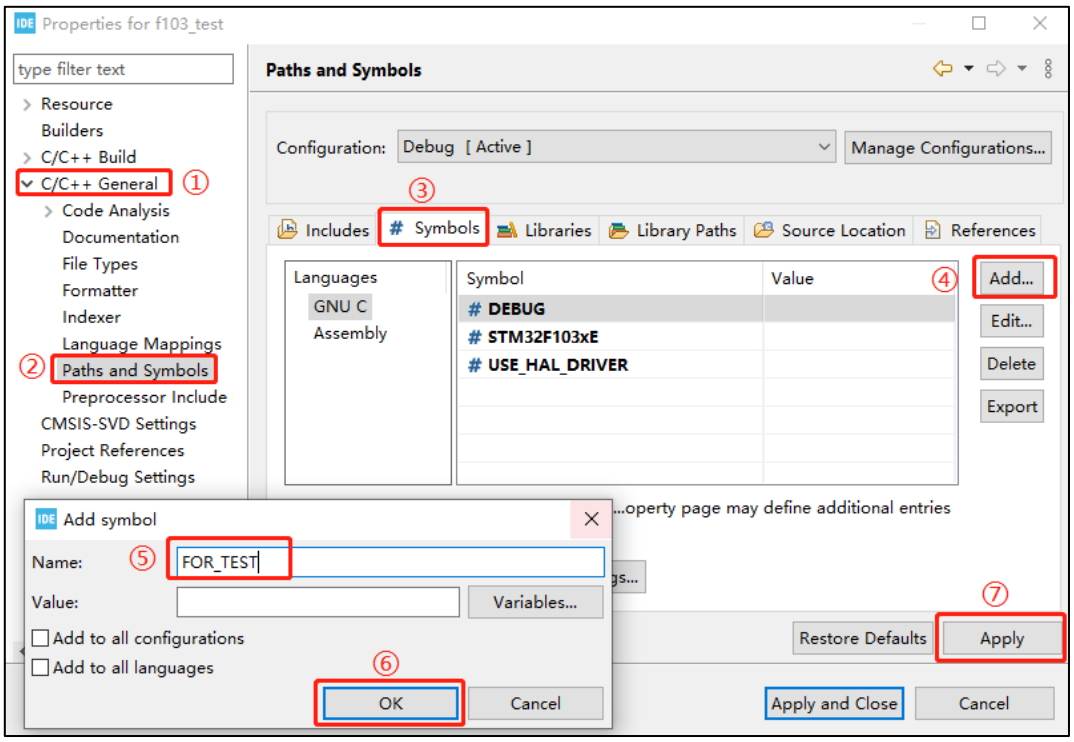


图 4.2.6.2 定义一个宏

添加完成后，在 Symbol 处可以看到已经添加的宏定义，添加完宏定义以后，记得重新编译工程。

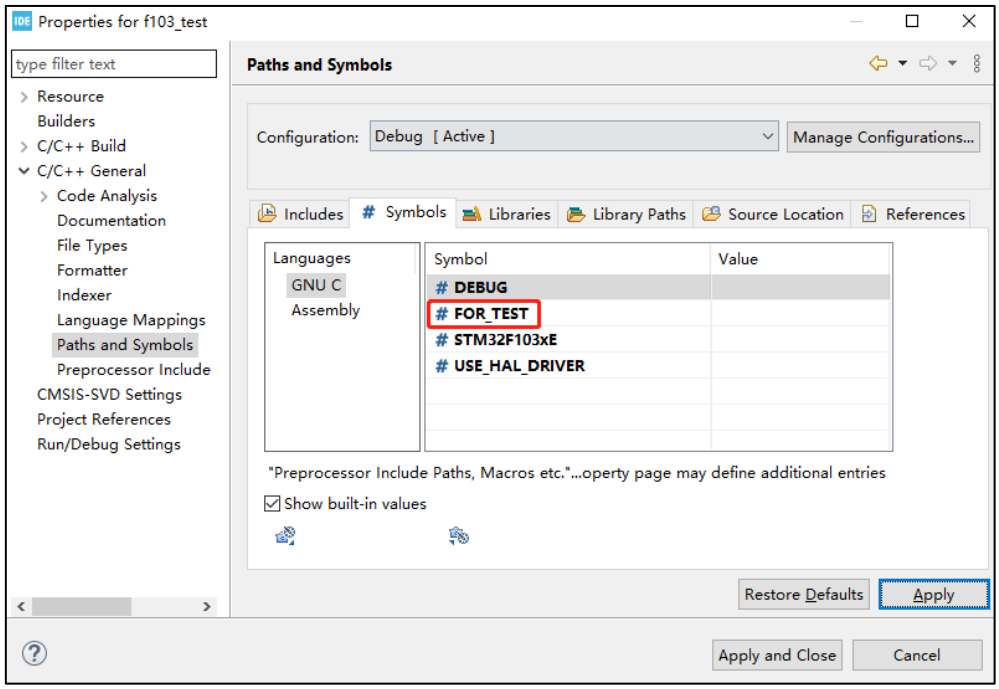


图 4.2.6.3 已经添加的宏

3. Debug 模式和 Release 模式切换

STM32CubeIDE 默认采用 Debug 模式。

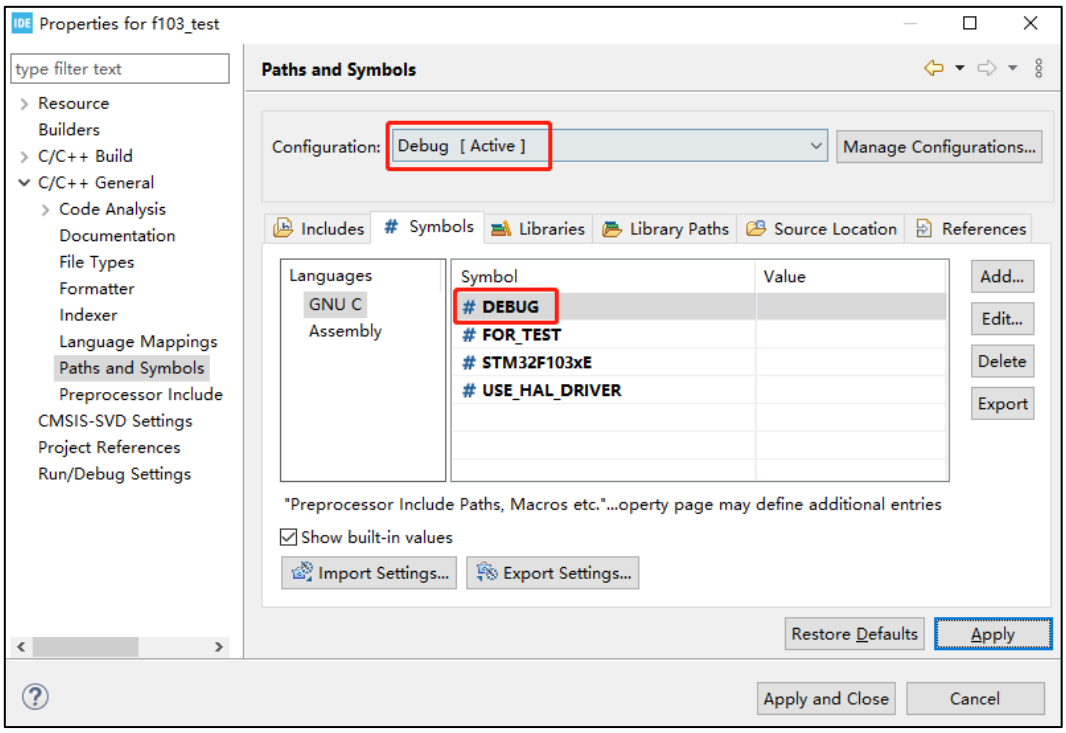


图 4.2.6.4 CubeIDE 默认采用 Debug 模式

Debug 模式，称为调试版本，编译结果通常包含调试信息，而且不做任何优化，只有 Debug 模式下才能设置断点、进行单步调试。另一个是 Release 模式，Release 称为发布版本，它往往是进行了各种优化，使得程序在代码大小和运行速度上都是最优的，以便用户很好地使用，Release 模式下，在线点击调试的时候，程序直接运行，但不能单步调试。如果要改为 Release 模式，可以按照如下步骤进行配置：

点击 Manage Configurations 处，在弹出的配置窗口选中 Release，然后选择 Set Active，表示默认激活点击 Release 模式，然后点击 OK 确认，修改完配置，记得点击 Apply and Close 使能配置：

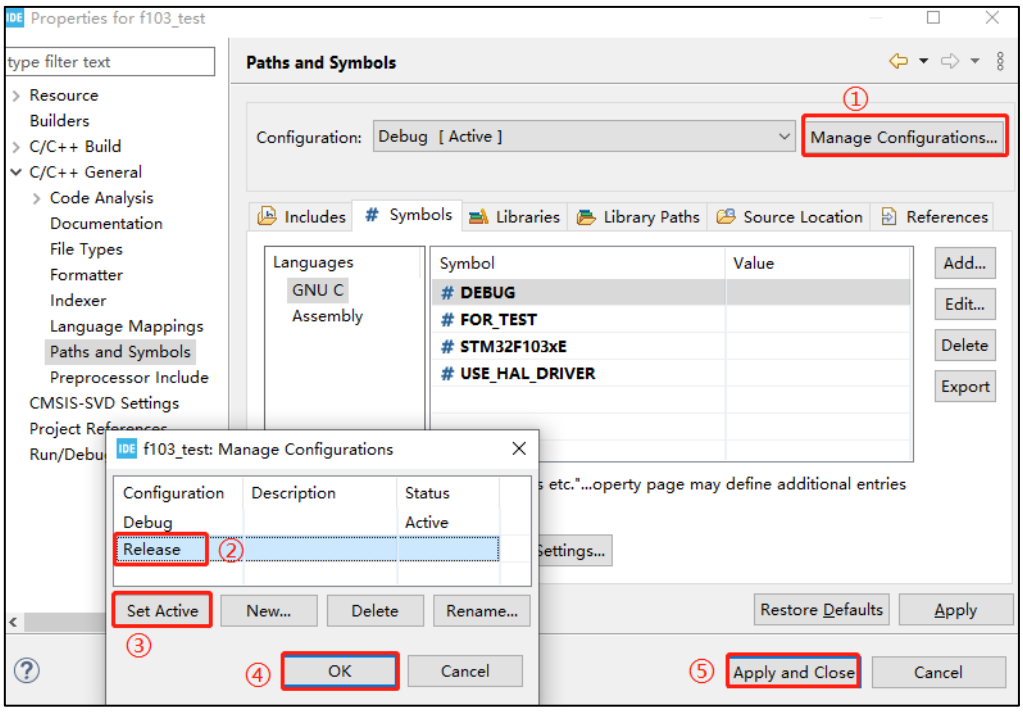


图 4.2.6.5 选择 Release 模式

配置完成后，再次进入 f103_test 工程的 Properties 配置界面，看到之前的 DEBUG 宏定义不见了：

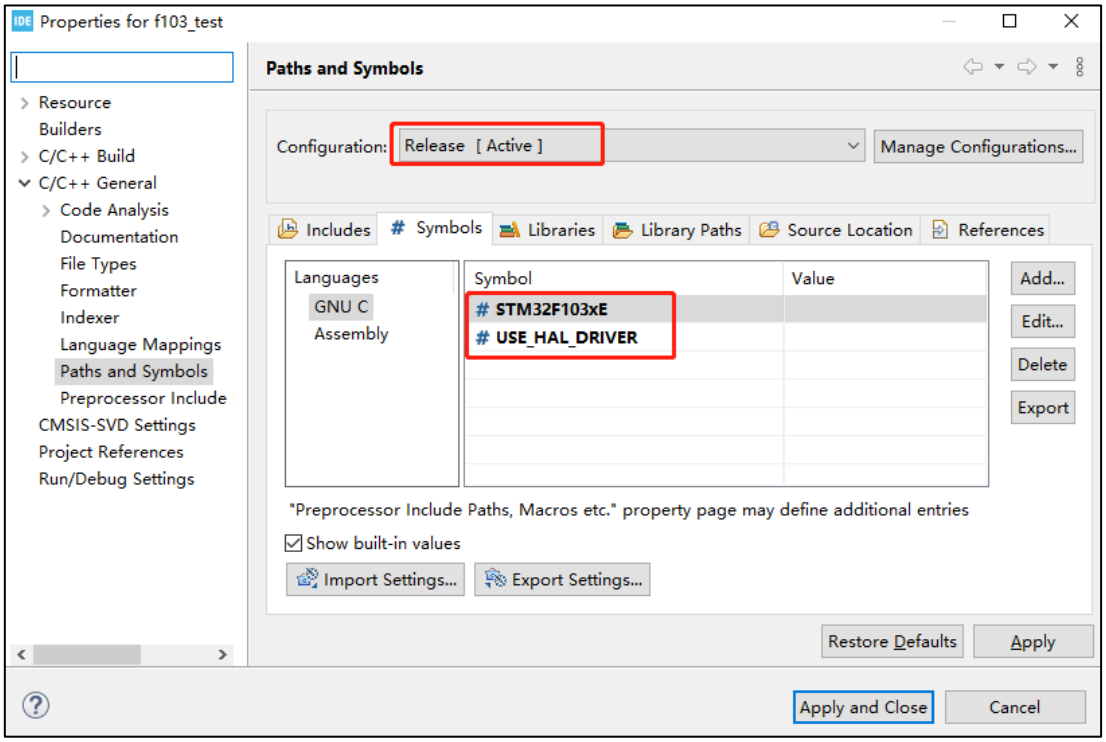


图 4.2.6.6 Release 模式配置成功

配置完成后，重新编译工程。不管是 Debug 模式还是 Release 模式，在工程的目录下会单独生成一个文件夹来存放编译生成的工程文件，在编译之后，可以对比一下生成文件的大小，可见 Release 模式下编译生成的文件比 Debug 模式下编译生成的文件还要小：

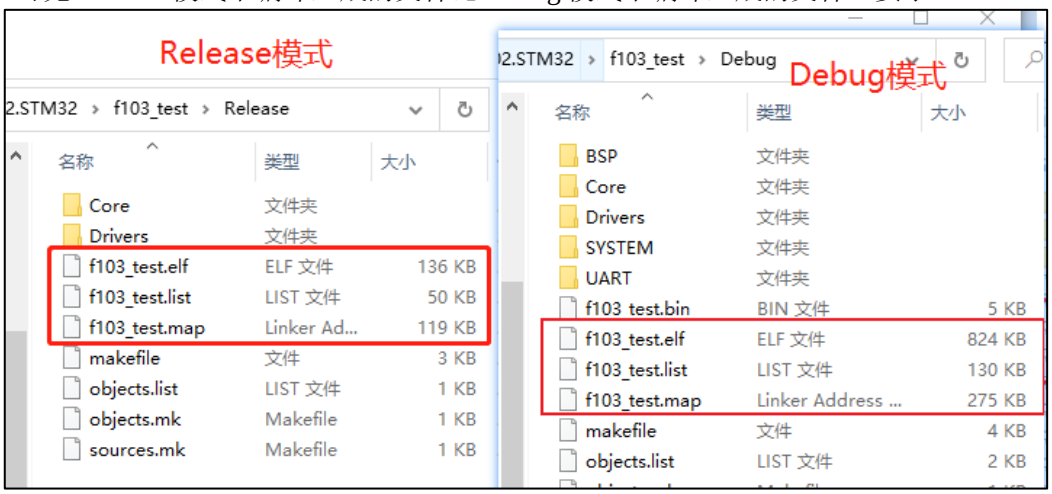


图 4.2.6.7 Debug 模式和 Release 模式编译生成的文件对比

以后，如果要编译工程，可以在小锤子下面选择是编译 Debug 版本还是 Release 版本

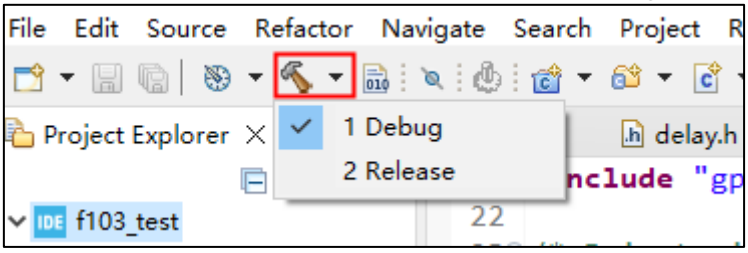


图 4.2.6.8 编译版本选择

或者可以直接在 Project 菜单栏选择设置激活 Debug 或者 Release 版本：

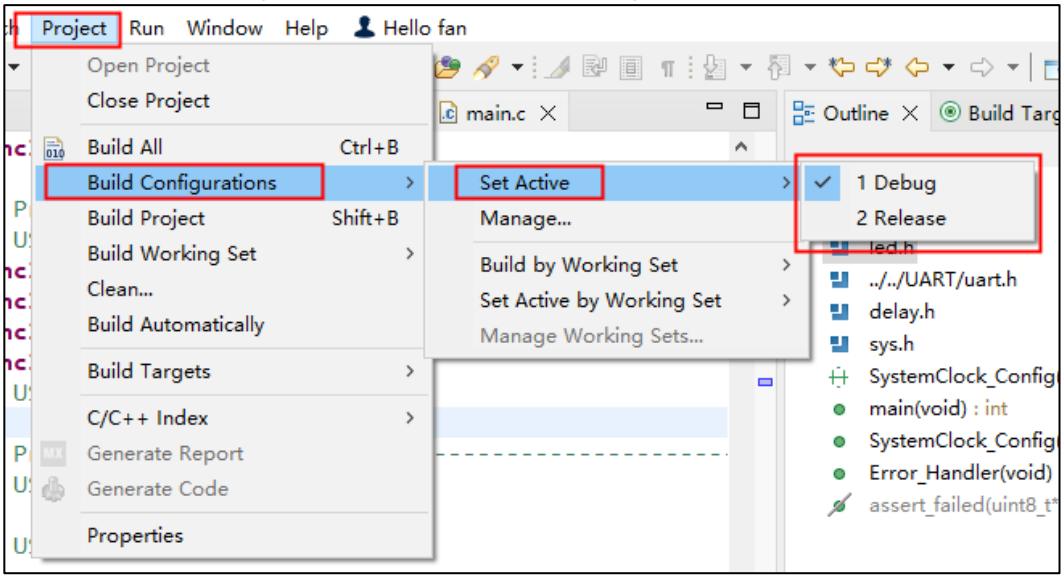


图 4.2.6.9 在 Project 中选择编译的版本

同样的，如果要调试，也可以选择 Debug 版本或者 Release 版本，要注意的是，如果要进行单步调试的话，一定要选择 Debug 版本。

4. 关闭工程

如果 STM32CubeIDE 中打开了多个工程，可能会干扰我们对某个工程的操作，那么我们可以将其他不用的工程关闭。如下图，笔者打开了 f103_test 工程和 g4_lcd 工程，如果要关闭 g4_lcd 工程，先选择 g4_lcd 工程，然后点击右键，再点击 Close Project 将其关闭：

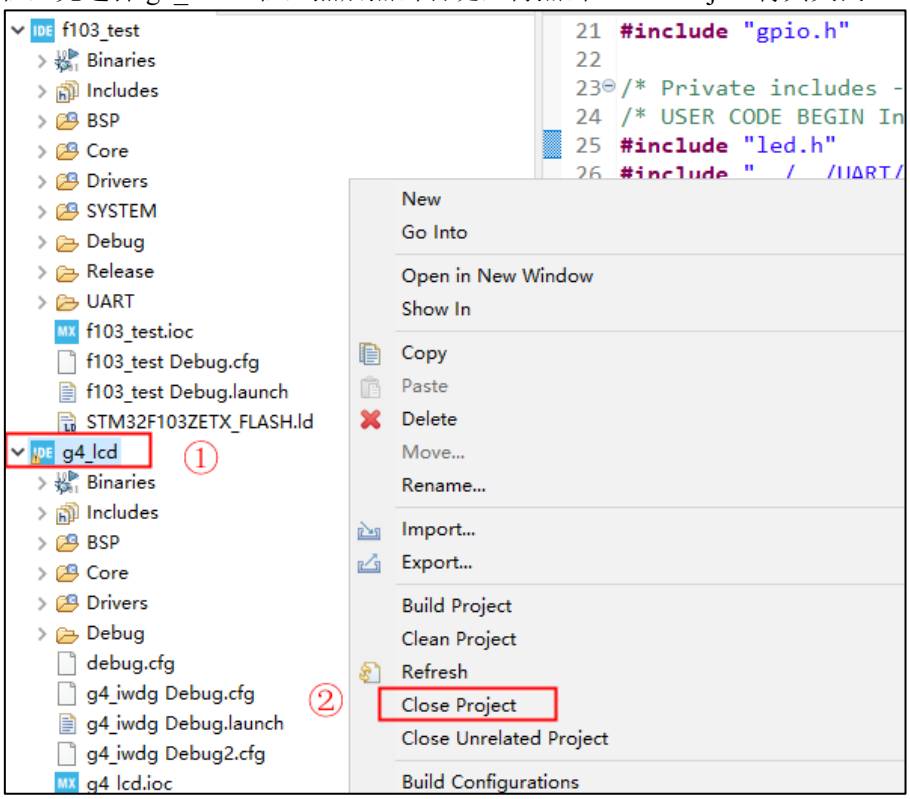


图 4.2.6.10 关闭工程

关闭后的效果如图：

图 4.2.6.11 已经关闭工程

5. 打开观察窗口

STM32CubeIDE 上集成了非常多的窗口，通过这些窗口我们可以获取一些信息。特别是在 Debug 的时候，可以选择打开这些窗口：

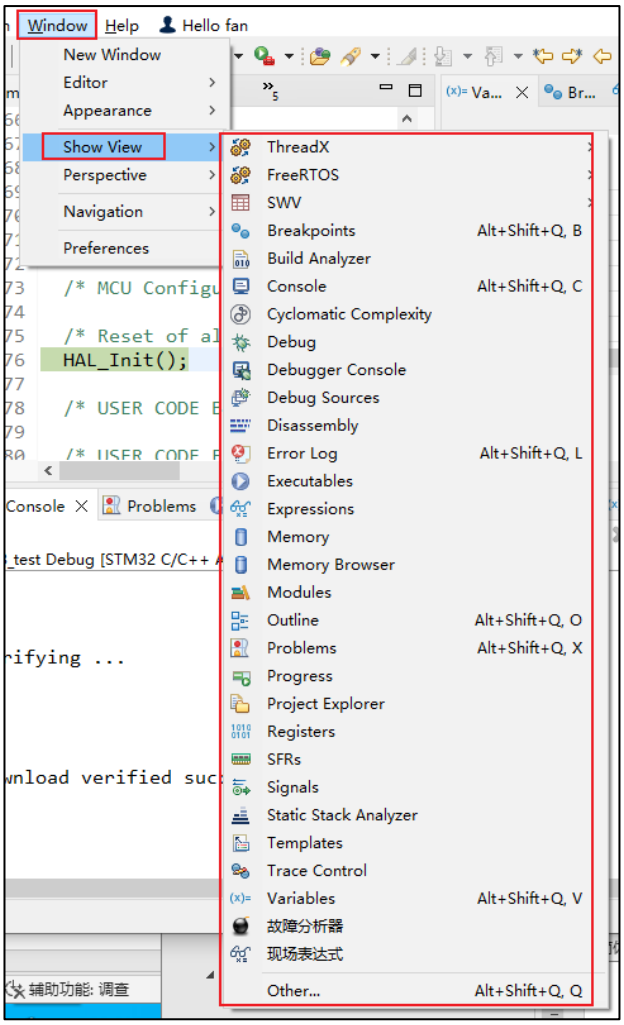


图 4.2.6.12 Debug 时可以打开的观察窗口

下面只是列举一部分常用的窗口，在后期，我们在实验调试中还会遇见一些窗口，结合实验来调试更容易理解。

（1）查看函数调用关系

如果想查看一个文件中有哪些函数，可以在项目浏览器那里打开此文件，如下图查看 main.c 文件中有哪些文件和函数：

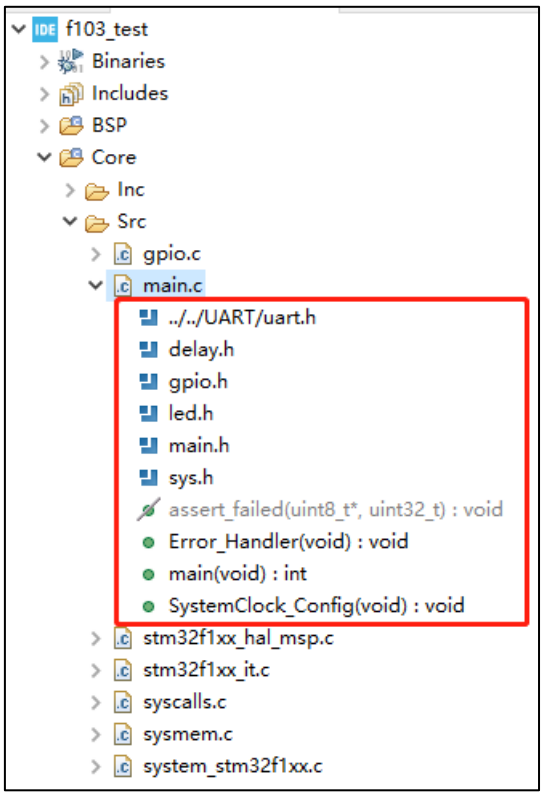


图 4.2.6.13 查看文件调用的函数

如果想看一个函数调用了那些函数，可以先选中此函数，然后右键，选中打开 Open Call Hierarchy：

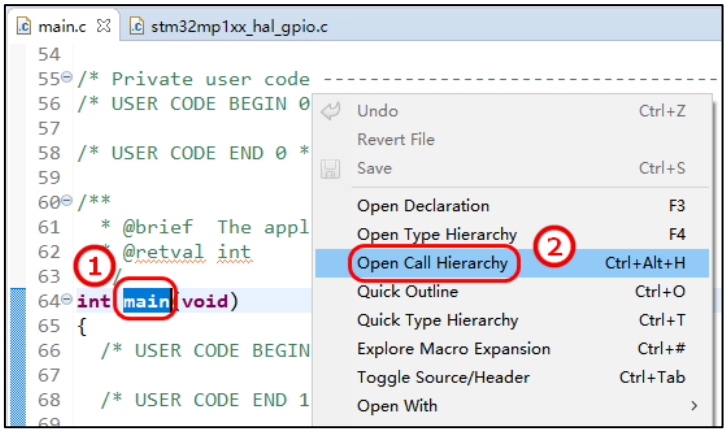


图 4.2.6.14 查看 main 函数调用关系

如下图，打开了 Open Call Hierarchy 窗口以后，可以查看各个函数的具体调用关系，点开>括号还可以层层查看函数的调用层级关系。

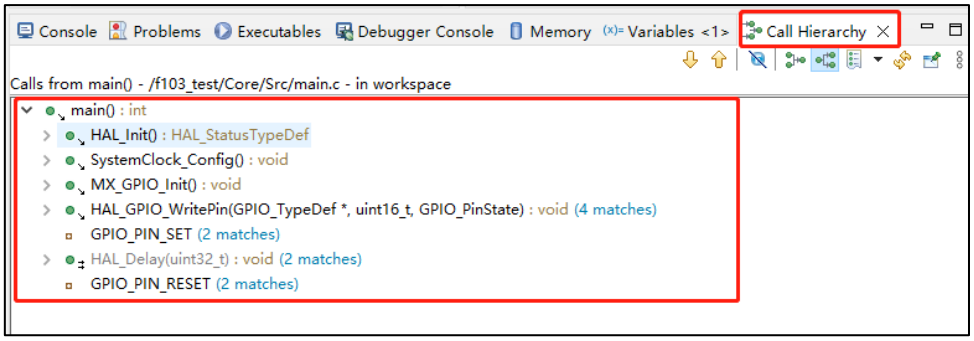


图 4.2.6.15 查看函数调用层级关系

（2）查看头文件调用关系

如果想查看一个文件调用了哪些头文件，以及这些头文件的调用关系，可以先选中此文件，右键，然后选择 Show In→Include Browser 打开包含关系列表：

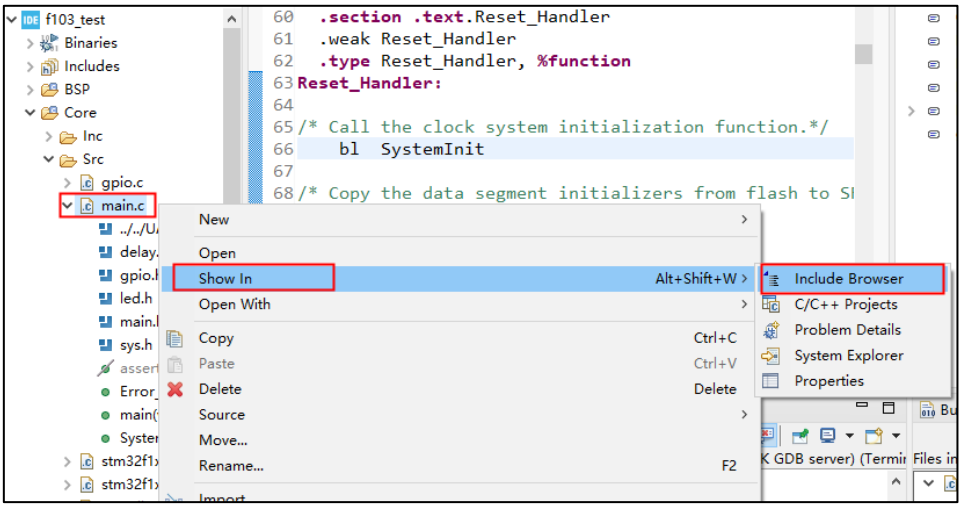


图 4.2.6.16 选择头文件包含列表

如下图，可以查看头文件的调用关系：

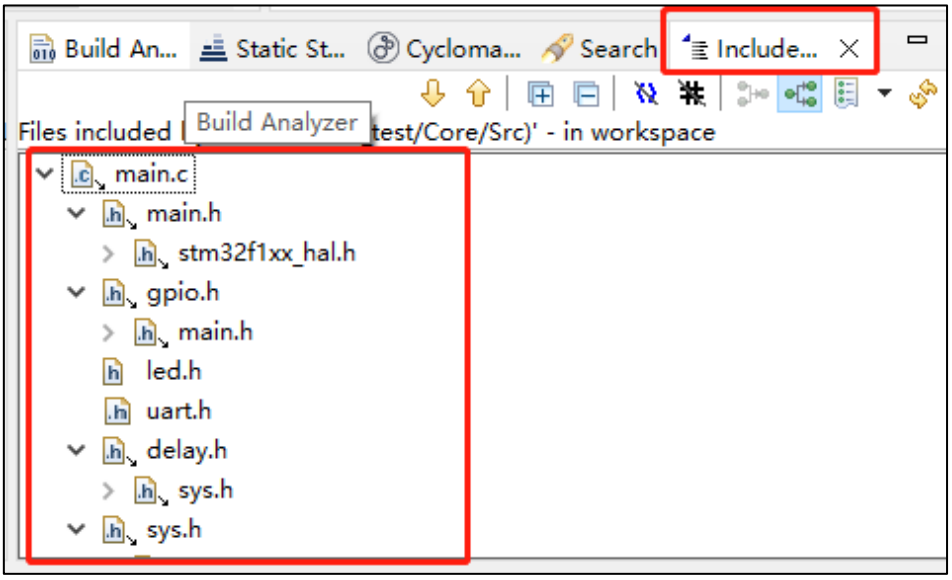


图 4.2.6.17 显示函数包含关系列表

（3）查找工程中某个文件

查找工程中某个文件，可以在项目类别里查询，如果工程中文件很多，也可以直接通过通　配符*来搜索。打开菜单栏 Navigate→open Resource→：

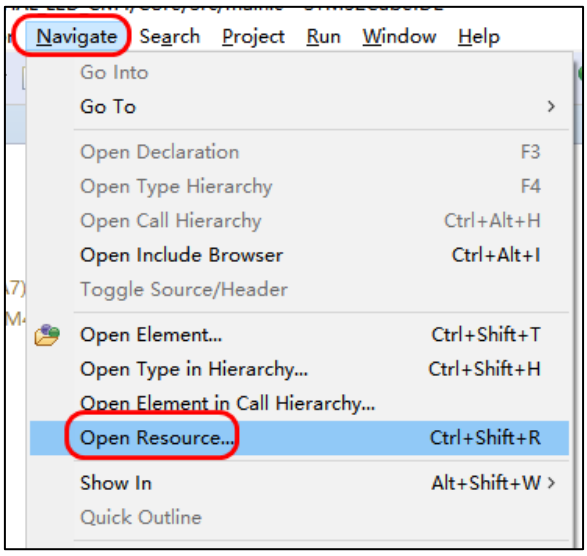


图 4.2.6.18 打开资源文件搜索窗口

例如搜索 gpio 相关的文件，可以在搜索框中输入*gpio，这样就会列出 gpio 相关的函数。

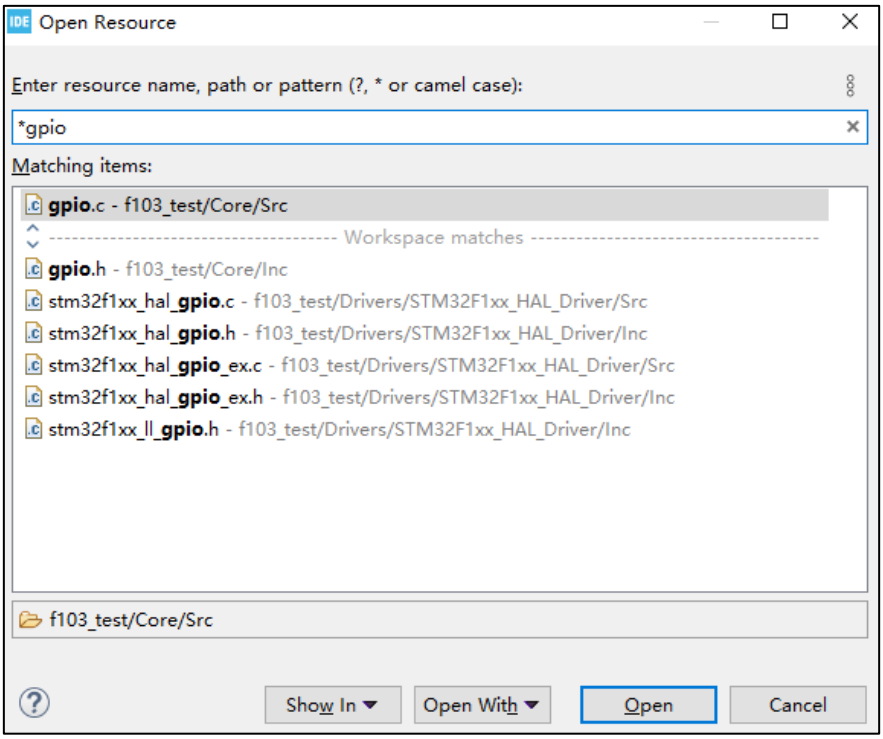


图 4.2.6.19 查找文件

（4）打开 Build Analyzer 窗口

Build Analyzer 窗口可以查看内存信息。点击菜单栏的 Window→Show View→Build Analyzer 可以打开 Build Analyzer 窗口。

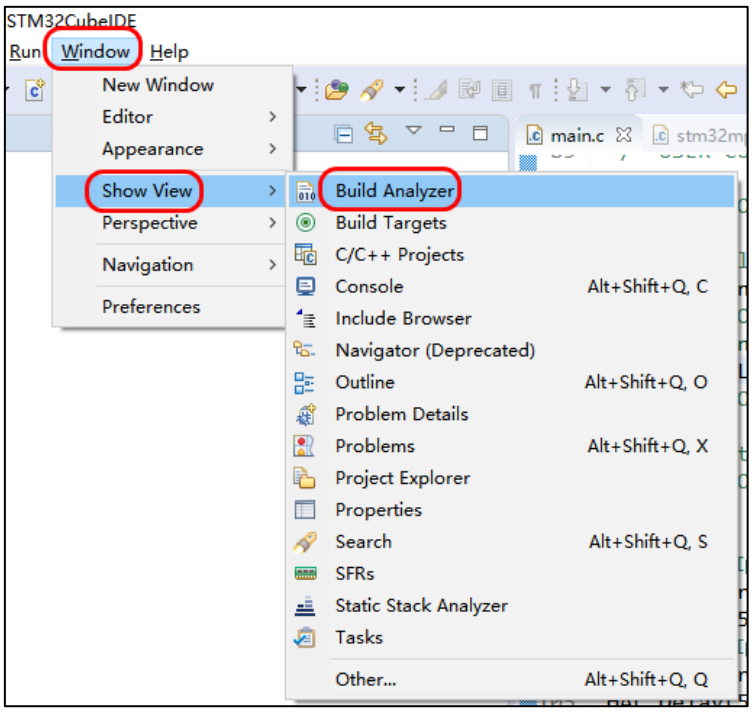


图 4.2.6.20 打开 Build Analyzer 窗口

在 Memory Regions 可以查看内存的分配区域，例如下图，内存地址 0x20000000~0x2000ffff 是 RAM 的一个范围，大小是 64KB，FLASH 内存地址范围是 0x08000000~0x0807ffff，大小是 512KB。

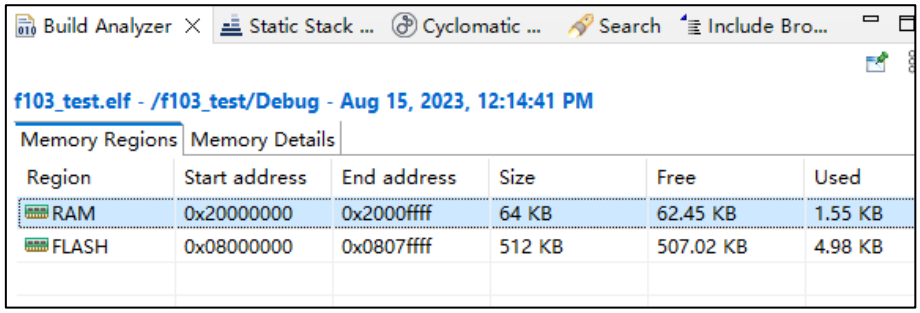


图 4.2.6.21 Memory Regions 列表

Memory Details 下是具体的内存映射关系，其中 VMA 表示虚拟地址，一般是 RAM 位置，LMA 一般是加载地址：

Build Analyzer | Static Stack Analyzer | Cyclomatic Complexity | Search | Include Browser

f103_test.elf - /f103_test/Debug - Aug 15, 2023, 12:14:41 PM

Memory Regions | Memory Details

Name	Run address (VMA)	Load address (LMA)	Size
FLASH	0x08000000		512 KB
.isr_vector	0x08000000	0x08000000	484 B
.text	0x080001e4	0x080001e4	4.45 KB
.rodata	0x080013b4	0x080013b4	36 B
.preinit_array	0x080013d8	0x080013d8	0 B
.init_array	0x080013d8	0x080013d8	4 B
.fini_array	0x080013dc	0x080013dc	4 B
.data	0x20000000	0x080013e0	12 B
RAM	0x20000000		64 KB
.data	0x20000000	0x080013e0	12 B
.bss	0x2000000c		36 B
._user_heap_stack	0x20000030		1.5 KB

图 4.2.6.22 Memory Details 列表

（5）Memory 窗口

Memory 窗口和上面的 Build Analyzer 窗口的内存映射关系窗口不一样，Memory 窗口可以查看具体的某个地址的数据，此项是在调试的时候才可以用。例如我们进入 Debug 调试模式，然后点击菜单栏的 Window→Show View→ Memory：

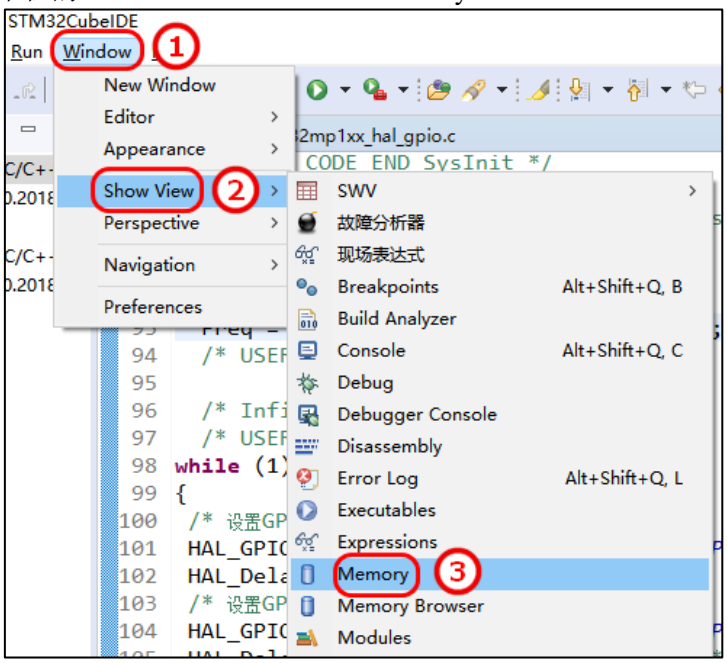


图 4.2.6.23 选择打开 Memory 窗口

在 Debug 处打开的 Memory 窗口如下：

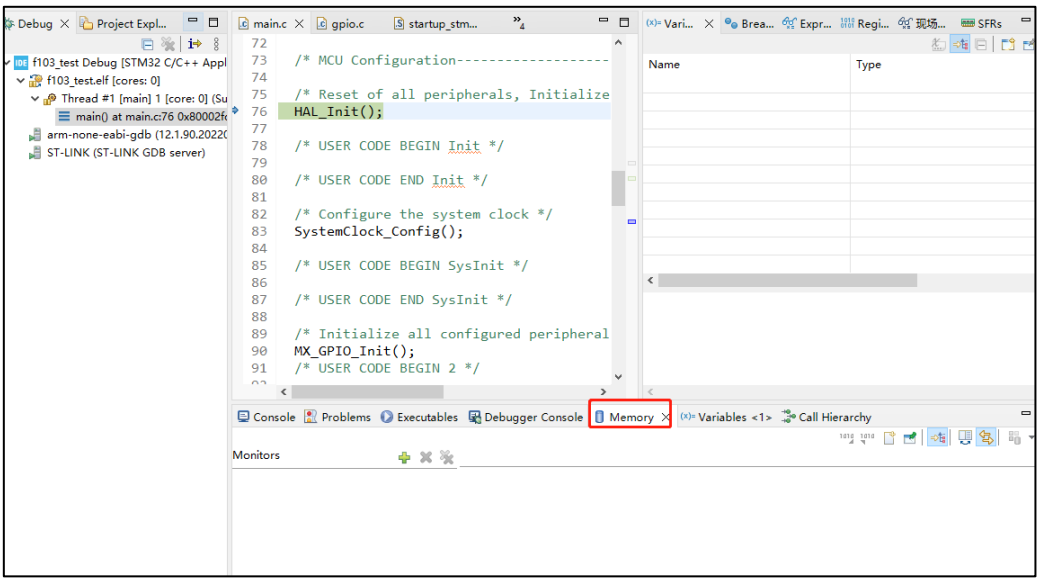


图 4.2.6.24 已经打开的 Memory 窗口

下面我们要观察某一段内存的数据，可以点击+号图标，然后再弹出的窗口写入地址，例如地址 0x20008000，然后点击 OK：

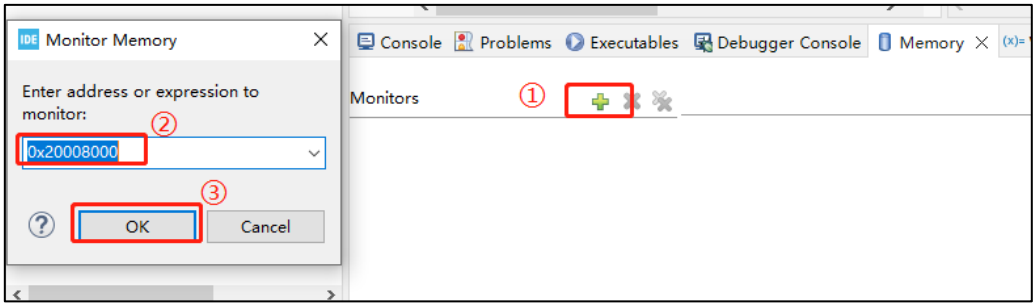


图 4.2.6.25 添加要观察的地址

在 Memory 窗口会列出从地址 0x20008000 开始以后的地址数据，是 16 进制的数据，且每 4 位为一个单元。

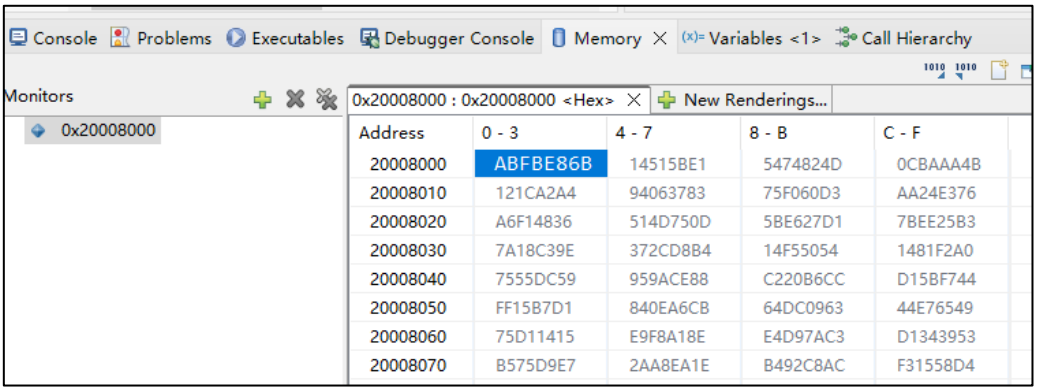


图 4.2.6.26 每 4 位为一个单元显示

如果觉得这样看着不是很方便，也可以将 4 位为一个单元改成每 1 位为一个单元，先将鼠标放在旁边的空白处，然后右键点击，选择 Format:

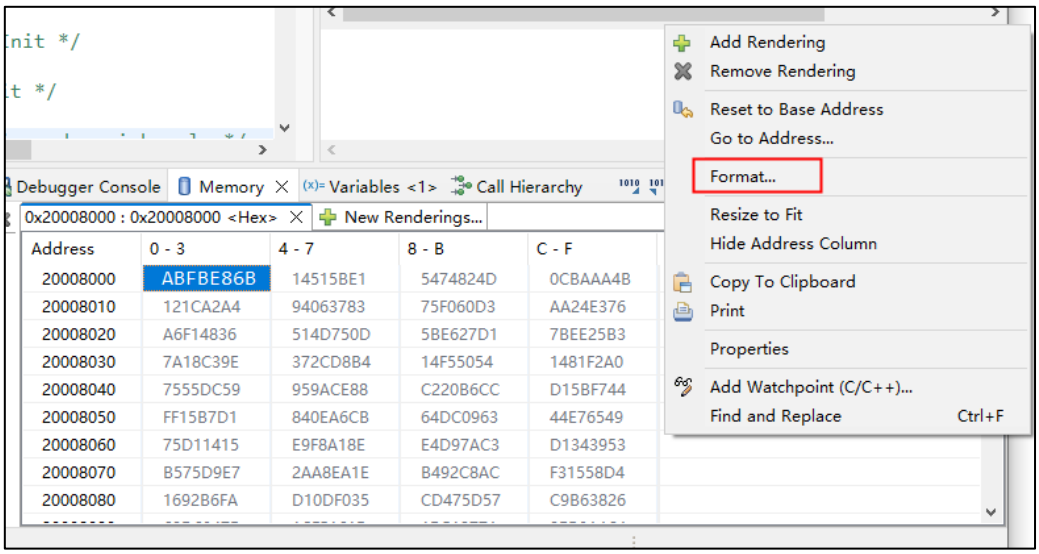


图 4.2.6.27 每 1 位为一个单元显示

然后可以选择进制数，可以是 8 进制、16 进制、2 进制或者其他，可以选择多少位来显示，这里我们选择还是 16 进制，以 1 位单元来显示：

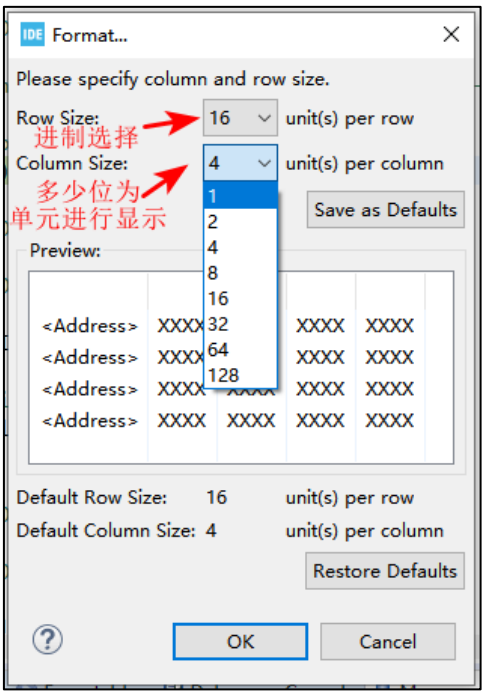


图 4.2.6.28 配置显示方式

效果如图：

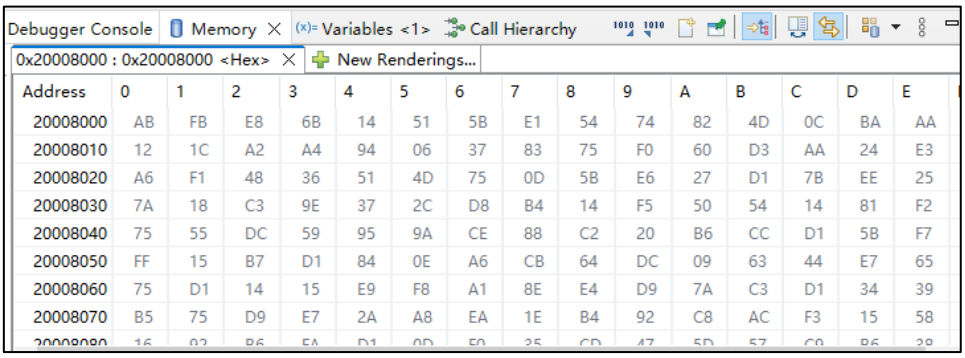


图 4.2.6.29 每 1 位为一个单元显示

（6）Static Stack Analyzer 窗口

Static Stack Analyzer 是静态堆栈分析窗口，如果有必要，也可以打开此窗口进行分析。点击菜单栏 Window→Show View→Static Stack Analyzer 打开静态堆栈分析窗口：

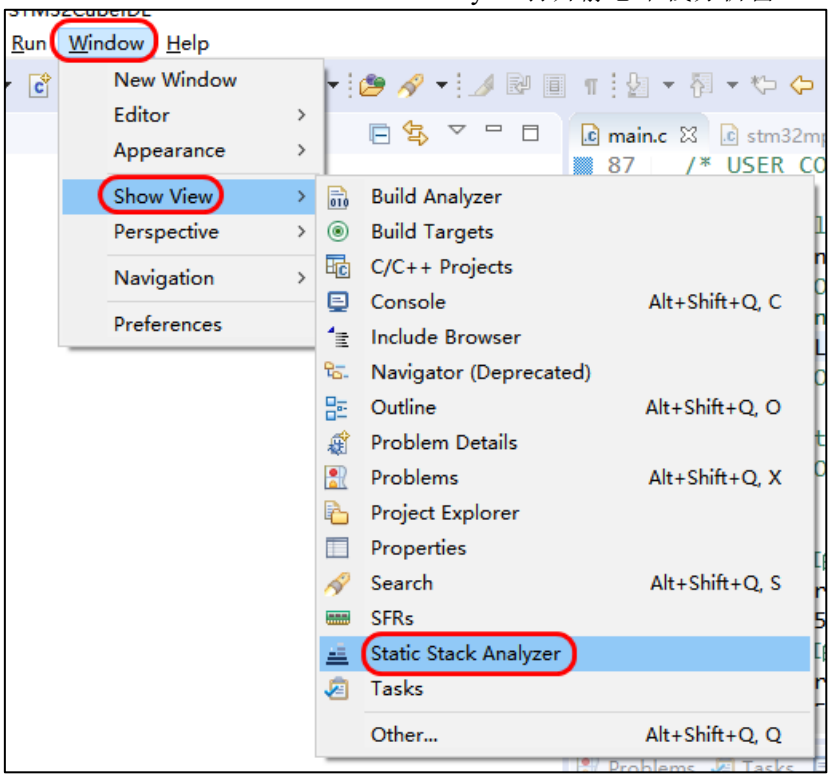
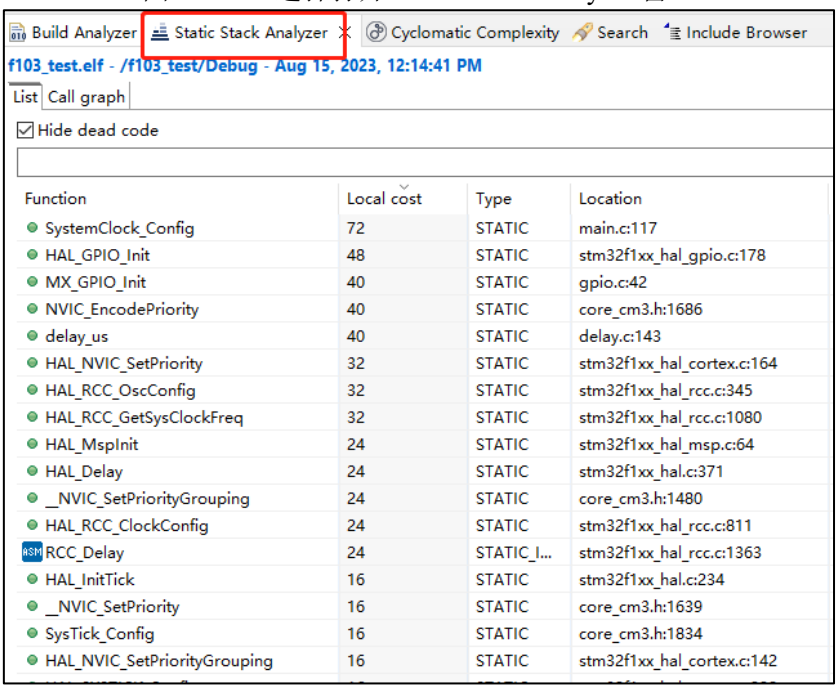


图 4.2.6.30 选择打开 Static Stack Analyzer 窗口



Function	Local cost	Type	Location
SystemClock_Config	72	STATIC	main.c:117
HAL_GPIO_Init	48	STATIC	stm32f1xx_hal_gpio.c:178
MX_GPIO_Init	40	STATIC	gpio.c:42
NVIC_EncodePriority	40	STATIC	core_cm3.h:1686
delay_us	40	STATIC	delay.c:143
HAL_NVIC_SetPriority	32	STATIC	stm32f1xx_hal_cortex.c:164
HAL_RCC_OscConfig	32	STATIC	stm32f1xx_hal_rcc.c:345
HAL_RCC_GetSysClockFreq	32	STATIC	stm32f1xx_hal_rcc.c:1080
HAL_MspInit	24	STATIC	stm32f1xx_hal_msp.c:64
HAL_Delay	24	STATIC	stm32f1xx_hal.c:371
__NVIC_SetPriorityGrouping	24	STATIC	core_cm3.h:1480
HAL_RCC_ClockConfig	24	STATIC	stm32f1xx_hal_rcc.c:811
RCC_Delay	24	STATIC_I...	stm32f1xx_hal_rcc.c:1363
HAL_InitTick	16	STATIC	stm32f1xx_hal.c:234
__NVIC_SetPriority	16	STATIC	core_cm3.h:1639
SysTick_Config	16	STATIC	core_cm3.h:1834
HAL_NVIC_SetPriorityGrouping	16	STATIC	stm32f1xx_hal_cortex.c:142

图 4.2.6.31 List 列表

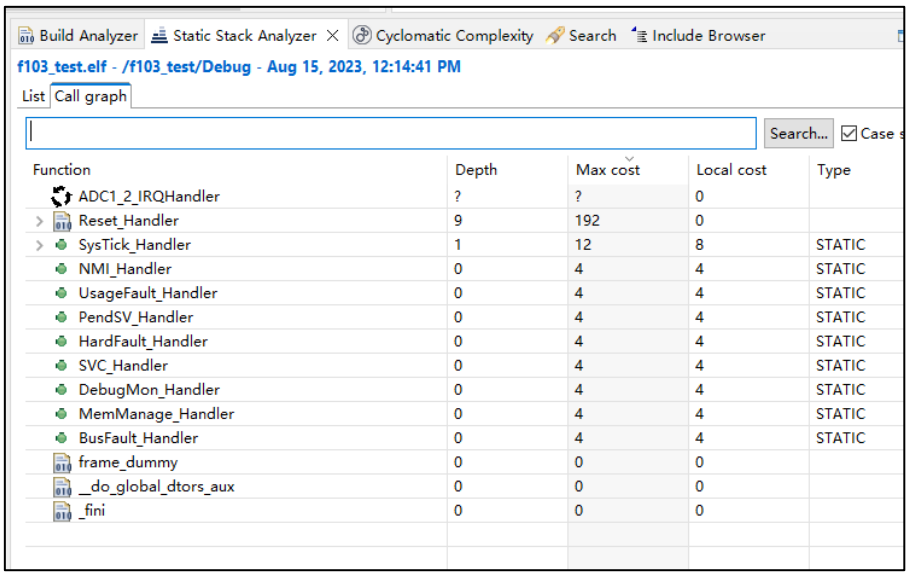


图 4.2.6.32 Call graph 列表

6. 单步调试

为了方便讲解调试部分内容，我们在 main.c 文件中添加如下代码，然后重新编译工程：

```
static uint32_t Freq = 0;
Freq = HAL_RCC_GetSysClockFreq();
```

这里定义一个静态局部变量 Freq，初始值为 0。HAL_RCC_GetSysClockFreq 是 HAL 库的一个函数，表示获取系统时钟。这里表示将系统时钟的值赋值给 Freq 变量，然后我们通过后面的调试来观察此变量。

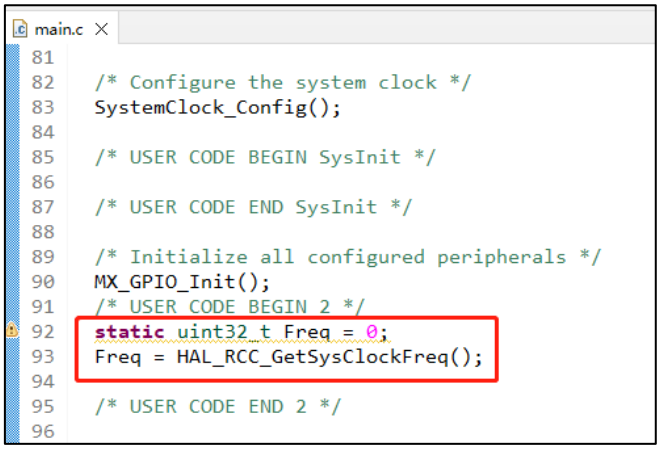


图 4.2.6.33 添加代码

编译不报错以后，开发板通过转接板和 ST-Link 相连，ST-Link 通过 USB 线接在电脑 USB 口上，具体操作请看第 4.1.4 小节。连接好硬件以后，打开开发板的开关，点击 Debug Configurations..进入调试配置界面（如果之前在工程中已经配置过调试选项，可以直接点击 进行调试）：

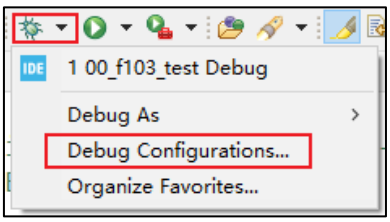


图 4.2.6.34 配置调试

进入调试配置项以后，按照第 4.1.4 小节操作，最后进入 Debug 界面。进入 Debug 界面

以后，箭头默认指向 HAL_Init();函数处，表示程序从此处开始执行，而且可以看到此时的 Freq 变量的值为 0：

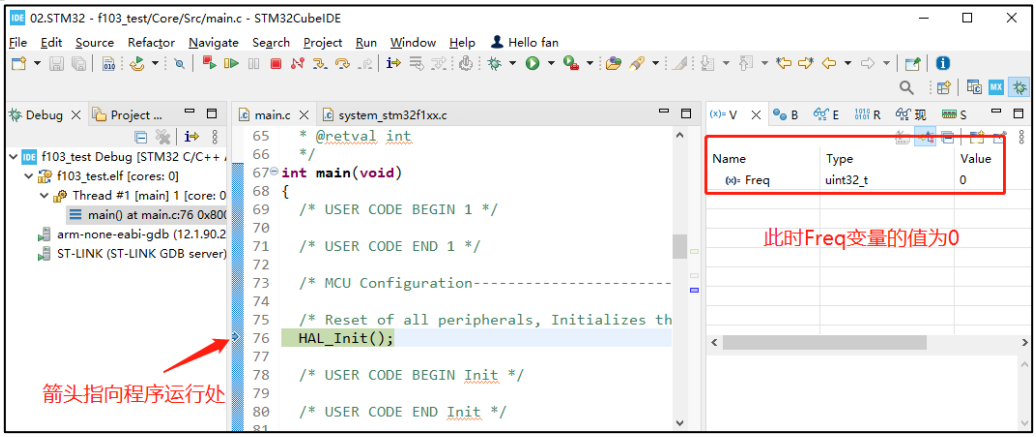


图 4.2.6.35 进入 Debug 界面

按下 F6 键或者点击 Step Over 符号（表示执行过去），箭头会跟着移动，继续点击 Step Over 符号，待箭头到了代码第 101 行以后，发现 Freq 变量的值变成了 72000000。

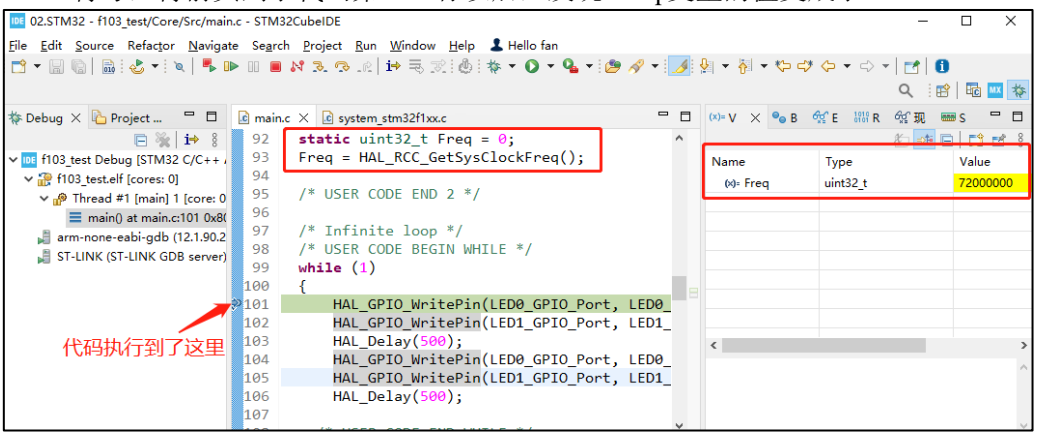


图 4.2.6.36 观察变量 Freq 的变化

再点一下 Step Over 符号，箭头到了代码第 102 行以后，开发板底板的 DS0 灯灭了。继续点击 Step Over 符号，箭头到了第 104 行代码以后 DS0 亮了，再继续点击，箭头到了第 105 行后 DS1 也亮了，如果此时继续点击 Step Over，箭头就又跳到代码第 101 行处，如果继续点击 Step Over，箭头在 whlie 函数里循环。

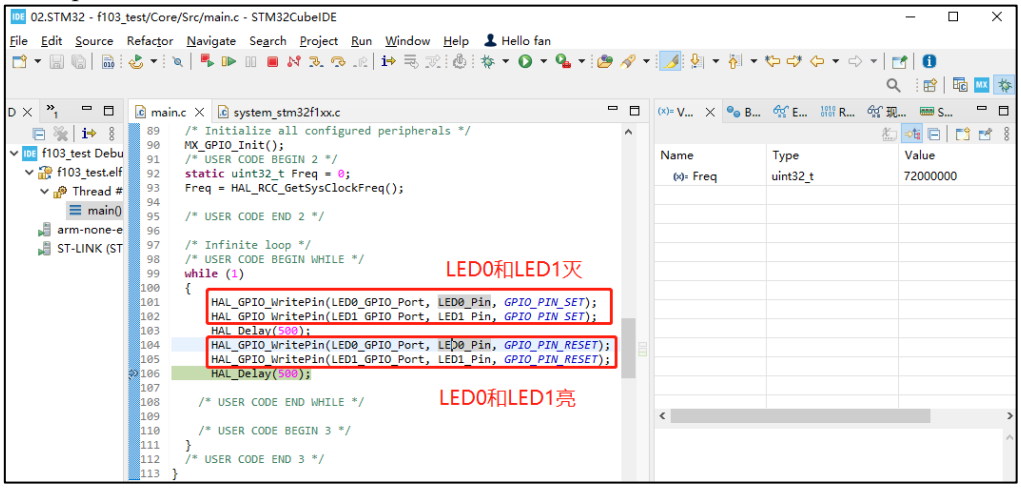


图 4.2.6.37 执行 Step Over

如果在代码第 101 行，点击 Step Into 符号，或者按下快捷键 F5，表示执行进去，也

就是进入 HAL_GPIO_WritePin 函数里边运行。

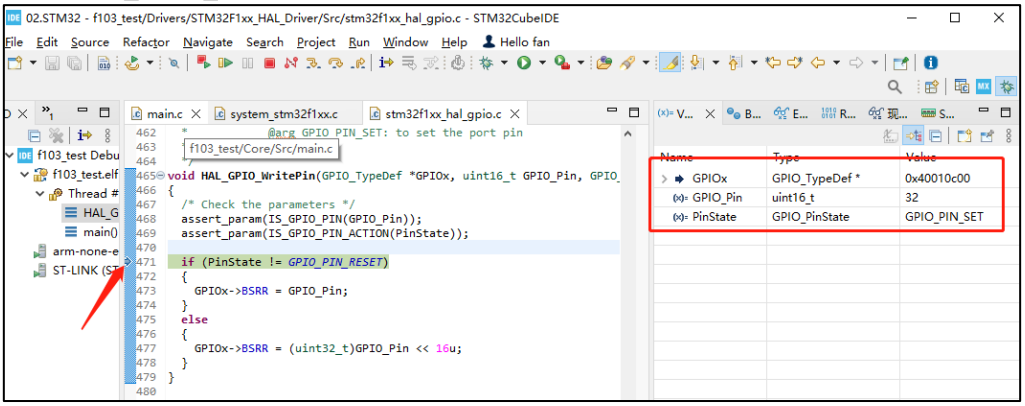


图 4.2.6.38 执行 Step Into

如果要跳出 HAL_GPIO_WritePin 函数，则按下快捷键 F7，或者点击 Step Return 返回符号。

7. 设置断点

调试过程中也可以通过设置断点来观察程序运行情况。在代码窗口行号的旁边，双击即可调价一个断点，再双击即可去掉一个断点。例如在 HAL_Delay (500)；函数行号旁边双击，即可添加一个断点：

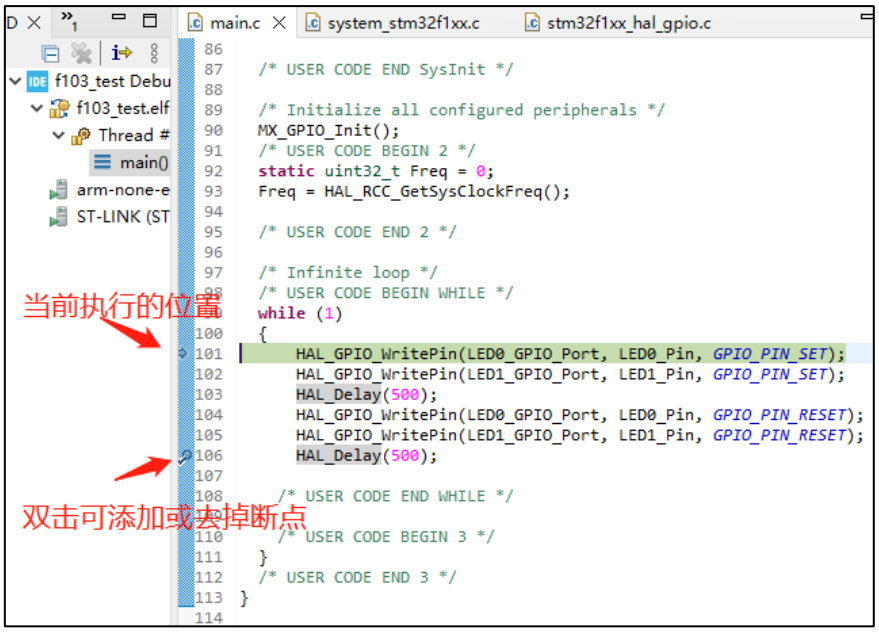


图 4.2.6.39 添加一个断点

点击运行，程序运行到断点处：

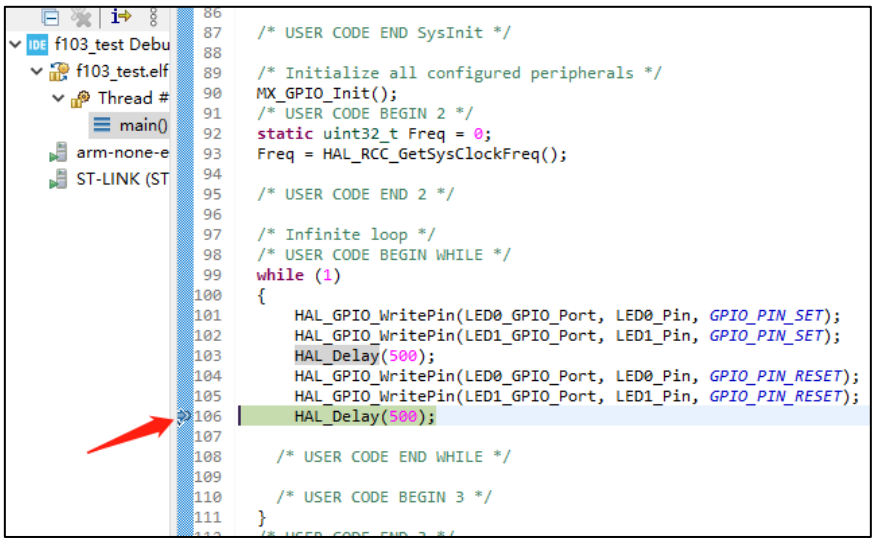


图 4.2.6.40 程序执行到断点处

当然，也可以在 STM32CubeIDE 中设置条件断点，即符合某个条件时才中断断点，不过要编写条件代码，这里就不列举了。

4.2.7 CubeIDE 支持打印浮点数设置

STM32CubeIDE 的默认设置不支持使用 sprintf/printf 函数来发送浮点数。如果需要支持浮点数，需要进行以下设置。首先选中需要的工程，右键，选择 Properties，选择 C/C++Build→Settings→MCU Settings 将④处的选框勾选上即可，如图：

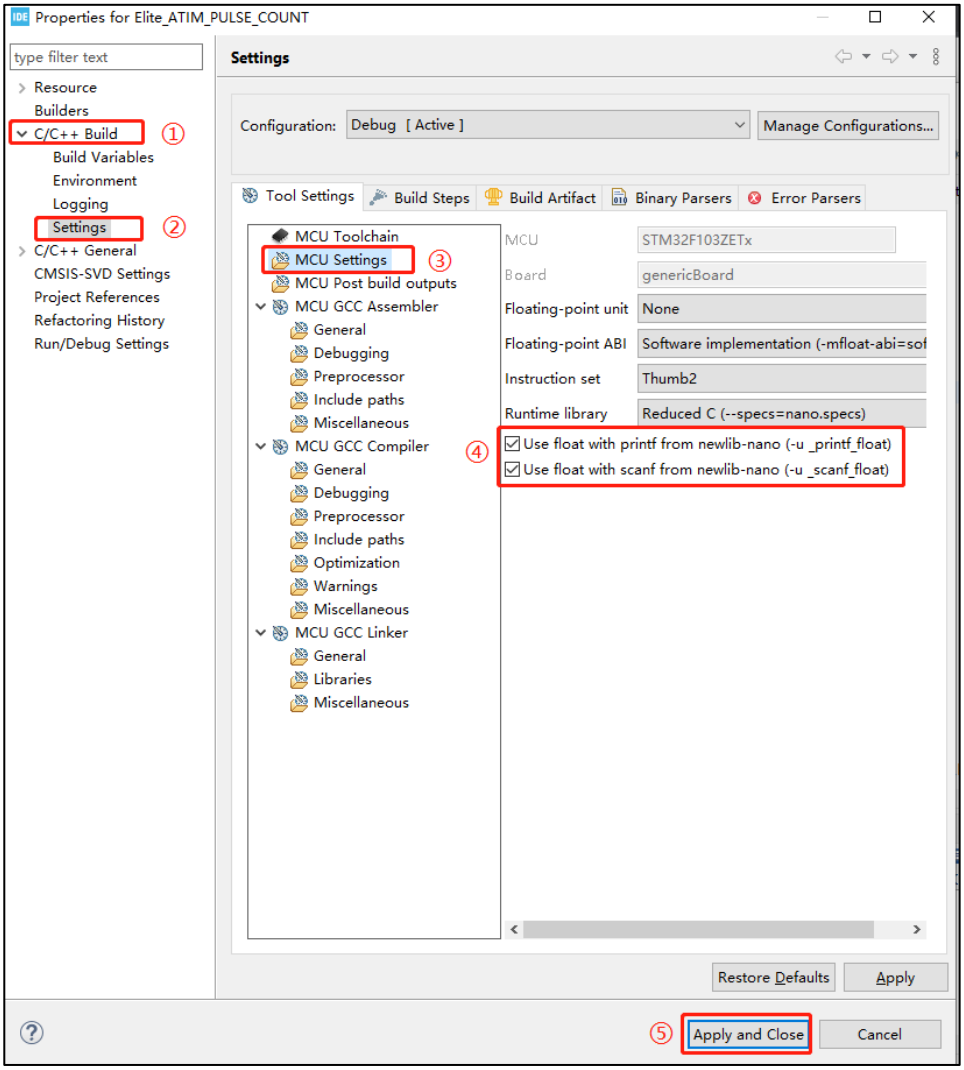


图 4.2.7.1 浮点数支持设置

4.3 无线调试器和 DAP 仿真器的使用

本节主要讲解如何使用正点原子的 DAP 仿真器以及无线调试器在 STM32（精英开发板为例）上进行仿真，正点原子的 DAP 仿真器和无线调试器分为高速版和普速版，关于以上仿真器的用户手册等相关资料，大家可以到正点原子资料下载中心进行下载。**注意，使用仿真器前一定要先认真阅读用户手册！**用户手册中讲解了仿真器的基本使用方法。

资料链接：http://www.openedv.com/docs/tool/dap/index.html

图 4.3.1 仿真器和调试器资料下载

4.3.1 DAP 仿真器的使用

1. 开发板连接

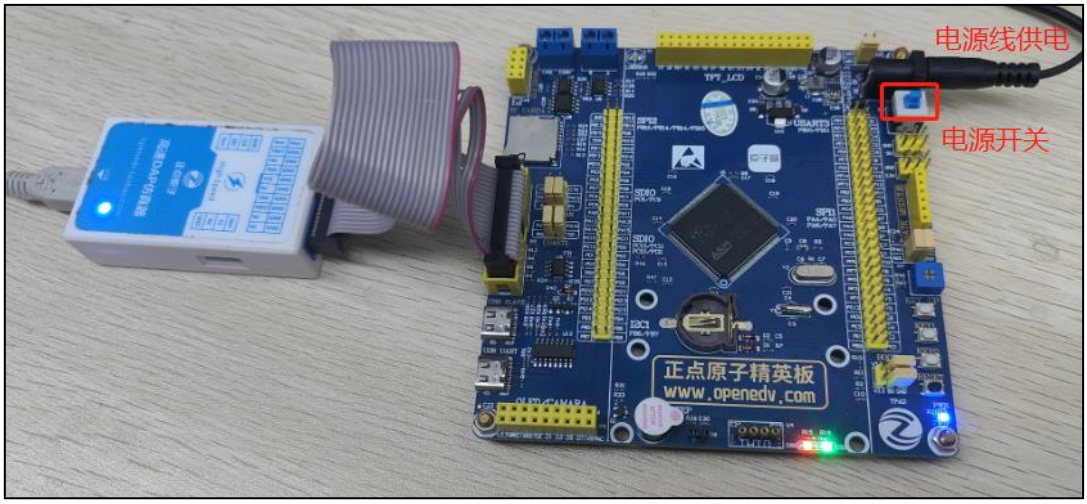


图 4.3.1.1 开发板连接

正常的情况是：开发板上电以后，开发板的电源指示灯（蓝灯）正常亮，DAP 仿真器的灯显示为蓝色，如果 DAP 仿真器的灯始终是红色或者不亮的话，说明仿真器没正常连接，请检查 USB 线是否已经接好，可以重新拔插下 USB 线。请注意，如果此前自己更新过 DAP 仿真器的固件的话，如果没有配置好，也可能会导致 DAP 仿真器的灯不亮，那么请参考正点原子的 DAP 用户使用手册进行操作。

2. STM32CubeIDE 配置

本节实验使用了上一小节我们新建的工程为例。成功编译工程后，按照如下步骤进入工程的下载配置界面：

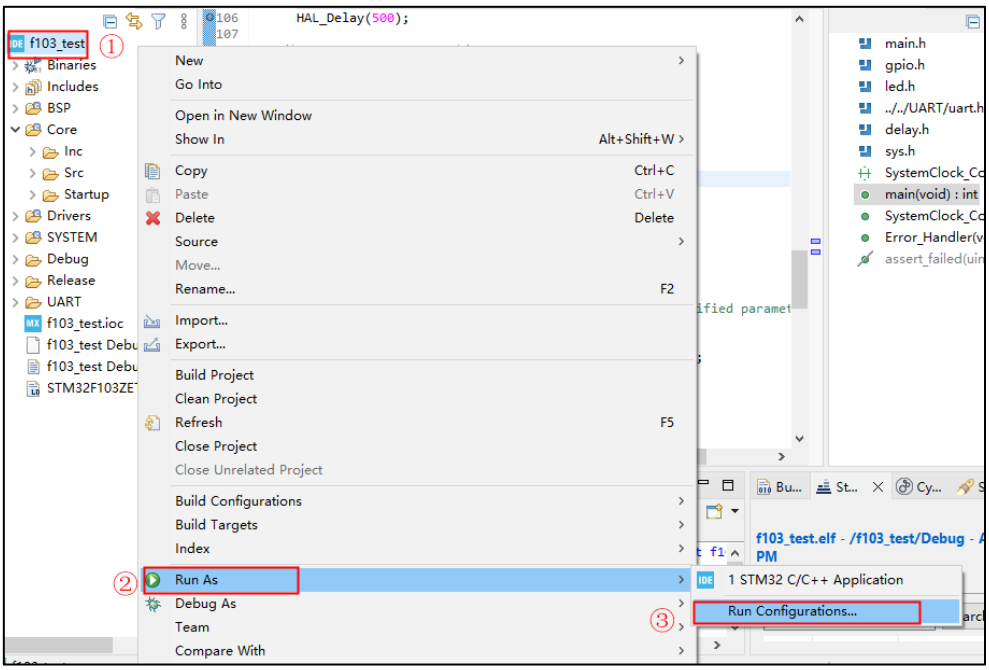


图 4.3.1.2 进入 Debug 配置界面

进入工程下载配置界面后，选择要用到的编译生成的文件，如下：

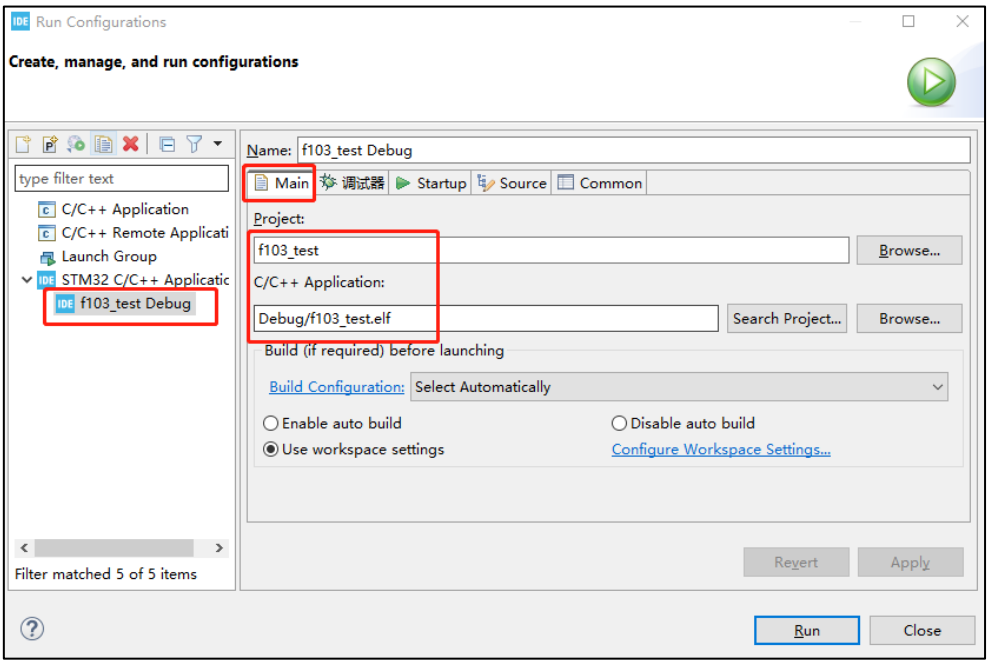


图 4.3.1.3 选择仿真的程序

进入调试器配置页面，按照如下配置：

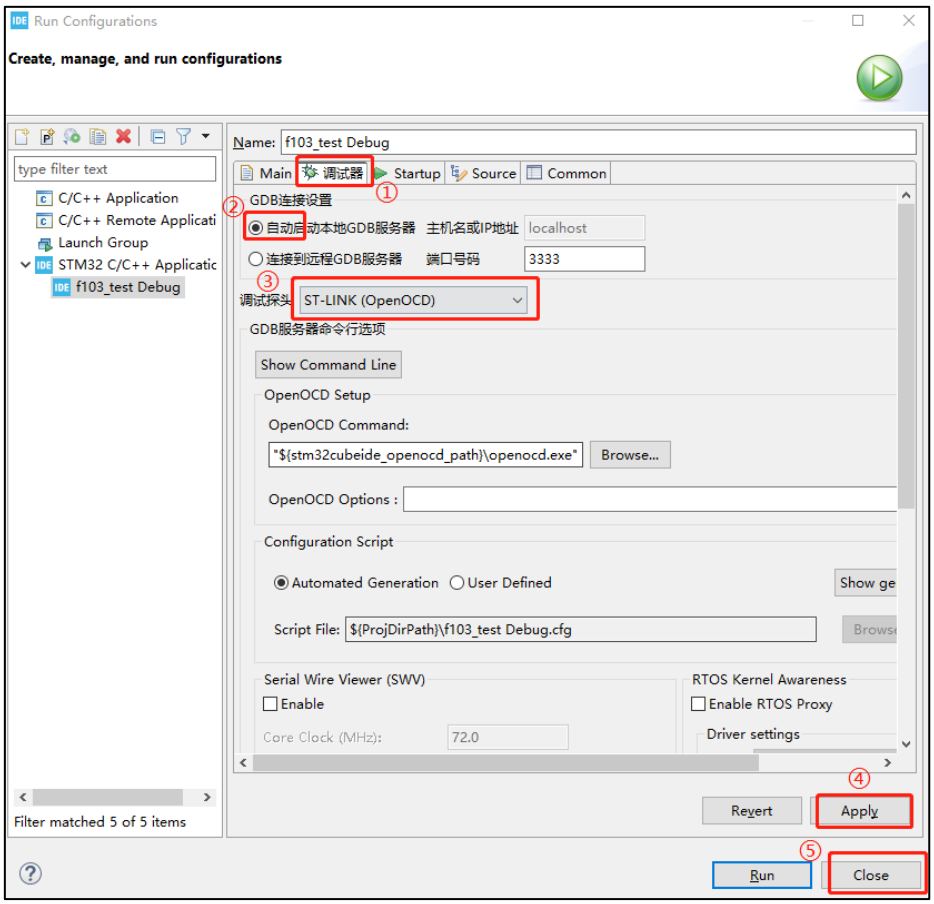


图 4.3.1.4 调试器配置图

在完成配置后，可以在 STM32CubeIDE 界面的左侧看到刚生成的.cfg 调试器配置文件。

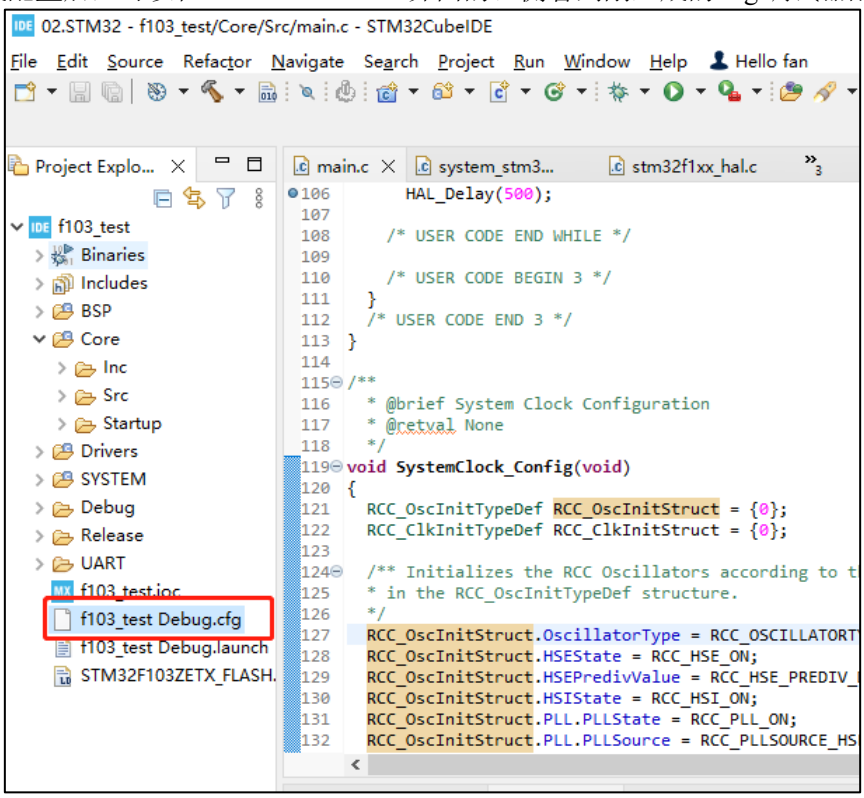


图 4.3.1.5 生成的 cfg 文件

我们可以在自己的 IDE 工程目录下找到它，并使用 Notepad++或 VSCode 等文本编辑器打开。**请务必在打开文件后将其另存为一个新的.cfg 文件，以免后续修改被 IDE 直接覆盖。**

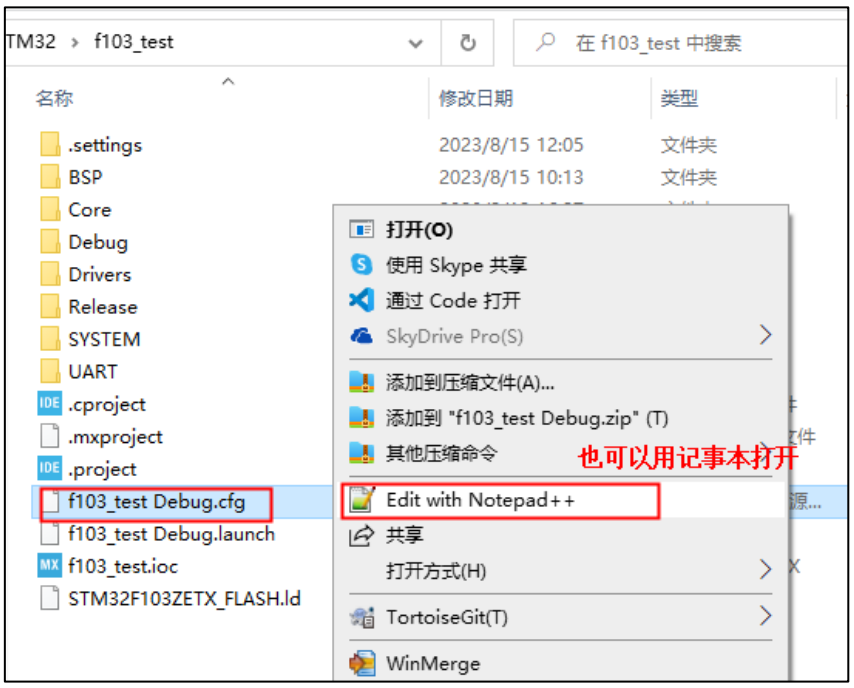


图 4.3.1.6 使用文本编辑器打开 cfg 文件

由于我们使用的是 DAP 仿真器 CubeIDE 默认是不支持的，需要进行一些改动才可，具体改动如下：

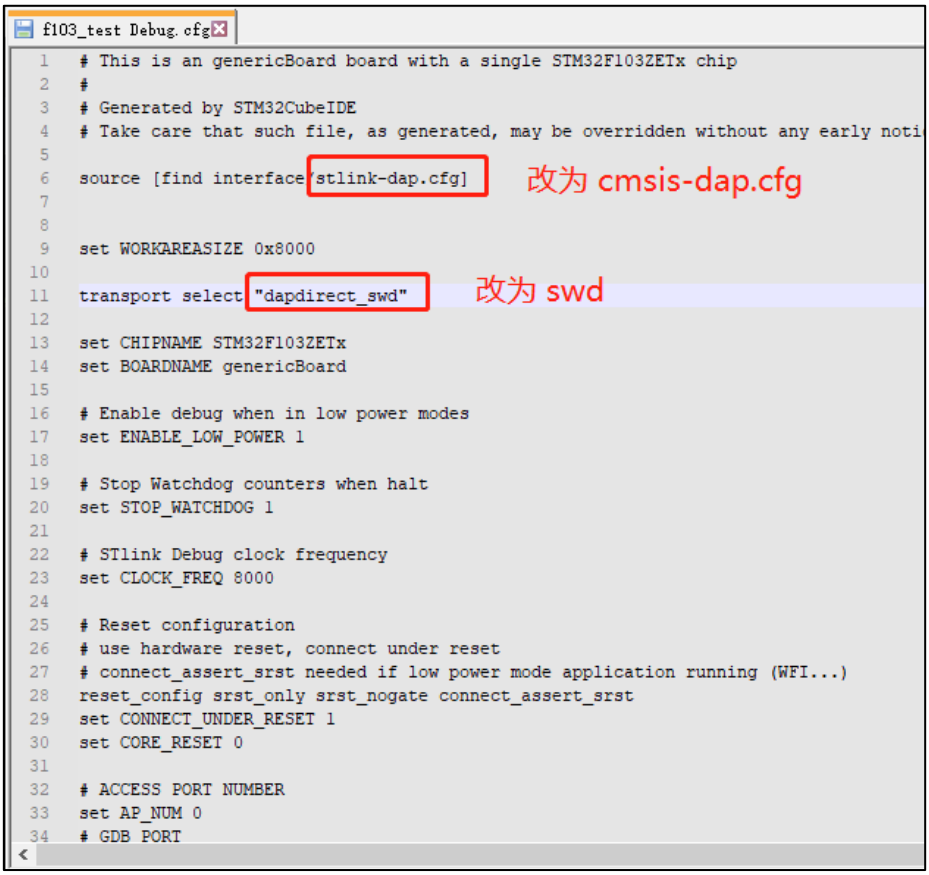


图 4.3.1.7 修改 cfg 文件

仅需按上图修改 2 处即可，修改内容：首先将 stlink-dap.cfg 改为 cmsis-dap.cfg；然后将 dapdirect_swd 修改为 swd 即可；接着注意一定要将该文件另存，如下：

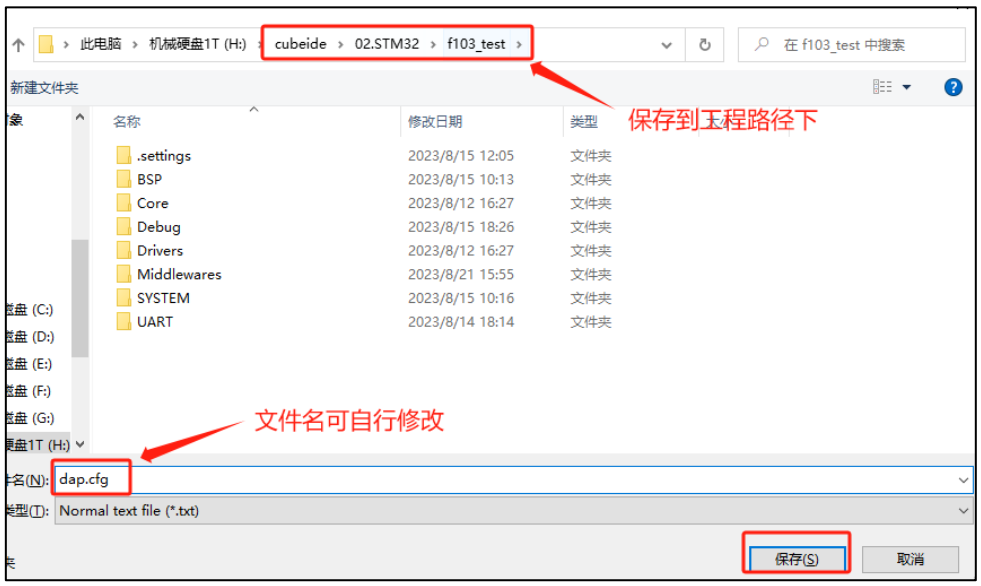


图 4.3.1.8 另存 cfg 文件

接着再次进入工程的下载配置界面，在这里把刚才修改的文件添加进来，整个设置工作就做完了，如下图：

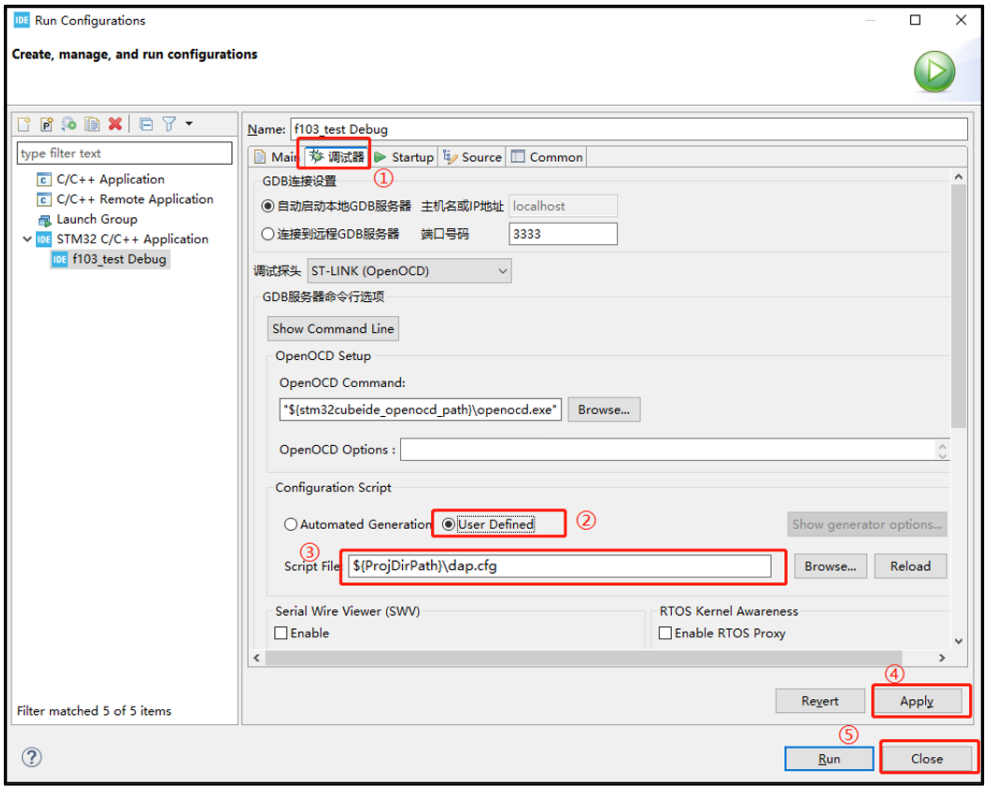


图 4.3.1.9 进入下载配置界面

此时我们回到 CubeIDE 主界面可以直接点击 下载程序。不过要注意使用 DAP 下载程序，**当下载成功后，程序不能直接运行，需要按下开发板的复位按键，才能正常工作**。尚未发现解决该问题的方法，如有相关建议或方法，欢迎联系我们！

关于 Debug 的操作和前面的 4.1.5 小节一样。大家可自行往上翻阅！

4.3.2 无线调试器的使用

无线调试器的使用步骤大部分和 DAP 仿真器的使用类似，只不过硬件连接方式不一样。

1. 开发板连接

如下图，无线调试器的发送端和接收端通过 USB 线和电脑进行通信，无线调试器的接收端通过 JTAG 接口和开发板连接，两个调试器的灯显示蓝色，说明连接正常。如调试器的灯是红色的或者灯不亮，说明连接不正常，请检查 USB 线是否接好，可以重新接好试试。请注意，如果此前自己更新过 DAP 仿真器的固件的话，如果没有配置好，也可能会导致 DAP 仿真器的灯**不亮**，那么请参考正点原子的 DAP 用户使用手册进行操作。

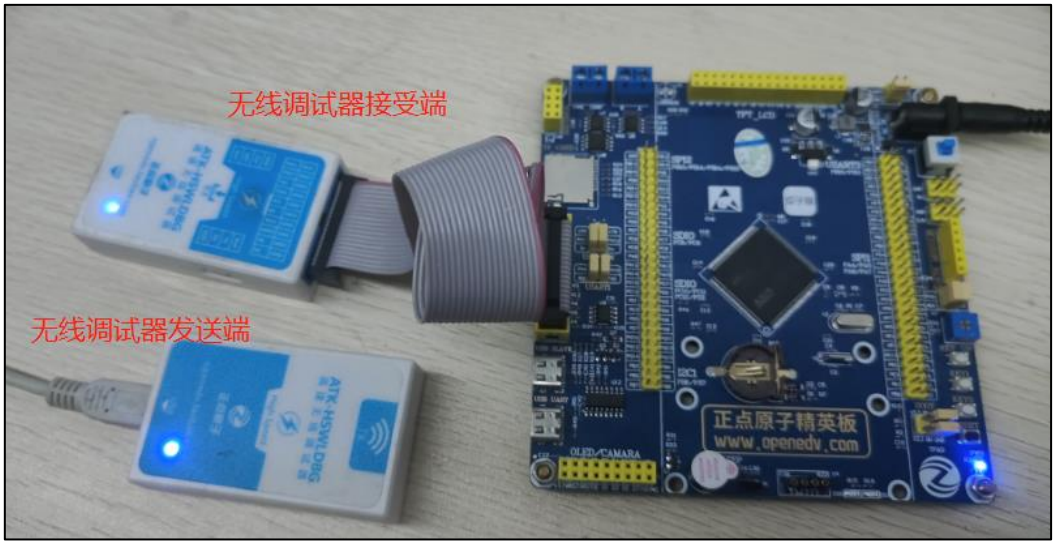


图 4.3.2.10 无线调试器链接

2. 仿真前配置

连接好硬件以后，剩下的操作和前面的 4.3.1 小节一样。

第五章 STM32Cube 固件包

第四章我们通过 STM32CubeIDE 在线下载了一个 STM32Cube 固件包，STM32CubeMX 插件就是利用这个固件包来生成初始化代码的，大家肯定好奇这个固件包里有什么？是做什么用的？和 HAL 库有什么关系？本章节我们就来分析这个固件包。

本章将分为如下几个小节：

5.1、获取 STM32Cube 固件包；

5.2、STM32Cube 固件包目录结构；

5.3、CMSIS 文件夹关键文件介绍；

5.1 获取 STM32Cube 固件包

STM32Cube 是 ST 公司提供的一套免费的开发工具和 STM32Cube 固件包，覆盖了整个 STM32 产品，可在 STM32 平台上进行快速轻松的开发，从而简化了开发人员的工作。STM32Cube 由以下组件组成，这些组件可以一起使用或独立使用：

- 允许用户通过图形化向导来生成 C 语言工程的图形配置工具 STM32CubeMX。
- 适用于每个 STM32 MCU 和 MPU 系列的 STM32Cube MCU 和 MPU 软件包（也叫 STM32Cube 固件包或者 STM32Cube 包）。

在前面 STM32CubeIDE 第一个工程实验的创建工程环节，STM32CubeIDE 已经自动在 C:\Users\自己的用户名称\STM32Cube\Repository\ STM32Cube_FW_F1_V1.8.5 路径中下载好了 STM32Cube_FW_F1_V1.8.5 包，这个包就是 STM32F1 的固件包，此固件包也可以从 ST 官网上下载，目前最新版本是 1.8.5 版本。

进入 ST 官网 https://www.st.com/content/st_com/en.html 以后，在搜索框中输入 STM32CubeF1 进行搜索。

图 5.1.1 搜索 STM32CubeF1 固件包

在开发板光盘 A 盘 \6，软件资料\1，软件\STM32CubeIDE 中我们也有提供 STM32F1 的固件包：

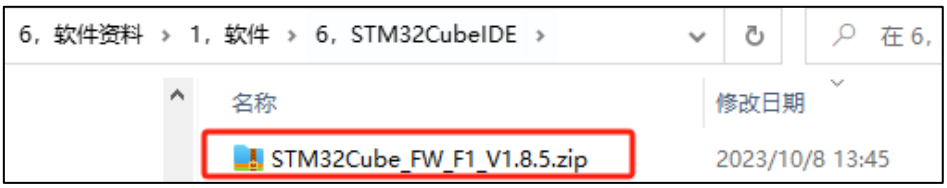


图 5.1.2 A 盘中下载好的固件包

我们打开 Drivers 文件夹，看到的 STM32F1xx_HAL_Driver 就是 HAL 库。

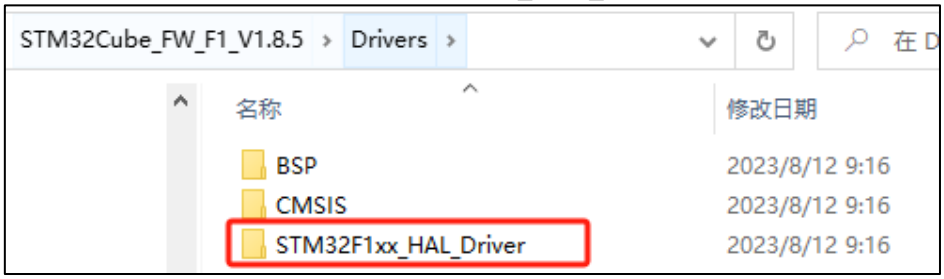


图 5.1.3 HAL 库

5.2 STM32CubeF1 固件包目录结构

接下来，我们看看前面下载好的 STM32CubeF1 固件包目录结构，打开 STM32Cube_FW_F1_V1.8.5 固件包，目录结构如下图。

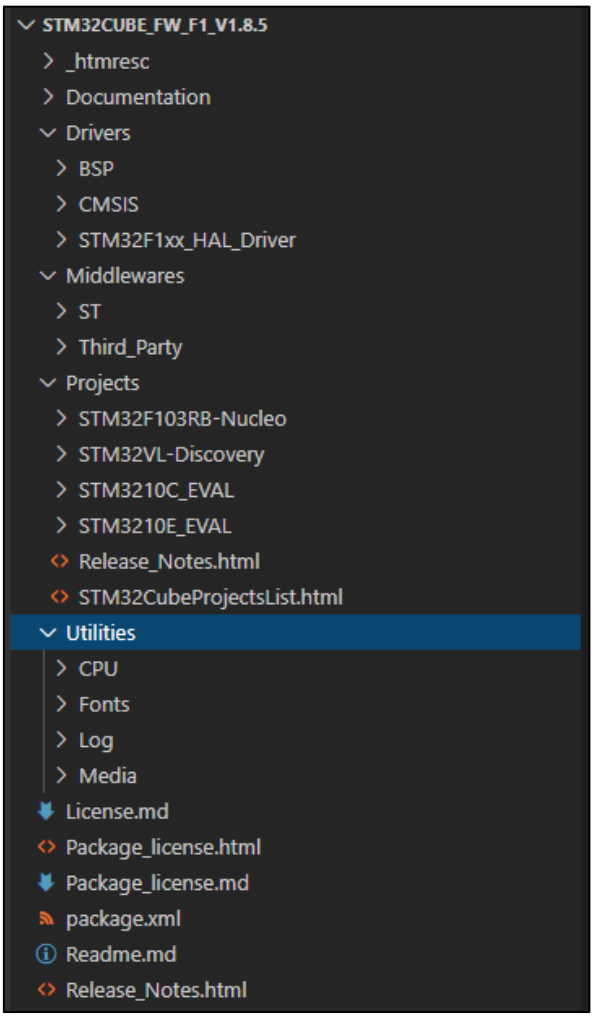


图 5.1.4 固件包目录结构

_htmresc 文件夹下是 ST 公司的 LOGO 图片和一些网站的资料，其实是用不到的，我们不去关注。对比较重要的文件夹，我们按照顺序进行介绍：

5.2.1 Drivers 文件夹

Drivers 文件夹包含 BSP，CMSIS 和 STM32F1xx_HAL_Driver 三个子文件夹。三个子文件夹具体说明请参考下表

BSP 文件夹	BSP 也叫板级支持包，此支持包提供的是直接与硬件打交道的 API，例如触摸屏，LCD，SRAM 以及 EEPROM 等板载硬件资源等驱动。目前在 STM32cubeF1 固件包中，ST 还未添加这部分内容（目前有 LED、COM 端口以及按钮相关的 API），后期 ST 应该会逐渐添加这些文件。 BSP 文件夹下还给了 ST 官方 DISCO 和 EVAL 开发板的硬件驱动 API 文件，每一种板对应一个文件夹。可以打开开发板文件夹，根据里边帮助文档查看 API 文件都有什么内容	
CMSIS 文件夹	CMSIS 文件夹用于存放符合 CMSIS 标准的文件，包括 STM32 启动文件、ARM Cortex 内核文件和对应外设头文件。关于 CMSIS 文件夹里的文件，我们后面会专门讲解。	
	Core	用于 Cortex-M 处理器内核和外围设备的 API
	Core_A	用于 Cortex-A5 / A7 / A9 处理器内核和外围设备的 API
	Device	微控制器专用头文件/启动代码/专用系统文件
	DSP	适用于各种数据类型的 DSP 库集合
	Include	STM32f1xx 外围设备访问层头文件
	Lib	ARM、GCC 和 IAR 格式的 DSP 库文件
	NN	神经网络库集合，目的是在 Cortex-M 处理器内核上最大化神经网络的性能并最小化其内存占用
	RTOS	实时操作系统通用 API 相关文件（V1 版本），兼容 RTX4
	RTOS2	对 RTOS V1 的拓展，兼容 RTX5
STM32F1xx_HAL_Driver 文件夹	HAL 库文件夹，处理 STM32“内部”设备,它包含了所有的 STM32F1xx 系列 HAL 库头文件和源文件，也就是所有底层硬件抽象层 API 声明和定义。它的作用是屏蔽了复杂的硬件寄存器操作，统一了外设的接口函数。该文件夹包含 Src 和 Inc 两个子文件夹，其中 Src 子文件夹存放的是.c 源文件，Inc 子文件夹存放的是与之对应的.h 头文件。每个.c 源文件对应一个.h 头文件。在前面的 STM32CubeIDE 第一个工程实验中就有用到该文件夹的文件，我们后面会重点介绍该文件夹的文件。	

表 5.2.1.1 Drivers 文件夹简介

5.2.2 Middlewares 文件夹

该文件夹下面有 ST 和 Third_Party 2 个子文件夹。ST 文件夹下面存放的是 STM32 相关的一些文件，包括 STemWin 和 USB 库等。Third_Party 文件夹是第三方中间件，这些中间价都是非常成熟的开源解决方案。具体说明请见下表 5.2.2.2：

Middlewares 文件夹	ST 子文件夹	STemWin 文件夹	STemWin 工具包。Segger 提供。
		STM32_USB_Device_Library 文件夹	USB 从机设备支持包。
		STM32_USB_Host_Library 文件夹	USB 主机设备支持包。
	Third_Party	FatFs 文件夹	FAT 文件系统支持包。采用

	子文件夹		的 FATFS 文件系统。
		FreeRTOS 文件夹	FreeRTOS 实时系统支持包。
		LwIP 文件夹	LwIP 网络通信协议支持包。

表 5.2.2.2 Middlewares 文件夹介绍

5.2.3 Projects 文件夹

该文件夹存放的是 ST 官方的开发板的适配例程，每个文件夹对应一个 ST 官方的 Demo 板，根据型号的不同提供 MDK 和 IAR 等类型的例程。里面有很多实例，读者可以根据自己的需要来作为参考。

5.2.4 Utilities 文件夹

该文件夹是一些公用组件，也是主要为 ST 官方的 DEMO 板提供的，在我们的例程中使用得不多。有兴趣的同学可以深入研究一下，这里我们不做过多介绍。

5.2.5 其它文件

文件夹中还有几个单独的文件，用于声明软件版本或者版权信息，我们使用 ST 的芯片已经默认得到这个软件的版权使用授权，可以简单了解一下各文件的内容，实际项目中我们一般不添加。

License.md：用于声明软件版权信息的文件。

package.xml：描述固件包版本信息的文件。

Release_Notes.html：超文本文件，用浏览器打开可知它是对固件包的补充描述和固件版本更新的记录说明。

5.3 CMSIS 文件夹关键文件介绍

随着 32 位处理器在嵌入式市场需求量逐渐增多，各家芯片公司推出新型芯片，伴随而来的是开发工具、软件兼容以及代码移植等问题。在这种情况下，各个硬件平台的供应商都寻求易于使用且高效的解决方案，其中，ARM 与 Atmel、IAR、KEIL、SEGGER 和 ST 等诸多芯片和软件工具厂商合作，发布了一套 CMSIS 标准。

CMSIS（Cortex Microcontroller Software Interface Standard），即 ARM Cortex 微控制器软件接口标准。CMSIS 标准提供了内核和外围设备、实时操作系统和中间组件之间的通用 API 接口，从而简化了软件的重复使用，缩短了微控制器开发人员的学习时间，并缩短了新设备的上市时间。下图是 ARM 公司的 CMSIS 标准结构框图：

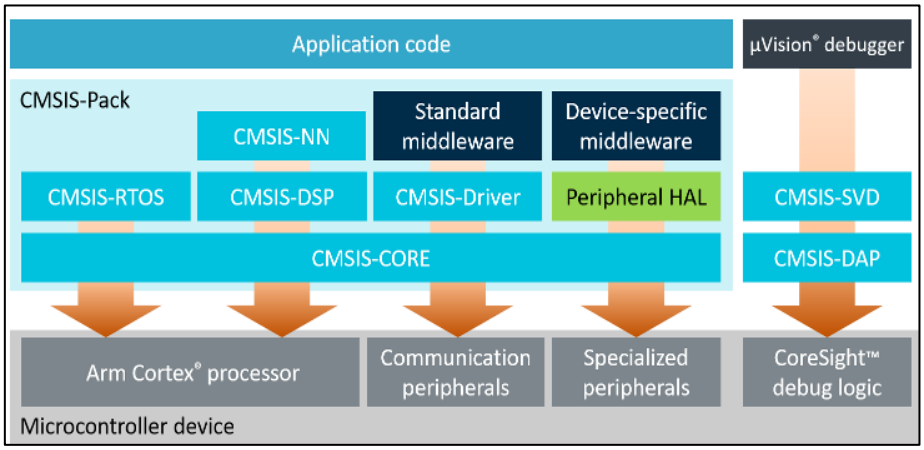


图 5.3.1 CMSIS 标准结构框图

其中，CMSIS-CORE 层定义了 Cortex-M 以及 Cortex-A 处理器（Cortex-A5/A7/A9）内核和外围设备的标准化 API。CMSIS-Pack 层包含了 CMSIS-Driver 驱动框架、CMSIS-DSP 相关库、CMSIS-RTOS 操作系统 API、中间件 API 和 Peripheral HAL 层 API 等。根据 CMSIS 的标

准，ARM 公司整合并提供了 CMSIS 软件包模板，目前最新的是 5.7.0 版本，感兴趣的小伙伴可以在 CMSIS 官网浏览更多信息： https://developer.arm.com/tools-and-software/embedded/cmsis

基于 ARM 提供的 CMSIS 软件包模板，ST 官方结合自己芯片的差异进行了修改，并将其整合到了 STM32Cube 固件包中的 CMSIS 文件夹里。CMSIS 版本 5.7.0 的规定软件包目录如表 5.3.1 所示：

文件/目录		描述
LICENSE.txt		Apache 2.0 授权的许可文件
Device		基于 Arm Cortex-M 处理器设备的 CMSIS 参考实现
ARM.CMSIS.pdsc		描述该 CMSIS 包的文件
CMSIS 组件	Documentation	这个数据包的描述文档
	Core	CMSIS-Core（Cortex-M）相关文件的用户代码模板,在 ARM.CMSIS.pdsc 中引用
	Core_A	CMSIS-Core（Cortex-A）相关文件的用户代码模板，在 ARM.CMSIS.pdsc 中引用
	DAP	CMSIS-DAP 调试访问端口源代码和参考实现
	Driver	CMSIS 驱动程序外设接口 API 的头文件
	DSP_Lib	CMSIS-DSP 软件库源代码
	NN	CMSIS-NN 软件库源代码
	Include	CMSIS-Core(Cortex-M)和 CMSIS-DSP 需要包括的头文件等。
	Lib	包括 CMSIS 核心（Cortex-M）和 CMSIS-DSP 的文件
	Pack	CMSIS-Pack 示例，包含包含设备支持、板支持和软件组件的软件包示例。
	RTOS	CMSIS-RTOS 版本 1 以及 RTX4 参考实现
	RTOS2	CMSIS-RTOS 版本 2 以及 RTX5 参考实现
	SVD	CMSIS-SVD 样例，规定开发者、制造商、工具制造商的分工和职能
	Utilities	PACK.xsd(CMSIS-Pack 架构文件),PackChk.exe(检查软件包的工具),CMSIS-SVD.xsd(MSIS-SVD 架构文件),SVDConv.exe(SVD 文件的转换工具)

表 5.3.1 CMSIS v5.7.0 的文件夹规范

知道了 CMSIS 规定的组件及其文件目录的大概内容后，我们再来看看 ST 提供的 CMSIS 文件夹，如上节提到的，它的位置是“STM32Cube_FW_F1_V1.8.0\Drivers\CMSIS”。打开文件夹内容如表 5.3.1 所示，可以发现它的目录结构完全按照 CMSIS 标准执行，仅仅是作了部分删减。

打开固件包中 STM32Cube_FW_F1_V1.8.5\Drivers\CMSIS 目录，其中，Device 文件夹和 Include 文件夹是每个工程都要用到的。

图 5.3.2 CMSIS 目录

CMSIS 文件夹中的 Device 和 Include 这两个文件夹中的文件是我们工程中最常用到的。下面对这两个文件夹作简单的介绍：

（1）Device 文件夹

Device 文件夹关键文件介绍如下表 5.3.2 所示：

文件	描述
stm32f1xx.h CMSIS └Device └ST └STM32F1xx └Include stm32f1xx.h	包含了很多条件定义和常用的枚举变量类型，与宏定义配合，选择性包含某一特定的 STM32F1 系列芯片的头文件。这个文件使我们在使用 STM32F1 系列的不同型号芯片时，不需要每次都修改工程头文件，只需要修改宏定义并增加特定型号芯片的头文件即可快速选择使用不同类型的 F1 芯片。
stm32f103xg.h CMSIS └Device └ST └STM32F1xx └Include stm32f103xe.h	STM32F1 大容量系列芯片通用的片上外设访问层头文件。Include 目录下有多个这样具体到型号的名字类似的头文件，我们具体使用哪个文件时需要根据实际使用的型号来确定。这个文件的主要作用是定义声明寄存器以及封装内存操作，以结构体和宏定义标识符的形式。
startup_stm32f103xe.s CMSIS └Device └ST └STM32F1xx └Source └Templates └arm startup_stm32f103xe.s	STM32F103 系列芯片的启动文件，每个系列都有与之对应的启动文件。启动文件的作用主要是进行堆栈的初始化，中断向量表以及中断函数定义等。启动文件有一个很重要的作用就是系统复位后引导进入 main 函数。后面会细讲。
system_stm32f1xx.h system_stm32f1xx.c CMSIS └Device └ST └STM32F1xx └Include system_stm32f1xx.h └Source └Templates system_stm32f1xx.c	主要是声明和定义系统初始化函数 SystemInit 以及系统时钟更新函数 SystemCoreClockUpdate。SystemInit 函数的作用是进行时钟系统的一些初始化操作以及中断向量表偏移地址设置，但它并没有设置具体的时钟值，这是与标准库的最大区别，在使用标准库的时候，SystemInit 函数会帮我们配置好系统时钟配置相关的各个寄存器。在启动文件 startup_stm32103xe.s 中会设置系统复位后，直接调用 SystemInit 函数进行系统初始化。SystemCoreClockUpdate 函数是在系统时钟配置进行修改后，调用这个函数来更新 SystemCoreClock 的值，变量 SystemCoreClock 是一个全局变量，开放这个变量可以方便我们在用户代码中直接使用这个变量来进行一些时钟运算。

表 5.3.2 Device 文件夹关键文件介绍

表 5.3.2 列出的文件都是正式工程中必须的文件。固件包的 CMSIS 文件包括了所有 STM32F1 芯片型号的文件，而我们只用到 STM32F103 系列，所以只针对我们用到的系列文

件来讲。

（2）Include 文件夹

Include 文件夹存放了符合 CMSIS 标准的 Cortex-M 内核头文件。 想要深入学习内核的朋友可以配合内核相关的手册去学习。对于 STM32F1 的工程，我们只要把我们需要的添加到工程即可，需要的头文件有：cmsis_armcc.h、cmsis_armclang.h、cmsis_compiler.h、cmsis_version.h、core_cm3.h 和 mpu_armv7.h。这几个头文件，对比起来，我们会比较多接触的是 core_cm3.h。

core_cm3.h 是内核底层的文件，由 ARM 公司提供，包含一些 AMR 内核指令，如软件复位，开关中断等功能。今后在需要的例程再去讲解其程序。

第六章 认识 HAL 库

在前面我们通过第一个工程实验熟悉了 STM32CubeIDE 的基本使用方法。本章节，我们来认识 HAL 库。HAL 库文件夹是 STM32Cube 固件包中重要的一部分，因为 HAL 库比较特殊，所以我们将其作为独立的章节来专门讲解。

在讲解之前我们需要说明一点，分析 HAL 库中的源码或者工程中的文件，不管它有多么复杂，无非就是一些.c 源文件和.h 头文件，还有一些类似.s 的启动文件，而头文件中遇见比较多的就是一些宏定义和函数声明，.c 文件比较常见的就是一些函数的定义。在分析过程中，我们去理解这些文件重要的宏定义和函数的作用，然后把这些函数和文件的关系弄清楚，在日后的项目开发中，我们才可以亲手操刀去打造属于自己的项目设计，在使用 HAL 库的时候才不至于逻辑混乱，犹如庖丁解牛，游刃有余。

本章将分为如下几个小节：

6.1、初识 HAL 库；
6.2、HAL 库框架结构；
6.3、如何使用 HAL 库；
6.4、HAL 库使用注意事项；

6.1 初识 HAL 库

6.1.1 获取 HAL 库

在上一章节的 STM32Cube 固件包中有一个 STM32F1xx_HAL_Driver 文件夹，该文件夹下存放的就是 HAL 库，我们所说的 HAL 库就是指里边的库文件。

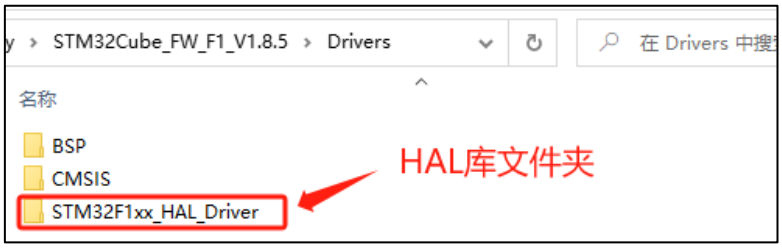


图 6.1.1.1 HAL 库在固件包中

在介绍 HAL 库文件前，我们先来理解下面几个概念。

6.1.2 什么是 HAL 库

在介绍 HAL 库之前，我们先来理解两个概念，什么是 API？什么是句柄（Handle）？

前面我们多次提到 API 这个词，API 全称 Application Programming Interface，翻译过来就是应用程序编程接口。接口，可以理解为是一些已经封装好了的可以被调用的功能函数或者方法，我们把这些函数放到我们的工程中，当我们要实现某个功能时，就可以在工程中找到对应的函数，然后进行调用。

句柄（Handle），在 STM32 的手册上经常看到这个词，可以理解为它是一个指针，或者是一些表的索引，或者是用于描述和标记某些资源的的标识，这些资源可以是函数、可以是一段内存、可以是一组数字、可以是一个外设等等，总之很广泛，通过句柄我们可以访问到打开的资源。我们在调用 API 函数的时候，可以利用句柄来说明要操作哪些资源。

HAL 的含义，ST 官方手册里的解释是 Hardware abstraction layer，即硬件抽象层，单词的首字母组合起来就是 HAL。HAL 库是一些封装好的驱动程序，向下可以操作硬件，向上可以给用户提供可操作的接口。

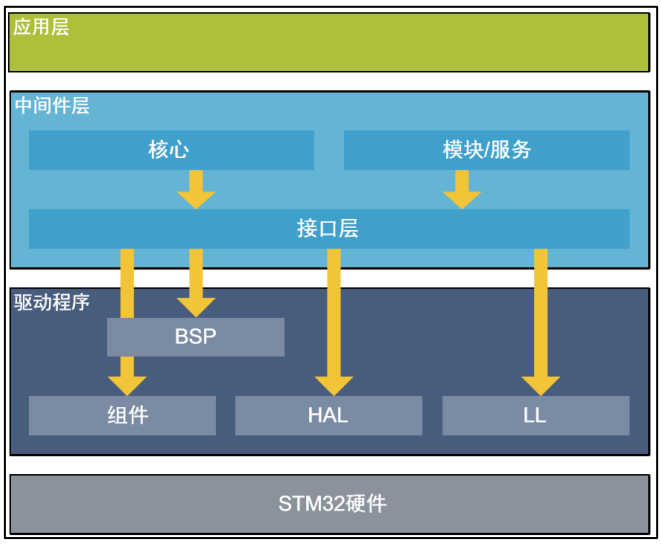


图 6.1.2.1 HAL 库属于驱动程序

HAL 库，笔者的理解是，ST 把对不同系列 MCU 的操作经过一层一层的封装，将硬件进行抽象化表达出来，最后呈现给我们的就是 HAL 库。硬件抽象化，也就是将对寄存器的操作做了一系列封装，将外设抽象组织为句柄，使我们看不到寄存器的影子，最后分离出可以调用的 API，使用者可以不去关注底层、不必关注复杂的硬件寄存器就可以进行编程，通过调用 API 和句柄操作可以实现对 MCU 的大部分外设进行操作，使用起来非常方便，而且很容易进行移植。

6.1.3 HAL 库能做什么

STM32 开发方式中，可以直接配置寄存器或者可以使用官方的库来实现。

我们在 51 单片机开发的时候就直接配置寄存器，但是到了 32 位单片机开发，如果开发大型项目，需要的功能外设很多，再使用这种方式就已经力不从心了。因为 STM32 的外设资源丰富，寄存器数量是 51 单片机寄存器的数十倍，那么多的寄存器根本无法记忆，而且开发中需要不停查找芯片手册，开发过程就显得机械和费力，完成的程序代码可读性差，可移植性不高，程序的维护成本变高了。当然了，采用直接配置寄存器方式开发会显得更直观，程序运行占用资源少。如果项目中只用少数几个外设，或者同一系列的芯片，项目组内部有一套成熟、可移植性高的祖传代码，开发起来就没那么困难了。

为了简化开发人员的工作，减少开发工作时间和成本，针对 STM32 系列芯片，ST 官方推出了标准外设库(STD 库)、HAL 库和 LL 库 。在这些库中，有很多用结构体封装好的寄存器参数，有常用的表示参数的宏，还有封装好的对寄存器操作的 API，开发者可以调用这些资源实现配置相应的寄存器，效率大大提高了。使用库的框架来开发，程序控制语句结构化，程序单元模块化，贴近人的思维，易于阅读，易于移植和维护。

可以这么说，库是架设于寄存器与用户驱动层之间的代码，向下是直达硬件相关的寄存器，向上为用户提供配置寄存器的接口，用户代码通过这些接口来间接操作寄存器。

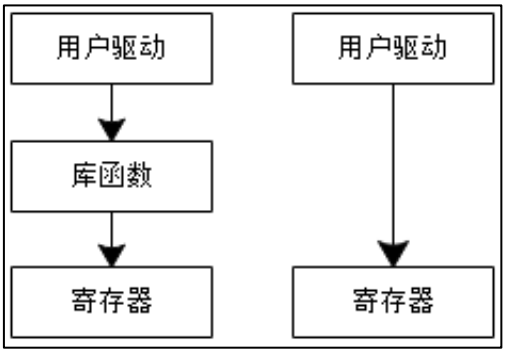


图 6.1.3.1 库和直接操作寄存器对比

6.1.4 HAL 库和其它库有什么不同

前面我们提到，针对 STM32 系列芯片，ST 官方推出了标准外设库(STD 库)、HAL 库和 LL 库，那么 HAL 库和其它两种库有什么差别？下图是三种库对 STM32 系列产品的支持。

Offer	Available for STM32									
	STM32 F0	STM32 F1	STM32 F3	STM32 F2	STM32 F4	STM32 F7	STM32 H7	STM32 L0	STM32 L1	STM32 L4
STM32Snippets	Now	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	Now	N.A.	N.A.
Standard Peripheral Library	Now	Now	Now	Now	Now	N.A.	N.A.	N.A.	Now	N.A.
STM32Cube HAL	Now	Now	Now	Now	Now	Now	Now	Now	Now	Now
STM32Cube LL	Now	Now	Now	Now	Now	Now	2018	Now	Now	Now

图 6.1.4.1 HAL 库对 ST 系列产品的支持情况

1. 标准外设库

STD（Standard Peripheral Libraries）标准外设库，它把实现功能中需要配置的寄存器以结构体的形式封装起来，使用者只需要配置结构体变量成员就可以修改外设的配置寄存器，比直接操作寄存器方便了不少。但标准外设库仍然接近于寄存器操作，它的方便也是针对某一系列芯片而言的，在不同系列芯片上使用标准外设库开发的程序可移植性比较差，例如，在 F4 上开发的程序移植到 F3 上，使用标准库是不通用的。目前 STM32 系列产品中仅 F0-F4 以及 L1 系列支持标准外设库。

2. HAL 库

为了解决标准库存在的问题，ST 在标准库的基础上又推出了 HAL 库。个人认为，HAL 库是用来取代之前的标准库的，因为这几年 ST 官方大力推广 HAL 库，而且在 ST 新出的 STM32 芯片中， ST 直接只提供 HAL 库。

HAL 库在设计的时候更注重软硬件分离，HAL 库的目的应该就是尽量抽离物理层，HAL 库的 API 集中关注各个外设的公共函数功能，以便定义通用性更好的 API 函数接口，从而具有更好的可移植性。HAL 库写的代码在不同的 STM32 产品上移植，非常方便，效率得到提升。目前 HAL 库支持 STM32 各个系列产品。

3. LL 库

LL 库（Low Layer）是 ST 继 HAL 库之后新增的库，与 HAL 库捆绑发布，在前面的 STM32CubeIDE 第一个工程实验中，我们看到在 STM32CubeMX 插件里有选择使用 HAL 库还是 LL 库的选项。

LL 库的英文名字翻译过来就是底层的意思，实际上 LL 库更接近硬件层，它和 STD 库类似，都是直接操作的寄存器，只不过 LL 库可以在 STM32Cube 中实现。LL 库提供一组轻量级、优化、面向专家的 API，具有最佳的性能和运行时效率。LL 库可以完全独立使用，也可以结合 HAL 库一起使用。当 HAL 库需要优化运行时，可以调用 LL 库来处理，而对于复杂的外设（例如：USB 驱动），两者混合使用，才能正常驱动这个复杂的外设。

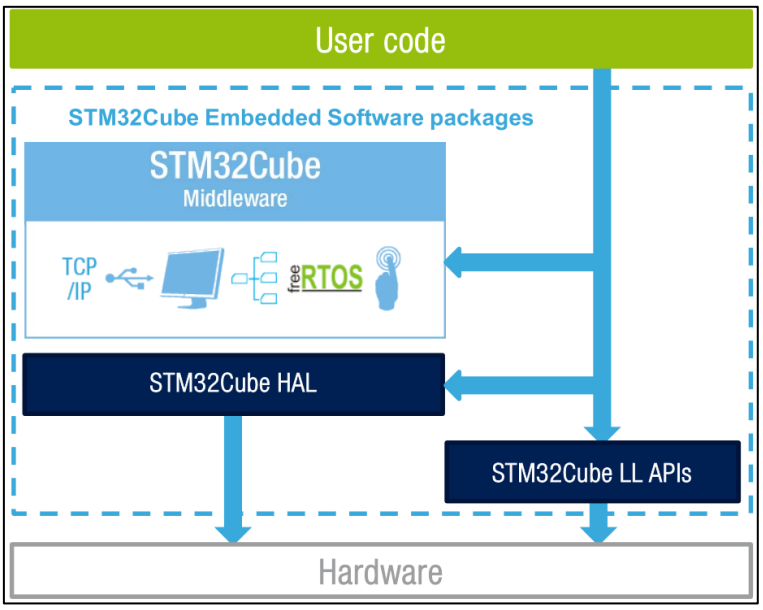


图 6.1.4.2 HAL 库和 LL 库

6.1.5 怎样学习 HAL 库

（1）不管 HAL 库封装的有多好，本质上还是通过配置 MCU/MPU 的寄存器来实现我们想要的功能。所以我们学习 HAL 库的同时，还需要学习外设的工作原理和寄存器的配置方法，通过原理来理解 HAL 库是怎样实现我们想要的功能，要知其然更要知其所以然。

（2）HAL 库不仅仅是底层驱动，它更是一套行业内可以公开和认可的架构。学习 HAL 库，我们要了解它的架构，了解库中每个文件夹下大概有哪些文件，文件之间的关系是什么，函数之间的调用关系是什么，调用条件是什么，常见的数据结构怎么用。

（3）学习 HAL 库，遇到疑问的地方可以查 HAL 的帮助文档，可以上网上查询相关说明，可以在 Wiki 上对共同的主题进行扩展或者探讨，多看相关的例程，跟着例程操作，多总结。

（4）要学会使用 ST 提供的优秀的开发工具，掌握 STM32Cube 工具套件的使用方法，熟话说得好，磨刀不误砍柴工。

不管怎样，我们的目的就是为了使用 HAL 库来开发，学会调用 HAL 库的 API 函数，配置对应外设按照我们的要求来工作，实现想要的功能。

下面，我们进入主题，分析 HAL 库。

6.2 HAL 库框架结构

这一节我们将简要分析一下 HAL 驱动文件夹下的驱动文件，帮助大家快速认识 HAL 库驱动的构成，旨在帮大家快速认识 HAL 库的函数的一些常用形式，帮助大家到遇到 HAL 库时能根据名字大致推断该函数的用法，本部分不要求大家在学习完本节后完全记住。

6.2.1 HAL 库文件夹结构

HAL 库头文件和源文件在 STM32Cube 固件包的 STM32F1xx_HAL_Driver 文件夹中，打开该文件夹，如图 6.2.1.1 所示。

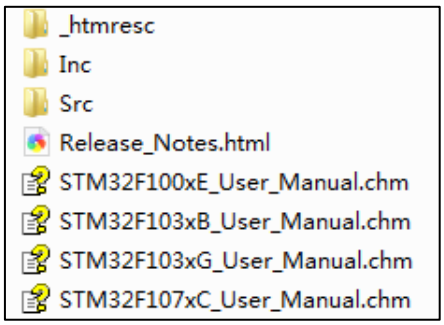


图 6.2.1.1 STM32F1xx_HAL_Driver 文件夹目录结构

STM32F1xx_HAL_Driver 文件夹下的 Src（Source 的简写）文件夹存放是所有外设的驱动程序源码，Inc（Include 的简写）文件夹存放的是对应源码的头文件。Release_Notes.html 是 HAL 库的版本更新信息。最后三个是库的用户手册，这个需要可以去熟悉一下，查阅起来很方便。

打开 Src 和 Inc 文件夹，大家会发现基本都是 stm32f1xx_hal_和 stm32f1xx_ll_开头的.c 和.h 文件。刚学 HAL 库的朋友可能会说，stm32f1xx_hal_开头的是 HAL 库，我能理解。那 stm32f1xx_ll_开头的文件又是什么？这就告诉大家，stm32f1xx_ll_开头的文件是 LL 库。

6.2.2 HAL 库文件介绍

HAL 库关键文件介绍如下表 6.2.2.1 所示，表中 ppp 代表任意外设。

文件	描述
sm32f1xx_hal.c stm32f1xx_hal.h	初始化 HAL 库，（比如 HAL_Init，HAL_DeInit，HAL_Delay 等），主要实现 HLA 库的初始化、系统滴答，HAL 库延时函数、IO 重映射和 DBGMCU 功能。HAL 的头文件应被用户代码所包含。
stm32f1xx_hal_conf.h HAL 库中本身没有这个文件，可以自行定义，也可以直接使用“Inc”文件夹下 stm32f1xx_hal_conf_template.h 的内容作为参考模版	HAL 用户配置文件，stm32f1xx_hal.h 引用了这个文件，用来对 HAL 库进行裁剪。由于 Hal 库的很多配置都是通过预编译的条件宏来决定是否使用这一 HAL 库的功能，这也是当前的主流库如 LWIP/FreeRTOS 等的做法，无需修改库函数的源码，通过使能/不使能一些宏来实现库函数的裁剪。
stm32f1xx_hal_def.h	通用 HAL 库资源定义，包含 HAL 的通用数据类型定义，声明、枚举，结构体和宏定义。如 HAL 函数操作结果返回值类型 HAL_StatusTypeDef 就是在这个文件中定义的。
stm32f1xx_hal_cortex.h stm32f1xx_hal_cortex.c	它是一些 Cortex 内核通用函数声明和定义，例如中断优先级 NVIC 配置，MPU，系统软复位以及 Systick 配置等，与前面 core_cm3.h 的功能类似。
stm32f1xx_hal_ppp.c stm32f1xx_hal_ppp.h	外设驱动函数。对于所有 STM32 的驱动名称都相同，ppp 代表一类外设，包含该外设的操作 API 函数。当 ppp 为 adc 时，这个函数就是 stm32f1xx_hal_adc.c/h，可以分别在 Src/Inc 目录下找到。
stm32f1xx_hal_ppp_ex.c stm32f1xx_hal_ppp_ex.h	外设特殊功能的 API 文件，作为标准外设驱动的功能补充和扩展。针对部分型号才有的特殊外设作功能扩展，或外设的实现功能与标准方式完全不同的情况下作重新初始化的备用接口。ppp 的含

	义同标准外设驱动
stm32f1xx_II_ppp.c stm32f1xx_II_ppp..h	LL 库文件，在一些复杂外设中实现底层功能，在部分 stm32f1xx_hal_ppp.c 中被调用

表 6.2.2.1 HAL 库关键文件介绍

以上是 HAL 库最常见的文件的列表，在 Src/Inc 下面还有 Legacy 文件夹，用于特殊外设的补充说明。我们的教程中用到的比较少，这里不展开描述。

不止文件命名有一定规则，stm32f1xx_hal_ppp (c/h)中的函数和变量命名也严格按照命名规则，如表 6.2.2.2 所示的命名规则在大部分情况下都是正确的：

文件名	stm32f1xx_hal_ppp (c/h)	stm32f1xx_hal_ppp_ex (c/h)
函数名	HAL_PPP_Function HAL_PPP_FeatureFuntion_MODE	HAL_PPPEx_Function HAL_PPPEx_FeatureFunction_MODE
外设句柄	PPP_HandleTypedef	无
初始化参数结构体	PPP_InitTypeDef	PPP_InitTypeDef
枚举类型	HAL_PPP_StructnameTypeDef	无

表 6.2.2.2 HAL 库函数、变量命名规则

对于 HAL 的 API 函数，常见的有以下几种：

- 初始化/反初始化函数:HAL_PPP_Init(), HAL_PPP_DeInit()
- 外设读写函数:HAL_PPP_Read(),HAL_PPP_Write(),HAL_PPP_Transmit(), HAL_PPP_Receive()
- 控制函数:HAL_PPP_Set (),HAL_PPP_Get ().
- 状态和错误:HAL_PPP_GetState (), HAL_PPP_GetError ().

HAL 库封装的很多函数都是通过定义好的结构体将参数一次性传给所需函数，参数也有一定的规律，主要有以下三种：

- 配置和初始化用的结构体:

一般为 PPP_InitTypeDef 或 PPP_ ConfTypeDef 的结构体类型，根据外设的寄存器设计成易于理解和记忆的结构体成员。

- 特殊处理的结构体

专为不同外设而设置的，带有“Process”的字样，实现一些特异化的中间处理操作等。

- 外设句柄结构体

HAL 驱动的重要参数，可以同时定义多个句柄结构以支持多外设多模式。HAL 驱动的操作结果也可以通过这个句柄获得。有些 HAL 驱动的头文件中还定义了一些跟这个句柄相关的一些外设操作。如用外设结构体句柄与 HAL 定义的一些宏操作配合，即可实现一些常用的寄存器位操作。比较常见的 HAL 库寄存器操作如表 6.2.2.3 所示:

宏定义结构	用途
__HAL_PPP_ENABLE_IT(__HANDLE__, __INTERRUPT__)	使能外设中断
__HAL_PPP_DISABLE_IT(__HANDLE__,__INTERRUPT__)	禁用外设中断
__HAL_PPP_GET_IT (__HANDLE__, __ INTERRUPT __)	获取外设某一中断源
__HAL_PPP_CLEAR_IT (__HANDLE__, __ INTERRUPT __)	清除外设中断
__HAL_PPP_GET_FLAG (__HANDLE__, __FLAG__)	获取外设的状态标记
__HAL_PPP_CLEAR_FLAG (__HANDLE__, __FLAG__)	清除外设的状态标记
__HAL_PPP_ENABLE(__HANDLE__)	使能某一外设
__HAL_PPP_DISABLE(__HANDLE__)	禁用某一外设
__HAL_PPP_XXXX (__HANDLE__, __PARAM__)	针对外设的特殊操作
__HAL_PPP_GET_IT_SOURCE (__HANDLE__,	检查外设的中断源

__INTERRUPT __)	

表 6.2.2.3 HAL 库驱动部分与外设句柄相关的宏

但是对于 SYSTICK/NVIC/RCC/FLASH/GPIO 这些内核外设或共享资源来说，不使用 PPP_HandleTypedef 这类外设句柄进行控制，如：HAL_GPIO_Init() 只需要初始化的 GPIO 编号和具体的初始化参数。

```
HAL_StatusTypeDef HAL_GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef *Init)
{
     /*GPIO 初始化程序·····*/
}
```

最后要分享的是 HAL 库的回调函数，这部分允许用户重定义，并在其中实现用户自定义的功能，也是我们使用 HAL 库的最常用的接口之一：

回调函数	举例
HAL_PPP_MspInit() / _DeInit()	举例: HAL_USART_MspInit() 由 HAL_PPP_Init()这个 API 调用，主要在这个函数中实现外设对应的 GPIO、时钟、DMA、中断开启的配置和操作。
HAL_PPP_ProcessCpltCallback	举例: HAL_USART_TxCpltCallback 由外设中断或 DMA 中断调用，调用时 API 内部已经实现中断标记的清除的操作，用户只需要专注于自己的软件功能实现即可。
HAL_PPP_ErrorCallback	举例: HAL_USART_ErrorCallback 外设或 DMA 中断中发生的错误，用于作错误处理。

表 6.2.2.4 HAL 库驱动中常用的回调函数接口

至此，我们大概对 HAL 库驱动文件的一些通用格式和命名规则有了初步印象，记住这些规则可以帮助我们快速对 HAL 库的驱动进行归类和判定这些驱动函数的用法。

ST 官方给我们提供了快速查找 API 函数的帮助文档。我们如果解压了 stm32cubef1 的固件包后，在路径“STM32Cube_FW_F1_V1.8.3\Drivers\STM32F1xx_HAL_Driver”下可以找到几个 chm 格式的文档，根据我们开发板主控芯片 STMF103ZE 我们没有找到直接可用的，但可以查看型号接近的：STM32F103xG_User_Manual.chm(因为 G 系列比 E 系列引脚功能更多，只是查看 API 函数不响应使用的)。双击打开后，可以看到左边目录下有四个主题，我们来查看 Modules。以外设 GPIO 为例，讲一下怎么使用这个文档。点击 GPIO 外设的主题下的 IO operation functions / functions 看看里面的 API 函数接口描述，如图 6.2.2.1 所示。

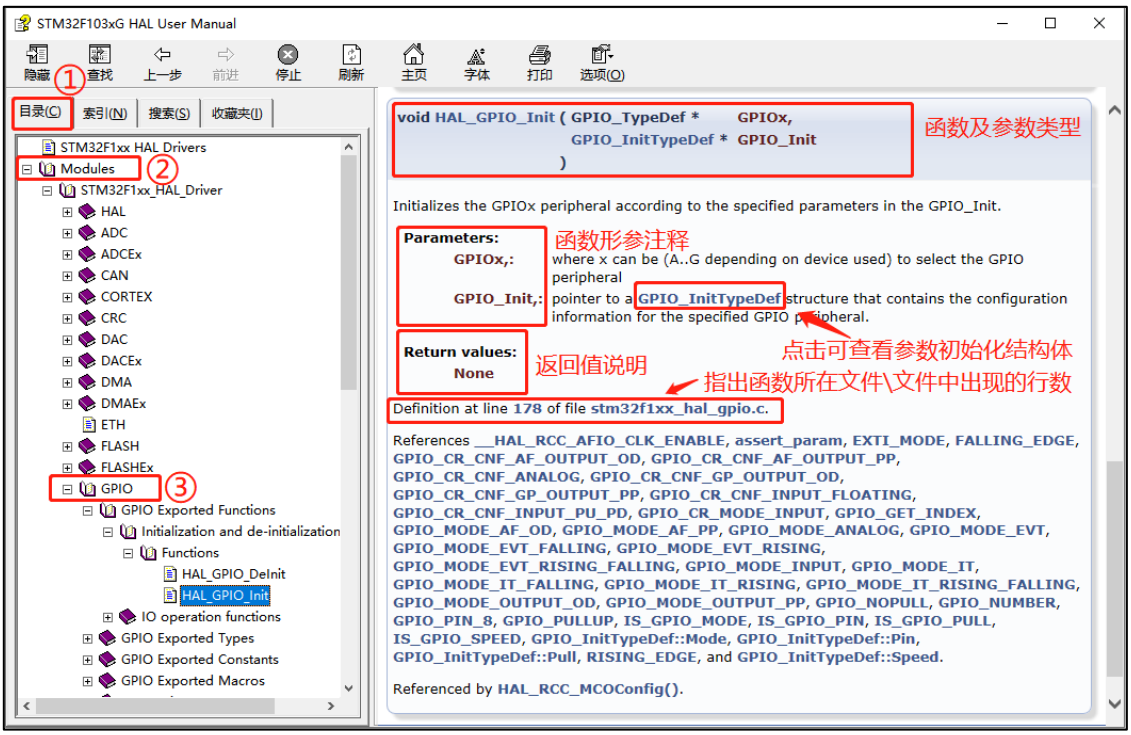


图 6.2.2.1 帮助文档的 API 函数描述

这个文档提供的信息很全，不看源码都可以直接使用它来编写代码，还给我们指示源码位置，非常方便。大家多翻一下其他主题了解一下文档的信息结构，很容易使用。

下面举个例子，比如我们要让 PB4 输出高电平。先看函数功能，HAL_GPIO_WritePin 函数就是我们的 GPIO 口输出设置函数，如图 6.2.2.2 所示：

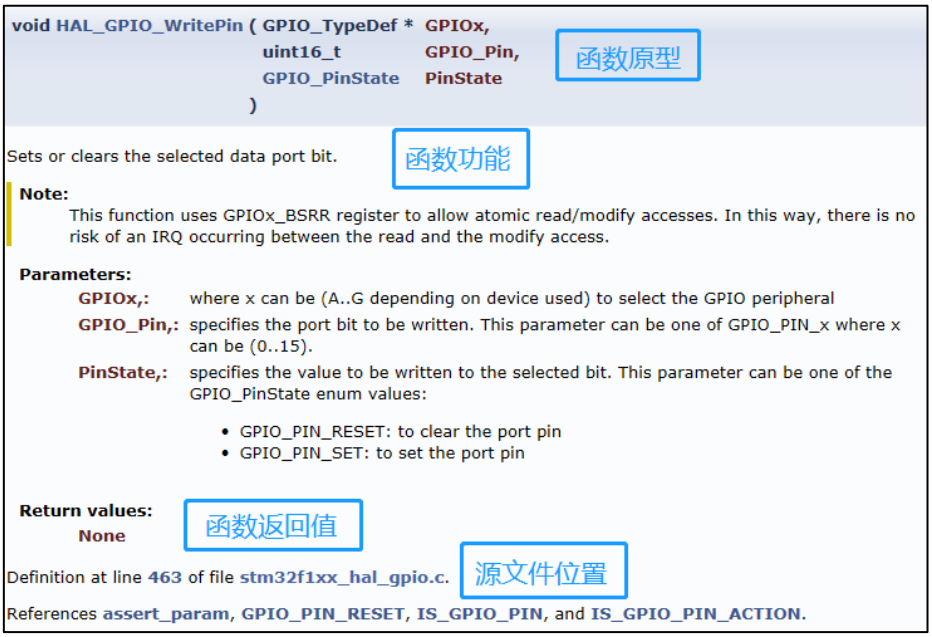


图 6.2.2.2 GPIO 口输出设置函数

函数有三个形参：

第一个形参是 GPIO_TypeDef *GPIOx，形参描述说：x 可以是 A 到 G 之间任何一个，而我们是 PB4 引脚，所以第一个形参确认是 GPIOB。

第二个形参是 uint16_t GPIO_Pin，看形参描述：该参数可以是 GPIO_PIN_x，x 可以 1 到 15，那么我们第二个形参就是 GPIO_PIN_4。

第三个形参是 GPIO_PinState PinState，看形参描述：该参数可以是枚举里的两个数，一

个是 GPIO_PIN_RESET：表示该位清零，另一个是 GPIO_PIN_SET：表示设置该位，即置 1，我们要输出 1，所以要置 1 该位，那么我们第三个形参就是 GPIO_PIN_SET。

最后看函数返回值：None，没有返回值。

所以最后得出我们要调用的函数是：

```
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);
```

帮助文档的使用就讲到这。

本节只针对 HAL 库作了一个简单的介绍，想要了解更多知识，ST 提供了关于 HAL 库 LL 库的更详细的说明文档，我们已经把它放到光盘资料 A 盘《8，STM32 参考资料》中了，大家可以自行查阅《Description of STM32F1 HAL and low-layer drivers.pdf》获取所需知识。

6.3 如何使用 HAL 库

我们要先知道 STM32 芯片的某个外设的性能和工作模式，才能借助 HAL 库来帮助我们编程，甚至修改 HAL 库来适配我们的开发项目。HAL 库的 API 虽多，但是查找和使用有规律可循，只要学会其中一个，其他的外设就是类似的，只是添加自己的特性的 API 而已。

6.3.1 学会用 HAL 库组织开发工具链

需要按照芯片使用手册建议的步骤去配置芯片。HAL 库驱动提供了芯片的驱动接口，但我们需要强调一个概念是使用 HAL 库的开发是对芯片功能的开发，而不是开发这个库，也不是有也这个库能就直接开发。如果我们对芯片的功能不作了解的话，仍然不知道按照怎样的步骤和寻找哪些可用的接口去实现想要实现的功能。ST 提供芯片使用手册《STM32F1xx 参考手册.pdf》告诉我们使用某一外设功能时如何具体地去操作每一个用到的寄存器的细节，后面我们的例程讲解过程也会结合这个手册来分析配置过程。

嵌入式的软件开发流程总遵循以下步骤：组织工具链、编写代码、生成可执行文件、烧录到芯片、芯片根据内部指令执行我们编程生成的可执行代码。

在 HAL 库学习前期，建议以模仿和操作体验为基础，通过例程来学习如何配置和驱动外设。下面根据我们后续要学习的工程梳理出来的基于 CMSIS 的一个 HAL 库应用程序文件结构，帮助读者学习和体会这些文件的组成意义，如下表 6.3.1.1 所示。

类别	文件名	描述	是否必须
用户程序文件	main.c	存放 main 函数，不一定要在这个文件	否
	main.h	包含头文件、声明等作用，已删除	否
	stm32f1xx_it.c	用户中断服务函数存放文件，不一定放到这个文件，可删除	否
	stm32f1xx_it.h		否
	stm32f1xx_hal_conf.h	用户配置文件	是
	stm32f1xx_hal_msp.c	回调函数存放文件，已删除	否
设备驱动层	sm32f1xx_hal.c	HAL 库的初始化、系统滴答，HAL 库延时函数等功能	是
	stm32f1xx_hal.h		是
	stm32hxx_hal_def.h	通用 HAL 库资源定义	是
	stm32f1xx_hal_ppp.c	外设的操作 API 函数文件	是
	stm32f1xx_hal_ppp.h		是
	stm32f1xx_hal_ppp_ex.c	拓展外设特性的 API 函数文件	是
	stm32f1xx_hal_ppp_ex.h		是
	stm32f1xx_II_ppp.c	LL 库文件，在一些复杂外设中实现底层功能	是
	stm32f1xx_II_ppp..h		是
	stm32f1xx.h	STM32F1 系列的顶层头文件	是
	stm32f103xe.h	STM32F103E 系列片上外设头文件	是
	system_stm32f1xx.c	主要存放系统初始化函数 SystemInit	是
	system_stm32f1xx.h		是

CMSIS 核心层	startup_stm32f1xx.s	启动文件，运行到 main 函数前的准备	是
	core_cm3.h	内核寄存器定义，如 Systick、SCB 等	是
	cmsis_armcc.h	内核头文件，一般都不需要去了解	是
	cmsis_armclang.h		
	cmsis_compiler.h		
	cmsis_version.h		
	mpu_armv7.h		

表 6.3.1.1 基于 CMSIS 应用程序文件描述

把这些文件组织起来的方法，我们会在后续章节新建工程中介绍，这只提前告诉一下大家组成我们需要的编译工具链大概需要哪些文件。

6.3.2 HAL 库的用户配置文件

stm32f1xx_hal_conf.h 用于裁剪 HAL 库和定义一些变量，官方没有直接提供这个文件，但在 STM32Cube_FW_F1_V1.8.3\Drivers\STM32F1xx_HAL_Driver\Inc 这个路径下提供了一个模版文件《stm32f1xx_hal_conf_template.h》，我们可以直接复制这个文件重命名为 stm32f1xx_hal_conf.h，做一些简单的修改即可，也可以从在官方的例程中直接复制过来。我们开发板使用的芯片是 STM32F103 的 E 系列，所以也可以从下面的路径获取这个配置文件：STM32Cube_FW_F1_V1.8.3\Projects\STM3210E_EVAL\Templates\Inc。

（1）配置外部高速晶振的频率。 HSE_VALUE 这个参数表示我们的外部高速晶振的频率。这个参数请务必根据我们板子外部焊接的晶振频率来修改，源码在 78 行开始，官方默认是 25M。正点原子 STM32F103 开发板外部高速晶振的频率是 8MHZ。我们没有在代码的其它地方定义过 HSE_VALUE 这个值，所以编译器最终会引用这里的值 8MHz 作为外部调整晶振的频率值。

注意事项：使用官方的开发板需要定义 USE_STM3210C_EVAL 这个宏，我们没有在代码的其它位置中或者编译器的预编译选项中定义过这个宏。

```
#if !defined  (HSE_VALUE)
#if defined(USE_STM3210C_EVAL)
#define HSE_VALUE    25000000U /*!< Value of the External oscillator in Hz */
#else
#define HSE_VALUE    8000000U /*!< Value of the External oscillator in Hz */
#endif
#endif /* HSE_VALUE */
```

还可以把上面的代码直接精简为一行，效果是一样的：

```
#define HSE_VALUE    8000000U /*!< Value of the External oscillator in Hz */
```

（2）还有一个参数就是外部低速晶振频率，这个用于 RTC 时钟，这个官方默认是 32.768KHZ，我们开发板的低速晶振也是这个频率，所以不用修改，源码在 111 行。

```
#if !defined  (LSE_VALUE)
  #define LSE_VALUE    ((uint32_t)32768) /* 外部低速振荡器的值，单位 HZ */
#endif /* LSE_VALUE */
```

（3）用户配置文件可以用来选择使能何种外设，源码配置在 31 行到 71 行，代码如下。

```
/* ########################## Module Selection ############################## */
/**
  * @brief This is the list of modules to be used in the HAL driver
  */
#define HAL_MODULE_ENABLED
#define HAL_ADC_MODULE_ENABLED
#define HAL_CEC_MODULE_ENABLED
#define HAL_COMP_MODULE_ENABLED
#define HAL_CORTEX_MODULE_ENABLED
...中间省略...
#define HAL_USART_MODULE_ENABLED
#define HAL_WWDG_MODULE_ENABLED
#define HAL_MMC_MODULE_ENABLED
```

比如要不使用 GPIO 的功能，把源码在 49 行这个宏注释掉即可，具体如下。

```
/* #define HAL_GPIO_MODULE_ENABLED */
```

结合同样在 stm32f1xx_hal_conf.h 中的 246 行的代码：

```
#ifdef HAL_GPIO_MODULE_ENABLED
#include "stm32f1xx_hal_gpio.h"
#endif /* HAL_GPIO_MODULE_ENABLED */
```

这是一个条件编译符，与#endif 配合使用。这里的要表达的意思是，只要工程中定义了 HAL_GPIO_MODULE_ENABLED 这个宏，就会包含 stm32f1xx_hal_gpio.h 这个头文件到我们的工程，同时 stm32f1xx_hal_gpio.c 中的#ifdef 到#endif 之间的程序（116 行到 579 行）就会参与编译，否则不编译。所以只要我们屏蔽了 stm32f1xx_hal_conf.h 文件 49 行的宏，GPIO 的驱动代码就不被编译。也就起到选择使能何种外设的功能，其他外设同理。使用时定义，否则不定义。这样就可以在不修改源码的前提下方便地裁剪 HAL 库代码的体积了。
注意第一个宏定义：

```
#define HAL_MODULE_ENABLED
```

它决定了 stm32f1xx_hal.c 中的第 47~587 行的代码是否能使用，也是根据条件编译来实现的。其中包含 HAL_Init()、HAL_Delay()、HAL_GetTick()这些其它驱动函数可能需要引用的函数，所以这个宏也是必须要定义的。

官方的示范例程，就是通过屏蔽外设的宏的方法来选择使能何种外设。表现上就是编译时间会变短，因为屏蔽了不使用的 HAL 库驱动，编译时间自然就短了。正点原子的例程选择另外一中方法，就是工程中只保留需要的 stm32f1xx_hal_ppp.c，不需要的不添加到工程里，由于找不到源文件且没有引用这些文件，同样编译器不会去编译这些代码。
关于配置文件我们暂时只讲这些，具体其它需要修改的地方，我们在例程讲解中再去说明。

（4）大家看到 STM32F1xx_hal_conf.h 文件的 127 行。

```
#define  TICK_INT_PRIORITY      ((uint32_t)0x0F) /*!< tick interrupt priority
*/
```

宏定义 TICK_INT_PRIORITY 是滴答定时器的优先级。这个优先级很重要，因为如果其它的外设驱动程序的延时是通过滴答定时器提供的时间基准，来实现延时的话，又由于实现方式是滴答定时器对寄存器进行计数，所以当我们在其它中断服务程序里调用基于此时间基准的延迟函数 HAL_Delay，那么假如该中断的优先级高于滴答定时器的优先级，就会导致滴答定时器中断服务函数一直得不到运行，从而程序卡死在这里。所以滴答定时器的中断优先级一定要比这些中断高。

请注意这个时间基准可以是滴答定时器提供，也可以是其他的定时器，默认是用滴答定时器。

（5）断言这个功能我们在使程中不使用。断言这个功能用来判断 HAL 函数的形参是否有效，并在参数错误时启用这个断言功能，告诉开发者代码错误的位置，断言功能由用户自己决定。这个功能的使能开关代码是一个宏，在源码的 160 行，默认是关闭的，代码如下。

```
/* #define USE_FULL_ASSERT    1 */
```

通过宏 USE_FULL_ASSERT 来选择功能，在源码 375 行到 389，代码如下。

```
#ifdef  USE_FULL_ASSERT
/**
  * @brief  The assert_param macro is used for function's parameters check.
  * @param  expr If expr is false, it calls assert_failed function
  *         which reports the name of the source file and the source
  *         line number of the call that failed.
  *         If expr is true, it returns no value.
  * @retval None
  */
#define assert_param(expr) ((expr)? (void)0U : \
                                assert_failed((uint8_t *)__FILE__,
__LINE__))
/* Exported functions ---------------------------------------------------------
*/
void assert_failed(uint8_t* file, uint32_t line);
#else
#define assert_param(expr) ((void)0U)
#endif /* USE_FULL_ASSERT */
```

也是通过条件编译符来选择对应的功能。当用户自己需要使用断言功能，怎么做呢？首

先需要定义宏 USE_FULL_ASSERT 来使能断言功能，即把源码的 160 行的注释去掉即可。然后看到源码 423 行的 assert_failed()这个函数。根据这个宏定义，我们还需要去定义这个函数的功能，可以按以下格式自定义这个函数，出错后使代码停留在这里：

```
#ifdef  USE_FULL_ASSERT

/**
 * @brief        当编译提示出错的时候此函数用来报告错误的文件和所在行
 * @param        file：指向源文件
 *               line：指向在文件中的行数
 * @retval       无
 */
void assert_failed(uint8_t* file, uint32_t line)
{
    while (1)
    {
    }
}
#endif
```

可以看到这个函数里面没有实现如何功能，就是一个什么不做的死循环，具体功能请根据自己的需求去实现。file 是指向源文件的指针，line 是指向源文件的行数。__FILE__是表示源文件名，__LINE__是表示在源文件中的行数。比如我们可以实现打印出这个错误的两个信息等等，但前提是你已经学会了如何使芯片输出打印信息这些功能。

总的来说断言功能就是，在 HAL 库中，如果定义了 USE_FULL_ASSERT 这个宏，那么所有的 HAL 库函数将会检查函数的形参是否正确。如果错误将会调用 assert_failed()这个函数，程序就会停留在这里，用户可以定位到出错的函数。这个功能实际上是在芯片上运行的时候的增加错误提示信息的功能，属于调试功能的一部分，实际我们的编译器就可以帮助定位到参数错误的问题并提示信息。在 F103 的工程中我们不使用这个功能。

6.3.3 stm32f1xx_hal.c 文件

这个文件内容比较多，包括 HAL 库的初始化、系统滴答、基准电压配置、IO 补偿、低功耗、EXTI 配置等都集合在这个文件里面。下面我们对该文件进行讲解。

1. HAL_Init()函数

源码在 142 行到 167 行，简化函数如下（下面的代码只针对 F1 的 HAL 固件 1.8.5 版本，其它版本可能有差异）：

```
HAL_StatusTypeDef HAL_Init(void)
{
  /* 配置 Flash 的预取控制器 */
#if (PREFETCH_ENABLE != 0)
#if defined(STM32F101x6) || defined(STM32F101xB) || defined(STM32F101xE) ||
defined(STM32F101xG) || \ defined(STM32F102x6) || defined(STM32F102xB) || \
    defined(STM32F103x6) || defined(STM32F103xB) || defined(STM32F103xE) ||
defined(STM32F103xG) || \ defined(STM32F105xC) || defined(STM32F107xC)

  /* Prefetch buffer is not available on value line devices */
  __HAL_FLASH_PREFETCH_BUFFER_ENABLE();
#endif
#endif /* 使能 Flash 的预取控制器 */

  /* 配置中断优先级顺序 */
  HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);

  /* 使用滴答定时器作为时钟基准，配置 1ms 滴答（重置后默认的时钟源为 HSI） */
  HAL_InitTick(TICK_INT_PRIORITY);

  /* 初始化其它底层硬件（如果必要） */
  HAL_MspInit();
```

```
  /* 返回函数状态 */
  return HAL_OK;
}
```

该函数是 HAL 库的初始化函数，原则上在程序中必须优先调用，其主要实现如下功能：

1）使能 Flash 的预取缓冲器（根据闪存编程手册，打开预取指令可以提高对 I-Code 总线的访问效率，且 AHB 时钟的预分频系数不为 1 时，必须打开预取缓冲器）

2）设置 NVIC 优先级分组为 4。

3）配置滴答定时器每 1ms 产生一个中断。

在这个阶段，系统时钟还没有配置好，因此系统还是默认使用内部高速时钟源 HSI 在跑程序。对于 F1 来说，HSI 的主频是 8MHZ。所以如果用户不配置系统时钟的话，那么系统将会使用 HSI 作为系统时钟源。

4）调用 HAL_MspInit 函数初始化底层硬件，HAL_MspInit 函数在 stm32f1xx_hal.c 文件里面做了弱定义。关于弱定义这个概念，后面会有讲解，现在不理解没关系。正点原子的 HAL 库例程是没有使用到这个函数去初始化底层硬件，而是单独调用需要用到的硬件初始化函数。用户可以根据自己的需求选择是否重新定义该函数来初始化自己的硬件。

注意事项：为了方便和兼容性，正点原子的 HAL 库例程中的中断优先级分组设置为分组 2，即把源码的 HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4)改为如下代码：

```
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_2);
```

中断优先级分组为 2，也就是 2 位抢占优先级，2 位响应优先级，抢占优先级和响应优先级的值的范围均为 0-3。

2. HAL_DeInit ()函数

源码在 175 行到 194 行，简化后函数如下：

```
HAL_StatusTypeDef HAL_DeInit(void)
{
  /* 重置所有外设 */
  __HAL_RCC_APB1_FORCE_RESET();
  __HAL_RCC_APB1_RELEASE_RESET();

  __HAL_RCC_APB2_FORCE_RESET();
  __HAL_RCC_APB2_RELEASE_RESET();

  /* 对底层硬件初始化 */
  HAL_MspDeInit();

  /* 返回函数状态 */
  return HAL_OK;
}
```

该函数取消初始化 HAL 库的公共部分，并且停止 systick，是一个可选的函数。该函数做了一下的事：

1）复位了 APB1、APB2 的时钟。

2）调用 HAL_MspDeInit 函数，对底层硬件初始化进行复位。HAL_MspDeInit 也在 STM32F1xx _hal.c 文件里面做了弱定义，并且与 HAL_MspInit 函数是一对存在。HAL_MspInit 函数负责对底层硬件初始化，HAL_MspDeInit 函数则是对底层硬件初始化进行复位。这两个函数都是需要用户根据自己的需求去实现功能，也可以不使用。

3. HAL_InitTick ()函数

源码在 234 行到 255 行，简化函数如下：

```
__weak HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
  /* uwTickFreq 是个枚举类型，如果检测到 uwTickFreq 为零，则返回 */
```

```
  if((uint32_t)uwTickFreq == 0UL)
  {
    return HAL_ERROR;
  }

    /* 配置滴答定时器 1ms 产生一次中断 */
  if (HAL_SYSTICK_Config(SystemCoreClock /(1000UL / (uint32_t)uwTickFreq))>
0U)
  {
    return HAL_ERROR;
  }

  /* 配置滴答定时器中断优先级 */
  if (TickPriority < (1UL << __NVIC_PRIO_BITS))
  {
    HAL_NVIC_SetPriority(SysTick_IRQn, TickPriority, 0U);
    uwTickPrio = TickPriority;
  }
  else
  {
    return HAL_ERROR;
  }

  /* 返回函数状态 */
  return HAL_OK;
}
```

该函数用于初始化滴答定时器的时钟基准，主要功能如下：

1）配置滴答定时器 1ms 产生一次中断。

2）配置滴答定时器的中断优先级。

3）该函数是__weak 修饰的函数，如果在关联工程中的其它地方没有定义__weak 后面的函数，这里是 HAL_InitTick()，则使用此处的定义，否则就使用其它地方定义好的函数功能。我们可以通过重定义这个函数来选择其它的时钟源（如定时器）作为 HAL 库函数的时基或者通过重定义不开启 Systick 的功能和中断等。

该函数可以通过 HAL_Init()或者 HAL_RCC_ClockConfig()重置时钟。在默认情况下，滴答定时器是时间基准的来源。如果其他中断服务函数调用了 HAL_Delay()，必须小心，滴答定时器中断必须具有比调用了 HAL_Delay()函数的其他中断服务函数的优先级高(数值较低)，否则会导致滴答定时器中断服务函数一直得不到执行，从而卡死在这里。

4. 滴答定时器相关的函数

源码在 293 行到 416 行，相关函数如下：

```
/* 该函数在滴答定时器时钟中断服务函数中被调用，一般滴答定时器 1ms 中断一次，
所以函数每 1ms 让全局变量 uwTick 计数值加 1 */
__weak void HAL_IncTick(void)
{
  uwTick += (uint32_t)uwTickFreq;
}

/* 获取全局变量 uwTick 当前计算值 */
__weak uint32_t HAL_GetTick(void)
{
  return uwTick;
}

/* 获取滴答时钟优先级 */
uint32_t HAL_GetTickPrio(void)
{
  return uwTickPrio;
}

/* 设置滴答定时器中断频率 */
```

```
HAL_StatusTypeDef HAL_SetTickFreq(HAL_TickFreqTypeDef Freq)
{
  HAL_StatusTypeDef status  = HAL_OK;
  HAL_TickFreqTypeDef prevTickFreq;

  assert_param(IS_TICKFREQ(Freq));

  if (uwTickFreq != Freq)
  {

    /* 备份滴答定时器中断频率 */
    prevTickFreq = uwTickFreq;

    /* 更新被 HAL_InitTick()调用的全局变量 uwTickFreq */
    uwTickFreq = Freq;

    /* 应用新的滴答定时器中断频率  */
    status = HAL_InitTick(uwTickPrio);
    if (status != HAL_OK)
    {
      /* 恢复之前的滴答定时器中断频率 */
      uwTickFreq = prevTickFreq;
    }
  }

  return status;
}


/* 获取滴答定时器中断频率 */
HAL_TickFreqTypeDef HAL_GetTickFreq(void)
{
  return uwTickFreq;
}

/*HAL 库的延时函数，默认延时单位 ms  */
__weak void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();
  uint32_t wait = Delay;

  /* Add a freq to guarantee minimum wait */
  if (wait < HAL_MAX_DELAY)
  {
    wait += (uint32_t)(uwTickFreq);
  }

  while ((HAL_GetTick() - tickstart) < wait)
  {
  }
}

/* 挂起滴答定时器中断，全局变量 uwTick 计数停止 */
__weak void HAL_SuspendTick(void)
{
  /* 禁止滴答定时器中断 */
  CLEAR_BIT(SysTick->CTRL, SysTick_CTRL_TICKINT_Msk);
}

/* 恢复滴答定时器中断，恢复全局变量 uwTick 计数 */
__weak void HAL_ResumeTick(void)
{
  /* 使能滴答定时器中断 */
  SET_BIT(SysTick->CTRL, SysTick_CTRL_TICKINT_Msk);
}
```

这些函数不是很难，请参照注释理解。注意：如果函数被前缀__weak 定义，则用户可以重新定义该函数。更多的内容可以参考 8.1.5 小节。

5. HAL 库版本相关的函数

源码在 422 行到 484 行，相关函数声明在 stm32f1xx_hal.h 中，详看如下：

```
uint32_t HAL_GetHalVersion(void);   /* 获取 HAL 库驱动程序版本 */
uint32_t HAL_GetREVID(void);        /* 获取设备修订标识符 */
uint32_t HAL_GetDEVID(void);        /* 获取设备标识符 */
uint32_t HAL_GetUIDw0(void);        /* 获取唯一设备标识符的第一个字 */
uint32_t HAL_GetUIDw1(void);        /* 获取唯一设备标识符的第二个字 */
uint32_t HAL_GetUIDw2(void);        /* 获取唯一设备标识符的第三个字 */
```

这些函数了解一下就好了，用得不多。

6. 调试功能相关函数

源码在 490 行到 587 行，函数声明如下：

```
void HAL_DBGMCU_EnableDBGSleepMode(void);
void HAL_DBGMCU_DisableDBGSleepMode(void);
void HAL_DBGMCU_EnableDBGStopMode(void);
void HAL_DBGMCU_DisableDBGStopMode(void);
void HAL_DBGMCU_EnableDBGStandbyMode(void);
void HAL_DBGMCU_DisableDBGStandbyMode(void);
```

这六个函数用于调试功能，默认调试器在睡眠模式下无法调试代码，开发过程中配合这些函数，可以在不同模式下(睡眠模式、停止模式和待机模式)，使能或者失能调试器，当我们使用到时再作详细。

6.3.4 HAL 库中断处理

中断是 STM32 开发的一个很重要的概念，这里我们可以简单地理解为：STM32 暂停了当前手中的事并优先去处理更重要的事务。而这些“更重要的事务”是由软件开发人员在软件中定义的。关于 STM32 中断的概念，我们会在中断例程的讲解再跟大家详细介绍。

由于 HAL 库中断处理的逻辑比较统一，我们将这个处理过程抽象为图 6.3.4.1 所表示的业务逻辑：

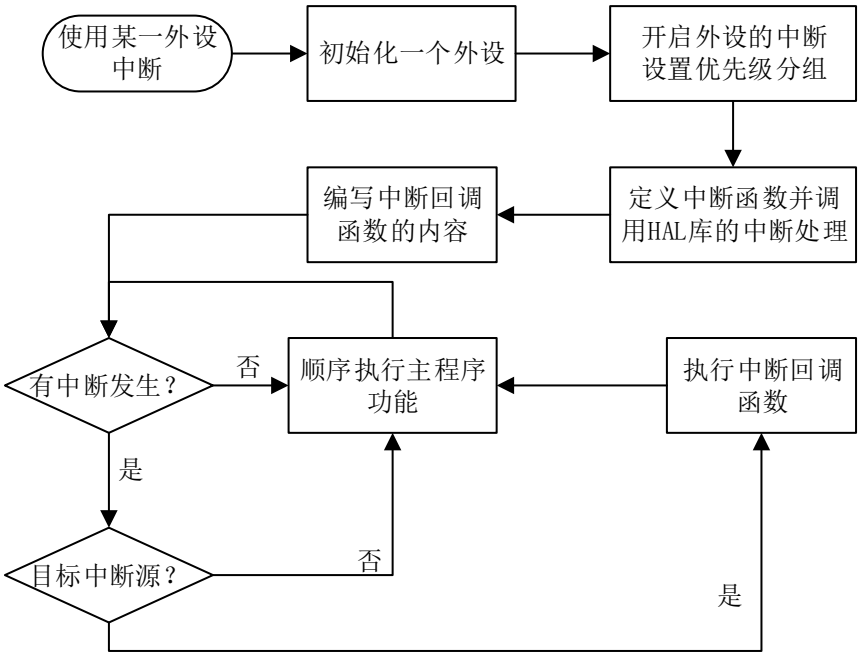


图 6.3.4.1 HAL 驱动中断处理流程

结合以前的 HAL 库文件介绍章节，以上的流程大概就是：设置外设的控制句柄结构体 PPP_HandleType 和初始化 PPP_InitType 结构体的参数，然后调用 HAL 库对应这个驱动的初始化 HAL_PPP_Init()，由于这个 API 中有针对外设初始化细节的接口 Hal_PPP_Mspinit()，我们需要重新实现这个函数并完成外设时钟、IO 等细节差异的设置，完成各细节处理后，使用 HAL_NVIC_SetPriority()、HAL_NVIC_EnableIRQ()来使能我们的外设中断；

定义中断处理函数 PPP_IRQHandler，并在中断函数中调用 HAL_ppp_function_IRQHandler()来判断和处理中断标记；HAL 库中断处理完成后，根据对应中的调用我们需要自定义的中断回调接口 HAL_PPP_ProcessCpltCallback();如串口接收函数 HAL_UART_RxCpltCallback()，我们在这个函数中实现我们对串口接收数据想做的处理;

中断响应处理完成后，stm32 芯片继续顺序执行我们定义的主程序功能，按照以上处理的标准流程完成了一次中断响应。

6.3.5 正点原子对 HAL 库用法的个性化修改

前面介绍了 ST 官方建议的 HAL 库的使用方法，这部分我们结合我们实际例程的编写，列出例程中对 HAL 库使用上的一些与官方推荐用法的差异，读者们请结合自己的使用习惯，辩证地去看待我们这种修改方式：

1，每个例程的 BSP 下都有一个 HAL 库，并且代码中对文件的引用尽量使用相对路径，保证每个复制完整的例程，在其他路径下也能编译通过；这种做法增加了 HAL 库全部例程的体积；

2，将中断处理函数独立到每个外设中，便于独立驱动；同类型的外设驱动处理函数不使用 HAL 回调函数接口处理操作而直接在中断函数中处理判断对应中断；

3，我们把原来的中断分组进行了修改，由抢占式无子优先级改为中断分组 2；便于管理同类外设的优先级响应；

4，我们编写的初始化函数或者芯片操作时序上的延时，我们使用到了 delay_ms()、delay_us()等函数进行初始化，使用的是 Systick 作的精准延时，而 HAL 库默认也使用 Systick 作延时处理，为解决这种冲突和兼容我们大部分的驱动代码，我们在例程中使用 delay.c 中的延时函数取代 Hal_Delay();取消原来 HAL 库的 Systick 延时设置。

6.4 HAL 库使用注意事项

本小节根据经验跟大家讲述一些关于 HAL 库使用的注意事项，供读者参考。

1，即使我们已经在使用库函数作为开发工具了，我们可以忽略很多芯片的硬件外设使用上的细节，但当发生问题时，我们仍需要回归到芯片使用手册查看当前操作是否违规或缺漏；

2，使用 HAL 库和其它第三方的库开发类似，把我们需要编写的软件和第三方的库分开成相互独立的文件，开发过程中我们尽量不去修改第三方的软件源码，需要修改的部分尽量在自己的代码中实现；这样一旦我们需要更新第三方库时，我们原来编写的功能也能很快地匹配新的库去执行功能；

3，即使 HAL 库目前较以前已经相对更完善了，但它仍无法覆盖我们要想实现的所有细节功能，甚至可能存在错误，我们要有怀疑精神，辩证地去使用好这个工具；如我们在 PWM 一节编码时发现 HAL 库中有个宏定义 TIM_RESET_CAPTUREPOLARITY 括号不匹配导致编译报错，这时我们不得不修改一下 HAL 库的源码了。

4，注意 HAL 库的执行效率。由于 HAL 库的驱动对相同外设大多是可重入的，在执行 HAL 驱动的 API 函数的效率没有直接寄存器操作来得高，如果在对时序要求比较严苛的代码，建议使用简洁的寄存器操作代替；

5，我们在例程中使用 delay.c 中的延时函数取代 Hal_Delay();取消原来 HAL 库的 Systick 延时设置；但这会有一个问题：原来 HAL 库的超时处理机制不再适用，所以对于设置了超时的函数，可能会导致停留在这个函数的处理中，无法按正常的超时退出；

6，我们建议如无致命 BUG 出现，尽量使用开发学习时已经测试稳定的 HAL 库来继续进行开发，不必要频繁更新 HAL 库，因为更新 HAL 库后可能会导致原来能正常运行的代

码，在更新 HAL 库之后反面反面无法运行的情况，有些函数操作的结构方式变化了等等，这里只是笔者的建议，大家自己的实际情况权衡。

第七章 SYSTEM 文件夹介绍

SYSTEM 文件夹由正点原子提供，里面包含了一系列系统底层核心驱动代码，目的是为了方便读者快速构建自己的工程。本章将介绍 SYSTEM 文件夹中的代码，也希望读者能够灵活地使用 SYSTEM 文件夹中提供的各种函数来快速构建工程，并应用到实际的项目中。

SYSTEM 文件夹下包含了 sys、delay 两个文件夹，分别包含了 sys.s 和 delay.c 以及相关的头文件，这两个 C 源文件提供了系统时钟设置、延时等功能，通过这些功能能够起到快速移植和辅助开发的作用。

本章分为如下几个小节：

7.1 sys 文件夹代码介绍

7.2 delay 文件夹代码介绍

7.1 sys 文件夹代码介绍

sys 文件夹内包含了 sys.c 和 sys.h 两个文件，在 sys.c 文件中主要实现了如下几个函数：

```
/* 函数声明 */
void sys_nvic_set_vector_table(uint32_t baseaddr, uint32_t offset);
void sys_standby(void);
void sys_soft_reset(void);
uint8_t sys_stm32_clock_init(uint32_t plln);

/* 汇编函数 */
void sys_wfi_set(void);
void sys_intx_disable(void);
void sys_intx_enable(void);
void sys_msr_msp(uint32_t addr);
```

下面讲一下函数的功能，sys_nvic_set_vector_table 函数主要是设置中断向量表偏移地址，sys_standby 函数用于进入待机模式，sys_soft_reset 函数用于系统软复位，sys_stm32_clock_init 函数是系统时钟初始化函数。

在 sys.h 文件中只是对于 sys.c 的函数进行声明。

7.2 delay 文件夹代码介绍

delay 文件夹中包含了 delay.c 和 delay.h 两个文件，在 delay.h 文件中声明了以下一个函数：

```
/* 函数声明 */
void delay_init(uint16_t sysclk);       /* 初始化延时功能 */
void delay_us(uint32_t nus);            /* 微秒级延时 */
void delay_ms(uint16_t nms);            /* 毫秒级延时 */
```

以上三个函数均在 delay.c 文件中实现，分别用于初始化延时功能、微秒级延时和毫秒级延时，该延时功能区分了裸机和操作系统的两种情况。

在裸机的情况下，该延时为阻塞延时，即 CPU 忙延时，说得再通俗一点就是“死等”，再延时期间，CPU 只会不断地判断延时时间是否超时，而不会做其他任何事情。具体的实现是利用了 SysTick，SysTick 是一个 24 位的向下递减计数器，在初始化延时功能的时候，会配置 SysTick 的计数频率为系统时钟频率，即 72MHz，也就是说 SysTick 的计数值减少 72 就耗时 1 微秒（不精确），如果要延时 n 微秒，那么只要让 SysTick 从 n*72 开始计数，当 SysTick 的计数值为 0 时，就说明延时了 n 微秒。而毫秒级延时就是循环延时多个 1000 微秒。

在操作系统的情况下，一般操作系统都会提供一个毫秒级的非阻塞延时，那么就可以使用操作系统提供的延时功能来进行毫秒级延时。而微秒级延时，则先将待延时的微秒数转换为毫秒数进行毫秒级延时，若还有余数，则使用裸机情况下的微秒级延时进行延时。

第八章 STM32CubeIDE 工程模板分析

在第四章的操作中，我们通过第一个工程实验熟悉了 STM32CubeIDE 的基本使用方法，本章节，我们以第一个工程为模板，分析一下工程的结构，了解工程中有哪些文件，他们的关系是什么，有什么作用。通过对工程的分析，我们可以了解工程和 STM32CubeF1 固件包有着怎样密切的关系。

本章将分为如下几个小节：

8.1、STM32CubeIDE 工程文件夹分析；

8.1 STM32CubeIDE 工程文件夹分析

我们打开前面第一个工程 00_f103_test 的保存目录，并对比 STM32CubeIDE 软件上的工程浏览窗口，它们的目录对应关系如下：

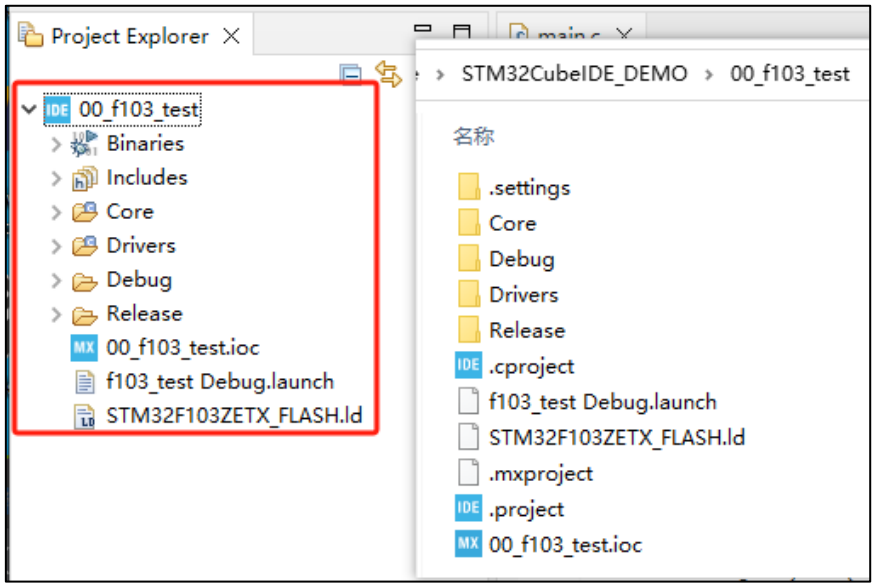


图 8.1.1 CubeIDE 工程目录

下表中是对 00_f103_test 工程文件的一个汇总：

文件	说明
.settings 文件夹	位于工程的保存目录中，是 IDE 的配置文件，一般不用管
Binaries 文件夹	STM32CubeIDE 工程列表中，显示编译生成的.elf 文件
Includes 文件夹	STM32CubeIDE 工程列表中，Includes 文件是工程的头文件路径，编译前可以配置，在前面 4.2.3 小节的 STM32CubeIDE 使用技巧章节我们有进行过配置。
Core 文件夹	STM32CubeIDE 自动生成的操作外设要用到的 API 函数文件和 main 函数文件。
Drivers 文件夹	HAL 库文件，**同时里边我们还会存放两个关于正点原子的一些 BSP 和 SYSTEM 文件夹，其中 BSP 用于存放板级驱动，SYSTEM 用于存放系统时钟设置、延时等功能驱动**
Debug 文件夹	编译生成的文件，对应 Debug 版本，在 4.2.5 小节有介绍
Release 文件夹	编译生成的文件，对应 Release 发行版，在 4.2.5 小节介绍
00_f103_test.ioc 文件	STM32CubeMX 插件工程文件，双击可以通过独立的

	STM32CubeMX 软件来打开工程。
.cproject .project	位于工程的保存目录中，双击可以在 STM32CubeIDE 中打开
f103_test Debug.launch	由 STM32CubeIDE 生成，调试时对应的配置文件，分别对应 Debug 版本和 Release 版本，这些文件可以不用关注。
STM32F103ZETX_FLASH.ld	工程的链接脚本，类似启动文件，堆栈的大小设置，以及 RAM 和 FLASH 的起始地址大小设置等均在此文件。
ATK_Middlewares（由我们自己添加的文件）	**该文件夹用于存放正点原子的中间层组件代码，包括：USMART、MALLOC、TEXT、T9INPUT、PICTURE、GUI、MJPEG、各种 ATK 开头的 LIB、NES、SMS、QR_ENCODE 等**

表 8.1.1 f103_test 工程列表

值得注意的是，我们会在 STM32CubeIDE 生成的工程文件中，添加属于自己的文件夹以及驱动文件（添加教程请查看 4.2.4 小节）。首先在 Drivers 文件夹中，我们会主动创建两个文件夹分别是：BSP 和 SYSTEM，其中 BSP 文件夹主要用于存放板级驱动，如 LED、KEY、LCD、OLED、CAN 等驱动；在 SYSTEM 文件夹主要用于存放系统时钟设置、延时等驱动代码。另外我们还会在 STM32CubeIDE 生成的工程文件的一级目录下，创建 ATK_Middlewares 文件夹，该文件夹用于存放正点原子的中间层组件代码，包括：USMART、MALLOC、TEXT、T9INPUT、PICTURE、GUI、MJPEG、各种 ATK 开头的 LIB、NES、SMS、QR_ENCODE 等。如下图所示：

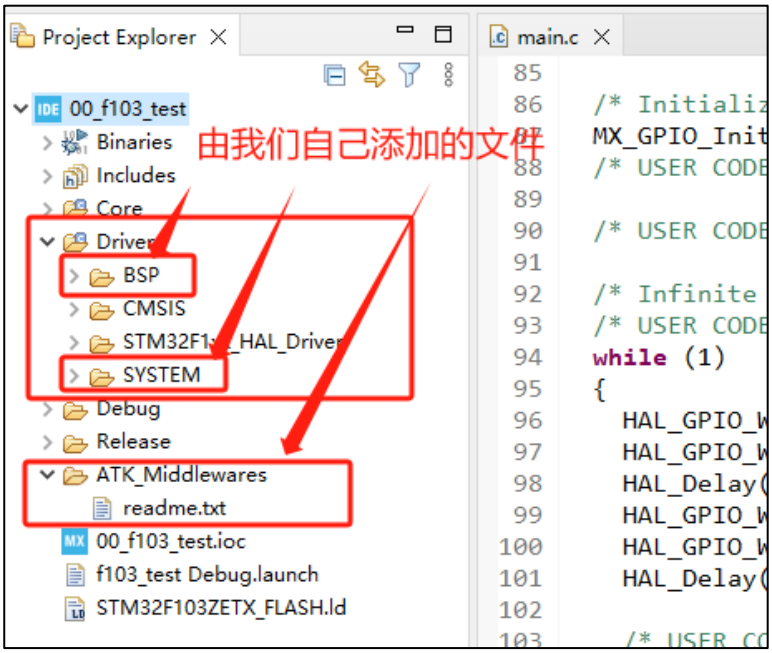


图 8.1.1 添加自己的文件夹及驱动文件

下面我们将详细介绍下 STM32CubeIDE 生成的工程中的文件作用：

1. Binaries

Binaries 下有 00_f103_test.elf 文件，它是指向 Debug 目录下生成的.elf 文件，我们编译完以后可以直接看 Binaries 下有没有.elf 文件，有的话编译成功。可以在工程中选中 00_f103_test.elf 文件然后右键查看 Properties 属性。

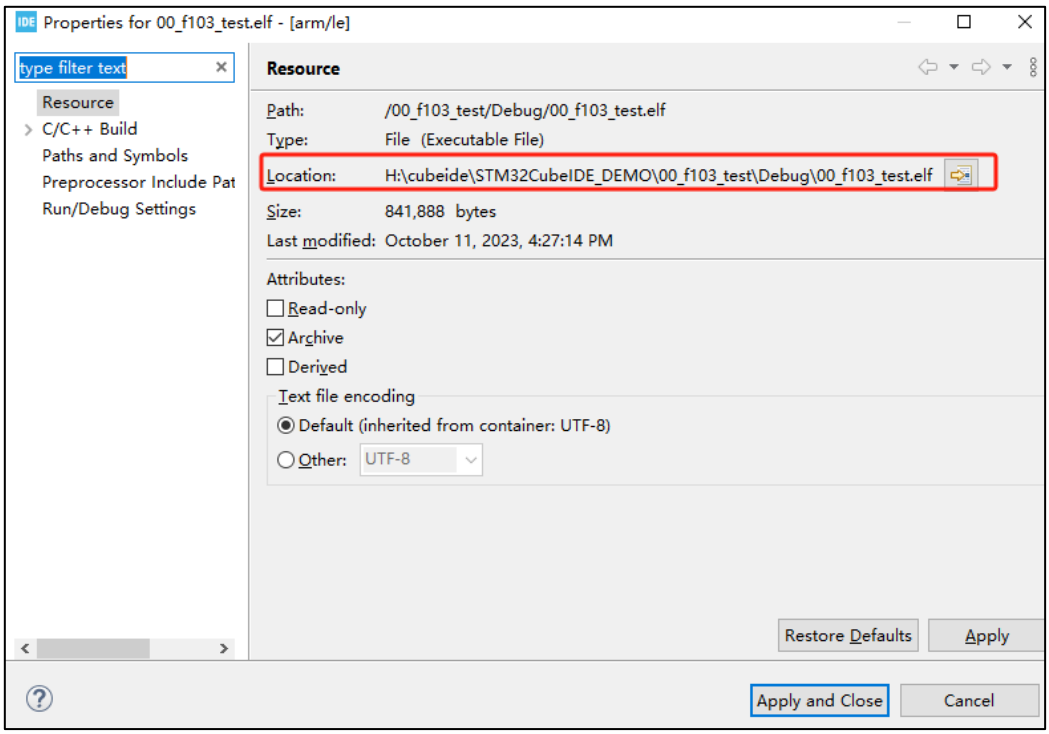


图 8.1.2 工程中 Binaries 文件夹

2. Core 文件夹

Core 文件夹下主要是 STM32CubeIDE 自动生成的操作外设要用到的 API 函数文件和 main 函数文件。其中 Inc 下主要是 API 的头文件，Src 下主要是 API 的源文件，实验中我们操作的是 GPIO，所以会看到有关 GPIO 的 API 函数 gpio.c 和 gpio.h，如果此时在工程中配置 UART，那么就会多出 uart.c 和 uart.h 文件。我们分析一下这些文件：

- gpio.c 文件是 GPIO 引脚配置的代码，例如 GPIO 的时钟使能和 GPIO 的模式配置代码。
- main.c 文件，毋庸置疑，就是主函数文件。
- stm32f1xx_hal_msp.c 文件，此文件提供 MSP 初始化和反初始化代码的代码。
- stm32f1xx_it.c 文件包含异常处理程序和外设中断服务程序，文件中有调用 HAL_IncTick 函数来保证 Systick 每隔 1ms 产生一次中断。中断服务函数也可以编写在其他文件中，比如各个硬件的驱动文件里面，既然 STM32CubeIDE 生成了此文件，那我们就利用此文件好了，在以后的项目开发中，我们的中断服务函数都可以在此文件中添加。
- syscalls.c 文件，syscalls 翻译过来是系统调用的意思。syscalls.c 是 MCU 最小系统调用文件，由 STM32CubeIDE 自动生成。打开文件，文件调用了一些 C 函数库，例如标准输入输出函数 stdio.h，还有返回错误事件错误码的 errno.h 函数等，这些函数的具体实现，可以参考 Newlib 或者 libc 说明手册。

注：

Newlib 和 libc 是一个开源的、面向嵌入式系统的 C 运行库，可以将这些库移植到我们工程代码中，详细信息可以参考官网链接：

http://www.gnu.org/software/libc/
https://sourceware.org/newlib/

syscalls.c 文件中还有 getpid（取得进程识别码）函数，例如_write 函数和_read 函数，如果我们做串口实验会遇到 printf 的重定向问题，printf 的调用流程是先调用_write 函数，再调用__io_putchar 或__io_getchar 函数，那么我们可以直接调用 syscalls.c 文件，然后根据需要可以修改里边的内容。

- sysmem.c 文件

sysmem.c 文件，翻译过来就是和系统内存有关的文件，此文件是 STM32CubeIDE 系统内存调用文件，文件中调用了_sbrk 函数，将内存分配给 newlib 堆，并由 malloc 使用。其实也

是 C 库函数，可以参考 Newlib 或者 libc 说明手册进一步了解，此函数不是重点介绍内容。

3. Startup 文件夹

Startup 下是启动文件 startup_stm32f103zetx.s，它是是系统上电后第一个运行的程序，主要用于：

①设置栈指针 SP；
②设置初始 PC= Reset_Handler；
③设置中断向量表入口地址，并初始化向量表；
④初始化.data 和 .bss 段；
⑤跳转到 C 库中的 main

4. Drivers 文件夹

Drivers 文件夹下就是 HAL 库了，里边的文件是 STM32CubeIDE 从之前下载的 STM32CubeF1 固件包中拷贝的。记得我们之前在创建工程的时候采用默认配置，在 STM32CubeMX 插件中配置了 Copy only the necessary library files 选项，意思就是，生成工程时，会从 STM32CubeF1 固件包中拷贝需要用到的 HAL 库文件，没用到的不会拷贝，这样生成的工程就比较精简和小巧。

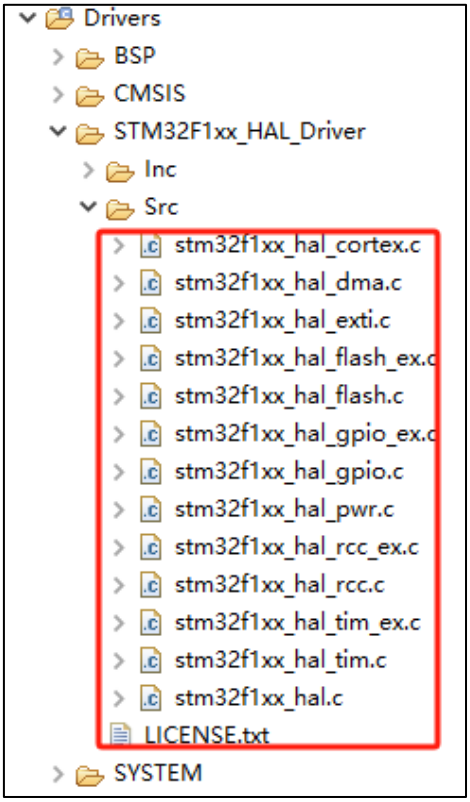


图 8.1.3 工程的 Drivers 文件夹

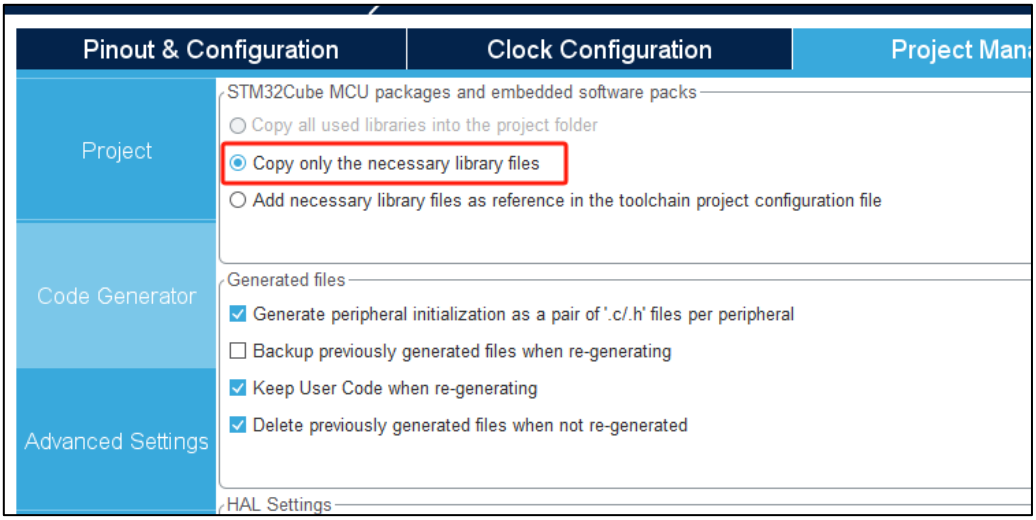


图 8.1.4 CubeMX 的配置

5. Debug 文件夹

Debug 文件夹下就是编译生成的文件。Core 和 Drivers 文件夹下有编译生成的中间文件。Debug 文件夹根目录下有编译生成的可执行.elf 文件和 bin 格式文件，这些格式的文件可以通过配置 STM32CubeIDE 来生成，我们在 4.2.5 小节有讲解怎么配置。.map 文件是地图文件，.list 文件是反汇编文件，makefile 文件是控制工程编译规则的文件。

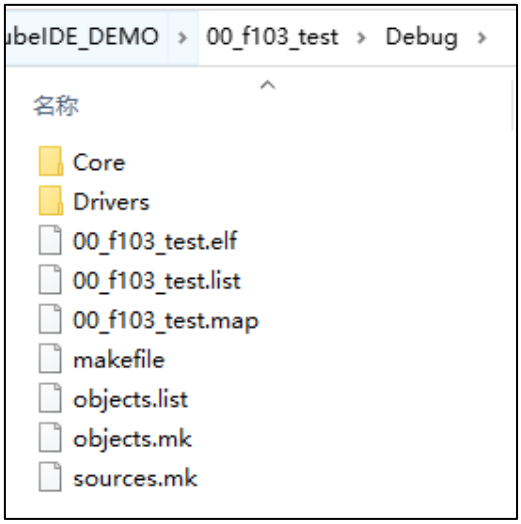


图 8.1.5 Debug 文件夹

6. Release 文件夹

对应 Release 发行版编译生成的文件，我们有在 4.2.6 小节介绍过。

第九章 STM32 时钟配置

MCU 都是基于时序控制的一个系统。这一讲将结合《STM32F10xxx 参考手册_V10（中文版）.pdf》和《Cortex-M3 权威指南》的知识，对 STM32F1 的整体架构作一个简单的介绍，帮助大家更全面、系统地认识 STM32F1 系统的主控结构。了解时钟系统在整个 STM32 系统的贯穿和驱动作用，学会设置 STM32 的系统时钟。

本章将分为如下几个小节：

9.1 认识时钟树

9.1 认识时钟树

数字电路的知识告诉我们：任意复杂的电路控制系统都可以经由门电路组成的组合电路实现。回顾《第五章 STM32 基础知识入门》的知识点，我们知道 STM32 内部也是由多种多样的电路模块组合在一起实现的。当一个电路越复杂，在达到正确的输出结果前，它可能因为延时会有一些短暂的中间状态，而这些中间状态有时会导致输出结果会有一个短暂的错误，这叫做电路中的“毛刺现象”，如果电路需要运行得足够快，那么这些错误状态会被其它电路作为输入采样，最终形成一系列的系统错误。为了解决这个问题，在单片机系统中，设计时以时序电路控制替代纯粹的组合电路，在每一级输出结果前对各个信号进行采样，从而使得电路中某些信号即使出现延时也可以保证各个信号的同步，可以避免电路中发生的“毛刺现象”，达到精确控制输出的效果。

由于时序电路的重要性，因此在 MCU 设计时就设计了专门用于控制时序的电路，在芯片设计中称为时钟树设计。由此设计出来的时钟，可以精确控制我们的单片机系统，这也是我们这节要展开分析的时钟分析。为什么是时钟树而不是时钟呢？一个 MCU 越复杂，时钟系统也会相应地变得复杂，如 STM32F1 的时钟系统比较复杂，不像简单的 51 单片机一个系统时钟就可以解决一切。对于 STM32F1 系列的芯片，正常工作的主频可以达到 72Mhz，但并不是所有外设都需要系统时钟这么高的频率，比如看门狗以及 RTC 只需要几十 kHZ 的时钟即可。同一个电路，时钟越快功耗越大，同时抗电磁干扰能力也会越弱，所以对于较为复杂的 MCU 一般都是采取多时钟源的方法来解决这些问题。

STM32 本身非常复杂，外设非常的多，为了保持低功耗工作，STM32 的主控默认不开启这些外设功能。用户可以根据自己的需要决定 STM32 芯片要使用的功能，这个功能开关在 STM32 主控中也就是各个外设的时钟。

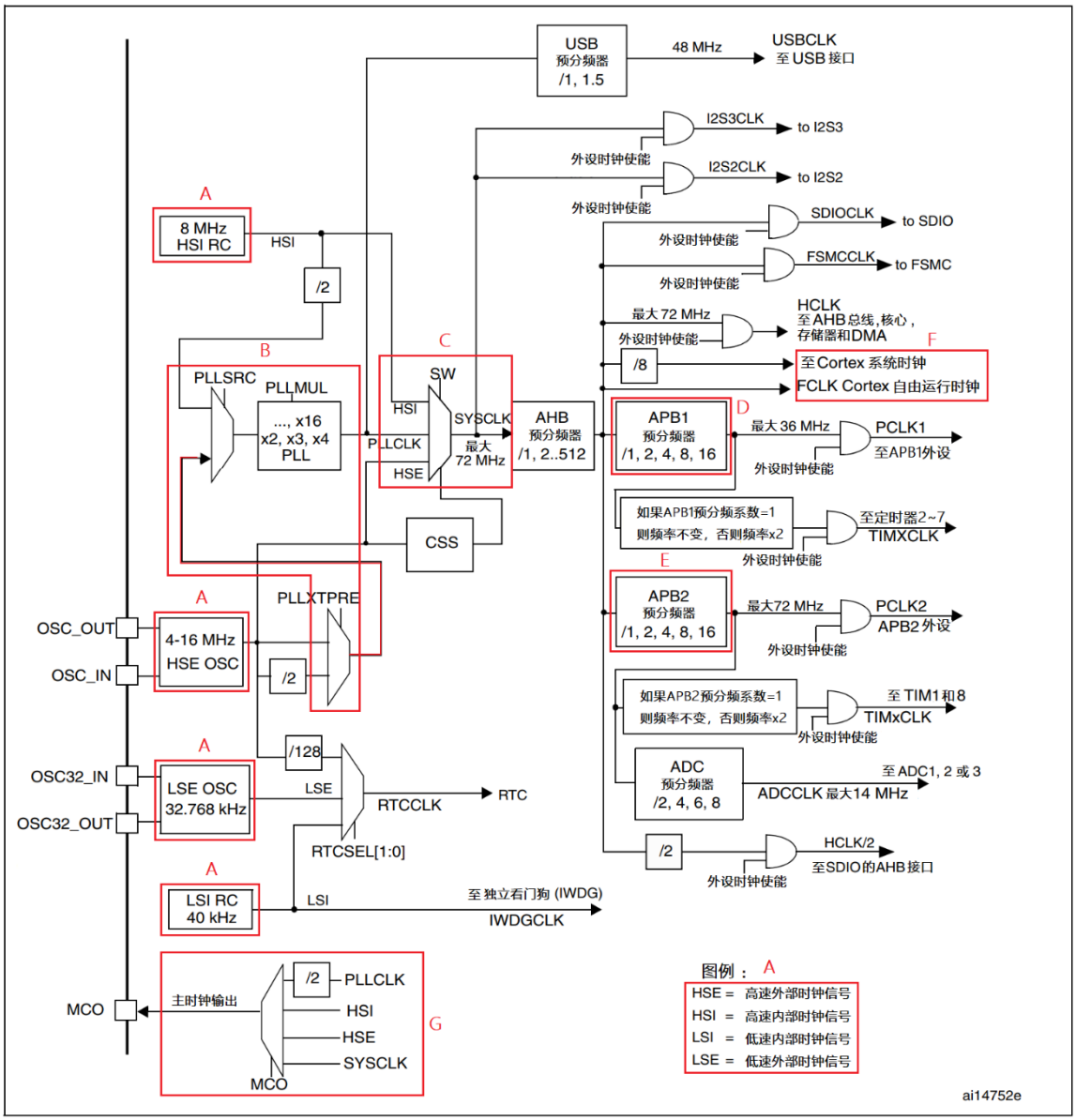


图 9.1.1 STM32F1 时钟系统图

如图 9.1.1 为一个简化的 STM32F1 时钟系统。图中已经把我们主要关注几处标注出来。A 部分表示其他电路需要的输入源时钟信号；B 为一个特殊的振荡电路“PLL”，由几个部分构成；C 为我们重点需要关注的 MCU 内的注释中“SYSCLK”；AHB 预分频器将 SYSCLK 分频或不分频后分发给其他外设进行处理，包括到 F 部分的 Cortex-M 内核系统的时钟。D 和 E 部分分别为定时器等外设的时钟源 APB1/APB2。G 是 STM32 的时钟输出功能，其他部分等我们学习到再详细探讨。接下来我们来详细了解这些部分的功能。

9.1.1 时钟源

对于 STM32F1，输入时钟源（Input Clock）主要包括 HSI，HSE，LSI，LSE。其中，从时钟频率来分可以分为高速时钟源和低速时钟源，其中 HSI、HSE 高速时钟，LSI 和 LSE 是低速时钟。从来源可分为外部时钟源和内部时钟源，外部时钟源就是从外部通过接晶振的方式获取时钟源，其中 HSE 和 LSE 是外部时钟源；其他是内部时钟源，芯片上电即可产生，不需要借助外部电路。下面我们看看 STM32 的时钟源。

（1）2 个外部时钟源：

- **高速外部振荡器 HSE (High Speed External Clock signal)**

外接石英/陶瓷谐振器，频率为 4MHz~16MHz。本开发板使用的是 8MHz。

- **低速外部振荡器 LSE (Low Speed External Clock signal)**

外接 32.768kHz 石英晶体，主要作用于 RTC 的时钟源。

（2）2 个内部时钟源：

- **高速内部振荡器 HSI(High Speed Internal Clock signal)**

由内部 RC 振荡器产生，频率为 8MHz。

- **低速内部振荡器 LSI(Low Speed Internal Clock signal)**

由内部 RC 振荡器产生，频率为 40kHz，可作为独立看门狗的时钟源。

芯片上电时默认由内部的 HSI 时钟启动，如果用户进行了硬件和软件的配置，芯片才会根据用户配置调试尝试切换到对应的外部时钟源，所以同时了解这几个时钟源信号还是很有必要的。如何设置时钟的方法我们会在后文提到。

9.1.2 锁相环 PLL

锁相环是自动控制系统中常用的一个反馈电路，在 STM32 主控中，锁相环的作用主要有两个部分：输入时钟净化和倍频。前者是利用锁相环电路的反馈机制实现，后者我们用于使芯片在更高且频率稳定的时钟下工作。

在 STM32 中，锁相环的输出也可以作为芯片系统的时钟源。根据图 9.1.1 的时钟结构，使用锁相环时只需要进行三个部分的配置。为了方便查看，截取了使用 PLL 作为系统时钟源的配置部分，如图 9.1.2.1 所示。

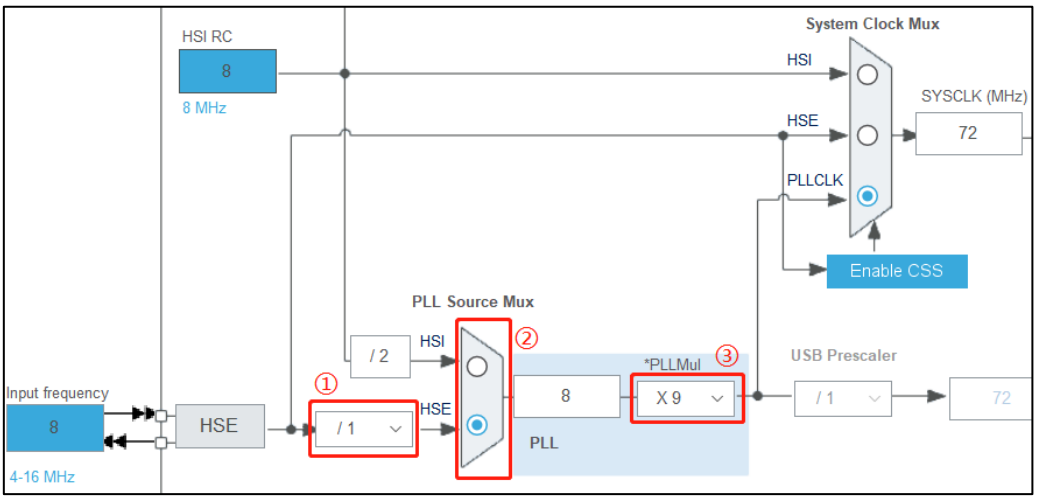


图 9.1.2.1 PLL 时钟配置图

图 9.1.2.1 借用了在 CubeMX 下用锁相环配置 72MHz 时钟的一个示例：

◆ PLLXTPRE：HSE 分频器作为 PLL 输入 (HSE divider for PLL entry)

即图 9.1.2.1 在标注为①的地方，它专门用于 HSE，ST 设计它有两种方式，并把它的控制功能放在 RCC_CFGR 寄存器中，我们引用如图 9.1.2.2。

位17	**PLLXTPRE**：HSE分频器作为PLL输入 (HSE divider for PLL entry) 由软件置'1'或清'0'来分频HSE后作为PLL输入时钟。只能在关闭PLL时才能写入此位。 0：HSE不分频 1：HSE 2分频

图 9.1.2.2 PLLXTPRE 设置选项值

从 F103 参考手册可知它的值有两个：一是 2 分频，另一种是 1 分频（不分频）。

经过 HSE 分频器处理后的输出振荡时钟信号比直接输入的时钟信号更稳定。

◆ PLLSRC： PLL 输入时钟源 (PLL entry clock source)

图中②表示的是 PLL 时钟源的选择器，同样的，参考 F103 参考手册：

位16	**PLLSRC**：PLL输入时钟源 (PLL entry clock source) 由软件置'1'或清'0'来选择PLL输入时钟源。只能在关闭PLL时才能写入此位。 0：HSI振荡器时钟经2分频后作为PLL输入时钟 1：HSE时钟作为PLL输入时钟。

图 9.1.2.3 PLLSRC 锁相环时钟源选择

它有两种可选择的输入源：设计为 HSI 的二分频时钟，另一个是 A 处的 PLLXTPRE 处理后的 HSE 信号。

◆ PLLMUL： PLL 倍频系数 (PLL multiplication factor)

图中③所表示的配置锁相环倍频系数，同样地可以查到在 STM32F1 系列中，ST 设置它的有效倍频范围为 2~16 倍。

结合图 9.1.2.1，要实现 72MHz 的主频率，我们通过选择 HSE 不分频作为 PLL 输入的时钟信号，即输入 8Mhz，通过标号③选择倍频因子,可选择 2-16 倍频，我们选择 9 倍频，这样可以得到时钟信号为 8*9=72MHz。

9.1.3 系统时钟 SYSCLK

STM32 的系统时钟 SYSCLK 为整个芯片提供了时序信号。我们已经大致知道 STM32 主控是时序电路链接起来的。对于相同的稳定运行的电路，时钟频率越高，指令的执行速度越快，单位时间能处理的功能越多。STM32 的系统时钟是可配置的，在 STM32F1 系列中，它可以为 HSI、PLLCLK、HSE 中的一个，通过 CFGR 的位 SW[1:0]设置。

讲解 PLL 作为系统时钟时，根据我们开发板的资源，可以把主频通过 PLL 设置为 72MHz。仍使用 PLL 作为系统时钟源，如果使用 HSI/2，那么可以得到最高主频 8MHz/2*16=64MHz。

从上面的图 9.1.1 时钟树图可知，AHB、APB1、APB2、内核时钟等时钟通过系统时钟分频得到。根据得到的这个系统时钟，下面我们结合外设来看一看各个外设时钟源。

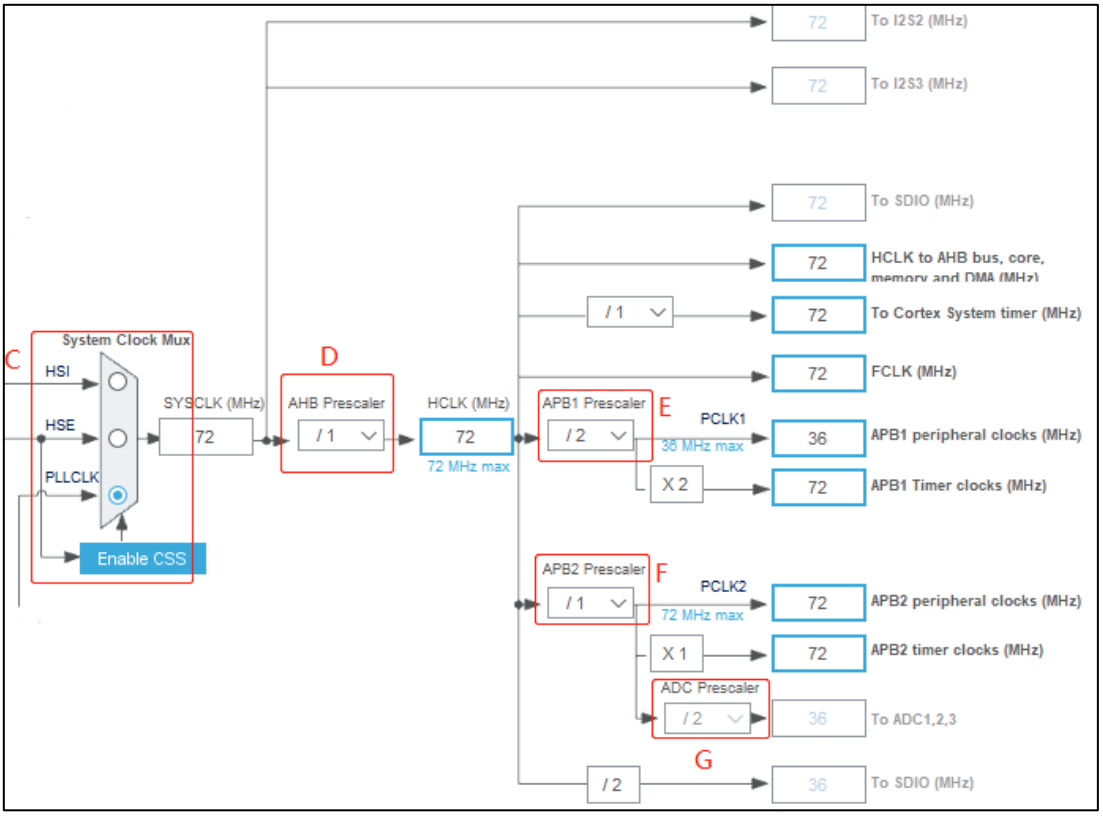


图 9.1.3.1 STM32F103 系统时钟生成图

大家看图 9.1.3.1 STM32F103 系统时钟，标号 C 为系统时钟输入选择，可选时钟信号有外部高速时钟 HSE(8M)、内部高速时钟 HSI(8M)和经过倍频的 PLL CLK(72M)，选择 PLL CLK 作为系统时钟，此时系统时钟的频率为 72MHz。系统时钟来到标号 D 的 AHB 预分频器，

其中可选择的分频系数为 1，2，4，8，16，32，64，128，256，我们选择不分频，所以 AHB 总线时钟达到最大的 72MHz。

下面介绍一下由 AHB 总线时钟得到的时钟：

APB1 总线时钟，由 HCLK 经过标号 E 的低速 APB1 预分频器得到，分频因子可以选择 1，2，4，8，16，这里我们选择的是 2 分频，所以 APB1 总线时钟为 36M。由于 APB1 是低速总线时钟，所以 APB1 总线最高频率为 36MHz，片上低速的外设就挂载在该总线上，例如有看门狗定时器、定时器 2/3/4/5/6/7、RTC 时钟、USART2/3/4/5、SPI2(I2S2)与 SPI3(I2S3)、I2C1 与 I2C2、CAN、USB 设备和 2 个 DAC。

APB2 总线时钟，由 HCLK 经过标号 F 的高速 APB2 预分频器得到，分频因子可以选择 1，2，4，8，16，这里我们选择的是 1 即不分频，所以 APB2 总线时钟频率为 72M。与 APB2 高速总线链接的外设有外部中断与唤醒控制、7 个通用目的输入/输出口(PA、PB、PC、PD、PE、PF 和 PG)、定时器 1、定时器 8、SPI1、USART1、3 个 ADC 和内部温度传感器。其中标号 G 是 ADC 的预分频器在后面 ADC 实验中详细说明。

此外，AHB 总线时钟直接作为 SDIO、FSMC、AHB 总线、Cortex 内核、存储器和 DMA 的 HCLK 时钟，并作为 Cortex 内核自由运行时钟 FCLK。

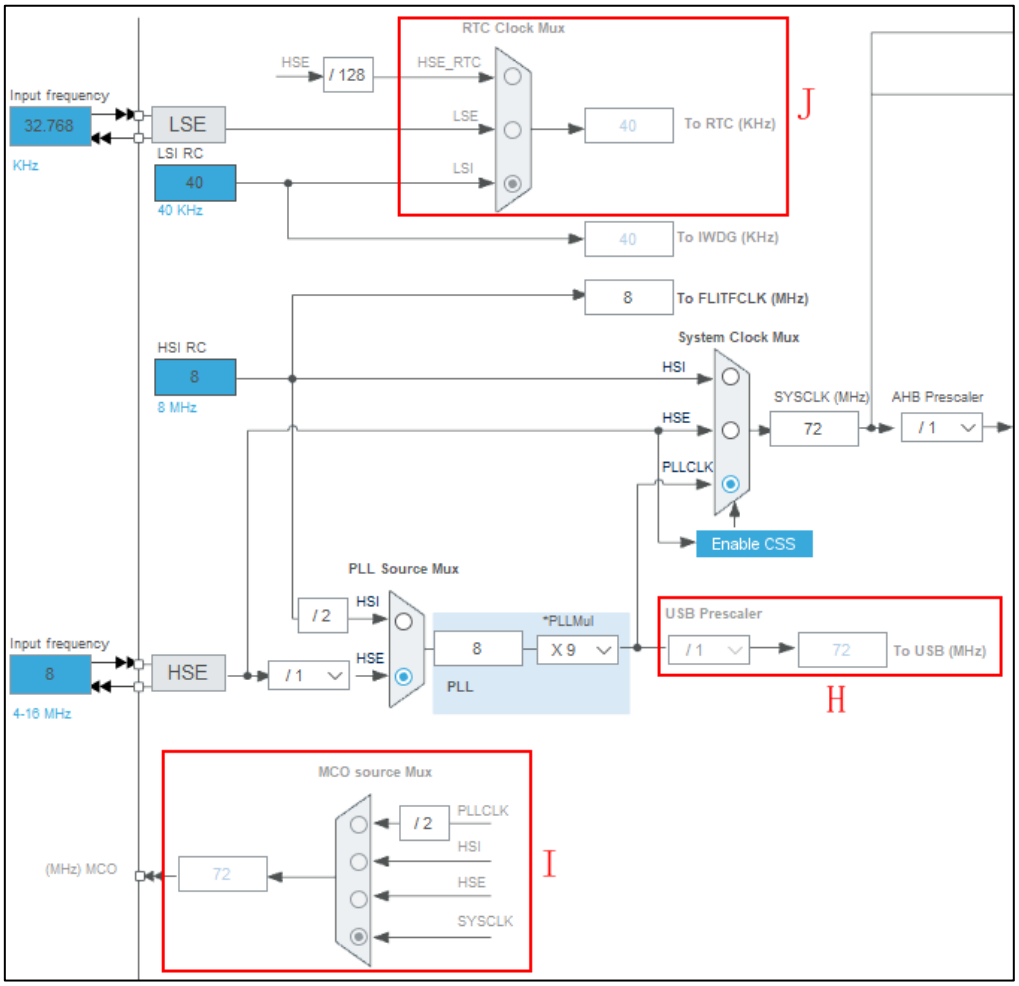


图 11.2.3.1 USB、RTC、MCO 相关时钟

标号 H 是 USBCLK，是一个通用串行接口时钟，时钟来源于 PLLCLK。STM32F103 内置全速功能的 USB 外设，其串行接口引擎需要一个频率为 48MHz 的时钟源。该时钟源只能从 PLL 输出端获取，可以选择为 1.5 分频或者 1 分频，也就是，当需要使用 USB 模块时，PLL 必须使能，并且时钟频率配置为 48MHz 或 72MHz。

标号 I 是 MCO 输出内部时钟，STM32 的一个时钟输出 IO(PA8)，它可以选择一个时钟信号输出，可以选择为 PLL 输出的 2 分频、HSI、HSE、或者系统时钟。这个时钟可以用来给外部其他系统提供时钟源。

标号 J 是 RTC 定时器，其时钟源为 HSE/128、LSE 或 LSI。

9.1.4 时钟信号输出 MCO

STM32 允许通过设置，通过 MCO 引脚输出一个稳定的时钟信号。在图 9.1.1 中标注为“G”的部分。以下四个时钟信号可被选作 MCO 时钟：

- SYSCLK
- HSI
- HSE
- 除 2 的 PLL 时钟

时钟的选择由时钟配置寄存器(RCC_CFGR)中的 MCO[2:0]位控制。

我们可以通过 MCO 引脚来输出时钟信号，测试输出时钟的频率，或作为其它需要时钟信号的外部电路的时钟。

第二篇 实战篇

功夫不负有心人，相信学习至此你已经掌握了基础篇介绍的知识。我们希望通过前面的章节你已经掌握了 STM32 开发的工具和方法。本篇我们将和大家一起来学习 STM32 的一些基础外设，这些外设实际项目中经常会用到，希望大家认真学习和掌握，以便将来更好、更快的完成实际项目开发。

本篇所涉及的实验，在原理部分并未进行详细介绍。在本篇中我们将重点关注如何使用 STM32CubeIDE 进行配置以实现 STM32 的外设功能。对于想要详细了解实验原理部分的读者，请查阅相应开发板 A 盘资料中的《xxx 开发指南》PDF 文档。

本篇将以一章一例的方式介绍如何使用 STM32CubeIDE 配置 STM32 常用外设。通过学习本篇内容，将熟练掌握使用 STM32CubeIDE 配置 STM32 外设功能。

第十章 跑马灯实验

本章，通过一个经典的跑马灯程序，带领大家了解 STM32 的 IO 口作为输出使用的方法，我们将通过 STM32CubeIDE 生成代码，并在此基础上加上我们的逻辑代码控制开发板上的两个 LED 灯 DS0 和 DS1 交替闪烁，实现类似跑马灯的效果。本章分为如下几个小节：

10.1 硬件设计
10.2 程序设计
10.3 下载验证

10.1 硬件设计

1. 例程功能

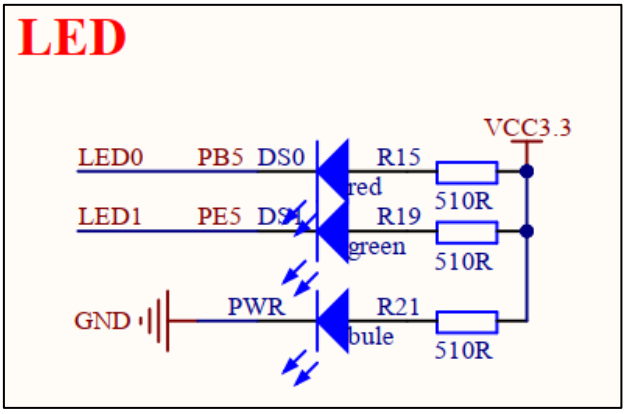
1. LED0 和 LED1 以 500 毫秒的频率交替闪烁，实现类似跑马灯的效果。

2. 硬件资源

1. LED
   LED0 - PB5
   LED1 - PE5

3. 原理图

本章实验用的两个 STM32 精英开发板板载 LED，分别为 LED0（红色）和 LED1（绿色），其与板载 MCU 的连接原理图，如下图所示：



10.1.1 LED 与 MCU 的连接原理图

根据上述原理图可见，LED0 和 LED1 的正极分别经由限流电阻与电源正极相连，而它们的负极则分别与 MCU 的 PB5 引脚和 PE5 引脚相连。因此，通过控制 PB5 引脚或 PE5 引脚输出低电平，即可控制 LED0 和 LED1 的点亮，反之，则熄灭。

10.2 程序设计

本章节我们通过 HAL 库的 API 函数来驱动 LED，实现 LED0 和 LED1 以 500ms 交替闪烁。其中，我们会用到 HAL_GPIO_WritePin 和 HAL_GPIO_TogglePin 函数。

10.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 01_LED。

10.2.2 GPIO 功能引脚配置

我们新建一个工程 LED，进入 STM32CubeMX 插件配置界面后，在 Pin 搜索管脚处输入 PB5，在引脚排列视图处可以看到一个引脚在闪烁，这个就是我们要找的 PB5。

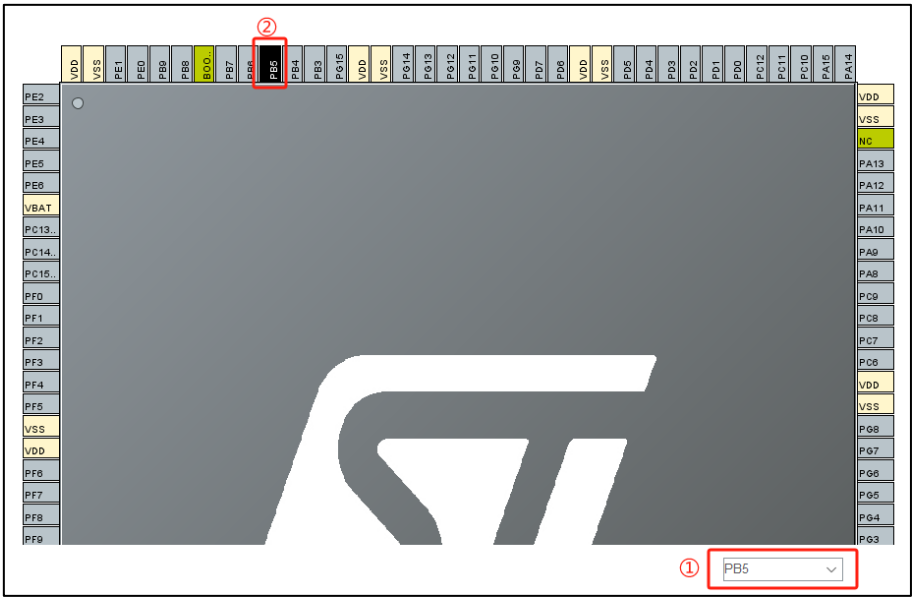


图 10.2.2.1 搜索 PB5 引脚

我们点击闪烁的引脚 PB5，在弹出的复用功能菜单中选择 GPIO_Output。

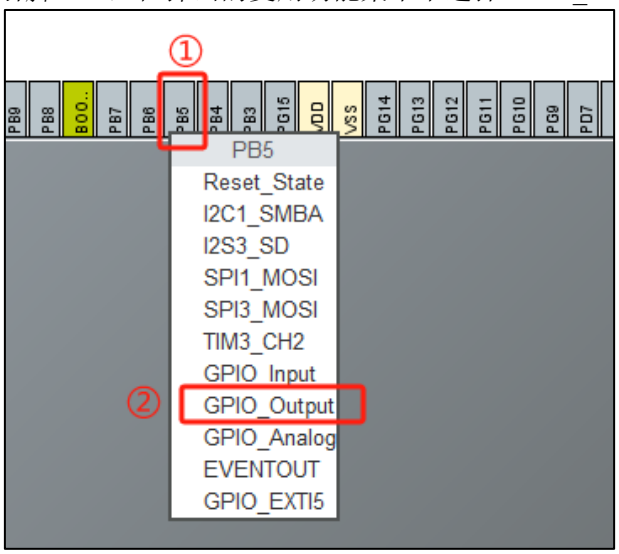


图 10.2.2.2 配置 PB5 引脚复用功能

按照前面同样的方法我们配置 PE5。接下来，我们配置 GPIO 的工作模式：

设置好即可看到引脚从灰色变成绿色，标识该管脚已经启用，默认标识是选择的功能，作为输出功能。同时通过 System Core 下的 GPIO 选项可以看到刚刚设置的管脚功能。

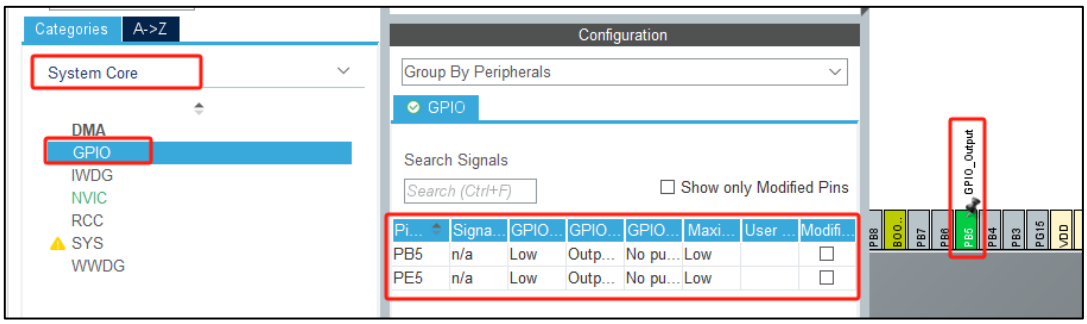


图 10.2.2.3 GPIO 设置菜单

单击 PB5 进入 IO 详情配置：

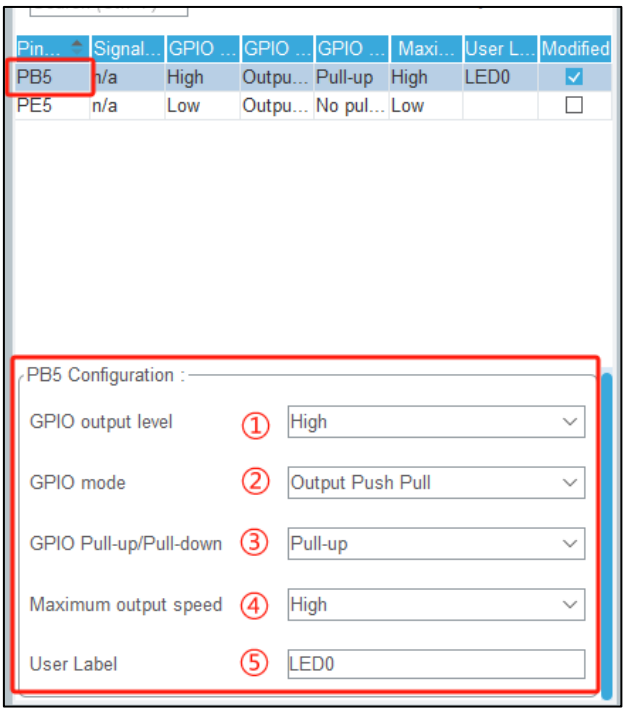


图 10.2.2.4 配置 LED0 工作模式

① 选项 GPIO output level 用于确定 GPIO 上电的时候，保持用户想要的电平，这里我们设置为输出高电平，保持 LED 默认状态为熄灭状态。

② 选项 GPIO mode 用来设置 IO 口输出模式为 Output Push Pull(推挽)还是 Output Open Drain（开漏）。本实验我们设置为推挽输出 Output Push Pull。

③ 选项 GPIO Pull-up/Pull-down 用来设置 IO 口是上拉/下拉/没有上下拉。本实验我们设置为上拉（Pull-up）。

④ 选项 Mzximum ouput speed 用来设置 IO 口输出速度为低速(Low)/中速(Medium)/高速(Hign)。本实验我们设置为高速 High 就可以了。

⑤ 选项 User Label 是用来设置初始化的 IO 口 Pin 值为我们自定义的宏，一般情况我们可以不用设置，这里我们填写为 LED0。

按照前面同样的方法我们配置 PE5，User Label 为 LED1：

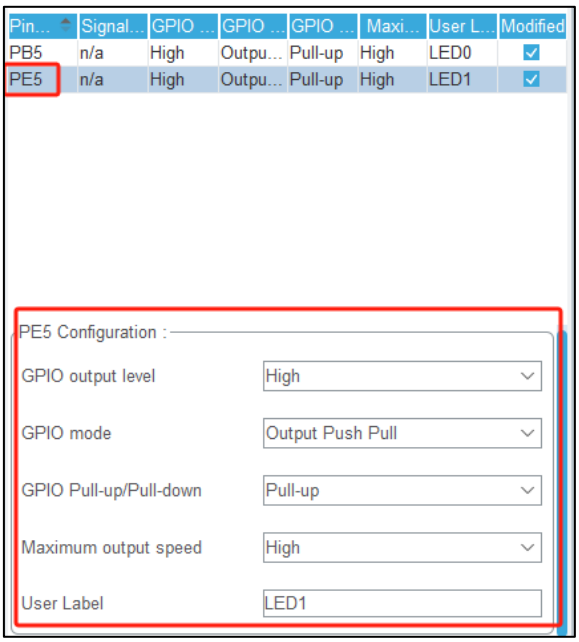


图 10.2.2.5 配置 LED1 工作模式

10.2.3 时钟和工程配置（后续实验此步骤基本一样的配置，后面不在过多介绍此步骤）

首先默认 RCC 时钟源都是使用的内部晶振，我们需要改成外部的晶体/陶瓷振荡器。如下图所示。

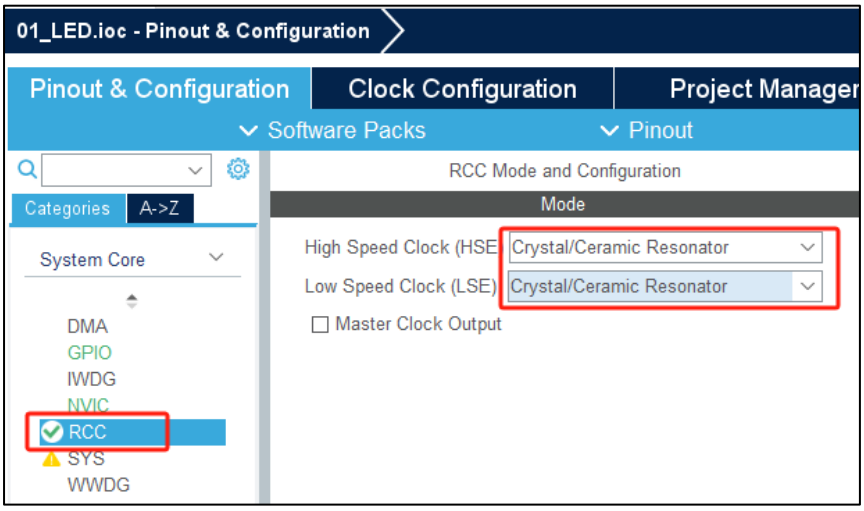


图 10.2.3.1 时钟源设置

此时可以看到芯片这两对管脚已经启用了，并且配置为晶振功能使用。

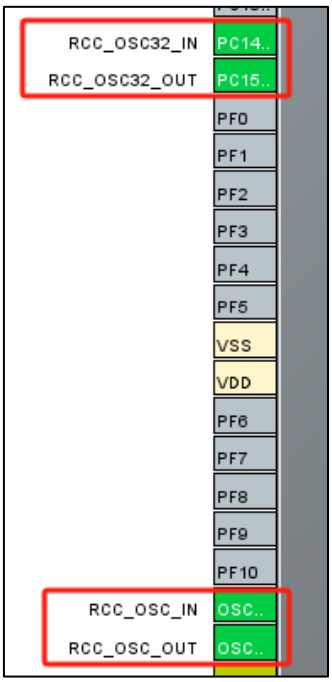


图 10.2.3.2 晶振管脚启用

接下来，我们将修改时钟设置。在 Clock Configuration 界面中，可以看到外部振荡器已经启用，并且框图变成蓝色表示该时钟源处于可用状态。如果框图显示为灰色，则表示该时钟源无法使用。

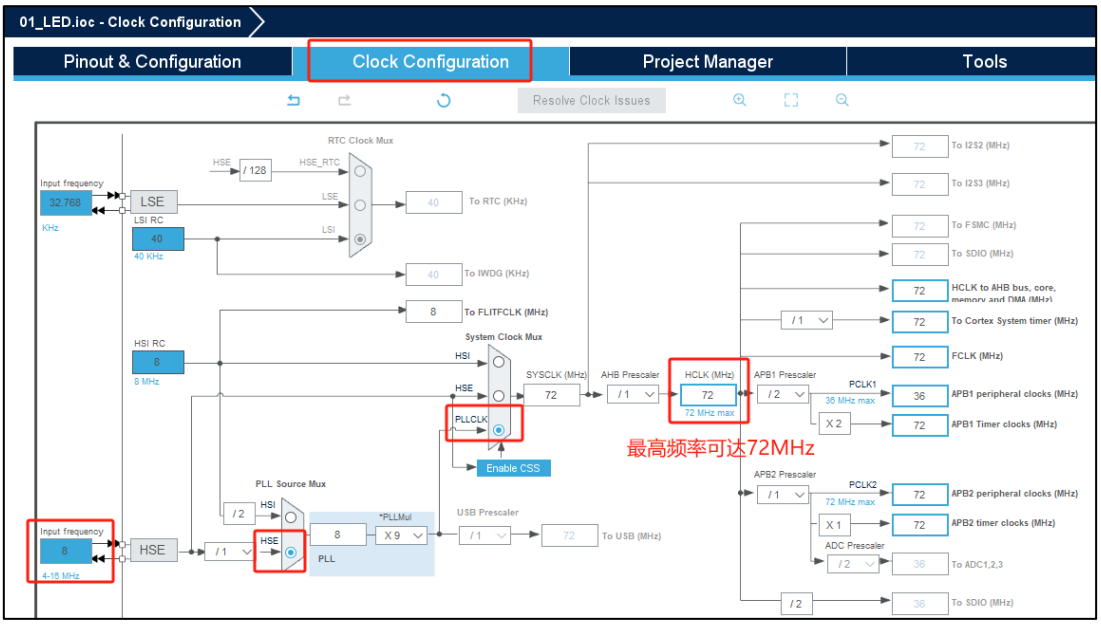


图 10.2.3.3 系统时钟设置

鉴于精英开发板板载的外部高速晶振为 8MHz，而官方推荐的主频时钟为 72MHz，因此我们将其配置为最高推荐主频 72MHz。当然，也可以选择较低的频率，但这将会降低运行速度并影响性能。除非需要进行低功耗设计，否则通常我们会直接将其设置为最大频率。注意如果配置的参数超出了最高允许范围，那么选项框将会显示红色警告。

接着配置仿真调试接口。在这里打开 SYS 选项，并进行以下设置：

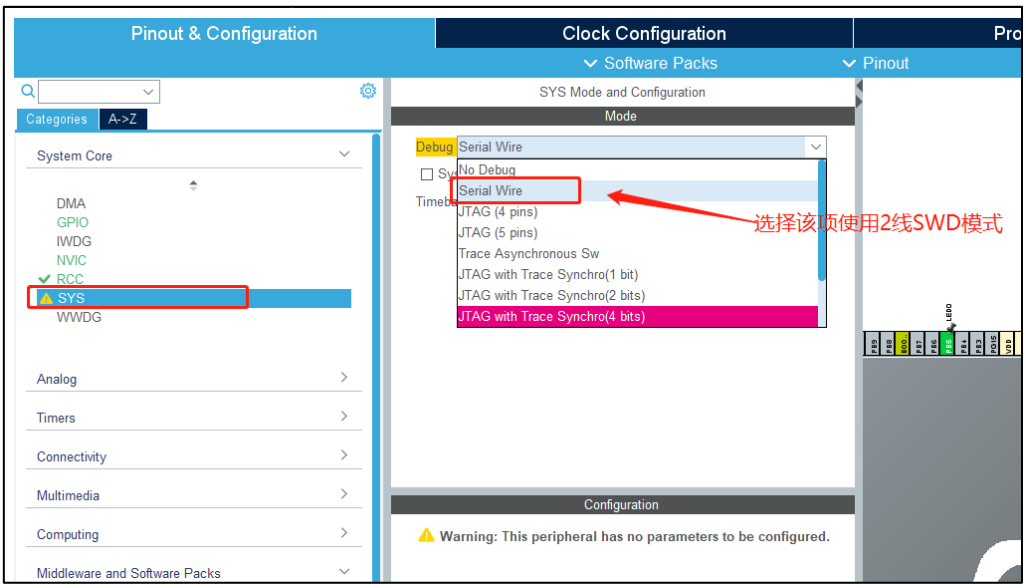


图 10.2.3.4 调试接口设置

SW 调试端口采用同步串行协议，SWCLK 引脚为从主机到从机的时钟信号，SWDIO 为双向数据信号。考虑成本效益，当采用引脚较少 SW 调试端口的方案显然具有更高的性价比。

接下来配置工程的格式。要注意勾选 Generate peripheral initialization as a pair of ".c/.h' files per peripheral 选项，这样可以独立生成对应外设的初始化.h 和.c 文件（这么做也是为了不让外设的初始化代码生成在 main.c 文件中，方便查看，也避免 main.c 文件变得臃肿）：

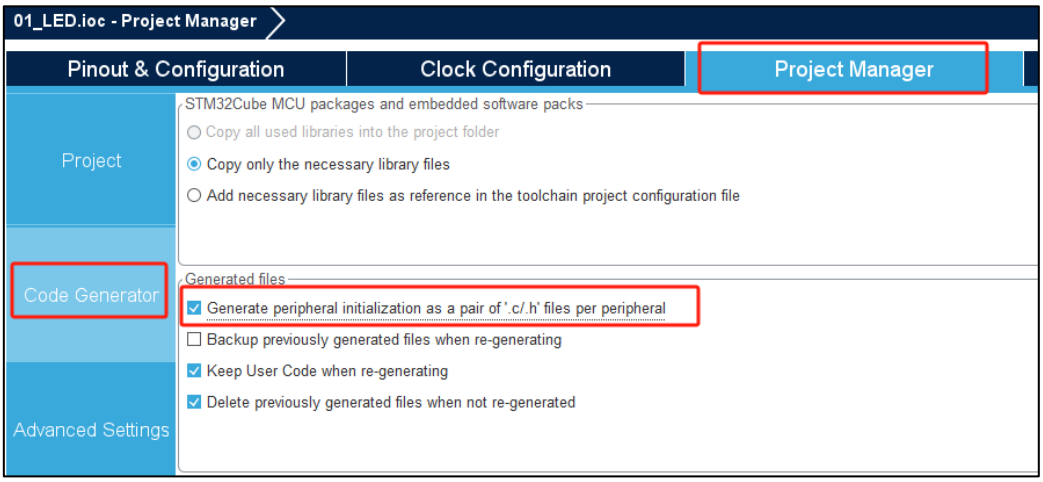


图 10.2.3.5 配置生成外设独立的初始化文件

10.2.4 生成工程

修改完之后，按下键盘的“Ctrl+S”组合键保存 01_LED.ioc 文件，系统开始生成初始化代码。生成工程文件目录如下图所示：

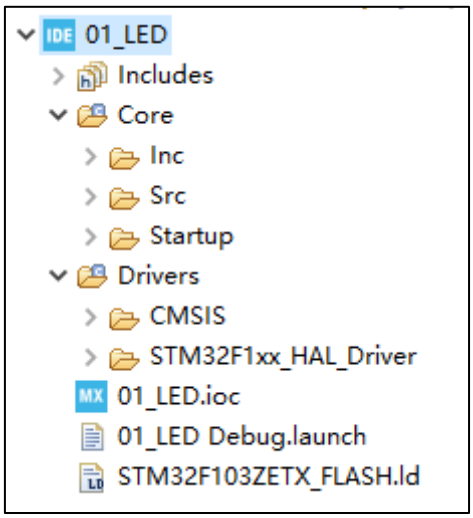


图 10.2.4.1 生成的 01_LED 工程

在 01_LED 工程的 Drivers 文件夹下新建 BSP 文件夹、SYSTEM 文件夹和 Readme 文件夹；其中 BSP 文件夹下在创建一个 LED 文件夹，LED 文件夹下创建两个文件，分别是 led.c 文件和 led.h 文件；SYSTEM 文件夹中则是正点原子自己写的一些关于系统操作函数以及 us 和 ms 延时函数的驱动文件（详情可查看第七章）；Readme 文件夹则是用于存放工程说明文件的，创建好后的工程文件目录如下所示：

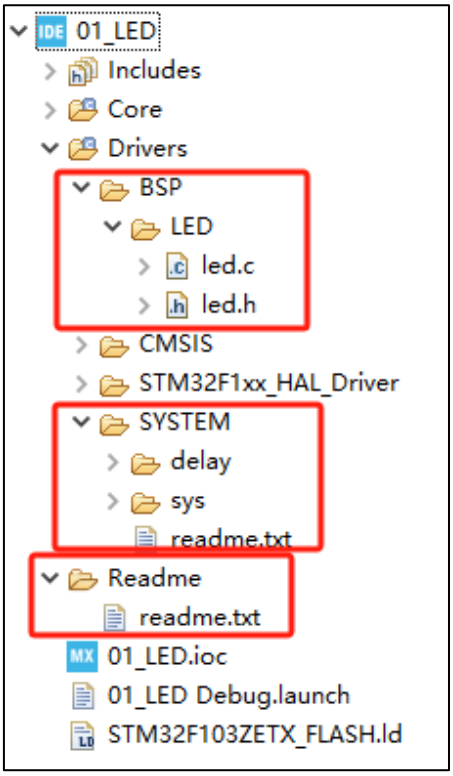


图 10.2.4.2 创建好所需文件夹后的 LED 工程

为了保持我们所提供的 STM32CubeIDE 的例程格式统一，我们还需在 01_LED 工程下新建 ATK_Middlewares 文件夹（本例中并未使用到），在后续工程中会用于存放正点原子自己写的一些中间层组件，包括：USMART、MALLOC、TEXT、T9INPUT、PICTURE 等驱动文件。如下图所示：

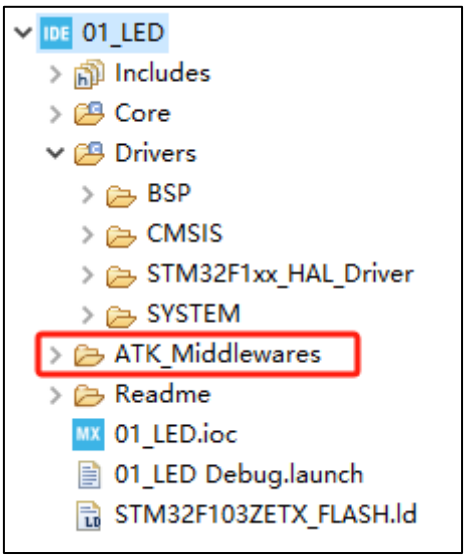


图 10.2.4.3 新建 ATK_Middlewares 文件夹

添加完以后，记得在工程中将源文件和工程关联（一级目录下新建的文件才需此操作，如 ATK_Middlewares），如下图所示：

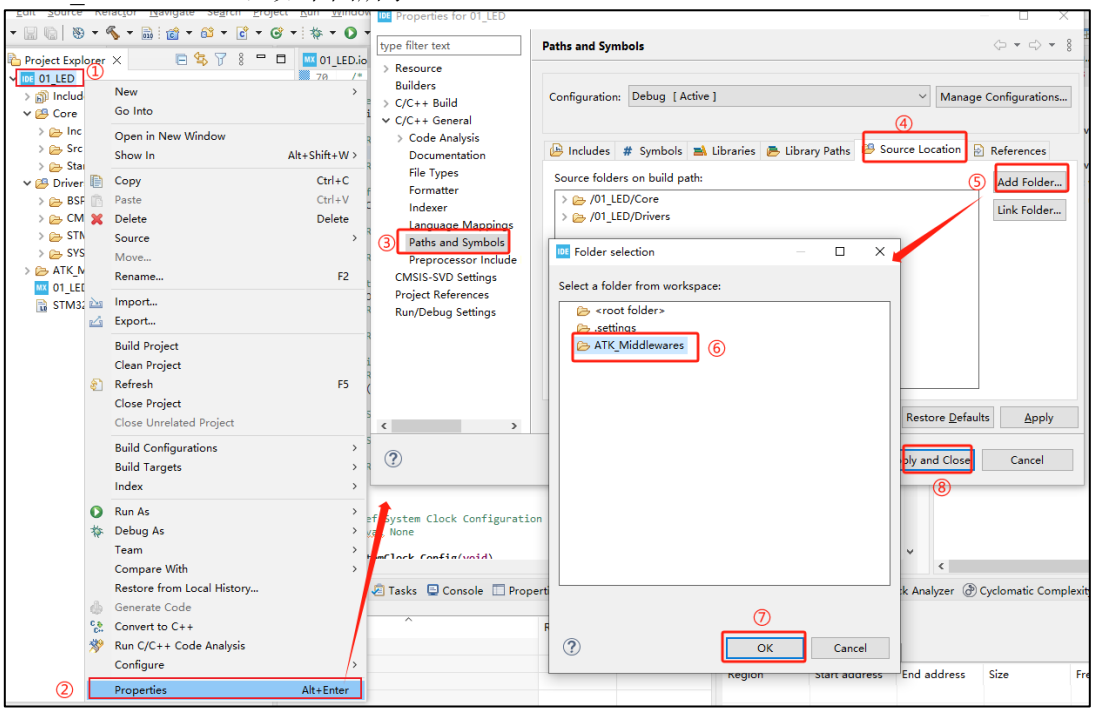


图 10.2.4.4 将源文件和工程关联

10.2.5 添加用户驱动代码

1. 初始化代码分析

前面的操作已经生成了 GPIO 的初始化代码，我们打开 01_LED 工程下的 Core→ Src→ gpio.c 文件，gpio.c 文件内容（以下代码已经把多余的注释部分删除），初始化 GPIO 外设的代码主要分为两个部分：

①先开启要操作的外设时钟。前面我们在时钟系统章节有分析过，复位以后，所有外设的时钟均关闭，要操作外设的话，必须先使能对应外设的时钟。

②根据用户的配置来初始化外设（即将这些配置写入对应的寄存器），完成外设配置。

```
1)  #include "gpio.h"
```

```
2)  void MX_GPIO_Init(void)
3)  {
4)  GPIO_InitTypeDef GPIO_InitStruct = {0};

5)  /* 使能 GPIO 时钟 */
6)  __HAL_RCC_GPIOE_CLK_ENABLE();
7)  __HAL_RCC_GPIOC_CLK_ENABLE();
8)  __HAL_RCC_GPIOA_CLK_ENABLE();
9)  __HAL_RCC_GPIOB_CLK_ENABLE();

10) /* 设置初始 IO 的状态 */
11) HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);

12) /* 设置初始 IO 的状态  */
13) HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET);

14) /* 配置 PB5 */
15) GPIO_InitStruct.Pin = LED1_Pin;
16) GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
17) GPIO_InitStruct.Pull = GPIO_PULLUP;
18) GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
19) HAL_GPIO_Init(LED1_GPIO_Port, &GPIO_InitStruct);

20) /* 配置 PE5 */
21) GPIO_InitStruct.Pin = LED0_Pin;
22) GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
23) GPIO_InitStruct.Pull = GPIO_PULLUP;
24) GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
25) HAL_GPIO_Init(LED0_GPIO_Port, &GPIO_InitStruct);

26) }
```

gpio.c 文件的内容主要就是实现了 LED0 和 LED1 对应的 IO 口的初始化。

第 4 行，定义一个 GPIO_InitTypeDef 结构体类型变量 GPIO_InitStruct，初始值设置为 0。选中 GPIO_InitTypeDef 然后按下 F2 键可以看到此结构体的声明，主要是对 GPIO 引脚的工作模式、上/下拉、速度等级以及引脚复用关系的配置。

第 6~9 行是使能 LED 以及外部晶振引脚相关的 GPIO 外设时钟

第 10-13 行是设置 IO 上电时的初始状态为高电平。

第 14-18 行表示要控制的 GPIOB 端口的某个 IO 口（LED0_Pin）引脚的输出类型为推挽输出，引脚为上拉模式，引脚速率为高速。

第 19 行调用库函数 HAL_GPIO_Init，根据上面对 GPIO_InitStructure 结构体成员赋的值来 再次初始化 LED0_GPIO_Port。

选中 LED0_GPIO_Port，按下 Ctrl 键并点击鼠标左键打开 LED0_GPIO_Port 声明，在 main.h 中看到声明，LED0_Pin 就是 GPIO_PIN_5，LED0_GPIO_Port 就是 GPIOB。

```
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

#include "stm32f1xx_hal.h"
void Error_Handler(void);

#define LED1_Pin GPIO_PIN_5
#define LED1_GPIO_Port GPIOE
#define LED0_Pin GPIO_PIN_5
#define LED0_GPIO_Port GPIOB

#ifdef __cplusplus
}
#endif
```

```
#endif /* __MAIN_H */
```

可以看到我们在 CubeMX 的插件配置的用户标签定义，其实就是使用宏定义来指向用户自定义的 IO，使生成的代码更具有可读性！

2. 逻辑代码实现

在前面，我们使用 STM32CubeIDE 生成了基于 HAL 库的 GPIO 初始化代码，还没有控制 LED0 和 LED1 点亮和熄灭的逻辑代码，接下来我们在这个工程的基础上去手动完成这部分内容。

在 01_LED 工程中的 Drivers 文件夹下新建 BSP 文件夹，此文件夹下专门存放自己的逻辑代码，然后在 BSP 下新建 LED 文件夹用于存放 led 的驱动文件，然后再 LED 文件夹中分别新建 led.h 和 led.c 文件。

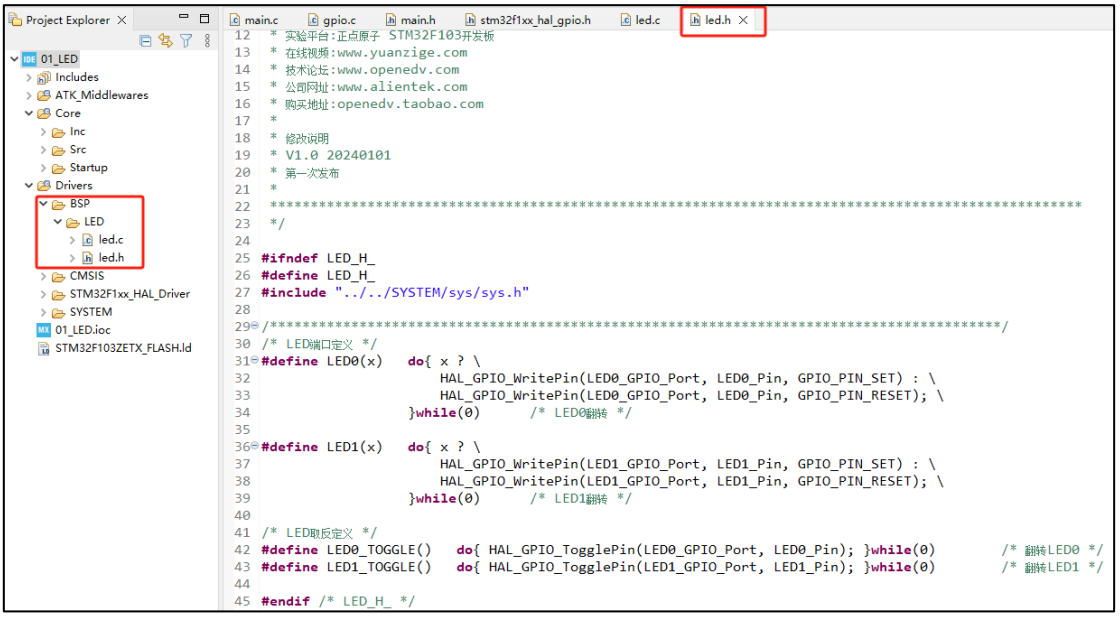


图 10.2.5.1 LED 文件夹下关于 led 的驱动文件

led.h 文件代码如下：

```
#ifndef LED_H_
#define LED_H_
#include "../../SYSTEM/sys/sys.h"

/*****************************************************************************/
/* LED 端口定义 */
#define LED0(x)    do{ x ? \
        HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET) : \
        HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET); \
                }while(0)       /* LED0 翻转 */

#define LED1(x)    do{ x ? \
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET) : \
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET); \
                }while(0)       /* LED1 翻转 */

/* LED 取反定义 */
#define LED0_TOGGLE()    do{ HAL_GPIO_TogglePin(LED0_GPIO_Port,
LED0_Pin); }while(0)         /* 翻转 LED0 */
#define LED1_TOGGLE()    do{ HAL_GPIO_TogglePin(LED1_GPIO_Port,
LED1_Pin); }while(0)         /* 翻转 LED1 */

#endif /* LED_H_ */
```

由上可知当 x 为 1 即大于 0 的时候，LED0(1)为 HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET)，表示设置 LED0 的端口位为 1，端口位 LED0_Pin 输出高电平，LED0 点熄灭。

当 x 为 0 的时候，LED0(0)为 HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET)，表示设置 LED0 的端口位为 0，端口位 LED0_Pin 输出低电平，LED 点亮。

另外调用 HAL_GPIO_TogglePin 函数将引脚的状态取反。

HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_SET)表示设置 GPIOB 的第 LED0_Pin 位所对应的管脚为高电平，那么 LED0 关闭。

HAL_GPIO_WritePin(LED0_GPIO_Port, LED0_Pin, GPIO_PIN_RESET)表示设置 GPIOB 的第 LED0_Pin 位所对应的管脚为低电平，那么 LED0 被点亮。

led.c 文件我们并未编写任何内容，仅包含了 led.h 头文件，是由于 LED 的 IO 初始化已经在生成的文件 gpio.c 中完成初始化了。所以为了保持文件的完整性，我们保留 led.c，后续如果要实现其他关于 led 的控制函数可以在 led.c 中添加即可！

3. main 函数调用

main.c 文件的实现：

```
1)  #include "../../BSP/LED/led.h"
2)  #include "../../SYSTEM/delay/delay.h"
3)  int main(void)
4)  {

5)   HAL_Init();
6)   /* 初始化系统时钟 */
7)   SystemClock_Config();

8)   /* 延时初始化（正点原子自编的延时初始化函数） */
9)   delay_init(72);

10)  /* GPIO 初始化 */
11)  MX_GPIO_Init();

12)  while (1)
13)  {
14)    LED0(0);          /* 打开 LED0 */
15)    LED1(1);          /* 关闭 LED1 */
16)    delay_ms(500);    /* 延时 500ms */
17)    LED0(1);          /* 关闭 LED0 */
18)    LED1(0);          /* 打开 LED1 */
19)    delay_ms(500);    /* 延时 500ms */
20)  }
21) }
```

首先在 main.c 中首先要引用对应的头文件，分别是 led.h 和 delay.h，led.h 中使用宏定义的方式控制 LED0 和 LED1 显著提高代码的可读性；delay.h 则是我们正点原子自己编写的延时函数，内部实现了精准的毫秒延时函数和微妙延时函数，可起到辅助开发的作用（HAL 库仅有毫秒延时函数，并没有微妙延时函数）。

第 5 行，初始化 HAL 库，使用 HAL 库函数前，必须在 main 函数中先对 HAL 库进行初始化。

第 7 行，初始化系统时钟。

第 9 行，延时初始化，该函数由正点原子自己编写实现的，大家直接移植即可。

第 11 行，初始化已配置的外围设备，这里是初始化 GPIO。

第 14~19 行是 LED0 和 LED1 交替闪烁的初始化代码，调用 delay_ms (500)函数实现每隔 500ms 交替闪烁一次。

10.3 下载验证

保存修改后点击工具栏的小锤子进行编译，编译无报错后，按照第 4.1.4 小节连接好开发板和 ST-Link，然后下载程序进去板子，下载完之后，运行结果如图 10.3.1 所示，可以看到 LED 灯的 LED0 和 LED1 交替闪烁。

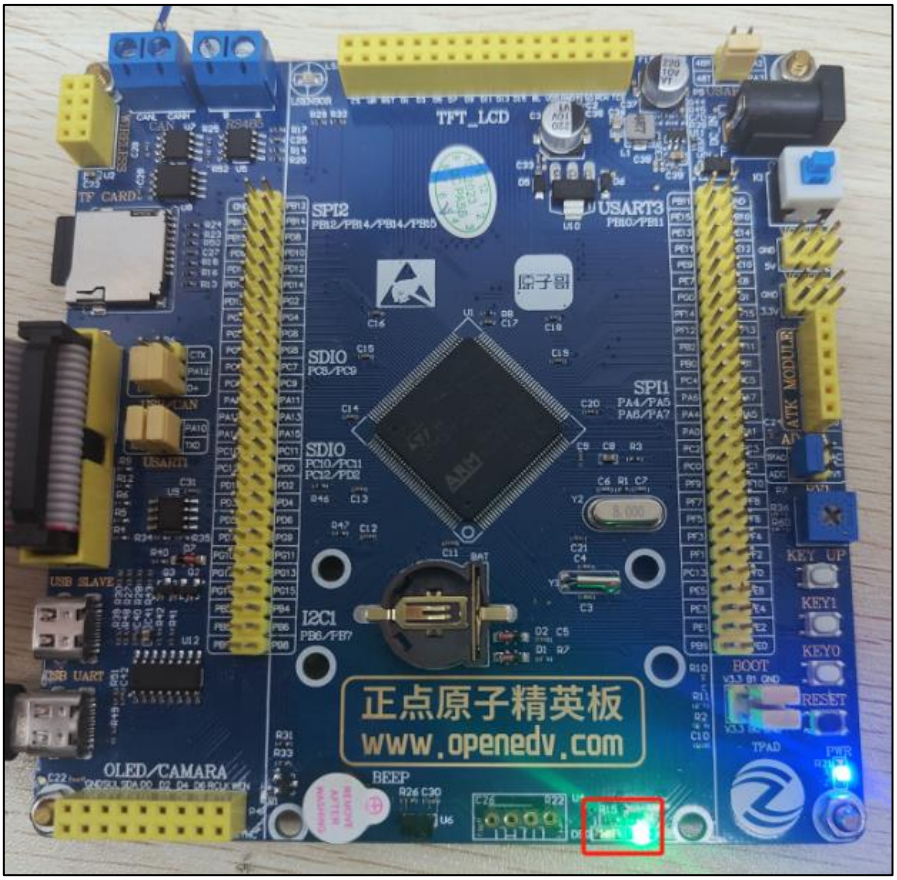


图 10.3.1 实验现象 LED0 和 LED1 交替闪烁

至此，我们的跑马灯实验的学习就结束了，本章介绍了 STM32F103 的 IO 口的使用及注意事项，是后面学习的基础，希望大家好好理解。

第十一章 蜂鸣器实验

上一章，我们介绍了 STM32 的 IO 口作为输出的使用。本章，我们将通过另外一个例子继续巩固 IO 口作为输出使用的操作方法，不同的是本章讲的不是用 IO 口直接驱动器件，而是通过三极管间接驱动。我们将利用一个 IO 口来控制板载的有源蜂鸣器。

本章将分为如下几个小节：

11.1 硬件设计
11.2 程序设计
11.3 下载验证

11.1 硬件设计

1. 例程功能

蜂鸣器每隔 500ms 响或者停一次。LED0 每隔 500ms 亮或者灭一次。LED0 亮的时候蜂鸣器不响，而 LED0 熄灭的时候，蜂鸣器响。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）蜂鸣器
BEEP - PB8

3. 原理图

蜂鸣器在硬件上是直接连接好了的，不需要经过任何设置，直接编写代码就可以了。蜂鸣器的驱动信号连接在 STM32F1 的 PB8 上。如图 11.1.1 所示：

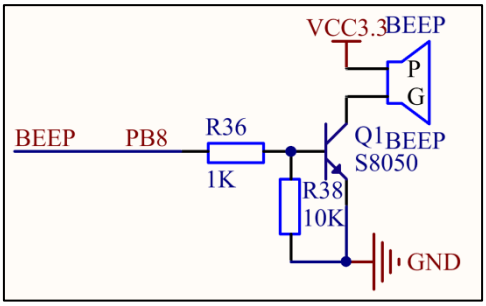


图 11.1.1 蜂鸣器与 STM32F1 连接原理图

我们用一个 NPN 三极管（S8050）来驱动蜂鸣器，驱动信号通过 R36 和 R38 间的电压获得，芯片上电时默认电平为低电平，故上电时蜂鸣器不会直接响起。当 PB8 输出高电平的时候，蜂鸣器将发声，当 PB8 输出低电平的时候，蜂鸣器停止发声。

11.2 程序设计

本章节我们通过 HAL 库的 API 函数来驱动 LED 和蜂鸣器，实现蜂鸣器、LED0 和蜂鸣器以 500ms 间隔交替开启和关闭。其中，我们会用到 HAL 库中的 HAL_GPIO_WritePin 和 HAL_GPIO_TogglePin 函数。

11.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 02_BEEP。

11.2.2 创建和配置工程

在跑马灯的基础上复制一份，如下图所示

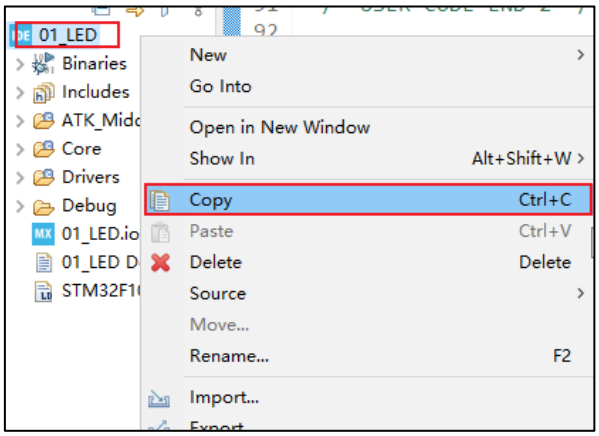


图 11.2.2.1 Copy 一份工程（快捷键“Ctrl+C”）

之后单击右侧空白处，使用快捷键“Ctrl+V”粘贴工程，并将工程命名为：02_BEEP 如下图所示：

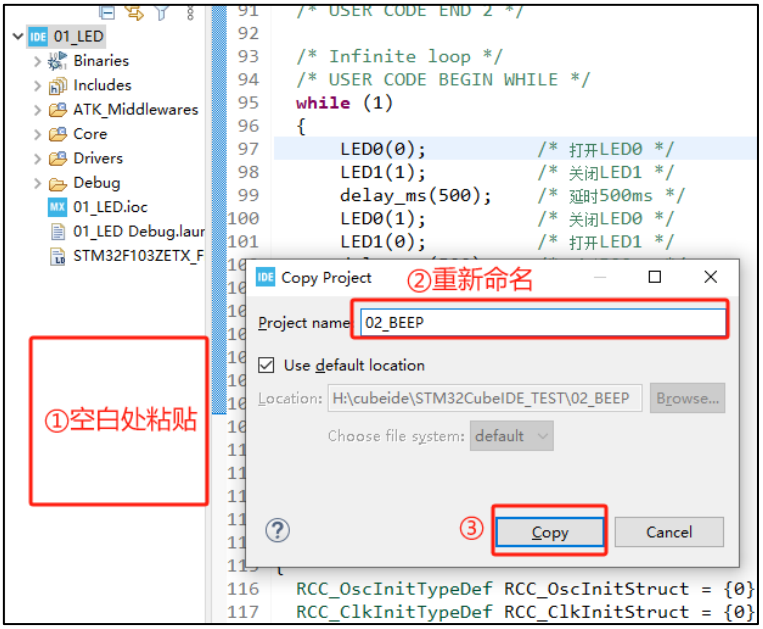


图 11.2.2.2 粘贴一份工程

接着我们可以看到右侧会出现 02_BEEP 的工程文件，但是可以看到它的.ioc 文件名与工程文件名不一致，这样双击.ioc 文件是无法正常打开的，需进行重命名，如下图所示：

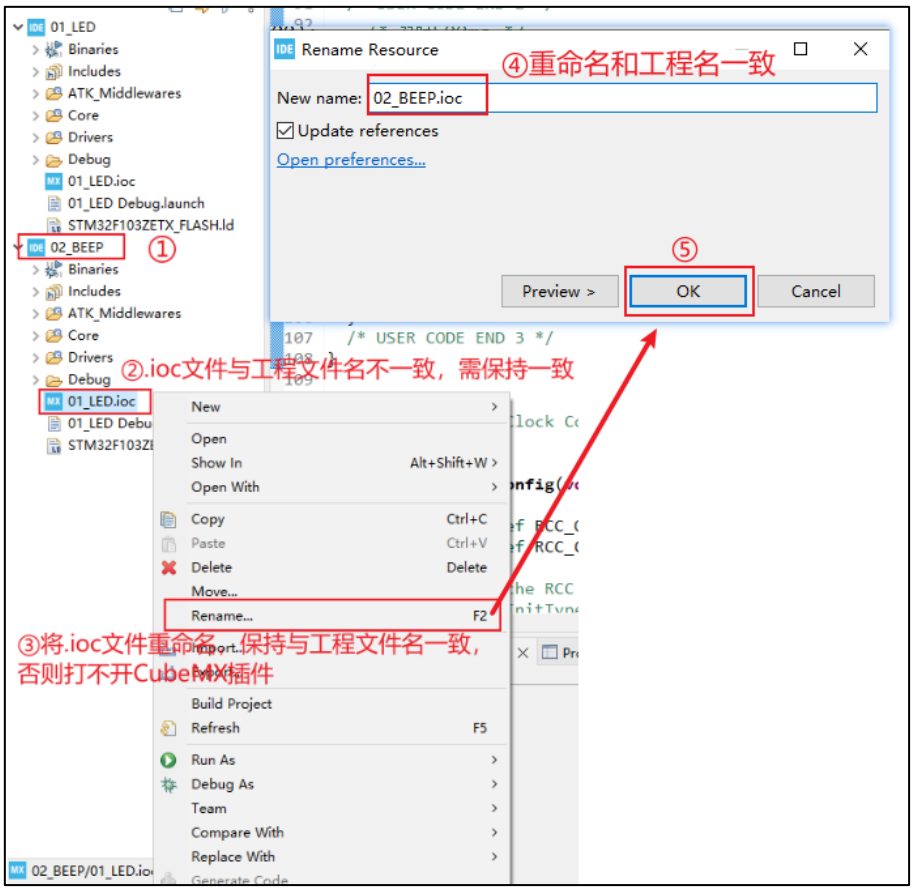


图 11.2.2.3 重命名.ioc 文件

然后我们就可以得到一个 02_BEEP 工程，工程目录如下图所示：

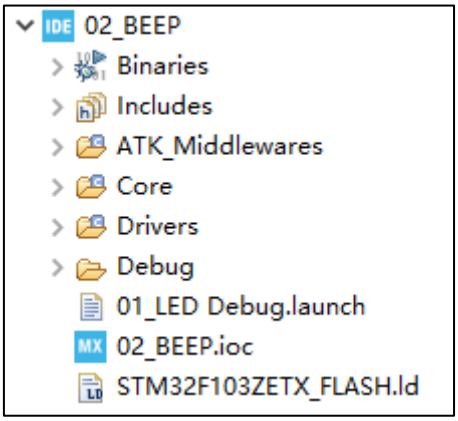


图 11.2.2.4 02_BEEP 工程文件目录

该工程继承了跑马灯的所有配置。我们将在此基础上添加与 BEEP 相关的功能，从而避免重复配置 LED 部分和系统时钟部分等步骤，可大大提高效率。

（1）GPIO 配置

接着双击 02_BEEP.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PB8，如下图所示：

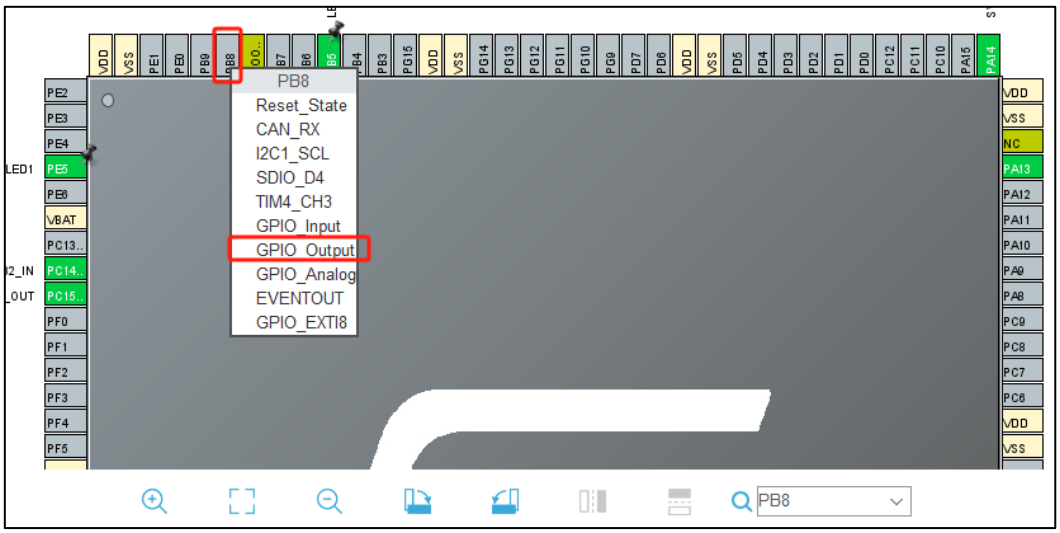


图 11.2.2.5 配置 PB8 引脚输出功能

接下来配置 GPIO 的工作模式，如下图所示：

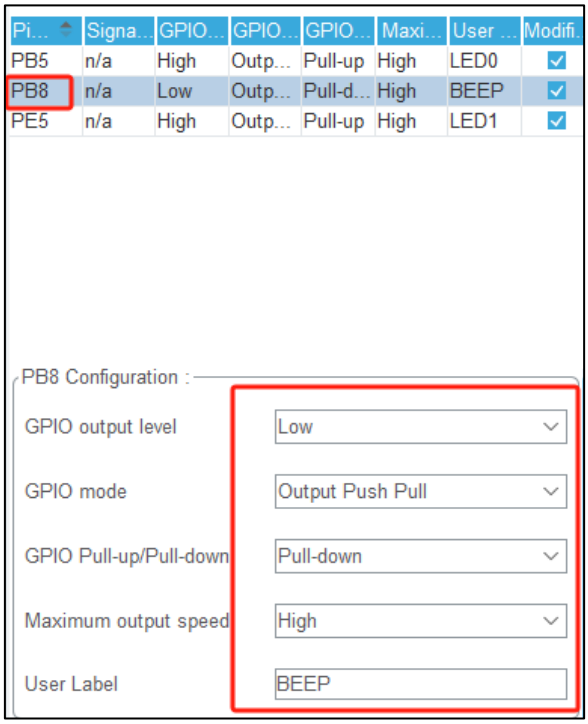


图 11.2.2.6 配置 PB8 的工作模式

11.2.3 时钟和工程配置

与跑马灯一致，此工程已继承其所有配置，所以无需另外配置，若想看详细配置步骤，可查看 10.2.3 小节。

11.2.4 生成工程

配置好之后，按下键盘的“Ctrl+S”组合键保存 BEEP.ioc 文件，系统开始生成初始化代码。

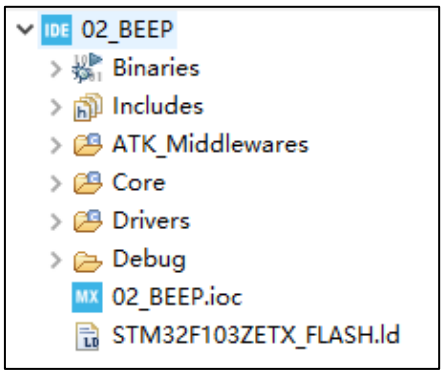


图 11.2.4.1 生成 BEEP 工程（相较跑马灯新增 PB8 的初始化部分）

11.2.5 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 BEEP 文件夹，然后再 BEEP 文件夹下新建两个文件，分别是 beep.c 和 beep.h，如下图所示：

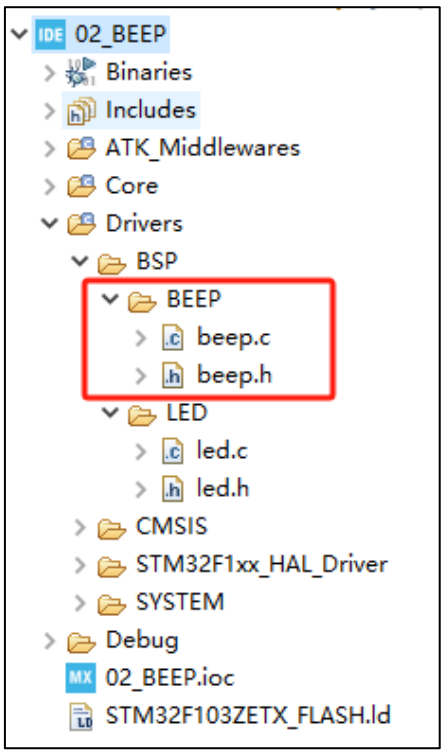


图 11.2.5.1 新建 BEEP 文件夹

(1)beep.h 文件

在 beep.h 文件中添加如下代码：

```
#ifndef BEEP_BEEP_H_
#define BEEP_BEEP_H_
#include "../../SYSTEM/sys/sys.h"

/*****************************************************************************/

/* 蜂鸣器控制 */
#define BEEP(x)         do{ x ? \
                HAL_GPIO_WritePin(BEEP_GPIO_Port, BEEP_Pin, GPIO_PIN_SET) : \
                HAL_GPIO_WritePin(BEEP_GPIO_Port, BEEP_Pin, GPIO_PIN_RESET); \
                }while(0)
```

```
/* BEEP 状态翻转 */
#define BEEP_TOGGLE()   do{
    HAL_GPIO_TogglePin(BEEP_GPIO_Port, BEEP_Pin); }while(0) /* BEEP = !BEEP */

#endif /* BEEP_BEEP_H_ */
```

beep.h 文件中调用 HAL_GPIO_WritePin 函数来对 GPIO 端口位写值，调用 HAL_GPIO_TogglePin 函数来对 GPIO 端口位取反。

其中宏 BEEP(x)，当 x 为 1 的时，蜂鸣器响；当 x 为 0 的时，蜂鸣器不响。

宏 BEEP_Toggle()，则是对 GPIO 端口位取反，如果蜂鸣器开启，执行这段语句后蜂鸣器关闭；如果此时蜂鸣器关闭，执行这段语句后打开蜂鸣器。

beep.c 文件我们并未编写任何内容，仅包含了 beep.h 头文件，是由于 BEEP 的 IO 初始化已经在生成的文件 gpio.c 中完成初始化了。所以为了保持文件的完整性，我们保留 beep.c，后续如果要实现其他关于 beep 的控制函数可以在 beep.c 中添加即可！

(2)main.c 文件

接着我们来看下 main.c 文件代码：

```
#include "../../BSP/LED/led.h"
#include "../../BSP/BEEP/beep.h"
#include "../../SYSTEM/delay/delay.h"
int main(void)
{

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  while (1)
  {
      LED0(0);          /* 打开 LED0 */
      BEEP(0);          /* 关闭蜂鸣器 */
      delay_ms(500);    /* 延时 500ms */
      LED0(1);          /* 关闭 LED0 */
      BEEP(1);          /* 打开蜂鸣器 */
      delay_ms(500);    /* 延时 500ms */
  }
}
```

首先是头文件的包含，这部分在 10.2.5 小节有分析过，这里我们不过多赘述。在 while 循环中，蜂鸣器以 0.5 秒的间隔发声，当蜂鸣器响的时候，LED0 灭，当蜂鸣器不响的时候，LED0 亮。

11.3 下载验证

编译下载完之后，可以看到 LED0 亮的时候蜂鸣器不叫，而 LED0 熄灭的时候，蜂鸣器叫。间隔为 0.5 秒左右，符合预期设计。

至此，本章的学习就结束了。通过本章，我们进一步学习 IO 作为输出的使用方法，同时巩固了前面知识的学习。希望大家在开发板上实际验证一下，从而加深印象

第十二章 按键输入实验

上一章，我们介绍了 STM32 的 IO 口作为输出的使用。本章，我们向大家介绍 IO 口作为输入使用的操作方法，我们将利用板载的 3 个按键来控制 LED 灯亮和灭以及和蜂鸣器的开和关。

本章将分为如下几个小节：

12.1 硬件设计
12.2 程序设计
12.3 下载验证

12.1 硬件设计

1. 例程功能

通过开发板上的三个独立按键控制 LED 灯和蜂鸣器： KEY_UP 控制蜂鸣器翻转，KEY1 控制 LED1 翻转，KEY0 控制 LED0/LED1 同时翻转。

2. 硬件资源

1）LED 灯
LED0 - PB5
LED1 - PE5
2）蜂鸣器
BEEP - PB8
2）独立按键
KEY0 - PE4
KEY1 - PE3
KEY_UP - PA0

3. 原理图

独立按键硬件部分的原理图，如图 12.1.1 所示：

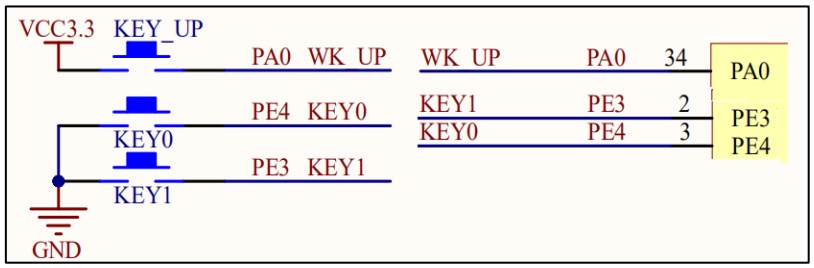


图 12.1.1 独立按键与 STM32F1 连接原理图

这里需要注意的是：KEY0 和 KEY1 是低电平有效的，而 KEY_UP 则是高电平有效的，并且外部都没有上下拉电阻，所以需要在 STM32F103 内部设置上下拉，来确定设置空闲电平状态。

12.2 程序设计

本章节我们通过 HAL 库的 HAL_GPIO_ReadPin 函数（实际上是操作 IDR 寄存器）来读取按键对应 GPIO 引脚电平状态，从而判断按键是否有按下。

12.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\03_KEY。

12.2.2 GPIO 功能引脚配置

同样的在 STM32CubeIDE 中复制上一章节的工程，然后将工程文件名和.ioc 文件名均修改为 03_KEY，如下图所示：

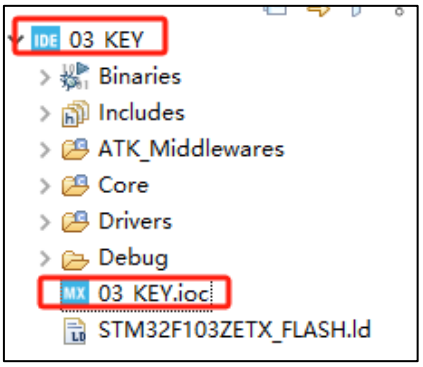


图 12.2.2.1 KEY 工程

接着双击 03_KEY.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PE4、PE3、PA0，要注意，需将按键这几个 IO 都设置成 GPIO_Intput 模式，如下图所示：

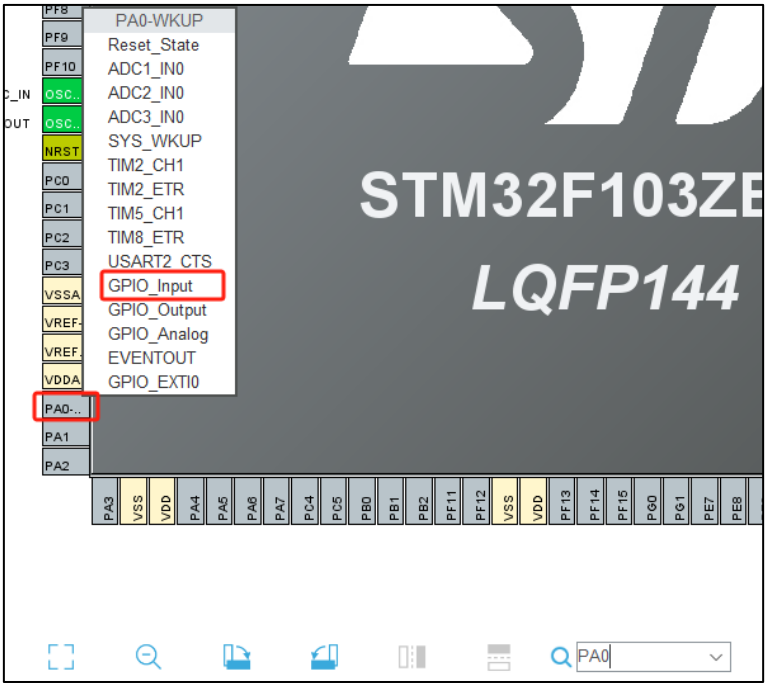


图 12.2.2.2 按键 IO 配置输入模式

接下来配置按键的 GPIO 的工作模式，需要注意：除了按键 WKUP 配置为下拉以外，其它按键引脚均配置为上拉了。引脚配置结果如下图所示：

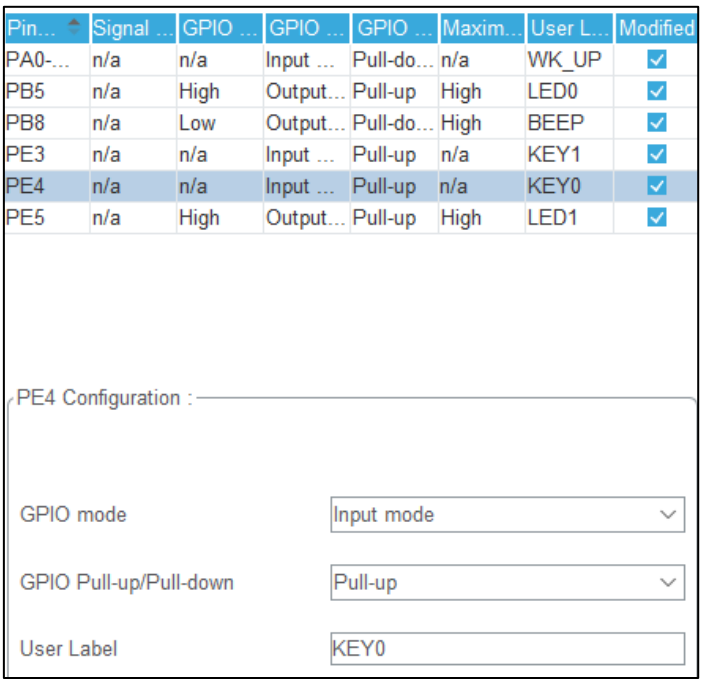


图 12.2.2.3 按键引脚配置

12.2.3 时钟和工程配置

该步骤的配置与前面章节相同，因为本工程已经继承了前面章节的所有配置，因此无需进行额外配置。如果需要查看详细的配置步骤，可以查看 10.2.3 小节。

需注意大部分实验在此步骤中的配置是相同的，因此在后续实验中将不再赘述此步骤的配置。除非有做修改才会提及此步骤！

12.2.4 生成工程

配置好之后，按下键盘的“Ctrl+S”组合键保存 KEY.ioc 文件，系统开始生成初始化代码。

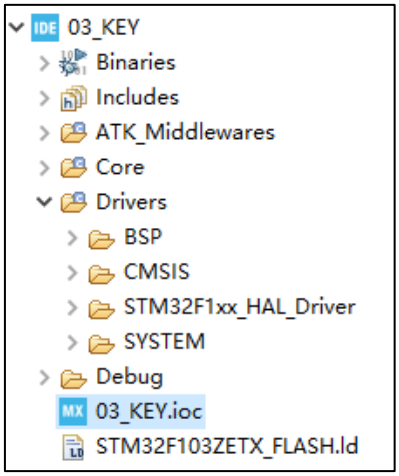


图 12.2.4.1 生成代码

12.2.5 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 KEY 文件夹，然后再 KEY 文件夹下新建两个文件，分别是 key.c 和 key.h，如下图所示：

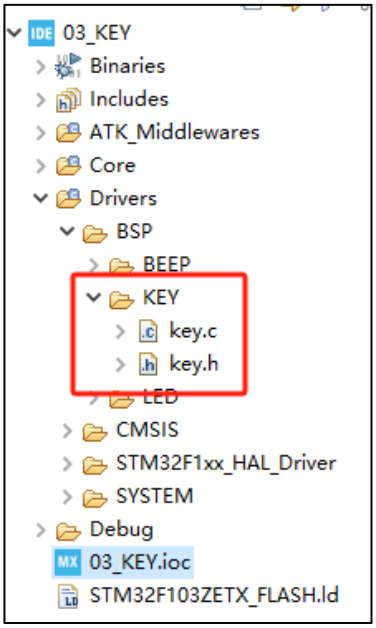


图 12.2.5.1 新建 KEY 文件夹

接着我们在 key.h、key.c 和 main.c 文件中添加如下代码。

（1）key.h 文件

key.h 文件代码如下：

```
1)  #ifndef KEY_KEY_H_
2)  #define KEY_KEY_H_
3)  #include "../../SYSTEM/sys/sys.h"
4)  /* 读取 KEY0 引脚 */
5)  #define KEY0        HAL_GPIO_ReadPin(KEY0_GPIO_Port, KEY0_Pin)

6)  /* 读取 KEY1 引脚 */
7)  #define KEY1        HAL_GPIO_ReadPin(KEY1_GPIO_Port, KEY1_Pin)

8)  /* 读取 WKUP 引脚 */
9)  #define WK_UP       HAL_GPIO_ReadPin(WK_UP_GPIO_Port, WK_UP_Pin)

10) #define KEY0_PRES    1              /* KEY0 按下 */
11) #define KEY1_PRES    2              /* KEY1 按下 */
12) #define WKUP_PRES    3              /* KEY_UP 按下(即 WK_UP) */

13) uint8_t key_scan(uint8_t mode);     /* 按键扫描函数 */

14) #endif /* KEY_KEY_H_ */
```

key.h 头文件很简单：

- 第 4~9 行，通过 HAL_GPIO_ReadPin 函数读取 GPIO 的电平值，也就是读取按键电平值。
- 第 10~12 行，定义 3 个宏，分别对应 3 个按键按下。
- 第 13 行，声明按键扫描函数 key_scan。

（2）key.c 文件

```
#ifndef KEY_KEY_C_
#define KEY_KEY_C_
#include "key.h"
#include "../../SYSTEM/delay/delay.h"

/**
 #ifndef KEY_KEY_C_
```

```
#define KEY_KEY_C_
#include "key.h"
#include "../../SYSTEM/delay/delay.h"

/**
 * @brief       按键扫描函数
 * @note        该函数有响应优先级(同时按下多个按键): WK_UP > KEY1 > KEY0!!
 * @param       mode:0 / 1, 具体含义如下:
 *   @arg       0,  不支持连续按(当按键按下不放时, 只有第一次调用会返回键值,
 *                  必须松开以后, 再次按下才会返回其他键值)
 *   @arg       1,  支持连续按(当按键按下不放时, 每次调用该函数都会返回键值)
 * @retval      键值, 定义如下:
 *              KEY0_PRES, 1, KEY0 按下
 *              KEY1_PRES, 2, KEY1 按下
 *              WKUP_PRES, 3, WKUP 按下
 */
uint8_t key_scan(uint8_t mode)
{
    static uint8_t key_up = 1;  /* 按键按松开标志 */
    uint8_t keyval = 0;

    if (mode) key_up = 1;       /* 支持连按 */
    /* 按键松开标志为1, 且有任意一个按键按下了 */
    if (key_up && (KEY0 == 0 || KEY1 == 0 || WK_UP == 1))
    {
        delay_ms(10);           /* 去抖动 */
        key_up = 0;

        if (KEY0 == 0)  keyval = KEY0_PRES;

        if (KEY1 == 0)  keyval = KEY1_PRES;

        if (WK_UP == 1) keyval = WKUP_PRES;
    }
    else if (KEY0 == 1 && KEY1 == 1 && WK_UP == 0) /*没有任何按键按下标记按键松开*/
    {
        key_up = 1;
    }

    return keyval;              /* 返回键值 */
}


#endif /* KEY_KEY_C_ */
```

key_scan 函数用于扫描这 3 个 IO 口是否有按键按下。key_scan 函数，支持两种扫描方式，通过 mode 参数来设置。

当 mode 为 0 的时候，key_scan 函数将不支持连续按，扫描某个按键，该按键按下之后必须要松开，才能第二次触发，否则不会再响应这个按键，这样的好处就是可以防止按一次多次触发，而坏处就是在需要长按的时候比较不合适。

当 mode 为 1 的时候，key_scan 函数将支持连续按，如果某个按键一直按下，则会一直返回这个按键的键值，这样可以方便的实现长按检测。

有了 mode 这个参数，大家就可以根据自己的需要，选择不同的方式。这里要提醒大家，因为该函数里面有 static 变量，所以该函数不是一个可重入函数，在有 OS 的情况下，这个大家要留意下。可以看到该函数的消抖延时是 10ms。同时还有一点要注意的是，该函数的按键扫描是有优先级的，最优先的是 KEY_UP，第二优先的是 KEY1，最后是按键 KEY0。该函数有返回值，如果有按键按下，则返回非 0 值，如果没有或者按键不正确，则返回 0。

（3）main.c 文件

main.c 文件函数如下：

```
#include "../../BSP/LED/led.h"
#include "../../BSP/BEEP/beep.h"
#include "../../BSP/KEY/key.h"
#include "../../SYSTEM/delay/delay.h"
int main(void)
{
  uint8_t key;
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  while (1)
  {
      key = key_scan(0);                    /* 得到键值 */
      if (key)
      {
          switch (key)
          {
              case WKUP_PRES:               /* 控制蜂鸣器 */
                  BEEP_TOGGLE();            /* BEEP 状态取反 */
                  break;

              case KEY1_PRES:               /* 控制 LED1(GREEN)翻转 */
                  LED1_TOGGLE();            /* LED1 状态取反 */
                  break;

              case KEY0_PRES:               /* 同时控制 LED0, LED1 翻转 */
                  LED0_TOGGLE();            /* LED0 状态取反 */
                  LED1_TOGGLE();            /* LED1 状态取反 */
                  break;
          }
      }
      else
      {
          delay_ms(10);
      }
  }
}
```

可以看到应用代码中，首先是调用系统级别的初始化：初始化 HAL 库、系统时钟和延时函数。接下来，GPIO 的初始化。最后在无限循环里面扫描获取键值，接着用键值判断哪个按键按下，如果是 KEY0 按下，LED0 和 LED1 同时翻转；KEY1 按下，LED1 翻转，WKUP 按下，蜂鸣器翻转。

12.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 处于熄灭状态，若此时按下 KEY0 按键，则能够看到 LED0 和 LED1 的亮灭状态发生了一次翻转，同样的，当我们按下 KEY1，LED1 状态翻转，若此时按下 WKUP 按键，蜂鸣器状态发生了一次翻转，与预期的实验现象效果相符。

第十三章 外部中断实验

在前面几章的学习中，我们掌握了 STM32 的 IO 口最基本的操作方法，在上一章节的实验中，我们通过按键扫描的方式来检测按键的状态，本章节，我们将通过中断的方式来检测按键的状态。通过本章节的学习，我们可以掌握 STM32 的 IO 口作为外部中断输入来使用的方法。

本章将分为如下几个小节：

13.1、硬件设计；

13.2、程序设计；

13.3、下载验证；

13.1 硬件设计

1. 例程功能

通过外部中断的方式让开发板上的三个独立按键控制 LED 灯：KEY_UP 控制 LED0 翻转，KEY1 控制 LED1 翻转，KEY0 控制 LED0 和 LED1 翻转。

2. 硬件资源

1）LED 灯

LED0 - PB5

LED1 - PE5

2）独立按键

KEY0 - PE4

KEY1 - PE3

KEY_UP - PA0

3. 原理图

独立按键硬件部分的原理图，如图 13.1.1 所示：

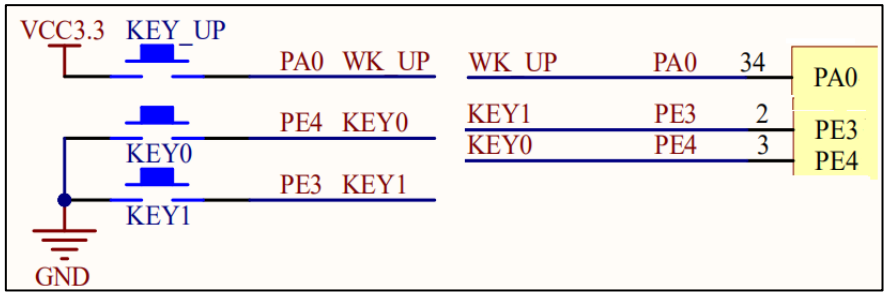


图 13.1.1 独立按键与 STM32F1 连接原理图

这里需要注意的是：KEY0 和 KEY1 设计为采样到按键另一端的低电平为有效，而 KEY_UP 则需要采样到高电平才为按键有效，并且外部都没有上下拉电阻，所以需要在 STM32F103 内部设置上下拉以设置空闲电平。

13.2 程序设计

13.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 04_EXTI。

13.2.2 GPIO 功能引脚配置

在 STM32CubeIDE 中复制上一章节的工程（03_KEY），然后将工程文件名和.ioc 文件名均修改为 04_EXTI，如下图所示：

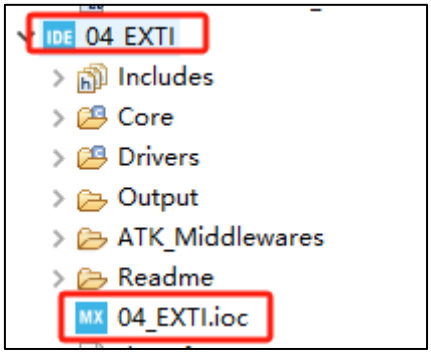


图 13.2.2.1 EXTI 工程

接着双击 04_EXTI.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PE4、PE3、PA0，要注意，需将按键 WKUP 和 KEY0 以及 KEY1 分别配置为 GPIO_EXTI0、GPIO_EXTI4、GPIO_EXTI3 外部中断模式，如下图所示：

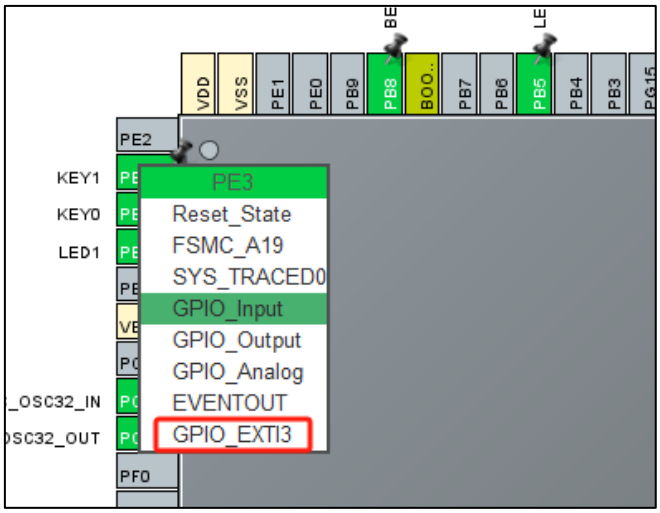


图 13.2.2.2 引脚复用配置（PE3 为例）

接下来配置按键的 GPIO 的工作模式，需要注意 3 点：

① 按键的 User Label 名字需和上一章节的保持一致。

② 除了按键 WKUP 配置为下拉模式，其它按键均配置为上拉模式。这里注意了，如果按键不配置上下拉的话，后面按下按键将会看不到效果。

③ 按键 WKUP 和 KEY0 以及 KEY1 触发模式分别配置为上升沿触发、下降沿触发、下降沿触发。

引脚配置结果如下图所示：

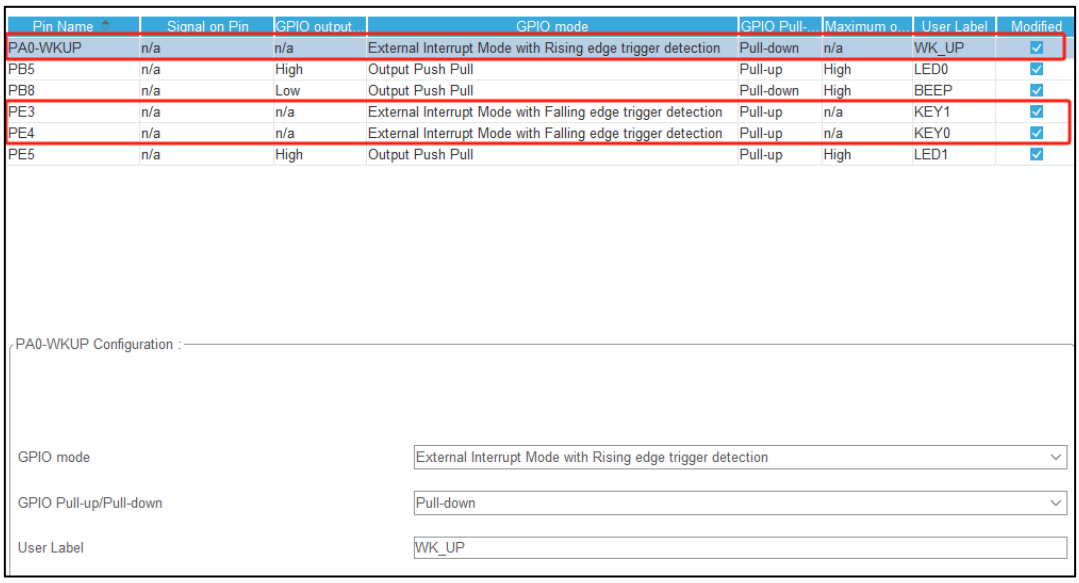


图 13.2.2.3 GPIO 配置

13.2.3 NVIC 配置

在 System Core→NVIC 选项中配置如下：

如下图，在 NVIC 的 NVIC 配置项下，配置外部中断的优先级， Preemption Priority 是抢占优先级，Sub Priority 是响应优先级（子优先级），Enabled 全局中断使能选项记得勾选。

首先中断优先级分组设置成组 2：即选择“2 bits for pre-emption priority 2”；

EXTI0 中断线：抢占优先级为 2，响应优先级为 0；

EXTI4 中断线：抢占优先级为 2，响应优先级为 1；

EXTI3 中断线：抢占优先级为 2，子优先级为 2。

Systick 中断我们配置成最低优先级即可，因为 HAL_Delay 延时函数是通过 Systick（滴答定时器）来提供时基的。

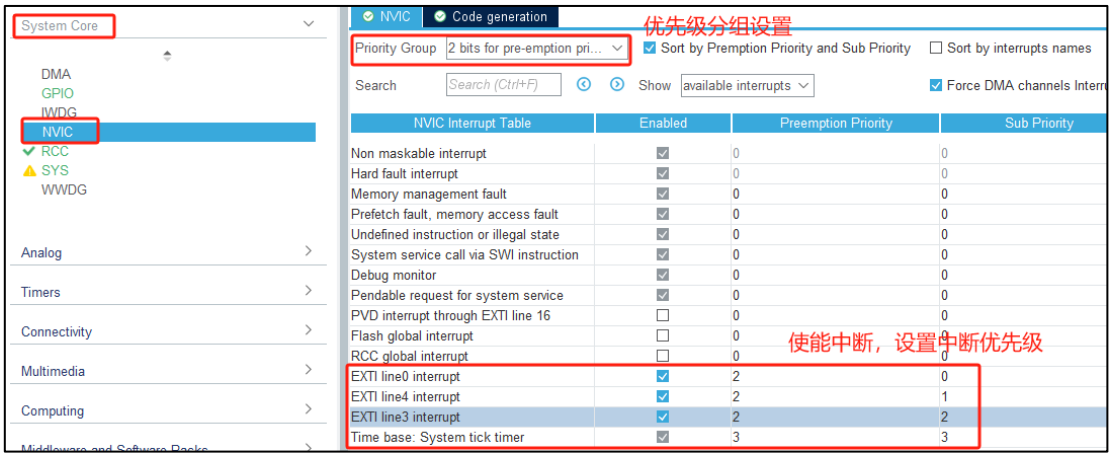


图 13.2.3.1 NVIC 设置

13.2.4 生成工程

配置好之后，按下键盘的“Ctrl+S”组合键保存 EXTI.ioc 文件，系统开始生成初始化代码。

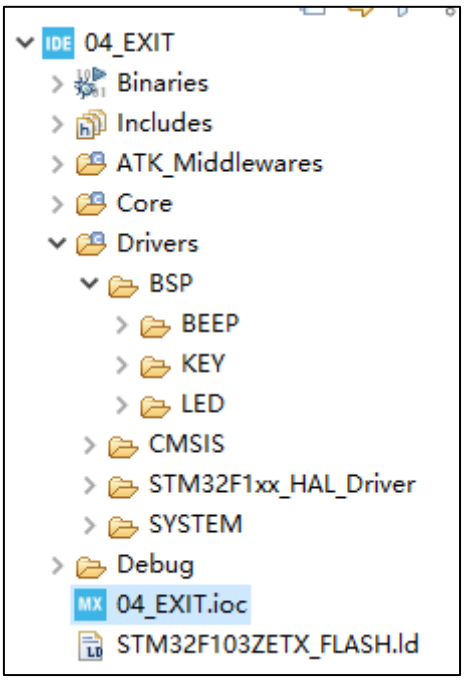


图 13.2.4.1 生成代码

13.2.5 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 EXTI 文件夹，然后再 EXTI 文件夹下新建两个文件，分别是 exti.c 和 exti.h，如下图所示：

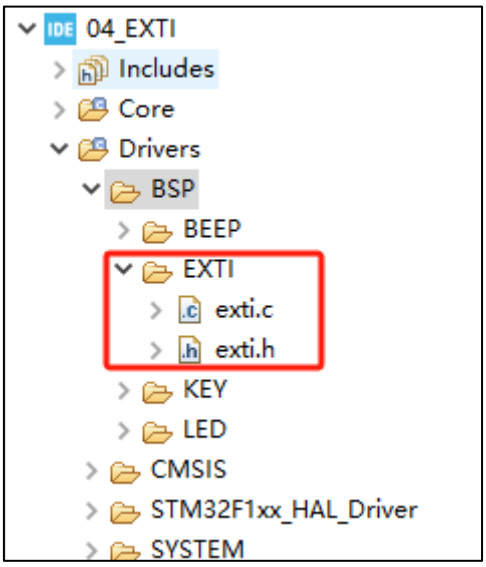


图 13.2.5.2 新建 EXTI 文件

接着我们在 exti.h、exti.c 和 main.c 文件中添加如下代码。

（1）exti.c 文件

exti.c 文件代码如下：

```
#include "exti.h"
#include "../../SYSTEM/delay/delay.h"
#include "../KEY/key.h"
#include "../LED/led.h"
#include "../BEEP/beep.h"
```

```
/**
 * @brief       中断服务程序中需要做的事情
                在 HAL 库中所有的外部中断服务函数都会调用此函数
 * @param       GPIO_Pin:中断引脚号
 * @retval      无
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    delay_ms(20);       /* 消抖 */
    switch(GPIO_Pin)
    {
        case KEY0_Pin:
            if (KEY0 == 0)
            {
                LED0_TOGGLE();  /* LED0 状态取反 */
                LED1_TOGGLE();  /* LED1 状态取反 */
            }
            break;
        case KEY1_Pin:
            if (KEY1 == 0)
            {
                LED0_TOGGLE();  /* LED0 状态取反 */
            }
            break;
        case WK_UP_Pin:
            if (WK_UP == 1)
            {
                BEEP_TOGGLE();  /* 蜂鸣器状态取反 */
            }
            break;
    }
}
```

中断回调函数 HAL_GPIO_EXTI_Callback，这个接口是一个__weak 的接口，我们通过重新实现这个函数来实现真正的外部中断控制逻辑。在该函数内部，通过判断 IO 引脚号来确定中断是来自哪个 IO 口，也就是哪个中断线，然后编写相应的控制逻辑。

如果是上升沿中断，则调用 HAL_GPIO_EXTI_Rising_Callback 回调函数，先使用 delay_ms(20)延时 20ms 进行消抖，然后再判断按键 WK_UP 的电平，如果为高电平，表示 WKUP 按键按下，蜂鸣器状态翻转。

如果是下降沿中断，先延时 20ms 进行消抖，然后再判断按键的电平，如果 KEY0 电平为 0，表示 KY0 按下，LED0 和 LED1 状态取反。如果 KEY1 电平为 0，表示 KEY1 按下，LED0 状态取反。

然后 exti.h 文件里边并未有任何内容，保持工程代码格式的完整性，所以保留其头文件。

（2）main.c 文件

接着我们在 main.c 里面编写如下代码：

```
#include "../../BSP/LED/led.h"
#include "../../BSP/BEEP/beep.h"
#include "../../BSP/KEY/key.h"
#include "../../SYSTEM/delay/delay.h"
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
```

```
  MX_GPIO_Init();

  while (1)
  {
      delay_ms(10);
  }
}
```

mian 函数比较简单，在 while 循环里边仅调用延时函数，主要的逻辑控制代码都在中断回调函数中完成。

13.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 处于熄灭状态，若此时按下 KEY0 按键，则能够看到 LED0 和 LED1 的亮灭状态发生了一次翻转，同样的，当我们按下 KEY1，LED1 状态翻转，若此时按下 WKUP 按键，蜂鸣器状态发生了一次翻转，与预期的实验现象效果相符。

第十四章 串口通信实验

本章我们将学习 STM32F1 的串口，教大家如何使用 STM32F1 的串口来发送和接收数据。本章将实现如下功能：STM32F1 通过串口和上位机的对话，STM32F1 在收到上位机发过来的字符串后，原原本本的返回给上位机。

本章分为如下几个小节：

14.1 硬件设计
14.2 程序设计
14.3 下载验证

14.1 硬件设计

1. 例程功能

LED0 闪烁，提示程序在运行。STM32 通过串口 1 和上位机对话，STM32 在收到上位机发过来的字符串(以回车换行结束)后，会返回给上位机。同时每隔一定时间，通过串口 1 输出一段信息到电脑。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340C 上面)，需要跳线帽连接。

3. 原理图

USB 转串口硬件部分的原理图，如图 14.1.1 所示：

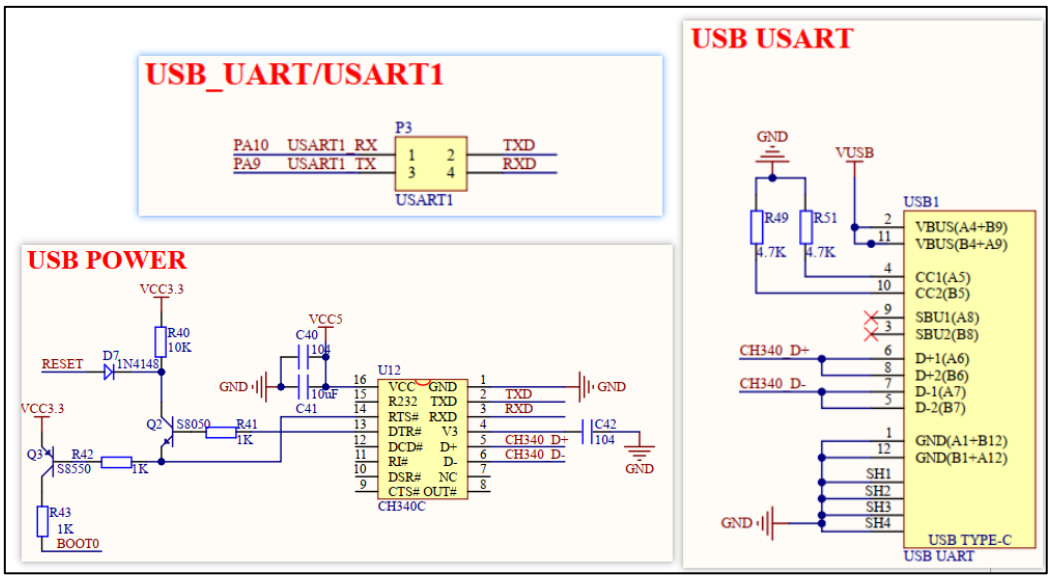


图 14.1.1 USB 转串口原理图

这里需要注意的是：上图中的 P3 的 RXD 和 PA9 用跳线帽连接，以及 TXD 和 PA10 也用跳线帽连接。如图 14.1.2 所示：

图 14.1.2 短路帽连接

14.2 程序设计

14.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\05_UART。

14.2.2 创建和配置工程

在 STM32CubeIDE 中复制 03_KEY 整个工程，然后将工程文件名和.ioc 文件名均修改为 05_UART，如下图所示：

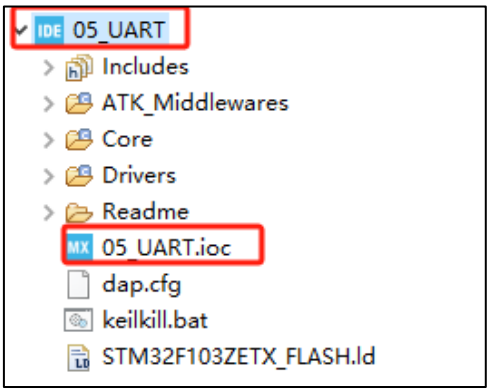


图 14.2.2.1 UART 工程

接着双击 05_UART.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PA9、PA10。需注意，配置 PA9 复用为 USART1_TX，PA10 复用为 USART1_RX.。如下图所示：

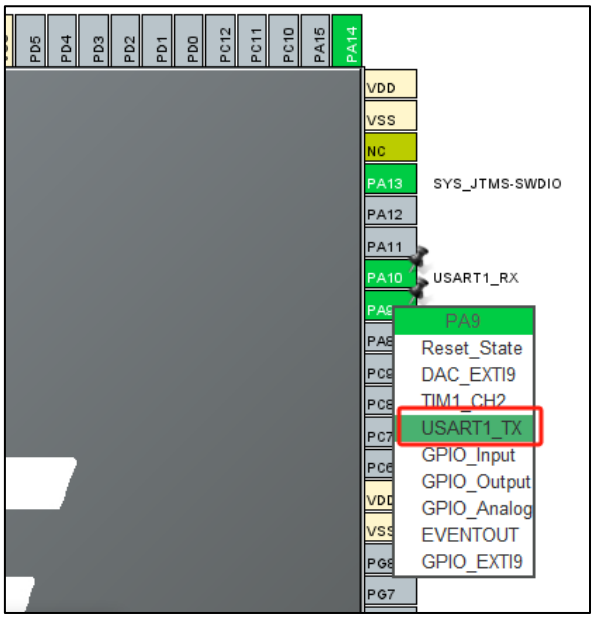


图 14.2.2.2 GPIO 复用

14.2.3 配置 USART1 参数

USART1 的参数我们选择模式：Asynchronous（异步通信模式），其余默认配置为：波特率为 115200Bit/s；字长为 8 位；无校验位；1 位停止位；数据方向设置为发送和接收；如下图所示：

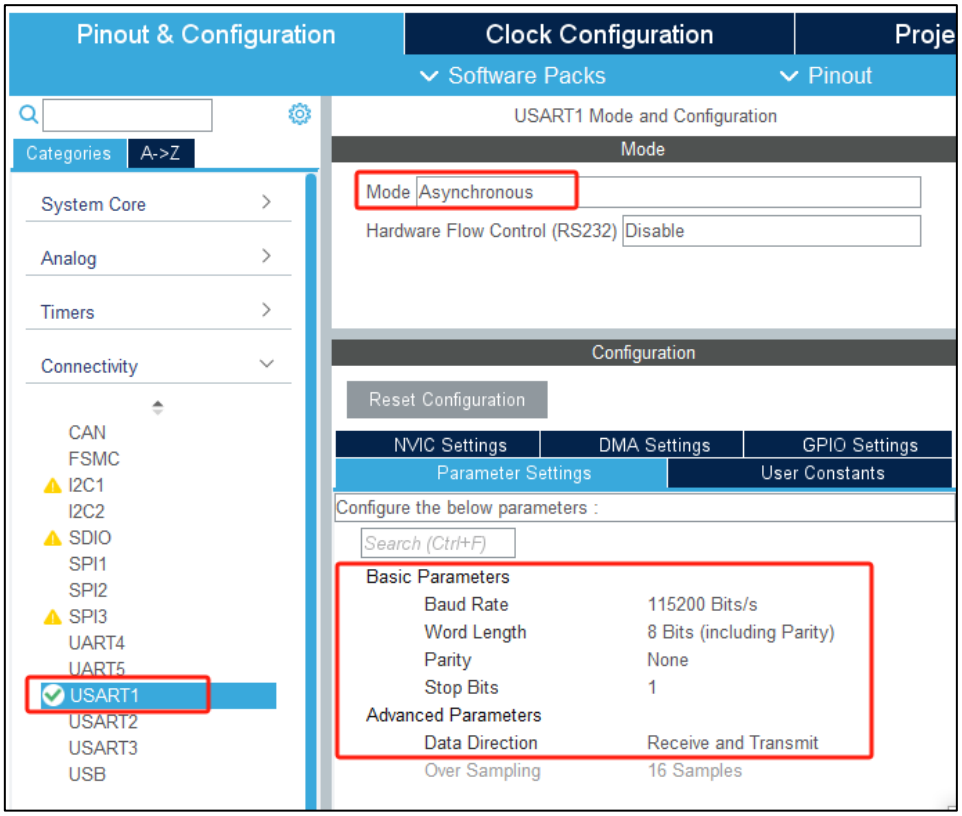


图 14.2.3.1 USART1 配置

14.2.4 配置 NVIC

本实验我们要使用串口接收中断，所以要配置 NVIC，首先开启串口全局中断，配置串口中断优先级分组为 2，抢占优先级和子优先级都是 2，如下图所示：

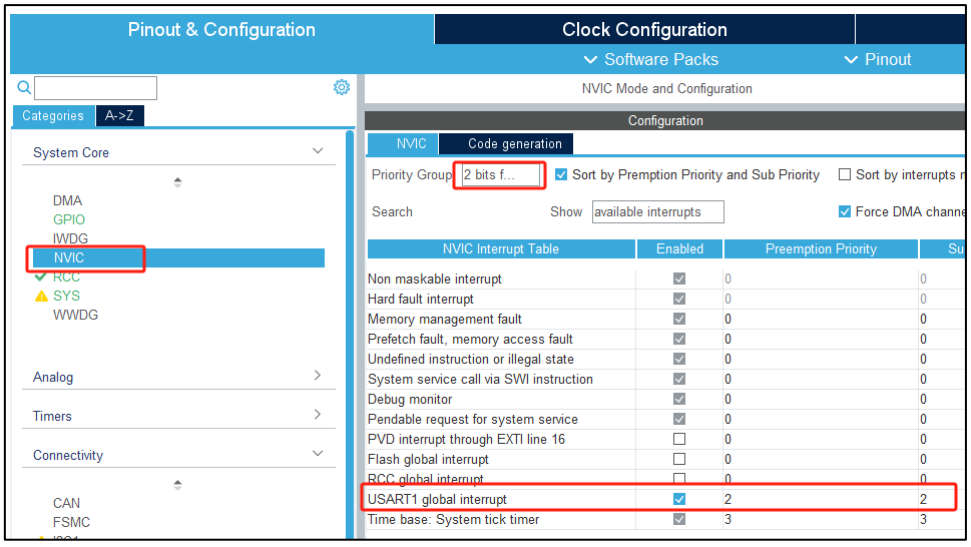


图 14.2.4.1 NVIC 配置

14.2.5 生成初始化代码

按下“Ctrl+S”保存配置，生成初始化代码：

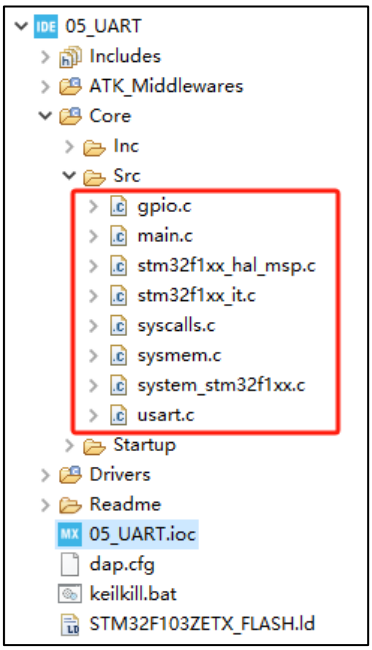


图 14.2.5.1 生成工程

14.2.6 添加用户驱动代码

（1）修改 usart.h 文件

```
#ifndef __USART_H__
#define __USART_H__

#ifdef __cplusplus
extern "C" {
#endif
#include "main.h"
#include "stdio.h"                                  /* 串口重定向使用到 */

extern UART_HandleTypeDef huart1;

/* USER CODE BEGIN Private defines */
#define USART_REC_LEN                200           /* 定义最大接收字节数 200 */
#define RXBUFFERSIZE                 1             /* 缓存大小 */
/* 接收缓冲,最大 USART_REC_LEN 个字节.末字节为换行符 */
extern uint8_t  g_usart_rx_buf[USART_REC_LEN];
extern uint16_t g_usart_rx_sta;                     /* 接收状态标记 */
extern uint8_t g_rx_buffer[RXBUFFERSIZE];           /* HAL 库 USART 接收 Buffer */

void MX_USART1_UART_Init(void);

#ifdef __cplusplus
}
#endif

#endif /* __USART_H__ */
```

在 usart.h 中我们添加了头文件 stdio.h，用于串口重定向；接着定义 USART_REC_LEN 表示最大接收字节数，这里定义的是 200 个字节，后续如果有需求要发送更大的数据包，可以改大这个值，这里不改太大，是避免浪费太多内存。RXBUFFERSIZE 是缓冲大小。

（2）修改 usart.c 文件

在 usart.c 中，为了让串口支持 printf，需要对串口进行重定向，在 usart.c 中添加的重定向代码如下：

```
/* 串口重定向 */
#ifdef __GNUC__
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif

PUTCHAR_PROTOTYPE
{
    HAL_UART_Transmit(&huart1, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
```

完成上述重定向串口操作后，就可以使用 printf 语句来打印数据。但是值得注意的是，**如果需要打印浮点数，则需要按照 4.2.7 小节的要求进行设置。**

由于本章内容是要实现串口的发送和接收，经过串口重定向后我们可以通过 printf 完成串口的数据发送了，所以接下来，我们开始完成串口数据接收部分内容。

```
void MX_USART1_UART_Init(void)
{

  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }

/* 该函数会开启接收中断：标志位 UART_IT_RXNE，并且设置接收缓冲以及接收缓冲接收最大数据量 */
  HAL_UART_Receive_IT(&huart1, (uint8_t *)g_rx_buffer, RXBUFFERSIZE);
}

/* 省略部分代码... ... */
```

在串口初始化 MX_USART1_UART_Init(void)函数中，调用函数 HAL_UART_Receive_IT，作用是开启接收中断，同时设置接收的缓存区以及接收的数据量。

在中断服务函数里主要就调用 HAL 库的串口中断公共处理函数 HAL_UART_IRQHandler()，然后该函数内部再调用相关的中断回调函数，我们这里用到的是串口接收完成中断回调函数 HAL_UART_RxCpltCallback 来处理用户的逻辑代码。

```
/**
 * @brief       串口数据接收回调函数
                数据处理在这里进行
 * @param       huart:串口句柄
 * @retval      无
 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)                      /* 如果是串口 1 */
    {
        if ((g_usart_rx_sta & 0x8000) == 0)             /* 接收未完成 */
        {
            if (g_usart_rx_sta & 0x4000)                /* 收到 0x0d（回车键）*/
            {
```

```
                /* 接收到的不是 0x0a（即不是换行键） */
                if (g_rx_buffer[0] != 0x0a)
                {
                    g_usart_rx_sta = 0;                 /* 接收错误,重新开始 */
                }
                else                                    /* 收到 0x0a（换行键） */
                {
                    g_usart_rx_sta |= 0x8000;           /* 接收完成了 */
                }
            }
            else                                        /*未收到 0X0d（回车键）*/
            {
                if (g_rx_buffer[0] == 0x0d)
                    g_usart_rx_sta |= 0x4000;
                else
                {
                    g_usart_rx_buf[g_usart_rx_sta & 0X3FFF] = g_rx_buffer[0];
                    g_usart_rx_sta++;

                    if (g_usart_rx_sta > (USART_REC_LEN - 1))
                    {
                        g_usart_rx_sta = 0;             /* 接收错误,重新接收 */
                    }
                }
            }
        }

        HAL_UART_Receive_IT(&huart1, (uint8_t *)g_rx_buffer, RXBUFFERSIZE);
    }
}
```

在 USART1 的中断回调函数中主要用于读取 USART1 接收到的数据，并将其逐一存入接收的缓冲区，并在接收到“回车”和“换行”后标志数据接收完成。

（3）main.c 文件

本实验的应用代码，如下所示：

```
int main(void)
{
  uint8_t len;
  uint16_t times = 0;

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

 /* 串口初始化 */
  MX_USART1_UART_Init();

  while (1)
  {
      if (g_usart_rx_sta & 0x8000)          /* 接收到了数据? */
      {
          len = g_usart_rx_sta & 0x3fff;  /* 得到此次接收到的数据长度 */
          printf("\r\n 您发送的消息为:\r\n");

          /* 发送接收到的数据 */
```

```
            HAL_UART_Transmit(&huart1,(uint8_t*)g_usart_rx_buf, len, 1000);
            /* 等待发送结束 */
            while(__HAL_UART_GET_FLAG(&huart1, UART_FLAG_TC) != SET);
            printf("\r\n\r\n");                 /* 插入换行 */
            g_usart_rx_sta = 0;
        }
        else
        {
            times++;

            if (times % 5000 == 0)
            {
                printf("\r\n 正点原子 STM32 开发板 串口实验\r\n");
                printf("正点原子@ALIENTEK\r\n\r\n\r\n");
            }

            if (times % 200 == 0) printf("请输入数据,以回车键结束\r\n");

            if (times % 30  == 0) LED0_TOGGLE(); /* 闪烁 LED,提示系统正在运行. */

            delay_ms(10);
        }
    }
}
```

本实验的实验代码很简单，在完成初始化后，就不断地通过串口通信驱动提供的数据接收完成标志判断数据是否接收完毕，若还未完成数据接收，则每间隔一段时间就使用 printf 函数通过 USART1 打印一段提示信息，若数据接收完毕，则将数据原原本本地使用 printf 函数通过 USART1 打印出去，实现数据的回显功能。

14.3 下载验证

在完成编译和烧录操作后，将开发板的 USB UART 接口与与电脑的 USB 接口进行连接，并保证 14.1 小节中指示跳线帽已正确安装。接着打开 PC 上的 ATK-XCOM 串口调试助手软件，选择好正确的 COM 端口和相关的配置后，就能看到串口调试助手上每间隔一段时间就打印一次“请输出数据，以回车键结束”，接下来就可以根据提示通过串口调试助手发送一段任意的数据（以回车换行结束），随后立马就能看到串口调试助手上显示发送出去的数据，这就是本实验实现的数据回显功能。

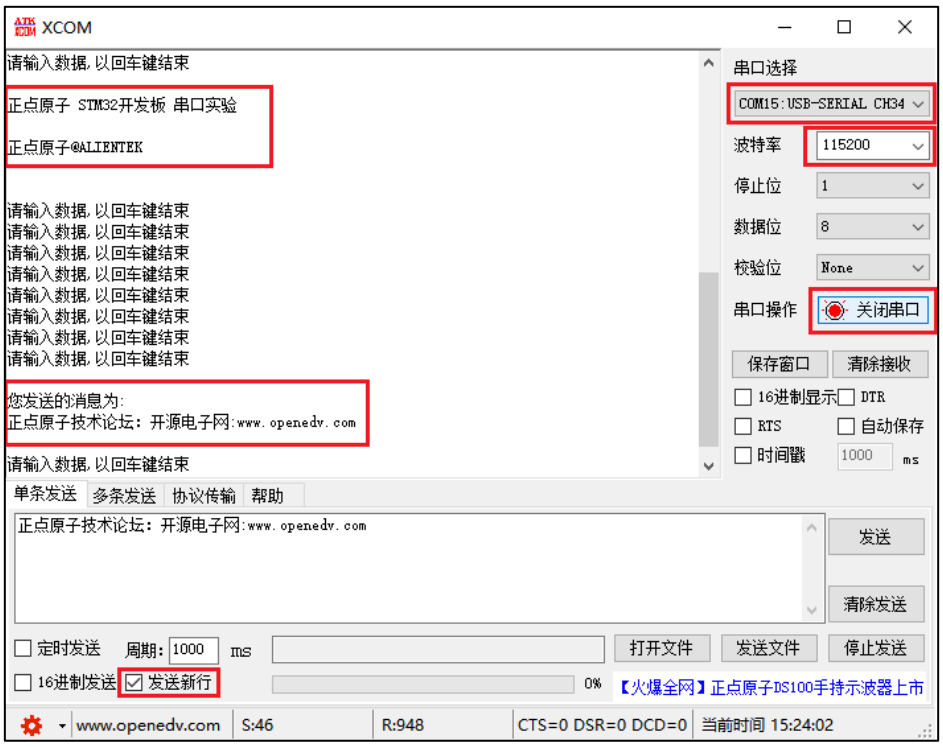


图 14.3.1 串口助手

可以看到，我们发送的消息被发送回来了。大家可以试试，如果不发送回车（取消发送新行），在输入内容之后，直接按发送是什么结果，大家测试一下吧。

第十五章 独立看门狗（IWDG）实验

本章我们学习如何使用 STM32F1 的独立看门狗（以下简称 IWDG）。STM32F1 内部自带了 2 个看门狗：独立看门狗（IWDG）和窗口看门狗（WWDG）。这一章我们只介绍独立看门狗，窗口看门狗将在下一章介绍。在本章中，我们将通过按键 KEY_UP 来喂狗，然后通过 LED0 提示复位状态。

本章分为如下几个小节：

15.1 硬件设计
15.2 程序设计
15.3 下载验证

15.1 硬件设计

1. 例程功能

在配置看门狗后，LED0 将常亮，如果 KEY_UP 按键按下，就喂狗，只要 KEY_UP 不停的按，看门狗就一直不会产生复位，保持 LED0 的常亮，一旦超过看门狗定溢出时间（Tout）还没按，那么将会导致程序重启，这将导致 LED0 熄灭一次。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
WK_UP - PA0
3）独立看门狗

3. 原理图

独立看门狗实验的核心是在 STM32F103 内部进行，并不需要外部电路。但是考虑到指示当前状态和喂狗等操作，我们需要 2 个 IO 口，一个用来触发喂狗信号，另外一个用来指示程序是否重启。喂狗我们采用板上的 KEY_UP 键来操作，而程序重启，则是通过 LED0 来指示的。

15.2 程序设计

15.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\06_IWDG。

15.2.2 创建和配置工程

在 STM32CubeIDE 中复制 05_UART 整个工程，然后将工程文件名和.ioc 文件名均修改为 06_IWDG，如下图所示：

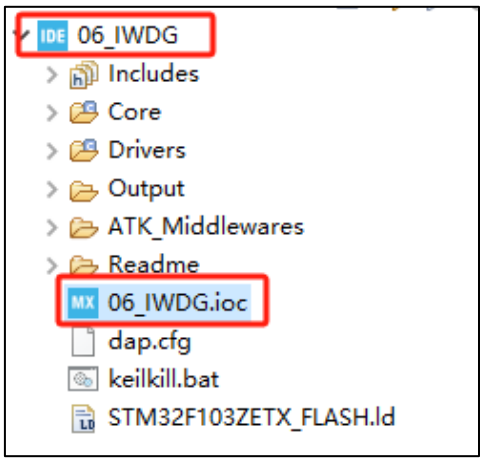


图 15.2.2.1 IWDG 工程

接着双击 06_IWDG.ioc 进入 STM32CubeMX 插件配置界面后，打开左边的 System Core→IWDG，配置 IWDG 参数，这里我们配置独立看门狗的溢出时间为 1s，所以需要进行如下配置，配置 IWDG counter clock prescaler 分频值为 64，IWDG down-counter reload value 重装载值为 625。如下图所示：

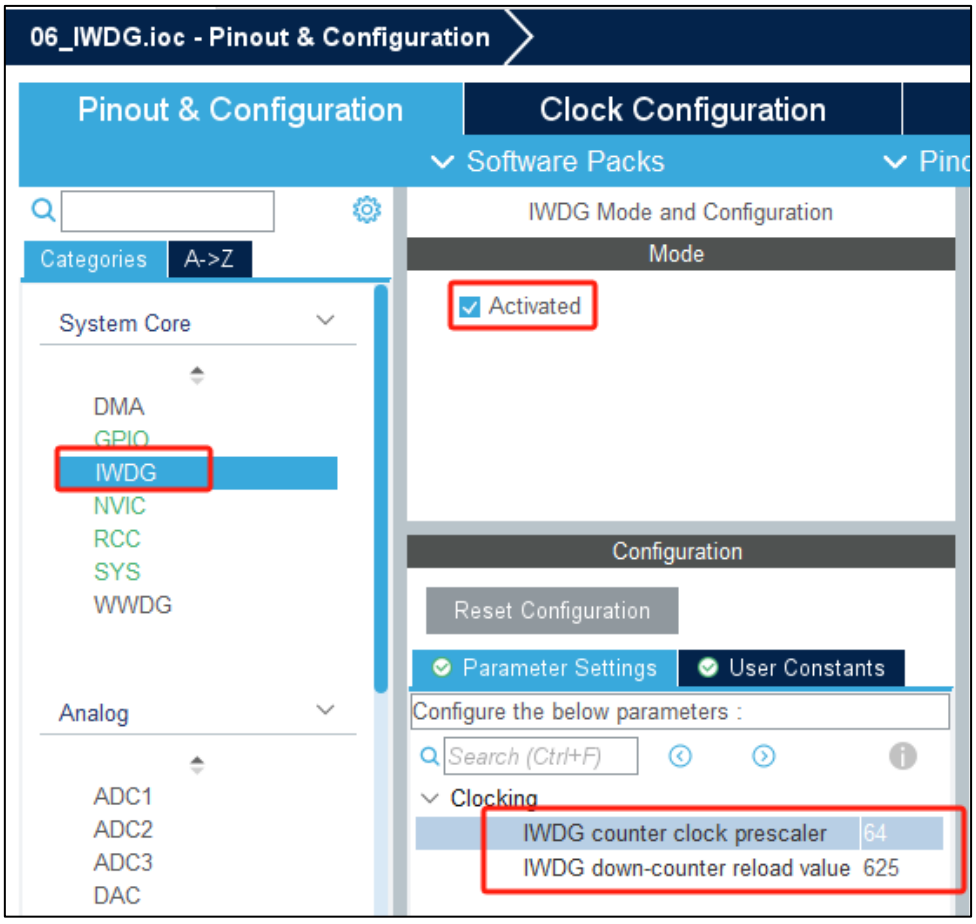


图 15.2.2.2 IWDG 参数配置

即预分频系数为 64，重装载值是 625，所以可由公式得到 Tout=64×625/40=1000ms，即溢出时间就是 1s。只要你在一秒钟之内，有一次写 0XAAAA 到 IWDG_KR，就不会导致看门狗复位（当然写入多次也是可以的）。这里需要提醒大家的是，看门狗的时钟不是准确的 40Khz，所以在喂狗的时候，最好不要太晚了，否则，有可能发生看门狗复位。

15.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 iwdg.c 文件。iwdg.c 主要是完成 IWDG 的初始化。

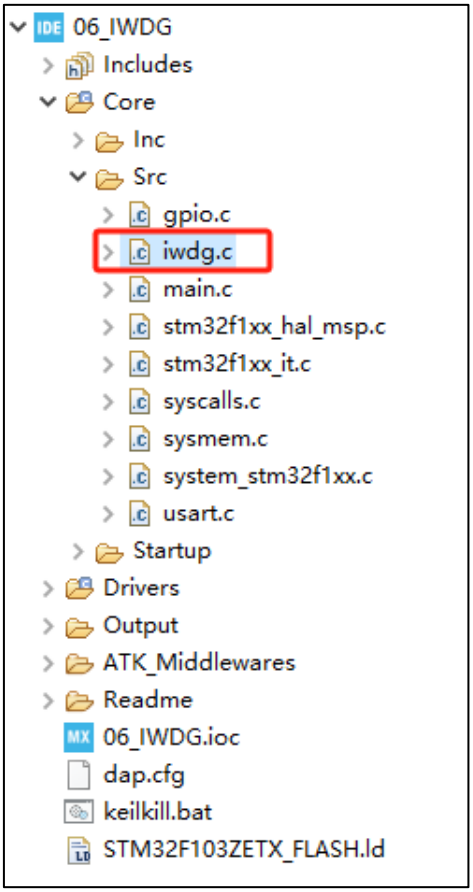


图 15.2.3.1 生成工程

15.2.4 添加用户驱动代码

我们本实验主要修改这三个文件，分别是：iwdg.c、iwdg.h、main.c，首先，我们将对 iwdg.c 文件进行修改。

（1）修改 iwdg.h 文件

```
#ifndef __IWDG_H__
#define __IWDG_H__

#ifdef __cplusplus
extern "C" {
#endif
#include "main.h"

extern IWDG_HandleTypeDef hiwdg;

void MX_IWDG_Init(void);

/* USER CODE BEGIN Prototypes */
void iwdg_feed(void);   /* ι' */
/* USER CODE END Prototypes */

#ifdef __cplusplus
}
```

```
#endif

#endif /* __IWDG_H__ */
```

该文件比较简单，我们只添加了一个函数声明，声明了 iwdg_free 函数，此函数用于喂狗。

（2）修改 iwdg.c 文件

```
#include "iwdg.h"

/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

IWDG_HandleTypeDef hiwdg;

/* IWDG init function */
void MX_IWDG_Init(void)
{

  hiwdg.Instance = IWDG;
  hiwdg.Init.Prescaler = IWDG_PRESCALER_64;
  hiwdg.Init.Reload = 625;
  if (HAL_IWDG_Init(&hiwdg) != HAL_OK)
  {
    Error_Handler();
  }
}

/* USER CODE BEGIN 1 */
/**
 * @brief       喂独立看门狗
 * @param       无
 * @retval      无
 */
void iwdg_feed(void)
{
    HAL_IWDG_Refresh(&hiwdg);   /* 重装载计数器 */
}
/* USER CODE END 1 */
```

在该文件中，我们添加了 iwdg_feed 函数用来喂狗，在该函数内部只需调用 HAL 库函数 HAL_IWDG_Refresh。

（3）修改 main.c 文件

```
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* 初始化独立看门狗 */
  MX_IWDG_Init();
```

```
  /* USER CODE BEGIN 2 */
  delay_ms(100);              /* 延时 100ms 再初始化看门狗,LED0 的变化"可见" */
  LED0(0);                    /* 点亮 LED0(红灯) */
  while (1)
  {
      if (key_scan(1) == WKUP_PRES)        /* 如果 WK_UP 按下,则喂狗 */
      {
          iwdg_feed();                     /* 喂狗 */
      }

      delay_ms(10);
  }
}
```

在 main 函数里，先初始化系统和用户的外设代码，然后先点亮 LED0，在无限循环里开始获取按键的键值，并判断是不是按键 WK_UP 按下，是的话就喂狗，不是则延时 10ms，继续上述操作。当 1 秒钟后都没测到按键 WK_UP 按下，IWDG 就会产生一次复位信号，系统复位，可以看到 LED0 因系统复位熄灭一次，再亮。反之，当按下按键 WK_UP 后，1 秒内再按下按键 WK_UP，就会及时喂狗，结果就是系统不会复位，LED0 也就不会闪烁。

15.3 下载验证

下载代码后，可以看到 LED0 不停的闪烁，证明系统在不停的复位，否则 LED0 常亮。这时我们试试不停的按 WK_UP 按键，可以看到 LED0 就常亮了，不会再闪烁。说明我们的实验设计成功了。

第十六章 窗口看门狗（WWDG）实验

本章我们学习如何使用 STM32F1 的另外一个看门狗，窗口看门狗（以下简称 WWDG）。我们将使用窗口看门狗的中断功能来喂狗，通过 LED0 和 LED1 提示程序的运行状态。

本章分为如下几个小节：

16.1 硬件设计
16.2 程序设计
16.3 下载验证

16.1 硬件设计

1. 例程功能

先点亮 LED0 延时 300ms 后，初始化窗口看门狗，进入死循环，关闭 LED0。然后等待窗口看门狗中断的到来，在中断里面喂狗，并执行 LED1 的翻转操作。我们将通过 LED0 来指示 STM32F1 是否被复位了，如果被复位了就会点亮 300ms。LED1 用来指示中断喂狗，每次中断喂狗翻转一次。

2. 硬件资源

1）LED 灯
LED0 - PB5
LED1 - PE5
2）窗口看门狗

3. 原理图

窗口看门狗属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 LED0 和 LED1 来指示 STM32F103 的复位情况和窗口看门狗的喂狗情况。

16.2 程序设计

16.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\07_WWDG。

16.2.2 创建和配置工程

在 STM32CubeIDE 中复制 05_UART 工程，然后将工程文件名和.ioc 文件名均修改为 07_WWDG，如下图所示：

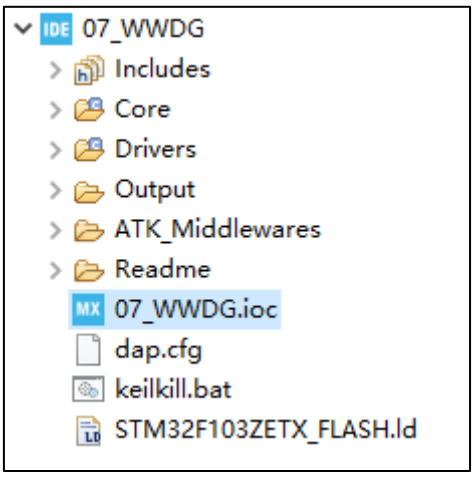


图 16.2.2.1 WWDG 工程

接着双击 07_WWDG.ioc 进入 STM32CubeMX 插件配置界面后，打开左边的 System Core→WWDG，配置 WWDG 参数，如下配置，配置 WWDG counter clock prescaler 分频值为 8，WWDG window value 窗口寄存器为 5f，配置 WWDG free-running downcounter value 计数器值为 7f，并使能窗口看门狗提前唤醒中断。如下图所示：

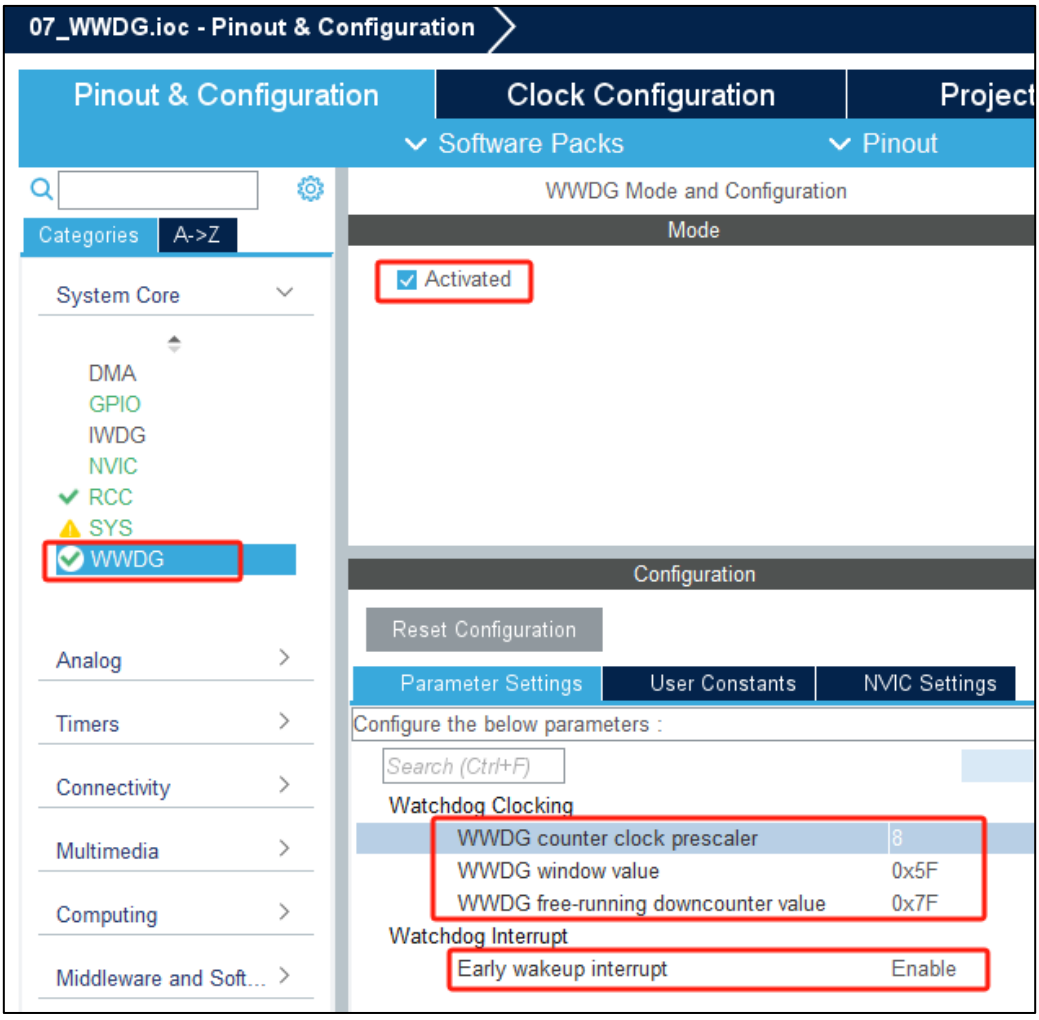


图 16.2.2.2 WWDG 参数配置

将我们配置的数值代入公式得到 Twwdg=4096×8×64/36MHz=58.25ms，即超时时间就是 58.25ms。

16.2.3 配置 NVIC

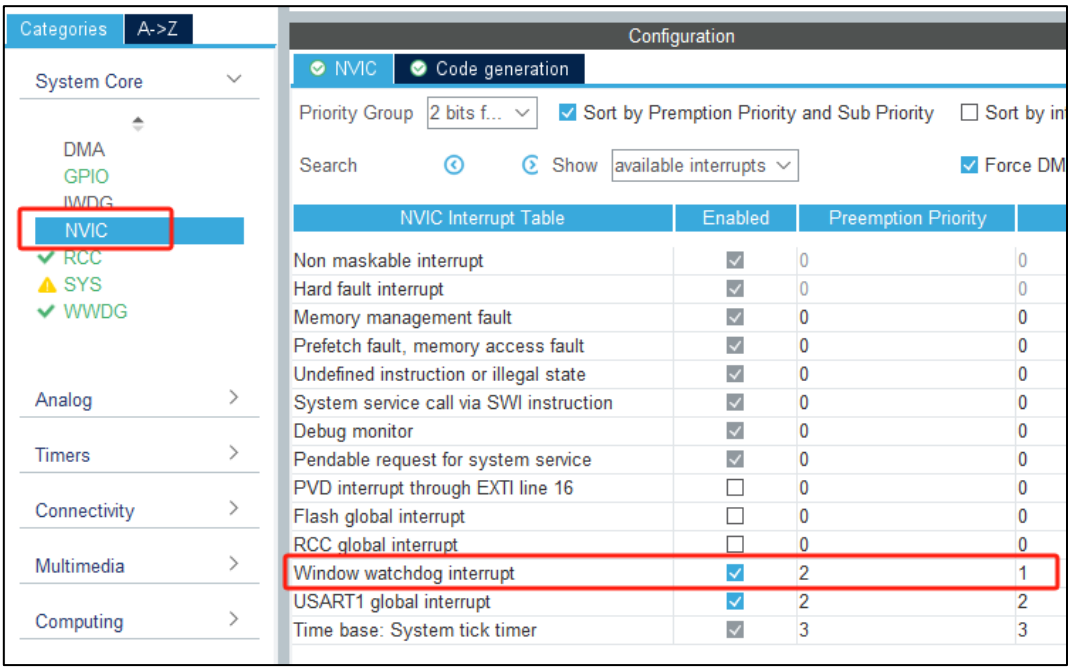


图 16.2.3.1 NVIC 配置

在 NVIC 窗口中，使能窗口看门狗中断，并设置窗口看门狗中断的抢占优先级为 2，响应优先级为 1。

16.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 wwdg.c 文件。wwdg.c 主要是完成 WWDG 的初始化。

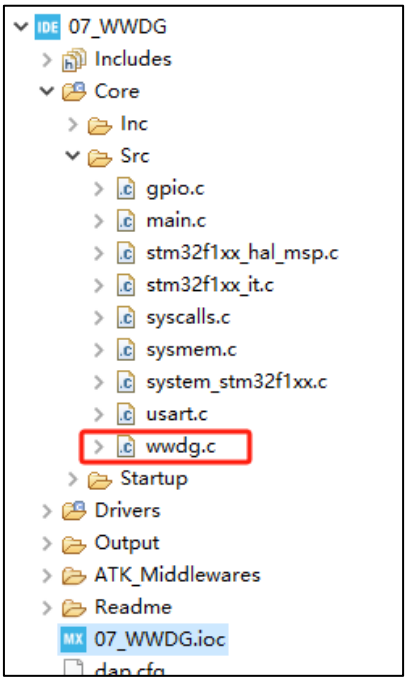


图 16.2.4.1 生成工程

16.2.5 添加用户驱动代码

我们本实验主要修改两个文件，分别是：wwdg.c、main.c，首先，我们将对 wwdg.c 文件进行修改。

（1）修改 wwdg.c 文件

```
#include "wwdg.h"

#include "../../BSP/LED/led.h"

WWDG_HandleTypeDef hwwdg;

/* WWDG init function */
void MX_WWDG_Init(void)
{
  hwwdg.Instance = WWDG;
  hwwdg.Init.Prescaler = WWDG_PRESCALER_8;
  hwwdg.Init.Window = 0x5F;
  hwwdg.Init.Counter = 0x7F;
  hwwdg.Init.EWIMode = WWDG_EWI_ENABLE;
  if (HAL_WWDG_Init(&hwwdg) != HAL_OK)
  {
    Error_Handler();
  }

}

void HAL_WWDG_MspInit(WWDG_HandleTypeDef* wwdgHandle)
{

  if(wwdgHandle->Instance==WWDG)
  {
    /* WWDG clock enable */
    __HAL_RCC_WWDG_CLK_ENABLE();

    /* WWDG interrupt Init */
    HAL_NVIC_SetPriority(WWDG_IRQn, 2, 1);
    HAL_NVIC_EnableIRQ(WWDG_IRQn);
  }
}

/* USER CODE BEGIN 1 */
/**
 * @brief        窗口看门狗喂狗提醒中断服务回调函数
 * @param        wwdg 句柄
 * @note         此函数会被 HAL_WWDG_IRQHandler()调用
 * @retval       无
 */
void HAL_WWDG_EarlyWakeupCallback(WWDG_HandleTypeDef *hwwdg)
{
    HAL_WWDG_Refresh(hwwdg);                /* 更新窗口看门狗值 */
    LED1_TOGGLE();                          /* LED1 闪烁 */
}
```

在窗口看门狗的初始化函数中，通过我们前面的配置，设置计数器值为 7f，窗口寄存器为 5f，分频数为 8，然后可由前面的公式得到 Twwdg=4096×8×64/36MHz=58.25ms，即超时时间就是 58.25ms。58.25ms 进中断一次，在中断回调函数中实现我们的操作逻辑，一中断就喂狗，并翻转 LED1。

（2）修改 main.c 文件

```
int main(void)
```

```
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  LED0(0);
  delay_ms(300);              /* 延时 100ms 再初始化看门狗,LED0 的变化"可见" */
  MX_WWDG_Init();             /* 注意看门狗初始化在 LED0 点亮后再初始化，否则效果不明显 */

  while (1)
  {
      LED0(1);                /* 关闭红灯 */;
  }
}
```

在 main 函数里，先初始化系统和外设代码，然后先点亮 LED0，延时 300ms 后，在初始化窗口看门狗，进入死循环，关闭 LED0。然后每间隔 58.25ms，会触发窗口看门狗的中断回调函数，在里边进行喂狗，并翻转 LED1。注意一定要先点亮 LED0 再去初始化窗口看门狗，否则现象不明显！

16.3 下载验证

下载代码后，可以看到 LED0 亮了一下就熄灭，紧接着 LED1 开始不停的闪烁。可以接入示波器测试得每秒钟闪烁 17 次左右，说明程序在中断不停的喂狗，和我们预期的一致。

第十七章 基本定时器实验

STM32F103 有众多的定时器，其中包括 2 个基本定时器（TIM6 和 TIM7）、4 个通用定时器（TIM2~TIM5）、2 个高级控制定时器（TIM1 和 TIM8），这些定时器彼此完全独立，不共享任何资源。本章我们学习如何使用 STM32F103 的基本定时器中断。我们将使用 TIM6 的定时器中断来控制 LED1 的翻转，在主函数用 LED0 的翻转来提示程序正在运行。

本章分为如下几个小节：

17.1 硬件设计
17.2 程序设计
17.3 下载验证

17.1 硬件设计

1. 例程功能

LED0 用来指示程序运行，每 200ms 翻转一次。我们在更新中断中，将 LED1 的状态取反。LED1 用于指示定时器发生更新事件的频率，500ms 取反一次。

2. 硬件资源

1）LED 灯
LED0 – PB5
LED1 – PE5
2）定时器 6

3. 原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 LED1 来指示 STM32F103 的定时器进入中断情况。

17.2 程序设计

17.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\08_BTIM。

17.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 08_BTIM，以进行下一步的实验操作。如下图所示：

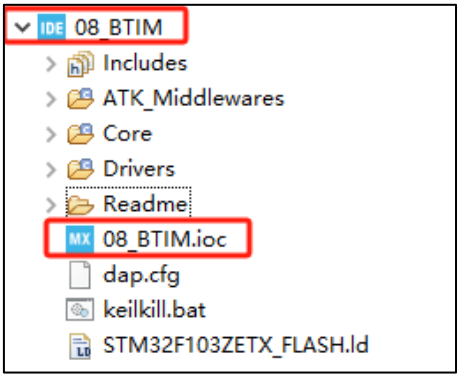


图 17.2.2.1 BTIM 工程

接着双击 08_BTIM.ioc 进入 STM32CubeMX 插件配置界面后，打开左边的 Timers→TIM6，配置 TIM6 参数，如下图所示：

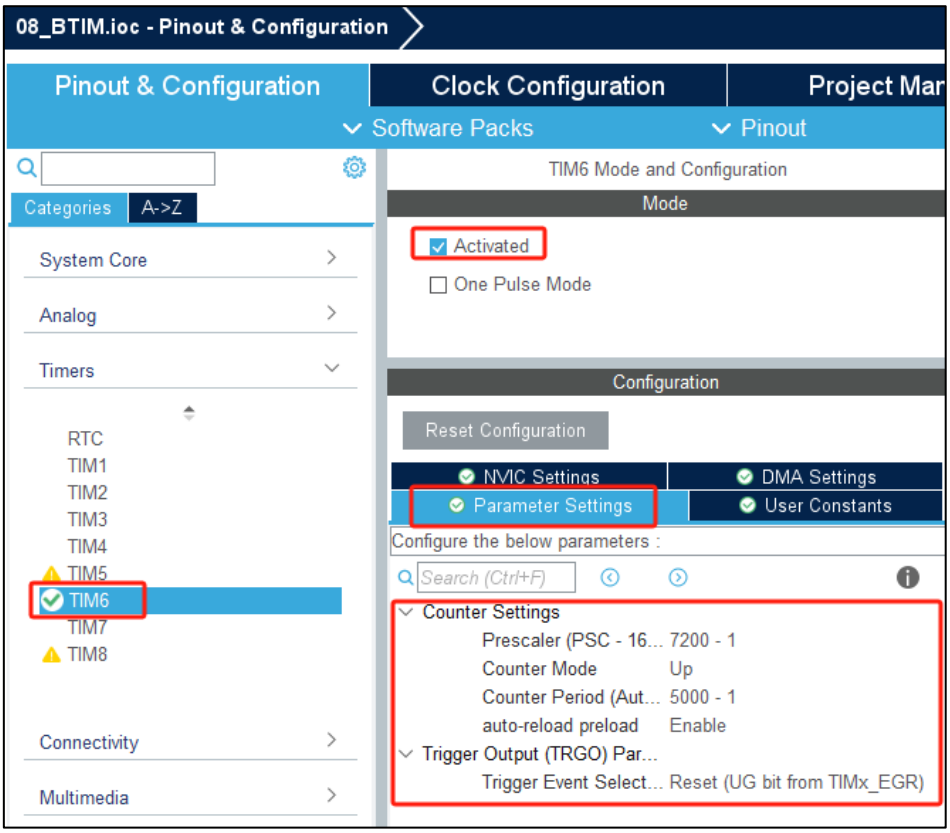


图 17.2.2.2 TIM6 参数配置

注意到上面的参数配置，我们配置预分频器寄存器(TIMx_PSC)的值为 7200-1；计数模式是向上递增计数，基本定时器的计数模式只有递增模式；自动重载寄存器 (TIMx_ARR)的值为 5000-1；开启自动重装载模式。这里注意的是，自动重载寄存器 (TIMx_ARR)的值不能设置为 0，否则计数器不工作（定时器也就不工作了），另外，要开启自动重装载模式，即定时器溢出时会自动重装初值。

Trigger Output (TRGO) Parameters 选项中，触发输出事件我们配置为 Reset(UG bit from TIMX_EGR)，即 TIMx_EGR 寄存器的 UG 位用做触发输出，触发事件选择为清零。

基本定时器的时钟频率是 72MHz，以上参数预分频器分频值为 7200-1，计算出计数器 CK_CNT 的时钟频率是：

$$f_{CK_CNT} = \frac{f_{CK_PSC}}{PSC[15:0]+1}\frac{72MHz}{7200} = 10KHz$$

那么计数器计数 5000 次就会溢出产生中断，所以每次溢出时间是：

$$5000 * \frac{1}{10KHz} = 0.5s$$

17.2.3 配置 NVIC

定时器要每 0.5 秒溢出产生中断，所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM6 定时器全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 1：

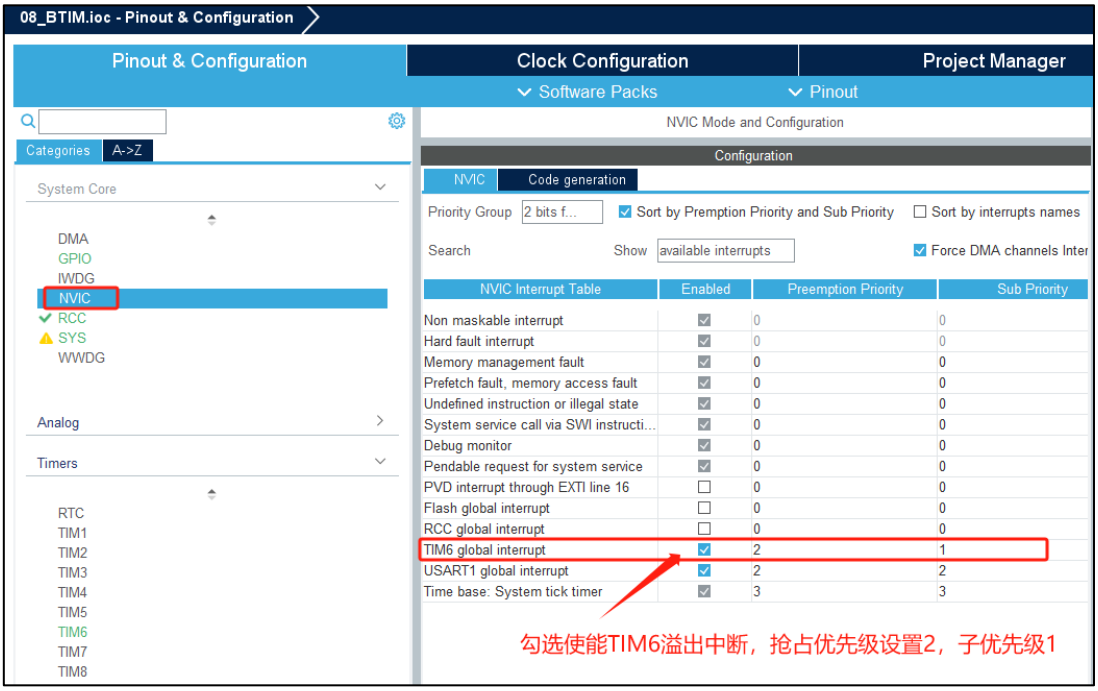


图 17.2.3.1 NVIC 配置

17.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM6 的初始化。

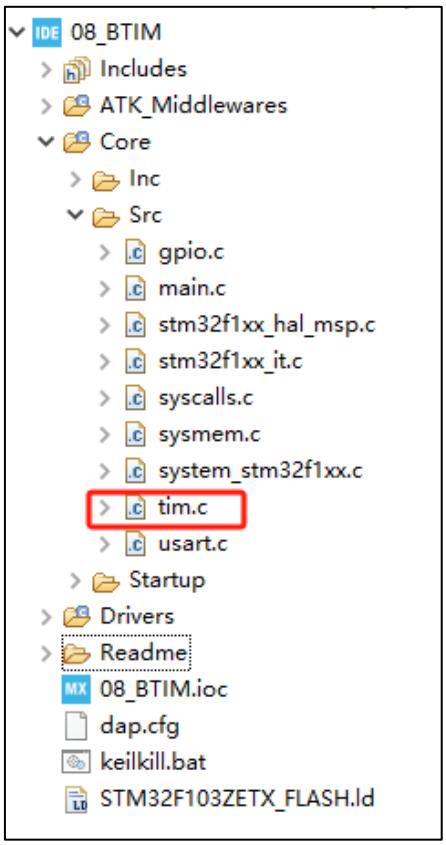


图 17.2.4.1 生成代码

17.2.5 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

```
#include "tim.h"

#include "../../BSP/LED/led.h"

TIM_HandleTypeDef htim6;

/* TIM6 init function */
void MX_TIM6_Init(void)
{
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  htim6.Instance = TIM6;
  htim6.Init.Prescaler = 7200 - 1;
  htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim6.Init.Period = 5000 - 1;
  htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim6) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim6, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
```

```
  /* USER CODE BEGIN TIM6_Init 2 */
  HAL_TIM_Base_Start_IT(&htim6);     /* 使能定时器 x 及其更新中断 */
  /* USER CODE END TIM6_Init 2 */

}

/* 省略部分代码… … */

/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        LED1_TOGGLE(); /* LED1 反转 */
    }
}
```

在上述代码中，添加 led 相关的头文件。是由于后续需要在中断回调函数中对 LED1 进行翻转操作；在 MX_TIM6_Init 函数中，我们通过前面的配置，设置 TIM6 每 500ms 溢出一次，并触发更新中断。同时，在该函数中调用 HAL_TIM_Base_Start_IT(&htim6)函数，用于使能 TIM6 并开启更新中断。在中断回调函数中，我们会判断是否为 TIM6 触发的更新中断，如果是，则执行一次 LED1 的翻转操作。

（2）修改 main.c 文件

```
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
  MX_TIM6_Init();
  while (1)
  {
      LED0_TOGGLE();
      delay_ms(200);
  }
}
```

在 main 函数中先初始化系统和用户的外设代码，然后在 wilhe(1)里每 200ms 翻转一次 LED0。实现的内容比较简单。在定时器中断回调函数中我们设置的每 500ms 中断一次，并翻转一次 LED1。

17.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 都在闪烁，但闪烁频率不同，LED0 每间隔 200 毫秒改变一次状态，LED1 每隔 500 毫秒改变一次状态。

第十八章 通用定时器中断实验

本章介绍 STM32F103 通用定时器的使用，通用定时器相较于基本定时器，拥有输入捕获和输出比较等功能，这些功能可以用来测量脉冲宽度、频率和占空比，并且可以产生并输出波形等。通过本章的学习，读者将学习到通用定时器的基本使用。

本章分为如下几个小节：

18.1 硬件设计
18.2 程序设计
18.3 下载验证

18.1 硬件设计

1. 例程功能

LED0 用来指示程序正在运行，200ms 翻转一次。LED1 在定时器中断中翻转，500ms 进入中断一次。

2. 硬件资源

1）LED 灯
LED0 - PB5
LED1 - PE5
2）定时器 3

3. 原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 LED1 来指示 STM32F103 的定时器进入中断的频率，LED0 则指示程序的运行状态。

18.2 程序设计

18.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\09_1_GTIM_Update。

18.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 09_1_GTIM_Update，以进行下一步的实验操作。如下图所示：

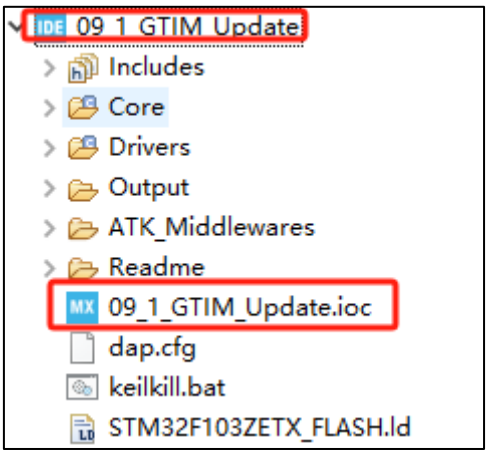


图 18.2.2.1 GTIM 工程

接着双击 09_1_GTIM_Update.ioc 进入 STM32CubeMX 插件配置界面后，打开左边的 Timers→TIM3，配置 TIM3 参数，如下图所示：

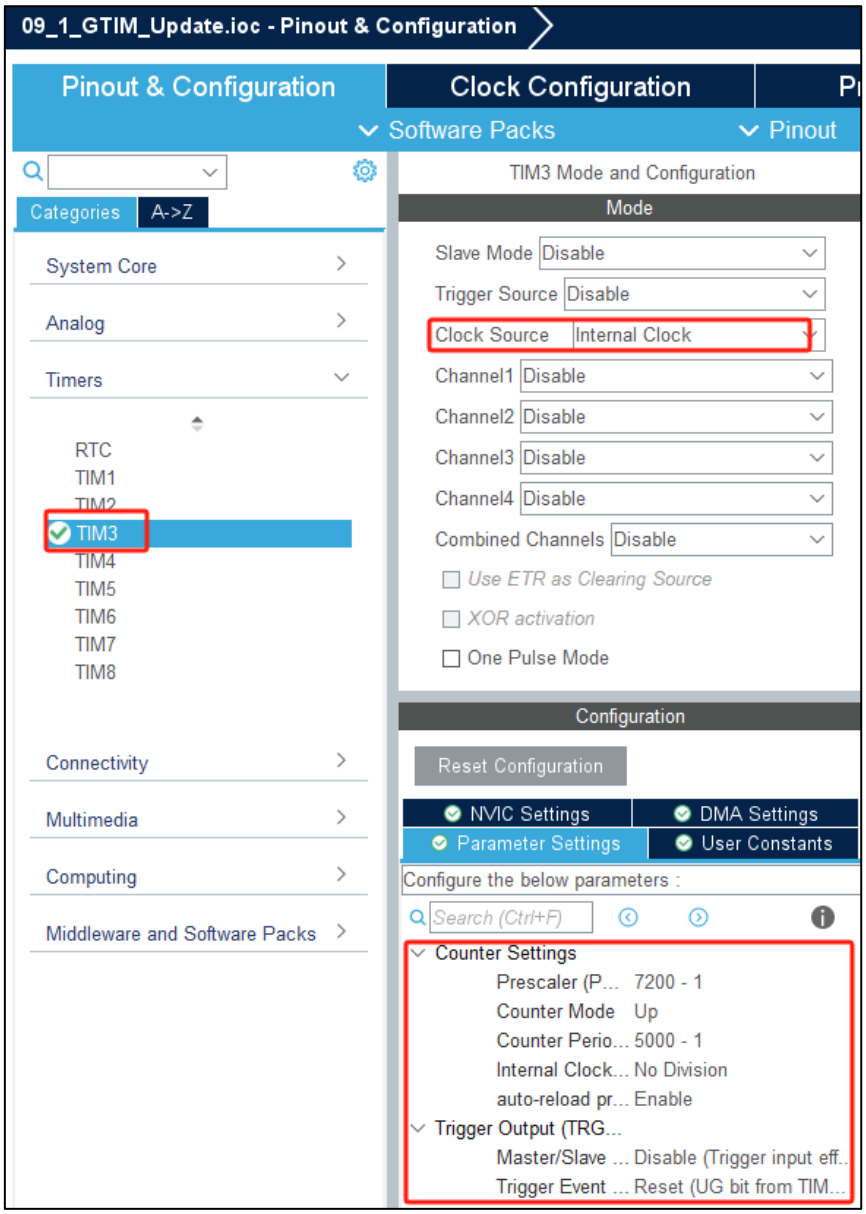


图 18.2.2.2 TIM3 参数配置

如上图，按照步骤配置通用定时器。Clock Source 我们选择 Internal Clock，表示使用内部时钟，即来自 APB1 的时钟。然后配置预分频器寄存器(TIMx_PSC)的值为 7200-1。计数模式是向上递增计数，前面我们提到基本定时器的计数模式只有递增（向上）计数模式，而通用定时器具有向上或者向下计数模式。对于向上计数模式，复位后计数器从 0 重新开始计数对于向下计数模式，复位后计数器从 ARR 开始向下递减计数。自动重载寄存器 (TIMx_ARR) 的值为 5000-1。开启自动重装载模式。

定时器 3 的时钟频率是 72MHz，计算出计数器 CK_CNT 的时钟频率是

$$f_{CK_CNT} = \frac{f_{CK_PSC}}{PSC[15:0]+1}\frac{72MHz}{7200} = 10KHz$$

那么计数器计数 5000 次就会溢出产生中断，所以每次溢出时间是：

$$5000 * \frac{1}{10KHz} = 0.5s$$

其它选项就保持默认状态，不需要配置。

18.2.3 配置 NVIC

定时器要每 0.5 秒溢出产生中断，所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM3 定时器全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 1：

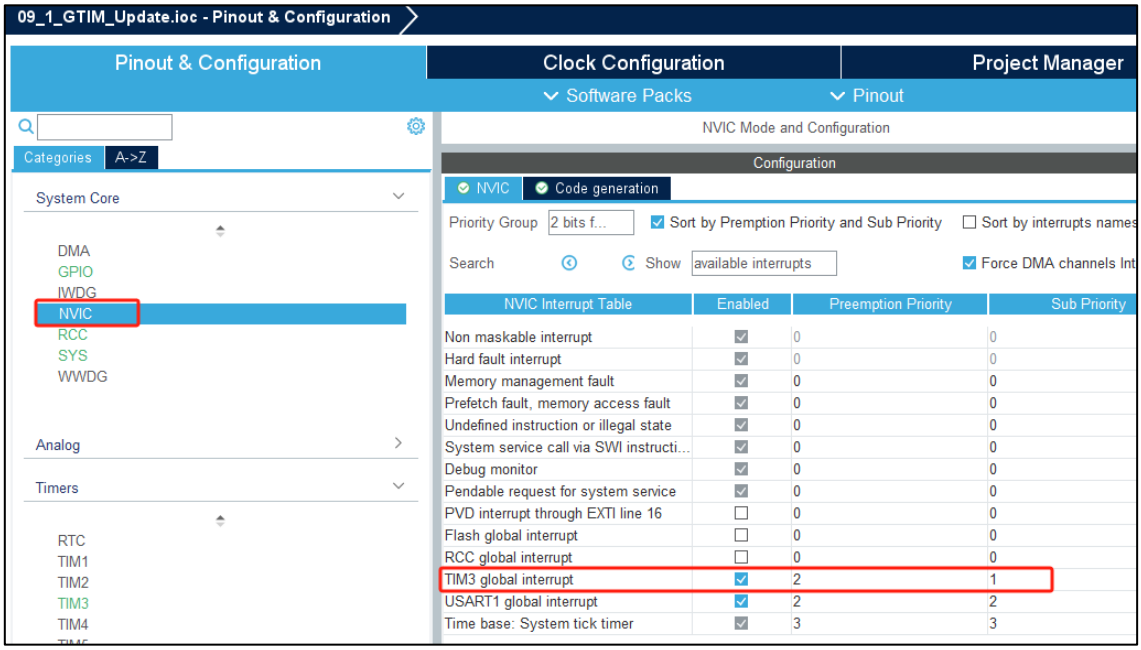


图 18.2.3.1 NVIC 配置

18.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM3 的初始化。

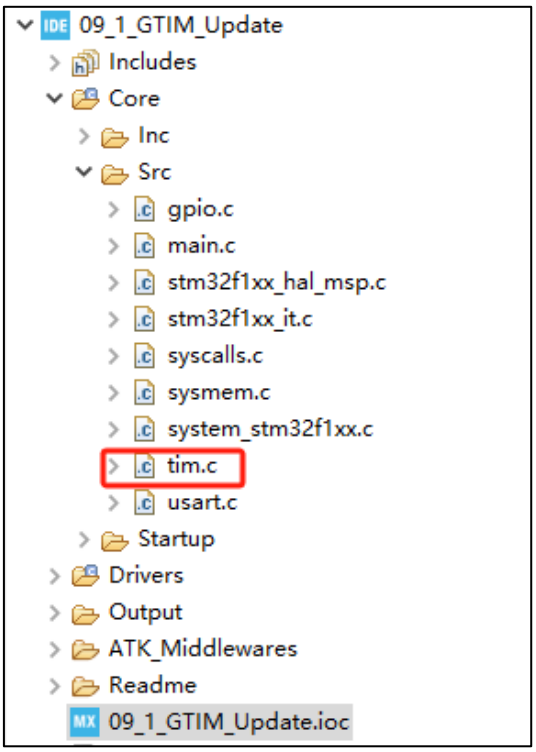


图 18.2.4.1 生成代码

18.2.5 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

```
void MX_TIM3_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 7200 - 1;
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 5000 - 1;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim3, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM3_Init 2 */
  HAL_TIM_Base_Start_IT(&htim3);        /* 使能定时器 x 和定时器 x 更新中断 */
  /* USER CODE END TIM3_Init 2 */
```

```
}

/* 省略部分代码… … */

/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3)
    {
        LED1_TOGGLE(); /* LED1 反转 */
    }
}
```

在 MX_TIM3_Init 函数中，我们通过前面的配置，设置 TIM3 每 500ms 溢出一次，并触发更新中断。同时，在该函数中调用 HAL_TIM_Base_Start_IT(&htim3)函数，用于使能 TIM3 并开启更新中断。在中断回调函数中，我们会判断是否为 TIM3 触发的更新中断，如果是，则执行一次 LED1 的翻转操作。

（2）修改 main.c 文件

```
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
  MX_TIM3_Init();
  while (1)
  {
      LED0_TOGGLE();
      delay_ms(200);
  }
}
```

在 main 函数中先初始化系统和用户的外设代码，然后在 wilhe(1)里每 200ms 翻转一次 LED0。实现的内容比较简单。在定时器中断回调函数中我们设置的每 500ms 中断一次，并翻转一次 LED1。

18.3 下载验证

在完成编译和烧录操作后，可以看到板子上的 LED0 和 LED1 都在闪烁，但闪烁频率不同，LED0 每间隔 200 毫秒改变一次状态，LED1 每隔 500 毫秒改变一次状态。

第十九章 通用定时器 PWM 输出实验

本章将介绍使用 STM32F103 的通用定时器输出 PWM，通过本章的学习，读者将学习到通用定时器输出比较的使用。

本章分为如下几个小节：

19.1 硬件设计
19.2 程序设计
19.3 下载验证

19.1 硬件设计

1，例程功能

使用 TIM3 通道 2（由 PB5 复用）输出 PWM， PB5 引脚连接了 LED0，从而实现 PWM 输出控制 LED0 亮度。

2，硬件资源

1）LED 灯
LED0 - PB5
2）定时器 3 输出通道 2（由 PB5 复用）

3，原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 LED0 来间接指示定时器的 PWM 输出情况。

19.2 程序设计

19.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\09_2_GTIM_PWM。

19.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 09_2_GTIM_PWM，以进行下一步的实验操作。如下图所示：

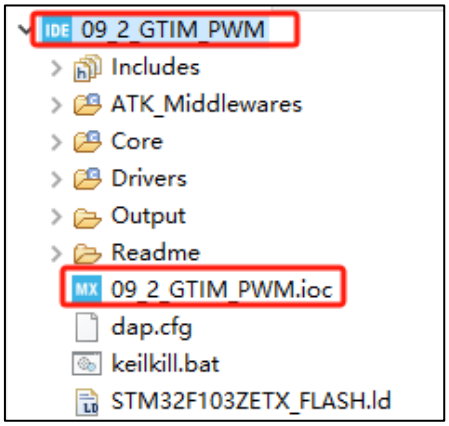


图 19.2.2.1 GTIM 工程

接着双击 09_2_GTIM_PWM.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PB5。需注意，配置 PB5 复用为 TIM3_CH2，如下图所示：

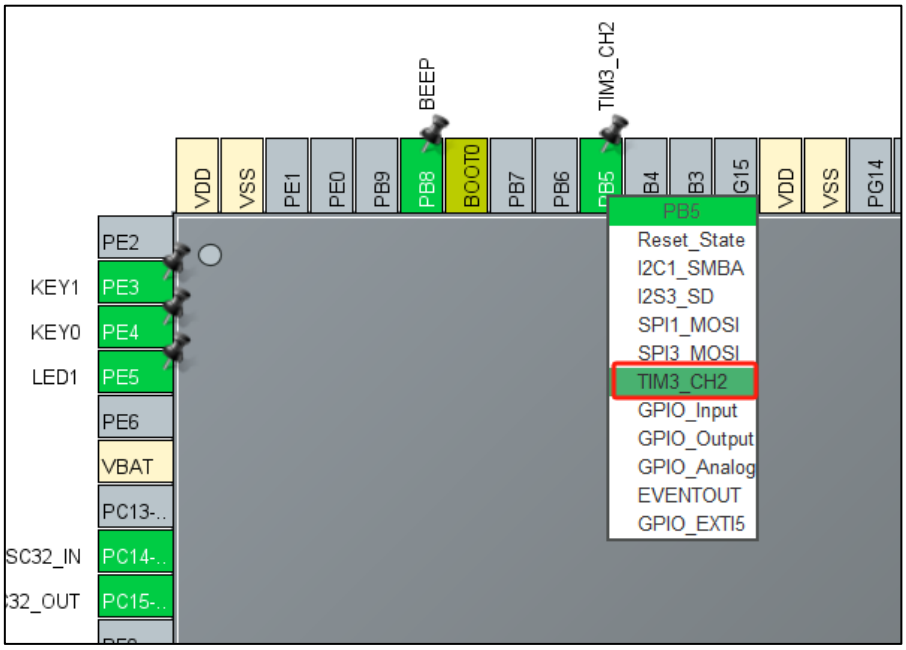


图 19.2.2.1 引脚复用

接着打开左边的 Timers→TIM3，配置 TIM3 参数，配置如下，Clock Source 选择 Internal Clock，表示选择内部时钟，Channel2 选择 PWM Generation CH2，表示使用 TIM3 的通道 2 产生 PWM 波形，其它选项保持默认配置。

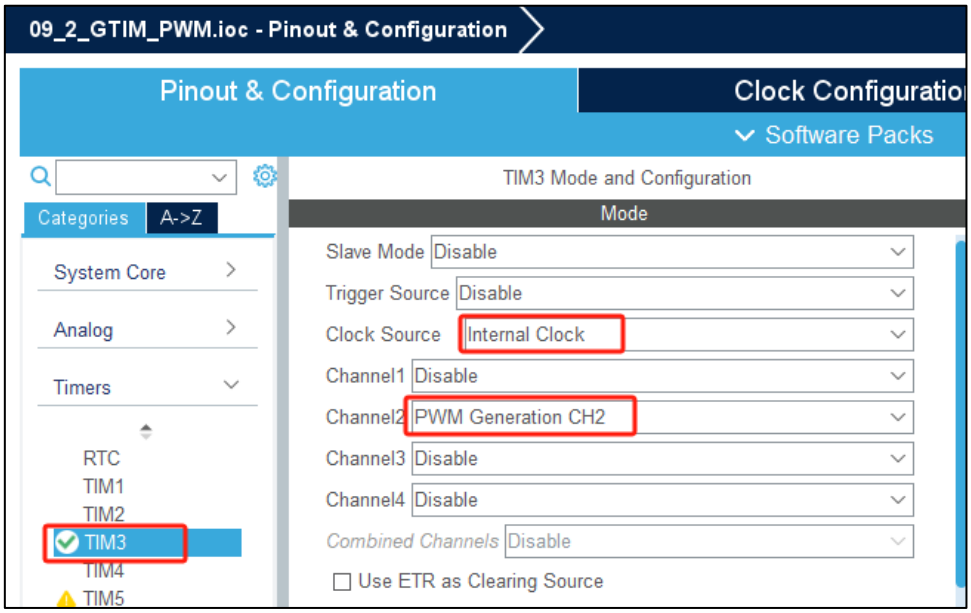


图 19.2.2.2 TIM3 内部时钟

在 Parameter Settings 处配置参数如下：

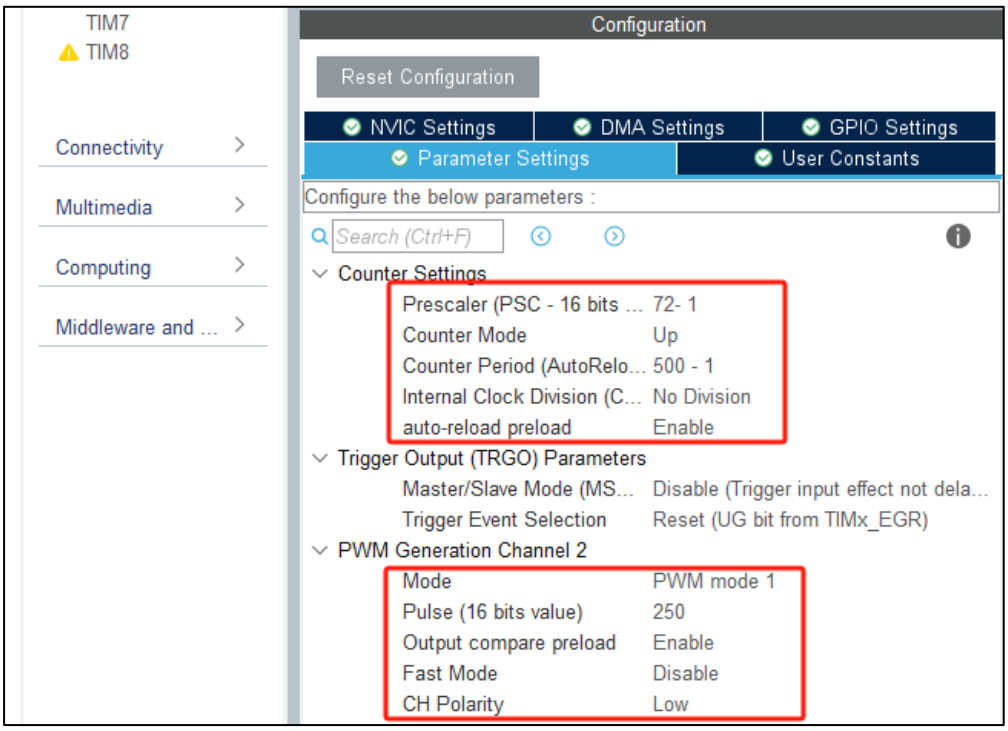


图 19.2.2.2 TIM3 参数配置

Counter Settings 用于配置计数器的参数：

- Prescaler：配置定时器的分频系数，这里配置 72-1；
- Counter Mode：定时器的计数模式，我们选择 UP，即向上计数模式；
- Counter Period：计数周期，即自动重装载值，也就是装在 TIM3_ARR 的值，这里配置为 500-1；
- Internal Clock Division (CKD)：内部时钟分频因子，这里就不设置分频了；
- auto-reload preload：这里设置为 Enable，定时器自动重装载使能，即使能 TIMx_ARR 寄存器进行缓冲；

上述参数可以计算定时器的时钟频率为：

$$\frac{72MHz}{(72-1)+1} = 1MHz$$

TIM3 的溢出时间：

$$\frac{500}{1MHz} = 0.0005s$$

- **Trigger Output(TRGO) Parameters** 用于配置触发输出（TRGO）参数，我们这里不配置。

PWM Generation Channel2 用于配置通道 2 的参数，其中：

- Mode：用于配置 PWM 的模式，这里选择 PWM mode 1，即 PWM 模式 1。另外还有 PWM 模式 2，可以理解 PWM mode l 是与 PWM mode 2 模式互补的波，PWM 模式 1 为高电平时 PWM 模式 2 为低电平，反之亦然。
- Pulse (32 bits value)：是占空比值，即 TIM3_CCR2 的值，也就是有效电平的值，可以配置在 0-500 之间，例如配置 0。这里配置 250，即占空比为 50%。在后面的实验中，我们会对 TIMx_CCR2 寄存器写入新的值来改变占空比，从而控制 LED 逐渐点亮和熄灭。
- Output compare preload：输出比较预加载项选择 Enable，即在定时器工作时是否能修改 Pulse 的值，如果禁用此项，表示定时器工作时不能进行修改，只能等到更新事件到来的时候才能进行修改，所以这里**选择使能。**
- CH Polarity：输出极性，这里我们选择 Low（LED0 是低电平有效）。

19.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM3 的初始化。

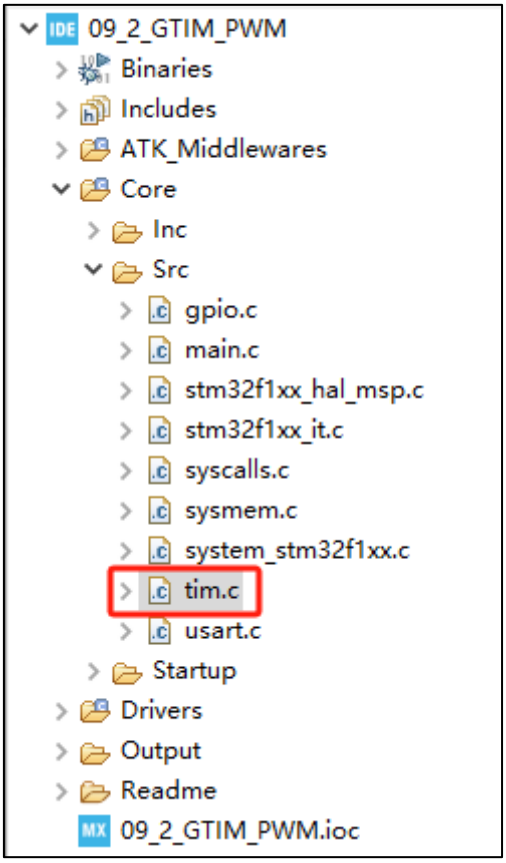


图 19.2.3.1 生成代码

19.2.4 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

```
void MX_TIM3_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 72- 1;
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 500 - 1;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim3, &sClockSourceConfig) != HAL_OK)
  {
```

```
    Error_Handler();
  }
  if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 250;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_LOW;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM3_Init 2 */
  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);        /* 开启对应 PWM 通道 */
  /* USER CODE END TIM3_Init 2 */
  HAL_TIM_MspPostInit(&htim3);

}
  /* 省略部分代码… … */
```

上述函数用于初始化 TIM3，配置参数在前面已经介绍过了，这里不在赘述。我们主要在该函数中添加了函数 HAL_TIM_PWM_Start，该函数用于使能 TIM3 以及使能 PWM 通道 TIM3_CH2 输出。

（2）修改 main.c 文件

```
int main(void)
{
  uint16_t ledrpwmval = 0;
  uint8_t dir = 1;
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM3 初始化 */
  MX_TIM3_Init();

  while (1)
  {
      delay_ms(10);

      if (dir)ledrpwmval++;                /* dir==1 ledrpwmval 递增 */
      else ledrpwmval--;                   /* dir==0 ledrpwmval 递减 */

      if (ledrpwmval > 300)dir = 0;        /* ledrpwmval 到达 300 后，方向为递减 */
      if (ledrpwmval == 0)dir = 1;         /* ledrpwmval 递减到 0 后，方向改为递增*/
```

```
        /* 修改比较值控制占空比 */
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, ledrpwmval);
    }
}
```

从上面的代码中可以看到，在初始化完 TIM3 输出 PWM 后，就不断地改变 TIM3 通道 2 的比较值，以达到改变 PWM 占功比的目的。又因为 PWM 由 PB5 引脚输出，PB5 引脚连接至 LED0，所以 LED0 的亮度也会随之发生变化，从而实现呼吸灯的效果。

19.3 下载验证

在完成编译和烧录后，可以看到板子上的 LED0 先由暗再逐渐变亮，以此循环，实现了呼吸灯的效果。

第二十章 通用定时器输入捕获实验

本章将介绍使用 STM32F103 通用定时器的输入捕获功能。通过本章的学习，读者将学习到通用定时器输入捕获的使用。

本章分为如下几个小节：

20.1 硬件设计
20.2 程序设计
20.3 下载验证

20.1 硬件设计

1，例程功能

1、使用 TIM5_CH1 来做输入捕获，捕获 PA0 上的高电平脉宽，并将脉宽时间通过串口打印出来，然后通过按 WK_UP 按键，模拟输入高电平，例程中能测试的最长高电平脉宽时间为：4194303 us。

2、LED0 闪烁指示程序运行。

2，硬件资源

1）LED 灯：
LED0 – PB5
2）独立按键：WK_UP - PA0
3）定时器 5，使用 TIM5 通道 1，将 PA0 复用为 TIM5_CH1。

3，原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们借助 WK_UP 做输入脉冲源并通过串口上位机来监测定时器输入捕获的情况。

20.2 程序设计

20.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\09_3_GTIM_CAP。

20.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 09_3_GTIM_CAP，以进行下一步的实验操作。如下图所示：

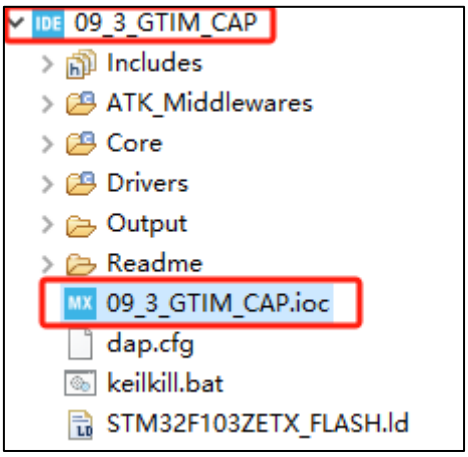


图 20.2.2.1 GTIM 工程

接着双击 09_3_GTIM_CAP.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PA0。需注意，配置 PA0 复用为 TIM5_CH1，如下图所示：

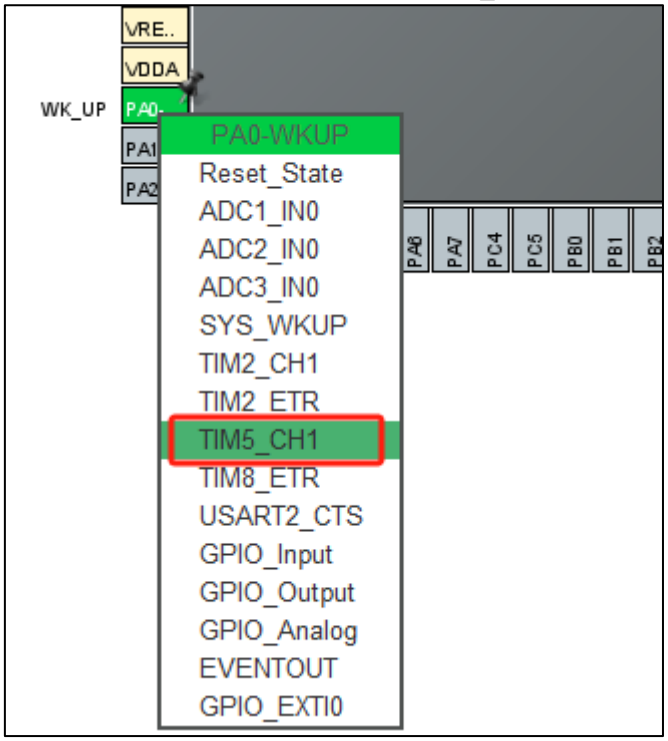


图 20.2.2.1 GPIO 复用配置

因为按键 WKUP 是高电平有效的，所以还需将 PA0 配置为下拉，User Label 配置为 WK_UP 保持一致，如下图所示：

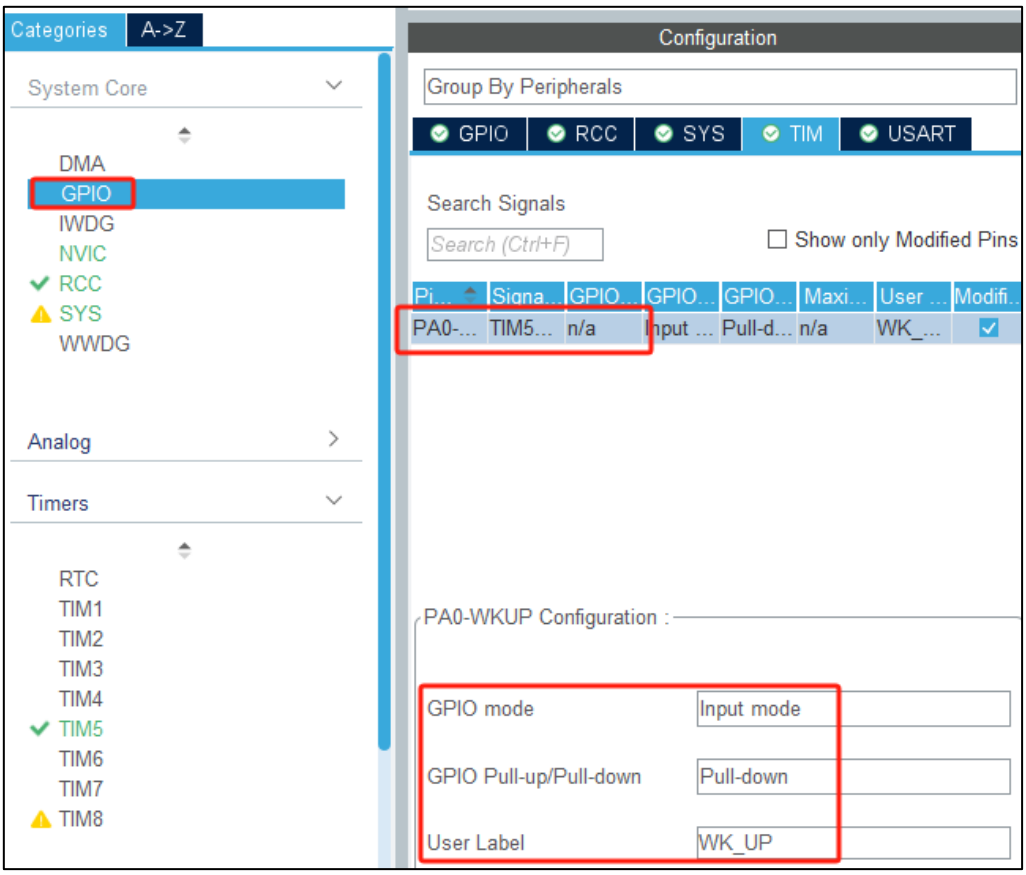


图 20.2.2.2 GPIO 配置

20.2.3 配置 TIM5 时基和捕获参数

接着打开左边的 Timers→TIM5，配置 TIM5 参数，配置如下，勾选 Internal Clock，表示选择内部时钟，Channel1 选择 Input Capture direct mode，表示使用 TIM5 的通道 1 输入捕获模式，其它选项保持默认配置。

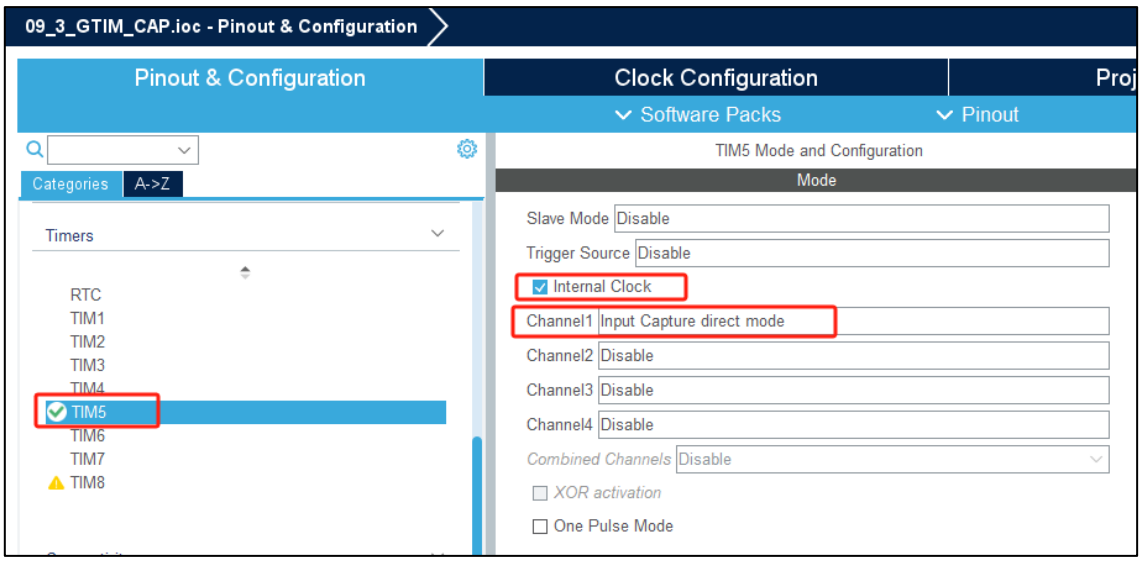


图 20.2.3.1 TIM5 时钟配置

在 Parameter Settings 处配置参数如下：

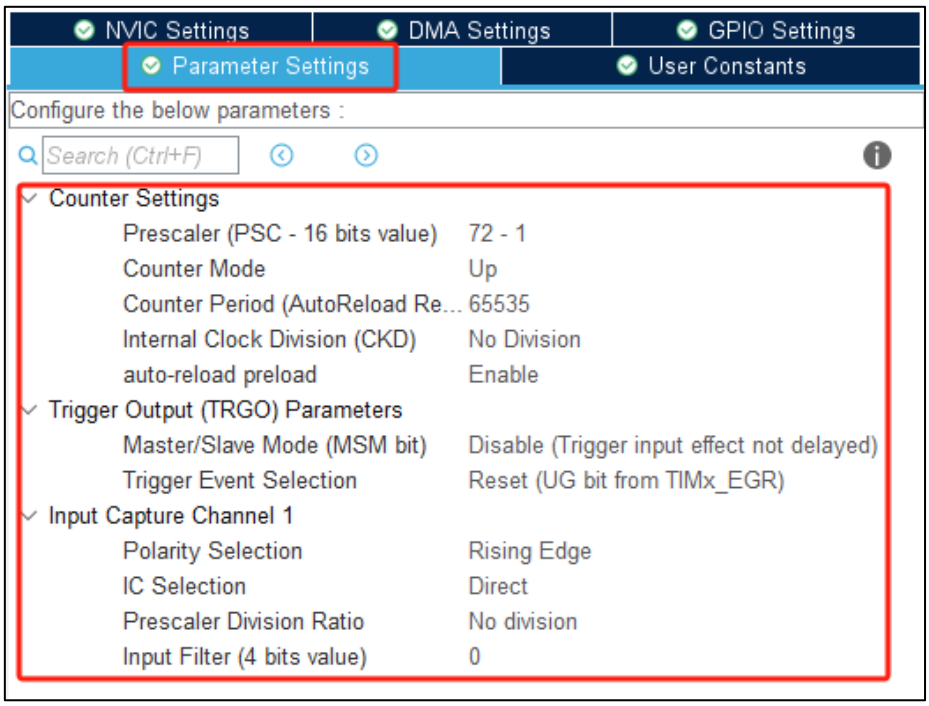


图 20.2.3.2 TIM5 通道一配置

上图的参数介绍如下：

Counter Settings（计数器）配置如下：

- Prescaler 用于配置定时器预分频值，这里配置为 72-1；
- Counter Mode 用于配置计数模式，我们选择向上计数 Up；
- Counter Period 用于配置定时器自动重装载值，我们设置为 65536-1（或者写为 0Xffff-1），即只是用了低 16 位。也可以设置最大为 0Xffffffff-1，因为 TIM5 是 32 位的。
- Internal Clock Division (CKD)配置为 No Division 即内部时钟不分频；
- auto-reload preload 用于配置自动重载是否使能，我们选择 Enable 使能自动重载，上述参数可以计算定时器的时钟频率为：

$$\frac{72MHz}{(72-1)+1} = 1MHz$$

即计数器每计数一下（一个节拍），时间为 1us。

Trigger Output (TRGO)Parameters 参数配置如下：

- Master/Slave Mode(MSM bit)用于配置主/从模式，这里不使用主从模式，所以配置为 Disable(Trigger input effect not delayed)；
- Trigger Event Selection TRGO 用于配置触发事件选择，这里选 Reset(UG bit from TIMX_EGR)

Input Capture Channel1 参数配置如下：

- Polarity Selection 用于配置极性选择，这里配置上升沿；
- IC Selection 配置为 Direct，即配置 IC1 直接映射在 TI1 上；
- Prescaler Division Ratio 用于配置配置输入分频，这里选择为 No division，即不分频；
- Input Filter (4 bits value) 配置输入滤波器，这里配置为 0，即不滤波。

20.2.4 配置 NVIC

本实验要用到定时器输入捕获中断来完成捕获功能。所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM5 定时器全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 1：

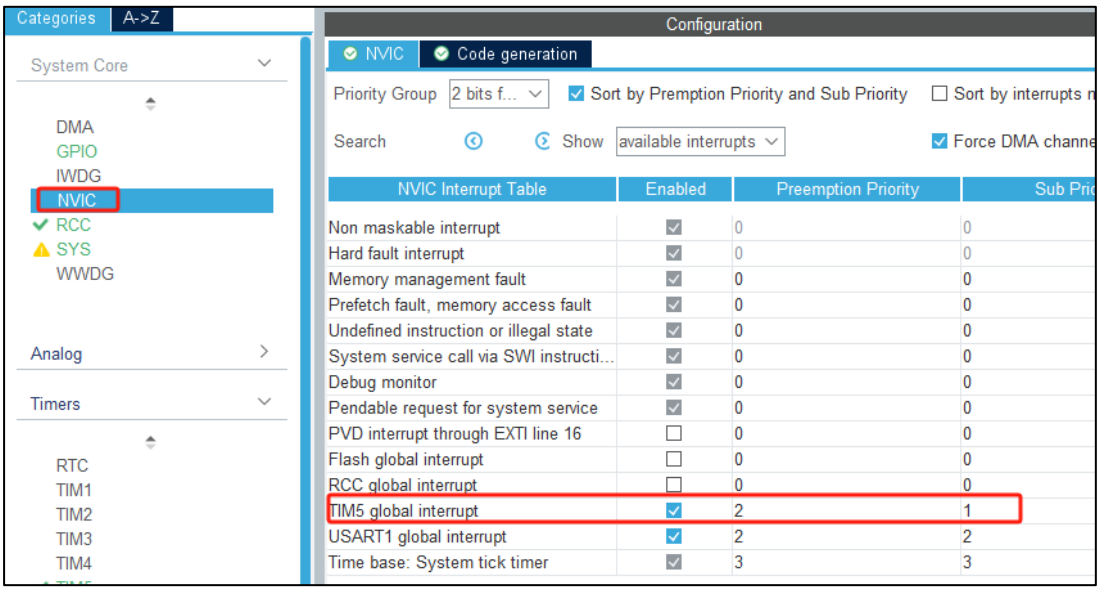


图 20.2.4.1 NVIC 配置

20.2.5 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM5 的初始化。

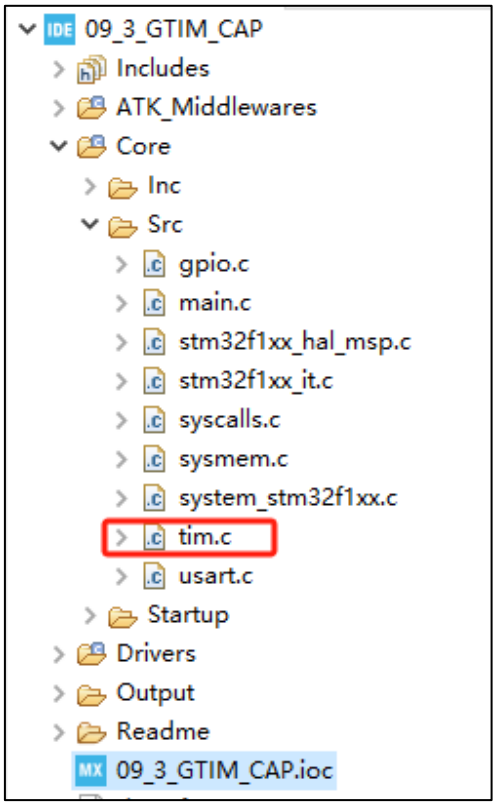


图 20.2.5.1 生成代码

20.2.6 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

```
void MX_TIM5_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_IC_InitTypeDef sConfigIC = {0};

  htim5.Instance = TIM5;
  htim5.Init.Prescaler = 72 - 1;
  htim5.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim5.Init.Period = 65535;
  htim5.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim5.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim5) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim5, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_IC_Init(&htim5) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim5, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
  sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
  sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
  sConfigIC.ICFilter = 0;
  if (HAL_TIM_IC_ConfigChannel(&htim5, &sConfigIC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM5_Init 2 */
  __HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);        /* 使能更新中断 */
  HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_1);        /* 开始捕获 TIM5 的通道 1 */
  /* USER CODE END TIM5_Init 2 */

}

  /* 省略部分代码… … */

/* 输入捕获状态(g_timxchy_cap_sta)
 * [7]  :0,没有成功的捕获;1,成功捕获到一次.
 * [6]  :0,还没捕获到高电平;1,已经捕获到高电平了.
 * [5:0]:捕获高电平后溢出的次数,最多溢出 63 次,所以最长捕获值 = 63*65536 + 65535 =
4194303
 *       注意:为了通用,我们默认 ARR 和 CCRy 都是 16 位寄存器,对于 32 位的定时器(如:TIM5),也
只按 16 位使用
 *       按 1us 的计数频率,最长溢出时间为:4194303 us, 约 4.19 秒
 *
 *      (说明一下：正常 32 位定时器来说,1us 计数器加 1,溢出时间:4294 秒)
 */
uint8_t g_timxchy_cap_sta = 0;    /* 输入捕获状态 */
uint16_t g_timxchy_cap_val = 0;   /* 输入捕获值 */
```

```
/**
 * @brief       定时器输入捕获中断处理回调函数
 * @param       htim:定时器句柄指针
 * @note        该函数在 HAL_TIM_IRQHandler 中会被调用
 * @retval      无
 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM5)
    {
        if ((g_timxchy_cap_sta & 0X80) == 0)        /* 还未成功捕获 */
        {
            if (g_timxchy_cap_sta & 0X40)           /* 捕获到一个下降沿 */
            {
                g_timxchy_cap_sta |= 0X80;          /* 标记成功捕获一次高电平脉宽 */
                g_timxchy_cap_val = HAL_TIM_ReadCapturedValue(&htim5,
                 TIM_CHANNEL_1);                    /* 获取当前的捕获值 */
                 /* 一定要先清除原来的设置 */
                TIM_RESET_CAPTUREPOLARITY(&htim5, TIM_CHANNEL_1);
                TIM_SET_CAPTUREPOLARITY(&htim5, TIM_CHANNEL_1,
                 TIM_ICPOLARITY_RISING);            /* 配置 TIM5 通道 1 上升沿捕获 */
            }
            else /* 还未开始,第一次捕获上升沿 */
            {
                g_timxchy_cap_sta = 0;              /* 清空 */
                g_timxchy_cap_val = 0;
                g_timxchy_cap_sta |= 0X40;              /* 标记捕获到了上升沿 */
                __HAL_TIM_DISABLE(&htim5);              /* 关闭定时器 5 */
                __HAL_TIM_SET_COUNTER(&htim5, 0);       /* 定时器 5 计数器清零 */
                 /* 一定要先清除原来的设置！！ */
                TIM_RESET_CAPTUREPOLARITY(&htim5, TIM_CHANNEL_1);
                TIM_SET_CAPTUREPOLARITY(&htim5, TIM_CHANNEL_1,
                 TIM_ICPOLARITY_FALLING);   /* 定时器 5 通道 1 设置为下降沿捕获 */
                __HAL_TIM_ENABLE(&htim5);   /* 使能定时器 5 */
            }
        }
    }
}

/**
 * @brief       定时器更新中断回调函数
 * @param        htim:定时器句柄指针
 * @note        此函数会被定时器中断函数共同调用的
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM5)
    {
        if ((g_timxchy_cap_sta & 0X80) == 0)            /* 还未成功捕获 */
        {
            if (g_timxchy_cap_sta & 0X40)               /* 已经捕获到高电平了 */
            {
                if ((g_timxchy_cap_sta & 0X3F) == 0X3F) /* 高电平太长了 */
                {
                    /* 一定要先清除原来的设置 */
                    TIM_RESET_CAPTUREPOLARITY(&htim5, TIM_CHANNEL_1);
                    TIM_SET_CAPTUREPOLARITY(&htim5, TIM_CHANNEL_1,
                    TIM_ICPOLARITY_RISING);     /* 配置 TIM5 通道 1 上升沿捕获 */
                    g_timxchy_cap_sta |= 0X80; /* 标记成功捕获了一次 */
                    g_timxchy_cap_val = 0XFFFF;
                }
```

```
                else                                    /* 累计定时器溢出次数 */
                {
                    g_timxchy_cap_sta++;
                }
            }
        }
    }
}
```

该文件中，首先在 MX_TIM5_Init 函数添加了两个函数：__HAL_TIM_ENABLE_IT 和 HAL_TIM_IC_Start_IT，一个用于使能更新中断，一个用于开启 TIM5 通道一的捕获并使能捕获中断。

另外两个中断回调函数中，就是处理我们捕获的主要逻辑了，在 TIM5 的中断回调函数中会依次捕获输入信号的上升沿和下降沿，并在第一次捕获到信号上升沿的时候清空 TIM5 的计数值。随后在捕获到信号下降沿的时候读取 TIM5 的计数值，该值就是该输入信号高电平脉宽对应的计数值，只要根据 TIM5 的计数频率，就能够计算出输入信号高电平脉宽的时间。TIM5 的更新中断是用于处理计数溢出的。

（2）修改 main.c 文件

```
int main(void)
{
  uint32_t temp = 0;
  uint8_t t = 0;
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM5 初始化 */
  MX_TIM5_Init();

  while (1)
  {
      if (g_timxchy_cap_sta & 0X80)         /* 成功捕获到了一次高电平 */
      {
          temp = g_timxchy_cap_sta & 0X3F;
          temp *= 65536;                    /* 溢出时间总和 */
          temp += g_timxchy_cap_val;        /* 得到总的高电平时间 */
          printf("HIGH:%lu us\r\n", temp); /* 打印总的高点平时间 */
          g_timxchy_cap_sta = 0;            /* 开启下一次捕获*/
      }

      t++;

      if (t > 20)                           /* 200ms 进入一次 */
      {
          t = 0;
          LED0_TOGGLE();                    /* LED0 闪烁 ,提示程序运行 */
      }
      delay_ms(10);
  }
```

```
}
```

经过前面的配置我们计算出 TIM5 的计数频率 1MHZ，即 1us 计数一次，所以我们的捕获时间精度是 1us。这里可以知道定时器的溢出时间是 65536us。

While（1）无限循环通过判断 g_timxchy_cap_sta 的第 7 位，来获知有没有成功捕获到一次高电平，如果成功捕获，先计算总的高电平时间，再通过串口传输到电脑。

20.3 下载验证

在完成编译和烧录后，可以看到 LED0 在闪烁，说明程序已经正常在跑了，我们再打开串口调试助手，选择对应的串口端口，我这边是 COM15，然后按 KEY_UP 按键，可以看到串口打印的高电平持续时间，如图 20.3.1 所示：

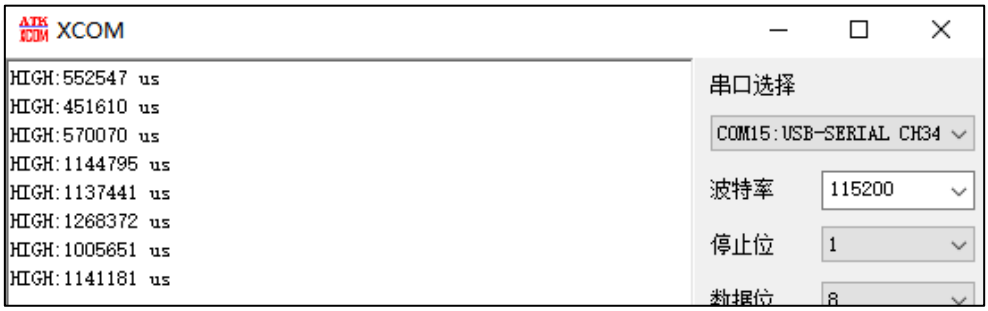


图 20.3.1 打印捕获到的高电平时间

第二十一章 通用定时器脉冲计数实验

本章将介绍使用 STM32F103 通用定时器对输入脉冲的个数进行计数。通过本章的学习，读者将学习到通用定时器从模式的使用。

本章分为如下几个小节：

21.1 硬件设计
21.2 程序设计
21.3 下载验证

21.1 硬件设计

1，例程功能

使用 TIM2_CH1 做输入捕获，我们将捕获 PA0 上的高电平脉宽，并将脉宽进行计数，通过串口打印出来。大家可以通过按 WK_UP 按键，输入高电平脉冲，通过按 KEY0 重设当前计数。LED0 闪烁，提示程序运行。

2，硬件资源

1）LED 灯
LED0 - PB5
2）独立按键：
KEY0 - PE4
WK_UP - PA0
3）定时器 2，使用 TIM2 通道 1，PA0 复用为 TIM2_CH1。

3，原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们借助 WK_UP 做输入脉冲源，捕获 PA0 上的高电平脉宽，然后对脉宽进行计数并通过串口上位机打印出来。还可以通过按 KEY0 重设当前计数。

21.2 程序设计

21.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\09_4_GTIM_CNT。

21.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 09_4_GTIM_CNT，以进行下一步的实验操作。如下图所示：

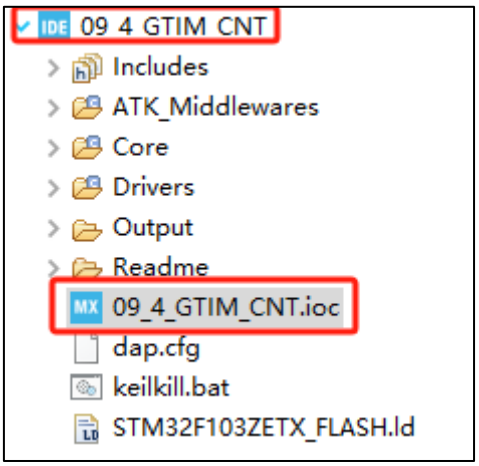


图 21.2.2.1 GTIM 工程

接着双击 09_4_GTIM_CNT.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PA0。需注意，配置 PA0 复用为 TIM2_CH1，如下图所示：

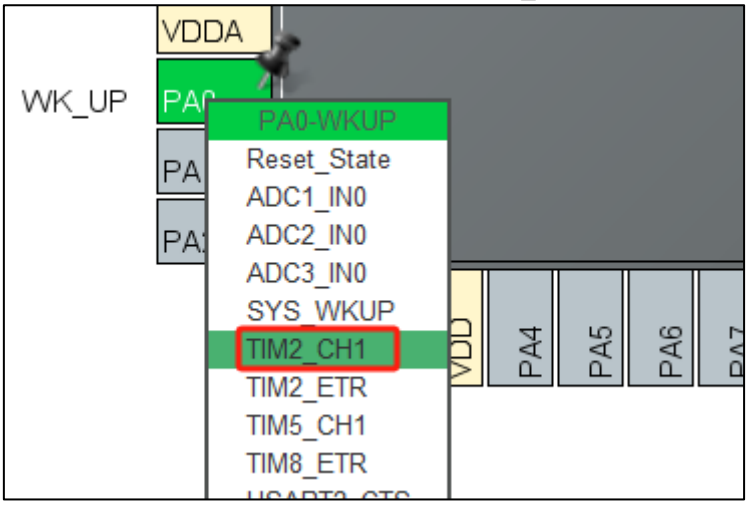


图 21.2.2.1 GPIO 复用配置

因为按键 WKUP 是高电平有效的，所以还需将 PA0 配置为下拉，User Label 配置为 WK_UP 保持一致，如下图所示：

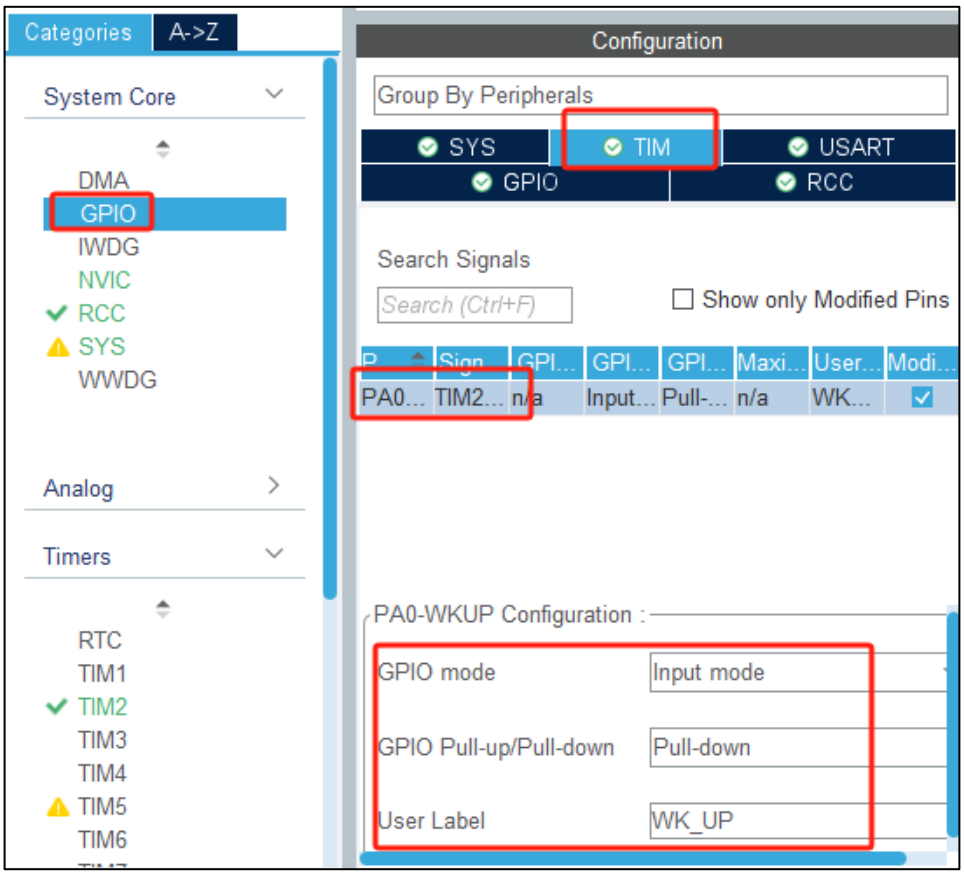


图 21.2.2.2 GPIO 配置

21.2.3 配置 TIM2 时基和捕获参数

接着打开左边的 Timers→TIM2，配置 TIM2 参数，配置如下：

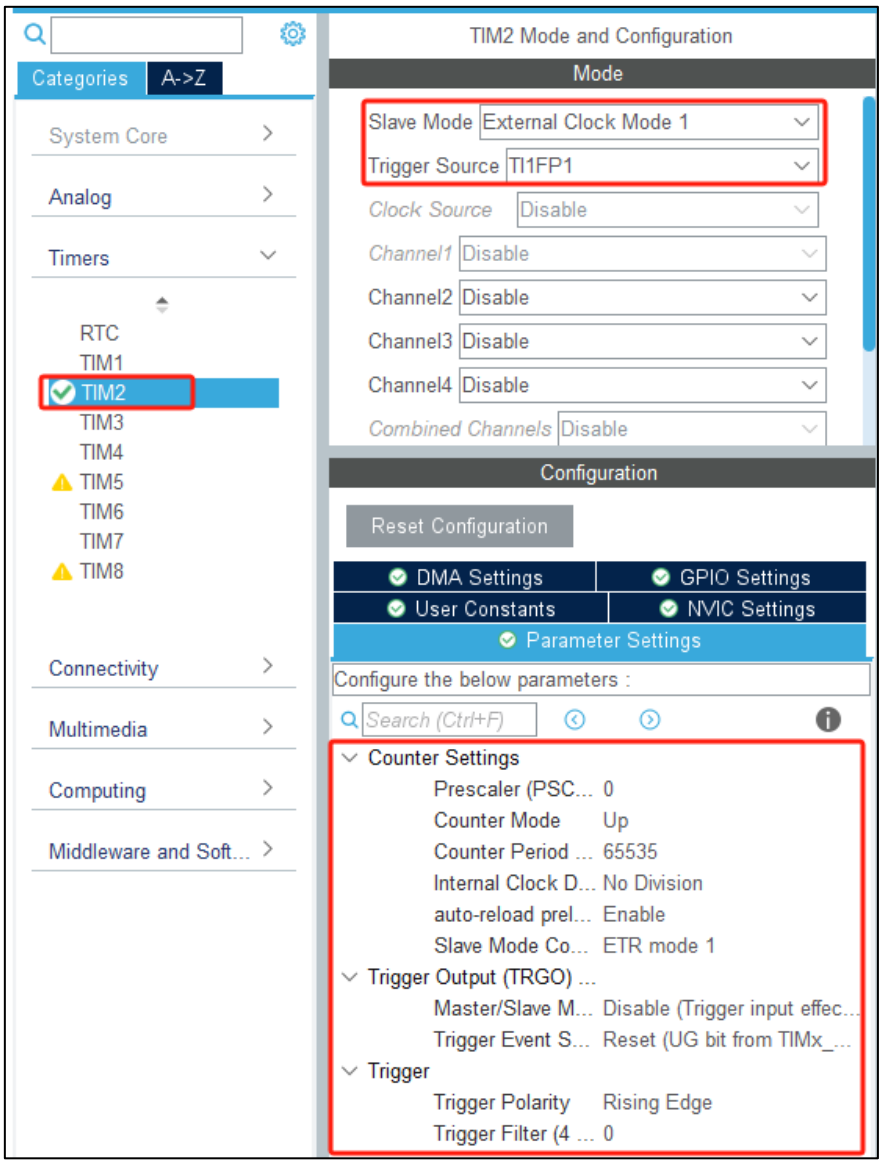


图 21.2.3.1 配置 TIM2 通道 1

上图的参数介绍如下：

定时器模式配置如下：

- Slave Mode 选择 External Clock Mode 1，即选择外部时钟模式 1；
- Trigger Source 表示触发源，我们选择 TI1FP1，表示触发信号来自滤波后的定时器输入 1(TI1FP1)；

Counter Settings 配置如下：

- Prescaler 用于配置定时器预分频值，这里配置为 0，表示每一个时钟都会计数一次，以提高精度；
- Counter Mode 用于配置计数模式，我们选择向上计数 Up；
- Counter Period 用于配置定时器自动重装载值，我们设置为 65536-1（或者写为 0XFFFF-1）；
- Internal Clock Division (CKD)配置为 No Division 即内部时钟不分频；
- auto-reload preload 用于配置自动重载是否使能，我们选择 Enable 使能自动重载；

Trigger Output (TRGO)Parameters 参数配置如下：

- Master/Slave Mode(MSM bit)用于配置主/从模式，这里是指是用一个定时器来触发另一个定时器，我们这里不用此模式。

- Trigger Event Selection TRGO 用于配置触发事件选择，这里选 Reset(UG bit from TIMX_EGR)

Trigger参数配置如下：

- Trigger Polarity 用于配置触发极性选择，这里配置上升沿 Rising Edge；
- Trigger Fiter (4 bits value)用于配置滤波，这里配置为 0，即不滤波。

21.2.4 配置 NVIC

本实验要用到定时器累计溢出次数，所以要用到定时器的更新中断。所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM2 定时器全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 1：

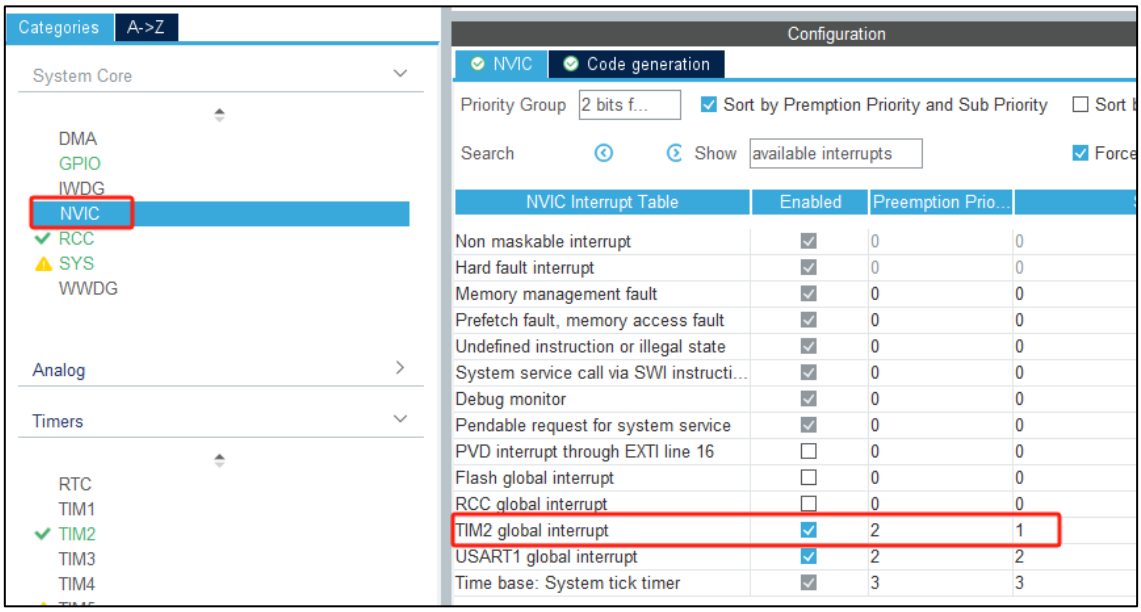


图 21.2.4.1 NVIC 配置

21.2.5 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM2 的初始化。

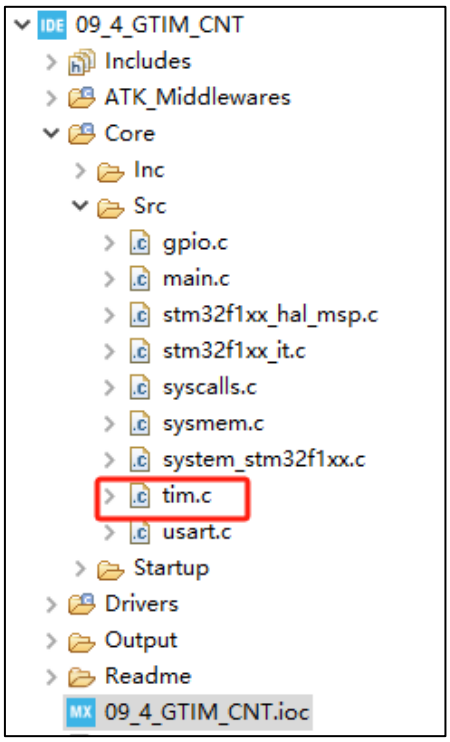


图 21.2.5.1 生成代码

21.2.6 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

```
void MX_TIM2_Init(void)
{
  TIM_SlaveConfigTypeDef sSlaveConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 0;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim2.Init.Period = 65535;
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sSlaveConfig.SlaveMode = TIM_SLAVEMODE_EXTERNAL1;
  sSlaveConfig.InputTrigger = TIM_TS_TI1FP1;
  sSlaveConfig.TriggerPolarity = TIM_TRIGGERPOLARITY_RISING;
  sSlaveConfig.TriggerFilter = 0;
  if (HAL_TIM_SlaveConfigSynchro(&htim2, &sSlaveConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM2_Init 2 */
```

```
  __HAL_TIM_ENABLE_IT(&htim2, TIM_IT_UPDATE);       /* 使能更新中断 */
  HAL_TIM_IC_Start(&htim2, TIM_CHANNEL_1);          /* 开始捕获 TIMx 的通道 y */
  /* USER CODE END TIM2_Init 2 */

}

/* 省略部分代码… … */

/* 记录定时器计数器的溢出次数，方便计算总脉冲个数 */
uint32_t g_timxchy_cnt_ofcnt = 0 ;                  /* 计数溢出次数 */

/**
 * @brief       通用定时器 TIMX 通道 Y 获取当前计数值
 * @param       无
 * @retval      当前计数值
 */
uint32_t gtim_timx_cnt_chy_get_count(void)
{
    uint32_t count = 0;
    count = g_timxchy_cnt_ofcnt * 65536;      /* 计算溢出次数对应的计数值 */
    count += __HAL_TIM_GET_COUNTER(&htim2);   /* 加上当前 CNT 的值 */
    return count;
}

/**
 * @brief       通用定时器 TIMX 通道 Y 重启计数器
 * @param       无
 * @retval      当前计数值
 */
void gtim_timx_cnt_chy_restart(void)
{
    __HAL_TIM_DISABLE(&htim2);              /* 关闭定时器 TIMX */
    g_timxchy_cnt_ofcnt = 0;                /* 累加器清零 */
    __HAL_TIM_SET_COUNTER(&htim2, 0);       /* 计数器清零 */
    __HAL_TIM_ENABLE(&htim2);               /* 使能定时器 TIMX */
}

/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄指针
 * @note        此函数会被定时器中断函数共同调用的
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        g_timxchy_cnt_ofcnt++;              /* 累计溢出次数 */
    }
}
```

在 TIM2 的初始化函数中，我， 添加了两个函数：__HAL_TIM_ENABLE_IT 函数和 HAL_TIM_IC_Start 函数，它们主要用于开启输入捕获以及开启更新中断。

gtim_timx_cnt_chy_restart 函数用于重启计数器，按下 KEY0 按键后就会调用此函数实现计数器清零；

gtim_timx_cnt_chy_get_count 用于计算计数器的值，通过此值可以知道计数器记录的高脉冲次数。程序运行后，先按下 KEY0 清零计数器，然后再按下 WK_UP 按键给 PA0 提供高电平脉冲，每按下一次 WK_UP 就给 PA0 一次高电平脉冲；

更新中断回调函数中，用于记录溢出次数。

（2）修改 main.c 文件

```
int main(void)
{
  uint32_t curcnt = 0;
  uint32_t oldcnt = 0;
  uint8_t key = 0;
  uint8_t t = 0;
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM2 初始化 */
  MX_TIM2_Init();
  gtim_timx_cnt_chy_restart();                  /* 重启计数 */

  while (1)
  {
      key = key_scan(0);                        /* 扫描按键 */

      if (key == KEY0_PRES)                     /* KEY0 按键按下,重启计数 */
      {
          gtim_timx_cnt_chy_restart();          /* 重新启动计数 */
      }

      curcnt = gtim_timx_cnt_chy_get_count(); /* 获取计数值 */

      if (oldcnt != curcnt)
      {
          oldcnt = curcnt;
          printf("CNT:%lu\r\n", oldcnt);        /* 打印脉冲个数 */
      }

      t++;

      if (t > 20)                               /* 200ms 进入一次 */
      {
          t = 0;
          LED0_TOGGLE();                        /* LED0 闪烁 ,提示程序运行 */
      }

      delay_ms(10);
  }
}
```

由于我们前面设置 TIM2 的预分频器为 0，也就是不分频，所以每个脉冲都会触发 TIM2 的计数值加 1，若 KEY0 按键被按下，则会清零 TIM2 的计数值；若 KEY_UP 按键被按下，则会触发 TIM2 的计数值加 1，同时会将新的计数值通过串口输出至串口调试助手。

21.3 下载验证

在完成编译和烧录后，连续按下 KEY_UP 按键，模拟产生多个脉冲，可以通过串口调试助手观察到 KEY_UP 按键被按下的次数，同时按下 KEY0 按键，可清零统计值。

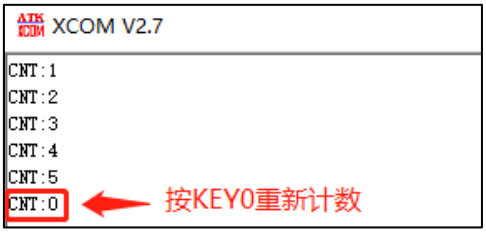


图 21.3.1 打印高电平脉冲次数

第二十二章 高级定时器输出指定个数 PWM 实验

本章将介绍使用 STM32F103 高级定时器输出指定个数的 PWM。通过本章的学习，读者将学习到高级定时器重复计数器的使用。

本章分为如下几个小节：

22.1 硬件设计
22.2 程序设计
22.3 下载验证

22.1 硬件设计

1， 例程功能

通过 TIM8_CH1（由 PC6 复用）输出 PWM，然后为了指示 PWM 的输出情况，我们用杜邦线将 PC6 和 PE5 引脚的排针连接起来，从而实现 PWM 输出控制 LED1（硬件已连接在 PPE5 引脚上）的亮灭。注意的点是：PE5 要设置成浮空输入，避免引脚冲突，我们在 main 函数中设置好了，请看源码。上电默认输出 5 个 PWM 波，连接好杜邦线后可以看见 LED1 亮灭五次。之后按一下按键 KEY0，就会输出 5 个 PWM 波控制 LED1 亮灭五次。LED0 闪烁提示系统正在运行。

2， 硬件资源

1）LED 灯：
LED0 – PB5
LED1 – PE5
2）独立按键：
KEY0 – PE4
3）定时器 8，使用 TIM8 通道 1，由 PC6 复用。用杜邦线将 PC6 和 PE5 引脚连接起来。

3， 原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们想通过 LED1 来指示 STM32F103 的定时器的 PWM 输出情况，所以需要用一根杜邦线连接 PC6 和 PE5，同时还用按键 KEY0 进行控制。

22.2 程序设计

22.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\10_1_ATIM_NPWM。

22.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 10_1_ATIM_NPWM，以进行下一步的实验操作。如下图所示：

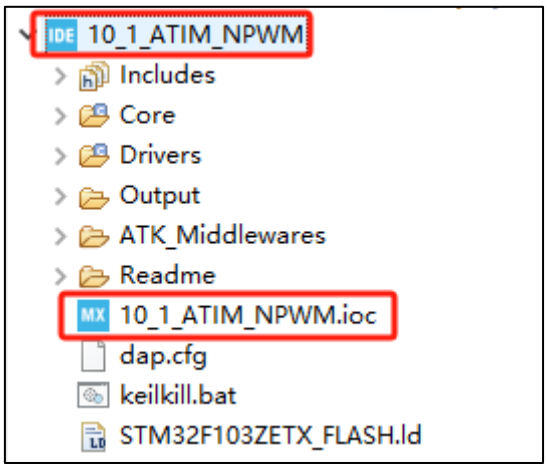


图 22.2.2.1 ATIM 工程

接着双击 10_1_ATIM_NPWM.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PC6。需注意，将 PC6 复用为 TIM8_CH1，如下图所示：

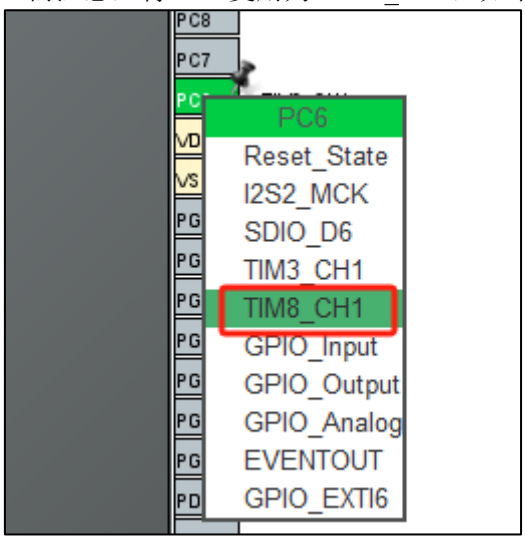


图 22.2.2.1 PC6 复用为 TIM8_CH1

22.2.3 配置 TIM8 时基等参数

在 Timers→TIM8 中配置 TIM8_CH1 参数，其中，模式配置如下：

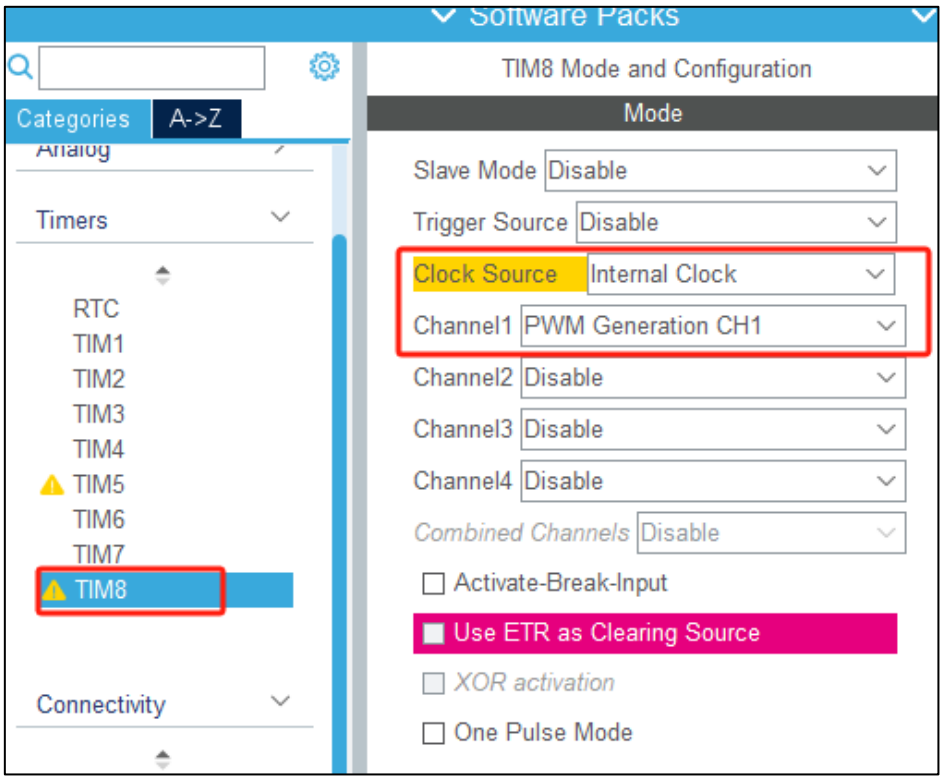


图 22.2.3.1 配置模式为 PWM 输出

这里选择定时器 8 的通道 1 生成 PWM 波形。配置完 TIM8_CH1 的模式后，我们配置 TIM8_CH1 的时基等参数，如下红框部分需要我们手动配置，其它部分就不需要配置了：

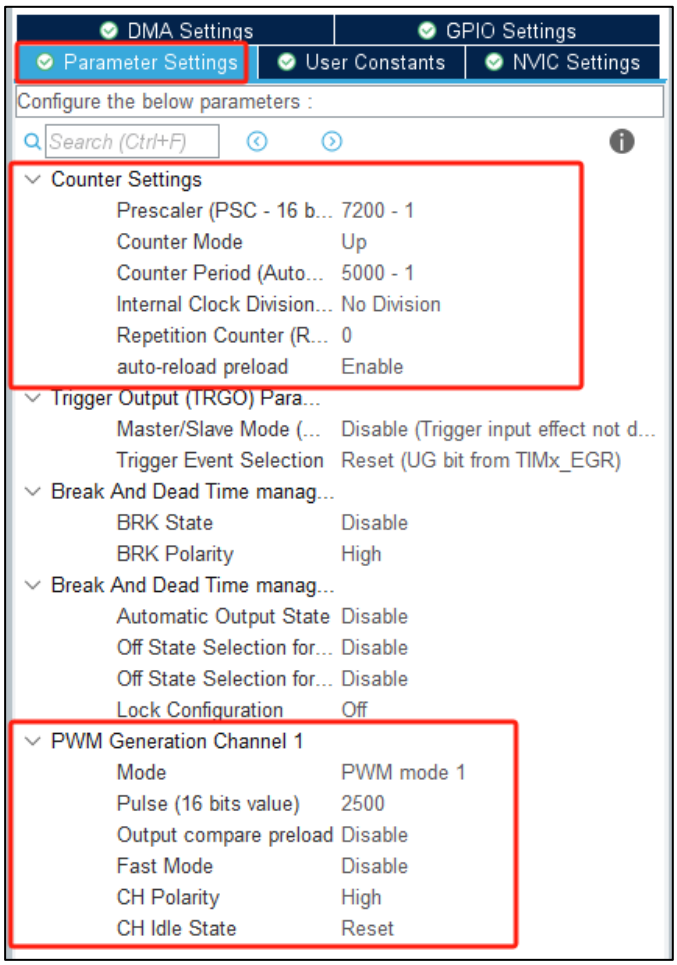


图 22.2.3.2 配置 TIM8 通道 1 参数

上图的配置参数，除了红框中的要配置以外，其它选项保持默认，本实验用不到刹车功能，所以就不会使能 BRK。配置部分的介绍如下：

Counter Settings（计数器配置）配置如下：

- Prescaler 用于配置定时器预分频值，这里配置为 7200-1；
- Counter Mode 用于配置计数模式，我们选择向上计数 Up；
- Counter Period 用于配置定时器自动重装载值，我们设置为 5000-1；
- Internal Clock Division (CKD)配置为 No Division 即时钟不分频；
- auto-reload preload 用于配置自动重载是否使能，我们选择 Enable 使能自动重载；

上述参数可以计算定时器的时钟频率为：

$$\frac{72MHz}{(7200-1)+1}=10KHz$$

计数器的溢出时间：

$$\frac{5000}{10kHz}=0.5\text{s}$$

- Repetition Counter(RCR-16 bits value)默认选择为 0，这里注意，高级定时器和基本定时器以及通用定时器不同，高级定时器的 TIMx_RCR 递减为 0 时才会发生更新事件。

PWM Generation Channel 1 用于配置通道 1 的参数，其中：

- Mode：用于配置 PWM 的模式，这里选择 PWM mode1，即 PWM 模式 1。另外还有 PWM 模式 2，可以理解 PWM mode 1 是与 PWM mode 2 模式互补的波，PWM 模式 1 为高电平时 PWM 模式 2 为低电平，反之亦然。
- Pulse (16 bits value)：是占空比值，也可以说是脉冲宽度，即 TIMx_CCR2 的值，也就是有效电平的值，可以配置在 0-5000 之间，例如配置 0。这里配置 2500，即占空比为 50%。

$$占空比=\frac{Pulse}{自动重载值}*100\%$$

- Output compare preload：输出比较预加载项选择 disable，即在定时器工作时是否能修改 Pulse 的值，如果禁用此项，表示定时器工作时不能进行修改，只能等到更新事件到来的时候才能进行修改。
- Fast Mode 用于配置 PWM 脉冲快速模式，这里我们也不需要，可以不配置。
- CH Polarity：输出极性，这里我们选择 High，即高电平有效（我们的蜂鸣器是低电平有效，即低电平时蜂鸣器发声）。
- CH Idle State 用于配置通道的空闲状态，也就是配置 PWM 不输出时的状态，这里默认选择 Reset，即 PWM 关闭状态下默认为低电平。如果配置为 Set 就是 PWM 关闭状态下默认为高电平。

22.2.4 配置 NVIC

本实验要用到定时器更新中断，所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM8 定时器全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 1：

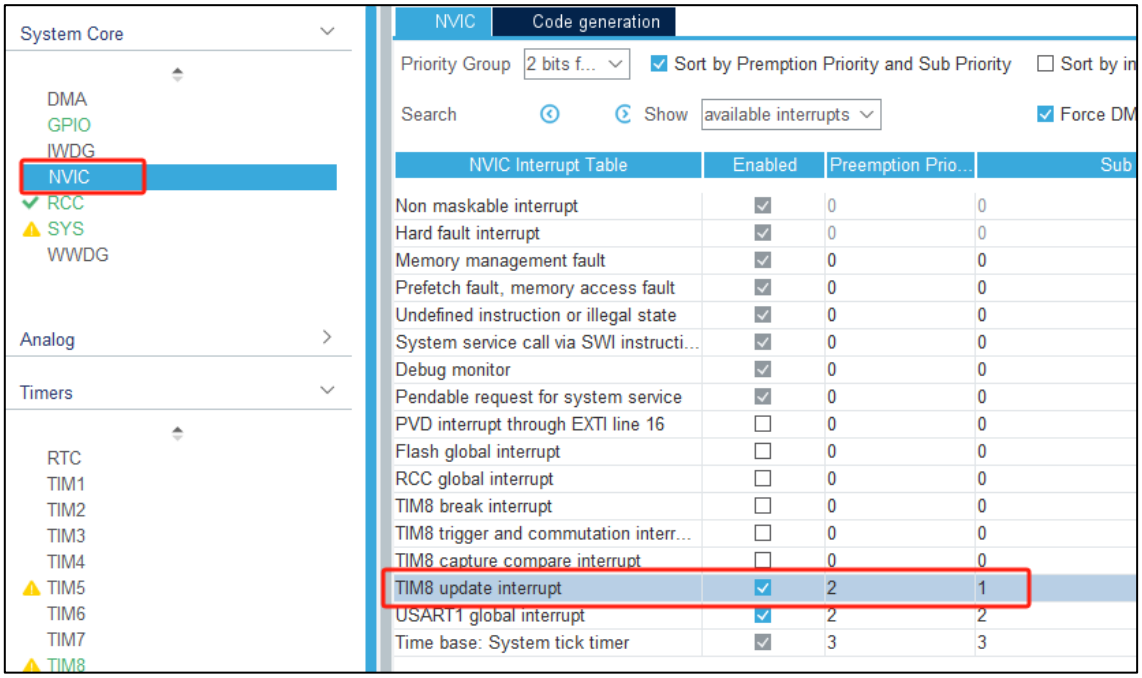


图 22.2.4.1 NVIC 配置

22.2.5 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM8 的初始化。

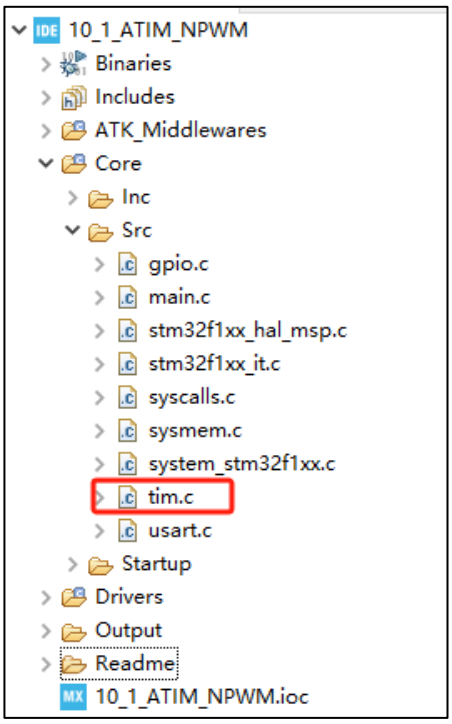


图 22.2.5.1 生成代码

22.2.6 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

如下，在标红的字体中间添加我们的代码：

```
/* USER CODE BEGIN 0 */
/* g_npwm_remain 表示当前还剩下多少个脉冲要发送
 * 每次最多发送 256 个脉冲
 */
volatile static uint32_t g_npwm_remain = 0;
/* USER CODE END 0 */

TIM_HandleTypeDef htim8;

/* TIM8 init function */
void MX_TIM8_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};
  htim8.Instance = TIM8;
  htim8.Init.Prescaler = 7200 - 1;
  htim8.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim8.Init.Period = 5000 - 1;
  htim8.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim8.Init.RepetitionCounter = 0;
  htim8.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim8) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
```

```
  if (HAL_TIM_ConfigClockSource(&htim8, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_Init(&htim8) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim8, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 2500;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
  sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
  if (HAL_TIM_PWM_ConfigChannel(&htim8, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  __HAL_TIM_DISABLE_OCxPRELOAD(&htim8, TIM_CHANNEL_1);
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
  sBreakDeadTimeConfig.DeadTime = 0;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
  sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
  if (HAL_TIMEx_ConfigBreakDeadTime(&htim8, &sBreakDeadTimeConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM8_Init 2 */

  HAL_TIM_PWM_Start(&htim8, TIM_CHANNEL_1);          /* 开启对应 PWM 通道 */

  /* USER CODE END TIM8_Init 2 */
  HAL_TIM_MspPostInit(&htim8);

}
  /* 省略部分代码… … */

/* USER CODE BEGIN 1 */

/**
 * @brief       高级定时器 TIMX NPWM 设置 PWM 个数
 * @param       rcr: PWM 的个数, 1~2^32 次方个
 * @retval      无
 */
void atim_timx_npwm_chy_set(uint32_t npwm)
{
    if (npwm == 0) return;

    g_npwm_remain = npwm;                           /* 保存脉冲个数 */
    __HAL_TIM_ENABLE_IT(&htim8, TIM_IT_UPDATE);     /* 数据修改后，在允许更新中断 */
    /* 产生一次更新事件,在中断里面处理脉冲输出 */
    HAL_TIM_GenerateEvent(&htim8, TIM_EVENTSOURCE_UPDATE);
    __HAL_TIM_ENABLE(&htim8);                       /* 使能定时器 TIMX */
}
/**
```

```
 * @brief       高级定时器 TIMX NPWM 中断服务函数
 * @param       无
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    uint16_t npwm = 0;
    if (g_npwm_remain >= 256)           /* 还有大于 256 个脉冲需要发送 */
    {
        g_npwm_remain = g_npwm_remain - 256;
        npwm = 256;
    }
    else if (g_npwm_remain % 256)       /* 还有位数（不到 256）个脉冲要发送 */
    {
        npwm = g_npwm_remain % 256;
        g_npwm_remain = 0;              /* 没有脉冲了 */
    }

    if (npwm) /* 有脉冲要发送 */
    {
        TIM8->RCR = npwm - 1;  /* 设置重复计数寄存器值为 npwm-1, 即 npwm 个脉冲 */
        /* 产生一次更新事件,在中断里面处理脉冲输出 */
        HAL_TIM_GenerateEvent(&htim8, TIM_EVENTSOURCE_UPDATE);
        __HAL_TIM_ENABLE(&htim8);       /* 使能定时器 TIMX */
    }
    else
    {
        /* 关闭定时器 TIMX，使用 HAL Disable 会清除 PWM 通道信息，此处不用 */
        TIM8->CR1 &= ~(1 << 0);
    }
}

/* USER CODE END 1 */
```

首先在 TIM8 的初始化函数中，调用函数 HAL_TIM_PWM_Start，用于开启 TIM8_CH1 的 PWM 输出；atim_timx_npwm_chy_set 函数用于用户设置要发送的 PWM 波个数；接着在更新中断回调函数 HAL_TIM_PeriodElapsedCallback 中，才是实现我们的主要判断逻辑，分析如下：

因为高级定时器的重复计数寄存器只有低八位有效，因此需要在需要产生大于 255 个 PWM 的情况下做特殊处理，随后将需要产生的 PWM 个数值减 1 写入 TIM8 的重复计数寄存器，并且由于写入重复计数寄存器的新值只有在下一次更新事件发生时才生效，因此需要手动产生一次更新事件，最后使能 TIM8，这么一来，TIM8 就会持续地输出 PWM 直到重复计数寄存器的数值为 0 后，产生一次更新中断，若此时已经输出完了指定个数的 PWM，那么就会调用 TIM8->CR1 &= ~(1 << 0)关闭高级定时器。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key = 0;
    uint8_t t = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);
```

```
  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM8 初始化 */
  MX_TIM8_Init();
  /* USER CODE BEGIN 2 */
  atim_timx_npwm_chy_set(5);              /* 输出 5 个 PWM 波 */
  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY0_PRES)               /* KEY0 按下 */
      {
          atim_timx_npwm_chy_set(5);      /* 输出 5 个 PWM 波 */
      }

      t++;
      delay_ms(10);

      if (t > 50)                         /* 控制 LED0 闪烁, 提示程序运行状态 */
      {
          t = 0;
          LED0_TOGGLE();
      }
  }
}
```

经过前面的配置我们已经设置 PWM 的频率为 2Hz。初始化完 TIM8 后，我们会调用 atim_timx_npwm_chy_set(5)输出 5 个 PWM 波；然后就重复地扫描按键，若 KEY0 按键被按下，则输出 5 个 PWM。

22.3 下载验证

在完成编译和烧录操作后，此时可以通过示波器或外接 LED 的方式观察 PC6 引脚输出了 5 个频率为 2Hz，占空比为 50%的 PWM。按下 KEY0 按键，同样也会输出 5 个 PWM 波。

第二十三章 高级定时器输出比较模式实验

本章将介绍使用 STM32F103 的高级定时器输出多个频率、占空比相同但相位不同的 PWM。通过本章的学习，读者将学习到高级定时器输出比较的匹配时输出翻转模式的使用。

本章分为如下几个小节：

23.1 硬件设计
23.2 程序设计
23.3 下载验证

23.1 硬件设计

1， 例程功能

1. 定时器 8 通道 1、2、3、4 在匹配时输出翻转模式下分别输出相位为 25%、50%、75%、100%，占空比为 50%的 PWM
2. LED0 闪烁，指示程序正在运行

2， 硬件资源

1）LED 灯
LED0 – PB5
2）PC6 复用为 TIM8_CH1
PC7 复用为 TIM8_CH2
PC8 复用为 TIM8_CH3
PC9 复用为 TIM8_CH4

3， 原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们需要通过示波器观察 PC6、PC7、PC8 和 PC9 引脚 PWM 输出的情况。

23.2 程序设计

23.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\10_2_ATIM_COMP。

23.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 10_2_ATIM_COMP，以进行下一步的实验操作。如下图所示：

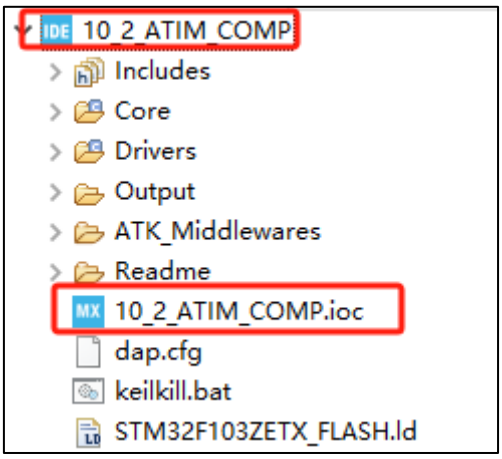


图 23.2.2.1 ATIM 工程

接着双击 10_2_ATIM_COMP.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PC6~PC9。将 PC6~PC9 分别复用为 TIM8_CH1~TIM8_CH4，如下图所示：

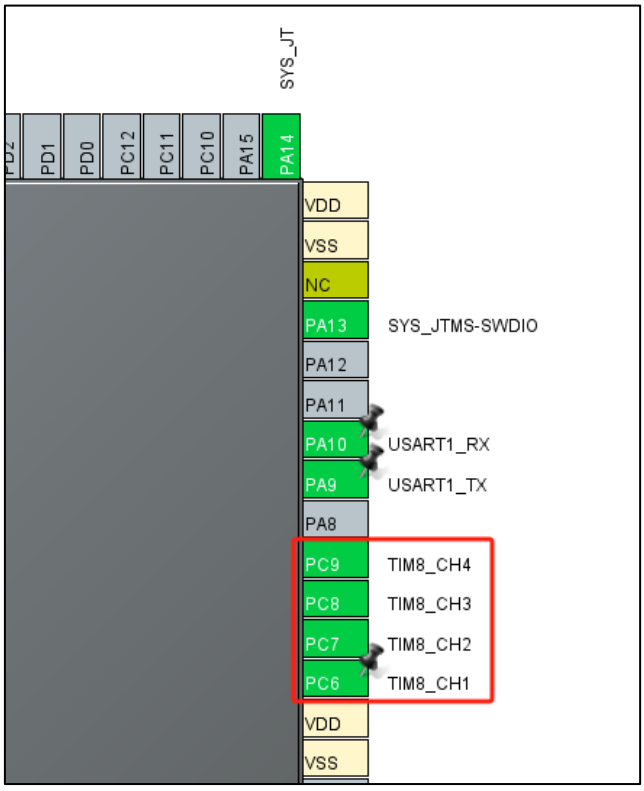


图 23.2.2.2 GPIO 复用

23.2.3 配置 TIM8 时基等参数

在 Timers→TIM8 中配置 TIM8 的模式，其中，模式配置如下图所示，我们选择内部时钟，通道 1~通道 4 选择为输出比较功能：

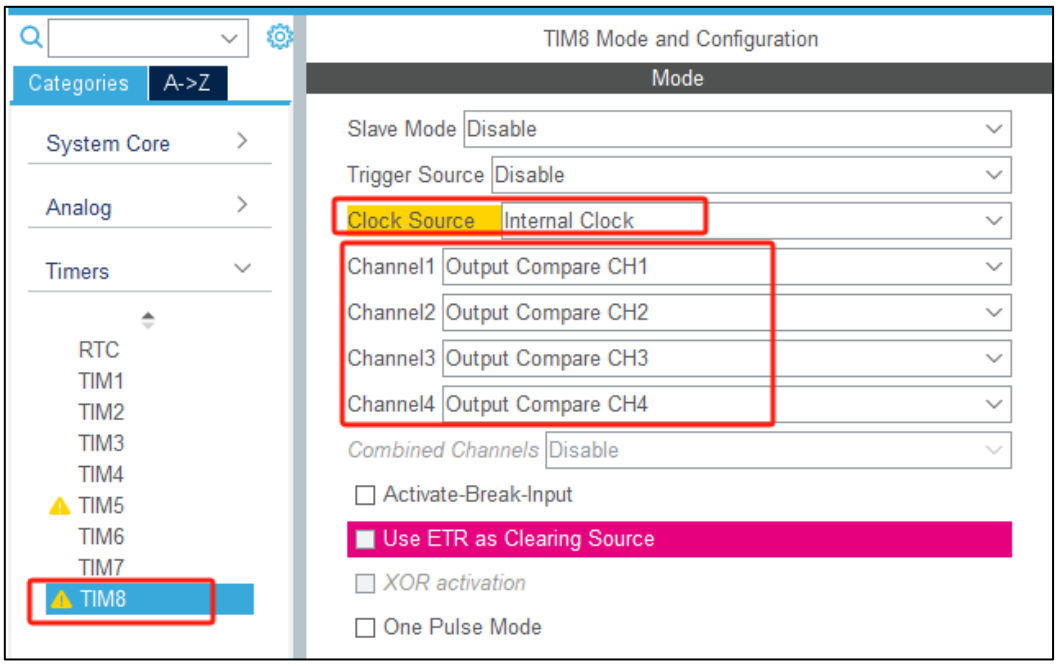


图 23.2.3.1 TIM8 模式配置

接下来我们配置 TIM8 的时基参数和通道 1~通道 4 的参数，如下红框部分需要我们手动配置，其它部分就保持默认配置：

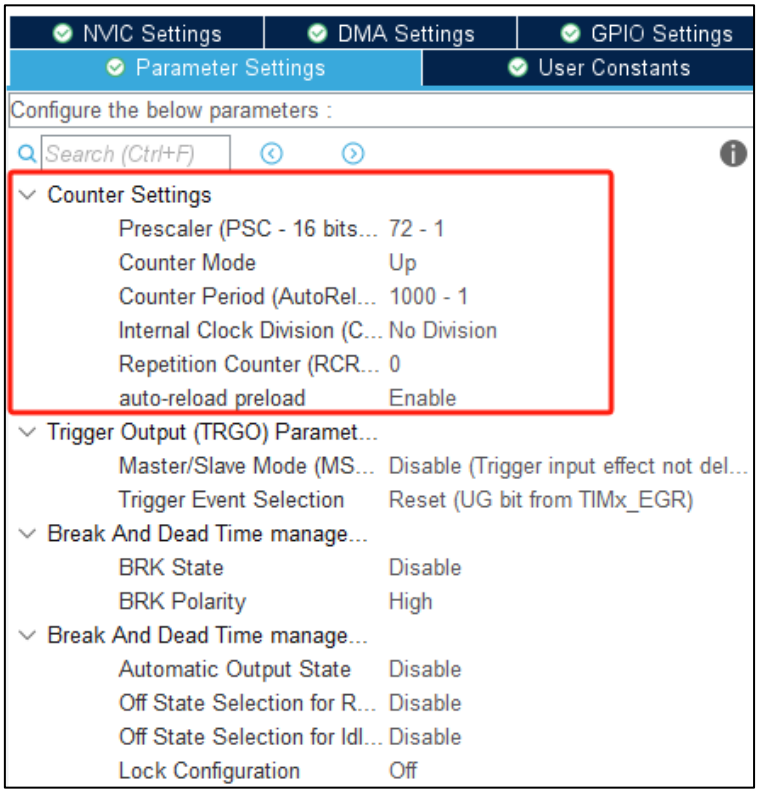


图 23.2.3.2 TIM8 参数配置 1

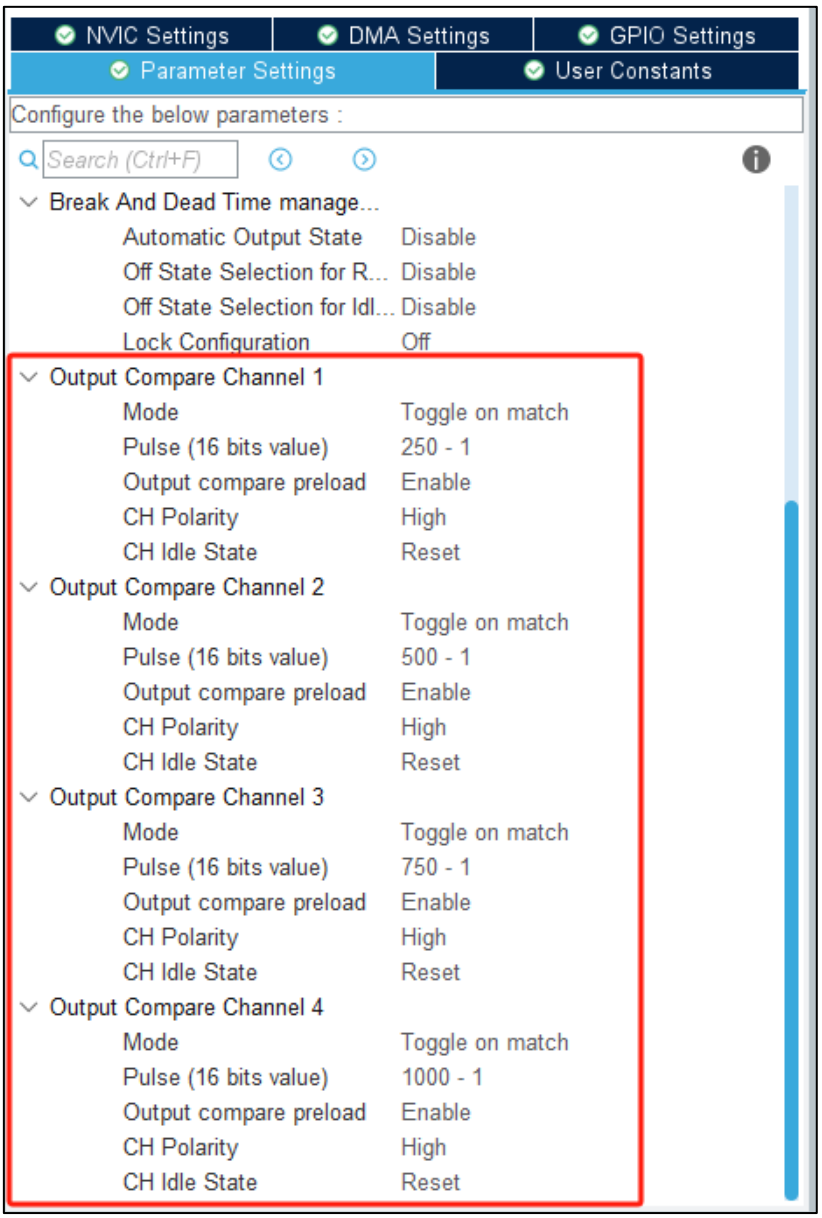


图 23.2.3.3 TIM8 参数配置 2

图 23.2.3.3 中的参数说明在前面章节已经讲解过了，这里不在赘述，直接将配置后的定时器溢出周期算出来：

$$T_{out} = (ARR[15:0] + 1) * \frac{1}{f_{CK_{CNT}}} = \frac{1000 * 72}{72MHz} = 1ms$$

比较模式中 PWM 波的周期是 2 倍的 ARR 计数器所用的计数器时间，所以得到 PWM 波的周期是 2ms，频率为 500HZ。

Output Compare Channel 1 用于配置通道 1 的 PWM 输出参数，其中：

- Mode：用于配置 PWM 的模式，这里选择 Toggle on match，即翻转。
- Pulse (16 bits value)：是 TIMx_CCRx 的值，也就是有效电平的值，可以配置在 0-1000 之间，例如配置 0。这里配置 250，即初相位为 25%。

$$相位=\frac{Pulse}{自动重载值} * 100\%$$

- Output compare preload：输出比较预加载项选择 Enable，即在定时器工作时是否能修改 Pulse 的值，如果禁用此项，表示定时器工作时不能进行修改，只能等到更新事件到来的时候才能进行修改，所以这里选择使能。

- CH Polarity：输出极性，这里默认选择 High，即高电平有效，修改为 Low 也是可以的，在本实验中无关紧要，只是想输出 PWM 波而已。
- CH Idle State 用于配置通道的空闲状态，也就是配置 PWM 不输出时的状态，这里默认选择 Reset，即 PWM 关闭状态下默认为低电平。如果配置为 Set 就是 PWM 关闭状态下默认为高电平。

注意其余通道的配置与上述基本一致，仅 Pulse (16 bits value)此项有区别，通道 2~3，分别配置为：500、750、1000。

23.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM8 的初始化。

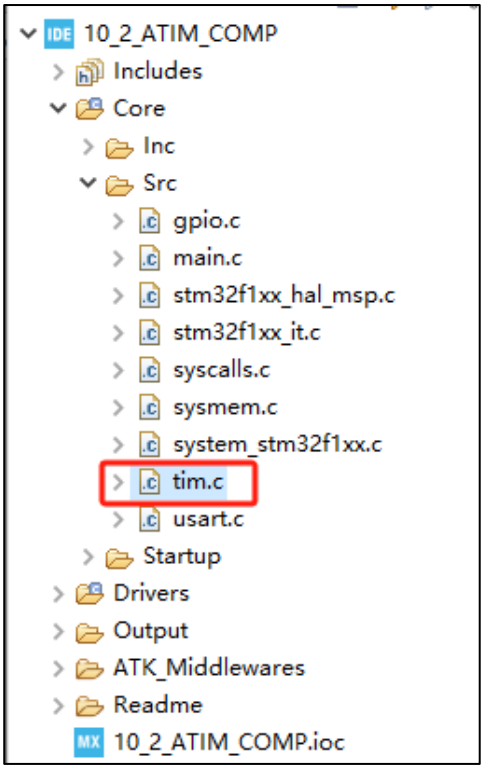


图 23.2.4.1 生成代码

23.2.5 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

```
void MX_TIM8_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};
  htim8.Instance = TIM8;
  htim8.Init.Prescaler = 72 - 1;
  htim8.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim8.Init.Period = 1000 - 1;
  htim8.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim8.Init.RepetitionCounter = 0;
```

```
  htim8.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim8) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim8, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_OC_Init(&htim8) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim8, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_TOGGLE;
  sConfigOC.Pulse = 250 - 1;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
  sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
  if (HAL_TIM_OC_ConfigChannel(&htim8, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  __HAL_TIM_ENABLE_OCxPRELOAD(&htim8, TIM_CHANNEL_1);
  sConfigOC.Pulse = 500 - 1;
  if (HAL_TIM_OC_ConfigChannel(&htim8, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  __HAL_TIM_ENABLE_OCxPRELOAD(&htim8, TIM_CHANNEL_2);
  sConfigOC.Pulse = 750 - 1;
  if (HAL_TIM_OC_ConfigChannel(&htim8, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
  {
    Error_Handler();
  }
  __HAL_TIM_ENABLE_OCxPRELOAD(&htim8, TIM_CHANNEL_3);
  sConfigOC.Pulse = 1000 - 1;
  if (HAL_TIM_OC_ConfigChannel(&htim8, &sConfigOC, TIM_CHANNEL_4) != HAL_OK)
  {
    Error_Handler();
  }
  __HAL_TIM_ENABLE_OCxPRELOAD(&htim8, TIM_CHANNEL_4);
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
  sBreakDeadTimeConfig.DeadTime = 0;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
  sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
  if (HAL_TIMEx_ConfigBreakDeadTime(&htim8, &sBreakDeadTimeConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM8_Init 2 */
  HAL_TIM_OC_Start(&htim8, TIM_CHANNEL_1);
  HAL_TIM_OC_Start(&htim8, TIM_CHANNEL_2);
  HAL_TIM_OC_Start(&htim8, TIM_CHANNEL_3);
  HAL_TIM_OC_Start(&htim8, TIM_CHANNEL_4);
  /* USER CODE END TIM8_Init 2 */
```

```
  HAL_TIM_MspPostInit(&htim8);

}
```

在 TIM8 的初始化函数中，我们通过调用函数 HAL_TIM_OC_Start 来使能 TIM1 通道 1~通道 4 输出。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM8 初始化 */
  MX_TIM8_Init();

  while (1)
  {
      delay_ms(10);
      t++;

      if (t >= 20)
      {
          LED0_TOGGLE(); /* LED0(RED)闪烁 */
          t = 0;
      }
  }
}
```

在 main 函数中，调用系统初始化以及外设初始化即可，然后 while 循环中实现 LED0 的间隔翻转，代表程序正常运行。

23.3 下载验证

下载代码后，可以看到 LED0 在闪烁，说明程序已经正常在跑了。 我们需要借助示波器观察 PC6、PC7、PC8 和 PC9 引脚 PWM 输出的情况，如下图所示：

图 23.3.1 相位为 25%、50%、75%、100%的 PWM 波

图 23.3.1 中，由上到下分别是引脚 PC6、PC7、PC8 和 PC9 输出的 PWM，即分别对应的是 TIM8_CH1、TIM8_CH2、TIM8_CH3 和 TIM8_CH4 输出的相位为 25%、50%、75%和 100%的 PWM。大家可以把其中一个通道的捕获/比较寄存器的值设置为 0，那么就可以得到 PWM 初相位的波形，即相位为 0%。

第二十四章 高级定时器互补输出带死区控制实验

本章将介绍使用 STM32F103 输出带死区和刹车控制的两路互补 PWM。通过本章的学习，读者将学习到高级定时器的互补输出、死区插入和刹车的功能的使用。

本章分为如下几个小节：

24.1 硬件设计
24.2 程序设计
24.3 下载验证

24.1 硬件设计

1， 例程功能

1、利用 TIM1_CH1(PE8)输出 70%占空比的 PWM 波，它的互补输出通道(PE9)则是输出 30%占空比的 PWM 波。

2、刹车功能，当给刹车输入引脚(PE15)输入高电平时，进行刹车，即 PE8 和 PE9 停止输出 PWM 波。

3、LED0 闪烁指示程序运行。

2， 硬件资源

1）LED 灯
LED0 – PB5
2）定时器 1
TIM1 正常输出通道 PE9
TIM1 互补输出通道 PE8
TIM1 刹车输入 PE15

3， 原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们需要通过示波器观察 PE8 和 PE9 引脚 PWM 输出的情况。还可以通过给 PE15 引脚接入高电平进行刹车。

24.2 程序设计

24.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\10_3_ATIM_CPLM_PWM。

24.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 10_3_ATIM_CPLM_PWM，以进行下一步的实验操作。如下图所示：

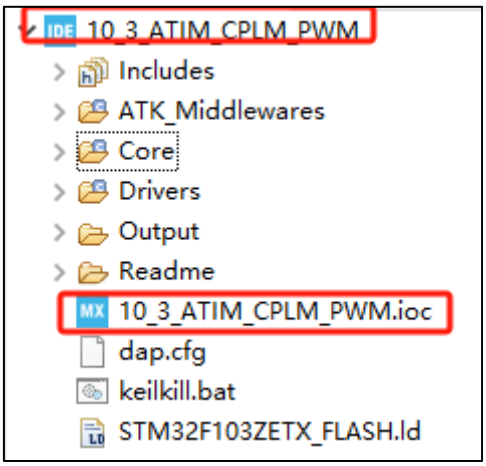


图 24.2.2.1 ATIM 工程

接着双击 10_3_ATIM_CPLM_PWM.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PE8、PE9、PE15。将 PE8、PE9 分别复用为 TIM1_CH1 和 TIM1_CH1N 用于互补输出；将 PE15 复用为 TIM1_BKIN 用于测试刹车功能，如下图所示：

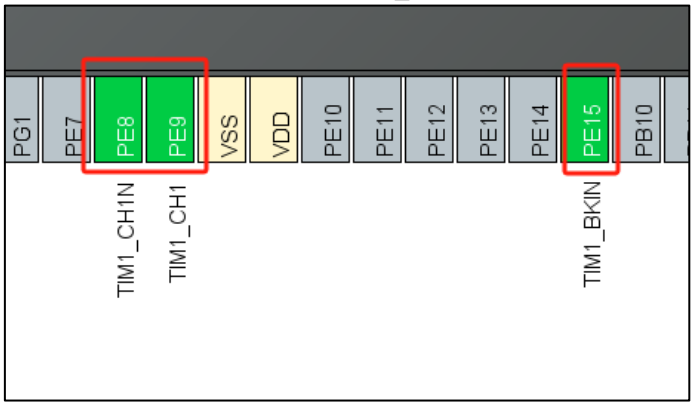


图 24.2.2.2 GPIO 复用

24.2.3 配置 TIM1 时基等参数

在 Timers→TIM1 中先配置 TIM1 的模式，模式配置如下图所示，我们选择内部时钟，选择 PWM Generation CH1 CH1N，即通道 1 和互补通道 1 产生 PWM：

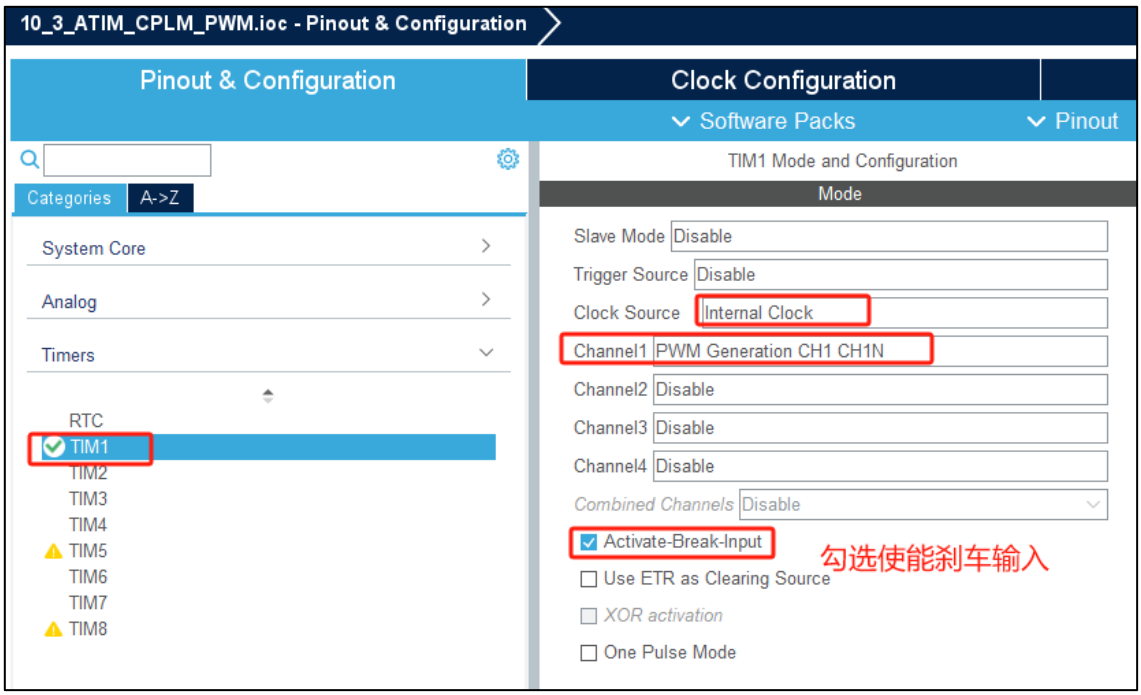


图 24.2.3.1 TIM1 模式配置

接下来我们配置 TIM1 的时基参数和通道 1 以及互补通道 1 的参数，如下红框部分需要我们手动配置，其它部分就保持默认配置：

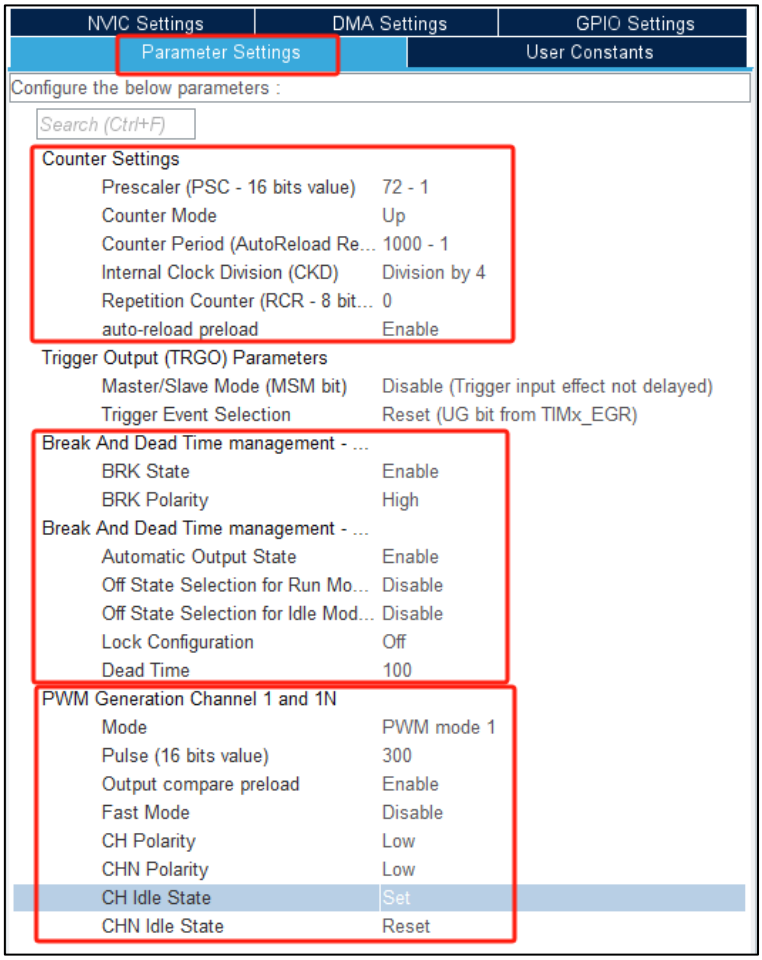


图 24.2.3.2 参数配置

Counter Settings参数我们前面已经分析过多次，这里不再赘述。要注意的是Internal Clock Division(CKD)我们选择Division by 4 ，即4分频，72/4=18MHz。

Break And Dead Time management-BRK Configuration 用于配置刹车输入，参数配置：

- BRK State 用于配置刹车状态，这里配置为 Enable，即打开刹车电平；
- BRK Polarity 用于配置刹车极性，这里配置为 High，即刹车电平为高；

Break And Dead Time management-Output Configuration 用于配置死区（停滞）时间

- Automatic Output State 用于配置自动输出状态，这里选择使能 Enable，即将 TIMx_BDTR 寄存器的 AOE 置 1，所以只要断路输入（BRK）为无效状态，使能主输出 MOE 位以后，OCx 和 OCxN 就有输出。
- Off State Selection for Run Mode (OSSR)即运行模式下“关闭状态”选择，这里选 Disable，即关闭
- Off State Selection for ldle Mode (OSSl) 即空闲模式下“关闭状态”选择，这里选择 Disable，即关闭
- Lock Configuration，锁定设置选择 Off，即关闭

Dead Time 用于配置死区延时时间，这里配置死区发生器为十进制数的 100，也就是本例程的 DTG[7:0]位的值设置为十进制 100，即二进制数 0110 0100。DTG[7:5]=011，符合第一条式子：DT=DTG[7:0] * t_{dtg}，其中 t_{dtg} = tDTS。DT 是死区时间，可以得到 DT = 100*55.56 ns = 5. 56us。到后面下载验证小节，我们通过示波器验证一下这个死区时间计算的理论值和实际值是否一样。

PWM Generation Channel 1 and 1N

- Mode 选择 PWM mode 1，即 PWM 模式 1；
- Pulse(16 bits value)用于配置通道 1 的占空比，这里配置为 300，即 30%的占空比，那么通道 1 对应的互补输出通道的占空比为 70%；
- Output compare preload 输出比较预加载项选择 Enable，即在定时器工作时是否能修改 Pulse 的值，如果禁用此项，表示定时器工作时不能进行修改，只能等到更新事件到来的时候才能进行修改，所以这里选择使能。
- Fast Mode 用于配置 PWM 脉冲快速模式，这里我们也不需要，可以不配置。
- CHN Polarity 用于配置互补输出通道电平极性，这里我们选择为 Low，即低电平有效；
- CH Polarity：输出极性，这里我们选择 Low，即低电平有效；
- CH Idle State 用于配置通道的空闲状态的电平，这里默认选择 Set 为低电平。
- CHN Idle StateSet 用于配置互补输出通道的空闲状态的电平，这里默认选择 Reset 为高电平。

24.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM1 的初始化。

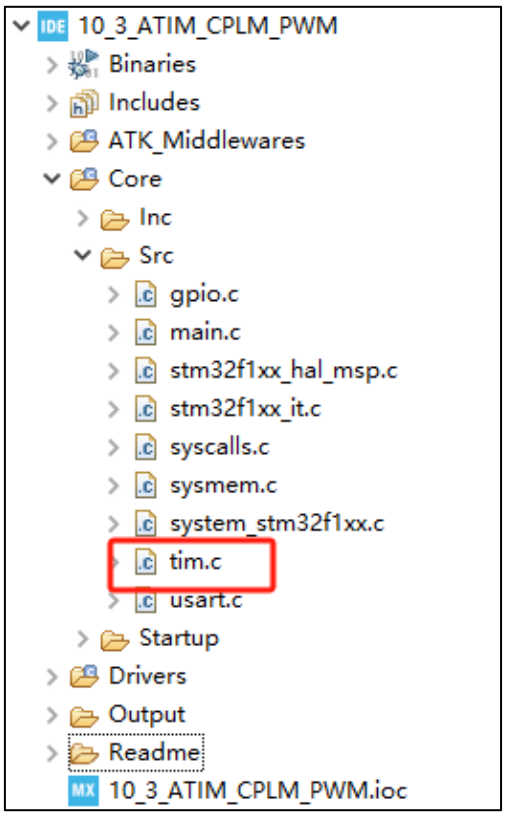


图 24.2.4.1 生成代码

24.2.5 添加用户驱动代码

我们本实验主要修改两个文件，分别是：tim.c、main.c，首先我们将对 tim.c 文件进行修改。

（1） 修改 tim.c 文件

```
void MX_TIM1_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};
  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 72 - 1;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 1000 - 1;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV4;
  htim1.Init.RepetitionCounter = 0;
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
```

```
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 300;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_LOW;
  sConfigOC.OCNPolarity = TIM_OCNPOLARITY_LOW;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  sConfigOC.OCIdleState = TIM_OCIDLESTATE_SET;
  sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
  if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
  sBreakDeadTimeConfig.DeadTime = 100;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_ENABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
  sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_ENABLE;
  if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM1_Init 2 */
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);          /* 使能 OCy 输出 */
  HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1);       /* 使能 OCyN 输出 */
  /* USER CODE END TIM1_Init 2 */
  HAL_TIM_MspPostInit(&htim1);

}
```

在 TIM1 的初始化函数中，记得要添加 HAL_TIM_PWM_Start 函数和 HAL_TIMEx_PWMN_Start 函数，用于启动通道输出和互补通道输出。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM1 初始化 */
  MX_TIM1_Init();
  /* USER CODE BEGIN 2 */
  while (1)
  {
```

```
        delay_ms(10);
        t++;

        if (t >= 20)
        {
            LED0_TOGGLE(); /* LED0(RED)闪烁 */
            t = 0;
        }
    }
}
```

在 main 函数中，调用系统初始化以及外设初始化即可，然后 while 循环中实现 LED0 的间隔翻转，代表程序正常运行。

24.3 下载验证

下载代码后，可以看到 LED0 在闪烁，说明程序已经正常在跑了。 我们需要借助示波器观察 PE9 正常输出和 PE8 互补输出 PWM 的情况，示波器显示截图如图 24.3.1 所示：

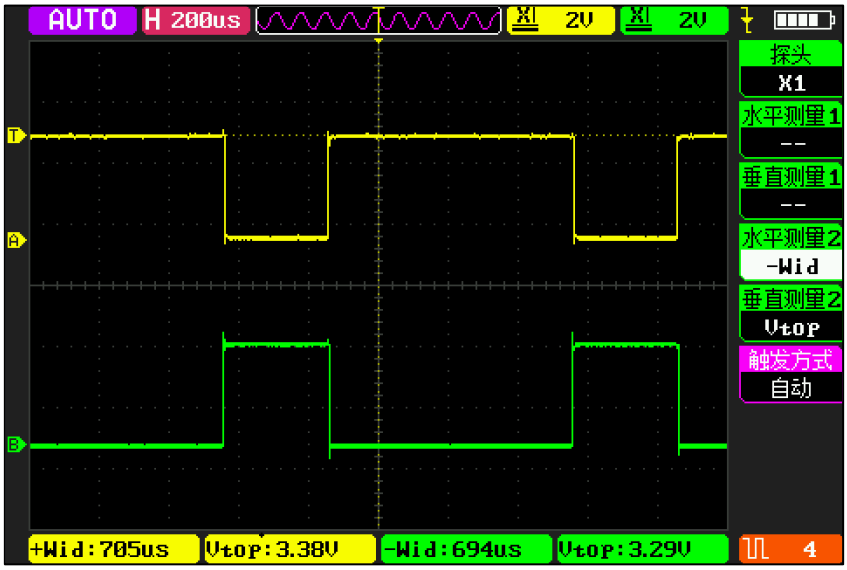


图 24.3.1 PE8 正常输出和 PE9 互补输出 PWM 的情况

图 24.3.1 中的由上到下分别是 PE9 输出 70%占空比的 PWM 波和 PE8 互补输出 30%占空比的 PWM 波。互补输出的 PWM 波的正脉宽减去正常的 PWM 的负脉宽的值除以 2 就是死区时间，也可以是正常的 PWM 的正脉宽减去互补输出的 PWM 波的负脉宽的值除以 2。我们使用第一种方法得到：死区时间 =（705 - 694）/2 us=5.5us。与我们理论到的的值 5.56us 基本一样，这样的误差是正常的。

因为使能了刹车和自动输出功能，因此将 PE15 引脚接入有效的高电平后，可以看到两路 PWM 都被禁止输出了，撤销 PE15 引脚接入的高电平后，可以看到两路 PWM 有自动恢复输出了。

第二十五章 高级定时器 PWM 输入模式实验

本章将介绍使用 STM32F103 的高级定时器检测输入 PWM 的占空比和周期。通过本章的学习，读者将学习到高级定时器 PWM 输入模式的使用。

本章分为如下几个小节：

25.1 硬件设计
25.2 程序设计
25.3 下载验证

25.1 硬件设计

1， 例程功能

首先通过 TIM3_CH2(PB5)输出 PWM 波。然后把 PB5 输出的 PWM 波用杜邦线接入 PC6（定时器 8 通道 1），最后通过串口打印 PWM 波的脉宽和频率等信息。通过 LED1 闪烁来提示程序正在运行。

2， 硬件资源

1）LED 灯
LED0 - PB5
2）定时器 3 通道 2（PB5）输出 PWM 波
定时器 8 通道 1（PC6）输入 PWM 波

3， 原理图

定时器属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们把 PB5 引脚输出的 PWM 波用杜邦线接入 PC6 引脚，然后通过电脑串口上位机软件观察打印出来的信息。

25.2 程序设计

25.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\10_4_ATIM_PWMIN。

25.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 10_4_ATIM_PWMIN，以进行下一步的实验操作。如下图所示：

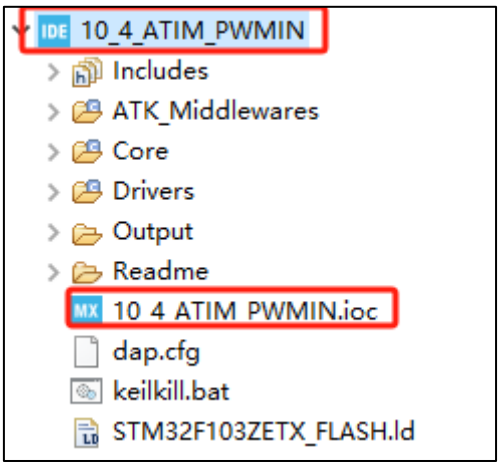


图 25.2.2.1 ATIM 工程

接着双击 10_4_ATIM_PWMIN.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PB5、PC6。将 PB5、PC6 分别复用为 TIM3_CH2（PWM 输出）和 TIM8_CH1（PWM 输入），如下图所示：

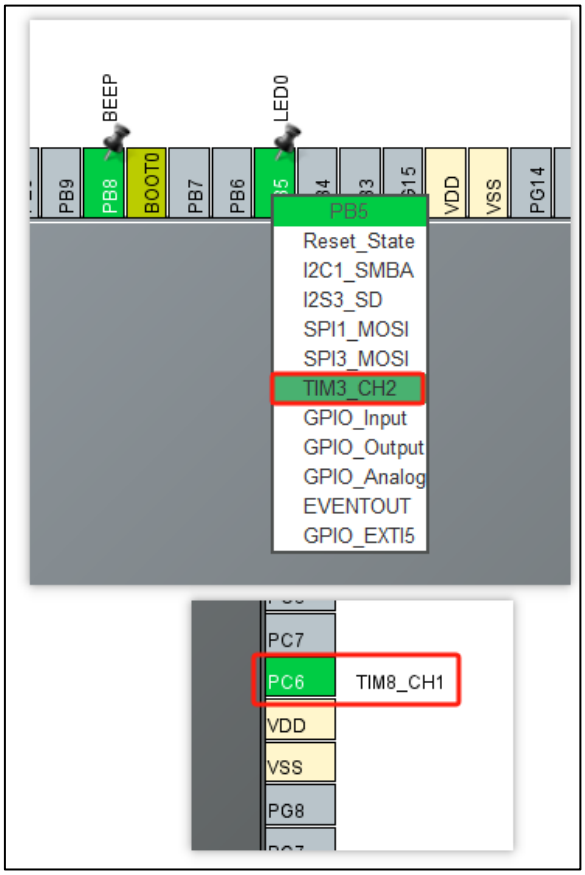


图 25.2.2.2 GPIO 复用

25.2.3 配置 TIM3 时基等参数

在 Timers→TIM3 中先配置 TIM3 的模式，模式配置如下图所示，我们选择内部时钟，选择 PWM Generation CH2，即通道 2 产生 PWM：

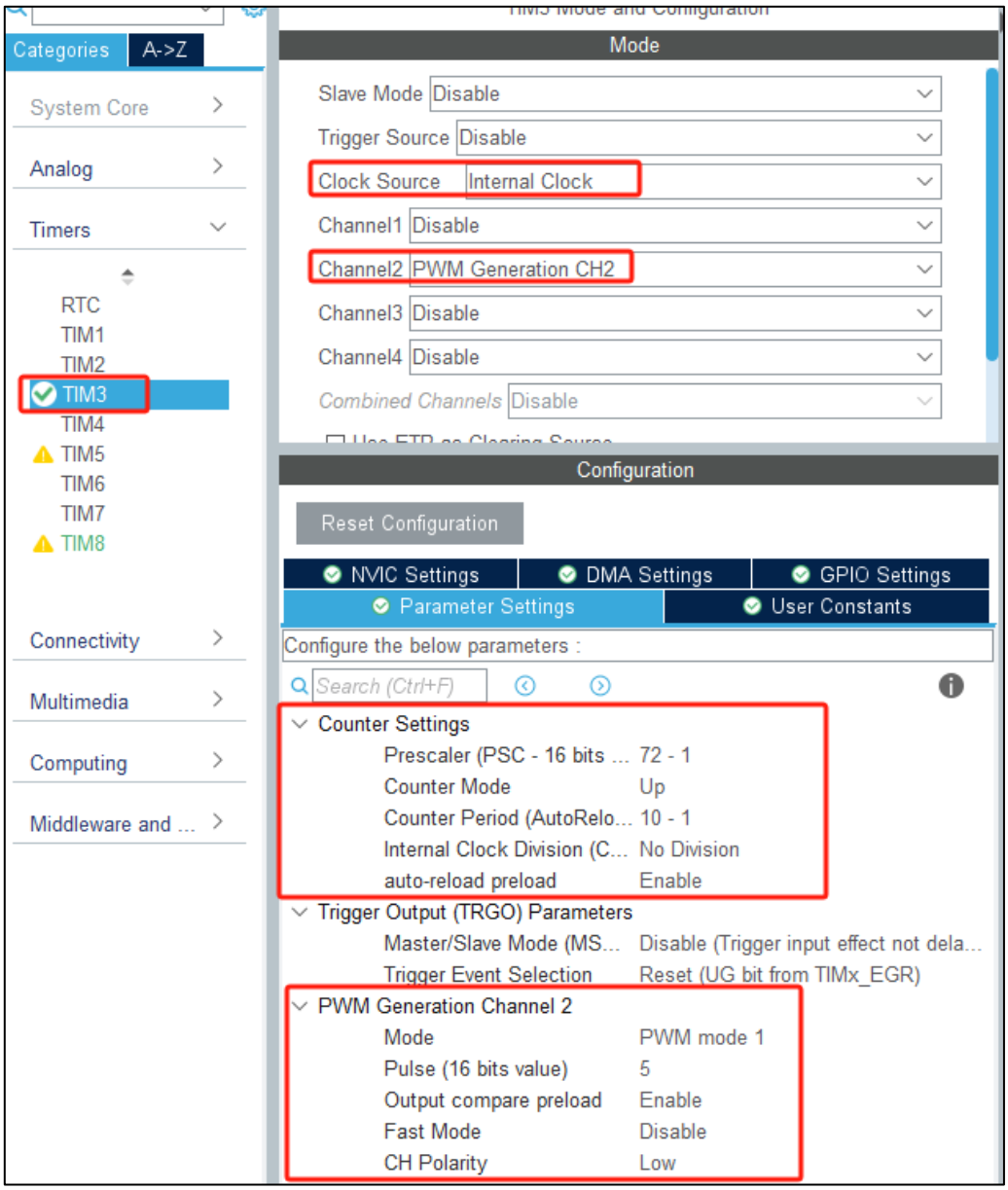


图 25.2.3.1 TIM3 参数配置

Counter Settings 参数我们前面已经分析过多次，这里不再赘述。直接将配置后的定时器溢出周期算出来：

$$T_{out} = (ARR[15:0] + 1) * \frac{1}{f_{CK_{CNT}}} = \frac{10 * 72}{72MHz} = 10us$$

即中断频率为：100kHZ。

PWM Generation Channel 2

- Mode 选择 PWM mode 1，即 PWM 模式 1；
- Pulse(16 bits value)用于配置通道 1 的占空比，这里配置为 5，即 50%的占空比。
- Output compare preload 输出比较预加载项选择 Enable，即在定时器工作时是否能修改 Pulse 的值，如果禁用此项，表示定时器工作时不能进行修改，只能等到更新事件到来的时候才能进行修改，所以这里选择使能。
- Fast Mode 用于配置 PWM 脉冲快速模式，这里我们也不需要，可以不配置。
- CH Polarity：输出极性，这里我们选择 Low，即低电平有效；

25.2.4 配置 TIM8 时基等参数

本实验我们要用到 TIM8_CH1 捕获 TIM3_CH2 产生的 PWM 波，然后计算出 PWM 波的高电平脉宽、PWM 波的周期和频率。这里配置两个捕获通道（IC1 和 IC2）。

在 Timers→TIM8 中先配置 TIM8 的模式，模式配置如下图所示，我们选择内部时钟，选择从模式为 Reset Mode，触发源为 TI1FP1，并配置两个捕获通道 IC1 和 IC2。

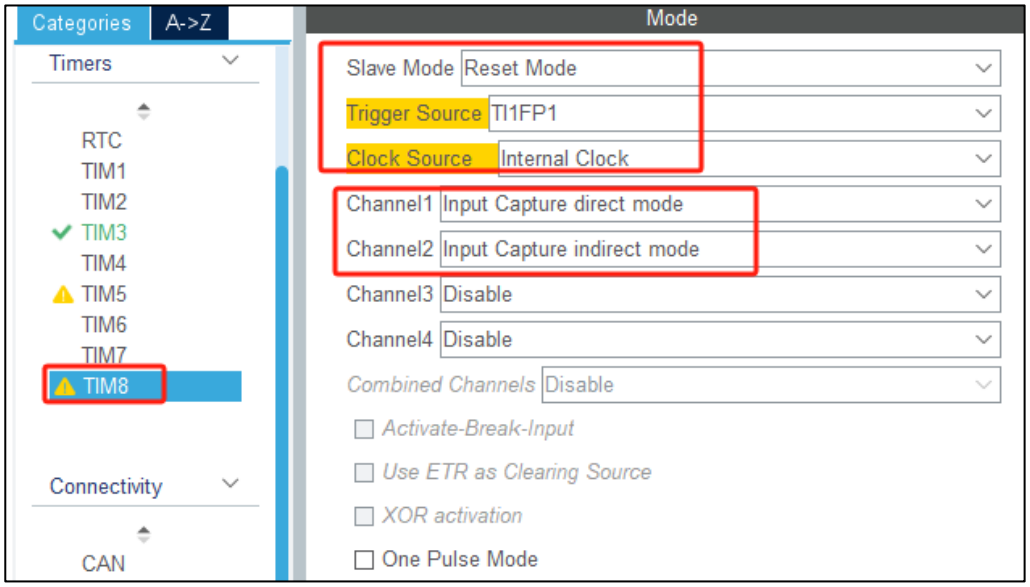


图 25.2.4.1 配置 TIM8 模式

这里的从模式为 Reset Mode 也就是复位模式，TIM8 做从定时器，TIM3 是主定时器。从模式定时器收到主模式定时器的信号后，当触发输入产生变化时，从模式定时器的计数器被复位，并更新所有的寄存器。此外还有其他的模式，如：

External Clock Mode1——外部时钟:使用外部时钟源作为定时器时钟；

Gated Mode——门控模式：使用门控的功能决定时钟是否工作；

Trigger Mode——触发模式：由主定时器信号变化触发是否工作。

接下来配置 TIM8_CH1 的时基和捕获通道参数：

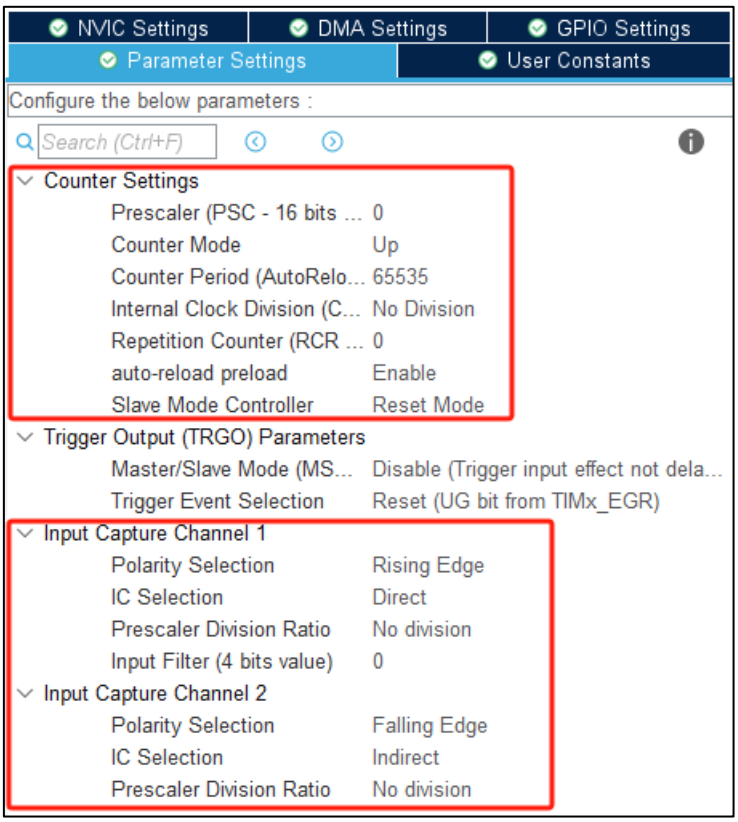


图 25.2.4.2 TIM8 参数配置

Counter Settings 配置：

这里配置 TIM8 的预分频值为 0，即时钟频率就是 PWM 波输入的频率；计数模式为向上计数；定时器自动重装载值默认选择最大，为 65536；内部时钟不分频；重复计数默认选择为 0，高级定时器和基本定时器以及通用定时器不同，高级定时器的 TIMx_RCR 递减为 0 时才会发生更新事件，这里配置为 0，表示当计数器计数到 65536 后就发生溢出；使能自动装载；从模式控制选择为复位模式。

Input Capture Channel 1 参数配置如下：

- Polarity Selection 用于配置极性选择，这里配置上升沿；
- IC Selection 配置为 Direct，为直连模式，即配置 IC1 直接映射在 TI1 上；
- Prescaler Division Ratio 用于配置配置输入分频，这里选择为 No division，即不分频。要注意的是，如果设置了分频比，例如设置为2，表示捕获到2个上升沿才产生一次中断；
- Input Filter (4 bits value) 配置输入滤波器参数，这里配置为 0，即不滤波。

Input Capture Channel 2 参数配置如下：

- Polarity Selection 这里配置下降沿；
- IC Selection 配置为 Direct，为直连模式，即配置 IC2 直接映射在 TI1 上；
- Prescaler Division Ratio 用于配置配置输入分频，这里选择为 No division，即不分频；

以上配置符合我们前面的设置，也就是捕获通道 1 用于测试 PWM 波的周期，捕获通道 2 用于配置 PWM 波的高电平脉宽。

25.2.5 配置 NVIC

本实验要用到定时器 8 的更新中断和输入捕获中断，所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM8 定时器更新中断和输入捕获中断，配置更新中断优先级分组为 2，抢占优先级和子优先级为 1；配置捕获中断优先级分组为 2，抢占优先级和子优先级为 1：

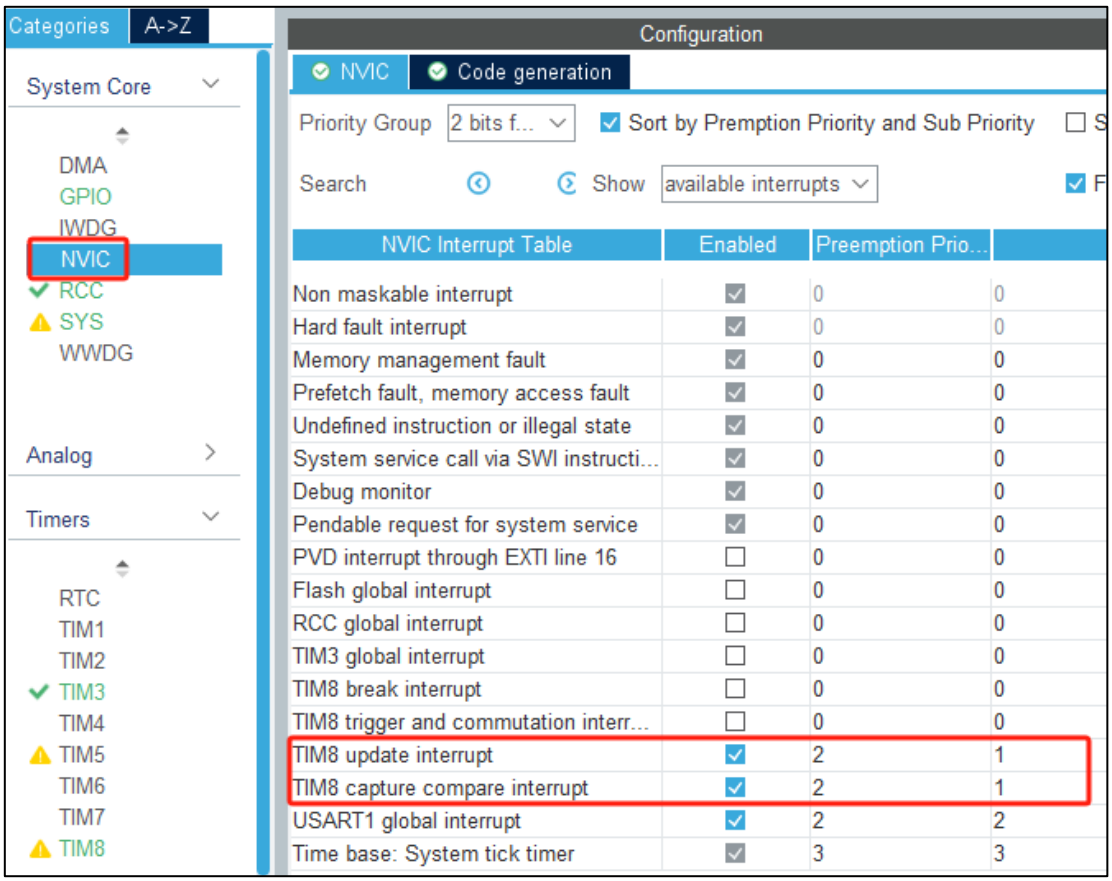


图 25.2.5.1 NVIC 配置

25.2.6 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM3 和 TIM8 的初始化。

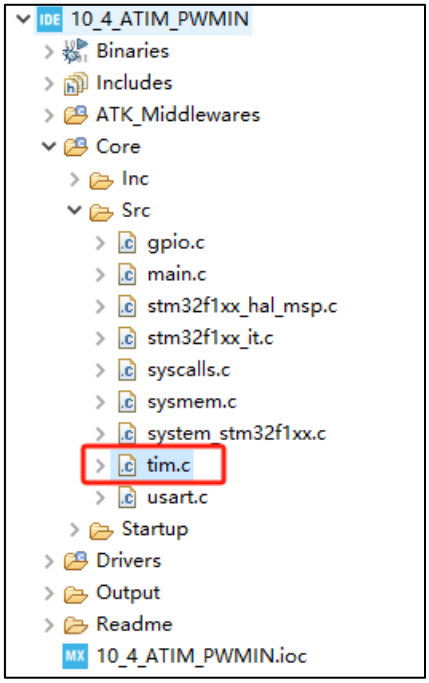


图 25.2.6.1 生成代码

25.2.7 添加用户驱动代码

我们本实验主要修改三个文件，分别是：tim.c、main.c、stm32f1xx_it.c，首先我们将对 tim.c 文件进行修改。

（1）修改 tim.c 文件

首先看下 TIM3 初始化部分我们添加了 HAL_TIM_PWM_Start 函数，用于开启 PWM 的输出。代码如下：

```
void MX_TIM3_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 72 - 1;
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 10 - 1;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim3, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 5;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_LOW;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM3_Init 2 */
  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);        /* 开启对应 PWM 通道 */
  /* USER CODE END TIM3_Init 2 */
  HAL_TIM_MspPostInit(&htim3);

}
```

接着来看下 TIM8 的初始化函数，我们添加了函数__HAL_TIM_ENABLE_IT 用于使能定时器 8 的更新中断，添加函数 HAL_TIM_IC_Start_IT 用于开启通道 1 和通道 2 的输入捕获，以及捕获中断。代码如下：

```
void MX_TIM8_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_SlaveConfigTypeDef sSlaveConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_IC_InitTypeDef sConfigIC = {0};
  htim8.Instance = TIM8;
```

```
  htim8.Init.Prescaler = 0;
  htim8.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim8.Init.Period = 65535;
  htim8.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim8.Init.RepetitionCounter = 0;
  htim8.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim8) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim8, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_IC_Init(&htim8) != HAL_OK)
  {
    Error_Handler();
  }
  sSlaveConfig.SlaveMode = TIM_SLAVEMODE_RESET;
  sSlaveConfig.InputTrigger = TIM_TS_TI1FP1;
  sSlaveConfig.TriggerPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
  sSlaveConfig.TriggerFilter = 0;
  if (HAL_TIM_SlaveConfigSynchro(&htim8, &sSlaveConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim8, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
  sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
  sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
  sConfigIC.ICFilter = 0;
  if (HAL_TIM_IC_ConfigChannel(&htim8, &sConfigIC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_FALLING;
  sConfigIC.ICSelection = TIM_ICSELECTION_INDIRECTTI;
  if (HAL_TIM_IC_ConfigChannel(&htim8, &sConfigIC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM8_Init 2 */
  __HAL_TIM_ENABLE_IT(&htim8, TIM_IT_UPDATE);
  HAL_TIM_IC_Start_IT(&htim8, TIM_CHANNEL_1);
  HAL_TIM_IC_Start_IT(&htim8, TIM_CHANNEL_2);
  /* USER CODE END TIM8_Init 2 */

}
```

接着我们还在 tim.c 文件中添加了两个独立函数，如下：

```
/**
 * @brief       定时器 TIMX PWM 输入模式 重新启动捕获
 * @param       无
 * @retval      无
 */
void atim_timx_pwmin_chy_restart(void)
{
    sys_intx_disable();                                 /* 关闭中断 */

    g_timxchy_pwmin_sta = 0;                            /* 清零状态,重新开始检测 */
```

```
    g_timxchy_pwmin_psc = 0;                                /* 分频系数清零 */

    __HAL_TIM_SET_PRESCALER(&htim8, 0);                     /* 以最大的计数频率采集 */
    __HAL_TIM_SET_COUNTER(&htim8, 0);                       /* 计数器清零 */

    __HAL_TIM_ENABLE_IT(&htim8, TIM_IT_CC1);                /* 使能通道 1 捕获中断 */
    __HAL_TIM_ENABLE_IT(&htim8, TIM_IT_UPDATE);             /* 使能溢出中断 */
    __HAL_TIM_ENABLE(&htim8);                               /* 使能定时器 TIMX */

    TIM8->SR = 0;                                           /* 清除所有中断标志位 */

    sys_intx_enable();                                      /* 打开中断 */
}


/**
 * @brief       定时器 TIMX 通道 Y PWM 输入模式 中断处理函数
 * @note
 *              因为 TIM1/TIM8 等有多个中断服务函数,而 TIM2~5/TIM12/TIM15 等普通定时器只有 1 个中断服务
 *              函数,为了更好的兼容,我们对中断处理统一放到 atim_timx_pwin_chy_process 函数里面进行处理
 *
 * @param       无
 * @retval      无
 */
void atim_timx_pwmin_chy_process(void)
{
    static uint8_t sflag = 0;                               /* 启动 PWMIN 输入检测 */

    if (g_timxchy_pwmin_sta)
    {
        g_timxchy_pwmin_psc = 0;
        TIM8->SR = 0;                                       /* 清除所有中断标志位 */
        __HAL_TIM_SET_COUNTER(&htim8, 0);                   /* 计数器清零 */
        return ;
    }

    if (__HAL_TIM_GET_FLAG(&htim8, TIM_FLAG_UPDATE))        /* 发生了溢出中断 */
    {
        __HAL_TIM_CLEAR_FLAG(&htim8, TIM_FLAG_UPDATE);      /* 清除溢出中断标记 */
        /* 没有发生周期捕获中断,且捕获未完成 */
        if (__HAL_TIM_GET_FLAG(&htim8, TIM_FLAG_CC1) == 0)
        {
            sflag = 0;
            if (g_timxchy_pwmin_psc == 0)                   /* 从 0 到 1 */
            {
                g_timxchy_pwmin_psc ++;
            }
            else
            {   /* 已经最大了,可能是无输入状态 */
                if (g_timxchy_pwmin_psc == 65535)
                {
                    g_timxchy_pwmin_psc = 0;                /* 重新恢复不分频 */
                }
                else if (g_timxchy_pwmin_psc > 32767)       /* 不能倍增了 */
                {
                    g_timxchy_pwmin_psc = 65535;            /* 直接等于最大分频系数 */
                }
                else
                {
                    g_timxchy_pwmin_psc += g_timxchy_pwmin_psc; /* 倍增 */
```

```
                }
            }
            /* 设置定时器预分频系数 */
            __HAL_TIM_SET_PRESCALER(&htim8, g_timxchy_pwmin_psc);
            __HAL_TIM_SET_COUNTER(&htim8, 0);          /* 计数器清零 */
            TIM8->SR = 0;                              /* 清除所有中断标志位 */
            return ;
        }
    }

    if (sflag == 0)     /* 第一次采集到捕获中断 */
    {
        if (__HAL_TIM_GET_FLAG(&htim8, TIM_FLAG_CC1)) /* 检测到第一次捕获中断 */
        {
            sflag = 1; /* 标记第一次周期已经捕获, 第二次周期捕获可以开始了 */
        }

        TIM8->SR = 0;                                  /* 清除所有中断标志位 */
        return ;                                       /* 完成此次操作 */
    }

    if (g_timxchy_pwmin_sta == 0)                      /* 还没有成功捕获 */
    {
        if (__HAL_TIM_GET_FLAG(&htim8, TIM_FLAG_CC1)) /* 检测到了周期捕获中断 */
        {
            g_timxchy_pwmin_hval = HAL_TIM_ReadCapturedValue(&htim8,
            TIM_CHANNEL_2) + 1;                        /* 高定平脉宽捕获值 */
            g_timxchy_pwmin_cval = HAL_TIM_ReadCapturedValue(&htim8,
            TIM_CHANNEL_1) + 1;                        /* 周期捕获值 */
            /* 高电平脉宽必定小于周期长度 */
            if (g_timxchy_pwmin_hval < g_timxchy_pwmin_cval)
            {
                g_timxchy_pwmin_sta = 1;               /* 标记捕获成功 */

                g_timxchy_pwmin_psc = TIM8->PSC;       /* 获取 PWM 输入分频系数 */

                if (g_timxchy_pwmin_psc == 0)          /* 分频系数 0, 修正数据 */
                {
                    g_timxchy_pwmin_hval++;            /* 修正系数为 1, 加 1 */
                    g_timxchy_pwmin_cval++;            /* 修正系数为 1, 加 1 */
                }

                sflag = 0;
                /* 每次捕获 PWM 输入成功后, 停止捕获,避免频繁中断影响系统正常代码运行 */
                TIM8->CR1   &= ~(1 << 0);              /* 关闭定时器 TIMX */
                __HAL_TIM_DISABLE_IT(&htim8, TIM_IT_CC1); /* 关闭捕获中断 */
                __HAL_TIM_DISABLE_IT(&htim8, TIM_IT_CC2); /* 关闭捕获中断 */
                __HAL_TIM_DISABLE_IT(&htim8, TIM_IT_UPDATE);  /* 关闭溢出中断 */

                TIM8->SR = 0;                          /* 清除所有中断标志位 */
            }
            else
            {
                atim_timx_pwmin_chy_restart();
            }
        }
    }

    TIM8->SR = 0;                                      /* 清除所有中断标志位 */
}
```

atim_timx_pwmin_chy_restart 该函数首先关闭所有中断，然后把一些状态标志位清零、设置定时器预分频系数、计数器值、使能相关中断、以及清除相关中断标志位，最后才允许

被中断。

atim_timx_pwmin_chy_process 函数包含了捕获中断程序和更新中断程序的处理。如果发生了更新中断（即定时器溢出），证明超出定时器量程，这里会加大预分频系数，以得到更大的量程。量程变大了，那么测量的精度就会降低，所谓鱼和熊掌不可兼得。代码中的“if (sflag == 0) /* 第一次采集到捕获中断 */”这个程序段，表示第一次采集到捕获中断。这时候相当于第一次捕获到上升沿，我们只是把 sflag 标志位置 1，然后清除所有中断标志位，等待下次的捕获中断发生。如果再次发生捕获中断，就会来到“if (g_timxchy_pwmin_sta == 0) /* 还没有成功捕获 */”程序段。通过 HAL_TIM_ReadCapturedValue 函数获取 CCR1 和 CCR2 寄存器的值，把这个获取到的寄存器值+1 才是对应的计数器计数个数。如果预分频系数为 0 的时候，还要把这两个寄存器的值再+1，这样计算的结果更准确。

（2）修改 stm32f1xx_it.c 文件

在 stm32f1xx_it.c 中断文件中，包含了 TIM8 的更新中断和捕获中断服务函数，我们需要在这两个函数中添加 atim_timx_pwmin_chy_process 函数，代码如下：

```
void TIM8_UP_IRQHandler(void)
{
  /* USER CODE BEGIN TIM8_UP_IRQn 0 */
  atim_timx_pwmin_chy_process();
  /* USER CODE END TIM8_UP_IRQn 0 */
  HAL_TIM_IRQHandler(&htim8);
}

void TIM8_CC_IRQHandler(void)
{
  /* USER CODE BEGIN TIM8_CC_IRQn 0 */
  atim_timx_pwmin_chy_process();
  /* USER CODE END TIM8_CC_IRQn 0 */
  HAL_TIM_IRQHandler(&htim8);
}
```

（3）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    float ht, ct, f, tpsc;
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM3 和 TIM8 初始化 */
  MX_TIM3_Init();
  MX_TIM8_Init();
  while (1)
  {
      delay_ms(10);
      t++;
```

```
        if (t >= 20)    /* 每 200ms 输出一次结果,并闪烁 LED0,提示程序运行 */
        {
            if (g_timxchy_pwmin_sta)    /* 捕获了一次数据 */
            {
                printf("\r\n");          /* 输出空,另起一行 */
                /* 打印分频系数 */
                printf("PWM PSC  :%u\r\n", g_timxchy_pwmin_psc);
                /* 打印高电平脉宽 */
                printf("PWM Hight:%lu\r\n", g_timxchy_pwmin_hval);
                /* 打印周期 */
                printf("PWM Cycle:%lu\r\n", g_timxchy_pwmin_cval);
                /* 得到 PWM 采样时钟周期时间 */
                tpsc = ((double)g_timxchy_pwmin_psc + 1) / 72;
                ht = g_timxchy_pwmin_hval * tpsc;        /* 计算高电平时间 */
                ct = g_timxchy_pwmin_cval * tpsc;        /* 计算周期长度 */
                f = (1 / ct) * 1000000;                  /* 计算频率 */
                printf("PWM Hight time:%.3fus\r\n", ht); /* 打印高电平脉宽长度 */
                printf("PWM Cycle time:%.3fus\r\n", ct); /* 打印周期时间长度 */
                printf("PWM Frequency :%.3fHz\r\n", f);  /* 打印频率 */
                atim_timx_pwmin_chy_restart();           /* 重启 PWM 输入检测 */
            }

            LED1_TOGGLE();                               /* LED1（GREEN）闪烁 */
            t = 0;
        }
    }
}
```

首先对系统以及外设进行初始化，当初始化完成后，就等待高级定时器中断函数中的捕获成功标志为真，捕获成功后，便将捕获到的 PWM 输入信号的周期和占空比输出值串口调试助手。

25.3 下载验证

在完成编译和烧录操作后，可以看到 LED0 在闪烁，说明程序已经正常在跑了，我们再打开串口调试助手，选择对应的串口端口。然后用杜邦线把 PB5 引脚连接到 PC6 引脚，就可以看到串口助手不断打印 PWM 波的信息，如图 25.3.1 所示：

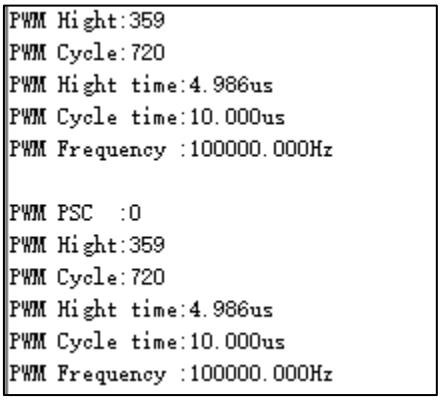


图 25.3.1 打印高电平脉冲次数

可以看到打印出来的 PWM 波信息为：周期是 10us，频率是 100KHz，占空比是 50%，和我们的预想结果一样。

第二十六章 电容触摸按键实验

在前面的章节，我们已经介绍了 STM32F1 的输入捕获功能及其使用。这一章，我们将向大家介绍如何通过输入捕获功能，来做一个电容触摸按键。在本章中，我们将用 TIM5 的通道 2（PA1）来做输入捕获，并实现一个简单的电容触摸按键，通过该按键控制 LED1 的亮灭。

从本章分为如下几个部分：

26.1 硬件设计
26.2 程序设计
26.3 下载验证

26.1 硬件设计

1， 例程功能

LED0 用来指示程序运行，150ms 变换一次状态，即约 300ms 一次闪烁。不断扫描按键的状态，如果判定了电容触摸按键按下，我们就把 LED1 的状态翻转一次。

2， 硬件资源

1）LED 灯：
LED0 – PB5
LED1 – PE5
2） 定时器 5，使用 TIM5 通道 2，将 PA1 复用为 TIM5_CH1
3） GPIO：PA1，用于控制触摸按键 TPAD

3， 原理图

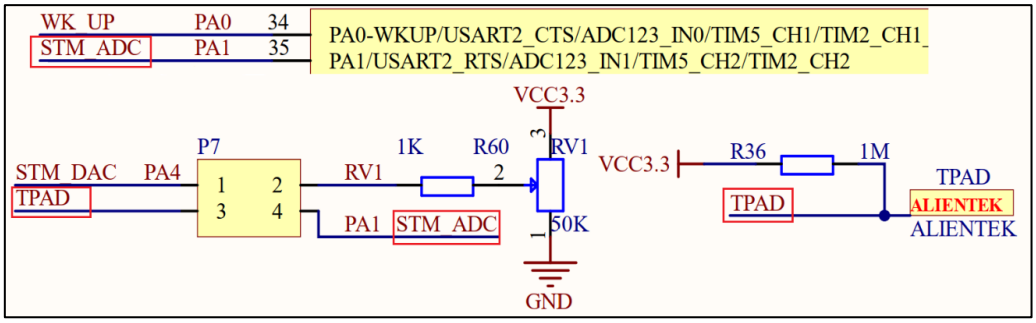


图 26.1.1 电容触摸按键 TPAD 连接原理图

由于设计时 PA1 不直接连接到电容触摸按键，而是引到了插针上，我们需要通过跳线帽把 P7 上标为 ADC 的引脚与标号为“TPAD”的标号连接到一起，如图 26.1.2 所示。

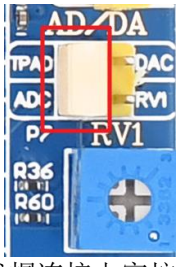


图 26.1.2 用跳线帽连接电容按键 TPAD 和 PA1

26.2 程序设计

26.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\11_TPAD。

26.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 11_TPAD，以进行下一步的实验操作。如下图所示：

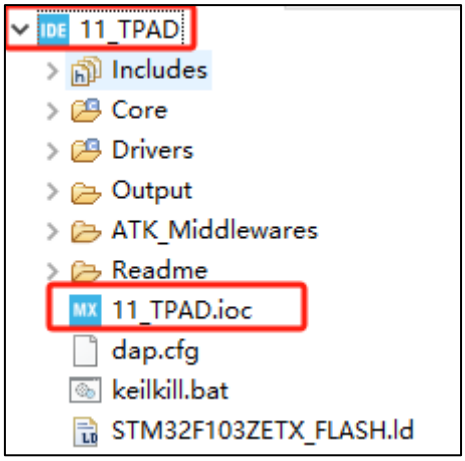


图 26.2.2.1 TPAD 工程

接着双击 11_TPAD.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PA1。将 PA1 复用为 TIM5_CH2，如下图所示：

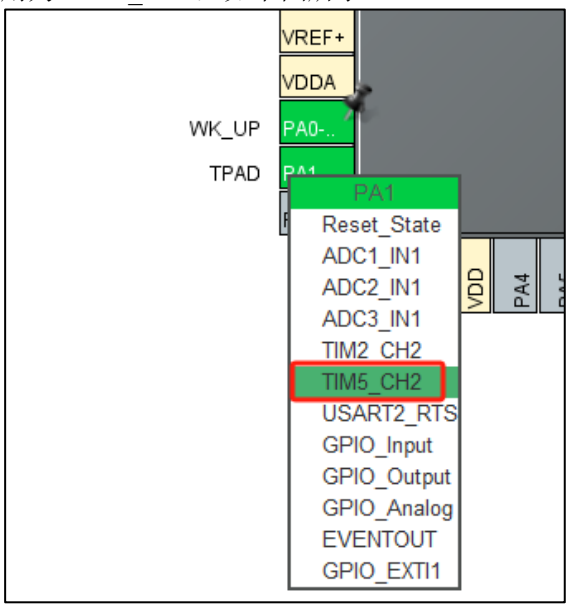


图 26.2.2.2 GPIO 复用

我们将 PA1 配置为下拉（浮空也可，不影响），User Label 配置为 TPAD 增强可读性，如下图所示：

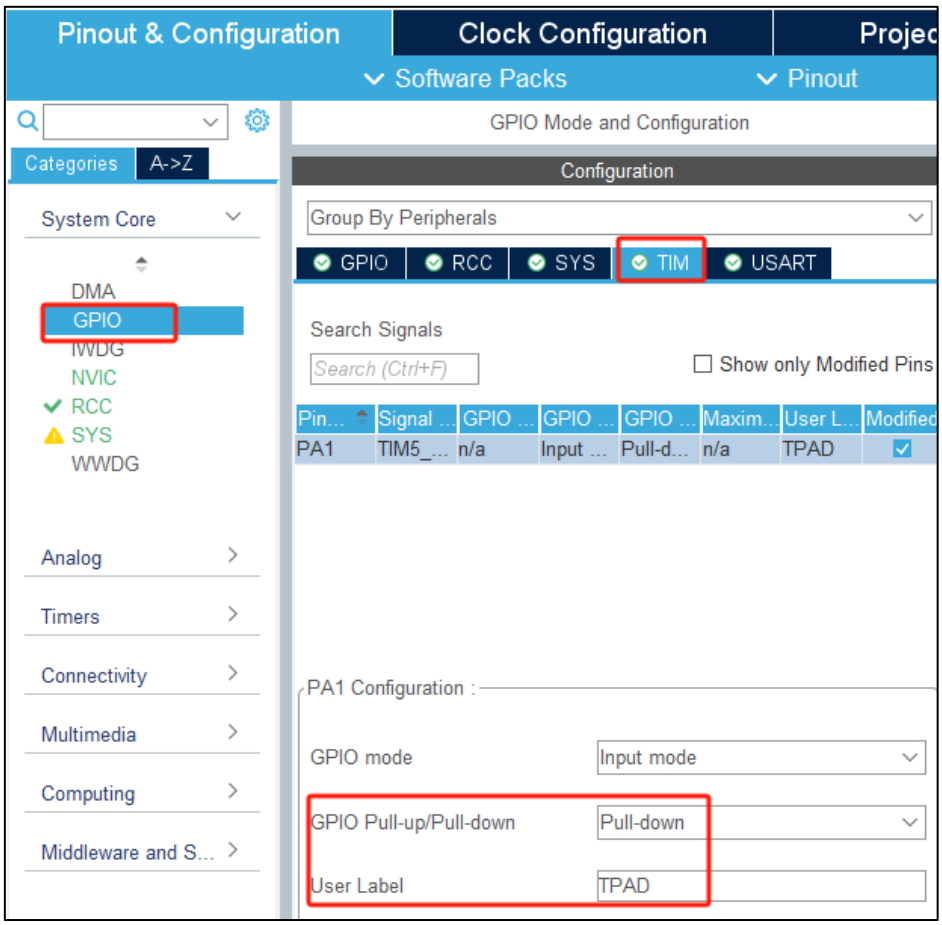


图 26.2.2.3 GPIO 配置

26.2.3 配置 TIM5 时基和捕获参数

接着打开左边的 Timers→TIM5，配置 TIM5 参数，配置如下，勾选 Internal Clock，表示选择内部时钟，Channel2 选择 Input Capture direct mode，表示使用 TIM5 的通道 2 输入捕获模式，其它选项保持默认配置。

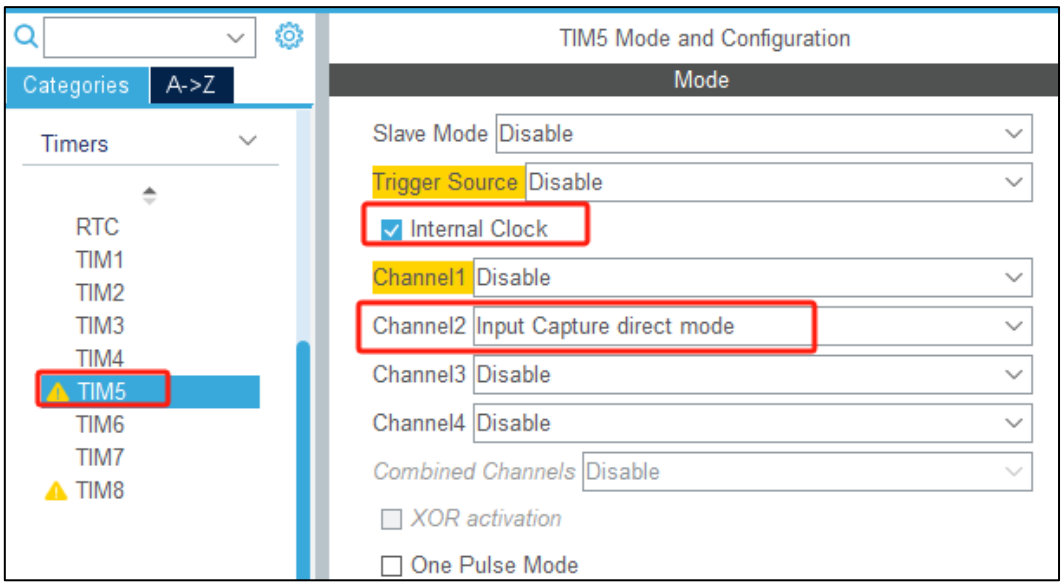


图 26.2.3.1 TIM5 时钟配置

在 Parameter Settings 处配置参数如下：

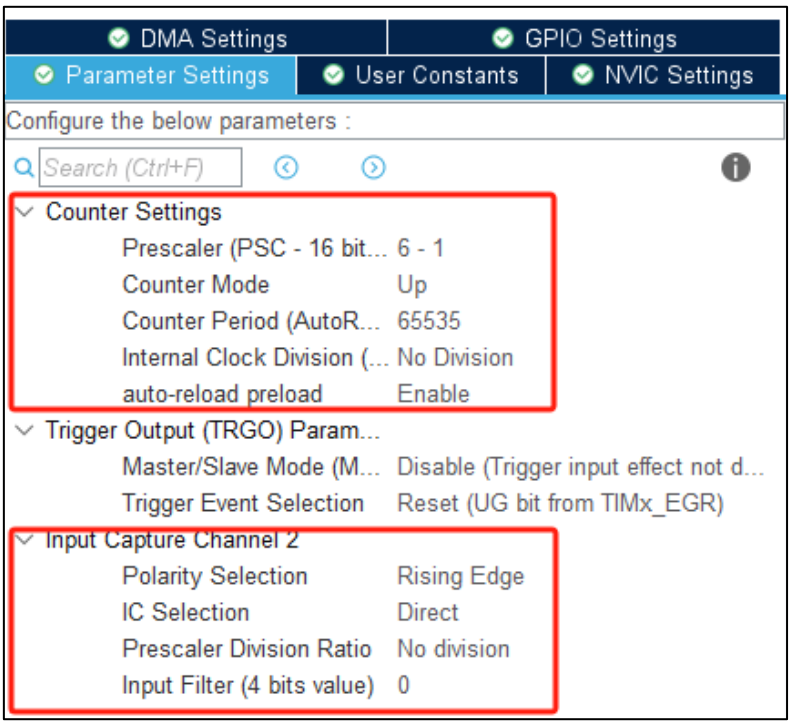


图 26.2.3.2 TIM5 通道一配置

上图的参数介绍，在输入捕获章节已经介绍过，这里不在赘述。需要注意的点是分频系数的值大小，该值越小，电容触摸按键越灵敏，最小值为：1。

26.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM5 的初始化。

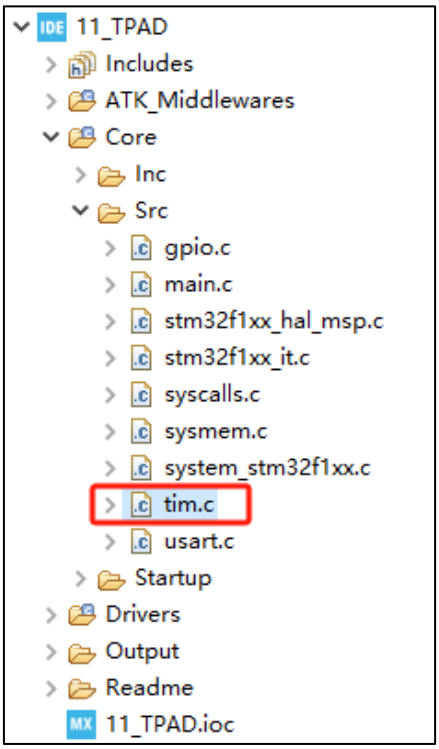


图 26.2.4.1 生成代码

26.2.5 添加用户驱动代码

1. 新建用户文件

在 Drivers/BSP 文件夹中，新建 TPAD 文件夹，然后再 TPAD 文件夹下新建两个文件，分别是 tpad.c 和 tpad.h，如下图所示：

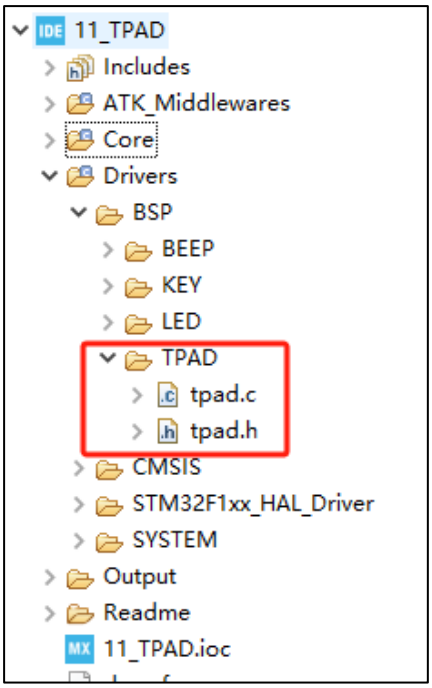


图 26.2.5.1 新建 TPAD 文件夹

2. 添加用户驱动代码

我们本实验主要对 tpad.c、tim.c 和 main.c 这几个文件进行修改和添加代码。首先，我们来看一下对 tim.c 文件的修改。

（1）修改 tim.c 文件

```
void MX_TIM5_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_IC_InitTypeDef sConfigIC = {0};
  htim5.Instance = TIM5;
  htim5.Init.Prescaler = 6 - 1;
  htim5.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim5.Init.Period = 65535;
  htim5.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim5.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim5) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim5, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_IC_Init(&htim5) != HAL_OK)
```

```
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim5, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
  sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
  sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
  sConfigIC.ICFilter = 0;
  if (HAL_TIM_IC_ConfigChannel(&htim5, &sConfigIC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM5_Init 2 */
  HAL_TIM_IC_Start(&htim5, TIM_CHANNEL_2);      /* 使能输入捕获和定时器 */
  /* USER CODE END TIM5_Init 2 */

}
```

在 TIM5 初始化函数中，添加了 HAL_TIM_IC_Start 函数，用于开启 TIM5_CH2 的输入捕获。

（2）在 tpad.c 文件添加如下代码

```
#include "tpad.h"
#include "../../SYSTEM/delay/delay.h"
#include "usart.h"
#include "tim.h"

/* 空载的时候(没有手按下),计数器需要的时间
 * 这个值应该在每次开机的时候被初始化一次
 */
volatile uint16_t g_tpad_default_val = 0;    /* 空载的时候(没按)计数器需要的时间 */

/* 静态函数, 仅限 tapd.c 调用 */
static void tpad_reset(void);                /* 复位 */
static uint16_t tpad_get_val(void);          /* 得到定时器捕获值 */
static uint16_t tpad_get_maxval(uint8_t n); /* 读取 n 次, 获取最大值 */

/**
 * @brief       初始化触摸按键
 * @param       psc     : 分频系数(值越小, 越灵敏, 最小值为: 1)
 * @retval      0, 初始化成功; 1, 初始化失败;
 */
uint8_t tpad_init(uint16_t psc)
{
    uint16_t buf[10];
    uint16_t temp;
    uint8_t j, i;
    MX_TIM5_Init();

    for (i = 0; i < 10; i++)     /* 连续读取 10 次 */
    {
        buf[i] = tpad_get_val();
        delay_ms(10);
    }

    for (i = 0; i < 9; i++)      /* 排序 */
    {
        for (j = i + 1; j < 10; j++)
```

```
        {
            if (buf[i] > buf[j])/* 升序排列 */
            {
                temp = buf[i];
                buf[i] = buf[j];
                buf[j] = temp;
            }
        }
    }

    temp = 0;

    for (i = 2; i < 8; i++)      /* 取中间的 6 个数据进行平均 */
    {
        temp += buf[i];
    }

    g_tpad_default_val = temp / 6;
    printf("g_tpad_default_val:%d\r\n", g_tpad_default_val);

    if (g_tpad_default_val > TPAD_ARR_MAX_VAL / 2)
    {
        return 1;    /* 初始化遇到超过 TPAD_ARR_MAX_VAL/2 的数值,不正常! */
    }

    return 0;
}

/**
 * @brief       复位 TPAD
 *   @note      我们将 TPAD 按键看做是一个电容, 当手指按下/不按下时容值有变化
 *              该函数将 GPIO 设置成推挽输出, 然后输出 0, 进行放电, 然后再设置
 *              GPIO 为浮空输入, 等待外部大电阻慢慢充电
 * @param       无
 * @retval      无
 */
static void tpad_reset(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    gpio_init_struct.Pin = TPAD_Pin;                    /* 输入捕获的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;        /* 复用推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;    /* 中速 */
    HAL_GPIO_Init(TPAD_GPIO_Port, &gpio_init_struct);
   /* TPAD 引脚输出 0, 放电 */
    HAL_GPIO_WritePin(TPAD_GPIO_Port, TPAD_Pin, GPIO_PIN_RESET);

    delay_ms(5);

    htim5.Instance->SR = 0;                             /* 清除标记 */
    htim5.Instance->CNT = 0;                            /* 归零 */

    gpio_init_struct.Pin = TPAD_Pin;                    /* 输入捕获的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;            /* 复用推挽输出 */
    gpio_init_struct.Pull = GPIO_NOPULL;                /* 浮空 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;    /* 中速 */
    HAL_GPIO_Init(TPAD_GPIO_Port, &gpio_init_struct);   /* TPAD 引脚浮空输入 */
}

/**
 * @brief       得到定时器捕获值
 *   @note      如果超时, 则直接返回定时器的计数值
 *              我们定义超时时间为: TPAD_ARR_MAX_VAL - 500
```

```
 * @param       无
 * @retval      捕获值/计数值（超时的情况下返回）
 */
uint16_t tpad_get_val(void)
{
    uint32_t flag = (TPAD_TIMX_CAP_CHY== TIM_CHANNEL_1)?TIM_FLAG_CC1:\
                    (TPAD_TIMX_CAP_CHY== TIM_CHANNEL_2)?TIM_FLAG_CC2:\
                    (TPAD_TIMX_CAP_CHY==
TIM_CHANNEL_3)?TIM_FLAG_CC3:TIM_FLAG_CC4;

    tpad_reset();
    while (__HAL_TIM_GET_FLAG(&htim5 ,flag) == RESET)  /* 等待通道 CHY 捕获上升沿
*/
    {
        if (htim5.Instance->CNT > TPAD_ARR_MAX_VAL - 500)
        {
            return htim5.Instance->CNT;                 /* 超时了,直接返回 CNT 值*/
        }
    }

    return __HAL_TIM_GET_COMPARE(&htim5, TPAD_TIMX_CAP_CHY);/* 返回捕获值 */
}

/**
 * @brief       读取 n 次, 取最大值
 * @param       n       : 连续获取的次数
 * @retval      n 次读数里面读到的最大读数值
 */
static uint16_t tpad_get_maxval(uint8_t n)
{
    uint16_t temp = 0;
    uint16_t maxval = 0;

    while (n--)
    {
        temp = tpad_get_val();  /* 得到一次值 */

        if (temp > maxval)maxval = temp;
    }

    return maxval;
}

/**
 * @brief       扫描触摸按键
 * @param       mode    : 扫描模式
 *   @arg       0, 不支持连续触发(按下一次必须松开才能按下一次);
 *   @arg       1, 支持连续触发(可以一直按下)
 * @retval      0, 没有按下; 1, 有按下;
 */
uint8_t tpad_scan(uint8_t mode)
{
    static uint8_t keyen = 0;   /* 0, 可以开始检测;  >0, 还不能开始检测; */
    uint8_t res = 0;
    uint8_t sample = 3;         /* 默认采样次数为 3 次 */
    uint16_t rval;

    if (mode)
    {
        sample = 6;             /* 支持连按的时候, 设置采样次数为 6 次 */
        keyen = 0;              /* 支持连按, 每次调用该函数都可以检测 */
    }

    rval = tpad_get_maxval(sample);
```

```
    if (rval > (g_tpad_default_val + TPAD_GATE_VAL))/* 大于
tpad_default_val+TPAD_GATE_VAL,有效 */
    {
        if (keyen == 0)
        {
            res = 1;     /* keyen==0, 有效 */
        }
        keyen = 3;       /* 至少要再过 3 次之后才能按键有效 */
    }

    if (keyen)keyen--;

    return res;
}
```

tpad_reset 函数，该函数顾名思义就是进行一次复位操作。先设置 PA1 输出低电平，电容放电，同时清除中断标志位并且计数器值清零，然后配置 PA1 为复用功能浮空输入，利用外部上拉电阻给电容 C 充电，同时开启 TIM5_CH2 的的输入捕获。

而实现充电时间的获取主要是通过调用 tpad_get_val 函数。函数 tpad_get_val 得到定时器的一次捕获值，函数内部先调用 tpad_reset，复位 TPAD 电容触摸按键，让其处于被充电状态，后面捕获到上升沿就把捕获到的值（或溢出值）作为返回值返回。

得到充电时间后，接下来我们要做的就是获取没有按下 TPAD 时的充电时间，并把它作为基准来确认后续有无按下操作，我们定义全局变量 g_tpad_default_val 来保存这个值，通过多次平均的滤波算法来减小误差，编写的初始化函数 tpad_init。

（3）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM5 初始化 */
  MX_TIM5_Init();
  /* USER CODE BEGIN 2 */
  tpad_init(6);                          /* 初始化触摸按键 */
  /* USER CODE END 2 */

  while (1)
  {
      if (tpad_scan(0)) /* 成功捕获到了一次上升沿(此函数执行时间至少 15ms) */
      {
          LED1_TOGGLE(); /* LED1 翻转 */
      }

      t++;
```

```
        if (t == 15)
        {
            t = 0;
            LED0_TOGGLE(); /* LED0 翻转 */
        }

        delay_ms(10);
    }
}
```

main 函数比较简单，tpad_init(6)函数执行之后，就开始触摸按键的扫描。当有触摸时，对 LED1 状态取反，而 LED0 则有规律的间隔取反，提示程序正在运行。

26.3 下载验证

下载代码后，可以看到 LED0 不停闪烁（每 300ms 闪烁一次），用手指按下电容按键时，LED1 的状态发生改变（亮灭交替一次）。这里记得 TPAD 引脚和 PA1 都是连接到开发板上的排针上的，开始测试前需要连接好，否则测试就不准了，如果下载代码前没有连接好，请连接后复位重新测试即可。

第二十七章 OLED 显示实验

本章我们来学习使用 OLED 液晶显示屏，在开发板上我们预留了 OLED 模块接口，需要准备一个 OLED 显示模块。下面我们一起来点亮 OLED，并实现 ASCII 字符的显示。

本章分为如下几个小节：

27.1 硬件设计
27.2 程序设计
27.3 下载验证

27.1 硬件设计

1， 例程功能

使用 8080 并口模式驱动 OLED 模块，不停的显示 ASCII 码和码值。LED0 闪烁，提示程序运行。

2， 硬件资源

1）LED 灯
DS0 : LED0 - PB5
2）ALIENTEK 0.96 寸 OLED 模块，在硬件上，OLED 与开发板的 IO 口对应关系如下：
OLED_CS 对应 OV_WRST，即：PD6；
OLED_RS 对应 OV_SCL，即：PD3，高电平为命令，低电平为数据；
OLED_WR 对应 OV_RRST，即：PG14；
OLED_RD 对应 DCMI_SDA，即：PG13；
OLED_RST 对应 DCMI_RESET，即：PG15；
OLED_D[7:0]对应 DCMI_D[7:0]，即：PC[7:0]

3， 原理图

OLED 模块的原理图在前面已有详细说明了，这里我们介绍 OLED 模块与我们开发板的连接，开发板上有一个 OLED/CAMERA 的接口（P4 接口）可以和正点原子 OLED 模块直接对插（**靠左插！**），连接如图 27.1.1 所示：

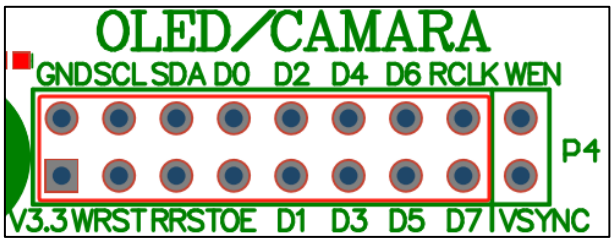


图 27.1.1 OLED 模块与开发板连接示意图

开发板上的这些 IO 与 OLED 模块的 IO 的对应关系，我们通过对比原理图，就可以得到以下表格的对应关系：

开发板 IO	OLED 模块 IO	STM32 芯片对应 GPIO
VCC3.3	VCC3.3	
OV_WRST	OLED_CS	PD6
OV_RRST	OLED_RW	PG14
OV_OE	OLED_RST	PG15
OV_D1	OLED_D1	PC1
OV_D3	OLED_D3	PC3

OV_D5	OLED_D5	PC5
OV_D7	OLED_D7	PC7
GND	GND	
OV_SCL	OLED_DC	PD3
OV_SDA	OLED_RD	PG13
OV_D0	OLED_D0	PC0
OV_D2	OLED_D2	PC2
OV_D4	OLED_D4	PC4
OV_D6	OLED_D6	PC6
OV_RCLK	悬空	悬空

表 27.1.1 OLED 模块与开发板连接示意图

这些线的连接，开发板的内部已经连接好了，我们只需要将 OLED 模块插上去就好了，注意，这里的 OLED_D[7:0] 我们接到连续的 GPIOC[7:0]上一一对应，所以只需要读一次 PC 就可以得到对应的显示数据，后续会介绍。实物连接如图 27.1.2 所示：

图 27.1.2 OLED 模块与开发板连接实物图

27.2 程序设计

27.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\12_OLED。

27.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 12_OLED，以进行下一步的实验操作。如下图所示：

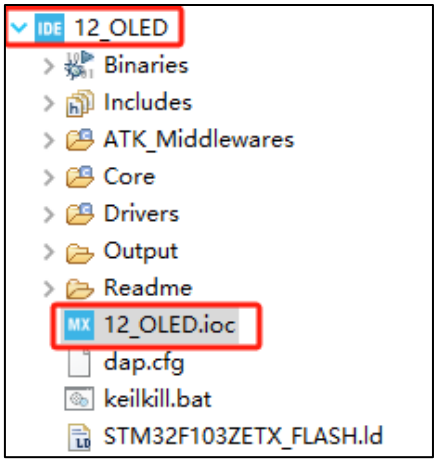


图 27.2.2.1 OLED 工程

接着双击 12_OLED.ioc 进入 STM32CubeMX 插件配置界面后，开始配置需要用到的 IO（硬件资源章节有说明）。由于配置 IO 口和时钟的配置步骤在前面我们都已经详细讲解，这里就不再重复讲解之前的步骤了。我们直接来看 IO 配置结果如下：

Pin N...	Signal o...	GPIO ou...	GPIO m...	GPIO Pu...	Maximu...	User Label	Modified
PA0-WK...	n/a	Low	Output P...	Pull-down	High	WK_UP	☑
PB5	n/a	High	Output P...	Pull-up	High	LED0	☑
PB8	n/a	Low	Output P...	Pull-down	High	BEEP	☑
PC0	n/a	High	Output P...	Pull-up	High	OLED_S...	☑
PC1	n/a	High	Output P...	Pull-up	High	OLED_S...	☑
PC2	n/a	High	Output P...	Pull-up	High		☑
PC3	n/a	High	Output P...	Pull-up	High		☑
PC4	n/a	High	Output P...	Pull-up	High		☑
PC5	n/a	High	Output P...	Pull-up	High		☑
PC6	n/a	High	Output P...	Pull-up	High		☑
PC7	n/a	High	Output P...	Pull-up	High		☑
PD3	n/a	High	Output P...	Pull-up	High	OLED_S...	☑
PD6	n/a	High	Output P...	Pull-up	High	OLED_S...	☑
PE3	n/a	High	Output P...	Pull-up	High	KEY1	☑
PE4	n/a	High	Output P...	Pull-up	High	KEY0	☑
PE5	n/a	High	Output P...	Pull-up	High	LED1	☑
PG13	n/a	High	Output P...	Pull-up	High		☑
PG14	n/a	High	Output P...	Pull-up	High		☑
PG15	n/a	High	Output P...	Pull-up	High	OLED_S...	☑

图 27.2.2.2 IO 配置

27.2.3 生成工程并添加文件

配置好之后，按下键盘的“Ctrl+S”组合键保存 12_OLED.ioc 文件，系统开始生成初始化代码。然后接着我们在 Drivers/BSP 文件夹中，新建 OLED 文件夹，然后再 OLED 文件夹下新建三个文件，分别是 oled.c 和 oled.h、font.c 和 font.h、oledfont.h 文件，如下图所示：

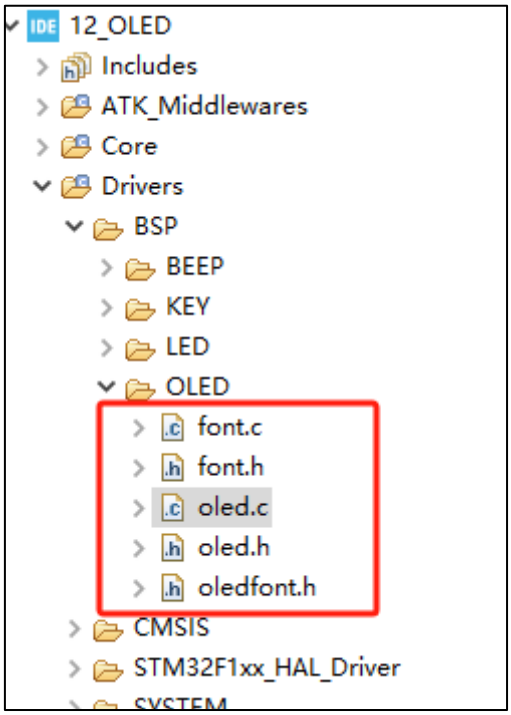


图 27.2.3.1 生成代码

其中 font.c 和 font.h 用于存放汉字和图片的字符集点阵数据，而 oledfont.h 用于存放

ASCII 码字符集点阵数据。

1，添加 oledfont.h 文件代码

oledfont.h 文件用于存放 ASCII 字符集点阵数据，也就是将 ASCII 字符集取模后得到的数据，这里我们把 ASCII 字符集按字宽和字高为 12*12、16*16 和 24*24 的大小取模出来。这里我们以按字宽和字高为 12*12 为例子做讲解。设置字体为隶书，字宽和字高都为 12，取模方式设置：阴码+逐列式+顺向+C51 格式，最后将以下 ASCII 码字符拷贝到输入框，注意，第一个字符是空格，ASCII 字符集: !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~

图 27.2.3.2 设置取模方式

最后将数据保存在一个.tex 文件中，得到的数据如下：

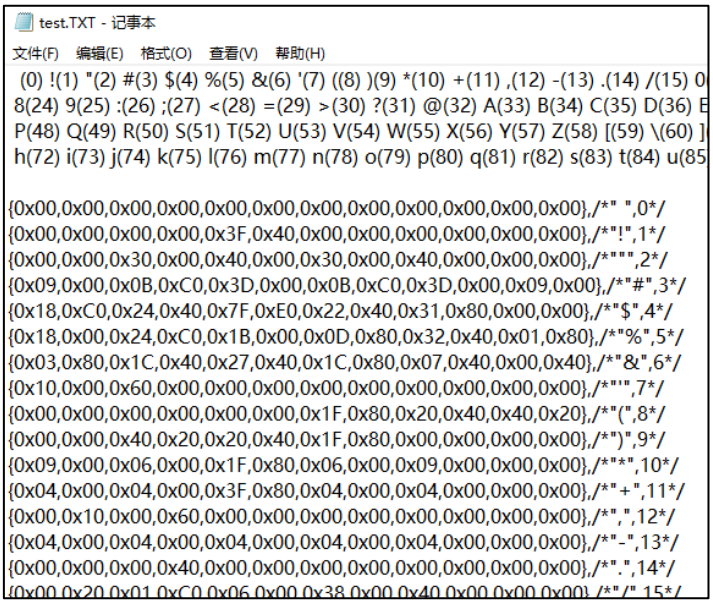


图 27.2.3.3 取模得到的点阵数据

其它字宽和字高为 16*16 和 24*24 的数据生成也是同样的方法。oledfont.h 文件的代码我们已经在本实验的工程中给出，大家可以直接使用。

2，添加 font.c 和 font.h 文件代码

font.c 和 font.h 用于存放汉字和图片的字符集点阵数据，这里以取汉字“正点原子”，大小 16*16 为例，以及图片大小 32*32 为例。

图 27.2.3.4 汉字取模

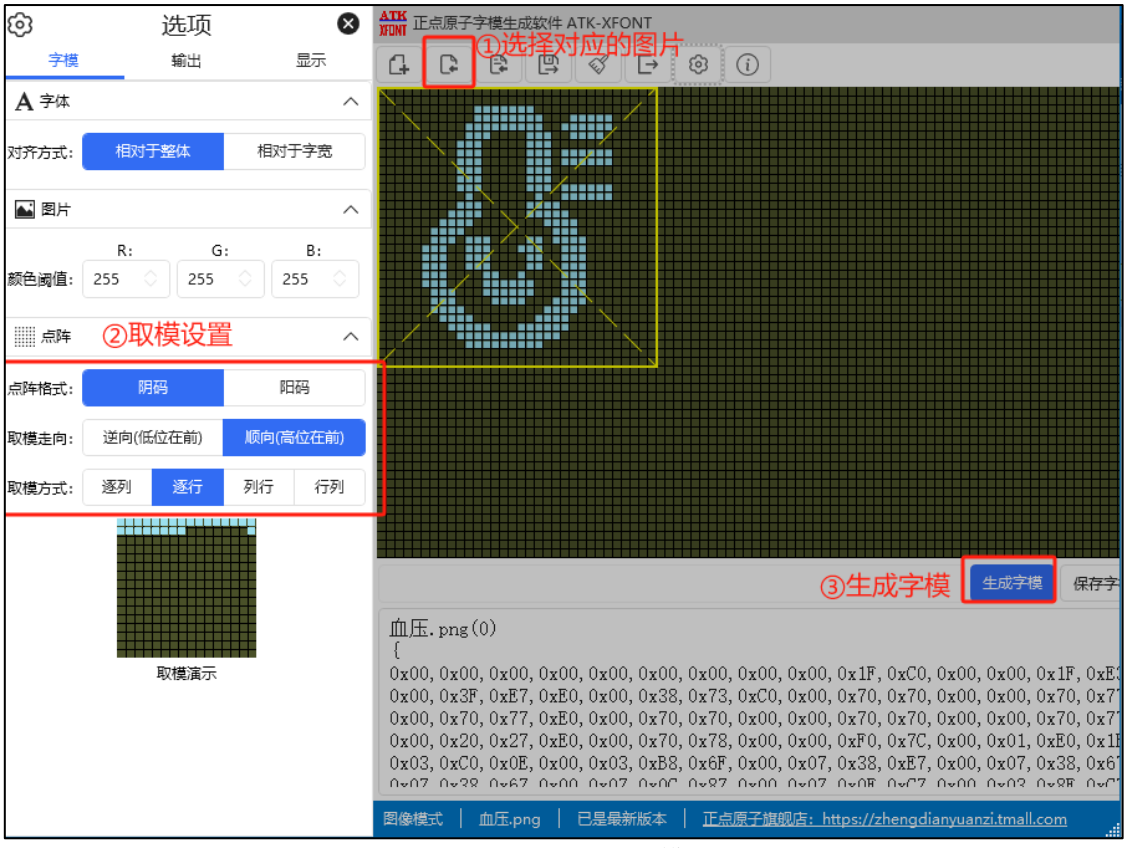


图 27.2.3.5 图片取模

其它大小的汉字和图片生成也是同样的方法。font.c 和 font.h 文件的代码我们已经在本实

验的工程中给出，大家可以直接使用。

3，添加 oled.h 文件代码

oled.h 文件代码如下：

```
/* OLED 8080 模式相关端口控制函数 定义
 * 注意:OLED_RST/OLED_CS/OLED_RS,这三个是和 80 并口模式共用的,即 80 模式也必须实现这 3 个
函数!
 */
#define OLED_RST(x)     do{ x ? \
                            HAL_GPIO_WritePin(OLED_RST_GPIO_Port,
                             OLED_RST_Pin, GPIO_PIN_SET) : \
                            HAL_GPIO_WritePin(OLED_RST_GPIO_Port,
                             OLED_RST_Pin, GPIO_PIN_RESET); \
                        }while(0)       /* 设置 RST 引脚 */

#define OLED_CS(x)      do{ x ? \
                            HAL_GPIO_WritePin(OLED_CS_GPIO_Port,
                             OLED_CS_Pin, GPIO_PIN_SET) : \
                            HAL_GPIO_WritePin(OLED_CS_GPIO_Port,
                             OLED_CS_Pin, GPIO_PIN_RESET); \
                        }while(0)       /* 设置 CS 引脚 */
#define OLED_RS(x)      do{ x ? \
                            HAL_GPIO_WritePin(OLED_RS_GPIO_Port,
                             OLED_RS_Pin, GPIO_PIN_SET) : \
                            HAL_GPIO_WritePin(OLED_RS_GPIO_Port,
                             OLED_RS_Pin, GPIO_PIN_RESET); \
                        }while(0)       /* 设置 RS 引脚 */

#define OLED_WR(x)      do{ x ? \
                            HAL_GPIO_WritePin(OLED_WR_GPIO_Port,
                             OLED_WR_Pin, GPIO_PIN_SET) :  \
                            HAL_GPIO_WritePin(OLED_WR_GPIO_Port,
                             OLED_WR_Pin, GPIO_PIN_RESET); \
                        } while (0)     /* 设置 WR 引脚 */

#define OLED_RD(x)      do{ x ? \
                            HAL_GPIO_WritePin(OLED_RD_GPIO_Port,
                             OLED_RD_Pin, GPIO_PIN_SET) : \
                            HAL_GPIO_WritePin(OLED_RD_GPIO_Port,
                             OLED_RD_Pin, GPIO_PIN_RESET); \
                        }while(0)       /* 设置 RD 引脚 */

/* 命令/数据 定义 */
#define OLED_CMD        0       /* 写命令 */
#define OLED_DATA       1       /* 写数据 */
/* OLED 分辨率 定义 */
#define OLED_H          64      /* 垂直分辨率 */
#define OLED_W          128     /* 水平分辨率 */
```

上述代码主要实现了 8080 并口模式相关端口控制函数定义；以及写命令和写数据宏定义；

4，添加 oled.c 文件代码

下面我们来重点分析 oled.c 文件的代码，此文件中有很多函数，我们选择分析几个重要的函数。所有函数的代码清查看工程文件中的 oled.c 文件。

（1）oled_init 初始化函数

```
/**
 * @brief       初始化 OLED(SSD1306)
 * @param       无
 * @retval      无
 */
```

```
void oled_init(void)
{
    /* 此处为 8080 方式的 IO 初始化,已在 main 函数中初始化对应 IO 了，这里无需在初始化 IO */
    OLED_WR(1);
    OLED_RD(1);

    OLED_CS(1);
    OLED_RS(1);

    OLED_RST(0);
    delay_ms(100);
    OLED_RST(1);

    oled_wr_byte(0xAE, OLED_CMD);    /* 关闭显示 */
    oled_wr_byte(0xD5, OLED_CMD);    /* 设置时钟分频因子,震荡频率 */
    oled_wr_byte(80, OLED_CMD);      /* [3:0],分频因子;[7:4],震荡频率 */
    oled_wr_byte(0xA8, OLED_CMD);    /* 设置驱动路数 */
    oled_wr_byte(0X3F, OLED_CMD);    /* 默认 0X3F(1/64) */
    oled_wr_byte(0xD3, OLED_CMD);    /* 设置显示偏移 */
    oled_wr_byte(0X00, OLED_CMD);    /* 默认为 0 */

    oled_wr_byte(0x40, OLED_CMD);    /* 设置显示开始行 [5:0],行数. */

    oled_wr_byte(0x8D, OLED_CMD);    /* 电荷泵设置 */
    oled_wr_byte(0x14, OLED_CMD);    /* bit2，开启/关闭 */
    oled_wr_byte(0x20, OLED_CMD);    /* 设置内存地址模式 */
    oled_wr_byte(0x02, OLED_CMD);    /* 00，列地址模式;01，行地址模式;10,页地址模式*/
    oled_wr_byte(0xA1, OLED_CMD);    /* 段重定义设置,bit0:0,0->0;1,0->127; */
    oled_wr_byte(0xC8, OLED_CMD);
    oled_wr_byte(0xDA, OLED_CMD);    /* 设置 COM 硬件引脚配置 */
    oled_wr_byte(0x12, OLED_CMD);    /* [5:4]配置 */

    oled_wr_byte(0x81, OLED_CMD);    /* 对比度设置 */
    oled_wr_byte(0xEF, OLED_CMD);    /* 1~255;默认 0X7F (亮度设置,越大越亮) */
    oled_wr_byte(0xD9, OLED_CMD);    /* 设置预充电周期 */
    oled_wr_byte(0xf1, OLED_CMD);    /* [3:0],PHASE 1;[7:4],PHASE 2; */
    oled_wr_byte(0xDB, OLED_CMD);    /* 设置 VCOMH 电压倍率 */
    oled_wr_byte(0x30, OLED_CMD);

    oled_wr_byte(0xA4, OLED_CMD);    /* 显示开启;bit0:1,开启;0,关闭;(白屏/黑屏) */
    oled_wr_byte(0xA6, OLED_CMD);    /* 设置显示方式;bit0:1,反相显示;0,正常显示 */
    oled_wr_byte(0xAF, OLED_CMD);    /* 开启显示 */
    oled_clear();
}
```

OLED 的初始化函数中，主要实现了一些初始化序列了，我们按照厂家提供的资料来做就可以。值得注意一点的是，因为 OLED 是无背光的，在初始化之后，我们把显存都清空了，所以我们在屏幕上是看不到任何内容的，就像没通电一样，不要以为这就是初始化失败，要写入数据模块才会显示的。

接着，要介绍的是 oled_refresh_gram 更新显存到 OLED 函数，该函数的作用是把我们在程序中定义的二维数组 g_oled_gram 的值一次性刷新到 OLED 的显存 GRAM 中。我们在 oled.c 文件开头定义了如下一个二维数组：

```
/*
 * OLED 的显存
 * 每个字节表示 8 个像素， 128,表示有 128 列， 8 表示有 64 行， 高位表示第行数.
 * 比如 g_oled_gram[0][0]包含了第一列,第 1~8 行的数据 g_oled_gram[0][0].0 即表示坐标
(0,0)
 * 类似的: g_oled_gram[1][0].6,表示坐标(1,1), g_oled_gram[10][1].2,表示坐标
(10,10),
 * 存放格式如下(高位表示高行数).
 * [0]0 1 2 3 ... 127
```

```
 * [1]0 1 2 3 ... 127
 * [2]0 1 2 3 ... 127
 * [3]0 1 2 3 ... 127
 * [4]0 1 2 3 ... 127
 * [5]0 1 2 3 ... 127
 * [6]0 1 2 3 ... 127
 * [7]0 1 2 3 ... 127
 */
static uint8_t g_oled_gram[128][8];
```

该数组值与 OLED 显存 GRAM 值一一对应。在操作的时候我们只需要先修改该数组的值，然后再通过调用 oled_refresh_gram 函数把数组的值一次性刷新到 OLED 的 GRAM 上即可。oled_refresh_gram 函数定义如下：

```
/**
 * @brief       更新显存到 OLED
 * @param       无
 * @retval      无
 */
void oled_refresh_gram(void)
{
    uint8_t i, n;
    for (i = 0; i < 8; i++)
    {
        oled_wr_byte (0xb0 + i, OLED_CMD); /* 设置页地址（0~7） */
        oled_wr_byte (0x00, OLED_CMD);      /* 设置显示位置—列低地址 */
        oled_wr_byte (0x10, OLED_CMD);      /* 设置显示位置—列高地址 */
        for (n = 0; n < 128; n++)
        {
            oled_wr_byte(g_oled_gram[n][i], OLED_DATA);
        }
    }
}
```

oled_refresh_gram 函数先设置页地址，然后写入列地址（也就是纵坐标），然后从 0 开始写入 128 个字节，写满该页，最后循环把 8 页的内容都写入，就实现了整个从 STM32 显存到 OLED 显存的拷贝。

oled_refresh_gram 函数还调用了 oled_wr_byte 这个函数，也就是我们接着要介绍的函数：该函数和硬件相关，8080 并口模式下该函数定义如下：

```
/**
 * @brief           向 OLED 写入一个字节
 * @param           data: 要输出的数据
 * @param           cmd: 数据/命令标志 0,表示命令;1,表示数据;
 * @retval          无
 */
static void oled_wr_byte(uint8_t data, uint8_t cmd)
{
    oled_data_out(data);
    OLED_RS(cmd);
    OLED_CS(0);
    OLED_WR(0);
    OLED_WR(1);
    OLED_CS(1);
    OLED_RS(1);
}
```

8080 并口模式下的 oled_wr_byte 函数还调用 oled_data_out 函数，其定义如下：

```
/**
 * @brief           通过拼凑的方法向 OLED 输出一个 8 位数据
 * @param           data: 要输出的数据
 * @retval          无
 */
static void oled_data_out(uint8_t data)
{
    GPIOC->ODR = (GPIOC->ODR & 0XFF00) | (data & 0X00FF);
```

```
}
```

oled_data_out 函数的处理方法，就是我们前面说的，因为 OLED 的 D0~D7 对于战舰开开发板来说正好顺序接到了 GPIOC[0:7]，所以我们只要向 GPIOC 的低位输出一次数据即可实现并口数据输出。

如果我们使用 SPI 模式，则操作模块时会调用的函数接口按以下的软件 SPI 方式实现：

```
/**
 * @brief       向 OLED 写入一个字节
 * @param       data: 要输出的数据
 * @param       cmd: 数据/命令标志 0,表示命令;1,表示数据;
 * @retval      无
 */
static void oled_wr_byte(uint8_t data, uint8_t cmd)
{
    uint8_t i;

    OLED_RS(cmd);    /* 写命令 */
    OLED_CS(0);
    for (i = 0; i < 8; i++)
    {
        OLED_SCLK(0);
        if (data & 0x80)
            OLED_SDIN(1);
        else
            OLED_SDIN(0);
        OLED_SCLK(1);
        data <<= 1;
    }
    OLED_CS(1);
    OLED_RS(1);
}
```

两种模式下的 oled_wr_byte 函数形参是一样的。第一个形参 data 就是要写的内容。第二个形参 cmd 是通过选用 OLED_CMD 和 OLED_DATA 两个宏定义的其中一个，控制选择写命令还是写数据。

接着是 OLED 画点函数，其定义如下：

```
/**
 * @brief       OLED 画点
 * @param       x  : 0~127
 * @param       y  : 0~63
 * @param       dot: 1 填充 0,清空
 * @retval      无
 */
void oled_draw_point(uint8_t x, uint8_t y, uint8_t dot)
{
    uint8_t pos, bx, temp = 0;

    if (x > 127 || y > 63) return;  /* 超出范围了 */

    pos = y / 8;    /* 计算 GRAM 里面的 y 坐标所在的字节, 每个字节可以存储 8 个行坐标 */

    bx = y % 8;     /* 取余数,方便计算 y 在对应字节里面的位置,及行(y)位置 */
    temp = 1 << bx;     /* 高位表示高行号, 得到 y 对应的 bit 位置,将该 bit 先置 1 */

    if (dot)    /* 画实心点 */
    {
        g_oled_gram[x][pos] |= temp;
    }
    else        /* 画空点,即不显示 */
    {
        g_oled_gram[x][pos] &= ~temp;
    }
}
```

该函数有 3 个形参，前两个是横纵坐标，第三个 t 为要写入 1 还是 0。该函数实现了我们在 OLED 模块上任意位置画点的功能。

前面我们知道取模方式是：从上到下，从左到右，高位在前。下面根据取模的方式来编写显示字符 oled_show_char 函数，其定义如下：

```
/**
 * @brief       在指定位置显示一个字符,包括部分字符
 * @param       x    : 0~127
 * @param       y    : 0~63
 * @param       size: 选择字体 12/16/24
 * @param       mode: 0,反白显示;1,正常显示
 * @retval      无
 */
void oled_show_char(uint8_t x,uint8_t y,uint8_t chr,uint8_t size,uint8_t mode)
{
    uint8_t temp, t, t1;
    uint8_t y0 = y;
   uint8_t *pfont = 0;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    uint8_t csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size / 2);
   chr = chr - ' ';    /* 得到偏移后的值,因为字库是从空格开始存储的,第一个字符是空格 */

    if (size == 12)             /* 调用 1206 字体 */
    {
        pfont = (uint8_t *)oled_asc2_1206[chr];
    }
    else if (size == 16)        /* 调用 1608 字体 */
    {
        pfont = (uint8_t *)oled_asc2_1608[chr];
    }
    else if (size == 24)        /* 调用 2412 字体 */
    {
        pfont = (uint8_t *)oled_asc2_2412[chr];
    }
    else                        /* 没有的字库 */
    {
        return;
    }

    for (t = 0; t < csize; t++)
    {
        temp = pfont[t];
        for (t1 = 0; t1 < 8; t1++)
        {
            if (temp & 0x80)oled_draw_point(x, y, mode);
            else oled_draw_point(x, y, !mode);

            temp <<= 1;
            y++;

            if ((y - y0) == size)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
}
```

该函数为字符以及字符串显示的核心部分，函数中 chr = chr - ' ';这句是要得到在字符点阵数据里面的实际地址，因为我们的取模是从空格键开始的，例如 oled_asc2_1206 [0][0]，代表的是空格符开始的点阵码。在接下来的代码，我们也是按照从上到小(先 y++)，从左到右（再 x++）的取模方式来编写的，先得到最高位，然后判断是写 1 还是 0，画点；接着读第二

位，如此循环，直到一个字符的点阵全部取完为止。这其中涉及到列地址和行地址的自增，根据取模方式来理解，就不难了。

接着是图片显示函数 oled_draw_bitmap：

```
/**
 * @brief       显示图片
 * @param       x,y     : 起始坐标
 * @param       *bitmap : 位图地址
 * @param       p_w     : 图片的宽度
 * @param       p_h     : 图片的高度
 * @param       mode    : 0,反白显示;1,正常显示
 * @retval      无
 */
void oled_draw_bitmap(int16_t x, int16_t y, uint8_t *bitmap, uint16_t p_w,
uint16_t p_h, uint8_t mode)
{
    if (x > (OLED_W - 1) || y > (OLED_H - 1)) return;

    uint8_t color;
    int16_t x0 = x;
    uint16_t col_bytes = (p_w >> 3) + ((p_w % 8) ? 1 : 0);  /* 一行字节数 */

    for (int col_i = 0; col_i < p_h; col_i++)               /* 行++ */
    {
        for (int col = 0; col < col_bytes; col++)           /* 显示一行 */
        {
            uint8_t temp = bitmap[col_i * col_bytes + col];

            for (int i = 0; i < 8; i++)                     /* 显示一个字节 */
            {
                if (x >= 0 && y >= 0)
                {
                    color = mode ? (temp & 0x80) : !(temp & 0x80);
                    oled_draw_point(x, y, color);
                }
                temp <<= 1;
                x++;
                /* 显示达到宽度或大于限制窗口，换行 */
                if ((x - x0) == p_w || x > (OLED_W - 1))
                {
                    x = x0;
                    y++;                                /* 指向下一行（逐行式） */
                    if (y > (OLED_H - 1)) return;       /* 达到限制窗口退出 */
                    break;
                }
            }

            if (x == x0) break; //换行
        }
    }
}
```

该函数为图片显示的核心部分，通过 bitmap 找到位图地址，之后先得到最高位，然后判断是写 1 还是 0，画点；接着读第二位，如此循环，直到一个图片的点阵全部取完为止。这其中涉及到列地址和行地址的自增，根据取模方式来理解，就不难了。

接着编写了指定大小的汉字显示函数 show_font：

```
/**
 * @brief   显示一个指定大小的汉字
 * @param   x,y     :   汉字的坐标
 * @param   font    :   汉字 GBK 码
 * @param   f_h     :   字高
 * @param   mode    :   0,反白显示;1,正常显示
 * @retval  无
```

```
 */
void show_font(uint16_t x, uint16_t y, const char *str, struct _font *font,
uint8_t mode)
{
    uint16_t fontset;
    uint8_t temp, t, t1;
    uint16_t y0 = y;
    /* 得到自由分辨字符所占的字节数 */
    uint8_t csize = (font->h / 8 + ((font->h % 8) ? 1 : 0)) * font->w;
     /* 计算 font_table 对应字库的数组下标 */
    for(fontset = 0; fontset < (strlen(font->index) >> 1); fontset++)
    {
        if(font->index[2 * fontset] == str[0] && font->index[2 * fontset + 1]
== str[1])
        {
            break;
        }
    }
    /* font_table 中没有 font 该字 */
    if(fontset >= (strlen(font->index) >> 1)) return;

    for(t = 0; t < csize; t++)
    {
        /* 找到某个汉字对应的字模数据 */
        temp = *(font->bitmap + fontset * csize + t);  /* 调用相应字库内容 */
        for(t1 = 0; t1 < 8; t1++)
        {
            if(temp & 0x80) oled_draw_point(x, y, mode);
            else oled_draw_point(x, y, !mode);
            temp <<= 1;
            y++;
            if((y - y0) == font->h)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
}
```

该函数为一个汉字以及汉字串显示的核心部分，首先通过 font->index 找到汉字索引，之后调用对应的汉字字库，先得到最高位，然后判断是写 1 还是 0，画点；接着读第二位，如此循环，直到一个汉字的点阵全部取完为止。

oled.c 的内容比较多，其他的函数请大家自行理解，下面开始 main.c 文件的介绍。

5，修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* 串口初始化 */
```

```
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */
  oled_init();                                /* 初始化 OLED */

  show_str(32, 4, "正点原子", 24, &font_24x24_mis, 1);
  oled_draw_bitmap(0, 0, (uint8_t *)zdyz_32x32_logo, 32, 32 , 1);
  oled_show_string(0, 36, "0.96' OLED TEST", 16);
  oled_show_string(0, 52, "ASCII:", 12);
  oled_show_string(64, 52, "CODE:", 12);
  oled_refresh_gram();                        /* 更新显示到 OLED */
  t = ' ';
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
      oled_show_char(36, 52, t, 12, 1);       /* 显示 ASCII 字符 */
      oled_show_num(94, 52, t, 3, 12);        /* 显示 ASCII 字符的码值 */
      oled_refresh_gram();                    /* 更新显示到 OLED */
      t++;

      if (t > '~')
      {
          t = ' ';
      }

      delay_ms(500);
      LED0_TOGGLE();                          /* LED0 闪烁 */
  }
}
```

main.c 主要功能就是在 OLED 上显示一些实验信息字符和汉字以及图片，然后开始从空格键开始不停的循环显示 ASCII 字符集，并显示该字符的 ASCII 值。最后 LED0 闪烁提示程序正在运行。

27.3 下载验证

下载代码后，LED0 不停的闪烁，提示程序已经在运行了。同时 OLED 模块显示 ASCII 字符集等信息，如下图所示（本实验使用的 OLED 模块是双色的，显示的颜色有两种）：

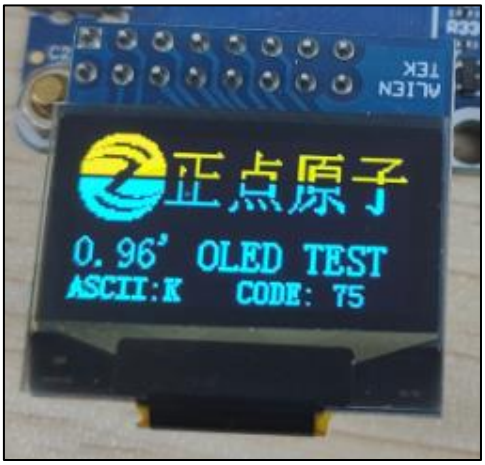


图 27.3.1 OLED 显示效果

OLED 显示了三种尺寸的字符：24*12（ALIENTEK）、16*8（0.96’OLED TEST）和 12*6（剩下的内容）。说明我们的实验是成功的，实现了三种不同尺寸 ASCII 字符的显示，在最后一行不停的显示 ASCII 字符以及其码值。

通过这一章的学习，我们学会了 ALIENTEK OLED 模块的使用，在调试代码的时候，又多了一种显示信息的途径，在以后的程序编写中，大家可以好好利用。

第二十八章 TFTLCD（MCU 屏）实验

前面我们介绍了 OLED 模块及其显示，但是该模块只能显示单色/双色，不能显示彩色，而且尺寸也较小。本章我们将介绍正点原子的 TFT LCD 模块（MCU 屏），该模块采用 TFTLCD 面板，可以显示 16 位色的真彩图片。在本章中，我们将使用开发板底板上的 TFTLCD 接口（仅支持 MCU 屏，本章仅介绍 MCU 屏的使用），来点亮 TFTLCD，并实现 ASCII 字符和彩色的显示等功能，并在串口打印 LCD 控制器 ID，同时在 LCD 上面显示。

本章分为如下几个小节：

28.1 硬件设计
28.2 程序设计
28.3 下载验证

28.1 硬件设计

1， 例程功能

使用开发板的 MCU 屏接口连接正点原子 TFTLCD 模块(仅限 MCU 屏模块)，实现 TFTLCD 模块的显示。通过把 LCD 模块插入底板上的 TFTLCD 模块接口，按下复位之后，就可以看到 LCD 模块不停的显示一些信息并不断切换底色。同时该实验会显示 LCD 驱动器的 ID，并且会在串口打印（按复位一次，打印一次）。LED0 闪烁用于提示程序正在运行。

2， 硬件资源

1）LED 灯
LED0 – PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3， 原理图

TFTLCD 模块的电路见图 28.1.2，而开发板的 LCD 接口和正点原子 TFTLCD 模块直接可以对插，开发板上的 LCD 接口如图 28.1.1 所示：

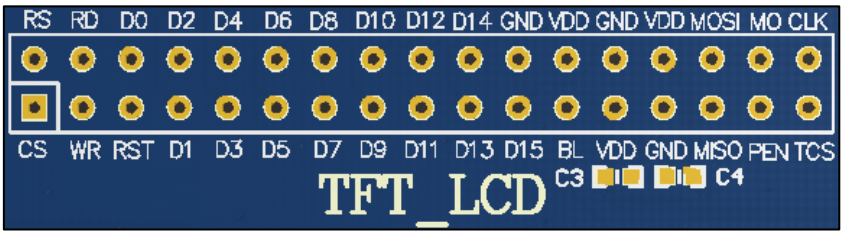


图 28.1.1 TFTLCD 模块与开发板对接的 LCD 接口示意图

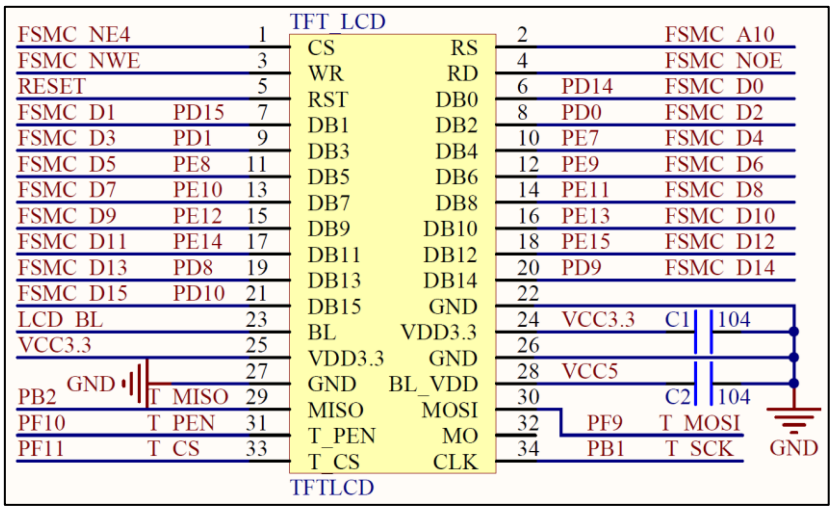


图 28.1.2 TFTLCD 模块与开发板的连接原理图

在硬件上，TFTLCD 模块与开发板的 IO 口对应关系如下：

LCD_BL(背光控制)对应 PB0；

LCD_CS 对应 PG12 即 FSMC_NE4；

LCD_RS 对应 PG0 即 FSMC_A10；

LCD_WR 对应 PD5 即 FSMC_NWE；

LCD_RD 对应 PD4 即 FSMC_NOE；

LCD _D[15:0]则直接连接在 FSMC_D15~FSMC_D0；

这些线的连接，开发板的内部已经连接好了，我们只需要将 TFTLCD 模块插上去就好了。

需要说明的是，开发板上设计的 TFT-LCD 模块插座，已经把模块模块的 RST 信号线直接接到我们开发板的复位脚上，所以不需要软件控制，这样可以省下来一个 IO 口。另外我们还需要一个背光控制线来控制 LCD 的背光灯，因为 LCD 不会自发光，没有背光灯的情况下我们是看不到 LCD 上显示的内容的。所以，我们总共需要的 IO 口数目为 22 个。

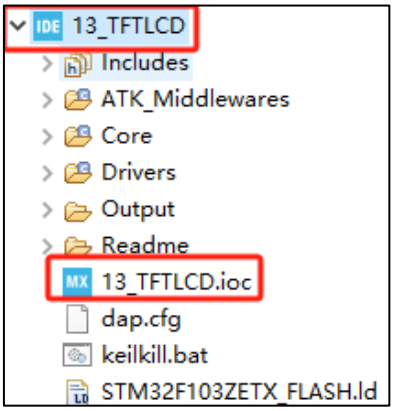
28.2 程序设计

28.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\13_TFTLCD。

28.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 05_UART 工程，并将工程文件名和.ioc 文件名均修改为 13_TFTLCD，以进行下一步的实验操作。如下图所示：



28.2.2.1 TFTLCD 工程

接着双击 13_TFTLCD.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处，配置 LCD 的背光引脚 PB0，如下图所示：

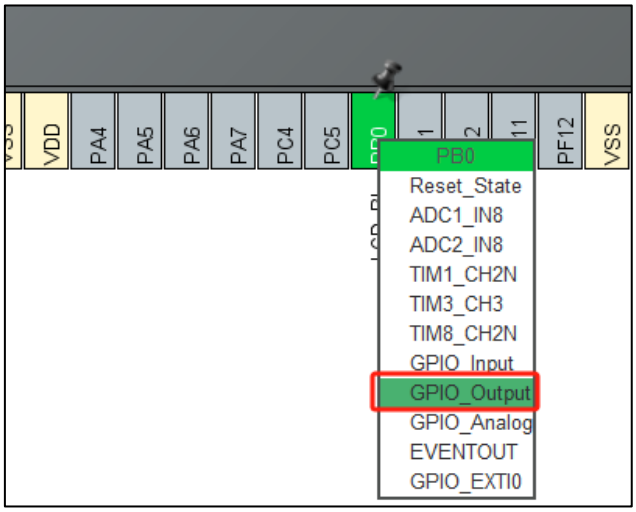


图 28.2.2.2 背光引脚配置

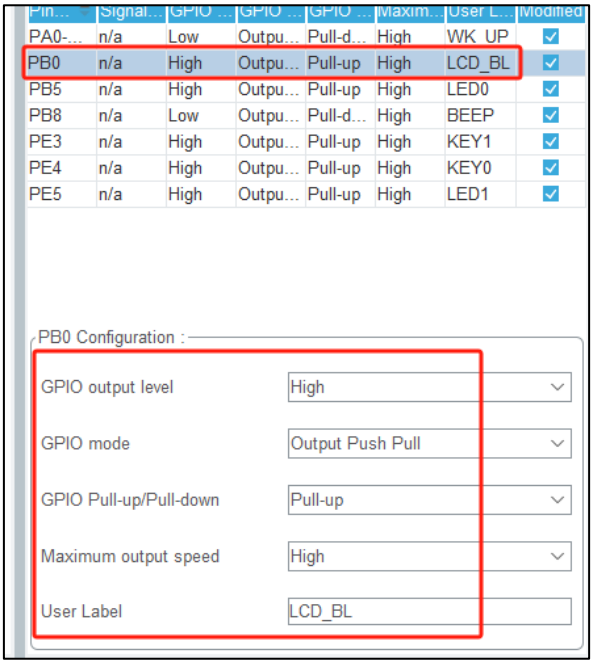


图 28.2.2.3 引脚配置

这个不属于 FSMC 的内容，如果下载程序后屏幕亮不起来那么八成是这里出问题了。

28.2.3 配置 FSMC 参数

接着，在 Connectivity→FSMC 中配置 FSMC 的参数：

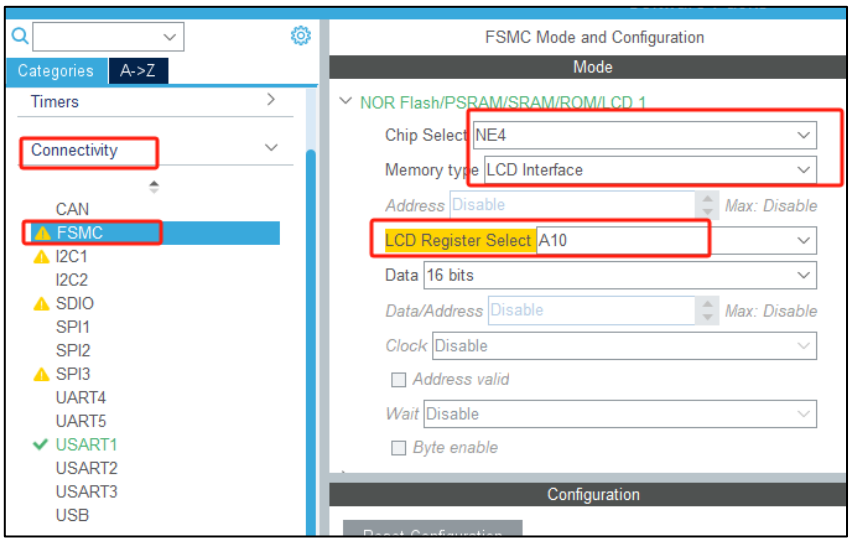


图 28.2.3.1 FSMC 配置

- **NOR Flash/PSRAM/SRAM/ROM/LCD 1**，这里选择这个代表我们使用的是 STM32 的 FSMC 存储块 1（Bank1）；
- **Chip Select**，我们这里选择 NE4，代表是 Bank1 的第四区，这部分是根据原理图的映射管脚进行选择的，这里选择不同区对应的引脚是不同的；
- **Memory type**，存储类型，这里我们驱动的是 LCD 设备，所以选择 LCD 接口。该选项还有其它类型的存储设备，感兴趣可自行了解；
- **LCD Register Select**，这里是选择 RS 脚，也就是命令/数据选择位，同样是根据原理图得知这里应该选择 A10；
- **Data**，数据位，我们的 TFTLCD 使用的是 16 位数据宽度，所以这里选择 16bits 就好。

接着来到 FSMC 的参数配置，如下图所示：

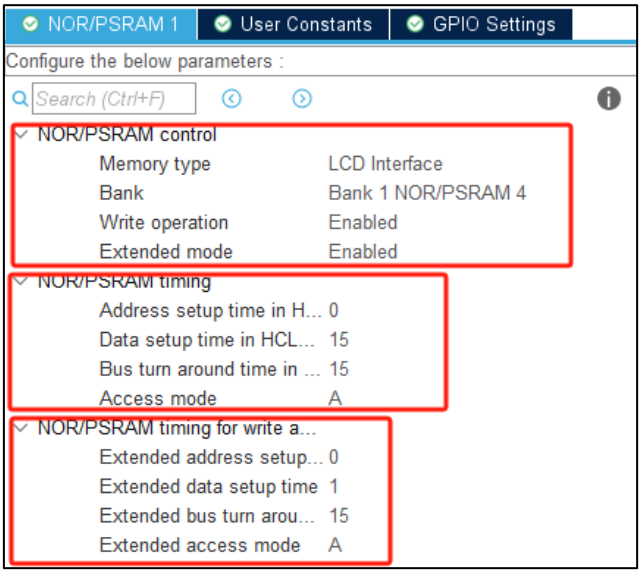


图 28.2.3.2 FSMC 参数配置

这里需要使能读写不同的时序，也就是使能 Write operation 和 Extended mode；接着由上往下介绍这些参数，分别是：

- Address setup time in HCLK clock cycles：地址建立的时钟周期
- Data setup time in HCLK clock cycles：数据建立的时钟周期

- Bus turn around time in HCLK clock cycles：总线转阶段持续时间；
- Extended address setup time：扩展地址建立时间；
- Extended data setup time：扩展数据建立时间；
- Extended bus turn around time：扩展总线建立时间；
- Access mode 和 Extended access mode 代表的读写时序控制模式，我们选择模式 A 代表支持独立的读写时序控制；

上述参数中，扩展属于写时序，其他的属于读时序。这些参数设置，是根据 LCD 芯片手册中的驱动时序填写的，不同的芯片需要根据不同的手册进行计算，但计算的方法都是一样的。

接下来这里有一件事情需要大家注意一下，就是最好将目前为止的引脚使能情况跟原理图中的一一对应，也就是检查一遍，虽然花不了多少时间，但是由于使用了大量的引脚，一旦出错那么整个工程就是失败的。

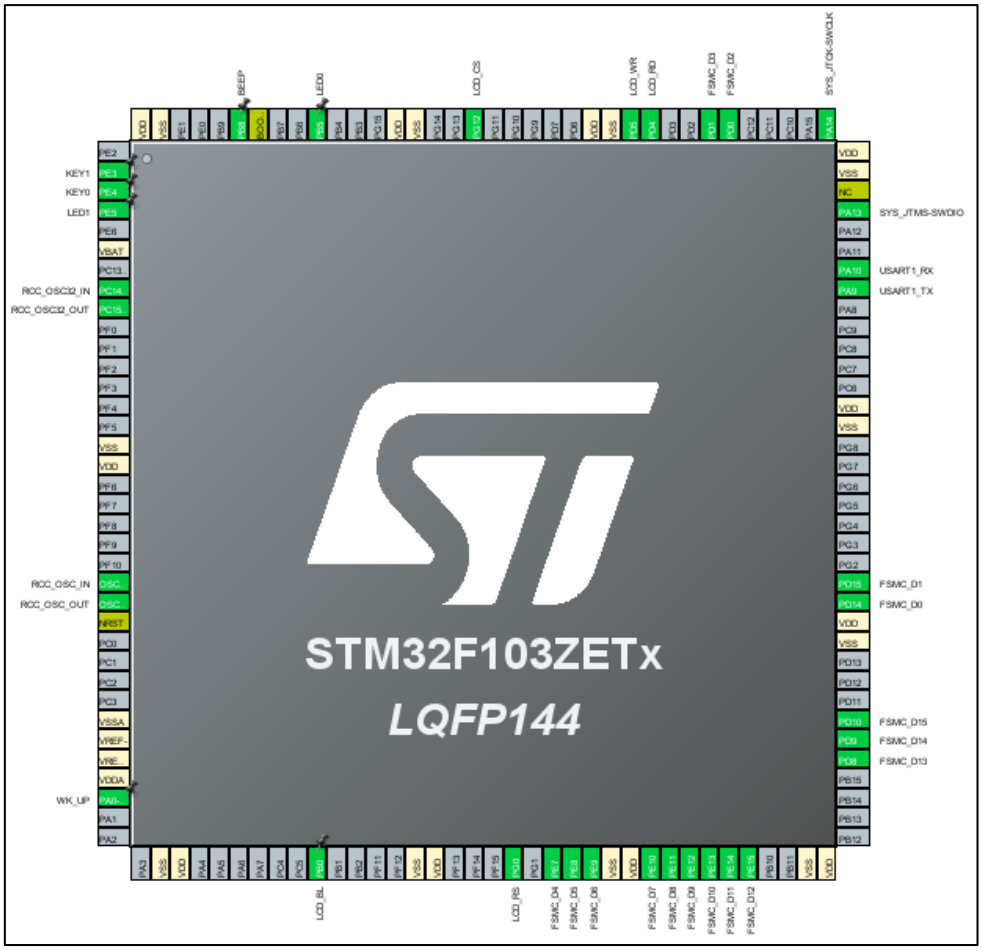


图 28.2.3.3 IO 检查与原理图一一对应

28.2.4 生成工程并添加文件

配置好之后，按下键盘的“Ctrl+S”组合键保存 13_TFTLCD.ioc 文件，系统开始生成初始化代码。然后接着我们在 Drivers/BSP 文件夹中，新建 LCD 文件夹，然后再 LCD 文件夹下新建 4 个文件，分别是 lcd.c 和 lcd.h、lcd_ex.c 和 lcdfont.h 文件，如下图所示：

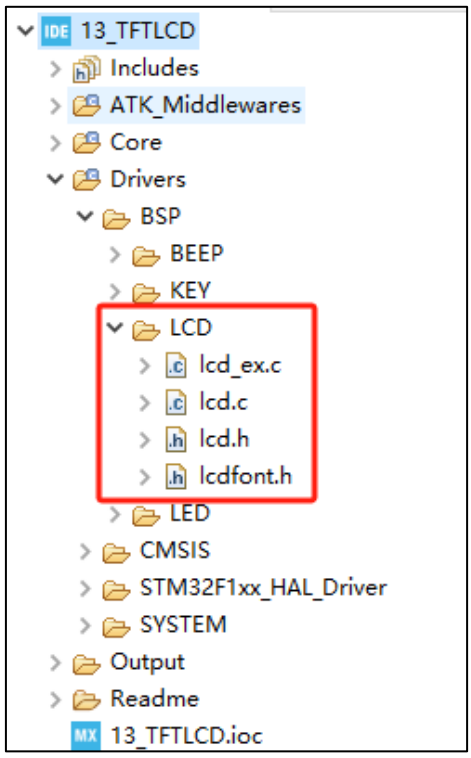


图 28.2.4.1 新建 LCD 文件夹

lcd.c 和 lcd.h 文件是驱动函数和引脚接口宏定义以及函数声明等。lcd_ex.c 存放各个 LCD 驱动 IC 的寄存器初始化部分代码，是 lcd.c 文件的补充文件，起到简化 lcd.c 文件的作用。lcdfont.h 头文件存放了 4 种字体大小不一样的 ASCII 字符集（12*12、16*16、24*24 和 32*32）。这个跟 oledfont.h 头文件一样的，只是这里多了 32*32 的 ASCII 字符集。

1，添加 lcd.h 文件代码

```
/******************************************************************************/
/* FSMC 相关参数 定义
 * 注意: 我们默认是通过 FSMC 块 1 来连接 LCD, 块 1 有 4 个片选: FSMC_NE1~4
 *
 * 修改 LCD_FSMC_NEX, 对应的 LCD_CS_GPIO 相关设置也得改
 * 修改 LCD_FSMC_AX , 对应的 LCD_RS_GPIO 相关设置也得改
 */
#define LCD_FSMC_NEX         4      /* 使用 FSMC_NE4 接 LCD_CS,取值范围只能是: 1~4*/
#define LCD_FSMC_AX          10     /* 使用 FSMC_A10 接 LCD_RS,取值范围是: 0 ~ 25*/
/* BCR 寄存器,根据 LCD_FSMC_NEX 自动计算 */
#define LCD_FSMC_BCRX        FSMC_Bank1->BTCR[(LCD_FSMC_NEX - 1) * 2]
/* BTR 寄存器,根据 LCD_FSMC_NEX 自动计算 */
#define LCD_FSMC_BTRX        FSMC_Bank1->BTCR[(LCD_FSMC_NEX - 1) * 2 + 1]
/* BWTR 寄存器,根据 LCD_FSMC_NEX 自动计算 */
#define LCD_FSMC_BWTRX       FSMC_Bank1E->BWTR[(LCD_FSMC_NEX - 1) * 2]

/******************************************************************************/

/* LCD 重要参数集 */
typedef struct
{
    uint16_t width;      /* LCD 宽度 */
    uint16_t height;     /* LCD 高度 */
    uint16_t id;         /* LCD ID */
    uint8_t dir;         /* 横屏还是竖屏控制：0，竖屏；1，横屏。 */
    uint16_t wramcmd;    /* 开始写 gram 指令 */
```

```
    uint16_t setxcmd;    /* 设置 x 坐标指令 */
    uint16_t setycmd;    /* 设置 y 坐标指令 */
} _lcd_dev;

/* LCD 参数 */
extern _lcd_dev lcddev; /* 管理 LCD 重要参数 */

/* LCD 的画笔颜色和背景色 */
extern uint32_t  g_point_color;      /* 默认红色 */
extern uint32_t  g_back_color;       /* 背景颜色.默认为白色 */

/* LCD 背光控制 */
#define LCD_BL(x)    do{ x ? \
                         HAL_GPIO_WritePin(LCD_BL_GPIO_Port, LCD_BL_Pin,
GPIO_PIN_SET) : \
                         HAL_GPIO_WritePin(LCD_BL_GPIO_Port, LCD_BL_Pin,
GPIO_PIN_RESET); \
                        }while(0)

/* LCD 地址结构体 */
typedef struct
{
    volatile uint16_t LCD_REG;
    volatile uint16_t LCD_RAM;
} LCD_TypeDef;


/* LCD_BASE 的详细解算方法:
 * 我们一般使用 FSMC 的块 1(BANK1)来驱动 TFTLCD 液晶屏(MCU 屏), 块 1 地址范围总大小为
256MB,均分成 4 块:
 * 存储块 1(FSMC_NE1)地址范围: 0X6000 0000 ~ 0X63FF FFFF
 * 存储块 2(FSMC_NE2)地址范围: 0X6400 0000 ~ 0X67FF FFFF
 * 存储块 3(FSMC_NE3)地址范围: 0X6800 0000 ~ 0X6BFF FFFF
 * 存储块 4(FSMC_NE4)地址范围: 0X6C00 0000 ~ 0X6FFF FFFF
 *
 * 我们需要根据硬件连接方式选择合适的片选(连接 LCD_CS)和地址线(连接 LCD_RS)
 * 战舰 F103 开发板使用 FSMC_NE4 连接 LCD_CS, FSMC_A10 连接 LCD_RS ,16 位数据线,计算方法
如下:
 * 首先 FSMC_NE4 的基地址为: 0X6C00 0000;      NEx 的基地址为(x=1/2/3/4): 0X6000 0000
+ (0X400 0000 * (x - 1))
 * FSMC_A10 对应地址值: 2^10 * 2 = 0X800;     FSMC_Ay 对应的地址为(y = 0 ~ 25): 2^y
* 2
 *
 * LCD->LCD_REG,对应 LCD_RS = 0(LCD 寄存器); LCD->LCD_RAM,对应 LCD_RS = 1(LCD 数据)
 * 则 LCD->LCD_RAM 的地址为:  0X6C00 0000 + 2^10 * 2 = 0X6C00 0800
 *    LCD->LCD_REG 的地址可以为 LCD->LCD_RAM 之外的任意地址.
 * 由于我们使用结构体管理 LCD_REG 和 LCD_RAM(REG 在前,RAM 在后,均为 16 位数据宽度)
 * 因此 结构体的基地址(LCD_BASE) = LCD_RAM - 2 = 0X6C00 0800 -2
 *
 * 更加通用的计算公式为((片选脚 FSMC_NEx)x=1/2/3/4, (RS 接地址线 FSMC_Ay)y=0~25):
 *          LCD_BASE = (0X6000 0000 + (0X400 0000 * (x - 1))) | (2^y * 2 -2)
 *          等效于(使用移位操作)
 *          LCD_BASE = (0X6000 0000 + (0X400 0000 * (x - 1))) | ((1 << y) * 2
-2)
 */
#define LCD_BASE        (uint32_t)((0X60000000 + (0X4000000 * (LCD_FSMC_NEX -
1))) | (((1 << LCD_FSMC_AX) * 2) -2))
#define LCD             ((LCD_TypeDef *) LCD_BASE)

/*****************************************************************************/
/* LCD 扫描方向和颜色 定义 */
```

```
/* 扫描方向定义 */
#define L2R_U2D         0           /* 从左到右,从上到下 */
#define L2R_D2U         1           /* 从左到右,从下到上 */
#define R2L_U2D         2           /* 从右到左,从上到下 */
#define R2L_D2U         3           /* 从右到左,从下到上 */

#define U2D_L2R         4           /* 从上到下,从左到右 */
#define U2D_R2L         5           /* 从上到下,从右到左 */
#define D2U_L2R         6           /* 从下到上,从左到右 */
#define D2U_R2L         7           /* 从下到上,从右到左 */

#define DFT_SCAN_DIR    L2R_U2D     /* 默认的扫描方向 */

/* 常用画笔颜色 */
#define WHITE           0xFFFF      /* 白色 */
#define BLACK           0x0000      /* 黑色 */
#define RED             0xF800      /* 红色 */
#define GREEN           0x07E0      /* 绿色 */
#define BLUE            0x001F      /* 蓝色 */
#define MAGENTA         0XF81F      /* 品红色/紫红色 = BLUE + RED */
#define YELLOW          0XFFE0      /* 黄色 = GREEN + RED */
#define CYAN            0X07FF      /* 青色 = GREEN + BLUE */

/* 非常用颜色 */
#define BROWN           0XBC40      /* 棕色 */
#define BRRED           0XFC07      /* 棕红色 */
#define GRAY            0X8430      /* 灰色 */
#define DARKBLUE        0X01CF      /* 深蓝色 */
#define LIGHTBLUE       0X7D7C      /* 浅蓝色 */
#define GRAYBLUE        0X5458      /* 灰蓝色 */
#define LIGHTGREEN      0X841F      /* 浅绿色 */
#define LGRAY           0XC618      /* 浅灰色(PANNEL),窗体背景色 */
#define LGRAYBLUE       0XA651      /* 浅灰蓝色(中间层颜色) */
#define LBBLUE          0X2B12      /* 浅棕蓝色(选择条目的反色) */

/*****************************************************************************/
/* SSD1963 相关配置参数(一般不用改) */

/* LCD 分辨率设置 */
#define SSD_HOR_RESOLUTION      800     /* LCD 水平分辨率 */
#define SSD_VER_RESOLUTION      480     /* LCD 垂直分辨率 */

/* LCD 驱动参数设置 */
#define SSD_HOR_PULSE_WIDTH     1       /* 水平脉宽 */
#define SSD_HOR_BACK_PORCH      46      /* 水平前廊 */
#define SSD_HOR_FRONT_PORCH     210     /* 水平后廊 */

#define SSD_VER_PULSE_WIDTH     1       /* 垂直脉宽 */
#define SSD_VER_BACK_PORCH      23      /* 垂直前廊 */
#define SSD_VER_FRONT_PORCH     22      /* 垂直前廊 */

/* 如下几个参数，自动计算 */
#define SSD_HT          (SSD_HOR_RESOLUTION + SSD_HOR_BACK_PORCH +
SSD_HOR_FRONT_PORCH)
#define SSD_HPS         (SSD_HOR_BACK_PORCH)
#define SSD_VT          (SSD_VER_RESOLUTION + SSD_VER_BACK_PORCH +
SSD_VER_FRONT_PORCH)
#define SSD_VPS         (SSD_VER_BACK_PORCH)

/*****************************************************************************/
```

```
/* 函数申明 */

void lcd_wr_data(volatile uint16_t data);              /* LCD 写数据 */
void lcd_wr_regno(volatile uint16_t regno);            /* LCD 写寄存器编号/地址 */
void lcd_write_reg(uint16_t regno, uint16_t data);     /* LCD 写寄存器的值 */


void lcd_init(void);                                   /* 初始化 LCD */
void lcd_display_on(void);                             /* 开显示 */
void lcd_display_off(void);                            /* 关显示 */
void lcd_scan_dir(uint8_t dir);                        /* 设置屏扫描方向 */
void lcd_display_dir(uint8_t dir);                     /* 设置屏幕显示方向 */
void lcd_ssd_backlight_set(uint8_t pwm);               /* SSD1963 背光控制 */

void lcd_write_ram_prepare(void);                      /* 准备些 GRAM */
void lcd_set_cursor(uint16_t x, uint16_t y);           /* 设置光标 */
uint32_t lcd_read_point(uint16_t x, uint16_t y);/* 读点(32 位颜色,兼容 LTDC)  */
void lcd_draw_point(uint16_t x, uint16_t y, uint32_t color);

void lcd_clear(uint16_t color);                        /* LCD 清屏 */
void lcd_fill_circle(uint16_t x, uint16_t y, uint16_t r, uint16_t color);
void lcd_draw_circle(uint16_t x0, uint16_t y0, uint8_t r, uint16_t color);
void lcd_draw_hline(uint16_t x, uint16_t y, uint16_t len, uint16_t color);
void lcd_set_window(uint16_t sx, uint16_t sy, uint16_t width, uint16_t
height);                                               /* 设置窗口 */
void lcd_fill(uint16_t sx, uint16_t sy, uint16_t ex, uint16_t ey, uint32_t
color);                                               /* 纯色填充矩形(32 位颜色,兼容 LTDC) */
void lcd_color_fill(uint16_t sx, uint16_t sy, uint16_t ex, uint16_t ey,
uint16_t *color);                                     /* 彩色填充矩形 */
void lcd_draw_line(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,
uint16_t color);                                      /* 画直线 */
void lcd_draw_rectangle(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,
uint16_t color);                                      /* 画矩形 */


void lcd_show_char(uint16_t x, uint16_t y, char chr, uint8_t size, uint8_t
mode, uint16_t color);                                /* 显示一个字符 */
void lcd_show_num(uint16_t x, uint16_t y, uint32_t num, uint8_t len, uint8_t
size, uint16_t color);                                /* 显示数字 */
void lcd_show_xnum(uint16_t x, uint16_t y, uint32_t num, uint8_t len, uint8_t
size, uint8_t mode, uint16_t color);                  /* 扩展显示数字 */
void lcd_show_string(uint16_t x, uint16_t y, uint16_t width, uint16_t height,
uint8_t size, char *p, uint16_t color);               /* 显示字符串 */
```

该文件有个重要的结构体_lcd_dev，该结构体用于保存一些 LCD 重要参数信息，比如 LCD 的长宽、LCD ID（驱动 IC 型号）、LCD 横竖屏状态等，这个结构体虽然占用了十几个字节的内存，但是却可以让我们的驱动函数支持不同尺寸的 LCD，同时可以实现 LCD 横竖屏切换等重要功能，所以还是利大于弊的。最后声明_lcd_dev 结构体类型变量 lcddev，lcddev 在 lcd.c 中定义。

紧接着就是 g_point_color 和 g_back_color 变量的声明，它们也是在 lcd.c 中被定义。g_point_color 变量用于保存 LCD 的画笔颜色，g_back_color 则是保存 LCD 的背景色。

然后还使用宏，定义的我们的背光引脚，使得操作更加便捷。

关于该文件我们这里只是简单介绍下，更详细的介绍可查看《精英开发指南》。

2，添加 lcd.c 文件代码

该文件包含大量内容，我们在此仅重点介绍其中一些关键函数。若需要了解该文件的全部内容，请查阅源代码。

```
/**
 * @brief       初始化 LCD
```

```
 *    @note       该初始化函数可以初始化各种型号的 LCD(详见本.c 文件最前面的描述)
 *
 * @param        无
 * @retval       无
 */
void lcd_init(void)
{
    delay_ms(50);         /* 初始化 FSMC 后,必须等待一定时间才能开始初始化 */

    /* 尝试 9341 ID 的读取 */
    lcd_wr_regno(0XD3);
    lcddev.id = lcd_rd_data();  /* dummy read */
    lcddev.id = lcd_rd_data();  /* 读到 0X00 */
    lcddev.id = lcd_rd_data();  /* 读取 0X93 */
    lcddev.id <<= 8;
    lcddev.id |= lcd_rd_data(); /* 读取 0X41 */

    if (lcddev.id != 0X9341)    /* 不是 9341 , 尝试看看是不是 ST7789 */
    {
        lcd_wr_regno(0X04);
        lcddev.id = lcd_rd_data();      /* dummy read */
        lcddev.id = lcd_rd_data();      /* 读到 0X85 */
        lcddev.id = lcd_rd_data();      /* 读取 0X85 */
        lcddev.id <<= 8;
        lcddev.id |= lcd_rd_data();     /* 读取 0X52 */

        if (lcddev.id == 0X8552)        /* 将 8552 的 ID 转换成 7789 */
        {
            lcddev.id = 0x7789;
        }

        if (lcddev.id != 0x7789)        /* 也不是 ST7789, 尝试是不是 NT35310 */
        {
            lcd_wr_regno(0xD4);
            lcddev.id = lcd_rd_data();  /* dummy read */
            lcddev.id = lcd_rd_data();  /* 读回 0x01 */
            lcddev.id = lcd_rd_data();  /* 读回 0x53 */
            lcddev.id <<= 8;
            lcddev.id |= lcd_rd_data(); /* 这里读回 0x10 */

            if (lcddev.id != 0x5310)    /* 也不是 NT35310,尝试看看是不是 ST7796 */
            {
                lcd_wr_regno(0XD3);
                lcddev.id = lcd_rd_data();  /* dummy read */
                lcddev.id = lcd_rd_data();  /* 读到 0X00 */
                lcddev.id = lcd_rd_data();  /* 读取 0X77 */
                lcddev.id <<= 8;
                lcddev.id |= lcd_rd_data(); /* 读取 0X96 */
                /* 也不是 ST7796,尝试看看是不是 NT35510 */
                if (lcddev.id != 0x7796)
                {
                    /* 发送密钥（厂家提供） */
                    lcd_write_reg(0xF000, 0x0055);
                    lcd_write_reg(0xF001, 0x00AA);
                    lcd_write_reg(0xF002, 0x0052);
                    lcd_write_reg(0xF003, 0x0008);
                    lcd_write_reg(0xF004, 0x0001);

                    lcd_wr_regno(0xC500);       /* 读取 ID 低八位 */
                    lcddev.id = lcd_rd_data();  /* 读回 0x55 */
                    lcddev.id <<= 8;
```

```
                    lcd_wr_regno(0xC501);       /* 读取 ID 高八位 */
                    lcddev.id |= lcd_rd_data(); /* 读回 0x10 */

                    delay_ms(5);
                    /* 也不是 NT5510,尝试看看是不是 ILI9806 */
                    if (lcddev.id != 0x5510)
                    {
                        lcd_wr_regno(0XD3);
                        lcddev.id = lcd_rd_data();  /* dummy read */
                        lcddev.id = lcd_rd_data();  /* 读回 0X00 */
                        lcddev.id = lcd_rd_data();  /* 读回 0X98 */
                        lcddev.id <<= 8;
                        lcddev.id |= lcd_rd_data(); /* 读回 0X06 */
                        /* 也不是 ILI9806,尝试看看是不是 SSD1963 */
                        if (lcddev.id != 0x9806)
                        {
                            lcd_wr_regno(0xA1);
                            lcddev.id = lcd_rd_data();
                            lcddev.id = lcd_rd_data();  /* 读回 0x57 */
                            lcddev.id <<= 8;
                            lcddev.id |= lcd_rd_data(); /* 读回 0x61 */
                            /* SSD1963 读回的 ID 是 5761H,我们强制设置为 1963 */
                            if (lcddev.id == 0x5761) lcddev.id = 0x1963;
                        }
                    }
                }
            }
        }
    }
```

该函数首先读取 LCD 控制器的型号，根据控制 IC 的型号执行不同的初始化代码，这样提高了整个程序的通用性。为了简化 lcd.c 的初始化程序，不同控制 IC 的芯片对应的初始化程序（如：lcd_ex_st7789_reginit()、lcd_ex_ili9341_reginit()等）我们放在 lcd_ex.c 文件中，这些初始化代码完成对 LCD 寄存器的初始化，由 LCD 厂家提供，一般是不需要做任何修改的，我们直接调用就可以了。

下面是 6 个简单，但是很重要的函数：

```
/**
 * @brief       LCD 写数据
 * @param           data: 要写入的数据
 * @retval          无
 */
void lcd_wr_data(volatile uint16_t data)
{
    data = data;            /* 使用-O2 优化的时候,必须插入的延时 */
    LCD->LCD_RAM = data;
}

/**
 * @brief           LCD 写寄存器编号/地址函数
 * @param           regno: 寄存器编号/地址
 * @retval          无
 */
void lcd_wr_regno(volatile uint16_t regno)
{
    regno = regno;              /* 使用-O2 优化的时候,必须插入的延时 */
    LCD->LCD_REG = regno;       /* 写入要写的寄存器序号 */
}

/**
 * @brief       LCD 写寄存器
 * @param       regno:寄存器编号/地址
 * @param           data:要写入的数据
```

```
 * @retval          无
 */
void lcd_write_reg(uint16_t regno, uint16_t data)
{
    LCD->LCD_REG = regno;    /* 写入要写的寄存器序号 */
    LCD->LCD_RAM = data;     /* 写入数据 */
}

/**
 * @brief       LCD 延时函数,仅用于部分在 mdk -O1 时间优化时需要设置的地方
 * @param           t:延时的数值
 * @retval      无
 */
static void lcd_opt_delay(uint32_t i)
{
    while (i--);     /*使用 AC6 时空循环可能被优化,可使用 while(1) __asm volatile("");
*/
}

/**
 * @brief           LCD 读数据
 * @param       无
 * @retval      读取到的数据
 */
static uint16_t lcd_rd_data(void)
{
   volatile uint16_t ram;  /* 防止被优化 */
   lcd_opt_delay(2);
    ram = LCD->LCD_RAM;
    return ram;
}

/**
 * @brief       准备写 GRAM
 * @param       无
 * @retval      无
 */
void lcd_write_ram_prepare(void)
{
    LCD->LCD_REG = lcddev.wramcmd;
}
```

因为 FSMC 自动控制了 WR/RD/CS 等这些信号，所以这 6 个函数实现起来都非常简单，我们就不多说。

下面要介绍的函数是坐标设置函数，该函数代码如下：

```
/**
 * @brief       设置光标位置(对 RGB 屏无效)
 * @param           x,y: 坐标
 * @retval          无
 */
void lcd_set_cursor(uint16_t x, uint16_t y)
{
    if (lcddev.id == 0X1963)
    {
        if (lcddev.dir == 0)        /* 竖屏模式, x 坐标需要变换 */
        {
            x = lcddev.width - 1 - x;
            lcd_wr_regno(lcddev.setxcmd);
            lcd_wr_data(0);
            lcd_wr_data(0);
            lcd_wr_data(x >> 8);
            lcd_wr_data(x & 0XFF);
        }
```

```
        else                            /* 横屏模式 */
        {
            lcd_wr_regno(lcddev.setxcmd);
            lcd_wr_data(x >> 8);
            lcd_wr_data(x & 0XFF);
            lcd_wr_data((lcddev.width - 1) >> 8);
            lcd_wr_data((lcddev.width - 1) & 0XFF);
        }

        lcd_wr_regno(lcddev.setycmd);
        lcd_wr_data(y >> 8);
        lcd_wr_data(y & 0XFF);
        lcd_wr_data((lcddev.height - 1) >> 8);
        lcd_wr_data((lcddev.height - 1) & 0XFF);

    }
    else if (lcddev.id == 0X5510)
    {
        lcd_wr_regno(lcddev.setxcmd);
        lcd_wr_data(x >> 8);
        lcd_wr_regno(lcddev.setxcmd + 1);
        lcd_wr_data(x & 0XFF);
        lcd_wr_regno(lcddev.setycmd);
        lcd_wr_data(y >> 8);
        lcd_wr_regno(lcddev.setycmd + 1);
        lcd_wr_data(y & 0XFF);
    }
    else    /* 9341/5310/7789/7796/9806 等 设置坐标 */
    {
        lcd_wr_regno(lcddev.setxcmd);
        lcd_wr_data(x >> 8);
        lcd_wr_data(x & 0XFF);
        lcd_wr_regno(lcddev.setycmd);
        lcd_wr_data(y >> 8);
        lcd_wr_data(y & 0XFF);
    }
}
```

该函数实现将 LCD 的当前操作点设置到指定坐标(x,y)。因为 9341/5310/1963/5510 等的设置有些不太一样，所以进行了区别对待。

接下来介绍画点函数，其定义如下：

```
/**
 * @brief       画点
 * @param           x,y: 坐标
 * @param           color: 点的颜色(32 位颜色,方便兼容 LTDC)
 * @retval          无
 */
void lcd_draw_point(uint16_t x, uint16_t y, uint32_t color)
{
    lcd_set_cursor(x, y);       /* 设置光标位置 */
    lcd_write_ram_prepare();    /* 开始写入 GRAM */
    LCD->LCD_RAM = color;
}
```

第十个要介绍的是字符显示函数 lcd_show_char，该函数同前面 OLED 模块的字符显示函数差不多，但是这里的字符显示函数多了 1 个功能，就是可以以叠加方式显示，或者以非叠加方式显示。叠加方式显示多用于在显示的图片上再显示字符。非叠加方式一般用于普通的显示。该函数实现代码如下：

```
/**
 * @brief       在指定位置显示一个字符
 * @param           x,y  : 坐标
 * @param           chr  : 要显示的字符:" "--->"~"
 * @param           size : 字体大小 12/16/24/32
 * @param           mode : 叠加方式(1); 非叠加方式(0);
```

```
 * @retval          无
 */
void lcd_show_char(uint16_t x, uint16_t y, char chr, uint8_t size,
                   uint8_t mode, uint16_t color)
{
    uint8_t temp, t1, t;
    uint16_t y0 = y;
    uint8_t csize = 0;
    uint8_t *pfont = 0;
   /* 得到字体一个字符对应点阵集所占的字节数 */
   csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size / 2);
   /* 得到偏移后的值（ASCII 字库是从空格开始取模，所以-' '就是对应字符的字库） */
    chr = chr - ' ';

    switch (size)
    {
        case 12:
            pfont = (uint8_t *)asc2_1206[chr];  /* 调用 1206 字体 */
            break;

        case 16:
            pfont = (uint8_t *)asc2_1608[chr];  /* 调用 1608 字体 */
            break;

        case 24:
            pfont = (uint8_t *)asc2_2412[chr];  /* 调用 2412 字体 */
            break;

        case 32:
            pfont = (uint8_t *)asc2_3216[chr];  /* 调用 3216 字体 */
            break;

        default:
            return ;
    }

    for (t = 0; t < csize; t++)
    {
        temp = pfont[t];                    /* 获取字符的点阵数据 */

        for (t1 = 0; t1 < 8; t1++)      /* 一个字节 8 个点 */
        {
            if (temp & 0x80)                /* 有效点,需要显示 */
            {
                lcd_draw_point(x, y, color);        /* 画点出来,要显示这个点 */
            }
            else if (mode == 0)             /* 无效点,不显示 */
            {
             /* 画背景色,相当于这个点不显示(注意背景色由全局变量控制) */
                lcd_draw_point(x, y, g_back_color);
            }

            temp <<= 1;         /* 移位, 以便获取下一个位的状态 */
            y++;

            if (y >= lcddev.height)return;  /* 超区域了 */

            if ((y - y0) == size)       /* 显示完一列了? */
            {
                y = y0;     /* y 坐标复位 */
                x++;        /* x 坐标递增 */

                if (x >= lcddev.width)return;   /* x 坐标超区域了 */
```

```
                break;
            }
        }
    }
}
```

在 lcd_show_char 函数里面，我们用到了四个字符集点阵数据数组 asc2_1206、asc2_1608、asc2_2412 和 asc2_3216。

lcd.c 的函数比较多，其他的函数请大家自行查看源码，都有详细的注释。

3，修改 main.c 代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
  uint8_t x = 0;
  uint8_t lcd_id[12];
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                          /* 初始化 LCD */
  g_point_color = RED;
  sprintf((char *)lcd_id, "LCD ID:%04X", lcddev.id);   /* ID 打印到 lcd_id 数组 */
  /* USER CODE END 2 */

  while (1)
  {
      switch (x)
      {
      case 0:
          lcd_clear(WHITE);
          break;

      case 1:
          lcd_clear(BLACK);
          break;

      case 2:
          lcd_clear(BLUE);
          break;

      case 3:
          lcd_clear(RED);
          break;

      case 4:
          lcd_clear(MAGENTA);
          break;
```

```
        case 5:
            lcd_clear(GREEN);
            break;

        case 6:
            lcd_clear(CYAN);
            break;

        case 7:
            lcd_clear(YELLOW);
            break;

        case 8:
            lcd_clear(BRRED);
            break;

        case 9:
            lcd_clear(GRAY);
            break;

        case 10:
            lcd_clear(LGRAY);
            break;

        case 11:
            lcd_clear(BROWN);
            break;
        }

        lcd_show_string(10, 40, 240, 32, 32, "STM32", RED);
        lcd_show_string(10, 80, 240, 24, 24, "TFTLCD TEST", RED);
        lcd_show_string(10, 110, 240, 16, 16, "ATOM@ALIENTEK", RED);
        lcd_show_string(10, 130, 240, 16, 16, (char *)lcd_id, RED);
        x++;

        if (x == 12)
            x = 0;

        LED0_TOGGLE(); /*红灯闪烁*/
        delay_ms(1000);
    }
}
```

main 函数功能主要是显示一些固定的字符，字体大小包括 32*16、24*12、16*8 和 12*6 四种，同时显示 LCD 驱动 IC 的型号，然后不停的切换背景颜色，每 1s 切换一次。而 LED0 也会不停的闪烁，指示程序已经在运行了。其中我们用到一个 sprintf 的函数，该函数用法同 printf，只是 sprintf 把打印内容输出到指定的内存区间上，最终在死循环中通过 lcd_show_strinig 函数进行屏幕显示，sprintf 的详细用法，请百度学习。

特别注意：

串口一记得要初始化，因为在 lcd_init 函数里面调用了 printf，所以一旦去掉这个初始化，就会死机！实际上，只要你的代码有用到 printf，就必须初始化串口，否则都会死机，停在 usart.c 里面的 fputc 函数出不来。

修改好上述这些文件后，直接编译工程会发现会编译错误，如下图所示：

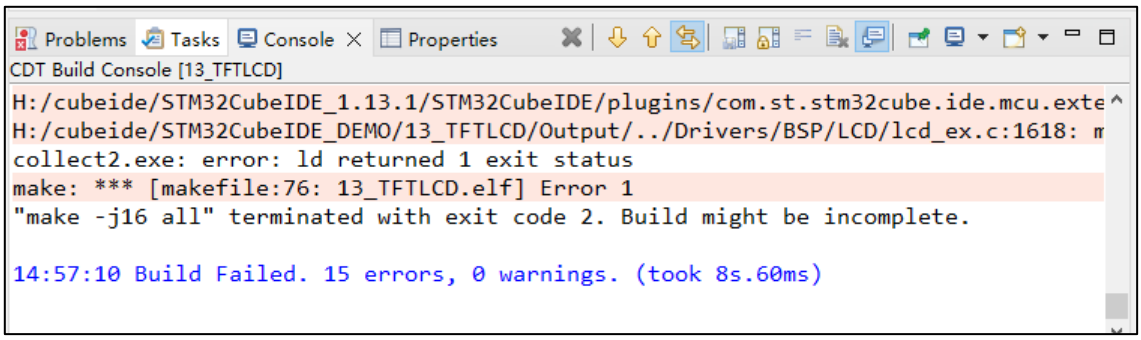


图 28.2.4.2 编译失败提示

这是因为在 lcd.c 中，我们使用了 include 的形式添加 lcd_ex.c 文件，所以不能再我们的工程中对它进行编译，那应该如何过滤掉该文件呢？可以通过以下步骤设置：

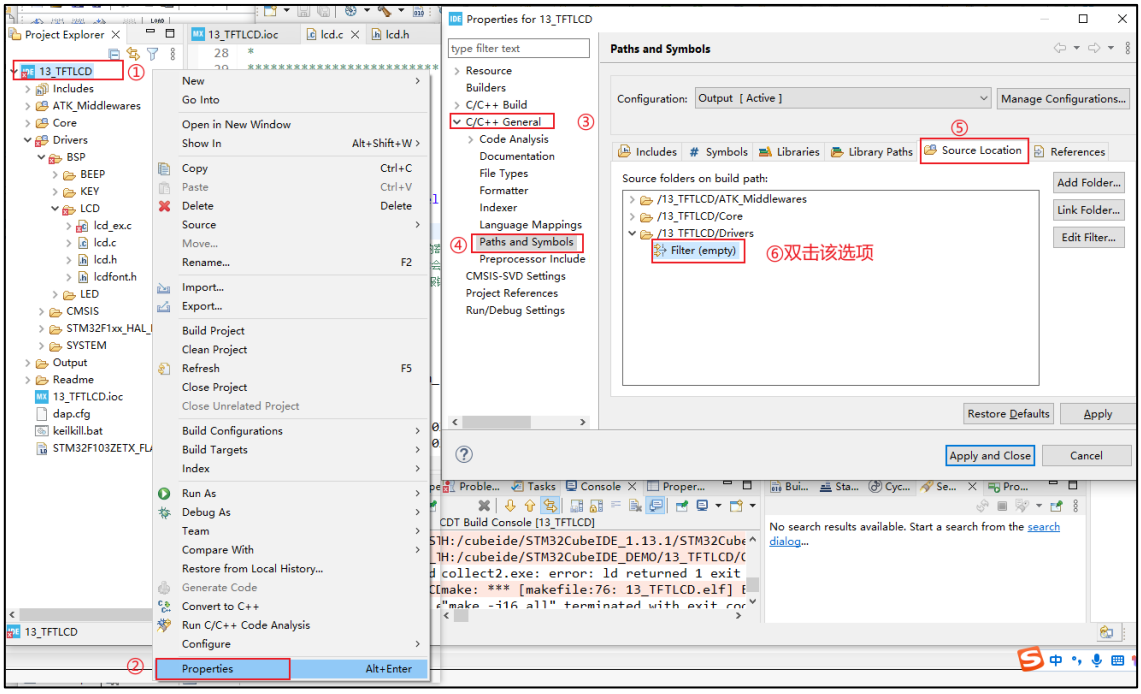


图 28.2.4.3 设置过滤器步骤一

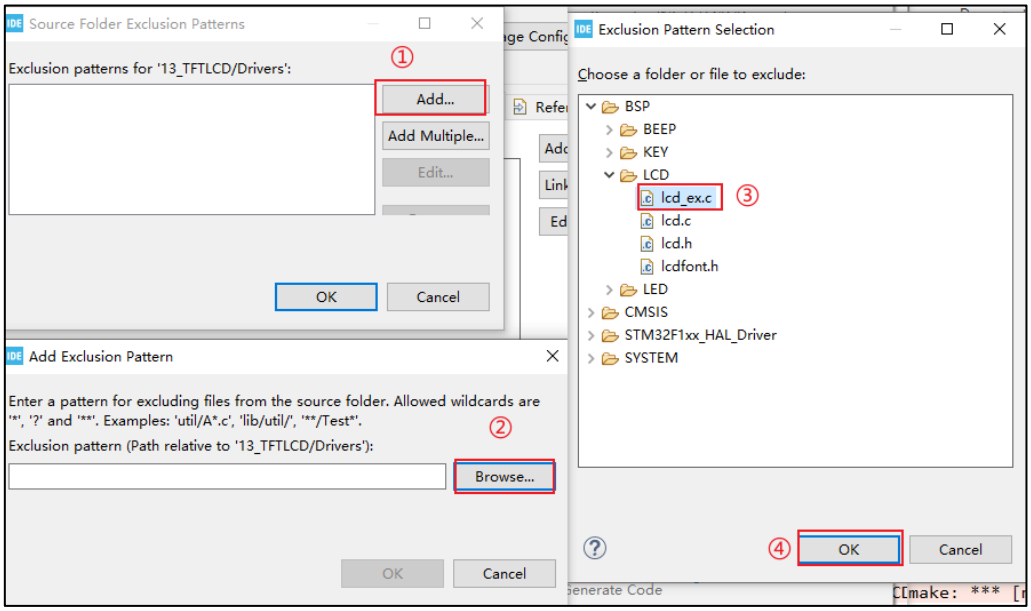


图 28.2.4.4 设置过滤器步骤二

通过以上设置，最后在点击 Apply and Close 选项，即可保存设置，保存后可以看到 13_TFTLCD 工程得 BSP/LCD 文件夹下的 lcd_ex.c 有个小斜杠，代表过滤器设置成功，该文件不会被编译，如下图显示：

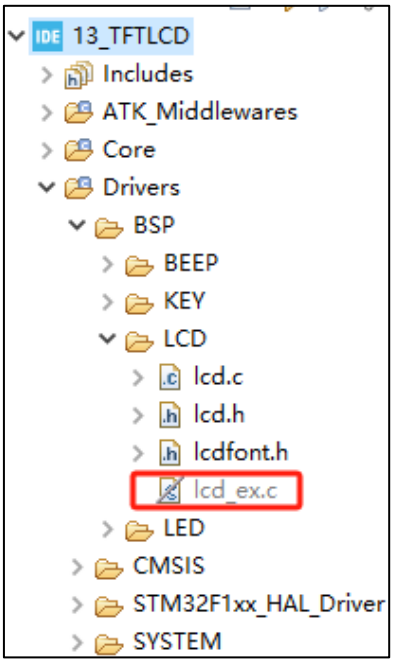


图 28.2.4.5 过滤器设置

通过以上设置后，再去编译即可成功！

28.3 下载验证

编译并下载代码成功后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。同时可以看到 TFTLCD 模块的显示背景色不停切换，如图 28.3.1 所示：

图 28.3.1 TFTLCD 显示效果图

第二十九章 USMART 调试组件实验

本章，我们将向大家介绍一个十分重要的辅助调试工具：USMART 调试组件。该组件由正点原子开发提供，功能类似 linux 的 shell（RTT 的 finsh 也属于此类）。USMART 最主要的功能就是通过串口调用单片机里面的函数，并执行，对我们调试代码是很有帮助的。

本章分为如下几个小节：

29.1 硬件设计
29.2 程序设计
29.3 下载验证

29.1 硬件设计

1. 例程功能

本实验通过 usmart 调用单片机里面的函数，实现对 LCD 显示和 LED 以及延时的控制。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
LED1 - PE5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）定时器 4
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

本章实验使用的 USMART 为软件库，因此没有对应的连接原理图。

29.2 程序设计

29.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\14_USMART。

29.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 14_USMART，以进行下一步的实验操作。如下图所示：

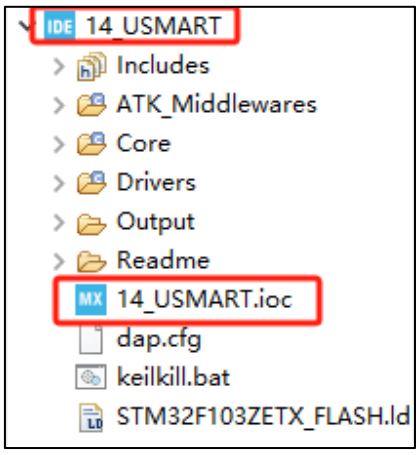


图 29.2.2.1 USMART 工程

接着双击 14_USMART.ioc 进入 STM32CubeMX 插件配置界面后，打开左边的 Timers→TIM4，配置 TIM4 参数，如下图所示：

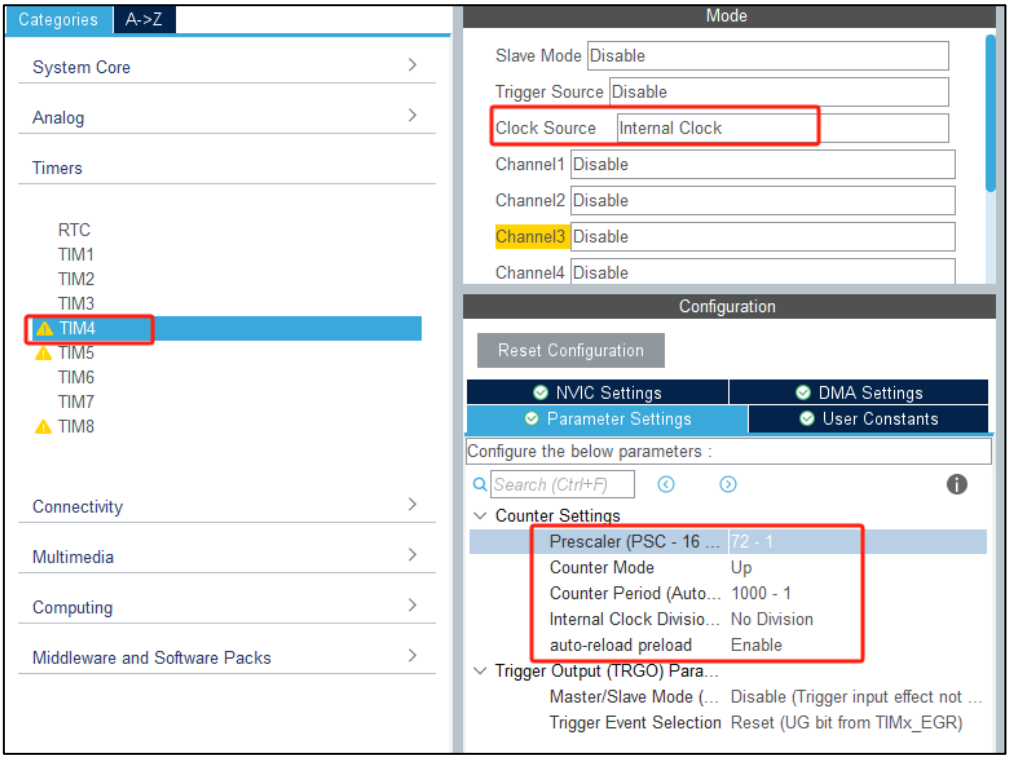


图 29.2.2.2 TIM4 配置

如上图，按照步骤配置通用定时器。Clock Source 我们选择 Internal Clock，表示使用内部时钟，定时器 4 的时钟频率是 72MHz，计算出计数器 CK_CNT 的时钟频率是

$$f_{CK_CNT} = \frac{f_{CK_PSC}}{PSC[15:0]+1}\frac{72MHz}{72} = 1MHz$$

那么计数器计数 5000 次就会溢出产生中断，所以每次溢出时间是：

$$1000 * \frac{1}{1MHz} = 1ms$$

其它选项就保持默认状态，不需要配置。

我们是使用 TIM4 的定时中断来定时扫描 SCAN 函数的，为了保证扫描函数的实时性，我们设置成 1ms（建议 1ms~100ms），扫描一次，大家可根据实际效果去调整该时间。

29.2.3 配置 NVIC

定时器要每 1ms 溢出产生中断，所以我们要使能定时器全局中断，并配置中断优先级。如下图，勾选 TIM4 定时器全局中断，配置中断优先级分组为 2，抢占优先级为 2 和子优先级为 1：

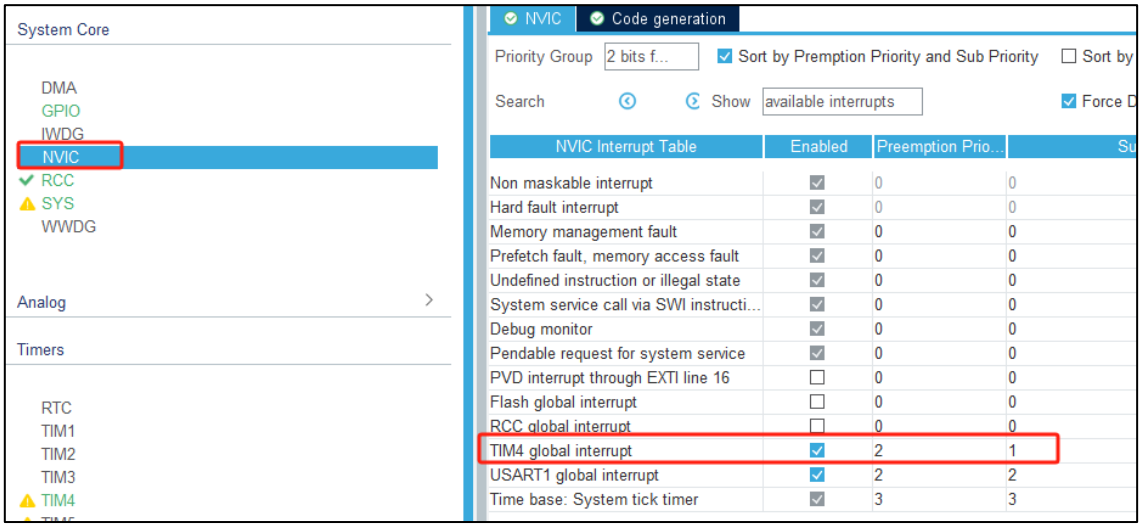


图 29.2.3.1 NVIC 配置

29.2.4 生成工程并添加文件

配置好之后，按下键盘的“Ctrl+S”组合键保存 14_USMART.ioc 文件，系统开始生成初始化代码。要使用 USMART，我们先得进行代码移植。我们在 ATK_Middlewares 文件夹中，移植 USMART 文件夹（第一次使用 USMART 可直接用我们源码里边修改好的），在 USMART 文件夹中包含以下几个文件，如下图所示：

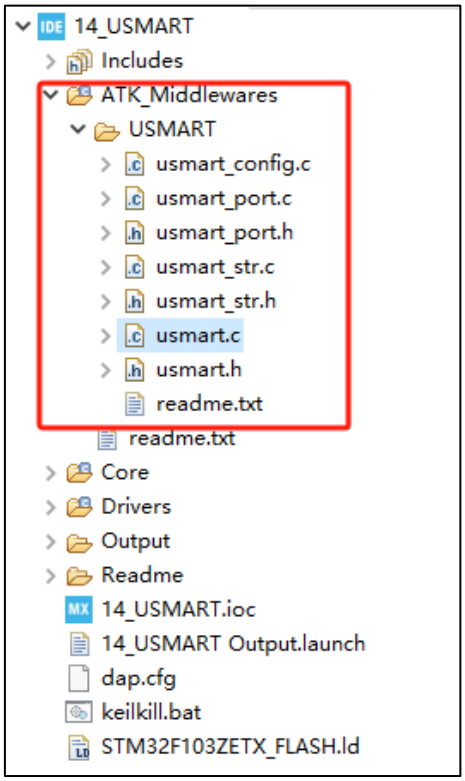


图 29.2.4.1 移植 USMART 文件

USMART 组件的移植也非常简单，只需要实现 usmart_port.c 文件中的 5 个函数即可完成移植。

第一个函数为 usmart_get_input_string()，该函数用于 USMART 获取串口输入的数据流，该函数的实现，如下所示：

```
/**
 * @brief       获取输入数据流(字符串)
 * @note        USMART 通过解析该函数返回的字符串以获取函数名及参数等信息
 * @param       无
 * @retval
 * @arg         0,  没有接收到数据
 * @arg         其他,数据流首地址(不能是 0)
 */
char *usmart_get_input_string(void)
{
    uint8_t len;
    char *pbuf = 0;

    if (g_usart_rx_sta & 0x8000)          /* 串口接收完成? */
    {
        len = g_usart_rx_sta & 0x3fff;    /* 得到此次接收到的数据长度 */
        g_usart_rx_buf[len] = '\0';       /* 在末尾加入结束符. */
        pbuf = (char*)g_usart_rx_buf;
        g_usart_rx_sta = 0;               /* 开启下一次接收 */
    }

    return pbuf;
}
```

从上面的代码中可以看出，该函数就是从 SYSTEM 文件夹 USART 驱动中获取 USART1 输入的数据。

剩余的四个函数，在宏 USMART_ENTIMX_SCAN 开启后才需要定义，该宏用于使能 runtime 统计等功能。

第二个函数为 usmart_timx_reset_time()，该函数用于复位 runtime，该函数的实现，如下所示：

```
/**
 * @brief       复位 runtime
 * @note        需要根据所移植到的 MCU 的定时器参数进行修改
 * @param       无
 * @retval      无
 */
void usmart_timx_reset_time(void)
{
    __HAL_TIM_CLEAR_FLAG(&htim4, TIM_FLAG_UPDATE); /* 清除中断标志位 */
    __HAL_TIM_SET_AUTORELOAD(&htim4, 0XFFFF);      /* 将重装载值设置到最大 */
    __HAL_TIM_SET_COUNTER(&htim4, 0);              /* 清空定时器的 CNT */
    usmart_dev.runtime = 0;
}
```

该函数复位了 runtime 功能和用于 runtime 功能的相关 TMR。

第三个函数为 usmart_timx_get_time()，用于 runtime 功能获取时间，该函数的实现，如下所示：

```
/**
 * @brief       获得 runtime 时间
 * @note        需要根据所移植到的 MCU 的定时器参数进行修改
 * @param       无
 * @retval      执行时间,单位:0.1ms,最大延时时间为定时器 CNT 值的 2 倍*0.1ms
 */
uint32_t usmart_timx_get_time(void)
{
    /* 在运行期间,产生了定时器溢出 */
    if (__HAL_TIM_GET_FLAG(&htim4, TIM_FLAG_UPDATE) == SET)
```

```
    {
        usmart_dev.runtime += 0XFFFF;
    }
    usmart_dev.runtime += __HAL_TIM_GET_COUNTER(&htim4);
    return usmart_dev.runtime;                                  /* 返回计数值 */
}
```

应为该函数能够处理一次定时器的溢出情况，因此能获取到的时间上限为定时器计数最大值的两倍。

第四个函数为 usmart_timx_init()，用于初始化用于 runtime 功能的定时器，该函数的实现，如下所示：

```
/**
 * @brief       定时器初始化函数
 * @param       arr:自动重装载值
 *              psc:定时器分频系数
 * @retval      无
 */
void usmart_timx_init(uint16_t arr, uint16_t psc)
{
    TMR_BaseConfig_T tmr_init_struct;

    USMART_TIMX_CLK_ENABLE();

    tmr_init_struct.countMode = TMR_COUNTER_MODE_UP;     /* 向上计数器 */
    tmr_init_struct.clockDivision = TMR_CLOCK_DIV_1;
    tmr_init_struct.period = arr;                        /* 自动装载值 */
    tmr_init_struct.division = psc;                      /* 分频系数 */
    TMR_ConfigTimeBase(USMART_TIMX, &tmr_init_struct);
    NVIC_EnableIRQRequest(USMART_TIMX_IRQn, 3, 0);

    TMR_EnableInterrupt(USMART_TIMX, TMR_INT_UPDATE);
    TMR_Enable(USMART_TIMX);
}
```

可以看到，该函数初始化了一个用于 runtime 功能的定时器，同时使能了该定时器的更新中断。

第五个函数就是用于runtime功能的定时器的中断服务函数，该函数的实现，如下所示：

```
/**
 * @brief       USMART 定时器中断服务函数
 * @param       无
 * @retval      无
 */
void TIM4_IRQHandler(void)
{
    /* 溢出中断 */
    if(__HAL_TIM_GET_IT_SOURCE(&htim4,TIM_IT_UPDATE)==SET)
    {

        usmart_dev.scan();                              /* 执行 usmart 扫描 */
        __HAL_TIM_SET_COUNTER(&htim4, 0);;              /* 清空定时器的 CNT */
        __HAL_TIM_SET_AUTORELOAD(&htim4, 100);          /* 恢复原来的设置 */
    }
    __HAL_TIM_CLEAR_IT(&htim4, TIM_IT_UPDATE);          /* 清除中断标志位 */
}
```

这里要注意，由于 TIM4 的中断服务函数，在使用 CubeIDE 生成代码时已在 stm32f1xx it.c 文件中被定义了，所以会导致重复定义，需要将 stm32f1xx it.c 文件中的定义屏蔽掉，如下图所示：

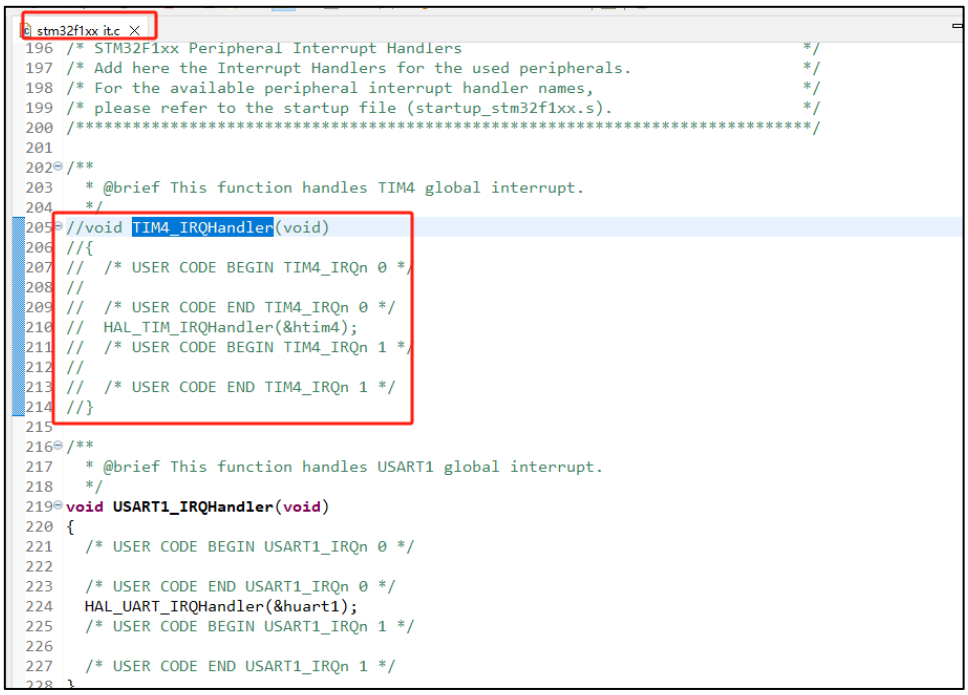


图 29.2.4.2 屏蔽 TIM4 中断服务函数

以上就是移植 USMART 组件时需要实现的五个函数，至此 USMART 组件的移植也就基本完成了，接下来便可在 usart_config.c 文件中的 usmart_nametab 数组中添加需要调试的函数。

1，修改 main.c 代码

```
/* LED 状态设置函数 */
void led_set(uint8_t sta)
{
    LED1(sta);
}

/* 函数参数调用测试函数 */
void test_fun(void(*ledset)(uint8_t), uint8_t sta)
{
    ledset(sta);
}
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();
  /* USER CODE BEGIN 2 */
```

```
  lcd_init();                             /* 初始化 LCD */
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "USMART TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  /* USER CODE END 2 */

  while (1)
  {
      LED0_TOGGLE();                      /* LED0(RED) 闪烁 */
      delay_ms(500);
  }
}
```

led_set 函数，用于设置 LED1 的状态，而第二个函数 test_fun 则是测试 USMART 对函数参数的支持的，test_fun 的第一个参数是函数，在 USMART 里面也是可以被调用的。

main 函数功能如下：经过一系列初始化，显示使用信息后，就是在无限循环中 LED0 翻转延时，并等待串口数据。

29.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。同时，屏幕上显示了一些字符（就是主函数里面要显示的字符）。

我们打开串口调试助手 XCOM，选择正确的串口号→多条发送→勾选发送新行（即发送回车键）选项，然后发送 list 指令，即可打印所有 usmart 可调用函数。如下图所示：

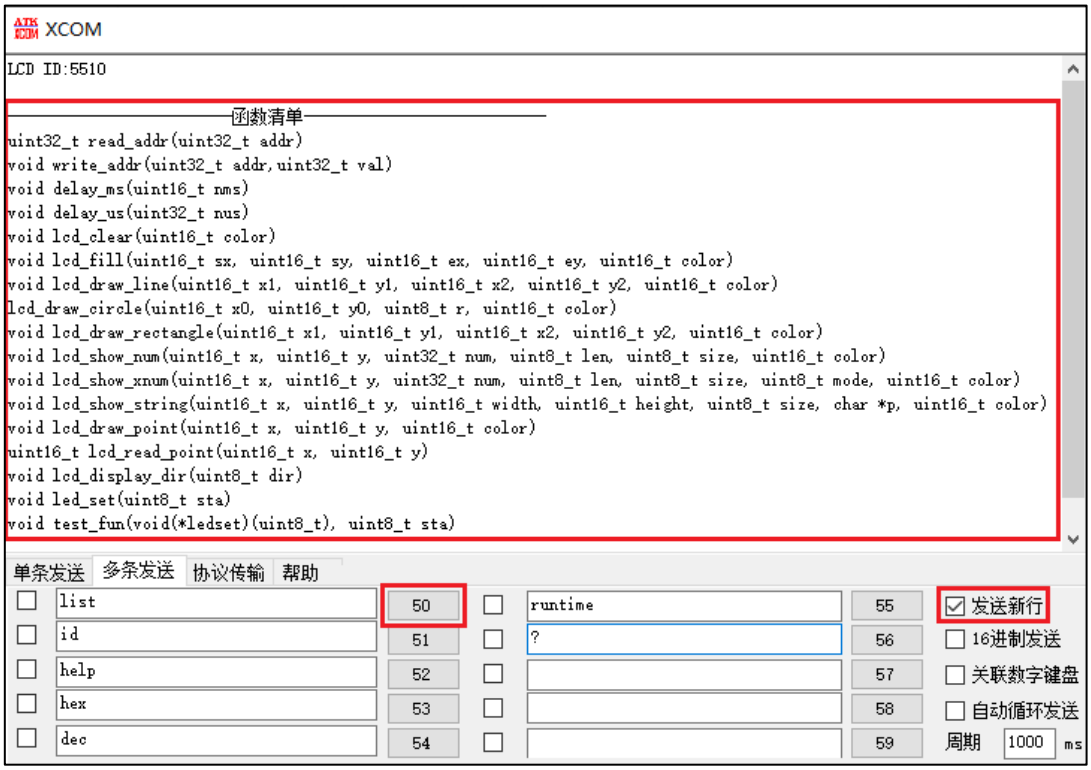


图 29.3.1 驱动串口调试助手

上图中 list、id、help、hex、dec、？和 runtime 都属于 usmart 自带的系统命令，点击后方的数字按钮，即可发送对应的指令。下面我们简单介绍下这几个命令：

list，该命令用于打印所有 usmart 可调用函数。发送该命令后，串口将受到所有能被 usmart 调用得到函数，如图 26.4.1 所示。

id，该指令用于获取各个函数的入口地址。比如前面写的 test_fun 函数，就有一个函数参数，我们需要先通过 id 指令，获取 led_set 函数的 id（即入口地址），然后将这个 id 作为函数参数，传递给 test_fun。

help（或者‘?’也可以），发送该指令后，串口将打印 usmart 使用的帮助信息。

hex 和 dec，这两个指令可以带参数，也可以不带参数。当不带参数的时候，hex 和 dec 分别用于设置串口显示数据格式为 16 进制/10 进制。当带参数的时候，hex 和 dec 就执行进制转换，比如输入：hex 1234，串口将打印：HEX:0X4D2，也就是将 1234 转换为 16 进制打印出来。又比如输入：dec 0X1234，串口将打印：DEC:4660，就是将 0X1234 转换为 10 进制打印出来。

runtime 指令，用于函数执行时间统计功能的开启和关闭，发送：runtime 1，可以开启函数执行时间统计功能；发送：runtime 0，可以关闭函数执行时间统计功能。函数执行时间统计功能，默认是关闭的。

大家可以亲自体验下这几个系统指令，不过要注意，所有的指令都是大小写敏感的，不要写错哦。

第三十章 RTC 实时时钟实验

本章介绍 STM32F103 实时时钟（RTC）的使用，实时时钟能为系统提供一个准确的时间，即时系统复位或主电源断电，RTC 依然能够运行，因此 RTC 也经常用于各种低功耗场景。通过本章的学习，读者将学习到 RTC 的使用。

本章分为如下几个小节：

30.1 硬件设计
30.2 程序设计
30.3 下载验证

30.1 硬件设计

1. 例程功能

本实验通过 LCD 显示 RTC 时间，并可以通过 usmart 设置 RTC 时间，从而调节时间，或设置 RTC 闹钟，还可以写入或者读取 RTC 后备区域 SRAM。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 – PE5
LED1 – PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）RTC(实时时钟)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

RTC 属于 STM32F103 内部资源，通过软件设置好就可以了。不过 RTC 不能断电，否则数据就丢失了，我们如果想让时间在断电后还可以继续走，那么必须确保开发板的电池有电。

30.2 程序设计

30.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\15_RTC。

30.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 14_USMART 工程，并将工程文件名和.ioc 文件名均修改为 15_RTC，以进行下一步的实验操作。如下图所示：

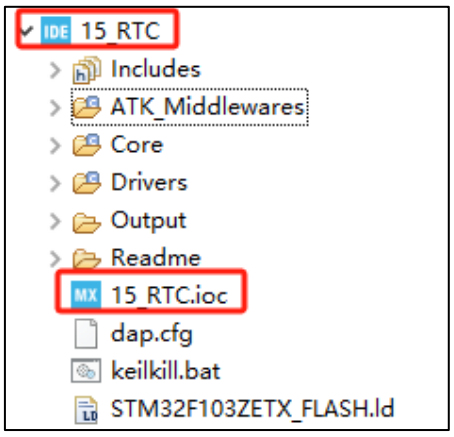


图 30.2.2.1 RTC 工程

接着双击 15_RTC.ioc 进入 STM32CubeMX 插件配置界面后，打开左边的 System Core →RCC，配置使用外部低速晶振，如下图所示：

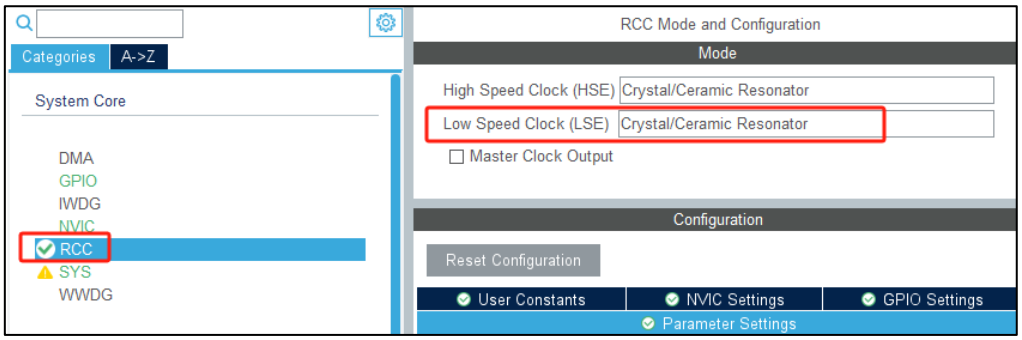


图 30.2.2.2 RCC 配置

配置 RTC 的时钟源为 LSE 提供：

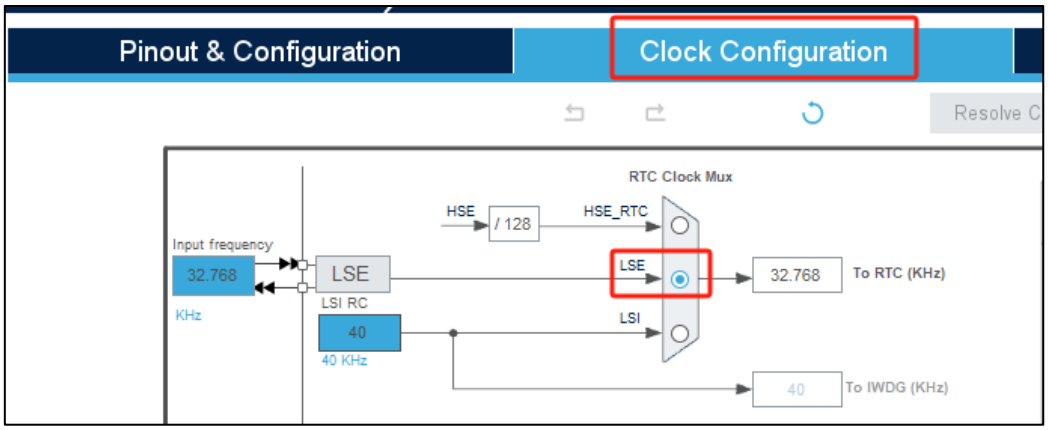


图 30.2.2.3 RTC 时钟源选择

接着打开左边的 Timers →RTC，配置 RTC，激活时钟源、激活日历，如下图所示

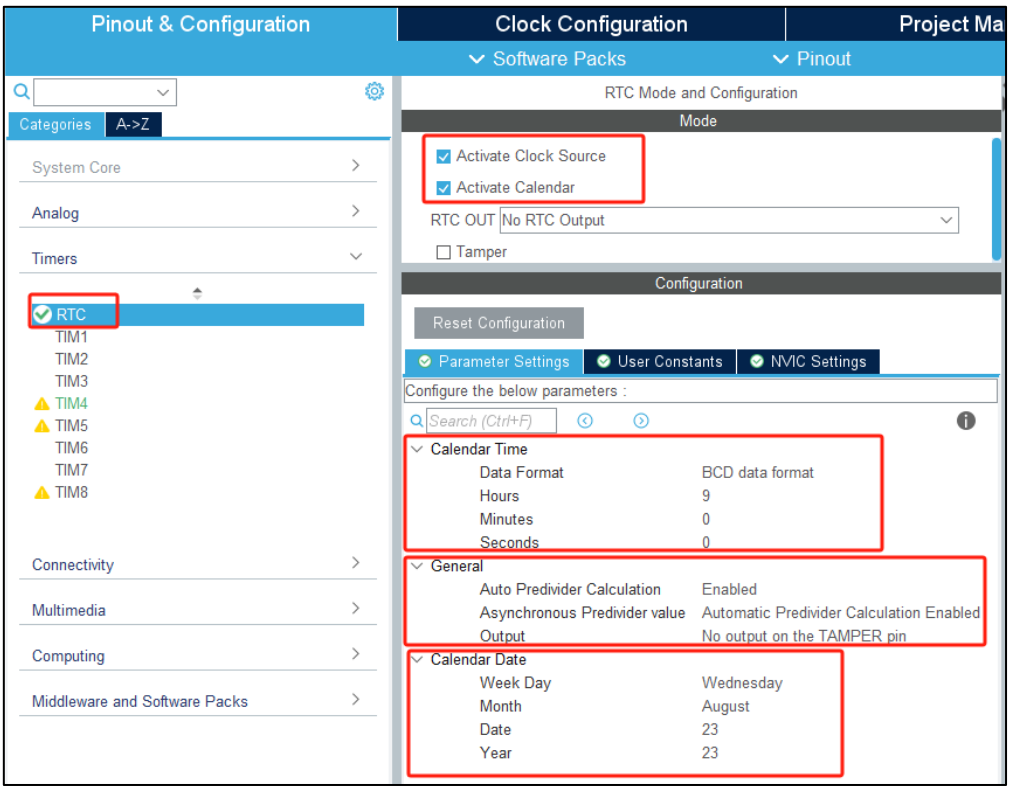


图 30.2.2.4 RTC 参数配置

本示例中将 RTC 配置为 2023 年 8 月 23 日 09:00:00，然而由于我们将使用自定义函数来实现日期和时间的设置，所以该设置将不会生效。具体原因如下：

注意：实验中没有使用 HAL 库自带的设置 RTC 时间的函数 HAL_RTC_SetTime、设置 RTC 日期的函数 HAL_RTC_SetDate、获取当前 RTC 日期的函数 HAL_RTC_GetTime。原因在于版本的 HAL 库函数不满足我们同时更新年月日时分秒的要求，且在实测中发现写时间会覆盖日期，写日期亦然，所以我们直接通过操作寄存器的方式去编写功能更加全面的函数。

30.2.3 配置 NVIC

使能 RTC 的全局中断，并配置中断优先级。如下图，勾选 RTC 全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 2：

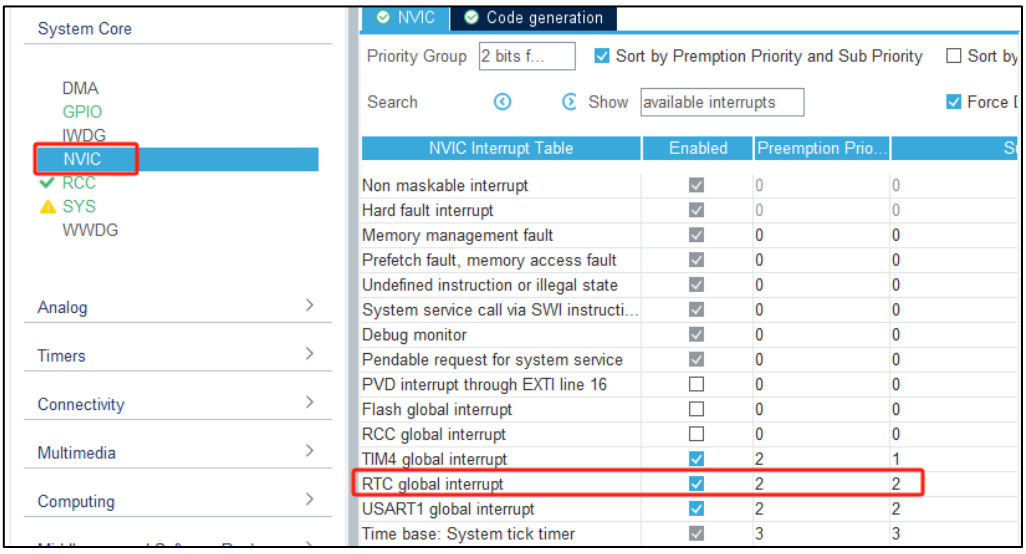


图 30.2.3.1 NVIC 配置

30.2.4 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 rtc.c 文件。rtc.c 主要是完成 RTC 的初始化。

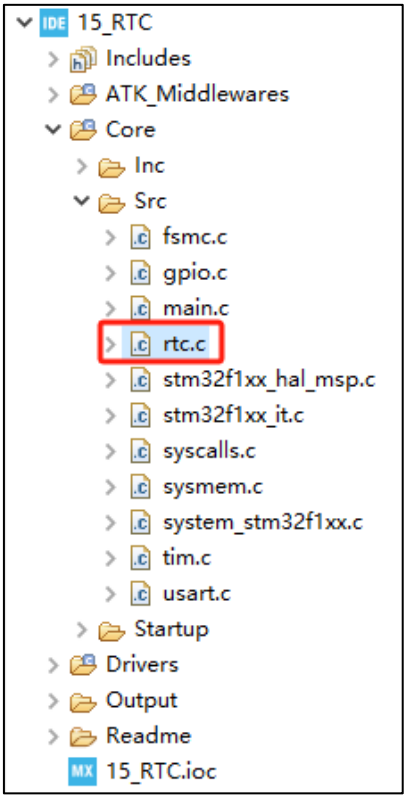


图 30.2.4.1 生成代码

30.2.5 添加用户驱动代码

我们本实验主要修改三个文件，分别是：rtc.c、main.c、stm32f1xx_it.c，首先我们将对 rtc.c 文件进行修改。

（1）修改 rtc.c 文件

由于篇幅过长，我们这里只列出比较重要的几个函数进行讲解，若想要详细的代码文件，可打开我们的源码查看。

首先我们先来看下设置时间函数：

```
/**
 * @brief       设置时间，包括年月日时分秒
 *  @note        以 1970 年 1 月 1 日为基准，往后累加时间
 *                合法年份范围为：1970 ~ 2105 年
 *                HAL 默认为年份起点为 2000 年
 * @param       syear : 年份
 * @param       smon  : 月份
 * @param       sday  : 日期
 * @param       hour  : 小时
 * @param       min   : 分钟
 * @param       sec   : 秒钟
 * @retval      0, 成功; 1, 失败;
 */
uint8_t rtc_set_time(uint16_t syear, uint8_t smon, uint8_t sday, uint8_t hour,
uint8_t min, uint8_t sec)
```

```
{
    uint32_t seccount = 0;

    /* 将年月日时分秒转换成总秒钟数 */
    seccount = rtc_date2sec(syear, smon, sday, hour, min, sec);

    __HAL_RCC_PWR_CLK_ENABLE();     /* 使能电源时钟 */
    __HAL_RCC_BKP_CLK_ENABLE();     /* 使能备份域时钟 */
    HAL_PWR_EnableBkUpAccess();     /* 取消备份域写保护 */
    /* 上面三步是必须的! */

    RTC->CRL |= 1 << 4;             /* 进入配置模式 */

    RTC->CNTL = seccount & 0xffff;
    RTC->CNTH = seccount >> 16;

    RTC->CRL &= ~(1 << 4);          /* 退出配置模式 */

    /* 等待 RTC 寄存器操作完成, 即等待 RTOFF == 1 */
    while (!__HAL_RTC_ALARM_GET_FLAG(&g_rtc_handle, RTC_FLAG_RTOFF));

    return 0;
}
```

该函数用于设置时间，把我们输入的时间，转换为以 1970 年 1 月 1 日 0 时 0 分 0 秒做起始时间的秒钟信号，后续的计算都以这个时间为基准，由于 STM32 的秒钟计数器可以保存 136 年的秒钟数据，这样我们就可以计时到 2106 年。

接着，我们介绍 rtc_set_alarma 函数，该函数用于设置闹钟时间，同 rtc_set_time 函数几乎一模一样，主要区别：就是将 RTC→CNTL 和 RTC→CNTH 换成了 RTC→ALRL 和 RTC→ALRH，用于设置闹钟时间。RTC 其实是有闹钟中断的，我们这里并没有用到，本实验用到了秒中断，所以在秒中断里顺带处理闹钟中断的事情。具体代码请参考本例程源码。

特别提醒：假如只是使用 HAL 库的__HAL_RTC_ALARM_ENABLE_IT 函数来使能闹钟中断，但是没有设置闹钟相关的 NVIC 和 EXTI，实际上不会产生闹钟中断，只会产生闹钟标志（RTC->CRL 的 ALRL 置位）。可以通过读取闹钟标志来判断是否发生闹钟事件。

接着，我们介绍一下 rtc_get_time 函数，其定义如下：

```
/**
 * @brief       得到当前的时间
 *   @note      该函数不直接返回时间, 时间数据保存在 calendar 结构体里面
 * @param           无
 * @retval      无
 */
void rtc_get_time(void)
{
    static uint16_t daycnt = 0;
    uint32_t seccount = 0;
    uint32_t temp = 0;
    uint16_t temp1 = 0;
    /* 平年的月份日期表 */
    const uint8_t month_table[12] = {31,28,31,30,31,30,31,31,30,31,30,31};

    seccount = RTC->CNTH;               /* 得到计数器中的值(秒钟数) */
    seccount <<= 16;
    seccount += RTC->CNTL;

    temp = seccount / 86400;            /* 得到天数(秒钟数对应的) */

    if (daycnt != temp)                 /* 超过一天了 */
    {
        daycnt = temp;
        temp1 = 1970;                   /* 从 1970 年开始 */
```

```
        while (temp >= 365)
        {
            if (rtc_is_leap_year(temp1)) /* 是闰年 */
            {
                if (temp >= 366)
                {
                    temp -= 366;        /* 闰年的秒钟数 */
                }
                else
                {
                    break;
                }
            }
            else
            {
                temp -= 365;            /* 平年 */
            }

            temp1++;
        }

        calendar.year = temp1;          /* 得到年份 */
        temp1 = 0;

        while (temp >= 28)              /* 超过了一个月 */
        {
          /* 当年是不是闰年/2 月份 */
            if (rtc_is_leap_year(calendar.year) && temp1 == 1)
            {
                if (temp >= 29)
                {
                    temp -= 29;         /* 闰年的秒钟数 */
                }
                else
                {
                    break;
                }
            }
            else
            {
                if (temp >= month_table[temp1])
                {
                    temp -= month_table[temp1]; /* 平年 */
                }
                else
                {
                    break;
                }
            }

            temp1++;
        }

        calendar.month = temp1 + 1;         /* 得到月份 */
        calendar.date = temp + 1;           /* 得到日期 */
    }

    temp = seccount % 86400;                /* 得到秒钟数 */
    calendar.hour = temp / 3600;            /* 小时 */
    calendar.min = (temp % 3600) / 60;      /* 分钟 */
   calendar.sec = (temp % 3600) % 60;       /* 秒钟 */
   /* 获取星期 */
    calendar.week = rtc_get_week(calendar.year, calendar.month,
calendar.date);
```

```
}
```

该函数其实就是将存储在秒钟寄存器 RTC->CNTL 和 RTC->CNTH 中的秒钟数据转换为真正的时间和日期。该代码还用到了一个 calendar 的结构体，calendar 是我们在 rtc.h 里面将要定义的一个时间结构体，用来存放时钟的年月日时分秒等信息。因为 STM32 的 RTC 只有秒钟计数器，而年月日，时分秒则需要我们自己软件计算。我们把计算好的值保存在 calendar 里面，方便其他函数调用。

接着，我们介绍一下使用多次数最多的函数 rtc_date2sec，该函数代码如下：

```
/**
 * @brief           将年月日时分秒转换成秒钟数
 *   @note      以 1970 年 1 月 1 日为基准，1970 年 1 月 1 日，0 时 0 分 0 秒，表示第 0 秒钟
 *                  最大表示到 2105 年，因为 uint32_t 最大表示 136 年的秒钟数(不包括闰
年)!
 *                  本代码参考只 linux mktime 函数，原理说明见此贴:
 *                  http://www.openedv.com/thread-63389-1-1.html
 * @param           syear : 年份
 * @param       smon  : 月份
 * @param           sday  : 日期
 * @param           hour  : 小时
 * @param           min   : 分钟
 * @param           sec   : 秒钟
 * @retval      转换后的秒钟数
 */
static long rtc_date2sec(uint16_t syear, uint8_t smon, uint8_t sday,
                         uint8_t hour, uint8_t min, uint8_t sec)
{
    uint32_t Y, M, D, X, T;
    signed char monx = smon;    /* 将月份转换成带符号的值，方便后面运算 */

    if (0 >= (monx -= 2)) /* 1..12 -> 11,12,1..10 */
    {
        monx += 12; /* Puts Feb last since it has leap day */
        syear -= 1;
    }

    /* 公元元年 1 到现在的闰年数 */
    Y = (syear - 1) * 365 + syear / 4 - syear / 100 + syear / 400;
    M = 367 * monx / 12 - 30 + 59;
    D = sday - 1;
    X = Y + M + D - 719162;                         /* 减去公元元年到 1970 年的天数 */
    T = ((X * 24 + hour) * 60 + min) * 60 + sec;    /* 总秒钟数 */
    return T;
}
```

该函数参考了 linux 的 mktime 函数，用于将年月日时分秒转化成秒钟数，进而被其他函数使用，例如 rtc_set_time 和 rtc_set_alarm，那两个函数的形参是需要使用 rtc_date2sec 函数获取秒钟数，进而操作寄存器的方法把总秒数写入特定的寄存器完成相对应的功能。

最后再来看下我们的 RTC 初始化函数：

```
void MX_RTC_Init(void)
{

  /* USER CODE BEGIN RTC_Init 0 */
    /* 检查是不是第一次配置时钟 */
    uint16_t bkpflag = 0;
  /* USER CODE END RTC_Init 0 */

  RTC_TimeTypeDef sTime = {0};
  RTC_DateTypeDef DateToUpdate = {0};

  hrtc.Instance = RTC;
  hrtc.Init.AsynchPrediv = RTC_AUTO_1_SECOND;
  hrtc.Init.OutPut = RTC_OUTPUTSOURCE_NONE;
  if (HAL_RTC_Init(&hrtc) != HAL_OK)
```

```
  {
    Error_Handler();
  }

  /* USER CODE BEGIN Check_RTC_BKUP */
  bkpflag = rtc_read_bkr(0);     /* 读取 BKP0 的值 */
  if (bkpflag != 0X5050)         /* 之前未初始化过，重新配置 */
  {
  /* USER CODE END Check_RTC_BKUP */
  sTime.Hours = 0x9;
  sTime.Minutes = 0x0;
  sTime.Seconds = 0x0;

  if (HAL_RTC_SetTime(&hrtc, &sTime, RTC_FORMAT_BCD) != HAL_OK)
  {
    Error_Handler();
  }
  DateToUpdate.WeekDay = RTC_WEEKDAY_WEDNESDAY;
  DateToUpdate.Month = RTC_MONTH_AUGUST;
  DateToUpdate.Date = 0x23;
  DateToUpdate.Year = 0x23;

  if (HAL_RTC_SetDate(&hrtc, &DateToUpdate, RTC_FORMAT_BCD) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN RTC_Init 2 */
      rtc_set_time(2023, 9, 10, 12, 0, 0);          /* 设置时间 */
      rtc_write_bkr(0, 0X5050);     /* 写入 0x5050，以便不会重复初始化 RTC 的时间 */
  }
  __HAL_RTC_ALARM_ENABLE_IT(&hrtc, RTC_IT_SEC);     /* 允许秒中断 */
  rtc_get_time();                                   /* 更新时间 */
  /* USER CODE END RTC_Init 2 */

}
```

该函数用来初始化 RTC 配置以及日期和时钟，但是只在第一次的时候设置时间，以后如果重新上电/复位都不会再进行时间设置了（前提是备份电池有电）。这里设置时间和日期，是通过 rtc_set_time 函数来实现的，我们之所以不是用 HAL 库自带的设置时间和日期的函数，在前面已经提到了这个原因，就不用多说了。那么 rtc_set_time 是我们直接操作寄存器，同时，它也可以为我们的 USMART 所调用，十分方便我们调试时候使用。

（2）修改 stm32f1xx_it.c 文件

由于我们重新在 STM32CubeIDE 中生成了代码，所以在 stm32f1xx_it.c 文件中的 TIM4 中断服务函数又被定义了，这样会和 USMART 部分的 TIM4 中断服务函数发生重定义，所以第一步屏蔽 stm32f1xx_it.c 中断文件中的 TIM4 中断服务函数，如下：

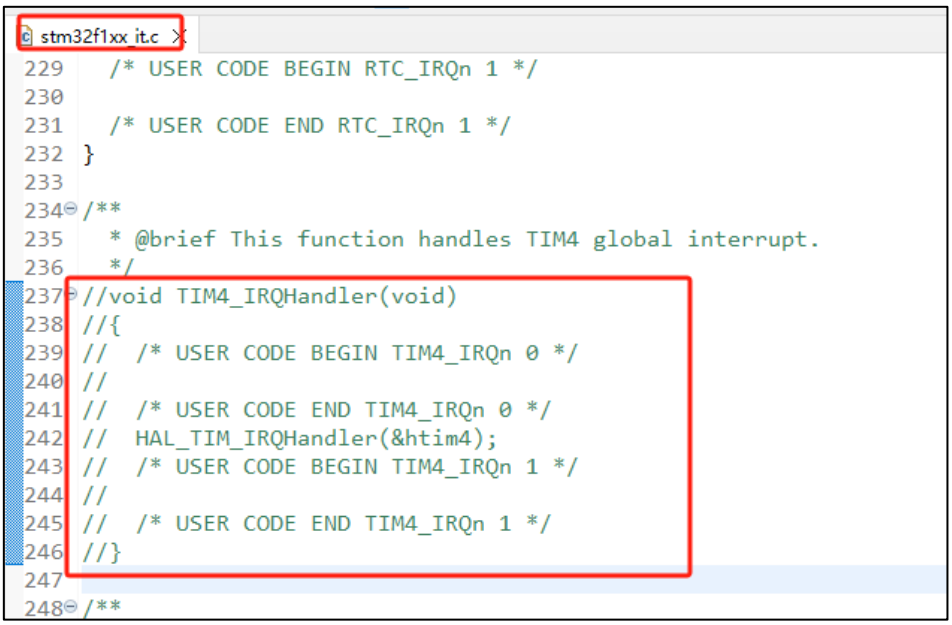


图 30.2.5.1 屏蔽 TIM4 中断服务函数

接着我们在 stm32f1xx_it.c 文件中，添加 RTC 秒中断以及闹钟中断处理逻辑的相关内容，如下：

```
void RTC_IRQHandler(void)
{
  /* USER CODE BEGIN RTC_IRQn 0 */
    if (__HAL_RTC_ALARM_GET_FLAG(&hrtc, RTC_FLAG_SEC) != RESET) /* 秒中断 */

    {
        rtc_get_time();                                         /* 更新时间 */
        __HAL_RTC_ALARM_CLEAR_FLAG(&hrtc, RTC_FLAG_SEC);        /* 清除秒中断 */
        //printf("sec:%d\r\n", calendar.sec);                   /* 打印秒钟 */
    }

    /* 顺带处理闹钟标志 */
    if (__HAL_RTC_ALARM_GET_FLAG(&hrtc, RTC_FLAG_ALRAF) != RESET)
    {
        __HAL_RTC_ALARM_CLEAR_FLAG(&hrtc, RTC_FLAG_ALRAF);/* 清除闹钟标志 */
        printf("Alarm Time:%d-%d-%d %d:%d:%d\n", calendar.year,
calendar.month, calendar.date, calendar.hour, calendar.min, calendar.sec);
    }

    __HAL_RTC_ALARM_CLEAR_FLAG(&hrtc, RTC_FLAG_OW);             /* 清除溢出中断标志 */
    /* 等待 RTC 寄存器操作完成，即等待 RTOFF == 1 */
    while (!__HAL_RTC_ALARM_GET_FLAG(&hrtc, RTC_FLAG_RTOFF));
  /* USER CODE END RTC_IRQn 0 */
  HAL_RTCEx_RTCIRQHandler(&hrtc);
  /* USER CODE BEGIN RTC_IRQn 1 */

  /* USER CODE END RTC_IRQn 1 */
}
```

RTC_IRQHandle 中断服务函数用于 RTC 秒中断的，由于在 MX_RTC_Init 中已经配置好了时钟周期为 1 秒，所以每一秒都会跳进 RTC 中断服务函数中。在函数中，判断秒中断是否触发，由于每一次都是秒中断触发，所以可以先更新时间，然后把 printf 的注释去掉看一下效果，是不是每一秒打印一下。接着判断闹钟标志是否置位，这个闹钟标志跟我们的 rtc_set_alarm 函数有关，假设时间到了闹钟设置的时间，就会跳进该秒中断中顺带处理闹钟标志，执行函数体的指令。执行完上述的任务之后，需要在最后清除溢出中断标志。

（3）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t tbuf[40];
    uint8_t t = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* RTC 初始化 */
  MX_RTC_Init();

  /* USER CODE BEGIN 2 */
  rtc_set_alarm(2023, 9, 10, 12, 3, 0); /* 设置一次闹钟 */
  lcd_init();                           /* 初始化 LCD */
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "RTC TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  /* USER CODE END 2 */
  while (1)
  {
      t++;

      if ((t % 10) == 0)                /* 每 100ms 更新一次显示数据 */
      {
          rtc_get_time();
          sprintf((char *)tbuf, "Time:%02d:%02d:%02d", calendar.hour,
          calendar.min, calendar.sec);
          lcd_show_string(30, 120, 210, 16, 16, (char *)tbuf, RED);
          sprintf((char *)tbuf, "Date:%04d-%02d-%02d", calendar.year,
          calendar.month, calendar.date);
          lcd_show_string(30, 140, 210, 16, 16, (char *)tbuf, RED);
          sprintf((char *)tbuf, "Week:%s", weekdays[calendar.week]);
          lcd_show_string(30, 160, 210, 16, 16, (char *)tbuf, RED);
      }

      if ((t % 20) == 0)
      {
          LED0_TOGGLE();                /* 每 200ms,翻转一次 LED0 */
      }

      delay_ms(10);
  }
}
```

我们在无限循环中每 100ms 读取 RTC 的时间和日期（一次），并显示在 LCD 上面。每 200ms，翻转一次 LED0。

（4）修改 usmart_config.c 文件

为方便 RTC 相关函数的调用验证，在 usmart_config.c 里面，修改了 usmart_nametab 如下：

```
/* 函数名列表初始化(用户自己添加)
 * 用户直接在这里输入要执行的函数名及其查找串
 */
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",

    (void *)rtc_read_bkr, "uint16_t rtc_read_bkr(uint32_t bkrx)",
    (void *)rtc_write_bkr, "void rtc_write_bkr(uint32_t bkrx, uint16_t data)",
    (void *)rtc_get_week, "uint8_t rtc_get_week(uint16_t year, uint8_t month,
                                                uint8_t day)",
    (void *)rtc_set_time, "uint8_t rtc_set_time(uint16_t syear, uint8_t smon,
                        uint8_t sday, uint8_t hour, uint8_t min, uint8_t sec)",
    (void *)rtc_set_alarm, "uint8_t rtc_set_alarm(uint16_t syear, uint8_t
                        smon, uint8_t sday, uint8_t hour, uint8_t min, uint8_t
                        sec)",
};
```

将 RTC 的一些相关函数加入了 usmart，这样通过串口就可以直接设置 RTC 时间、闹钟。至此，RTC 的软件设计就完成了，接下来就让我们来检验一下，程序是否正确。

30.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上实时地显示着 RTC 的时间，并且可以看到 LED0 在闪烁，此时可以通过串口调试助手调用 USMART 调试组件的 rtc_set_alarma()函数来设置 RTC 的闹钟，当通过 LCD 观察到 RTC 的时间达到设置的闹钟时间后，可以看到串口调试助手上打印了“ALARM A!\r\n”的字符串提示。

第三十一章 PVD 电压监控实验

本章介绍 STM32F103 电源电压监测器（PVD）的使用，PVD 可以设置一个电压阈值，当监测到电源电压低于该阈值后，可以触发中断，以完成一些紧急处理。通过本章的学习，读者将学习到 PVD 的使用。

本章分为如下几个小节：

31.1 硬件设计
31.2 程序设计
31.3 下载验证

31.1 硬件设计

1. 例程功能

开发板供电正常的话，LCD 屏会显示"PVD Voltage OK!"。当供电电压过低，则会通过 PVD 中断服务函数将 LED1 点亮；当供电电压正常后，会在 PVD 中断服务函数将 LED1 熄灭。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
LED1 - PE5
2）PVD(可编程电压监测器)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

PVD 属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 LED0 和 LCD 来指示进入 PVD 中断的情况。

31.2 程序设计

31.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 16_1_LowPower_PVD。

31.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 16_1_LowPower_PVD，以进行下一步的实验操作。如下图所示：

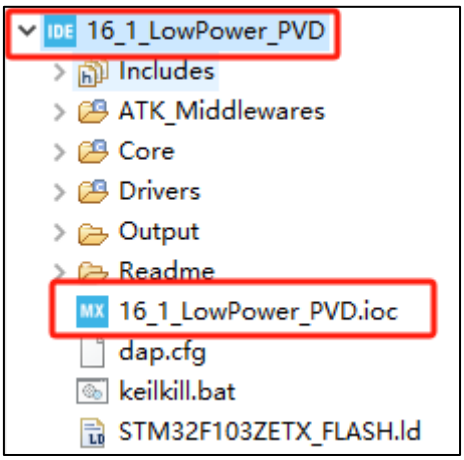


图 31.2.2.1 PVD 工程

31.2.3 添加文件

本实验在 13_TFTLCD 的基础上进行改动的，不需要另外去配置.ioc 文件。可直接添加驱动文件。我们在 Drivers/BSP 文件夹中，新建 PWR 文件夹，然后再 PWR 文件夹下新建 2 个文件，分别是 pwr.c 和 pwr.h 文件，如下图所示：

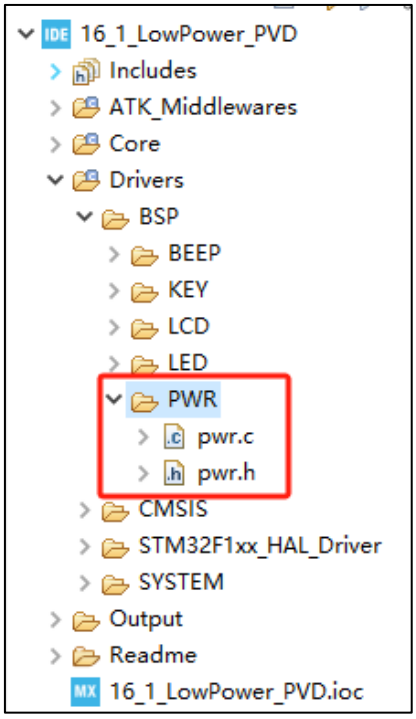


图 31.2.3.1 新建 PWR 文件夹

其中 pwr.h 头文件只有函数声明，下面直接开始介绍 pwr.c 的程序。

（1）修改 pwr.c 文件

```
#include "pwr.h"
#include "../LED/led.h"
#include "../LCD/lcd.h"


/**
 * @brief       初始化 PVD 电压监视器
 * @param       pls: 电压等级(PWR_PVD_detection_level)
 *   @arg       PWR_PVDLEVEL_0,2.2V;
```

```
 *    @arg        PWR_PVDLEVEL_1,2.3V;
 *    @arg        PWR_PVDLEVEL_2,2.4V;
 *    @arg        PWR_PVDLEVEL_3,2.5V;
 *    @arg        PWR_PVDLEVEL_4,2.6V;
 *    @arg        PWR_PVDLEVEL_5,2.7V;
 *    @arg        PWR_PVDLEVEL_6,2.8V;
 *    @arg        PWR_PVDLEVEL_7,2.9V;
 * @retval       无
 */
void pwr_pvd_init(uint32_t pls)
{
    PWR_PVDTypeDef pwr_pvd = {0};

    __HAL_RCC_PWR_CLK_ENABLE();                          /* 使能 PWR 时钟 */

    pwr_pvd.PVDLevel = pls;                              /* 检测电压级别 */
    /* 使用中断线的上升沿和下降沿双边缘触发 */
    pwr_pvd.Mode = PWR_PVD_MODE_IT_RISING_FALLING;
    HAL_PWR_ConfigPVD(&pwr_pvd);

    HAL_NVIC_SetPriority(PVD_IRQn, 3 ,3);
    HAL_NVIC_EnableIRQ(PVD_IRQn);
    HAL_PWR_EnablePVD();                                 /* 使能 PVD 检测 */
}

/**
 * @brief       PVD 中断服务函数
 * @param       无
 * @retval      无
 */
void PVD_IRQHandler(void)
{
    HAL_PWR_PVD_IRQHandler();
}

/**
 * @brief       PVD 中断服务回调函数
 * @param       无
 * @retval      无
 */
void HAL_PWR_PVDCallback(void)
{
    if (__HAL_PWR_GET_FLAG(PWR_FLAG_PVDO)) /* 电压比 PLS 所选电压还低 */
    {
        /* LCD 显示电压低 */
        lcd_show_string(30, 130, 200, 16, 16, "PVD Low Voltage!", RED);
        LED1(0);                                /* 点亮绿灯，表明电压低了 */
    }
    else
    {
        /* LCD 显示电压正常 */
        lcd_show_string(30, 130, 200, 16, 16, "PVD Voltage OK! ", BLUE);
        LED1(1);                                /* 灭掉绿灯 */
    }
}
```

首先是 PVD 初始化函数，设置 PVD 中断线选择的是上升沿和下降沿双边沿触发；然后就是 PVD 中断服务函数及其回调函数，HAL_PWR_PVDCallback 回调函数中首先是判断 V_{DD} 电压是否比 PLS 所选电压还低，是的话，就在 LCD 显示 PVD Low Voltage!并且点亮 LED1，否则，在 LCD 显示 PVD Voltage OK!并且关闭 LED1。

（2）修改 main.c 文件

```
int main(void)
```

```
{
  /* USER CODE BEGIN 1 */
  uint8_t t = 0;
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                             /* 初始化 LCD */
  pwr_pvd_init(PWR_PVDLEVEL_7);           /* PVD 2.9V 检测 */

  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "PVD TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  /* 默认 LCD 显示电压正常 */
  lcd_show_string(30, 110, 200, 16, 16, "PVD Voltage OK! ", BLUE);
  /* USER CODE END 2 */

  while (1)
  {
      if ((t % 20) == 0)
      {
          LED0_TOGGLE();                  /* 每 200ms,翻转一次 LED0 */
      }

      delay_ms(10);
      t++;
  }
}
```

这里我们选择 PVD 的检测电压阀值为 2.9V，其他的代码很好理解，最后下载验证一下。

31.3 下载验证

在完成编译和烧录操作后，若系统的供电正常，便可以在 LCD 上看到“PVD Voltage OK!”的提示，并且 LED1 也处于熄灭状态；若系统的供电低于设置的 PVD 电压阈值 2.9V 时，便可以在 LCD 上看到“PVD Low Voltage!”的提示，并且 LED1 也会亮起（系统供电过低时，可能出现跟多意想不到的结果，因此可能会看不到部分现象）。

第三十二章 睡眠模式实验

本章介绍 STM32F103 低功耗模式中的睡眠模式，通过调用 WFI 命令进入睡眠模式后，ARM Corten-M3 内核将停止以降低功耗，在该睡眠模式下可以被任意中断唤醒。通过本章的学习，读者将学习到低功耗模式中睡眠模式的使用。

本章分为如下几个小节：

32.1 硬件设计
32.2 程序设计
32.3 下载验证

32.1 硬件设计

1. 例程功能

LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，LED1 点亮，提示进入睡眠模式，此时 LED0 不再闪烁，说明已经进入睡眠模式。按下按键 WK_UP 后，LED1 熄灭，提示退出睡眠模式，此时 LED0 继续闪烁，说明已经退出睡眠模式。

2. 硬件资源

1）LED 灯
LED0 – PB5　　LED1 – PE5
2）独立按键
KEY0 – PE4　　WK_UP - PA0
3）电源管理(低功耗模式 - 睡眠模式)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

PWR 属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 KEY0 让 CPU 进入睡眠模式，再通过 WK_UP 触发 EXTI 中断来唤醒 CPU。LED0 指示程序是否执行，LED1 指示 CPU 是否进入睡眠模式。

32.2 程序设计

32.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 16_2_LowPower_Sleep。

32.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 16_1_LowPower_PVD 工程，并将工程文件名和.ioc 文件名均修改为 16_2_LowPower_Sleep，以进行下一步的实验操作。如下图所示：

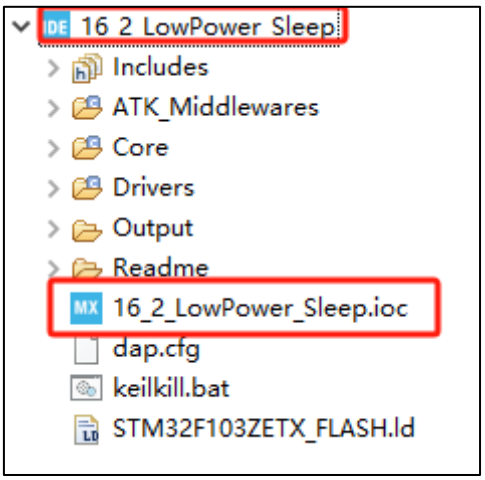


图 32.2.2.1 SLEEP 工程

接着双击 16_2_LowPower_Sleep.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处，配置 PA0，由于我们需要通过 PA0 的外部中断唤醒睡眠模式，所以我们需要将 PA0 复用为 GPIO_EXTI0，如下图所示：

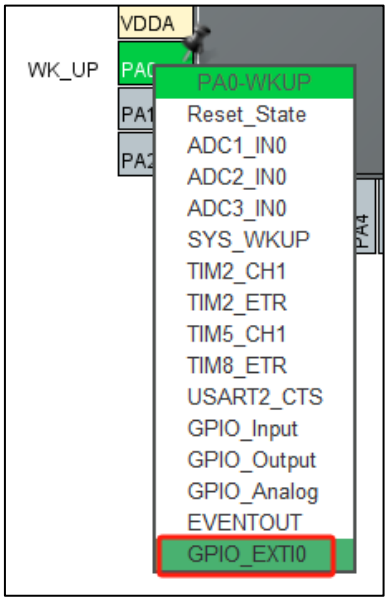


图 32.2.2.2 GPIO 复用

然后将 PA0 参数配置为下拉，上升沿触发，如下图所示：

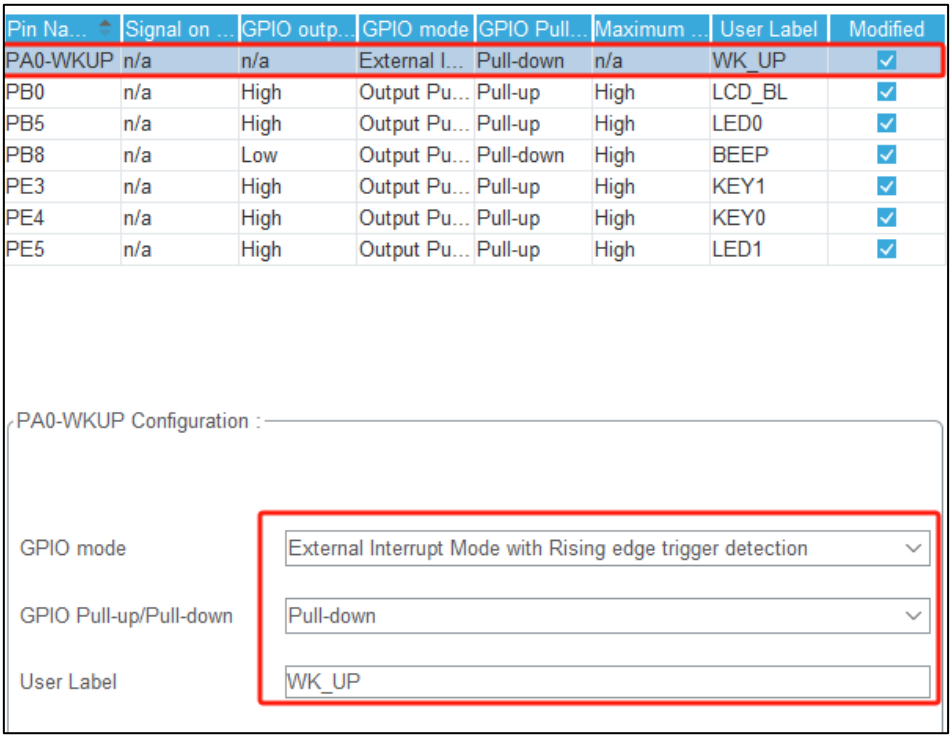


图 32.2.2.3 IO 配置

32.2.3 配置 NVIC

由于本实验是通过外部中断唤醒睡眠模式，所以我们要使能外部中断线 0，并配置中断优先级。如下图，勾选使能外部中断线 0，配置中断优先级分组为 2，抢占优先级和子优先级为 2：

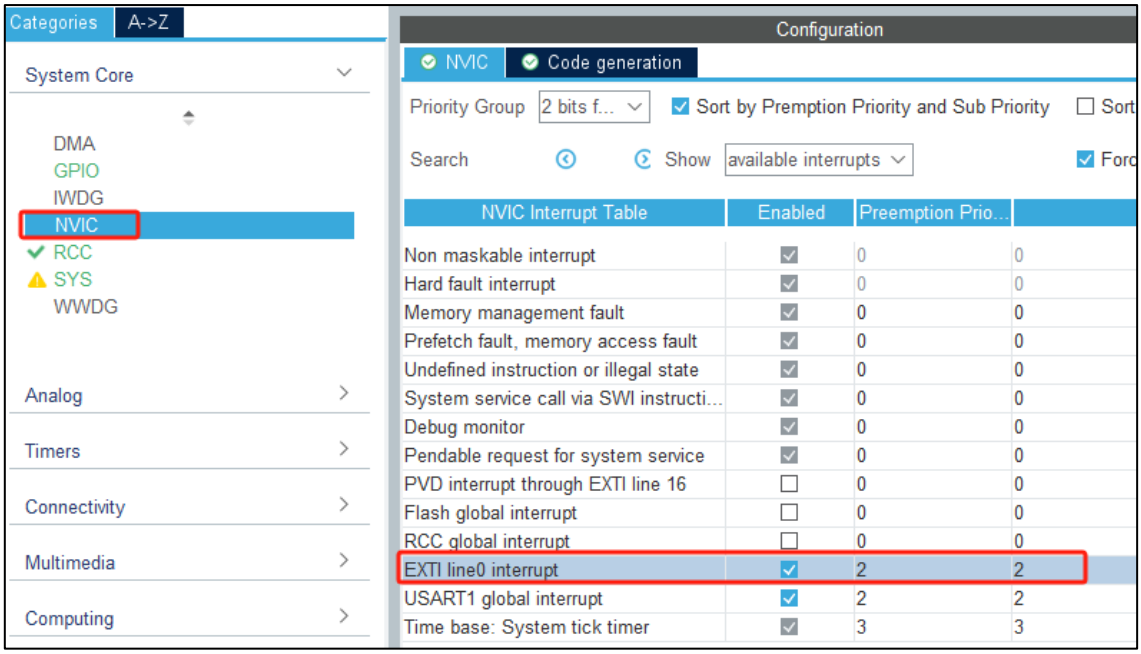


图 32.2.3.1 NVIC 配置

32.2.4 生成并修改代码

按下“Ctrl+S”保存配置，生成工程，由于本实验在 16_1_LowPower_PVD 的基础上进行改

动的，所以我们直接在 Drivers/BSP/PWR 文件夹中的 pwr.c 文件中添加本例所需的驱动代码即可，如下图所示：

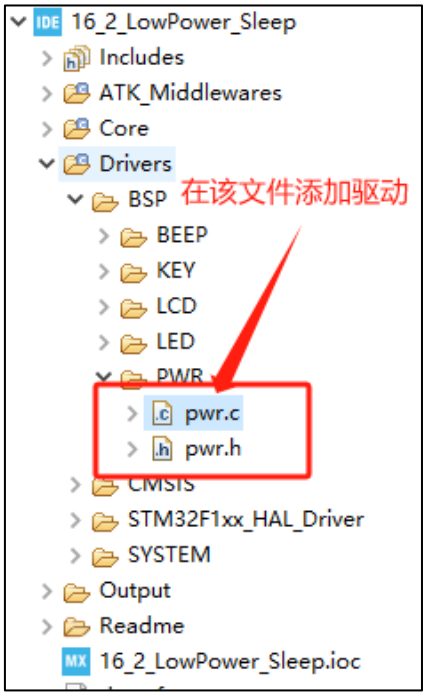


图 32.2.4.1 在 pwr 添加驱动

（1）修改 pwr.c 文件

```
/******************************睡眠模式实验程序************************/
/**
 * @brief       进入 CPU 睡眠模式
 * @param       无
 * @retval      无
 */
void pwr_enter_sleep(void)
{
    /* 暂停滴答时钟，防止通过滴答时钟中断唤醒 */
    HAL_SuspendTick();
    /* 执行 WFI 指令，进入睡眠模式 */
    HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
}

/**
 * @brief       外部中断回调函数
 * @param       GPIO_Pin:中断线引脚
 * @note        此函数会被 PWR_WKUP_INT_IRQHandler()调用
 * @retval      无
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == WK_UP_Pin)
    {
/* HAL_GPIO_EXTI_IRQHandler()函数已经为我们清除了中断标志位，所以进了回调函数可不操作*/
    }
}
```

pwr_enter_sleep 函数内直接调用 HAL_PWR_EnterSLEEPMode 函数使用 WFI 指令进入睡眠模式。

在 WK_UP 按键外部中断服务函数中我们调用 HAL 库的 HAL_GPIO_EXTI_IRQHandler 函数来处理外部中断。该函数会调用__HAL_GPIO_EXTI_CLEAR_IT 函数取消屏蔽对应的外

部中断线位，这里是 EXTI_CPUIMR1 寄存器相应位，还有其他寄存器控制其他外部中断线。我们只是唤醒睡眠模式而已，不需要其他的逻辑程序，所以 HAL_GPIO_EXTI_Callback 回调函数可以什么都不用做，甚至也可以不重新定义这个回调函数（屏蔽该回调函数也可以）。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    uint8_t key = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                             /* 初始化 LCD */

  lcd_show_string(30, 50, 200, 16, 16,  "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16,  "SLEEP TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16,  "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter SLEEP MODE", RED);
  lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit SLEEP MODE", RED);
  /* USER CODE END 2 */
  while (1)
  {
      key = key_scan(0);

      if (key == KEY0_PRES)
      {
          LED1(0);                        /* 点亮绿灯,提示进入睡眠模式 */

          pwr_enter_sleep();              /* 进入睡眠模式 */
          HAL_ResumeTick();               /* 恢复滴答时钟 */

          LED1(1);                        /* 关闭绿灯,提示退出睡眠模式 */
      }

      if ((t % 20) == 0)
      {
          LED0_TOGGLE();                  /* 每 200ms,翻转一次 LED0 */
      }

      delay_ms(10);
      t++;
  }
}
```

该部分程序，功能就是按下 KEY0 后，点亮 LED1，进入睡眠模式。然后一直等待外部

中断唤醒，当按下按键 WK_UP，就触发外部中断，睡眠模式就被唤醒，然后继续执行后面的程序，关闭 LED1 等。

32.3 下载验证

在完成编译和烧录操作后，可以看到 LED0 闪烁提示系统程序正在运行，此时可以按下 KEY0 按键，可以看到 LED1 亮起，但 LED0 不再闪烁，这是因为系统已经进入睡眠模式了，此时再按下 KEY_UP 按键，即可从睡眠模式下唤醒，可以看到 LED1 熄灭，LED0 继续闪烁。

第三十三章 停止模式实验

本章介绍 STM32F103 低功耗模式中的停止模式，进入停止模式后，所有的时钟都将被停止以降低功耗，在停止模式下可以被任意中断唤醒。通过本章的学习，读者将学习到低功耗模式中停止模式的使用。

本章分为如下几个小节：

33.1 硬件设计
33.2 程序设计
33.3 下载验证

33.1 硬件设计

1. 例程功能

LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，LED1 点亮，提示进入停止模式，此时 LED0 不再闪烁，说明已经进入停止模式。按下按键 WK_UP 后，LED1 熄灭，提示退出停止模式，此时 LED0 继续闪烁，说明已经退出停止模式。

2. 硬件资源

1）LED 灯
LED0 – PB5
LED1 – PE5
2）独立按键
KEY0 – PE4
WK_UP - PA0
3）电源管理(低功耗模式 – 停止模式)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

PWR 属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 KEY0 让 CPU 进入停止模式，再通过 WK_UP 触发 EXTI 中断来唤醒 CPU。LED0 指示程序是否执行，LED1 指示 CPU 是否进入停止模式。

33.2 程序设计

33.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 16_3_LowPower_Stop。

33.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 16_2_LowPower_Sleep 工程，并将工程文件名和.ioc 文件名均修改为 16_3_LowPower_Stop，以进行下一步的实验操作。如下图所示：

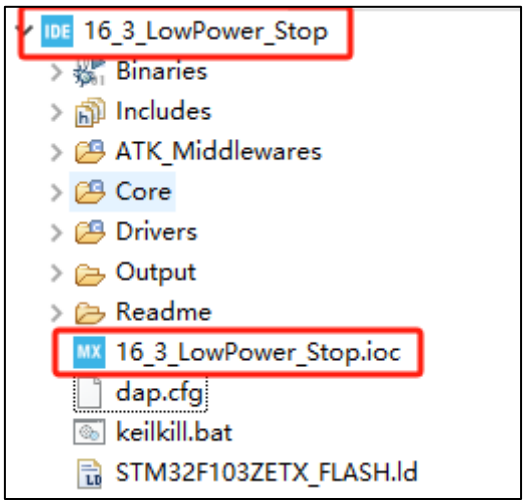


图 33.2.2.1 STOP 工程

本实验是在 16_2_LowPower_Sleep 的基础上复制而来的，所以外部中断配置这些都已经配置好了，我们不需要额外去修改.ioc 文件，直接在 pwr.c 文件中添加停止模式的驱动代码即可！

33.2.3 控制逻辑代码实现

（1）修改 pwr.c 文件

```
/************************停止模式实验程序*************************/

/**
 * @brief       进入停止模式
 * @param       无
 * @retval      无
 */
void pwr_enter_stop(void)
{
    __HAL_RCC_PWR_CLK_ENABLE();/* 使能电源时钟 */
    HAL_SuspendTick();          /* 暂停滴答时钟，防止通过滴答时钟中断唤醒 */
    /* 进入停止模式，设置稳压器为低功耗模式，等待中断唤醒 */
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
}
```

该函数因为涉及电源控制寄存器的操作，所以先调用__HAL_RCC_PWR_CLK_ENABLE 函数去使能 PWR 时钟，然后调用 HAL_PWR_EnterSTOPMode 函数进入停止模式，形参 1 即 PWR_LOWPOWERREGU LATOR_ON 设置稳压器为低功耗模式，形参 2 则是选择 WFI 指令。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    uint8_t key = 0;
  /* USER CODE END 1 */

   HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);
```

```
    /* GPIO 初始化 */
    MX_GPIO_Init();

    /* FSMC 初始化 */
    MX_FSMC_Init();

    /* 串口初始化 */
    MX_USART1_UART_Init();

    /* USER CODE BEGIN 2 */
    lcd_init();                             /* 初始化 LCD */

    lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30,  70, 200, 16, 16, "STOP TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STOP MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STOP MODE", RED);
    /* USER CODE END 2 */
    while (1)
    {
        key = key_scan(0);

        if (key == KEY0_PRES)
        {
            LED1(0);                                /* 点亮绿灯,提示进入停止模式 */
            pwr_enter_stop();                       /* 进入停止模式 */

            /* 从停止模式唤醒, 需要重新设置系统时钟, 72Mhz */
            SystemClock_Config();
            HAL_ResumeTick();                       /* 恢复滴答时钟 */
            LED1(1);                                /* 关闭绿灯,提示退出停止模式 */
        }

        if ((t % 20) == 0)
        {
            LED0_TOGGLE();                          /* 每 200ms,翻转一次 LED0 */
        }

        delay_ms(10);
        t++;
    }
}
```

该部分程序，功能就是按下 KEY0 后，点亮 LED1，进入停止模式。然后一直等待外部中断唤醒，当按下按键 WK_UP，就触发外部中断，停止模式就被唤醒，然后继续执行后面的程序，重新设置系统时钟 72MHZ 和延时初始化，关闭 LED1 等。

33.3 下载验证

在完成编译和烧录操作后，可以看到LED0闪烁提示系统正在运行，此时可以按下KEY0按按键，可以看到 LED1 亮起，但 LED0 不再闪烁，这是因为系统已经进入停止模式了，此时再按下 KEY_UP 按键，即可从停止模式下唤醒，可以看到 LED1 熄灭，LED0 继续闪烁。

第三十四章 待机模式实验

本章介绍 STM32F103 低功耗模式中的待机模式，进入待机模式后，MCU 内部的电压调压器将断开 1.8V 电源域的电源，这意味着内核和外设都将停止工作，并且内核寄存器和内存中的数据都将丢失，但这也是功耗最低的模式，待机模式下可被 WKUP 引脚的上升沿唤醒。通过本章的学习，读者将学习到低功耗模式下待机模式的使用。

本章分为如下几个小节：

34.1 硬件设计
34.2 程序设计
34.3 下载验证

34.1 硬件设计

1. 例程功能

LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，进入待机模式，待机模式下大部分引脚处于高阻态，所以说这时候 LED0 会熄灭，TFTLCD 也会熄灭。按下 WK_UP 按键后，退出待机模式（相当于复位操作），程序重新执行，LED0 继续闪烁，TFTLCD 屏点亮。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
KEY0 - PE4
WK_UP - PA0
3）电源管理(低功耗模式 - 待机模式)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

PWR 属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。我们通过 KEY0 让 CPU 进入待机模式，再通过 WK_UP 上升沿唤醒 CPU。LED0 指示程序是否执行。

34.2 程序设计

34.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 16_4_LowPower_Standby。

34.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 16_3_LowPower_Stop 工程，并将工程文件名和.ioc 文件名均修改为 16_4_LowPower_Standby，以进行下一步的实验操作。如下图所示：

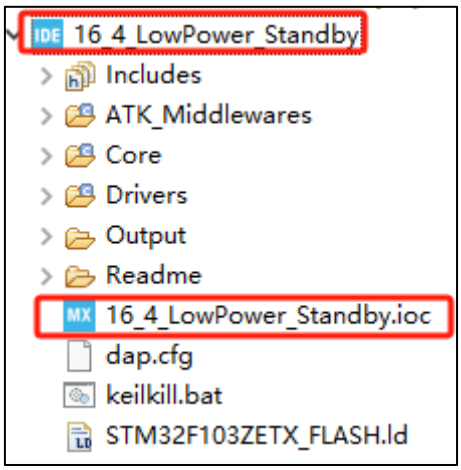


图 34.2.2.1 Standby 工程

本实验是在 16_3_LowPower_Stop 的基础上复制而来的，所以外部中断配置这些都已经配置好了，我们不需要额外去修改.ioc 文件，直接在 pwr.c 文件中添加待机模式的驱动代码即可！

34.2.3 添加用户驱动代码

（1）修改 pwr.c 文件

```
/*******************************待机模式*********************************/
/**
 * @brief       进入待机模式
 * @param       无
 * @retval      无
 */
void pwr_enter_standby(void)
{
    __HAL_RCC_PWR_CLK_ENABLE();                 /* 使能电源时钟 */

    HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1); /* 使能 KEY_UP 引脚的唤醒功能 */
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);          /* 需要清此标记，否则将保持唤醒状态*/
    HAL_PWR_EnterSTANDBYMode();                 /* 进入待机模式 */
}
```

该函数首先是调用__HAL_RCC_PWR_CLK_ENABLE 来使能 PWR 时钟，然后调用函数 HAL_PWR_EnableWakeUpPin 用来设置 WK_UP 引脚作为唤醒源。在进入待机模式前，还得调用__HAL_PWR_CLEAR_FLAG 函数清除一下唤醒标志，要不然会保持唤醒状态。最后调用函数 HAL_PWR_EnterSTANDBYMode 进入待机模式。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    uint8_t key = 0;
  /* USER CODE END 1 */

   HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);
```

```
    /* GPIO 初始化 */
    MX_GPIO_Init();

    /* FSMC 初始化 */
    MX_FSMC_Init();

    /* 串口初始化 */
    MX_USART1_UART_Init();

    /* USER CODE BEGIN 2 */
    lcd_init();                             /* 初始化 LCD */

    lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30,  70, 200, 16, 16, "STANDBY TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STANDBY MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STANDBY MODE", RED);
    /* USER CODE END 2 */

    while (1)
    {
        key = key_scan(0);

        if (key == KEY0_PRES)
        {
            pwr_enter_standby();            /* 进入待机模式 */
            /* 从待机模式唤醒相当于系统重启(复位), 因此不会执行到这里 */
        }

        if ((t % 20) == 0)
        {
            LED0_TOGGLE();                  /* 每 200ms,翻转一次 LED0 */
        }

        delay_ms(10);
        t++;
    }
}
```

该部分程序，经过一系列初始化后，判断到 KEY0 按下就调用 pwr_enter_standby 函数进入待机模式，然后等待按下 WK_UP 按键产生 WKUP 上升沿唤醒 CPU。注意待机模式唤醒后，系统会进行复位。

34.3 下载验证

在完成编译和烧录操作后，可以看到 LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，TFTLCD 屏熄灭，此时 LED0 不再闪烁，说明已经进入待机模式。按下按键 WK_UP 后，TFTLCD 屏点亮，LED0 闪烁，说明系统从待机模式中唤醒相当于复位。

第三十五章 DMA 实验

本章介绍 STM32F103 直接存储访问（DMA）的使用，DMA 能够在无 CPU 干预的情况下，实现外设与存储器或存储器与存储器之间数据的高速传输，从而节省 CPU 资源来做其他操作。通过本章的学习，读者将学习到 DMA 的使用。

本章分为如下几个小节：

35.1 硬件设计
35.2 程序设计
35.3 下载验证

35.1 硬件设计

1. 例程功能

每按下按键KEY0，串口1就会以DMA方式发送数据，同时在LCD上面显示传送进度。打开串口调试助手，可以收到 DMA 发送的内容。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
2）独立按键
KEY0 – PE4
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

DMA 属于 STM32F103 内部资源，通过软件设置好就可以了。

35.2 程序设计

35.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 17_DMA。

35.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 17_DMA，以进行下一步的实验操作。如下图所示：

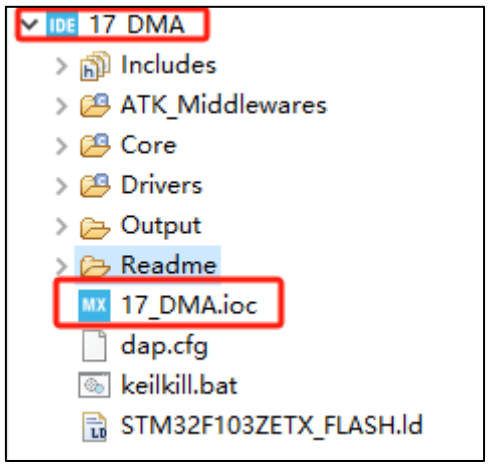


图 35.2.2.1 DMA 工程

接着双击 17_DMA.ioc 进入 STM32CubeMX 插件配置界面，串口一的配置在工程中已经配置好了，所以无须另外配，我们只需打开左边的 System Core →DMA，配置 DMA 参数即可，由于 USART1 与 DMA1 相关联，所以在 DMA1 处新增 DMA 请求，如下图所示：

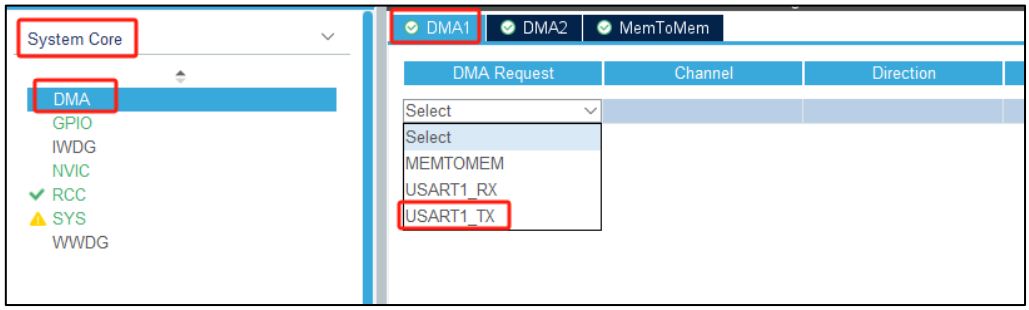


图 35.2.2.2 添加 DMA 请求

DMA1 请求选择 USART1_TX，通道选择 DMA1 Channel4，传输方向选择 Memory To Peripheral，传输速率设置为中速，如下图所示：

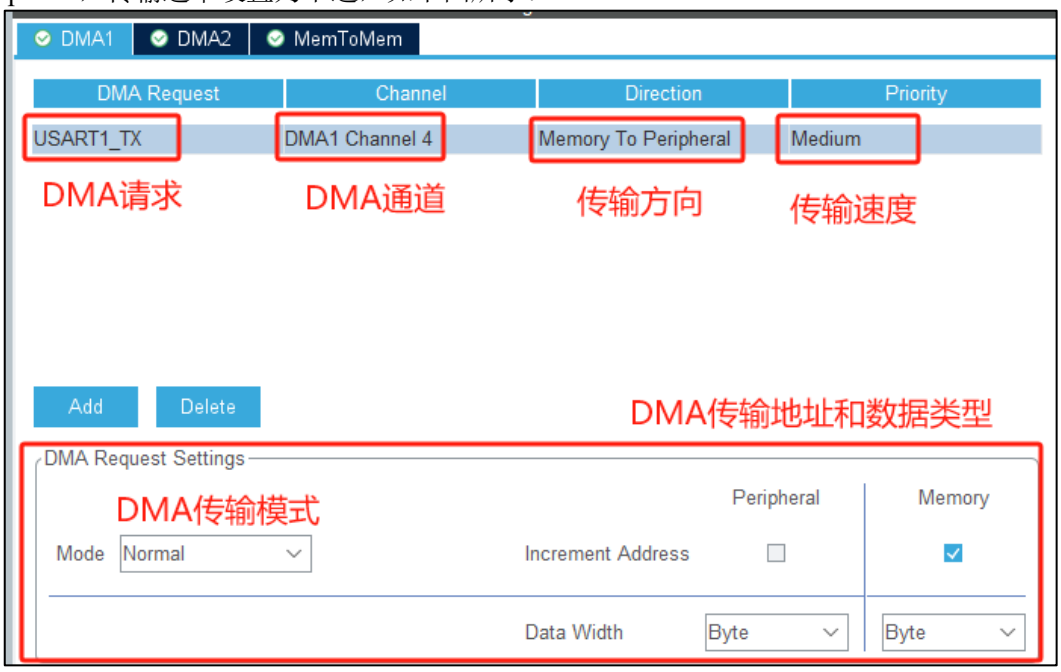


图 35.2.2.3 配置 DMA 参数

以上配置项解释如下：

- **DMA Request** 是 DMA 请求：

即 **DMA 传输的对应外设，这里我们选为 USART1 的发送端；**

- **Channel 是** DMA 通道：
  我们前面分析了 DMA1 有 1~7（共 7 个）通道，不同外设所匹配的通道不一样，可查看《STM32 中文参考手册》第 10.3.7 小节；
- **Dirction 是 DMA 传输方向，有 4 种传输方向：**
  外设到内存 **Peripheral To Memory**
  内存到外设 **Memory To Peripheral**
  内存到内存 **Memory To Memory**
  外设到外设 **Peripheral To Peripheral**
- **Priority 是传输速度：**
  可选最高优先级 **Very Hight**、高优先级 **Hight**、中等优先级 **Medium 和**低优先级 **Low；**
- Mode 是 DMA 的传输模式：

Normal 表示正常模式，当一次 DMA 数据传输完后就停止 DMA 传送，也就是只传输一次就停止了；

Circular 是循环模式，当传输完成后又重新开始继续传输，即不断循环永不停止传输

Src Memory 表示外设地址寄存器，设置传输数据的时候外设地址是不变还是递增。如果设置 为递增，那么下一次传输的时候地址加 Data Width 个字节。

Dst Memory 表示内存地址寄存器，设置传输数据时候内存地址是否递增。如果设置 为递增，那么下一次传输的时候地址加 Data Width 个字节。

由于串口数据发送寄存器只能存储 8bit，每次发送一个字节，所以数据长度选择 Byte。

35.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 dma.c 文件。dma.c 主要是完成 DMA 的初始化。

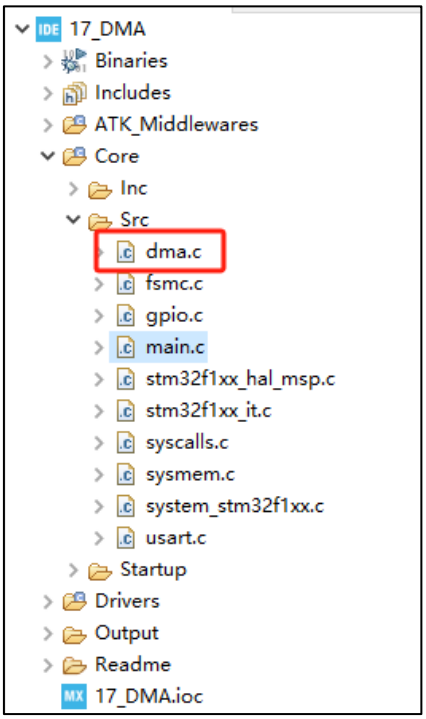


图 35.2.3.1 生成代码

35.2.4 添加用户驱动代码

本实验关于 DMA 的初始化代码在 dma.c 文件中生成了，USART1 的初始化代码在 usart.c 中生成了。下面我们直接手动添加用户代码，实现串口 DMA 发送一串字符。在 main.c 中添

加如下代码，其中标红的字体之间的代码是我们手动添加的部分：

```
/* USER CODE BEGIN PFP */
const uint8_t TEXT_TO_SEND[] = {"正点原子 STM32 DMA 串口实验"};  /* 字符串 */
/* 发送数据长度，等于 sizeof(TEXT_TO_SEND) + 2 的 200 倍. */
#define SEND_BUF_SIZE       (sizeof(TEXT_TO_SEND) + 2) * 200

uint8_t g_sendbuf[SEND_BUF_SIZE];   /* 发送数据缓冲区 */

extern DMA_HandleTypeDef hdma_usart1_tx;
/* USER CODE END PFP */

int main(void)
{
  /* USER CODE BEGIN 1 */
  uint8_t  key = 0;
  uint16_t i, k;
  uint16_t len;
  uint8_t  mask = 0;
  float pro = 0;           /* 进度 */
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* DMA 初始化 */
  MX_DMA_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                          /* 初始化 LCD */
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "DMA TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:Start", RED);

  len = sizeof(TEXT_TO_SEND);
  k = 0;

  for (i = 0; i < SEND_BUF_SIZE; i++) /* 填充 ASCII 字符集数据 */
  {
      if (k >= len)    /* 入换行符 */
      {
          if (mask)
          {
              g_sendbuf[i] = 0x0a;
              k = 0;
          }
          else
          {
              g_sendbuf[i] = 0x0d;
              mask++;
```

```
            }
        }
        else      /* 复制 TEXT_TO_SEND 语句 */
        {
            mask = 0;
            g_sendbuf[i] = TEXT_TO_SEND[k];
            k++;
        }
    }

    i = 0;
    /* USER CODE END 2 */

    /* USER CODE BEGIN WHILE */
    while (1)
    {
        key = key_scan(0);

        if (key == KEY0_PRES)        /* KEY0 按下 */
        {
            printf("\r\nDMA DATA:\r\n");
            lcd_show_string(30, 130, 200, 16, 16, "Start Transimit....", BLUE);
            lcd_show_string(30, 150, 200, 16, 16, "    %", BLUE);  /* 显示百分号 */

            HAL_UART_Transmit_DMA(&huart1, g_sendbuf, SEND_BUF_SIZE);
            /* 等待 DMA 传输完成，此时我们来做另外一些事情，比如点灯
             * 实际应用中，传输数据期间，可以执行另外的任务
             */
            while (1)
            {
                /* 等待 DMA1_Channel4 传输完成 */
                if ( __HAL_DMA_GET_FLAG(&hdma_usart1_tx, DMA_FLAG_TC4))
                {
                    /* 清除 DMA1_Channel4 传输完成标志 */
                    __HAL_DMA_CLEAR_FLAG(&hdma_usart1_tx, DMA_FLAG_TC4);
                    HAL_UART_DMAStop(&huart1);   /* 传输完成以后关闭串口 DMA */
                    break;
                }

                pro = __HAL_DMA_GET_COUNTER(&hdma_usart1_tx);
                len = SEND_BUF_SIZE;             /* 总长度 */
                pro = 1 - (pro / len);           /* 得到百分比 */
                pro *= 100;                      /* 扩大 100 倍 */
                lcd_show_num(30, 150, pro, 3, 16, BLUE);
            }
            lcd_show_num(30, 150, 100, 3, 16, BLUE);    /* 显示 100% */
            lcd_show_string(30, 130, 200, 16, 16, "Transimit Finished!", BLUE);
        }

        i++;
        delay_ms(10);

        if (i == 20)
        {
            LED0_TOGGLE();  /* LED0 闪烁,提示系统正在运行 */
            i = 0;
        }
    }
}
```

main 函数的流程大致是：先初始化发送数据缓冲区 g_sendbuf 的值，然后通过 KEY0 开启串口 DMA 发送，在发送过程中，通过__HAL_DMA_GET_COUNTER(&g_dma_handle)获取当前还剩余的数据量来计算传输百分比，最后在传输结束之后清除相应标志位，提示已经传输完成。

35.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 35.3.1 所示：

图 35.3.1 DMA 实验测试图

我们打开串口调试助手，然后按 KEY0，可以看到串口显示如图 35.3.2 所示的内容：

图 35.3.2 串口收到的数据内容

可以看到串口收到了开发板发送过来的数据，同时可以看到 TFTLCD 上显示了进度等信息，如图 35.3.3 所示：

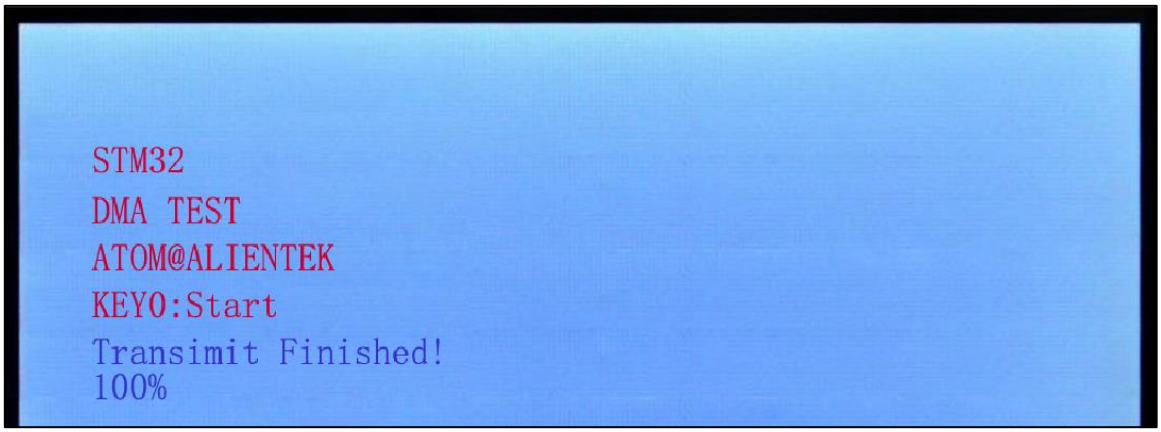


图 35.3.3 DMA 串口数据传输中

第三十六章 单通道 ADC 采集实验

本章介绍使用 STM32F103 模数转换器（ADC）进行带通道的电压采集。通过本章的学习，读者将学习到单通道 ADC 的使用。

本章分为如下几个小节：

36.1 硬件设计
36.2 程序设计
36.3 下载验证

36.1 硬件设计

1. 例程功能

使用 ADC1 采集通道 1（PA1）上面的电压，并在 LCD 模块上面显示 ADC 规则数据寄存器 12 位的转换值以及将该值换算成电压后的电压值。使用短路帽将 ADC 和 RV1 排针连接，使得 PA1 连接到电位器上，然后将 ADC 采集到的数据和转换后的电压值在 TFTLCD 屏中显示。用户可以通过调节电位器的旋钮改变电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）ADC1 ：通道 1 – PA1

3. 原理图

ADC 属于 STM32F103 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 的通道 1（PA1）来采集外部电压值，开发板有一个电位器，可调节的电压范围是：0~3.3V。我们可以通过跳线帽将 PA1 与电位器连接，如下图所示：

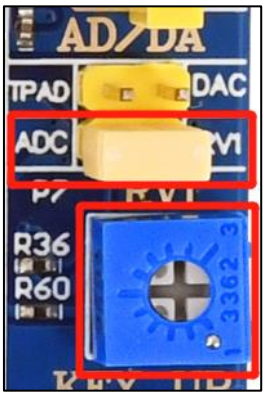


图 36.1.1 PA1（对应 ADC 排针）与电位器示意图

使用短路帽将 ADC 和 RV1 排针连接好后，并下载程序后，就可以用螺丝刀调节电位器变换多种电压值进行测试。

有的朋友可能还想测试其它地方的电压值，我们还可以通过杜邦线，一端接到 ADC 排针上，另外一端就接你要测试的电压点。**一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

36.2 程序设计

36.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 18_1_ADC_Single_Channel。

36.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 18_1_ADC_Single_Channel，以进行下一步的实验操作。如下图所示：

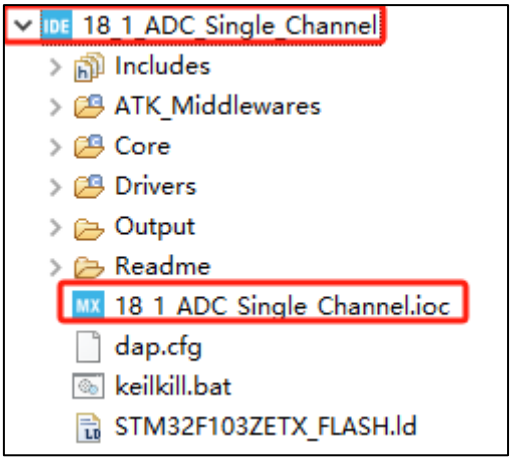


图 36.2.2.1 ADC_Single_Channel 工程

（1）引脚配置

接着双击 18_1_ADC_Single_Channel.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处配置 PA1。需注意，配置 PA1 复用为 ADC1_IN1，如下图所示：

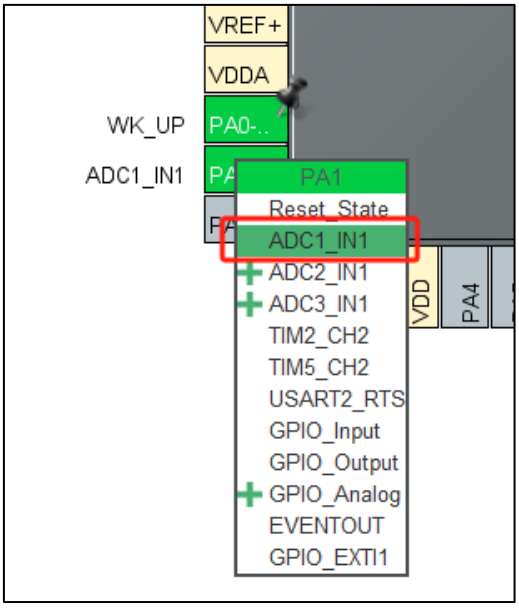


图 36.2.2.2 GPIO 复用

（2）配置 ADC 参数

在 Analog→ADC1 处配置 ADC。如下所示，有 IN0~IN15，表示 15 个通道，本实验用到 ADC1 的通道 1，所以勾选 IN1：

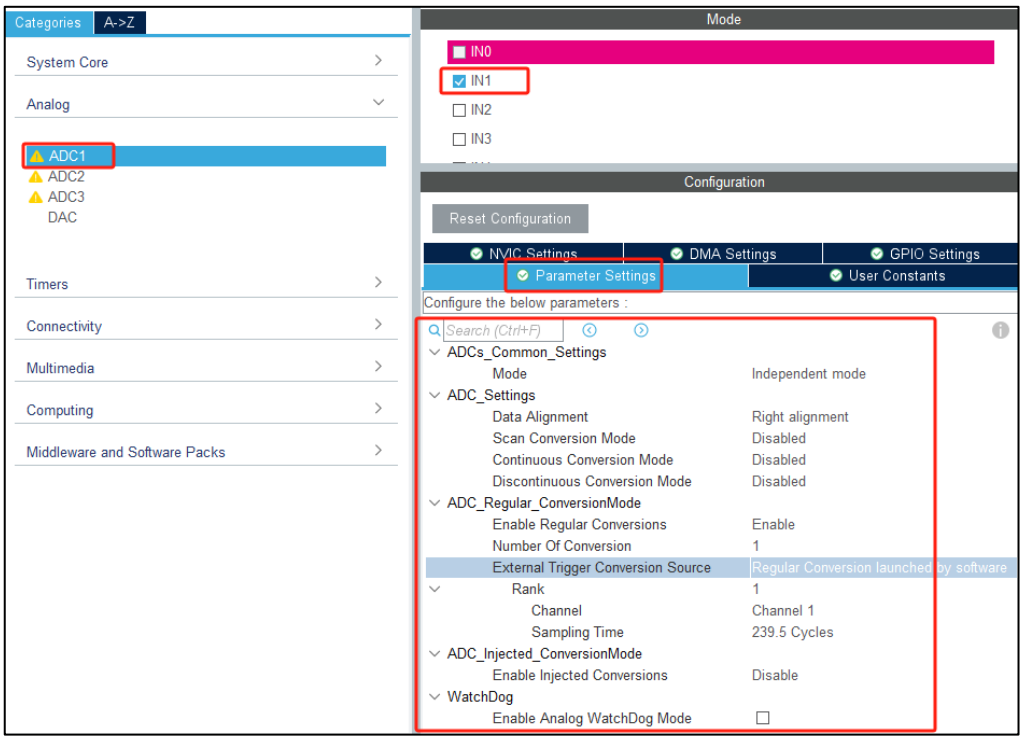


图 36.2.2.3 ADC1 模式配置

对以上的 ADC 参数配置选项进行介绍：

- ADCs_Common_Settings（ADC 工作模式配置）：

这里配置为独立模式，独立模式是指**在同一个管脚上只有一个 ADC 采集该管脚的电压信号**。如果只是用了一个 ADC 的时候就配置为独立模式。

除了独立模式，还有双重模式以及三重模式等多重模式，多重模式是指双 ADC 共同工作，如果需要两个 ADC 同步的话，则使用此模式。

- ADC_Settings（ADC 参数设置）

✧ Data Alignment 用于配置数据对齐方式，这里选择右对齐；

✧ Scan Conversion Mode 用于配置扫描模式，当有多个通道需要采集信号时必须开启扫描模式，此时 ADC 将会按设定的顺序轮流采集各通道信号，单通道转换不需要使用此功能，这里选择 Disabled；

✧ Continuous Conversion Mode 用于配置自动连续转换还是单次转换。使用 Enable 配置为使能自动连续转换；使用 Disabled 配置为单次转换，转换一次后停止需要手动控制才重新启动转换，我们选择 Disabled;

✧ Discontinuous Conversion Mode 用于配置是否使用不连续的转换模式，所谓不连续，比如要转换的通道有 1、2、5、7、8、9，那么第一次触发会进行通道 1 与通道 2，下次触发就是转换通道 5 与通道 7，这样不连续的转换，依次类推。这里我们选择禁用不连续的转换模式 Disabled；

- ADC Regular_ConversionMode（ADC常规通道转换模式）

✧ Enable Regular Conversions 选择 Enable，启用常规转换；

✧ Number Of Conversion 转换通道数量，此参数会影响可供设置的通道数，按实际使用的通道数来选择即可，这里是 1；

✧ External Trigger Conversion Source 外部触发转换模式配置，ADC 在接收到到触发信号后才开始进行模数转换，触发源可以是定时器触发、外部中断触发等硬件触发、也可以是软件控制触发，这里选择软件触发，工程中需要我们添加启动 ADC 的代码；

✧ Rank 配置模拟信号采集及转换的次序，默认是 1，其中：

✧ Channel 用于选择转换的通道，这里选择通道 1；

✧ Sampling Time 采样周期选择 239.5Cycles；

- ADC_Injected_ConversionMode（ADC注入通道转换模式）
  - Enable Injected Conversions 用于配置注入通道转换模式，实验中我们不需要使用注入通道，所以此项配为 Disable。
    后面的是模拟量看门狗的设置，本实验我们不需要，不配置即可。

36.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 adc.c 文件。adc.c 主要是完成 ADC1 的初始化。

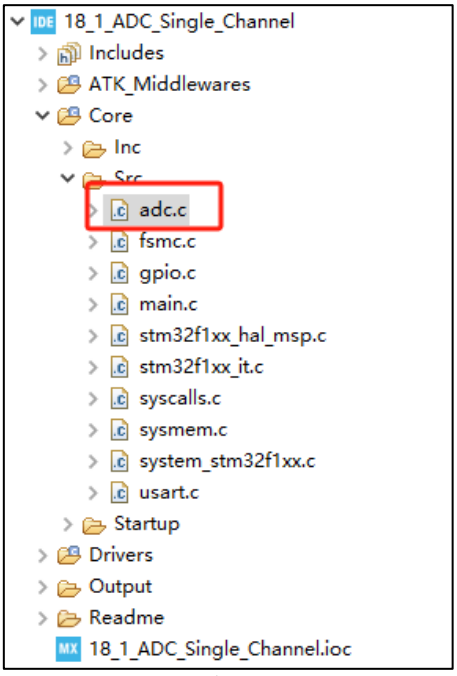


图 36.2.3.1 生成 ADC 工程

36.2.4 添加用户驱动代码

我们本实验主要修改三个文件，分别是：adc.c、adc.h、main.c，其中 adc.h 仅声明 adc.c 中的一些功能函数，这里不过多赘述此文件。我们首先对 adc.c 文件进行修改。

（1）修改 adc.c 文件

```
void MX_ADC1_Init(void)
{
  ADC_ChannelConfTypeDef sConfig = {0};

  hadc1.Instance = ADC1;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.NbrOfConversion = 1;
  if (HAL_ADC_Init(&hadc1) != HAL_OK)
  {
    Error_Handler();
  }

  sConfig.Channel = ADC_CHANNEL_1;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
```

```
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN ADC1_Init 2 */
  HAL_ADCEx_Calibration_Start(&hadc1);                    /* 校准 ADC */
  /* USER CODE END ADC1_Init 2 */

}

/* 省略部分代码… … */

/* USER CODE BEGIN 1 */
/**
 * @brief       设置 ADC 通道采样时间
 * @param       adcx : adc 句柄指针,ADC_HandleTypeDef
 * @param       ch   : 通道号, ADC_CHANNEL_0~ADC_CHANNEL_17
 * @param       stime: 采样时间  0~7, 对应关系为:
 *   @arg       ADC_SAMPLETIME_1CYCLE_5, 1.5 个 ADC 时钟周期
ADC_SAMPLETIME_7CYCLES_5, 7.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_13CYCLES_5, 13.5 个 ADC 时钟周期
ADC_SAMPLETIME_28CYCLES_5, 28.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_41CYCLES_5, 41.5 个 ADC 时钟周期
ADC_SAMPLETIME_55CYCLES_5, 55.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_71CYCLES_5, 71.5 个 ADC 时钟周期
ADC_SAMPLETIME_239CYCLES_5, 239.5 个 ADC 时钟周期
 * @param       rank: 多通道采集时需要设置的采集编号,
                假设你定义 channle1 的 rank=1，channle2 的 rank=2,
                那么对应你在 DMA 缓存空间的变量数组 AdcDMA[0] 就 i 是 channle1 的转换结
果，AdcDMA[1]就是通道 2 的转换结果。
                单通道 DMA 设置为 ADC_REGULAR_RANK_1
 *   @arg       编号 1~16: ADC_REGULAR_RANK_1~ADC_REGULAR_RANK_16
 * @retval      无
 */
void adc_channel_set(ADC_HandleTypeDef *adc_handle, uint32_t ch, uint32_t
rank, uint32_t stime)
{
    ADC_ChannelConfTypeDef adc_ch_conf;

    adc_ch_conf.Channel = ch;                               /* 通道 */
    adc_ch_conf.Rank = rank;                                /* 序列 */
    adc_ch_conf.SamplingTime = stime;                       /* 采样时间 */
    HAL_ADC_ConfigChannel(adc_handle, &adc_ch_conf);        /* 通道配置 */
}

/**
 * @brief       获得 ADC 转换后的结果
 * @param       ch: 通道值 0~17，取值范围为：ADC_CHANNEL_0~ADC_CHANNEL_17
 * @retval      无
 */
uint32_t adc_get_result(uint32_t ch)
{
    adc_channel_set(&hadc1 , ch, ADC_REGULAR_RANK_1,
    ADC_SAMPLETIME_239CYCLES_5);                /* 设置通道，序列和采样时间 */

    HAL_ADC_Start(&hadc1);                      /* 开启 ADC */
    HAL_ADC_PollForConversion(&hadc1, 10);      /* 轮询转换 */
    return (uint16_t)HAL_ADC_GetValue(&hadc1); /* 返回最近一次 ADC1 转换结果 */
}

/**
```

```
 * @brief       获取通道 ch 的转换值，取 times 次,然后平均
 * @param       ch      : 通道号, 0~17
 * @param       times   : 获取次数
 * @retval      通道 ch 的 times 次转换结果平均值
 */
uint32_t adc_get_result_average(uint32_t ch, uint8_t times)
{
    uint32_t temp_val = 0;
    uint8_t t;

    for (t = 0; t < times; t++)      /* 获取 times 次数据 */
    {
        temp_val += adc_get_result(ch);
        delay_ms(5);
    }

    return temp_val / times;         /* 返回平均值 */
}
/* USER CODE END 1 */
```

首先在函数 MX_ADC1_Init 中，初始化 ADC，参数配置在前面已经介绍过，这里不在赘述，然后我们在其中添加了 HAL_ADCEx_Calibration_Start 函数用于校准 ADC。

接着我们介绍我们自己编写的几个函数，第一个函数是 adc_channel_set，该函数主要是通过 HAL_ADC_ConfigChannel 函数选择要配置的 ADC 规则组通道，并设置通道的序列号和采样时间。

第二个函数是 adc_get_result，该函数先是调用我们自己定义的 adc_channel_set 函数选择 ADC 通道、设置转换序列号和采样时间等，接着调用 HAL_ADC_Start 启动转换，然后调用 HAL_ADC_PollForConversion 函数等待转换完成，最后调用 HAL_ADC_GetValue 函数获取转换结果。

第三个函数是 adc_get_result_average，该函数用于获取 ADC 多次转换结果的平均值，从而提高准确度。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint16_t adcx;
    float temp;
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* ADC1 初始化 */
  MX_ADC1_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                          /* 初始化 LCD */
```

```
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "ADC TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "ADC1_CH1_VAL:", BLUE);
  lcd_show_string(30, 130, 200, 16, 16, "ADC1_CH1_VOL:0.000V", BLUE);

  /* USER CODE END 2 */

  while (1)
  {
      /* 获取通道 5 的转换值，10 次取平均 */
      adcx = adc_get_result_average(ADC_CHANNEL_1, 10);
      /* 显示 ADCC 采样后的原始值 */
      lcd_show_xnum(134, 110, adcx, 5, 16, 0, BLUE);
      /* 获取计算后的带小数的实际电压值，比如 3.1111 */
      temp = (float)adcx * (3.3 / 4096);
      adcx = temp;       /* 赋值整数部分给 adcx 变量，因为 adcx 为 u16 整形 */
      /* 显示电压值的整数部分，3.1111 的话，这里就是显示 3 */
      lcd_show_xnum(134, 130, adcx, 1, 16, 0, BLUE);

      temp -= adcx;      /* 把已经显示的整数部分去掉，留下小数部分 */
      temp *= 1000;      /* 小数部分乘以 1000，相当于保留三位小数。 */
      /* 显示小数部分（前面转换为了整形显示），这里显示的就是 111. */
      lcd_show_xnum(150, 130, temp, 3, 16, 0X80, BLUE);

      LED0_TOGGLE();
      delay_ms(100);
  }
}
```

此部分代码，我们在 TFTLCD 模块上显示一些提示信息后，将每隔 100ms 读取一次 ADC 通道 1 的转换值，并显示读到的 ADC 值（数字量），以及其转换成模拟量后的电压值。同时控制 LED0 闪烁，以提示程序正在运行。ADC 值的显示简单介绍一下：首先在液晶固定位置显示了小数点，先计算出整数部分在小数点前面显示，然后计算出小数部分，在小数点后面显示。这样就能在液晶上面显示转换结果的整数和小数部分。

36.3 下载验证

下载代码后，可以看到 LCD 显示如图 36.3.1 所示：

图 36.3.1 单通道 ADC 采集实验测试图

上图中，我们使用短路帽将 ADC 和 RV1 排针连接，使得 PA1 连接到电位器上，测试电位器的电压，并可以通过螺丝刀调节电位器改变电压值，范围：0~3.3V。LED0 闪烁，提示程序运行。

大家也可以用杜邦线将 ADC 排针接到其它待测量的电压点，看看测量到的电压值是否准确？**但是要注意：一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

第三十七章 单通道 ADC 采集（DMA 读取）实验

本章介绍使用 STM32F103 的 DMA 进行单通道的 ADC 采集。通过本章的学习，读者将学习到 DMA、ADC 的使用。

本章分为如下几个小节：

37.1 硬件设计
37.2 程序设计
37.3 下载验证

37.1 硬件设计

1. 例程功能

使用 ADC 采集（DMA 读取）通道 1（PA1）上面的电压，在 LCD 模块上面显示 ADC 转换值以及换算成电压后的电压值。使用杜邦线将 ADC 和 RV1 排针连接，使得 PA1 连接到电位器上，然后将 ADC 采集到的数据和转换后的电压值在 TFTLCD 屏中显示。用户可以通过调节电位器的旋钮改变电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）ADC1：通道 1 - PA1
5）DMA（DMA1 通道 1）

3. 原理图

ADC 属于 STM32F103 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 的通道 1（PA1）来采集外部电压值，开发板有一个电位器，可调节的电压范围是：0~3.3V。我们可以通过跳线帽将 PA1 与电位器连接，如下图所示：

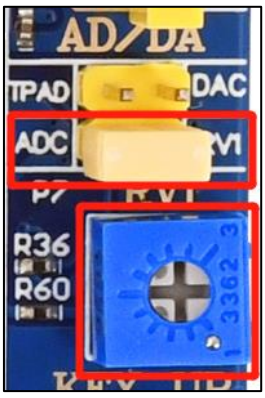


图 37.1.1 PA1（对应 ADC 排针）与电位器示意图

使用短路帽将 ADC 和 RV1 排针连接好后，并下载程序后，就可以用螺丝刀调节电位器变换多种电压值进行测量。

有的朋友可能还想测试其它地方的电压值，我们还可以通过杜邦线，一端接到 P7 的 ADC 排针上，另外一端就接你要测试的电压点。**一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

37.2 程序设计

37.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 18_2_ADC_DMA_Single_Channel。

37.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 18_1_ADC_Single_Channel 工程，并将工程文件名和.ioc 文件名均修改为 18_2_ADC_DMA_Single_Channel，以进行下一步的实验操作。如下图所示：

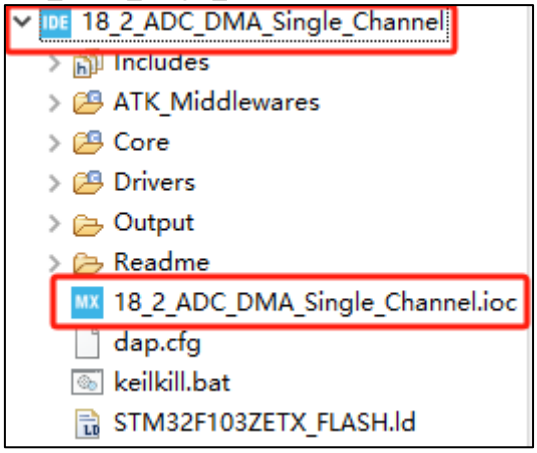


图 37.2.2.1 ADC_DMA_Single_Channel 工程

由于我们是基于上个工程来实现的，所以 ADC 引脚，无需另外去配置，我们主要修改下 ADC 参数以及配置 DMA 即可。

（1）配置 ADC 参数

该配置相较于上一章节只需修改一处，将 Continuous Conversion Mode 设置为 Enable，表示使能自动连续转换。如下图所示：

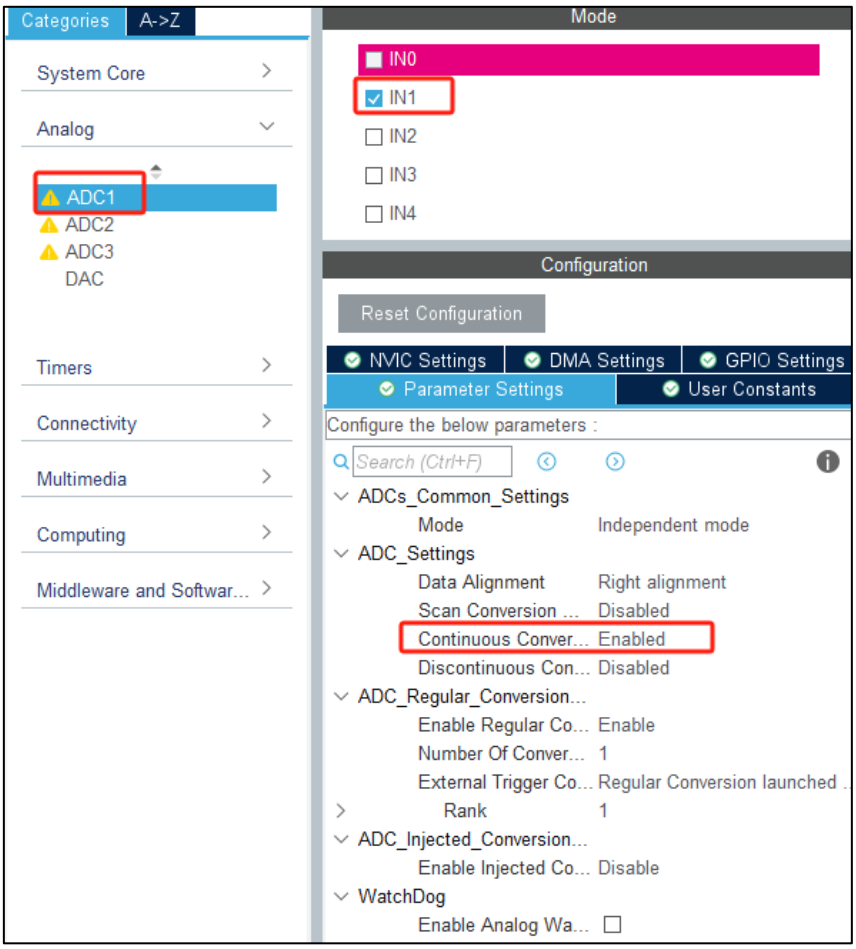


图 37.2.2.2 ADC 参数配置

其余配置与上一章节基本一致，我们这里不在赘述！详情可查看 36.2.2 小节。

（2）配置 DMA 参数

本实验我们要使用 DMA 将 ADC 转换后存在 ADC_DR 寄存器的数据，传输到自定义的一段内存中，所以需要配置 DMA。

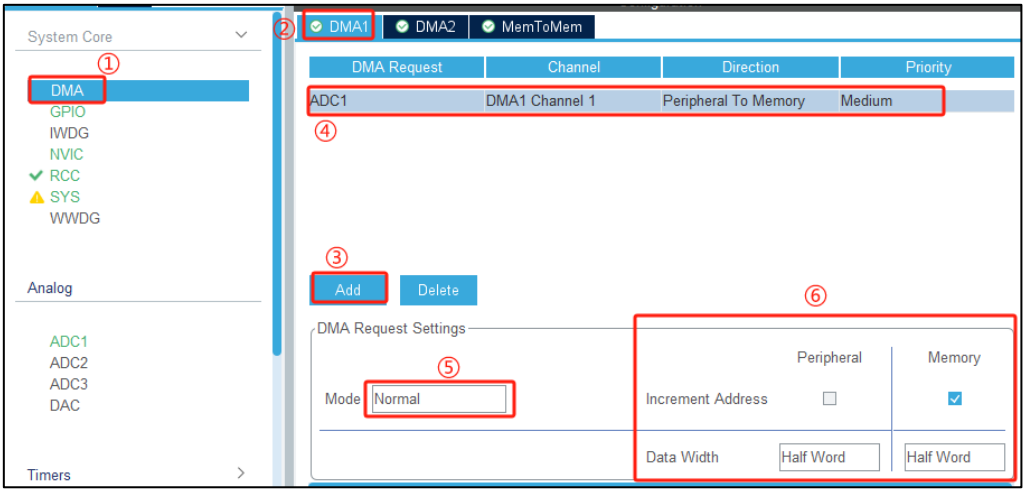


图 37.2.2.3 DMA 参数配置

我们选择通道为 DMA1_Channel 1，数据方向为外设到内存，速度等级为中等，DMA 请求为正常模式（Normal），勾选内存递增，要注意，这里选择字宽为半字。

（3）配置 NVIC 参数

下面设置 DMA 的中断优先级：

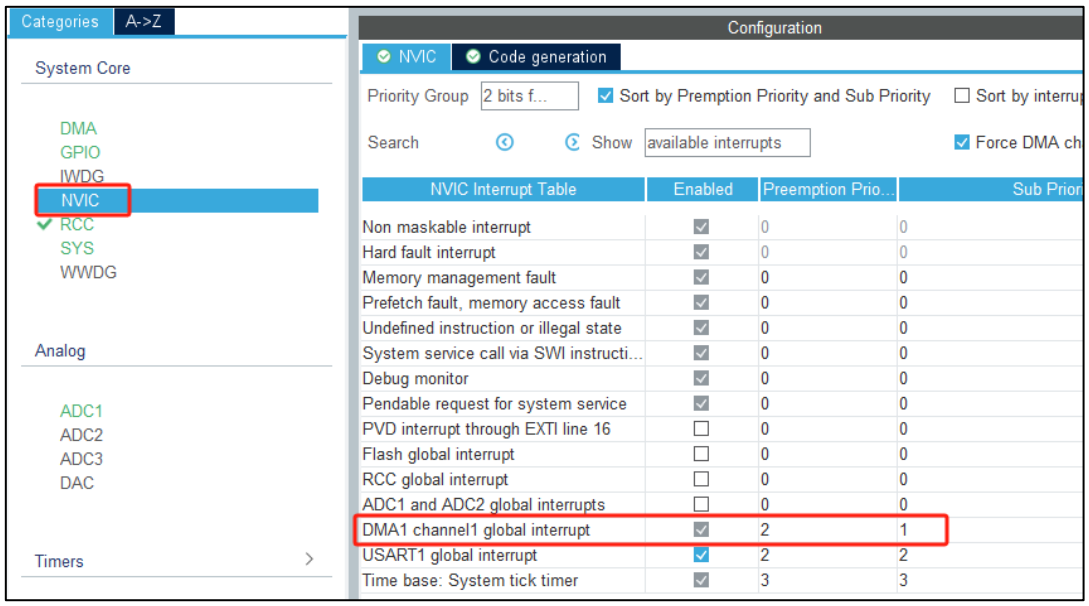


图 37.2.2.4 DMA 中断优先级配置

37.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图：

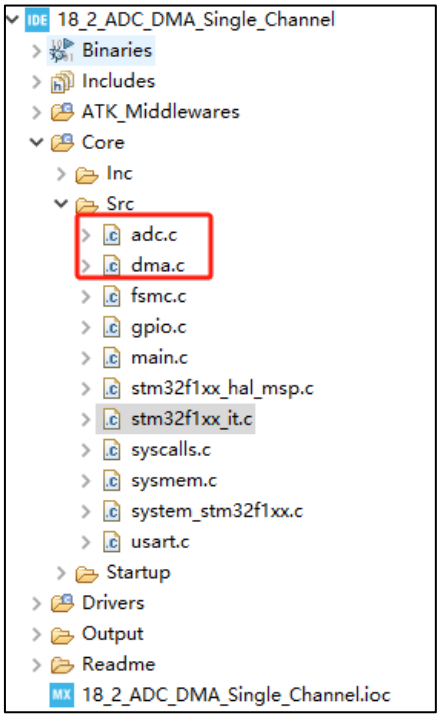


图 37.2.3.1 生成工程

37.2.4 添加用户驱动代码

我们本实验主要讲解以下几个文件，分别是：adc.c、dma.c、main.c、stm32f1xx_it.c。我们首先对 dma.c 文件进行讲解。

（1）dma.c 文件

```
void MX_DMA_Init(void)
{
```

```
  /* DMA controller clock enable */
  __HAL_RCC_DMA1_CLK_ENABLE();

  /* DMA interrupt init */
  /* DMA1_Channel1_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 2, 1);
  HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);

}
```

在 DMA 的初始化函数中，主要就是使能 DMA1 的时钟，并使能 DMA1 通道一的中断以及配置中断优先级。该文件不需要改动。

（2） 修改 adc.c 文件

```
void MX_ADC1_Init(void)
{
  ADC_ChannelConfTypeDef sConfig = {0};
  hadc1.Instance = ADC1;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode = ENABLE;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.NbrOfConversion = 1;
  if (HAL_ADC_Init(&hadc1) != HAL_OK)
  {
    Error_Handler();
  }
  sConfig.Channel = ADC_CHANNEL_1;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN ADC1_Init 2 */
  HAL_ADCEx_Calibration_Start(&hadc1);              /* 校准 ADC */
  /* USER CODE END ADC1_Init 2 */

}

/* 省略部分代码… … */

/* USER CODE BEGIN 1 */
/**
 * @brief       使能一次 ADC DMA 传输
 *   @note      该函数用寄存器来操作，防止用 HAL 库操作对其他参数有修改,也为了兼容性
 * @param       ndtr: DMA 传输的次数
 * @retval      无
 */
void adc_dma_enable(uint16_t cndtr)
{
    ADC1->CR2 &= ~(1 << 0);                     /* 先关闭 ADC */

    DMA1_Channel1->CCR &= ~(1 << 0);            /* 关闭 DMA 传输 */
    while (DMA1_Channel1->CCR & (1 << 0));      /* 确保 DMA 可以被设置 */
    DMA1_Channel1->CNDTR = cndtr;               /* DMA 传输数据量 */
    DMA1_Channel1->CCR |= 1 << 0;               /* 开启 DMA 传输 */

    ADC1->CR2 |= 1 << 0;                        /* 重新启动 ADC */
    ADC1->CR2 |= 1 << 22;                       /* 启动规则转换通道 */
}
```

```
/* USER CODE END 1 */
```

首先在函数 MX_ADC1_Init 中，初始化 ADC 以及 DMA 的一些参数，参数配置在前面已经介绍过，这里不在赘述，然后我们在其中添加了 HAL_ADCEx_Calibration_Start 函数用于校准 ADC。

接着是函数 adc_dma_enable，该函数我们使用寄存器来操作，防止用 HAL 库相关宏操作会对其它参数进行修改，同时也是为了兼容后面的实验。HAL_DMA_Start_IT 函数已经配置好了 DMA 传输的源地址和目标地址，本函数只需要调用 ADC_ADCX_DMACx->CNDTR = cndtr;语句给 DMA_CNDTRx 寄存器写入要传输的数据量，然后启动 DMA 就可以传输了。

（3） 修改 stm32f1xx_it.c 文件

```
void DMA1_Channel1_IRQHandler(void)
{
  /* USER CODE BEGIN DMA1_Channel1_IRQn 0 */
    if (ADC_ADCX_DMACx_IS_TC())
    {
        g_adc_dma_sta = 1;                  /* 标记 DMA 传输完成 */
        ADC_ADCX_DMACx_CLR_TC();            /* 清除 DMA1 数据流 7 传输完成中断 */
    }
    HAL_DMA_IRQHandler(&hdma_adc1);

  /* USER CODE END DMA1_Channel1_IRQn 1 */
}
```

在 DMA 的采集中断中我们添加了以上代码。在该函数里，通过判断 DMA 传输完成标志位是否是 1，是 1 就给 g_adc_dma_sta 变量赋值为 1，标记 DMA 传输完成，最后清除 DMA 的传输完成标志位。

（4） 修改 main.c 文件

```
/* USER CODE BEGIN PFP */
#define ADC_DMA_BUF_SIZE        100         /* ADC DMA 采集 BUF 大小 */
uint16_t g_adc_dma_buf[ADC_DMA_BUF_SIZE];  /* ADC DMA BUF */
extern uint8_t g_adc_dma_sta;              /* DMA 状态标志, 0,未完成; 1, 已完成 */
/* USER CODE END PFP */

int main(void)
{
  /* USER CODE BEGIN 1 */
    uint16_t i;
    uint16_t adcx;
    uint32_t sum;
    float temp;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* DMA 初始化 */
  MX_DMA_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();
```

```
  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* ADC1 初始化 */
  MX_ADC1_Init();

  /* USER CODE BEGIN 2 */
  /* 启动 DMA，并开启中断 */
HAL_DMA_Start_IT(&hdma_adc1, (uint32_t)&ADC1->DR, (uint32_t)&g_adc_dma_buf,0);
  /* 开启 ADC，通过 DMA 传输结果 */
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&g_adc_dma_buf, 0);

  lcd_init();                             /* 初始化 LCD */
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "ADC DMA TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "ADC1_CH1_VAL:", BLUE);
  lcd_show_string(30, 130, 200, 16, 16, "ADC1_CH1_VOL:0.000V", BLUE);

  adc_dma_enable(ADC_DMA_BUF_SIZE);     /* 启动 ADC DMA 采集 */
  /* USER CODE END 2 */

  while (1)
  {
      if (g_adc_dma_sta == 1)
      {
          /* 计算 DMA 采集到的 ADC 数据的平均值 */
          sum = 0;

          for (i = 0; i < ADC_DMA_BUF_SIZE; i++)    /* 累加 */
          {
              sum += g_adc_dma_buf[i];
          }

          adcx = sum / ADC_DMA_BUF_SIZE;            /* 取平均值 */

          /* 显示结果 */
          lcd_show_xnum(134, 110, adcx, 4, 16, 0, BLUE);

          temp = (float)adcx * (3.3 / 4096);   /* 获取计算后的带小数的实际电压值*/
          adcx = temp;                  /* 赋值整数部分给 adcx 变量，因为 adcx 为 u16 整形 */
          lcd_show_xnum(134, 130, adcx, 1, 16, 0, BLUE);

          temp -= adcx;                 /* 把已经显示的整数部分去掉，留下小数部分*/
          temp *= 1000;                 /* 小数部分乘以 1000 */
          lcd_show_xnum(150, 130, temp, 3, 16, 0X80, BLUE);

          g_adc_dma_sta = 0;    /* 清除 DMA 采集完成状态标志 */
          adc_dma_enable(ADC_DMA_BUF_SIZE);         /* 启动下一次 ADC DMA 采集 */
      }

      LED0_TOGGLE();
      delay_ms(100);
  }
}
```

此部分代码，和单通道 ADC 采集实验十分相似，只是这里使能了 DMA 传输数据，DMA 传输的数据存放在 g_adc_dma_buf 数组里，这里我们对数组的数据取平均值，减少误差。在 LCD 屏显示结果的处理和单通道 ADC 采集实验一样。首先在液晶固定位置显示了小数点，先计算出整数部分在小数点前面显示，然后计算出小数部分，在小数点后面显示。这样就能在液晶上面显示转换结果的整数和小数部分。

37.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 显示如图 37.3.1 所示：

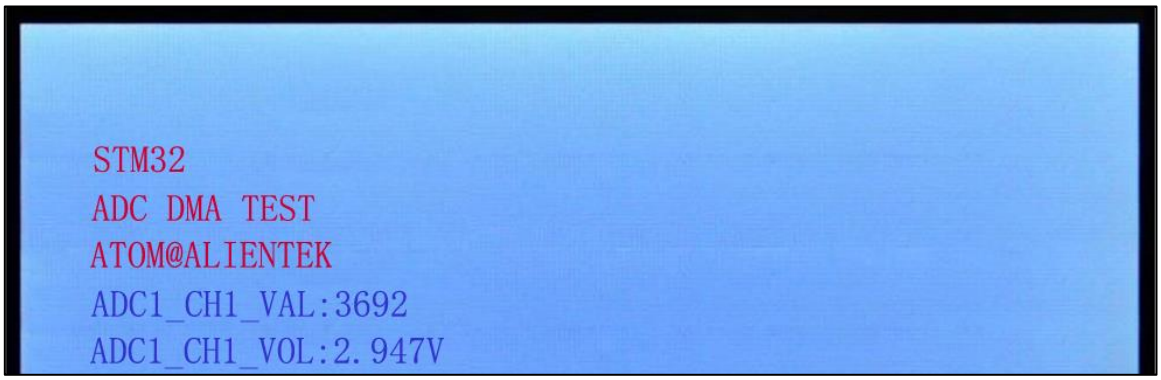


图 37.3.1 单通道 ADC 采集（DMA 读取）实验测试图

上图中，我们使用短路帽将 P7 的 ADC 和 RV1 连接，使得 PA1 连接到电位器上，测试电位器的电压，并可以通过螺丝刀调节电位器改变电压值，范围：0~3.3V。LED0 闪烁，提示程序运行。

大家也可以用杜邦线将 ADC 排针接到其它待测量的电压点，看看测量到的电压值是否准确？**但是要注意：一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

第三十八章 多通道 ADC 采集（DMA 读取）实验

本章介绍 STM32F103 的 DMA 进行多通道的 ADC 采集。通过本章的学习，读者将学习到 DMA、ADC 的使用。

本章分为如下几个小节：

38.1 硬件设计
38.2 程序设计
38.3 下载验证

38.1 硬件设计

1. 例程功能

使用 ADC1 采集（DMA 读取）通道 1\2\3\4\5 的电压，在 LCD 模块上面显示对应的 ADC 转换值以及换算成电压后的电压值。可以使用杜邦线连接 PA1\PA2\PA3\PA4\PA5 到你想测量的电压源（0~3.3V），然后通过 TFTLCD 显示的电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PE5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）ADC1： 通道 1 - PA1、通道 2 - PA2、通道 3 - PA3、通道 4 - PA4、通道 5 - PA5
5）DMA（DMA1 通道 1）

3. 原理图

ADC 和 DMA 属于 STM32F103 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 的通道 1\2\3\4\5 来采集外部电压值，并通过 DMA 来读取。

38.2 程序设计

38.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 18_3_ADC_DMA_Multi_Channel。

38.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 18_2_ADC_DMA_Single_Channel 工程，并将工程文件名和.ioc 文件名均修改为 18_3_ADC_DMA_Multi_Channel，以进行下一步的实验操作。如下图所示：

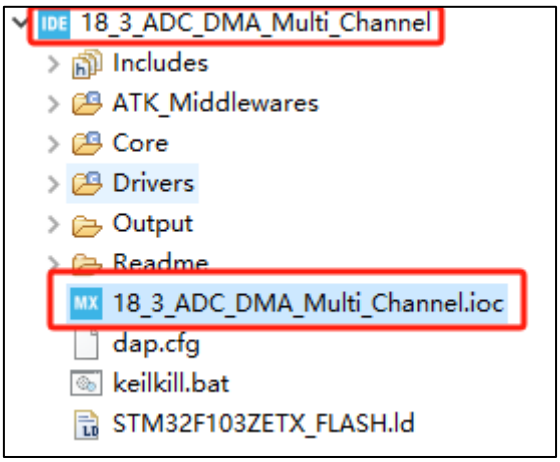


图 38.2.2.1 ADC_DMA_ Multi _Channel 工程

（1）配置 ADC1 的通道

接下来先配置 ADC1 通道的引脚，本实验我们使用 ADC1 的输入通道 1\2\3\4\5，根据上面的硬件资源表格来配置 ADC1 各个通道为单端输入模式模式，如下图所示：

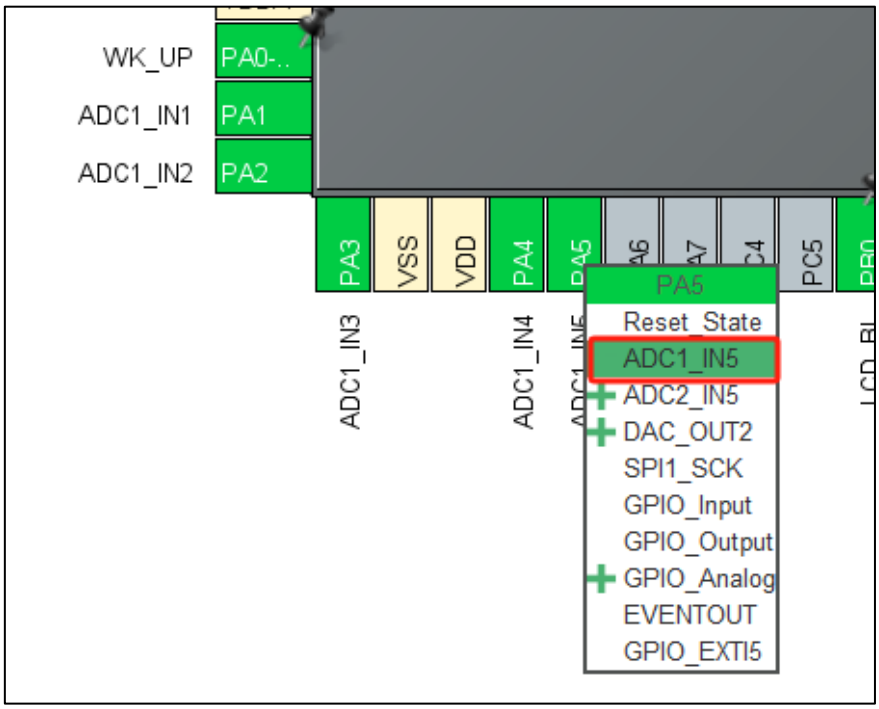


图 38.2.2.2 ADC 引脚配置

在 Analog→ADC1 处配置 ADC。如下所示，有 IN0~IN15，表示 15 个通道，本实验用到 ADC1 的通道 1~通道 5，所以勾选 IN1~IN5：

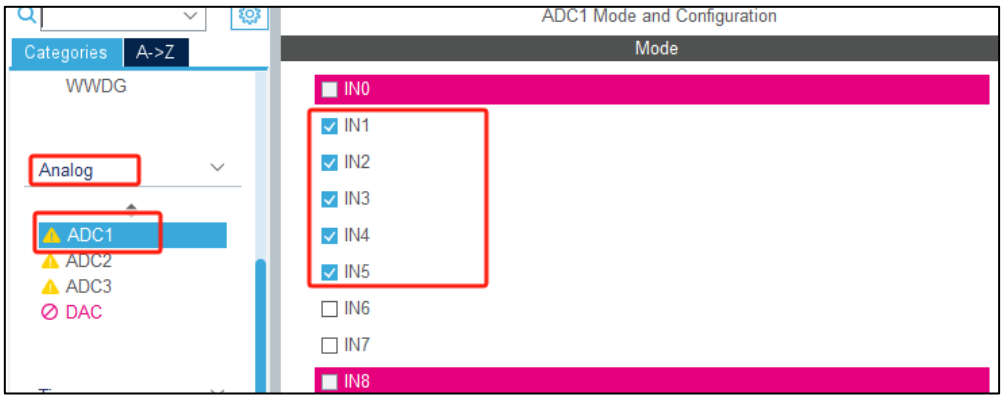


图 38.2.2.3 ADC 通道选择

接下来配置 ADC1 的参数，如下：

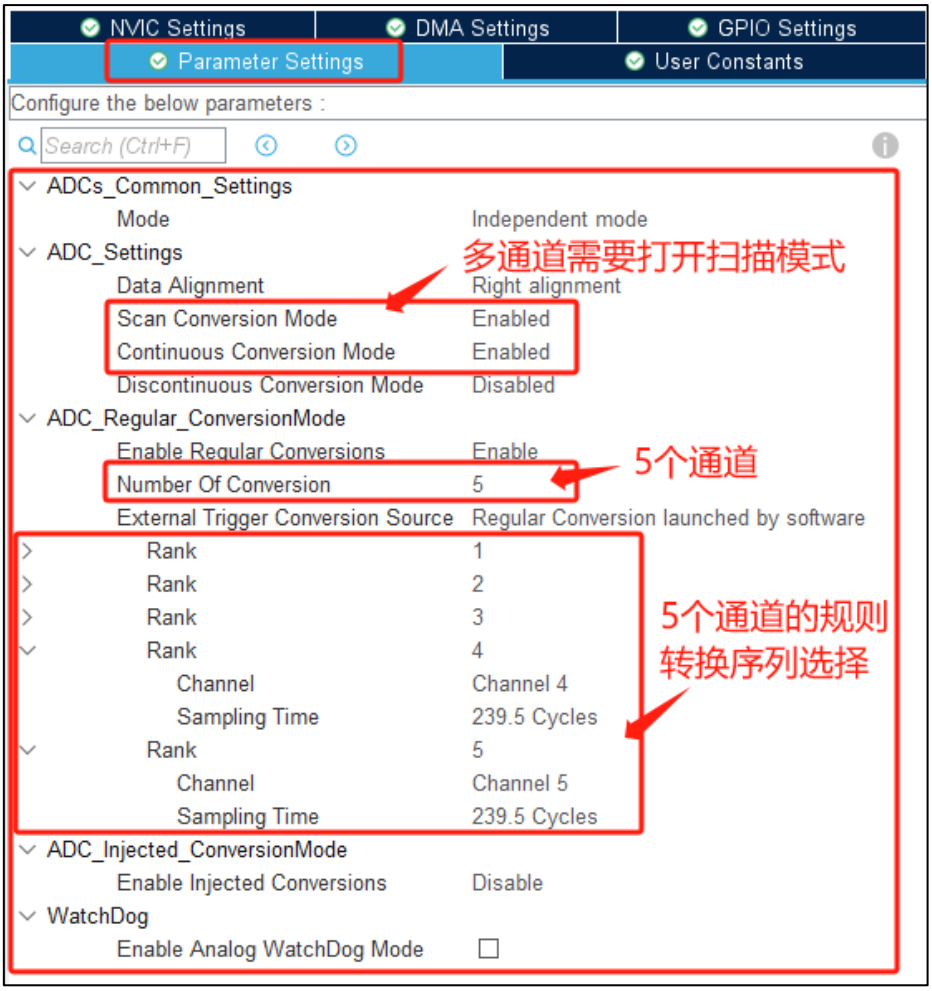


图 38.2.2.4 ADC 参数配置

以上部分配置参数我们已经讲过，下面介绍与前面配置不一样的地方：

- ✧ Scan Conversion Mode 用于配置扫描模式，当有多个通道需要采集信号时此项会需开启；
- ✧ Number Of Conversion 转换通道数量，此参数会影响可供设置的通道数，按实际使用的通道数来选择即可，这里我们使用 5 个通道，所以配置为 5；
- ✧ Rank 配置模拟信号采集及转换的次序，总共有 5 组，我们设置通道 1\2\3\4\5 按照先后顺序转换，最先转换的是通道 1，所以设置序列 Rank1 处是 Channel1，其它通道先后顺序依次类推。
- ✧ Channel 用于选择转换的通道；
- ✧ Sampling Time 采样周期选择 239.5Cycles；

（2）配置 DMA

实验我们要使用 DMA 将 ADC 转换后存在 ADC_DR 寄存器的数据传输到自定义的一段内存中，所以需要配置 DMA。

该步骤的配置与上一章节完全一致，参数详情这里不过多赘述。参数配置如下图：

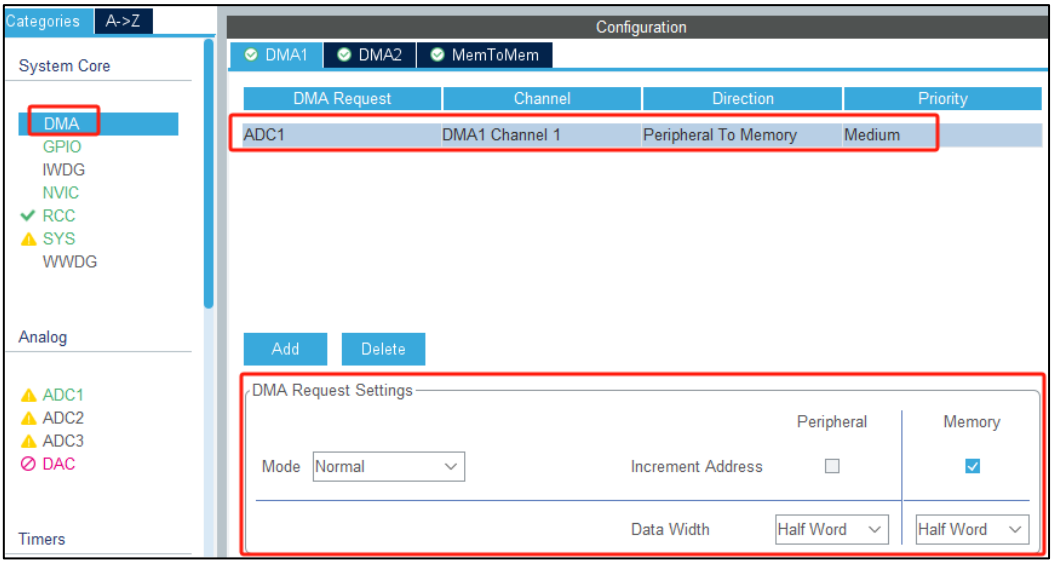


图 38.2.2.5 配置 DMA 模式

38.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图：

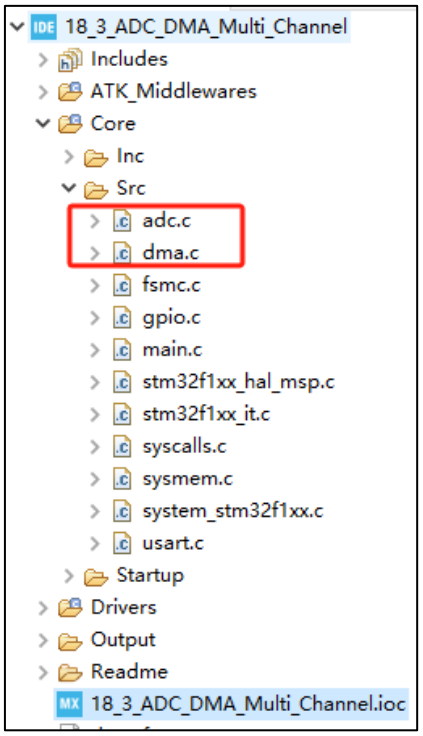


图 38.2.3.1 生成工程

38.2.4 添加用户驱动代码

ADC1 和 DMA 的初始化代码在 adc.c 和 dma.c 文件及其头文件中，初始化代码我们这里

就不再分析了，前面的实验我们已经分析过，下面我们直接添加测试代码。

（1）修改 stm32f1xx_it.c 文件

```
void DMA1_Channel1_IRQHandler(void)
{
  /* USER CODE BEGIN DMA1_Channel1_IRQn 0 */
    if (ADC_ADCX_DMACx_IS_TC())
    {
        g_adc_dma_sta = 1;              /* 标记 DMA 传输完成 */
        ADC_ADCX_DMACx_CLR_TC();        /* 清除 DMA1 数据流 7 传输完成中断 */
    }
    HAL_DMA_IRQHandler(&hdma_adc1);
  /* USER CODE END DMA1_Channel1_IRQn 1 */
}
```

和上一章节的实验一样，我们通过判断 DMA 传输完成标志位是否是 1，是 1 就给 g_adc_dma_sta 变量赋值为 1，标记 DMA 传输完成，最后清除 DMA 的传输完成标志位。

（2）修改 main.c 文件

```
/* USER CODE BEGIN PFP */
#define ADC_DMA_BUF_SIZE        50 * 5      /* ADC DMA 采集 BUF 大小*/
uint16_t g_adc_dma_buf[ADC_DMA_BUF_SIZE];   /* ADC DMA BUF */
extern uint8_t g_adc_dma_sta;               /* DMA 传输状态标志*/
/* USER CODE END PFP */

int main(void)
{
  /* USER CODE BEGIN 1 */
    uint16_t i,j;
    uint16_t adcx;
    uint32_t sum;
    float temp;
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* DMA 初始化 */
  MX_DMA_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* ADC1 初始化 */
  MX_ADC1_Init();

  /* USER CODE BEGIN 2 */
  /* 启动 DMA，并开启中断 */
  HAL_DMA_Start_IT(&hdma_adc1, (uint32_t)&ADC1->DR, (uint32_t)&g_adc_dma_buf,
  0);
  /* 开启 ADC，通过 DMA 传输结果 */
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&g_adc_dma_buf, 0);
```

```
    lcd_init();                     /* 初始化 LCD */
    lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30,  70, 200, 16, 16, "ADC 6CH DMA TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 12, 12, "ADC1_CH1_VAL:", BLUE);
    lcd_show_string(30, 122, 200, 12, 12, "ADC1_CH1_VOL:0.000V", BLUE);

    lcd_show_string(30, 140, 200, 12, 12, "ADC1_CH2_VAL:", BLUE);
    lcd_show_string(30, 152, 200, 12, 12, "ADC1_CH2_VOL:0.000V", BLUE);

    lcd_show_string(30, 170, 200, 12, 12, "ADC1_CH3_VAL:", BLUE);
    lcd_show_string(30, 182, 200, 12, 12, "ADC1_CH3_VOL:0.000V", BLUE);
    lcd_show_string(30, 200, 200, 12, 12, "ADC1_CH4_VAL:", BLUE);
    lcd_show_string(30, 212, 200, 12, 12, "ADC1_CH4_VOL:0.000V", BLUE);
    lcd_show_string(30, 230, 200, 12, 12, "ADC1_CH5_VAL:", BLUE);
    lcd_show_string(30, 242, 200, 12, 12, "ADC1_CH5_VOL:0.000V", BLUE);
    adc_dma_enable(ADC_DMA_BUF_SIZE);    /* 启动 ADC DMA 采集 */
    /* USER CODE END 2 */

    while (1)
    {
        if (g_adc_dma_sta == 1)
        {
            /* 循环显示通道 4~通道 5 的结果 */
            for(j = 0; j < 5; j++)  /* 遍历 5 个通道 */
            {
                sum = 0; /* 清零 */
                /* 每个通道采集了 10 次数据,进行 10 次累加 */
                for (i = 0; i < ADC_DMA_BUF_SIZE / 5; i++)
                {
                    sum += g_adc_dma_buf[(5 * i) + j];   /* 相同通道的转换数据累加*/
                }
                adcx = sum / (ADC_DMA_BUF_SIZE / 5);     /* 取平均值 */

                /* 显示结果 */
                lcd_show_xnum(108, 110 + (j * 30), adcx, 4, 12, 0, BLUE);
                temp = (float)adcx * (3.3 / 4096);  /* 计算后的带小数的实际电压值*/
                adcx = temp;                        /* 赋值整数部分给 adcx 变量*/
                lcd_show_xnum(108, 122 + (j * 30), adcx, 1, 12, 0, BLUE);

                temp -= adcx;
                temp *= 1000;
                lcd_show_xnum(120, 122 + (j * 30), temp, 3, 12, 0X80, BLUE);
            }

            g_adc_dma_sta = 0;                      /* 清除 DMA 采集完成状态标志 */
            adc_dma_enable(ADC_DMA_BUF_SIZE);       /* 启动下一次 ADC DMA 采集 */
        }

        LED0_TOGGLE();
        delay_ms(100);
    }
}
```

这里使用了 DMA 传输数据，DMA 传输的数据存放在 g_adc_dma_buf 数组里，该数组的大小是 50 * 5 本实验用到 5 通道，每个通道使用 50 个 uint16_t 大小的空间存放 ADC 的结果。输入通道 1 转换数据存放在 g_adc_dma_buf[0]到 g_adc_dma_buf[49]，输入通道 2 转换数据存放在 g_adc_dma_buf[50]到 g_adc_dma_buf[99]，后面的以此类推。然后对数组的每个通道的数据取平均值，减少误差。最后在 LCD 屏上显示 ADC 的转换值和换算成电压后的电压值。

38.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上不断地刷新显示 ADC1 通道 1~通道 5（PA1 引脚~PA5 引脚）采集到电压的数字量和模拟量，此时可以通过杜邦线给 PA1 至 PA5 中的任意一个或多个引脚接入不同的电压值（注意共地，且输入电压不能超过 3.3V，否则可能损坏开发板），可以看到 LCD 上显示的电压数字量和模拟量也随之改变。

第三十九章 内部温度传感器实验

本章介绍使用 ADC 采集 STM32F103 内部温度传感器输出的电压值，并根据该电压值计算出 STM32F103 芯片的大致温度。通过本章的学习，读者将学习到 ADC、内部温度传感器的使用。

本章分为如下几个小节：

39.1 硬件设计
39.2 程序设计
39.3 下载验证

39.1 硬件设计

1. 例程功能

通过 ADC 的通道 16 读取 STM32F103 内部温度传感器的电压值，并将其转换为温度值，显示在 TFTLCD 屏上。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）ADC1 通道 16
5）内部温度传感器

3. 原理图

ADC 和内部温度传感器都属于 STM32F103 内部资源，实际上我们只需要软件设置就可以正常工作，我们需要用到 TFTLCD 模块显示结果。

39.2 程序设计

39.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 19_ADC_Temperature。

39.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 19_ADC_Temperature，以进行下一步的实验操作。如下图所示：

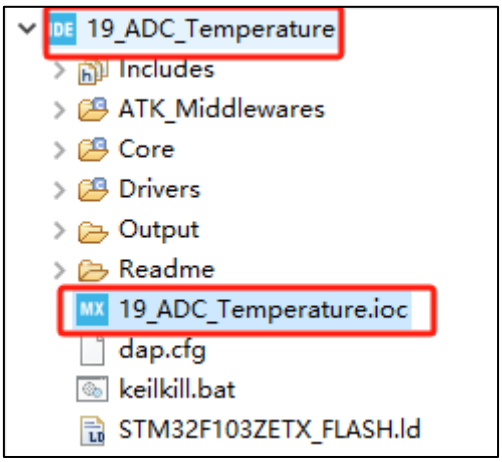


图 39.2.2.1 ADC_Temperature 工程

（1）配置 ADC1 的通道

接下来先配置 ADC1 通道的引脚，本实验我们使用 ADC1 的通道 16，因为温度传感器在内部和 ADC1_IN16 输入通道相连接，配置如下图所示：

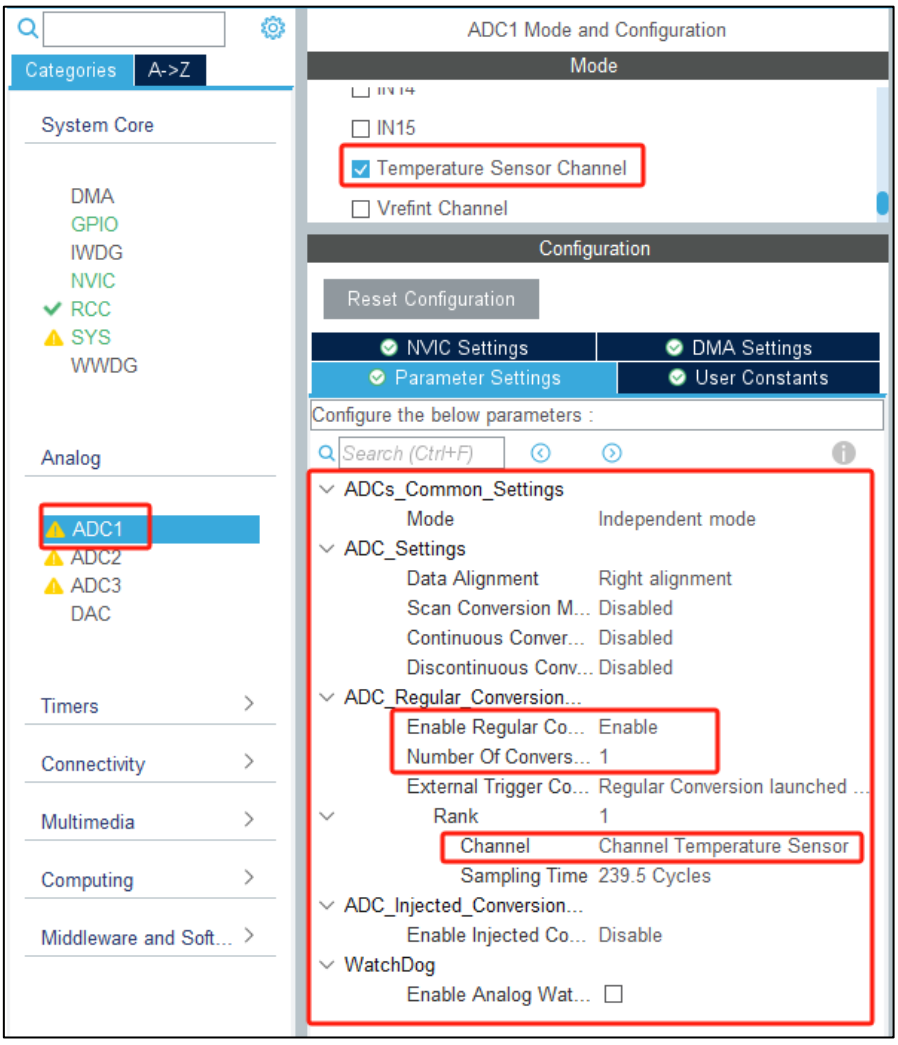


图 39.2.2.2 ADC1 参数配置

前面章节中已对这些参数进行了详细介绍，因此在此处不再赘述。需要注意的是，在配置过程中需要选择 Temperature Sensor Channel。

39.2.3 生成初始化代码

按下“Ctrl+S”保存配置，生成工程，如下图：

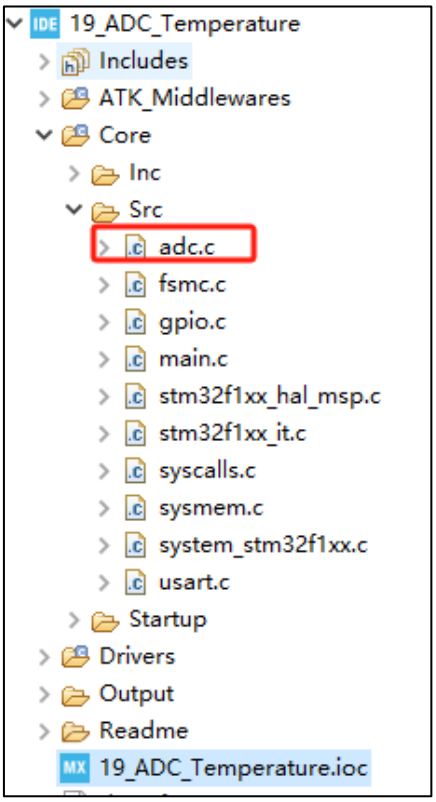


图 39.2.3.1 生成工程

39.2.4 添加用户驱动代码

我们本实验主要修改三个文件，分别是：adc.c、adc.h、main.c，其中 adc.h 仅声明 adc.c 中的一些功能函数，这里不过多赘述此文件。我们首先对 adc.c 文件进行修改。

（1）修改 adc.c 文件

```
void MX_ADC1_Init(void)
{
  ADC_ChannelConfTypeDef sConfig = {0};
  hadc1.Instance = ADC1;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.NbrOfConversion = 1;
  if (HAL_ADC_Init(&hadc1) != HAL_OK)
  {
    Error_Handler();
  }
  sConfig.Channel = ADC_CHANNEL_TEMPSENSOR;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN ADC1_Init 2 */
```

```
  HAL_ADCEx_Calibration_Start(&hadc1);                    /* 校准 ADC */
  /* USER CODE END ADC1_Init 2 */

}
/* 省略部分代码… … */

/* USER CODE BEGIN 1 */
/**********************内部温度传感器实验代码**********************/
/**
 * @brief       设置 ADC 通道采样时间
 * @param       adcx : adc 句柄指针,ADC_HandleTypeDef
 * @param       ch   : 通道号, ADC_CHANNEL_0~ADC_CHANNEL_17
 * @param       stime: 采样时间  0~7, 对应关系为:
 *   @arg       ADC_SAMPLETIME_1CYCLE_5, 1.5 个 ADC 时钟周期
ADC_SAMPLETIME_7CYCLES_5, 7.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_13CYCLES_5, 13.5 个 ADC 时钟周期
ADC_SAMPLETIME_28CYCLES_5, 28.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_41CYCLES_5, 41.5 个 ADC 时钟周期
ADC_SAMPLETIME_55CYCLES_5, 55.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_71CYCLES_5, 71.5 个 ADC 时钟周期
ADC_SAMPLETIME_239CYCLES_5, 239.5 个 ADC 时钟周期
 * @param       rank: 多通道采集时需要设置的采集编号,
                假设你定义 channle1 的 rank=1，channle2 的 rank=2,
                那么对应你在 DMA 缓存空间的变量数组 AdcDMA[0] 就 i 是 channle1 的转换结
果，AdcDMA[1]就是通道 2 的转换结果。
                单通道 DMA 设置为 ADC_REGULAR_RANK_1
 *   @arg       编号 1~16: ADC_REGULAR_RANK_1~ADC_REGULAR_RANK_16
 * @retval      无
 */
void adc_channel_set(ADC_HandleTypeDef *adc_handle, uint32_t ch, uint32_t
rank, uint32_t stime)
{
    ADC_ChannelConfTypeDef adc_ch_conf;

    adc_ch_conf.Channel = ch;                               /* 通道 */
    adc_ch_conf.Rank = rank;                                /* 序列 */
    adc_ch_conf.SamplingTime = stime;                       /* 采样时间 */
    HAL_ADC_ConfigChannel(adc_handle, &adc_ch_conf);        /* 通道配置 */
}

/**
 * @brief       获得 ADC 转换后的结果
 * @param       ch: 通道值 0~17，取值范围为: ADC_CHANNEL_0~ADC_CHANNEL_17
 * @retval      无
 */
uint32_t adc_get_result(uint32_t ch)
{
    adc_channel_set(&hadc1 , ch, ADC_REGULAR_RANK_1,
    ADC_SAMPLETIME_239CYCLES_5);                        /* 设置通道，序列和采样时间 */

    HAL_ADC_Start(&hadc1);                              /* 开启 ADC */
    HAL_ADC_PollForConversion(&hadc1, 10);              /* 轮询转换 */
    return (uint16_t)HAL_ADC_GetValue(&hadc1);          /* 返回 ADC1 规则组的转换结果 */
}

/**
 * @brief       获取通道 ch 的转换值，取 times 次,然后平均
 * @param       ch      : 通道号, 0~17
 * @param       times   : 获取次数
 * @retval      通道 ch 的 times 次转换结果平均值
 */
```

```
uint32_t adc_get_result_average(uint32_t ch, uint8_t times)
{
    uint32_t temp_val = 0;
    uint8_t t;

    for (t = 0; t < times; t++)      /* 获取 times 次数据 */
    {
        temp_val += adc_get_result(ch);
        delay_ms(5);
    }

    return temp_val / times;         /* 返回平均值 */
}

/**
 * @brief       获取内部温度传感器温度值
 * @param       无
 * @retval      温度值(扩大了 100 倍,单位:℃.)
 */
short adc_get_temperature(void)
{
    uint32_t adcx;
    short result;
    double temperature;
    /* 读取内部温度传感器通道,10 次取平均 */
    adcx = adc_get_result_average(ADC_CHANNEL_TEMPSENSOR, 20);
    temperature = (float)adcx * (3.3 / 4096);              /* 转化为电压值 */
    temperature = (1.43 - temperature) / 0.0043 + 25;      /* 计算温度 */
    result = temperature *= 100;     /* 扩大 100 倍. */
    return result;

}
/* USER CODE END 1 */
```

ADC 的初始化函数 MX_ADC1_Init，与 ADC 单通道采集基本一致，仅改变了通道值，本实验用的是通道 16 即 ADC_CHANNEL_TEMPSENSOR。

接着是函数 adc_get_temperature，该函数先是调用前面 ADC 实验章节写好的 adc_get_result_average 函数取获取通道 ch 的转换值，然后通过温度转换公式，返回温度值。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
  short temp;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
```

```
  /* ADC1 初始化 */
  MX_ADC1_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                              /* 初始化 LCD */
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "Temperature TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 120, 200, 16, 16, "TEMPERATE: 00.00C", BLUE);
  /* USER CODE END 2 */

  while (1)
  {
      temp = adc_get_temperature();    /* 得到温度值 */

      if (temp < 0)
      {
          temp = -temp;
          /* 显示负号 */
          lcd_show_string(30 + 10 * 8, 120, 16, 16, 16, "-", BLUE);
      }
      else
      {
          lcd_show_string(30 + 10 * 8, 120, 16, 16, 16, " ", BLUE);
      }
      /* 显示整数部分 */
      lcd_show_xnum(30 + 11 * 8, 120, temp / 100, 2, 16, 0, BLUE);
       /* 显示小数部分 */
      lcd_show_xnum(30 + 14 * 8, 120, temp % 100, 2, 16, 0X80, BLUE);

      LED0_TOGGLE();  /* LED0 闪烁,提示程序运行 */
      delay_ms(250);
  }
}
```

该部分的代码逻辑很简单，先是得到温度值，再根据温度值判断正负值，来显示温度符号，再显示整数和小数部分。

39.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 39.3.1 所示：

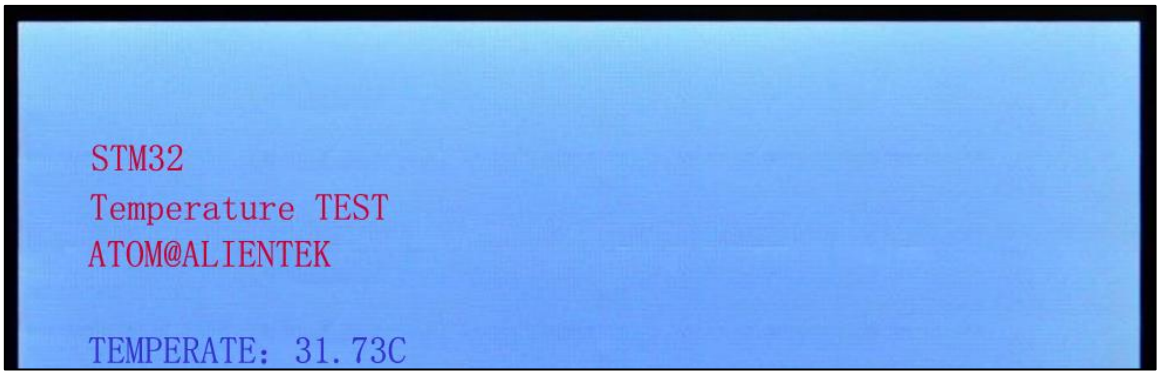


图 39.3.1 内部温度传感器实验测试图

大家可以看看你的温度值与实际是否相符合（因为芯片会发热，所以一般会比实际温度偏高）？

第四十章 光敏传感器实验

本章，我们将学习使用 STM32 开发板板载的一个光敏传感器。我们还是要使用到 ADC 采集，通过 ADC 采集电压，获取光敏传感器的电阻变化，从而得出环境光线的变化，并在 TFTLCD 上面显示出来。

本章分为如下几个小节：

40.1 硬件设计
40.2 程序设计
40.3 下载验证

40.1 硬件设计

1. 例程功能

通过 ADC3 的通道 6（PF8）读取光敏传感器（LS1）的电压值，并转换为 0~100 的光线强度值，显示在 LCD 模块上面。光线越亮，值越大；光线越暗，值越小。大家可以用手指遮挡 LS1 和用手电筒照射 LS1，来查看光强变化。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED : LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）ADC3：通道 6 - PF8
5）光敏传感器

3. 原理图

我们主要来看看光敏传感器和开发板的连接，如下图所示：

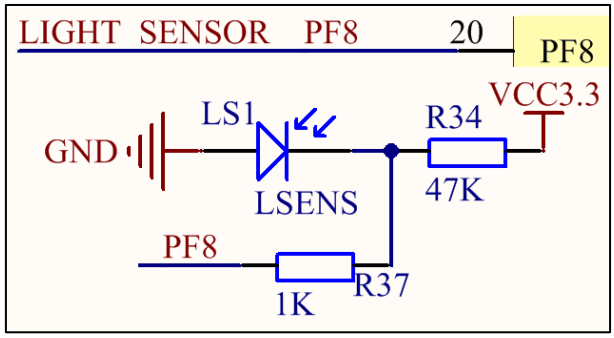


图 40.1.1 光敏传感器与开发板连接示意图

图中，LS1 是光敏二极管，外观看起像与贴片 LED 类似（位于 OLED 插座旁边，LS1），R34 为其提供反向电压，当环境光线变化时，LS1 两端的电压也会随之改变，通过 ADC3_IN6 通道读取 LIGHT_SENSOR（PF8）上面的电压，即可得到环境光线的强弱。光线越强，电压越低，光线越暗，电压越高。

40.2 程序设计

40.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料

\1、程序源码\4，CubeIDE_project\ 20_LSENS。

40.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 20_LSENS，以进行下一步的实验操作。如下图所示：

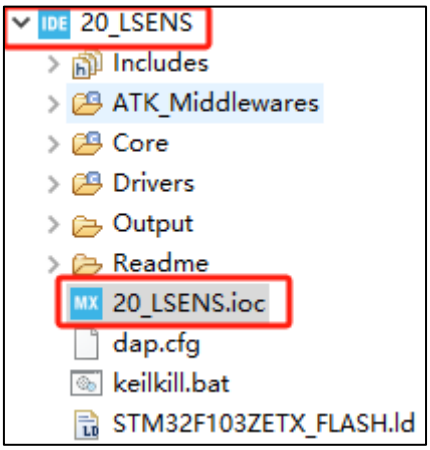


图 40.2.2.1 LSENS 工程

（1）配置 ADC3 的通道

接下来先配置 ADC3 通道的引脚，本实验我们使用 ADC3 的通道 6，配置如下图所示：

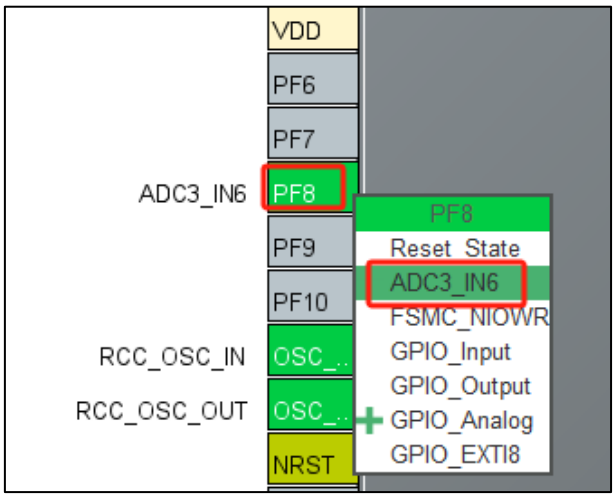


图 40.2.2.2 ADC3 通道 IO 配置

（2）配置 ADC3 的参数

在 Analog→ADC3 处配置 ADC。如下所示，本实验用到 ADC3 的通道 6，所以勾选 IN6：

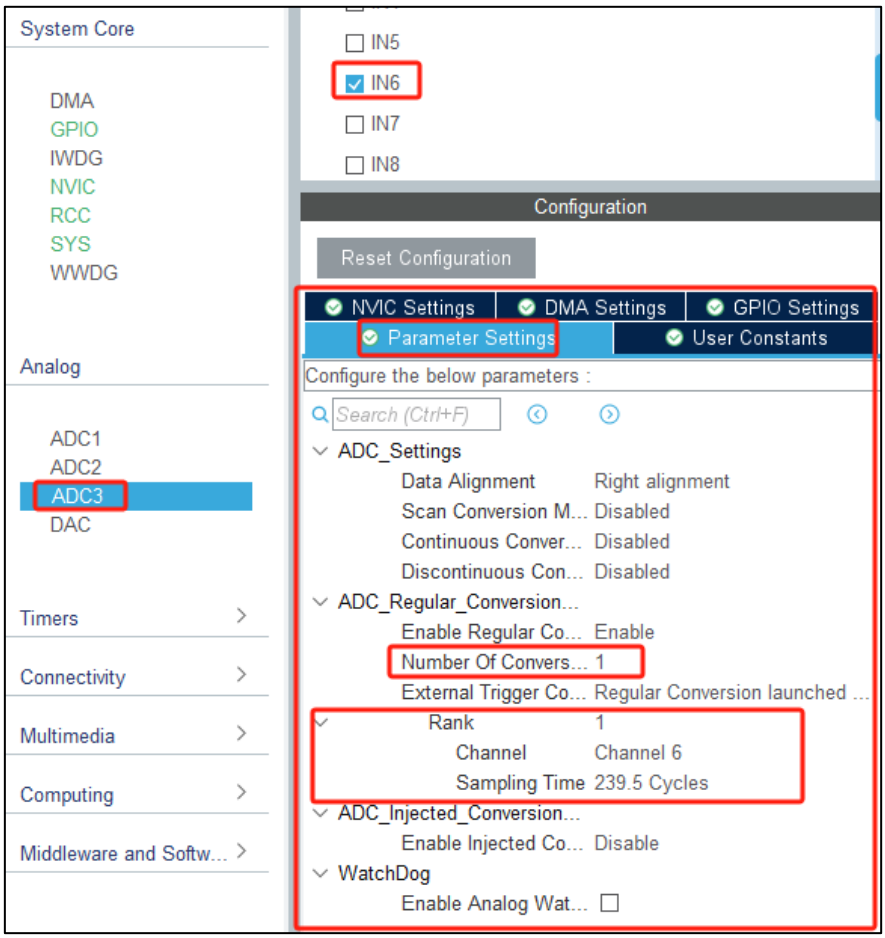


图 40.2.2.3 ADC3 参数配置

前面章节中已对这些参数进行了详细介绍（36.2.2 小节），因此在此处不再赘述。需要注意的是，由于我们只使用单个通道，所以 Scan Conversion Mode 配置为 Disabled，并且 Number Of Conversion 转换通道数量配置为 1，然后在配置过程中需要选择 Channel 6。

40.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 adc.c 文件。adc.c 主要是完成 ADC3 的初始化。

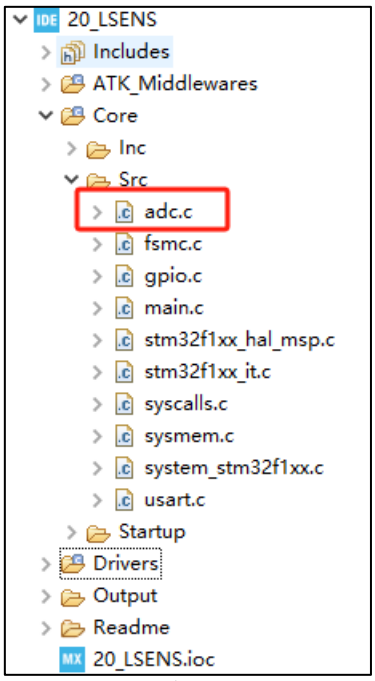


图 40.2.3.1 生成 ADC 工程

40.2.4 添加用户驱动代码

然后接着我们在 Drivers/BSP 文件夹中，新建 LSENS 文件夹，然后再 LSENS 文件夹下新建两个文件，分别是 lsens.c 和 lsens.h 文件，如下图所示：

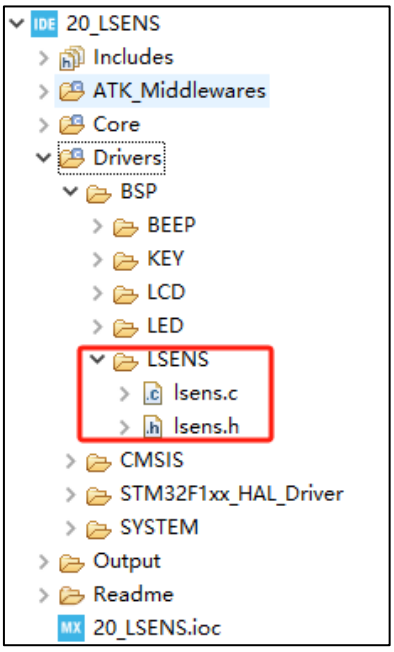


图 40.2.4.1 添加文件

我们本实验主要修改 5 个文件，分别是：adc.c、adc.h、lsens.c、lsens.h 和 main.c，其中 adc.h 仅声明 adc.c 中的一些功能函数，lsens.h 仅声明 lsens.c 中的一些功能函数，这里不过多赘述此文件。我们首先对 adc.c 文件进行修改。

（1）修改 adc.c 文件

```
void MX_ADC3_Init(void)
```

```
{
  ADC_ChannelConfTypeDef sConfig = {0};
  hadc3.Instance = ADC3;
  hadc3.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc3.Init.ContinuousConvMode = DISABLE;
  hadc3.Init.DiscontinuousConvMode = DISABLE;
  hadc3.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc3.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc3.Init.NbrOfConversion = 1;
  if (HAL_ADC_Init(&hadc3) != HAL_OK)
  {
    Error_Handler();
  }

  sConfig.Channel = ADC_CHANNEL_6;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
  if (HAL_ADC_ConfigChannel(&hadc3, &sConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN ADC3_Init 2 */
  HAL_ADCEx_Calibration_Start(&hadc3);                /* 校准 ADC */
  /* USER CODE END ADC3_Init 2 */

}

/* 省略部分代码… … */

/* USER CODE BEGIN 1 */
/**
 * @brief       设置 ADC 通道采样时间
 * @param       adcx : adc 句柄指针,ADC_HandleTypeDef
 * @param       ch   : 通道号, ADC_CHANNEL_0~ADC_CHANNEL_17
 * @param       stime: 采样时间  0~7, 对应关系为:
 *   @arg       ADC_SAMPLETIME_1CYCLE_5, 1.5 个 ADC 时钟周期
ADC_SAMPLETIME_7CYCLES_5, 7.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_13CYCLES_5, 13.5 个 ADC 时钟周期
ADC_SAMPLETIME_28CYCLES_5, 28.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_41CYCLES_5, 41.5 个 ADC 时钟周期
ADC_SAMPLETIME_55CYCLES_5, 55.5 个 ADC 时钟周期
 *   @arg       ADC_SAMPLETIME_71CYCLES_5, 71.5 个 ADC 时钟周期
ADC_SAMPLETIME_239CYCLES_5, 239.5 个 ADC 时钟周期
 * @param       rank: 多通道采集时需要设置的采集编号,
                假设你定义 channle1 的 rank=1，channle2 的 rank=2,
                那么对应你在 DMA 缓存空间的变量数组 AdcDMA[0] 就 i 是 channle1 的转换结
果，AdcDMA[1]就是通道 2 的转换结果。
                单通道 DMA 设置为 ADC_REGULAR_RANK_1
 *   @arg       编号 1~16: ADC_REGULAR_RANK_1~ADC_REGULAR_RANK_16
 * @retval      无
 */
void adc3_channel_set(ADC_HandleTypeDef *adc_handle, uint32_t ch, uint32_t
rank, uint32_t stime)
{
    ADC_ChannelConfTypeDef adc_ch_conf;

    adc_ch_conf.Channel = ch;                               /* 通道 */
    adc_ch_conf.Rank = rank;                                /* 序列 */
    adc_ch_conf.SamplingTime = stime;                       /* 采样时间 */
    HAL_ADC_ConfigChannel(adc_handle, &adc_ch_conf);        /* 通道配置 */
}

/**
```

```
 * @brief       获得 ADC 转换后的结果
 * @param       ch: 通道值 0~17，取值范围为：ADC_CHANNEL_0~ADC_CHANNEL_17
 * @retval      无
 */
uint32_t adc3_get_result(uint32_t ch)
{
    adc3_channel_set(&hadc3 , ch, ADC_REGULAR_RANK_1,
    ADC_SAMPLETIME_239CYCLES_5);                    /* 设置通道，序列和采样时间 */

    HAL_ADC_Start(&hadc3);                          /* 开启 ADC */
    HAL_ADC_PollForConversion(&hadc3, 10);          /* 轮询转换 */
    return (uint16_t)HAL_ADC_GetValue(&hadc3);      /* 返回 ADC3 规则组的转换结果 */
}

/**
 * @brief       获取通道 ch 的转换值，取 times 次,然后平均
 * @param       ch      : 通道号, 0~17
 * @param       times   : 获取次数
 * @retval      通道 ch 的 times 次转换结果平均值
 */
uint32_t adc3_get_result_average(uint32_t ch, uint8_t times)
{
    uint32_t temp_val = 0;
    uint8_t t;

    for (t = 0; t < times; t++)      /* 获取 times 次数据 */
    {
        temp_val += adc3_get_result(ch);
        delay_ms(5);
    }

    return temp_val / times;         /* 返回平均值 */
}
/* USER CODE END 1 */
```

adc.c 文件的代码和单通道 ADC 采集实验的 adc.c\h 文件的代码几乎一样，这里就不再赘述了。

（2）添加 lsens.c 文件

```
#include "adc.h"
#include "lsens.h"

/**
 * @brief       读取光敏传感器值
 * @param       无
 * @retval      0~100:0,最暗;100,最亮
 */
uint8_t lsens_get_val(void)
{
    uint32_t temp_val = 0;
    temp_val = adc3_get_result_average(LSENS_ADC3_CHX, 10);  /* 读取平均值 */
    temp_val /= 40;

    if (temp_val > 100)temp_val = 100;

    return (uint8_t)(100 - temp_val);
}
```

lsens_get_val 函数用于获取当前光照强度，该函数通过 adc3_get_result_average 函数得到通道 6 转换的电压值，经过简单量化后，处理成 0~100 的光强值。0 对应最暗，100 对应最亮。

（3）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    short adcx;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* ADC3 初始化 */
  MX_ADC3_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                          /* 初始化 LCD */
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "LSENS TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "LSENS_VAL:", BLUE);
  /* USER CODE END 2 */

  while (1)
  {
      adcx = lsens_get_val();
      lcd_show_xnum(30 + 10 * 8, 110, adcx, 3, 16, 0, BLUE); /* 显示 ADC 的值 */
      LED0_TOGGLE();                                   /* LED0 闪烁,提示程序运行 */
      delay_ms(250);
  }
}
```

该部分的代码逻辑很简单，初始化各个外设之后，进入死循环，通过 lsens_get_val 获取光敏传感器得到的光强值（0~100），并显示在 TFTLCD 上面。

40.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 40.3.1 所示：

图 40.3.1 光敏传感器实验测试图

我们可以通过给 LS1 不同的光照强度，来观察 LSENS_VAL 值的变化，光照越强，该值越大，光照越弱，该值越小，LSENS_VAL 值的范围是 0~100。

第四十一章 DAC 输出实验

本章介绍使用 STM32F103 的 DAC 输出指定的电压值。通过本章的学习。读者将学习到 DAC 的使用。

本章分为如下几个小节：

41.1 硬件设计
41.2 程序设计
41.3 下载验证

41.1 硬件设计

1. 例程功能

使用 KEY1/KEY_UP 两个按键，控制 STM32 内部 DAC 的通道 1 输出电压大小，然后通过 ADC3 的通道 1 采集 DAC 输出的电压，在 LCD 模块上面显示 ADC 采集到的电压值以及 DAC 的设定输出电压值等信息。也可以通过 usmart 调用 dac_set_voltage 函数，来直接设置 DAC 输出电压。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键：
KEY1 - PE3
WK_UP - PA0
5）ADC3：通道 1 - PA1
6）DAC1：通道 1 - PA4

3. 原理图

我们来看看原理图上 ADC3 通道 1（PA1）和 DAC1 通道 1（PA4）引出来的引脚，如下图所示：

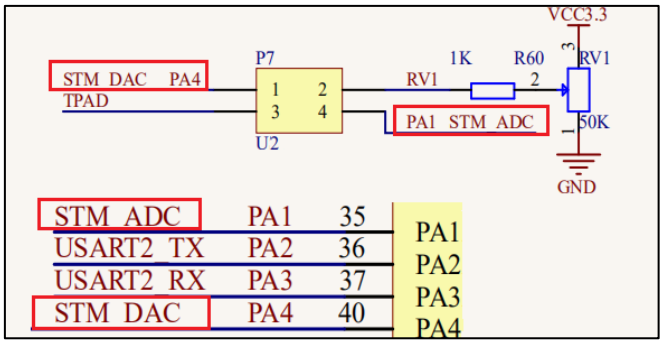


图 41.1.1 ADC 和 DAC 在开发板上的连接关系原理图

我们只需要通过杜邦线连接 ADC 和 DAC，就可以使得 ADC3 通道 1（PA1）和 DAC1 通道 1（PA4）连接起来。对应的硬件连接如图 41.1.2 所示：

图 41.1.2 硬件连接示意图

41.2 程序设计

41.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 21_1_DAC_Output。

41.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 21_1_DAC_Output，以进行下一步的实验操作。如下图所示：

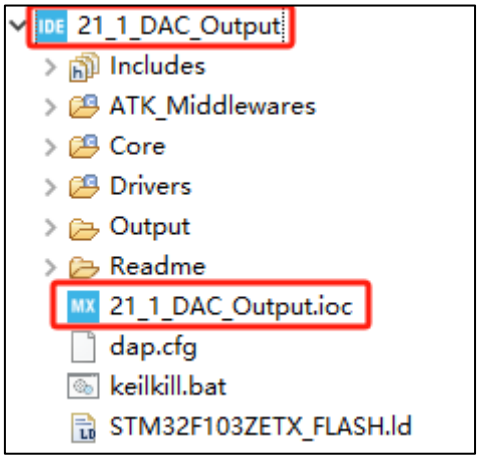


图 41.2.2.1 DAC_Output 工程

（1）配置 ADC3 的通道以及参数

双击 21_1_DAC_Output.ioc 文件，进入配置界面，先配置 ADC3 通道的引脚，本实验我们使用 ADC3 的通道 1，配置如下图所示：

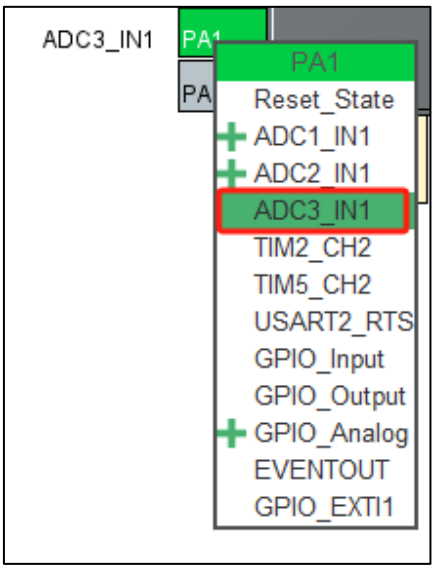


图 41.2.2.2 ADC3 通道 IO 配置

接着来看下 ADC3 的参数配置，如下图所示：

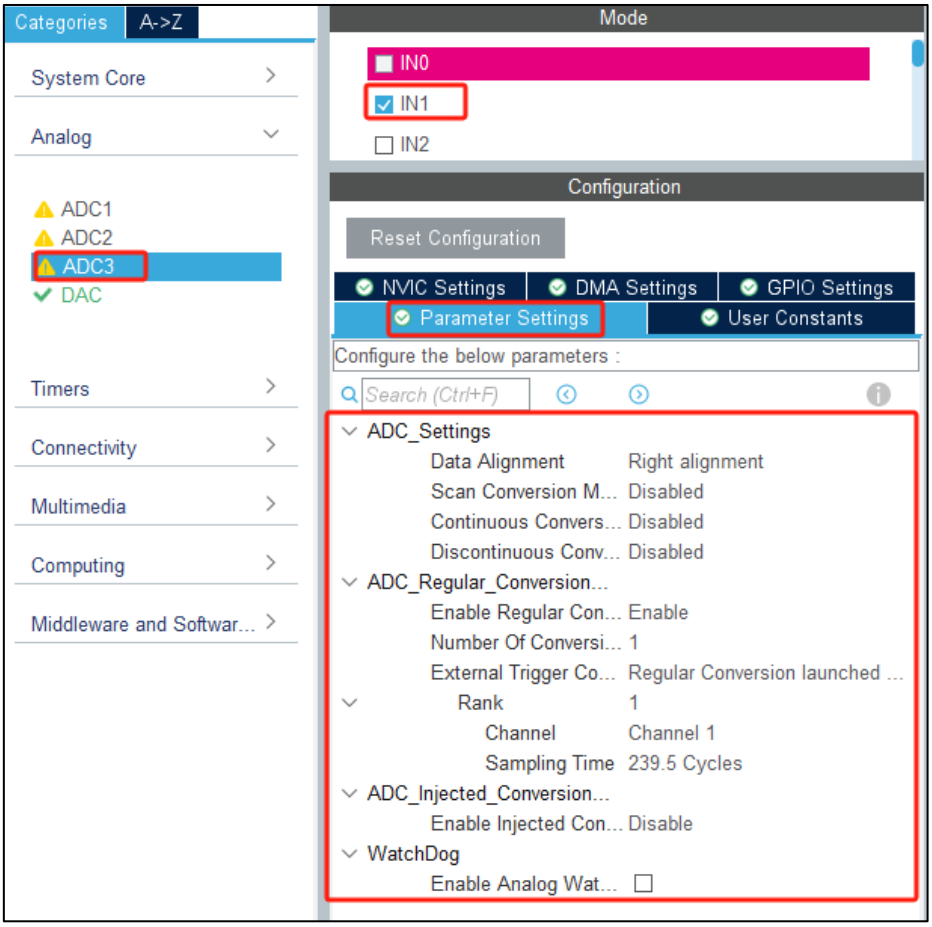


图 41.2.2.3 ADC3 参数配置

前面章节中已对这些参数进行了详细介绍（36.2.2 小节），因此在此处不再赘述。

（2）配置 DAC 的通道以及参数

本实验用到了 DAC 的输出通道 1，所以如下进行配置：

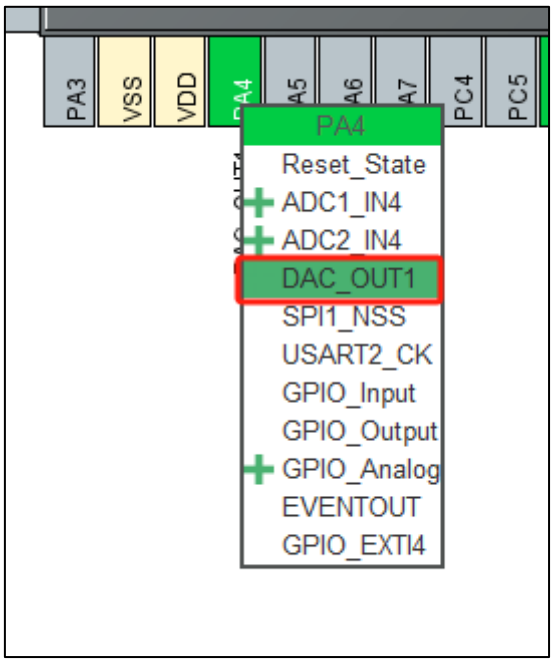


图 41.2.2.4 DAC 通道配置

接着配置 DAC 的参数，如下进行设置：

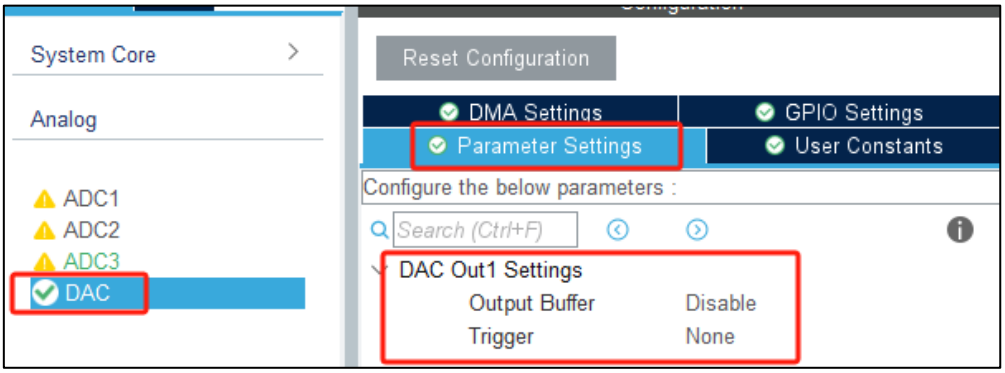


图 41.2.2.5 DAC 参数配置

DAC 参数配置部分：

- Output Buffer 配置不使能；
- Trigger 配置触发源，可以配置软件触发和硬件触发以及不触发，我们配置不触发；

41.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 adc.c、dac.c 文件。adc.c 主要是完成 ADC3 的初始化；dac.c 主要是完成 DAC 通道一的初始化。如下图：

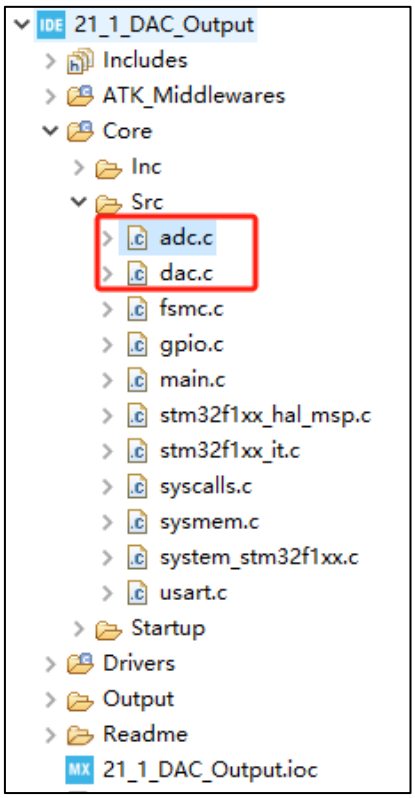


图 41.2.3.1 生成初始化工程

41.2.4 添加用户驱动代码

（1）修改 dac.c 文件

ADC 的初始化代码我们在前面章节的实验已经介绍过，这里就不再介绍了。DAC 的初始化代码在 dac.c 文件及其头文件中，如下：

```
void MX_DAC_Init(void)
{
  DAC_ChannelConfTypeDef sConfig = {0};
  hdac.Instance = DAC;
  if (HAL_DAC_Init(&hdac) != HAL_OK)
  {
    Error_Handler();
  }
  sConfig.DAC_Trigger = DAC_TRIGGER_NONE;
  sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_DISABLE;
  if (HAL_DAC_ConfigChannel(&hdac, &sConfig, DAC_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
}

/* 省略部分代码... ... */

/* USER CODE BEGIN 1 */
/**
 * @brief       设置通道 1/2 输出电压
 * @param       outx: 1,通道 1; 2,通道 2
 * @param       vol : 0~3300,代表 0~3.3V
 * @retval      无
 */
```

```
void dac_set_voltage(uint8_t outx, uint16_t vol)
{
    double temp = vol;
    temp /= 1000;
    temp = temp * 4096 / 3.3;

    if (temp >= 4096)temp = 4095;    /* 如果值大于等于 4096，则取 4095 */

    if (outx == 1)    /* 通道 1 */
    {
        /* 12 位右对齐数据格式设置 DAC 值 */
        HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, temp);
    }
    else             /* 通道 2 */
    {
        /* 12 位右对齐数据格式设置 DAC 值 */
        HAL_DAC_SetValue(&hdac, DAC_CHANNEL_2, DAC_ALIGN_12B_R, temp);
    }
}
/* USER CODE END 1 */
```

MX_DAC_Init 该函数用于初始化 DAC，参数配置在前面已经讲过，不再赘述。

dac_set_voltage 该函数实际就是将电压值转换为 DAC 输入值，形参 1 用于设置通道，形参 2 设置要输出的电压值，设置的范围：0~3300，代表 0~3.3V。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint16_t adcx;
    float temp;
    uint8_t t = 0;
    uint16_t dacval = 0;
    uint8_t key;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* ADC3 初始化 */
  MX_ADC3_Init();

  /* DAC 初始化 */
  MX_DAC_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                  /* 初始化 LCD */
  HAL_DAC_Start(&hdac,DAC_CHANNEL_1);          /* 开启 DAC 通道 1 */
```

```
    lcd_show_string(30,  50, 200, 16, 16, "STM32F103", RED);
    lcd_show_string(30,  70, 200, 16, 16, "DAC TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "WK_UP:+  KEY1:-", RED);

    lcd_show_string(30, 130, 200, 16, 16, "DAC VAL:", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "DAC VOL:0.000V", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "ADC VOL:0.000V", BLUE);
    /* USER CODE END 2 */

    while (1)
    {
        t++;
        key = key_scan(0);          /* 按键扫描 */

        if (key == WKUP_PRES)
        {
            if (dacval < 4000)dacval += 200;
            /* 输出增大 200 */
            HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, dacval);
        }
        else if (key == KEY1_PRES)
        {
            if (dacval > 200)dacval -= 200;
            else dacval = 0;
            /* 输出减少 200 */
            HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, dacval);
        }
        /* WKUP/KEY1 按下了,或者定时时间到了 */
        if (t == 10 || key == KEY1_PRES || key == WKUP_PRES)
        {
            /* 读取前面设置 DAC1_OUT1 的值 */
            adcx = HAL_DAC_GetValue(&hdac, DAC_CHANNEL_1);
            lcd_show_xnum(94, 130, adcx, 4, 16, 0, BLUE);    /* 显示 DAC 寄存器值 */

            temp = (float)adcx * (3.3 / 4096);               /* 得到 DAC 电压值 */
            adcx = temp;
            lcd_show_xnum(94, 150, temp, 1, 16, 0, BLUE);    /* 显示电压值整数部分*/

            temp -= adcx;
            temp *= 1000;
            /* 显示电压值的小数部分 */
            lcd_show_xnum(110, 150, temp, 3, 16, 0X80, BLUE);
            /* 得到 ADC3 通道 1 的转换结果 */
            adcx = adc3_get_result_average(ADC_CHANNEL_1, 20);
            temp = (float)adcx * (3.3 / 4096);
            adcx = temp;
            lcd_show_xnum(94, 170, temp, 1, 16, 0, BLUE);    /* 显示电压值整数部分*/

            temp -= adcx;
            temp *= 1000;
            lcd_show_xnum(110, 170, temp, 3, 16, 0X80, BLUE);

            LED0_TOGGLE();  /* LED0 闪烁 */
            t = 0;
        }

        delay_ms(10);
    }
}
```

此部分代码，我们通过 KEY_UP（WKUP 按键）和 KEY1（也就是上下键）来实现对 DAC 输出的幅值控制。按下 KEY_UP 增加，按 KEY1 减小。同时在 LCD 上面显示 DHR12R1 寄存器的值、DAC 设置输出电压以及 ADC 采集到的 DAC 输出电压。

41.3 下载验证

在完成编译和烧录操作后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示如图 41.3.1 所示：

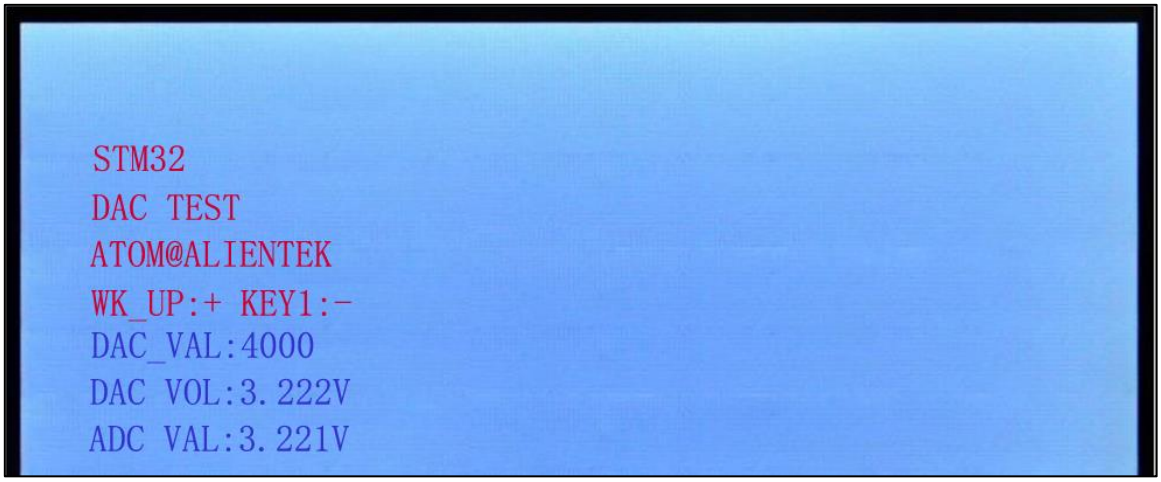


图 41.3.1 DAC 输出实验测试图

验证试验前，记得先通过跳线帽连接 ADC 和 DAC 排针，然后我们可以通过按 WK_UP 按键，增加 DAC 输出的电压，这时 ADC 采集到的电压也会增大，通过按 KEY1 减小 DAC 输出的电压，这时 ADC 采集到的电压也会减小。

第四十二章 DAC 输出三角波实验

本章将使用软件的方式控制 STM32F103 的 DAC 输出指定幅值、频率和个数的三角波。通过本章的学习，读者将学习到 DAC 的使用。

本章分为如下几个小节：

42.1 硬件设计
42.2 程序设计
42.3 下载验证

42.1 硬件设计

1. 例程功能

使用 DAC 输出三角波，通过 KEY0/KEY1 两个按键，控制 DAC1 的通道 1 输出两种三角波，需要通过示波器接 PA4 进行观察。也可以通过 usmart 调用 dac_triangular_wave 函数，来控制输出哪种三角波。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）ALIENTEK2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键：
KEY0 - PE4
KEY1 - PE3
5）DAC1：通道 1 - PA4

3. 原理图

我们只需要把示波器的探头接到 DAC1 通道 1（PA4）引脚，就可以在示波器上显示 DAC 输出的波形。PA4 在 P10 的 DAC 标志排针已经引出，硬件连接如图 42.1.1 所示：

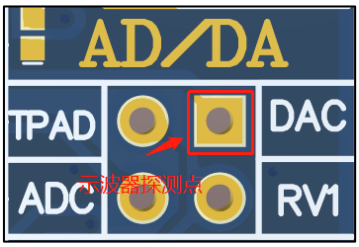


图 42.1.1 硬件连接示意图

42.2 程序设计

42.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 21_2_DAC_Triangular_Wave。

42.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修

改为 21_2_DAC_Triangular_Wave，以进行下一步的实验操作。如下图所示：

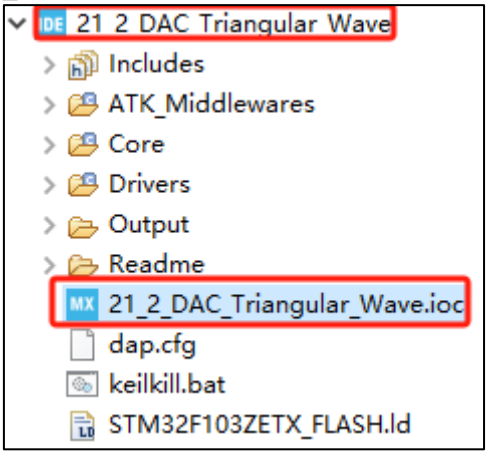


图 42.2.2.1 DAC_Triangular_Wave 工程

（1）配置 DAC 的通道以及参数

本实验 DAC 工程的配置和上一章节的一样，都用到了 DAC 的输出通道 1，所以如下进行配置：

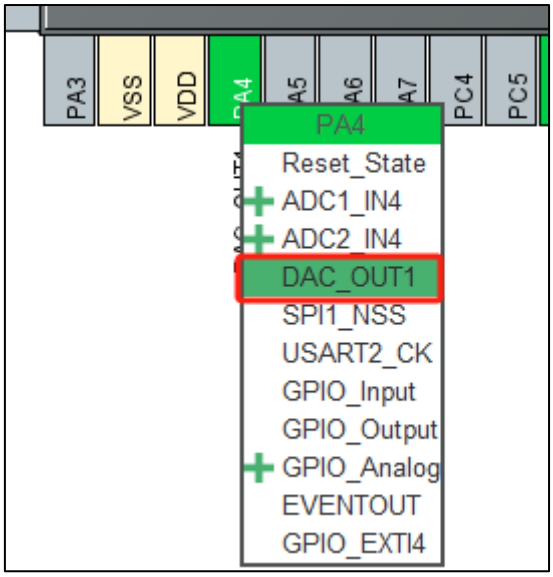


图 42.2.2.2 DAC 通道配置

接着配置 DAC 的参数，如下进行设置：

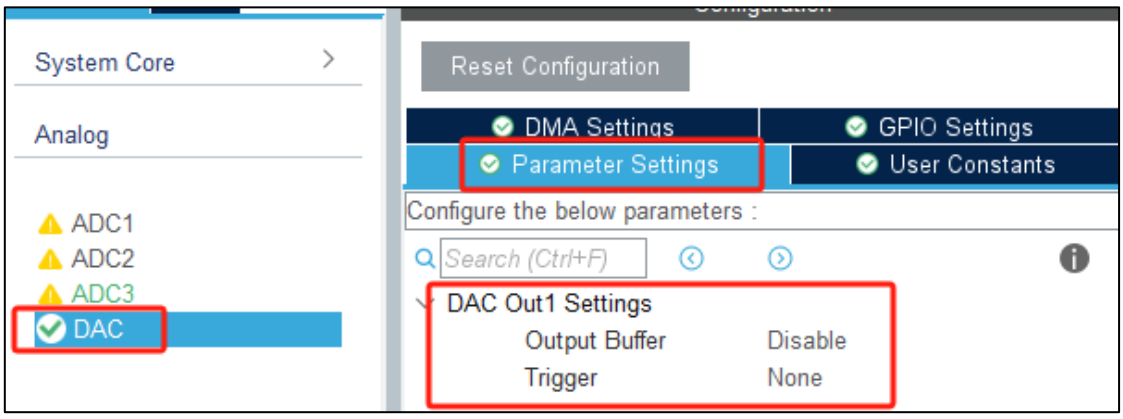


图 42.2.2.3 DAC 参数配置

DAC 参数配置部分：

- Output Buffer 配置不使能；
- Trigger 配置触发源，可以配置软件触发和硬件触发以及不触发，我们配置不触发；

42.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，可以看到在工程中多了一个 dac.c 文件。dac.c 主要是完成 DAC 通道一的初始化。如下图：

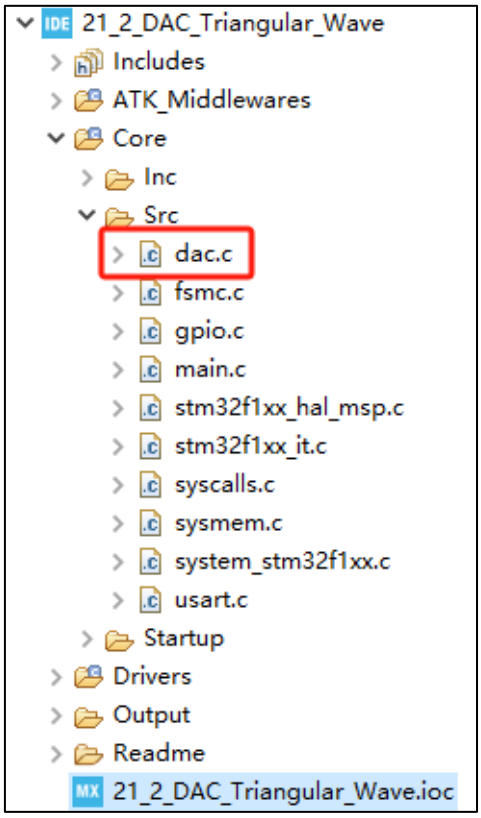


图 42.2.3.1 生成初始化工程

42.2.4 添加用户驱动代码

（1）添加三角波生成函数

如下，我们在 dac.c 文件中添加如下代码：

```
/**************************DAC 输出三角波实验代码**************************/

/**
 * @brief       设置 DAC_OUT1 输出三角波
 *   @note      输出频率 ≈ 1000 / (dt * samples) Khz, 不过在 dt 较小的时候,比如小于
    5us 时, 由于 delay_us
 *              本身就不准了(调用函数,计算等都需要时间,延时很小的时候,这些时间会影响到延
    时), 频率会偏小.
 *
 * @param       maxval : 最大值(0 < maxval < 4096), (maxval + 1)必须大于等于
   samples/2
 * @param       dt     : 每个采样点的延时时间(单位: us)
 * @param       samples: 采样点的个数, samples 必须小于等于(maxval + 1) * 2 , 且
   maxval 不能等于 0
 * @param       n      : 输出波形个数,0~65535
 *
 * @retval      无
```

```
 */
void dac_triangular_wave(uint16_t maxval, uint16_t dt, uint16_t samples,
uint16_t n)
{
    uint16_t i, j;
    float incval;                                   /* 递增量 */
    float Curval;                                   /* 当前值 */

    if((maxval + 1) <= samples)return ;             /* 数据不合法 */

    incval = (maxval + 1) / (samples / 2);          /* 计算递增量 */

    for(j = 0; j < n; j++)
    {
        Curval = 0;
        /* 先输出 0 */
        HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, Curval);
        for(i = 0; i < (samples / 2); i++)          /* 输出上升沿 */
        {
            Curval  +=  incval;                     /* 新的输出值 */
            HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, Curval);
            delay_us(dt);
        }
        for(i = 0; i < (samples / 2); i++)          /* 输出下降沿 */
        {
            Curval  -=  incval;                     /* 新的输出值 */
            HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, Curval);
            delay_us(dt);
        }
    }
}
```

该函数用于设置 DAC 通道 1 输出三角波，输出频率 ≈ 1000 / (dt * samples) Khz，形参含义在源码已经有详细注释。该函数中，我们使用 HAL_DAC_SetValue 函数来设置 DAC 的输出值，这样得到的三角波在示波器上可以看到。如果有跳动现象（不平稳），是正常的，因为调用函数，计算等都需要时间，这样就会导致输出的波形是不太稳定的。越高性能的 MCU，得到的波形会越稳定。而且用 HAL 库函数操作效率没有直接操作寄存器高，所以可以像寄存器版本实验一样，直接操作 DHR12R1 寄存器，得到的波形会相对稳定些。

（2）修改 main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    uint8_t key;
  /* USER CODE END 1 */

    HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
```

```
    MX_USART1_UART_Init();

    /* DAC 初始化 */
    MX_DAC_Init();

    /* USER CODE BEGIN 2 */
    lcd_init();                                             /* 初始化 LCD */
    HAL_DAC_Start(&hdac, DAC_CHANNEL_1);

    lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30,  70, 200, 16, 16, "DAC Triangular WAVE TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Wave1  KEY1:Wave2", RED);
    lcd_show_string(30, 130, 200, 16, 16, "DAC None", BLUE); /* 提示无输出 */
    /* USER CODE END 2 */

    while (1)
    {
        t++;
        key = key_scan(0);                                  /* 按键扫描 */

        if (key == KEY0_PRES)                               /* 高采样率 , 约 1Khz 波形 */
        {
            lcd_show_string(30, 130, 200, 16, 16, "DAC Wave1 ", BLUE);
            /* 幅值 4095, 采样点间隔 5us, 200 个采样点, 100 个波形 */
            dac_triangular_wave(4095, 5, 2000, 100);
            lcd_show_string(30, 130, 200, 16, 16, "DAC None  ", BLUE);
        }
        else if (key == KEY1_PRES)                          /* 低采样率 , 约 1Khz 波形 */
        {
            lcd_show_string(30, 130, 200, 16, 16, "DAC Wave2 ", BLUE);
            /* 幅值 4095, 采样点间隔 500us, 20 个采样点, 100 个波形 */
            dac_triangular_wave(4095, 500, 20, 100);
            lcd_show_string(30, 130, 200, 16, 16, "DAC None  ", BLUE);
        }

        if (t == 10)                                        /* 定时时间到了 */
        {
            LED0_TOGGLE();                                  /* LED0 闪烁 */
            t = 0;
        }

        delay_ms(10);
    }
}
```

该部分代码功能是，按下 KEY0 后，DAC 输出三角波 1，按下 KEY1 后，DAC 输出三角波 2，将 dac_triangular_wave 的形参代入公式：输出频率 ≈ 1000 / (dt * samples) KHz，得到三角波 1 和三角波 2 的频率都是 0.1KHz。

42.3 下载验证

下载代码后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示如图 42.3.1 所示：

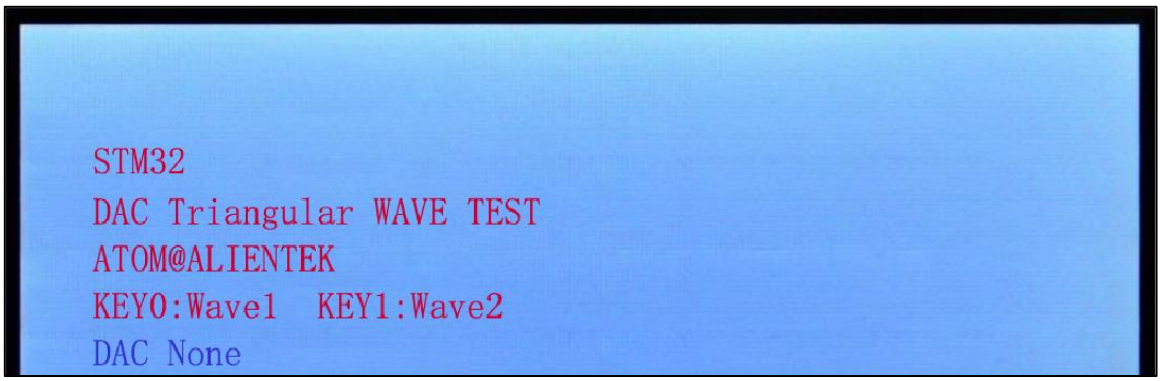


图 42.3.1 DAC 输出三角波实验测试图

没有按下任何按键之前，LCD屏显示DAC None，当按下KEY0后，DAC输出三角波1，LCD 屏显示 DAC Wave1 ，三角波 1 输出完成后 LCD 屏继续显示 DAC None，当按下 KEY1 后，DAC 输出三角波 2，LCD 屏显示 DAC Wave2，三角波 2 输出完成后 LCD 屏继续显示 DAC None。

其中三角波 1 和三角波 2 在示波器的显示情况如下图所示：

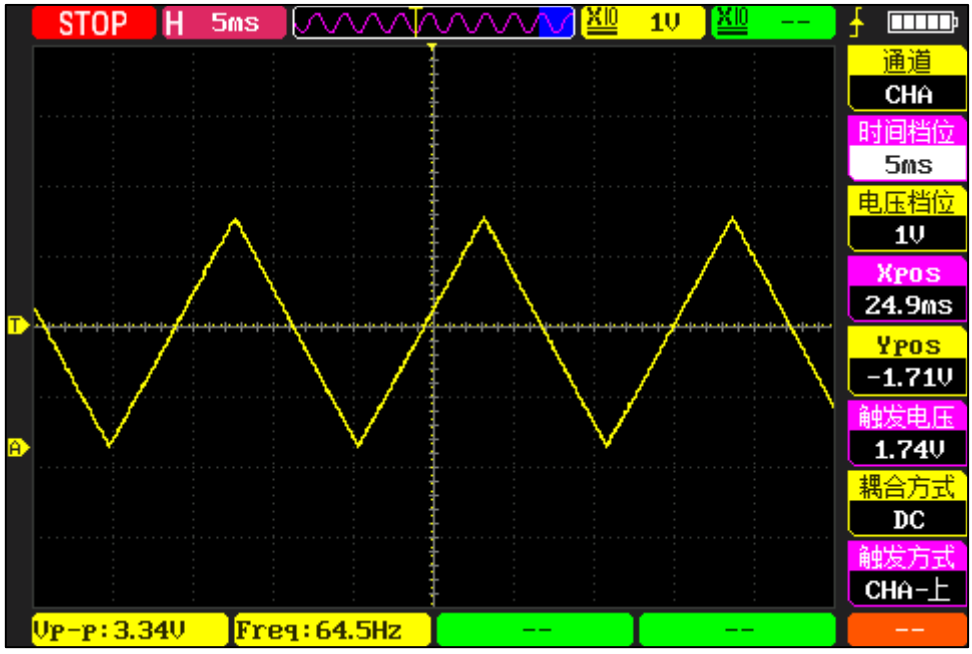


图 42.3.2 DAC 输出的三角波 1

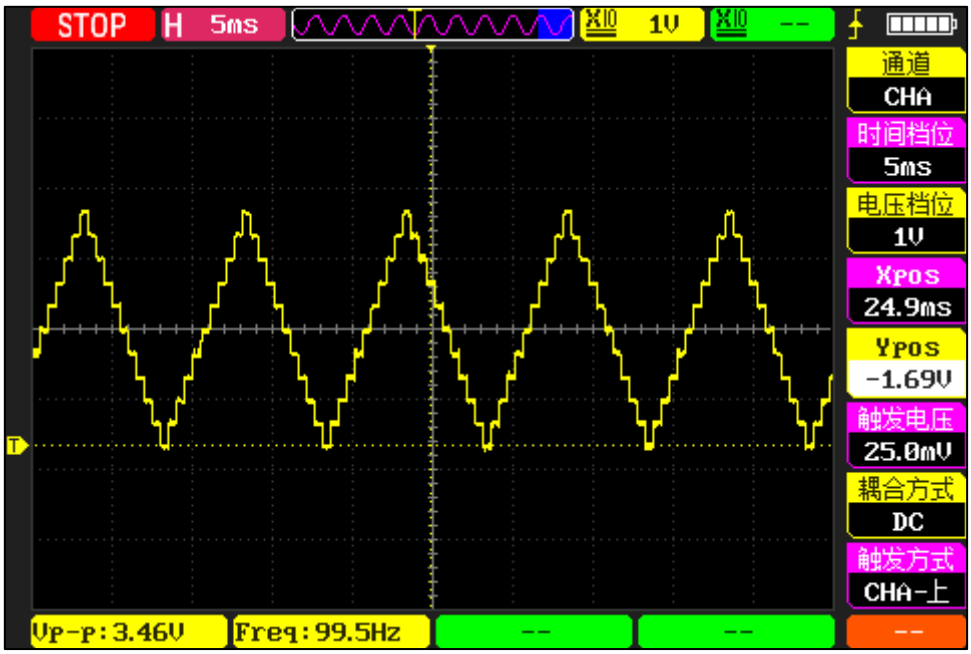


图 42.3.3 DAC 输出的三角波 2

由上面两副测试图可以知道，三角波 1 的频率是 64.5Hz，三角波 2 的频率是 99.5Hz。三角波 2 基本接近我们算出来的结果 0.1KHz，三角波 1 有较大误差，在介绍 dac_triangular_wave 函数时也说了原因，加上三角波 1 的采样率比较高，所以误差就会比较大。

第四十三章 DAC 输出正弦波实验

本章将使用定时器 7 来触发 DAC 进行转换输出正弦波，以 DMA 传输数据的方式。通过本章的学习，读者将学习到 DAC、TIM7、DMA 的使用。

本章分为如下几个小节：

43.1 硬件设计
43.2 程序设计
43.3 下载验证

43.1 硬件设计

1. 例程功能

使用 DAC 输出正弦波，通过 KEY0/KEY1 两个按键，控制 DAC1 的通道 1 输出两种正弦波，需要通过示波器接 PA4 进行观察。TFTLCD 显示 DAC 转换值、电压值和 ADC 的电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键：
KEY0 - PE4
KEY1 - PE3
5）ADC1：通道 1 - PA1
6）DAC1：通道 1 - PA4
7）DMA(DMA2_Channel13)
8）定时器 7

3. 原理图

我们只需要把示波器的探头接到 DAC1 通道 1（PA4）引脚，就可以在示波器上显示 DAC 输出的波形。PA4 在 P7 已经引出，硬件连接如图 43.1.1 所示：

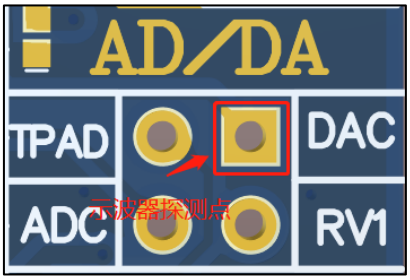


图 43.1.1 硬件连接示意图

43.2 程序设计

43.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 21_3_DAC_SineWave。

43.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 21_3_DAC_SineWave，以进行下一步的实验操作。如下图所示：

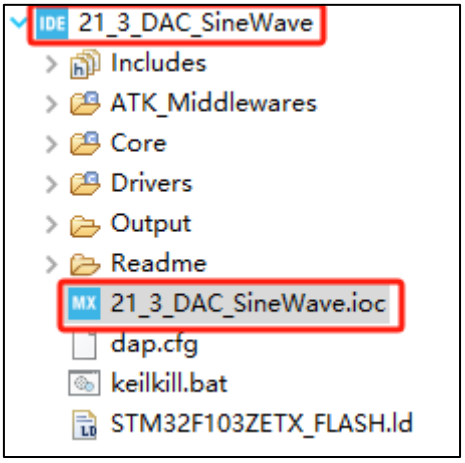


图 43.2.2.1 DAC_Triangular_Wave 工程

（1）配置 DAC

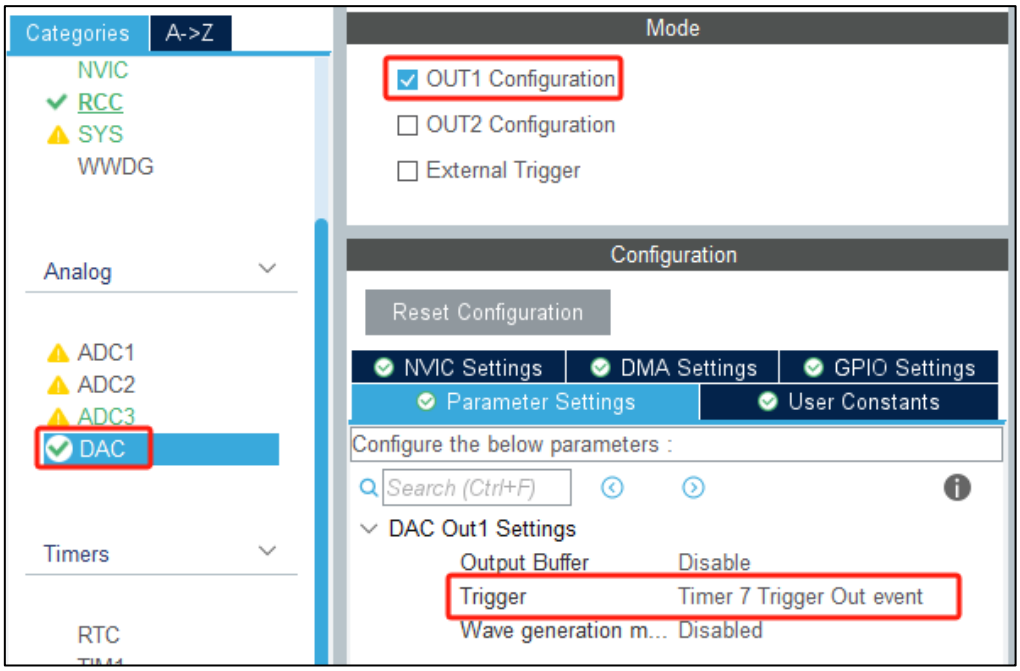


图 43.2.2.2 DAC 参数配置

以上配置中，注意 Trigger 选项我们配置为使用片上定时器的内部信号来触发 DAC 通道，这里选择定时器 7。接着配置 DMA。如下：

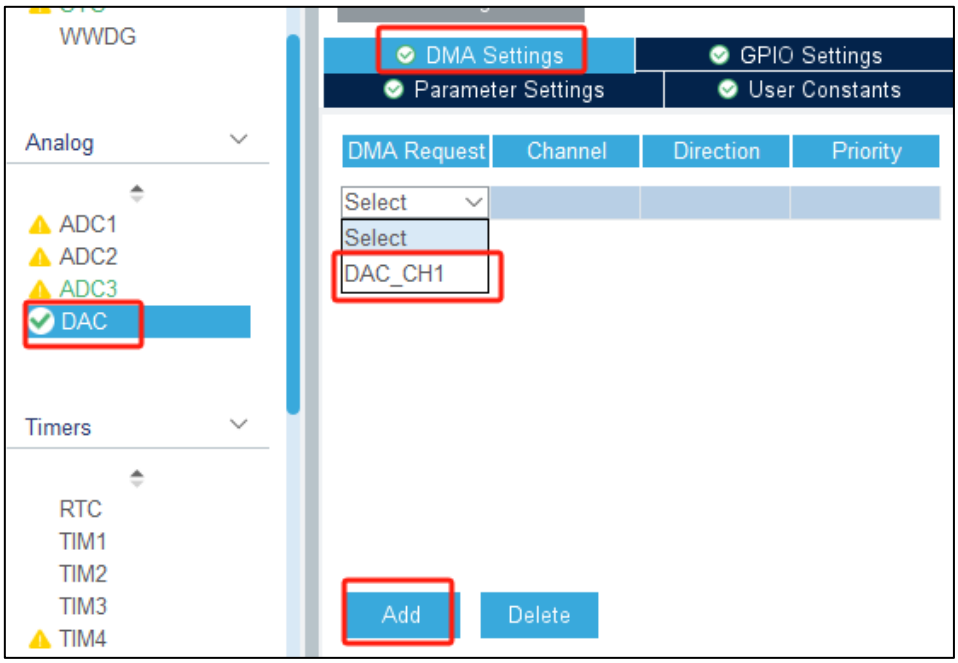


图 43.2.2.3 DMA 选择

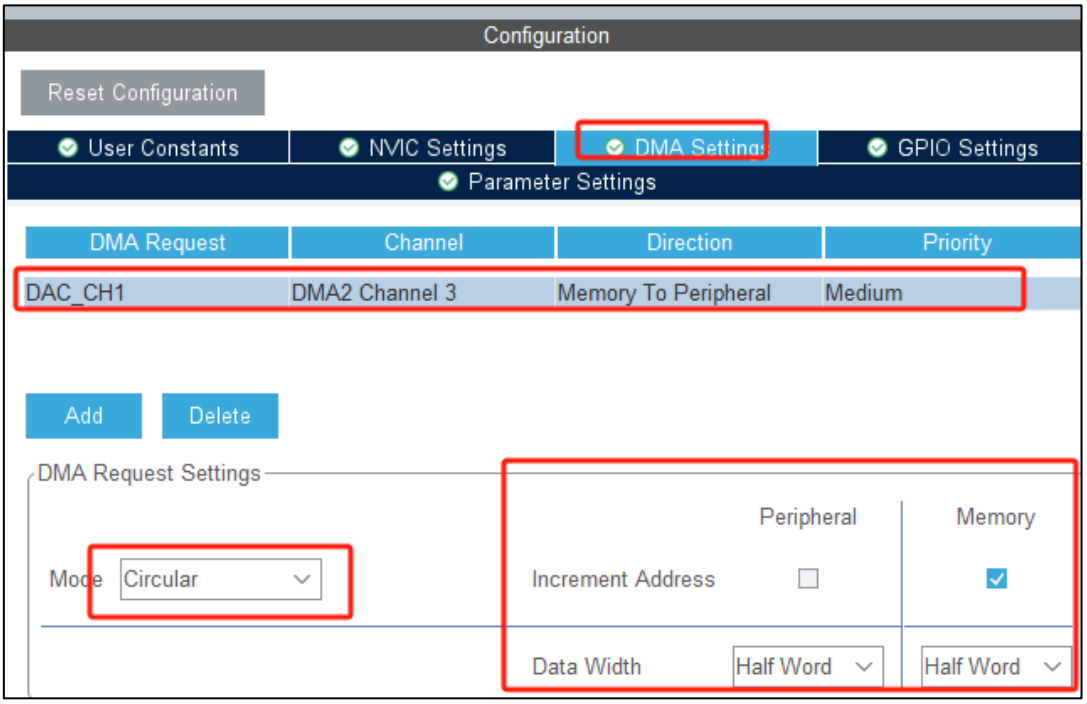


图 43.2.2.4 DMA 参数设置

（2）配置 ADC3

前面章节中关于 ADC3 的配置参数已经做了详细介绍因此在此处不再赘述，我们直接来看我们的配置结果，如下图所示：

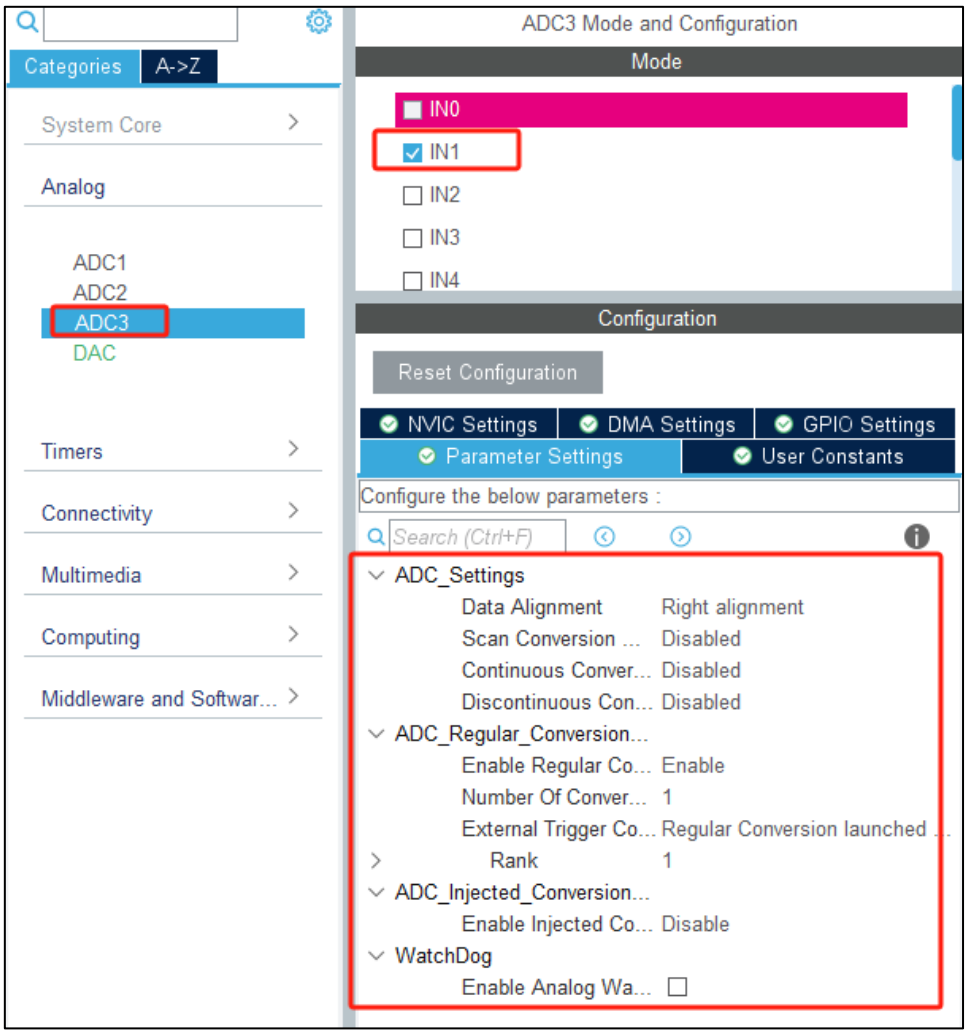


图 43.2.2.5 ADC 参数配置

（3）配置 TIM7

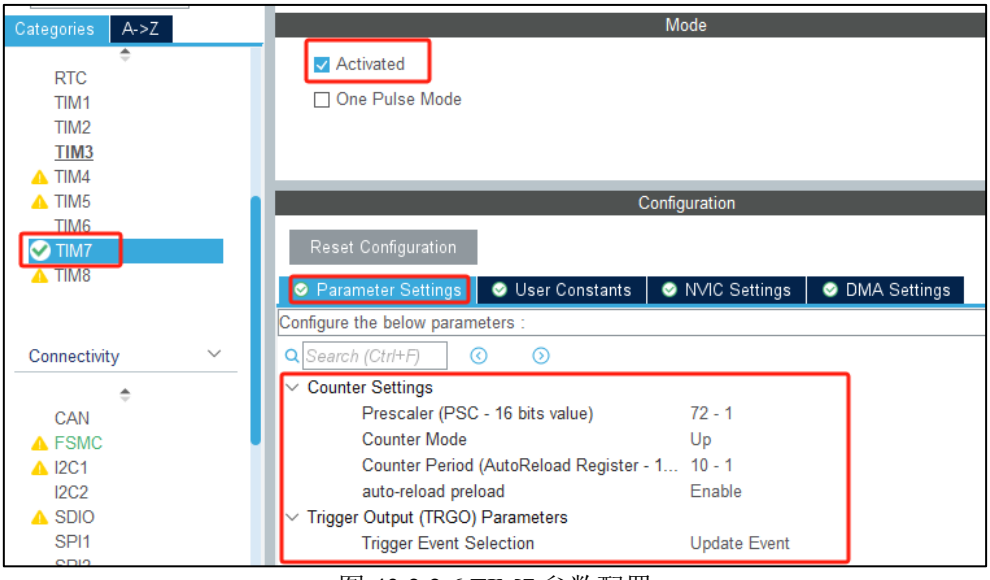


图 43.2.2.6 TIM7 参数配置

以上配置中，TIM7 的分频值为 72-1，向上计数模式，计数周期（TIMx_ARR 的值）为

1，自动重载使能，触发事件为上溢更新事件。也就是计数器每计数一次就触发 DAC 通道，那么触发信号的频率怎么计算呢？还记得我们前面学习基本定时器时，基本定时器的溢出时间计算方法：

$$T_{out} = (ARR[15:0] + 1) * \frac{1}{f_{CK_CNT}} = \frac{(ARR[15:0] + 1) * (PSC[15:0] + 1)}{f_{CK_PSC}}$$

这里，f_{CK_PSC}为 72MHz，$ARR[15:0]$=1，$PSC[15:0]$=72-1，所以：

$$f_{out} == \frac{f_{CK_PSC}}{(ARR[15:0]+1)*(PSC[15:0]+1)} = \frac{72MHz}{2*72} = 500KHz$$

也就是说，由定时器触发的触发信号频率为 500KHz。

43.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程。如下图：

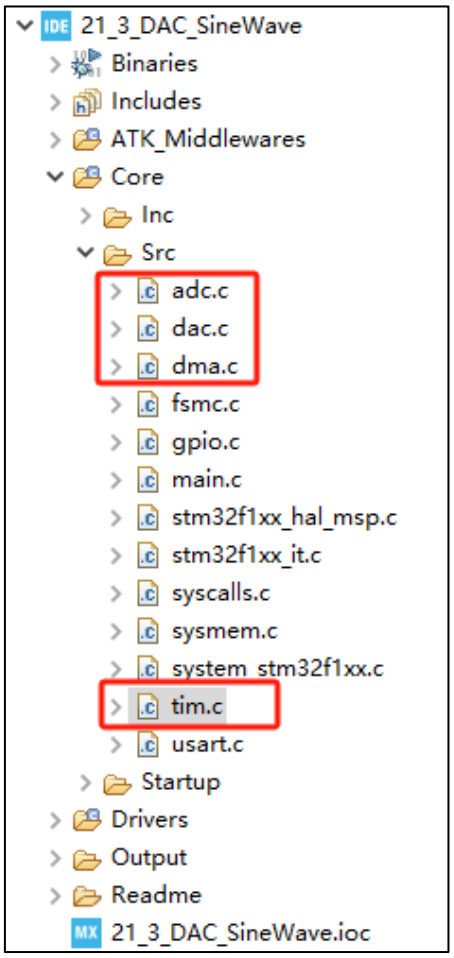


图 43.2.3.1 生成初始化工程

43.2.4 添加用户驱动代码

其中 adc.c 和 tim.c 文件在前面的章节实验，已经做了介绍，添加内容基本一致，感兴趣可打开源码自行查阅。接下来我们添加本实验主要的一些逻辑代码。

（1）添加产生正弦波序列函数

在 dac.c 文件中添加如下代码，dac_creat_sin_buf 函数用于产生正弦波需要的序列。

```
uint16_t g_dac_sin_buf[4096];            /* 发送数据缓冲区 */

/*********************DAC 输出三角波实验代码************************/
```

```
/**
 * @brief       产生正弦波函序列
 *   @note      需保证: maxval > samples/2
 *
 * @param       maxval : 最大值(0 < maxval < 2048)
 * @param       samples: 采样点的个数
 *
 * @retval      无
 */
void dac_creat_sin_buf(uint16_t maxval, uint16_t samples)
{
    uint8_t i;
    float inc = (2 * 3.1415962) / samples; /* 计算增量（一个周期 DAC_SIN_BUF 个）*/
    float outdata = 0;

    for (i = 0; i < samples; i++)
    {
        /* 计算以 dots 个点为周期的每个点的值，放大 maxval 倍，并偏移到正数区域 */
        outdata = maxval * (1 + sin(inc * i));
        if (outdata > 4095)
            outdata = 4095; /* 上限限定 */
        g_dac_sin_buf[i] = outdata;
    }
}

/**
 * @brief       DAC DMA 使能波形输出
 *   @note      TIM7 的输入时钟频率(f)来自 APB1, f = 36M * 2 = 72Mhz.
 *              DAC 触发频率 ftrgo = f / ((psc + 1) * (arr + 1))
 *              波形频率 = ftrgo / ndtr;
 *
 * @param       ndtr        : DMA 通道单次传输数据量
 * @param       arr         : TIM7 的自动重装载值
 * @param       psc         : TIM7 的分频系数
 * @retval      无
 */
void dac_dma_wave_enable(uint16_t cndtr, uint16_t arr, uint16_t psc)
{
    __HAL_TIM_SET_PRESCALER(&htim7, psc);
    __HAL_TIM_SET_AUTORELOAD(&htim7, arr);
    HAL_TIM_Base_Start(&htim7);                     /* 启动定时器 7 */

    HAL_DAC_Stop_DMA(&hdac, DAC_CHANNEL_1);         /* 先停止之前的传输 */
    HAL_DAC_Start_DMA(&hdac, DAC_CHANNEL_1, (uint32_t *)g_dac_sin_buf, cndtr,
    DAC_ALIGN_12B_R);
}
```

dac_creat_sin_buf 函数用于产生正弦波序列，并保存在 g_dac_sin_buf 数组中，供给 DAC 转换。

dac_dma_wave_enable 函数用于使能波形输出，利用定时器 7 的更新事件来触发 DAC 转换输出。启动定时器 7，然后调用 HAL_DAC_Stop_DMA 函数停止 DAC 转换以及 DMA 传输，最后再调用 HAL_DAC_Start_DMA 函数重新配置并启动 DAC 和 DMA。

（2）main.c 文件实现代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint16_t adcx;
    float temp;
    uint8_t t = 0;
    uint8_t key;
```

```
  /* USER CODE END 1 */

    HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* DMA 初始化 */
  MX_DMA_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* DAC 初始化 */
  MX_DAC_Init();

  /* ADC3 初始化 */
  MX_ADC3_Init();

  /* TIM7 初始化 */
  MX_TIM7_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                 /* 初始化 LCD */
  adc3_channel_set(&hadc3, ADC_CHANNEL_1, ADC_REGULAR_RANK_1,
  ADC_SAMPLETIME_1CYCLE_5);
  HAL_DMA_Start(&hdma_dac_ch1, (uint32_t)&DAC1->DHR12R1,
  (uint32_t)g_dac_sin_buf, 0);                  /* 配置 DMA 传输参数 */

  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "DAC DMA Sine WAVE TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:3Khz  KEY1:30Khz", RED);

  lcd_show_string(30, 130, 200, 16, 16, "DAC VAL:", BLUE);
  lcd_show_string(30, 150, 200, 16, 16, "DAC VOL:0.000V", BLUE);
  lcd_show_string(30, 170, 200, 16, 16, "ADC VOL:0.000V", BLUE);
  dac_creat_sin_buf(2048, 100);
  /* 100Khz 触发频率, 100 个点, 得到 1Khz 的正弦波 */
  dac_dma_wave_enable(100, 10 - 1, 72 - 1);

  /* USER CODE END 2 */

  while (1)
  {
      t++;
      key = key_scan(0);                                /* 按键扫描 */

      if (key == KEY0_PRES)                             /* 高采样率 , 1Khz 波形 */
      {
          dac_creat_sin_buf(2048, 100);
          /* 300Khz 触发频率, 100 个点, 得到最高 3KHz 的正弦波. */
          dac_dma_wave_enable(100, 10 - 1, 24 - 1);
      }
      else if (key == KEY1_PRES)                        /* 低采样率 , 1Khz 波形 */
```

```
        {
            dac_creat_sin_buf(2048, 10);
            /* 300Khz 触发频率, 10 个点, 可以得到最高 30KHz 的正弦波. */
            dac_dma_wave_enable(10, 10 - 1, 24 - 1);
        }

        adcx = DAC1->DHR12R1;                                   /* 获取 DAC1_OUT1 的状态*/
        lcd_show_xnum(94, 130, adcx, 4, 16, 0, BLUE);    /* 显示 DAC 寄存器值 */

        temp = (float)adcx * (3.3 / 4096);                      /* 得到 DAC 电压值 */
        adcx = temp;
        lcd_show_xnum(94, 150, temp, 1, 16, 0, BLUE);    /* 显示电压值整数部分 */

        temp -= adcx;
        temp *= 1000;
        lcd_show_xnum(110, 150, temp, 3, 16, 0X80, BLUE); /* 显示电压值的小数部分*/

        adcx = adc3_get_result_average(ADC_CHANNEL_1, 10);
        temp = (float)adcx * (3.3 / 4096);
        adcx = temp;
        lcd_show_xnum(94, 170, temp, 1, 16, 0, BLUE);

        temp -= adcx;
        temp *= 1000;
        lcd_show_xnum(110, 170, temp, 3, 16, 0X80, BLUE);

        if (t == 40)         /* 定时时间到了 */
        {
            LED0_TOGGLE();   /* LED0 闪烁 */
            t = 0;
        }

        delay_ms(5);
    }
}
```

可以看到，在实验应用代码中，定义了函数 dac_creat_sin_buf()，该函数用于生成正弦波数据，并保存至数组 g_dac_sin_buf 中。在完成 DAC 初始化后，便生成了一组正弦波数据，并调用函数 dac_dma_wave_enable()使能 DAC 通道 1 输出指定的正弦波，随后便通过扫描到的不同按键值，输出不同频率的正弦波。同时也还使能了 ADC 便于观察 DAC 输出的电压，DAC 输出电压的模拟量和数字量以及 ADC 采集到电压的模拟量都将被实时地在 LCD 上进行显示。

43.3 下载验证

下载代码后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示如图 43.3.1 所示：

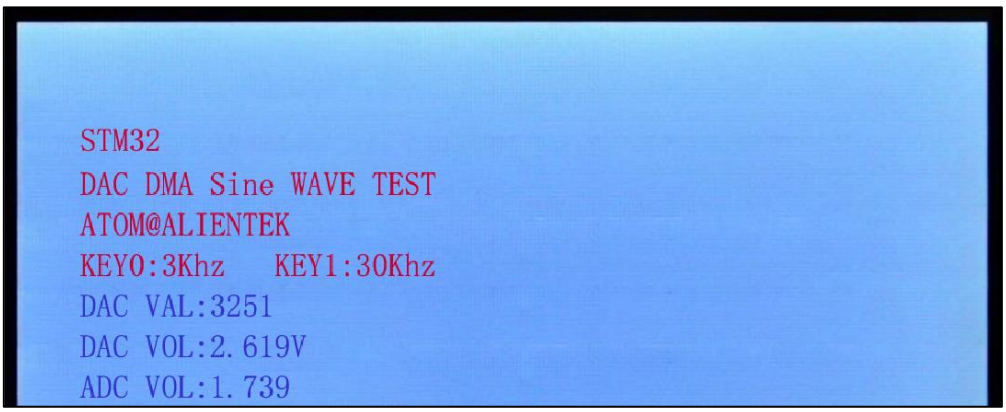


图 43.3.1 DAC 输出正弦波实验测试图

用短路帽将 ADC 和 DAC 排针连接后，可以看到 ADC VOL 的值随着 DAC 的输出变化而变化，即 ADC 采集到的值是不停变化的。由于变化太快了，这样看不出采集到值形成什么波形，下面我们借用示波器来进行观察，首先将探头接到 DAC 的排针上。

没有按下任何按键之前，默认输出 1KHz（100 个采样点）的正弦波，如下图所示：

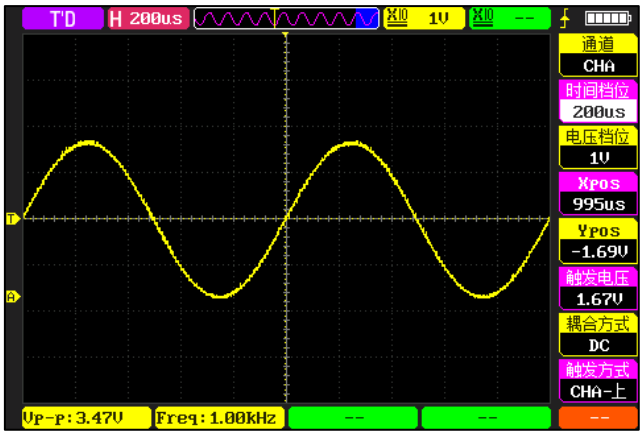


图 43.3.2 默认 DAC 输出的的正弦波

当按下 KEY0 后，DAC 输出 3KHz（100 个采样点）的正弦波，如下图所示：

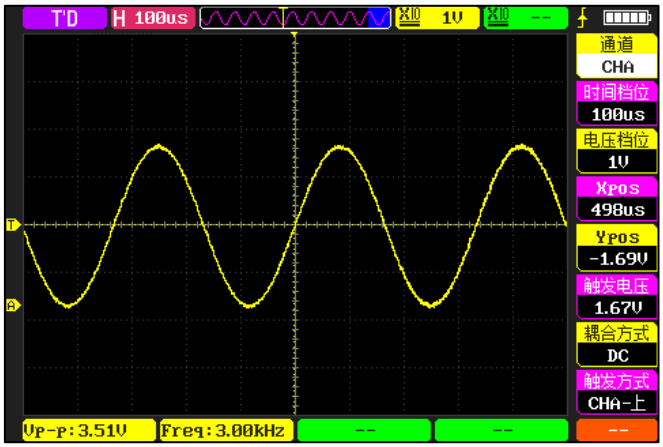


图 43.3.3 按下 KEY0，DAC 输出的的正弦波

当按下 KEY1 后，DAC 输出 30KHz（10 个采样点）的正弦波，如下图所示：

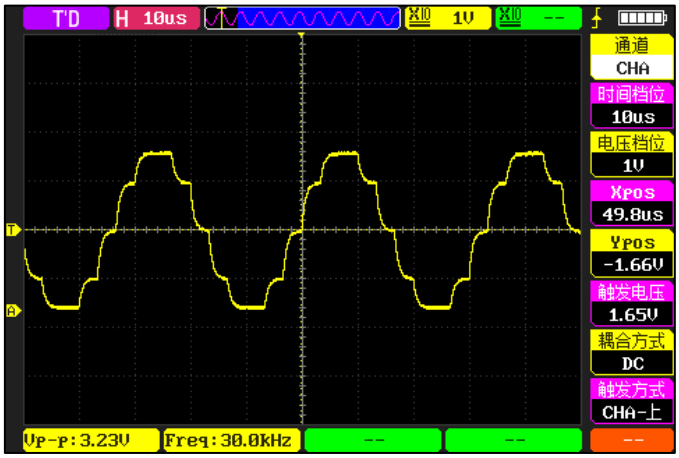


图 43.3.4 按下 KEY1，DAC 输出的的正弦波

第四十四章 IIC 实验

本章将介绍使用 STM32F103 驱动板载的 EEPROM 进行读写操作。由于 F1 的硬件 IIC 有 BUG（为了绕开飞利浦的专利），所以本章将使用软件 IIC 来驱动 EEPROM。通过本章的学习，读者将学习到使用 GPIO 模拟 IIC 时序以及 EEPROM 的驱动。

本章分为如下几个小节：

44.1 硬件设计
44.2 程序设计
44.3 下载验证

44.1 硬件设计

1. 例程功能

每按下 KEY1，MCU 通过 IIC 总线向 24C02 写入数据，通过按下 KEY0 来控制 24C02 读取数据。同时在 LCD 上面显示相关信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
KEY0 - PE4
KEY1 - PE3
3）EEPROM AT24C02
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)(USMART 使用)

3. 原理图

我们主要来看看 24C02 和开发板的连接，如下图所示：

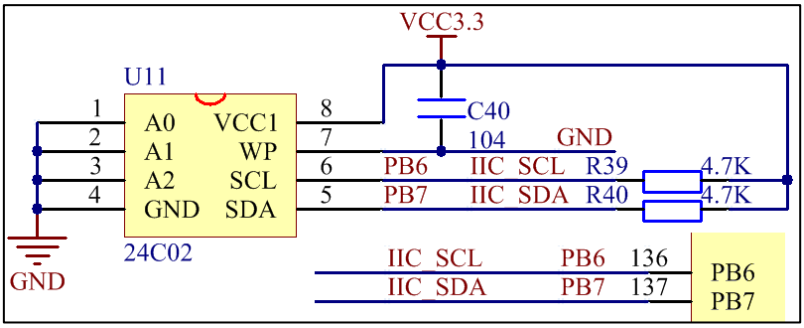


图 44.1.1 24C02 与开发板连接示意图

24C02 的 SCL 和 SDA 分别连接在 STM32 的 PB6 和 PB7 上。本实验通过软件模拟 IIC 信号建立起与 24C02 的通信，进行数据发送与接收，使用按键 KEY0 和 KEY1 去触发，LCD 屏幕进行显示。

44.2 程序设计

44.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料

\1、程序源码\4，CubeIDE_project\ 22_IIC。

44.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 22_IIC，以进行下一步的实验操作。如下图所示：

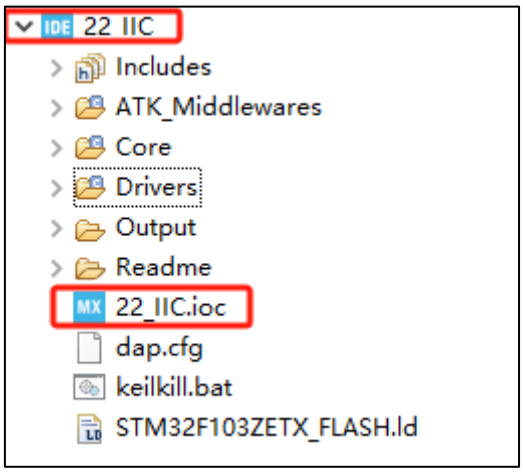


图 44.2.2.1 IIC 工程

（1）GPIO 配置

接着双击 22_IIC.ioc 进入 STM32CubeMX 插件配置界面后，在 Pinout & Configuration 处我们配置模拟 I2C 要用的两个 IO 口：PB6 和 PB7:，如下图所示：

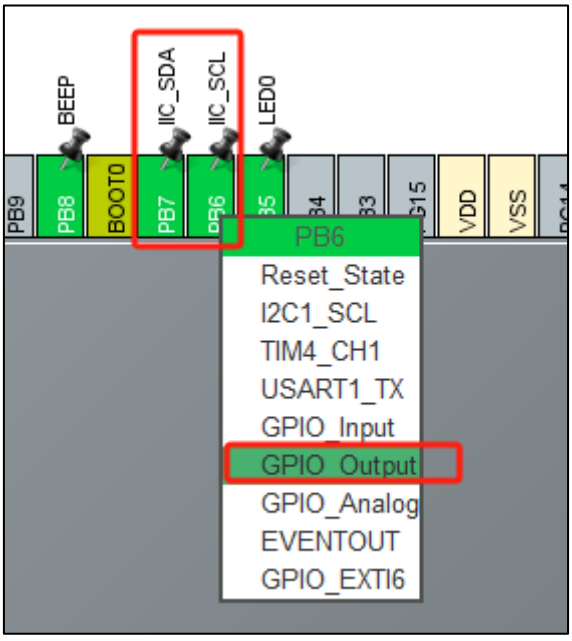
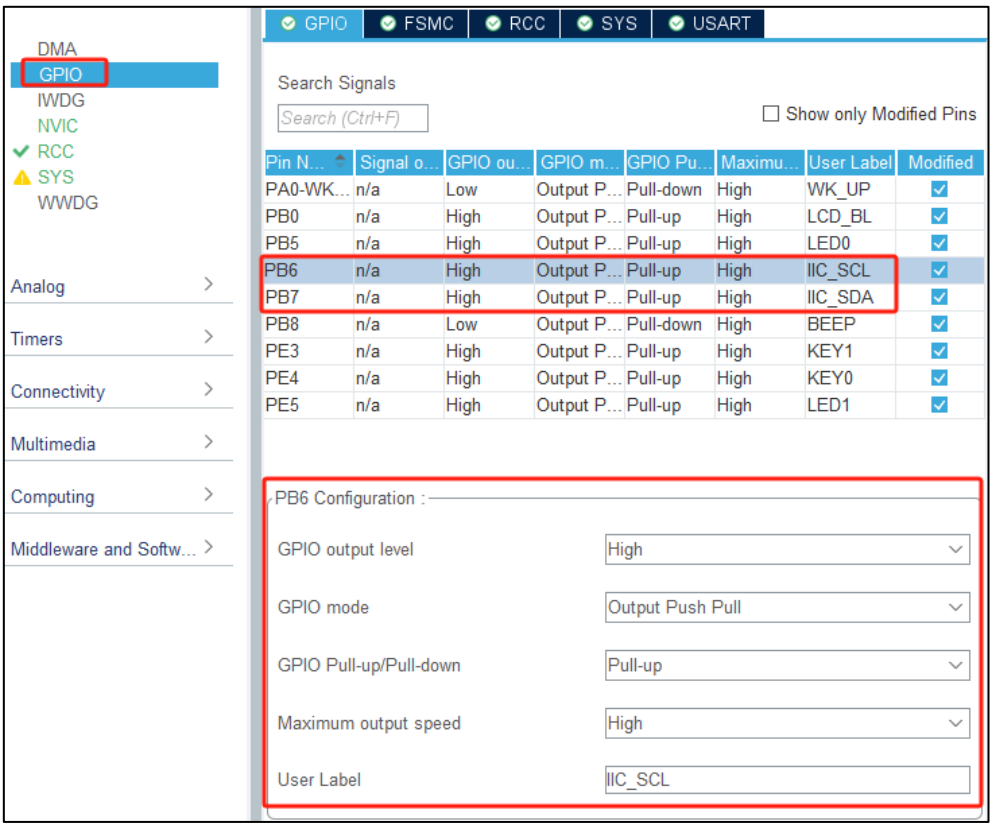


图 44.2.2.2 GPIO 引脚配置输出模式

再将 PB6、PB7 配置为推挽输出、上拉、高速模式；User Label 则分别配置为：IIC_SCL、IIC_SDA。如下图：



44.2.2.3 配置 GPIO 引脚模式

44.2.3 添加用户驱动代码

按下“Ctrl+S”保存配置，生成工程。在 Drivers/BSP 文件夹中，新建 IIC 和 24CXX 文件夹，然后再 IIC 文件夹下新建两个文件，分别是 myiic.c 和 myiic.h；在 24CXX 文件夹下，新建两个文件，分别是：24cxx.c 和 24cxx.h。如下图所示：

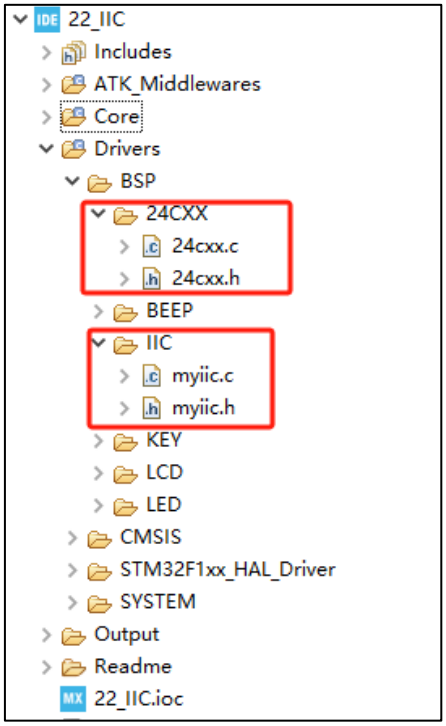


图 44.2.3.1 添加用户文件

下面我们先在 myiic.h 中添加 GPIO 口模拟 I2C 的驱动代码：

（1）myiic.h 文件代码

```
#ifndef __MYIIC_H
#define __MYIIC_H

#include "../../SYSTEM/sys/sys.h"

/*****************************************************************************/

/* IO 操作 */
#define IIC_SCL(x)         do{ x ? \
                               HAL_GPIO_WritePin(IIC_SCL_GPIO_Port,
IIC_SCL_Pin, GPIO_PIN_SET) : \
                               HAL_GPIO_WritePin(IIC_SCL_GPIO_Port,
IIC_SCL_Pin, GPIO_PIN_RESET); \
                           }while(0)       /* SCL */

#define IIC_SDA(x)         do{ x ? \
                               HAL_GPIO_WritePin(IIC_SDA_GPIO_Port,
IIC_SDA_Pin, GPIO_PIN_SET) : \
                               HAL_GPIO_WritePin(IIC_SDA_GPIO_Port,
IIC_SDA_Pin, GPIO_PIN_RESET); \
                           }while(0)       /* SDA */
/* 读取 SDA */
#define IIC_READ_SDA     HAL_GPIO_ReadPin(IIC_SDA_GPIO_Port, IIC_SDA_Pin)


/* IIC 所有操作函数 */
void iic_init(void);                    /* 初始化 IIC 的 IO 口 */
void iic_start(void);                   /* 发送 IIC 开始信号 */
void iic_stop(void);                    /* 发送 IIC 停止信号 */
void iic_ack(void);                     /* IIC 发送 ACK 信号 */
void iic_nack(void);                    /* IIC 不发送 ACK 信号 */
uint8_t iic_wait_ack(void);             /* IIC 等待 ACK 信号 */
void iic_send_byte(uint8_t txd);        /* IIC 发送一个字节 */
uint8_t iic_read_byte(unsigned char ack);  /* IIC 读取一个字节 */

#endif
```

我们通过宏定义标识符的方式去定义 SCL 和 SDA 两个引脚，同时通过宏定义的方式定义了 IIC_SCL()和 IIC_SDA()设置这两个管脚可以输出 0 或者 1，主要还是通过 HAL 库的 GPIO 操作函数实现的。另外方便在 iic 操作函数中调用读取 SDA 管脚的数据，这里直接宏定义 IIC_READ_SDA 实现，在后面 iic 模拟信号实现中会频繁调用。

（2）myiic.c 文件代码

```
/**
 * @brief       初始化 IIC
 * @param       无
 * @retval      无
 */
void iic_init(void)
{
    iic_stop();     /* 停止总线上所有设备 */
}

/**
 * @brief       IIC 延时函数,用于控制 IIC 读写速度
 * @param       无
 * @retval      无
 */
```

```
static void iic_delay(void)
{
    delay_us(2);     /* 2us 的延时, 读写速度在 250Khz 以内 */
}

/**
 * @brief       产生 IIC 起始信号
 * @param       无
 * @retval      无
 */
void iic_start(void)
{
    IIC_SDA(1);
    IIC_SCL(1);
    iic_delay();
    IIC_SDA(0);      /* START 信号: 当 SCL 为高时, SDA 从高变成低, 表示起始信号 */
    iic_delay();
    IIC_SCL(0);      /* 钳住 I2C 总线, 准备发送或接收数据 */
    iic_delay();
}

/**
 * @brief       产生 IIC 停止信号
 * @param       无
 * @retval      无
 */
void iic_stop(void)
{
    IIC_SDA(0);      /* STOP 信号: 当 SCL 为高时, SDA 从低变成高, 表示停止信号 */
    iic_delay();
    IIC_SCL(1);
    iic_delay();
    IIC_SDA(1);      /* 发送 I2C 总线结束信号 */
    iic_delay();
}

/**
 * @brief       等待应答信号到来
 * @param       无
 * @retval      1, 接收应答失败
 *              0, 接收应答成功
 */
uint8_t iic_wait_ack(void)
{
    uint8_t waittime = 0;
    uint8_t rack = 0;

    IIC_SDA(1);      /* 主机释放 SDA 线(此时外部器件可以拉低 SDA 线) */
    iic_delay();
    IIC_SCL(1);      /* SCL=1, 此时从机可以返回 ACK */
    iic_delay();

    while (IIC_READ_SDA)     /* 等待应答 */
    {
        waittime++;

        if (waittime > 250)
        {
            iic_stop();
            rack = 1;
            break;
        }
    }
```

```
    IIC_SCL(0);     /* SCL=0, 结束 ACK 检查 */
    iic_delay();
    return rack;
}

/**
 * @brief       产生 ACK 应答
 * @param       无
 * @retval      无
 */
void iic_ack(void)
{
    IIC_SDA(0);     /* SCL 0 -> 1  时 SDA = 0,表示应答 */
    iic_delay();
    IIC_SCL(1);     /* 产生一个时钟 */
    iic_delay();
    IIC_SCL(0);
    iic_delay();
    IIC_SDA(1);     /* 主机释放 SDA 线 */
    iic_delay();
}

/**
 * @brief       不产生 ACK 应答
 * @param       无
 * @retval      无
 */
void iic_nack(void)
{
    IIC_SDA(1);     /* SCL 0 -> 1  时 SDA = 1,表示不应答 */
    iic_delay();
    IIC_SCL(1);     /* 产生一个时钟 */
    iic_delay();
    IIC_SCL(0);
    iic_delay();
}

/**
 * @brief       IIC 发送一个字节
 * @param       data: 要发送的数据
 * @retval      无
 */
void iic_send_byte(uint8_t data)
{
    uint8_t t;

    for (t = 0; t < 8; t++)
    {
        IIC_SDA((data & 0x80) >> 7);    /* 高位先发送 */
        iic_delay();
        IIC_SCL(1);
        iic_delay();
        IIC_SCL(0);
        data <<= 1;     /* 左移 1 位,用于下一次发送 */
    }
    IIC_SDA(1);         /* 发送完成, 主机释放 SDA 线 */
}

/**
 * @brief       IIC 读取一个字节
 * @param       ack:  ack=1 时, 发送 ack; ack=0 时, 发送 nack
 * @retval      接收到的数据
 */
```

```
uint8_t iic_read_byte(uint8_t ack)
{
    uint8_t i, receive = 0;

    for (i = 0; i < 8; i++ )    /* 接收 1 个字节数据 */
    {
        receive <<= 1;          /* 高位先输出,所以先收到的数据位要左移 */
        IIC_SCL(1);
        iic_delay();

        if (IIC_READ_SDA)
        {
            receive++;
        }

        IIC_SCL(0);
        iic_delay();
    }

    if (!ack)
    {
        iic_nack();             /* 发送 nACK */
    }
    else
    {
        iic_ack();              /* 发送 ACK */
    }

    return receive;
}
```

IIC 驱动中对 IIC 的各种操作，例如产生 IIC 起始信号、产生 IIC 停止信号等，请读者结合 IIC 的时序规定查看本实验的配套实验源码。

（3）24cxx.h 文件代码

```
#ifndef __24CXX_H
#define __24CXX_H

#include "../../SYSTEM/sys/sys.h"


#define AT24C01     127
#define AT24C02     255
#define AT24C04     511
#define AT24C08     1023
#define AT24C16     2047
#define AT24C32     4095
#define AT24C64     8191
#define AT24C128    16383
#define AT24C256    32767

/* 开发板使用的是 24c02，所以定义 EE_TYPE 为 AT24C02 */

#define EE_TYPE     AT24C02

void at24cxx_init(void);                                    /* 初始化 IIC */
uint8_t at24cxx_check(void);                                /* 检查器件 */
uint8_t at24cxx_read_one_byte(uint16_t addr);               /* 指定地址读取一个字节*/
void at24cxx_write_one_byte(uint16_t addr,uint8_t data);/* 指定地址写入一个字节*/
/* 从指定地址开始写入指定长度的数据 */
void at24cxx_write(uint16_t addr, uint8_t *pbuf, uint16_t datalen);
/* 从指定地址开始读出指定长度的数据 */
void at24cxx_read(uint16_t addr, uint8_t *pbuf, uint16_t datalen);
#endif
```

为了使代码功能更加健全，在 24cxx.h 中宏定义了不同容量大小的 24C 系列型号。

（4）24cxx.c 文件代码

```
/**
 * @brief       初始化 IIC 接口
 * @param       无
 * @retval      无
 */
void at24cxx_init(void)
{
    iic_init();
}

/**
 * @brief       在 AT24CXX 指定地址读出一个数据
 * @param       readaddr: 开始读数的地址
 * @retval      读到的数据
 */
uint8_t at24cxx_read_one_byte(uint16_t addr)
{
    uint8_t temp = 0;
    iic_start();                    /* 发送起始信号 */

    /* 根据不同的 24CXX 型号, 发送高位地址
     * 1, 24C16 以上的型号, 分 2 个字节发送地址
     * 2, 24C16 及以下的型号, 分 1 个低字节地址 + 占用器件地址的 bit1~bit3 位 用于表示高位
     地址, 最多 11 位地址
     *    对于 24C01/02, 其器件地址格式(8bit)为: 1   0   1   0   A2   A1   A0   R/W
     *    对于 24C04,    其器件地址格式(8bit)为: 1   0   1   0   A2   A1   a8   R/W
     *    对于 24C08,    其器件地址格式(8bit)为: 1   0   1   0   A2   a9   a8   R/W
     *    对于 24C16,    其器件地址格式(8bit)为: 1   0   1   0   a10  a9   a8   R/W
     *    R/W       : 读/写控制位 0,表示写; 1,表示读;
     *    A0/A1/A2  : 对应器件的 1,2,3 引脚(只有 24C01/02/04/8 有这些脚)
     *    a8/a9/a10: 对应存储整列的高位地址, 11bit 地址最多可以表示 2048 个位置,可以寻址
     24C16 及以内的型号
     */
    if (EE_TYPE > AT24C16)          /* 24C16 以上的型号, 分 2 个字节发送地址 */
    {
        iic_send_byte(0XA0);        /* 发送写命令, IIC 规定最低位是 0, 表示写入 */
        iic_wait_ack();             /* 每次发送完一个字节,都要等待 ACK */
        iic_send_byte(addr >> 8);/* 发送高字节地址 */
    }
    else
    {
        /* 发送器件 0XA0 + 高位 a8/a9/a10 地址,写数据 */
        iic_send_byte(0XA0 + ((addr >> 8) << 1));
    }

    iic_wait_ack();                 /* 每次发送完一个字节,都要等待 ACK */
    iic_send_byte(addr % 256);      /* 发送低位地址 */
    iic_wait_ack();                 /* 等待 ACK, 此时地址发送完成了 */

    iic_start();                    /* 重新发送起始信号 */
    iic_send_byte(0XA1);            /* 进入接收模式, IIC 规定最低位是 0, 表示读取 */
    iic_wait_ack();                 /* 每次发送完一个字节,都要等待 ACK */
    temp = iic_read_byte(0);        /* 接收一个字节数据 */
    iic_stop();                     /* 产生一个停止条件 */
    return temp;
}
```

```
/**
 * @brief       在 AT24CXX 指定地址写入一个数据
 * @param       addr: 写入数据的目的地址
 * @param       data: 要写入的数据
 * @retval      无
 */
void at24cxx_write_one_byte(uint16_t addr, uint8_t data)
{
    /* 原理说明见:at24cxx_read_one_byte 函数, 本函数完全类似 */
    iic_start();                    /* 发送起始信号 */

    if (EE_TYPE > AT24C16)          /* 24C16 以上的型号, 分 2 个字节发送地址 */
    {
        iic_send_byte(0XA0);        /* 发送写命令, IIC 规定最低位是 0, 表示写入 */
        iic_wait_ack();             /* 每次发送完一个字节,都要等待 ACK */
        iic_send_byte(addr >> 8);   /* 发送高字节地址 */
    }
    else
    {
        /* 发送器件 0XA0 + 高位 a8/a9/a10 地址,写数据 */
        iic_send_byte(0XA0 + ((addr >> 8) << 1));
    }

    iic_wait_ack();                 /* 每次发送完一个字节,都要等待 ACK */
    iic_send_byte(addr % 256);      /* 发送低位地址 */
    iic_wait_ack();                 /* 等待 ACK, 此时地址发送完成了 */

    /* 因为写数据的时候,不需要进入接收模式了,所以这里不用重新发送起始信号了 */
    iic_send_byte(data);            /* 发送 1 字节 */
    iic_wait_ack();                 /* 等待 ACK */
    iic_stop();                     /* 产生一个停止条件 */
    delay_ms(10);                   /* 注意: EEPROM 写入慢,须等 10ms 后再写下一个字节 */
}

/**
 * @brief       检查 AT24CXX 是否正常
 *   @note      检测原理: 在器件的末地址写如 0X55, 然后再读取, 如果读取值为 0X55
 *              则表示检测正常. 否则,则表示检测失败.
 *
 * @param       无
 * @retval      检测结果
 *              0: 检测成功
 *              1: 检测失败
 */
uint8_t at24cxx_check(void)
{
    uint8_t temp;
    uint16_t addr = EE_TYPE;
    temp = at24cxx_read_one_byte(addr);         /* 避免每次开机都写 AT24CXX */

    if (temp == 0X55)                           /* 读取数据正常 */
    {
        return 0;
    }
    else                                        /* 排除第一次初始化的情况 */
    {
        at24cxx_write_one_byte(addr, 0X55);     /* 先写入数据 */
        temp = at24cxx_read_one_byte(255);      /* 再读取数据 */

        if (temp == 0X55)return 0;
    }
```

```
    return 1;
}

/**
 * @brief       在 AT24CXX 里面的指定地址开始读出指定个数的数据
 * @param       addr    : 开始读出的地址 对 24c02 为 0~255
 * @param       pbuf    : 数据数组首地址
 * @param       datalen : 要读出数据的个数
 * @retval      无
 */
void at24cxx_read(uint16_t addr, uint8_t *pbuf, uint16_t datalen)
{
    while (datalen--)
    {
        *pbuf++ = at24cxx_read_one_byte(addr++);
    }
}

/**
 * @brief       在 AT24CXX 里面的指定地址开始写入指定个数的数据
 * @param       addr    : 开始写入的地址 对 24c02 为 0~255
 * @param       pbuf    : 数据数组首地址
 * @param       datalen : 要写入数据的个数
 * @retval      无
 */
void at24cxx_write(uint16_t addr, uint8_t *pbuf, uint16_t datalen)
{
    while (datalen--)
    {
        at24cxx_write_one_byte(addr, *pbuf);
        addr++;
        pbuf++;
    }
}
```

24C02 驱动中对 EEPROM 的各种操作函数，例如：EEPROM 的读写函数，请读者结合 24C02 EEPROM 芯片的数据手册查看本实验的配套实验源码。

（5）main 函数逻辑代码实现

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint16_t i = 0;
    uint8_t datatemp[TEXT_SIZE];
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
```

```
  /* USER CODE BEGIN 2 */
  lcd_init();                                     /* 初始化 LCD */
  at24cxx_init();                                 /* 初始化 24CXX */

  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "IIC TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY1:Write  KEY0:Read", RED);
  while (at24cxx_check()) /* 检测不到 24c02 */
  {
      lcd_show_string(30, 130, 200, 16, 16, "24C02 Check Failed!", RED);
      delay_ms(500);
      lcd_show_string(30, 130, 200, 16, 16, "Please Check!      ", RED);
      delay_ms(500);
      LED0_TOGGLE();       /* 红灯闪烁 */
  }

  lcd_show_string(30, 130, 200, 16, 16, "24C02 Ready!", RED);
  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY1_PRES)    /* KEY1 按下,写入 24C02 */
      {
        lcd_fill(0, 150, 239, 319, WHITE);  /* 清除半屏 */
        lcd_show_string(30, 150, 200, 16, 16, "Start Write 24C02....", BLUE);
        at24cxx_write(0, (uint8_t *)g_text_buf, TEXT_SIZE);
        lcd_show_string(30, 150, 200, 16, 16, "24C02 Write Finished!", BLUE);
      }

      if (key == KEY0_PRES)    /* KEY0 按下,读取字符串并显示 */
      {
        lcd_show_string(30, 150, 200, 16, 16, "Start Read 24C02.... ", BLUE);
        at24cxx_read(0, datatemp, TEXT_SIZE);
        lcd_show_string(30, 150, 200, 16, 16, "The Data Readed Is:  ", BLUE);
        lcd_show_string(30, 170, 200, 16, 16, (char *)datatemp, BLUE);
      }

      i++;

      if (i == 20)
      {
          LED0_TOGGLE();        /* 红灯闪烁 */
          i = 0;
      }

      delay_ms(10);
  }
}
```

main 函数的流程大致是：在 main 函数外部定义要写入 24C02 的字符串数组 g_text_buf。在完成系统级和用户级初始化工作后，检测 24c02 是否存在，然后通过 KEY0 去读取 0 地址存放得数据并把数据显示在 LCD 上；另外还可以通过 KEY1 去 0 地址处写入 g_text_buf 数据并在 LCD 界面中显示传输中，完成后并显示“24C02 Write Finished!”。

44.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了，先按下 KEY1 写入数据，然后再按 KEY0 读取数据，最终 LCD 显示的内容如图 44.3.1 所示：

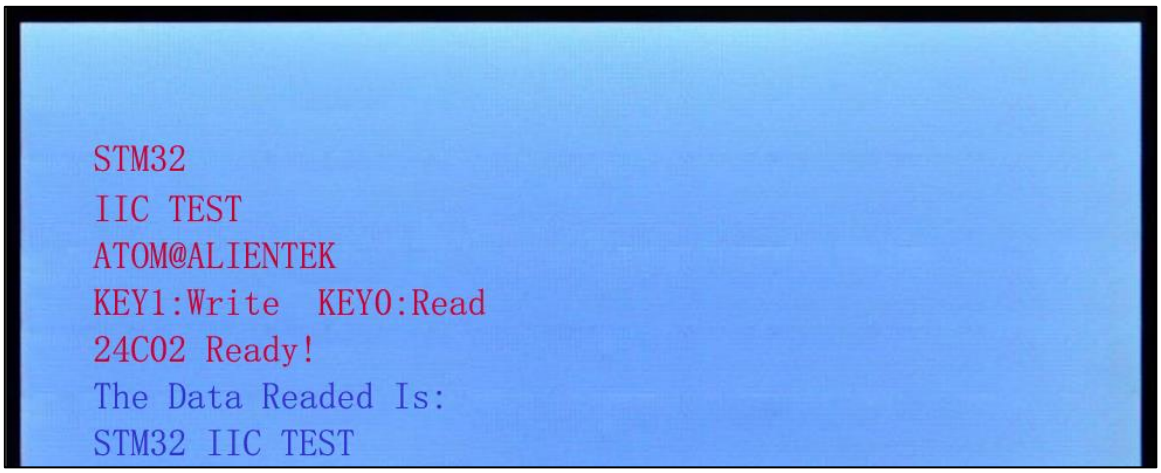


图 44.3.1 IIC 实验程序运行效果图

假如大家需要验证 24C02 的自检函数，可以用跟杜邦线把 PB6 和 PB7 短接，看是否能看到报错。

第四十五章 SPI 实验

本章，我们将介绍如何使用 STM32F103 的 SPI 功能，并实现对外部 NOR FLASH 的读写并把结果显示在 TFTLCD 模块上。

本章分为如下几个小节：

45.1 硬件设计
45.2 程序设计
45.3 下载验证

45.1 硬件设计

1. 例程功能

通过 KEY1 按键来控制 norflash 的写入，通过按键 KEY0 来控制 norflash 的读取。并在 LCD 模块上显示相关信息。我们还可以通过 USMART 控制读取 norflash 的 ID、擦除某个扇区或整片擦除。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
2）独立按键
KEY0 – PE4
KEY1 – PE3
3）NOR FLASH NM25Q128
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)（USMART 使用）

3. 原理图

我们主要来看看 norflash 和开发板的连接，如下图所示：

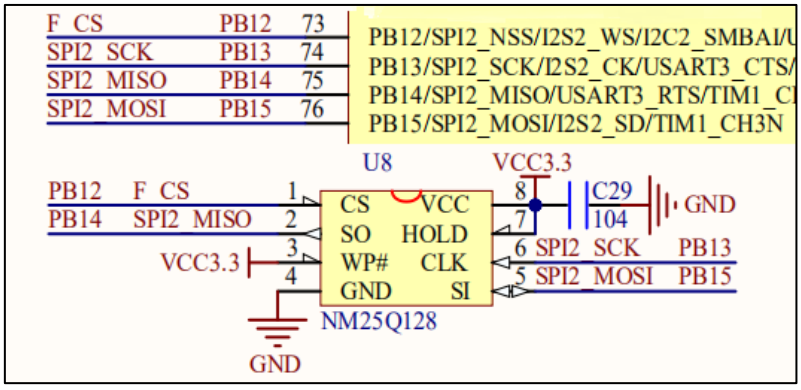


图 45.1.1 NOR FLASH 和开发板的连接

通过上图可知，NM25Q128 的 CS、SCK、MISO 和 MOSI 分别连接在 PB12、PB13、PB14 和 PB15 上。本实验还支持多种型号的 SPI FLASH 芯片，比如：BY25Q128/NM25Q128/W25Q128 等等，具体请看 norflash.h 文件的宏定义，在程序上只需要稍微修改一下，后面讲解程序的时候会提到。

45.2 程序设计

45.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 23_SPI。

45.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 23_SPI，以进行下一步的实验操作。如下图所示：

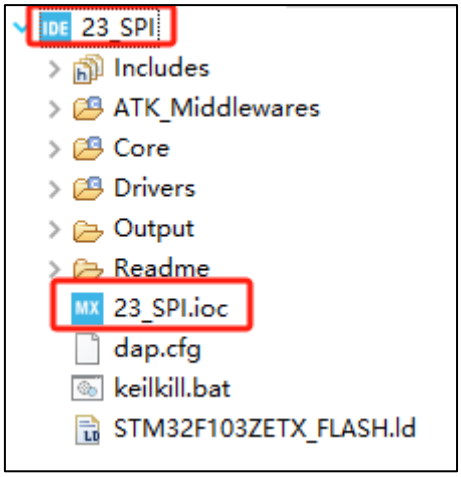


图 45.2.2.1 SPI 工程

（1）SPI2 配置

本实验采用 SPI2 驱动 noflash，并使用软件片选引脚。为配置 SPI2 参数，需要双击 23_SPI.ioc 进入 STM32CubeMX 插件的 Connectivity->SPI2 界面，注意开发板上的 norflash 仅支持 SPI 模式 0 和模式 3，这里我们使用模式 3，如下图所示。

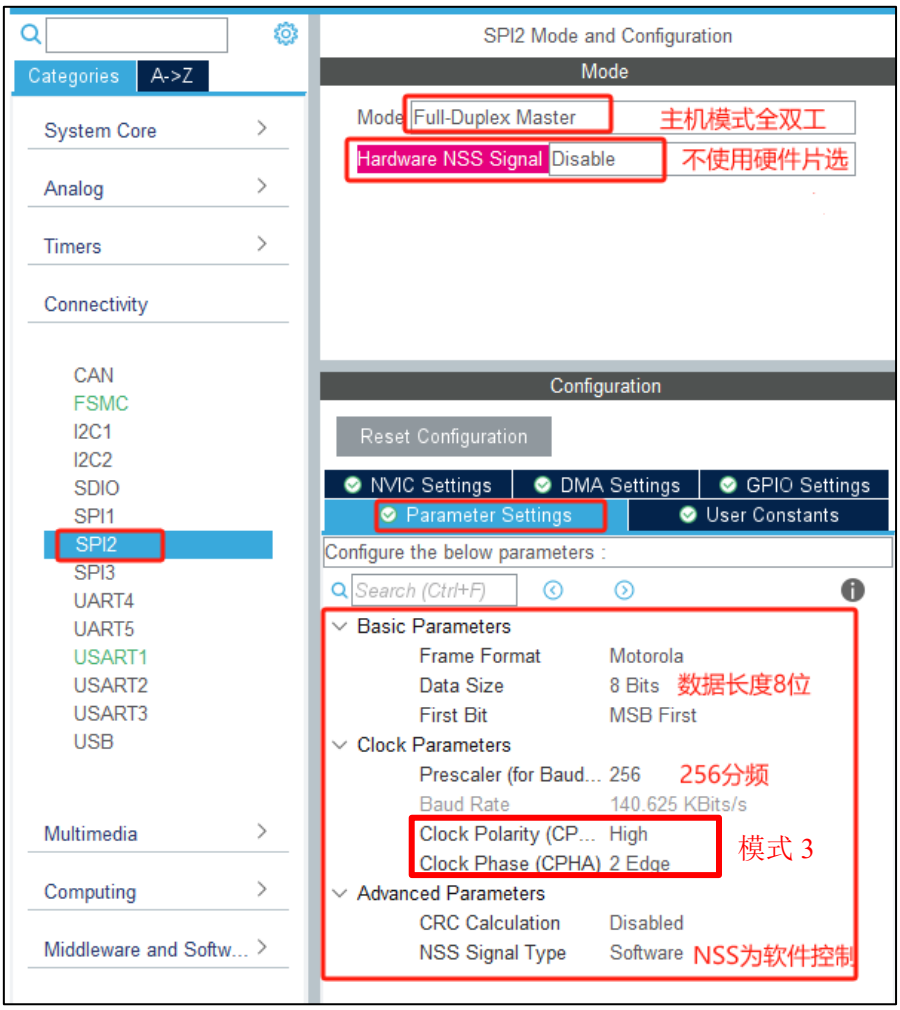


图 45.2.2.2 SPI 参数配置

（2）NSS 片选引脚配置

接着开始配置的软件 NSS 片选引脚，结合我们的硬件原理图，我们的片选引脚使用的是 PB12，所以进行如下配置：

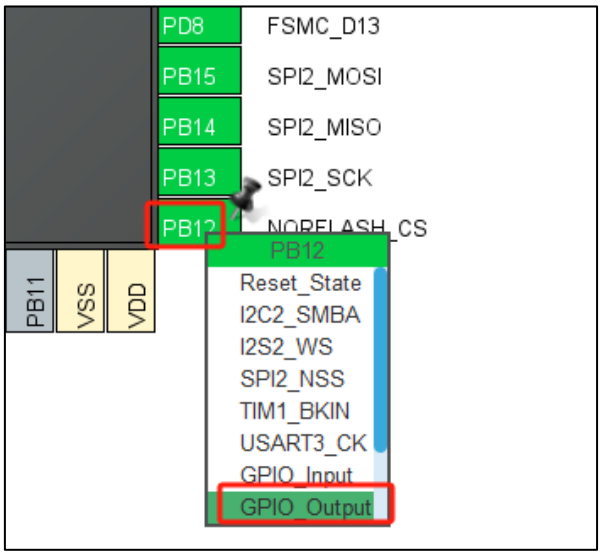


图 45.2.2.3 NSS 引脚复用

然后点击 GPIO 配置该 NSS 引脚的参数，如下：

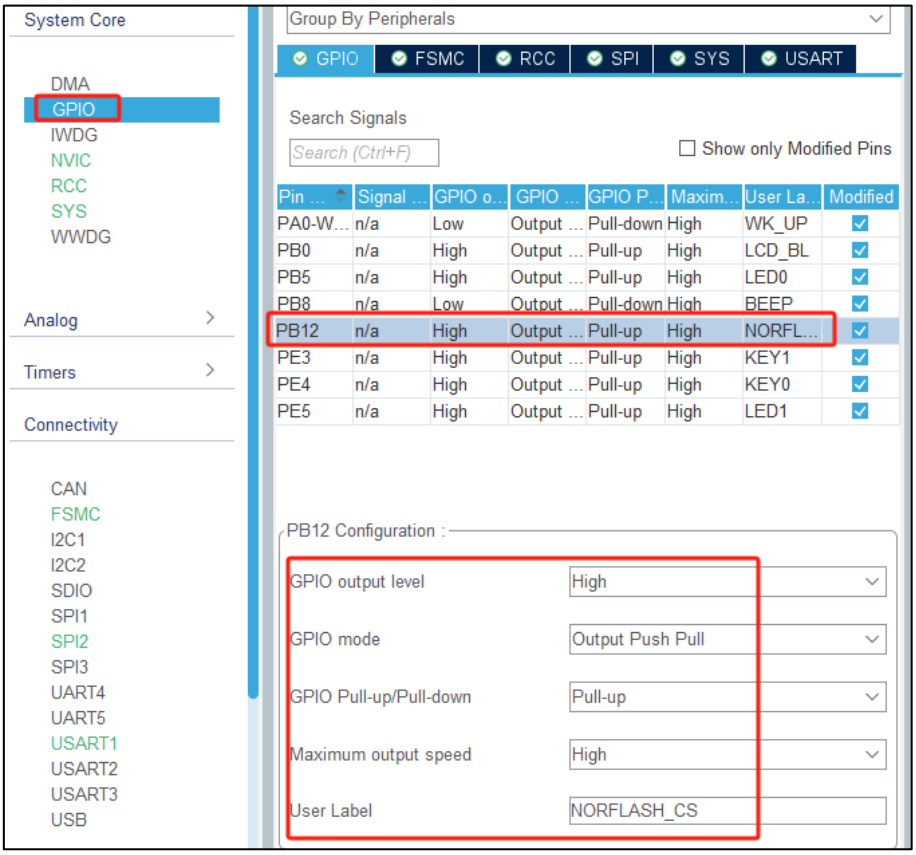


图 45.2.2.4 NSS 引脚参数配置

45.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程。如下图：

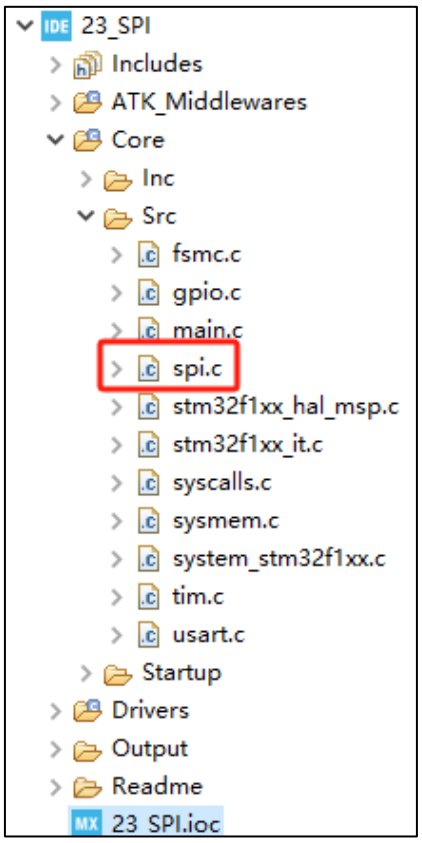


图 45.2.3.1 生成初始化工程

45.2.4 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 NORFLASH 文件夹，然后再 NORFLASH 文件夹下新建两个文件，分别是 norflash.c 和 norflash.h，如下图所示：

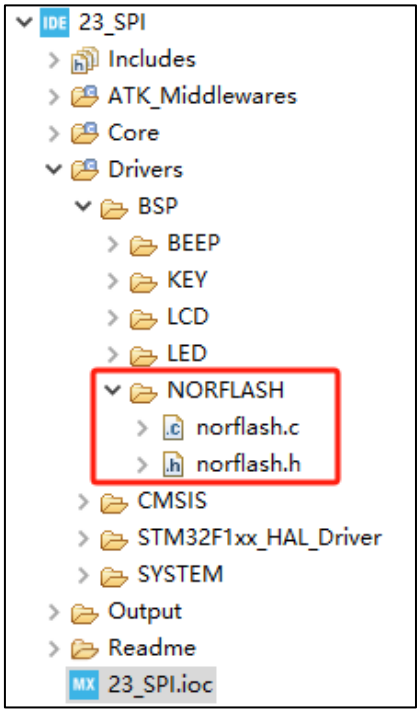


图 45.2.4.1 添加用户文件

本实验主要修改以下几个文件，分别是：spi.c、spi.h、norflash.c、norflash.h、main.c。下面我们先在 spi.h 中添加 SPI 速度设置相关的宏定义。

（1）spi.h 文件代码

```
#ifndef __SPI_H__
#define __SPI_H__

#ifdef __cplusplus
extern "C" {
#endif

#include "main.h"

extern SPI_HandleTypeDef hspi2;

/* USER CODE BEGIN Private defines */
/* SPI 总线速度设置 */
#define SPI_SPEED_2         0
#define SPI_SPEED_4         1
#define SPI_SPEED_8         2
#define SPI_SPEED_16        3
#define SPI_SPEED_32        4
#define SPI_SPEED_64        5
#define SPI_SPEED_128       6
#define SPI_SPEED_256       7
/* USER CODE END Private defines */

void MX_SPI2_Init(void);

/* USER CODE BEGIN Prototypes */
void spi2_set_speed(uint8_t speed);
uint8_t spi2_read_write_byte(uint8_t txdata);
/* USER CODE END Prototypes */

#ifdef __cplusplus
}
#endif

#endif /* __SPI_H__ */
```

在 spi.h 文件中，主要定义了 SPI 通信速率的宏定义，并声明了一些功能函数，以便于更加便捷地设置 SPI 通信速率。

（2）spi.c 文件代码

```
void MX_SPI2_Init(void)
{
  hspi2.Instance = SPI2;
  hspi2.Init.Mode = SPI_MODE_MASTER;
  hspi2.Init.Direction = SPI_DIRECTION_2LINES;
  hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
  hspi2.Init.CLKPolarity = SPI_POLARITY_HIGH;
  hspi2.Init.CLKPhase = SPI_PHASE_2EDGE;
  hspi2.Init.NSS = SPI_NSS_SOFT;
  hspi2.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
  hspi2.Init.FirstBit = SPI_FIRSTBIT_MSB;
  hspi2.Init.TIMode = SPI_TIMODE_DISABLE;
  hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
  hspi2.Init.CRCPolynomial = 10;
  if (HAL_SPI_Init(&hspi2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN SPI2_Init 2 */
```

```
  __HAL_SPI_ENABLE(&hspi2);      /* 使能 SPI2 */
  spi2_read_write_byte(0Xff);    /* 启动传输,产生 8 个时钟脉冲,清空 DR 的作用 非必需 */
  /* USER CODE END SPI2_Init 2 */

}

/* 省略部分代码... ... */

/* USER CODE BEGIN 1 */

/**
 * @brief       SPI2 速度设置函数
 *   @note      SPI2 时钟选择来自 APB1, 即 PCLK1, 为 36Mhz
 *              SPI 速度 = PCLK1 / 2^(speed + 1)
 * @param       speed   : SPI2 时钟分频系数
                        取值为 SPI_BAUDRATEPRESCALER_2~SPI_BAUDRATEPRESCALER_2
256
 * @retval      无
 */
void spi2_set_speed(uint8_t speed)
{
    assert_param(IS_SPI_BAUDRATE_PRESCALER(speed)); /* 判断有效性 */
    __HAL_SPI_DISABLE(&hspi2);              /* 关闭 SPI */
    hspi2.Instance->CR1 &= 0XFFC7;          /* 位 3-5 清零，用来设置波特率 */
    hspi2.Instance->CR1 |= speed << 3;      /* 设置 SPI 速度 */
    __HAL_SPI_ENABLE(&hspi2);               /* 使能 SPI */
}

/**
 * @brief       SPI2 读写一个字节数据
 * @param       txdata  : 要发送的数据(1 字节)
 * @retval      接收到的数据(1 字节)
 */
uint8_t spi2_read_write_byte(uint8_t txdata)
{
    uint8_t rxdata;
    HAL_SPI_TransmitReceive(&hspi2, &txdata, &rxdata, 1, 1000);
    return rxdata;                          /* 返回收到的数据 */
}
/* USER CODE END 1 */
```

在 SPI 的初始化函数中，我们添加了函数__HAL_SPI_ENABLE 用于使能 SPI。以及函数 spi2_read_write_byte(0Xff)，用于启动发送。

函数 spi2_set_speed 是 SPI 的速度设置函数，通过操作寄存器的方式去实现。

函数 spi2_read_write_byte 直接调用了 HAL 库内置的函数进行接收发送操作。

（3）norflash.h 文件代码

```
#ifndef __norflash_H
#define __norflash_H

#include "../../SYSTEM/sys/sys.h"


/*****************************************************************************/
/* NORFLASH 片选信号 */
#define NORFLASH_CS(x)      do{ x ? \
                                HAL_GPIO_WritePin(NORFLASH_CS_GPIO_Port,
NORFLASH_CS_Pin, GPIO_PIN_SET) : \
                                HAL_GPIO_WritePin(NORFLASH_CS_GPIO_Port,
NORFLASH_CS_Pin, GPIO_PIN_RESET); \
                            }while(0)
```

```
/* FLASH 芯片列表 */
#define W25Q80      0XEF13          /* W25Q80   芯片 ID */
#define W25Q16      0XEF14          /* W25Q16   芯片 ID */
#define W25Q32      0XEF15          /* W25Q32   芯片 ID */
#define W25Q64      0XEF16          /* W25Q64   芯片 ID */
#define W25Q128     0XEF17          /* W25Q128  芯片 ID */
#define W25Q256     0XEF18          /* W25Q256  芯片 ID */
#define BY25Q64     0X6816          /* BY25Q64  芯片 ID */
#define BY25Q128    0X6817          /* BY25Q128 芯片 ID */
#define NM25Q64     0X5216          /* NM25Q64  芯片 ID */
#define NM25Q128    0X5217          /* NM25Q128 芯片 ID */

extern uint16_t norflash_TYPE;      /* 定义 FLASH 芯片型号 */

/* 指令表 */
#define FLASH_WriteEnable           0x06
#define FLASH_WriteDisable          0x04
#define FLASH_ReadStatusReg1        0x05
#define FLASH_ReadStatusReg2        0x35
#define FLASH_ReadStatusReg3        0x15
#define FLASH_WriteStatusReg1       0x01
#define FLASH_WriteStatusReg2       0x31
#define FLASH_WriteStatusReg3       0x11
#define FLASH_ReadData              0x03
#define FLASH_FastReadData          0x0B
#define FLASH_FastReadDual          0x3B
#define FLASH_FastReadQuad          0xEB
#define FLASH_PageProgram           0x02
#define FLASH_PageProgramQuad       0x32
#define FLASH_BlockErase            0xD8
#define FLASH_SectorErase           0x20
#define FLASH_ChipErase             0xC7
#define FLASH_PowerDown             0xB9
#define FLASH_ReleasePowerDown      0xAB
#define FLASH_DeviceID              0xAB
#define FLASH_ManufactDeviceID      0x90
#define FLASH_JedecDeviceID         0x9F
#define FLASH_Enable4ByteAddr       0xB7
#define FLASH_Exit4ByteAddr         0xE9
#define FLASH_SetReadParam          0xC0
#define FLASH_EnterQPIMode          0x38
#define FLASH_ExitQPIMode           0xFF

/* 普通函数 */
void norflash_init(void);                   /* 初始化 25QXX */
uint16_t norflash_read_id(void);            /* 读取 FLASH ID */
void norflash_write_enable(void);           /* 写使能 */
uint8_t norflash_read_sr(uint8_t regno);    /* 读取状态寄存器 */
void norflash_write_sr(uint8_t regno,uint8_t sr);   /* 写状态寄存器 */

void norflash_erase_chip(void);             /* 整片擦除 */
void norflash_erase_sector(uint32_t saddr); /* 扇区擦除 */
void norflash_read(uint8_t *pbuf, uint32_t addr, uint16_t datalen);     /* 读取
flash */
void norflash_write(uint8_t *pbuf, uint32_t addr, uint16_t datalen);    /* 写入
flash */

#endif
```

使用宏定义的方式， 定义了 NSS 引脚，便于片选操作的简洁性。然后为了兼容不同的 noflash 我们使用宏定义的方式，定义了多种芯片 ID，并且为了代码的可读性，我们还使用宏定义的方式定义了指令码等。

（4）norflash.c 文件代码

```
/**
 * @brief       初始化 SPI NOR FLASH
 * @param       无
 * @retval      无
 */
void norflash_init(void)
{
    uint8_t temp;

    NORFLASH_CS(1);                         /* 取消片选 */

    MX_SPI2_Init();                         /* 初始化 SPI2 */
    spi2_set_speed(SPI_SPEED_2);            /* SPI2 切换到高速状态 18Mhz */

    g_norflash_type = norflash_read_id();/* 读取 FLASH ID. */

    if (g_norflash_type == W25Q256)         /* W25Q256, 使能 4 字节地址模式 */
    {
        temp = norflash_read_sr(3);         /* 读取状态寄存器 3，判断地址模式 */

        if ((temp & 0X01) == 0)             /* 不是 4 字节地址模式,进入 4 字节地址模式 */
        {
            norflash_write_enable();        /* 写使能 */
            temp |= 1 << 1;                 /* ADP=1, 上电 4 位地址模式 */
            norflash_write_sr(3, temp); /* 写 SR3 */

            NORFLASH_CS(0);
            spi2_read_write_byte(FLASH_Enable4ByteAddr); /* 使能 4 字节地址指令 */
            NORFLASH_CS(1);
        }
    }
}
```

可以看到，在 norflash 的初始化函数中，先拉高 norflash 片选的 GPIO 引脚，然后还会根据不同容量的 norflash 做相应的配置操作。

norflash 驱动中其他对 norflash 的操作函数，例如，norflash 的读写函数、擦除函数等，请读者结合 25Q128 NOR Flash 芯片的数据手册查看本实验的配套实验源码。

（5）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint16_t i = 0;
    uint8_t datatemp[TEXT_SIZE+2];
    uint32_t flashsize;
    uint16_t id = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();
```

```
/* FSMC 初始化 */
MX_FSMC_Init();

/* 串口初始化 */
MX_USART1_UART_Init();

/* SPI2 初始化 */
MX_SPI2_Init();

/* USER CODE BEGIN 2 */
lcd_init();                                     /* 初始化 LCD */
norflash_init();                                /* 初始化 NORFLASH */

lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
lcd_show_string(30,  70, 200, 16, 16, "SPI TEST", RED);
lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
lcd_show_string(30, 110, 200, 16, 16, "KEY1:Write  KEY0:Read", RED);
id = norflash_read_id(); /* 读取 FLASH ID */

while ((id == 0) || (id == 0XFFFF)) /* 检测不到 FLASH 芯片 */
{
    lcd_show_string(30, 130, 200, 16, 16, "FLASH Check Failed!", RED);
    delay_ms(500);
    lcd_show_string(30, 130, 200, 16, 16, "Please Check!      ", RED);
    delay_ms(500);
    LED0_TOGGLE(); /* LED0 闪烁 */
}

lcd_show_string(30, 130, 200, 16, 16, "SPI FLASH Ready!", BLUE);
flashsize = 16 * 1024 * 1024; /* FLASH 大小为 16M 字节 */
/* USER CODE END 2 */

while (1)
{
    key = key_scan(0);

    if (key == KEY1_PRES) /* KEY1 按下,写入 */
    {
      lcd_fill(0, 150, 239, 319, WHITE); /* 清除半屏 */
      lcd_show_string(30, 150, 200, 16, 16, "Start Write FLASH....", BLUE);
      printf("111:%d\r\n",i);
      sprintf((char *)datatemp, "%s%d", (char *)g_text_buf, i);
      norflash_write((uint8_t *)datatemp, flashsize - 100, TEXT_SIZE);
      lcd_show_string(30, 150, 200, 16, 16, "FLASH Write Finished!", BLUE);
    }

    if (key == KEY0_PRES) /* KEY0 按下,读取字符串并显示 */
    {
      lcd_show_string(30, 150, 200, 16, 16, "Start Read FLASH... . ", BLUE);
      norflash_read(datatemp, flashsize - 100, TEXT_SIZE);
      lcd_show_string(30, 150, 200, 16, 16, "The Data Readed Is:   ", BLUE);
      lcd_show_string(30, 170, 200, 16, 16, (char *)datatemp, BLUE);
    }

    i++;

    if (i == 20)
    {
        LED0_TOGGLE(); /* LED0 闪烁 */
        i = 0;
    }

    delay_ms(10);
```

```
    }
}
```

在 main 函数前面，我们定义了 g_text_buf 数组，用来存放要写入到 FLASH 的字符串。main 函数代码和 IIC 实验那部分代码大同小异，具体流程大致是：在完成系统级和用户级初始化工作后，读取 FLASH 的 ID，然后通过 KEY0 去读取倒数第 100 个地址处开始的数据并把数据显示在 LCD 上；另外还可以通过 KEY1 去倒数第 100 个地址处写入 g_text_buf 数据并在 LCD 界面中显示传输中，完成后并显示“FLASH Write Finished!”。

45.3 下载验证

在完成编译和烧录操作后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 45.3.1 所示：

图 45.3.1 SPI 实验程序运行效果图

通过先按下 KEY1 写入数据，然后再按 KEY0 读取数据，得到如图 45.3.2 所示：

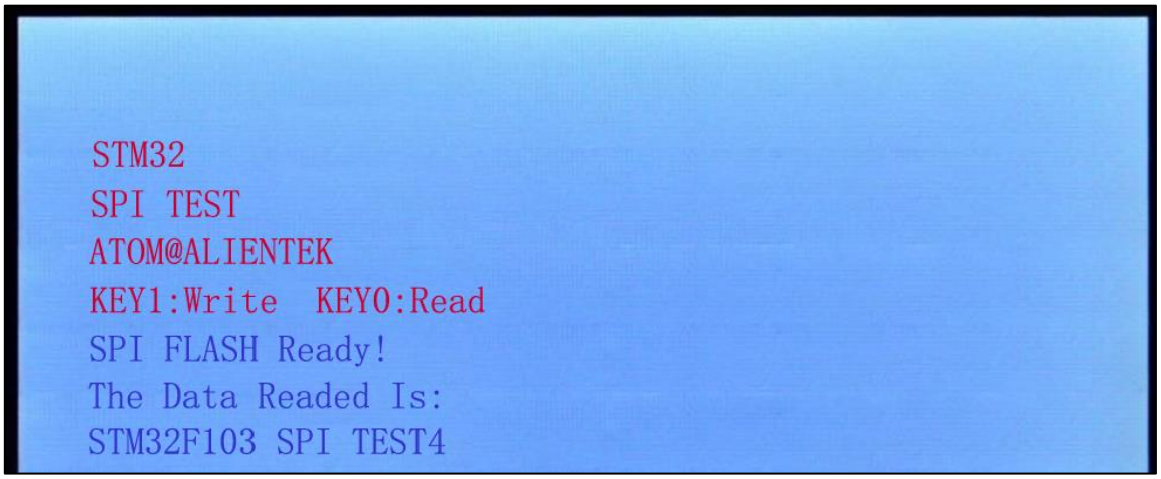


图 45.3.2 操作后的显示效果图

程序在开机的时候会检测 NOR FLASH 是否存在，如果不存在则会在 LCD 模块上显示错误信息，同时 LED0 慢闪。大家可以通过跳线帽把 PB14 和 PB15 短接就可以看到报错了。

第四十六章 485 实验

本章我们将向大家介绍如何使用 STM32F1 的串口实现 485 通信（半双工）。在本章中，我们将使用 STM32F1 的串口 2 来实现两块开发板之间的 485 通信，并将结果显示在 TFTLCD 模块上。

本章分为如下几个部分：

46.1 硬件设计
46.2 程序设计
46.3 下载验证

46.1 硬件设计

1. 例程功能

本章将实现这样的功能：通过连接两个精英 STM32F103 的 RS485 接口，然后由 KEY0 控制发送，当按下一个开发板的 KEY0 的时候，就发送 5 个数据给另外一个开发板，并在两个开发板上分别显示发送的值和接收到的值。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）USART2，用于实际的 485 信号串行通讯。
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）RS485 收发芯片 TP8485/SP3485
5）开发板两块（485 半双式模式无法自收发，我们需要用两个开发板或者 USB 转 485 调试器+串口助手来帮助我们完成测试，大家根据自己的实际条件选择）

3. 原理图

根据我们需要实现的程序功能，我们设计电路原理如下：

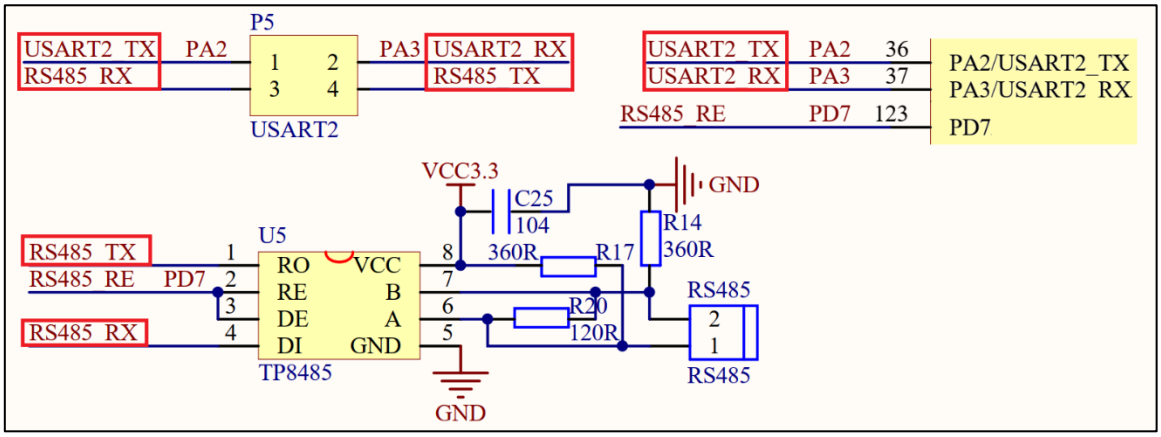


图 46.1.1 RS485 连接原理设计

从上图可以看出：开发板的串口 2 和 TP8485 上的引脚连接到 P5 端上的端子，但不直接相连，所以测试 485 功能时我们需要用跳线帽短接 P5 上的两组排针使之连通。STM32F1 的 PD7 控制 RS485 的收发模式：

当 PD7=0 的时候，为接收模式；当 PD7=1 的时候，为发送模式。

另外，图中的 R14 和 R17 是两个偏置电阻，用来保证总线空闲时，A、B 之间的电压差都会大于 200mV（逻辑 1）。从而避免因总线空闲时因 A、B 压差不稳定，可能出现的乱码。

最后，我们用 2 根导线将两个开发板 RS485 端子的 A 和 A，B 和 B 连接起来。这里注意不要接反了（A 接 B），接反了会导致通讯异常!!

46.2 程序设计

46.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 24_RS485。

46.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 24_RS485，以进行下一步的实验操作。如下图所示：

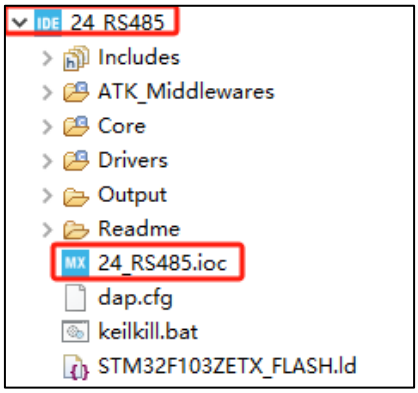


图 46.2.2.1 RS485 工程

（1）USART2 配置

本章，我们只需要配置好串口 2，就可以实现正常的 485 通信，串口 2 的配置和串口 1 基本类似，只是串口 2 的时钟来自 APB1，最大频率为 36Mhz。配置如下图所示：

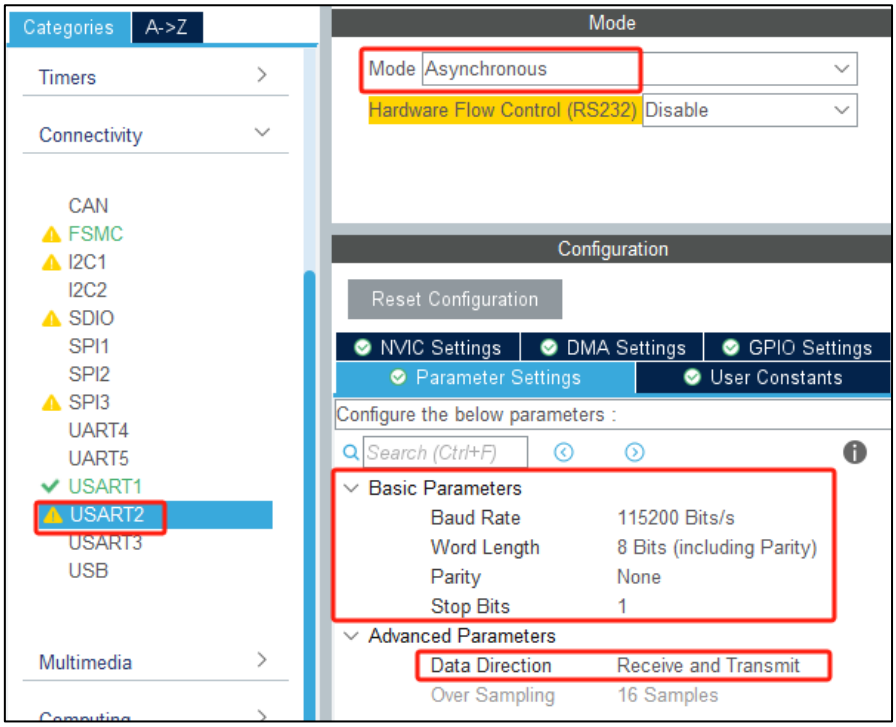


图 46.2.2.2 串口 2 配置

USART2 的参数我们选择模式：Asynchronous（异步通信模式），其余默认配置为：波特率为 115200Bit/s；字长为 8 位；无校验位；1 位停止位；数据方向为发和收；

（2）NVIC 配置

本实验我们要使用串口接收中断，所以要配置 NVIC，首先开启串口全局中断，配置串口中断优先级分组为 2，抢占优先级 2，子优先级 1，如下图所示：

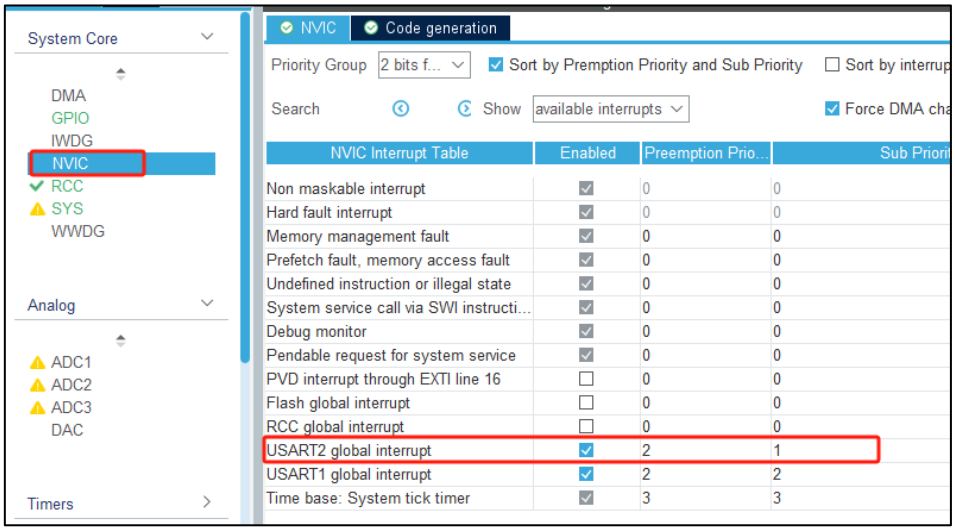


图 46.2.2.3 NVIC 配置

46.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程。如下图。其中在 usart.c 中，主要用于初始化 USART1 和 USART2。

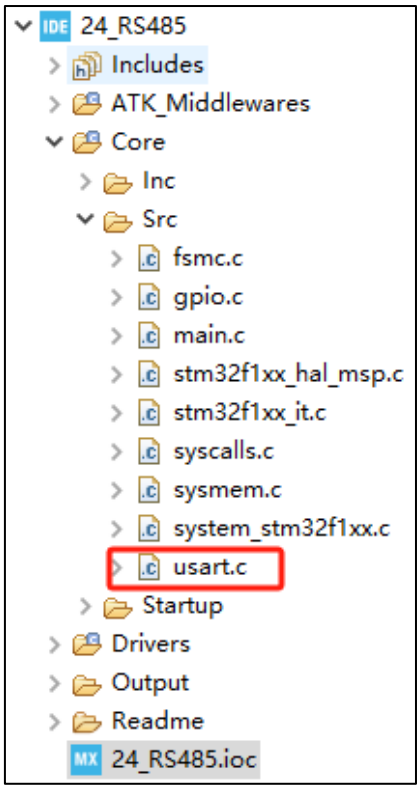


图 46.2.3.1 生成初始化工程

46.2.4 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 RS485 文件夹，然后再 RS485 文件夹下新建两个文件，分别是 rs485.c 和 rs485.h，如下图所示：

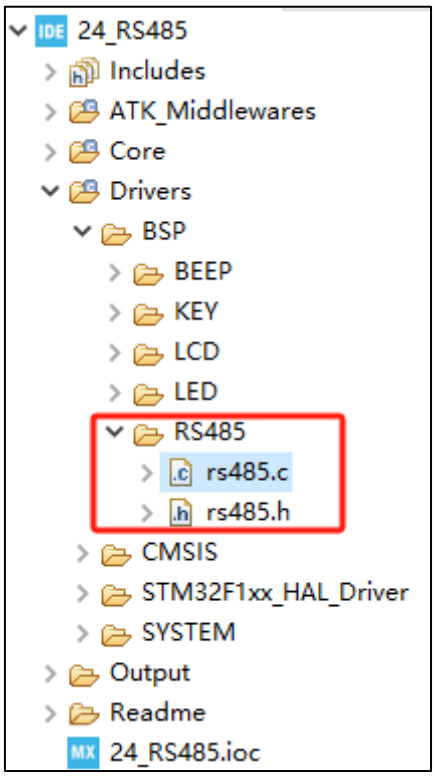


图 46.2.4.1 添加用户文件

本实验主要修改以下几个文件，分别是：rs485.c、rs485.h、stm32f1xx_it.c、main.c。下面我们先看下 rs485.h 中。

（1）rs485.h 文件代码

```
#ifndef __RS485_H
#define __RS485_H

#include "../../SYSTEM/sys/sys.h"

/******************************************************************************/

/* 控制 RS485_RE 脚, 控制 RS485 发送/接收状态
 * RS485_RE = 0, 进入接收模式
 * RS485_RE = 1, 进入发送模式
 */
#define RS485_RE(x)    do{ x ? \
                           HAL_GPIO_WritePin(RS485_RE_GPIO_Port, RS485_RE_Pin,
                            GPIO_PIN_SET) : \
                           HAL_GPIO_WritePin(RS485_RE_GPIO_Port, RS485_RE_Pin,
                            GPIO_PIN_RESET); \
                       }while(0)

#define RS485_REC_LEN               64          /* 定义最大接收字节数 64 */

extern uint8_t g_RS485_rx_buf[RS485_REC_LEN];    /* 缓冲,最大 RS485_REC_LEN 字节*/
extern uint8_t g_RS485_rx_cnt;                   /* 接收数据长度 */
```

```
void rs485_init( uint32_t baudrate);                    /* RS485 初始化 */
void rs485_send_data(uint8_t *buf, uint8_t len);        /* RS485 发送数据 */
void rs485_receive_data(uint8_t *buf, uint8_t *len);    /* RS485 接收数据 */

#endif
```

在该文件中，主要定义了 RE 引脚的宏定义，该引脚控制 RS485 的接收或发送。

（2）rs485.c 文件代码

```
uint8_t g_RS485_rx_buf[RS485_REC_LEN]; /* 接收缓冲, 最大 RS485_REC_LEN 个字节. */
uint8_t g_RS485_rx_cnt = 0;            /* 接收到的数据长度 */

/**
 * @brief       RS485 初始化函数
 *   @note      该函数主要是初始化串口
 * @param       baudrate: 波特率, 根据自己需要设置波特率值
 * @retval      无
 */
void rs485_init(uint32_t baudrate)
{
    __HAL_UART_ENABLE_IT(&huart2, UART_IT_RXNE);     /* 开启接收中断 */
    RS485_RE(0);                                     /* 默认为接收模式 */
}

/**
 * @brief       RS485 发送 len 个字节
 * @param       buf     : 发送区首地址
 * @param       len     : 发送的字节数(为了和本代码的接收匹配,这里建议不要超过
RS485_REC_LEN 个字节)
 * @retval      无
 */
void rs485_send_data(uint8_t *buf, uint8_t len)
{
    RS485_RE(1);                                    /* 进入发送模式 */
    HAL_UART_Transmit(&huart2, buf, len, 1000); /* 串口 2 发送数据 */
    g_RS485_rx_cnt = 0;
    RS485_RE(0); /* 进入接收模式 */
}

/**
 * @brief       RS485 查询接收到的数据
 * @param       buf     : 接收缓冲区首地址
 * @param       len     : 接收到的数据长度
 *   @arg               0    , 表示没有接收到任何数据
 *   @arg               其他, 表示接收到的数据长度
 * @retval      无
 */
void rs485_receive_data(uint8_t *buf, uint8_t *len)
{
    uint8_t rxlen = g_RS485_rx_cnt;
    uint8_t i = 0;
    *len = 0;      /* 默认为 0 */
    delay_ms(10); /* 等待 10ms,连续超过 10ms 没有接收到一个数据,则认为接收结束 */

    if (rxlen == g_RS485_rx_cnt && rxlen)  /* 接收到了数据,且接收完成了 */
    {
        for (i = 0; i < rxlen; i++)
        {
            buf[i] = g_RS485_rx_buf[i];
        }
```

```
        *len = g_RS485_rx_cnt;                  /* 记录本次数据长度 */
        g_RS485_rx_cnt = 0;                     /* 清零 */
    }
}
```

在 rs485_init 中，开启串口 2 的接收中断，关于串口 2 的初始化，在 usart.c 中已完成，感兴趣可以打开实验源码查看。拉低 RE 引脚，即默认 485 为接收状态。

rs485_send_data 为发送函数用于输出 485 信号到 485 总线上，我的默认的 485 方式一般空闲时为接收状态，只有发送数据时我们才控制 485 芯片进入发送状态，发送完成后马上回到空闲接收状态，这样可以保证操作过程中 485 的数据丢失最小。

rs485_receive_data 查询接收数据函数，该函数用于查询 485 总线上接收到的数据，主要实现的逻辑是：一开始进入函数时，先记录下当前接收计数器的值，再来一个延时去判断接收是否结束（即该期间有无接收到数据），假如说接收计数器的值没有改变，就证明接收结束，我们就可以把当前接收缓冲区传递出去。

（3）stm32f1xx_it.c 文件代码

```
void USART2_IRQHandler(void)
{
  /* USER CODE BEGIN USART2_IRQn 0 */
    uint8_t res;

    if ((__HAL_UART_GET_FLAG(&huart2, UART_FLAG_RXNE) != RESET)) /* 接收到数据*/
    {
        HAL_UART_Receive(&huart2, &res, 1, 1000);

        if (g_RS485_rx_cnt < RS485_REC_LEN)          /* 缓冲区未满 */
        {
            g_RS485_rx_buf[g_RS485_rx_cnt] = res;    /* 记录接收到的值 */
            g_RS485_rx_cnt++;                        /* 接收数据增加 1 */
        }
    }
  HAL_UART_IRQHandler(&huart2);
  /* USER CODE END USART2_IRQn 1 */
}
```

RS485 的接收就与串口中断一样了，不过要注意空闲时要切换回接收状态，否则会收不到数据。我们定义了一个全局的缓冲区 g_RS485_rx_buf 进行接收测试，通过串口中断接收数据。

（4）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint8_t i = 0, t = 0;
    uint8_t cnt = 0;
    uint8_t rs485buf[5];
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();
```

```
  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                                     /* 初始化 LCD */
  rs485_init(9600);
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "RS485 TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:Send", RED);

  lcd_show_string(30, 130, 200, 16, 16, "Count:", RED);
  lcd_show_string(30, 150, 200, 16, 16, "Send Data:", RED);
  lcd_show_string(30, 190, 200, 16, 16, "Receive Data:", RED);

  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY0_PRES)                       /* KEY0 按下,发送一次数据 */
      {
          for (i = 0; i < 5; i++)
          {
              rs485buf[i] = cnt + i;              /* 填充发送缓冲区 */
              lcd_show_xnum(30 + i * 32, 170, rs485buf[i], 3, 16, 0X80, BLUE);
          }

          rs485_send_data(rs485buf, 5);           /* 发送 5 个字节 */
      }

      rs485_receive_data(rs485buf, &key);

      if (key)                                    /* 接收到有数据 */
      {
          if (key > 5)key = 5;                    /* 最大是 5 个数据. */

          for (i = 0; i < key; i++)
          {
              lcd_show_xnum(30 + i * 32, 210, rs485buf[i], 3, 16, 0X80, BLUE);
          }
      }

      t++;
      delay_ms(10);

      if (t == 20)
      {
          LED0_TOGGLE();                          /* LED0 闪烁, 提示系统正在运行 */
          t = 0;
          cnt++;
          lcd_show_xnum(30 + 48, 130, cnt, 3, 16, 0X80, BLUE);
      }
  }
}
```

我们是通过按键控制数据的发送。在此部分代码中，cnt 是一个累加数，一旦 KEY0 按下，就以这个数位基准连续发送 5 个数据。当 485 总线收到数据得时候，就将收到的数据直接显示在 LCD 屏幕上。

46.3 下载验证

在代码编译成功之后，我们通过下载代码到正点原子精英 STM32F103 上（注意要 2 个开发板都下载这个代码），得到如图 46.3.1 所示：

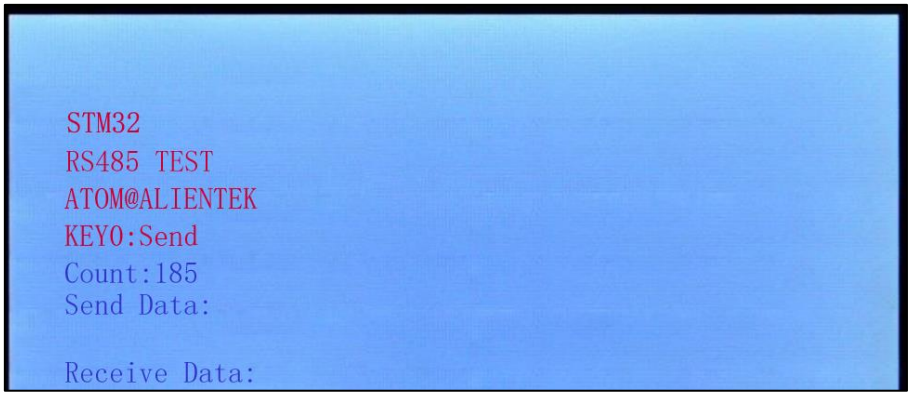


图 46.3.1 程序运行效果图

伴随 DS0 的不停闪烁，提示程序在运行。此时，我们按下 KEY0 就可以在另外一个开发板上面收到这个开发板发送的数据了。如图 46.3.2 和图 46.3.3 所示：

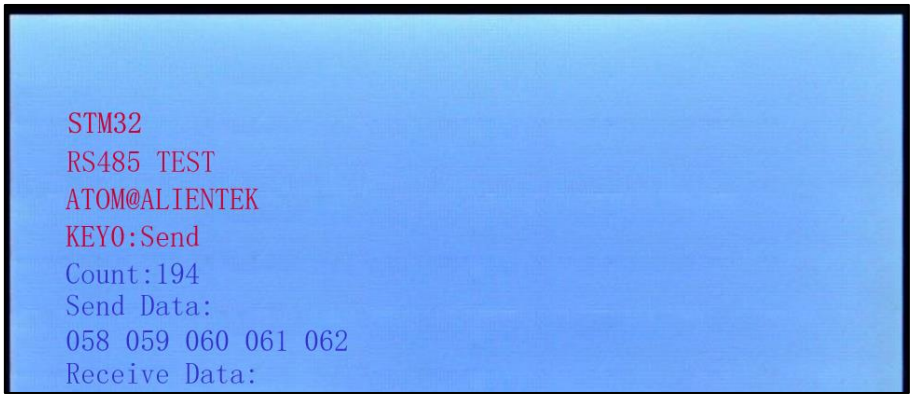


图 46.3.2 发送 RS485 数据的开发板界面

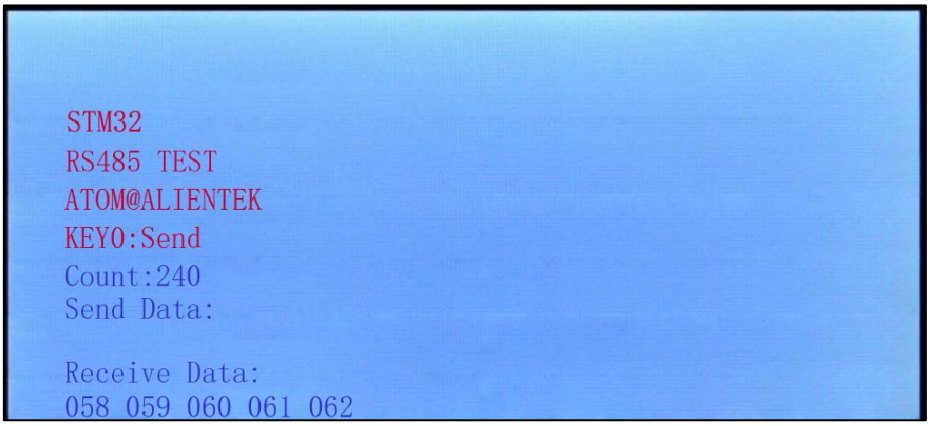


图 46.3.3 接收 RS485 数据的开发板

图 46.3.2 来自开发板 A，发送了 5 个数据，图 46.3.3 来自开发板 B，接收到了来自开发板 A 的 5 个数据。

第四十七章 CAN 通讯实验

本章我们将向大家介绍如何使用 STM32 自带的 CAN 控制器来实现 CAN 的收发功能，并将结果显示在 TFTLCD 模块上。本章分为如下几个部分：

47.1 硬件设计
47.2 程序设计
47.3 下载验证

47.1 硬件设计

1. 例程功能

通过 KEY_UP 按键（即 WK_UP 按键）选择 CAN 的工作模式（正常模式/环回模式），然后通过 KEY0 控制数据发送，接着查询是否有数据接收到，假如接收到数据，就将接收到的数据显示在 LCD 模块上。如果是环回模式，我们不需要 2 个开发板。如果是正常模式，我们就需要 2 个精英开发板，并且将他们的 CAN 接口对接起来，然后一个开发板发送数据，另外一个开发板将接收到的数据显示在 LCD 模块上。

2. 硬件资源

1） LED 灯
LED0 - PB5
2） 独立按键
KEY0 - PE4
KEY_UP - PA0
3） 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4） STM32 自带 CAN 控制器
5） CAN 收发芯片 TJA1050/SIT1050T

3. 原理图

STM32 有 CAN 的控制器，但要实现 CAN 通讯的差分电平，我们还需要借助外围电路来实现，根据我们需要实现的程序功能，我们设计电路原理如下：

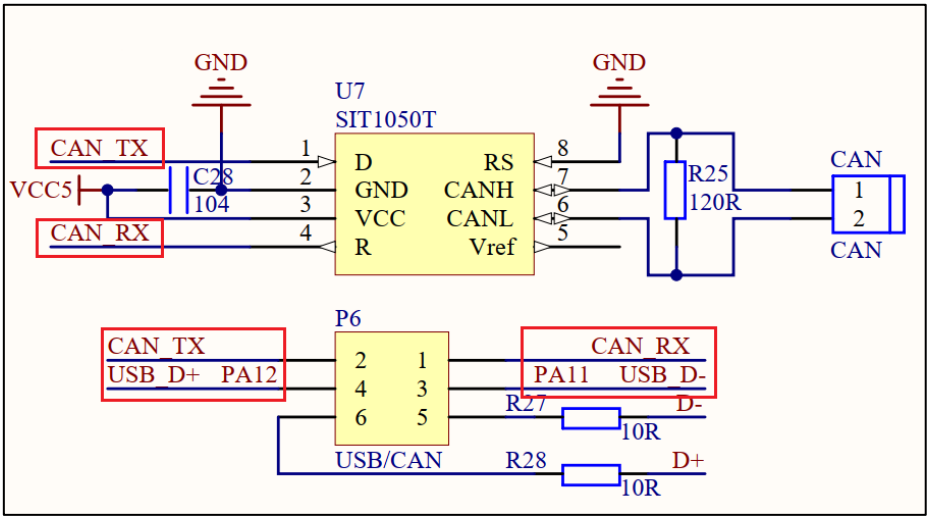


图 47.1.2 CAN 连接原理设计

从上图可以看出：STM32F1 的 CAN 通过 P6 的设置，连接到 TJA1050/SIT1050T 收发芯

片，然后通过接线端子（CAN）同外部的 CAN 总线连接。图中可以看出，在精英 STM32F103 开发板上面是带有 120Ω的终端电阻的，如果我们的开发板不是作为 CAN 的终端的话，需要把这个电阻去掉，以免影响通信。另外，需要注意：CAN 和 USB 共用了 PA11 和 PA12，所以他们不能同时使用。

这里还要注意，我们要设置好开发板上 P6 排针的连接，通过跳线帽将 PA11 和 PA12 分别连接到 CRX 和 CTX 上面，如图 47.1.2 所示。

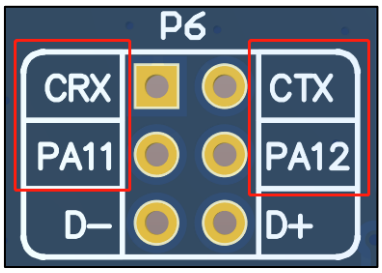


图 47.1.2 CAN 实验需要跳线连接的位置

最后，我们用 2 根导线将两个开发板 CAN 端子的 CAN_L 和 CAN_L，CAN_H 和 CAN_H 连接起来。这里注意不要接反了（CAN_L 接 CAN_H），接反了会导致通讯异常!!

47.2 程序设计

47.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 25_CAN。

47.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 25_CAN，以进行下一步的实验操作。如下图所示：

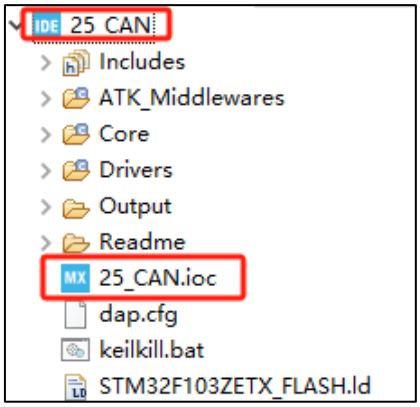


图 47.2.2.1 CAN 工程

（1）CAN 配置

双击 25_CAN.ioc 进入 STM32CubeMX 插件的，在 Pinout & Configuration 处我们配置 CAN 的相关 IO，如下图所示

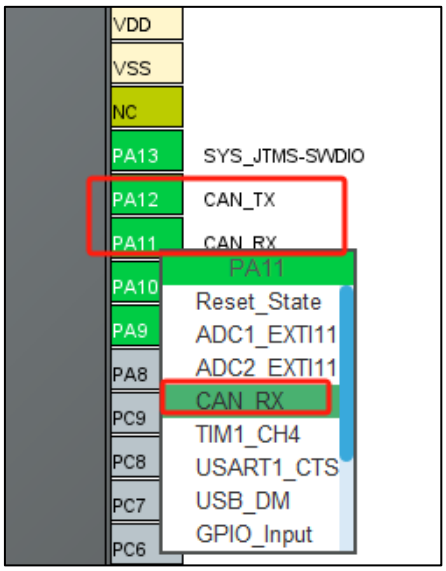


图 47.2.2.2 GPIO 复用

接着在 Connectivity->CAN 界面，配置 CAN 参数，如下图所示：

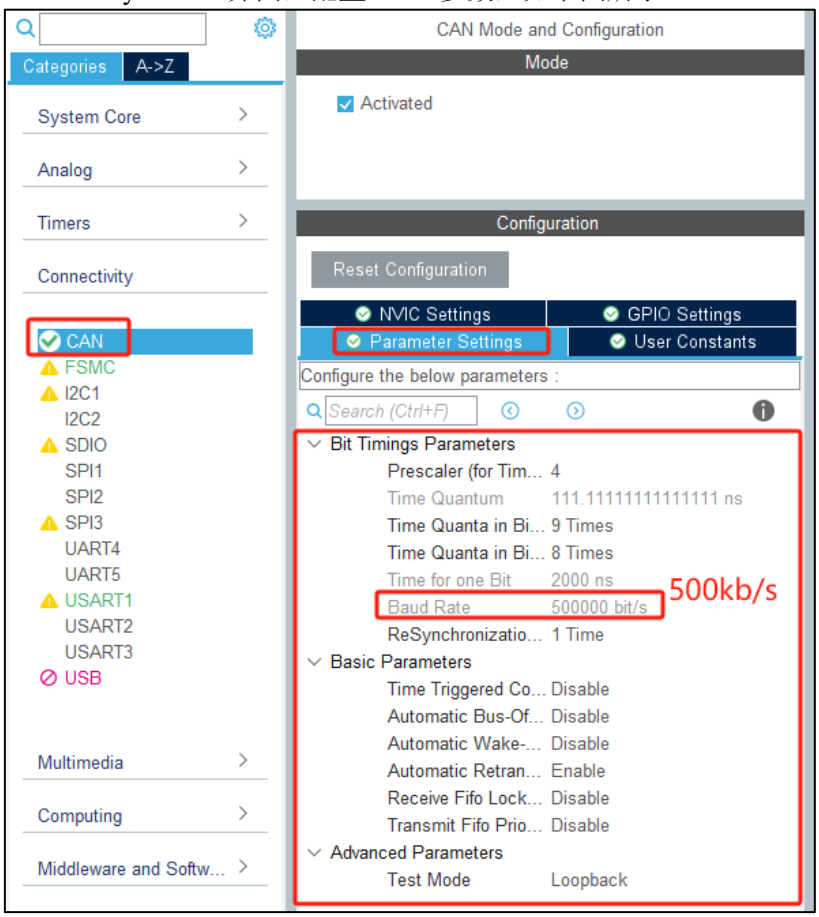


图 47.2.2.3 CAN 参数配置

通过该配置可设置 CAN 的通信波特率，其波特率公式如下：

```
波特率 = Fpclk1 / ((tbs1 + tbs2 + 1) * brp);
```

经过上述配置后，波特率 = 36M/((9 + 8 + 1) * 4) = 500kbps，然后通信模式默认配置为环回模式（自发自收）。

47.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 can.c 文件。can.c 主要是完成 CAN 的初始化。

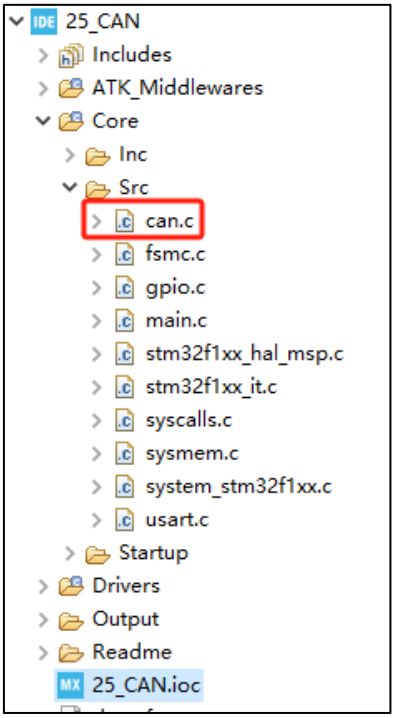


图 47.2.3.1 生成工程

47.2.4 添加用户驱动代码

在本章实验中，我们主要修改以下几个文件，分别是：can.c、can.h、main.c。其中 can.h 文件中，仅添加了在 can.c 中实现的一些功能函数的声明，这里不过多赘述，我们先来看下 can.c 文件。

（1）can.c 文件代码

```
void MX_CAN_Init(void)
{
  hcan.Instance = CAN1;
  hcan.Init.Prescaler = 4;
  hcan.Init.Mode = CAN_MODE_LOOPBACK;
  hcan.Init.SyncJumpWidth = CAN_SJW_1TQ;
  hcan.Init.TimeSeg1 = CAN_BS1_9TQ;
  hcan.Init.TimeSeg2 = CAN_BS2_8TQ;
  hcan.Init.TimeTriggeredMode = DISABLE;
  hcan.Init.AutoBusOff = DISABLE;
  hcan.Init.AutoWakeUp = DISABLE;
  hcan.Init.AutoRetransmission = ENABLE;
  hcan.Init.ReceiveFifoLocked = DISABLE;
  hcan.Init.TransmitFifoPriority = DISABLE;
  if (HAL_CAN_Init(&hcan) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN CAN_Init 2 */
  /*配置 CAN 过滤器*/
  sFilterConfig.FilterBank = 0;                       /* 过滤器 0 */
```

```
  sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;          /* 标识符屏蔽位模式
*/
  sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;         /* 长度 32 位位宽*/
  sFilterConfig.FilterIdHigh = 0x0000;                       /* 32 位 ID */
  sFilterConfig.FilterIdLow = 0x0000;
  sFilterConfig.FilterMaskIdHigh = 0x0000;                   /* 32 位 MASK */
  sFilterConfig.FilterMaskIdLow = 0x0000;
  sFilterConfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;/* 过滤器 0 关联到 FIFO0*/
  sFilterConfig.FilterActivation = CAN_FILTER_ENABLE;    /* 激活滤波器 0 */
  sFilterConfig.SlaveStartFilterBank = 14;

  /* 过滤器配置 */
  if (HAL_CAN_ConfigFilter(&hcan, &sFilterConfig) != HAL_OK)
  {

  }

  /* 启动 CAN 外围设备 */
  if (HAL_CAN_Start(&hcan) != HAL_OK)
  {

  }
  /* USER CODE END CAN_Init 2 */

}

/* 省略部分代码… … */

/* USER CODE BEGIN 1 */
/**
 * @brief       CAN 模式设置
 * @note        模式选择：环回模式、普通模式
 * @param       mode     : CAN_MODE_NORMAL,  正常模式;
 *                         CAN_MODE_LOOPBACK,回环模式;
 * @retval      无
 */
void can_set_mode(uint32_t mode)
{
    hcan.Instance = CAN1;
    hcan.Init.Prescaler = 4;
    hcan.Init.Mode = mode;
    hcan.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan.Init.TimeSeg1 = CAN_BS1_9TQ;
    hcan.Init.TimeSeg2 = CAN_BS2_8TQ;
    hcan.Init.TimeTriggeredMode = DISABLE;
    hcan.Init.AutoBusOff = DISABLE;
    hcan.Init.AutoWakeUp = DISABLE;
    hcan.Init.AutoRetransmission = ENABLE;
    hcan.Init.ReceiveFifoLocked = DISABLE;
    hcan.Init.TransmitFifoPriority = DISABLE;
    if (HAL_CAN_Init(&hcan) != HAL_OK)
    {
        Error_Handler();
    }
    /*配置 CAN 过滤器*/
    sFilterConfig.FilterBank = 0;                            /* 过滤器 0 */
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;        /* 标识符屏蔽位模式*/
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;       /* 长度 32 位位宽*/
    sFilterConfig.FilterIdHigh = 0x0000;                     /* 32 位 ID */
    sFilterConfig.FilterIdLow = 0x0000;
    sFilterConfig.FilterMaskIdHigh = 0x0000;                 /* 32 位 MASK */
    sFilterConfig.FilterMaskIdLow = 0x0000;
    sFilterConfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;   /* 关联到 FIFO0 */
```

```
    sFilterConfig.FilterActivation = CAN_FILTER_ENABLE;      /* 激活滤波器 0 */
    sFilterConfig.SlaveStartFilterBank = 14;

    /* 过滤器配置 */
    if (HAL_CAN_ConfigFilter(&hcan, &sFilterConfig) != HAL_OK)
    {

    }

    /* 启动 CAN 外围设备 */
    if (HAL_CAN_Start(&hcan) != HAL_OK)
    {

    }
}


/**
 * @brief       CAN 发送一组数据
 *   @note      发送格式固定为: 标准 ID, 数据帧
 * @param       id      : 标准 ID(11 位)
 * @retval      发送状态 0, 成功; 1, 失败;
 */
uint8_t can_send_msg(uint32_t id, uint8_t *msg, uint8_t len)
{
  uint32_t TxMailbox = CAN_TX_MAILBOX0;

  g_canx_txheader.StdId = id;          /* 标准标识符 */
  g_canx_txheader.ExtId = id;          /* 扩展标识符(29 位) 标准标识符情况下，该成员无
效*/
  g_canx_txheader.IDE = CAN_ID_STD;    /* 使用标准标识符 */
  g_canx_txheader.RTR = CAN_RTR_DATA; /* 数据帧 */
  g_canx_txheader.DLC = len;

  if (HAL_CAN_AddTxMessage(&hcan, &g_canx_txheader, msg, &TxMailbox) !=
HAL_OK) /* 发送消息 */
  {
    return 1;
  }
  /* 等待发送完成,所有邮箱(有三个邮箱)为空 */
  while (HAL_CAN_GetTxMailboxesFreeLevel(&hcan) != 3);

  return 0;
}

/**
 * @brief       CAN 接收数据查询
 *   @note      接收数据格式固定为: 标准 ID, 数据帧
 * @param       id      : 要查询的 标准 ID(11 位)
 * @param       buf     : 数据缓存区
 * @retval      接收结果
 *   @arg       0    , 无数据被接收到;
 *   @arg       其他, 接收的数据长度
 */
uint8_t can_receive_msg(uint32_t id, uint8_t *buf)
{
  if (HAL_CAN_GetRxFifoFillLevel(&hcan, CAN_RX_FIFO0) == 0)/* 没有接收到数据 */
  {
    return 0;
  }

  if (HAL_CAN_GetRxMessage(&hcan, CAN_RX_FIFO0, &g_canx_rxheader, buf) !=
HAL_OK)  /* 读取数据 */
```

```
  {
    return 0;
  }

  if (g_canx_rxheader.StdId!= id || g_canx_rxheader.IDE != CAN_ID_STD ||
g_canx_rxheader.RTR != CAN_RTR_DATA)/* 接收到的 ID 不对 / 不是标准帧 / 不是数据帧 */
  {
    return 0;
  }

  return g_canx_rxheader.DLC;
}
/* USER CODE END 1 */
```

在 CAN 的初始化函数中，我们添加的部分为：配置过滤器以使能硬件自动过滤功能，最后使能 CAN 以开始 CAN 控制器的工作。

can_set_mode 该函数用于设置 CAN 的通信模式，支持环回模式和普通模式切换。并且我们还实现了另外两个 CAN 的收发函数，分别是：can_send_msg 和 can_receive_msg。

（2）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint8_t i = 0, t = 0;
    uint8_t cnt = 0;
    uint8_t canbuf[8];
    uint8_t rxlen = 0;
    uint8_t res;
    uint8_t mode = 1; /* CAN 工作模式: 0,正常模式; 1,回环模式 */
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* CAN 初始化 */
  MX_CAN_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                                    /* 初始化 LCD */
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "CAN TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "LoopBack Mode", RED);
  lcd_show_string(30, 130, 200, 16, 16, "KEY0:Send KEK_UP:Mode", RED);

  lcd_show_string(30, 150, 200, 16, 16, "Count:", RED);      /* 显示当前计数值 */
  lcd_show_string(30, 170, 200, 16, 16, "Send Data:", RED); /* 提示发送的数据 */
  lcd_show_string(30, 230, 200, 16, 16, "Receive Data:", RED);
  /* USER CODE END 2 */
```

```
while (1)
{
    key = key_scan(0);

    if (key == KEY0_PRES)              /* KEY0 按下,发送一次数据 */
    {
        for (i = 0; i < 8; i++)
        {
            canbuf[i] = cnt + i;       /* 填充发送缓冲区 */

            if (i < 4)
            {
                lcd_show_xnum(30 + i * 32, 190, canbuf[i], 3, 16, 0X80,
                BLUE);                 /* 显示数据 */
            }
            else
            {
                lcd_show_xnum(30 + (i - 4) * 32, 210, canbuf[i], 3, 16,
                0X80, BLUE);           /* 显示数据 */
            }
        }

        res = can_send_msg(0X12, canbuf, 8); /* ID = 0X12, 发送 8 个字节 */

        if (res)
        {
            lcd_show_string(30 + 80, 170, 200, 16, 16, "Failed", BLUE);
        }
        else
        {
            lcd_show_string(30 + 80, 170, 200, 16, 16, "OK    ", BLUE);
        }
    }
    else if (key == WKUP_PRES)         /* WK_UP 按下，改变 CAN 的工作模式 */
    {
        mode = !mode;

        if (mode == 0)                 /* 正常模式，需要 2 个开发板 */
        {
            /* CAN 正常模式初始化， 正常模式， 波特率 500Kbps */
            can_set_mode(CAN_MODE_NORMAL);
            lcd_show_string(30, 110, 200, 16, 16, "Nnormal Mode ", RED);
        }
        else /* 回环模式,一个开发板就可以测试了. */
        {
            /* CAN 回环模式初始化， 回环模式， 波特率 500Kbps */
            can_set_mode(CAN_MODE_LOOPBACK);
            lcd_show_string(30, 110, 200, 16, 16, "LoopBack Mode", RED);
        }
    }

    rxlen = can_receive_msg(0X12, canbuf);   /* CAN ID = 0X12, 接收数据查询 */

    if (rxlen)                                /* 接收到有数据 */
    {
        lcd_fill(30, 270, 130, 310, WHITE);  /* 清除之前的显示 */

        for (i = 0; i < rxlen; i++)
        {
            if (i < 4)
            {
                lcd_show_xnum(30 + i * 32, 250, canbuf[i], 3, 16, 0X80,
                BLUE);              /* 显示数据 */
```

```
                }
                else
                {
                    lcd_show_xnum(30 + (i - 4) * 32, 270, canbuf[i], 3, 16,
                    0X80, BLUE);     /* 显示数据 */
                }
            }
        }

        t++;
        delay_ms(10);

        if (t == 20)
        {
            LED0_TOGGLE();           /* 提示系统正在运行 */
            t = 0;
            cnt++;
            lcd_show_xnum(30 + 48, 150, cnt, 3, 16, 0X80, BLUE); /* 显示数据 */
        }
    }
}
```

main 函数的执行过程比较简单通过按键控制通信模式以及发送数据等。这里需要注意：在选择正常模式的情况下，要使两个开发板通信成功，必须保持一致的波特率。其它细节，参考光盘资料中的源码即可。

47.3 下载验证

在代码编译成功之后，我们通过下载代码到开发板上，得到如图 47.3.1 所示：

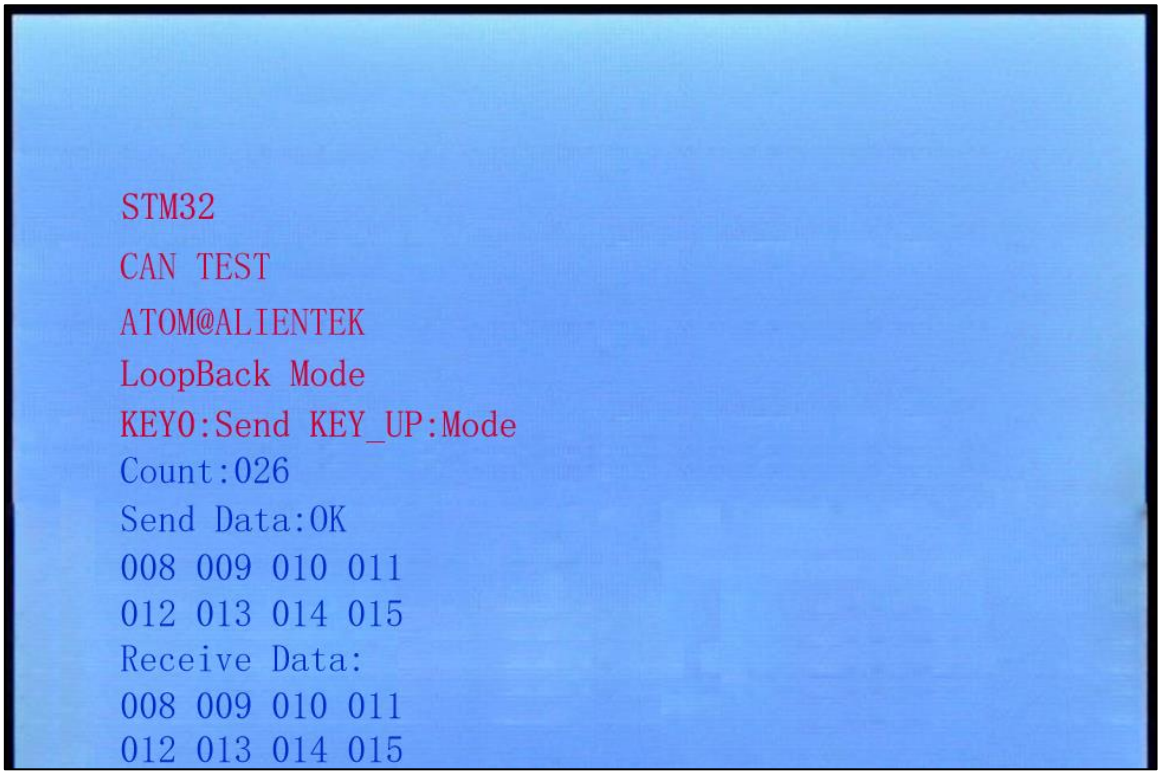


图 47.3.1 程序运行效果图

伴随 LED0 的不停闪烁，提示程序在运行。默认设置的回环模式，按下 KEY0 就可以在 LCD 模块上面看到自发自收的数据（如上图所示），如果我们选择正常模式（KEY_UP 按键切换），就必须连接两个开发板的 CAN 接口，然后就可以互发数据了。如图 47.3.2 和图 47.3.3 所示：

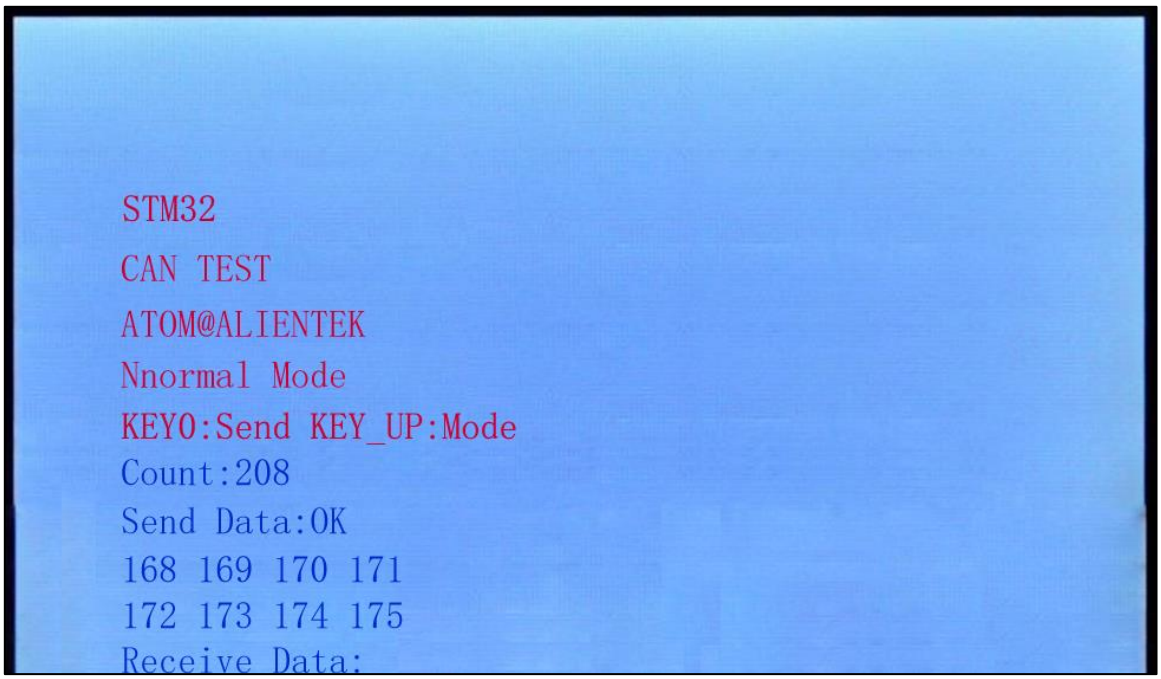


图 47.3.2 CAN 正常模式发送数据

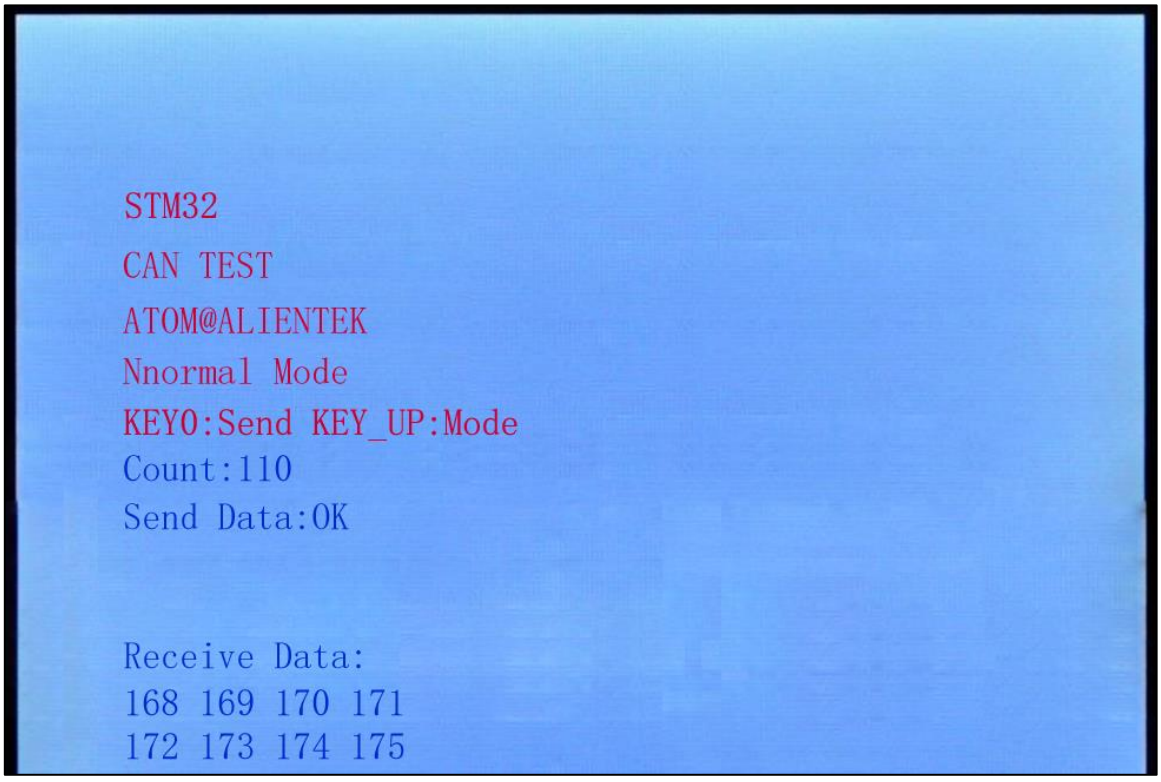


图 47.3.3 CAN 正常模式接收数据

图 47.3.2 来自开发板 A，发送了 8 个数据，图 47.3.3 来自开发板 B，收到了来自开发板 A 的个数据。

第四十八章 触摸屏实验

本章，我们将介绍如何使用 STM32F1 来驱动触摸屏，正点原子精英 STM32F103 本身并没有触摸屏控制器，但是它支持触摸屏，可以通过外接带触摸屏的 LCD 模块（比如正点原子 TFTLCD 模块），来实现触摸屏控制。在本章中，我们将向大家介绍 STM32 控制正点原子 TFTLCD 模块（包括电阻触摸与电容触摸），实现触摸屏驱动，最终实现一个手写板的功能。

本章分为如下几个部分：

48.1 硬件设计
48.2 程序设计
48.3 下载验证

48.1 硬件设计

1. 例程功能

1. 程序运行后，先显示实验信息，随后在 LCD 上绘制出触摸屏被触摸的轨迹
2. 可通过触摸屏幕右上角的“RST”来清空屏幕
3. 使用电阻式触摸屏的时候，可按下 KEY0 按键，进行触摸校准
4. LED0 闪烁，指示程序正在运行

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
KEY0 - PE4
3）EEPROM AT24C02
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3. 原理图

所有这些资源与 STM32F1 的连接图，在前面都已经介绍了，这里我们只针对 TFTLCD 模块与 STM32F1 的连接端口再说明一下，TFTLCD 模块的触摸屏（电阻触摸屏）总共有 5 根线与 STM32F1 连接，连接电路图如图 48.1.1 所示：

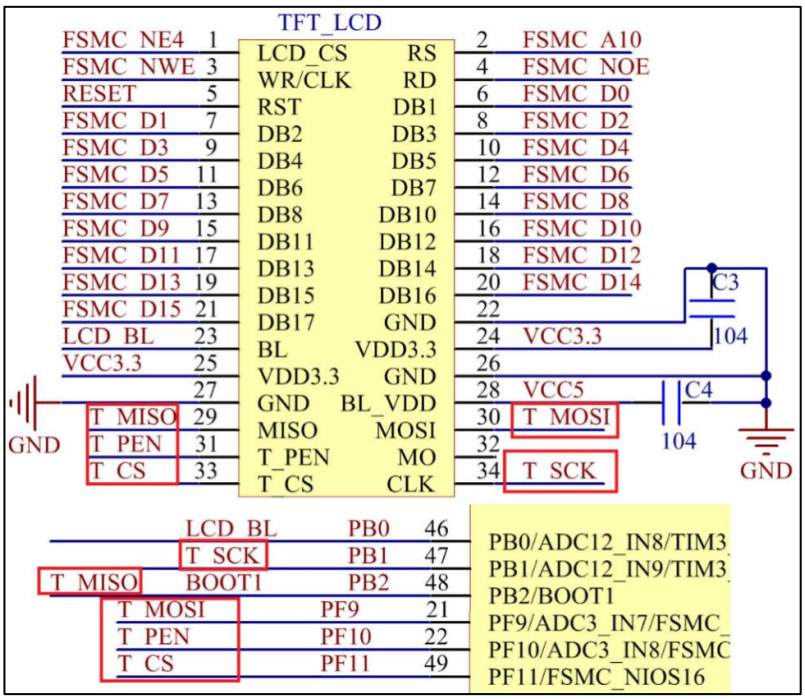


图 48.1.1 触摸屏与 STM32F1 的连接图

从图中可以看出,T_SCK、T_MISO、T_MOSI、T_PEN 和 T_CS 分别连接在 STM32F1 的：PB1、PB2、PF9、PF10 和 PF11 上。

如果是电容式触摸屏，我们的接口和电阻式触摸屏一样（上图右侧接口），只是没有用到五根线了，而是四根线，分别是：T_PEN(CT_INT)、T_CS(CT_RST)、T_CLK(CT_SCL)和 T_MOSI(CT_SDA)。其中：CT_INT、CT_RST、CT_SCL 和 CT_SDA 分别是 GT9147/FT5206 的：中断输出信号、复位信号，IIC 的 SCL 和 SDA 信号。我们用查询的方式读取 GT9147/FT5206 的数据，对于 FT5206 没有用到中断信号（CT_INT），所以同 STM32F1 的连接，最少只需要 3 根线即可，不过 GT9147 等 IC 还需要用到 CT_INT 做 IIC 地址设定，所以需要 4 根线连接。

48.2 程序设计

48.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 26_TOUCH。

48.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 26_TOUCH，以进行下一步的实验操作。如下图所示：

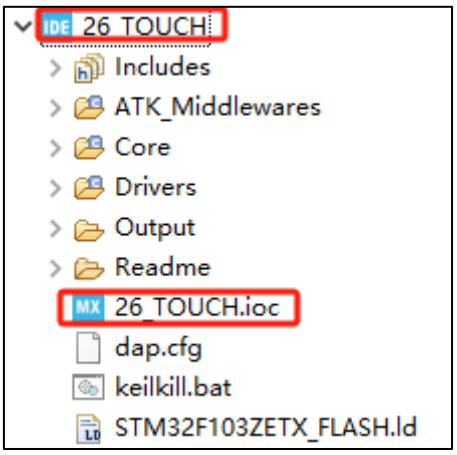


图 48.2.2.1 TOUCH 工程

本实验所涉及到触摸屏部分的 IO 我们不在软件上直接配置，主要为了兼容电阻屏和电容屏的驱动，所以触摸相关部分的 IO，本章在代码中自己实现。

48.2.3 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 TOUCH 文件夹，在 TOUCH 中，我们主要添加了触摸屏的驱动代码，包括 touch.c、touch.h、ctiic.c、ctiic.h、ft5206.c、ft5206.h、gt9xxx.c、gt9xxx.h 八个文件。

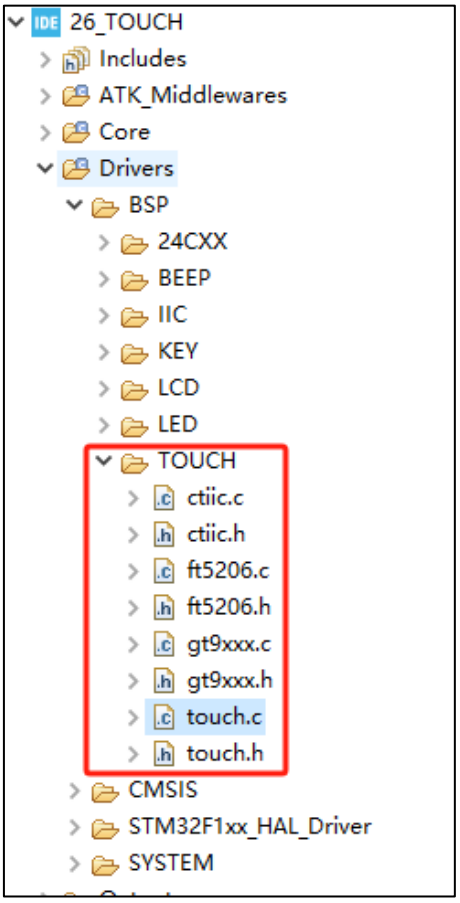


图 48.2.3.1 添加用户文件

（1）touch.c 文件代码

触摸屏的驱动中，我们主要看下在 touch.c 中的触摸屏的初始化函数，如下图所示：

```
/**
 * @brief       触摸屏初始化
 * @param       无
 * @retval      0,没有进行校准
 *              1,进行过校准
 */
uint8_t tp_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    tp_dev.touchtype = 0;                       /* 默认设置(电阻屏 & 竖屏) */
    tp_dev.touchtype |= lcddev.dir & 0X01;  /* 根据 LCD 判定是横屏还是竖屏 */

    if (lcddev.id == 0x7796)
    {
        /* 初始化 GT 系列触摸屏成功,即当前 3.5 寸屏为电容触摸屏 */
        if (gt9xxx_init() == 0)
        {
            tp_dev.scan = gt9xxx_scan;      /* 扫描函数指向 GT9147 触摸屏扫描 */
            tp_dev.touchtype |= 0X80;       /* 电容屏 */
            return 0;
        }
    }

    if (lcddev.id == 0X5510 || lcddev.id == 0X9806 || lcddev.id == 0X4342 ||
lcddev.id == 0X4384 || lcddev.id == 0X1018)  /* 电容触摸屏,4.3 寸/10.1 寸屏 */
    {
        gt9xxx_init();
        tp_dev.scan = gt9xxx_scan;      /* 扫描函数指向 GT9147 触摸屏扫描 */
        tp_dev.touchtype |= 0X80;       /* 电容屏 */
        return 0;
    }
    else if (lcddev.id == 0X1963 || lcddev.id == 0X7084 || lcddev.id ==
0X7016)     /* SSD1963 7 寸屏或者 7 寸 800*480/1024*600 RGB 屏 */
    {
        if (!ft5206_init())
        {
            tp_dev.scan = ft5206_scan;  /* 扫描函数指向 FT5206 触摸屏扫描 */
        }
        else
        {
            gt9xxx_init();
            tp_dev.scan = gt9xxx_scan;  /* 扫描函数指向 GT9147 触摸屏扫描 */
        }
        tp_dev.touchtype |= 0X80;       /* 电容屏 */
        return 0;
    }
    else
    {
        T_PEN_GPIO_CLK_ENABLE();    /* T_PEN 脚时钟使能 */
        T_CS_GPIO_CLK_ENABLE();     /* T_CS 脚时钟使能 */
        T_MISO_GPIO_CLK_ENABLE();   /* T_MISO 脚时钟使能 */
        T_MOSI_GPIO_CLK_ENABLE();   /* T_MOSI 脚时钟使能 */
        T_CLK_GPIO_CLK_ENABLE();    /* T_CLK 脚时钟使能 */

        gpio_init_struct.Pin = T_PEN_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_INPUT;                /* 输入 */
        gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;          /* 高速 */
```

```
        HAL_GPIO_Init(T_PEN_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = T_MISO_GPIO_PIN;
        HAL_GPIO_Init(T_MISO_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = T_MOSI_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;              /* 推挽输出 */
        gpio_init_struct.Pull = GPIO_PULLUP;                      /* 上拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;            /* 高速 */
        HAL_GPIO_Init(T_MOSI_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = T_CLK_GPIO_PIN;
        HAL_GPIO_Init(T_CLK_GPIO_PORT, &gpio_init_struct);
        gpio_init_struct.Pin = T_CS_GPIO_PIN;
        HAL_GPIO_Init(T_CS_GPIO_PORT, &gpio_init_struct);

        tp_read_xy(&tp_dev.x[0], &tp_dev.y[0]);    /* 第一次读取初始化 */
        at24cxx_init();                            /* 初始化 24CXX */

        if (tp_get_adjust_data())
        {
            return 0;           /* 已经校准 */
        }
        else                    /* 未校准? */
        {
            lcd_clear(WHITE);   /* 清屏 */
            tp_adjust();        /* 屏幕校准 */
            tp_save_adjust_data();
        }

        tp_get_adjust_data();
    }

    return 1;
}
```

从上面的代码中可以看出，触摸屏的初始化会读取 TFTLCD 模块 LCD 的 ID 号，来判断触摸屏的型号，因此在使用本触摸屏驱动初始化触摸屏前，需要先进行 LCD 的初始化。触摸屏的初始化函数会依据 LCD 的 ID 对不同型号的触摸屏进行初始化。
对于触摸屏驱动中的其他内容，请读者自行结合实际使用的 TFTLCD 的用户手册产看本章配套实验例程的源码。

（2）main.c 文件代码

```
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
```

```
    lcd_init();                                 /* 初始化 LCD */
    tp_dev.init();                              /* 触摸屏初始化 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "TOUCH TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    if (tp_dev.touchtype != 0XFF)
    {
        lcd_show_string(30, 110, 200, 16, 16, "Press KEY0 to Adjust", RED);
    }

    delay_ms(1500);

    load_draw_dialog();

    if (tp_dev.touchtype & 0X80)
    {
        ctp_test(); /* 电容屏测试 */
    }
    else
    {
        rtp_test(); /* 电阻屏测试 */
    }
    /* USER CODE END 2 */

    while (1)
    {
        delay_ms(10);
    }
}
```

在进行完触摸屏的初始化后，便根据触摸屏的不同类型调用了不同的测试函数，这是因为电容屏是支持多点触控的，而电阻屏并不支持。

电容屏测试测试函数，如下所示：

```
/**
 * @brief       电容触摸屏测试函数
 * @param       无
 * @retval      无
 */
void ctp_test(void)
{
    uint8_t t = 0;
    uint8_t i = 0;
    uint16_t lastpos[10][2];         /* 最后一次的数据 */
    uint8_t maxp = 5;

    if (lcddev.id == 0X1018)maxp = 10;

    while (1)
    {
        tp_dev.scan(0);

        for (t = 0; t < maxp; t++)
        {
            if ((tp_dev.sta) & (1 << t))
            {
                if (tp_dev.x[t] < lcddev.width && tp_dev.y[t] < lcddev.height)
                {
                    if (lastpos[t][0] == 0XFFFF)
                    {
                        lastpos[t][0] = tp_dev.x[t];
                        lastpos[t][1] = tp_dev.y[t];
                    }

                    lcd_draw_bline(lastpos[t][0], lastpos[t][1], tp_dev.x[t],
```

```
                    tp_dev.y[t], 2, POINT_COLOR_TBL[t]); /* 画线 */
                    lastpos[t][0] = tp_dev.x[t];
                    lastpos[t][1] = tp_dev.y[t];

                    if (tp_dev.x[t] > (lcddev.width - 24) && tp_dev.y[t] < 20)
                    {
                        load_draw_dialog();/* 清除 */
                    }
                }
            }
            else
            {
                lastpos[t][0] = 0XFFFF;
            }
        }

        delay_ms(5);
        i++;

        if (i % 20 == 0)LED0_TOGGLE();
    }
}
```

电阻触摸屏就相对简单，因为电阻触摸屏仅支持单点触控，因此仅需将触摸的 1 个触摸点对应 LCD 屏幕上坐标的点进行绘制即可，同时因为电阻触摸屏是需要校准的，因此当检测到 KEY0 按键被按下时，便会进行电阻触摸屏校准。

48.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的实验信息，随后便进入“白板”界面，此时便可在 LCD 上通过触摸屏绘制出任意的图案，若是电容屏，还支持多点触控，若是电阻屏出现触摸点与 LCD 上显示的绘制点坐标不吻合，可以按下 KEY0 按键进行电阻触摸屏的校准。

第四十九章 红外遥控实验

本章，我们将介绍 STM32F103 对红外遥控器的信号解码。STM32 板子上标配的红外接收头和一个小巧的红外遥控器。我们将利用 STM32 的输入捕获功能，解码开发板标配的红外遥控器的编码信号，并将编码后的键值在 LCD 模块中显示出来。

本章分为如下几个小节：

49.1 硬件设计
49.2 程序设计
49.3 下载验证

49.1 硬件设计

1. 例程功能

本实验开机在 LCD 上显示一些信息之后，即进入等待红外触发，如果接收到正确的红外信号，则解码，并在 LCD 上显示键值和所代表的意义，以及按键次数等信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）红外接收头
REMOTE_IN - PB9
3）正点原子红外遥控器
4）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
5）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

红外遥控接收头与 STM32 的连接关系，如下图所示：

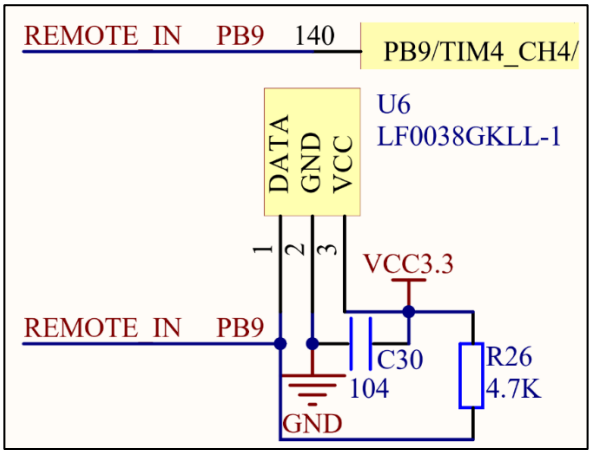


图 49.1.1 红外遥控接收头与 STM32 的连接电路图

红外遥控接收头连接在 STM32 的 PB9(TIM4_CH4)上。硬件上不需要变动，只需要程序将 TIM4_CH4 设计输入捕获，然后将接收到的脉冲信号解码就可以了。

开发板配套的红外遥控器外观如图 49.1.2 所示。

图 49.1.2 红外遥控器

开发板上接收红外遥控器信号的红外管外观如图 49.1.3 所示。使用时需要遥控器有红外管的一端对准开发板上的红外管才能正确收到信号。

图 49.1.3 开发板上的红外接收管位置

49.2 程序设计

49.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 27_REMOTE。

49.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 27_REMOTE，以进行下一步的实验操作。如下图所示：

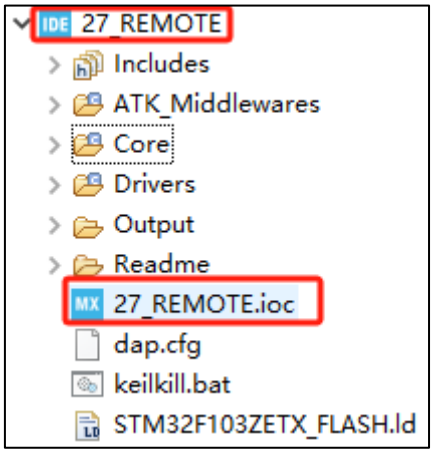


图 49.2.2.1 REMOTE 工程

（1）TIM4 配置

双击 27_REMOTE.ioc 进入 STM32CubeMX 插件的，在 Pinout & Configuration 处我们配置 TIM4 通道 4 的相关 IO，如下图所示

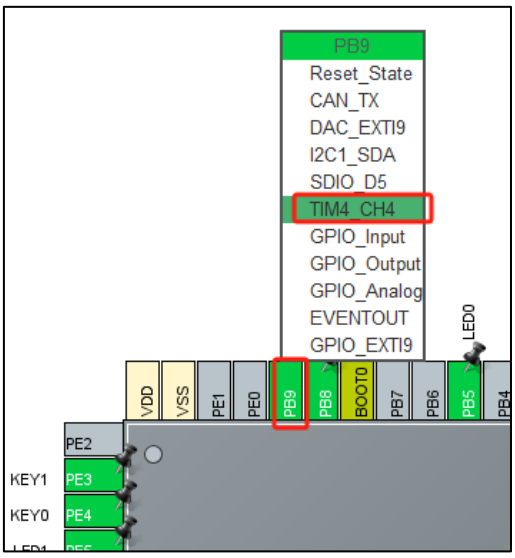


图 49.2.2.2 GPIO 复用

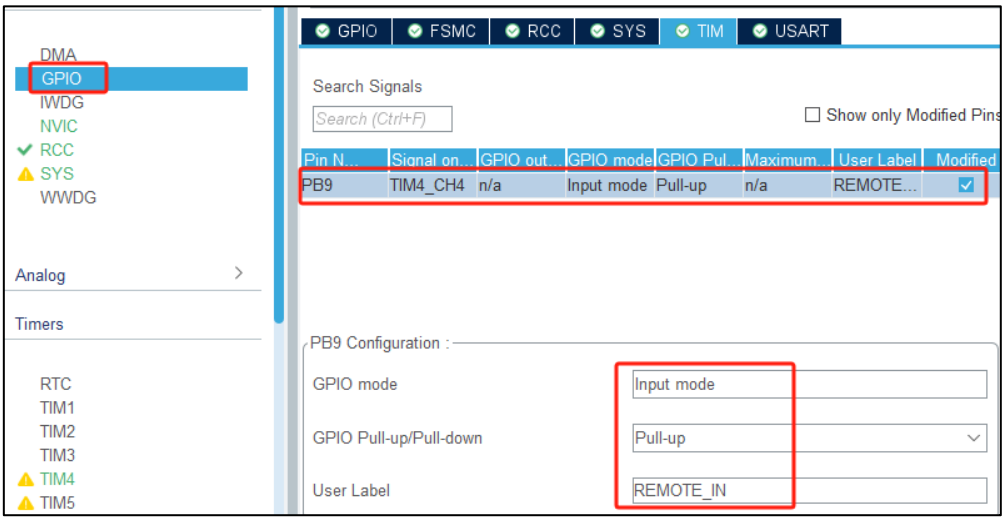


图 49.2.2.3 GPIO 参数配置

接着在 Timers->TIM4 界面，配置 TIM4 参数，如下图所示：

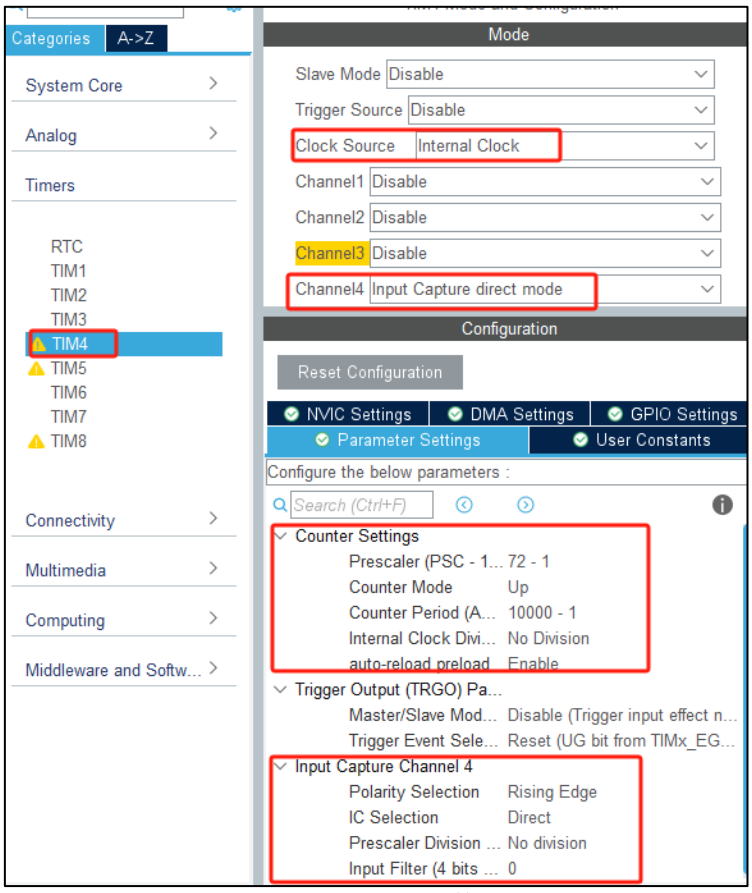


图 49.2.2.4 TIM4 参数配置

关于定时器的配置参数，我们在前面的定时器章节已经介绍过多遍，这里不在赘述。这里我们配置 TIM4 的通道 4 为输入捕获模式，并设置上升沿捕获。

（2）NVIC 配置

由于本实验需用到定时器的输入捕获中断，所以要配置 NVIC，首先开启定时器全局中断，配置定时器中断优先级分组为 2，抢占优先级 2，子优先级 1，如下图所示：

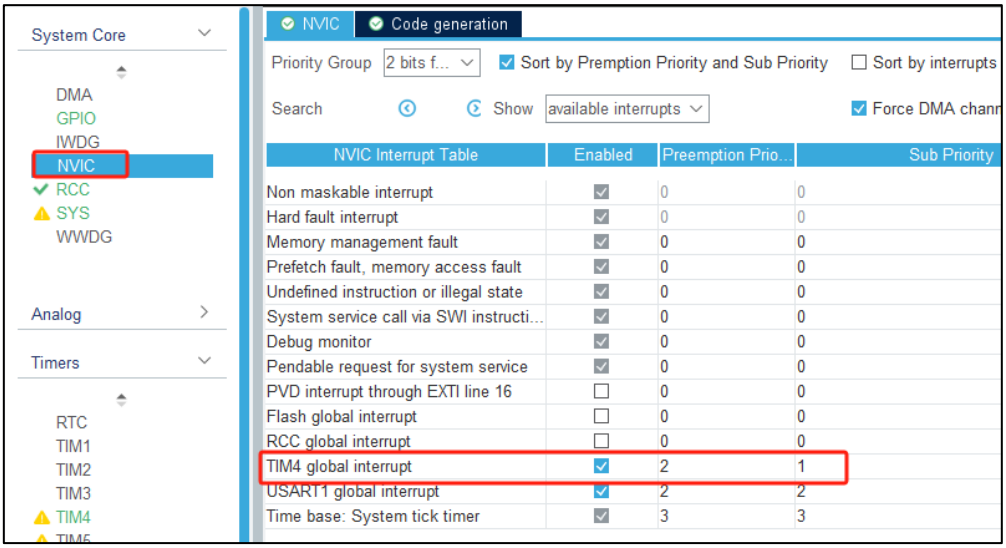


图 49.2.2.5 NVIC 配置

49.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 tim.c 文件。tim.c 主要是完成 TIM4 通道 4 的初始化。

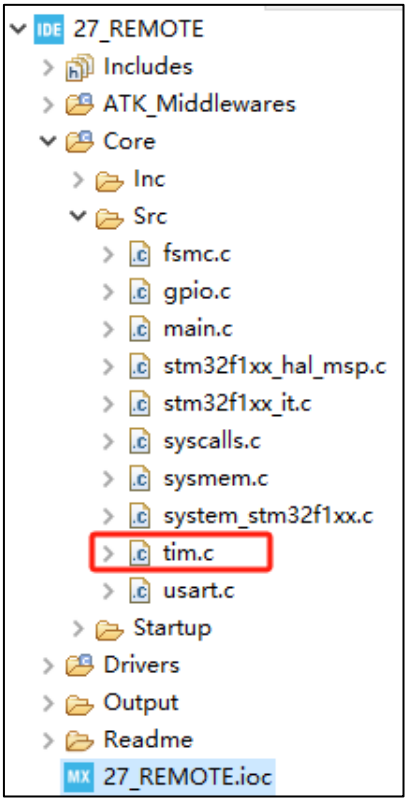


图 49.2.3.1 生成工程

49.2.4 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 REMOTE 文件夹，然后再 REMOTE 文件夹下新建两个文件，分别是 remote.c 和 remote.h，如下图所示：

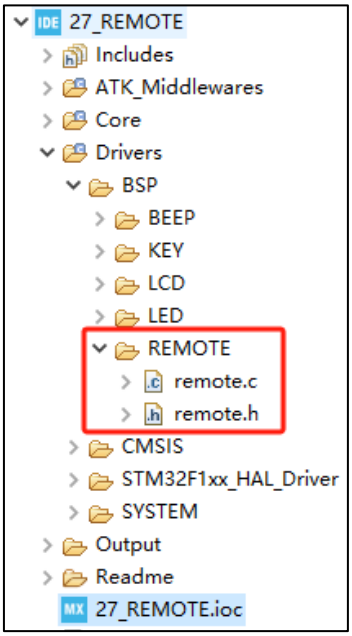


图 49.2.4.1 添加用户文件

本实验主要修改以下几个文件，分别是：tim.c、remote.h、remote.c、main.c。下面我们先看下 tim.c 中。

（1）tim.c 文件代码

```
void MX_TIM4_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_IC_InitTypeDef sConfigIC = {0};

  htim4.Instance = TIM4;
  htim4.Init.Prescaler = 72 - 1;
  htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim4.Init.Period = 10000 - 1;
  htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
  if (HAL_TIM_Base_Init(&htim4) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim4, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_IC_Init(&htim4) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
  sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
  sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
  sConfigIC.ICFilter = 0;
  if (HAL_TIM_IC_ConfigChannel(&htim4, &sConfigIC, TIM_CHANNEL_4) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM4_Init 2 */
  HAL_TIM_IC_Start_IT(&htim4, TIM_CHANNEL_4);        /* 开始捕获 TIM 的通道值 */
  __HAL_TIM_ENABLE_IT(&htim4, TIM_IT_UPDATE);        /* 使能更新中断 */
  /* USER CODE END TIM4_Init 2 */

}
```

在 TIM4 的初始化函数中，我们添加了函数 HAL_TIM_IC_Start_IT，用于开启并使能 TIM4 通道 4 的输入捕获中断；函数__HAL_TIM_ENABLE_IT，用于使能定时器的更新中断。

（2）remote.h 文件代码

```
#ifndef __REMOTE_H
#define __REMOTE_H

#include "../../SYSTEM/sys/sys.h"
#include "tim.h"

/******************************************************************************/
/* 红外输入引脚及定时器 定义 */
```

```
#define REMOTE_IN_TIMX                          TIM4
#define REMOTE_IN_TIMX_IRQn                     TIM4_IRQn
#define REMOTE_IN_TIMX_CHY                      TIM_CHANNEL_4
/* 通道 Y,  1<= Y <=2*/
#define REMOTE_IN_TIMX_CCRY                     REMOTE_IN_TIMX->CCR4
#define REMOTE_IN_TIMX_CHY_CLK_ENABLE()
do{ __HAL_RCC_TIM4_CLK_ENABLE(); }while(0)       /* TIMX 时钟使能 */

/******************************************************************************/


#define RDATA           HAL_GPIO_ReadPin(REMOTE_IN_GPIO_Port, REMOTE_IN_Pin)
/* 红外数据输入脚 */


/* 红外遥控识别码(ID),每款遥控器的该值基本都不一样,但也有一样的.
 * 我们选用的遥控器识别码为 0
*/
#define REMOTE_ID       0

extern uint8_t g_remote_cnt;     /* 按键按下的次数 */

uint8_t remote_scan(void);

#endif
```

该文件采用宏定义的方式，定义了定时器相关的一些内容，如通道和中断线等。同时，还使用宏定义的方式，定义了红外数据输入引脚。

（3）remote.c 文件代码

```
/* 遥控器接收状态
 * [7]  : 收到了引导码标志
 * [6]  : 得到了一个按键的所有信息
 * [5]  : 保留
 * [4]  : 标记上升沿是否已经被捕获
 * [3:0]: 溢出计时器
 */
uint8_t g_remote_sta = 0;
uint32_t g_remote_data = 0; /* 红外接收到的数据 */
uint8_t  g_remote_cnt = 0;  /* 按键按下的次数 */

/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == REMOTE_IN_TIMX)
    {
        if (g_remote_sta & 0x80)            /* 上次有数据被接收到了 */
        {
            g_remote_sta &= ~0X10;          /* 取消上升沿已经被捕获标记 */

            if ((g_remote_sta & 0X0F) == 0X00)
            {
                g_remote_sta |= 1 << 6;     /* 标记已经完成一次按键的键值信息采集 */
            }

            if ((g_remote_sta & 0X0F) < 14)
            {
                g_remote_sta++;
```

```
            }
            else
            {
                g_remote_sta &= ~(1 << 7);      /* 清空引导标识 */
                g_remote_sta &= 0XF0;           /* 清空计数器 */
            }
        }
    }
}

/**
 * @brief       定时器输入捕获中断回调函数
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == REMOTE_IN_TIMX)
    {
        uint16_t dval;  /* 下降沿时计数器的值 */

        if (RDATA)      /* 上升沿捕获 */
        {

__HAL_TIM_SET_CAPTUREPOLARITY(&htim4,REMOTE_IN_TIMX_CHY,TIM_INPUTCHANNELPOLARI
TY_FALLING);//CC4P=1 设置为下降沿捕获
            __HAL_TIM_SET_COUNTER(&htim4, 0);  /* 清空定时器值 */
            g_remote_sta |= 0X10;              /* 标记上升沿已经被捕获 */
        }
        else            /* 下降沿捕获 */
        {
            dval=HAL_TIM_ReadCapturedValue(&htim4, REMOTE_IN_TIMX_CHY);
                        /* 读取 CCR4 也可以清 CC4IF 标志位 */
            __HAL_TIM_SET_CAPTUREPOLARITY(&htim4, REMOTE_IN_TIMX_CHY,
             TIM_INPUTCHANNELPOLARITY_RISING); /* 配置 TIM4 通道 4 上升沿捕获 */

            if (g_remote_sta & 0X10)           /* 完成一次高电平捕获 */
            {
                if (g_remote_sta & 0X80)       /* 接收到了引导码 */
                {

                    if (dval > 300 && dval < 800)      /* 560 为标准值,560us */
                    {
                        g_remote_data >>= 1;                /* 右移一位 */
                        g_remote_data &= ~(0x80000000);     /* 接收到 0 */
                    }
                    /* 1680 为标准值,1680us */
                    else if (dval > 1400 && dval < 1800)
                    {
                        g_remote_data >>= 1;                /* 右移一位 */
                        g_remote_data |= 0x80000000;        /* 接收到 1 */
                    }
                    else if (dval > 2000 && dval < 3000)
                    {
                        g_remote_cnt++;                 /* 按键次数增加 1 次 */
                        g_remote_sta &= 0XF0;           /* 清空计时器 */
                    }
                }
                else if (dval > 4200 && dval < 4700)   /* 4500 为标准值 4.5ms */
                {
                    g_remote_sta |= 1 << 7;             /* 标记成功接收到了引导码*/
                    g_remote_cnt = 0;                   /* 清除按键次数计数器 */
```

```
                }
            }

            g_remote_sta&=~(1<<4);
        }
    }
}

/**
 * @brief       处理红外按键(类似按键扫描)
 * @param       无
 * @retval      0    , 没有任何按键按下
 *              其他, 按下的按键键值
 */
uint8_t remote_scan(void)
{
    uint8_t sta = 0;
    uint8_t t1, t2;

    if (g_remote_sta & (1 << 6))                /* 得到一个按键的所有信息了 */
    {
        t1 = g_remote_data;                     /* 得到地址码 */
        t2 = (g_remote_data >> 8) & 0xff;       /* 得到地址反码 */
        /* 检验遥控识别码(ID)及地址 */
        if ((t1 == (uint8_t)~t2) && t1 == REMOTE_ID)
        {
            t1 = (g_remote_data >> 16) & 0xff;
            t2 = (g_remote_data >> 24) & 0xff;

            if (t1 == (uint8_t)~t2)
            {
                sta = t1;                       /* 键值正确 */
            }
        }
         /* 按键数据错误/遥控已经没有按下了 */
        if ((sta == 0) || ((g_remote_sta & 0X80) == 0))
        {
            g_remote_sta &= ~(1 << 6);          /* 清除接收到有效按键标识 */
            g_remote_cnt = 0;                   /* 清除按键次数计数器 */
        }
    }

    return sta;
}
```

定时器更新中断回调函数主要是对标志位进行管理；输入捕获中断用于捕获其高电平的脉宽；remote_scan 函数是用来扫描解码结果的，相当于我们的按键扫描，输入捕获解码的红外数据，通过该函数传送给其他程序。

（4）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint8_t t = 0;
    char *str = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();
```

```
/* 延时初始化 */
delay_init(72);

/* GPIO 初始化 */
MX_GPIO_Init();

/* FSMC 初始化 */
MX_FSMC_Init();

/* 串口初始化 */
MX_USART1_UART_Init();

/* TIM4 初始化 */
MX_TIM4_Init();

/* USER CODE BEGIN 2 */
lcd_init();                                                 /* 初始化 LCD */
lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
lcd_show_string(30,  70, 200, 16, 16, "REMOTE TEST", RED);
lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
lcd_show_string(30, 110, 200, 16, 16, "KEYVAL:", RED);
lcd_show_string(30, 130, 200, 16, 16, "KEYCNT:", RED);
/* USER CODE END 2 */

while (1)
{
    key = remote_scan();

    if (key)
    {
        lcd_show_num(86, 110, key, 3, 16, BLUE);          /* 显示键值 */
        lcd_show_num(86, 130, g_remote_cnt, 3, 16, BLUE); /* 显示按键次数 */

        switch (key)
        {
            case 0:
                str = "ERROR";
                break;

            case 69:
                str = "POWER";
                break;

            case 70:
                str = "UP";
                break;

            case 64:
                str = "PLAY";
                break;

            case 71:
                str = "ALIENTEK";
                break;

            case 67:
                str = "RIGHT";
                break;

            case 68:
                str = "LEFT";
                break;

            case 7:
                str = "VOL-";
```

```
                break;

            case 21:
                str = "DOWN";
                break;

            case 9:
                str = "VOL+";
                break;

            case 22:
                str = "1";
                break;

            case 25:
                str = "2";
                break;

            case 13:
                str = "3";
                break;

            case 12:
                str = "4";
                break;

            case 24:
                str = "5";
                break;

            case 94:
                str = "6";
                break;

            case 8:
                str = "7";
                break;

            case 28:
                str = "8";
                break;

            case 90:
                str = "9";
                break;

            case 66:
                str = "0";
                break;

            case 74:
                str = "DELETE";
                break;
        }

        lcd_fill(86, 150, 116 + 8 * 8, 170 + 16, WHITE);
        lcd_show_string(86, 150, 200, 16, 16, str, BLUE);    /* 显示 SYMBOL */
    }
    else
    {
        delay_ms(10);
    }

    t++;

    if (t == 20)
```

```
        {
            t = 0;
            LED0_TOGGLE();  /* LED0 闪烁 */
        }
    }
}
```

main 函数代码比较简单，主要是通过 remote_scan 函数获得红外遥控输入的数据（控制码），然后显示在 LCD 上面。

49.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 49.3.1 所示：

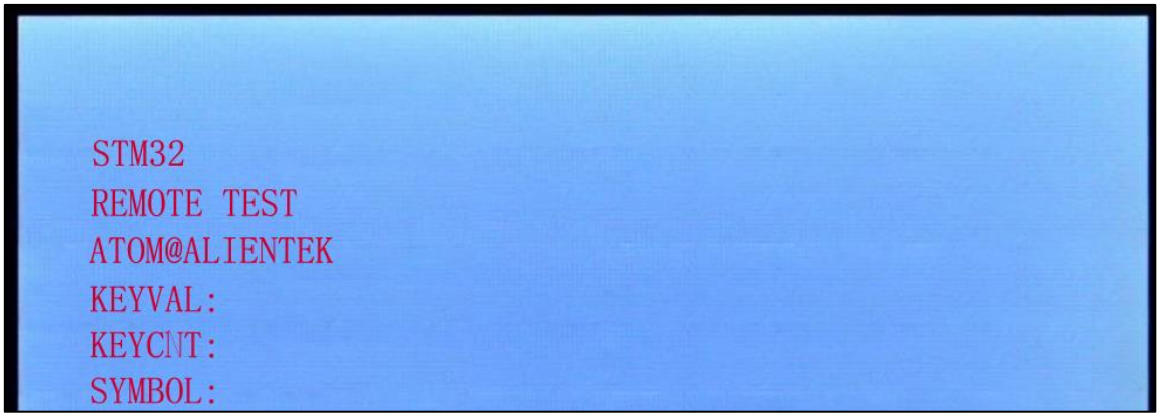


图 49.3.1 程序运行效果图

此时我们通过遥控器按下不同的按键，则可以看到 LCD 上显示了不同按键的键值以及按键次数和对应的遥控器上的符号。如图 49.3.2 所示。

图 49.3.2 解码成功

第五十章 DS18B20 数字温度传感器实验

本章，我们将介绍 STM32F103 如何读取外部温度传感器的温度，来得到较为准确的环境温度。我们将学习单总线技术，通过它来实现 STM32 和外部温度传感器 DS18B20 的通信，并把从温度传感器得到的温度显示在 LCD 上。

本章分为如下几个小节：

50.1 硬件设计
50.2 程序设计
50.3 下载验证

50.1 硬件设计

1. 例程功能

本实验开机的时候先检测是否有 DS18B20 存在，如果没有，则提示错误。只有在检测到 DS18B20 之后才开始读取温度并显示在 LCD 上，如果发现了 DS18B20，则程序每隔 100ms 左右读取一次数据，并把温度显示在 LCD 上。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
2）DS18B20 温度传感器 – PG11
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

DS18B20 接口与 STM32 的连接关系，如下图所示：

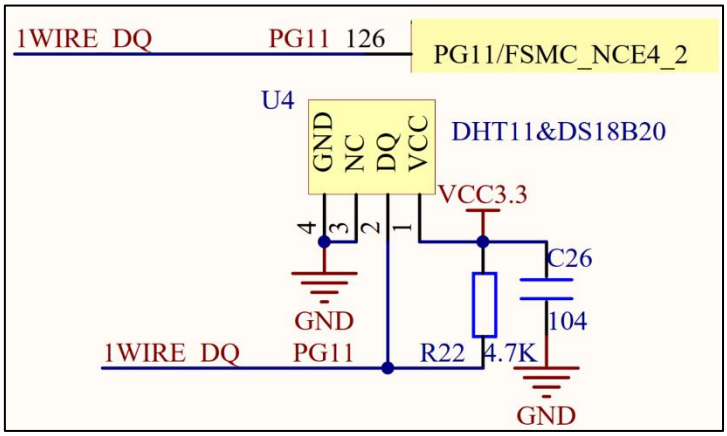


图 50.1.1 DS18B20 接口与 STM32 的连接电路图

从上图可以看出，我们使用得是 STM32 的 PG11 来连接 U4 的 DQ 引脚，图中 U4 为 DHT11（温湿度传感器）和 DS18B20 公用的一个接口，DHT11 我们将在下一章介绍。

DS18B20 只用到 U4 的 3 个引脚（U6 的 1、2 和 3 脚），将 DS18B20 传感器插入到这个上面（靠左插）就可以通过 STM32 来读取 DS18B20 的温度了。连接示意图如图 50.1.2 所示：

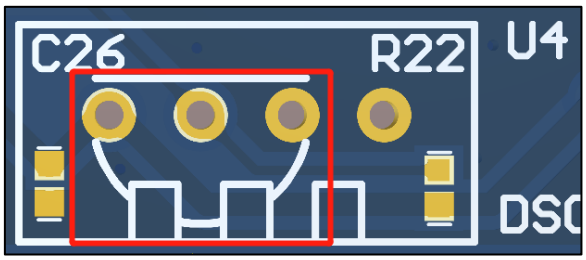


图 50.1.2 DS18B20 连接示意图

从上图可以看出，DS18B20 的平面部分（有字的那面）应该朝内，而曲面部分朝外。然后插入如图所示的三个孔内。

50.2 程序设计

50.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 28_DS18B20。

50.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 28_DS18B20，以进行下一步的实验操作。如下图所示：

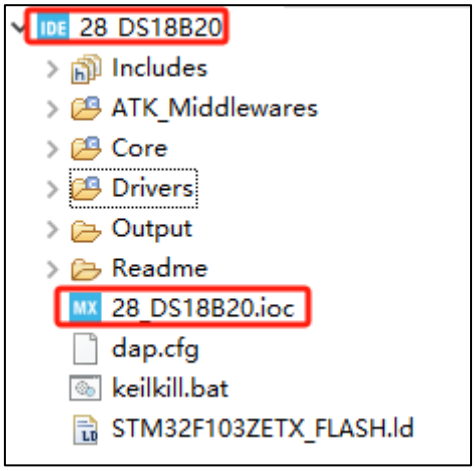


图 50.2.2.1 DS18B20 工程

（1）GPIO 配置

双击 28_DS18B20.ioc 进入 STM32CubeMX 插件的，在 Pinout & Configuration 处我们配置 PG11 为 GPIO_Output 模式，然后参数配置为开漏输出、上拉、高速模式，如下图所示

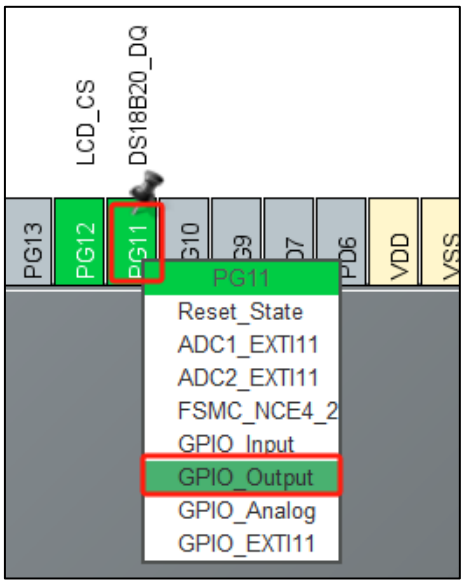


图 50.2.2.2 GPIO 复用

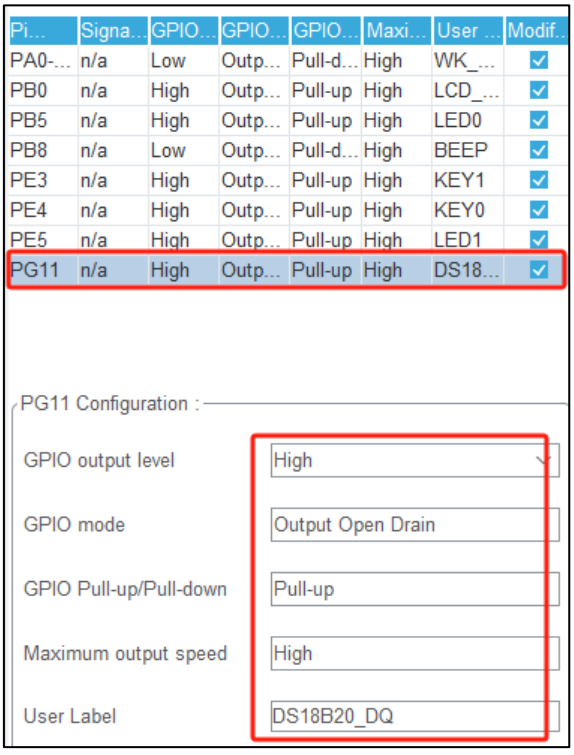


图 50.2.2.3 GPIO 参数配置

50.2.3 添加用户驱动代码

按下“Ctrl+S”保存配置，生成工程，然后在 Drivers/BSP 文件夹中，新建 DS18B20 文件夹，然后再 DS18B20 文件夹下新建两个文件，分别是 ds18b20.c 和 ds18b20.h，如下图所示：

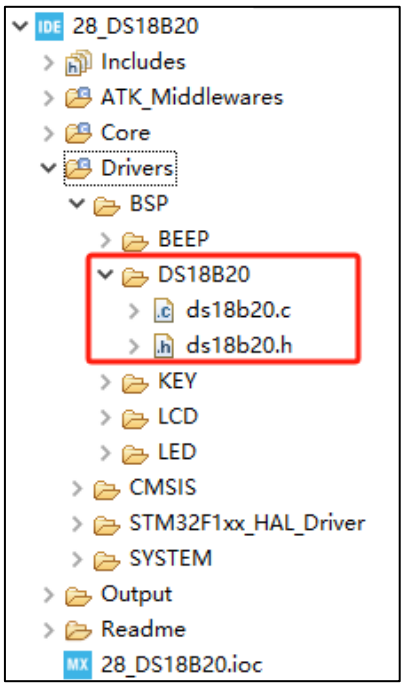


图 50.2.3.1 添加用户文件

下面我们开始添加用户代码：

（1）ds18b20.h 文件代码

```
#ifndef __DS18B20_H
#define __DS18B20_H

#include "../../SYSTEM/sys/sys.h"

/*********************************************************************************/

/* IO 操作函数 */
#define DS18B20_DQ_OUT(x)    do{ x ? \
                                 HAL_GPIO_WritePin(DS18B20_DQ_GPIO_Port,
                                 DS18B20_DQ_Pin, GPIO_PIN_SET) : \
                                 HAL_GPIO_WritePin(DS18B20_DQ_GPIO_Port,
                                 DS18B20_DQ_Pin, GPIO_PIN_RESET); \
                               }while(0)
/* 数据端口输出 */
#define DS18B20_DQ_IN        HAL_GPIO_ReadPin(DS18B20_DQ_GPIO_Port,
                                              DS18B20_DQ_Pin)      /* 数据端口输入 */


uint8_t ds18b20_init(void);          /* 初始化 DS18B20 */
uint8_t ds18b20_check(void);         /* 检测是否存在 DS18B20 */
short ds18b20_get_temperature(void);/* 获取温度 */

#endif
```

DS18B20_DQ_IN 用于读取 PG11 引脚（总线）的电平，DS18B20_DQ_OUT 通过参数 x 设置引脚 PG11 的电平，当 x 为 0 时，设置 PG11 输出 0，当 x 为 1 时，设置 PG11 输出 1。

（2）ds18b20.c 文件代码

```
/**
 * @brief       复位 DS18B20
 * @param       data: 要写入的数据
 * @retval      无
```

```
 */
static void ds18b20_reset(void)
{
    DS18B20_DQ_OUT(0);   /* 拉低 DQ,复位 */
    delay_us(750);       /* 拉低 750us */
    DS18B20_DQ_OUT(1);   /* DQ=1, 释放复位 */
    delay_us(15);        /* 延迟 15US */
}

/**
 * @brief       等待 DS18B20 的回应
 * @param       无
 * @retval      0, DS18B20 正常
 *              1, DS18B20 异常/不存在
 */
uint8_t ds18b20_check(void)
{
    uint8_t retry = 0;
    uint8_t rval = 0;

    while (DS18B20_DQ_IN && retry < 200)     /* 等待 DQ 变低, 等待 200us */
    {
        retry++;
        delay_us(1);
    }

    if (retry >= 200)
    {
        rval = 1;
    }
    else
    {
        retry = 0;

        while (!DS18B20_DQ_IN && retry < 240)    /* 等待 DQ 变高, 等待 240us */
        {
            retry++;
            delay_us(1);
        }

        if (retry >= 240) rval = 1;
    }

    return rval;
}

/**
 * @brief       从 DS18B20 读取一个位
 * @param       无
 * @retval      读取到的位值: 0 / 1
 */
static uint8_t ds18b20_read_bit(void)
{
    uint8_t data = 0;
    DS18B20_DQ_OUT(0);
    delay_us(2);
    DS18B20_DQ_OUT(1);
    delay_us(12);

    if (DS18B20_DQ_IN)
    {
        data = 1;
    }
```

```
    delay_us(50);
    return data;
}

/**
 * @brief       从 DS18B20 读取一个字节
 * @param       无
 * @retval      读到的数据
 */
static uint8_t ds18b20_read_byte(void)
{
    uint8_t i, b, data = 0;

    for (i = 0; i < 8; i++)
    {
        b = ds18b20_read_bit(); /* DS18B20 先输出低位数据 ,高位数据后输出 */

        data |= b << i;         /* 填充 data 的每一位 */
    }

    return data;
}

/**
 * @brief       写一个字节到 DS18B20
 * @param       data: 要写入的字节
 * @retval      无
 */
static void ds18b20_write_byte(uint8_t data)
{
    uint8_t j;

    for (j = 1; j <= 8; j++)
    {
        if (data & 0x01)
        {
            DS18B20_DQ_OUT(0);  /*  Write 1 */
            delay_us(2);
            DS18B20_DQ_OUT(1);
            delay_us(60);
        }
        else
        {
            DS18B20_DQ_OUT(0);  /*  Write 0 */
            delay_us(60);
            DS18B20_DQ_OUT(1);
            delay_us(2);
        }

        data >>= 1;             /* 右移,获取高一位数据 */
    }
}

/**
 * @brief       开始温度转换
 * @param       无
 * @retval      无
 */
static void ds18b20_start(void)
{
    ds18b20_reset();
    ds18b20_check();
    ds18b20_write_byte(0xcc);   /*  skip rom */
    ds18b20_write_byte(0x44);   /*  convert */
}
```

```
/**
 * @brief       检测 DS18B20 的存在
 * @param       无
 * @retval      0, 正常
 *              1, 不存在/不正常
 */
uint8_t ds18b20_init(void)
{
    ds18b20_reset();
    return ds18b20_check();
}

/**
 * @brief       从 ds18b20 得到温度值(精度：0.1C)
 * @param       无
 * @retval      温度值 （-550~1250）
 *   @note      返回的温度值放大了 10 倍.
 *              实际使用的时候,要除以 10 才是实际温度.
 */
short ds18b20_get_temperature(void)
{
    uint8_t flag = 1;           /* 默认温度为正数 */
    uint8_t TL, TH;
    short temp;

    ds18b20_start();            /*  ds1820 start convert */
    ds18b20_reset();
    ds18b20_check();
    ds18b20_write_byte(0xcc);   /*  skip rom */
    ds18b20_write_byte(0xbe);   /*  convert */
    TL = ds18b20_read_byte();   /*  LSB */
    TH = ds18b20_read_byte();   /*  MSB */

    if (TH > 7)
    {/* 温度为负，查看 DS18B20 的温度表示法与计算机存储正负数据的原理一致：
        正数补码为寄存器存储的数据自身，负数补码为寄存器存储值按位取反后+1
        所以我们直接取它实际的负数部分，但负数的补码为取反后加一，但考虑到低位可能+1 后有进
        位和代码冗余，
        我们这里先暂时没有作+1 的处理，这里需要留意 */
        TH = ~TH;
        TL = ~TL;
        flag = 0;    /* 温度为负 */
    }

    temp = TH;       /* 获得高八位 */
    temp <<= 8;
    temp += TL;      /* 获得底八位 */

    /* 转换成实际温度 */
    if (flag == 0)
    {   /* 将温度转换成负温度，这里的+1 参考前面的说明 */
        temp = (double)(temp+1) * 0.625;
        temp = -temp;
    }
    else
    {
        temp = (double)temp * 0.625;
    }

    return temp;
}
```

该文件中的函数，基本都是根据 ds18b20 的时序图去设计的，大家可结合

DS18B20 的数据手册去看这部分的代码。

（3）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    short temperature;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                                          /* 初始化 LCD */
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "DS18B20 TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

  while (ds18b20_init()) /* DS18B20 初始化 */
  {
      lcd_show_string(30, 110, 200, 16, 16, "DS18B20 Error", RED);
      delay_ms(200);
      lcd_fill(30, 110, 239, 130 + 16, WHITE);
      delay_ms(200);
  }

  lcd_show_string(30, 110, 200, 16, 16, "DS18B20 OK", RED);
  lcd_show_string(30, 130, 200, 16, 16, "Temp:    . C", BLUE);
  /* USER CODE END 2 */

  while (1)
  {
      if (t % 10 == 0) /* 每 100ms 读取一次 */
      {
          temperature = ds18b20_get_temperature();

          if (temperature < 0)
          {
              lcd_show_char(30 + 40, 130, '-', 16, 0, BLUE); /* 显示负号 */
              temperature = -temperature;                    /* 转为正数 */
          }
          else
          {
              lcd_show_char(30 + 40, 130, ' ', 16, 0, BLUE); /* 去掉负号 */
          }
          /* 显示正数部分 */
          lcd_show_num(30 + 40 + 8, 130, temperature / 10, 2, 16, BLUE);
          /* 显示小数部分 */
          lcd_show_num(30 + 40 + 32, 130, temperature % 10, 1, 16, BLUE);
```

```
        }

        delay_ms(10);
        t++;

        if (t == 20)
        {
            t = 0;
            LED0_TOGGLE(); /* LED0 闪烁 */
        }
    }
}
```

主函数代码比较简单，一系列硬件初始化后，在循环中调用 ds18b20_get_temperature 函数获取温度值，然后显示在 LCD 上。

50.3 下载验证

假定 DS18B20 传感器已经接上去正确的位置，将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示当前的温度值的内容如图 50.3.1 所示：

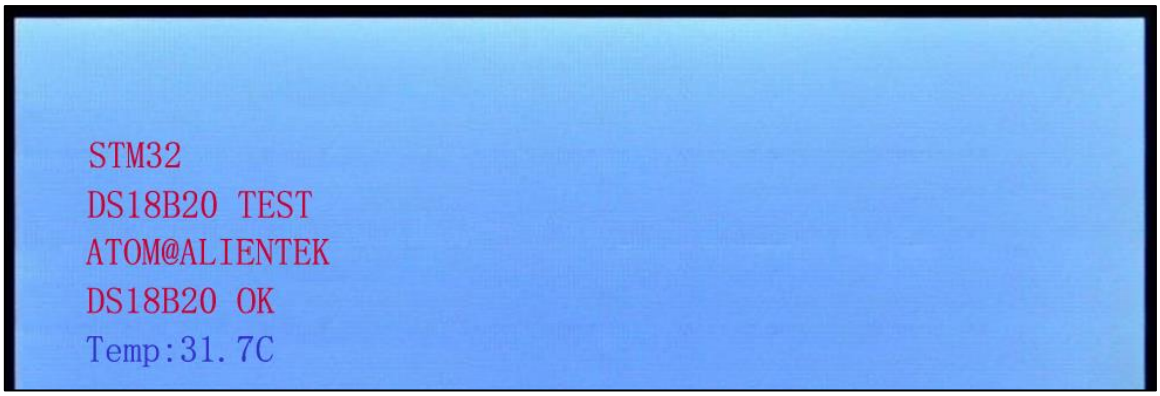


图 50.3.1 程序运行效果图

该程序还可以读取并显示负温度值，具备零下温度条件可以测试一下。

第五十一章 DHT11 数字温湿度传感器

本章，我们将介绍数字温湿度传感器 DHT11 的使用，与前一章的温度传感器相比，该传感器不但能测温度，还能测湿度。我们将学习如何获取 DHT11 传感器的温湿度数据，并把数据显示在 LCD 上。

本章分为如下几个小节：

51.1 硬件设计
51.2 程序设计
51.2 下载验证

51.1 硬件设计

1. 例程功能

本实验开机的时候先检测是否有 DHT11 存在，如果没有，则提示错误。只有在检测到 DHT11 之后才开始读取温湿度值并显示在 LCD 上，如果发现了 DHT11，则程序每隔 100ms 左右读取一次数据，并把温湿度显示在 LCD 上。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
2）DHT11 温湿度传感器 – PG11
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

DHT11 接口与 STM32 的连接关系跟上一章节中 DS18B20 和 STM32 的关系是一样的，使用到的 GPIO 口是 PG11。这里原理图就不列出来了，可以翻看上一章节原理图。

DHT11 和 DS18B20 的接口是共用一个的，不过 DHT11 有 4 条腿，需要把 U4 的 4 个接口都用上，将 DHT11 传感器插入到这个上面就可以通过 STM32F1 来读取温湿度值了。连接示意图如图 51.1.1 所示：

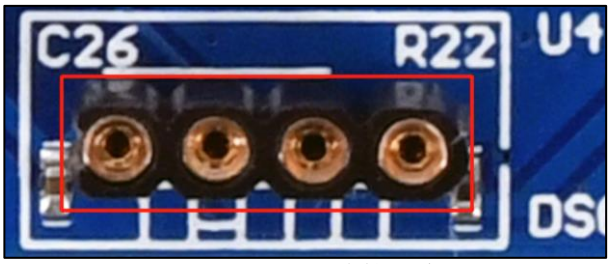


图 51.1.1 DHT11 连接示意图

这里要注意，将 DHT11 贴有字的一面朝内，而有很多孔的一面(网面)朝外，然后插入上图所示的四个孔内就可以了。

51.2 程序设计

51.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 29_DHT11。

51.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 29_DHT11，以进行下一步的实验操作。如下图所示：

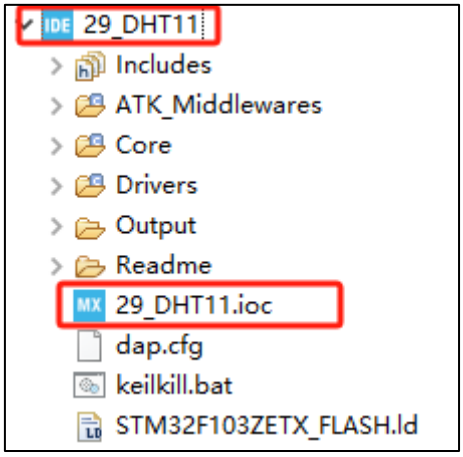


图 51.2.2.1 DHT11 工程

（1）GPIO 配置

双击 29_DHT11.ioc 进入 STM32CubeMX 插件的，在 Pinout & Configuration 处我们配置 PG11 为 GPIO_Output 模式，然后参数配置为开漏输出、上拉、高速模式，如下图所示

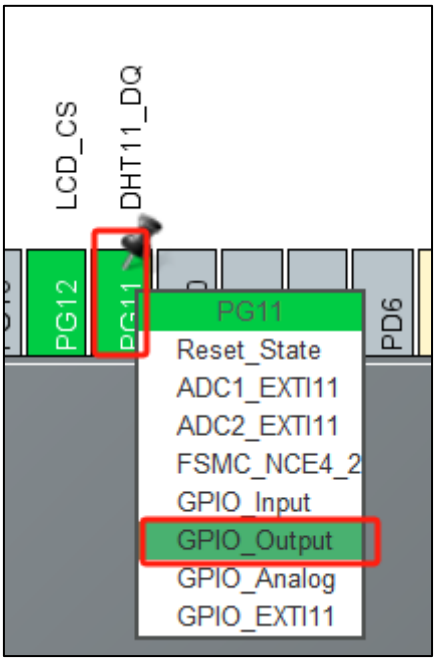


图 51.2.2.2 GPIO 复用

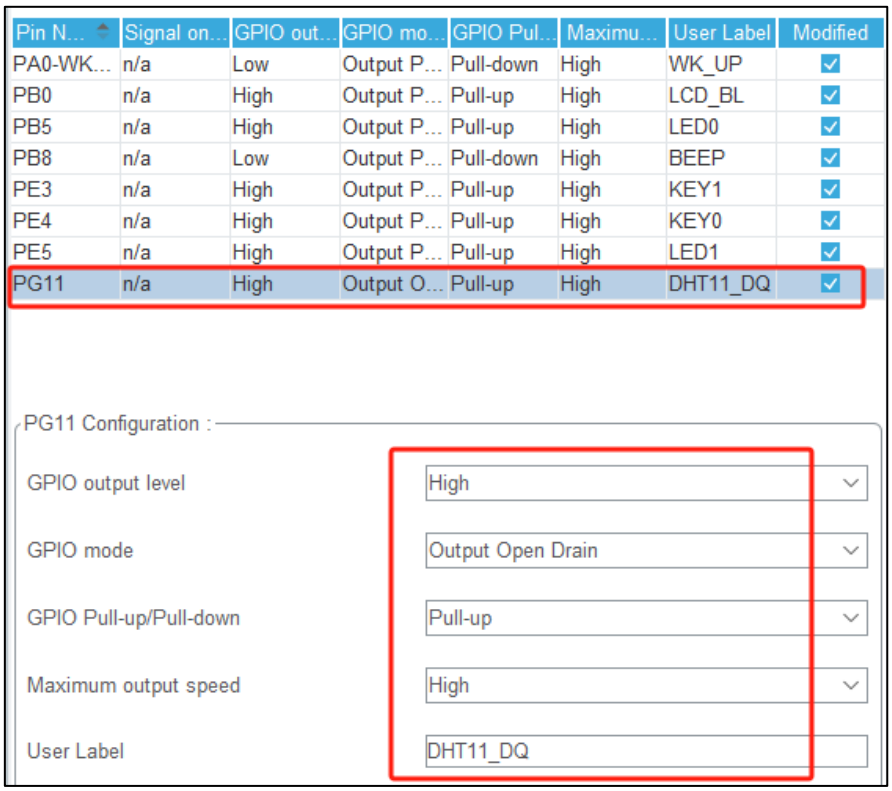


图 51.2.2.3 GPIO 参数配置

51.2.3 添加用户驱动代码

按下“Ctrl+S”保存配置，生成工程，然后在 Drivers/BSP 文件夹中，新建 DHT11 文件夹，然后再 DHT11 文件夹下新建两个文件，分别是 dht11.c 和 dht11.h，如下图所示：

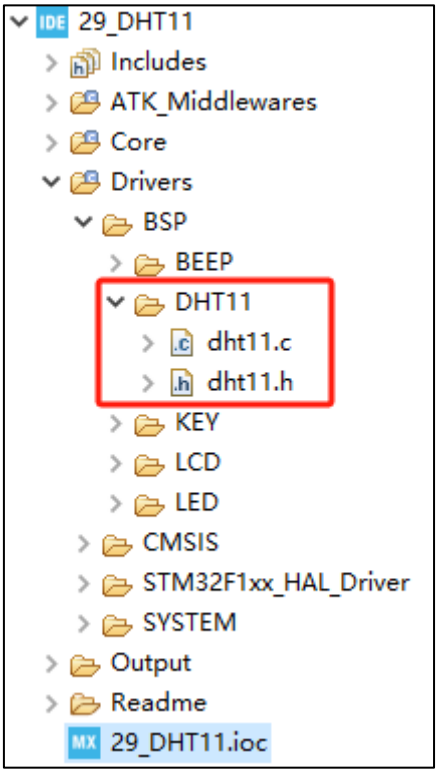


图 51.2.3.1 添加用户文件

下面我们开始添加用户代码：

（1）dht11.h 文件代码

```
#ifndef __DHT11_H
#define __DHT11_H

#include "../../SYSTEM/sys/sys.h"

/*****************************************************************************/

/* IO 操作函数 */
#define DHT11_DQ_OUT(x)      do{ x ? \
                                 HAL_GPIO_WritePin(DHT11_DQ_GPIO_Port,
                                DHT11_DQ_Pin, GPIO_PIN_SET) : \
                                 HAL_GPIO_WritePin(DHT11_DQ_GPIO_Port,
                                  DHT11_DQ_Pin, GPIO_PIN_RESET); \
                               }while(0)
/* 数据端口输出 */
#define DHT11_DQ_IN          HAL_GPIO_ReadPin(DHT11_DQ_GPIO_Port, DHT11_DQ_Pin)
/* 数据端口输入 */


uint8_t dht11_init(void);                                /* 初始化 DHT11 */
uint8_t dht11_check(void);                               /* 检测是否存在 DHT11 */
uint8_t dht11_read_data(uint8_t *temp,uint8_t *humi);  /* 读取温湿度 */

#endif
```

DHT11_DQ_IN 用于读取 PG11 引脚（总线）的电平，DHT11_DQ_OUT 通过参数 x 设置引脚 PG11 的电平，当 x 为 0 时，设置 PG11 输出 0，当 x 为 1 时，设置 PG11 输出 1。

（2）dht11.c 文件代码

```
/**
 * @brief       复位 DHT11
 * @param       无
 * @retval      无
 */
static void dht11_reset(void)
{
    DHT11_DQ_OUT(0);     /* 拉低 DQ */
    delay_ms(20);        /* 拉低至少 18ms */
    DHT11_DQ_OUT(1);     /* DQ=1 */
    delay_us(30);        /* 主机拉高 10~35us */
}

/**
 * @brief       等待 DHT11 的回应
 * @param       无
 * @retval      0, DHT11 正常
 *              1, DHT11 异常/不存在
 */
uint8_t dht11_check(void)
{
    uint8_t retry = 0;
    uint8_t rval = 0;

    while (DHT11_DQ_IN && retry < 100)  /* DHT11 会拉低约 83us */
    {
        retry++;
        delay_us(1);
    }
```

```
    if (retry >= 100)
    {
        rval = 1;
    }
    else
    {
        retry = 0;

        while (!DHT11_DQ_IN && retry < 100) /* DHT11 拉低后会再次拉高约 87us */
        {
            retry++;
            delay_us(1);
        }
        if (retry >= 100) rval = 1;
    }

    return rval;
}

/**
 * @brief       从 DHT11 读取一个位
 * @param       无
 * @retval      读取到的位值: 0 / 1
 */
uint8_t dht11_read_bit(void)
{
    uint8_t retry = 0;

    while (DHT11_DQ_IN && retry < 100)  /* 等待变为低电平 */
    {
        retry++;
        delay_us(1);
    }

    retry = 0;

    while (!DHT11_DQ_IN && retry < 100) /* 等待变高电平 */
    {
        retry++;
        delay_us(1);
    }

    delay_us(40);       /* 等待 40us */

    if (DHT11_DQ_IN)    /* 根据引脚状态返回 bit */
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

/**
 * @brief       从 DHT11 读取一个字节
 * @param       无
 * @retval      读到的数据
 */
static uint8_t dht11_read_byte(void)
{
    uint8_t i, data = 0;

    for (i = 0; i < 8; i++)         /* 循环读取 8 位数据 */
    {
```

```
        data <<= 1;                 /* 高位数据先输出，先左移一位 */
        data |= dht11_read_bit();   /* 读取 1bit 数据 */
    }

    return data;
}

/**
 * @brief       从 DHT11 读取一次数据
 * @param       temp: 温度值(范围:-20~60°)
 * @param       humi: 湿度值(范围:5%~95%)
 * @retval      0, 正常.
 *              1, 失败
 */
uint8_t dht11_read_data(uint8_t *temp, uint8_t *humi)
{
    uint8_t buf[5];
    uint8_t i;
    dht11_reset();

    if (dht11_check() == 0)
    {
        for (i = 0; i < 5; i++)     /* 读取 40 位数据 */
        {
            buf[i] = dht11_read_byte();
        }

        if ((buf[0] + buf[1] + buf[2] + buf[3]) == buf[4])
        {
            *humi = buf[0];
            *temp = buf[2];
        }
    }
    else
    {
        return 1;
    }

    return 0;
}

/**
 * @brief       初始化 DHT11 的 IO 口 DQ 同时检测 DHT11 的存在
 * @param       无
 * @retval      0, 正常
 *              1, 不存在/不正常
 */
uint8_t dht11_init(void)
{
    dht11_reset();
    return dht11_check();
}
```

该文件中的函数，基本都是根据 dht11 的时序图去设计的，大家可结合 DHT11 的数据手册去看这部分的代码。

（3）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t = 0;
    uint8_t temperature;
    uint8_t humidity;
  /* USER CODE END 1 */
```

```
    HAL_Init();

    /* 系统时钟初始化 */
    SystemClock_Config();

    /* 延时初始化 */
    delay_init(72);

    /* GPIO 初始化 */
    MX_GPIO_Init();

    /* FSMC 初始化 */
    MX_FSMC_Init();

    /* 串口初始化 */
    MX_USART1_UART_Init();

    /* USER CODE BEGIN 2 */
    lcd_init();                                             /* 初始化 LCD */
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DHT11 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (dht11_init()) /* DHT11 初始化 */
    {
        lcd_show_string(30, 110, 200, 16, 16, "DHT11 Error", RED);
        delay_ms(200);
        lcd_fill(30, 110, 239, 130 + 16, WHITE);
        delay_ms(200);
    }

    lcd_show_string(30, 110, 200, 16, 16, "DHT11 OK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "Temp:  C", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "Humi:  %", BLUE);
    /* USER CODE END 2 */

    while (1)
    {
        if (t % 10 == 0) /* 每 100ms 读取一次 */
        {
            dht11_read_data(&temperature, &humidity);           /* 读取温湿度值 */
            lcd_show_num(30 + 40, 130, temperature, 2, 16, BLUE); /* 显示温度 */
            lcd_show_num(30 + 40, 150, humidity, 2, 16, BLUE);    /* 显示湿度 */
        }

        delay_ms(10);
        t++;

        if (t == 20)
        {
            t = 0;
            LED0_TOGGLE(); /* LED0 闪烁 */
        }
    }
}
```

主函数代码比较简单，一系列硬件初始化后，如果 DHT11 初始化成功，那么在循环中调用 dht11_read_data 函数获取温湿度值，每隔 100ms 读取数据并显示在 LCD 上。

51.3 下载验证

假定 DHT11 传感器已经接上去正确的位置，将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示当前的温度值的内容如图 51.3.1 所示：

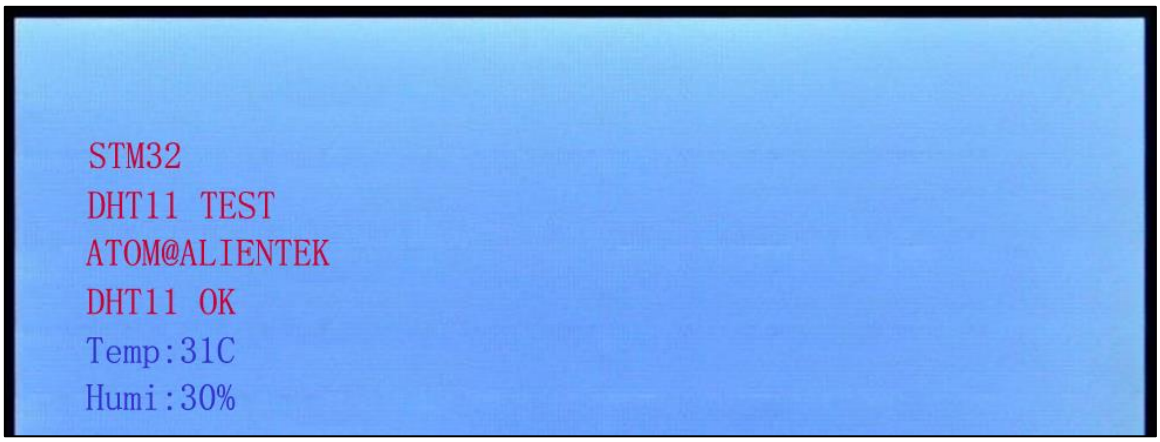


图 51.3.1 程序运行效果图

至此，本章实验结束。大家可以将本章通过 DHT11 读取到的温度值，和前一章的通过 DS18B20 读取到的温度值对比一下，看看哪个更准确？

第五十二章 无线通信实验

本章，我们将介绍如何使用 2.4G 无线模块 NRF24L01 实现无线通信。将使用两块 STM32 开发板，一块用于发送，一块用于接收，从而实现无线数据传输，并把数据显示在 LCD 上。本章分为如下几个小节：

52.1 硬件设计
52.2 程序设计
52.3 下载验证

52.1 硬件设计

1. 例程功能

开机的时候先检测 NRF24L01 模块是否存在，在检测到 NRF24L01 模块之后，根据 KEY0 和 KEY1 的设置来决定模块的工作模式。在设定好工作模式之后，就会不停的发送/接收数据，同时在 LCD 上面显示相关信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2) 独立按键
KEY0 - PE4
KEY1 - PE3
3）2.4G 无线模块 NRF24L01 模块
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
6）SPI2(连接在 PB13/ PB14/ PB15)

3. 原理图

NRF24L01 模块与 STM32 的连接关系，如下图所示：

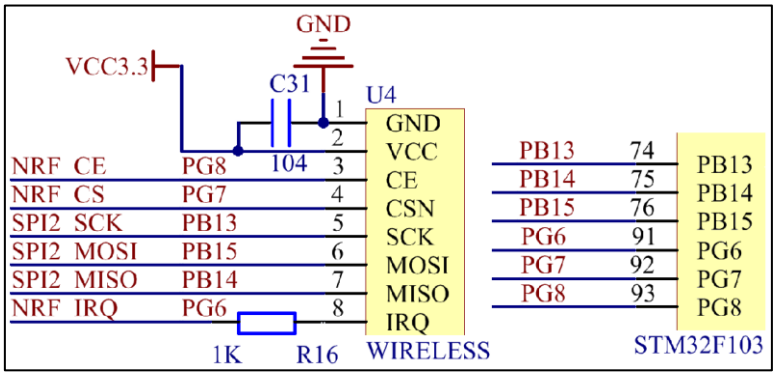


图 52.1.1 NRF24L01 模块接口与 STM32 连接原理图

NRF24L01 使用的是 SPI2，与 NorFlash 共用一个 SPI 接口，所以如果同时使用这些设备的时候，必须分时复用。为了防止其他器件对 NRF24L01 的通信造成干扰，最好可以把 NorFlash 的片选信号引脚拉高。

由于无线通信实验是双向的，所以至少需要有两个模块同时能工作，这里我们使用 2 套开发板来向大家演示。

52.2 程序设计

52.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 30_NRF24C01。

52.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 30_NRF24C01，以进行下一步的实验操作。如下图所示：

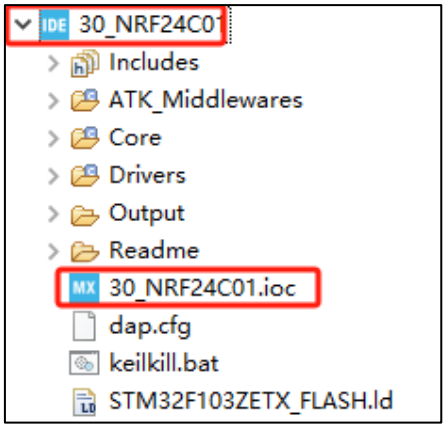


图 52.2.2.1 NRF24C01 工程

（1）GPIO 配置

双击 30_NRF24C01.ioc 进入 STM32CubeMX 插件，在 Pinout & Configuration 处我们配置所需的 IO，NRF24L01 的 CE、CSN 和 IRQ 分别对应 PG8，PG7 和 PG6 进行如下配置：

Pin Name	Signal on Pin	GPIO output le...	GPIO mode	GPIO Pull-up/...	Maximum out...	User Label	Modified
PA0-WKUP	n/a	Low	Output Push ...	Pull-down	High	WK_UP	☑
PB0	n/a	High	Output Push ...	Pull-up	High	LCD_BL	☑
PB5	n/a	High	Output Push ...	Pull-up	High	LED0	☑
PB8	n/a	Low	Output Push ...	Pull-down	High	BEEP	☑
PE3	n/a	High	Output Push ...	Pull-up	High	KEY1	☑
PE4	n/a	High	Output Push ...	Pull-up	High	KEY0	☑
PE5	n/a	High	Output Push ...	Pull-up	High	LED1	☑
PG6	n/a	n/a	Input mode	Pull-up	n/a	NRF24L01_IRQ	☑
PG7	n/a	High	Output Push ...	Pull-up	High	NRF24L01_CSN	☑
PG8	n/a	High	Output Push ...	Pull-up	High	NRF24L01_CE	☑

图 52.2.2.2 GPIO 复用

（2）SPI2 配置

本实验用到 SPI2 驱动 NRF24L01，使用 PB13、PB14 和 PB15 作为 SPI_SCK、SPI_MISO 和 SPI_MOSI，如下进行配置：

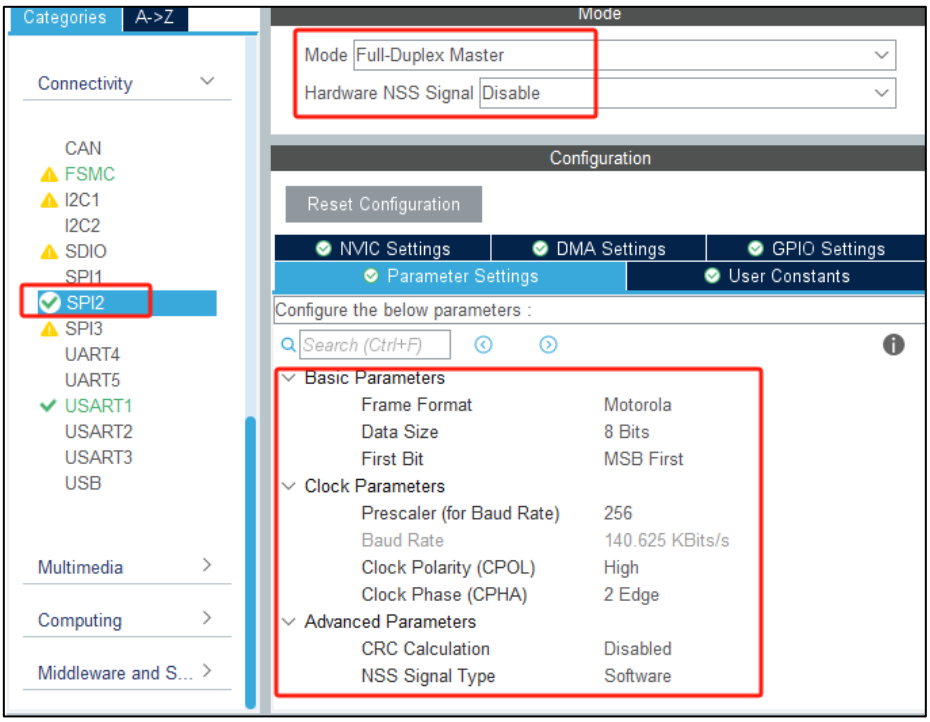


图 52.2.2.3 SPI2 参数配置

接着需要检查下所配置的所有 IO 是否与硬件原理图相对应。

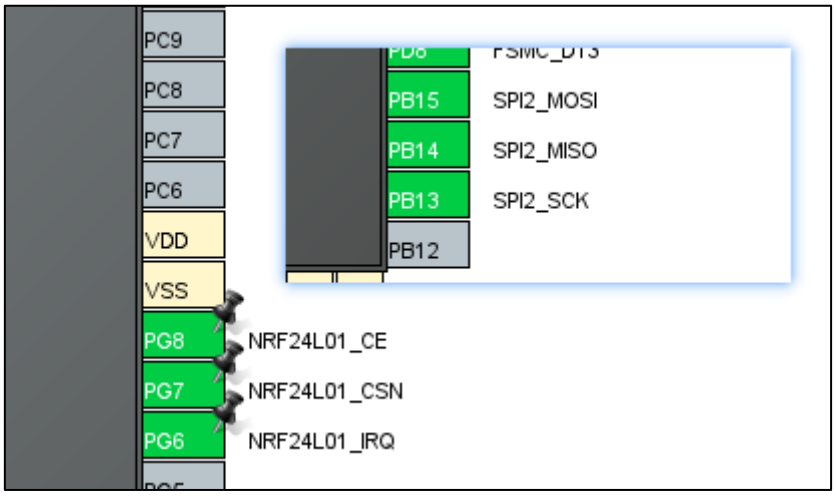


图 52.2.2.4 IO 检查

52.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 spi.c 文件。spi.c 主要是完成 SPI2 的初始化。

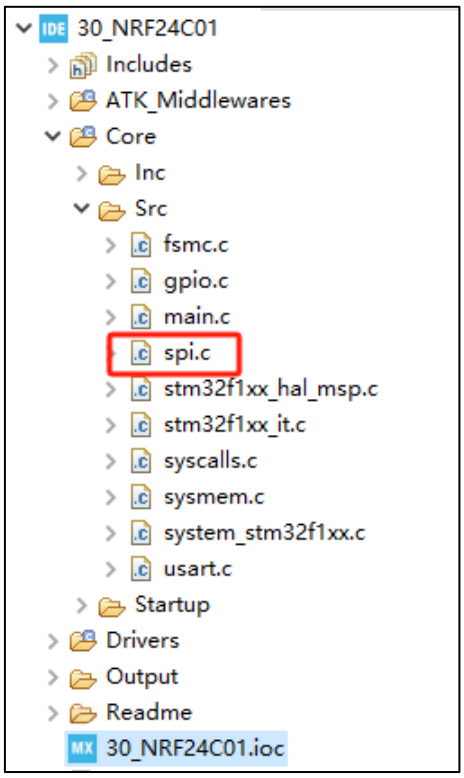


图 52.2.3.1 生成工程

52.2.4 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 NRF24L01 文件夹，然后再 NRF24L01 文件夹下新建两个文件，分别是 nrf24l01.c 和 nrf24l01.h，如下图所示：

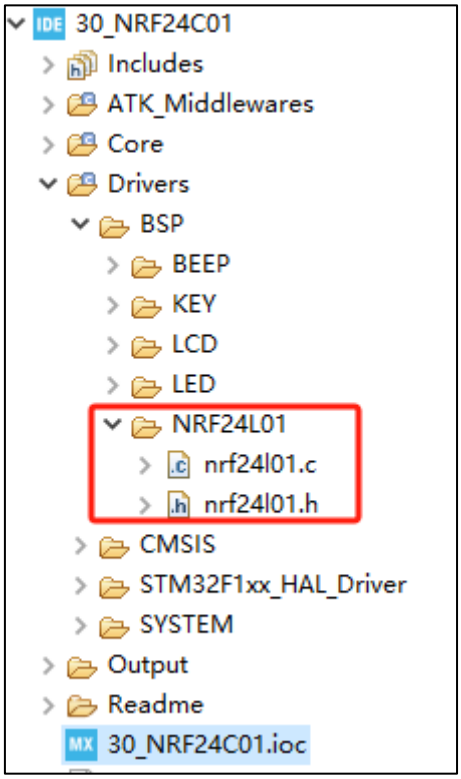


图 52.2.4.1 添加用户文件

下面我们开始添加用户代码：

（1）spi.c 文件代码

```
void MX_SPI2_Init(void)
{
  hspi2.Instance = SPI2;
  hspi2.Init.Mode = SPI_MODE_MASTER;
  hspi2.Init.Direction = SPI_DIRECTION_2LINES;
  hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
  hspi2.Init.CLKPolarity = SPI_POLARITY_HIGH;
  hspi2.Init.CLKPhase = SPI_PHASE_2EDGE;
  hspi2.Init.NSS = SPI_NSS_SOFT;
  hspi2.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
  hspi2.Init.FirstBit = SPI_FIRSTBIT_MSB;
  hspi2.Init.TIMode = SPI_TIMODE_DISABLE;
  hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
  hspi2.Init.CRCPolynomial = 10;
  if (HAL_SPI_Init(&hspi2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN SPI2_Init 2 */
  __HAL_SPI_ENABLE(&hspi2);      /* 使能 SPI2 */
  spi2_read_write_byte(0Xff);    /* 启动传输，达到清空 DR 的作用，非必需 */
  /* USER CODE END SPI2_Init 2 */

}

/* 省略部分代码… … */

/* USER CODE BEGIN 1 */
/**
 * @brief       SPI2 速度设置函数
 *   @note      SPI2 时钟选择来自 APB1，即 PCLK1，为 36Mhz
 *              SPI 速度 = PCLK1 / 2^(speed + 1)
 * @param       speed   : SPI2 时钟分频系数
                        取值为 SPI_BAUDRATEPRESCALER_2~SPI_BAUDRATEPRESCALER_2
256
 * @retval      无
 */
void spi2_set_speed(uint8_t speed)
{
    assert_param(IS_SPI_BAUDRATE_PRESCALER(speed)); /* 判断有效性 */
    __HAL_SPI_DISABLE(&hspi2);                      /* 关闭 SPI */
    hspi2.Instance->CR1 &= 0XFFC7;                  /* 位 3-5 清零，用来设置波特率
*/
    hspi2.Instance->CR1 |= speed << 3;              /* 设置 SPI 速度 */
    __HAL_SPI_ENABLE(&hspi2);                       /* 使能 SPI */
}

/**
 * @brief       SPI2 读写一个字节数据
 * @param       txdata  : 要发送的数据(1 字节)
 * @retval      接收到的数据(1 字节)
 */
uint8_t spi2_read_write_byte(uint8_t txdata)
{
    uint8_t rxdata;
    HAL_SPI_TransmitReceive(&hspi2, &txdata, &rxdata, 1, 1000);
    return rxdata; /* 返回收到的数据 */
}
/* USER CODE END 1 */
```

在 SPI 的初始化函数中，我们添加了函数__HAL_SPI_ENABLE 用于使能 SPI。以及函数 spi2_read_write_byte(0Xff)，用于启动发送。

函数 spi2_set_speed 是 SPI 的速度设置函数，通过操作寄存器的方式去实现。

函数 spi2_read_write_byte 直接调用了 HAL 库内置的函数进行接收发送操作。

（2）nrf24l01.h 文件代码

```
#ifndef __24L01_H
#define __24L01_H

#include "../../SYSTEM/sys/sys.h"
/*****************************************************************************/

/* 24L01 操作线 */
#define NRF24L01_CE(x)    do{ x ? \
                              HAL_GPIO_WritePin(NRF24L01_CE_GPIO_Port,
NRF24L01_CE_Pin, GPIO_PIN_SET) : \
                              HAL_GPIO_WritePin(NRF24L01_CE_GPIO_Port,
NRF24L01_CE_Pin, GPIO_PIN_RESET); \
                          }while(0)       /* 24L01 模式选择信号 */

#define NRF24L01_CSN(x)   do{ x ? \
                              HAL_GPIO_WritePin(NRF24L01_CSN_GPIO_Port,
NRF24L01_CSN_Pin, GPIO_PIN_SET) : \
                              HAL_GPIO_WritePin(NRF24L01_CSN_GPIO_Port,
NRF24L01_CSN_Pin, GPIO_PIN_RESET); \
                          }while(0)       /* 24L01 片选信号 */

#define NRF24L01_IRQ      HAL_GPIO_ReadPin(NRF24L01_IRQ_GPIO_Port,
NRF24L01_IRQ_Pin) /* IRQ 主机数据输入 */


/* 24L01 发送接收数据宽度定义
 * 用户必须根据实际情况设置正确的数据宽度和数据长度
 * 发送端&接收端必须保持一致，否则将导致通信失败!!!!
 */
#define TX_ADR_WIDTH    5       /* 5 字节的地址宽度 */
#define RX_ADR_WIDTH    5       /* 5 字节的地址宽度 */
#define TX_PLOAD_WIDTH  32      /* 32 字节的用户数据宽度 */
#define RX_PLOAD_WIDTH  32      /* 32 字节的用户数据宽度 */


/*****************************************************************************/
/* NRF24L01 寄存器操作命令 */
#define NRF_READ_REG    0x00    /* 读配置寄存器,低 5 位为寄存器地址 */
#define NRF_WRITE_REG   0x20    /* 写配置寄存器,低 5 位为寄存器地址 */
#define RD_RX_PLOAD     0x61    /* 读 RX 有效数据,1~32 字节 */
#define WR_TX_PLOAD     0xA0    /* 写 TX 有效数据,1~32 字节 */
#define FLUSH_TX        0xE1    /* 清除 TX FIFO 寄存器.发射模式下用 */
#define FLUSH_RX        0xE2    /* 清除 RX FIFO 寄存器.接收模式下用 */
#define REUSE_TX_PL     0xE3    /* 重新使用上一包数据,CE 为高,数据包被不断发送. */
#define NOP             0xFF    /* 空操作,可以用来读状态寄存器 */

/* SPI(NRF24L01)寄存器地址 */
#define MAX_TX          0x10    /* 达到最大发送次数中断 */
#define TX_OK           0x20    /* TX 发送完成中断 */
#define RX_OK           0x40    /* 接收到数据中断 */

/* 省略部分宏定义... ... */

/*****************************************************************************/
```

```
void nrf24l01_spi_init(void);    /* 针对 NRF24L01 修改 SPI2 驱动 */
void nrf24l01_init(void);        /* 初始化 */
void nrf24l01_rx_mode(void);     /* 配置为接收模式 */
void nrf24l01_tx_mode(void);     /* 配置为发送模式 */
uint8_t nrf24l01_check(void);    /* 检查 24L01 是否存在 */
uint8_t nrf24l01_tx_packet(uint8_t *ptxbuf);      /* 发送一个包的数据 */
uint8_t nrf24l01_rx_packet(uint8_t *prxbuf);      /* 接收一个包的数据 */

#endif
```

上述文件主要是引脚操作函数，此外还有一些 NRF24L01 寄存器操作命令以及其寄存器地址，由于篇幅太大，所以这里就不一一列出来了

（3）nrf24l01.c 文件代码

```
const uint8_t TX_ADDRESS[TX_ADR_WIDTH] = {0x34, 0x43, 0x10, 0x10, 0x01};    /*
发送地址 */
const uint8_t RX_ADDRESS[RX_ADR_WIDTH] = {0x34, 0x43, 0x10, 0x10, 0x01};    /*
发送地址 */

/**
 * @brief       针对 NRF24L01 修改 SPI2 驱动
 * @param       无
 * @retval      无
 */
void nrf24l01_spi_init(void)
{
    __HAL_SPI_DISABLE(&hspi2);                  /* 先关闭 SPI2 */
    hspi2.Init.CLKPolarity = SPI_POLARITY_LOW; /* 串行同步时钟的空闲状态为低电平 */
    hspi2.Init.CLKPhase = SPI_PHASE_1EDGE;      /* 第 1 个跳变沿数据被采样 */
    HAL_SPI_Init(&hspi2);
    __HAL_SPI_ENABLE(&hspi2);                   /* 使能 SPI2 */
}

/**
 * @brief       初始化 24L01 的 IO 口
 *   @note      将 SPI2 模式改成 SCK 空闲低电平,及 SPI 模式 0
 * @param       无
 * @retval      无
 */
void nrf24l01_init(void)
{
    MX_SPI2_Init();             /* 初始化 SPI2 */
    nrf24l01_spi_init();        /* 针对 NRF 的特点修改 SPI 的设置 */
    NRF24L01_CE(0);             /* 使能 24L01 */
    NRF24L01_CSN(1);            /* SPI 片选取消 */
}

/**
 * @brief       检测 24L01 是否存在
 * @param       无
 * @retval      0, 成功; 1, 失败;
 */
uint8_t nrf24l01_check(void)
{
    uint8_t buf[5] = {0XA5, 0XA5, 0XA5, 0XA5, 0XA5};
    uint8_t i;
    spi2_set_speed(SPI_SPEED_32);/* spi 速度为 7.5Mhz,24L01 的最大 SPI 时钟 10Mhz*/
    nrf24l01_write_buf(NRF_WRITE_REG + TX_ADDR, buf, 5);  /* 写入 5 个字节的地址*/
    nrf24l01_read_buf(TX_ADDR, buf, 5);                   /* 读出写入的地址 */
```

```
    for (i = 0; i < 5; i++)
    {
        if (buf[i] != 0XA5) break;
    }

    if (i != 5) return 1;    /* 检测 24L01 错误 */

    return 0;                /* 检测到 24L01 */
}

/**
 * @brief       NRF24L01 写寄存器
 * @param       reg   : 寄存器地址
 * @param       value : 写入寄存器的值
 * @retval      状态寄存器值
 */
static uint8_t nrf24l01_write_reg(uint8_t reg, uint8_t value)
{
    uint8_t status;
    NRF24L01_CSN(0);                    /* 使能 SPI 传输 */
    status = spi2_read_write_byte(reg); /* 发送寄存器号 */
    spi2_read_write_byte(value);        /* 写入寄存器的值 */
    NRF24L01_CSN(1);                    /* 禁止 SPI 传输 */
    return status;                      /* 返回状态值 */
}

/**
 * @brief       NRF24L01 读寄存器
 * @param       reg   : 寄存器地址
 * @retval      读取到的寄存器值;
 */
static uint8_t nrf24l01_read_reg(uint8_t reg)
{
    uint8_t reg_val;
    NRF24L01_CSN(0);            /* 使能 SPI 传输 */
    spi2_read_write_byte(reg);  /* 发送寄存器号 */
    reg_val = spi2_read_write_byte(0XFF);   /* 读取寄存器内容 */
    NRF24L01_CSN(1);            /* 禁止 SPI 传输 */
    return reg_val;             /* 返回状态值 */
}

/**
 * @brief       在指定位置读出指定长度的数据
 * @param       reg   : 寄存器地址
 * @param       pbuf  : 数据指针
 * @param       len   : 数据长度
 * @retval      状态寄存器值
 */
static uint8_t nrf24l01_read_buf(uint8_t reg, uint8_t *pbuf, uint8_t len)
{
    uint8_t status, i;
    NRF24L01_CSN(0);    /* 使能 SPI 传输 */
    status = spi2_read_write_byte(reg);         /* 发送寄存器值(位置),并读取状态值
*/

    for (i = 0; i < len; i++)
    {
        pbuf[i] = spi2_read_write_byte(0XFF);   /* 读出数据 */
    }

    NRF24L01_CSN(1);    /* 关闭 SPI 传输 */
    return status;      /* 返回读到的状态值 */
```

```
}

/**
 * @brief       在指定位置写指定长度的数据
 * @param       reg   : 寄存器地址
 * @param       pbuf  : 数据指针
 * @param       len   : 数据长度
 * @retval      状态寄存器值
 */
static uint8_t nrf24l01_write_buf(uint8_t reg, uint8_t *pbuf, uint8_t len)
{
    uint8_t status, i;
    NRF24L01_CSN(0);    /* 使能 SPI 传输 */
    status = spi2_read_write_byte(reg);/* 发送寄存器值(位置),并读取状态值 */

    for (i = 0; i < len; i++)
    {
        spi2_read_write_byte(*pbuf++); /* 写入数据 */
    }

    NRF24L01_CSN(1);    /* 关闭 SPI 传输 */
    return status;      /* 返回读到的状态值 */
}

/**
 * @brief       启动 NRF24L01 发送一次数据(数据长度 = TX_PLOAD_WIDTH)
 * @param       ptxbuf : 待发送数据首地址
 * @retval      发送完成状态
 *   @arg       0    : 发送成功
 *   @arg       1    : 达到最大发送次数,失败
 *   @arg       0XFF : 其他错误
 */
uint8_t nrf24l01_tx_packet(uint8_t *ptxbuf)
{
    uint8_t sta;
    uint8_t rval = 0XFF;

    NRF24L01_CE(0);
    nrf24l01_write_buf(WR_TX_PLOAD, ptxbuf, TX_PLOAD_WIDTH);
    NRF24L01_CE(1);                     /* 启动发送 */

    while (NRF24L01_IRQ != 0);          /* 等待发送完成 */

    sta = nrf24l01_read_reg(STATUS);    /* 读取状态寄存器的值 */
    /* 清除 TX_DS 或 MAX_RT 中断标志 */
    nrf24l01_write_reg(NRF_WRITE_REG + STATUS, sta);

    if (sta & MAX_TX)                   /* 达到最大重发次数 */
    {
        nrf24l01_write_reg(FLUSH_TX, 0xff); /* 清除 TX FIFO 寄存器 */
        rval = 1;
    }

    if (sta & TX_OK)/* 发送完成 */
    {
        rval = 0;   /* 标记发送成功 */
    }

    return rval;    /* 返回结果 */
}

/**
```

```
 * @brief       启动 NRF24L01 接收一次数据(数据长度 = RX_PLOAD_WIDTH)
 * @param       prxbuf : 接收数据缓冲区首地址
 * @retval      接收完成状态
 *   @arg       0 : 接收成功
 *   @arg       1 : 失败
 */
uint8_t nrf24l01_rx_packet(uint8_t *prxbuf)
{
    uint8_t sta;
    uint8_t rval = 1;

    sta = nrf24l01_read_reg(STATUS);                    /* 读取状态寄存器的值 */
    nrf24l01_write_reg(NRF_WRITE_REG + STATUS, sta);    /* 清除 RX_OK 中断标志 */

    if (sta & RX_OK)                                    /* 接收到数据 */
    {
        nrf24l01_read_buf(RD_RX_PLOAD, prxbuf, RX_PLOAD_WIDTH); /* 读取数据 */
        nrf24l01_write_reg(FLUSH_RX, 0xff);             /* 清除 RX FIFO 寄存器 */
        rval = 0;                                       /* 标记接收完成 */
    }

    return rval;                                        /* 返回结果 */
}

/**
 * @brief       NRF24L01 进入接收模式
 *   @note      设置 RX 地址,写 RX 数据宽度,选择 RF 频道,波特率
 *              当 CE 变高后,即进入 RX 模式,并可以接收数据了
 * @param       无
 * @retval      无
 */
void nrf24l01_rx_mode(void)
{
    NRF24L01_CE(0);
    nrf24l01_write_buf(NRF_WRITE_REG + RX_ADDR_P0, (uint8_t *)RX_ADDRESS,
RX_ADR_WIDTH);     /* 写 RX 节点地址 */

    nrf24l01_write_reg(NRF_WRITE_REG + EN_AA, 0x01);    /* 使能通道 0 的自动应答 */
    nrf24l01_write_reg(NRF_WRITE_REG + EN_RXADDR, 0x01);/* 使能通道 0 的接收地址*/
    nrf24l01_write_reg(NRF_WRITE_REG + RF_CH, 40);          /* 设置 RF 通信频率*/
    /* 选择通道 0 的有效数据宽度 */
    nrf24l01_write_reg(NRF_WRITE_REG + RX_PW_P0, RX_PLOAD_WIDTH);
    /* 设置 TX 发射参数,0db 增益,2Mbps */
    nrf24l01_write_reg(NRF_WRITE_REG + RF_SETUP, 0x0f);
    /* 配置基本工作模式的参数;PWR_UP,EN_CRC,16BIT_CRC,接收模式 */
    nrf24l01_write_reg(NRF_WRITE_REG + CONFIG, 0x0f);
    NRF24L01_CE(1); /* CE 为高,进入接收模式 */
}

/**
 * @brief       NRF24L01 进入发送模式
 *   @note      设置 TX 地址,写 TX 数据宽度,设置 RX 自动应答的地址,填充 TX 发送数据,选择 RF
频道,波特率和
 *              PWR_UP,CRC 使能
 *              当 CE 变高后,即进入 TX 模式,并可以发送数据了, CE 为高大于 10us,则启动发送.
 * @param       无
 * @retval      无
 */
void nrf24l01_tx_mode(void)
{
    NRF24L01_CE(0);
    nrf24l01_write_buf(NRF_WRITE_REG + TX_ADDR, (uint8_t *)TX_ADDRESS,
```

```
    TX_ADR_WIDTH);                          /* 写 TX 节点地址 */
    nrf24l01_write_buf(NRF_WRITE_REG + RX_ADDR_P0, (uint8_t *)RX_ADDRESS,
    RX_ADR_WIDTH);                          /* 设置 RX 节点地址,主要为了使能 ACK */

    nrf24l01_write_reg(NRF_WRITE_REG + EN_AA, 0x01);   /* 使能通道 0 的自动应答 */
    /* 使能通道 0 的接收地址 */
    nrf24l01_write_reg(NRF_WRITE_REG + EN_RXADDR, 0x01);
    /* 设置自动重发间隔时间:500us + 86us;最大自动重发次数:10 次 */
    nrf24l01_write_reg(NRF_WRITE_REG + SETUP_RETR, 0x1a);
    nrf24l01_write_reg(NRF_WRITE_REG + RF_CH, 40);     /* 设置 RF 通道为 40 */
    /* 设置 TX 发射参数,0db 增益,2Mbps */
    nrf24l01_write_reg(NRF_WRITE_REG + RF_SETUP, 0x0f);
    /* 配置基本工作模式的参数;PWR_UP,EN_CRC,16BIT_CRC,接收模式,开启所有中断 */
    nrf24l01_write_reg(NRF_WRITE_REG + CONFIG, 0x0e);
    NRF24L01_CE(1); /* CE 为高,10us 后启动发送 */
}
```

以上是 NRF24L01 的写寄存器函数和读寄存器函数，以及扩展的函数：在指定位置写入指定长度的数据函数和指定位置读取指定长度的数据函数。

NRF24L01 读写寄存器函数实现的具体过程：

先拉低片选线→发送寄存器号→发送数据/接收数据→拉高片选线。

nrf24l01_rx_mode -- Rx 模式初始化过程：

1） 写 Rx 节点的地址
2） 使能通道 x 自动应答
3） 使能通道 x 接收地址
4） 设置通信频率
5） 选择通道 x 的有效数据宽度
6） 配置发射参数（发射功率、无线速率）
7） 配置 NRF24L01 的基本参数以及工作模式

nrf24l01_tx_mode -- Tx 模式初始化过程：

1） 写 Tx 节点的地址
2） 写 Rx 节点的地址，主要为了使能硬件的自动应答
3） 使能通道 x 的自动应答
4） 使能通道 x 接收地址
5） 配置自动重发次数
6） 配置通信频率
7） 选择通道 x 的有效数据宽度
8） 配置发射参数（发射功率、无线速率）
9） 配置 NRF24L01 的基本参数以及切换工作模式

（4）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key, mode;
    uint16_t t = 0;
    uint8_t tmp_buf[33];
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);
```

```
/* GPIO 初始化 */
MX_GPIO_Init();

/* FSMC 初始化 */
MX_FSMC_Init();

/* 串口初始化 */
MX_USART1_UART_Init();

/* SPI2 初始化 */
MX_SPI2_Init();

/* USER CODE BEGIN 2 */
lcd_init();                             /* 初始化 LCD */
nrf24l01_init();                        /* 初始化 NRF24L01 */

lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
lcd_show_string(30, 70, 200, 16, 16, "NRF24L01 TEST", RED);
lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
while (nrf24l01_check())                /* 检查 NRF24L01 是否在线 */
{
    lcd_show_string(30, 110, 200, 16, 16, "NRF24L01 Error", RED);
    delay_ms(200);
    lcd_fill(30, 110, 239, 130 + 16, WHITE);
    delay_ms(200);
}

lcd_show_string(30, 110, 200, 16, 16, "NRF24L01 OK", RED);
/* USER CODE END 2 */

while (1)
{
    key = key_scan(0);

    if (key == KEY0_PRES)
    {
        mode = 0; /* 接收模式 */
        break;
    }
    else if (key == KEY1_PRES)
    {
        mode = 1; /* 发送模式 */
        break;
    }

    t++;

    if (t == 100) /* 显示提示信息 */
    {
        lcd_show_string(10, 130, 230, 16, 16, "KEY0:RX_Mode  KEY1:TX_Mode",
        RED);
    }

    if (t == 200) /* 关闭提示信息 */
    {
        lcd_fill(10, 130, 230, 150 + 16, WHITE);
        t = 0;
    }

    delay_ms(5);
}

lcd_fill(10, 130, 240, 166, WHITE);  /* 清空上面的显示 */
```

```
if (mode == 0)                          /* RX 模式 */
{
    lcd_show_string(30, 130, 200, 16, 16, "NRF24L01 RX_Mode", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "Received DATA:", BLUE);
    nrf24l01_rx_mode(); /* 进入 RX 模式 */

    while (1)
    {
        if (nrf24l01_rx_packet(tmp_buf) == 0) /* 一旦接收到信息,则显示出来. */
        {
            tmp_buf[32] = 0; /* 加入字符串结束符 */
            lcd_show_string(0, 170, lcddev.width - 1, 32, 16, (char
            *)tmp_buf, BLUE);
        }
        else
            delay_us(100);

        t++;

        if (t == 10000)         /* 大约 1s 钟改变一次状态 */
        {
            t = 0;
            LED0_TOGGLE();
        }
    }
}
else                            /* TX 模式 */
{
    lcd_show_string(30, 130, 200, 16, 16, "NRF24L01 TX_Mode", BLUE);
    nrf24l01_tx_mode();         /* 进入 TX 模式 */
    mode = ' ';                 /* 从空格键开始发送 */

    while (1)
    {
        if (nrf24l01_tx_packet(tmp_buf) == 0) /* 发送成功 */
        {
            lcd_show_string(30, 150, 239, 32, 16, "Sended DATA:", BLUE);
            lcd_show_string(0, 170, lcddev.width - 1, 32, 16, (char
            *)tmp_buf, BLUE);
            key = mode;

            for (t = 0; t < 32; t++)
            {
                key++;

                if (key > ('~'))
                    key = ' ';

                tmp_buf[t] = key;
            }

            mode++;

            if (mode > '~')
                mode = ' ';

            tmp_buf[32] = 0; /* 加入结束符 */
        }
        else
        {
            lcd_fill(0, 150, lcddev.width, 170 + 16 * 3, WHITE); /*清空显示*/
            lcd_show_string(30, 150, lcddev.width - 1, 32, 16, "Send Failed
            ", BLUE);
        }
```

```
            LED0_TOGGLE();
            delay_ms(200);
        }
    }
}
```

程序运行时先通过 nrf24l01_cheak 函数检测 NRF24L01 是否存在，如果存在，则让用户选择发送模式还是接收模式，在确定模式之后，设置 NRF24L01 的工作模式，然后执行相对应的数据发送/接收处理。

52.3 下载验证

将程序下载到开发板后，可以看到 LCD 显示的内容如图 52.3.1 所示：

图 52.3.1 选择工作模式图

通过 KEY0 和 KEY1 来选择 NRF24L01 模块所要进入的工作模式，我们两个开发板一个选择发送模式，一个选择接收模式就可以了。设置好的通信界面如图 52.3.2 和图 52.3.3 所示：

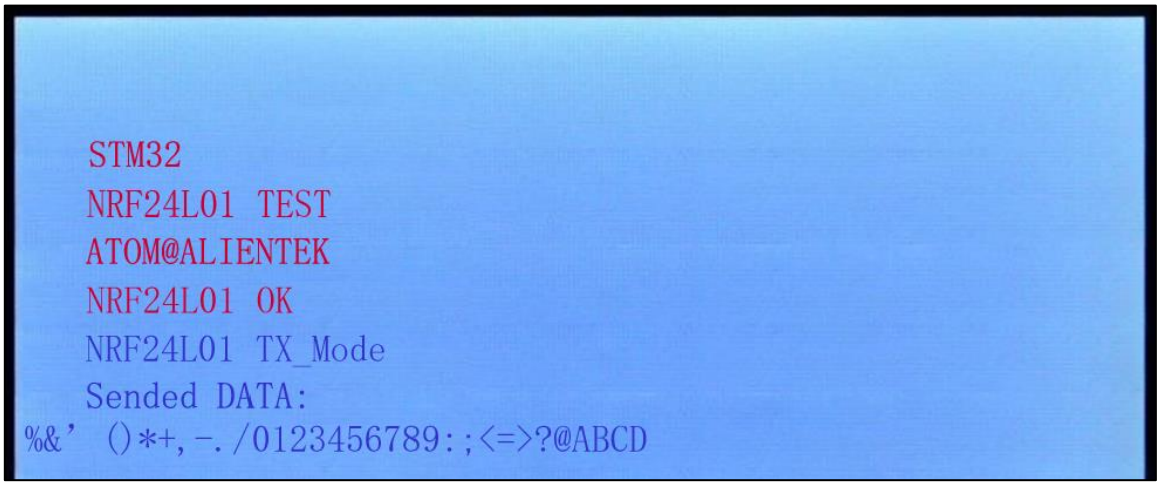


图 52.3.2 开发板 A 发送数据

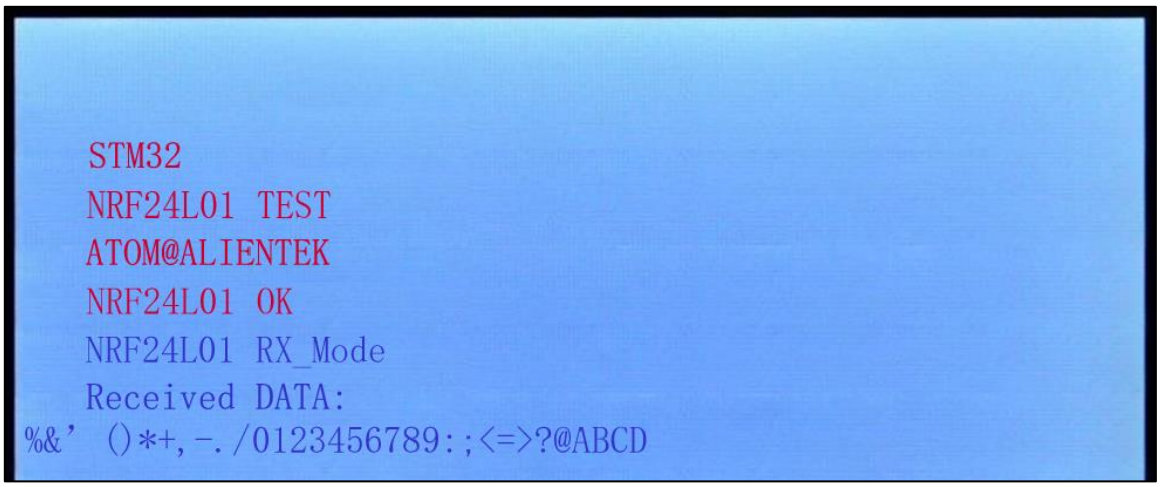


图 52.3.3 开发板 B 接收数据

图 52.3.2 来自于开发板 A，工作在发送模式。图 52.3.3 来自于开发板 B，工作在接收模式，A 发送，B 接收。发送和接收图片的数据不一样，是因为我们拍照的时间不一样导致的。大家看到收发数据是一致，那就说明实验成功。

第五十三章 FLASH 模拟 EEPROM 实验

STM32 本身没有自带 EEPROM，但是 STM32 具有 IAP（在应用编程）功能，所以我们可以把它的 FLASH 当成 EEPROM 来使用。

本章分为如下几个小节：

53.1 硬件设计
53.2 软件设计
53.3 下载验证

53.1 硬件设计

1. 例程功能

按键 KEY1 控制写入 FLASH 的操作，按键 KEY0 控制读出操作，并在 TFTLCD 模块上显示相关信息，还可以借助 USMART 进行读取或者写入操作。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键
KEY0 - PE4 KEY1 - PE3

3. 原理图

FLASH 属于 STM32F103 的内部资源，只需要软件设置好即可正常工作。

53.2 程序设计

53.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 31_FLASH_EEPROM。

53.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 14_USMART 工程，并将工程文件名和.ioc 文件名均修改为 31_FLASH_EEPROM，以进行下一步的实验操作。如下图所示：

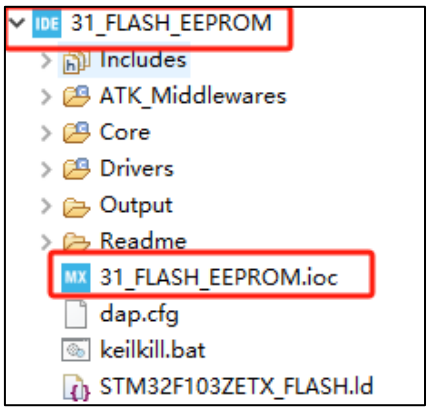


图 53.2.2.1 FLASH_EEPROM 工程

本实验关于 FLASH 模拟 EEPROM 的驱动内容均在代码中自己实现，无需再 STM32CubeIDE 上另外配置。所以我们复制好工程后，直接添加用户驱动代码。

53.2.3 添加用户驱动代码

在 Drivers/BSP 文件夹中，新建 STMFLASH 文件夹，在 STMFLASH 中，我们添加 2 个驱动代码，分别是：stmflash.c、stmflash.h。如下图所示：

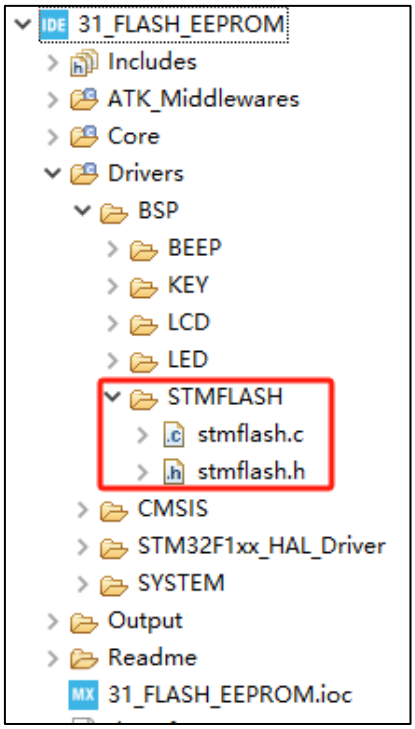


图 53.2.3.1 添加用户文件

下面我们开始添加用户代码：

（1）stmflash.h 文件代码

```
#ifndef __STMFLASH_H
#define __STMFLASH_H

#include "../../SYSTEM/sys/sys.h"


/* FLASH 起始地址 */
#define STM32_FLASH_SIZE        0x80000         /* STM32 FLASH 总大小 */
```

```
#define STM32_FLASH_BASE        0x08000000      /* STM32 FLASH 起始地址 */

 /* STM32F103 扇区大小 */
#if STM32_FLASH_SIZE < 256 * 1024
#define STM32_SECTOR_SIZE   1024    /* 容量小于 256K 的 F103, 扇区大小为 1K 字节 */
#else
#define STM32_SECTOR_SIZE   2048    /* 容量大于等于 256K 的 F103, 扇区大小为 2K 字节*/
#endif

/* FLASH 解锁键值 */
#define STM32_FLASH_KEY1        0X45670123
#define STM32_FLASH_KEY2        0XCDEF89AB

/* 接口函数(外部可调用) */
uint16_t stmflash_read_halfword(uint32_t faddr);   /* FLASH 读半字 */
/* 从指定地址开始读出指定长度的数据 */
void stmflash_read(uint32_t raddr, uint16_t *pbuf, uint16_t length);
/* 在 FLASH 指定位置, 写入指定长度的数据(自动擦除) */
void stmflash_write(uint32_t waddr, uint16_t *pbuf, uint16_t length);

/* 测试函数 */
void test_write(uint32_t waddr, uint16_t wdata);

#endif
```

STM32_FLASH_BASE 和 STM32_FLASH_SIZE 分别是 FLASH 的起始地址和 FLASH 总大小，这两个宏定义随着芯片是固定的，我们开发板 F103 芯片的 FLASH 是 512K 字节，所以 STM32_FLASH_SIZE 宏定义值为 0x80000。

（2）stmflash.c 文件代码

```
/**
 * @brief       从指定地址读取一个半字 (16 位数据)
 * @param       faddr   : 读取地址 (此地址必须为 2 的倍数!!)
 * @retval      读取到的数据 (16 位)
 */
uint16_t stmflash_read_halfword(uint32_t faddr)
{
    return *(volatile uint16_t *)faddr;
}

/**
 * @brief       从指定地址开始读出指定长度的数据
 * @param       raddr : 起始地址
 * @param       pbuf  : 数据指针
 * @param       length: 要读取的半字(16 位)数,即 2 个字节的整数倍
 * @retval      无
 */
void stmflash_read(uint32_t raddr, uint16_t *pbuf, uint16_t length)
{
    uint16_t i;

    for (i = 0; i < length; i++)
    {
        pbuf[i] = stmflash_read_halfword(raddr);    /* 读取 2 个字节 */
        raddr += 2; /* 偏移 2 个字节 */
    }
}

/**
 * @brief       不检查的写入
 *              这个函数的假设已经把原来的扇区擦除过再写入
 * @param       waddr   : 起始地址 (此地址必须为 2 的倍数!!,否则写入出错!)
```

```
 * @param       pbuf    : 数据指针
 * @param       length  : 要写入的 半字(16 位)数
 * @retval      无
 */
void stmflash_write_nocheck(uint32_t waddr, uint16_t *pbuf, uint16_t length)
{
    uint16_t i;
    for (i = 0; i < length; i++)
    {
        HAL_FLASH_Program(FLASH_TYPEPROGRAM_HALFWORD, waddr, pbuf[i]);
        waddr += 2; /* 指向下一个半字 */
    }
}

/**
 * @brief       在 FLASH 指定位置, 写入指定长度的数据(自动擦除)
 *   @note      该函数往 STM32 内部 FLASH 指定位置写入指定长度的数据
 *              该函数会先检测要写入的扇区是否是空(全 0XFFFF)的?, 如果
 *              不是, 则先擦除, 如果是, 则直接往扇区里面写入数据.
 *              数据长度不足扇区时, 自动被回擦除前的数据
 * @param       waddr   : 起始地址 (此地址必须为 2 的倍数!!,否则写入出错!)
 * @param       pbuf    : 数据指针
 * @param       length  : 要写入的 半字(16 位)数
 * @retval      无
 */
uint16_t g_flashbuf[STM32_SECTOR_SIZE / 2]; /* 最多是 2K 字节 */
void stmflash_write(uint32_t waddr, uint16_t *pbuf, uint16_t length)
{
    uint32_t secpos;         /* 扇区地址 */
    uint16_t secoff;         /* 扇区内偏移地址(16 位字计算) */
    uint16_t secremain;      /* 扇区内剩余地址(16 位字计算) */
    uint16_t i;
    uint32_t offaddr;        /* 去掉 0X08000000 后的地址 */
    FLASH_EraseInitTypeDef flash_eraseop;
    uint32_t erase_addr;     /* 擦除错误，这个值为发生错误的扇区地址 */

    if (waddr < STM32_FLASH_BASE || (waddr >= (STM32_FLASH_BASE + 1024 *
    STM32_FLASH_SIZE)))
    {
        return;                                     /* 非法地址 */
    }

    HAL_FLASH_Unlock();                             /* FLASH 解锁 */

    offaddr = waddr - STM32_FLASH_BASE;             /* 实际偏移地址. */
    secpos = offaddr / STM32_SECTOR_SIZE;           /* 扇区地址  0~255 */
    secoff = (offaddr % STM32_SECTOR_SIZE) / 2; /* 在扇区内的偏移 2 个字节为单位 */
    secremain = STM32_SECTOR_SIZE / 2 - secoff; /* 扇区剩余空间大小 */
    if (length <= secremain)
    {
        secremain = length; /* 不大于该扇区范围 */
    }

    while (1)
    {
        stmflash_read(secpos * STM32_SECTOR_SIZE + STM32_FLASH_BASE,
        g_flashbuf, STM32_SECTOR_SIZE / 2);     /* 读出整个扇区的内容 */
        for (i = 0; i < secremain; i++)         /* 校验数据 */
        {
            if (g_flashbuf[secoff + i] != 0XFFFF)
            {
                break;      /* 需要擦除 */
```

```
            }
        }
        if (i < secremain)  /* 需要擦除 */
        {
            flash_eraseop.TypeErase = FLASH_TYPEERASE_PAGES;  /* 选择页擦除 */
            flash_eraseop.Banks = FLASH_BANK_1;
            flash_eraseop.NbPages = 1;
            flash_eraseop.PageAddress = secpos * STM32_SECTOR_SIZE +
            STM32_FLASH_BASE;                                 /* 要擦除的扇区 */
            HAL_FLASHEx_Erase( &flash_eraseop, &erase_addr);

            for (i = 0; i < secremain; i++)                   /* 复制 */
            {
                g_flashbuf[i + secoff] = pbuf[i];
            }
            stmflash_write_nocheck(secpos * STM32_SECTOR_SIZE +
        STM32_FLASH_BASE, g_flashbuf, STM32_SECTOR_SIZE / 2); /* 写入整个扇区 */
        }
        else
        {
            /* 写已经擦除了的,直接写入扇区剩余区间. */
            stmflash_write_nocheck(waddr, pbuf, secremain);
        }
        if (length == secremain)
        {
            break; /* 写入结束了 */
        }
        else        /* 写入未结束 */
        {
            secpos++;                /* 扇区地址增 1 */
            secoff = 0;              /* 偏移位置为 0 */
            pbuf += secremain;       /* 指针偏移 */
            waddr += secremain * 2; /* 写地址偏移(16 位数据地址,需要*2) */
            length -= secremain;     /* 字节(16 位)数递减 */
            if (length > (STM32_SECTOR_SIZE / 2))
            {
                secremain = STM32_SECTOR_SIZE / 2; /* 下一个扇区还是写不完 */
            }
            else
            {
                secremain = length; /* 下一个扇区可以写完了 */
            }
        }
    }

    HAL_FLASH_Lock();    /* 上锁 */
}

/*****************************************************************************/
/* 测试用代码 */

/**
 * @brief       测试写数据(写 1 个字)
 * @param       waddr : 起始地址
 * @param       wdata : 要写入的数据
 * @retval      读取到的数据
 */
void test_write(uint32_t waddr, uint16_t wdata)
{
    stmflash_write(waddr, &wdata, 1);    /* 写入一个半字 */
}
```

该函数主要包含了片上 FLASH 的读写函数，首先 Flash 的读取十分简单，仅需读取对应

地址的数据即可。写入函数相对复杂些，需要注意几点：

1，写入地址必须是用户代码区以外的地址

2，写入地址必须是 2 的倍数。

3，在写 Flash 前需要先判断待写入的比特位是否为 1，若不为 1 则需要先进行擦除操作，否则将写入失败。

第 1 点比较好理解，如果把用户代码给擦了，可想而知你运行的程序可能就被废了，从而很可能出现死机的情况。第 2 点则是 STM32 FLASH 的要求，每次必须写入 16 位，如果你写的地址不是 2 的倍数，那么写入的数据，可能就不是写在你要写的地址了。第 3 点则是 FLASH 的写入特性。

（3）main.c 文件代码

```
const uint8_t g_text_buf[] = {"STM32 FLASH TEST"};

#define TEXT_LENTH sizeof(g_text_buf) /* 数组长度 */

/*SIZE 表示半字长(2 字节)，大小必须是 2 的整数倍，如果不是的话，强制对齐到 2 的整数倍 */
#define SIZE TEXT_LENTH / 2 + ((TEXT_LENTH % 2) ? 1 : 0)
/* 设置 FLASH 保存地址(必须为偶数，且其值要大于本代码所占用 FLASH 的大小 + 0X08000000) */
#define FLASH_SAVE_ADDR 0X08070000

int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key = 0;
    uint8_t i = 0;
    uint8_t datatemp[SIZE];
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                    /* 初始化 LCD */
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "FLASH EEPROM TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY1:Write  KEY0:Read", RED);
  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY1_PRES) /* KEY1 按下,写入 STM32 FLASH */
```

```
        {
            lcd_fill(0, 150, 239, 319, WHITE); /* 清除半屏 */
            lcd_show_string(30, 150, 200, 16, 16, "Start Write FLASH....", RED);
            stmflash_write(FLASH_SAVE_ADDR, (uint16_t *)g_text_buf, SIZE);
            lcd_show_string(30, 150, 200, 16, 16, "FLASH Write Finished!", RED);
        }

        if (key == KEY0_PRES) /* KEY0 按下,读取字符串并显示 */
        {
            lcd_show_string(30, 150, 200, 16, 16, "Start Read FLASH.... ", RED);
            stmflash_read(FLASH_SAVE_ADDR, (uint16_t *)datatemp, SIZE);
            lcd_show_string(30, 150, 200, 16, 16, "The Data Readed Is:  ", RED);
            lcd_show_string(30, 170, 200, 16, 16, (char *)datatemp, BLUE);
        }

        i++;
        delay_ms(10);

        if (i % 20 == 0)
        {
            LED0_TOGGLE(); /* 提示系统正在运行 */
            i = 0;
        }
    }
}
```

主函数代码逻辑比较简单，当检测到按键 KEY1 按下后往 FLASH 指定地址开始的连续地址空间写入一段数据，当检测到按键 KEY0 按下后读取 FLASH 指定地址开始的连续空间数据。最后，我们将 stmflash_read_word 和 test_write 函数加入 USMART 控制，这样，我们就可以通过串口调试助手，调用 STM32F103 的 FLASH 读写函数，方便测试。

53.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 53.3.1 所示：

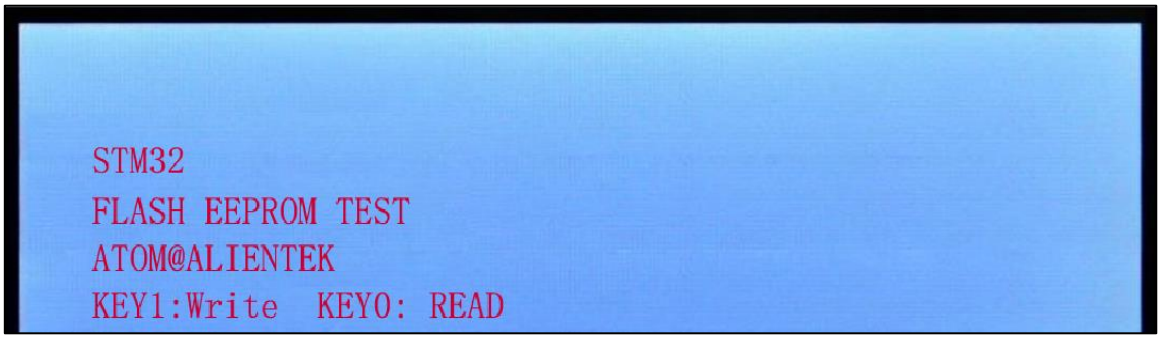


图 53.3.1 程序运行效果图

通过先按 KEY1 按键写入数据，然后按 KEY0 读取数据，得到如图 53.3.2 所示：

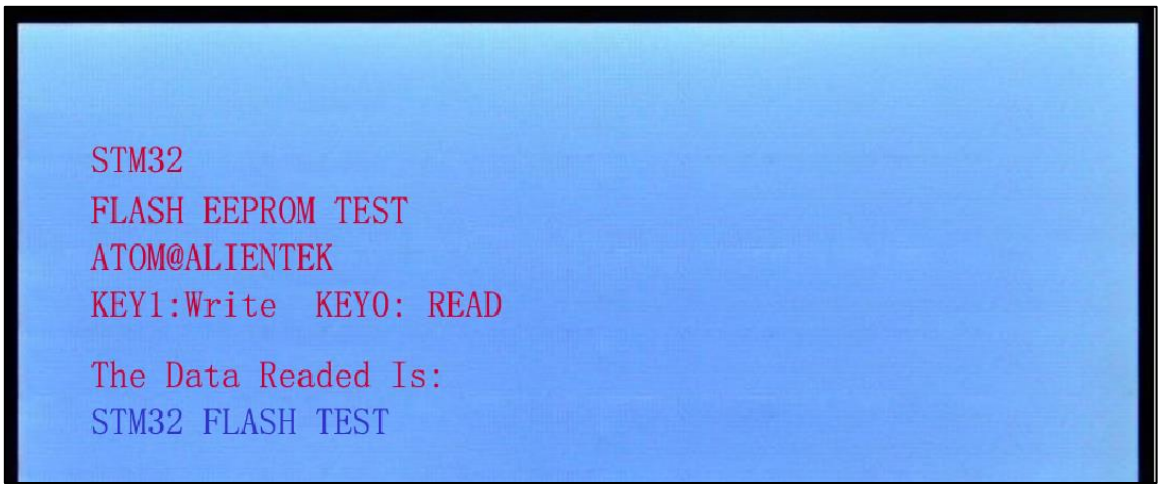


图 53.3.2 操作后的显示效果图

本实验的测试，我们还可以借助 USMART，调用：stmflash_read_word 和 test_write 函数进行测试！

第五十四章 摄像头实验

正点原子精英 STM32 开发板板载了一个摄像头接口（P4），该接口可以用来连接正点原子 OV7725 摄像头模块。本章，我们将使用 STM32 驱动正点原子 OV7725 摄像头模块，实现摄像头功能。

本章分为如下几个部分：

54.1 硬件设计
54.2 程序设计
54.3 下载验证

54.1 硬件设计

1. 例程功能

开机后，检测和初始化 OV7725 摄像头模块。初始化成功后需要先通过 KEY0 和 KEY1 选择为 QVGA 或 VGA 输出模式，然后 LCD 才会显示拍摄到的画面。

正常显示拍摄画面后，我们可以通过 KEY0 设置光照模式、通过 KEY1 设置色饱和度，通过 KEY2 设置亮度，通过 KEY_UP 设置对比度，通过 TPAD 设置特效（总共 7 种特效）。通过串口，我们可以查看当前的帧率（这里是指 LCD 显示的帧率，而不是指 OV7725 的输出帧率），同时可以借助 USMART 设置 OV7725 的寄存器，方便大家调试。LED0 指示程序运行状态。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键: KEY0 - PE4、KEY1 - PE3、KEY2 - PE2、WK_UP - PA0
5）电容按键: PA1，用于控制触摸按键 TPAD。
6）外部中断 8（连接 PA8，用于检测 OV7725 的帧信号）
7）定时器 6（用于打印摄像头帧率等信息）
8）正点原子 OV7725 摄像头模块，通讯线的连接关系为：

OV7725 模块	STM32 开发板	OV7725 模块	STM32 开发板
OV_D0~D7	PC0~7	FIFO_OE	PG15
OV_SCL	PD3	FIFO_WRST	PD6
OV_SDA	PG13	FIFO_WEN	PB3
OV_VSYNC	PA8	FIFO_RCLK	PB4
FIFO_RRST	PG14		

表 54.1.1 OV7725 模块与开发板连接关系

这部分的连线，模块与开发板上的 P4 座子已经对应好了，安装时直接把模块镜头背离开发板的方向安装即可(建议安装所有模块的时候断电安装)。

图 54.1.1 开发板上连接 OV7725 模块的座子

3. 原理图

本章实验使用了一个正点原子 OV7725 摄像头模块，可直接与板载的 CAMERA 接口进行连接，该接口与板载 MCU 的连接原理图，如下图所示：

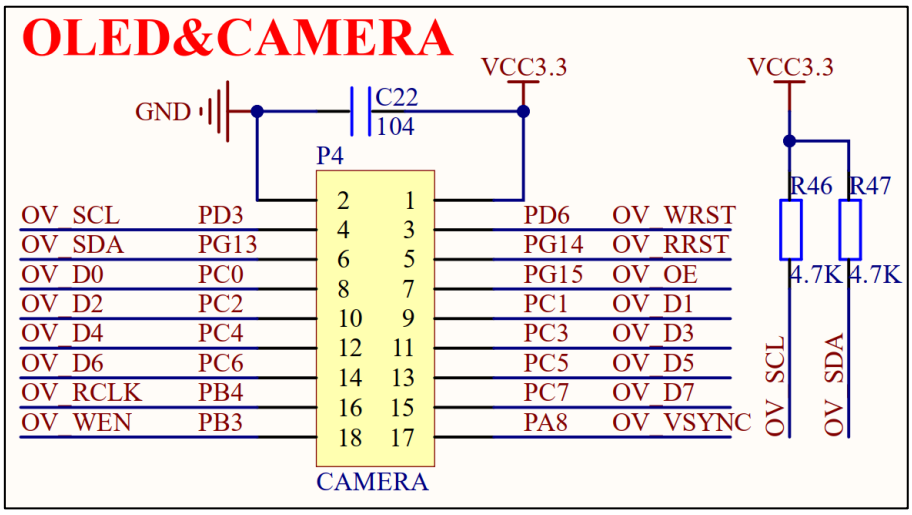


图 54.1.2 摄像头接口原理图

54.2 程序设计

54.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 32_OV7725_Camera。

54.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 32_OV7725_Camera，以进行下一步的实验操作。如下图所示：

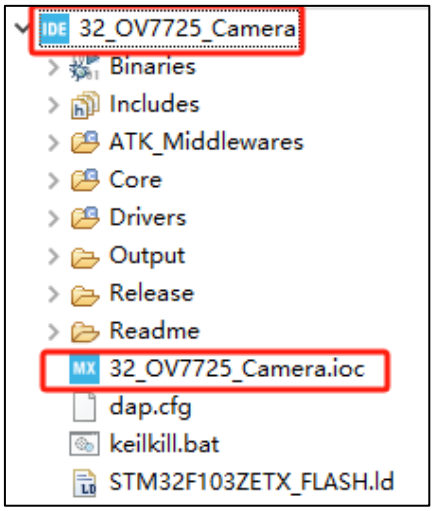


图 54.2.2.1 OV7725_Camera 工程

本实验所涉及的 IO 非常多，所以大家配置时一定要细心检查下，下面我们开始配置 GPIO。

（1）GPIO 配置

双击 32_OV7725_Camera.ioc 文件，进入 STM32CubeMX 插件。首先，将 PC0 到 PC7 这 8 个 IO 口设置为 GPIO_Input 模式，用于读取摄像头数据。然后，将 PA8 设置为外部中断，上升沿触发，用于检测帧信号。此外，将 PG13 设置为开漏输出，其余引脚均为推挽输出。具体配置详见源码配套例程。

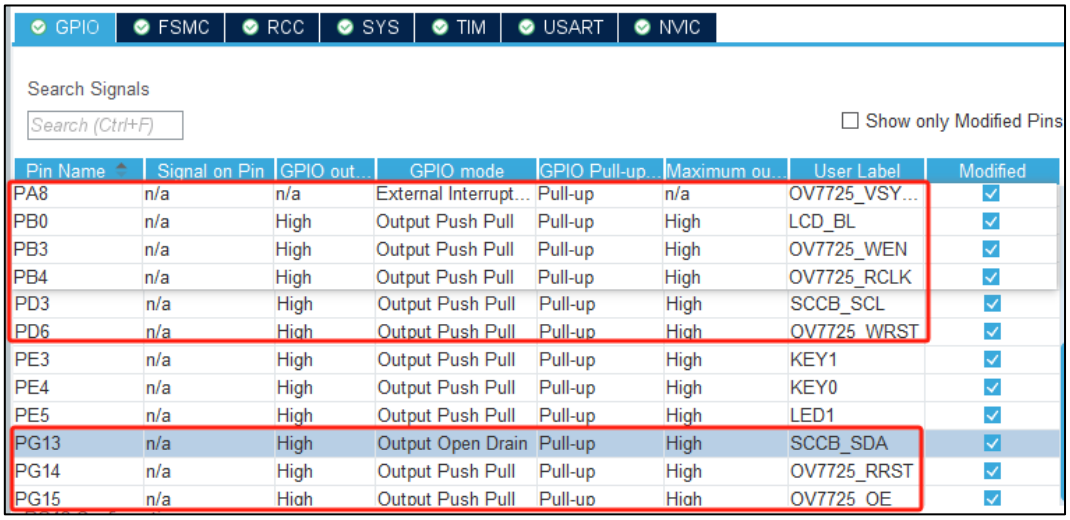


图 54.2.2.2 GPIO 配置

（2）TIM 配置

在 Timers->TIM6 处我们配置定时器 6 的参数，如下配置：

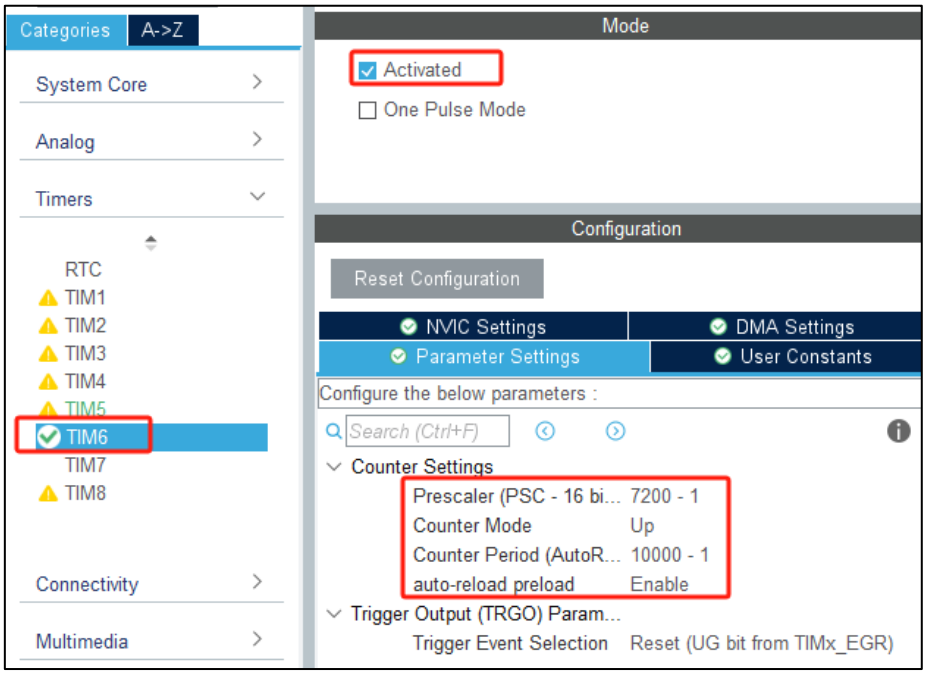


图 54.2.2.3 TIM6 参数配置

关于定时器的参数介绍，前面的章节已经介绍过了，这里不在赘述。我们只需注意我这里配置的是 1s 溢出一次，用于统计帧率。

接着配置 TPAD 电容触摸按键所需要用到的定时器 TIM5_CH2（PA1），如下：

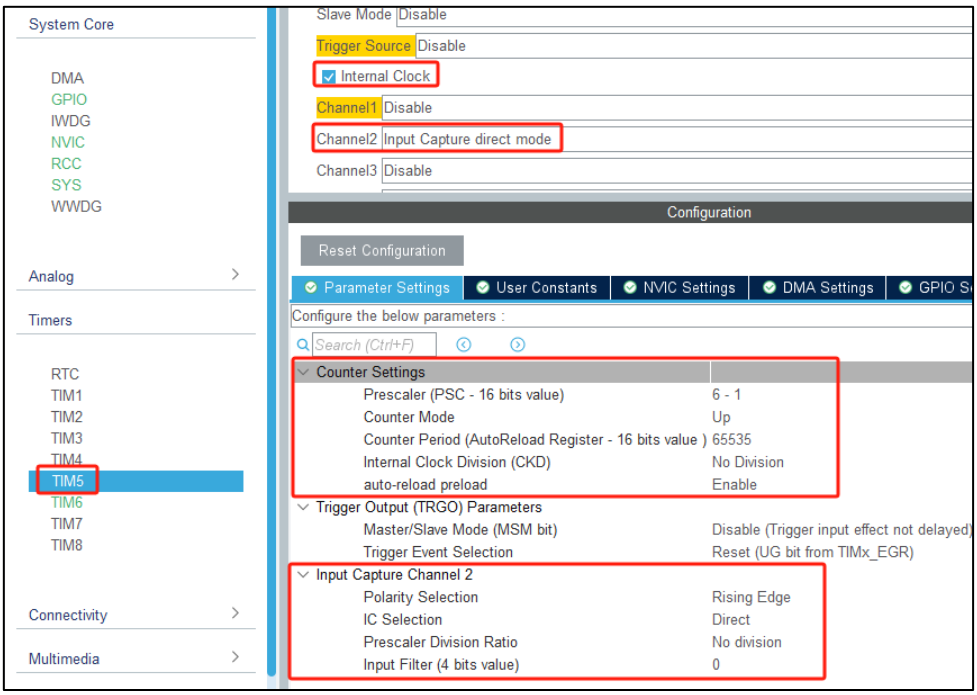


图 54.2.2.4 TIM5 参数配置

这部分的配置在 TPAD 章节已详细说明，感兴趣可以翻看下 TPAD 章节。

（3）NVIC 配置

定时器 6 要每 1 秒溢出产生中断；外部中断线则是用于检测帧信号，所以我们要使能对应中断，并配置中断优先级。如下图，

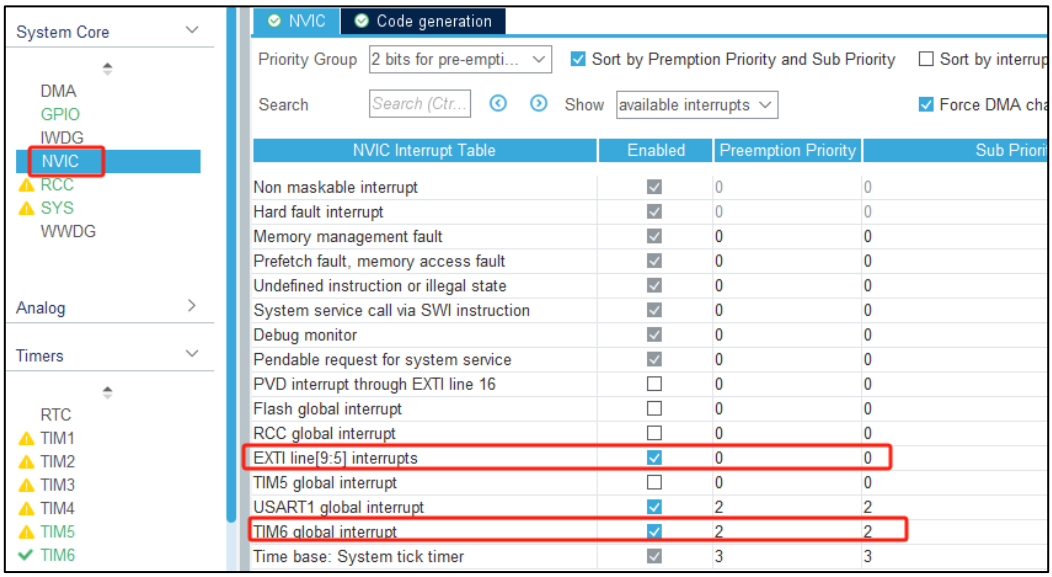


图 54.2.2.5 NVIC 配置

54.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，在 gpio.c 中就初始化了我们前面所配置的所有 IO，tim.c 则是将 TIM6 进行了初始化。

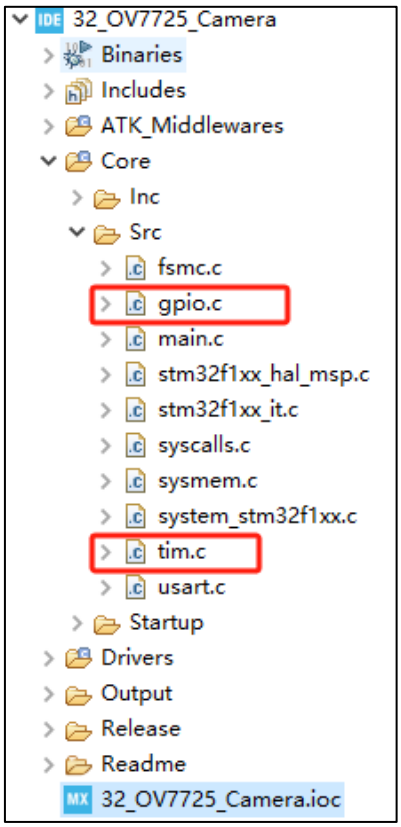


图 54.2.3.1 生成工程

54.2.4 添加用户驱动代码

首先将 TPAD 实验中的 TPAD 文件夹拷贝到此工程的 Drivers/BSP 文件夹中。然后在 BSP 文件夹下新建 OV7725 和 EXTI 文件夹，接着再 OV7725 文件夹下新建 ov7725.c、ov7725.h、ov7725cfg.h、sccb.c 和 sccb.h；再 EXTI 文件夹下新建 exit.c 和 exit.h，如下图所示：

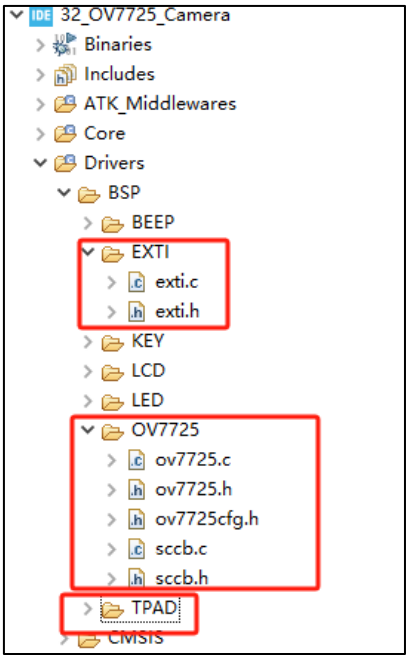


图 54.2.4.1 添加用户文件

（1）sccb.c 和 sccb.h 文件代码

本章实验的 SCCB 驱动主要负责向 OV7725 驱动提供配置 OV7725 摄像头的各种函数，SCCB 协议与 IIC 协议十分相似，也可兼容 IIC 协议，因此请读者结合 SCCB 和 IIC 协议的相关文档查看本章实验配套实验源码中 SCCB 的相关驱动文件。本章实验中，SCCB 的驱动代码包括 sccb.c 和 sccb.h 两个文件。

（2）ov7725cfg.h 文件代码

该文件主要用于定义 OV7725 摄像头的寄存器以及对应的寄存器值。由于篇幅过长这里不展示，感兴趣可以查看本章实验配套实验源码中 ov7725cfg.h 文件。

（3）ov7725.h 文件代码

```
#ifndef _OV7725_H
#define _OV7725_H

#include "../../SYSTEM/sys/sys.h"


/*****************************************************************************/
/* OV7725 模块的的 VSYNC/WRST/RRST/OE/RCLK/WEN 引脚 定义
 * D0~D7, 由于引脚多, 且连续, 这里就不在这里定义了,直接在 ov7725_init 里面修改.所以在移植
 * 的时候, 除了改这 6 个 IO 口, 还得改 ov7725_init 里面的 D0~D7 所在的 IO 口.
 */
/*****************************************************************************/

/* OV7725 相关端口定义 */
#define OV7725_VSYNC(x)    do{ x ? \
                               HAL_GPIO_WritePin(OV7725_VSYNC_GPIO_Port,
                               OV7725_VSYNC_Pin, GPIO_PIN_SET) : \
                               HAL_GPIO_WritePin(OV7725_VSYNC_GPIO_Port,
                               OV7725_VSYNC_Pin, GPIO_PIN_RESET); \
                           }while(0)         /* VSYNC */

#define OV7725_WRST(x)     do{ x ? \
                               HAL_GPIO_WritePin(OV7725_WRST_GPIO_Port,
                               OV7725_WRST_Pin, GPIO_PIN_SET) : \
                               HAL_GPIO_WritePin(OV7725_WRST_GPIO_Port,
                               OV7725_WRST_Pin, GPIO_PIN_RESET); \
                           }while(0)         /* WRST */

#define OV7725_RRST(x)     do{ x ? \
                               HAL_GPIO_WritePin(OV7725_RRST_GPIO_Port,
                               OV7725_RRST_Pin, GPIO_PIN_SET) : \
                               HAL_GPIO_WritePin(OV7725_RRST_GPIO_Port,
                               OV7725_RRST_Pin, GPIO_PIN_RESET); \
                           }while(0)         /* RRST */

#define OV7725_OE(x)       do{ x ? \
                               HAL_GPIO_WritePin(OV7725_OE_GPIO_Port,
                               OV7725_OE_Pin, GPIO_PIN_SET) : \
                               HAL_GPIO_WritePin(OV7725_OE_GPIO_Port,
                               OV7725_OE_Pin, GPIO_PIN_RESET); \
                           }while(0)         /* OE */

#define OV7725_RCLK(x)       x ? (OV7725_RCLK_GPIO_Port->BSRR =
OV7725_RCLK_Pin) : (OV7725_RCLK_GPIO_Port->BRR = OV7725_RCLK_Pin)     /* RCLK
*/

#define OV7725_WEN(x)      do{ x ? \
                               HAL_GPIO_WritePin(OV7725_WEN_GPIO_Port,
                               OV7725_WEN_Pin, GPIO_PIN_SET) : \
```

```
                            HAL_GPIO_WritePin(OV7725_WEN_GPIO_Port,
                            OV7725_WEN_Pin, GPIO_PIN_RESET); \
                          }while(0)          /* WEN */

#define OV7725_DATA         GPIOC->IDR & 0X00FF
/* D0~D7 */


/* OV7725 ID */
#define OV7725_MID              0X7FA2      /* MID , 第一个 MID*/
#define OV7725_MID1             0X7FFF      /* MID1, 第二个 MID */
#define OV7725_PID              0X7721

/* OV7670 的 SCCB 地址 */
#define OV7725_ADDR             0X42


uint8_t ov7725_init(void);                          /* OV7725 初始化 */
uint8_t ov7725_read_reg(uint16_t reg);              /* OV7725 读寄存器 */
uint8_t ov7725_write_reg(uint8_t reg, uint8_t data);/* OV7725 写寄存器 */

void ov7725_light_mode(uint8_t mode);       /* OV7725 灯光模式设置 */
void ov7725_color_saturation(uint8_t sat);  /* OV7725 色彩饱和度设置 */
void ov7725_brightness(uint8_t bright);     /* OV7725 亮度设置 */
void ov7725_contrast(uint8_t contrast);     /* OV7725 对比度设置 */
void ov7725_special_effects(uint8_t eft);   /* OV7725 特效设置 */
void ov7725_window_set(uint16_t width, uint16_t height, uint8_t mode);

#endif
```

该文件包含了引脚操作的宏定义，以及 7725 摄像头相关的一些 ID 和器件地址的宏定义，便于操作。

（4）ov7725.c 文件代码

```
/**
 * @brief       OV7725 读寄存器
 * @param       reg : 寄存器地址
 * @retval      读到的寄存器值
 */
uint8_t ov7725_read_reg(uint16_t reg)
{
    uint8_t data = 0;

    sccb_start();                       /* 起始信号 */
    sccb_send_byte(OV7725_ADDR);        /* 写通讯地址 */
    sccb_send_byte(reg);                /* 寄存器地址 */
    sccb_stop();                        /* 停止信号 */

    /* 设置寄存器地址后，才是读 */
    sccb_start();                       /* 起始信号 */
    sccb_send_byte(OV7725_ADDR | 0X01); /* 读通讯地址 */
    data = sccb_read_byte();            /* 读取数据 */
    sccb_nack();                        /* 非应答信号 */
    sccb_stop();                        /* 停止信号 */

    return data;
}

/**
 * @brief       OV7725 写寄存器
 * @param       reg : 寄存器地址
```

```
 * @param       data: 要写入寄存器的值
 * @retval      0, 成功; 1, 失败;
 */
uint8_t ov7725_write_reg(uint8_t reg, uint8_t data)
{
    uint8_t res = 0;

    sccb_start();   /* 起始信号 */

    if (sccb_send_byte(OV7725_ADDR))res = 1;     /* 写通讯地址 */

    if (sccb_send_byte(reg))res = 1;             /* 寄存器地址 */

    if (sccb_send_byte(data))res = 1;            /* 写数据 */

    sccb_stop();    /* 停止信号 */

    return res;
}

/**
 * @brief       初始化 OV7725
 * @param       无
 * @retval      0, 成功; 1, 失败;
 */
uint8_t ov7725_init(void)
{
    uint16_t i = 0;
    uint16_t reg = 0;

    OV7725_WRST(1);             /* WRST =1 */
    OV7725_RRST(1);             /* RRST =1 */
    OV7725_OE(1);               /* OE =1 */
    OV7725_RCLK(1);             /* RCLK =1 */
    OV7725_WEN(1);              /* WEN =1 */

    sccb_init();                /* 初始化 SCCB 的 IO 口 */

    if (ov7725_write_reg(0x12, 0x80))   /* 复位 */
    {
        return 1;
    }

    delay_ms(50);
    reg = ov7725_read_reg(0X1c);    /* 读取厂家 ID 高八位 */
    reg <<= 8;
    reg |= ov7725_read_reg(0X1d);   /* 读取厂家 ID 低八位 */

    if ((reg != OV7725_MID) && (reg != OV7725_MID1))    /* MID 不正确 ? */
    {
        printf("MID:%d\r\n", reg);
        return 1;
    }

    reg = ov7725_read_reg(0X0a);    /* 读取厂家 ID 高八位 */
    reg <<= 8;
    reg |= ov7725_read_reg(0X0b);   /* 读取厂家 ID 低八位 */

    if (reg != OV7725_PID)          /* PID 不正确 ? */
    {
        printf("HID:%d\r\n", reg);
        return 2;
    }
```

```
    /* 初始化 OV7725,采用 QVGA 分辨率(320*240) */
    for (i = 0; i < sizeof(ov7725_init_reg_tb1) /
    sizeof(ov7725_init_reg_tb1[0]); i++)
    {
        ov7725_write_reg(ov7725_init_reg_tb1[i][0],
         ov7725_init_reg_tb1[i][1]);
    }

    return 0;    /* ok */
}

/* 省略部分代码... ... */
```

通过 ov7725_init 函数就完成了 OV7725 的基本配置，OV7725 就会以配置的 QVGA 模式输出图像数据。每当 OV7725 输出一个帧信号 VSYNC，就代表一帧图像数据就要通过数据引脚进行输出了，因此我们可以利用 STM32 的外部中断来捕获这个信号，进而在中断服务函数里，把图像数据写入到 FIFO 中。

关于外部中断捕获帧信号的这部分操作在 exit.c 中实现。

（5）exit.c 文件代码

```
/* OV7725 帧中断标志
 * 0, 表示上一帧数据已经处理完了, 新的帧中断到来, 可以往 FIFO 写数据
 * 1, 表示上一帧数据还没有处理完, 新的帧中断到来, 不能往 FIFO 写数据
 */
uint8_t g_ov7725_vsta = 0;

/**
 * @brief       OV7725 VSYNC 外部中断服务程序
 * @param       无
 * @retval      无
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == OV7725_VSYNC_Pin)  /* 是 OV7725_VSYNC_Pin 线的中断 */
    {
        if (g_ov7725_vsta == 0)        /* 上一帧数据已经处理了? */
        {
            OV7725_WRST(0);            /* 复位写指针 */
            OV7725_WRST(1);            /* 结束复位 */
            OV7725_WEN(1);             /* 允许写入 FIFO */
            g_ov7725_vsta = 1;         /* 标记帧中断 */
        }
        else
        {
            OV7725_WEN(0);             /* 禁止写入 FIFO */
        }
    }
    /* 清除 OV7725_VSYNC_Pin 上的中断标志位 */
    __HAL_GPIO_EXTI_CLEAR_IT(OV7725_VSYNC_Pin);
}
```

在外部中断回调函数中，需要先判断中断是不是来自中断线 8 再进行处理。

中断处理的部分流程：每当帧中断到来后，先判断 g_ov7725_vsta 的值是否为 0，如果是 0，说明可以往 FIFO 里面写入数据，执行复位 FIFO 写指针，并允许 FIFO 写入。此时，AL422B 将从地址 0 开始，存储新一帧的图像数据。然后设置 g_ov7725_vsta 为 1，标记新的一帧数据正在存储中。如果 g_ov7725_vsta 不为 0，说明之前存储在 FIFO 里面的一帧数据还未被读取过，直接禁止 FIFO 写入，等待 MCU 读取 FIFO 数据，以免数据覆盖。

然而，STM32 只需要判断 g_ov7725_vsta 是否为 1，来读取 FIFO 里面的数据，读完一帧

后，设置 g_ov7725_vsta 为 0，以免重复读取，同时还可以使能 FIFO 新一帧数据的写入。

exit.h 头文件仅包含了相关的一些头文件，这里不展开讲解，详情可查看配套源码。

（6）main.c 文件代码

```
/**
 * @brief       更新 LCD 显示
 *   @note      该函数将 OV7725 模块 FIFO 里面的数据拷贝到 LCD 屏幕上
 * @param       无
 * @retval      无
 */
void ov7725_camera_refresh(void)
{
    uint32_t i, j;
    uint16_t color;

    if (g_ov7725_vsta)              /* 有帧中断更新 */
    {
        lcd_scan_dir(U2D_L2R);      /* 从上到下, 从左到右 */
        lcd_set_window((lcddev.width - g_ov7725_wwidth) / 2, (lcddev.height -
        g_ov7725_wheight) / 2,
        g_ov7725_wwidth, g_ov7725_wheight); /* 将显示区域设置到屏幕中央 */


        lcd_write_ram_prepare();    /* 开始写入 GRAM */

        OV7725_RRST(0);             /* 开始复位读指针 */
        OV7725_RCLK(0);
        OV7725_RCLK(1);
        OV7725_RCLK(0);
        OV7725_RRST(1);             /* 复位读指针结束 */
        OV7725_RCLK(1);

        for (i = 0; i < g_ov7725_wheight; i++)
        {
            for (j = 0; j < g_ov7725_wwidth; j++)
            {
                OV7725_RCLK(0);
                color = OV7725_DATA;    /* 读数据 */
                OV7725_RCLK(1);
                color <<= 8;
                OV7725_RCLK(0);
                color |= OV7725_DATA;   /* 读数据 */
                OV7725_RCLK(1);
                LCD->LCD_RAM = color;
            }
        }

        g_ov7725_vsta = 0;          /* 清零帧中断标记 */
        g_ov7725_frame++;
        lcd_scan_dir(DFT_SCAN_DIR);/* 恢复默认扫描方向 */
    }
}


int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint8_t i = 0;
    char msgbuf[15];                /* 消息缓存区 */
    uint8_t tm = 0;
    uint8_t lightmode = 0, effect = 0;
```

```
  uint8_t saturation = 4, brightness = 4, contrast = 4;
/* USER CODE END 1 */

HAL_Init();

/* 系统时钟初始化 */
SystemClock_Config();

/* 延时初始化 */
delay_init(72);

/* GPIO 初始化 */
MX_GPIO_Init();

/* FSMC 初始化 */
MX_FSMC_Init();

/* 串口初始化 */
MX_USART1_UART_Init();

/* TIM5/6 初始化 */
MX_TIM5_Init();
MX_TIM6_Init();

/* USER CODE BEGIN 2 */
lcd_init();                             /* 初始化 LCD */
tpad_init();                            /* TPAD 初始化 */
lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
lcd_show_string(30,  70, 200, 16, 16, "OV7725 TEST", RED);
lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
lcd_show_string(30, 110, 200, 16, 16, "KEY0:Light Mode", RED);
lcd_show_string(30, 130, 200, 16, 16, "KEY1:Saturation", RED);
lcd_show_string(30, 150, 200, 16, 16, "KEY_UP:Contrast", RED);
lcd_show_string(30, 170, 200, 16, 16, "TPAD:Effects", RED);
lcd_show_string(30, 190, 200, 16, 16, "OV7725 Init...", RED);
/* USER CODE END 2 */

while (1)
{
    if (ov7725_init() == 0)
    {
        lcd_show_string(30, 190, 200, 16, 16, "OV7725 Init OK       ", RED);

        while (1)
        {
            key = key_scan(0);

            if (key == KEY0_PRES)
            {
                g_ov7725_wwidth = 320;          /* 默认窗口宽度为 320 */
                g_ov7725_wheight = 240;         /* 默认窗口高度为 240 */
                ov7725_window_set(g_ov7725_wwidth, g_ov7725_wheight, 0);
                break;
            }
            else if (key == KEY1_PRES)
            {
                g_ov7725_wwidth = 320;          /* 默认窗口宽度为 320 */
                g_ov7725_wheight = 240;         /* 默认窗口高度为 240 */
                /* VGA 模式输出 */
                ov7725_window_set(g_ov7725_wwidth, g_ov7725_wheight, 1);
                break;
            }

            i++;
```

```
            if (i == 100)
            lcd_show_string(30, 210, 210, 16, 16, "KEY0:QVGA  KEY1:VGA", RED);

            if (i == 200)
            {
                lcd_fill(30, 230, 210, 250 + 16, WHITE);
                i = 0;
            }

            delay_ms(5);
        }

        ov7725_light_mode(lightmode);
        ov7725_color_saturation(saturation);
        ov7725_brightness(brightness);
        ov7725_contrast(contrast);
        ov7725_special_effects(effect);

        OV7725_OE(0); /* 使能 OV7725 FIFO 数据输出 */

        break;
    }
    else
    {
        lcd_show_string(30, 190, 200, 16, 16, "OV7725 Error!!", RED);
        delay_ms(200);
        lcd_fill(30, 190, 239, 246, WHITE);
        delay_ms(200);
    }
}
lcd_clear(BLACK);

while (1)
{
    key = key_scan(0); /* 不支持连按 */

    if (key)
    {
        tm = 20;

        switch (key)
        {
            case KEY0_PRES: /* 灯光模式 Light Mode */
                lightmode++;

                if (lightmode > 5)
                    lightmode = 0;

                ov7725_light_mode(lightmode);
                sprintf((char *)msgbuf, "%s", LMODE_TBL[lightmode]);
                break;

            case KEY1_PRES: /* 饱和度 Saturation */
                saturation++;

                if (saturation > 8)
                    saturation = 0;

                ov7725_color_saturation(saturation);
                sprintf((char *)msgbuf, "Saturation:%d", saturation);
                break;

            case WKUP_PRES: /* 对比度 Contrast */
                contrast++;
```

```
                if (contrast > 8)
                    contrast = 0;

                ov7725_contrast(contrast);
                sprintf((char *)msgbuf, "Contrast:%d", contrast);
                break;
            }
        }

        if (tpad_scan(0)) /* 检测到触摸按键 */
        {
            effect++;

            if (effect > 6)
                effect = 0;

            ov7725_special_effects(effect); /* 设置特效 */
            sprintf((char *)msgbuf, "%s", EFFECTS_TBL[effect]);
            tm = 20;
        }

        ov7725_camera_refresh(); /* 更新显示 */

        if (tm)
        {
            lcd_show_string((lcddev.width - 240) / 2 + 30, (lcddev.height - 320)
            / 2 + 60, 200, 16, 16, msgbuf, BLUE);
            tm--;
        }

        i++;

        if (i >= 15) /* DS0 闪烁 */
        {
            i = 0;
            LED0_TOGGLE(); /* LED0 闪烁 */
        }
    }
}
```

ov7725_camera_refresh 该函数用于将摄像头的数据，更新到 LCD 上显示出来，实现的效果就是类似摄像头。

main 函数的代码比较简单，这里我们就不展开说明了。

最后，为了得到最快的显示速度，我们可以编译时选择 Release 进行编译并下载，这样 OV7725 的显示帧率可达 22 帧左右。注意：因为 tpad_scan 扫描会占用比较多的时间，所以帧率比较慢，屏蔽该函数，也可以提高帧率

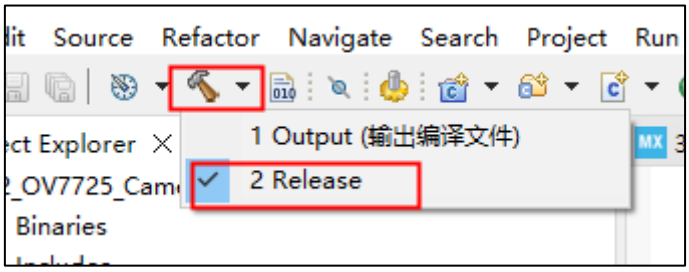


图 54.2.4.2 使用 Release 编译提高代码效率

54.3 下载验证

在完成编译和烧录操作后，我们通过下载代码到正点原子精英 STM32 开发板上，得到如图 54.3.1 所示界面：

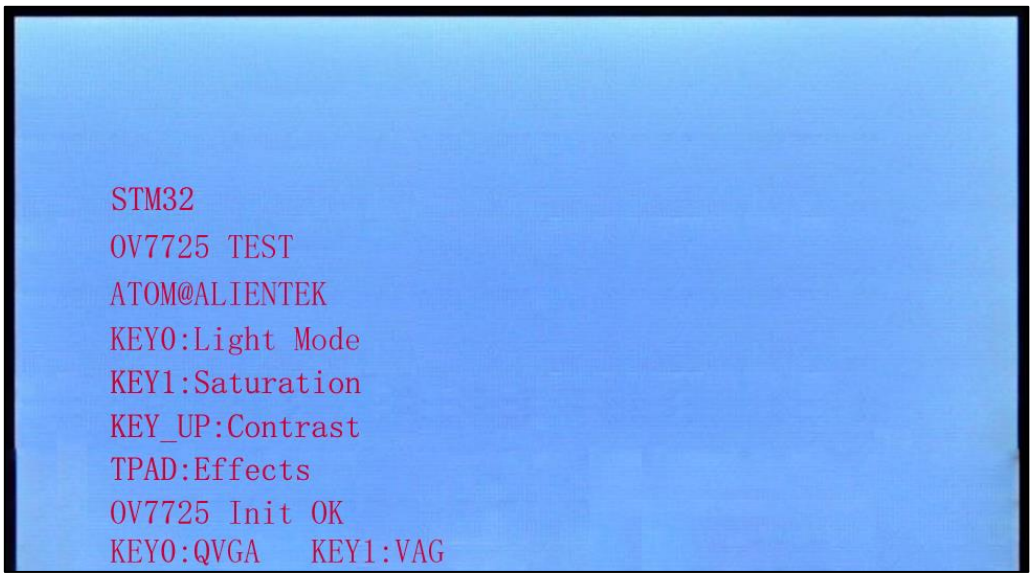


图 54.3.1 程序运行效果图

随后，我们通过 KEY0 和 KEY1 选择模式。当选择 QVGA 模式时，OV7725 直接输出 320*240 分辨率图像数据，该模式相对 VGA 模式视角较广但画面没有那么清晰细腻。当选择 VGA 模式时，实质是将 640*480 窗口截取中间 320*240 的图像输出，该模式拍出的图像较为清晰细腻但视角较小。

第五十五章 内存管理实验

本章，我们将介绍内存管理。我们将使用内存的动态管理减少对内存的浪费。本章分为如下几个小节：

55.1 硬件设计
55.2 程序设计
55.3 下载验证

55.1 硬件设计

1. 例程功能

开机后，LCD 显示提示信息，等待外部输入。KEY0 用于申请内存，每次申请 2K 字节内容，还向申请到的内存写入数据。KEY1 用于释放内存。WK_UP 用于更新数据到申请到的内存里。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
KEY0 - PE4
KEY1 - PE3
WK_UP - PA0
3）串口 1（PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面）
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

本章实验使用的内存管理库为软件库，因此没有对应的连接原理图。

55.2 程序设计

55.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 33_MALLOC。

55.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 14_USMART 工程，并将工程文件名和.ioc 文件名均修改为 33_MALLOC，以进行下一步的实验操作。如下图所示：

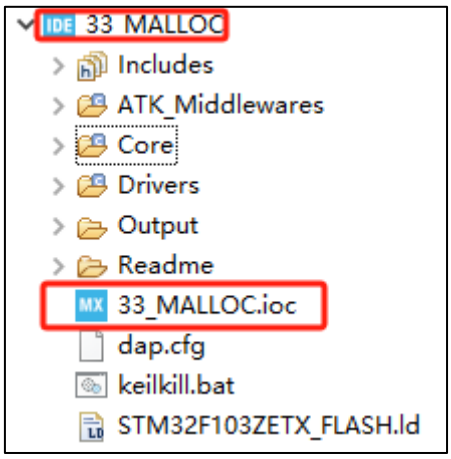


图 55.2.2.1 MALLOC 工程

本章实验使用的内存管理库为软件库，因此不涉及硬件以及外设上的配置。所以直接在工程中添加我们的用户文件。

55.2.3 添加用户驱动代码

在 ATK_Middlewares 文件夹中，新建 MALLOC 文件夹，在 MALLOC 中，我们添加了 malloc.c 和 malloc.h 文件。

（1）malloc.h 文件代码

下面我们直接介绍 malloc.h 中比较重要的一个结构体和内存参数宏定义，其定义如下：

```
/* mem1 内存参数设定.mem1 是 F103 内部的 SRAM. */
#define MEM1_BLOCK_SIZE    32          /* 内存块大小为 32 字节 */
#define MEM1_MAX_SIZE      40 * 1024 /* 最大管理内存 40K, F103ZE 内部 SRAM 共 512KB*/
#define MEM1_ALLOC_TABLE_SIZE    MEM1_MAX_SIZE/MEM1_BLOCK_SIZE    /* 内存表大小 */

/* mem2 内存参数设定.mem3 是 F103 外扩 SRAM */
#define MEM2_BLOCK_SIZE          32           /* 内存块大小为 32 字节 */
#define MEM2_MAX_SIZE            1 * 32       /* 精英板没有外扩内存，故设置一个最小值*/
#define MEM2_ALLOC_TABLE_SIZE    MEM2_MAX_SIZE/MEM2_BLOCK_SIZE    /* 内存表大小 */
/* 内存管理控制器 */
struct _m_mallco_dev
{
    void (*init)(uint8_t);                /* 初始化 */
    uint16_t (*perused)(uint8_t);         /* 内存使用率 */
    uint8_t *membase[SRAMBANK];           /* 内存池 管理 SRAMBANK 个区域的内存 */
    MT_TYPE *memmap[SRAMBANK];            /* 内存管理状态表 */
    uint8_t  memrdy[SRAMBANK];            /* 内存管理是否就绪 */
};
```

我们可以定义几个不同的内存管理表，再分配相应的指针给到管理控制器即可。程序中我们用宏定义 MEM1_BLOCK_SIZE 来定义 malloc 可以管理的内部内存池总大小，实际上我们定义为一个大小为 MEM1_BLOCK_SIZE 的数组，这样编译后就能获得一块实际的连续内存区域，这里是 40KB，MEM1_ALLOC_TABLE_SIZE 代表内存池的内存管理表大小。我们可以定义多个内存管理表，这样就可以同时管理多块内存。

从这里可以看出，如果内存分块越小，那么内存管理表就越大，当分块为 2 字节 1 个块的时候，内存管理表就和内存池一样大了（管理表的每项都是 uint16_t 类型），显然是不合适。我们这里取 32 字节，比例为 1：16，内存管理表相对就比较小了。

通过这个内存管理控制器_m_malloc_dev 结构体，我们把分块式内存管理的相关信息，其初始化函数、获取使用率、内存池、内存管理表以及内存管理的状态保存下来，实现对内存池的管理控制。

（2）malloc.c 文件代码

整个 malloc 代码的核心函数：my_mem_malloc 和 my_mem_free，分别用于内存申请和内存释放。这两个函数只是内部调用，外部调用我们另外定义了 mymalloc 和 myfree 两个函数，其他函数我们就不多介绍了。下面看一下分配内存和释放内存相关函数，其定义如下：

```
/**
 * @brief       内存分配(内部调用)
 * @param           memx : 所属内存块
 * @param           size : 要分配的内存大小(字节)
 * @retval          内存偏移地址
 *   @arg           0 ~ 0XFFFFFFFE : 有效的内存偏移地址
 *   @arg           0XFFFFFFFF     : 无效的内存偏移地址
 */
static uint32_t my_mem_malloc(uint8_t memx, uint32_t size)
{
    signed long offset = 0;
    uint32_t nmemb;         /* 需要的内存块数 */
    uint32_t cmemb = 0;     /* 连续空内存块数 */
    uint32_t i;

    if (!mallco_dev.memrdy[memx])
    {
        mallco_dev.init(memx);              /* 未初始化,先执行初始化 */
    }

    if (size == 0) return 0XFFFFFFFF;   /* 不需要分配 */

    nmemb = size / memblksize[memx];    /* 获取需要分配的连续内存块数 */

    if (size % memblksize[memx]) nmemb++;

    for (offset=memtblsize[memx] - 1; offset >= 0; offset--) /*搜索整个内存控制区
*/
    {
        if (!mallco_dev.memmap[memx][offset])
        {
            cmemb++;            /* 连续空内存块数增加 */
        }
        else
        {
            cmemb = 0;          /* 连续内存块清零 */
        }

        if (cmemb == nmemb) /* 找到了连续 nmemb 个空内存块 */
        {
            for (i = 0; i < nmemb; i++)             /* 标注内存块非空 */
            {
                mallco_dev.memmap[memx][offset + i] = nmemb;
            }
            return (offset * memblksize[memx]); /* 返回偏移地址 */
        }
    }
    return 0XFFFFFFFF;  /* 未找到符合分配条件的内存块 */
}

/**
 * @brief       释放内存(内部调用)
 * @param       memx    : 所属内存块
 * @param       offset : 内存地址偏移
 * @retval      释放结果
 *   @arg       0, 释放成功;
```

```
 *   @arg       1, 释放失败;
 *   @arg       2, 超区域了(失败);
 */
static uint8_t my_mem_free(uint8_t memx, uint32_t offset)
{
    int i;

    if (!mallco_dev.memrdy[memx])       /* 未初始化,先执行初始化 */
    {
        mallco_dev.init(memx);
        return 1;                       /* 未初始化 */
    }
    if (offset < memsize[memx])         /* 偏移在内存池内. */
    {
        int index = offset / memblksize[memx];              /* 偏移所在内存块号码
*/
        int nmemb = mallco_dev.memmap[memx][index];         /* 内存块数量 */

        for (i = 0; i < nmemb; i++)                         /* 内存块清零 */
        {
            mallco_dev.memmap[memx][index + i] = 0;
        }

        return 0;
    }
    else
    {
        return 2;   /* 偏移超区了. */
    }
}

/**
 * @brief       释放内存(外部调用)
 * @param       memx : 所属内存块
 * @param       ptr  : 内存首地址
 * @retval      无
 */
void myfree(uint8_t memx, void *ptr)
{
   uint32_t offset;

   if (ptr == NULL)return;          /* 地址为0. */

    offset = (uint32_t)ptr - (uint32_t)mallco_dev.membase[memx];
    my_mem_free(memx, offset);  /* 释放内存 */
}

/**
 * @brief       分配内存(外部调用)
 * @param       memx : 所属内存块
 * @param       size : 要分配的内存大小(字节)
 * @retval  分配到的内存首地址.
 */
void *mymalloc(uint8_t memx, uint32_t size)
{
    uint32_t offset;
   offset = my_mem_malloc(memx, size);

    if (offset == 0XFFFFFFFF)    /* 申请出错 */
    {
        return NULL;             /* 返回空(0) */
    }
    else     /* 申请没问题, 返回首地址 */
```

```
    {
        return (void *)((uint32_t)mallco_dev.membase[memx] + offset);
    }
}
```

（3）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t paddr[20];  /* 存放 P Addr:+p 地址的 ASCII 值 */
    uint16_t memused = 0;
    uint8_t key;
    uint8_t i = 0;
    uint8_t *p = 0;
    uint8_t *tp = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                                 /* 初始化 LCD */
  my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */

  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "MALLOC TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:Malloc KEY1:Free", RED);
  lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Write ", RED);
  lcd_show_string(30, 160, 200, 16, 16, "SRAMIN ", BLUE);
  lcd_show_string(30, 176, 200, 16, 16, "SRAMIN   USED:", BLUE);
  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);        /* 不支持连按 */

      switch (key)
      {
          case KEY0_PRES:       /* KEY0 按下 */
              /* 申请 2K 字节,并写入内容,显示在 lcd 屏幕上面 */
              p = mymalloc(SRAMIN, 2048);

              if (p != NULL)
              {
                  /* 向 p 写入一些内容 */
                  sprintf((char *)p, "Memory Malloc Test%03d", i);
```

```
                lcd_show_string(30, 260, 209, 16, 16, (char *)p, BLUE);
            }

            break;

        case KEY1_PRES:          /* KEY1 按下 */
            myfree(SRAMIN, p);   /* 释放内存 */
            p = 0;               /* 指向空地址 */
            break;

        case WKUP_PRES:          /* KEY UP 按下 */
        if (p != NULL )
        {
          sprintf((char *)p, "Memory Malloc Test%03d", i);
          lcd_show_string(30, 260, 209, 16, 16, (char *)p, BLUE);
        }
        break;
    }

    if (tp != p)
    {
        tp = p;
        sprintf((char *)paddr, "P Addr:0X%08X", (uint32_t)tp);
        /* 显示 p 的地址 */
        lcd_show_string(30, 240, 209, 16, 16, (char *)paddr, BLUE);

        if (p)
        {   /* 显示 P 的内容 */
            lcd_show_string(30, 260, 280, 16, 16, (char *)p, BLUE);
        }
        else
        {
            lcd_fill(30, 260, 209, 296, WHITE); /* p=0,清除显示 */
        }
    }

    delay_ms(10);
    i++;

    if ((i % 20) == 0)  /* DS0 闪烁. */
    {
        memused = my_mem_perused(SRAMIN);
        sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
        /* 显示内部内存使用率 */
        lcd_show_string(30 + 112, 176, 200, 16, 16, (char *)paddr, BLUE);

        LED0_TOGGLE();  /* LED0 闪烁 */
    }
  }
}
```

该部分代码比较简单，主要是对 mymalloc 和 myfree 的应用。不过这里提醒大家，如果对一个指针进行多次内存申请，而之前的申请又没释放，那么将造成“内存泄露”，这是内存管理所不希望发生的，久而久之，可能导致无内存可用的情况！所以，在使用的时候，请大家一定记得，申请的内存在用完以后，一定要释放。

55.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 55.3.1 所示：

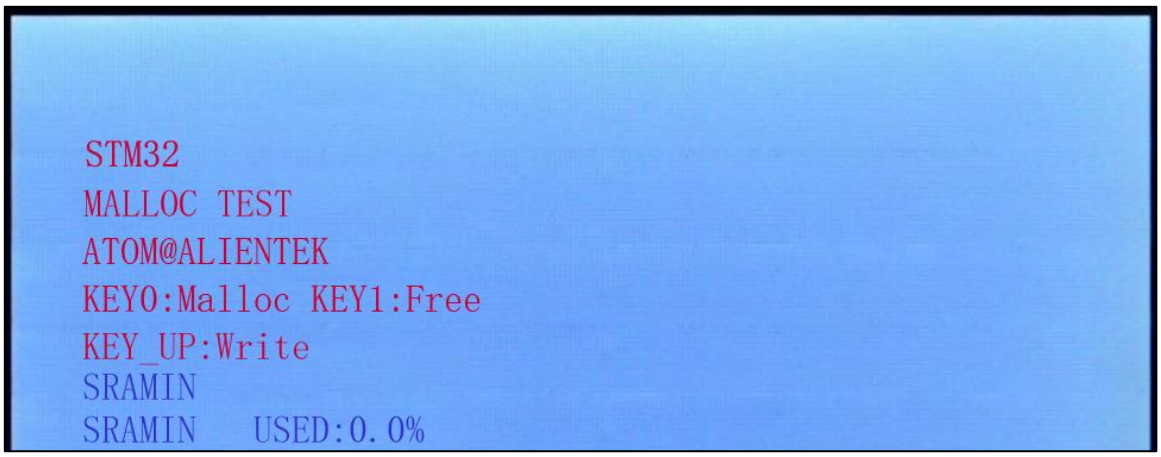


图 55.3.1 内存管理实验测试图

可以看到，内存的使用率均为 0%，说明还没有任何内存被使用。此时我们按下 KEY0，就可以看到内存使用了 5%了，同时看到下面提示了指针 p 所指向的地址（其实就是被分配到的内存地址）和内容。多按几次 KEY0，可以看到内存使用率持续上升（注意比对 p 的值，可以发现是递减的，说明是从顶部开始分配内存！），此时如果按下 KEY1，可以发现内存使用率降低了 5%，但是再按 KEY1 将不再降低，说明“内存泄漏”，在实际使用的时候，必须避免内存泄漏。

第五十六章 SD 卡实验

本章将介绍使用 STM32F103 驱动 SD 卡进行 SD 卡的识别、读写等操作。通过本章的学习，读者将学习到 SD 卡的使用。

本章分为如下几个小节：

56.1 硬件设计
56.2 程序设计
56.3 下载验证

56.1 硬件设计

1. 例程功能

1. 程序运行后，若初始化 SD 卡成功，则会通过串口输出 SD 卡的相关参数
2. 按下 KEY_0 按键，可读取 SD 卡扇区 0 的数据，并通过串口显示
3. 可通过 USMART 进行 SD 卡数据读写操作
4. LED0 闪烁，指示程序正在运行

2. 硬件资源

1）LED 灯
LED0 - PB5
2）按键
KEY0 - PE4
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）microSD Card

3. 原理图

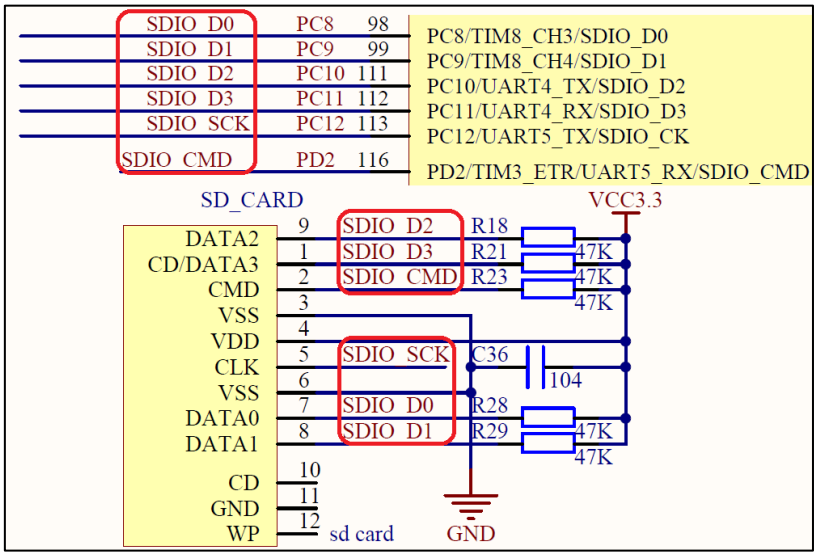


图56.1.1 SD卡接口与STM32F1连接原理图

SD 卡座在 JTAG 插座附近，TF 卡座与开发板的连接在开发板上是直接连接在一起的，硬件上不需要任何改动。大家准备好 TF 卡就可以开始我们的程序设计和验证了。

56.2 程序设计

56.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 34_SD。

56.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 14_USMART 工程，并将工程文件名和.ioc 文件名均修改为 34_SD，以进行下一步的实验操作。如下图所示：

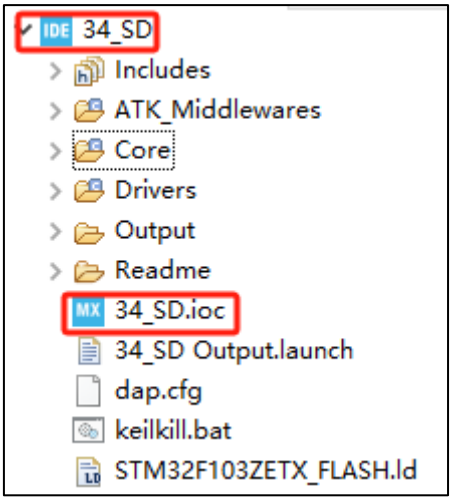


图 56.2.2.1 SD 工程

（1）SDIO 配置

双击 34_SD.ioc 进入 STM32CubeMX 插件界面，在 Connectivity->SDIO 处我们配置 SDIO 参数，如下图所示：

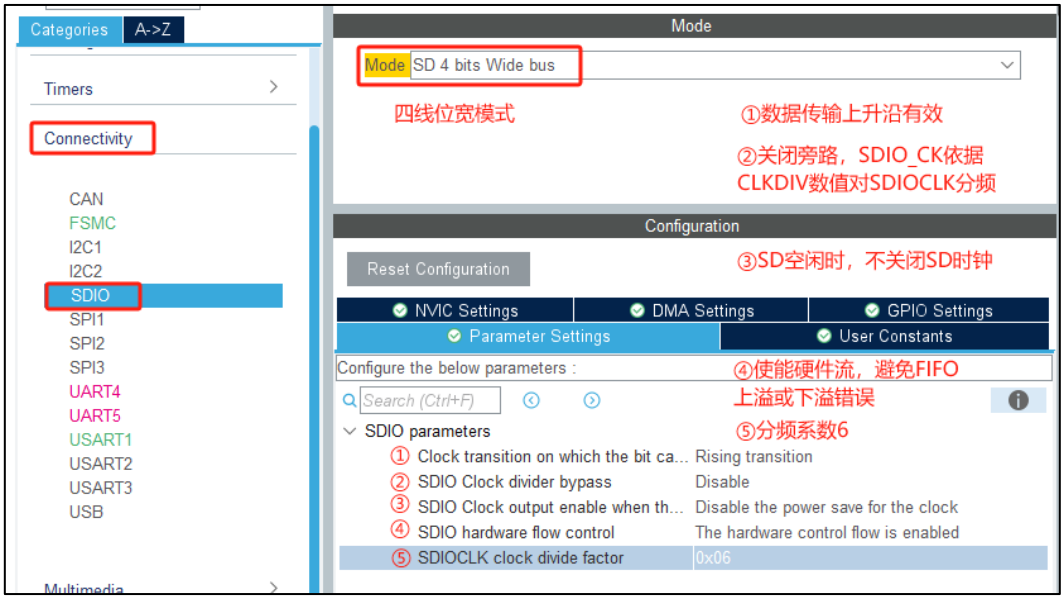


图 56.2.2.2 SDIO 参数配置

注意到上面的参数配置，由上到下进行介绍。首先我们配置的是四线位宽模式；

1、SD 的双向传输均为 CLK 时钟的上升沿有效。

2、关闭旁路，即 SDIO_CK 就依据 CLKDIV 数值对 SDIOCLK 分频，公式如下：

$$SDIO_CK = \frac{\text{SDIOCLK}}{(2 + \text{CLKDIV})}$$

其中，SDIOCLK 为 HCLK，一般是 72Mhz，而⑤CLKDIV 则是分频系数，可以通过 SDIO 的 SDIO_CLKCR 寄存器进行设置（确保 SDIO_CK 不超过卡的最大操作频率）。在这里我们设置的是 6 分频，即 SDIO_CK 为 9M。

3、当 SD 空闲时，不关闭 SD 的时钟，即禁用时钟的省电功能；

4、启用硬件流控制，避免 FIFO 上溢或下溢错误。

5、当关闭旁路功能时，分频系数将对 SDIO_CLK 进行分频，从而产生 SDIO_CK。

完成上述配置后，请细心检查下引脚是否与原理图一一对应。

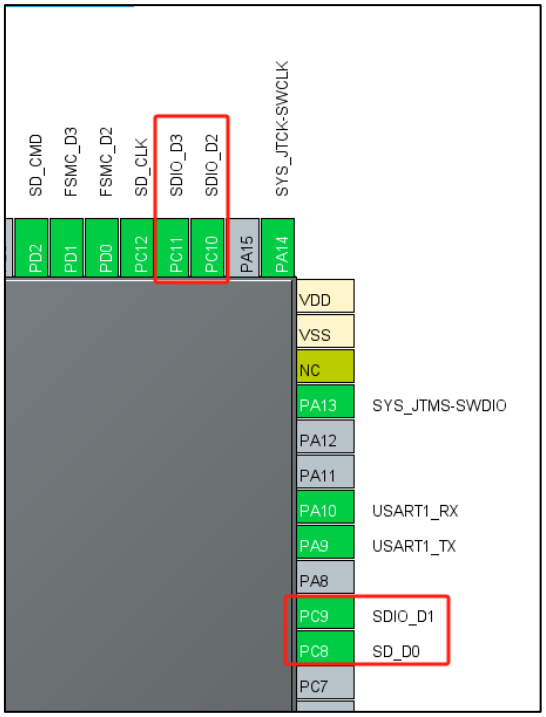


图 56.2.2.3 引脚检查

56.2.3 生成初始化工程

按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了一个 sdio.c 文件。sdio.c 主要是完成 SD 的初始化。

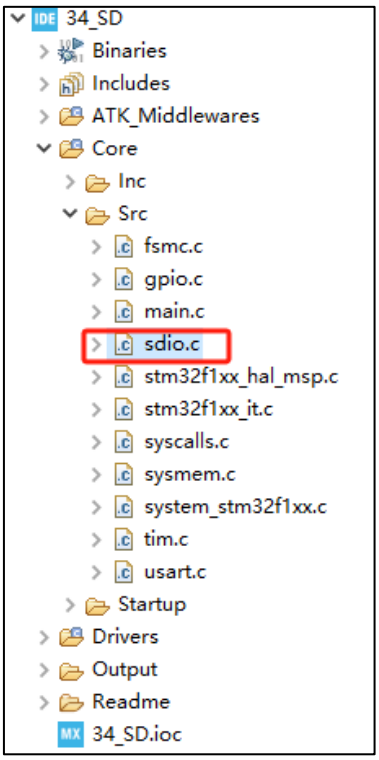


图 56.2.3.1 生成工程

56.2.4 添加用户驱动代码

在本章实验中，我们主要修改以下几个文件，分别是：sdio.c、sdio.h、main.c。其中 sdio.h 文件中，仅添加了在 sdio.c 中实现的一些功能函数的声明，这里不过多赘述，我们先来看下 sdio.c 文件。

（1）sdio.c 文件代码

注意，每次生成新代码时，该步骤会被置位，都需要重新进行修改，修改后如下：

```
void MX_SDIO_SD_Init(void)
{
  hsd.Instance = SDIO;
  hsd.Init.ClockEdge = SDIO_CLOCK_EDGE_RISING;
  hsd.Init.ClockBypass = SDIO_CLOCK_BYPASS_DISABLE;
  hsd.Init.ClockPowerSave = SDIO_CLOCK_POWER_SAVE_DISABLE;
  hsd.Init.BusWide = SDIO_BUS_WIDE_1B; /* 人为修改为 1 位宽，初始化时需保持一位宽 */
  hsd.Init.HardwareFlowControl = SDIO_HARDWARE_FLOW_CONTROL_ENABLE;
  hsd.Init.ClockDiv = 0x06;
  if (HAL_SD_Init(&hsd) != HAL_OK)
  {
    g_sd_error = 1; /* 用于标记 SD 卡是否初始化成功，若失败则置一 */
  }
  /* 初始化完成后，主机才可修改数据总线位宽 */
  if (HAL_SD_ConfigWideBusOperation(&hsd, SDIO_BUS_WIDE_4B) != HAL_OK)
  {
    Error_Handler();
  }
}
```

以上配置参数已在前面详细介绍，这里不再赘述。需要注意的是，在复位后，SDIO_D0 默认用于数据传输（1 位宽）。只有在初始化后，主机才能改变数据总线的宽度。

接着我们还在 sdio.c 文件中，添加以下函数：

```
/**
```

```
 * @brief       获取卡信息函数
 * @param       cardinfo:SD 卡信息句柄
 * @retval      返回值:读取卡信息状态值
 */
uint8_t get_sd_card_info(HAL_SD_CardInfoTypeDef *cardinfo)
{
    uint8_t sta;
    sta = HAL_SD_GetCardInfo(&hsd, cardinfo);
    return sta;
}

/**
 * @brief       判断 SD 卡是否可以传输(读写)数据
 * @param       无
 * @retval      返回值:SD_TRANSFER_OK       传输完成，可以继续下一次传输
                       SD_TRANSFER_BUSY SD 卡正忙，不可以进行下一次传输
 */
uint8_t get_sd_card_state(void)
{
    return ((HAL_SD_GetCardState(&hsd) == HAL_SD_CARD_TRANSFER) ?
SD_TRANSFER_OK : SD_TRANSFER_BUSY);
}

/**
 * @brief       读 SD 卡(fatfs/usb 调用)
 * @param       pbuf  : 数据缓存区
 * @param       saddr : 扇区地址
 * @param       cnt   : 扇区个数
 * @retval      0, 正常;  其他, 错误代码(详见 SD_Error 定义);
 */
uint8_t sd_read_disk(uint8_t *pbuf, uint32_t saddr, uint32_t cnt)
{
    uint8_t sta = HAL_OK;
    uint32_t timeout = SD_TIMEOUT;
    long long lsector = saddr;

    __disable_irq();
     /* 关闭总中断(POLLING 模式,严禁中断打断 SDIO 读写操作!!!) */
    sta = HAL_SD_ReadBlocks(&hsd, (uint8_t *)pbuf, lsector, cnt, SD_TIMEOUT);
     /* 多个 sector 的读操作 */

    /* 等待 SD 卡读完 */
    while (get_sd_card_state() != SD_TRANSFER_OK)
    {
        if (timeout-- == 0)
        {
            sta = SD_TRANSFER_BUSY;
        }
    }
    __enable_irq(); /* 开启总中断 */
    return sta;
}

/**
 * @brief       写 SD 卡(fatfs/usb 调用)
 * @param       pbuf  : 数据缓存区
 * @param       saddr : 扇区地址
 * @param       cnt   : 扇区个数
 * @retval      0, 正常;  其他, 错误代码(详见 SD_Error 定义);
 */
uint8_t sd_write_disk(uint8_t *pbuf, uint32_t saddr, uint32_t cnt)
{
    uint8_t sta = HAL_OK;
```

```
    uint32_t timeout = SD_TIMEOUT;
    long long lsector = saddr;
    __disable_irq();
   /* 关闭总中断(POLLING 模式,严禁中断打断 SDIO 读写操作!!!) */
    sta = HAL_SD_WriteBlocks(&hsd, (uint8_t *)pbuf, lsector, cnt, SD_TIMEOUT);
   /* 多个 sector 的写操作 */

    /* 等待 SD 卡写完 */
    while (get_sd_card_state() != SD_TRANSFER_OK)
    {
        if (timeout-- == 0)
        {
            sta = SD_TRANSFER_BUSY;
        }
    }
    __enable_irq(); /* 开启总中断 */
    return sta;
}
```

1，sd_read_disk 函数，该函数比较简单，实际上我们使用它来对 HAL 库的读函数 HAL_SD_ReadBlocks 进行了二次封装，并在最后加入了状态判断以使后续操作（实际上这部分代码也可以省略），直接根据读函数返回值自己作其它处理。为了保护 SD 卡的数据操作，我们在进行操作时暂时关闭了中断以防止数据读过程发生意外。

2，sd_write_disk 函数，该函数比较简单，实际上我们使用它来对 HAL 库的读函数 HAL_SD_WriteBlocks 进行了二次封装，并在最后加入了状态判断以使后续操作（实际上这部分代码也可以省略），直接根据读函数返回值自己作其它处理。为了保护 SD 卡的数据操作，我们在进行操作时暂时关闭了中断以防止数据写过程发生意外。

get_sd_card_info 函数和 get_sd_card_state 函数的作用分别是：获取 SD 卡的信息和获取 SD 卡的状态。

（2）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t key;
    uint8_t t = 0;
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* SDIO 初始化 */
  MX_SDIO_SD_Init();
```

```
  /* USER CODE BEGIN 2 */
  lcd_init();                                 /* 初始化 LCD */
  my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */

  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "SD TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:Read Sector 0", RED);
  while (g_sd_error == 1) /* 检测不到 SD 卡 */
  {
      g_sd_error = 0;
      MX_SDIO_SD_Init();
      lcd_show_string(30, 130, 200, 16, 16, "SD Card Error!", RED);
      delay_ms(500);
      lcd_show_string(30, 130, 200, 16, 16, "Please Check! ", RED);
      delay_ms(500);
      LED0_TOGGLE(); /* 红灯闪烁 */
  }
  /* 打印 SD 卡相关信息 */
  show_sdcard_info();

  /* 检测 SD 卡成功 */
  lcd_show_string(30, 130, 200, 16, 16, "SD Card OK    ", BLUE);
  lcd_show_string(30, 150, 200, 16, 16, "SD Card Size:     MB", BLUE);
  lcd_show_num(30 + 13 * 8, 150, SD_TOTAL_SIZE_MB(&hsd), 5, 16, BLUE);
  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY0_PRES)      /* KEY0 按下了 */
      {
          sd_test_read(0, 1);    /* 从扇区 0 读取 1*512 字节的内容 */
      }else if(key == KEY1_PRES)
      {
          sd_test_write(0, 1);   /* 写数据到扇区 0 */
      }

      t++;
      delay_ms(10);

      if (t == 20)
      {
          LED0_TOGGLE();         /* 红灯闪烁 */
          t = 0;
      }
  }
}
```

main.c 里面的代码比较简单，这里就不展开讲解。然后为了方便测试，我们编写了 sd_test_read()\sd_test_write()\show_sdcard_info ()三个函数分别用于读写测试和卡信息打印，主要就是调用 sdio_sdcard.c 文件中的函数接口，代码也比较容易看懂，这里就不单独介绍这几个函数了。

main 函数先初始化相关外设和 SD 卡，初始化成功后，通过调用 show_sdcard_info 函数输出 SD 卡相关信息（卡类型、制造商 ID、卡相对地址、容量和块大小等），并在 LCD 上显示 SD 卡容量。然后进入死循环，如果有按键 KEY0 按下，则通过 sd_test_read 函数读取 SD 卡的扇区 0（物理磁盘，扇区 0），并将数据通过串口打印出来。

56.3 下载验证

在代码编译下载成功之后，我们测试使用的是 16GB 标有“SDHC”标志的卡，安装方法

如图 56.3.1 所示：

图 56.3.1 测试用的 microSD 卡与开发板的连接方式

SD 卡成功初始化后，LCD 显示本程序的一些必要信息，如图 56.3.2 所示：

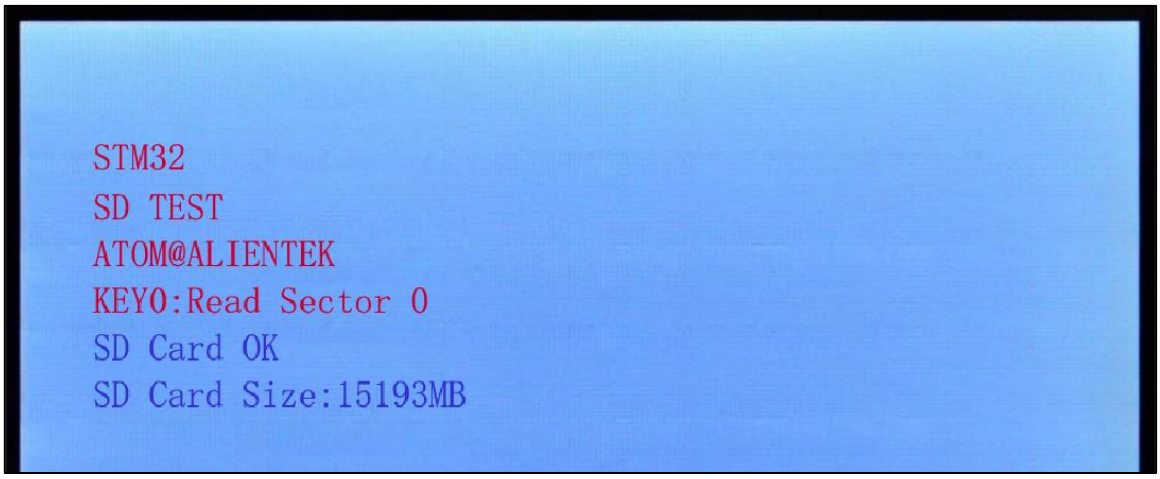


图 56.3.2 程序运行效果图

在进入测试的主循环前，我们如果已经通过 USB 连接开发板的串口 1 和电脑，可以看到串口端打印出 SD 卡的相关信息（也可以在接好 SD 卡后按 Reset 复位开发板），我们测试使用的是 16GB 标有“SDHC”标志的卡，SD 卡成功初始化后的信息，如图 56.3.3 所示：

图 56.3.3 串口助手打印信息

可见我们用程序读到的 SD 卡信息与我们使用的 SD 卡一致。伴随 LED0 的不停闪烁，提示程序在运行。此时，我们按下 KEY0，调用我们编写的 SD 卡测试函数，这里我们只用到了读函数，写函数的测试大家可以添加代码进行演示。按下 KEY0 后，LCD 显示信息如图 56.3.4 所示，读取到的扇区数据将通过串口打印出来，具体数据情况如图 56.3.5 所示：

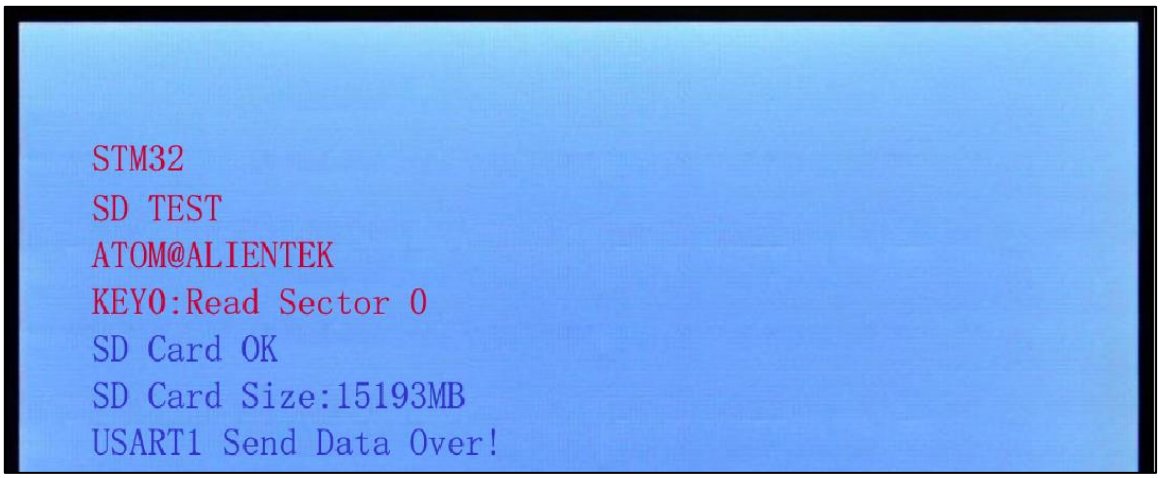
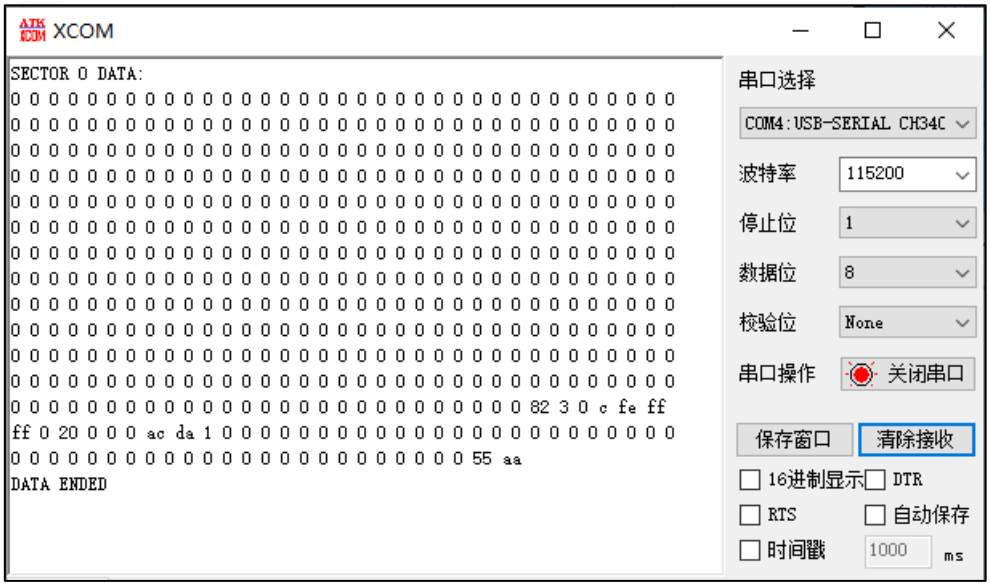


图 56.3.4 按下 KEY0 的开发板界面


XCOM
SECTOR 0 DATA:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

第五十七章 FATFS 实验

上一章实验中已经成功驱动 SD 卡，并可对 SD 卡进行读写操作，但读写 SD 卡时都是直接读出或写入二进制数据，这样使用起来显得十分不方便，因此本章将介绍 FATFS，FATFS 是一个通用的 FAT 文件系统模块，能够帮助实现文件系统，方便对 SD 卡、NOR Flash 或其他存储介质中数据的管理。通过本章的学习，读者将学习到 FATFA 的基本使用。

本章分为如下几个小节：

57.1 硬件设计
57.2 程序设计
57.3 下载验证

57.1 硬件设计

1. 例程功能

本章实验功能简介：开机的时候先初始化 SD 卡，初始化成功之后，注册两个磁盘：一个给 SD 卡用，一个给 SPI FLASH 用，之所以把 SPI FLASH 也当成磁盘来用，一方面是为了演示大容量的 SPI Flash 也可以用 FATFS 管理，说明 FATFS 的灵活性；另一方面可以展示 FATFS 方式比原来直接按地址管理数据便利性，使板载 SPI Flash 的使用更具灵活性。挂载成功后获取 SD 卡的容量和剩余空间，并显示在 LCD 模块上，最后等待 USMART 输入指令进行各项测试。通过 LED0 指示程序运行状态。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）micro SD 卡
5）NOR FLASH

3. 原理图

本章实验使用的 FATFS 为软件库，因此没有对应的连接原理图。

57.2 程序设计

57.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 35_FATFS。

57.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制上节课 34_SD 工程，并将工程文件名和.ioc 文件名均修改为 35_FATFS，以进行下一步的实验操作。如下图所示：

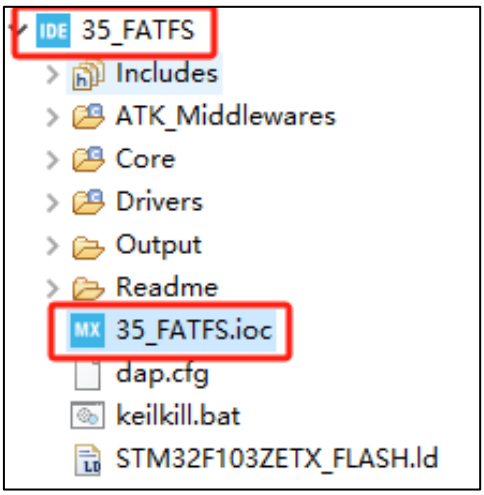


图 57.2.2.1 FATFS 工程

在本次实验中，我们将使用 FATFS 来读写两个磁盘，分别为 SD 卡和 NOR FLASH。由于 SD 卡驱动在本次实验之前已经配置完成，所以我们还需要将 NOR FLASH 的驱动进行配置。关于 NOR FLASH 的配置，请参照 SPI 实验的章节，并将 SPI 工程中的 NOFFLASH 文件夹复制到本次实验的 BSP 文件夹下。如下图所示：

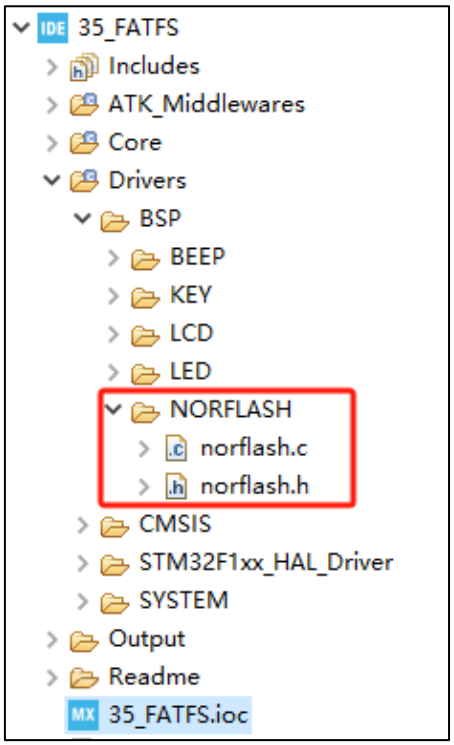


图 57.2.2.2 复制 NORFLASH 文件夹

注意 SPI2 的配置以及 spi.c 和 spi.h 文件的修改这里我们不在赘述，大家可参考 SPI 实验章节进行配置。接着我们开始配置 FATFS 部分。

（1）FATFS 配置

在 STM32CubeMX 插件界面的 Middleware and Software Packs->FATFS 处我们配置 FATFS 参数，如下图所示：

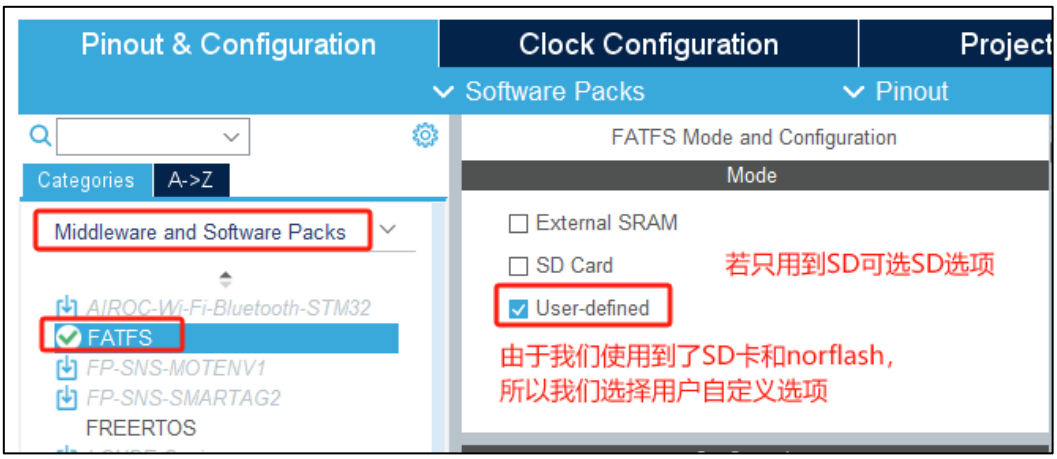


图 57.2.2.3 选择用户自定义选项

接着来到 FATFS 参数配置， 配置如下图：

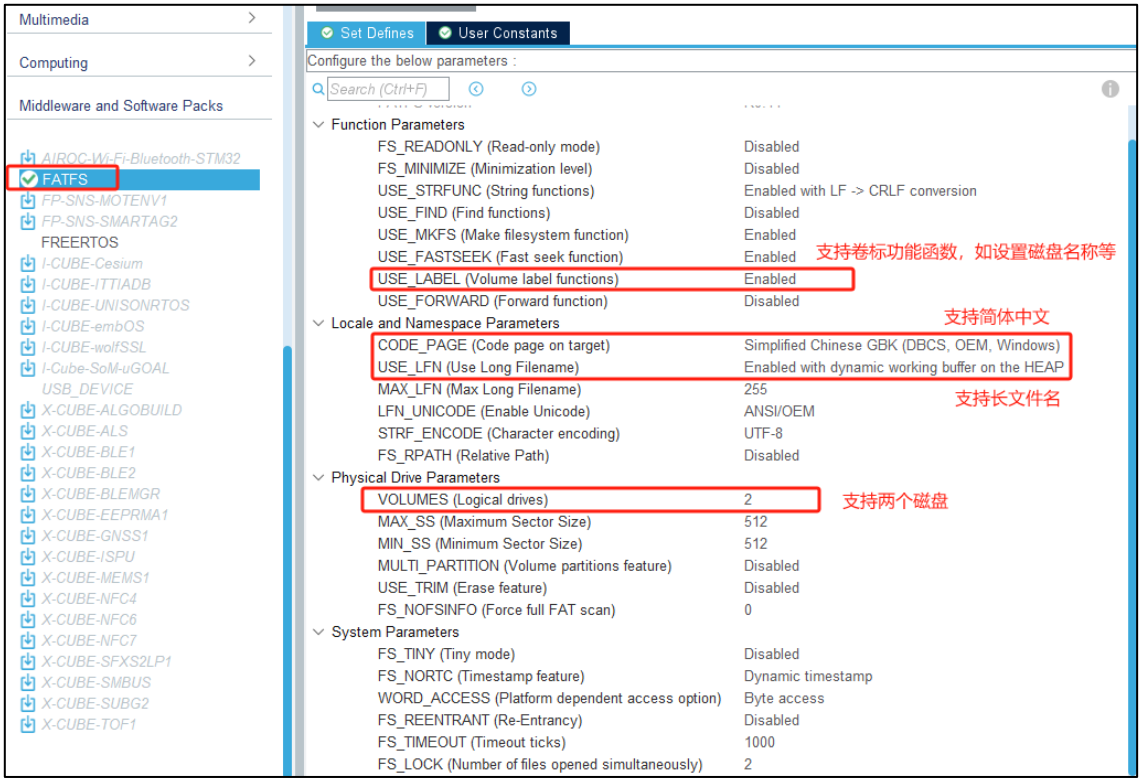


图 57.2.2.4 FATFS 参数配置

由于参数过多，我们这里不一一介绍，主要讲解几个我们需要修改的参数：

USE_LABEL：主要用于禁用（0）或启用（1）相应的 API 函数：f_getlabel 和 f_setlabel。可用于获取磁盘名称和设置磁盘名称。

CODE_PAGE：此项用于设置支持简体中文文件名；

USE_LFN：此项用于设置是否支持长文件名，还可设置堆栈作为工作缓冲区，请注意堆栈溢出问题。可适当增加堆栈大小。

VOLUMES：此项用于设置逻辑驱动器的数量（磁盘数量）

接下来，需要设置堆栈大小。考虑到我们已经配置了 FATFS 以支持长文件名和中文字符，为了避免堆栈溢出，需要将堆栈大小设置得更大，具体操作如下图所示：

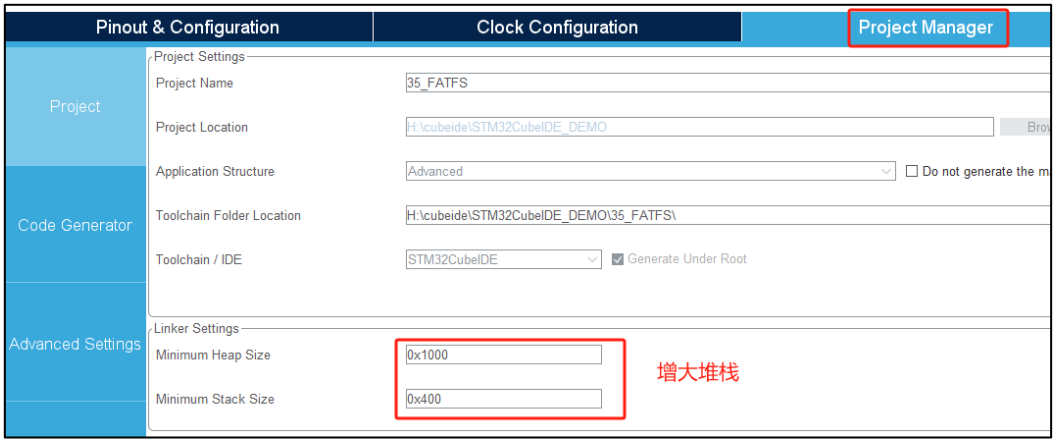


图 57.2.2.5 增大堆栈设置

57.2.3 生成初始化工程

配置完成后，按下“Ctrl+S”保存配置，生成工程，如下图，可以看到在工程中多了两个文件夹，分别是 FATFS 和 Middlewares。其中 FATFS 用于存放用户应用层文件，Middlewares 用于存放第三方中间层组件。工程结构如下图所示。

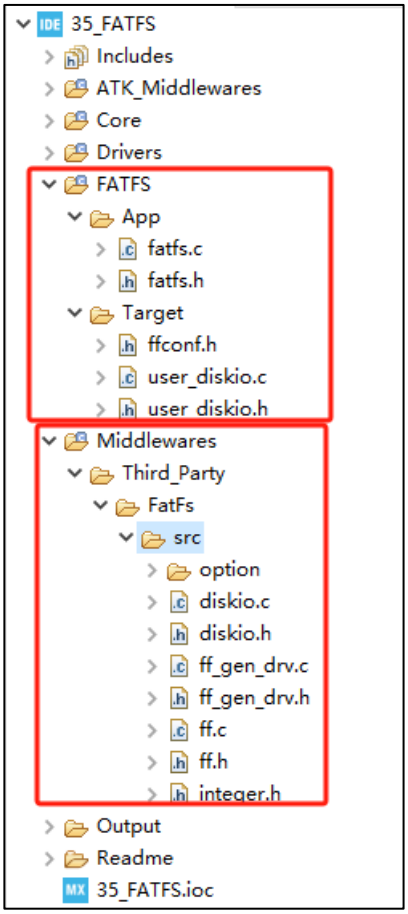


图 57.2.3.1 生成工程

57.2.4 添加用户驱动代码

在本章实验中，关于 spi 和 norflash 部分的移植以及修改我们不在赘述，可直接参考 SPI 实验章节。

我们主要关注 FATFS 部分的修改。首先，user_diskio.c 文件是 ST 官方提供的一个用户应用层代码文件。ST 官方的代码逻辑是：user_diskio.c->ff_gen _drv.c-> diskio.c 这样的一个处理步骤，即一层套一层的封装。通常，磁盘的读写以及初始化等操作，均在 user_diskio.c 文件中进行编写，但是该文件仅适用于单个磁盘的情况。而本实验使用到两个磁盘，因此，我们考虑跳过前面的一层接一层的封装步骤，直接修改 diskio.c 文件，以实现一步到位的效果。**但是需要注意的是，每次我们重新生成代码时，diskio.c 文件的内容都会被替换为初始代码状态，因此我们需要记得将修改内容重新添加回来。**

接着我们开始在 diskio.c 文件中添加用户代码。

（1）diskio.c 文件代码

```
#include "diskio.h"
#include "ff_gen_drv.h"
#include "sdio.h"
#include "../../BSP/NORFLASH/norflash.h"
extern Disk_drvTypeDef  disk;

#define SD_CARD     0       /* SD 卡,卷标为 0 */
#define EX_FLASH    1       /* 外部 spi flash,卷标为 1 */


/**
 * 对于 25Q128 FLASH 芯片, 我们规定前 12M 给 FATFS 使用, 12M 以后
 * 紧跟字库, 3 个字库 + UNIGBK.BIN, 总大小 3.09M, 共占用 15.09M
 * 15.09M 以后的存储空间大家可以随便使用.
 */

#define SPI_FLASH_SECTOR_SIZE   512
#define SPI_FLASH_SECTOR_COUNT  12 * 1024 * 2  /*前 12M 字节给 FATFS 占用 */
#define SPI_FLASH_BLOCK_SIZE    8       /* 每个 BLOCK 有 8 个扇区 */
#define SPI_FLASH_FATFS_BASE    0       /* FATFS 在外部 FLASH 的起始地址 从 0 开始 */

/**
  * @brief  Gets Disk Status
  * @param  pdrv: Physical drive number (0..)
  * @retval DSTATUS: Operation status
  */
DSTATUS disk_status (
    BYTE pdrv
)
{
    return RES_OK;
}

/**
  * @brief  Initializes a Drive
  * @param  pdrv: Physical drive number (0..)
  * @retval DSTATUS: Operation status
  */
DSTATUS disk_initialize (
    BYTE pdrv               /* Physical drive nmuber to identify the drive */
)
{
    uint8_t res = 0;

    switch (pdrv)
    {
        case SD_CARD:           /* SD 卡 */
            MX_SDIO_SD_Init();  /* SD 卡初始化 */
            break;
```

```
        case EX_FLASH:          /* 外部 flash */
            norflash_init();
            break;

        default:
            res = 1;
    }

    if (res)
    {
        return  STA_NOINIT;
    }
    else
    {
        return 0; /* 初始化成功*/
    }
}

/**
  * @brief  Reads Sector(s)
  * @param  pdrv: Physical drive number (0..)
  * @param  *buff: Data buffer to store read data
  * @param  sector: Sector address (LBA)
  * @param  count: Number of sectors to read (1..128)
  * @retval DRESULT: Operation result
  */
DRESULT disk_read (
    BYTE pdrv,      /* Physical drive nmuber to identify the drive */
    BYTE *buff,     /* Data buffer to store read data */
    DWORD sector,    /* Sector address in LBA */
    UINT count      /* Number of sectors to read */
)
{
    uint8_t res = 0;

    if (!count)return RES_PARERR;    /* count 不能等于 0，否则返回参数错误 */

    switch (pdrv)
    {
        case SD_CARD:       /* SD 卡 */
            res = sd_read_disk(buff, sector, count);

            while (res)     /* 读出错 */
            {
                if (res != 2) MX_SDIO_SD_Init();     /* 重新初始化 SD 卡 */

                res = sd_read_disk(buff, sector, count);
            }

            break;

        case EX_FLASH:      /* 外部 flash */
            for (; count > 0; count--)
            {
                norflash_read(buff, SPI_FLASH_FATFS_BASE + sector *
                 SPI_FLASH_SECTOR_SIZE, SPI_FLASH_SECTOR_SIZE);
                sector++;
                buff += SPI_FLASH_SECTOR_SIZE;
            }

            res = 0;
            break;

        default:
            res = 1;
```

```
    }

    /* 处理返回值，将返回值转成 ff.c 的返回值 */
    if (res == 0x00)
    {
        return RES_OK;
    }
    else
    {
        return RES_ERROR;
    }
}

/**
  * @brief  Writes Sector(s)
  * @param  pdrv: Physical drive number (0..)
  * @param  *buff: Data to be written
  * @param  sector: Sector address (LBA)
  * @param  count: Number of sectors to write (1..128)
  * @retval DRESULT: Operation result
  */
#if _USE_WRITE == 1
DRESULT disk_write (
    BYTE pdrv,      /* Physical drive nmuber to identify the drive */
    const BYTE *buff,   /* Data to be written */
    DWORD sector,       /* Sector address in LBA */
    UINT count          /* Number of sectors to write */
)
{
    uint8_t res = 0;

    if (!count)return RES_PARERR;  /* count 不能等于 0，否则返回参数错误 */

    switch (pdrv)
    {
        case SD_CARD:               /* SD 卡 */
            res = sd_write_disk((uint8_t *)buff, sector, count);

            while (res)             /* 写出错 */
            {
                MX_SDIO_SD_Init(); /* 重新初始化 SD 卡 */
                res = sd_write_disk((uint8_t *)buff, sector, count);
            }

            break;

        case EX_FLASH:              /* 外部 flash */
            for (; count > 0; count--)
            {
                norflash_write((uint8_t *)buff, SPI_FLASH_FATFS_BASE + sector
                 * SPI_FLASH_SECTOR_SIZE, SPI_FLASH_SECTOR_SIZE);
                sector++;
                buff += SPI_FLASH_SECTOR_SIZE;
            }

            res = 0;
            break;

        default:
            res = 1;
    }

    /* 处理返回值，将返回值转成 ff.c 的返回值 */
    if (res == 0x00)
    {
```

```
        return RES_OK;
    }
    else
    {
        return RES_ERROR;
    }
}
#endif /* _USE_WRITE == 1 */

/**
  * @brief  I/O control operation
  * @param  pdrv: Physical drive number (0..)
  * @param  cmd: Control code
  * @param  *buff: Buffer to send/receive control data
  * @retval DRESULT: Operation result
  */
#if _USE_IOCTL == 1
DRESULT disk_ioctl (
    BYTE pdrv,      /* Physical drive nmuber (0..) */
    BYTE cmd,       /* Control code */
    void *buff      /* Buffer to send/receive control data */
)
{
    DRESULT res;

    if (pdrv == SD_CARD)     /* SD卡 */
    {
        switch (cmd)
        {
            case CTRL_SYNC:
                res = RES_OK;
                break;

            case GET_SECTOR_SIZE:
                *(DWORD *)buff = 512;
                res = RES_OK;
                break;

            case GET_BLOCK_SIZE:
                *(WORD *)buff = hsd.SdCard.BlockSize;
                res = RES_OK;
                break;

            case GET_SECTOR_COUNT:
                *(DWORD *)buff = ((long long)hsd.SdCard.BlockNbr *
                 hsd.SdCard.BlockSize) / 512;
                res = RES_OK;
                break;

            default:
                res = RES_PARERR;
                break;
        }
    }
    else if (pdrv == EX_FLASH)  /* 外部FLASH */
    {
        switch (cmd)
        {
            case CTRL_SYNC:
                res = RES_OK;
                break;

            case GET_SECTOR_SIZE:
                *(WORD *)buff = SPI_FLASH_SECTOR_SIZE;
                res = RES_OK;
                break;
```

```
            case GET_BLOCK_SIZE:
                *(WORD *)buff = SPI_FLASH_BLOCK_SIZE;
                res = RES_OK;
                break;

            case GET_SECTOR_COUNT:
                *(DWORD *)buff = SPI_FLASH_SECTOR_COUNT;
                res = RES_OK;
                break;

            default:
                res = RES_PARERR;
                break;
        }
    }
    else
    {
        res = RES_ERROR;    /* 其他的不支持 */
    }

    return res;
}
#endif /* _USE_IOCTL == 1 */

/**
  * @brief  Gets Time from RTC
  * @param  None
  * @retval Time in DWORD
  */
__weak DWORD get_fattime (void)
{
  return 0;
}
```

该文件我们主要修改了几个函数，分别是：disk_initialize、disk_read、disk_wirte、disk_ioctl，下面我们分别来讲解下这几个函数的作用：

1）disk_initialize 作用：初始化指定编号的磁盘，磁盘所指定的存储区。该函数只有一个形参，即 FATFS 管理的磁盘编号 pdrv：磁盘编号 0~9，我们配置_VOLUMES 为 2 来支持两个磁盘，因此可选值为 0 和 1。

2）disk_read 作用：从磁盘驱动器上读取扇区数据。该函数有四个形参：

3）disk_wirte 作用：向磁盘驱动器上写入扇区数据。

4）disk_ioctl 作用：控制设备指定特性和除了读/写外得杂项功能。

（2）添加 exfuns 文件代码

本章，我们编写了 4 个文件，分别是：exfuns.c、exfuns.h、fattester.c 和 fattester.h。这 4 个文件存放于文件夹 exfuns 中。

exfuns.c 主要定义了一些全局变量，方便 FATFS 的使用，同时实现了磁盘容量获取等函数。而 fattester.c 文件则主要是为了测试 FATFS 用，因为 FATFS 的很多函数无法直接通过 USMART 调用，所以我们在 fattester.c 里面对这些函数进行了一次再封装，使得可以通过 USMART 调用。

我们 exfuns 文件夹保存在本实验的 Middlewares/Third_Party/FatFs 路径下，如下图所示：

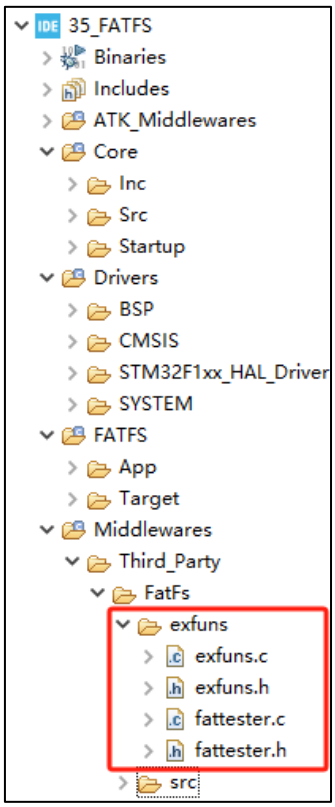


图 57.2.4.1 添加 exfuns 文件

关于这些文件的内容，大家可打开本例程源码查看。

（3）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint32_t total, free;
    uint8_t t = 0, key = 0;
    UINT bw = 0, br = 0;
    uint8_t res = 0;
    FIL *f_fil;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();
```

```
/* SDIO 初始化 */
MX_SDIO_SD_Init();

/* FATFS 初始化 */
MX_FATFS_Init();

/* SPI2 初始化 */
MX_SPI2_Init();

/* USER CODE BEGIN 2 */
lcd_init();                                 /* 初始化 LCD */
norflash_init();                            /* 初始化 NORFLASH */
my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */
lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
lcd_show_string(30,  70, 200, 16, 16, "FATFS TEST", RED);
lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
lcd_show_string(30, 110, 200, 16, 16, "Use USMART for test", RED);
while (g_sd_error == 1)                     /* 检测不到 SD 卡 */
{
    g_sd_error = 0;
    MX_SDIO_SD_Init();
    lcd_show_string(30, 130, 200, 16, 16, "SD Card Error!", RED);
    delay_ms(500);
    lcd_show_string(30, 130, 200, 16, 16, "Please Check! ", RED);
    delay_ms(500);
    LED0_TOGGLE(); /* LED0 闪烁 */
}
exfuns_init();                              /* 为 fatfs 相关变量申请内存 */

f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
res = f_mount(fs[1], "1:", 1);              /* 挂载 SPI_FLASH 卡 */
if (res == FR_NO_FILESYSTEM) /* FLASH 磁盘,FAT 文件系统错误,重新格式化 FLASH */
{
    lcd_show_string(30, 130, 200, 16, 16, "Flash Disk Formatting...", RED);
     /* 格式化 FLASH */
    res = f_mkfs("1:", 0, _MAX_SS);
     /* 格式化 FLASH,1:,盘符;0,使用默认格式化参数 */

    if (res == 0)
    {
        /* 设置 Flash 磁盘的名字为：ALIENTEK */
        f_setlabel((const TCHAR *)"1:ALIENTEK");
        lcd_show_string(30, 130, 200, 16, 16, "Flash Disk Format Finish",
        RED); /* 格式化完成 */
    }
    else
    {
        lcd_show_string(30, 130, 200, 16, 16, "Flash Disk Format Error ",
        RED); /* 格式化失败 */
    }

    delay_ms(1000);
}

lcd_fill(30, 130, 240, 150 + 16, WHITE);     /* 清除显示 */
/* 得到 SD 总容量和剩余容量 */
while (exfuns_get_free((uint8_t*)"0", &total, &free))
{
    lcd_show_string(30, 130, 200, 16, 16, "SD Card Fatfs Error!", RED);
    delay_ms(200);
    lcd_fill(30, 130, 240, 150 + 16, WHITE); /* 清除显示 */
    delay_ms(200);
    LED0_TOGGLE();                           /* LED0 闪烁 */
```

```
    }
    lcd_show_string(30, 130, 200, 16, 16, "FATFS OK!", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "SD Total Size:     MB", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "SD  Free Size:     MB", BLUE);
    lcd_show_num(30 + 8 * 14, 150, total >> 10, 5, 16, BLUE);
    lcd_show_num(30 + 8 * 14, 170, free >> 10, 5, 16, BLUE);

    f_fil = (FIL *)mymalloc(SRAMIN, sizeof(f_fil));  /* 为 file 申请内存 */
    if (f_fil == NULL)                               /* 内存申请失败 */
    {
        myfree(SRAMIN, f_fil);
    }
    printf("文件系统挂载成功\r\n");
    printf("************ 即将进行文件读写测试 ************\r\n");

    /* USER CODE END 2 */

    while (1)
    {
        /*-------------------- 文件系统测试：读写测试 ----------------------*/
       key = key_scan(0);
       if(key == KEY0_PRES)
       {
           res = f_open(f_fil, "1:test.txt", FA_OPEN_ALWAYS | FA_WRITE |
            FA_READ);/* 打开文件，没有文件就创建新文件 */
           if(res == FR_OK)
           {
               f_lseek (f_fil, 0);             /* 偏移到初始位置写数据 */
               res = f_write(f_fil, write_buf, sizeof(write_buf), &bw);
               if(res == FR_OK)
               {
                   printf("数据写入成功！！\r\n");
                   lcd_show_string(30, 190, 200, 16, 16, "write OK", BLUE);
               }
           }else
           {
               lcd_show_string(30, 190, 200, 16, 16, "read and write Error",
               BLUE);
           }
       }else if(key == KEY1_PRES)
       {
           f_lseek (f_fil, 0);                 /* 偏移到初始位置读数据 */
           res = f_read(f_fil, read_buf, sizeof(read_buf), &br);
           lcd_show_string(30, 210, 200, 16, 16, (char *)read_buf, BLUE);
           printf("读取到的数据位：%s\r\n",read_buf);
           printf("读取字节数据：%d\r\n", br);
           f_close(f_fil);
       }

        t++;
        if(t % 20 == 0)
        {
            LED0_TOGGLE(); /* LED0 闪烁 */
        }
        delay_ms(10);
    }
}
```

在 main 函数里面，我们为 SD 卡和 FLASH 都注册了工作区（挂载），在初始化 SD 卡并显示其容量信息后，进入死循环，按键判断，当 KEY0 按下，则通过 FATFS 打开 SD 卡路径并新建 test.txt 文件，并在该文件写入“Hello test demo”，接着按下 KEY1 读出刚刚写入的内容到串口打印出来。大家可自行测试。

我们也可以通过 USMART 测试。

在 usmart_config.c 里面的 usmart_nametab 数组添加如下内容：

```
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",
    (void *)mf_mount, "uint8_t mf_mount(uint8_t* path,uint8_t mt)",
    (void *)mf_open, "uint8_t mf_open(uint8_t*path,uint8_t mode)",
    (void *)mf_close, "uint8_t mf_close(void)",
    (void *)mf_read, "uint8_t mf_read(uint16_t len)",
    (void *)mf_write, "uint8_t mf_write(uint8_t*dat,uint16_t len)",
    (void *)mf_opendir, "uint8_t mf_opendir(uint8_t* path)",
    (void *)mf_closedir, "uint8_t mf_closedir(void)",
    (void *)mf_readdir, "uint8_t mf_readdir(void)",
    (void *)mf_scan_files, "uint8_t mf_scan_files(uint8_t * path)",
    (void *)mf_showfree, "uint32_t mf_showfree(uint8_t *path)",
    (void *)mf_lseek, "uint8_t mf_lseek(uint32_t offset)",
    (void *)mf_tell, "uint32_t mf_tell(void)",
    (void *)mf_size, "uint32_t mf_size(void)",
    (void *)mf_mkdir, "uint8_t mf_mkdir(uint8_t*path)",
    (void *)mf_fmkfs, "uint8_t mf_fmkfs(uint8_t* path,uint8_t opt,uint16_t
au)",
    (void *)mf_unlink, "uint8_t mf_unlink(uint8_t *path)",
    (void *)mf_rename, "uint8_t mf_rename(uint8_t *oldname,uint8_t* newname)",
    (void *)mf_getlabel, "void mf_getlabel(uint8_t *path)",
    (void *)mf_setlabel, "void mf_setlabel(uint8_t *path)",
    (void *)mf_gets, "void mf_gets(uint16_t size)",
    (void *)mf_putc, "uint8_t mf_putc(uint8_t c)",
    (void *)mf_puts, "uint8_t mf_puts(uint8_t *str)",
};
```

57.3 下载验证

在代码编译成功之后，我们通过下载代码到正点原子精英 STM32F103 上，我们测试使用的是 16GB 标有“SDHC”标志的 micorSD 卡，可以看到 LCD 显示如图 57.3.1 所示：

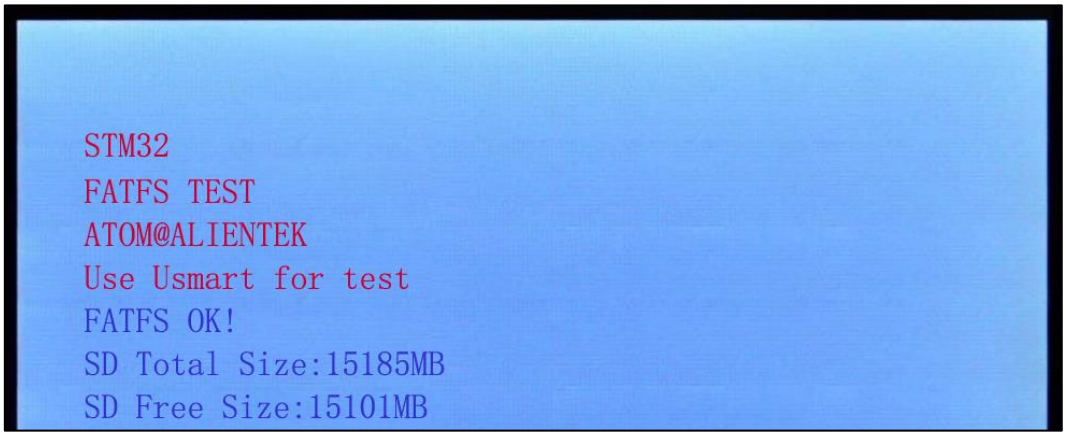


图 57.3.1 程序运行效果图

打开串口调试助手，我们就可以串口调用前面添加的各种 FATFS 测试函数了，比如我们输入 mf_scan_files("0:")，即可扫描 SD 卡根目录的所有文件，如图 57.3.2 所示：

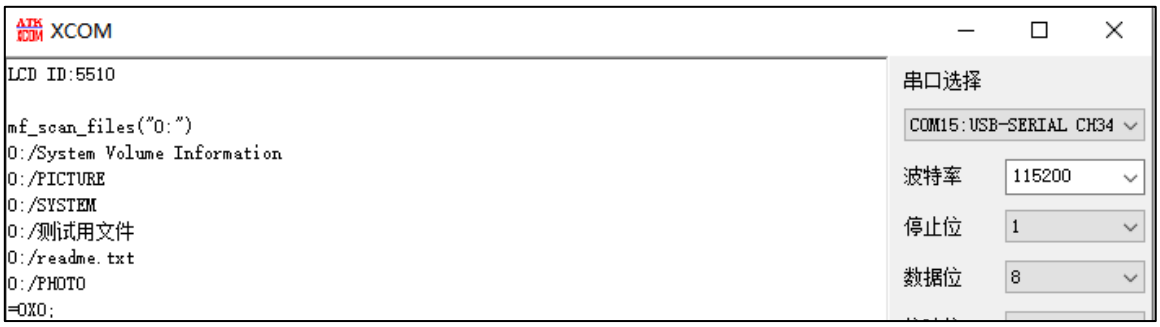


图 57.3.2 扫描 SD 卡根目录所有文件

其他函数的测试，用类似的办法即可实现。注意这里 0 代表 SD 卡，1 代表 SPI FLASH。另外，提醒大家，mf_unlink 函数，在删除文件夹的时候，必须保证文件夹是空的，才可以正常删除，否则不能删除。

第五十八章 汉字显示实验

本章将介绍正点原子提供的字库管理库的使用，通过使用字库能够在 LCD 或其他显示设备上显示中文字符，对应中文应用的开发，是一个很有用的工具。通过本章的学习，读者将学习到正点原子字库管理库的使用。

本章分为如下几个小节：

58.1 硬件设计
58.2 程序设计
58.3 下载验证

58.1 硬件设计

1. 例程功能

1. 程序运行后，会检测 NOR Flash 中是否存在字库，若不存在则会从 SD 卡中更新字库到 NOR Flash
2. NOR Flash 中的字库准备好后，就开始在 LCD 上显示汉字
3. 按下 KEY0 按键可强制更新字库
4. 可通过 USMART 操作 LCD 显示中文
5. LED0 闪烁，指示程序正在运行

2. 硬件资源

1）LED 灯
LED0 – PB5
2）独立按键
KEY0 – PE4
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）micro SD 卡
6）NOR FLASH，需要用到它来存储汉字库

3. 原理图

本章实验使用的字库管理库为软件库，因此没有对应的连接原理图。

58.2 程序设计

58.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 36_HZ。

58.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制上节课 35_FATFS 工程，并将工程文件名和.ioc 文件名均修改为 36_HZ，以进行下一步的实验操作。如下图所示：

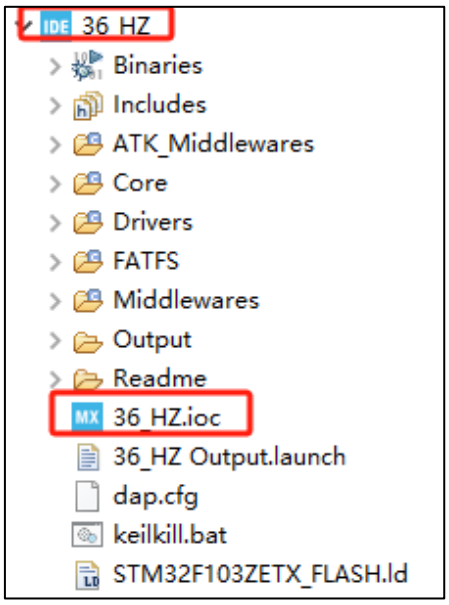


图 58.2.2.1 HZ 工程

本章实验使用的字库管理库为软件库，因此不需在 STM32CubeIDE 上进行其他配置，直接添加相应的驱动代码即可。

58.2.3 添加用户驱动代码

在 ATK_Middlewares 文件夹中，新建 TEXT 文件夹，在 TEXT 中，我们新建 4 个文件分别是 text.c、text.h、fonts.c 和 fonts.h。

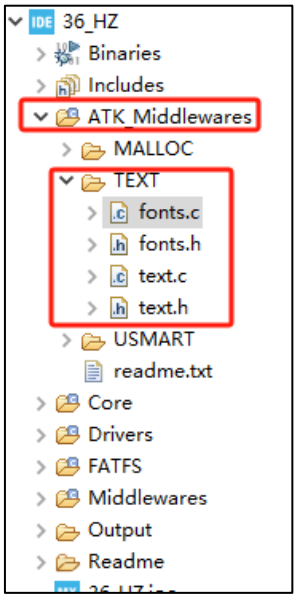


图 58.2.3.1 添加用户文件

汉字显示实验代码主要分为两部分：一部分就是对字库的更新，另一部分就是对汉字的显示。字库的更新代码放在 font.c 和 font.h 文件中，汉字的显示代码就放在 text.c 和 text.h 中。关于这几个文件的源码，大家可打开本实验查看，这里不过多讲解。

（1）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
```

```
    uint32_t fontcnt;
    uint8_t i, j;
    uint8_t fontx[2]; /* GBK 码 */
    uint8_t key, t;
/* USER CODE END 1 */

HAL_Init();

/* 系统时钟初始化 */
SystemClock_Config();

/* 延时初始化 */
delay_init(72);

/* GPIO 初始化 */
MX_GPIO_Init();

/* FSMC 初始化 */
MX_FSMC_Init();

/* 串口初始化 */
MX_USART1_UART_Init();

/* TIM4 初始化 */
MX_TIM4_Init();

/* SDIO 初始化 */
MX_SDIO_SD_Init();

/* FATFS 初始化 */
MX_FATFS_Init();

/* SPI2 初始化 */
MX_SPI2_Init();

/* USER CODE BEGIN 2 */
lcd_init();                                 /* 初始化 LCD */
norflash_init();                            /* 初始化 NORFLASH */
my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */
exfuns_init();                              /* 为 fatfs 相关变量申请内存 */

f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
f_mount(fs[1], "1:", 1);                    /* 挂载 SPI_FLASH 卡 */

while (fonts_init())                        /* 检查字库 */
{
UPD:
    lcd_clear(WHITE);                        /* 清屏 */
    lcd_show_string(30, 30, 200, 16, 16, "STM32", RED);

    while (g_sd_sta == 1)                /* 检测 SD 卡 */
    {
        g_sd_sta = 0;
        MX_SDIO_SD_Init();
        lcd_show_string(30, 50, 200, 16, 16, "SD Card Failed!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
    }

    lcd_show_string(30, 50, 200, 16, 16, "SD Card OK", RED);
    lcd_show_string(30, 70, 200, 16, 16, "Font Updating...", RED);
    key = fonts_update_font(20, 90, 16, (uint8_t *)"0:", RED); /* 更新字库 */
```

```
    while (key) /* 更新失败 */
    {
        lcd_show_string(30, 90, 200, 16, 16, "Font Update Failed!", RED);
        delay_ms(200);
        lcd_fill(20, 90, 200 + 20, 90 + 16, WHITE);
        delay_ms(200);
    }

    lcd_show_string(30, 90, 200, 16, 16, "Font Update Success!   ", RED);
    delay_ms(1500);
    lcd_clear(WHITE); /* 清屏 */
}

text_show_string(30, 30, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30, 50, 200, 16, "GBK 字库测试程序", 16, 0, RED);
text_show_string(30, 70, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
text_show_string(30, 90, 200, 16, "按 KEY0,更新字库", 16, 0, RED);


text_show_string(30, 110, 200, 16, "内码高字节:", 16, 0, BLUE);
text_show_string(30, 130, 200, 16, "内码低字节:", 16, 0, BLUE);
text_show_string(30, 150, 200, 16, "汉字计数器:", 16, 0, BLUE);

text_show_string(30, 180, 200, 24, "对应汉字为:", 24, 0, BLUE);
text_show_string(30, 204, 200, 16, "对应汉字(16*16)为:", 16, 0, BLUE);
text_show_string(30, 220, 200, 16, "对应汉字(12*12)为:", 12, 0, BLUE);
/* USER CODE END 2 */

while (1)
{
    fontcnt = 0;

    for (i = 0x81; i < 0xff; i++)        /* GBK 内码高字节范围为 0X81~0XFE */
    {
        fontx[0] = i;
        lcd_show_num(118, 130, i, 3, 16, BLUE); /* 显示内码高字节 */
        /* GBK 内码低字节范围为 0X40~0X7E, 0X80~0XFE) */
        for (j = 0x40; j < 0xfe; j++)
        {
            if (j == 0x7f)
                continue;

            fontcnt++;
            lcd_show_num(118, 130, j, 3, 16, BLUE);       /* 显示内码低字节 */
            lcd_show_num(118, 150, fontcnt, 5, 16, BLUE); /* 汉字计数显示 */
            fontx[1] = j;
            text_show_font(30 + 132, 180, fontx, 24, 0, BLUE);
            text_show_font(30 + 144, 204, fontx, 16, 0, BLUE);
            text_show_font(30 + 108, 220, fontx, 12, 0, BLUE);
            t = 200;

            while (t--)          /* 延时,同时扫描按键 */
            {
                delay_ms(1);
                key = key_scan(0);

                if (key == KEY0_PRES)
                {
                    goto UPD;    /* 跳转到 UPD 位置(强制更新字库) */
                }
            }

            LED0_TOGGLE();
```

```
            }
        }
    }
}
```

从上面的代码中可以看出，本实验的应用代码中，在完成文件系统等一系列初始化后，便调用函数 fonts_init()初始化字库管理库，若初始化失败则说明 NOR Flash 中没有对应的字库数据，因此需要进行更新字库，更新字库时，会从 SD 卡中读入字库数据并将其写入 NOR Flash 中，因此需要在 SD 中提前准备好字库文件。

当 NOR Flash 有了字库数据后，便在 LCD 上使用中文显示本实验的实验信息，并不断的显示中文字符，同时检测按键输入，若检测到 KEY0 按键被按下，则强制进行字库更新。

58.3 下载验证

在完成编译和烧录操作后，将 A 盘资料\5，SD 卡根目录文件 文件夹下面的 SYSTEM 文件夹拷贝到 SD 卡根目录下，插入开发板，若 NOR Flash 中没有字库数据，则能够看到 LCD 显示了正在更新字库的提示，更新完成后，便可以在 LCD 上看到中文显示的实验信息和不断刷新显示的中文字符，此时可以按下 KEY0 按键强制更新字库。

第五十九章 图片显示实验

本章将介绍使用 STM32F103 软件解码 BMP、JPG 和 GIF 等格式的图片，并在 LCD 进行显示。通过本章的学习，读者将学习到图片解码库的使用。

59.1 硬件设计
59.2 程序设计
59.3 下载验证

59.1 硬件设计

1. 例程功能

1. 程序运行后，先检测字库，然后检测 SD 卡是否存在，如果 SD 卡存在，则会查找 SD 卡根目录下的 PICTURE 文件夹，接着会显示 SD 卡中 PICTURE 文件夹中的第一张图片（支持 bmp、jpg、jpeg 或 gif 格式）
2. 按下 KEY0 或 KEY_UP 按键，可分别进行切换上一张图片和切换下一张图片的操作
3. 可通过 USMART 操作解码任意路径下的图片
4. LED0 闪烁，指示程序正在运行

2. 硬件资源

1）LED 灯
LED0 – PB5
LED1 – PE5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键：
KEY0 - PE4
KEY1 – PE3
WK_UP - PA0
5）SD 卡，通过 SDIO 连接
6）NOR FLASH(SPI FLASH 芯片，连接在 SPI 上)

3. 原理图

本章实验使用的图片解码库为软件库，因此没有对应的连接原理图。

59.2 程序设计

59.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 37_PICTURE。

59.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制上节课 36_HZ 工程，并将工程文件名和.ioc 文件名均修改为 37_PICTURE，以进行下一步的实验操作。如下图所示：

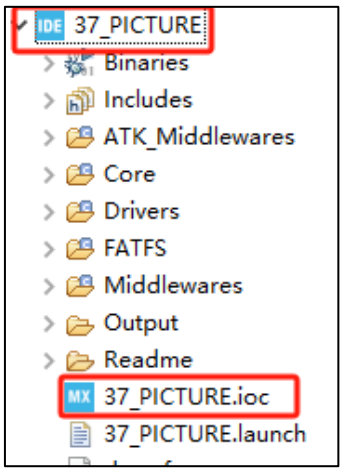


图 59.2.2.1 PICTURE 工程

本章实验使用的图片解码库为软件库，因此不需在 STM32CubeIDE 上进行其他配置，直接添加相应的驱动代码即可。

59.2.3 添加用户驱动代码

在 ATK_Middlewares 文件夹中，新建 PICTURE 文件夹，PICTURE 驱动源码包括九个文件：bmp.c、bmp.h、tjpgd.c、tjpgd.h、gif.c、gif.h、piclib.c 和 piclib.h。

其中：

bmp.c 和 bmp.h 用于实现对 bmp 文件的解码；

tjpgdcnf.h、tjpgd.c 和 tjpgd.h 用于实现对 jpeg/jpg 文件的解码；

gif.c 和 gif.h 用于实现对 gif 文件的解码；

piclib.c 和 piclib.h 为正点原子图片解码库文件，用于实现图片解码库和画图的初始化和根据文件类型自动解码并画图的函数等；

这几个代码太长了，而且也有规定的标准，需要结合各个图片编码的格式来编写，所以我们在这里不贴出来，大家查看光盘中的源码的实现过程即可。

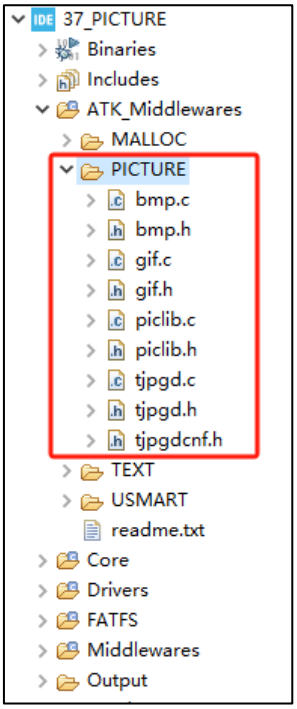


图 59.2.3.1 添加用户文件

（1）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t res;
    DIR picdir;             /* 图片目录 */
    FILINFO picfileinfo;    /* 文件信息 */
    char *pname;            /* 带路径的文件名 */
    uint8_t *fn;            /* 长文件名 */
    uint16_t totpicnum;     /* 图片文件总数 */
    uint16_t curindex;      /* 图片当前索引 */
    uint8_t key;            /* 键值 */
    uint8_t pause = 0;      /* 暂停标记 */
    uint8_t t;
    uint16_t temp;
    uint32_t *picoffsettbl; /* 图片文件 offset 索引表 */
  /* USER CODE END 1 */
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* SDIO 初始化 */
  MX_SDIO_SD_Init();

  /* FATFS 初始化 */
  MX_FATFS_Init();

  /* SPI2 初始化 */
  MX_SPI2_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                                 /* 初始化 LCD */
  norflash_init();                            /* 初始化 NORFLASH */
  my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */
  exfuns_init();                              /* 为 fatfs 相关变量申请内存 */

  f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
  f_mount(fs[1], "1:", 1);                    /* 挂载 SPI_FLASH 卡 */

  while (fonts_init())                        /* 检查字库 */
  {
      lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
      delay_ms(200);
      lcd_fill(30, 50, 240, 66, WHITE);       /* 清除显示 */
      delay_ms(200);
  }
```

```
text_show_string(30,  50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30,  70, 200, 16, "图片显示 实验", 16, 0, RED);
text_show_string(30,  90, 200, 16, "KEY0:NEXT KEY1:PREV", 16, 0, RED);
text_show_string(30, 110, 200, 16, "KEY_UP:PAUSE", 16, 0, RED);
text_show_string(30, 130, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);

while (f_opendir(&picdir, "0:/PICTURE")) /* 打开图片文件夹 */
{
    text_show_string(30, 150, 240, 16, "PICTURE 文件夹错误!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 150, 240, 186, WHITE);    /* 清除显示 */
    delay_ms(200);
}

totpicnum = pic_get_tnum("0:/PICTURE");    /* 得到总有效文件数 */

while (totpicnum == NULL)                  /* 图片文件为 0 */
{
    text_show_string(30, 150, 240, 16, "没有图片文件!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 150, 240, 186, WHITE);                    /* 清除显示 */
    delay_ms(200);
}
picfileinfo.lfsize = _MAX_LFN * 2 + 1;                     /* 长文件名最大长度 */
/* 为长文件缓存区分配内存 */
picfileinfo.lfname = mymalloc(SRAMIN, picfileinfo.lfsize);
/* 为带路径的文件名分配内存 */
pname=mymalloc(SRAMIN, picfileinfo.lfsize);
/* 申请 4*totpicnum 个字节的内存,用于存放图片索引 */
picoffsettbl = mymalloc(SRAMIN, 2 * totpicnum);

while (!picfileinfo.lfname || !pname || !picoffsettbl)   /* 内存分配出错 */
{
    text_show_string(30, 150, 240, 16, "内存分配失败!", 16, 0, RED);
    delay_ms(200);
    lcd_fill(30, 150, 240, 186, WHITE); /* 清除显示 */
    delay_ms(200);
}

/* 记录索引 */
res = f_opendir(&picdir, "0:/PICTURE");  /* 打开目录 */

if (res == FR_OK)
{
    curindex = 0;                          /* 当前索引为 0 */

    while (1)                              /* 全部查询一遍 */
    {
        temp = picdir.index;                      /* 记录当前 dptr 偏移 */
        res = f_readdir(&picdir, &picfileinfo); /* 读取目录下的一个文件 */

        if (res != FR_OK || picfileinfo.fname[0] == 0)
            break; /* 错误了/到末尾了,退出 */
        fn = (uint8_t*)(*picfileinfo.lfname? picfileinfo.lfname :
        picfileinfo.fname);
        res = exfuns_file_type((char *)fn);

        if ((res & 0XF0) == 0X50)              /* 取高四位,看看是不是图片文件 */
        {
            picoffsettbl[curindex] = temp;     /* 记录索引 */
            curindex++;
```

```
        }
    }
}

text_show_string(30, 150, 240, 16, "开始显示...", 16, 0, RED);
delay_ms(1500);
piclib_init();                                          /* 初始化画图 */
curindex = 0;                                           /* 从 0 开始显示 */
res = f_opendir(&picdir, (const TCHAR *)"0:/PICTURE"); /* 打开目录 */

while (res == FR_OK) /* 打开成功 */
{
    dir_sdi(&picdir, picoffsettbl[curindex]);    /* 改变当前目录索引 */
    res = f_readdir(&picdir, &picfileinfo);      /* 读取目录下的一个文件 */

    if (res != FR_OK || picfileinfo.fname[0] == 0)
        break; /* 错误了/到末尾了,退出 */
    fn = (uint8_t*)(*picfileinfo.lfname? picfileinfo.lfname :
     picfileinfo.fname);
    strcpy((char *)pname, "0:/PICTURE/");             /* 复制路径(目录) */
    strcat((char *)pname, (const char *)fn);          /* 将文件名接在后面 */
    lcd_clear(BLACK);
    piclib_ai_load_picfile(pname, 0, 0, lcddev.width, lcddev.height, 1);
    text_show_string(2, 2, lcddev.width, 16, (char *)pname, 16, 1, RED);
    t = 0;
/* USER CODE END 2 */

while (1)
{
    key = key_scan(0);                /* 扫描按键 */

    if (t > 250)
        key = 1;                      /* 模拟一次按下 KEY0 */

    if ((t % 20) == 0)
    {
        LED0_TOGGLE();                /* LED0 闪烁,提示程序正在运行. */
    }

    if (key == KEY1_PRES)             /* 上一张 */
    {
        if (curindex)
        {
            curindex--;
        }
        else
        {
            curindex = totpicnum - 1;
        }

        break;
    }
    else if (key == KEY0_PRES)        /* 下一张 */
    {
        curindex++;

        if (curindex >= totpicnum)
            curindex = 0;             /* 到末尾的时候,自动从头开始 */

        break;
    }
    else if (key == WKUP_PRES)
    {
        pause = !pause;
```

```
            LED1(!pause);                   /* 暂停的时候 LED1 亮. */
        }

        if (pause == 0)
            t++;

        delay_ms(10);
    }
    res = 0;
    }
    myfree(SRAMIN, picfileinfo.lfname);     /* 释放内存 */
    myfree(SRAMIN, pname);                  /* 释放内存 */
    myfree(SRAMIN, picoffsettbl);           /* 释放内存 */
}
```

可以看到整个设计思路是跟据图片解码库来设计的，piclib_ai_load_picfile()是这套代码的核心，其它的交互是围绕它和图片解码后的图片信息作的显示。大家再仔细对照光盘中的源码进一步了解整个设置思路。另外，我们的程序中只分配了 4 个文件索引，故更多数量的图片无法直接在本程序下演示，大家根据自己的需要再进行修改即可。

59.3 下载验证

在代码编译成功之后，我们下载代码到正点原子精英 STM32F103 上，可以看到 LCD 开始显示图片（假设 SD 卡及文件都准备好了，即：在 SD 卡根目录新建：PICTURE 文件夹，并存放一些图片文件(.bmp/.jpg/.gif)在该文件夹内），如图 59.3.1 所示：

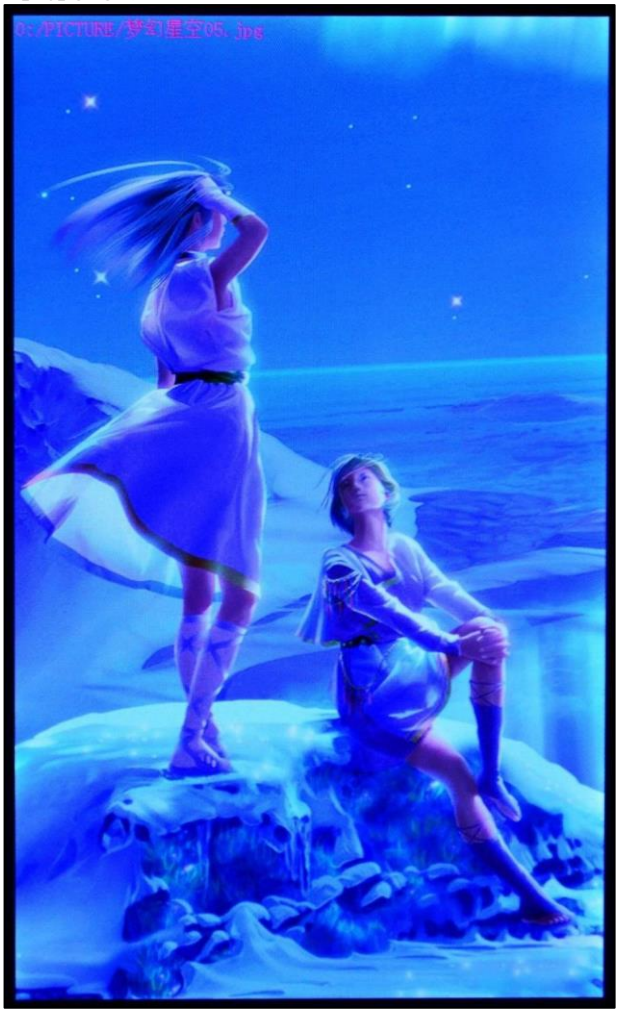


图 59.3.1 图片显示实验显示效果

按 KEY0 和 KEY2 可以快速切换到下一张或上一张，KEY_UP 按键可以暂停自动播放，

同时 DS1 亮，指示处于暂停状态，再按一次 KEY_UP 则继续播放。同时，由于我们的代码支持 gif 格式的图片显示（注意尺寸不能超过 LCD 屏幕尺寸），所以可以放一些 gif 图片到 PICTURE 文件夹，来看动画了。

需注意的是，解码 JPEG 格式的图片时速度较慢，是由于软件解码需要消耗一定的时间，加上库的效率较低，所以属于正常现象。

第六十章 照相机实验

上一章，我们学习了图片解码，本章我们将学习 BMP 编码，结合前面的摄像头实验，实现一个简单的照相机。本章分为如下几个部分：

60.1 硬件设计
60.2 软件设计
60.3 下载验证

60.1 硬件设计

1. 例程功能

开机的时候先检测字库，然后检测 SD 卡根目录是否存在 PHOTO 文件夹，如果不存在则创建，如果创建失败，则报错（提示拍照功能不可用）。在找到 SD 卡的 PHOTO 文件夹后，开始初始化 OV7725，在初始化成功之后，就一直在屏幕显示 OV7725 拍到的内容。当按下 KEY0 按键的时候，即进行拍照，此时 LED1 亮，拍照保存成功之后，蜂鸣器会发出“滴”的一声，提示拍照成功，同时 LED1 灭。LED0 还是用于指示程序运行状态。

2. 硬件资源

1) LED 灯
LED0 – PB5
LED1 – PE5
2) 独立按键
KEY0 - PE4
3) 串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4) 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5) micro SD 卡
6) NOR FLASH
7) 外部中断 8（PA8，用于检测 OV7725 的帧信号）
8) 定时器 6（用于打印摄像头帧率）
9) 正点原子 OV7725 摄像头模块，连接关系为:

OV7725 模块	STM32 开发板	OV7725 模块	STM32 开发板
OV_D0~D7	PC0~7	FIFO_OE	PG15
OV_SCL	PD3	FIFO_WRST	PD6
OV_SDA	PG13	FIFO_WEN	PB3
OV_VSYNC	PA8	FIFO_RCLK	PB4
FIFO_RRST	PG14		

3. 原理图

本章实验涉及的 SD 卡、摄像头、 LCD 等的连接原理图，请读者自行查看前面对应章节中的连接原理图。

60.2 程序设计

60.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 38_PHOTO。

60.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制上节课 37_PICTURE 工程，并将工程文件名和.ioc 文件名均修改为 38_PHOTO，以进行下一步的实验操作。如下图所示：

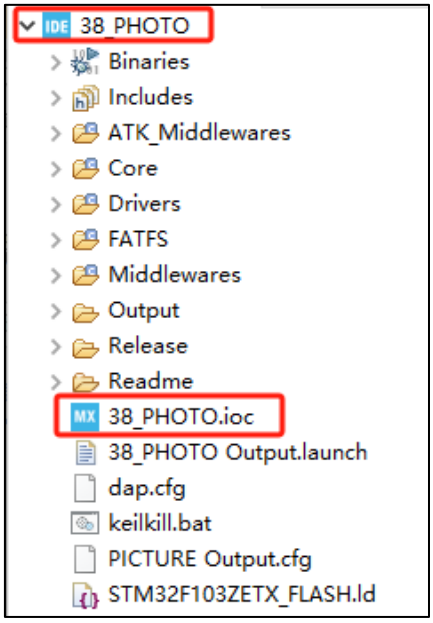


图 60.2.2.1 PHOTO 工程

本章实验是在图片显示实验的基础上进行修改的，因此在 SD 卡和图片编解码等方面无需进行额外的配置和修改。但是，由于本实验还使用了 7725 摄像头，因此需要进行摄像头的配置。有关摄像头配置的详细信息，请参阅第 54.2.2 节。

60.2.3 添加用户驱动代码

按照上述步骤配置后，工程文件如下图所示：

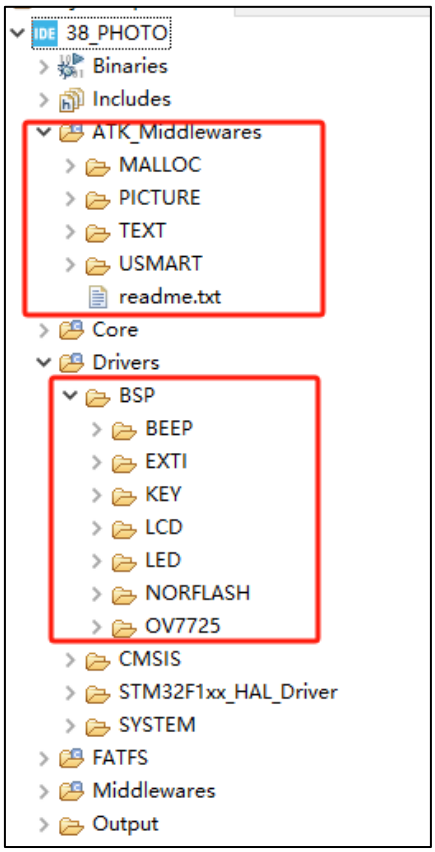


图 60.2.3.1 添加用户文件

接着添加照相机相关的应用程序，直接在 mian.c 中添加。

（1）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t res;
    char *pname;                            /* 带路径的文件名 */
    uint8_t key;                            /* 键值 */
    uint8_t i;
    uint8_t sd_ok = 1;                      /* 0, sd 卡不正常; 1, SD 卡正常 */
    uint8_t vga_mode = 0;   /* 0, QVGA 模式(320 * 240); 1, VGA 模式(640 * 480) */

  /* USER CODE END 1 */

  HAL_Init();
  SystemClock_Config();

  delay_init(72);                           /* 初始化延时函数 */

  MX_GPIO_Init();
  MX_FSMC_Init();
  MX_USART1_UART_Init();
  MX_SDIO_SD_Init();
  MX_FATFS_Init();
  MX_SPI2_Init();
  MX_TIM4_Init();
  MX_TIM6_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                               /* 初始化 LCD */
  norflash_init();                          /* 初始化 NORFLASH */
```

```
my_mem_init(SRAMIN);                            /* 初始化内部 SRAM 内存池 */
exfuns_init();                                  /* 为 fatfs 相关变量申请内存 */

f_mount(fs[0], "0:", 1);                        /* 挂载 SD 卡 */
f_mount(fs[1], "1:", 1);                        /* 挂载 SPI_FLASH 卡 */

piclib_init();                                  /* 初始化画图 */
while (fonts_init())                            /* 检查字库 */
{
    lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
    delay_ms(200);
    lcd_fill(30, 50, 240, 66, WHITE);           /* 清除显示 */
    delay_ms(200);
}

text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30, 70, 200, 16, "照相机 实验", 16, 0, RED);
text_show_string(30, 90, 200, 16, "KEY0:拍照(bmp 格式)", 16, 0, RED);
res = f_mkdir("0:/PHOTO");                      /* 创建 PHOTO 文件夹 */

if (res != FR_EXIST && res != FR_OK)            /* 发生了错误 */
{
    res = f_mkdir("0:/PHOTO");                  /* 创建 PHOTO 文件夹 */
    text_show_string(30, 110, 240, 16, "SD 卡错误!", 16, 0, RED);
    delay_ms(200);
    text_show_string(30, 110, 240, 16, "拍照功能将不可用!", 16, 0, RED);
    delay_ms(200);
    sd_ok = 0;
}

while (ov7725_init() != 0)                      /* 初始化 OV7725 失败? */
{
    lcd_show_string(30, 130, 200, 16, 16, "OV7725 Error!!", RED);
    delay_ms(200);
    lcd_fill(30, 130, 239, 246, WHITE);
    delay_ms(200);
}

lcd_show_string(30, 130, 200, 16, 16, "OV7725 Init OK       ", RED);

delay_ms(1500);

/* 输出窗口大小设置 QVGA / VGA 模式 */
g_ov7725_wwidth = 320;                      /* 默认窗口宽度为 320 */
g_ov7725_wheight = 240;                     /* 默认窗口高度为 240 */
ov7725_window_set(g_ov7725_wwidth, g_ov7725_wheight, vga_mode);

ov7725_light_mode(0);                       /* 自动 灯光模式 */
ov7725_color_saturation(4);                 /* 默认 色彩饱和度 */
ov7725_brightness(4);                       /* 默认 亮度 */
ov7725_contrast(4);                         /* 默认 对比度 */
ov7725_special_effects(0);                  /* 默认 特效 */

OV7725_OE(0);                               /* 使能 OV7725 FIFO 数据输出 */

pname = mymalloc(SRAMIN, 30);               /* 为带路径的文件名分配 30 个字节的内存 */
lcd_clear(BLACK);

/* USER CODE END 2 */

while (1)
{
```

```
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            if (sd_ok)
            {
                LED1(0);                        /* 点亮 LED1,提示正在拍照 */
                camera_new_pathname(pname); /* 得到文件名 */

                /* 编码成 bmp 图片 */
                if (bmp_encode((uint8_t *)pname, (lcddev.width -
                 g_ov7725_wheight) / 2, (lcddev.height - g_ov7725_wwidth) / 2,
                 g_ov7725_wheight, g_ov7725_wwidth, 0))
                {
                    text_show_string(40, 110, 240, 12, "写入文件错误!", 12, 0,
                     RED);
                }
                else
                {
                    text_show_string(40, 110, 240, 12, "拍照成功!", 12, 0, BLUE);
                    text_show_string(40, 130, 240, 12, "保存为:", 12, 0, BLUE);
                    text_show_string(40 + 42, 130, 240, 12, pname, 12, 0, BLUE);
                    BEEP(1);      /* 蜂鸣器短叫，提示拍照完成 */
                    delay_ms(100);
                }
            }
            else                        /* 提示 SD 卡错误 */
            {
                text_show_string(40, 110, 240, 12, "SD 卡错误!", 12, 0, RED);
                text_show_string(40, 130, 240, 12, "拍照功能不可用!", 12, 0, RED);
            }

            BEEP(0);                    /* 关闭蜂鸣器 */
            LED1(1);                    /* 关闭 LED1 */
            delay_ms(1800);             /* 等待 1.8 秒钟 */
            lcd_clear(BLACK);
        }
        else
        {
            delay_ms(5);
        }

        ov7725_camera_refresh(); /* 更新显示 */

        i++;

        if (i >= 15)
        {
            i = 0;
            LED0_TOGGLE();          /* LED0 闪烁 */
        }
    }
}
```

可以看到，本章实验实际上是前面章节中 SD 卡驱动、摄像头驱动、 FATFS、字库管理库、图片编解码库等的综合应用，并没有涉及新的内容。

然后为了模拟照相机的效果，我们需要把 LCD 上显示的画像读取出来并用 bmp_encode()函数编码成*.bmp 格式的图片进行存储，以模拟实时的拍照效果。

60.3 下载验证

将程序下载到开发板后，可以看到 LCD 首先显示一些实验相关的信息，如图 60.3.1 所示：

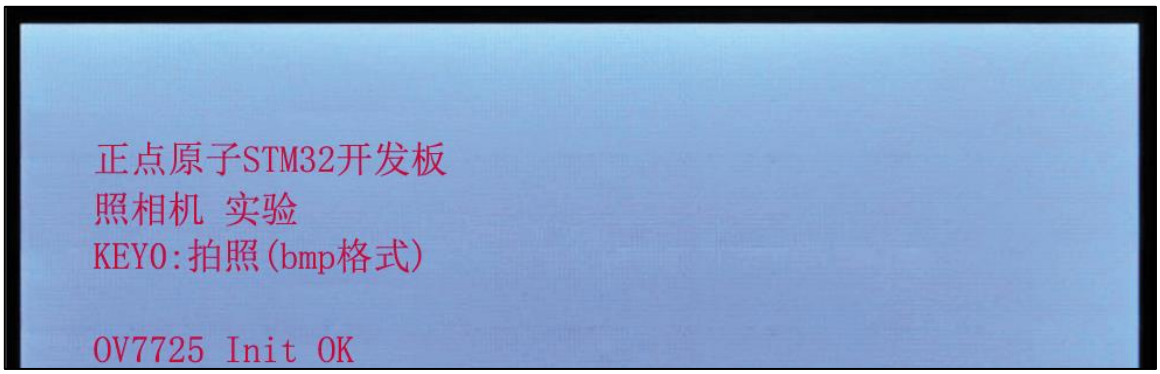


图 60.3.1 显示实验相关信息

随后，进入监控界面。此时，我们可以按下 KEY0 即可进行拍照。拍照得到的照片效果如图 60.3.2 所示：

图 60.3.2 拍照样图

最后，为了得到最快的显示速度，我们可以编译时选择 Release 进行编译并下载，这样 OV7725 的显示帧率可达 25 帧左右。

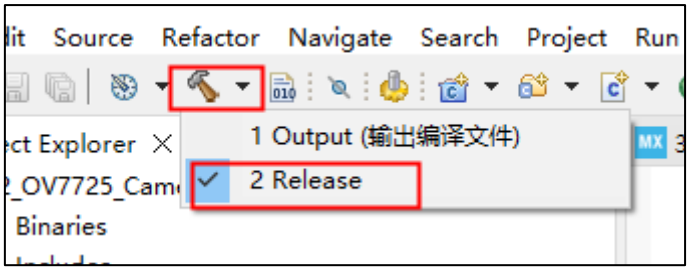


图 60.3.3 使用 Release 编译提高代码效率

第六十一章 DSP 测试实验

本章，我们将指导大家入门 STM32F103 的 DSP，手把手教大家搭建 DSP 库测试环境，同时通过对 DSP 库中的几个基本数学功能函数和 FFT 快速傅里叶变换函数的测试，让大家对 STM32F103 的 DSP 库有个基本的了解。

本章分为如下几个小节：

61.1 硬件设计
61.2 程序设计
61.3 下载验证

61.1 硬件设计

1. 例程功能

本章包含 2 个源码：**39_1_DSP_BasicMath** 和 **39_2_DSP_FFT**，他们除了 main.c 里面内容不一样外，其他源码完全一模一样（包括配置）。

实验 39_1_DSP_BasicMath 功能简介：测试 STM32F103 的 DSP 库基础数学函数：arm_cos_f32 和 arm_sin_f32 和标准库基础数学函数：cosf 和 sinf 的速度差别，并在 LCD 屏幕上面显示两者计算所用时间。LED0 闪烁，提示程序运行。

实验 39_2_DSP_FFT 功能简介：测试 STM32F103 的 DSP 库的 FFT 函数，程序运行后，自动生成 1024 点测试序列，然后，每当 KEY0 按下后，调用 DSP 库的 FFT 算法（基 4 法）执行 FFT 运算，在 LCD 屏幕上面显示运算时间，同时将 FFT 结果输出到串口。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键
KEY0 – PE4
5）定时器 6

3. 原理图

本章实验使用的数学库为软件库，因此没有对应的连接原理图。

61.2 程序设计

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 39_1_DSP_BasicMath 和 39_2_DSP_FFT。

61.2.1 创建和配置工程

在 STM32CubeIDE 中，通过复制 13_TFTLCD 工程，并将工程文件名和.ioc 文件名均修改为 39_1_DSP_BasicMath（39_2_DSP_FFT 同理），以进行下一步的实验操作。如下图所示：

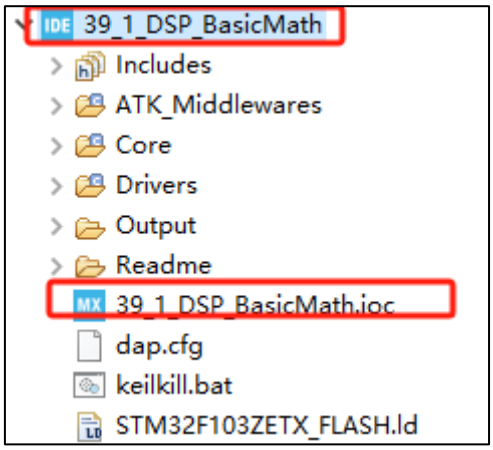


图 61.2.2.1 DSP_BasicMath 工程

本章实验使用的 DSP 库为软件库，因此不需在 STM32CubeIDE 上进行其他配置，直接添加相应的驱动代码即可。

61.2.2 添加 DSP 库

由于 STM32F103 采用 Cortex-M3 内核，没有内置硬件 FPU 单元，也没有 DSP 指令集。所以我们这里只是通过调用 DSP 库进行测试。那 DSP 库我们应该去哪移植呢？在 ST 提供的 HAL 库固件包里就有提供（该文件可以在 www.st.com 网站下载，搜索 STM32CubeF1 即可找到最新版本）。我们需要找到下图路径所示文件：

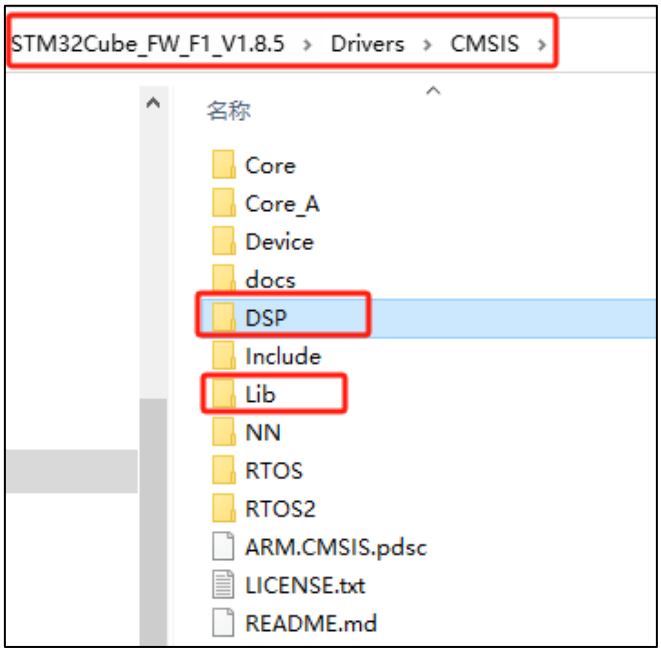


图 61.2.2.1 DAP 库路径

在该文件夹中，DSP 文件夹中 Include 文件是我们需要的，Lib 目录中我们只需要 GCC 文件夹中的库即可（因为 CubeIDE 使用 gcc 编译器）。将上述提到的文件拷贝到 39_1 DSP

_BasicMath\Drivers\CMSIS（39_2_DSP_FFT 同理）目录下:

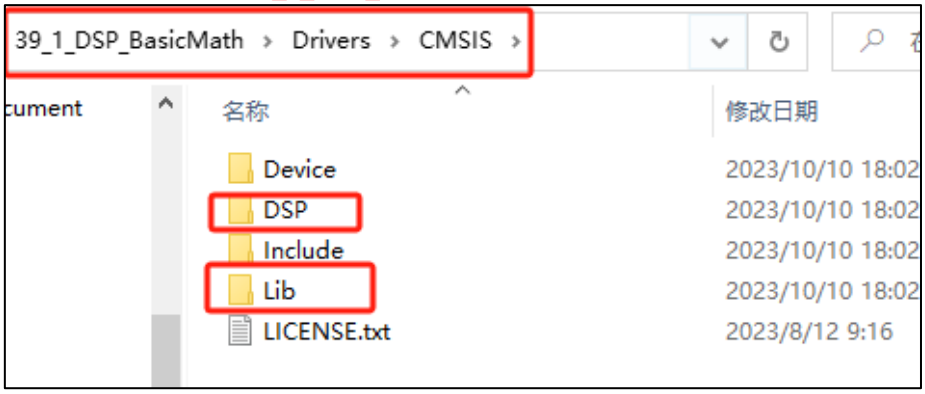


图 61.2.2.2 添加 DSP 库

打开 DSP 和 Lib 查看下里边的内容，如下图所示：

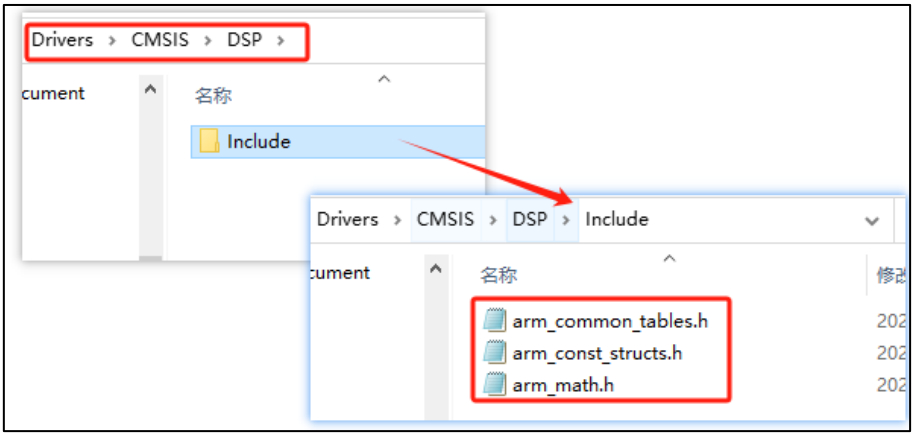


图 61.2.2.3 DSP 文件夹内容

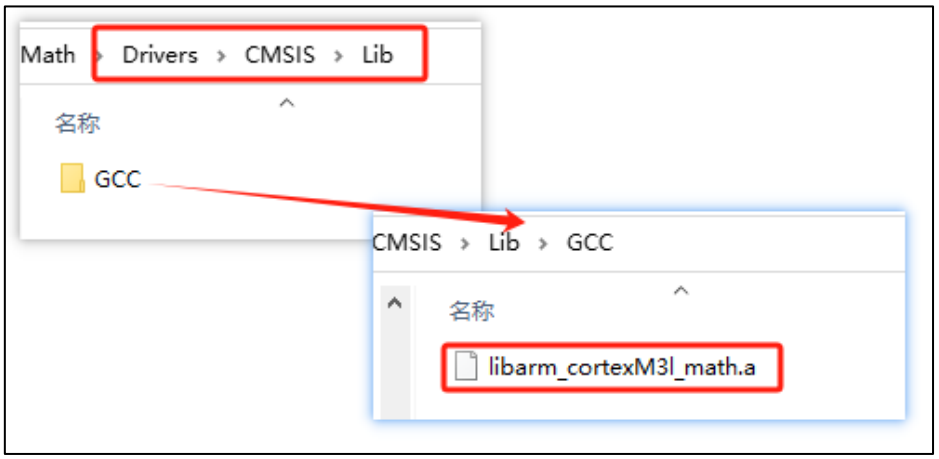


图 61.2.2.4 Lib 文件夹内容

61.2.3 添加宏定义

在 Symbols 中添加宏定义 ARM_MATH_CM3 告诉 DSP 库使用的 arm 内核类型，如下图所示：

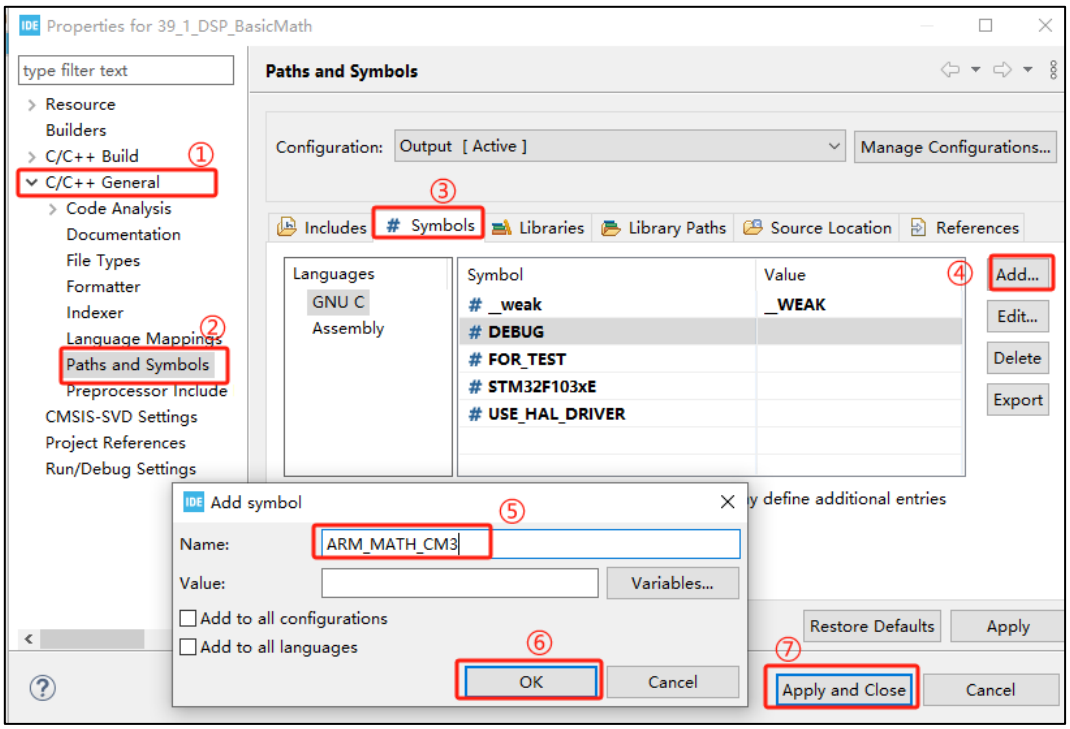


图 61.2.3.1 添加宏定义

61.2.4 添加头文件路径

右键项目→Properties。C/C++General→Paths and Symbols。在 Includes 选项卡中添加 DSP 的头文件：Drivers/CMSIS/DSP/Include 。

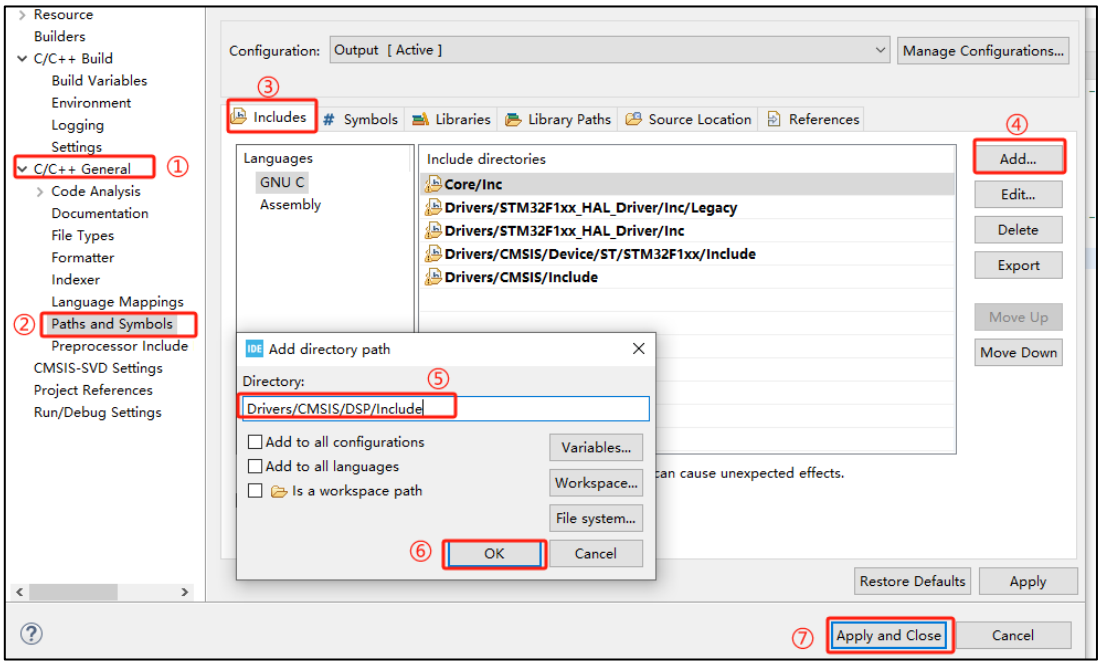


图 61.2.4.1 添加头文件路径

61.2.5 添加 Lib 库路径

为了让 STM32CubeIDE 能找到我们前面添加的 Lib 库，我们还需要进行以下操作：

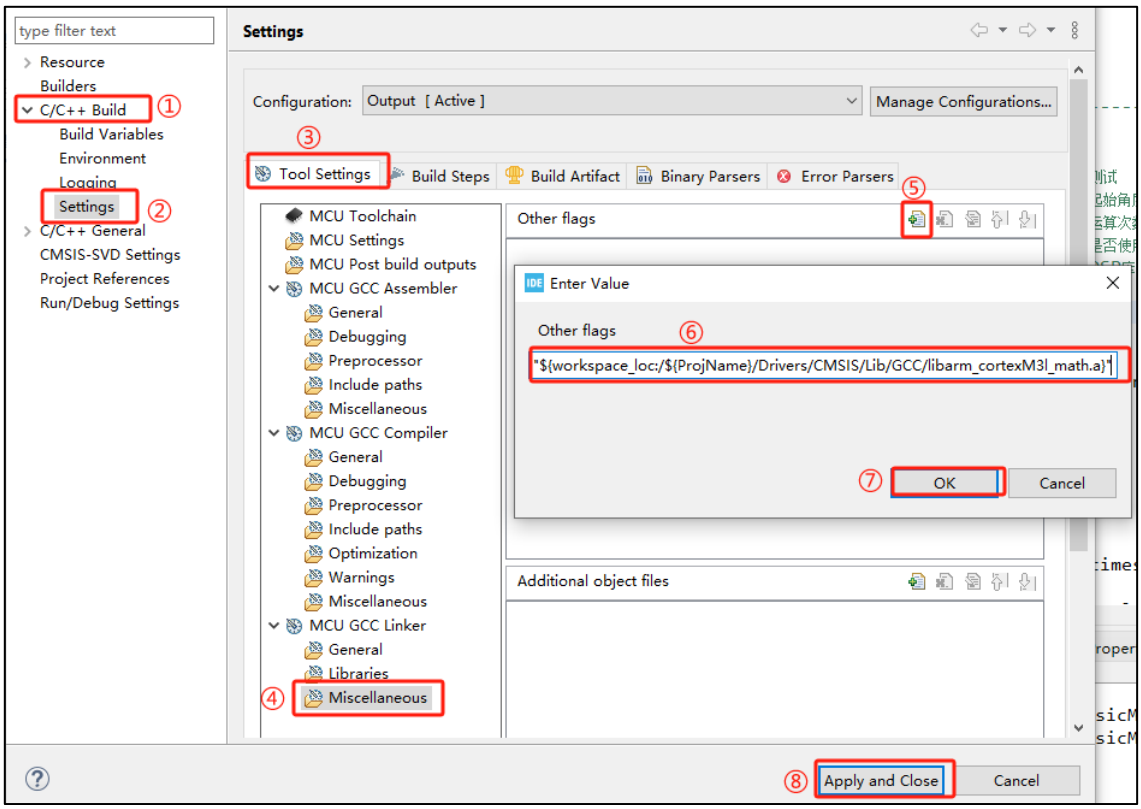


图 61.2.5.1 添加 Lib 库路径

Lib 库的路径比较长，自己手动输入容易出错，所以这里建议大家直接复制我们提供的路径："${workspace_loc:/${ProjName}/Drivers/CMSIS/Lib/GCC/libarm_cortexM3l_math.a}"

61.2.6 配置 TIM6

通过上述步骤，我们已成功将 DSP 库移植至工程中。接下来，我们需要进行工程配置。为了比较使用和不使用 DSP 库的计算速度时间差异，我们需要配置 TIM6 以进行计时。

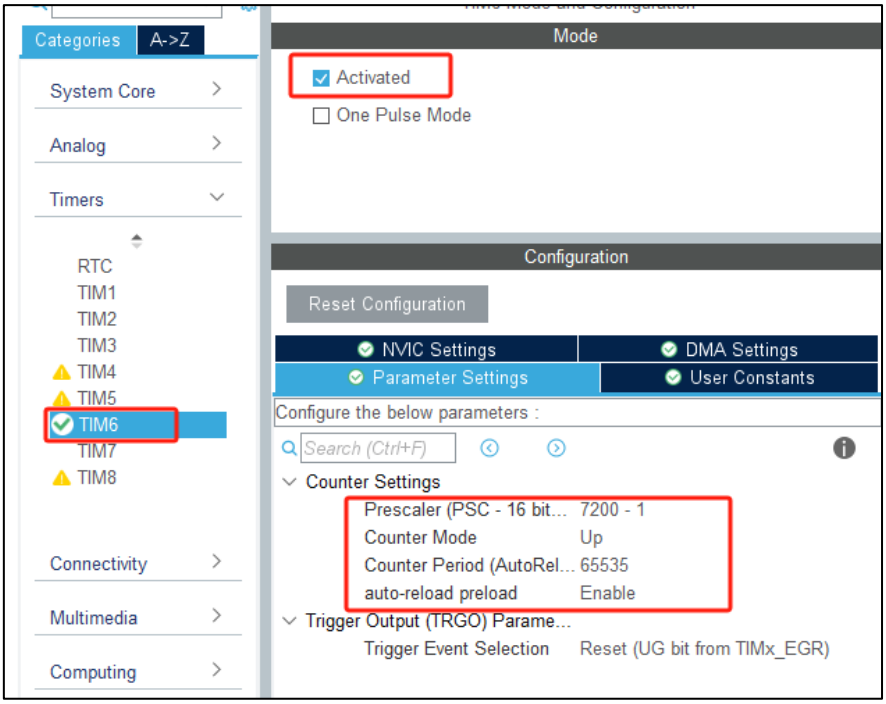


图 61.2.6.1 TIM6 配置

关于定时器的配置参数这里不在赘述，详情查阅定时器章节。这里我们配置定时器的计数频率为 10kHz。

61.2.7 配置 NVIC

为了防止时间溢出，所以我们要使能定时器更新中断，并配置中断优先级。如下图，勾选 TIM6 定时器全局中断，配置中断优先级分组为 2，抢占优先级和子优先级为 1：

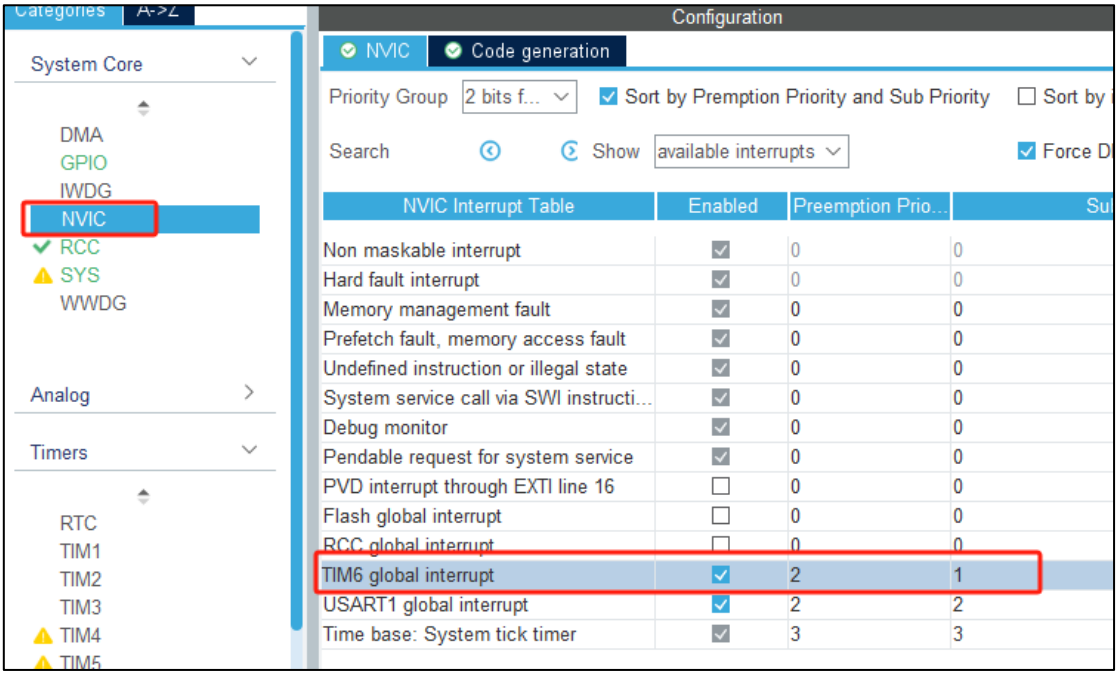


图 61.2.7.1 NVIC 配置

完成上述配置后，按下“Ctrl+S”保存配置并生成工程。注意，在 tim.c 文件中需要使能 TIM6 的更新中断并开启 TIM6。我们将不再赘述此步骤。接下来，开始添加用户驱动代码。

61.2.8 添加用户驱动代码

这部分两个实验仅 main.c 的内容有所区别，所以我们这里分开讲解。

61.2.8.1 实验 39_1_DSP_BasicMath

（1）main.c 文件代码

```
/* USER CODE BEGIN PTD */
#define DELTA    0.0001f                    /* 误差值 */
uint8_t g_timeout;
/* USER CODE END PTD */

/* USER CODE BEGIN PD */
/**
 * @brief       sin cos 测试
 * @param       angle : 起始角度
 * @param       times : 运算次数
 * @param       mode  : 是否使用 DSP 库
 *   @arg       0 , 不使用 DSP 库;
 *   @arg       1 , 使用 DSP 库;
 *
 * @retval      无
 */
```

```
uint8_t sin_cos_test(float angle, uint32_t times, uint8_t mode)
{
    float sinx, cosx;
    float result;
    uint32_t i = 0;

    if (mode == 0)
    {
        for (i = 0; i < times; i++)
        {
            cosx = cosf(angle);                  /* 不使用 DSP 优化的 sin cos 函数*/
            sinx = sinf(angle);
            result = sinx * sinx + cosx * cosx; /* 计算结果应该等于 1 */
            result = fabsf(result - 1.0f);       /* 对比与 1 的差值 */

            if (result > DELTA)return 0XFF;      /* 判断失败 */

            angle += 0.001f;                     /* 角度自增 */
        }
    }
    else
    {
        for (i = 0; i < times; i++)
        {

            cosx = arm_cos_f32(angle);           /* 使用 DSP 优化的 sin，cos 函数 */
            sinx = arm_sin_f32(angle);
            result = sinx * sinx + cosx * cosx; /* 计算结果应该等于 1 */
            result = fabsf(result - 1.0f);       /* 对比与 1 的差值 */

            if (result > DELTA)return 0XFF;      /* 判断失败 */

            angle += 0.001f;                     /* 角度自增 */
        }
    }

    return 0;                                    /* 任务完成 */
}
/* USER CODE END PD */

int main(void)
{
  /* USER CODE BEGIN 1 */
    float time;
    char buf[50];
    uint8_t res;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
```

```
  /* TIM6 初始化 */
  MX_TIM6_Init();
  /* USER CODE BEGIN 2 */
  lcd_init();                                             /* 初始化 LCD */
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "DSP BasicMath TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, " No DSP runtime:", RED);
  lcd_show_string(30, 150, 200, 16, 16, "Use DSP runtime:", RED);
  /* USER CODE END 2 */

  while (1)
  {
      /* 不使用 DSP 优化 */
      __HAL_TIM_SET_COUNTER(&htim6,0);      /* 重设 TIM6 定时器的计数器值 */
      g_timeout = 0;
      res = sin_cos_test(PI / 6, 10000, 0);
      time = __HAL_TIM_GET_COUNTER(&htim6) + (uint32_t)g_timeout * 65536;
      sprintf(buf, "%0.1fms\r\n", time / 10);

      if (res == 0)
      {
          lcd_show_string(30 + 16 * 8, 110, 100, 16, 16, buf, BLUE);
      }
      else
      {
          lcd_show_string(30 + 16 * 8, 110, 100, 16, 16, "error! ", BLUE);
      }

      /* 使用 DSP 优化 */
      __HAL_TIM_SET_COUNTER(&htim6,0);      /* 重设 TIM6 定时器的计数器值 */
      g_timeout = 0;
      res = sin_cos_test(PI / 6, 10000, 1);
      time = __HAL_TIM_GET_COUNTER(&htim6) + (uint32_t)g_timeout * 65536;
      sprintf(buf, "%0.1fms\r\n", time / 10);

      if (res == 0)
      {
          lcd_show_string(30 + 16 * 8, 150, 100, 16, 16, buf, BLUE);
      }
      else
      {
          lcd_show_string(30 + 16 * 8, 150, 100, 16, 16, "error! ", BLUE);
      }

      LED0_TOGGLE();
  }
}
```

这里包括 2 个函数：sin_cos_test 和 main 函数，sin_cos_test 函数用于根据给定参数，执行 $sin(x)^2+cos(x)^2=1$ 的计算。计算完后，计算结果同给定的误差值（DELTA）对比，如果不大于误差值，则认为计算成功，否则计算失败。该函数可以根据给定的模式参数(mode)来决定使用哪个基础数学函数执行运算，从而得出对比。

main 函数则比较简单，这里我们通过定时器 6 来统计 sin_cos_test 运行时间，从而得出对比数据。主循环里面，每次循环都会两次调用 sin_cos_test 函数，首先采用不使用 DSP 库方式计算，然后采用使用 DSP 库方式计算，并得出两次计算的时间，显示在 LCD 上面。

DSP 基础数学函数测试的程序设计就讲解到这里。

61.2.8.2 实验 39_2_DSP_FFT

（1）main.c 文件代码

```
/* USER CODE BEGIN PTD */
/* FFT 长度，默认是 1024 点 FFT
 * 可选范围为: 16, 64, 256, 1024.
 */
#define FFT_LENGTH      1024
float fft_inputbuf[FFT_LENGTH * 2];      /* FFT 输入数组 */
float fft_outputbuf[FFT_LENGTH];         /* FFT 输出数组 */

uint8_t g_timeout;
/* USER CODE END PTD */

int main(void)
{
  /* USER CODE BEGIN 1 */
    float time;
    char buf[50];
    arm_cfft_radix4_instance_f32 scfft;
    uint8_t key, t = 0;
    uint16_t i;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM6 初始化 */
  MX_TIM6_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                                    /* 初始化 LCD */
  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "DSP FFT TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY0:Run FFT", RED); /* 显示提示信息 */
  lcd_show_string(30, 140, 200, 16, 16, "FFT runtime:", RED); /* 显示提示信息 */
  /* 初始化 scfft 结构体，设定 FFT 相关参数 */
  arm_cfft_radix4_init_f32(&scfft, FFT_LENGTH, 0, 1);
  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY0_PRES)
      {
          for (i = 0; i < FFT_LENGTH; i++)  /* 生成信号序列 */
```

```
            {
                fft_inputbuf[2 * i] = 100 +
                                      10 * arm_sin_f32(2 * PI * i / FFT_LENGTH)
+
                                      30 * arm_sin_f32(2 * PI * i * 4 /
                                      FFT_LENGTH) +
                                      50 * arm_cos_f32(2 * PI * i * 8 /
                                      FFT_LENGTH);     /* 生成输入信号实部 */
                fft_inputbuf[2 * i + 1] = 0; /* 虚部全部为 0 */
            }

            TIM6->CNT = 0;                    /* 重设 TIM6 定时器的计数器值 */
            g_timeout = 0;

            arm_cfft_radix4_f32(&scfft, fft_inputbuf);       /* FFT 计算（基 4） */

            time = TIM6->CNT + (uint32_t)g_timeout * 65536; /* 计算所用时间 */
            sprintf((char *)buf, "%0.3fms\r\n", time / 10);
            lcd_show_string(30 + 12 * 8, 140, 100, 16, 16, buf, BLUE);

            arm_cmplx_mag_f32(fft_inputbuf, fft_outputbuf, FFT_LENGTH);

            printf("\r\n%d point FFT runtime:%0.3fms\r\n", FFT_LENGTH, time /
            1000);
            printf("FFT Result:\r\n");

            for (i = 0; i < FFT_LENGTH; i++)
            {
                printf("fft_outputbuf[%d]:%f\r\n", i, fft_outputbuf[i]);
            }
        }
        else
        {
            delay_ms(10);
        }

        t++;

        if ((t % 20) == 0)
        {
            LED0_TOGGLE();
        }
    }
}
```

通过三个函数：arm_cfft_radix4_init_f32、arm_cfft_radix4_f32 和 arm_cmplx_mag_f32 来执行 FFT 变换并取模值。每当按下 KEY0 就会重新生成一个输入信号序列，并执行一次 FFT 计算，将 arm_cfft_radix4_f32 所用时间统计出来，显示在 LCD 屏幕上面，同时将取模后的模值通过串口打印出来。

61.3 下载验证

将实验 39_1_DSP_BasicMath 测试实验的程序下载到开发板后，可以在屏幕看到两种实现方式的速度差别，使用 DSP 库的基础数学函数计算所用时间比不使用 DSP 库的短，使用 STM32F103 的 DSP 库，速度上面比传统的实现方式提升了约 23%。

对于实验 39_2_DSP_FFT 测试实验，下载后，屏幕显示提示信息，然后我们按下 KEY0 就可以看到 FFT 运算所耗时间。

第六十二章 手写识别实验

本章将介绍正点原子提供的手写识别库的使用，通过使用手写识别库可以实现一些简单的数字字母手写识别的应用。通过本章的学习，读者将学习到正点原子手写识别库的使用。

本章分为如下几个小节：

62.1 硬件设计
62.2 程序设计
62.3 下载验证

62.1 硬件设计

1. 例程功能

开机的时候先初始化手写识别器，然后检测字库，之后进入等待输入状态。此时，我们在手写区写数字/字符，在每次写入结束后，自动进入识别状态，进行识别，然后将识别结果输出在 LCD 模块上，同时打印到串口。通过按 KEY0 可以进行模式切换（4 种模式都可以测试），通过按 KEY_UP，可以进入触摸屏校准（仅电阻屏需要校准，如果发现触摸屏不准，请执行此操作）。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
KEY0 - PE4
KEY1 - PE3
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）NOR FLASH：通过 SPI2 连接

3. 原理图

本章实验使用的手写识别库为软件库，因此没有对应的连接原理图。

62.2 程序设计

62.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 40_ATKNCR。

62.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 36_HZ 工程，并将工程文件名和.ioc 文件名均修改为 40_ATKNCR，以进行下一步的实验操作。如下图所示：

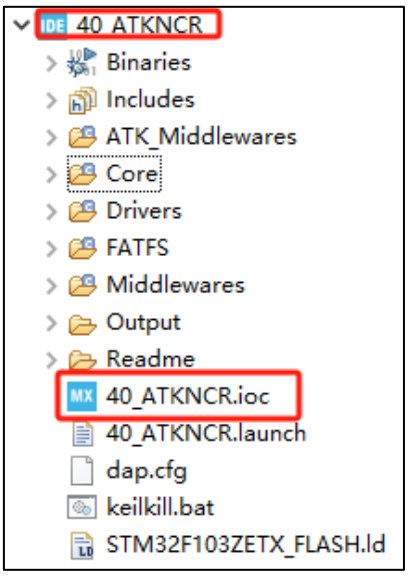


图 62.2.2.1 ATKNCR 工程

本章实验使用的手写识别库为软件库，因此不需在 STM32CubeIDE 上进行其他配置，直接添加相应的驱动代码即可。

62.2.3 添加用户驱动代码

在 ATK_Middlewares 文件夹中，新建 ATKNCR 文件夹，ATKNCR 驱动源码包括 ATKNCR_M_V2.0.a、atk_ncr.c 和 atk_ncr.h。

ATKNCR_M_V2.0.a 为手写识别库，我们的第一步，就是在我们本工程中添加手写识别库的路径，否则 STM32CubeIDE 编译将有错误。添加过程如下：

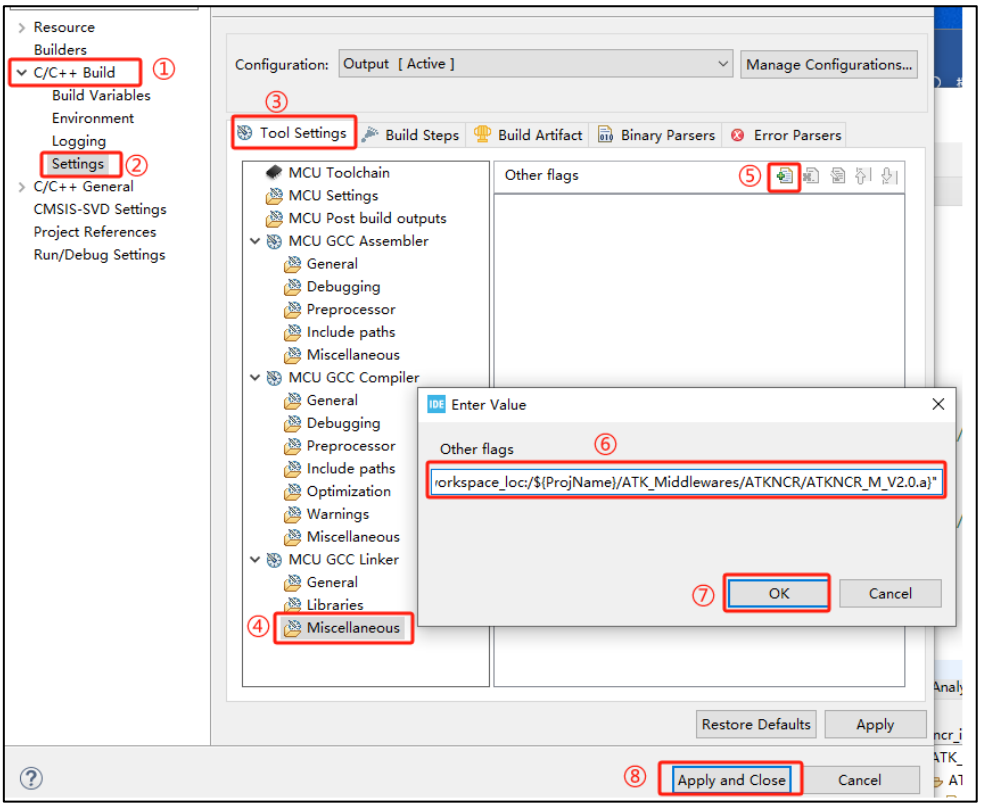


图 62.2.3.1 添加手写识别库

Lib 库的路径比较长，自己手动输入容易出错，所以这里建议大家直接复制我们提供的

路径："${workspace_loc:/${ProjName}/Drivers/CMSIS/Lib/GCC/ ATKNCR_M_V2.0.a}"

（1）atk_nrc.c/h 文件代码

接着看一下 atk_nrc.h 头文件中比较重要部分，其代码如下：

```
/* 输入轨迹坐标类型 */
__packed typedef struct _atk_ncr_point
{
    short x;          /* x 轴坐标 */
    short y;          /* y 轴坐标 */
}atk_ncr_point;

/* 外部调用函数
 * 初始化识别器
 * 返回值 : 0, 初始化成功
 *          1, 初始化失败
 */
unsigned char alientek_ncr_init(void);

/* 停止识别器 */
void alientek_ncr_stop(void);

/* 识别器识别
 * track    : 输入点阵集合
 * potnum   : 输入点阵的点数,就是 track 的大小
 * charnum  : 期望输出的结果数,就是你希望输出多少个匹配结果
 * mode     : 识别模式
 *            1,仅识别数字
 *            2,仅识别大写字母
 *            3,仅识别小写字母
 *            4,混合识别(全部识别)
 *
 * result   : 结果缓存区(至少为:charnum+1 个字节)
 */
void alientek_ncr(atk_ncr_point * track, int potnum, Int charnum,
                                                    unsigned char
mode,char*result);
```

在上面的代码中，我们定义了一些外部接口函数以及轨迹结构体等。

alientek_ncr_init 函数用与初始化识别器，该函数在.lib 文件实现，在识别开始之前，我们应该调用该函数。

alientek_ncr_stop 函数用于停止识别器，在识别完成之后（不需要再识别），我们调用该函数，如果一直处于识别状态，则没必要调用。该函数也是在.lib 文件实现。

alientek_ncr 函数就是识别函数了。它有 5 个参数，第一个参数 track，为输入轨迹点的坐标集（最好 200 以内）；第二个参数 potnum，为坐标集点坐标的个数；第三个参数 charnum，为期望输出的结果数，即希望输出多少个匹配结果，识别器按匹配程度排序输出（最佳匹配排第一）；第四个参数 mode，该函数用于设置模式，识别器总共支持 4 种模式：

1，仅识别数字

2，进识别大写字母

3，仅识别小写字母

4，混合识别(全部识别)

最后一个参数是 result，用来输出结果，注意这个结果是 ASCII 码格式的。

下面我们直接来介绍 atk_ncr.c 中内存管理部分，其代码如下：

```
/**
 * @brief       内存设置函数
 * @param           *p     : 内存首地址
```

```
 * @param           c     : 要设置的值
 * @param           len   : 需要设置的内存大小(字节为单位)
 * @retval          无
 */
void alientek_ncr_memset(char *p, char c, unsigned long len)
{
    my_mem_set((uint8_t*)p, (uint8_t)c, (uint32_t)len);
}

/**
 * @brief       分配内存
 * @param       size : 要分配的内存大小(字节)
 * @retval      分配到的内存首地址.
 */
void *alientek_ncr_malloc(unsigned int size)
{
    return mymalloc(SRAMIN,size);
}

/**
 * @brief       释放内存
 * @param       ptr  : 内存首地址
 * @retval      无
 */
void alientek_ncr_free(void *ptr)
{
    myfree(SRAMIN,ptr);
}
```

alientek_ncr_memset、alientek_ncr_free 和 alientek_ncr_free 三个函数的实现主要调用 malloc 中的函数实现。这里就不多讲，忘记这些函数的实现，可以回顾一下内存管理实验章节。

（2）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t i = 0;
    uint8_t tcnt = 0;
    char sbuf[10];
    uint8_t key;
    uint16_t pcnt = 0;
    uint8_t mode = 4;                   /* 默认是混合模式 */
    uint16_t lastpos[2];                /* 最后一次的数据 */
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();
```

```
  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* SDIO 初始化 */
  MX_SDIO_SD_Init();

  /* FATFS 初始化 */
  MX_FATFS_Init();

  /* SPI2 初始化 */
  MX_SPI2_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                 /* 初始化 LCD */
  norflash_init();                            /* 初始化 NORFLASH */
  tp_dev.init();                              /* 初始化触摸屏 */
  my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */
  exfuns_init();                              /* 为 fatfs 相关变量申请内存 */

  f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
  f_mount(fs[1], "1:", 1);                    /* 挂载 SPI_FLASH 卡 */
  alientek_ncr_init();                        /* 初始化手写识别 */
  while (fonts_init())                        /* 检查字库 */
  {
      lcd_show_string(60, 50, 200, 16, 16, "Font Error!", RED);
      delay_ms(200);
      lcd_fill(60, 50, 240, 66, WHITE); /* 清除显示 */
      delay_ms(200);
  }

RESTART:
  text_show_string(60, 10, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
  text_show_string(60, 30, 200, 16, "手写识别实验", 16, 0, RED);
  text_show_string(60, 50, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
  text_show_string(60, 70, 200, 16, "KEY0:MODE KEY1:Adjust", 16, 0, RED);
  text_show_string(60, 90, 200, 16, "识别结果:", 16, 0, RED);
  lcd_draw_rectangle(19, 114, lcddev.width - 20, lcddev.height - 5, RED);

  text_show_string(96, 207, 200, 16, "手写区", 16, 0, BLUE);
  tcnt = 100;

  /* USER CODE END 2 */

  while (1)
  {
      key = key_scan(0);

      if (key == KEY1_PRES && (tp_dev.touchtype & 0X80) == 0)
      {
          tp_adjust();        /* 屏幕校准 */
          lcd_clear(WHITE);
          goto RESTART;       /* 重新加载界面 */
      }

      if (key == KEY0_PRES)
      {
          lcd_fill(20, 115, 219, 314, WHITE); /* 清除当前显示 */
          mode++;

          if (mode > 4)mode = 1;

          switch (mode)
          {
```

```
            case 1:
                text_show_string(80, 207, 200, 16, "仅识别数字", 16, 0, BLUE);
                break;

            case 2:
                text_show_string(64, 207, 200, 16, "仅识别大写字母", 16, 0,
                BLUE);
                break;

            case 3:
                text_show_string(64, 207, 200, 16, "仅识别小写字母", 16, 0,
                BLUE);
                break;

            case 4:
                text_show_string(88, 207, 200, 16, "全部识别", 16, 0, BLUE);
                break;
        }

        tcnt = 100;
    }

    tp_dev.scan(0); /* 扫描 */

    if (tp_dev.sta & TP_PRES_DOWN)  /* 有按键被按下 */
    {
        delay_ms(1);     /* 必要的延时，否则老认为有按键按下 */
        tcnt = 0;        /* 松开时的计数器清空 */

        if ((tp_dev.x[0] < (lcddev.width - 20 - 2) && tp_dev.x[0] >= (20 +
2)) && (tp_dev.y[0] < (lcddev.height - 5 - 2) && tp_dev.y[0] >= (115 + 2)))
        {
            if (lastpos[0] == 0XFFFF)
            {
                lastpos[0] = tp_dev.x[0];
                lastpos[1] = tp_dev.y[0];
            }

            lcd_draw_bline(lastpos[0], lastpos[1], tp_dev.x[0], tp_dev.y[0],
            2, BLUE);  /* 画线 */
            lastpos[0] = tp_dev.x[0];
            lastpos[1] = tp_dev.y[0];

            if (pcnt < 200) /* 总点数少于 200 */
            {
                if (pcnt)
                {
                    if ((ncr_input_buf[pcnt - 1].y != tp_dev.y[0]) &&
                    (ncr_input_buf[pcnt - 1].x != tp_dev.x[0])) /* x,y 不相等 */
                    {
                        ncr_input_buf[pcnt].x = tp_dev.x[0];
                        ncr_input_buf[pcnt].y = tp_dev.y[0];
                        pcnt++;
                    }
                }
                else
                {
                    ncr_input_buf[pcnt].x = tp_dev.x[0];
                    ncr_input_buf[pcnt].y = tp_dev.y[0];
                    pcnt++;
                }
            }
        }
    }
    else     /* 按键松开了 */
```

```
        {
            lastpos[0] = 0XFFFF;
            tcnt++;
            delay_ms(10);
            /* 延时识别 */
            i++;

            if (tcnt == 40)
            {
                if (pcnt)    /* 有有效的输入 */
                {
                    printf("总点数:%d\r\n", pcnt);
                    alientek_ncr(ncr_input_buf, pcnt, 6, mode, sbuf);
                    printf("识别结果:%s\r\n", sbuf);
                    pcnt = 0;
                    lcd_show_string(60 + 72, 90, 200, 16, 16, sbuf, BLUE);
                }

                lcd_fill(20, 115, lcddev.width - 20 - 1, lcddev.height - 5 - 1,
                WHITE);
            }
        }

        if (i == 30)
        {
            i = 0;
            LED0_TOGGLE();
        }
    }
}
```

main 函数代码实现手写识别功能的步骤跟前面所说的一致的。其中使用到了 lcd_draw_bline 函数，该函数是用来画粗线的，函数的实现也是通过调用 lcd_fill_circle 实现，这里就不做展开讲解该函数了。在获取触点数据需要注意的是：这里我们采用的都是不重复的点阵（即相邻的坐标不相等）。这样可以避免重复数据，而重复的点阵数对于识别是没有帮助的。

62.3 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 62.3.1 所示：

图 62.3.1 手写识别界面图

此时，我们在手写区写数字/字母，即可得到识别结果，如图 62.3.2 所示：

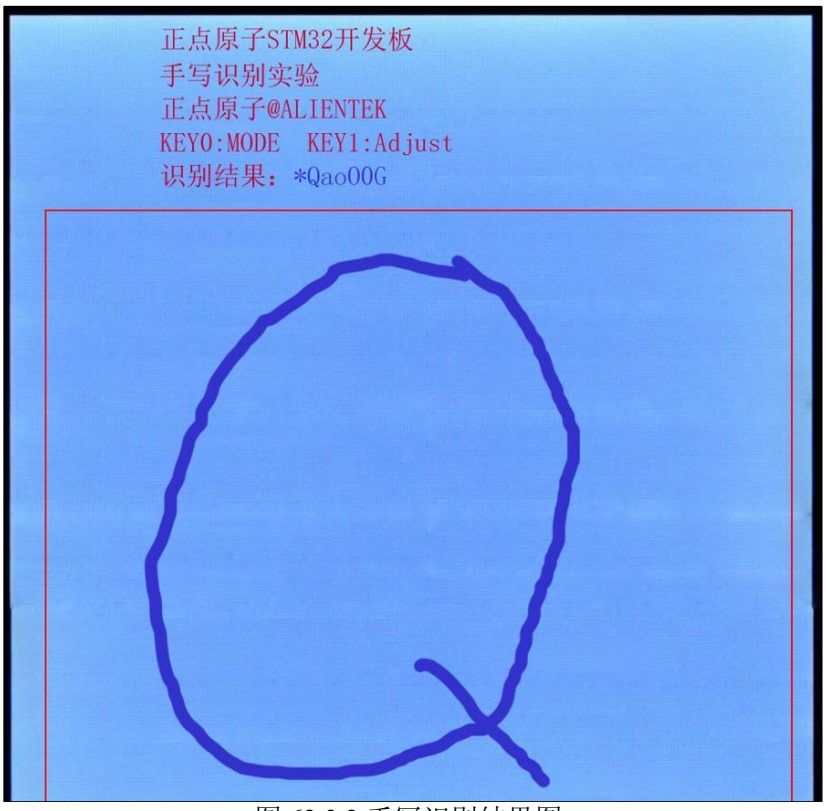


图 62.3.2 手写识别结果图

按下 KEY0 可以切换识别模式，同时在识别区提示当前模式。按下 KEY1 可以对屏幕进行校准（仅限电阻屏，电容屏无需校准）。每次识别结束，会在串口打印本次识别的输入点数和识别结果，大家可以通过串口助手查看。

第六十三章 T9 拼音输入法实验

本章，我们将介绍如何在 STM32 板子上实现一个简单的 T9 中文拼音输入法。本章分为如下几个小节：

63.1 硬件设计
63.2 程序设计
63.3 下载验证

63.1 硬件设计

1. 例程功能

开机的时候先检测字库，然后显示提示信息和绘制拼音输入表，之后进入等待输入状态。此时用户可以通过屏幕上的拼音输入表输入拼音数字串（通过 DEL 可以实现退格），然后程序自动检测与之对应的拼音和汉字，并显示在屏幕上（同时输出到串口）。如果有多个匹配的拼音，则通过 KEY_UP 和 KEY1 进行选择。按键 KEY0 用于清除一次输入，按键 KEY2 用于触摸屏校准。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）独立按键
KEY0 - PE4
KEY1 - PE3
WK_UP - PA0
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
5）NOR FLASH，通过 SPI2 驱动，我们需要里面的汉字库
6）触摸屏（电阻式\电容式）

3. 原理图

本章实验使用的 T9 拼音输入法库为软件库，因此没有对应的连接原理图。

63.2 程序设计

63.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 41_T9INPUT。

63.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 36_HZ 工程，并将工程文件名和.ioc 文件名均修改为 41_T9INPUT，以进行下一步的实验操作。如下图所示：

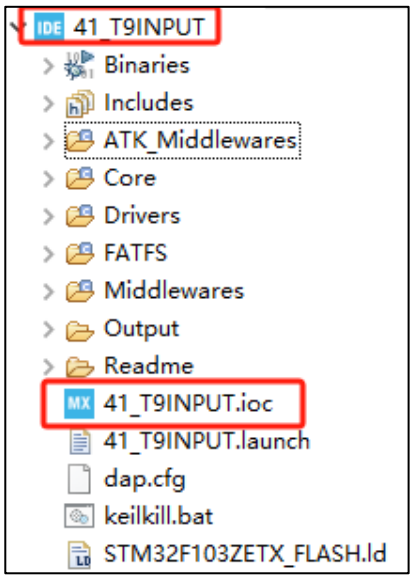


图 63.2.2.1 T9INPUT 工程

本章实验使用的 T9 拼音输入法库为软件库，因此不需在 STM32CubeIDE 上进行其他配置，直接添加相应的驱动代码即可。

63.2.3 添加用户驱动代码

在 ATK_Middlewares 文件夹中，新建 T9INPUT 文件夹，并将正点原子提供的 T9 拼音输入法库的三个文件移植到 T9INPUT 文件夹中。这三个文件分别为 pyinput.c、pyinput.h 和 pymb.h。

（1）pymb.h 文件代码

首先我们先介绍一下定义在 pymb.h 中的拼音索引。先介绍一下汉字排列表，该表将汉字拼音所有可能的组合都列出来了，如下所示：

```
/* 汉字排列表 */
const uint8_t PY_mb_space []={""};
const uint8_t PY_mb_a     []={"啊阿腌吖锕屙嘎錒呵腌"};
const uint8_t PY_mb_ai    []={"爱埃挨哎唉哀皑癌蔼矮艾碍隘捱嗳嗌嫒瑷暧砹锿霭"};
……此处省略 N 多个组合
const uint8_t PY_mb_zu    []={"足组卒族租祖诅阻俎菹镞"};
const uint8_t PY_mb_zuan  []={"钻攥纂缵躜"};
const uint8_t PY_mb_zui   []={"最罪嘴醉蕞觜"};
const uint8_t PY_mb_zun   []={"尊遵樽鳟撙"};
const uint8_t PY_mb_zuo   []={"左佐做作坐座昨撮唑柞阼琢嘬怍胙祚砟酢"};
```

这里我们只列出了部分组合，我们将这些组合称之为码表，然后将这些码表和其对应的数字串对应起来，组成一个拼音索引表，如下所示：

```
/* 拼音索引表 */
const py_index py_index3[]=
{
{"" ,"",(uint8_t*)PY_mb_space},
{"2","a",(uint8_t*)PY_mb_a},
{"3","e",(uint8_t*)PY_mb_e},
{"6","o",(uint8_t*)PY_mb_o},
{"24","ai",(uint8_t*)PY_mb_ai},
……此处省略 N 多个组合
{"94664","zhong",(uint8_t*)PY_mb_zhong},
{"94824","zhuai",(uint8_t*)PY_mb_zhuai},
{"94826","zhuan",(uint8_t*)PY_mb_zhuan},
{"248264","chuang",(uint8_t*)PY_mb_chuang},
```

```
{"748264","shuang",(uint8_t*)PY_mb_shuang},
{"948264","zhuang",(uint8_t*)PY_mb_zhuang},
};
```

其中 py_index 是一个结构体，定义如下：

```
/* 拼音码表与拼音的对应表 */
typedef struct
{
  uint8_t *py_input;     /* 输入的字符串 */
  uint8_t *py;           /* 对应的拼音 */
  uint8_t *pymb;         /* 码表 */
}py_index;
```

其中 py_input，即与拼音对应的数字串，比如“94824”。py，即与 py_input 数字串对应的拼音，如果 py_input = “94824”，那么 py 就是“zhuai”。最后 pymb 就是我们前面说到的码表。注意，一个数字串可以对应多个拼音，也可以对应多个码表。

（2）pyinput.c 文件代码

下面介绍一下 pyinput.c 中比较核心的函数 get_matched_pymb，代码如下：

```
/**
 * @brief       获取匹配的拼音码表
 * @param       strin      : 输入的字符串,形如:"726"
 * @param       matchlist  : 输出的匹配表
 * @retval      匹配状态
 *              [7]   , 0,表示完全匹配; 1，表示部分匹配（仅在没有完全匹配的时候才会出现）
 *              [6:0], 完全匹配的时候，表示完全匹配的拼音个数
 *                     部分匹配的时候，表示有效匹配的位数
 */
uint8_t get_matched_pymb(uint8_t *strin, py_index **matchlist)
{
    py_index *bestmatch = 0;            /* 最佳匹配 */
    uint16_t pyindex_len = 0;
    uint16_t i = 0;
    uint8_t temp, mcnt = 0, bmcnt = 0;

    bestmatch = (py_index *)&py_index3[0];  /* 默认为 a 的匹配 */
    pyindex_len = sizeof(py_index3) / sizeof(py_index3[0]); /*得到 py 索引表的大小
*/

    for (i = 0; i < pyindex_len; i++)
    {
        temp = str_match(strin, (uint8_t *)py_index3[i].py_input);

        if (temp)
        {
            if (temp == 0XFF)
            {
                matchlist[mcnt++] = (py_index *)&py_index3[i];
            }
            else if (temp > bmcnt)  /* 找最佳匹配 */
            {
                bmcnt = temp;
                bestmatch = (py_index *)&py_index3[i];  /* 最好的匹配 */
            }
        }
    }

    if (mcnt == 0 && bmcnt)      /* 没有完全匹配的结果,但是有部分匹配的结果 */
    {
        matchlist[0] = bestmatch;
        mcnt = bmcnt | 0X80;     /* 返回部分匹配的有效位数 */
    }
    return mcnt;     /* 返回匹配的个数 */
```

```
}
```

该函数实现的是将用户输入拼音数字串同拼音索引表里面的各个项对比，找出匹配结果，并将完全匹配的项目存放在 matchlist 里面，同时记录匹配数。对于那些没有完全匹配的输入串，则查找与其最佳匹配的项目，并将匹配的长度返回。

（3）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t i = 0;
    uint8_t result_num;
    uint8_t cur_index;
    uint8_t key;
    uint8_t temp;
    uint8_t inputstr[7];                          /* 最大输入 6 个字符+结束符 */
    uint8_t inputlen;                             /* 输入长度 */
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* SDIO 初始化 */
  MX_SDIO_SD_Init();

  /* FATFS 初始化 */
  MX_FATFS_Init();

  /* SPI2 初始化 */
  MX_SPI2_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                     /* 初始化 LCD */
  norflash_init();                                /* 初始化 NORFLASH */
  tp_dev.init();                                  /* 初始化触摸屏 */
  my_mem_init(SRAMIN);                            /* 初始化内部 SRAM 内存池 */
  exfuns_init();                                  /* 为 fatfs 相关变量申请内存 */

  f_mount(fs[0], "0:", 1);                        /* 挂载 SD 卡 */
  f_mount(fs[1], "1:", 1);                        /* 挂载 SPI_FLASH 卡 */

RESTART:
  while (fonts_init())                            /* 检查字库 */
  {
      lcd_show_string(60, 50, 200, 16, 16, "Font Error!", RED);
```

```
    delay_ms(200);
    lcd_fill(60, 50, 240, 66, WHITE);    /* 清除显示 */
    delay_ms(200);
}
text_show_string(30, 5, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30, 25, 200, 16, "拼音输入法实验", 16, 0, RED);
text_show_string(30, 45, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
text_show_string(30, 65, 200, 16, "KEY_UP:校准", 16, 0, RED);
text_show_string(30, 85, 200, 16, "KEY0:翻页  KEY1:清除", 16, 0, RED);
text_show_string(30, 105, 200, 16, "输入:         匹配:  ", 16, 0, RED);
text_show_string(30, 125, 200, 16, "拼音:         当前:  ", 16, 0, RED);
text_show_string(30, 145, 210, 32, "结果:", 16, 0, RED);

/* 根据 LCD 分辨率设置按键大小 */
if (lcddev.id == 0X5310)
{
    kbdxsize = 86;
    kbdysize = 43;
}
else if (lcddev.id == 0X5510)
{
    kbdxsize = 140;
    kbdysize = 70;
}
else
{
    kbdxsize = 60;
    kbdysize = 40;
}

py_load_ui(30, 195);
my_mem_set(inputstr, 0, 7);  /* 全部清零 */
inputlen = 0;                /* 输入长度为 0 */
result_num = 0;              /* 总匹配数清零 */
cur_index = 0;

/* USER CODE END 2 */

while (1)
{
    i++;
    delay_ms(10);
    key = py_get_keynum(30, 195);

    if (key)
    {
        if (key == 1)                          /* 删除 */
        {
            if (inputlen)inputlen--;

            inputstr[inputlen] = '\0';         /* 添加结束符 */
        }
        else
        {
            inputstr[inputlen] = key + '0';    /* 输入字符 */

            if (inputlen < 7)inputlen++;
        }

        if (inputstr[0] != NULL)
        {
            temp = t9.getpymb(inputstr);       /* 得到匹配的结果数 */

            if (temp)                          /* 有部分匹配/完全匹配的结果 */
```

```
            {
                result_num = temp & 0X7F;          /* 总匹配结果 */
                cur_index = 1;                     /* 当前为第一个索引 */

                if (temp & 0X80)                   /* 是部分匹配 */
                {
                    inputlen = temp & 0X7F;        /* 有效匹配位数 */
                    inputstr[inputlen] = '\0';     /* 不匹配的位数去掉 */

                    if (inputlen > 1)
                    {
                        temp = t9.getpymb(inputstr); /* 重新获取完全匹配字符数*/
                        result_num = (temp & 0X80)? 0 : (temp & 0X7F);
                    }
                }
            }
            else    /* 没有任何匹配 */
            {
                inputlen--;
                inputstr[inputlen] = '\0';
            }
        }
        else
        {
            cur_index = 0;
            result_num = 0;
        }

        lcd_fill(30 + 40, 105, 30 + 40 + 48, 110 + 16, WHITE);
        lcd_show_num(30 + 144, 105, result_num, 1, 16, BLUE);
        text_show_string(30 + 40, 105, 200, 16, (char *)inputstr, 16, 0,
        BLUE); /* 显示有效的数字串 */
        py_show_result(cur_index);              /* 显示第 cur_index 的匹配结果 */
    }

    key = key_scan(0);

    if (key == WKUP_PRES && tp_dev.touchtype == 0)  /* KEYUP 按下,且是电阻屏 */
    {
        tp_dev.adjust();
        lcd_clear(WHITE);
        goto RESTART;
    }

    switch (key)
    {
        case KEY0_PRES:       /* 下翻 */
            if (result_num)   /* 存在匹配的结果 */
            {
                if (cur_index < result_num)cur_index++;
                else cur_index = 1;

                py_show_result(cur_index);  /* 显示第 cur_index 的匹配结果 */
            }
            break;

        case KEY1_PRES: /* 清除输入 */
            lcd_fill(30 + 40, 145, lcddev.width - 1, 145 + 48, WHITE);
            goto RESTART;
    }


    if (i == 30)
    {
```

```
            i = 0;
            LED0_TOGGLE();
        }
    }
}
```

从上面的代码中可以看出，本实验实验的应用中，通过触摸屏获取拼音的键值，然后调用函数 t9. getpymb()进行码表匹配，有匹配结果时，则在 LCD 上显示匹配结果，若有多个匹配结果，则可通过 KEY0 按键进行翻页，通过也可通过 KEY_UP 按键清除输入。

63.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关实验信息，通过也显示了 T9 拼音的输入键盘，此时便可通过键盘输入拼音，随即便可在 LCD 上看到输入拼音对应的汉字匹配结果，若有多个匹配结果，例如输入“64”，则会匹配到拼音“mi”和“ni”，此时按下 KEY0 按键，可对多个匹配结果进行翻页，若按下 KEY_UP 按键则会清除所有输入。

第六十四章 串口 IAP 实验

本章将介绍在 STM32F103 上使用串口进行 IAP，以实现简单的 IAP 功能。通过本章的学习，读者将学习到 IAP 的使用。

本章分为如下几个小节：

64.1 硬件设计
64.2 程序设计
64.3 下载验证

64.1 硬件设计

1. 例程功能

本章实验（Bootloader 部分）功能简介：开机的时候先显示提示信息，然后等待串口输入接收 APP 程序（无校验，一次性接收），在串口接收到 APP 程序之后，即可执行 IAP。如果是 SRAM APP，通过按下 KEY0 即可执行这个收到的 SRAM APP 程序。如果是 FLASH APP，则需要先按下 KEY_UP 按键，将串口接收到的 APP 程序存放到 STM32F1 的 FLASH，之后再按 KEY1 即可以执行这个 FLASH APP 程序。LED0 用于指示程序运行状态。

2. 硬件资源

1）LED 灯
LED0 - PB5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）独立按键
KEY0 - PE4
KEY1 - PE3
WK_UP - PA0

3. 原理图

本章实验使用的 IAP 为软件算法，因此没有对应的连接原理图。

64.2 程序设计

64.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 42_1_IAP_Bootloader。

本实验，我们总共需要 3 个程序（1 个 Bootloader，2 个 APP）：

1、FLASH IAP Bootloader，起始地址为 0X08000000，设置为我们用于升级的跳转的程序，我们将用串口 1 来作数据接收程序，通过按键功能手动跳转到指定 APP。

2、FLASH APP，仅使用 STM32 内部 FLASH，我们使用 Release 编译优化，生成的 bin 大小为 46KB 左右。本程序使用：42_2_RTC_FLASH_APP 实验，作为 FLASH APP 程序（起始地址为 0X08010000）

3、SRAM APP，使用 STM32 内部 SRAM，我们使用 Release 编译优化，生成的 bin 大小为 54KB。本程序使用：42_3_TOUCH_SRAM_APP 实验，作为 SRAM APP 程序（起始地址为 0X20002000）。

本章关于 APP 程序的生成和修改比较简单，我们就不细说，请大家结合光盘源码对应板

子的开发指南相应章节的介绍，自行理解。本章程序解析小节仅针对 Bootloader 程序。

64.2.2 创建和配置工程

由于本章实验要用到 STM32 内部的 FLASH，所以在 STM32CubeIDE 中，通过复制 31_FLASH_EEPROM 工程，并将工程文件名和.ioc 文件名均修改为 42_1_IAP_Bootloader，以进行下一步的实验操作。如下图所示：

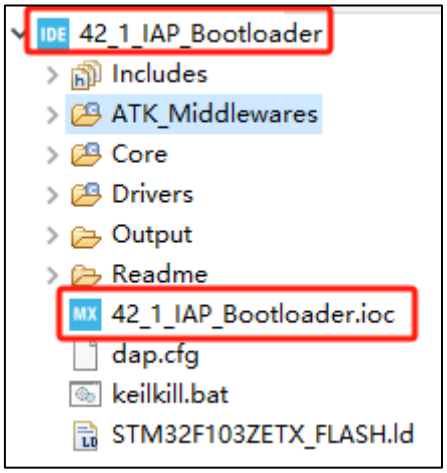


图 64.2.2.1 IAP_Bootloader 工程

本章实验使用的 IAP 为软件算法，因此不需在 STM32CubeIDE 上进行其他配置，直接添加相应的驱动代码即可。

64.2.3 添加用户驱动代码

在 ATK_Middlewares 文件夹中，新建 IAP 文件夹，并将正点原子提供的 IAP 的软件算法文件移植到 IAP 文件夹中。这两个文件分别为 iap.h 和 iap.h。如下图所示：

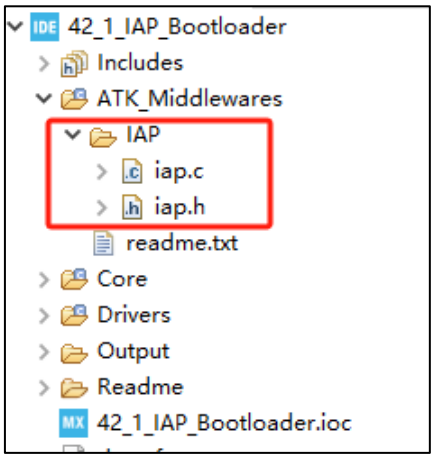


图 64.2.3.1 添加用户文件

（1）iap.h 文件代码

```
#ifndef __IAP_H
#define __IAP_H

#include "../../SYSTEM/sys/sys.h"

typedef void (*iapfun)(void);                    /* 定义一个函数类型的参数 */
```

```
#define FLASH_APP1_ADDR         0x08010000      /* 第一个应用程序起始地址(存放在内
                                                   部 FLASH) * 保留
                                                   0X08000000~0X08008FFF(36KB
)
                                                    的空间为 Bootloader 使用
                                                 */
void iap_load_app(uint32_t appxaddr);           /* 跳转到 APP 程序执行 */
/* 在指定地址开始,写入 bin */
void iap_write_appbin(uint32_t appxaddr,uint8_t *appbuf,uint32_t applen);

#endif
```

这里我们主要看下我们宏定义 FLASH_APP1_ADDR，它指向的是 APP 存放在 FLASH 的地址，即 APP 存放在地址 0x08010000，后续也是根据该地址跳转到相应 APP。

（2）iap.c 文件代码

```
iapfun jump2app;
uint16_t g_iapbuf[1024];        /* 2K 字节缓存 */

/**
 * @brief       IAP 写入 APP BIN
 * @param       appxaddr : 应用程序的起始地址
 * @param       appbuf   : 应用程序 CODE
 * @param       appsize  : 应用程序大小(字节)
 * @retval      无
 */
void iap_write_appbin(uint32_t appxaddr, uint8_t *appbuf, uint32_t appsize)
{
    uint16_t t;
    uint16_t i = 0;
    uint16_t temp;
    uint32_t fwaddr = appxaddr; /* 当前写入的地址 */
    uint8_t *dfu = appbuf;

    for (t = 0; t < appsize; t += 2)
    {
        temp = (uint16_t)dfu[1] << 8;
        temp |= (uint16_t)dfu[0];
        dfu += 2;                   /* 偏移 2 个字节 */
        g_iapbuf[i++] = temp;

        if (i == 1024)
        {
            i = 0;
            stmflash_write(fwaddr, g_iapbuf, 1024);
            fwaddr += 2048;     /* 偏移 2048  16 = 2 * 8  所以要乘以 2 */
        }
    }

    if (i)
    {
        stmflash_write(fwaddr, g_iapbuf, i);  /* 将最后的一些内容字节写进去 */
    }
}

/**
 * @brief       跳转到应用程序段(执行 APP)
 * @param       appxaddr : 应用程序的起始地址

 * @retval      无
 */
void iap_load_app(uint32_t appxaddr)
{
```

```
    /* 检查栈顶地址是否合法.可以放在内部 SRAM 共 64KB(0x20000000) */
    if (((*(volatile  uint32_t *)appxaddr) & 0x2FFE0000) == 0x20000000)
    {
        /* 用户代码区第二个字为程序开始地址(复位地址) */
        jump2app = (iapfun) * (volatile uint32_t *)(appxaddr + 4);

        /* 初始化 APP 堆栈指针(用户代码区的第一个字用于存放栈顶地址) */
        sys_msr_msp(*(volatile uint32_t *)appxaddr);

        /* 跳转到 APP */
        jump2app();
    }
}
```

在保存了一个完整的 APP 到了对应的位置后(这个位置需我们指定)，我们需要对栈顶进行检查操作，初步检查程序设置正确再进行跳转。

好那现在问题来了，APP 缓存位置如何指定？这里我们通过串口来接收 APP 的 bin 文件，并存放在一个数组内，通过__attribute__指定数组的所在位置。接着修改 usart.c 文件，如下：

（3）usart.c 文件代码

```
/* USER CODE BEGIN 0 */
#ifdef __GNUC__
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif

PUTCHAR_PROTOTYPE
{
    HAL_UART_Transmit(&huart1, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

/* 接收缓冲, 最大 USART_REC_LEN 个字节. */
uint8_t g_usart_rx_buf[USART_REC_LEN] __attribute__((section(".app_addr")));
/*  接收状态
 *  bit15,      接收完成标志
 *  bit14,      接收到 0x0d
 *  bit13~0,    接收到的有效字节数目
*/
uint16_t g_usart_rx_sta = 0;

uint8_t g_rx_buffer[RXBUFFERSIZE];  /* HAL 库使用的串口接收缓冲 */

/* 接收的字节数 */
uint32_t g_usart_rx_cnt = 0;

/* USER CODE END 0 */
```

__attribute__((section(".app_addr")))，这句话的含义是将数组 g_usart_rx_buf 放置到名为".app_addr"的段中。该段的地址指向是在 xxx.ld 文件中进行定义的。

（4）STM32F103ZETX_FLASH.ld 件代码

打开 STM32F103ZETX_FLASH.ld 文件，添加如下代码：

```
/* Memories definition */
MEMORY
{
  RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 64K
  APP_ADDR    (xrw)    : ORIGIN = 0x20002000,   LENGTH = 55K
  FLASH    (rx)    : ORIGIN = 0x8000000,   LENGTH = 512K
```

```
}

/* Sections */
SECTIONS
{
  /* 需要定义数组所在位置从 0x20002000 开始 */
    .app_addr :
    {
      . = ALIGN(4);
      KEEP(*(.app_addr)) /* Startup code */
      . = ALIGN(4);
    } >APP_ADDR

   /* 省略部分代码… … */
```

标红部分是我们添加的代码，通过该操作，将数组 g_usart_rx_buf 的地址指向 0x20002000。这意味着，当我们使用串口接收 APP 的 bin 文件时，该文件将被存储在数组 g_usart_rx_buf 中，即地址 0x20002000 处。注意 APP 程序同样需要修改此文件，将代码存于指定位置，然后再生成 bin 文件，详情看源码。

（5）main.c 文件代码

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t t;
    uint8_t key;
    uint32_t oldcount = 0;       /* 老的串口接收数据值 */
    uint32_t applenth = 0;       /* 接收到的 app 代码长度 */
    uint8_t clearflag = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                          /* 初始化 LCD */
  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "IAP TEST", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "KEY_UP: Copy APP2FLASH!", RED);
  lcd_show_string(30, 130, 200, 16, 16, "KEY1: Run FLASH APP", RED);
  lcd_show_string(30, 150, 200, 16, 16, "KEY0: Run SRAM APP", RED);
  /* USER CODE END 2 */

  while (1)
  {
      if (g_usart_rx_cnt)
      {
```

```
            /* 新周期内,没有收到任何数据,认为本次数据接收完成 */
            if (oldcount == g_usart_rx_cnt)
            {
                applenth = g_usart_rx_cnt;
                oldcount = 0;
                g_usart_rx_cnt = 0;
                printf("用户程序接收完成!\r\n");
                printf("代码长度:%luBytes\r\n", applenth);
            }
            else oldcount = g_usart_rx_cnt;
        }

        t++;
        delay_ms(100);

        if (t == 3)
        {
            LED0_TOGGLE();
            t = 0;

            if (clearflag)
            {
                clearflag--;

                if (clearflag == 0)
                {
                    lcd_fill(30, 190, 240, 210 + 16, WHITE);     /* 清除显示 */
                }
            }
        }

        key = key_scan(0);

        if (key == WKUP_PRES)    /* WKUP 按下,更新固件到 FLASH */
        {
            if (applenth)
            {
                printf("开始更新固件...\r\n");
                lcd_show_string(30, 190, 200, 16, 16, "Copying APP2FLASH...",
                BLUE);

                if (((*(volatile uint32_t *)(0X20002000 + 4)) & 0xFF000000) ==
                     0x08000000)  /* 判断是否为 0X08XXXXXX */
                {
                    iap_write_appbin(FLASH_APP1_ADDR, g_usart_rx_buf, applenth);
                    /* 更新 FLASH 代码 */
                    lcd_show_string(30, 190, 200, 16, 16, "Copy APP
Successed!!",
                     BLUE);
                    printf("固件更新完成!\r\n");
                }
                else
                {
                    lcd_show_string(30, 190, 200, 16, 16, "Illegal FLASH APP!
",
                     BLUE);
                    printf("非 FLASH 应用程序!\r\n");
                }
            }
            else
            {
                printf("没有可以更新的固件!\r\n");
                lcd_show_string(30, 190, 200, 16, 16, "No APP!", BLUE);
            }
```

```
            clearflag = 7; /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
        }

        if (key == KEY1_PRES)    /* KEY1 按键按下, 运行 FLASH APP 代码 */
        {
            if (((*(volatile uint32_t *)(FLASH_APP1_ADDR + 4)) & 0xFF000000) ==
                0x08000000) /* 判断 FLASH 里面是否有 APP,有的话执行 */
            {
                printf("开始执行 FLASH 用户代码!!\r\n\r\n");
                delay_ms(10);
                iap_load_app(FLASH_APP1_ADDR);/* 执行 FLASH APP 代码 */

            }
            else
            {
                printf("没有可以运行的固件!\r\n");
                lcd_show_string(30, 190, 200, 16, 16, "No APP!", BLUE);
            }

            clearflag = 7; /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
        }

        if (key == KEY0_PRES)    /* KEY0 按下 */
        {
            printf("开始执行 SRAM 用户代码!!\r\n\r\n");
            delay_ms(10);

            if (((*(volatile uint32_t *)(0X20002000 + 4)) & 0xFF000000) ==
            0x20000000)    /* 判断是否为 0X20XXXXXX */
            {
                iap_load_app(0X20002000);    /* SRAM 地址 */
            }
            else
            {
                printf("非 SRAM 应用程序,无法执行!\r\n");
                lcd_show_string(30, 190, 200, 16, 16, "Illegal SRAM APP!",
BLUE);
            }

            clearflag = 7; /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
        }
    }
}
```

APP 代码我们在这里就不做介绍了，大家可以参考本例程提供的源代码，注意在 mian 函数起始处重新设置中断向量表（寄存器 SCB-> VTOR）的偏移量，否则 APP 无法正常运行，仍以 Flash APP 为例，我们通过 Release 编译优化后生成 bin 文件，如图 64.2.3.2 所示：

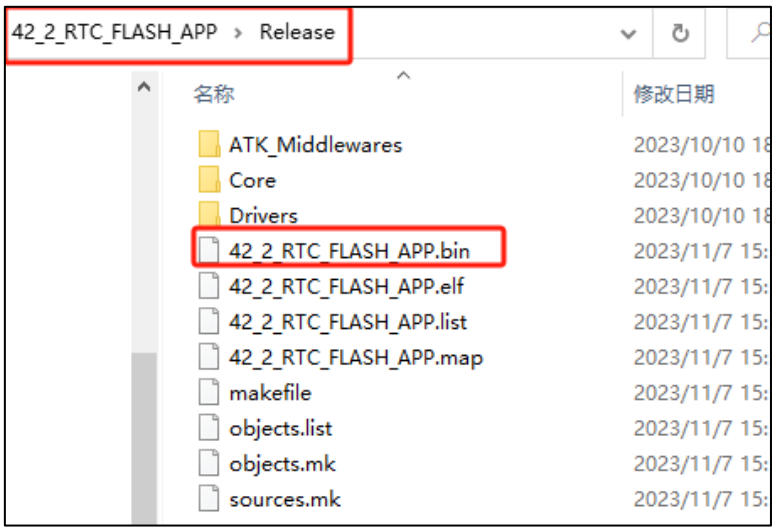


图 64.2.3.2 多存储段 APP 程序生成多个.bin 文件

64.3 下载验证

将程序下载到开发板后，可以看到 LCD 首先显示一些实验相关的信息，如图 64.3.1 所示：

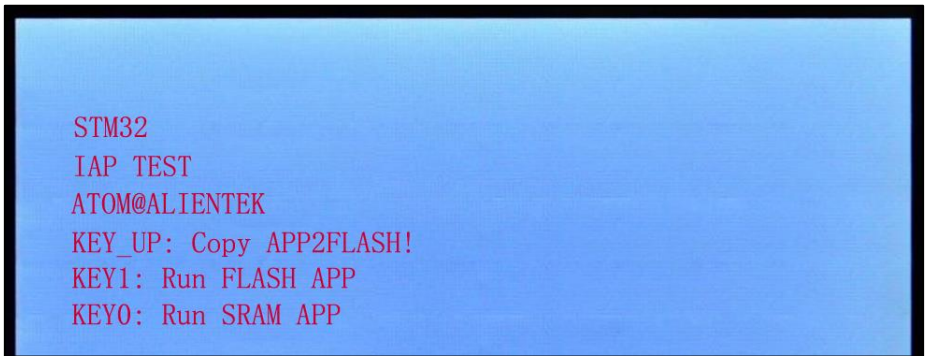


图 64.3.1 IAP 程序界面

此时，我们可以通过 XCOM，发送 FLASH APP、SRAM APP 到开发板，我们以 FLASH APP 为例进行演示，如图 64.3.2 所示：

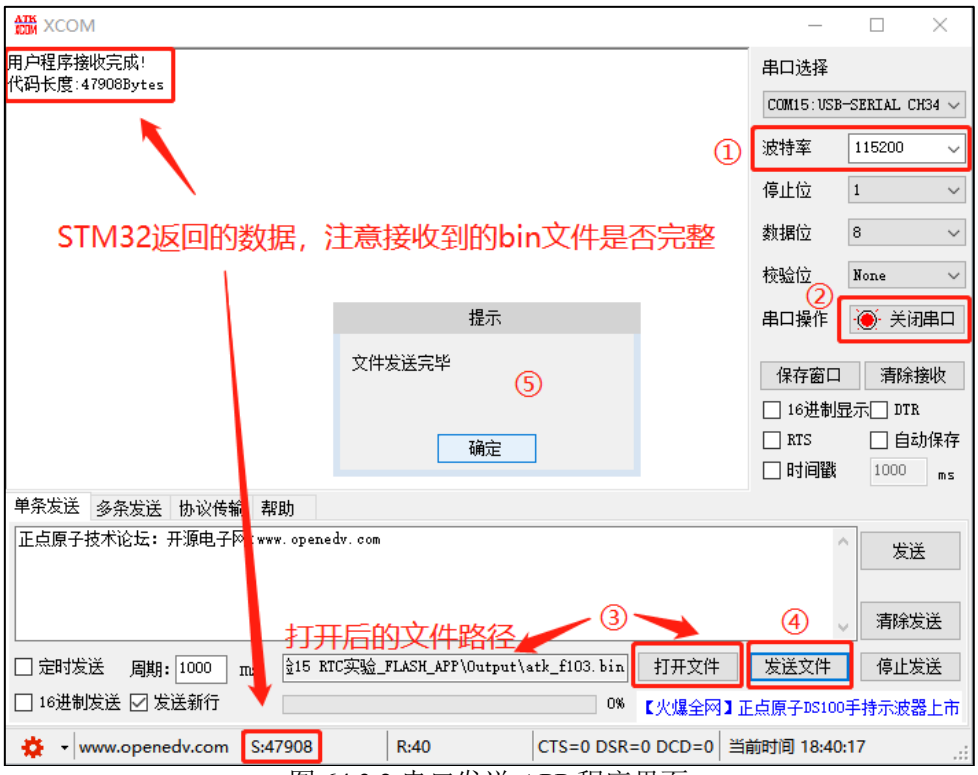


图 64.3.2 串口发送 APP 程序界面

首先找到开发板 USB 转串口的串口号，打开串口（我电脑是 COM15），然后设置波特率为 115200 并打开串口，然后，点击打开文件按钮（图中标号 3 所示），找到 APP 程序生成的.bin 文件（注意：文件类型得选择所有文件！默认是只打开 txt 文件的），最后点击发送文件（图中标号 4 所示），将.bin 文件发送给 STM32 开发板，发送完成后，XCOM 会提示文件发送完毕（图中标号 5 所示）。

开发板收到 APP 程序之后会打印提示信息，我们可以根据发送的数据与开发板的提示信息确认开发板接收到的 bin 文件是否完整，我们就可以通过 KEY0/KEY1 运行这个 APP 程序了（如果是 FLASH APP，需要通过 KEY1 将其存入对应 FLASH 区域），此时我们根据程序设计，按下 KEY1 即可执行 FLASH APP 程序，更新 SRAM APP 的过程类似，大家自行测试即可。

第六十五章 USB 读卡器（Slave）实验

本章将介绍使用 STM32F103 作为 USB 从设备，模拟出 USB 读卡器，使得 PC 与 STM32F103 通过 USB 连接后，能够访问与 STM32F103 连接的 SD 和 Norflash 中的数据。通过本章的学习，读者将学习到 STM32F103 作为 USB 从设备模拟出 USB 读卡器的使用。

本章分为如下几个小节：

65.1 硬件设计
65.2 程序设计
65.3 下载验证

65.1 硬件设计

1. 例程功能

本节实验功能简介：开机的时候先检测 SD 卡、SPI FLASH 是否存在，如果存在则获取其容量，并显示在 LCD 上（如果不存在，则报错）。之后开始 USB 配置，在配置成功之后，通过 USB 连接线可以在电脑上发现两个可移动磁盘。我们用 LED1 来指示 USB 正在读写，并在液晶上显示出来，同样，我们还是用 DS0 来指示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB5
LED1 – PE5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）microSD Card（使用大卡的情况类似，大家可根据自己设计的硬件匹配选择）
5）SPI Flash
6）STM32 自带的 USB Slave 功能，要通过跳线帽连接 PA11 和 D-以及 PA12 和 D+。

图 65.1.1 USB SLAVE 接口

3. 原理图

本章实验使用 USB 接口与 PC 进行连接，开发板板载了一个 USB SLAVE 接口，用于连接其他 USB 设备，USB SLAVE 接口与 MCU 的连接原理图，如下图所示：

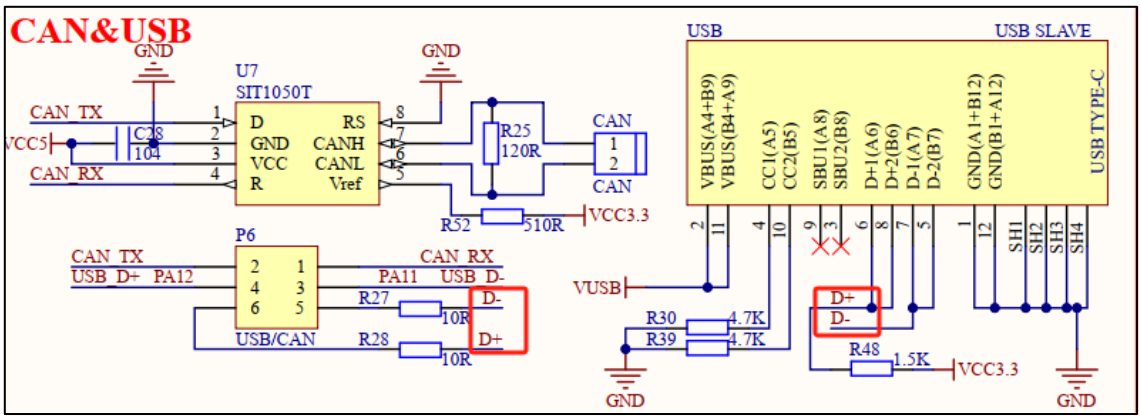


图 65.1.2 USB SLAVE 接口与 MCU 的连接原理图

65.2 程序设计

65.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 43_USB_CARD_READER。

65.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 34_SD 工程，并将工程文件名和.ioc 文件名均修改为 43_USB_CARD_READER，以进行下一步的实验操作。如下图所示：

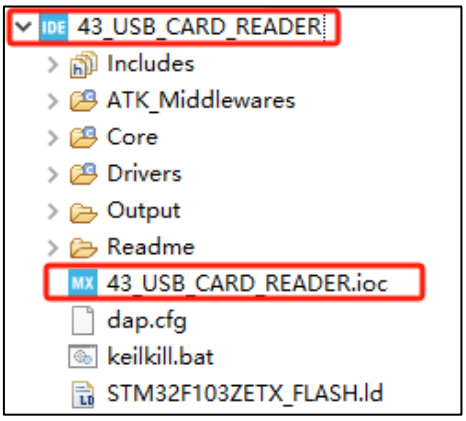


图 65.2.2.1 USB_CARD_READER 工程

在本次实验中，我们旨在实现 USB 读卡器的读写功能，以读写两个磁盘，即 SD 卡和 NOR FLASH。SD 卡的配置已经在之前的实验中继承，因此本次实验的还需配置 NOR FLASH 的驱动。关于 NOR FLASH 的配置，请参考 SPI 实验的章节，并将 SPI 工程中的 NOFFLASH 文件夹复制到本次实验的 BSP 文件夹下。具体操作可参考下图：

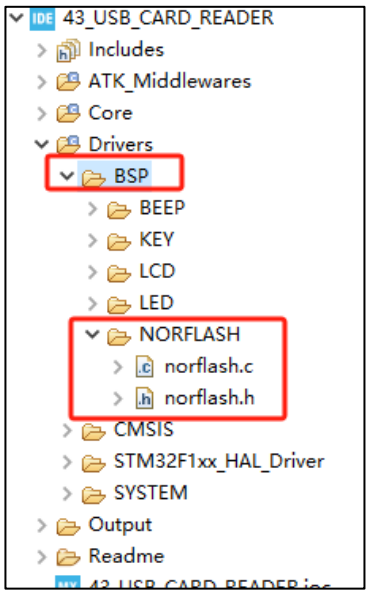


图 65.2.2.2 添加 norflash 驱动文件

注意 SPI2 的配置以及 spi.c 和 spi.h 文件的修改这里我们不在赘述，大家可参考 SPI 实验章节进行配置。接着我们开始配置 USB 部分。

（1）USB 配置

在 STM32CubeMX 插件界面的 Connectivity→USB 处激活 USB 从设备：

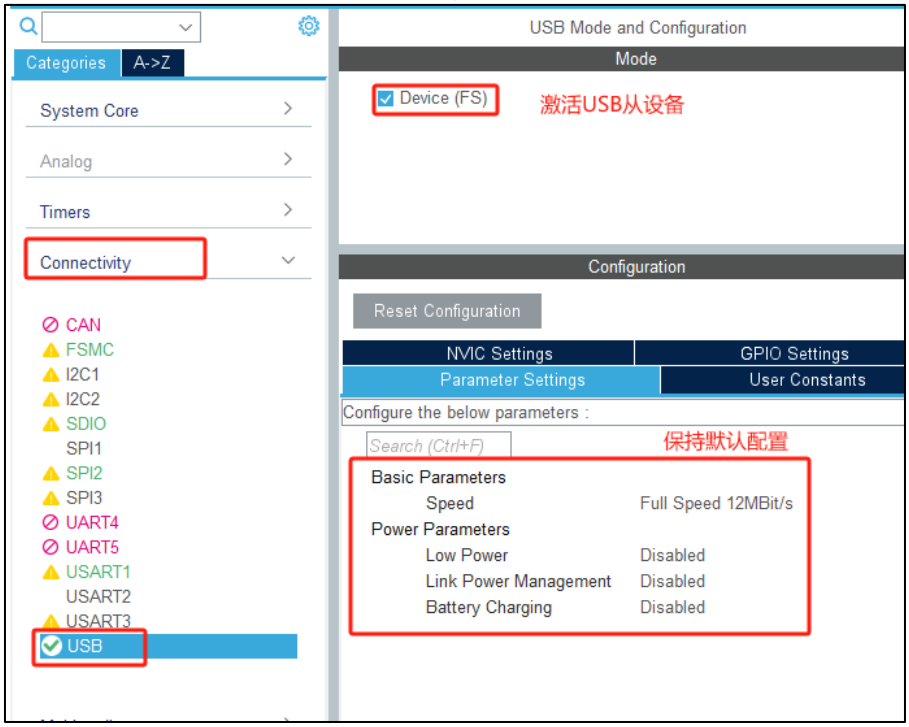


图 65.2.2.3 USB 配置

在 USB 参数部分，我们设定了 USB 传输速度为全速，并且禁用了低功耗模式。

接着开始配置 USB 的中间层组件，打开 Middleware and Software Packs-> USB_DEVICE 处我们配置 USB 组件参数，如下图所示：

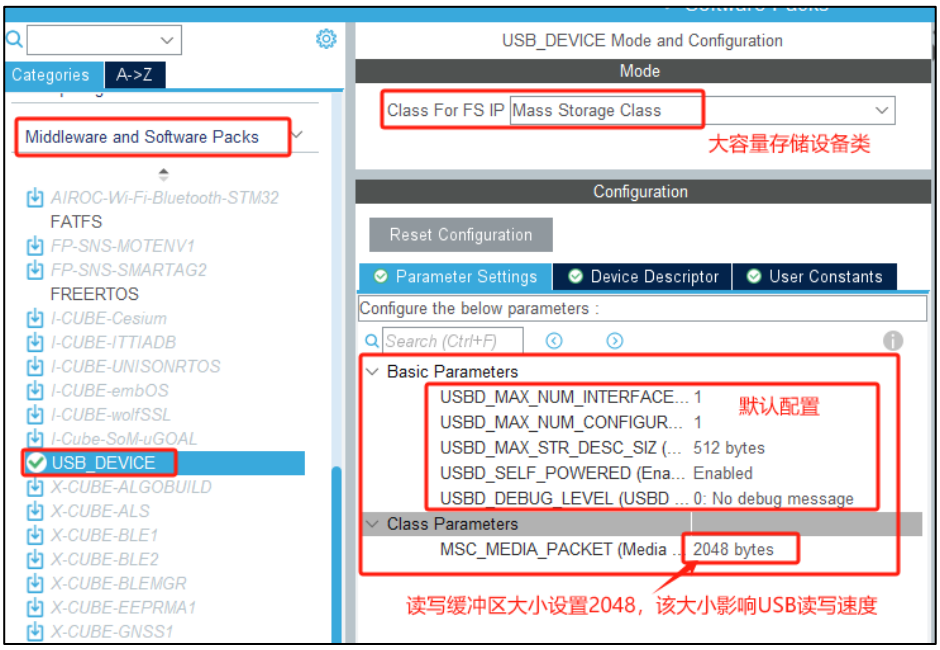


图 65.2.2.4 USB 组件配置

接着设备描述符保持默认配置。如下图所示：

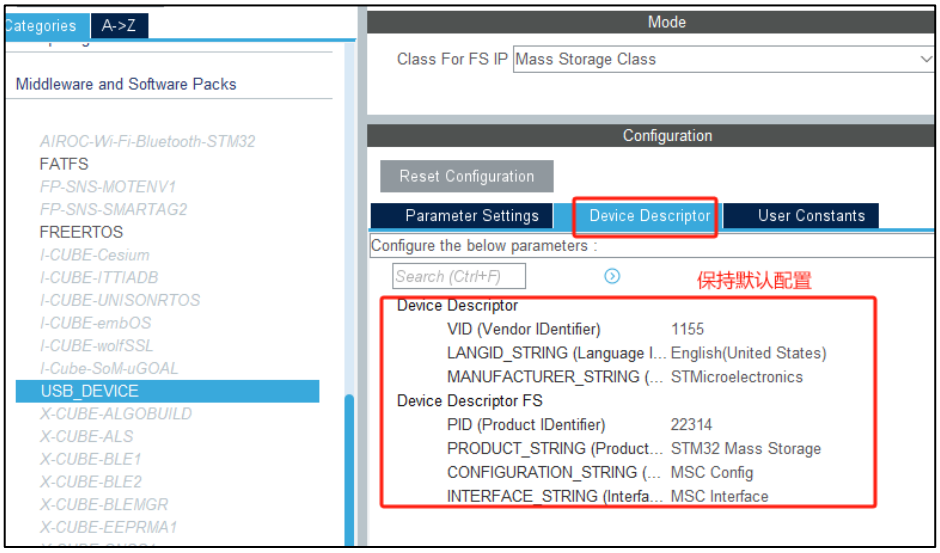


图 65.2.2.5 设备描述符默认配置

（2）USB 时钟配置

经过上述配置后，发现时钟配置部分存在错误，时钟频率超过了 48MHz，因此出现了红色警告提示，具体如下图所示：

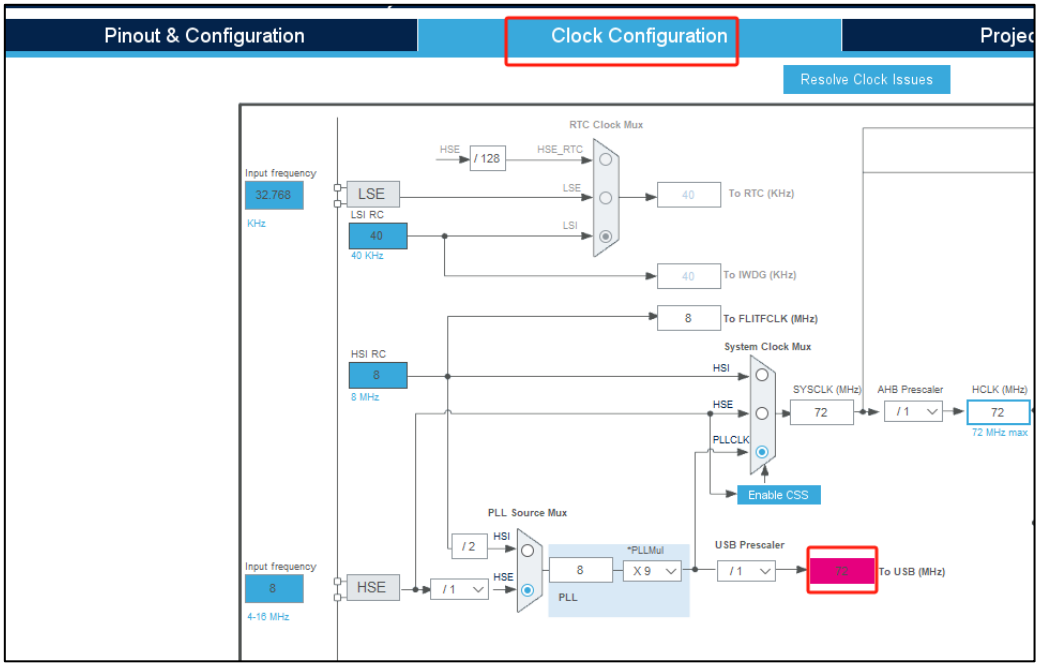


图 65.2.2.6 USB 时钟报错

进行如下配置修改为 48MHz：

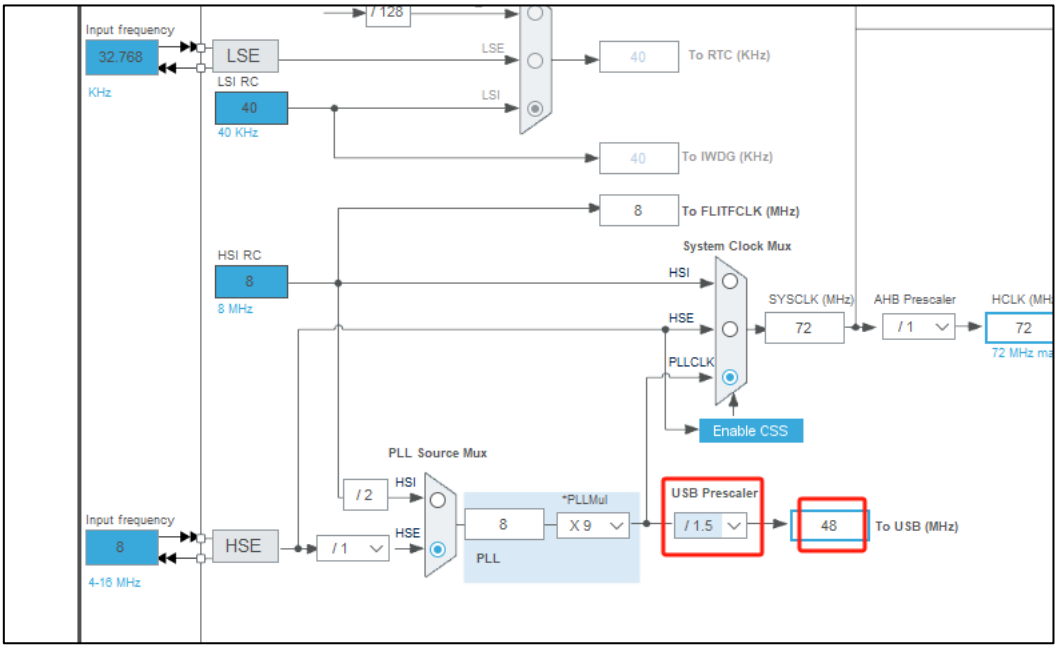


图 65.2.2.7 USB 时钟配置

（3）NVIC 配置

我们需要再 USB 中断回调函数中获取 USB 的连接状态，所以我们要使能 USB 低优先级中断（默认打开的），并配置中断优先级。如下图，配置中断优先级分组为 2，抢占优先级 2，子优先级为 1。

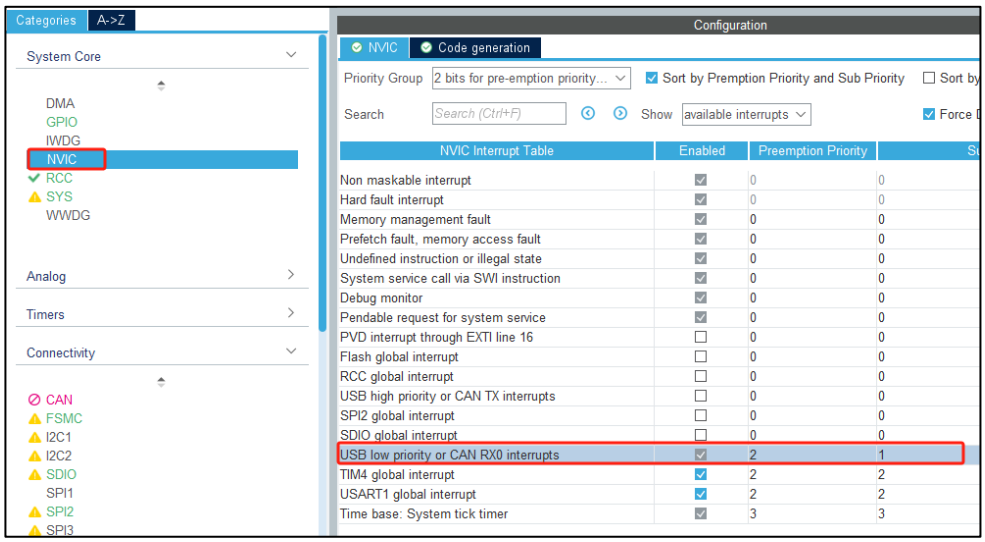


图 65.2.2.8 NVIC 配置

65.2.3 生成初始化工程

配置完成后，按下“Ctrl+S”保存配置并生成工程。在生成的工程中，可以看到多了两个文件夹，分别是 USB_DEVICE 和 Middlewares。USB_DEVICE 文件夹用于存放 USB 硬件抽象层和 USB 通信相关的文件，而 Middlewares 文件夹用于存放关于 USB 第三方中间层组件。工程的结构如下图所示。

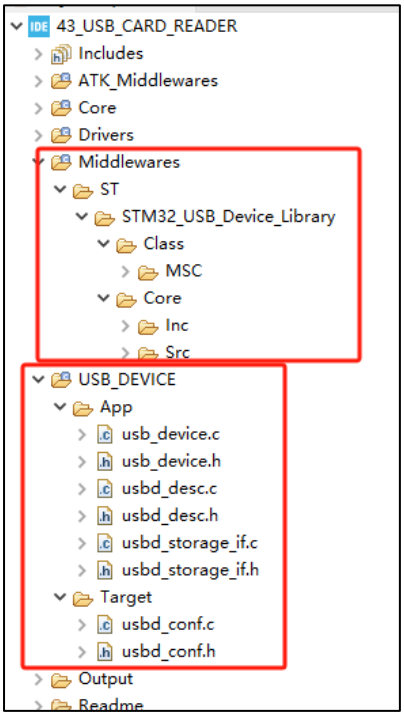


图 65.2.3.1 生成工程

65.2.4 添加用户驱动代码

在本章实验中，我们将不涉及 SPI 和 NORFLASH 部分的移植和修改。有关这些内容的详细信息，请参阅 SPI 实验章节。此外，sdio.c 初始化部分注意改为 1 位宽。

然后在 USB_DEVICE 文件夹中需要修改的几个文件，包括 usbd_storage_if.c、usbd_conf.c 和 usbd_conf.h。

其中 usbd_conf.c/.h 主要实现 USB 的硬件初始化和中断操作，当 USB 状态机处理完不同事务的时候，会调用这些回调函数，我们通过这些回调函数，就可以知道 USB 当前状态，比如：是否枚举成功了？是否连接上了？是否断开了？根据这些状态，用户应用程序可以执行不同操作，完成特定功能。

usbd_storage_if.c 主要实现大容量存储设备的一个标识信息，作为一个标准接口被封装在 USB 大容量存储设备的操作结构体 USBD_Storage_Interface_fops_FS 中，它是在 usbd_msc.h 定义好的 USBD_StorageTypeDef 类型，基本为函数指针和数据指针，我们需要为这些 USB 操作实现跟我们硬件相关的底层代码。

关于这三个文件的修改大家可打开源码查看，篇幅过长这里不列出来了，我们来看下 main.c 文件的实现。

（1）main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint8_t offline_cnt = 0;
    uint8_t tct = 0;
    uint8_t usb_sta;
    uint8_t device_sta;
    uint16_t id;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* TIM4 初始化 */
  MX_TIM4_Init();

  /* SDIO 初始化 */
  MX_SDIO_SD_Init();

  /* SPI2 初始化 */
  MX_SPI2_Init();

  /* USB 初始化 */
  MX_USB_DEVICE_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                    /* 初始化 LCD */
  norflash_init();               /* 初始化 NOR FLASH */
  my_mem_init(SRAMIN);           /* 初始化内部 SRAM 内存池 */

  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "USB Card Reader TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
```

```
if (g_sd_sta == 1)          /* 检测 SD 卡错误 */  {
    g_sd_sta = 0;
    MX_SDIO_SD_Init();
    lcd_show_string(30, 110, 200, 16, 16, "SD Card Error!", RED);
}
else                        /* SD 卡正常 */
{
    lcd_show_string(30, 110, 200, 16, 16, "SD Card Size:     MB", RED);
    lcd_show_num(134, 110, SD_TOTAL_SIZE_MB(&hsd), 5, 16, RED);
}

id = norflash_read_id();
if ((id == 0) || (id == 0XFFFF))
{
    lcd_show_string(30, 110, 200, 16, 16, "NorFlash Error!", RED);
}
else                        /* SPI FLASH 正常 */
{
    lcd_show_string(30, 130, 200, 16, 16, "SPI FLASH Size:7.25MB", RED);
}

usbd_port_config(0);        /* USB 先断开 */
delay_ms(500);
usbd_port_config(1);        /* USB 再次连接 */
delay_ms(500);

lcd_show_string(30, 170, 200, 16, 16, "USB Connecting...", RED);
delay_ms(1800);
/* USER CODE END 2 */

while (1)
{
    delay_ms(1);

    if (usb_sta != g_usb_state_reg)              /* 状态改变了 */
    {
        lcd_fill(30, 190, 240, 210 + 16, WHITE); /* 清除显示 */

        if (g_usb_state_reg & 0x01)              /* 正在写 */
        {
            LED1(0);
            /* 提示 USB 正在写入数据 */
            lcd_show_string(30, 190, 200, 16, 16, "USB Writing...", RED);
        }

        if (g_usb_state_reg & 0x02)    /* 正在读 */
        {
            LED1(0);
           /* 提示 USB 正在读出数据 */
            lcd_show_string(30, 190, 200, 16, 16, "USB Reading...", RED);
        }

        if (g_usb_state_reg & 0x04)
        {
           /* 提示写入错误 */
            lcd_show_string(30, 210, 200, 16, 16, "USB Write Err ", RED);
        }
        else
        {
            lcd_fill(30, 210, 240, 230 + 16, WHITE); /* 清除显示 */
        }

        if (g_usb_state_reg & 0x08)
        {
```

```
            /* 提示读出错误 */
            lcd_show_string(30, 230, 200, 16, 16, "USB Read  Err ", RED);
        }
        else
        {
            lcd_fill(30, 230, 240, 250 + 16, WHITE); /* 清除显示 */
        }

        usb_sta = g_usb_state_reg;                  /* 记录最后的状态 */
    }

    if (device_sta != g_device_state)
    {
        if (g_device_state == 1)
        {
            /* 提示 USB 连接已经建立 */
            lcd_show_string(30, 170, 200, 16, 16, "USB Connected    ", RED);
        }
        else
        {
            /* 提示 USB 被拔出了 */
            lcd_show_string(30, 170, 200, 16, 16, "USB DisConnected ", RED);
        }

        device_sta = g_device_state;
    }

    tct++;

    if (tct == 200)
    {
        tct = 0;
        LED1(1);                        /* 关闭 LED1 */
        LED0_TOGGLE();                  /* LED0 闪烁 */

        if (g_usb_state_reg & 0x10)
        {
            offline_cnt = 0;            /* USB 连接了,则清除 offline 计数器 */
            g_device_state = 1;
        }
        else                            /* 没有得到轮询 */
        {
            offline_cnt++;

            if (offline_cnt > 100)
            {
                g_device_state = 0;/* 20s 内没收到在线标记,代表 USB 被拔出了 */
            }
        }

        g_usb_state_reg = 0;
    }
  }
}
```

根据上述代码，可以观察到在调用 MX_USB_DEVICE_Init()函数之后，系统会持续地获取 USB 的读写状态和连接状态，并在 LCD 上进行显示。

在调用完函数 MX_USB_DEVICE_Init()后，USB 驱动库便会自动模拟出 1 个 USB 读卡器，当然也需要实现配置好 USB 读卡器模拟出的设备信息，以及读写、初始化 SD 卡和 norflash 等的操作，这些操作全部在 usbd_storage_if.c 文件中完成了，请读者自行参考该文件。

65.3 下载验证

在代码编译成功之后，我们通过下载代码到开发板上，在 USB 配置成功后（假设已经插

入 SD 卡，注意：USB 数据线，要插在开发板的 USB_SLAVE 口！而不是 USB_232 端口，且 P9 必须用跳线帽连接 PA11 和 D-以及 PA12 和 D+!!），LCD 显示效果如图 65.3.1 所示：

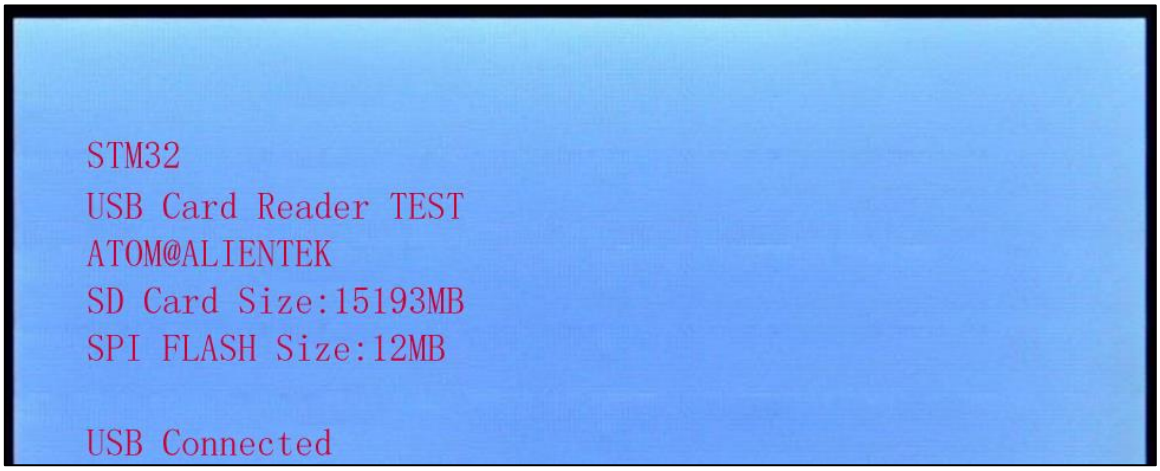


图 65.3.1 USB 连接成功

USB 识别成功后，电脑提示发现新硬件，并自动安装驱动，并且在设备管理器中可看到我们命名的驱动设备如下图所示：

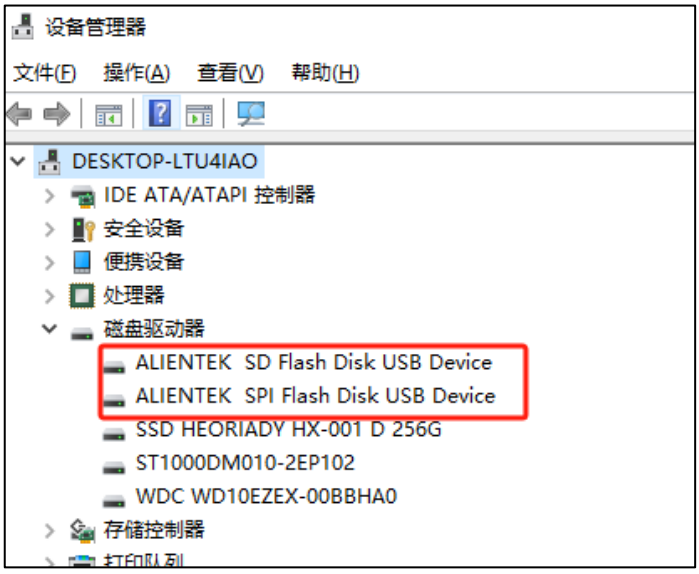


图 65.3.2 设备管理器显示

等 USB 配置成功后，DS1 不亮，DS0 闪烁，并且在电脑上可以看到我们的磁盘，如图 65.3.3 所示：

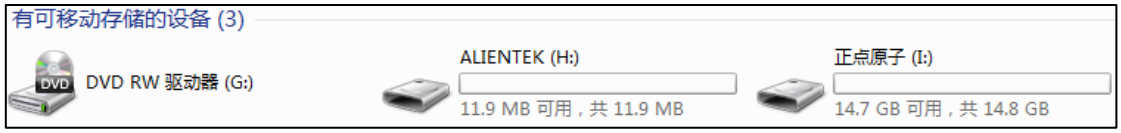


图 65.3.3 电脑找到 USB 读卡器的两个盘符

此时，我们就可以通过电脑读写 SD 卡或者 SPI FLASH 里面的内容了。在执行读写操作的时候，就可以看到 LED1 亮，并且会在液晶上显示当前的读写状态。

注意，因为 SPI FLASH 有写次数据限制，最好不要频繁的往里面写数据，否则很容易将 SPI FLASH 写爆!!

第六十六章 USB 虚拟串口（Slave）实验

本章将介绍使用 STM32F103 作为 USB 从设备，虚拟出串口与 PC 进行通信。通过本章的学习，读者将学习到 STM32F103 作为 USB 从设备虚拟出串口的使用。

本章分为如下几个小节：

66.1 硬件设计
66.2 程序设计
66.3 下载验证

66.1 硬件设计

1. 例程功能

1）LED 灯
LED0 – PB5
LED1 – PE5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）STM32 自带的 USB Slave 功能

这几个外设原理图我们在之前的章节已经介绍过了，这里就不重复介绍了，不清楚的话可以查看本文之前章节的描述或对光盘资料提供的开发板原理图。这里再次提醒大家，P3 的连接，要通过跳线帽连接 PA11 和 D-以及 PA12 和 D+。

2. 硬件资源

1）LED 灯
LED0 – PB5
LED1 – PE5
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)
4）microSD Card（使用大卡的情况类似，大家可根据自己设计的硬件匹配选择）
5）SPI Flash
6）STM32 自带的 USB Slave 功能，要通过跳线帽连接 PA11 和 D-以及 PA12 和 D+。

图 66.1.1 USB SLAVE 接口

3. 原理图

请见第 53.1.3 小节中 USB SLAVE 接口与 MCU 的连接原理图的相关内容。

66.2 程序设计

66.2.1 配套源码路径

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 44_USB_VSP。

我们按流程图编写的初始化顺序，在 STM32 注册 USB 内核，最后通过 USB 的中断和回调函数得到 USB 的操作状态和操作结果，主程序通过查询设定的标记变量的状态值后，在 LCD 上显示对应的 USB 操作状态。

66.2.2 创建和配置工程

在 STM32CubeIDE 中，通过复制 33_MALLOC 工程，并将工程文件名和.ioc 文件名均修改为 44_USB_VSP，以进行下一步的实验操作。如下图所示：

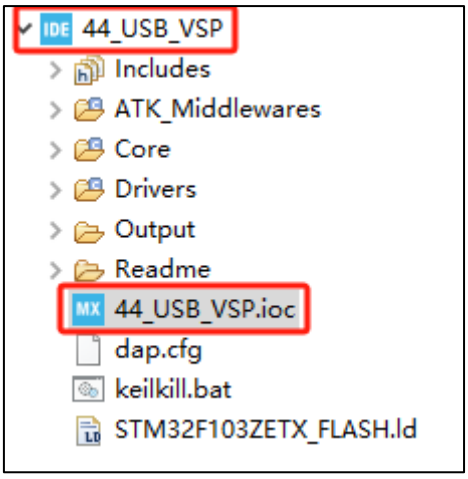


图 66.2.2.1 USB_VSP 工程

（1）USB 配置

双击 44_USB_VSP.ioc 进入 STM32CubeMX 插件界面，在 Connectivity->USB 处激活 USB 从设备，如下图所示：

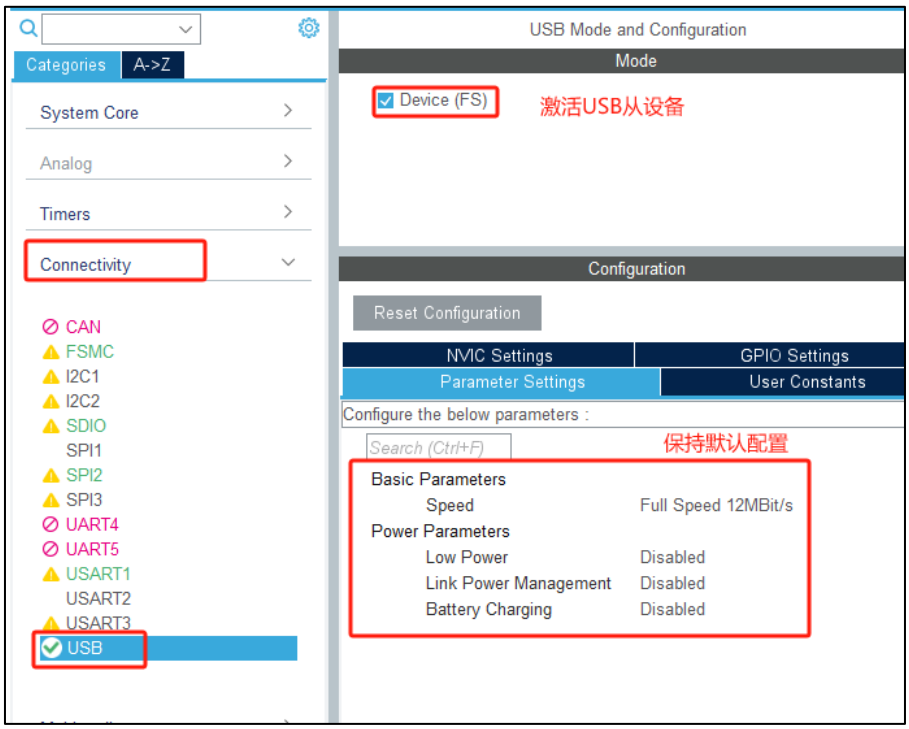


图 66.2.2.2 USB 配置

接着开始配置 USB 的中间层组件，打开 Middleware and Software Packs-> USB_DEVICE 处我们配置 USB 组件参数，如下图所示：

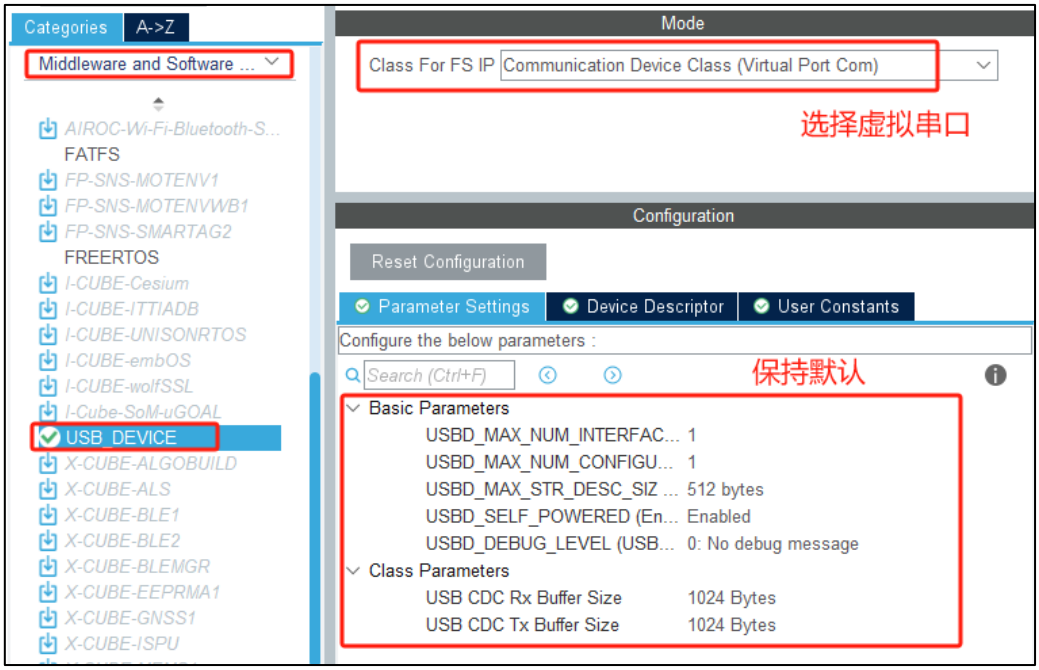


图 66.2.2.3 USB 组件配置

USB 的设备描述符保持默认配置。如下图所示：

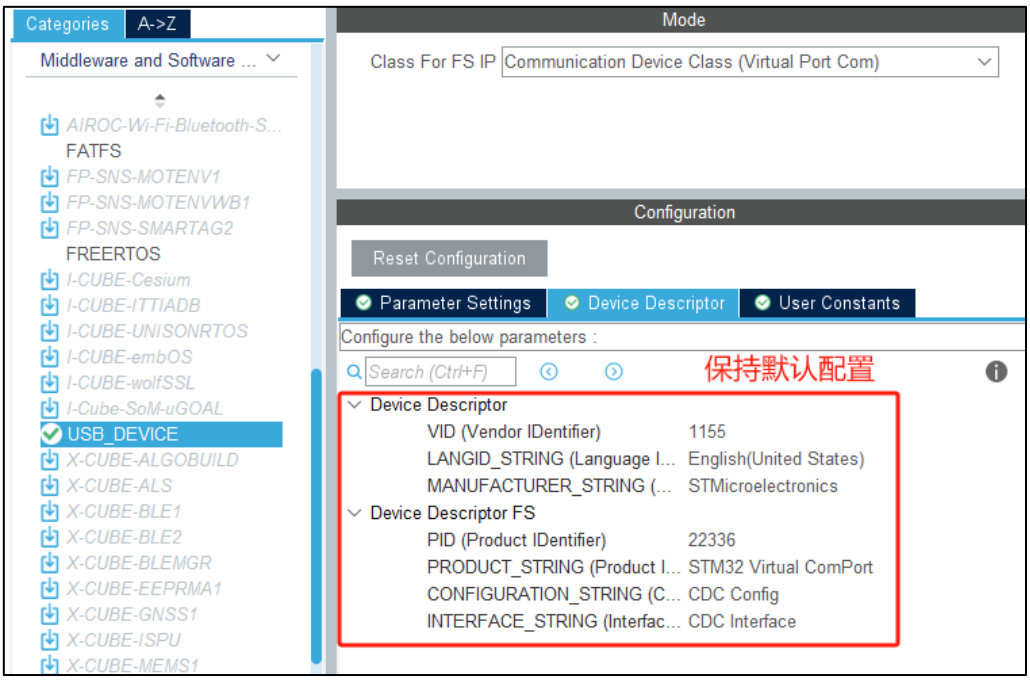


图 66.2.2.4 设备描述符默认配置

（2）USB 时钟配置

将 USB 时钟配置为 48M。如下图所示：

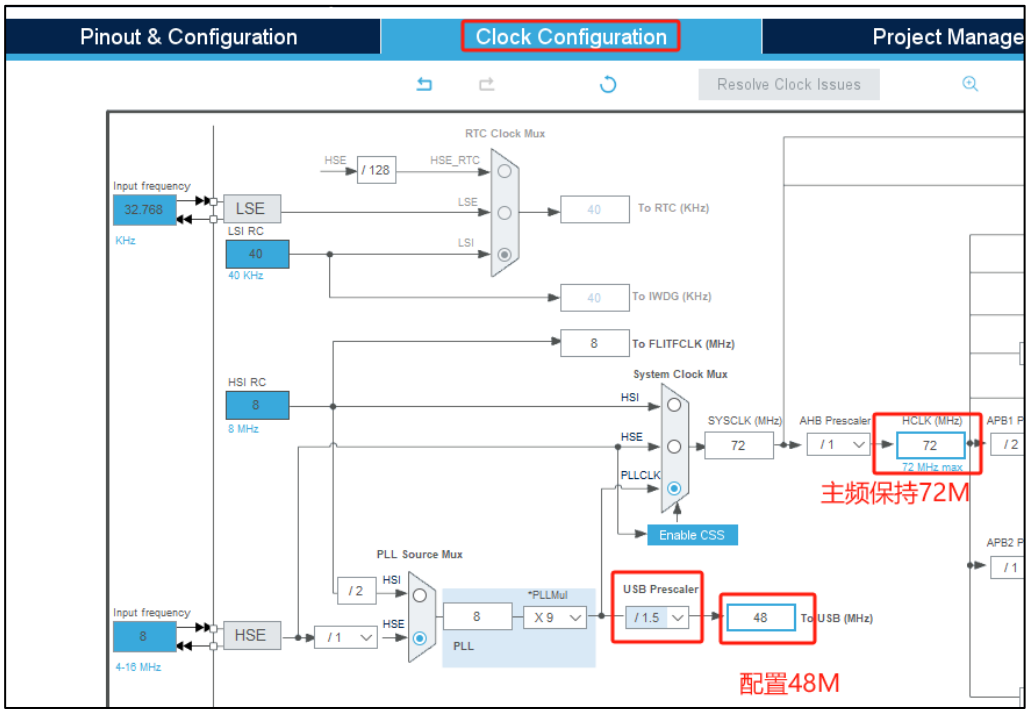


图 66.2.2.5 USB 时钟配置

（3）NVIC 配置

我们需要再 USB 中断回调函数中获取 USB 的连接状态，所以我们要使能 USB 低优先级中断（默认打开的），并配置中断优先级。如下图，配置中断优先级分组为 2，抢占优先级 2，子优先级为 1。

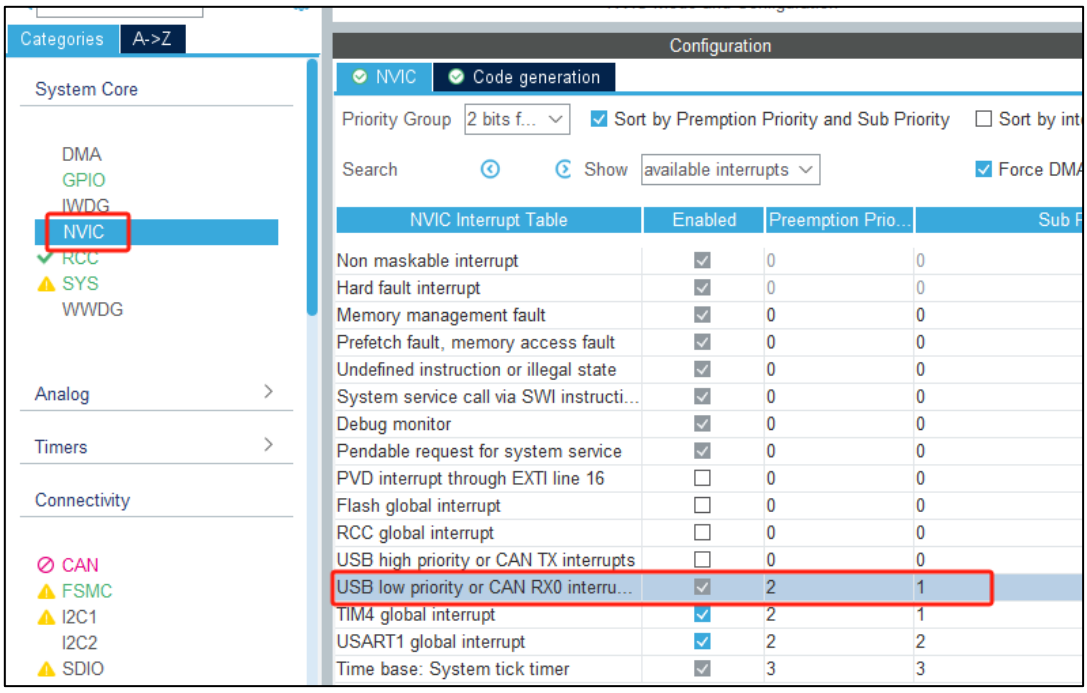


图 66.2.2.6 NVIC 配置

66.2.3 生成初始化工程

配置完成后，按下“Ctrl+S”保存配置并生成工程。在生成的工程中，可以看到多了两个文件夹，分别是 USB_DEVICE 和 Middlewares。USB_DEVICE 文件夹用于存放 USB 硬件抽象层和 USB 通信相关的文件，而 Middlewares 文件夹用于存放关于 USB 第三方中间层组件。工程的结构如下图所示。

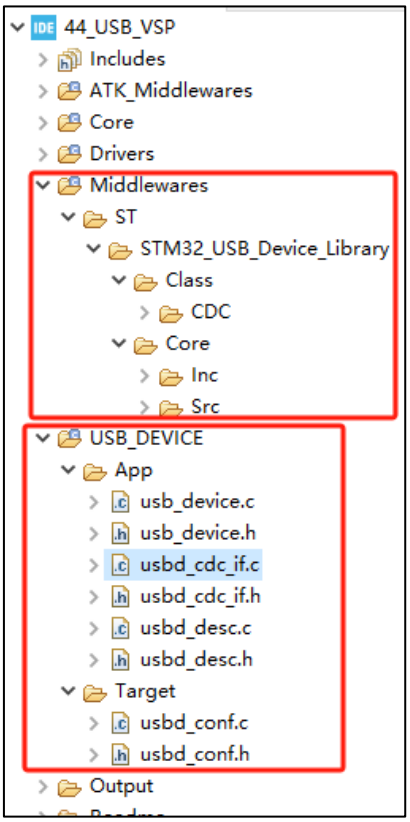


图 66.2.3.1 生成工程

66.2.4 添加用户驱动代码

在 USB_DEVICE 文件夹中需要修改的 4 个文件，包括 usbd_cdc_if.c、usbd_cdc_if.h、usbd_conf.c 和 usbd_conf.h。

usbd_cdc_if.c/.h 这两个文件使用到开发板的硬件底层驱动，需要适配我们的硬件信息。本例中，我们需要把 USB 操作与我们的串口操作对应起来。对此接口的详细修改大家可以参考对照光盘的源码即可。

usbd_conf.c/.h 主要实现 USB 的硬件初始化和中断操作，当 USB 状态机处理完不同事务的时候，会调用这些回调函数，我们通过这些回调函数，获取 USB 当前状态。

关于上述四个文件的修改细节，请自行查阅本实验的源代码。接着我们来看下 main.c 文件的实现。

（1）main.c 文件

```
int main(void)
{
  /* USER CODE BEGIN 1 */
    uint16_t len;
    uint16_t times = 0;
    uint8_t usbstatus = 0;
  /* USER CODE END 1 */

  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USB 初始化 */
  MX_USB_DEVICE_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                    /* 初始化 LCD */
  my_mem_init(SRAMIN);                           /* 初始化内部 SRAM 内存池 */

  lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30, 70, 200, 16, 16, "USB Virtual USART TEST", RED);
  lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
  lcd_show_string(30, 110, 200, 16, 16, "USB Connecting...", RED);

  usbd_port_config(0);     /* USB 先断开 */
  delay_ms(500);
  usbd_port_config(1);     /* USB 再次连接 */
  delay_ms(500);
  /* USER CODE END 2 */

  while (1)
  {
      if (usbstatus != g_device_state)      /* USB 连接状态发生了改变 */
```

```
        {
            usbstatus = g_device_state;      /* 记录新的状态 */

            if (usbstatus == 1)
            {
                /* 提示 USB 连接成功 */
                lcd_show_string(30, 110, 200, 16, 16, "USB Connected    ", RED);
                LED1(0);    /* 绿灯亮 */
            }
            else
            {
                /* 提示 USB 断开 */
                lcd_show_string(30, 110, 200, 16, 16, "USB disConnected ", RED);
                LED1(1);    /* 绿灯灭 */
            }
        }

        if (g_usb_usart_rx_sta & 0x8000)
        {
            len = g_usb_usart_rx_sta & 0x3FFF;   /* 得到此次接收到的数据长度 */
            usb_printf("\r\n 您发送的消息长度为:%d\r\n\r\n", len);
            cdc_vcp_data_tx(g_usb_usart_rx_buffer, len);;
            usb_printf("\r\n\r\n");              /* 插入换行 */
            g_usb_usart_rx_sta = 0;
        }
        else
        {
            times++;

            if (times % 5000 == 0)
            {
                usb_printf("\r\nSTM32 开发板 USB 虚拟串口实验\r\n");
                usb_printf("正点原子@ALIENTEK\r\n\r\n");
            }

            if (times % 200 == 0)usb_printf("请输入数据,以回车键结束\r\n");

            if (times % 30 == 0)
            {
                LED0_TOGGLE();  /* 闪烁 LED,提示系统正在运行 */
            }

            delay_ms(10);
        }
    }
}
```

main.c 就比较简单了，按照我们的流程图的思路编写即可，我们初始化按键，LCD 和 LED 等外设辅助程序显示，同样地在 while 循环之前调用我们之前编写的 usbd_port_config 接口以保证每次复位后 USB 都能重连成功。

66.3 下载验证

在完成编译和烧录操作后，可以看到 LCD 上显示了本实验的相关实验信息，此时可以将开发板通过 USB SLAVE 接口与 PC 进行连接，待 PC 自动完成驱动安装后，便可看到 PC 上多出了一个端口设备，此时可以通过串口调试助手与该端口进行通信，可以看到，我们的串口调试助手，收到了来自 STM32 开发板的数据，同时，按发送按钮（串口助手必须勾选：发送新行），也可以收到电脑发送给 STM32 的数据（原样返回），说明我们的实验是成功的。实验现象同串口通讯实验完全一样。

至此，USB 虚拟串口实验就完成了，通过本实验，我们就可以利用 STM32 的 USB，直接和电脑进行数据互传了，具有广泛的应用前景。

第六十七章 FreeRTOS 移植实验

前面章节中的实验都是在裸机环境下实现，本章将介绍 FreeRTOS 的简单使用，对 RTOS 感兴趣的读者，可以参考正点原子推出的 FreeRTOS 或 μC/OS-III 的全套教程资料，包含教学视频、开发指南并配套有例程源码。通过本章的学习，读者将学习到 FreeRTOS 的简单应用。

本章分为如下几个小节：

67.1 硬件设计
67.2 程序设计
67.3 下载验证

67.1 硬件设计

1. 例程功能

1. LED0 由 task01 任务控制每 1000ms 状态翻转一次
2. USART1 由 task02 任务控制每 500ms 打印一次浮点数，且浮点数每次自加 0.01。

2. 硬件资源

1. LED
   LED0 - PB5
   LED1 - PE5
2. 串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340C 上面)，需要跳线帽连接。

3. 原理图

本章实验使用的 FreeRTOS 为软件库，因此没有对应的连接原理图。

67.2 程序设计

本实验配置好的实验工程已经放到了开发板光盘中，路径为：开发板光盘 A-基础资料\1、程序源码\4，CubeIDE_project\ 45_FreeRTOS_Demo。

67.2.1 创建和配置工程

在 STM32CubeIDE 中，通过复制 33_MALLOC 工程，并将工程文件名和.ioc 文件名均修改为 45_FreeRTOS_Demo，以进行下一步的实验操作。如下图所示：

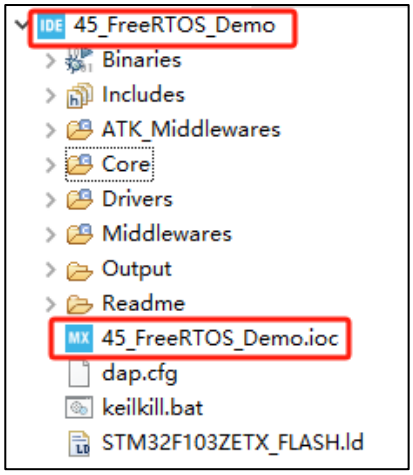


图 67.2.1.1 FreeRTOS_Demo 工程

（1）FreeRTOS 配置

双击 45_FreeRTOS_Demo.ioc 进入 CubeMX 插件界面，在 Connectivity->FREERTOS 处选择 CMSIS_V2，如下图所示：

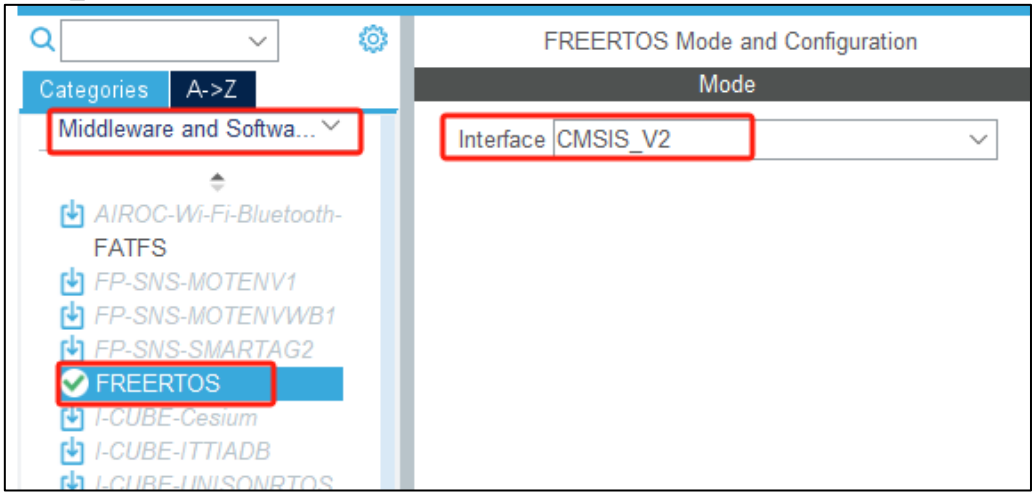


图 67.2.1.2 FreeRTOS 配置

该选项包含三个选项：Disable、CMSIS_V1 和 CMSIS_V2。由于我们将使用 FreeRTOS，因此需要从 CMSIS_V1 和 CMSIS_V2 这两个选项中进行选择。关于这两者之间的区别，详细信息请参见 ARM 官方网站上关于微控制器软件接口标准 CMSIS 的定义。

关于 FreeRTOS 的其他配置，保持默认配置即可。

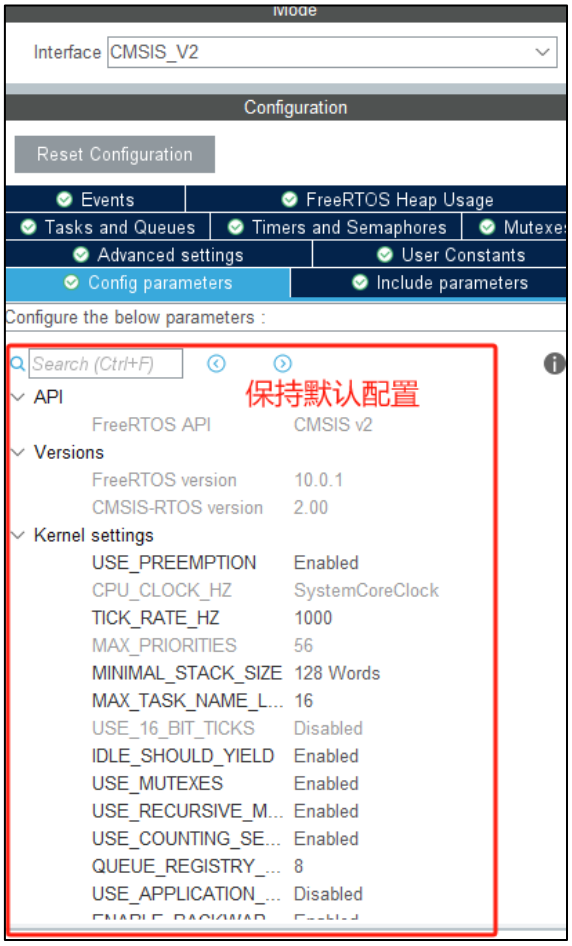


图 67.2.1.3 FreeRTOS 参数配置

（2）任务参数配置

本实验我们需要创建两个任务，分别是 task01 和 task02，接下来我们配置这两个任务，如下：

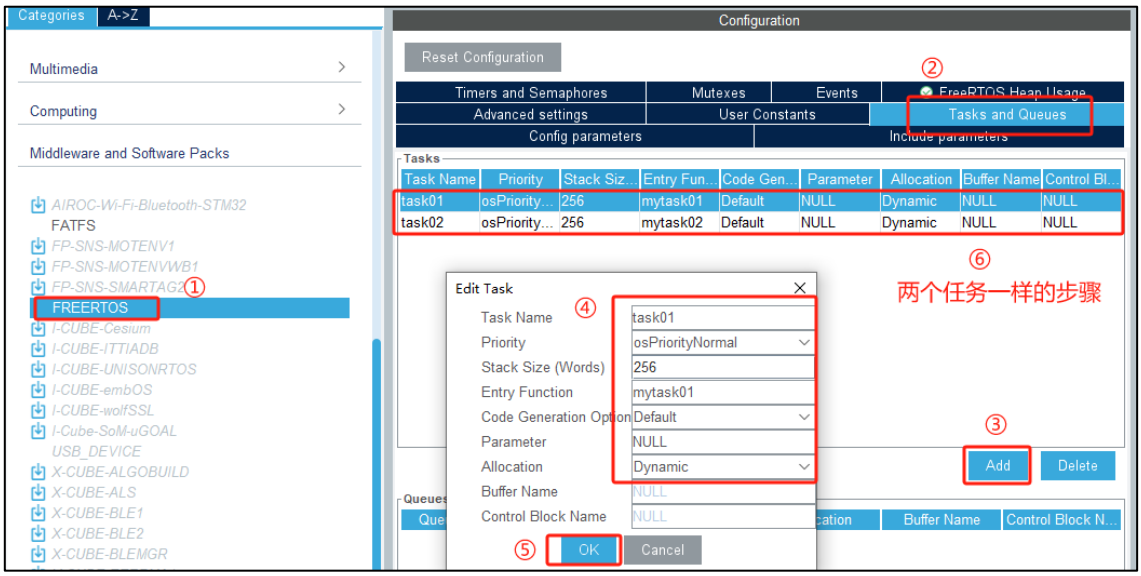


图 67.2.1.4 任务配置

按照上述步骤。即可完成任务名字、任务函数、以及任务堆栈等设置。

（3）SYS 配置

接着点击 System Core→SYS，配置 hal 库需要使用到的 tick 计数源。

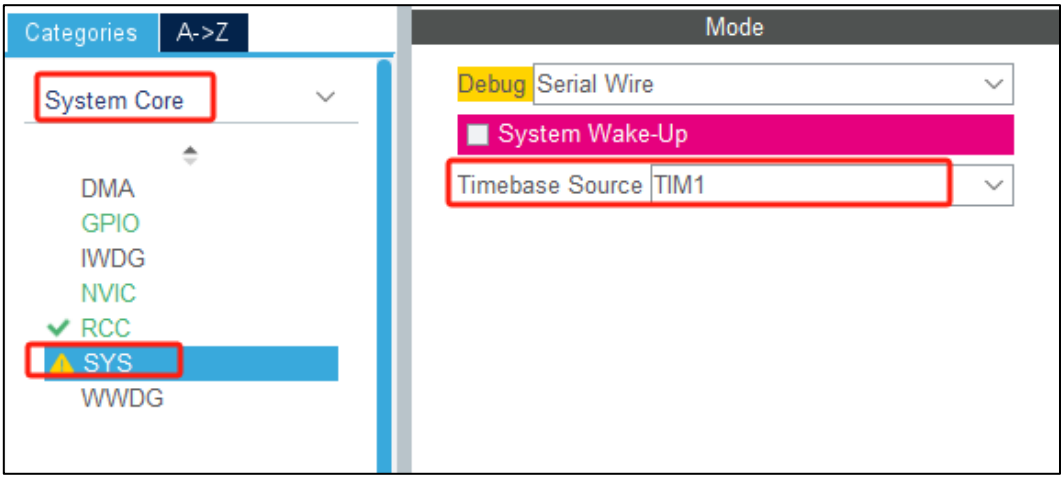


图 67.2.1.5 HAL 库延时函数基准时钟源配置

这里不使用 systick，是因为 freertos 的调度时钟是强制性使用 systick 的并且 systick 的中断优先级被设置为最低，这样就会导致 HAL 库提供的延时函数产生计数不准的情况。所以我们选择 TIM1 来提供 HAL 库的时间基准。

67.2.2 生成初始化工程

配置完成后，按下“Ctrl+S”保存配置并生成工程。在生成的工程中，可以看到多一个 Middlewares 文件夹。该文件夹用于存放 FreeRTOS 的源码以及库相关的文件。工程的结构如下图所示。

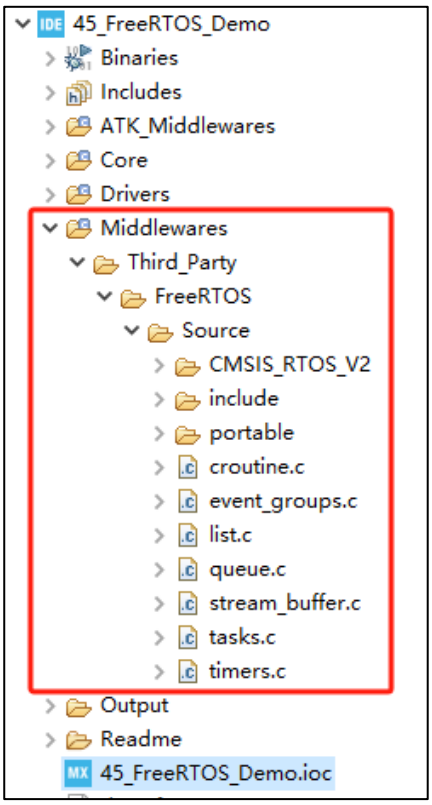


图 67.2.2.1 生成工程

67.2.3 添加用户驱动代码

需要注意的是，在 FreeRTOS 生成的工程中，会自动生成 systick 的中断服务函数，而我们在 delay.c 中也已经定义了该函数，这将导致重定义错误。因此，我们需要删除其中一个函数的定义。在这里，我们将删除 FreeRTOS 中自动生成的 systick 的中断服务函数的定义。

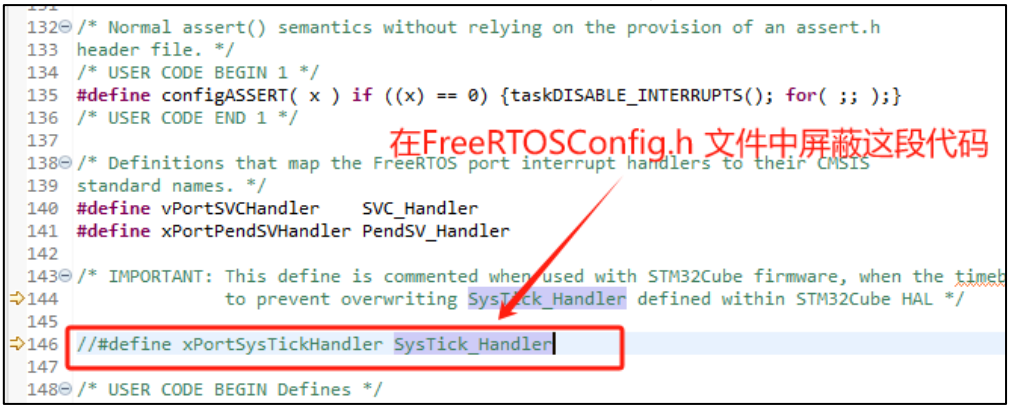


图 67.2.3.1 屏蔽 systick 宏定义

然后我们还需使能在 Drivers/SYSTEM/sys/sys.h 文件中的宏定义 SYS_SUPPORT_OS，表示支持 OS，如下图所示：

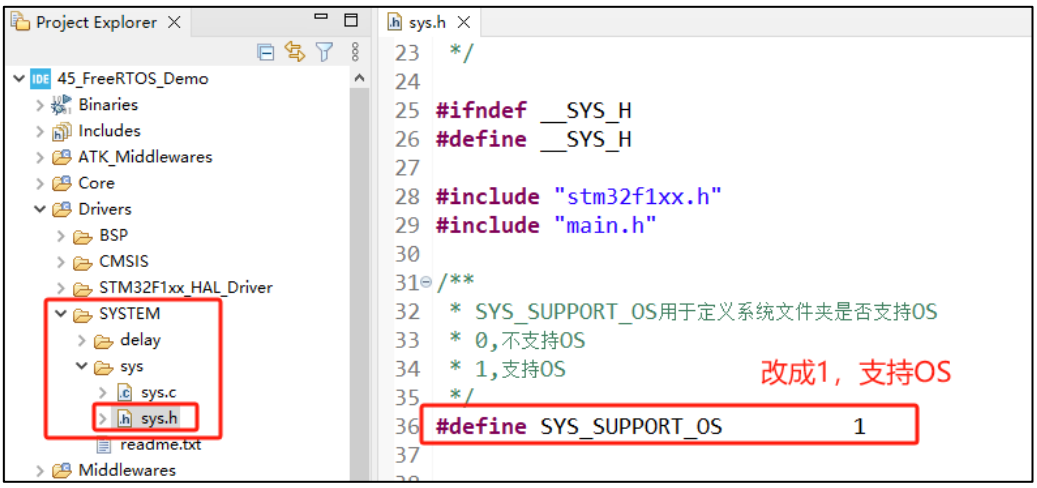


图 67.2.3.2 使能宏以支持 OS

接着我们来修改两个文件分别是：main.c 和 freertos.c。

（1）main.c 文件

```
int main(void)
{
  HAL_Init();

  /* 系统时钟初始化 */
  SystemClock_Config();

  /* 延时初始化 */
  delay_init(72);

  /* GPIO 初始化 */
  MX_GPIO_Init();

  /* FSMC 初始化 */
  MX_FSMC_Init();

  /* 串口初始化 */
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  lcd_init();                                  /* 初始化 LCD */
  my_mem_init(SRAMIN);                         /* 初始化内部 SRAM 内存池 */

  lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
  lcd_show_string(30,  70, 200, 16, 16, "FreeRTOS Porting", RED);
  lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);

  /* USER CODE END 2 */

  osKernelInitialize();
  MX_FREERTOS_Init();

  /* Start scheduler */
  osKernelStart();
  while (1)
  {
  }
}
```

在 main 函数中，我们只添加了一些用于显示 LCD 提示信息的代码。从源代码中可以看出，CubeIDE 自动生成的 FreeRTOS API 接口函数并不是原生的 FreeRTOS 接口函数，而是经过 CMSIS-RTOS 包装后的。这一点需要特别注意。

接着看下 freertos.c 文件的修改。

（2）freertos.c 文件

```
/* Includes ------------------------------------------------------------------*/
#include "FreeRTOS.h"
#include "task.h"
#include "main.h"
#include "cmsis_os.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "../../BSP/LED/led.h"
#include "usart.h"
/* USER CODE END Includes */

/* Definitions for task01 */
osThreadId_t task01Handle;
const osThreadAttr_t task01_attributes = {
  .name = "task01",
  .stack_size = 256 * 4,
  .priority = (osPriority_t) osPriorityNormal,
};

/* Definitions for task02 */
osThreadId_t task02Handle;
const osThreadAttr_t task02_attributes = {
  .name = "task02",
  .stack_size = 256 * 4,
  .priority = (osPriority_t) osPriorityLow2,
};

void mytask01(void *argument);
void mytask02(void *argument);

void MX_FREERTOS_Init(void); /* (MISRA C 2004 rule 8.1) */

/**
  * @brief  FreeRTOS initialization
  * @param  None
  * @retval None
  */
void MX_FREERTOS_Init(void) {

  /* Create the thread(s) */
  /* creation of task01 */
  task01Handle = osThreadNew(mytask01, NULL, &task01_attributes);

  /* creation of task02 */
  task02Handle = osThreadNew(mytask02, NULL, &task02_attributes);

}

/* USER CODE BEGIN Header_mytask01 */
/* USER CODE END Header_mytask01 */
void mytask01(void *argument)
{
  /* USER CODE BEGIN mytask01 */
    while(1)
    {
        LED0_TOGGLE();
        vTaskDelay(1000);
    }

  /* USER CODE END mytask01 */
}
```

```
/* USER CODE BEGIN Header_mytask02 */
/* USER CODE END Header_mytask02 */
void mytask02(void *argument)
{
  /* USER CODE BEGIN mytask02 */
    float num_float = 0.0;
    while(1)
    {
        num_float += 0.01;
        printf("num_float:%.2f\r\n",num_float);
        vTaskDelay(500);
    }
  /* USER CODE END mytask02 */
}
```

在 freertos.c 文件中，我们实现了两个任务的逻辑。第一个任务是让 LED0 每隔 1 秒钟翻转一次，第二个任务则是每隔 500ms 打印一次浮点数（需要注意，支持浮点数打印需要进行相关设置，详见第 4.2.7 节）。

67.3 下载验证

在完成编译和烧录操作后，可以看到开发板板载的 LED0 每隔 1 秒钟翻转一次，串口一每隔 500ms 打印一次浮点数。